

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME V OF EIGHT

Trees and Priorities

DAYS 098 – 124

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

27 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME V · TREES AND PRIORITIES

DAYS 98 TO 124

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume V of eight: **Trees and Priorities**. Days 98 to 124.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67
PART IX	Stacks and queues	Days 68–77
PART X	Linked lists	Days 78–86

PART XI	Recursion and backtracking	Days 87–97
Part XII	Trees and binary search trees	Days 98–112
Part XIII	Heaps and priority queues	Days 113–119
Part XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART XII · TREES AND BINARY SEARCH TREES	1
098 What a tree is, and the vocabulary you need	2
DSA What a tree is, and the vocabulary you need	3
DESIGN Vertical versus horizontal scaling	23
PRACTICPractice — What a tree is, and the vocabulary you need	39
099 Binary trees in code	45
DSA Binary trees in code	46
DESIGN Load balancers	67
PRACTICPractice — Binary trees in code	84
100 Depth-first traversal: preorder, inorder, postorder	90
DSA Depth-first traversal: preorder, inorder, postorder	91
DESIGN Stateless services and why they scale	113
PRACTICPractice — Depth-first traversal: preorder, inorder, postorder	129
101 Breadth-first traversal: level order	135
DSA Breadth-first traversal: level order	136
DESIGN Caching: the single biggest win	157
PRACTICPractice — Breadth-first traversal: level order	174
102 Height, depth, and diameter	180
DSA Height, depth, and diameter	181
DESIGN Cache invalidation and eviction policies	201
PRACTICPractice — Height, depth, and diameter	219
103 Same tree, symmetric tree, and subtree	225
DSA Same tree, symmetric tree, and subtree	226
DESIGN Content delivery networks	246
PRACTICPractice — Same tree, symmetric tree, and subtree	264
104 Path problems, and the return-value trick	270
DSA Path problems, and the return-value trick	271
DESIGN Database replication	293
PRACTICPractice — Path problems, and the return-value trick	309
105 Lowest common ancestor	315
DSA Lowest common ancestor	316
DESIGN Read replicas and replication lag	337
PRACTICPractice — Lowest common ancestor	354
106 Binary search trees: the ordering property	360
DSA Binary search trees: the ordering property	361
DESIGN Sharding, part one: choosing the key	382

	PRACTICE — Binary search trees: the ordering property	400
107	BST insert, search, and delete	406
	DSA BST insert, search, and delete	407
	DESIGN Sharding, part two: rebalancing and hot spots	429
	PRACTICE — BST insert, search, and delete	446
108	Validating a binary search tree	452
	DSA Validating a binary search tree	453
	DESIGN Consistent hashing	474
	PRACTICE — Validating a binary search tree	491
109	Balanced trees, and why balance matters	497
	DSA Balanced trees, and why balance matters	498
	DESIGN Back-of-the-envelope estimation	521
	PRACTICE — Balanced trees, and why balance matters	539
110	Building a tree from its traversals	545
	DSA Building a tree from its traversals	546
	DESIGN Capacity planning: QPS, storage, bandwidth	567
	PRACTICE — Building a tree from its traversals	585
111	Serialising and deserialising a tree	591
	DSA Serialising and deserialising a tree	592
	DESIGN Single points of failure	613
	PRACTICE — Serialising and deserialising a tree	631
112	Trees revision and mock round	637
	DSA Trees revision and mock round	638
	DESIGN Scaling revision and interview questions	661
	PRACTICE — Trees revision and mock round	681
PART XIII · HEAPS AND PRIORITY QUEUES		687
113	The heap: a tree stored in an array	688
	DSA The heap: a tree stored in an array	689
	DESIGN Why distributed systems are hard	710
	PRACTICE — The heap: a tree stored in an array	727
114	Push, pop, and heapify	733
	DSA Push, pop, and heapify	734
	DESIGN The CAP theorem, honestly	757
	PRACTICE — Push, pop, and heapify	774
115	Python's heapq, and the min-heap-only problem	780
	DSA Python's heapq, and the min-heap-only problem	781
	DESIGN Consistency models	803
	PRACTICE — Python's heapq, and the min-heap-only problem	820
116	Top K problems	826
	DSA Top K problems	827

	DESIGN Eventual consistency in practice	847
	PRACTICPractice — Top K problems	866
117	Merging K sorted lists	872
	DSA Merging K sorted lists	873
	DESIGN Quorums: why R plus W must exceed N	895
	PRACTICPractice — Merging K sorted lists	913
118	Two heaps: the running median	919
	DSA Two heaps: the running median	920
	DESIGN Leader election	943
	PRACTICPractice — Two heaps: the running median	961
119	Heaps revision and mock round	967
	DSA Heaps revision and mock round	968
	DESIGN Consensus, and Raft in plain English	989
	PRACTICPractice — Heaps revision and mock round	1009
PART XIV · TRIES		1016
120	The trie: a tree of characters	1017
	DSA The trie: a tree of characters	1018
	DESIGN Distributed transactions and two-phase commit	1039
	PRACTICPractice — The trie: a tree of characters	1059
121	Insert, search, and prefix search	1066
	DSA Insert, search, and prefix search	1067
	DESIGN The saga pattern	1089
	PRACTICPractice — Insert, search, and prefix search	1112
122	Autocomplete and word dictionaries	1119
	DSA Autocomplete and word dictionaries	1120
	DESIGN Idempotency and exactly-once delivery	1141
	PRACTICPractice — Autocomplete and word dictionaries	1159
123	Tries in interviews: word search II	1165
	DSA Tries in interviews: word search II	1166
	DESIGN Clocks, ordering, and why time is a lie	1184
	PRACTICPractice — Tries in interviews: word search II	1201
124	Tries revision and mock round	1207
	DSA Tries revision and mock round	1208
	DESIGN Failure detection, heartbeats, and timeouts	1227
	PRACTICPractice — Tries revision and mock round	1245
BACK MATTER		
A	Appendix A - The nine sections, and what each is for	1251
B	Appendix B - Recall cards	1253
C	Appendix C - The complete 180-day map	1291

D	Appendix D - Glossary	1294
E	Appendix E - References and further reading, by section	1313
F	Appendix F - Colophon: how this book was made	1315
G	Appendix G - Problem index	1316
H	Index	1324

List of figures

98.1 Data structures and algorithms - What a tree is, and the vocabulary you need	2
98.2 System design - Vertical versus horizontal scaling	2
99.1 System design - Load balancers	45
100.1 System design - Stateless services and why they scale	90
101.1 System design - Caching: the single biggest win	135
103.1 System design - Content delivery networks	225
104.1 System design - Database replication	270
105.1 System design - Read replicas and replication lag	315
111.1 System design - Single points of failure	591
112.1 Data structures and algorithms - Trees revision and mock round	637
112.2 System design - Scaling revision and interview questions	637
119.1 Data structures and algorithms - Heaps revision and mock round	967
119.2 System design - Consensus, and Raft in plain English	967
119.3 System design - Consensus, and Raft in plain English	967
120.1 Data structures and algorithms - The trie: a tree of characters	1017
120.2 System design - Distributed transactions and two-phase commit	1017
121.1 Data structures and algorithms - Insert, search, and prefix search	1066
121.2 System design - The saga pattern	1066
122.1 Data structures and algorithms - Autocomplete and word dictionaries	1119
122.2 System design - Idempotency and exactly-once delivery	1119
122.3 System design - Idempotency and exactly-once delivery	1119
123.1 Data structures and algorithms - Tries in interviews: word search II	1165
123.2 System design - Clocks, ordering, and why time is a lie	1165
124.1 Data structures and algorithms - Tries revision and mock round	1207
124.2 System design - Failure detection, heartbeats, and timeouts	1207
124.3 System design - Failure detection, heartbeats, and timeouts	1207

PART XII

Trees and binary search trees

Days 98 to 112 · 15 chapters

Alongside, on the system design track:

Scaling fundamentals

098 What a tree is, and the vocabulary you need

099 Binary trees in code

100 Depth-first traversal: preorder, inorder, postorder

101 Breadth-first traversal: level order

102 Height, depth, and diameter

103 Same tree, symmetric tree, and subtree

104 Path problems, and the return-value trick

105 Lowest common ancestor

106 Binary search trees: the ordering property

107 BST insert, search, and delete

108 Validating a binary search tree

109 Balanced trees, and why balance matters

110 Building a tree from its traversals

111 Serialising and deserialising a tree

112 Trees revision and mock round

What a tree is, and the vocabulary you need

DATA STRUCTURES AND ALGORITHMS

What a tree is, and the vocabulary you need

You can use root, leaf, height, depth, parent and subtree correctly and without hesitation.

SYSTEM DESIGN

Vertical versus horizontal scaling

You can say when buying a bigger machine is the right answer.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is the height of this tree? What is the depth of that node?*
- *Your database is at capacity. Bigger box, or more boxes?*

What a tree is, and the vocabulary you need

1 What this is, and why they ask it

A **tree** is a way of arranging things where each item has one thing above it and any number of things below it, and there are no loops. One item at the very top has nothing above it.

Three sentences. Everything you have stored so far has been **linear** — a list, a string, a stack, a queue — where each item has at most one thing before it and one thing after it. A tree is the first structure where an item can have *several* things after it, and that single change is what makes searching a million items cost twenty steps instead of a million. And the reason a tree can do that is that it has no cycles: from the top, there is **exactly one path** to any item.

They ask about the vocabulary directly, and they ask early, because the rest of the topic is unusable without it. **Height and depth are the two that get confused**, and confusing them in the first minute of a tree question makes every sentence afterwards ambiguous. An interviewer who hears “the depth of the tree” when you mean the height will start checking whether you know the difference, and you will spend the round proving vocabulary instead of solving the problem.

This is the first day of fifteen on trees. Everything in the next two weeks — traversals, balance, binary search trees, tries, heaps — is this shape with rules added.

2 The story

The wedding was in April and there were eleven days of work before it, and Ravi’s uncle Krishnan took charge of all of it the way he had for every wedding in that family since 1994.

He did not do any of it himself. What he did was sit in the front room with a cup of tea and give four people a job each.

Food went to Ravi’s father. The hall and everything in it went to Meena aunty. Guests — that is, receiving them, feeding them, putting the outstation ones somewhere — went to Ravi’s cousin Sudhir. Money went to Krishnan’s own son, because money always goes to your own son.

Then each of those four did the same thing.

Ravi's father did not cook. He gave the main meals to the caterer, the coffee and snacks to two of the younger cousins, and the sweets to a woman in the next street who had made the sweets for four weddings already. Meena aunty split hers into the decoration, the chairs and the lights, and gave each one to somebody.

By the fourth day, if you stood in that front room, you could ask any question and get an answer in about a minute, and the way it worked was always the same. You asked Krishnan. He said, that is Meena's. You asked Meena. She said, lights, that is Bhaskar. You asked Bhaskar, and Bhaskar actually knew, because Bhaskar was standing on a ladder holding the lights.

There were two things Ravi noticed that week.

The first was that everybody had exactly one person above them. Nobody was working for two people. When somebody tried it — one of the cousins doing both snacks and chairs — there was a whole argument about who he had to tell when he went home, and it was sorted out by giving the chairs to somebody else.

The second was about counting. Somebody asked how many steps it took to get from Krishnan to the man tying the flowers, and the answer was three: Krishnan, Meena, Bhaskar, flowers. And then somebody else asked a different question — what is the longest chain anywhere in this whole arrangement — and that was four, through the food side, because the caterer had his own assistant and his assistant had a boy.

Two different questions. Everybody kept answering the second one when they had been asked the first.

3 The idea in plain English

Krishnan has built a tree, and every word in the topic is in that front room.

- Krishnan is the **root** — the one item with nothing above it. A tree has exactly one.
- Everybody else has exactly one person above them: their **parent**.
- The people directly below someone are their **children**. Children of the same parent are **siblings**.
- Bhaskar on the ladder has nobody below him. He is a **leaf** — an item with no children.
- The line from Krishnan down to anybody is a **path**, and there is exactly one such path to each person. That is what “no loops” buys you.

- Meena and everybody under her, taken together, is a **subtree**. Every item in a tree is the root of its own subtree, and that fact is why almost every tree function is recursive.
- The connection between a parent and a child is an **edge**.

Depth and height, which are the two questions from the story

Depth is measured downwards from the root. Height is measured upwards from the lowest leaf.

```
depth of a node    = the number of edges from the ROOT to that node
                    (the root has depth 0)

height of a node   = the number of edges on the LONGEST path from that node
                    down to a leaf
                    (a leaf has height 0)

height of a TREE   = the height of its root
```

Say them as sentences until they stick:

- **Depth is about a node.** “How far down is this one?” Every node has its own depth.
- **Height is about the deepest thing below.** “How far does it go from here?” A leaf’s height is 0 whatever its depth.
- **The height of the tree is the height of the root**, which equals the largest depth in the tree.

The most common error is off by one, and it comes from counting nodes instead of edges. The convention in interviews, and on LeetCode, is:

```
a single node alone:  height 0 by the EDGE count
                      height 1 by the NODE count
```

Both conventions exist and both are used. LeetCode’s *Maximum Depth of Binary Tree* counts **nodes** and answers 1 for a single node. The textbook definition counts **edges** and answers 0. The safe move is one sentence: “*I will count edges, so a single node has height zero — tell me if you want the node count and I will add one.*” That sentence takes four seconds and removes the entire class of off-by-one arguments.

The rest of the vocabulary, defined once

- **Level** — all the nodes at the same depth. The root is level 0. “Level order” means one whole level at a time.

- **Ancestor** — anybody on the path from a node up to the root. Krishnan is an ancestor of the flower man.
- **Descendant** — anybody in your subtree.
- **Degree** — how many children a node has.
- **Internal node** — a node that is not a leaf.
- **Forest** — a collection of trees. What you get if you delete the root.

The two facts that are always true

One: a tree with n nodes has exactly $n - 1$ edges.

Every node except the root has exactly one edge going up to its parent. That is the whole proof, and it is worth being able to say in one line.

Two: there is exactly one path between any two nodes. More than one path would be a loop, and a loop is what makes something not a tree. This is why you never need a `visited` set when walking down a tree from the root — unlike a graph, where you always do.

Binary trees

A **binary tree** is a tree where **every node has at most two children**, called `left` and `right`. The two are distinguishable: a node with only a right child is a different tree from a node with only a left child, even though both have one child.

That restriction is what makes trees usable in code, because a node becomes a fixed-size object:

```
class TreeNode:
    def __init__(self, value: int) → None:
        self.value = value
        self.left: "TreeNode | None" = None
        self.right: "TreeNode | None" = None
```

Almost every interview tree is a binary tree, and [tomorrow](#) is entirely about writing them.

The named shapes, which come up in the same breath as balance:

FULL	every node has 0 or 2 children — never exactly 1
COMPLETE	every level filled except possibly the last, which fills LEFT to right
PERFECT	every level completely filled; $2^{h+1} - 1$ nodes for height h
BALANCED	the left and right heights differ by at most 1, at every node
DEGENERATE	every node has one child — a linked list wearing a hat

Complete is the one that matters most, because it is the shape that lets a tree live in an array with no pointers at all, which is the [heap](#).

Why any of this matters

a linked list of 1,000,000 items:

up to 1,000,000 steps to find something

a balanced tree of 1,000,000:

about 20 steps

Because each step down a balanced binary tree can halve what is left. **20 is $\log_2(1,000,000)$** . That is the entire reason trees exist, and it only holds while the tree is balanced — a degenerate tree is a linked list and gives you the million back. That tension is [day 109](#).

4 The picture

Krishnan’s wedding, drawn as a tree.

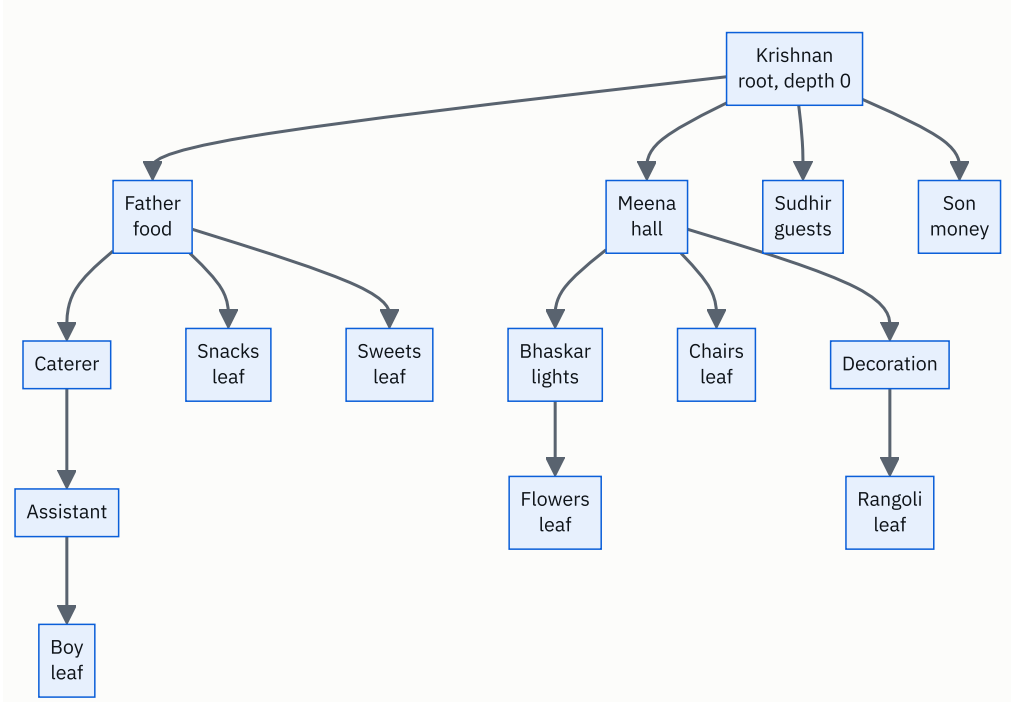
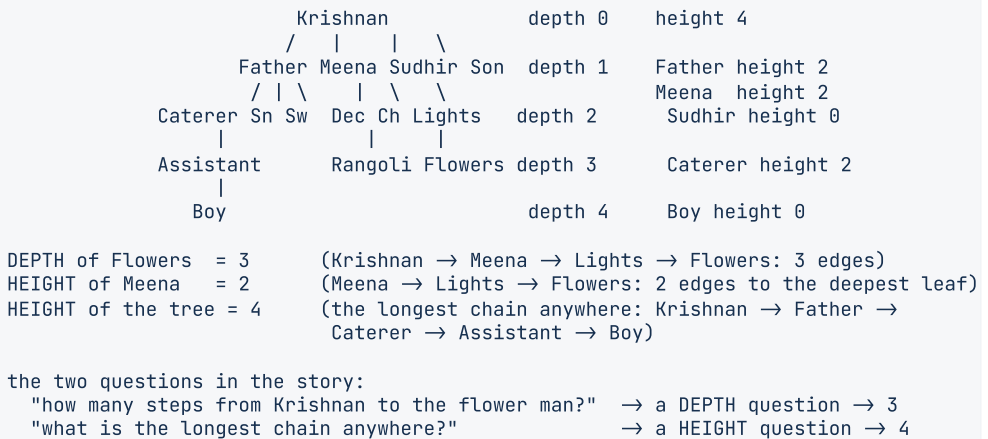


FIGURE 98.1

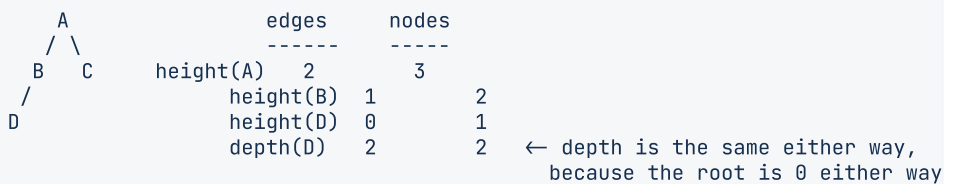
What to notice: **every node has exactly one arrow coming into it, except Krishnan**. That is what makes it a tree rather than a general graph, and it is what the cousin doing both snacks and chairs broke.

The same tree with the two measurements marked:

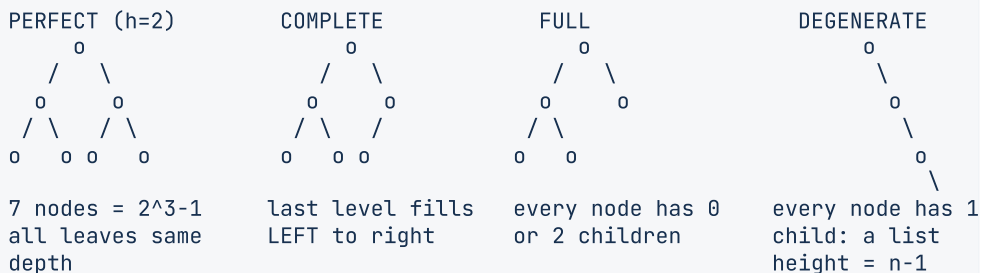


Depth is a property of one node. Height is a property of a node and everything under it. They are equal only for the root, and even then only because the height of the root is the maximum depth.

The two conventions, side by side, so the off-by-one never surprises you:



And the shapes:



5 The code, built step by step

Step 1 — the node

```
class TreeNode:
    def __init__(self, value: int) → None:
        self.value = value
        self.left: "TreeNode | None" = None
        self.right: "TreeNode | None" = None
```

Four lines, and that is the whole data structure. **A tree is not an object — it is a reference to its root node.** A function that takes “a tree” takes a `TreeNode | None`, and `None` is the empty tree.

Step 2 — the shape every tree function takes

```
def something(node):
    if node is None:
        return <the answer for an empty tree>
    left = something(node.left)
    right = something(node.right)
    return <combine left, right and node.value>
```

Every function in the next fifteen days is that shape. The base case is `None`, the recursion is on the two children, and the work is in the combining step. Write those five lines first, every time, and then fill in the two blanks.

Why it is always recursive: **every child is itself the root of a subtree**, so the question you are asking about the tree is exactly the question you ask about each child. That is the [leap of faith](#), and trees are where it feels natural for the first time.

Step 3 — height, which is the base case that catches people

```
def height(node) → int:
    if node is None:
        return -1 # EDGE convention: empty tree is -1
    return 1 + max(height(node.left), height(node.right))
```

-1 for the empty tree, not 0. Then a single leaf is $1 + \max(-1, -1) = 0$, which is correct for the edge convention. If you return `0` for `None`, you get the node count instead — which is also a valid convention, and is what LeetCode’s *Maximum Depth* wants. **Pick one, say which, and be consistent.**

Step 4 — depth needs something height does not

Height is computed by looking *down*, so a node can compute it alone. Depth is measured from the root, so a node cannot know its own depth — **the information has to be carried down from above**.

```
def depth_of(node, target, current_depth=0):
    if node is None:
        return -1                    # not found on this branch
    if node is target:
        return current_depth
    ...
```

That difference — **height comes up from the children, depth goes down from the parent** — is the single most useful structural idea in the whole topic. It reappears on [day 102](#) as “return one thing to the parent while tracking another”, and on [day 104](#) as the return-value trick.

Step 5 — counting, which is the same shape again

```
def count(node) → int:
    if node is None:
        return 0
    return 1 + count(node.left) + count(node.right)
```

Base case `0`, combine by adding one for yourself. **Compare it with `height` and notice that only the combining line changed** — `max` became `+`. That is what “every tree function is the same shape” means in practice.

The complete solution

```
from collections import deque

class TreeNode:
    """A binary tree node. The whole data structure.

    A "tree" is just a reference to its root. `None` is the empty tree, and
    every function must handle it – it is the base case of everything.
    """

    def __init__(self, value: int,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.value = value
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.value})"

def height(node: TreeNode | None) → int:
    """Edges on the longest downward path. A LEAF has height 0.

    The empty tree is -1, so that a leaf computes  $1 + \max(-1, -1) = 0$ .
    Returning 0 for None gives the NODE count instead – also valid, and what
    LeetCode 104 wants. Say which convention you are using.
    """
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))

def max_depth_nodes(node: TreeNode | None) → int:
    """LeetCode 104's convention: count NODES, so a single node is 1.
    The only difference from `height` is the base case."""
    if node is None:
        return 0
    return 1 + max(max_depth_nodes(node.left), max_depth_nodes(node.right))

def count_nodes(node: TreeNode | None) → int:
    """Same shape as height. Only the combining line differs: + instead of max."""
    if node is None:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)

def count_leaves(node: TreeNode | None) → int:
    """A leaf has no children. Note the empty tree is NOT a leaf."""
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)

def count_edges(node: TreeNode | None) → int:
    """Always exactly count_nodes - 1, for a non-empty tree. Every node except
    the root has exactly one edge going up to its parent."""
    n = count_nodes(node)
    return max(n - 1, 0)
```

```

def depth_of_value(node: TreeNode | None, target: int, depth: int = 0) → int:
    """DEPTH is measured from the root, so it must be carried DOWN as an
    argument. Contrast with height, which is computed UP from the children.

    Returns -1 if the value is not in the tree.
    """
    if node is None:
        return -1
    if node.value == target:
        return depth
    left = depth_of_value(node.left, target, depth + 1)
    if left ≠ -1:
        return left
    return depth_of_value(node.right, target, depth + 1)

def nodes_at_depth(node: TreeNode | None, want: int, depth: int = 0) → list[int]:
    """Everything on one level. Depth carried down again."""
    if node is None:
        return []
    if depth == want:
        return [node.value]
    return (nodes_at_depth(node.left, want, depth + 1)
        + nodes_at_depth(node.right, want, depth + 1))

def levels(root: TreeNode | None) → list[list[int]]:
    """Level by level, using a queue rather than recursion.

    The trick is to record the queue's length BEFORE the loop: that count is
    exactly one level, because nothing added inside the loop belongs to it.
    Day 101 is this idea in full.
    """
    if root is None:
        return []
    result: list[list[int]] = []
    queue = deque([root])
    while queue:
        level_size = len(queue)          # the boundary – capture it first
        level: list[int] = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.value)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result

def is_leaf(node: TreeNode | None) → bool:
    return node is not None and node.left is None and node.right is None

def is_full(node: TreeNode | None) → bool:
    """Every node has 0 or 2 children – never exactly 1."""
    if node is None:
        return True
    if (node.left is None) ≠ (node.right is None):
        return False          # exactly one child
    return is_full(node.left) and is_full(node.right)

```



```

def is_perfect(root: TreeNode | None) → bool:
    """Every level completely filled:  $n = 2^{(h+1)} - 1$ ."""
    return count_nodes(root) == 2 ** (height(root) + 1) - 1

def is_balanced(node: TreeNode | None) → bool:
    """Left and right heights differ by at most 1, AT EVERY NODE.

    Written the obvious way, which is  $O(n^2)$  because height is recomputed at
    every node. Day 109 does it in one pass.
    """
    if node is None:
        return True
    if abs(height(node.left) - height(node.right)) > 1:
        return False
    return is_balanced(node.left) and is_balanced(node.right)

def ancestors_of(node: TreeNode | None, target: int,
                 trail: list[int] | None = None) → list[int] | None:
    """Everybody on the path from the root down to `target`, exclusive.

    This is choose-recurse-un-choose from day 094, on a tree: append on the
    way down, pop on the way back up.
    """
    if node is None:
        return None
    if node.value == target:
        return list(trail or [])
    trail = trail if trail is not None else []
    trail.append(node.value) # choose
    found = (ancestors_of(node.left, target, trail)
             or ancestors_of(node.right, target, trail))
    trail.pop() # un-choose
    return found

def build_sample() → TreeNode:
    """Krishnan's wedding, with numbers instead of names.

    1
   / \
  2   3
 / \  \
4  5  6
/
7

    depth 0
    depth 1
    depth 2
    depth 3
    """
    return TreeNode(1,
                    TreeNode(2, TreeNode(4, TreeNode(7)), TreeNode(5)),
                    TreeNode(3, None, TreeNode(6)))

if __name__ == "__main__":
    root = build_sample()

    print(height(root), max_depth_nodes(root)) # 3 4 ← the two conventions
    print(height(None), max_depth_nodes(None)) # -1 0
    print(height(TreeNode(1)), max_depth_nodes(TreeNode(1))) # 0 1

    print(count_nodes(root), count_edges(root)) # 7 6 ← always n-1
    print(count_leaves(root)) # 3 (7, 5, 6)

    print(depth_of_value(root, 7)) # 3

```

```

print(depth_of_value(root, 3))          # 1
print(depth_of_value(root, 99))         # -1

print(nodes_at_depth(root, 2))          # [4, 5, 6]
print(levels(root))                     # [[1], [2, 3], [4, 5, 6], [7]]

print(is_full(root), is_perfect(root), is_balanced(root)) # False False False
perfect = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)),
                  TreeNode(3, TreeNode(6), TreeNode(7)))
print(is_full(perfect), is_perfect(perfect), is_balanced(perfect)) # True True True

print(ancestors_of(root, 7))            # [1, 2, 4]

# a degenerate tree: a linked list wearing a hat
chain = TreeNode(1, TreeNode(2, TreeNode(3, TreeNode(4))))
print(height(chain), count_nodes(chain)) # 3 4 ← height = n-1

```

6 What it costs

Every function in this lesson

height, count_nodes, count_leaves, is_full	visit every node once	$O(n)$
depth_of_value	worst case every node	$O(n)$
nodes_at_depth	every node	$O(n)$
levels (queue)	every node once	$O(n)$
is_balanced, as written	height at every node	$O(n^2)$

$O(n)$ is the default for a tree walk, because there are n nodes and each is visited once. That is the sentence to say. Anything worse than $O(n)$ in a tree function means you are recomputing something, and `is_balanced` above is the standard example — it calls `height` at every node, and `height` itself walks a whole subtree.

`is_balanced` on a balanced tree of n nodes:
`height()` is called once per node
 each call walks that node's subtree
 → total work = sum of subtree sizes = $O(n \log n)$ for a balanced tree
 → $O(n^2)$ for a degenerate one

Fixing that is [day 109](#), and the fix is the return-value trick: compute the height and the balance in the same pass.

Space

recursion stack:	$O(\text{height})$	
balanced tree of n nodes	→ $O(\log n)$	$n = 1,000,000 \rightarrow \sim 20$ frames
degenerate tree	→ $O(n)$	$n = 1,000,000 \rightarrow 1,000,000$ frames
level-order queue: $O(\text{width})$		
widest level of a perfect tree	= $n/2$	

The stack depth is the height, not the node count — and that is exactly why balance matters. On a balanced tree, recursion is free. On a degenerate one:

```
RecursionError: maximum recursion depth exceeded
```

at about a thousand nodes, with Python's default limit. **A tree problem with $n \leq 10^5$ and no balance guarantee is telling you that a recursive solution may blow the stack**, and that an iterative version is expected.

Note the queue and the stack trade places: **depth-first uses $O(\text{height})$ and breadth-first uses $O(\text{width})$** , and for a perfect tree the widest level holds half the nodes. So level-order on a million- node perfect tree holds 500,000 nodes in the queue at once, where the recursive walk holds 20 frames. Neither is universally cheaper.

The numbers that justify trees at all

```
height of a PERFECT binary tree with n nodes =  $\log_2(n + 1) - 1$ 
```

```
n = 1,000           height ≈ 9
n = 1,000,000       height ≈ 19
n = 1,000,000,000   height ≈ 29
```

A billion items, thirty steps. And the same billion in a degenerate tree is a billion steps. The entire value of the structure sits on that one condition, which is why “is it balanced?” is the first question about any tree in a real system.

```
n = 1,000,000
balanced:    ~20 comparisons per lookup
degenerate:  1,000,000 comparisons per lookup
ratio:       50,000×
```

7 The traps

Trap 1 — height and depth swapped

The single most common vocabulary error. “The depth of the tree” is not a thing — trees have a **height**; **nodes** have a depth.

```
wrong: "this tree has depth 4"
right: "this tree has height 4" or "the deepest node is at depth 4"
```

Both sentences describe the same number here, which is exactly why the mistake survives — until a question asks for the depth of a *particular node* and the two stop agreeing.

Trap 2 — the off-by-one from the two conventions

```
def height(node):  
    if node is None:  
        return 0 # NODE convention  
    return 1 + max(height(node.left), height(node.right))
```

This returns 1 for a single node, not 0. It is not wrong — it is a different convention — but mixing it with an edge-based definition inside one solution gives answers that are off by one in some branches and not others.

Say the convention out loud before writing the base case, and then never think about it again.

Trap 3 — forgetting the `None` case

```
def height(node):  
    return 1 + max(height(node.left), height(node.right))
```

```
AttributeError: 'NoneType' object has no attribute 'left'
```

Every tree function begins with the `None` check. It is not defensive coding — **`None` is the base case**, and a recursion without a base case is not a recursion.

Trap 4 — treating “no children” and “None” as the same thing

```
def count_leaves(node):  
    if node is None:  
        return 1 # WRONG: the empty tree is not a leaf
```

On a node with one child, this counts the missing child as a leaf:

```
count_leaves(TreeNode(1, TreeNode(2))) → 3    correct answer: 1
```

A leaf is a real node with no children. The empty tree is not a leaf; it is nothing.

Trap 5 — assuming the tree is balanced

Nothing in the definition of a tree says anything about balance. An interviewer who says “a binary tree” means a possibly-degenerate one, and any argument that starts “since the height is $\log n$ ” is wrong unless they said “balanced” or “binary search tree with balancing”.

```
building a tree by inserting 1, 2, 3, ..., 1000 in order → a chain of height 999
```

Trap 6 — assuming a binary tree is a binary search tree

They are different. A binary tree has at most two children per node. A **binary search tree** adds a rule about the *values* — everything left is smaller, everything right is larger — and that rule is what lets you skip half the tree.

You cannot search a plain binary tree in $O(\log n)$. Finding a value in one is $O(n)$, because you have no idea which way to go. Saying “I would search the left subtree, and if I do not find it, the right” is the correct answer for a plain binary tree and the wrong answer for a BST.

Trap 7 — recursing on a degenerate tree

```
height(chain_of_10000_nodes)
```

```
RecursionError: maximum recursion depth exceeded
```

Python’s default limit is 1000 frames. A tree question with n up to 10^5 and no balance guarantee is a hint that an iterative solution is expected — or at least that you should mention the risk.

Trap 8 — a node with two parents

If a node has two things pointing at it, it is **not a tree**. It might be a directed acyclic graph, and every algorithm in this topic will either loop for ever or count things twice.

```
shared = TreeNode(5)
root = TreeNode(1, TreeNode(2, shared), TreeNode(3, shared))
count_nodes(root) → 5      but there are only 4 distinct nodes
```

The cousin doing both snacks and chairs. **One parent each, or it is not a tree.**

8 In the interview

How it gets asked

- Straight, as a warm-up: “What is the height of this tree? What is the depth of that node?”
- The definitional probe: “What makes something a tree rather than a graph?”
- The convention check: “Is the height of a single node 0 or 1?”
- The first real problem: “Return the maximum depth of a binary tree.” LeetCode 104.
- The trap: “Can you find a value in a binary tree in $O(\log n)$?”

What to say out loud, in the first ninety seconds

1. **Define a tree by its constraints.** “Every node has exactly one parent except the root, which has none, and there are no cycles — so there is exactly one path from the root to any node. That is why I never need a visited set walking down a tree, unlike a graph.”
2. **Separate the two measurements immediately.** “Depth is a property of a node, measured downwards from the root. Height is a property of a node measured to its deepest leaf, and the height of the *tree* is the height of its root.”
3. **Declare the convention.** “I will count edges, so a leaf has height 0 and a single-node tree has height 0. If you want the node count — which is what LeetCode’s maximum-depth problem uses — that is the same code with a base case of 0 instead of -1 .”
4. **Name the structural asymmetry.** “Height is computed on the way *up* from the children, so a node can work it out alone. Depth has to be passed *down* from the parent, because a node cannot see the root. That distinction drives most tree solutions.”
5. **State the default complexity.** “Any full walk is $O(n)$ time and $O(\text{height})$ space for the stack — about 20 frames on a balanced million-node tree, and a million on a degenerate one.”
6. **Do not assume balance.** “Unless you tell me the tree is balanced, I will assume the worst case is a chain, so $O(\log n)$ claims need the balance to be given.”

The follow-ups

“Is the height of a single node 0 or 1?” “Both conventions are in use and I would state mine rather than guess yours. Counting **edges**, a single node has height 0 and an empty tree is -1 — that is the textbook definition and it makes the arithmetic clean, because height plus depth relationships work out. Counting **nodes**, a single node is 1 and empty is 0 — that is what LeetCode’s *Maximum Depth of Binary Tree* expects. The

code is identical except for the base case, so I will say which I am using and it will be consistent throughout. What I would not do is mix them, because then only some branches are off by one and it is very hard to see.”

“**What makes a tree different from a graph?**” “Two things, and the second follows from the first. Every node has exactly one parent except the root, and there are no cycles. Together those give the property I actually use: **exactly one path from the root to any node**. That is why tree traversals need no visited set — you cannot arrive at the same node twice — and why every tree function is naturally recursive, since each child is the root of a subtree that looks exactly like the original problem. A tree with n nodes has exactly $n - 1$ edges, because every node except the root contributes exactly one edge upward. The moment a node has two parents, all of that stops being true: walks double-count, and the recursion can loop.”

“**Find a value in a binary tree.**” “In a plain binary tree, $O(n)$ — I have to search the left subtree and, if it is not there, the right, because nothing about the values tells me which way to go. You only get $O(\log n)$ in a **binary search tree**, where every value in the left subtree is smaller and every value in the right is larger, so one comparison discards half the remaining tree. And even then, $O(\log n)$ needs the tree to be balanced: if it was built by inserting sorted data, it is a chain and lookup is $O(n)$ again. So the honest answer is $O(n)$ for a binary tree, $O(\text{height})$ for a BST, and $O(\log n)$ only for a *balanced* BST.”

“**What is the space complexity of your recursive solution?**” “ $O(\text{height})$ for the call stack — one frame per level. On a balanced tree of a million nodes that is about twenty frames, which is nothing. On a degenerate tree it is a million frames, and Python’s default recursion limit is a thousand, so it raises `RecursionError`. If the constraint says n up to 10^5 with no balance guarantee, I would either write it iteratively with an explicit stack or say the risk out loud. Worth adding: depth-first costs $O(\text{height})$ and breadth-first costs $O(\text{width})$, and for a perfect tree the widest level is half the nodes — so on a million-node tree, level order holds half a million nodes in the queue while the recursive walk holds twenty frames. Neither is always cheaper.”

“**Why do trees exist at all?**” “Because each step down can eliminate a large fraction of what is left. A balanced binary tree of a million nodes has height about twenty, so a lookup is twenty comparisons instead of a million — a factor of fifty thousand. And unlike a sorted array, which also gives $\log n$ lookup, a tree keeps that guarantee while you insert and delete, because you only rearrange a few pointers instead of shifting everything. That is the trade trees are actually making: a bit more memory and pointer-chasing, in exchange for insertion and lookup both being logarithmic.”

“How would you check whether a tree is balanced?” “The obvious version compares the heights of the two subtrees at every node — but computing a height is itself a full walk of that subtree, so calling it at every node makes the whole thing $O(n \log n)$ on a balanced tree and $O(n^2)$ on a chain. The fix is to compute the height and the balance in **one** pass: a function that returns the height, or a sentinel meaning ‘already unbalanced’, so a failure short-circuits all the way up. That is the return-value trick, and it is the same idea that makes diameter and maximum path sum one pass instead of two.”

A model answer

Asked: *what is the height of this tree, and what is the depth of that node?*

“Let me separate the two, because they are measured in opposite directions and it is worth being precise before I give you numbers.

Depth belongs to a node. It is the number of edges from the root down to that node, so the root is at depth 0, its children are at depth 1, and so on. Every node has its own depth, and it only makes sense relative to the root.

Height also belongs to a node, but it looks the other way. It is the number of edges on the longest path from that node *down* to a leaf. So every leaf has height 0, whatever its depth. And when people say ‘the height of the tree’, they mean the height of the root — which happens to equal the largest depth in the tree.

That difference is not just terminology; it decides how the code is written. **Height is computed upwards** — a node asks its two children for their heights and returns one more than the larger — so a node can work it out with no information from above. **Depth has to be carried downwards** as an argument, because a node has no way of seeing the root. Almost every tree problem is one of those two shapes, and knowing which one you are in tells you immediately whether the answer comes back from the recursion or goes in as a parameter.

One thing I would pin down before writing code: I will count **edges**, so a leaf has height 0 and a single-node tree has height 0. The other convention counts nodes, and gives 1 — that is what LeetCode’s maximum-depth problem uses. The code is identical apart from the base case, `-1` versus `0` for the empty tree, and the only real mistake is mixing them.

For this tree specifically: the height is 4, because the longest chain from the root to a leaf has four edges. The node you pointed at is at depth 3 — three edges from the root. Those are different numbers answering genuinely different questions, and if I had said ‘the depth of the tree’ I would have hidden that.

On cost: computing either is $O(n)$, since I visit every node once, and $O(\text{height})$ space for the call stack. On a balanced million-node tree that stack is about twenty frames; on a degenerate one — a tree built by inserting sorted values — it is a million, and Python raises a `RecursionError` at about a thousand. So unless you tell me the tree is balanced, I will assume it might be a chain.”

9 Recall card

- **A tree: every node has exactly one parent except the root, and no cycles — so there is exactly one path from the root to any node.** That is why tree walks need **no visited set** (a graph does), and why every tree function is recursive: **each child is the root of a subtree that looks like the original problem.** A tree with n nodes has exactly $n - 1$ edges.
- **Depth belongs to a node and is measured DOWN from the root (root = 0).** **Height belongs to a node and is measured UP from its deepest leaf (leaf = 0).** **The height of the tree is the height of the root.** “The depth of the tree” is not a thing.
- **Height is computed UP from the children — a node can work it out alone.** **Depth must be passed DOWN as an argument.** That asymmetry decides how nearly every tree problem is written.
- **Say the convention before the base case: edges (None → -1, single node → 0) or nodes (None → 0, single node → 1).** LeetCode 104 uses nodes. The code differs by one character; mixing them makes only *some* branches off by one.
- **Every full walk is $O(n)$ time and $O(\text{height})$ space — ~20 frames on a balanced million-node tree, 1,000,000 on a degenerate one (RecursionError at ~1,000).** **Never assume balance**, and never confuse a **binary tree** (≤ 2 children) with a **binary search tree** (ordered values): finding a value in a plain binary tree is $O(n)$, not $O(\log n)$. DFS costs $O(\text{height})$; **BFS costs $O(\text{width})$** , and the widest level of a perfect tree is $n/2$.

Vertical versus horizontal scaling

1 What this is, and why they ask it

When a system runs out of capacity there are exactly two things you can do. **Vertical scaling** means replacing the machine with a bigger one — more cores, more memory, faster disk. **Horizontal scaling** means putting more machines beside it and dividing the work.

Three sentences. Vertical scaling changes no code and creates no new problems, and it runs out — there is a largest machine you can buy, and the price stops being proportional to the power long before you reach it. Horizontal scaling has no ceiling, and it buys something vertical scaling can never buy: **surviving the failure of one machine**. But it only works if the work can actually be divided, which for stateless application servers is easy and for a database is the hardest problem in the subject.

They ask it because the expected answer is not “horizontal” and most candidates say “horizontal” immediately. The good answer is **“vertical first, and here is the specific number at which I would stop”** — because vertical scaling is one command and a restart, and horizontal scaling for a database means replication, sharding, and giving up transactions across shards. Reaching for the hard one when the easy one still has room is a real engineering mistake, and interviewers are checking whether you know that.

2 The story

The photocopy shop was next to the college gate and it was Nafisa's, and for eight years it had one machine.

In the last week of every semester the queue went out of the door and along the wall. Students with notes to copy for the whole class, forty pages at a time, and everybody wanting it now.

She had thought about this properly, twice.

The first time, a salesman came and offered her a bigger machine. Faster, he said. Twenty-two pages a minute instead of fourteen. She worked out that this would clear the queue about half an hour sooner on a bad day, and it cost eighty thousand rupees, and it took four hours to install during which the shop would do nothing at all. She

bought it anyway, because it was one decision and she did not have to change anything else about how she worked. Same counter, same person, same everything, just faster.

Two years later she was in the same position again, and this time the salesman offered her the top model. Thirty-five pages a minute. Two lakh forty thousand rupees.

That was the number that stopped her. Twice the speed of what she had, for three times the money. Whereas a second machine of the kind she already owned was eighty thousand, and two of those would do forty-four pages a minute, which was more than the expensive one.

The second machine was more work. It would not fit where the first one was, so the table had to move. Somebody had to stand at it, which meant paying her nephew. And students had to be sent to the right counter, which meant she now had to think about who was waiting for what, which she had never had to do before.

She bought the second machine.

What decided it, in the end, was not the money and not the speed. It was that in March the one machine had broken on a Tuesday and she had been shut for two days waiting for the part. Two days, in the last week of the semester.

With two machines, a Tuesday like that is a slow day. With one machine it is a closed shop, and no amount of buying a faster machine had ever changed that.

3 The idea in plain English

Nafisa has just worked through the whole decision, in the right order, for the right reasons.

- The faster machine is **vertical scaling** — also called *scaling up*. Same shape, more power.
- The second machine is **horizontal scaling** — *scaling out*. More units, work divided between them.
- The nephew and the moving table are the **operational cost** of going horizontal: it is never only the hardware.
- Sending students to the right counter is a **load balancer**.
- Two days shut in March is **availability**, and it is the real reason to go horizontal.

Vertical scaling: what it is and what it buys

You stop the process, move to a bigger machine, and start it again. On a cloud provider that is literally one command and a reboot.

What it buys:

- **No code changes.** Nothing about your program knows how big the machine is.
- **No new failure modes.** One machine still means one machine; there is no network between parts of your system that was not there before.
- **It is fast to do.** Minutes, not a quarter's project.

What it costs, and where it stops:

- **Downtime to resize.** Usually a few minutes, and always at least a restart.
- **A hard ceiling.** There is a largest machine, and you can look up what it is.
- **A price curve that bends.** This is the one people do not expect: past a certain size, doubling the machine costs much more than double.
- **Still one machine.** If it dies, everything is down. **No amount of vertical scaling improves availability** — that is the sentence to say.

Horizontal scaling: what it is and what it demands

You put `n` machines behind a **load balancer** — a component that receives every request and hands it to one of them.

What it buys:

- **No ceiling.** Ten machines, a hundred, a thousand.
- **Availability.** One dies, the rest carry on. This is the real prize.
- **Cheaper per unit of work at scale,** because you buy many ordinary machines rather than one extraordinary one.
- **You can scale down again,** which matters when the load is spiky.

What it demands:

- **The work must be divisible.** Ten machines only help if a request can be handled by any one of them.
- **The machines must be stateless,** or requests must always return to the same one — which is [tomorrow's](#) and [day 100's](#) subject.
- **You now have a distributed system,** with everything that implies: partial failures, network delays, and two machines that disagree. The load balancer itself needs to not be a single point of failure.

The rule that decides it

Scale vertically until it stops being cheap or until you need availability. Then scale horizontally.

For **application servers** the answer is almost always horizontal, and quickly, because they are naturally stateless — any server can handle any request — so dividing the work is nearly free.

For **databases** the answer is vertical for much longer than people expect. A single modern database machine handles a very large business. Horizontal scaling of a database means **replication** (copies for reading) and then **sharding** (splitting the data), and sharding is the point at which you lose cross-shard transactions and cross-shard joins. That is a permanent tax on every feature you build afterwards.

The two things that are actually different

People argue about cost and ceilings, but the two properties that genuinely differ in kind are these.

Availability. One machine has one failure. Two machines, either of which can serve, have a combined failure probability that is the product of the two — which is a completely different number.

one machine, 99% available → 3.65 days down per year
two machines, either can serve → $1 - (0.01 \times 0.01) = 99.99\%$ → 53 minutes per year

Elasticity. You can add and remove horizontal capacity in minutes and pay only for what is running. A big machine is a big machine at 3 a.m. as well.

The thing that is *not* different

Horizontal scaling does not make one request faster. A hundred machines serve a hundred times as many requests; each individual request takes exactly as long, and possibly slightly longer because there is now a load balancer hop in front. If your problem is that a single query takes eight seconds, adding machines does nothing at all — that is a vertical, or an indexing, or an algorithm problem.

Throughput scales horizontally. Latency does not. Say that sentence; it catches out a lot of people.

4 The picture

The two directions.

VERTICAL (scale up)

```
+-----+
|  2 CPU |
|  8 GB  |
+-----+
      |
      v  replace
+-----+
|  8 CPU |
| 32 GB  |
+-----+
      |
      v  replace
+-----+
| 64 CPU | ← and then you stop,
| 512 GB |   because there is no
+-----+   bigger one to buy
```

one machine, always
no code change
restart to resize
IT DIES → EVERYTHING IS DOWN

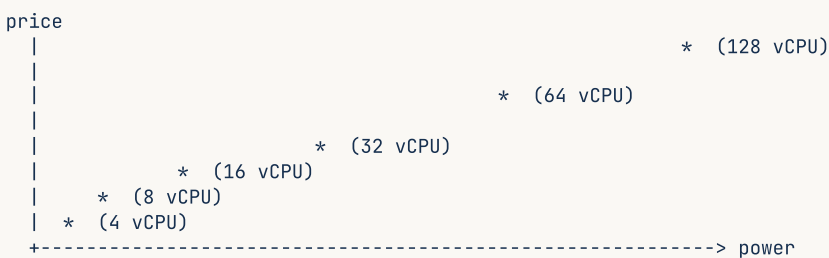
HORIZONTAL (scale out)

```
+-----+
| Load balancer |
+---+-----+---+
      |         |
+---v---+ +---v---+
| 2CPU | | 2CPU |
| 8 GB | | 8 GB |
+-----+ +-----+
      |         |
+---v-----v---+
| more, later |
+-----+
```

n machines, any number
requests must be divisible
add one with no downtime
one dies → the rest carry on

The cost curve, which is the part candidates do not expect:

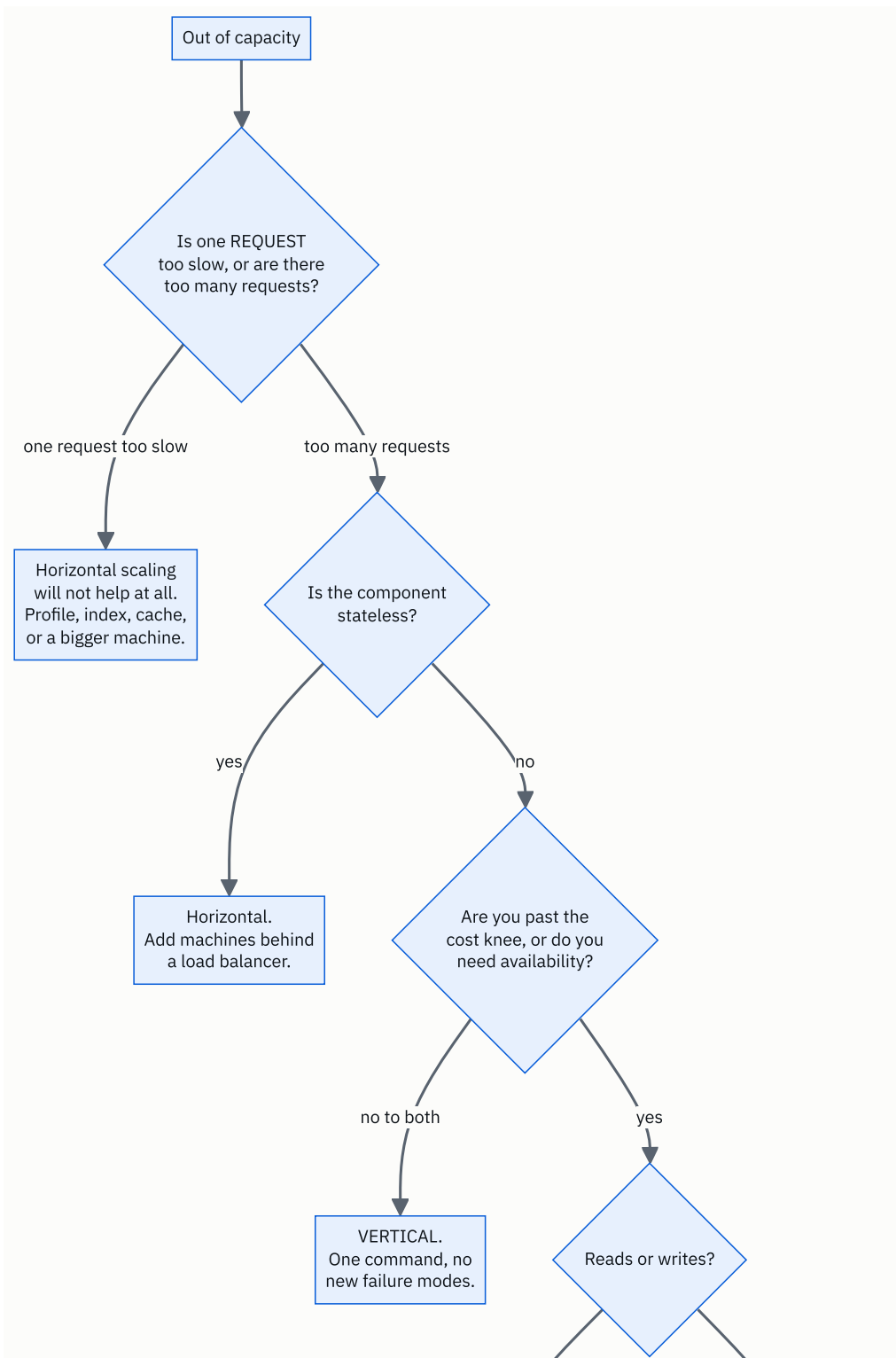
relative price against relative power, one cloud provider's general-purpose range



up to about 32 vCPU the line is roughly straight: twice the machine, twice the price. past that it bends upwards, and the very largest instances cost several times what their raw specification suggests, because few people buy them.

THE KNEE IS THE DECISION POINT.

The decision, as a procedure:



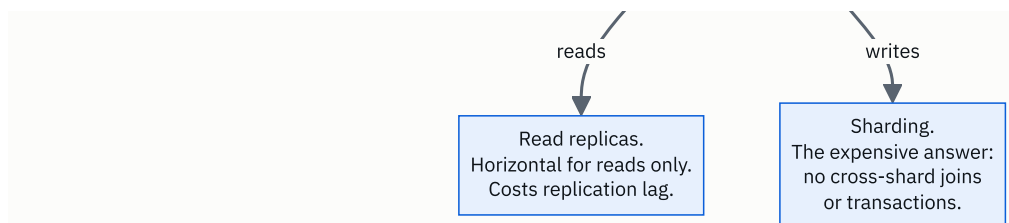


FIGURE 98.2

What to notice: **the first question is not ‘how big is the load’**. It is whether one request is slow or there are too many of them, because horizontal scaling only ever answers the second.

5 How it actually works

Vertical scaling, mechanically

On a cloud provider:

1. stop the instance
 2. change the instance type
 3. start it
- typically 2-5 minutes of downtime
- (or use live resize, where supported)
t3.large → m6i.4xlarge

For a managed database (Amazon RDS, Cloud SQL) it is a console setting, and with a standby replica the provider can do it as a **failover**: resize the standby, promote it, resize the old primary. Downtime drops to the failover time, usually under a minute.

The important operational detail: some things do not automatically use the new capacity. A database that was configured with a 4 GB buffer pool still uses 4 GB on a 64 GB machine. A JVM with a fixed heap still has that heap. **Resizing the machine without re-tuning the configuration is a very common way to pay for capacity you do not use**, and mentioning it is a strong signal of having actually done this.

The ceiling, in real numbers

The largest generally available cloud instances, as an order of magnitude:

general purpose	~128 vCPU,	~512 GB RAM	
memory optimised	~128 vCPU,	~4 TB RAM	
the extreme end	~448 vCPU,	~24 TB RAM	(specialised, very expensive)

A 4 TB machine holds a very large database entirely in memory. That is the honest reason vertical scaling lasts longer for databases than people assume: most companies' entire transactional dataset fits in the memory of one large machine, and a database that fits in memory is extremely fast.

Horizontal scaling, mechanically

Three things have to exist:

1. **A load balancer** in front, distributing requests. [Day 099](#) is entirely about how it chooses.
2. **Stateless machines**, so any of them can serve any request. Anything a request needs must come from a shared store, not from the machine's memory. That is [day 100](#).
3. **Health checks**, so a broken machine stops receiving traffic. Without this, adding a second machine makes availability *worse*: you have doubled the chance that some requests hit a broken box.

For a **database** the same three are much harder, and it happens in two stages:

- **Replication** — one primary takes writes, several replicas serve reads. This scales reads only, and it introduces **replication lag**: a replica can be a few milliseconds to a few seconds behind. [Days 104 and 105](#).
- **Sharding** — the data is split across machines by a key, so each machine holds a slice and takes both reads and writes for it. This scales writes, and it is the expensive one. [Days 106 and 107](#).

What real companies did

- **Stack Overflow** ran one of the busiest sites on the internet on a famously small number of machines, with a handful of very large SQL Servers — a deliberate, public argument for vertical scaling and careful engineering over horizontal sprawl.
- **Shopify, Instagram and Notion** all sharded their main database only after years, and each described it as a major multi-quarter project. Instagram ran on a single PostgreSQL primary with read replicas for a long time at very large scale.
- **Netflix** is the horizontal extreme: thousands of stateless instances, auto-scaling on demand, and the assumption that any instance can vanish at any moment — which is what [Chaos Monkey](#) exists to enforce.
- **DynamoDB, Cassandra and Kafka** are designed horizontally from the start: they have no single primary, and adding a node is a normal operation rather than a project.

The pattern in all of them: **stateless tiers went horizontal early and easily; the database went vertical for as long as it possibly could.**

6 The numbers

The cost knee

Approximate on-demand pricing for a general-purpose cloud instance family, normalised:

vCPU	RAM	relative price	price per vCPU
2	8 GB	1.0×	0.50
4	16 GB	2.0×	0.50
8	32 GB	4.0×	0.50
16	64 GB	8.0×	0.50
32	128 GB	16.0×	0.50
64	256 GB	33.0×	0.52
128	512 GB	70.0×	0.55

Up to about 32 vCPU, the price per unit of power is flat. Past that it climbs, and at the very top of the range — the specialised high-memory instances — it climbs steeply, because those machines are built for people who have no alternative.

So the honest answer to “when do I stop scaling up” is:

STOP when either:
(a) the price per unit of work starts rising – around 32-64 vCPU on general hardware, or
(b) you need to survive a machine failure – which is at ANY size

Availability, which is the argument that actually wins

one machine at 99.9% available	→ 8.8 hours down per year
two, either can serve	→ $1 - 0.001^2 = 99.9999\%$ → 32 seconds per year
three	→ effectively never, from this cause alone

That is a factor of a thousand from one extra machine, and no amount of vertical scaling produces any of it. This is the number to lead with when someone insists that a bigger box is always simpler.

The caveat worth adding, because it is where naive redundancy math goes wrong: the two machines must fail **independently**. Two instances in the same rack, on the same power feed, in the same availability zone, running the same buggy deploy, do not fail independently — and the arithmetic above quietly stops applying.

Sizing a real decision

GIVEN: 6,000 peak QPS, application tier

one app server, real work ~1,000 QPS

→ 6 machines at 100% utilisation

→ 9-10 machines at a sane 60-70% utilisation

vertical alternative: one machine at 6,000 QPS

needs ~6× the cores of a 1,000-QPS box

exists, costs about the same in raw price

and gives 8.8 hours of downtime a year instead of 32 seconds

The horizontal answer here is not about cost — it is about the 8.8 hours.

GIVEN: a database at 4,000 writes/second on an 8 vCPU box, at 90% CPU

vertical: move to 32 vCPU. ~4× headroom. One command, ~1 minute of failover.
cost: 4× the instance price, still on the flat part of the curve.

horizontal: shard by user id across 4 machines.

cost: 3-6 months of engineering, no cross-shard transactions afterwards, and every future feature pays that tax.

→ vertical, obviously, and it buys you two years

That comparison, stated with those numbers, is the answer to the interview question.

When vertical genuinely runs out

working set larger than the biggest machine's RAM

4 TB is the practical top end → a 10 TB hot dataset cannot fit

write throughput above one machine's disk

~50,000 IOPS on a good SSD; a write-heavy workload above that needs more disks

a single-country business becoming global

no machine is close to everybody; distance is not a capacity problem

Those three are the real triggers, and each of them is a **fact about the workload**, not a preference.

7 The trade-offs

Vertical: simplicity for a ceiling and a single failure

Take vertical when the component is hard to distribute (a database), the load is well under the cost knee, and you can tolerate the downtime — or when you simply do not yet know whether the growth is real.

I would not scale vertically if availability is a stated requirement. One box cannot be highly available, at any price, and buying a bigger one is answering a different question from the one asked.

Horizontal: no ceiling, for a distributed system you now have to run

Take horizontal when the component is stateless, when you need to survive a machine failure, or when the load is spiky enough that elasticity saves real money.

I would not scale horizontally if the work cannot be divided — a single slow query, a single write stream that must be ordered — or if the component is a database that is still comfortably within one machine. **Sharding a database you did not need to shard is one of the most expensive mistakes available in this subject**, because the cost is not the migration, it is every feature afterwards.

The hybrid, which is what almost everybody actually runs

stateless web/app tier	HORIZONTAL, auto-scaled, small machines
cache	HORIZONTAL, sharded by key
primary database	VERTICAL, one large machine, scaled up as needed
read replicas	HORIZONTAL, a few copies of that machine
object storage	somebody else's horizontal problem

Say this out loud in the interview. “Vertical or horizontal” is a false choice at the system level; different tiers get different answers, and naming which tier gets which is the actual skill.

Where the argument gets subtle

- **Utilisation.** Ten machines at 30 percent utilisation cost more than three at 90 percent, and are more available. There is a real trade between cost and headroom, and “how much headroom” is a business decision, not an engineering one.
- **The load balancer is a component too.** It has capacity, and it can fail. In practice it is either a managed service, or several of them behind DNS — but “what if the load balancer dies” is a fair question and “it is managed and multi-zone” is a fair answer.

- **Restarts get slower as machines get bigger.** A 512 GB cache takes a long time to warm up. A machine that takes twenty minutes to become useful is a machine you cannot casually replace, which quietly reduces the availability benefit of having several.
 - **Horizontal scaling can make things slower.** More machines means more connections to the database, more cache clients, more coordination. A hundred app servers each holding a pool of twenty database connections is two thousand connections, which many databases will not accept — and the fix is a connection pooler, which is another component.
-

8 In the interview

How it gets asked

- The direct one: *“Your database is at capacity. Bigger box, or more boxes?”*
- The general one: *“How would you scale this?”* — and the expected answer is per tier, not one word.
- The trap: *“Just add more servers, right?”* — they want to hear the stateless requirement, or the fact that one slow request stays slow.
- The number probe: *“When would you stop scaling up?”*
- The availability probe: *“You have one very large database server. What is your availability?”*

What to say out loud, in the first ninety seconds

1. **Ask which problem it is.** “Is one request too slow, or are there too many requests? Horizontal scaling only ever answers the second. If a single query takes eight seconds, more machines change nothing.”
2. **Answer per tier, not for the system.** “The application tier is stateless, so that is horizontal and easy. The database is a different conversation.”
3. **Default to vertical for the database, and give the stopping condition.** “I would scale the database up first. It is one command and a short failover, it changes no code, and it introduces no new failure modes. I would stop when either the price per unit of power starts bending — around 32 to 64 vCPU on general-purpose hardware — or when availability becomes a requirement.”
4. **Name what horizontal actually buys.** “The reason to go horizontal is not throughput, it is **surviving a machine failure**. One box at 99.9 percent is 8.8 hours a year; two boxes where either can serve is about thirty seconds. No amount of vertical

scaling produces that.”

5. **Name what it costs.** “Horizontal means a load balancer, health checks, and statelessness. For a database it means replication first, which scales reads and introduces lag, and then sharding, which scales writes and costs you cross-shard joins and transactions for ever.”
6. **Say the sentence that separates the two.** “Throughput scales horizontally; latency does not.”

The follow-ups

“**When would you stop scaling up?**” “Two conditions, and either one is enough. The first is the **cost knee**: up to roughly 32 vCPU the price per unit of power is basically flat, so doubling the machine doubles the price — that is a fair deal and I would keep taking it. Past that it bends, and at the top of the range you pay several times what the specification suggests. The second is **availability**, and it applies at any size: one machine is one failure, and if the requirement says we survive a machine dying, then no instance type answers it. There is also a hard third condition, which is a fact rather than a judgement: when the working set no longer fits in the biggest machine’s memory, or the write rate exceeds one machine’s disk, vertical is simply finished.”

“**Just add more servers, right?**” “Only if two things are true. First, the work has to be divisible — if the problem is that one query takes eight seconds, ten machines give me ten eight-second queries running at once, which helps throughput and does nothing for the user waiting. Throughput scales horizontally; latency does not. Second, the machines have to be **stateless**, so any request can go to any of them. If a server keeps the user’s session in its own memory, then adding machines breaks logins for two-thirds of users, and the usual patch — sticky sessions — reintroduces the single point of failure you were trying to remove. So my answer is yes for the app tier, and ‘let me look at the state’ for anything else.”

“**You have one very large database server. What is your availability?**” “Whatever that one machine’s availability is — call it 99.9 percent, which is about 8.8 hours of downtime a year, and that ignores the planned downtime for patching and resizing. And the important point is that **it does not improve if I buy a bigger machine**. Availability is a function of how many independent things can serve the request, not how powerful any one of them is. The cheapest real improvement is a standby replica with automatic failover: it does not add capacity, it turns an outage into a failover of maybe thirty seconds. I would do that before I did anything else, because it is the one change that alters the shape of the risk rather than the size of the box.”

“Why not shard the database now and be done with it?” “Because sharding is not a one-time cost, it is a permanent tax. Once the data is split across machines by a key, you lose transactions that span shards and joins that span shards — so every feature built afterwards has to be designed around the shard key you chose today, and choosing it wrongly is very expensive to undo. Meanwhile the alternative is one command. So my rule is: shard when the workload proves you must — the write rate exceeds one machine, or the working set exceeds the biggest machine’s memory — and not because we expect growth. Companies like Instagram and Shopify ran a single primary with read replicas for years at very serious scale, and each described sharding afterwards as a multi-quarter project.”

“So which is better?” “Neither, and the question is usually the wrong shape — the answer is per tier. In practice almost every real system is a hybrid: the stateless web and application tier is horizontal and auto-scaled on small machines, the cache is horizontal and sharded by key, the primary database is vertical and as large as it needs to be, read replicas are a small horizontal fan-out from it, and media sits in object storage, which is somebody else’s horizontal problem. Naming which tier gets which, and why, is the actual answer.”

“What breaks when you go from one app server to fifty?” “Three things people usually meet in this order. **Connections:** fifty servers with a pool of twenty database connections each is a thousand connections, and many databases start struggling well before that — so you need a connection pooler, which is a new component. **State:** anything held in a single process’s memory — sessions, in-process caches, rate-limit counters, scheduled jobs — is now wrong, either inconsistent or running fifty times. **Observability:** a bug that happens on one machine out of fifty is very hard to find without centralised logging and request tracing. None of those are reasons not to scale out; they are the actual work of scaling out, and they are why it is not free.”

A model answer

Asked: *your database is at capacity. Bigger box, or more boxes?*

“Before choosing, I want to know which problem it is, because the two answers solve different things. **Is one query too slow, or are there too many queries?** If a single statement takes eight seconds, more machines give me more eight-second statements — throughput scales horizontally, latency does not. That case is an indexing or query problem, or possibly a bigger machine, but it is not a sharding problem.

Assuming it is genuinely volume: **I would scale up first, and I would say exactly when I would stop.**

The reason to scale up first is that it is close to free in engineering terms. On a managed database it is a configuration change and a failover — under a minute of disruption if there is a standby, a few minutes if not. No code changes. No new failure modes, because one machine is still one machine. And the price is fair: up to around 32 to 64 vCPU, the cost per unit of power is essentially flat, so doubling the machine costs about double. If we are on an 8 vCPU box at 90 percent CPU, moving to 32 vCPU gives roughly four times the headroom for four times the price, and that will very likely buy two years.

Compare that with the horizontal answer for a database, which is not one thing but two. **Read replicas** scale reads and are relatively cheap, and they cost you replication lag — a user who writes and immediately reads may not see their own write, which is a real product problem you have to design around. **Sharding** scales writes, and it is a multi-quarter project whose real cost is not the migration but everything afterwards: no transactions across shards, no joins across shards, and every future feature constrained by the shard key you pick now.

So I would stop scaling up on one of three triggers. Two are judgements: the price curve bends, or we need headroom faster than a resize can give it. One is a fact: **the working set no longer fits in the largest machine’s memory**, which is around 4 TB at the top of the range, or the write rate exceeds one machine’s disk. Those are not preferences — at that point vertical is finished.

There is one thing I would do immediately regardless, though, because it answers a question the capacity discussion is hiding. **A single database server, at any size, is a single point of failure.** At 99.9 percent that is nearly nine hours a year, and a bigger box does not change it. So I would add a standby replica with automatic failover before anything else. It adds no capacity at all — it turns an outage into a thirty-second failover, which is the change that actually alters the shape of the risk.

And for the application tier the answer is the opposite and much easier: it is stateless, so it is horizontal, behind a load balancer with health checks, sized from peak QPS. Almost every real system is that hybrid — horizontal where the work divides cleanly, vertical where it does not.”

9 Recall card

- **Vertical = a bigger machine. Horizontal = more machines.** Ask first: *is one request slow, or are there too many?* **Horizontal only answers the second — throughput scales horizontally, latency does not.**
- **Default to vertical, and name the stopping condition.** It is one command and a restart, changes no code, and adds no failure modes. Stop when **(a)** the price curve bends — flat cost per vCPU to about **32–64 vCPU**, rising sharply above — **(b)** availability is required, at any size, or **(c)** the working set exceeds the biggest machine (**~4 TB RAM**) or one disk’s write rate (**~50,000 IOPS**).
- **The real prize of horizontal is availability, not throughput.** One machine at 99.9% is **8.8 hours down a year**; two where either can serve is **~32 seconds** — a $1000\times$ change no bigger box can buy. But only if they fail **independently**: same rack, same zone, same bad deploy breaks the arithmetic.
- **Horizontal demands statelessness, a load balancer and health checks.** For a database it is two separate steps: **replication** (scales reads, costs lag) then **sharding** (scales writes, costs cross-shard joins and transactions **for ever**). **Sharding a database you did not need to shard is one of the most expensive mistakes in the subject.**
- **The answer is per tier, not per system.** App tier horizontal and auto-scaled; cache horizontal, sharded by key; **primary database vertical, as large as it needs to be**; read replicas a small fan-out; media in object storage. Going from 1 to 50 app servers breaks three things in order: **connections** (need a pooler), **in-process state** (sessions, counters, cron), and **observability**.

PRACTICE

Practice — What a tree is, and the vocabulary you need

DSA topic: What a tree is, and the vocabulary you need **System design topic:** Vertical versus horizontal scaling

Code these, in this order

One rule for the whole set: **say the convention before you write the base case.** Edges or nodes. Every off-by-one in this topic comes from not deciding that in advance, and it costs four seconds to decide.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Maximum Depth of Binary Tree	LeetCode 104 (Easy)	The five-line shape, and which convention LeetCode uses.
2	Count Complete Tree Nodes	LeetCode 222 (Easy)	The same shape with <code>+</code> instead of <code>max</code> — then the follow-up about doing better.
3	Minimum Depth of Binary Tree	LeetCode 111 (Easy)	The trap: <code>min</code> is wrong when one child is missing.
4	Binary Tree Level Order Traversal	LeetCode 102 (Medium)	Capturing the queue length before the loop, which is the level boundary.

On problem 1, write both conventions

Write it returning `-1` for `None` and again returning `0`. Run both on an empty tree, a single node, and a chain of three. Record all six answers and say which one LeetCode accepts.

On problem 2, then answer the follow-up

The naive version is $O(n)$. The problem says the tree is *complete*. Say what that shape guarantees, and what complexity is achievable because of it. You do not have to write it today — say the idea.

On problem 3, find the trap before running it

Write `1 + min(left, right)` and run it on a tree where the root has only a right child. Record the answer, then say why it is wrong, then fix it. This is the single most-failed easy tree problem.

On problem 4, do not use recursion

Use a queue. Capture `len(queue)` into a variable before the inner loop and say, in one sentence, why that variable is exactly one level.

The vocabulary drill

Define each in one sentence, then give an example from the wedding tree:

1. root, leaf, parent, child, sibling
2. edge, path, level, subtree
3. ancestor, descendant, degree, internal node, forest
4. Then say which two of the above are properties of a *node* and which is a property of the *tree*.

The height-and-depth drill

1. Define depth and height, in opposite directions, in one sentence each.
2. Say which one is computed on the way up and which is carried down, and why.
3. For a given tree, state the depth of three nodes and the height of three nodes.
4. Say why a leaf's height is always 0 whatever its depth.
5. Say what "the depth of the tree" should have been.

The convention drill

1. Give both base cases and what each returns for a single node.
2. Say which one LeetCode 104 expects.
3. Write the sentence you would say to an interviewer to settle it in four seconds.
4. Mix the two inside one function and describe how the bug shows up.

The shape drill

1. Write the five-line shape that every tree function takes.
2. Write `height`, `count_nodes` and `count_leaves` against it, and say what differs in each.
3. Say what the base case always is, and why it is not defensive coding.
4. Say why `count_leaves` cannot return 1 for `None`, with the input that proves it.

The two-facts drill

1. State the edge-count fact and prove it in one line.
2. State the one-path fact and say what it lets you leave out of every traversal.

3. Build a structure where a node has two parents, run `count_nodes`, and say what goes wrong.
4. Say what such a structure is called instead.

The shapes drill

1. Draw full, complete, perfect, balanced and degenerate, each with five to seven nodes.
2. Give the node count of a perfect tree of height `h`.
3. Say which shape lets a tree live in an array, and name the structure that uses it.
4. Say which shape makes a tree behave like a linked list, and how you would build one by accident.

The cost drill

1. State the time and space of a full tree walk.
2. Give the stack depth for a balanced and a degenerate tree of a million nodes.
3. Say which traversal costs `O(height)` and which costs `O(width)`, and what the widest level of a perfect tree holds.
4. Explain why `is_balanced` as written is `O(n2)` and say what the fix is called.
5. Give the height of a perfect tree for $n = 1,000$, a million and a billion.

The break-it drill

Trigger each and record the exact output or error:

1. `height` with no `None` check.
 2. `count_leaves` returning 1 for `None`.
 3. A recursive walk on a chain of 10,000 nodes.
 4. Two subtrees sharing one node, then counting.
 5. `min(left, right)` for minimum depth on a one-sided tree.
-

The two-directions drill

1. Define both directions in one sentence each.
2. Give three things vertical scaling buys and three it costs.
3. Give three things horizontal scaling buys and three it demands.
4. State the one-sentence rule for choosing.

The first-question drill

1. Say the question you ask before anything else, and why.
2. Give an example where horizontal scaling helps nothing at all.
3. State the sentence about throughput and latency.
4. Say what you would do instead for the case where it helps nothing.

The knee drill

1. Reproduce the price-per-vCPU table and say where it stops being flat.
2. State the three stopping conditions for vertical scaling.
3. Say which of the three is a fact rather than a judgement.
4. Give the practical top-end RAM figure and say what it implies for most companies' databases.

The availability drill

Compute each, showing the working:

1. Downtime per year at 99%, 99.9%, 99.99%.
2. Combined availability of two independent machines at 99.9%, either of which can serve.
3. The improvement factor from adding that second machine.
4. Say what assumption the calculation makes, and three ways real deployments break it.

The per-tier drill

For each tier, say vertical or horizontal and give the reason:

1. Stateless application servers.
2. The cache.
3. The primary database.
4. Read replicas.
5. Media files.

Then say why “vertical or horizontal” is the wrong shape of question at the system level.

The database drill

1. Name the two stages of scaling a database horizontally, in order.
2. Say what each one scales and what each one costs.

3. Say what sharding takes away permanently.
4. Give the sizing comparison for a database at 4,000 writes/second on 8 vCPU, both ways.
5. Say what you would do first regardless of the capacity answer, and why.

The fifty-servers drill

Name the three things that break when going from one app server to fifty, in the order you meet them, and say what each one's fix is. Then say why none of them is a reason not to scale out.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is the height of this tree? What is the depth of that node?* Both defined in opposite directions, the convention declared in advance, the up-versus-down asymmetry named, the numbers given, and the cost stated with the degenerate-tree caveat.
2. *Your database is at capacity. Bigger box, or more boxes?* The first question about slow-versus-many, vertical first with the three stopping conditions, the availability number that no bigger box can buy, what sharding costs permanently, and the standby replica you would add regardless.
3. *What makes something a tree rather than a graph?* One parent each and no cycles, therefore exactly one path from the root, therefore no visited set and therefore natural recursion — plus the `n - 1` edges fact with its one-line proof.

Before you move on

- [] I can define all fifteen vocabulary terms without hesitating.
- [] I can define depth and height in opposite directions and say which is which.
- [] I know which one is computed upward and which is passed downward.
- [] I say the convention out loud before writing a base case.
- [] I can write the five-line shape that every tree function takes.
- [] I can write `height`, `count_nodes` and `count_leaves` and say what differs.
- [] I can prove the `n - 1` edges fact in one line.
- [] I can say what the one-path property lets me leave out of every traversal.

- [] I can draw all five named shapes.
- [] I know which shape lets a tree live in an array.
- [] I can state the stack depth for balanced and degenerate trees of a million nodes.
- [] I know why `is_balanced` as written is $O(n^2)$.
- [] I never assume a binary tree is balanced or a search tree.
- [] I can state the first question to ask before choosing a scaling direction.
- [] I can say the sentence about throughput and latency.
- [] I can give the three stopping conditions for vertical scaling.
- [] I can compute the availability of one machine versus two and give the factor.
- [] I can assign a direction to each of the five tiers with a reason.
- [] I can say what sharding costs permanently, and why it is not a migration cost.
- [] I answered all three questions above out loud.

Binary trees in code

DATA STRUCTURES AND ALGORITHMS

Binary trees in code

You can define a node class and build a tree by hand for testing.

SYSTEM DESIGN

Load balancers

You can name the layer, the algorithms, and what happens on health-check failure.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Build this tree in code. Now print it.*
- *How does a load balancer decide which server gets the request?*

1 What this is, and why they ask it

Yesterday was the words. Today is the four lines of Python that hold a tree, and the two things you will do with them in every single tree interview: **build a small tree by hand to test on**, and **read the tree the interviewer has written on the whiteboard**.

Three sentences. A binary tree in code is one class with a value and two references, and there is no `Tree` object at all — **a tree is a reference to its root node**, and `None` is the empty tree. The interviewer will hand you a tree written as a flat list like `[3, 9, 20, None, None, 15, 7]`, and that list is the tree read out **level by level, including the gaps**, which is a format worth being able to convert both ways without thinking. And a node has no reference to its parent unless you add one, which is the single most consequential fact about the structure.

They ask it because a candidate who cannot construct a test tree in thirty seconds ends up debugging their algorithm on the interviewer's one example, in their head, with no way to try anything. It is the plumbing, and being slow at plumbing costs you the part of the round where you were supposed to be thinking.

2 The story

The school sent a message home in June saying that if the school ever had to shut suddenly, there was now a chain for telling people, and Sudha's mother had been put in it.

The way it worked was this. The headmistress rang two people. Those two each rang two more. Those four each rang two more, and so on down, until everybody had been rung. Nobody had to make more than two calls, and the whole school of about two hundred and forty families would know inside twenty minutes.

Sudha's mother was one of the four in the third row, and she had two names under her: Mrs Pillai and a number that belonged to a family in the flats behind the temple.

In August they tried it out on a Saturday morning to see whether it worked, and it mostly did, except for one thing.

Somebody in the second row had moved to Bangalore in July. Her two names had been given to somebody else, but nobody had told the woman above her, so the woman above her rang a number that no longer worked, said "no answer", and

stopped. Eleven families never got the call.

At the meeting afterwards they decided that everyone should have the whole chain, not just their own two names, so that if a call did not connect you could go round it.

That turned out to be harder than anybody expected, because saying the chain out loud on the phone is not easy. Sudha's mother tried it with a neighbour and got confused within about a minute, because she was going down one branch all the way to the bottom and then coming back up, and the neighbour could not tell where she was.

The teacher who had made the chain in the first place had a better way. She read it **row by row**. The headmistress. Then the two people she calls. Then those two people's four. Then the eight.

And the important part, which she was strict about, was that when somebody in a row had nobody under them, you still said so. You said "nobody" and moved on. Because if you skipped them, the person listening would put the next name in the wrong place and the whole row after that would be shifted across by one.

Nobody, nobody, then the next two names.

Read that way, on the phone, in order, with the gaps said out loud, a person could write down the entire chain correctly the first time.

3 The idea in plain English

The phone chain is a binary tree, and the teacher's rules are the two things you have to be able to do with one in code.

The node, which is the whole data structure

```
class TreeNode:
    def __init__(self, value: int) → None:
        self.value = value
        self.left: "TreeNode | None" = None
        self.right: "TreeNode | None" = None
```

Four lines. **There is no `Tree` class.** A tree is a node — the root — and every node is the root of its own subtree. `None` means “no subtree here”, which is both “this branch is empty” and “the whole tree is empty”, depending on where you are.

That double meaning is deliberate and useful: every tree function's base case is `if node is None`, and it correctly handles both an empty tree and the bottom of a branch with the same line.

LeetCode's version has a different constructor, and knowing it saves you thirty seconds:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # `val`, not `value`
        self.left = left
        self.right = right
```

`val`, and the children can be passed in. Use their names when solving their problems; nothing is more irritating than an `AttributeError` on `.value` at minute twenty.

Building one by hand

Two ways, and you want the second.

Nested, which reads like the tree looks:

```
root = TreeNode(3,
                TreeNode(9),
                TreeNode(20, TreeNode(15), TreeNode(7)))
```

Indent it so the shape is visible and it is genuinely readable. One expression, no temporary names.

Assignment, which is what you write when the shape is irregular:

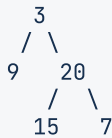
```
root = TreeNode(3)
root.left = TreeNode(9)
root.right = TreeNode(20)
root.right.left = TreeNode(15)
```

Verbose, but you can put a node anywhere without restructuring the expression. Use **nested for a tree you are typing once**, and **assignment for a tree you are going to modify**.

The list format, which is what the interviewer will hand you

```
[3, 9, 20, None, None, 15, 7]
```

That is the tree **read level by level, left to right**, with `None` for every missing child — the teacher's rule about saying "nobody" out loud.



level 0: 3

level 1: 9, 20

level 2: None, None, 15, 7

← 9's two missing children ARE recorded

The None s are load-bearing. Drop them and 15 and 7 become 9's children instead of 20's, which is a different tree. That is exactly the mistake the neighbour made with the phone chain.

Two things to know about the format, because both come up:

- **LeetCode trims trailing None s.** `[1, 2]` means a root with only a left child; there is no need to write `[1, 2, None]`.
- **It is not the array-index formula.** For a *complete* tree you can store a tree in an array where node `i`'s children are at `2i + 1` and `2i + 2` — that is the **heap** layout, and it is compact. But for a sparse tree that wastes a catastrophic amount of space: a degenerate tree of 20 nodes needs an array of about a million entries. **LeetCode's format skips the gaps below a None**, so it stays proportional to the number of nodes.

The fact that decides everything: no parent pointer

```

node.left      # yes
node.right     # yes
node.parent    # AttributeError – it does not exist
  
```

A node knows its children and **nothing about who points at it**. Three consequences that come up constantly:

1. **You cannot go up.** Anything needing an ancestor either passes information down as an argument, or reconstructs the path on the way back up the recursion.
2. **You cannot delete a node given only that node.** You need its parent to change a pointer, so deletion functions take the root and search from the top.
3. **A node cannot know its own depth.** Depth is carried down; *yesterday's* asymmetry, restated as a fact about the object.

You *can* add a `parent` field, and some designs do — a doubly linked tree. The price is that every insertion and every removal has to maintain it, and a stale parent pointer is a silent bug. **Do not add one in an interview unless the problem needs it; say**

instead that you would carry the parent as an argument.

Printing a tree, which you will want within five minutes

Three ways, in increasing order of usefulness:

```
print(root.value, root.left.value, root.right.value)    # useless past 3 nodes
```

```
def as_list(node):                                     # matches the interviewer's format
    ...                                                # level order with Nones
```

```
def show(node, indent=0):                             # sideways, and instantly readable
    if node is None:
        return
    show(node.right, indent + 1)
    print("    " * indent + str(node.value))
    show(node.left, indent + 1)
```

That third one is nine lines and it is the best thirty seconds you can spend in a tree interview. It prints the tree rotated ninety degrees — right subtree on top, root in the middle, left subtree below — so you can actually see what your code built. Write it once, remember it.

The equality trap

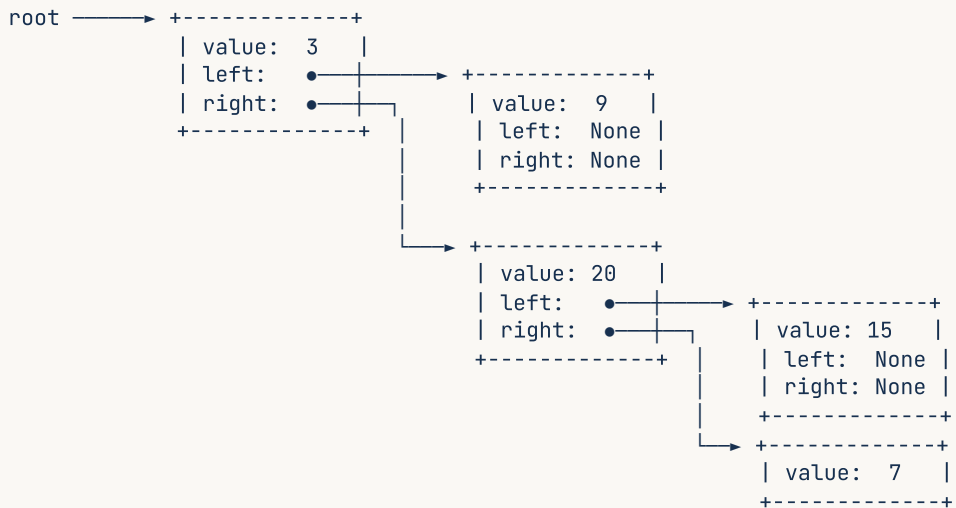
```
TreeNode(1) == TreeNode(1)                          # False
```

Two nodes with the same value are different objects, and Python compares objects by identity unless you say otherwise. That is *correct* — a tree is about structure, not values — and it is why [comparing two trees](#) is its own problem with its own recursion, rather than a `=`.

The same applies to `in`, to `set()`, and to using a node as a dictionary key: they all work, and they all work **by identity**, which is usually what you want and occasionally is not.

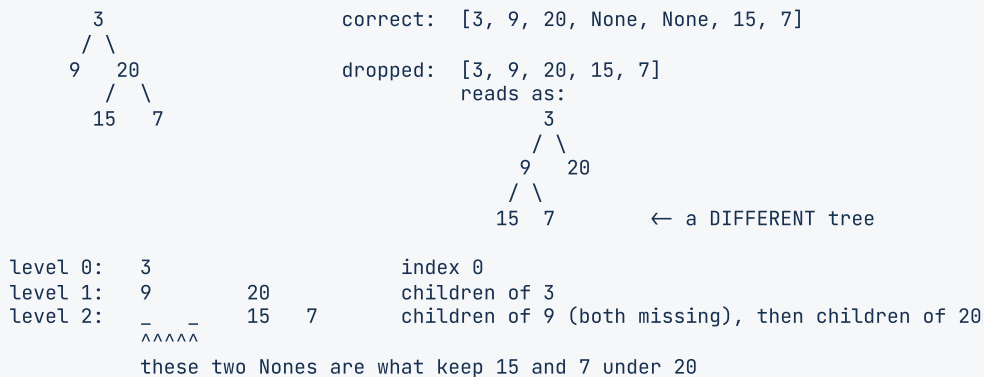
4 The picture

The object graph, drawn as it is in memory. This is what “a tree is a reference to its root” means.



Nothing points UPWARD. Every arrow goes down.
 `root` is an ordinary variable; lose it and the whole tree is unreachable.

The list format, and why the `None`s matter:



The sideways printer's output, which is what you actually look at while debugging:

```
show(root) prints:
```

```
      7
     20
    15
   3
  9
```

read it with your head tilted left:

- the deeper a node, the further right it is printed
- the RIGHT subtree appears ABOVE, the left below
- a 12-node tree fits on a screen; a horizontal drawing does not

Building the same tree three ways:

NESTED

```
root = TreeNode(3,
  TreeNode(9),
  TreeNode(20,
    TreeNode(15),
    TreeNode(7)))
```

one expression
shape is visible
hard to change

ASSIGNMENT

```
root = TreeNode(3)
root.left = TreeNode(9)
root.right = TreeNode(20)
root.right.left = TreeNode(15)
root.right.right = TreeNode(7)
```

one node per line
easy to modify
verbose

FROM A LIST

```
root = from_list(
  [3,9,20,None,None,15,7])
```

one line, and it
matches the problem
write it ONCE, reuse

5 The code, built step by step

Step 1 — the node, with LeetCode's names

```
class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right
```

val, and **children in the constructor**. Matching their names means you can paste a solution straight in, and it makes the nested building style possible in one expression.

Step 2 — `__repr__`, which pays for itself immediately

```
def __repr__(self) → str:
    return f"TreeNode({self.val})"
```


Without it, printing a list of nodes gives you `[<__main__.TreeNode object at 0x7f8b...>, ...]`, which tells you nothing. One line, and every debugging print becomes readable.

Step 3 — from a list, which you write once and reuse for ever

```
def from_list(values):
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        ...
```

The idea: **walk the list left to right, and give each value in turn to the next node that is waiting for a child.** A queue holds the nodes waiting. Every node consumes exactly two entries from the list — its left and its right — and a `None` entry means “no child”, so nothing is enqueued for it.

That “nothing is enqueued for a `None`” is exactly why the format stays compact: a missing node has no children to describe.

Step 4 — back to a list, which is how you check your work

```
while queue:
    node = queue.popleft()
    if node is None:
        out.append(None)
        continue
    out.append(node.val)
    queue.append(node.left)
    queue.append(node.right)
```

Note this enqueues `None`s deliberately, so that the gaps are recorded. Then trim the trailing `None`s at the end, to match LeetCode.

Being able to go both ways is what lets you test. Build from the interviewer’s list, run your function, convert back, and compare with what they expect.

Step 5 — the sideways printer

```
def show(node, indent=0):
    if node is None:
        return
    show(node.right, indent + 1)           # right subtree ABOVE
    print("    " * indent + str(node.val))
    show(node.left, indent + 1)           # left subtree BELOW
```

Right, then self, then left — a **reverse in-order** walk, which is [tomorrow's](#) subject arriving early. Nine lines, and it is the difference between debugging by reasoning and debugging by looking.

The complete solution

```
from collections import deque

class TreeNode:
    """A binary tree node. Field names match LeetCode's (`val`, not `value`)
    so that solutions paste straight in.

    There is no Tree class. A tree IS a reference to its root, and `None` is
    the empty tree – which is also what "no child here" means, which is why
    one base case handles both.
    """

    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"      # or debugging prints are unreadable

def from_list(values: list[int | None]) → TreeNode | None:
    """Build a tree from LeetCode's level-order-with-gaps format.

    [3, 9, 20, None, None, 15, 7] →
        3
       / \
      9  20
     /  \
    15   7

    Walk the list left to right; each waiting node consumes the next TWO
    entries as its children. A None entry creates no node and therefore
    enqueues nothing, which is why the format stays proportional to the
    number of real nodes rather than to 2^height.

    Write this ONCE. You will use it in every tree problem you ever practise.
    """
    if not values or values[0] is None:
        return None

    root = TreeNode(values[0])
    queue = deque([root])
    i = 1

    while queue and i < len(values):
        node = queue.popleft()

        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
            i += 1

        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
            i += 1

    return root
```

```

def to_list(root: TreeNode | None) → list[int | None]:
    """The inverse. Enqueues None deliberately so gaps are recorded, then
    trims the trailing Nones to match LeetCode's output."""
    if root is None:
        return []

    out: list[int | None] = []
    queue: deque[TreeNode | None] = deque([root])

    while queue:
        node = queue.popleft()
        if node is None:
            out.append(None)
            continue
        out.append(node.val)
        queue.append(node.left)
        queue.append(node.right)

    while out and out[-1] is None:        # LeetCode trims trailing gaps
        out.pop()
    return out

```

```

def show(node: TreeNode | None, indent: int = 0) → None:
    """Print the tree rotated 90 degrees: right subtree above, left below.

    Nine lines, and the single most useful debugging tool in this topic –
    a 12-node tree fits on a screen where a horizontal drawing does not.
    """
    if node is None:
        return
    show(node.right, indent + 1)
    print(" " * indent + str(node.val))
    show(node.left, indent + 1)

```

```

def build_nested() → TreeNode:
    """One expression, indented so the shape is visible.
    Use this for a tree you type once."""
    return TreeNode(3,
                    TreeNode(9),
                    TreeNode(20,
                            TreeNode(15),
                            TreeNode(7)))

```

```

def build_by_assignment() → TreeNode:
    """One node per line. Use this when the shape is irregular or when you
    are going to modify it afterwards."""
    root = TreeNode(3)
    root.left = TreeNode(9)
    root.right = TreeNode(20)
    root.right.left = TreeNode(15)
    root.right.right = TreeNode(7)
    return root

```

```

def find(node: TreeNode | None, target: int) → TreeNode | None:
    """Find a node by value in a PLAIN binary tree: O(n).

```

You must search both sides, because nothing about the values tells you which way to go. That is only $O(\log n)$ in a binary SEARCH tree.

```

"""
if node is None:
    return None
if node.val == target:
    return node
return find(node.left, target) or find(node.right, target)

def find_with_parent(node: TreeNode | None, target: int,
                    parent: TreeNode | None = None
                    ) → tuple[TreeNode | None, TreeNode | None]:
    """A node has NO parent pointer, so if you need the parent you carry it
    down as an argument. This is the standard workaround, and it is what you
    should say rather than adding a `parent` field."""
    if node is None:
        return None, None
    if node.val == target:
        return node, parent
    found, p = find_with_parent(node.left, target, node)
    if found:
        return found, p
    return find_with_parent(node.right, target, node)

def insert_level_order(root: TreeNode | None, value: int) → TreeNode:
    """Add a node at the first free position, level by level.

    This is how you grow a COMPLETE tree, and it is the only "insert" that
    makes sense for a plain binary tree – a binary SEARCH tree inserts by
    value instead, which is day 107.
    """
    node = TreeNode(value)
    if root is None:
        return node

    queue = deque([root])
    while queue:
        current = queue.popleft()
        if current.left is None:
            current.left = node
            return root
        queue.append(current.left)
        if current.right is None:
            current.right = node
            return root
        queue.append(current.right)
    return root

def equal(a: TreeNode | None, b: TreeNode | None) → bool:
    """Two nodes with the same value are DIFFERENT objects, so `==` is
    identity and structural comparison needs its own recursion. Day 103."""
    if a is None and b is None:
        return True
    if a is None or b is None:
        return False
    return a.val == b.val and equal(a.left, b.left) and equal(a.right, b.right)

def copy_tree(node: TreeNode | None) → TreeNode | None:
    """A deep copy. Note that `root2 = root1` copies the REFERENCE – both
    names then point at the same tree and a change through one is visible
    through the other."""
    if node is None:

```

```

        return None
    return TreeNode(node.val, copy_tree(node.left), copy_tree(node.right))

if __name__ == "__main__":
    values = [3, 9, 20, None, None, 15, 7]
    root = from_list(values)

    print(to_list(root))                # [3, 9, 20, None, None, 15, 7]
    print(to_list(root) == values)      # True - the round trip works

    show(root)
    #      7
    #     20
    #    15
    #   3
    #  9

    print(equal(root, build_nested()))   # True
    print(equal(root, build_by_assignment())) # True

    print(TreeNode(1) == TreeNode(1))    # False - identity, not value
    print(equal(TreeNode(1), TreeNode(1))) # True - structure

    node, parent = find_with_parent(root, 15)
    print(node, parent)                  # TreeNode(15) TreeNode(20)

    print(to_list(from_list([1, 2])))    # [1, 2] - trailing Nones trimmed
    print(from_list([]), from_list([None])) # None None

    lopsided = None
    for v in (1, 2, 3, 4, 5, 6):
        lopsided = insert_level_order(lopsided, v)
    print(to_list(lopsided))              # [1, 2, 3, 4, 5, 6]

    copy = copy_tree(root)
    copy.val = 999
    print(root.val, copy.val)             # 3 999 - genuinely separate

    alias = root
    alias.val = 42
    print(root.val)                      # 42 - the reference trap

```

6 What it costs

Space per node

Python object header	~16 bytes
__dict__ for three attributes	~104 bytes
the three references themselves	included above

one TreeNode	~120 bytes

1,000,000 nodes × ~120 B = ~120 MB

A million-node tree is about 120 MB in Python, and almost all of it is object overhead rather than data. `__slots__` cuts it substantially:

```
class TreeNode:
    __slots__ = ("val", "left", "right")    # ~56 bytes per node instead of ~120
```

Worth knowing and worth mentioning; not worth writing in an interview unless memory is the question.

Compare with the array layout that a [heap](#) uses:

pointer-based tree, 1M nodes	~120 MB	works for ANY shape
array-based complete tree, 1M	~8 MB	only for a COMPLETE tree

Fifteen times smaller, and it only works for one shape. That is the trade the heap makes, and it is why heaps are stored in arrays and general trees are not.

The operations

<code>from_list(k values)</code>	<code>O(k)</code>	each value handled once
<code>to_list(n nodes)</code>	<code>O(n)</code>	plus up to <code>n+1</code> Nones enqueued
<code>show(n nodes)</code>	<code>O(n)</code>	
<code>find in a plain tree</code>	<code>O(n)</code>	both sides must be searched
<code>insert_level_order</code>	<code>O(n)</code>	scans levels for the first gap
<code>copy_tree</code>	<code>O(n)</code>	
<code>equal</code>	<code>O(n)</code>	stops early on the first difference

Everything is `O(n)`, and `find` being `O(n)` is the one to say out loud, because people expect `O(log n)` from anything shaped like a tree. You only get `O(log n)` in a binary *search* tree, and only while it is balanced.

Why the array format would be a disaster for sparse trees

The index formula — children of `i` at `2i + 1` and `2i + 2` — needs a slot for every possible position:

array size needed = $2^{(\text{height}+1)} - 1$		
a COMPLETE tree of 1,000 nodes	→ ~1,024 slots	99% used
a DEGENERATE tree of 20 nodes	→ $2^{21} = 2,097,151$	0.001% used
a degenerate tree of 40 nodes	→ $\sim 1.1 \times 10^{12}$	impossible

Forty nodes, a trillion slots. LeetCode's format avoids this entirely by not describing the children of a `None`, so its length is proportional to the number of real nodes plus their missing children — `O(n)`, not `O(2n)`.

Recursion depth

```
show, copy_tree, equal, find: 0(height) stack  
  
balanced 1M-node tree      ~20 frames  
degenerate 10,000-node    10,000 frames → RecursionError
```

Same warning as yesterday, and it applies to every helper on this page.

7 The traps

Trap 1 — `val` versus `value`

```
root.value
```

```
AttributeError: 'TreeNode' object has no attribute 'value'
```

LeetCode's class uses `val`. If you define your own with `value` and then paste in one of their snippets, or write a solution against `val` and test with your own class, you get this at minute twenty when you least want it. **Pick `val` and never think about it again.**

Trap 2 — dropping the `None`s from the list format

```
[3, 9, 20, 15, 7]          is NOT   [3, 9, 20, None, None, 15, 7]
```

The first puts 15 and 7 under 9; the second puts them under 20. **No error, a different tree, and every answer wrong.** The gaps are part of the description, exactly as the teacher insisted.

Trap 3 — assignment copies the reference, not the tree

```
backup = root  
backup.val = 999  
print(root.val)                                # 999 — there was never a backup
```

`backup` and `root` are two names for one object. A real copy is a recursive walk, or `copy.deepcopy(root)`. This is the same reference trap as [day 091's](#) `current[:]`, wearing a tree costume.

Trap 4 — expecting `==` to compare structure

```
TreeNode(1) == TreeNode(1)           # False
build_nested() == build_by_assignment() # False, and they ARE the same tree
```

Python compares by identity. Structural comparison is its own recursion. This trips people writing tests more than it trips people writing solutions.

Trap 5 — reaching for `.parent`

```
node.parent
```

```
AttributeError: 'TreeNode' object has no attribute 'parent'
```

There is no parent pointer. **Carry the parent down as an argument** — which is four extra characters — rather than adding a field you then have to maintain on every insertion and deletion.

Trap 6 — assuming the values are unique

```
def find(node, target): ...
```

Nothing says the values in a binary tree are distinct. `find` returns the first match in its traversal order, which may not be the one meant. **Ask** — “are values unique?” — because the answer changes several problems, including [lowest common ancestor](#).

Trap 7 — the index formula on a sparse tree

```
tree = [None] * (2 ** (height + 1) - 1)    # for a degenerate tree of 40 nodes
```

```
MemoryError
```

The array layout only works for complete trees. For anything else it is exponential in the height, and forty nodes is already a trillion slots.

Trap 8 — building a test tree that is too symmetric

```
root = TreeNode(1, TreeNode(2), TreeNode(3))
```

Perfectly balanced, three nodes, both children present everywhere. It will pass almost any wrong solution. **A good test tree is lopsided, has a node with exactly one child, and has a value repeated** — because the one-child case is where `min`, `max`

and null-handling bugs live.

```
root = from_list([1, 2, 3, None, 4, None, None, 5]) # asymmetric, one-child, deep
```

8 In the interview

How it gets asked

- The plumbing question, usually first: “Build this tree in code.”
- The follow-up: “Now print it so I can see it.”
- The format question: “What does `[3, 9, 20, null, null, 15, 7]` mean?”
- The design probe: “Would you add a parent pointer?”
- The trap: “Find the node with value 15.” — checking whether you say `O(n)` or wrongly say `O(log n)`.

What to say out loud, in the first ninety seconds

1. **Say what a tree is, as an object.** “There is no `Tree` class — a tree is a reference to its root node, and `None` is the empty tree. That is also what ‘no child here’ means, which is why one base case handles both.”
2. **Use their field names.** “I will use `val`, `left` and `right`, matching the standard definition.”
3. **Read the list format out loud.** “That list is the tree level by level, left to right, with `null` for missing children. The nulls matter — dropping them moves 15 and 7 under 9 instead of 20.”
4. **Say the parent fact before it bites.** “Nodes have no parent pointer, so anything needing an ancestor either carries it down as an argument or reconstructs it on the way back up.”
5. **Build a deliberately awkward test tree.** “I will make it lopsided with a node that has exactly one child, because that is where the null-handling bugs are.”
6. **Write the printer.** “Let me spend thirty seconds on a sideways print so we can both see what the code is doing.”

The follow-ups

“What does `[3, 9, 20, null, null, 15, 7]` mean?” “It is the tree read level by level, left to right, and every missing child is written as `null`. So the root is 3; level one is 9 and 20; level two is 9’s two children, both missing, then 20’s children, 15 and 7. The nulls are load-bearing — if you drop them, 15 and 7 become children of 9 and it is a completely different tree with no error to tell you. Two details: trailing nulls are

trimmed, so `[1, 2]` is a root with only a left child. And it is **not** the array-index layout where node `i`'s children are at `2i + 1` and `2i + 2` — that one needs a slot for every possible position, so a degenerate tree of forty nodes would need about a trillion entries. This format describes no children for a null, so its length stays proportional to the number of real nodes.”

“Would you add a parent pointer?” “Not by default. It is genuinely useful for a few things — walking up from a node, deleting a node you already hold, finding a successor in a BST iterator — but it has to be maintained on every insertion, deletion and rotation, and a stale parent pointer is a silent bug that shows up somewhere far from where it was created. In an interview I would instead carry the parent down as an argument, which is four characters and cannot go stale. I would add the field if the problem required repeated upward traversal from arbitrary nodes, and then I would say that maintaining it is now part of every mutation.”

“Find the node with value 15.” “`O(n)`, and I want to be explicit about that because the shape suggests otherwise. In a plain binary tree nothing about the values tells me which way to go, so I have to search the left subtree and, if it is not there, the right — worst case every node. It is only `O(log n)` in a binary **search** tree, where the ordering lets one comparison discard half the tree, and even then only while the tree is balanced. One thing I would ask: are the values unique? If not, ‘the node with value 15’ is ambiguous and I would need to know whether you want the first in traversal order or all of them.”

“How would you test your solution?” “I write two helpers first and reuse them for every tree problem. `from_list`, which builds a tree from the level-order format you gave me, and `to_list`, which converts back — so I can build your example, run my function, convert the result, and compare against what you expect. Then a nine-line sideways printer that draws the tree rotated ninety degrees, right subtree above and left below, so I can see what I actually built rather than reason about it. And I deliberately make my test tree awkward: lopsided, with at least one node that has exactly one child and one repeated value. A perfectly balanced three-node tree passes almost any wrong solution.”

“How much memory does a million-node tree use?” “In Python, roughly 120 MB — about 120 bytes a node, and most of that is object overhead rather than the data. `__slots__` would take it to about 56 bytes a node. Worth comparing against the array layout used by a heap: a complete tree of a million integers is about 8 MB with no pointers at all, so fifteen times smaller — but that only works because a heap is always complete. The moment the tree can be sparse, the array layout needs a slot for every

possible position and becomes exponential in the height. That trade — compact but shape-restricted, versus general but pointer-heavy — is the whole reason heaps and binary search trees are stored differently.”

A model answer

Asked: *build this tree in code, then print it.*

“First, what I am building. There is no `Tree` class — **a tree is a reference to its root node**, and `None` is the empty tree. That double meaning is useful rather than sloppy: `None` also means ‘no child here’, so a single base case, `if node is None`, correctly handles both the empty tree and the bottom of every branch.

The node is four lines: a value and two references, `left` and `right`. I will call the value field `val` rather than `value`, because that is the standard definition and it means anything you hand me pastes straight in. I will also add a one-line `__repr__`, because without it printing a list of nodes gives me memory addresses and tells me nothing.

The tree you have written as `[3, 9, 20, null, null, 15, 7]` is level order, left to right, with a `null` for each missing child. So: root 3; then its children 9 and 20; then 9’s two children, which are both missing, and then 20’s children, 15 and 7. **Those two nulls are doing real work** — if I dropped them the list would read as 15 and 7 being 9’s children, which is a different tree and there would be no error to warn me.

I will write `from_list` once rather than typing trees by hand, because I will reuse it for every tree problem. It walks the list left to right with a queue of nodes waiting for children; each waiting node consumes the next two entries. A `null` entry creates no node and therefore enqueues nothing, which is exactly why this format stays proportional to the number of real nodes instead of exploding like the array-index layout would.

Then thirty seconds on a printer, because I would rather look at the tree than reason about it. Nine lines: recurse right with one more level of indent, print the value, recurse left. That draws the tree rotated ninety degrees — right subtree above, left below, depth going rightwards — and a dozen nodes fit on a screen where a proper drawing would not.

Two things I would flag before we go further. **Nodes have no parent pointer**, so anything that needs an ancestor either carries it down as an argument or rebuilds the path on the way back up — I would not add a `parent` field unless the problem needs repeated upward walks, because then every mutation has to maintain it. And when I make my own test tree I will make it deliberately awkward: lopsided, with a node that has exactly one child, because a symmetric three-node tree will pass almost any wrong solution and the one-child case is where the bugs are.”

9 Recall card

- **Four lines:** `val`, `left`, `right`. There is no `Tree` class — a tree is a reference to its root, and `None` is both “empty tree” and “no child here”, which is why one base case handles both. Use `val`, not `value`, and add a one-line `__repr__` or every debugging print is a memory address.
- `[3, 9, 20, null, null, 15, 7]` is level order with a `null` for every missing child, and the nulls are load-bearing — drop them and 15 and 7 move under 9, silently. Trailing nulls are trimmed. It is **not** the $2^{i+1} / 2^{i+2}$ array layout: that needs $2^{(h+1)} - 1$ slots, so a **degenerate tree of 40 nodes would need $\sim 10^{12}$ entries**.
- Write `from_list` and `to_list` once and reuse them for every tree problem, plus the **nine-line sideways printer** (recurse right, print, recurse left) — the best thirty seconds available in a tree round. Make test trees **lopsided, with a one-child node and a repeated value**; a symmetric three-node tree passes almost any wrong solution.
- **There is no parent pointer.** So you cannot go up, cannot delete a node given only that node, and a node cannot know its own depth. **Carry the parent down as an argument** rather than adding a field that every mutation must maintain.
- `TreeNode(1) == TreeNode(1)` is `False` — Python compares identity, so structural equality is its own recursion — and `backup = root` **copies the reference, not the tree**. `find` in a plain binary tree is $O(n)$, not $O(\log n)$. A million nodes is **~ 120 MB** in Python (~ 56 with `__slots__`), against **~ 8 MB** for the same as a complete-tree array.

1 What this is, and why they ask it

A **load balancer** is the machine that sits in front of your servers, receives every request, and sends it to one of them. Clients only ever talk to it; they never know how many servers exist or which one answered.

Three sentences. It exists for two separate reasons that get muddled — **spreading load**, which is the obvious one, and **hiding failure**, which is the one that actually matters, because a load balancer that keeps sending requests to a dead server is worse than having no load balancer at all. How it chooses is a short list of algorithms, and **the right one depends on whether your requests are all the same size**. And the whole thing rests on **health checks**, which are the part candidates forget and interviewers always ask about.

They ask it because it is the component that makes [yesterday's](#) horizontal scaling possible, and because “round robin” is where most candidates stop. The follow-up is always the same: *what happens when one server is slow rather than dead?* Round robin keeps sending it one request in every `n`, for ever, and every one of those users waits. Knowing that failure mode, and which algorithm fixes it, is the difference between having read about load balancers and having thought about them.

2 The story

The ration shop on Thursday mornings had four counters open and one man standing at the door, and the man at the door was the whole system.

His name was Pandian and he had been doing it for years. People came in with their cards and he pointed — you, three. You, one. You, one again.

He was not being fair in the sense of taking turns. He had tried taking turns, years ago, and it had not worked, because the counters were not the same.

Counter one was the fastest man in the shop. Counter four was a boy who had started in March and was still checking every entry twice. If Pandian sent one person to each counter in turn, the queue at four grew all morning while the queue at one emptied and the fastest man stood there doing nothing.

So what he actually did was look. He looked at the four queues and sent the next person to the shortest one. That was it, and it handled everything by itself — if the boy was slow, his queue stayed long and Pandian stopped sending him people, without ever having to decide that the boy was slow.

There was one exception he made. If somebody came in with a big monthly card for a whole joint family, he sent them to counter one whatever the queues looked like, because that transaction took eight minutes at counter one and twenty-five at counter four.

The part that took him longest to work out was what to do about a counter that had stopped.

Twice a morning somebody would leave their counter — tea, or the toilet, or a supervisor calling them over — and Pandian, watching from the door twenty feet away, would not notice for a while. He kept sending people. Those people stood there. And then they came back to the door, angry, and he had to put them at the front of another queue, which made the people in that queue angry.

His solution was to glance at each counter every so often rather than only at the queues. If a counter had not moved at all since the last time he looked, he stopped sending people there.

He learned to be careful about two things with that. If he checked too rarely, people waited at a dead counter for a long time. If he reacted to the very first glance, he would pull people away from a counter whose clerk had just bent down to pick up a dropped pen, and then the queue at the other three jumped for no reason. So he waited until he had seen it still twice in a row before he stopped sending people, and he waited until he had seen it working twice in a row before he started again.

3 The idea in plain English

Pandian is a load balancer, and he arrived at every important part of one by trial and error.

- Pointing people at counters is **load balancing**. Pandian is the load balancer; the counters are the **backends** (also called *servers*, *instances*, or the *target group*).
- Taking strict turns is **round robin**.
- Sending to the shortest queue is **least connections**.
- The joint-family card going to counter one is **request-aware routing** — some requests are much more expensive than others.

- Glancing at each counter is a **health check**.
- Waiting for two bad glances before pulling a counter out, and two good ones before putting it back, is the **unhealthy and healthy threshold**, and it exists to prevent **flapping**.

The two jobs, which are not the same job

Spreading load. Requests are divided so no single server is overwhelmed. This is what people mean by “load balancer”.

Hiding failure. When a server dies, the load balancer stops sending it traffic and clients notice nothing. **This is the more important job**, and it is why a load balancer improves availability rather than only throughput.

A useful way to say it: a load balancer turns n unreliable servers into one apparently reliable service. That only works if it knows which of the n are currently reliable, which is what health checks are for.

Layer 4 and layer 7

The two kinds, and this is the first thing an interviewer checks.

Layer 4 works at the level of connections. It sees an incoming connection with an address and a port and forwards the bytes to a backend, without reading them.

- Very fast, because there is nothing to parse — millions of connections per machine.
- Cannot make decisions based on the content: it cannot route `/api` differently from `/images`.
- Cannot retry a failed request, because it does not know where one request ends.

Layer 7 understands the requests themselves. It reads the request line and headers, so it can route on the path, the host name, a cookie, or a header.

- Path-based routing, host-based routing, header-based routing.
- Can **retry** a failed request on another backend, because it knows what a request is.
- Can terminate encryption, compress responses, rewrite headers, and enforce rate limits.
- Costs more CPU per request and adds a little delay — sub-millisecond in practice.

Layer 7 unless you need the raw speed. The retry ability alone usually decides it: a layer 4 balancer that forwards a connection to a server which dies mid-request gives the user an error, where a layer 7 one quietly tries the next server.

The algorithms, and the failure each one has

ALGORITHM	HOW IT CHOOSES	FAILS WHEN
Round robin	strict rotation	requests differ in cost, or a server is slow but alive
Weighted round robin	rotation, but bigger servers get more	the weights are static and reality is not
Least connections	fewest in-flight requests	requests are very short — the count is stale before you use it
Least response time	fewest connections, tie-broken by latency	needs measurement; slightly more state
Random	pick one uniformly	uneven in the short run, fine in the long run
Power of two choices	pick two at random, send to the less loaded of them	almost nothing — see below
IP hash / consistent hash	the same client always lands on the same backend	a backend leaves and its clients all move

Round robin is the default and it is the wrong default whenever requests differ in cost. Pandian’s original mistake: it is perfectly fair in *number of requests* and completely unfair in *work*.

Least connections is the right default for most real systems, because it is self-correcting. A slow server keeps its requests longer, so its in-flight count stays high, so it receives fewer new ones — with nobody having to decide anything. That is Pandian looking at queue lengths.

Power of two choices deserves its own sentence, because it is a genuinely surprising result: pick two backends at random and send the request to whichever has fewer in-flight requests. That single extra comparison gets you nearly all the benefit of checking all n , with none of the cost, and it avoids the herd problem where every balancer sends the next request to the same “least loaded” server at the same moment. It is what several large systems actually use.

Consistent hashing is the one to reach for when the backends have per-client state worth preserving — a cache, or a session. [Day 108](#) is entirely about it.

Health checks, which are the real subject

A health check is the balancer asking each backend “are you alive?” on a schedule.

Active — the balancer sends a request, usually `GET /health`, every few seconds and looks at the response. **Passive** — the balancer watches real traffic and marks a backend unhealthy after several failed or timed-out responses.

Real systems use both: active to notice a server that has gone quiet, passive to notice one that is failing requests it was given.

Four settings, and the interviewer will ask about the interaction between them:

interval	how often to check	e.g. every 5 seconds
timeout	how long to wait for a reply	e.g. 2 seconds
unhealthy threshold	consecutive failures before removal	e.g. 2
healthy threshold	consecutive successes before return	e.g. 2

The thresholds exist to prevent flapping — a server bouncing in and out of the pool because one check happened to time out. Pandian's dropped pen.

And the arithmetic those settings imply is the thing to be able to do on the spot:

```
worst case time to notice a dead server
= interval × unhealthy_threshold + timeout
= 5 × 2 + 2
= 12 seconds
```

For twelve seconds, that dead server keeps receiving one request in every `n`. With four servers at 1,000 requests a second, that is 3,000 failed requests. Saying that number is what shows you understand that health checks are not free.

What the health endpoint should actually check

This is a small question with a genuinely good answer, and it comes up.

- **Too shallow** — `return 200` — reports healthy while the database connection is dead. The balancer keeps sending traffic to a server that fails every request.
- **Too deep** — check the database, the cache, and three downstream services — and when the shared database has a hiccup, **every server fails its health check at once and the balancer removes all of them**. You have turned a slow database into a total outage.

The right answer is: check what this instance alone needs to serve a request — its own process, its own connection pool — **and expose the deep checks on a separate endpoint for monitoring, not for the balancer**. That distinction is worth stating; it is a real production failure that has taken down real systems.

Where load balancers sit

There is usually more than one, at different levels:

DNS	→ spreads across regions or data centres (crude, cached, slow to change)
edge / global LB	→ anycast, picks the nearest healthy region
layer 7 LB	→ the one you draw in the diagram; routes by path and host
internal LB	→ in front of each internal service, service to service
client-side LB	→ the caller holds the list and chooses; no extra hop

Drawing exactly one load balancer box and never mentioning the others is fine for an interview — just do not claim it is the only one, because “who balances the load balancer?” is a standard follow-up and the answer is DNS and multiple balancer instances.

4 The picture

Where it sits and what it decides.

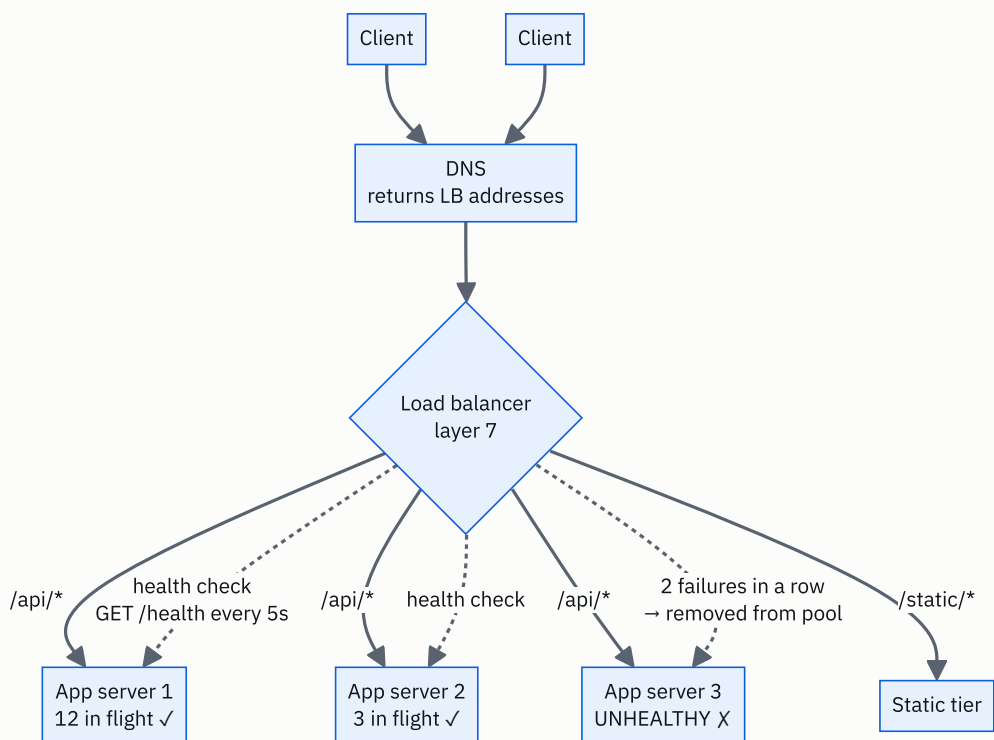


FIGURE 99.1

What to notice: **the health checks are a separate set of arrows from the traffic.** They run on their own schedule whether or not requests are arriving, which is what lets the balancer notice a server that has gone quiet rather than one that is failing.

The algorithms, on the same four backends:

```
four backends, current in-flight requests:  A:12  B:3  C:7  D:0
                                              (D just joined)

ROUND ROBIN          next → A, then B, then C, then D, then A...
                    ignores that A has 12 in flight and D has none.

LEAST CONNECTIONS   next → D (0), then D or C, ...
                    self-correcting: A stops receiving until it drains.

RANDOM               next → any of the four, 25% each.
                    fine over thousands of requests, lumpy over ten.

POWER OF TWO        pick two at random, say C and A → choose C (7 < 12).
                    one extra comparison, nearly the quality of least-connections,
                    and no herd effect from every balancer picking D at once.

IP HASH             hash(client_ip) % 4 → the SAME client always gets the same
                    backend. Good for per-client state; terrible for balance
                    if one client is much busier than the others.
```

The health-check timeline, drawn, because the timing is the whole point:

```
interval 5s · timeout 2s · unhealthy threshold 2 · healthy threshold 2

t=0    check → 200 OK          healthy, receiving traffic
t=5    check → 200 OK          healthy
t=8    ** the server dies **
t=10   check → timeout (2s)     failure 1 of 2   STILL IN THE POOL
t=15   check → timeout (2s)     failure 2 of 2   REMOVED at t=17
|-----|
t=8 ..... 9 seconds of traffic to a dead server ..... t=17

at 4,000 req/s across 4 servers, that is 1,000 req/s to the dead one
→ ~9,000 failed requests before it is removed

shorten the interval to 2s and the threshold stays at 2:
  worst case = 2 × 2 + 2 = 6 seconds → ~6,000 failures
  but the checks now cost 4 servers × 0.5/s = 2 extra requests per second,
  and a single slow moment is twice as likely to trigger a removal.
```

That trade — faster detection against more false removals and more check traffic — is the answer to “how would you tune it”.

5 How it actually works

The request path, step by step

- | | |
|---|--|
| 1. client resolves your domain name | → DNS returns one or more LB addresses |
| 2. client opens a connection to the LB | |
| 3. LB terminates TLS | (decrypts here; see the trade-offs) |
| 4. LB parses the request line and headers | (layer 7 only) |
| 5. LB picks a backend | algorithm + health state |
| 6. LB forwards the request | usually over a reused, warm connection |
| 7. backend responds | |
| 8. LB may rewrite headers, compress, log | |
| 9. LB returns the response to the client | |

Steps 3 to 6 add well under a millisecond in practice. **The one to notice is step 6:** the balancer keeps a pool of already-open connections to each backend, so a new client connection does not cause a new backend connection. That is why a hundred thousand clients do not become a hundred thousand database connections.

What happens when a backend fails mid-request

This is the difference between the two layers, made concrete.

- LAYER 4: the connection is broken. The client sees a reset. There is no retry, because the balancer does not know what a "request" was.
- LAYER 7: the balancer knows the request. If the backend fails BEFORE sending any response bytes, it can transparently retry on another backend and the client never knows.

And the caveat that separates a careful candidate: retries are only safe for idempotent requests. A `GET` can be retried freely. A `POST` that charges a card cannot — if the first backend actually processed it before dying, the retry charges twice. So layer 7 balancers retry `GET`, `HEAD` and `PUT` by default and do **not** retry `POST` unless you explicitly say it is safe.

Sticky sessions, and why they are a compromise

If a server keeps the user's session in its own memory, requests from that user must return to the same server. A balancer can do that with a cookie, or by hashing the client address.

It works, and it costs you three things:

- **Uneven load**, because users are not uniform and sessions last a long time.
- **A broken session when that server dies** — which is precisely the failure the balancer existed to hide.

- **You cannot drain a server** for deployment without disrupting the users pinned to it.

Stickiness is a workaround for state in the wrong place. The real fix is to move the session into a shared store, which is [tomorrow's](#) lesson.

Draining, which is how deployments stop hurting

When you take a server out for a deploy, you do not kill it. You mark it **draining**: it stops receiving new requests and keeps serving the ones already in flight, then shuts down when they finish or after a timeout.

```
without draining: in-flight requests are killed → user-visible errors on every deploy
with draining:    zero-error deploys, at the cost of a slower rollout
```

“How do you deploy without dropping requests?” is a very common follow-up and “connection draining, with a grace period longer than my slowest request” is the whole answer.

Real products

- **Hardware and software:** **HAProxy** and **nginx** are the classic software balancers. **Envoy** is the modern one and is the data plane inside most service meshes.
- **Cloud:** **AWS ALB** (layer 7), **NLB** (layer 4, millions of connections, very low latency), **Google Cloud Load Balancing** (global anycast), **Azure Front Door**.
- **Edge:** **Cloudflare** and **Fastly** are load balancers with a cache in front, distributed to hundreds of locations.
- **Client-side:** **gRPC** and service meshes often put the balancing in the *caller*, which removes a network hop entirely — the caller holds the list of healthy backends and chooses one itself. That is how internal service-to-service traffic is usually balanced at large companies.

6 The numbers

What one balancer handles

layer 4 (NLB-class)	millions of concurrent connections, ~100 µs added
layer 7 (nginx/ALB-class)	50,000 - 200,000 requests/second per instance
	~0.5 - 1 ms added latency
TLS handshakes	~5,000 - 20,000 new handshakes/s per core

The TLS number is the one that surprises people. A new HTTPS connection is far more expensive than a request on an existing one, which is why connection reuse — keep-alive — matters so much:

```
10,000 req/s, all on new connections → 10,000 handshakes/s → several cores of pure crypto
10,000 req/s, 100 requests per conn → 100 handshakes/s → negligible
```

Health check overhead

```
20 backends × 1 check every 5 s = 4 checks/second
negligible in traffic terms
```

but the DETECTION time is the number that matters:

```
interval 5 s, threshold 2, timeout 2 s → worst case 12 s
interval 2 s, threshold 2, timeout 1 s → worst case 6 s
interval 1 s, threshold 3, timeout 1 s → worst case 4 s, and far more
likely to remove a healthy-but-busy server
```

Cost of the detection window:

```
4,000 req/s across 4 backends = 1,000 req/s to each
12-second detection window    = 12,000 failed requests
6-second detection window     = 6,000 failed requests
```

Halving the interval halves the damage and doubles the false-positive risk. That is the trade, and stating it in those terms is the answer.

Sizing the balancer tier

```
peak 6,000 QPS
one layer 7 instance handles ~100,000 QPS → one is plenty on capacity

but: one instance = one point of failure
→ at least TWO, in different availability zones, behind DNS or anycast
```

You never run one load balancer, and the reason is availability rather than capacity — the same argument as [yesterday](#). A managed balancer already does this and presents a single name.

Connection multiplication

```
100,000 client connections at the balancer
× reused backend connections (keep-alive pool of ~100 per backend)
= a few hundred backend connections, not 100,000
```


The balancer is a connection funnel, and this is a genuine reason to have one even at low request rates. Without it, every client's connection becomes a backend connection, and databases and application servers run out of file descriptors long before they run out of CPU.

Uneven distribution, measured

Round robin with equal-cost requests is perfectly even. With unequal costs it is not:

4 backends, requests: 90% take 10 ms, 10% take 1,000 ms

round robin: each backend gets 25% of the SLOW requests by luck.
variance is high; one backend can hold several slow ones at once.
least connections: a backend holding a slow request has a high in-flight count,
so it stops receiving new ones until it drains.

measured effect on p99 latency in this kind of workload: often 2-5× better
with least connections.

7 The trade-offs

Layer 4 or layer 7?

Layer 4 is faster, cheaper, and handles far more connections. It is the right choice for raw TCP, for very high connection counts, and for anything where you must not decrypt the traffic.

Layer 7 can route by path, retry failed requests, terminate TLS, and enforce limits. **Take layer 7 for HTTP services** — the retry ability alone usually justifies it. **I would not use layer 7** if the traffic is not HTTP, if end-to-end encryption is a requirement, or if the connection count is in the millions and the per-request parsing cost matters.

Terminate TLS at the balancer, or pass it through?

Terminating means the balancer decrypts, which is what lets it read paths and headers at all — and it means traffic between the balancer and the backends is unencrypted unless you re-encrypt.

Terminate, and re-encrypt to the backends if the network is not trusted. The pure pass-through option gives you end-to-end encryption and gives up every layer 7 feature at once.

Least connections, or round robin?

Least connections by default. It is self-correcting, it needs no configuration, and it handles the case round robin cannot: a server that is slow but alive.

I would use round robin if every request costs the same and is very short, because then the in-flight count is stale by the time you act on it and the extra bookkeeping buys nothing. **And I would use power-of-two-choices** if there are several balancer instances, because the herd problem — every balancer independently deciding the same backend is least loaded — is real.

How aggressive should health checks be?

Covered in the numbers, and the shape of the answer is: **detect faster, and you remove healthy servers more often**. A removal is not free — the remaining servers absorb that load, and if the removal was caused by a load spike, removing a server makes the spike worse on the others and can cascade.

The specific failure to name: a health check that is too deep. If `/health` checks the shared database, then one database hiccup fails every server's health check simultaneously and the balancer removes the entire pool. The system goes from degraded to completely down, caused by the safety mechanism.

Where this design breaks

- **The balancer is on the path of every request**, so its failure is total. The answer is several instances behind DNS or anycast, and it is why managed balancers exist.
- **It cannot fix a slow backend dependency**. If every server is slow because the database is slow, the balancer has nowhere good to send anything. Load balancing distributes load; it does not create capacity.
- **Stickiness undermines the whole thing**. Every sticky session is a small piece of the availability guarantee given back.
- **Long-lived connections rebalance badly**. WebSockets last for hours, so a newly added server receives nothing until existing connections close. Solving that means forcing periodic reconnection, which is a real design decision rather than a setting.

8 In the interview

How it gets asked

- The direct one: *“How does a load balancer decide which server gets the request?”*
- The one that separates people: *“One server is slow but not dead. What happens?”*
- The mechanism probe: *“How does it know a server is down, and how long does that take?”*
- The layers question: *“Layer 4 or layer 7? Why?”*

- The recursion: “*What if the load balancer fails?*”
- The deployment question: “*How do you deploy without dropping requests?*”

What to say out loud, in the first ninety seconds

1. **Give both jobs, and rank them.** “It spreads load, and it hides failure. The second matters more — a balancer that keeps routing to a dead server is worse than none.”
2. **Pick a layer and justify it.** “Layer 7 for an HTTP service, because it can route on path and, more importantly, retry a failed request on another backend. Layer 4 if I need millions of connections or must not decrypt.”
3. **Name the algorithm and its failure mode together.** “Least connections by default. Round robin is the common answer, and it breaks when requests differ in cost or a server is slow but alive — it keeps sending that server one request in every n , for ever.”
4. **Go straight to health checks.** “The choice is only over *healthy* backends, so the health check is the real mechanism: active checks on an interval, plus thresholds so one slow response does not remove a good server.”
5. **Do the detection arithmetic.** “With a five-second interval, a threshold of two and a two-second timeout, worst case detection is twelve seconds — and at a thousand requests a second to that backend, that is twelve thousand failed requests before it is removed.”
6. **Say what the health endpoint must not do.** “It should check only what this instance needs to serve a request. If it checks the shared database, one database hiccup fails every server at once and the balancer removes the whole pool.”

The follow-ups

“**One server is slow but not dead. What happens?**” “With **round robin**, the worst possible thing: it is still passing health checks, so it keeps getting one request in every n , indefinitely, and every one of those users waits. Round robin is fair in request *count* and blind to request *cost*. With **least connections**, it fixes itself — a slow server holds its requests longer, so its in-flight count stays high, so the balancer sends it fewer new ones, with nobody deciding anything. That self-correction is exactly why I would default to least connections. If I wanted to go further I would add **outlier detection**: track per-backend latency and eject a backend whose p99 is far above its peers, then let it back in after a cool-down. And I would make the health check itself latency-aware, so a server that responds correctly but slowly counts as unhealthy.”

“How does it know a server is down, and how long does it take?” “Active health checks — usually a `GET /health` on a fixed interval — plus passive observation of real traffic failures. Four settings matter: the interval, the timeout, the unhealthy threshold and the healthy threshold. The two thresholds exist to prevent flapping: without them a single timed-out check pulls a healthy server out, and a single good one puts it straight back. The detection time is $\text{interval} \times \text{unhealthy_threshold} + \text{timeout}$ — with five seconds, two and two, that is twelve seconds worst case. That number matters because during it the dead server is still receiving its share: at four thousand requests a second across four servers, twelve seconds is about twelve thousand failed requests. I can halve that by halving the interval, and the price is more check traffic and a higher chance of removing a server that was merely busy.”

“What should the health endpoint check?” “Only what this instance needs in order to serve a request — the process is up, its own connection pool is usable. **Not** the shared database, and this is a real production failure rather than a theoretical one: if every server’s health check depends on the same database, then one database hiccup fails all of them simultaneously and the balancer removes the entire pool. You have converted a degraded system into a complete outage using the safety mechanism. The deep checks — database, cache, downstream services — belong on a separate endpoint that monitoring and alerting read, and that no balancer acts on.”

“What if the load balancer itself fails?” “It is on the path of every request, so its failure is total — which is why you never run one. In practice you run at least two instances in different availability zones and put them behind either DNS with multiple A records, or **anycast**, where the same address is announced from several locations and the network routes to the nearest healthy one. A managed balancer does this for you and presents a single name, which is most of why people use managed ones. DNS is the crudest layer and the slowest to react, because clients cache the answer for the record’s TTL — so DNS gives you regional failover in minutes, not seconds, and you should not rely on it for instance-level failure.”

“How do you deploy without dropping requests?” “Connection draining. When I take a server out, I mark it draining rather than killing it: the balancer stops sending it new requests, and it keeps serving the ones already in flight until they finish or a grace period expires. The grace period has to be longer than my slowest normal request, otherwise I am just killing requests on a delay. Then I roll through the fleet a few servers at a time, waiting for the new ones to pass their **healthy threshold** before continuing — which is why that setting matters in both directions. The failure mode without draining is that every deploy produces a burst of user-visible errors, and it is often mistaken for an application bug.”

“Would you use sticky sessions?” “Only as a temporary measure, and I would name it as a compromise rather than a design. Stickiness means the balancer sends a given user back to the same server, usually via a cookie, and it exists because that server is holding state in its own memory. It costs three things: load becomes uneven, because users are not uniform and sessions are long; when that server dies, those users lose their session — which is exactly the failure the balancer was supposed to hide; and you cannot drain a server for deployment without disrupting the users pinned to it. The real fix is to move the session into a shared store so any server can serve any request, which makes the balancer’s job trivial again.”

A model answer

Asked: how does a load balancer decide which server gets the request?

“Before the algorithm, I would separate the two jobs, because the second one is the one people skip. A load balancer **spreads load**, and it **hides failure**. Hiding failure is the more important job — a balancer that happily keeps routing to a dead server is worse than not having one, because at least without it you would notice.

So the decision is really two decisions: *which backends are eligible*, and *which of those gets this request*.

Eligibility is health checking. Active checks, typically a `GET /health` every few seconds, plus passive observation of real failures. Two thresholds matter as much as the interval: a server has to fail several checks in a row before it is removed, and succeed several in a row before it comes back, otherwise one unlucky timeout pulls a healthy server out and one lucky response puts it straight back. The number I would put on the board is the detection time: interval times the unhealthy threshold, plus the timeout. Five seconds, threshold two, two-second timeout is twelve seconds — and at a thousand requests a second reaching that backend, that is twelve thousand failed requests before it is removed. I can halve it by halving the interval, and I pay in false removals.

One thing about the health endpoint that is worth saying explicitly: it should check only what **this instance** needs to serve a request. If it checks the shared database, then a single database hiccup fails every server’s check at the same moment and the balancer removes the entire pool — turning a degraded system into a total outage, using the safety mechanism.

Then the choice among healthy backends. **Round robin** is the usual answer and it is the wrong default: it is fair in the number of requests and blind to their cost, so a server that is slow but alive keeps receiving one request in every `n`, indefinitely. **Least connections** is what I would use, because it is self-correcting — a slow server holds its requests longer, its in-flight count stays high, and it therefore receives fewer new ones without anyone deciding that it is slow. If there are several balancer instances I would prefer **power of two choices**: pick two backends at random and take the less loaded one. That single comparison gets nearly the quality of least-connections without every balancer stampeding onto the same idle server at once. **Consistent hashing** is the one I would use if backends hold per-client state, like a cache.

On the layer: **layer 7** for an HTTP service, so I can route by path and, more importantly, **retry** a failed request on another backend — with the caveat that retries are only safe for idempotent requests, so `GET` yes and `POST` only if the

caller says so. Layer 4 if the traffic is not HTTP, or the connection count is in the millions and I cannot afford to parse anything.

And I would never draw exactly one of these. It is on the path of every request, so its failure is total: at least two instances in different zones, behind DNS or anycast.”

9 Recall card

- **Two jobs, and the second matters more: spread load, and hide failure.** A balancer turns n unreliable servers into one apparently reliable service — but only if it knows which are healthy, so **health checks are the real mechanism**, not the algorithm.
- **Least connections by default, not round robin.** Round robin is fair in request *count* and blind to *cost*, so **a server that is slow but alive keeps getting one request in every n , for ever**. Least connections self-corrects. **Power of two choices** (pick two at random, take the lighter) gets nearly the same quality with no herd effect. **Consistent hashing** when backends hold per-client state.
- **Detection time** = `interval * unhealthy_threshold + timeout` — $5\text{ s} \times 2 + 2 = 12\text{ seconds}$, which at 1,000 req/s to that backend is **~12,000 failed requests**. Thresholds exist to stop **flapping**. Halving the interval halves the damage and doubles the false removals.
- **The health endpoint must check only what THIS instance needs.** A check that touches the shared database fails **every** server at once during one hiccup, and the balancer removes the whole pool — degraded becomes down, caused by the safety mechanism. Deep checks go on a separate endpoint for monitoring.
- **Layer 7 for HTTP** — path routing, TLS termination, and **retries** (safe for `GET`, not for `POST`); **layer 4** for raw speed and millions of connections. **Never run one balancer** — two zones behind DNS or anycast. **Connection draining** is how you deploy without dropping requests. **Sticky sessions are a compromise**, not a design: uneven load, sessions lost on failure, and no clean draining.

DSA topic: Binary trees in code **System design topic:** Load balancers

Code these, in this order

One rule for the whole set: **write** `from_list`, `to_list` and `show` once, keep them in a file, and use them for every tree problem for the next two weeks. Fifteen minutes today saves an hour a day afterwards.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Invert Binary Tree	LeetCode 226 (Easy)	Mutating a tree in place, and whether you can see the result.
2	Merge Two Binary Trees	LeetCode 617 (Easy)	Building new nodes, and the four <code>None</code> cases.
3	Search in a Binary Search Tree	LeetCode 700 (Easy)	Why this is <code>O(height)</code> and plain-tree search is <code>O(n)</code> .
4	Construct String from Binary Tree	LeetCode 606 (Medium)	A different serialisation format, and why the brackets are load-bearing.

On problem 1, use your printer

Print the tree before and after. If you cannot see the inversion in the output, your printer is wrong and that is worth fixing today rather than on day 104.

On problem 2, enumerate the cases before writing

There are four combinations of the two nodes being `None` or not. Write all four down, then write the code, then check that the code has exactly those four outcomes.

On problem 3, say the difference out loud

This is a search *tree*, so it is `O(height)`. Say what you would have had to do if it were a plain binary tree, and what the complexity would have been.

On problem 4, notice what the format keeps

LeetCode's list format and this bracket format both describe a tree. Say what each one does about missing children, and why neither can simply drop them.

The node drill

1. Write the class from memory, with LeetCode's field names.
2. Say why there is no `Tree` class.
3. Say the two things `None` means, and why one base case covers both.
4. Add `__repr__` and say what it saves you from.
5. Say what `__slots__` would change, in bytes per node.

The building drill

1. Build the same five-node tree three ways: nested, by assignment, and from a list.
2. Say when you would use each.
3. Build a tree with a node that has only a right child, both ways.
4. Say why that shape is the one worth putting in a test.

The list-format drill

1. Convert `[1, 2, 3, None, None, 4, 5]` into a drawing.
2. Convert your drawing back into a list and check it round-trips.
3. Drop the `None`s from the list and draw what it now means.
4. Say what LeetCode does with trailing `None`s, and give an example.
5. Say why this format is $O(n)$ where the index formula is $O(2^h)$.

The index-formula drill

1. Give the child and parent index formulas.
2. Compute the array size needed for a complete tree of 1,000 nodes.
3. Compute it for a degenerate tree of 20 nodes, and then 40.
4. Say which structure uses this layout deliberately, and why it can.
5. Compare the memory of a million-node pointer tree against a million-entry array.

The no-parent drill

1. State the three consequences of having no parent pointer.
2. Write `find_with_parent` and say what it does instead.
3. Say what maintaining a real `parent` field would cost.
4. Say when you would actually add one.

The reference drill

1. Run `backup = root; backup.val = 999` and print `root.val`.
2. Say why that happened, and name the day-091 trap it matches.

3. Write a real deep copy.

4. Run `TreeNode(1) == TreeNode(1)` and say why it is `False`.

5. Say what structural equality requires instead.

The printer drill

1. Write the nine-line sideways printer from memory.

2. Say which subtree appears above and which below, and why.

3. Say which traversal order the three lines form.

4. Print a seven-node tree and a degenerate one, and say which is easier to read.

The break-it drill

Trigger each and record the exact output or error:

1. `root.value` on a LeetCode-style node.

2. `root.parent`.

3. `from_list` on `[]` and on `[None]`.

4. Dropping the `None`s from a list and comparing the two trees.

5. Building the index-formula array for a degenerate tree of forty nodes.

6. A recursive `copy_tree` on a chain of 10,000 nodes.

The two-jobs drill

1. Name both jobs of a load balancer and say which matters more, with the reason.

2. Say what a balancer turns n unreliable servers into.

3. Say what that claim depends on.

The layers drill

1. Define layer 4 and layer 7 in one sentence each.

2. Give three things layer 7 can do that layer 4 cannot.

3. Give two reasons to choose layer 4 anyway.

4. Say which requests may be retried and which may not, and why.

The algorithms drill

For each of the seven algorithms, say how it chooses and give its failure mode:

round robin · weighted round robin · least connections · least response time · random
· power of two choices · consistent hashing

Then answer: four backends hold 12, 3, 7 and 0 in-flight requests. Where does each algorithm send the next request?

The slow-server drill

1. Say what round robin does with a server that is slow but alive.
2. Say what least connections does, and why it needs no decision.
3. Name the extra mechanism you would add on top, and what it tracks.
4. Say what you would change about the health check itself.

The health-check drill

1. Name the four settings.
2. Write the detection-time formula and compute it for $5\text{ s} / 2 / 2\text{ s}$.
3. Compute the failed requests during that window at 4,000 req/s across 4 backends.
4. Recompute both for a 2-second interval and say what you paid for the improvement.
5. Say why the two thresholds exist and what they are preventing.

The health-endpoint drill

1. Say what a too-shallow check reports, and what it costs.
2. Say what a too-deep check does when the shared database hiccups.
3. State the rule for what the endpoint should check.
4. Say where the deep checks belong instead.

The failure drill

For each, say what happens and what you would add:

1. The load balancer itself fails.
2. A deploy kills servers with requests in flight.
3. Sessions live in each server's memory.
4. Every backend is slow because the database is slow.
5. WebSocket connections have been open for six hours and a new server joins.
6. 100,000 clients connect and each one becomes a backend connection.

Two of the six are not solved by the balancer at all. Name them.

The numbers drill

Compute or state each:

1. Requests per second one layer 7 instance handles.
 2. TLS handshakes per second per core, and the effect of keep-alive at 10,000 req/s.
 3. Health-check traffic for 20 backends at a 5-second interval.
 4. How many balancer instances you run at 6,000 peak QPS, and why that number is not about capacity.
 5. Backend connections for 100,000 client connections with a keep-alive pool.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Build this tree in code. Now print it.* No `Tree` class and what `None` means, LeetCode's field names, the list format read out with the nulls justified, the helpers you would write once, the sideways printer, and the no-parent fact stated before it bites.
 2. *How does a load balancer decide which server gets the request?* Both jobs with the ranking, eligibility separated from choice, health checks with the detection-time arithmetic, least connections over round robin with the slow-server reason, the layer with the retry justification, and never running just one.
 3. *One server is slow but not dead. What happens?* Round robin's specific failure, least connections' self-correction, outlier detection as the next step, and what you would change about the health check.
-

Before you move on

- [] I have `from_list`, `to_list` and `show` saved in a file I will reuse.
- [] I can write the node class from memory with the right field names.
- [] I can say the two things `None` means in a tree.
- [] I can read a level-order list aloud and draw the tree correctly.
- [] I can say why the nulls cannot be dropped, with the specific wrong tree.
- [] I know why this format is $O(n)$ and the index formula is $O(2^h)$.
- [] I can state the three consequences of having no parent pointer.
- [] I know what `backup = root` actually does.
- [] I know why `TreeNode(1) == TreeNode(1)` is `False`.
- [] I can build a deliberately awkward test tree and say why it is better.
- [] I can name both jobs of a load balancer and rank them.

- [] I can define layer 4 and layer 7 and justify a choice.
- [] I can give the failure mode of every algorithm in the table.
- [] I can say what round robin does with a slow-but-alive server.
- [] I can write the detection-time formula and compute the failed-request count.
- [] I can say what the health endpoint must not check, and what happens if it does.
- [] I can explain connection draining and why the grace period matters.
- [] I can say why sticky sessions are a compromise rather than a design.
- [] I answered all three questions above out loud.

Depth-first traversal: preorder, inorder, postorder

DATA STRUCTURES AND ALGORITHMS

Depth-first traversal: preorder, inorder, postorder

You can write all three recursively and say which problem needs which order.

SYSTEM DESIGN

Stateless services and why they scale

You can explain why sticky sessions are a scaling bug.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Print the tree in inorder. Now do it iteratively.*
- *Why should a web server be stateless?*

Depth-first traversal: preorder, inorder, postorder

1 What this is, and why they ask it

To **traverse** a tree is to visit every node exactly once. Going **depth-first** means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

Three sentences. Those three placements are called **preorder**, **inorder** and **post-order**, and the only difference between them is **which line of the function the work is on** — the recursion is identical. They are not three techniques to memorise; they are one technique with the work at three different moments. And the choice is never arbitrary: **preorder is for anything the parent must decide before the children, post-order is for anything computed from the children, and inorder gives you a binary search tree in sorted order.**

They ask it because the recursive versions are four lines each and the iterative versions are genuinely different from one another — preorder is easy, inorder needs a specific loop shape, and postorder is awkward enough that there are three standard tricks for it. “*Now do it iteratively*” is one of the most reliable follow-ups in tree interviews, and it is the point at which people who memorised three functions come apart.

2 The story

The wedding date moved.

The hall had a problem with its licence and the whole thing shifted from the eleventh to the eighteenth, and Krishnan found out on a Monday evening. Everybody had to be told — the caterer, the lights man, the woman making the sweets, all thirty-odd people who had been given a job.

He did it in about forty minutes and he did it without ringing thirty people.

He rang his four. Told each of them the new date, and told them to tell theirs. Ravi's father rang the caterer, the two cousins doing the snacks, and the sweet woman, and told them; the caterer rang his assistant, and the assistant rang the boy.

Each person heard it, and then passed it on. That order mattered. You cannot tell somebody to pass on a message you have not been given yet.

Three weeks later, after the wedding, they did the accounts, and that went the other way round entirely.

Krishnan wanted one number: what the whole thing had cost. He rang his four and asked. Ravi's father could not answer. He had not spent the money himself — the caterer had, and the sweet woman had, and the two cousins had. So he rang them first and asked what each had spent, and only when he had all four numbers could he add his own and give Krishnan a total for food.

The caterer had the same problem one level further down. He could not say what he had spent until his assistant had told him, and the assistant could not say until the boy had.

So the answer came up from the bottom. The boy first, then the assistant, then the caterer, then Ravi's father, then Krishnan. And it took two days rather than forty minutes, because every person had to wait for everybody under them.

Krishnan's sister said afterwards that it was the same tree of people both times, so why had one taken forty minutes and the other two days.

Krishnan said it was the same people but the opposite direction. Telling goes down and each person can speak the moment they hear. Asking goes up and nobody can speak until everyone below them has.

3 The idea in plain English

Telling the date is a **preorder** traversal. Collecting the accounts is a **postorder** traversal. Same tree, same visits, opposite moments.

The three orders are one function with the work in three places

```
def walk(node):
    if node is None:
        return
    # A ← preorder: visit HERE, before the children
    walk(node.left)
    # B ← inorder: visit HERE, between the children
    walk(node.right)
    # C ← postorder: visit HERE, after the children
```


That is the entire lesson. The recursion never changes. Move one line and the order changes.

preorder	node, left, right	"tell them, then pass it on"
inorder	left, node, right	"left half, me, right half"
postorder	left, right, node	"hear from everyone below, then speak"

Say them as *where the node goes*: **pre** = node first, **in** = node in the middle, **post** = node last. The children are always left before right in all three.

Which order for which problem — this is the part that matters

Preorder — the parent decides something the children need.

- Copying a tree: you must create a node before you can attach its children to it.
- Serialising a tree: the reader needs the root before it can place anything under it.
- Anything carrying information *down* — a depth, a running path, a running sum, a permitted range.

Postorder — the answer is computed from the children.

- Height, size, sum of a subtree: you cannot know yours until you know theirs.
- Deleting a tree: free the children before the parent, or you lose the pointers to them.
- Diameter, balance, maximum path sum — all of [days 102](#) and [104](#) are postorder.

Inorder — only interesting for a binary search tree, where it produces sorted order.

That single fact is why inorder exists as a named thing. In a BST everything left is smaller and everything right is larger, so “left, me, right” is exactly ascending order. It is the basis of [validating a BST](#), of finding the k -th smallest element, and of the BST iterator. **On a tree that is not a BST, inorder has no particular meaning**, and saying so is a good sign.

The one-line test to say out loud:

Does the parent need something from the children? Postorder. Do the children need something from the parent? Preorder. Is it a BST and do I want sorted order? Inorder.

Why the recursion works at all

Because every child is the root of its own subtree, and “visit every node in this tree” is exactly “visit this node, and visit every node in each subtree”. That is [the leap of faith](#), and trees are where it stops feeling like a trick.

The base case is `None`, and it is doing real work: it is both “this branch is empty” and “we have reached the bottom”.

Doing it without recursion

The recursion is using the call stack to remember where to come back to. To do it iteratively you keep that stack yourself.

Preorder is easy, because you visit a node the moment you meet it:

```
stack = [root]
while stack:
    node = stack.pop()
    visit(node)
    if node.right: stack.append(node.right)    # right FIRST
    if node.left:  stack.append(node.left)     # so left comes off first
```

Push right before left, because a stack reverses what you put in it.

Inorder is the one with a specific shape, and it is worth learning as a shape rather than deriving it each time:

```
stack, node = [], root
while stack or node:
    while node:                # go as far left as possible, remembering the way
        stack.append(node)
        node = node.left
    node = stack.pop()         # nothing further left: this one is next
    visit(node)
    node = node.right          # then do its right subtree
```

The idea in one sentence: **walk left as far as you can, pushing everything you pass; when you cannot go further left, the top of the stack is the next node in order; then start again from its right child.**

Postorder is awkward, because a node must be visited *after* both children — so when you pop it you do not yet know whether you have already done its right subtree. Three standard answers:

1. **The trick**: do a preorder as *node, right, left*, then **reverse the whole output**. That reversed order is exactly `left, right, node`. Two lines longer than preorder, and it is what to write under time pressure.

2. **Two stacks:** push to a second stack instead of a list, then drain it. Same idea, more obvious.
3. **The honest one:** keep a `last_visited` pointer, and only visit a node when its right child is `None` or was the last thing visited. This is the version to describe if the interviewer says “without reversing”.

Say the reversal trick, then offer the honest version. Interviewers like the trick and then ask whether you can do it properly.

The unified template, if you want one function for all three

```
stack = [(root, False)]          # (node, have its children been queued?)
while stack:
    node, expanded = stack.pop()
    if node is None:
        continue
    if expanded:
        visit(node)              # second time we see it: do the work
    else:
        # push in REVERSE of the order you want
        stack.append((node.right, False))
        stack.append((node, True))  # ← move this line for pre/in/post
        stack.append((node.left, False))
```

Moving that one line changes the order, exactly as in the recursive version. It is slower than the specialised versions and it is one thing to remember instead of three, which is a fair trade in an interview.

Depth-first and breadth-first

All three of today's are **depth-first**: go deep, then back up. The other family is **breadth-first** — one whole level at a time — which needs a **queue** instead of a stack and is [tomorrow](#).

The distinction to hold on to: **depth-first costs $O(\text{height})$ memory**, **breadth-first costs $O(\text{width})$** . For a perfect tree the widest level holds half the nodes, so on a million-node tree depth-first holds about twenty frames and breadth-first holds half a million nodes.

4 The picture

One tree, three orders.

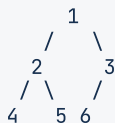


PREORDER (node, left, right) → 1 2 4 5 3 6
 INORDER (left, node, right) → 4 2 5 1 6 3
 POSTORDER (left, right, node) → 4 5 2 6 3 1

^ the root is LAST in postorder
 ^ the root is FIRST in preorder

The way to read them off a drawing without running any code:

Draw a loop around the whole tree, starting left of the root, going anticlockwise, hugging every node. Each node is passed THREE times.



PREORDER: write a node the FIRST time the loop passes it (on its left)
 INORDER: write a node the SECOND time (underneath it)
 POSTORDER: write a node the THIRD time (on its right)

One drawing, all three answers, and it is how to check yourself in an interview.

Krishnan's two jobs, drawn:

TELLING THE DATE (preorder)
information flows DOWN

```

      Krishnan (1st)
     /   |   \
  Father Meena Sudhir
  (2nd)
   /   |   \
Caterer Snacks Sweets
(3rd)
  |
Assistant (4th)
  |
  Boy (5th)
  
```

nobody can pass on a message
they have not received

COLLECTING THE ACCOUNTS (postorder)
information flows UP

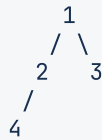
```

      Krishnan (last)
     /   |   \
  Father Meena Sudhir
  (after its children)
   /   |   \
Caterer Snacks Sweets
(must answer BEFORE Father can)
  |
Assistant
  |
  Boy (FIRST to answer)
  
```

nobody can give a total
until everyone below has

The iterative inorder, traced, because the shape is the thing to remember:

tree:



stack	node	action

[]	1	go left: push 1
[1]	2	go left: push 2
[1,2]	4	go left: push 4
[1,2,4]	None	cannot go left → pop 4, VISIT 4, node = 4.right = None
[1,2]	None	cannot go left → pop 2, VISIT 2, node = 2.right = None
[1]	None	cannot go left → pop 1, VISIT 1, node = 1.right = 3
[]	3	go left: push 3
[3]	None	cannot go left → pop 3, VISIT 3, node = None
[]	None	stack empty and node is None → done

output: 4 2 1 3

And the postorder trick:

```
modified preorder: node, RIGHT, left    → 1 3 6 2 5 4
reverse it                               → 4 5 2 6 3 1
                                         ^^^^^^^^^^^^^^^^^ = postorder
```

5 The code, built step by step

Step 1 — write the skeleton once and move one line

```
def walk(node):
    if node is None:
        return
    walk(node.left)
    walk(node.right)
```

Then put `out.append(node.val)` in one of the three positions. **Do not memorise three functions.** Write the skeleton, then decide where the visit goes by asking who needs what.

Step 2 — say which order the problem wants, and why

“The parent has to exist before I can attach children to it, so this is preorder.” / “I cannot know my height until I know my children’s, so this is postorder.” **Say the reason, not the name** — the name is a label on the reason.

Step 3 — iterative preorder, and the reversal detail

```
node = stack.pop()
out.append(node.val)
if node.right:
    stack.append(node.right)      # RIGHT first
if node.left:
    stack.append(node.left)      # so LEFT is popped first
```

Push right then left. Getting this backwards gives you `node, right, left`, which is a real order — it is just not the one asked for, and the output looks plausible enough to miss.

Step 4 — iterative inorder, as a shape

```
while node:
    stack.append(node)
    node = node.left
node = stack.pop()
out.append(node.val)
node = node.right
```

Learn these five lines as one unit. The outer condition is `while stack or node`, and forgetting the `or node` means the loop never starts, because the stack is empty at the beginning.

Step 5 — postorder, the trick and the honest version

```
# trick: preorder as node, RIGHT, left – then reverse
out.append(node.val)
if node.left: stack.append(node.left)
if node.right: stack.append(node.right)
...
return out[::-1]
```

Then, if asked for it without the reversal:

```
peeked = stack[-1]
if peeked.right and last_visited is not peeked.right:
    node = peeked.right      # right subtree not done yet
else:
    out.append(peeked.val)
    last_visited = stack.pop()
```

The condition means “**my right subtree is either empty or already finished, so it is my turn**”.

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

# -----
# Recursive: ONE function, the visit line in three different places.
# -----

def preorder(root: TreeNode | None) → list[int]:
    """node, left, right. For anything the PARENT must do first:
    copying, serialising, carrying a depth or a path DOWN."""
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        out.append(node.val)          # ← visit BEFORE the children
        walk(node.left)
        walk(node.right)

    walk(root)
    return out

def inorder(root: TreeNode | None) → list[int]:
    """left, node, right. On a BST this is SORTED ORDER, which is the only
    reason inorder is a named thing."""
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        walk(node.left)
        out.append(node.val)          # ← visit BETWEEN the children
        walk(node.right)

    walk(root)
    return out

def postorder(root: TreeNode | None) → list[int]:
    """left, right, node. For anything computed FROM the children:
    height, size, subtree sums, deletion, diameter, max path sum."""
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        walk(node.left)
        walk(node.right)
        out.append(node.val)          # ← visit AFTER the children
```

```

walk(root)
return out

# -----
# Iterative: the interviewer's follow-up.
# -----

def preorder_iterative(root: TreeNode | None) → list[int]:
    """Easiest of the three: visit a node the moment you pop it.

    Push RIGHT before LEFT, because a stack reverses. Getting that backwards
    gives node-right-left, which looks plausible and is wrong.
    """
    if root is None:
        return []
    out: list[int] = []
    stack = [root]
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return out

def inorder_iterative(root: TreeNode | None) → list[int]:
    """The one with a specific shape. Learn it as a shape.

    Walk left as far as possible, pushing everything you pass. When you can
    go no further left, the top of the stack is the next node in order.
    Then start again from its right child.

    Note `while stack or node` – with only `while stack` the loop never
    starts, because the stack is empty at the beginning.
    """
    out: list[int] = []
    stack: list[TreeNode] = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.append(node.val)
        node = node.right
    return out

def postorder_iterative(root: TreeNode | None) → list[int]:
    """The trick: preorder as node-RIGHT-left, then reverse the output.
    Reversing node-right-left gives left-right-node, which is postorder.

    Two lines longer than preorder. This is what to write under time
    pressure; postorder_iterative_honest is what to describe if the
    interviewer asks for it without the reversal.
    """
    if root is None:
        return []
    out: list[int] = []
    stack = [root]

```



```

while stack:
    node = stack.pop()
    out.append(node.val)
    if node.left:
        stack.append(node.left)          # LEFT first now – deliberately mirrored
    if node.right:
        stack.append(node.right)
return out[::-1]

def postorder_iterative_honest(root: TreeNode | None) → list[int]:
    """Genuine postorder, no reversal.

    A node may only be visited when its right subtree is empty or was the
    last thing visited – that is exactly what `last_visited` is tracking.
    """
    out: list[int] = []
    stack: list[TreeNode] = []
    node = root
    last_visited: TreeNode | None = None

    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        peeked = stack[-1]
        if peeked.right is not None and last_visited is not peeked.right:
            node = peeked.right          # right subtree still to do
        else:
            out.append(peeked.val)
            last_visited = stack.pop()
    return out

def traverse(root: TreeNode | None, order: str = "in") → list[int]:
    """All three from one loop, using a flag meaning "children already queued".

    Move the (node, True) line to change the order – exactly as moving the
    visit line does in the recursive version. Slower than the specialised
    versions; one thing to remember instead of three.
    """
    out: list[int] = []
    stack: list[tuple[TreeNode | None, bool]] = [(root, False)]

    while stack:
        node, expanded = stack.pop()
        if node is None:
            continue
        if expanded:
            out.append(node.val)
            continue
        if order == "pre":
            stack.append((node.right, False))
            stack.append((node.left, False))
            stack.append((node, True))
        elif order == "in":
            stack.append((node.right, False))
            stack.append((node, True))
            stack.append((node.left, False))
        else:
            # post
            stack.append((node, True))
            stack.append((node.right, False))
            stack.append((node.left, False))

```

```

    return out

# -----
# Why the order matters: the same walk, three real jobs.
# -----

def copy_tree(node: TreeNode | None) → TreeNode | None:
    """PREORDER: the node must exist before its children can be attached."""
    if node is None:
        return None
    new = TreeNode(node.val)          # create FIRST
    new.left = copy_tree(node.left)
    new.right = copy_tree(node.right)
    return new

def height(node: TreeNode | None) → int:
    """POSTORDER: cannot know mine until I know both of theirs."""
    if node is None:
        return -1
    left = height(node.left)          # ask the children FIRST
    right = height(node.right)
    return 1 + max(left, right)

def subtree_sums(node: TreeNode | None, out: list[int]) → int:
    """POSTORDER, and Krishnan's accounts exactly: each total needs the
    totals below it before it can be computed."""
    if node is None:
        return 0
    total = node.val + subtree_sums(node.left, out) + subtree_sums(node.right, out)
    out.append(total)                 # recorded AFTER the children
    return total

def paths_to_leaves(node: TreeNode | None,
                    trail: list[int] | None = None,
                    out: list[list[int]] | None = None) → list[list[int]]:
    """PREORDER: the path is carried DOWN, so the node is appended before
    the recursion. This is choose-recurse-un-choose from day 094."""
    trail = [] if trail is None else trail
    out = [] if out is None else out
    if node is None:
        return out
    trail.append(node.val)             # choose (preorder position)
    if node.left is None and node.right is None:
        out.append(trail[:])           # COPY
    else:
        paths_to_leaves(node.left, trail, out)
        paths_to_leaves(node.right, trail, out)
    trail.pop()                        # un-choose
    return out

def kth_smallest(root: TreeNode | None, k: int) → int | None:
    """INORDER on a BST is sorted order, so this stops after k nodes rather
    than walking the whole tree. 0(height + k), not 0(n)."""
    stack: list[TreeNode] = []
    node = root
    while stack or node:
        while node:
            stack.append(node)

```

```

        node = node.left
    node = stack.pop()
    k -= 1
    if k == 0:
        return node.val
    node = node.right
return None

def morris_inorder(root: TreeNode | None) → list[int]:
    """Inorder in O(1) extra space – no stack and no recursion.

    It temporarily links each node's rightmost left-descendant back to the
    node ("a threaded tree"), walks, then removes the link. Worth being able
    to NAME; rarely worth writing, because it mutates the tree while running,
    which is unsafe if anything else can see it.
    """
    out: list[int] = []
    node = root
    while node:
        if node.left is None:
            out.append(node.val)
            node = node.right
            continue
        predecessor = node.left
        while predecessor.right is not None and predecessor.right is not node:
            predecessor = predecessor.right
        if predecessor.right is None:
            predecessor.right = node          # make the temporary link
            node = node.left
        else:
            predecessor.right = None          # remove it again
            out.append(node.val)
            node = node.right
    return out

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
            i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
            i += 1
    return root

if __name__ == "__main__":
    root = from_list([1, 2, 3, 4, 5, 6])

    print(preorder(root))      # [1, 2, 4, 5, 3, 6]
    print(inorder(root))      # [4, 2, 5, 1, 6, 3]
    print(postorder(root))    # [4, 5, 2, 6, 3, 1]

    print(preorder_iterative(root) == preorder(root))    # True
    print(inorder_iterative(root) == inorder(root))      # True

```

```

print(postorder_iterative(root) == postorder(root))      # True
print(postorder_iterative_honest(root) == postorder(root)) # True
print(morris_inorder(root) == inorder(root))            # True

for order in ("pre", "in", "post"):
    print(order, traverse(root, order))

# a BST: inorder is sorted
bst = from_list([5, 3, 8, 2, 4, 7, 9])
print(inorder(bst))                                     # [2, 3, 4, 5, 7, 8, 9] ← sorted
print(kth_smallest(bst, 3))                             # 4

# the same tree, three jobs
sums: list[int] = []
subtree_sums(root, sums)
print(sums)                                              # [4, 5, 11, 6, 9, 21] ← children first
print(paths_to_leaves(root))                            # [[1,2,4],[1,2,5],[1,3,6]]
print(inorder(copy_tree(root)) == inorder(root))        # True

print(preorder(None), inorder(None), postorder(None))   # [] [] []
single = TreeNode(7)
print(preorder(single), inorder(single), postorder(single)) # [7] [7] [7]

```

6 What it costs

Time

every traversal: each node is visited exactly once → $O(n)$

All six functions are $O(n)$, recursive and iterative alike. There is no order that is faster; they differ only in *when* the work happens. If an interviewer asks which is fastest, that is the answer.

More precisely, the recursion makes $2n + 1$ calls — one per node plus one per `None` child — so the constant factor includes the empty calls. That is why some implementations check the child for `None` before recursing rather than at the top of the function; it halves the call count and makes the code uglier.

Space

recursive:	$O(\text{height})$	call stack
iterative pre/in/post	$O(\text{height})$	explicit stack
Morris inorder	$O(1)$	– no stack at all
output list	$O(n)$	– usually not counted as "extra"

balanced tree, $n = 1,000,000$	height ~ 20	→ ~ 20 frames
degenerate tree, $n = 10,000$	height 9,999	→ RecursionError

$O(\text{height})$, not $O(n)$ — and that is the difference from tomorrow's level-order walk, which is $O(\text{width})$:

```
perfect tree, n = 1,000,000
depth-first (today)      ~20 nodes held at once
breadth-first (tomorrow) ~500,000 nodes held at once
```

A factor of twenty-five thousand, in opposite directions depending on the shape. On a degenerate tree the numbers swap: depth-first holds a million frames and breadth-first holds one node.

The iterative versions, compared

preorder iterative	one stack, visit on pop	simplest
inorder iterative	one stack, the go-left loop	the shape to memorise
postorder by reversal	one stack + one reversal	2 extra lines
postorder honest	one stack + last_visited pointer	~6 extra lines
unified template	one stack of (node, flag) pairs	~2x slower, one thing to remember

The unified version pushes each node twice and allocates a tuple per push, so it is roughly twice as slow and allocates far more. **Fine in an interview, not what you would ship.**

Morris traversal, since it always gets mentioned

```
time    O(n)  - each edge is traversed at most three times, so still linear
space   O(1)  - no stack, no recursion
```

It achieves $O(1)$ by **temporarily modifying the tree**: linking a node's rightmost left-descendant back to it, then removing the link on the way through. That is the catch worth stating: **the tree is briefly in an inconsistent state, so it is unsafe if anything else can read the tree concurrently, and it cannot be used on a read-only structure.**

Know the name, know the cost, know the catch. Writing it in an interview is almost never the right use of ten minutes.

The recursion limit, stated as a rule

```
Python default recursion limit: 1000
→ a recursive traversal fails on any tree taller than ~1000
→ a tree of n nodes can be up to n tall
```

If the constraint says `n ≤ 105` and does not promise balance, the recursive version can fail. Either write it iteratively or say the risk out loud. Interviewers accept the recursive version plus that sentence; they do not accept the recursive version plus silence.

7 The traps

Trap 1 — pushing left before right in iterative preorder

```
if node.left: stack.append(node.left)
if node.right: stack.append(node.right)
```

A stack reverses, so this produces `node, right, left`. **No error, and the output looks like a traversal** — just not the one asked for. Check with a three-node tree: `[1,2,3]` must give `1 2 3`, and this gives `1 3 2`.

Trap 2 — `while stack` instead of `while stack or node` in iterative inorder

```
while stack:                                # WRONG
    while node:
        ...
```

```
inorder_iterative(root) → []
```

The stack is empty at the start, so the loop body never runs and you get nothing back. The `or node` is what lets the first iteration happen.

Trap 3 — thinking inorder means something on a non-BST

Inorder on an arbitrary binary tree produces a valid sequence with no useful property. If a candidate says “I will use inorder to get the values in sorted order” about a tree that was never a BST, that is a real misunderstanding. **Inorder is sorted order because of the BST property, not because of the traversal.**

Trap 4 — using preorder where the answer comes from the children

```
def height(node):
    if node is None:
        return -1
    h = 1 + max(...)           # cannot be computed here
    height(node.left)
    height(node.right)
    return h
```

You cannot compute a height before asking the children. Anything of the form “combine my children’s answers” is postorder, and trying to do it in preorder position produces code that either does not compile in your head or quietly returns nonsense.

Trap 5 — freeing a node before its children

```
def destroy(node):
    free(node)           # preorder
    destroy(node.left)   # the pointer is already gone
```

In a language with manual memory management this is a use-after-free. **Deletion is postorder**, always: children first, then the parent. Python’s garbage collector hides this, which is exactly why it is worth knowing — it is the standard C++ interview version of the same question.

Trap 6 — forgetting the copy when carrying a path down

```
if is_leaf(node):
    out.append(trail)           # a REFERENCE to the mutating list
```

```
paths_to_leaves(root) → [[], [], []]
```

[Day 091’s](#) trap, in a tree. Preorder traversals that carry state down are backtracking, and they need the copy and the un-choose.

Trap 7 — recursion on a deep tree

```
preorder(chain_of_10000_nodes)
```

```
RecursionError: maximum recursion depth exceeded
```

Not a hypothetical: a BST built by inserting sorted values is exactly this shape, and it is a common test case.

Trap 8 — assuming the traversal order identifies the tree

One traversal is not enough to reconstruct a tree. `[1, 2, 3]` as a preorder could be several different trees. You need **two** traversals, one of which must be inorder — which is [day 110](#) — or a traversal that records the `None`s, which is [day 111](#). Claiming that preorder alone determines the tree is a common and confident error.

8 In the interview

How it gets asked

- The pair: “Print the tree in inorder. Now do it iteratively.” LeetCode 94, and 144 and 145 for the other two.
- The choice question: “Which traversal would you use for this, and why?”
- The BST link: “Find the k -th smallest element in a BST.” LeetCode 230.
- The space probe: “Can you do it in $O(1)$ space?”
- The stack probe: “What is the space complexity of your recursion?”

What to say out loud, in the first ninety seconds

1. **Say that it is one function, not three.** “All three traversals are the same recursion with the visit on a different line — before the children, between them, or after. So I will write the skeleton and then decide where the work goes.”
2. **Choose by reason, not by name.** “The parent needs to exist before I attach children, so this is preorder.” Or: “I cannot compute my answer until I have both children’s, so this is postorder.”
3. **Say what inorder is actually for.** “Inorder is only interesting on a binary search tree, where it gives sorted order. On an arbitrary tree it has no particular meaning.”
4. **Give the complexity in both parts.** “ $O(n)$ time — every node once — and $O(\text{height})$ space for the stack. That is about twenty frames on a balanced million-node tree and a million on a degenerate one.”
5. **Flag the recursion limit before they do.** “If the tree can be a chain and n is 10^5 , the recursive version exceeds Python’s limit, so I would either write it iteratively or say I would raise the limit.”
6. **Offer the iterative version.** “Preorder is the easy one; inorder has a specific loop shape; postorder I would do as a reversed node-right-left preorder, and I can do it properly with a last-visited pointer if you prefer.”

The follow-ups

“Now do it iteratively.” “Which one? They are genuinely different. **Preorder** is easiest: a stack, pop, visit, push right then left — right first because a stack reverses. **Inorder** has a specific shape: walk left as far as you can pushing everything you pass, and when you cannot go further left, the top of the stack is the next node in order; visit it and move to its right child. The detail that catches people is the loop condition — it has to be `while stack or node`, because the stack is empty when you start, so `while stack` alone never enters the loop and you get an empty list. **Postorder** is the awkward one, because when you pop a node you do not know whether its right subtree is done. The quick answer is to do a preorder as node-right-left and reverse the output — reversing node-right-left gives left-right-node. If you want it without reversing, I keep a `last_visited` pointer and only visit a node when its right child is empty or was the last thing visited.”

“Which traversal would you use, and why?” “I decide by asking who needs what. If the **parent must do something before the children** — create a node so children can be attached to it, or carry a depth or a running path downward — that is **pre-order**. If the **answer is computed from the children** — height, subtree size, subtree sum, diameter, whether the subtree is balanced — that is **postorder**, because you cannot combine answers you have not asked for yet. And **inorder** if it is a binary search tree and I want the values in sorted order. Deletion is also postorder, and that one is worth saying because in a language without garbage collection, freeing a parent before its children is a use-after-free.”

“What is the space complexity?” “`O(height)`, not `O(n)` — the stack holds one frame per level of the current path. On a balanced million-node tree that is about twenty; on a degenerate tree it is a million, and Python’s default recursion limit is a thousand, so a chain of ten thousand nodes raises `RecursionError`. Worth contrasting with breadth-first, which holds `O(width)` — on a perfect million-node tree the widest level has half a million nodes, so level-order holds five hundred thousand where depth-first holds twenty. The two are opposite, and which is cheaper depends entirely on the shape.”

“Can you do it in `O(1)` space?” “For inorder, yes — Morris traversal. It temporarily links each node’s rightmost left-descendant back to the node, so it can find its way back up without a stack, then removes the link on the way through. Still `O(n)` time, because each edge is traversed at most three times. The catch, and I would state it rather than presenting it as free: **it mutates the tree while running**, so the tree is

briefly in an inconsistent state. That makes it unsafe if anything else can read the tree at the same time, and impossible on a read-only structure. I would name it as the answer and write the stack version unless you specifically want it.”

“**Find the k-th smallest in a BST.**” “Inorder gives sorted order in a BST, so I walk inorder and stop after k nodes. Written iteratively that is $O(\text{height} + k)$ rather than $O(n)$, because I never touch the right part of the tree — which is the real reason to do this iteratively rather than recursively, since a recursive inorder naturally wants to finish. If the question is asked many times on a changing tree, I would augment each node with the size of its subtree, which makes each query $O(\text{height})$ and makes insertion and deletion maintain one extra field.”

“**Can you rebuild the tree from its preorder traversal?**” “No, not from one traversal alone — several different trees have the same preorder. You need two, and one of them must be **inorder**, because inorder is what tells you where the split between the left and right subtrees is. Preorder plus postorder is not enough for a general binary tree, though it is enough if every node has zero or two children. The other way to make one traversal sufficient is to record the `None`s as well — then a single preorder with null markers determines the tree completely, which is how serialisation works.”

A model answer

Asked: *print the tree in inorder — and then do it iteratively.*

“First, the thing that makes all three traversals one idea rather than three. The recursion is identical in every case: handle `None`, recurse left, recurse right. **The only difference is which line the visit is on** — before the children is preorder, between them is inorder, after them is postorder. So I write the skeleton and then move one line, rather than remembering three functions.

Inorder specifically is *left, node, right*. And I want to say why anyone cares, because on an arbitrary binary tree it has no particular meaning: **on a binary search tree, inorder is sorted order**, since everything to the left is smaller and everything to the right is larger. That is the whole reason it is a named traversal, and it is the basis of validating a BST, of finding the k-th smallest, and of a BST iterator.

Recursively it is four lines. $O(n)$ time, because every node is visited once, and $O(\text{height})$ space for the call stack — about twenty frames on a balanced million-node tree, but a million on a degenerate one, and Python’s limit is a thousand. So if this tree can be a chain and `n` is large, the recursive version is a real risk and I would go iterative anyway.

Iteratively, inorder is the one with a shape worth memorising. I keep a stack and a current node. **Walk left as far as I can, pushing every node I pass. When I cannot go further left, the top of the stack is the next node in order** — pop it, visit it, and then continue from its right child. Repeat until both the stack and the current node are exhausted.

The detail that catches people is the loop condition: it must be `while stack or node`. With `while stack` alone the loop never starts, because the stack is empty at the beginning, and the function silently returns an empty list.

If you want the other two iteratively: preorder is the easy one — pop, visit, push right then left, right first because a stack reverses. Postorder is the awkward one, because when you pop a node you do not know whether its right subtree is finished. The quick way is to run a preorder as node-right-left and reverse the output, since reversing that gives left-right-node. If you want it done properly I keep a `last_visited` pointer and only visit a node once its right child is empty or was the last thing visited.

And if you want $O(1)$ space for inorder, that is Morris traversal — it threads each node's rightmost left-descendant back to the node so it can find its way up without a stack, and unthreads it on the way through. Still linear time. The honest caveat is that it mutates the tree while running, so it is unsafe if anything else can read the tree concurrently.”

9 Recall card

- **All three traversals are ONE function with the visit line in a different place** — before the children (**preorder**: node, left, right), between them (**inorder**: left, node, right), after them (**postorder**: left, right, node). Do not memorise three functions; move one line.
- **Choose by reason, not by name. Parent needs to act first** → **preorder** (copying, serialising, carrying a depth/path/range *down*). **Answer computed from the children** → **postorder** (height, size, sums, diameter, balance, and **deletion** — freeing a parent first is a use-after-free). **BST and you want sorted order** → **inorder**; on a non-BST, inorder means nothing.
- **Iterative: preorder pops and visits, pushing RIGHT then LEFT** (a stack reverses). **Inorder is a shape** — go left pushing everything, pop when you cannot, then go right — and the condition must be `while stack or node` or the loop never starts and you get `[]`. **Postorder** = **node-right-left preorder, reversed**; the honest version needs a `last_visited` pointer.
- **$O(n)$ time for all of them; $O(\text{height})$ space** — ~20 frames on a balanced million-node tree, **1,000,000 on a degenerate one** (`RecursionError` past ~1,000). **Depth-first is $O(\text{height})$, breadth-first is $O(\text{width})$** — on a perfect million-node tree that is 20 against 500,000, in opposite directions depending on shape.
- **Morris inorder is $O(1)$ space** by temporarily threading each node's rightmost left-descendant back to it — still $O(n)$ time, and the catch is that it **mutates the tree while running**. And **one traversal never determines the tree**: you need two, one of which must be **inorder**, or a traversal that records the `None` s.

1 What this is, and why they ask it

A service is **stateless** when handling a request needs nothing that is only in *that particular machine's* memory. Everything the request needs either arrives with the request or comes from a shared store that all the machines can reach.

Three sentences. Stateless does not mean there is no state — **the state exists, it just does not live in the server** — and that distinction is the whole lesson. It matters because it is the precondition for everything in [yesterday's](#) load balancer and [the day before's](#) horizontal scaling: if any server can serve any request, then adding machines works, removing machines works, and a machine dying is invisible. And the moment one server holds something the others do not, all three of those stop being true at once.

They ask it because **“stateless” is the word candidates use most and understand least**. The good answer is not “so it scales” — it is the specific list of things that break: you cannot add a server without redistributing users, you cannot remove one without losing their sessions, you cannot deploy without disrupting people, and auto-scaling stops working. And the follow-up is always sticky sessions, which are the workaround that quietly gives all of that back.

2 The story

There were three doctors at the clinic near the bus stand, and for years it had worked one way and then it changed.

The old way was that each doctor kept their own patients. Dr Menon had a wooden cupboard behind his chair with his patients' folders in it. Dr Raghavan had his own cupboard. The third doctor, who changed every year or so, had a shelf.

If you were Dr Menon's patient you waited for Dr Menon. It did not matter that Dr Raghavan was sitting there with nobody in front of him — he could not see you, because he did not have your folder and had no idea what you had been given last time.

Two things went wrong with that, regularly.

The queues were never even. On a Tuesday there would be fourteen people waiting for Menon and two for Raghavan, and everybody could see it, and it made people angry in a way that was hard to argue with.

And when Dr Menon's mother fell ill in Trichy and he was away for eleven days, his patients simply could not be seen. Not slowly — at all. Somebody found the cupboard key eventually, and then the folders were on a desk in a heap and nobody could find anything.

The new receptionist changed it, and it took her about two months to push through because the doctors did not like it.

She put every folder in one cabinet at the front desk. When your name was called, whoever gave you your folder gave it to whichever doctor was free, and that doctor read it and saw you.

Nobody's folder belonged to a doctor any more. It belonged to the cabinet.

What that bought was not obvious until it happened. In December they were very busy and they brought in a fourth doctor for three weeks — a young woman doing a locum — and she just started working. There was no handover, no list of which patients were hers. She called a name, took the folder, read it, and saw the patient. On her last day she stopped, and nothing had to be untangled.

The old receptionist, who still came in sometimes, said the folders had always been in the cupboards and what was the difference.

The new one said the difference was that before, you had to ask which doctor, and now you only had to ask which folder.

3 The idea in plain English

The clinic has just gone from stateful servers to stateless ones, and every consequence in the topic appeared in the story.

- Each doctor's own cupboard is **server-local state**. The folder exists only on that machine.
- The shared cabinet at the front desk is a **shared store** — a database, or Redis.
- “Whichever doctor is free” is the **load balancer**, and it can only work that way once the folders moved.
- The locum starting with no handover is **elastic scaling**: add a machine and it is immediately useful.

- Dr Menon away for eleven days is a **server failure**, and the difference between “his patients wait” and “nobody notices” is the entire argument.

What stateless actually means

Stateless does not mean stateless. The state is still there. It has moved.

STATEFUL SERVER

request → server 2
server 2 holds the
session in memory
and answers

STATELESS SERVER

request → any server
server reads state from
a shared store, uses it,
writes back, and forgets

The precise definition worth memorising:

A service is stateless if any instance can handle any request, and killing an instance mid-life loses nothing that mattered.

That is a testable claim. Take one machine out at random; if some users are affected differently from others, it was not stateless.

The four things that break without it

This is the answer to “why should a web server be stateless”, and it is a list, not a slogan.

One: you cannot add a machine. New machine, no sessions, so a third of users hitting it are logged out. Adding capacity causes an incident.

Two: you cannot remove a machine. Whatever it held is gone. Scaling down is destructive, so you stop doing it, so you pay for peak capacity all night.

Three: you cannot deploy. Every deployment restarts every process, and each restart drops whatever those processes were holding. Deployment becomes a scheduled outage, which means you deploy rarely, which means each deployment is bigger and riskier.

Four: a crash is user-visible. The load balancer’s health check removes the machine correctly and promptly — and its users are still logged out, because the thing they needed was inside it.

Say all four. “It scales better” is one sentence; those four are the actual content.

What counts as state, including the ones people miss

The obvious one is the login session. The others are where real systems go wrong:

STATE	WHERE IT WRONGLY LIVES	WHERE IT BELONGS
Login session	server memory	Redis, or a signed token in the cookie
Shopping cart	server memory	database or Redis
In-process cache	server memory	fine, if it is a pure optimisation
Uploaded file, mid-upload	the local disk	object storage, or a resumable upload
Rate-limit counters	server memory	Redis — otherwise the limit is <code>n × limit</code>
WebSocket connection	that machine, unavoidably	see below
Scheduled jobs	every machine's timer	a leader, or a scheduler service
Generated ids from a counter	a per-process counter	UUIDs, or a shared allocator

The rate-limit one is worth its own sentence because it is silent: with ten servers each keeping their own count, a limit of “100 requests per minute” is really 1,000, and nothing in the logs says so.

Scheduled jobs are the other silent one: a nightly job in every process means the job runs once per machine. Ten servers, ten billing runs.

The in-process cache exception, which is worth getting right

An in-memory cache is state, and it is usually **fine**:

```
acceptable:  a cache — if it is missing, the server fetches from the source and is
              merely slower. Correctness does not depend on it.
not acceptable: anything where a miss produces a WRONG ANSWER rather than a slow one.
```

The test: if this machine lost its memory right now, would the answer be wrong, or just slow? Slow is fine. Wrong is state that must move.

Sticky sessions: the workaround, and why it is a bug

Sticky sessions (also *session affinity*) mean the load balancer sends each user back to the same server every time — usually with a cookie, or by hashing the client address.

It works. It also gives back most of what the load balancer was for:

- **Load becomes uneven.** Users are not identical and sessions are long, so one machine ends up with the heavy users and the balancer cannot correct it.
- **A crash is user-visible again.** The server dies and its users lose their sessions — precisely the failure the balancer existed to hide.

- **You cannot drain for deployment** without disrupting the users pinned to that machine.
- **Auto-scaling is broken.** A new machine attracts only new users, so it stays cold while the old ones stay hot; and removing a machine is destructive.

The sentence to say: sticky sessions are not a scaling technique, they are a symptom of state in the wrong place. They are a legitimate short-term measure — sometimes the only option for a legacy application — and they should be described that way rather than as a design.

Where to put the session instead

Three options, and each is a real trade.

In a shared store (Redis). The server reads the session by id on every request. Simple, revocable instantly, and it costs one network round trip — sub-millisecond in the same data centre. **This is the default.**

In a signed token in the cookie — a JWT, from [day 020](#). The state travels with the request, so there is no store at all and no lookup. The price is real:

- **You cannot revoke it.** It is valid until it expires, so “log out everywhere” and “this account is compromised” do not work without adding a revocation list — which is a shared store, which is the thing you were avoiding.
- **It is sent on every request,** so a large token is bandwidth on every call.
- **It cannot be updated by the server** — changing a permission means the old token keeps the old permission until it expires.

In the database. Correct, durable, and slower than Redis for something read on every single request.

The usual answer: short-lived signed token for identity, Redis for anything that must be revocable or mutable.

The special case: WebSockets

A WebSocket connection is genuinely pinned to one machine — the socket exists there and nowhere else. You cannot make that stateless.

What you do instead is **keep only the connection on the machine and none of the meaning:** the conversation, the presence, the undelivered messages all live in a shared store, and the machine holds a socket and a user id. Then losing a machine drops connections, clients reconnect — possibly to a different machine — and nothing is lost but a second of downtime.

“Connections are stateful; the application does not have to be” is the sentence.

4 The picture

The two designs, side by side.

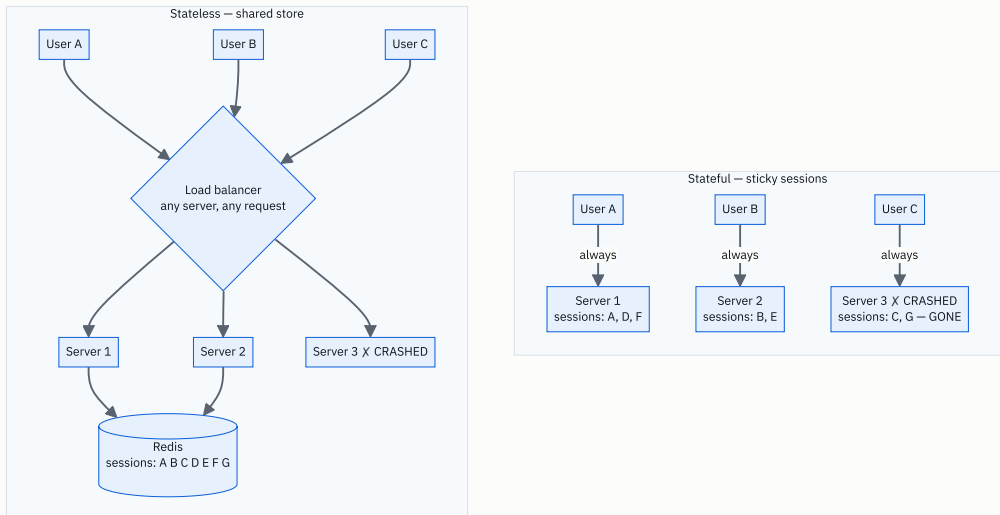


FIGURE 100.1

What to notice: in the stateful design, **the crash of server 3 is visible to users C and G and to nobody else** — an unfair, unpredictable, hard-to-explain failure. In the stateless design the crash costs one retry.

What actually moves where:

```
REQUEST ARRIVES
|
+-- carried IN the request:  the user's identity token, the request body,
|                           everything about what to do
|
+-- read from SHARED STORE:  session, cart, permissions, counters
|
+-- held ON THIS MACHINE:    nothing that matters
|                           (a cache is allowed: a miss is SLOW, not WRONG)
|
+-- written back:            to the shared store, before responding

TEST: kill this machine now. Was anything lost that mattered?
```

The cost of the two session designs, drawn:

SESSION IN REDIS

request carries a session id
server does 1 lookup ~0.5 ms
store: ~1 KB × active users
revoke: delete the key instant
update: write the key instant
bandwidth: ~40 bytes/request

1M active users:

Redis: 1 GB, and one round trip per request
token: 0 GB, and 800 MB/day of extra upload at 1,000 req/s

SESSION IN A SIGNED TOKEN

request carries the whole session
server verifies a signature ~0.05 ms
store: NOTHING
revoke: IMPOSSIBLE until it expires
update: not until the next login
bandwidth: ~800 bytes on EVERY request

5 How it actually works

Making a service stateless, in order

Step 1 — find the state. Search the codebase for module-level variables, singletons, in-memory dictionaries, `session[...]` backed by memory, local file writes, and timers. That list is the work.

Step 2 — classify each one. For each: *if this machine lost its memory now, would the answer be wrong or just slow?* Slow can stay. Wrong must move.

Step 3 — move it. Sessions and counters to Redis. Uploads to object storage. Scheduled jobs to a scheduler, or behind a leader election. Generated ids to UUIDs or a shared allocator.

Step 4 — verify by killing something. Terminate one instance during a load test and watch the error rate. If it is a blip, you are stateless. If a fraction of users see errors proportional to your instance count, you are not.

Step 5 — turn off stickiness. If it was on, it can now be off, and only now.

The session lookup, concretely

```
GET /orders  
Cookie: sid=8f2a...
```

1. server extracts sid	~0 µs
2. GET session:8f2a from Redis	~0.3-0.5 ms in the same DC
3. server handles the request	
4. SET session:8f2a with a refreshed TTL	~0.3 ms (often skipped)

Under a millisecond, and that is the entire cost of statelessness. Compare it against the cost of not being able to deploy without an outage, and the trade is not close.

Two implementation details worth mentioning:

- **The TTL is the session expiry.** Redis expires it automatically, so there is no cleanup job.
- **Do not write the session on every request** unless something changed. A read-only request that refreshes the TTL doubles your Redis traffic for very little.

The shared store is now the critical dependency

This is the honest cost, and a good candidate raises it before the interviewer does.

Every request now depends on Redis being up. You have removed a per-machine failure and created a shared one. Three things follow:

- **Redis must be replicated**, with automatic failover — otherwise you replaced ten small failures with one total one.
- **Decide what happens when it is down.** Fail the request, or fall back to a signed token, or serve logged-out content? That is a product decision and having an answer is a strong signal.
- **It is now a hot dependency** at the full request rate, so size it: 6,000 QPS against Redis is nothing (it does 100,000+), but it is a real capacity line.

What real systems do

- **Twelve-Factor App**, the widely cited deployment guide, states it directly: *processes are stateless and share nothing; any data that must persist goes in a stateful backing service*. That phrasing is worth borrowing in an interview.
 - **Kubernetes** encodes the distinction in its object types: a `Deployment` for stateless pods, which are interchangeable and can be replaced freely, and a `StatefulSet` for pods with stable identities and attached storage — used for databases, and deliberately harder to operate.
 - **AWS auto-scaling groups** assume statelessness. Instances are launched and terminated on a metric, and scale-in picks an instance without asking what is on it.
 - **Netflix's Chaos Monkey** terminates production instances at random during working hours. It only makes sense as a practice if the services are stateless — it is statelessness enforced by experiment rather than by policy.
 - **Session stores in practice:** Redis or Memcached for server-side sessions, and signed cookies (JWT or a framework's own signed session) where revocation is not needed.
-

6 The numbers

The cost of moving the session out

```
Redis GET, same data centre      ~0.3 - 0.5 ms
typical request handling         ~50 - 200 ms
→ the lookup is 0.25% - 1% of the request
```

Under one percent. That is the number that ends the argument.

```
6,000 peak QPS × 1 session read    = 6,000 Redis ops/s
one Redis instance                 = 100,000+ ops/s
→ ~6% of one instance
```

Session storage

```
session record: user id, roles, expiry, a little context  ~1 KB
1,000,000 active sessions × 1 KB                          = 1 GB
with a 30-minute TTL, "active" means the last 30 minutes
```

One gigabyte for a million concurrent sessions, on a machine with a hundred. This is not a capacity problem, and saying so pre-empt the objection.

Signed tokens instead

```
a JWT with a few claims          ~600 - 900 bytes
sent on EVERY request
1,000 req/s × 800 B              = 800 KB/s = ~69 GB/day of extra upload
→ at ~£7/GB egress-equivalent, a real number if it crosses a billing boundary
→ and mobile users pay it on their own data
```

Trade one Redis lookup for 800 bytes on every request. For a chatty API that is a bad trade; for a service with few, large requests it is a good one.

The cost of stickiness, measured

```
10 servers, sticky sessions, one dies:
  users affected          10% of logged-in users, immediately
  they see               logged out, cart lost, form contents gone

10 servers, stateless, one dies:
  users affected          the in-flight requests only
  they see               one retry, ~200 ms, or nothing at all
```

And in normal operation:

deploys per day, stateful with stickiness	~1, in a maintenance window
deploys per day, stateless with draining	as many as you like, zero errors

That second table is the real argument. The failure case is dramatic; the deployment case is what you live with every day.

Load imbalance under stickiness

- 10 servers, sticky, sessions lasting ~30 minutes, heavy users are 5% of traffic
- observed spread between busiest and quietest server: commonly 2-3×
- you must provision every server for the BUSIEST one
- ~30-50% wasted capacity

Stickiness costs money continuously, not only during incidents, and that is the argument that lands with people who are not worried about failure.

The rate-limiter bug, in numbers

intended limit	100 requests per minute per user
in-memory counters, 10 servers, round-robin balancing	
→ each server sees ~10% of that user's requests	
→ each allows 100	
→ effective limit	1,000 requests per minute

Ten times the intended limit, with nothing in the logs to say so. This is the most common concrete example of accidental state, and it is worth naming because it is a correctness bug rather than a performance one.

7 The trade-offs

Session in Redis, or in a signed token?

Redis costs one sub-millisecond lookup per request and gives you instant revocation, server-side updates, and small cookies. **Take it by default.**

A **signed token** removes the lookup and the store entirely, which genuinely matters for a service that must work across regions without a shared store, or at extreme scale where the lookup is a real line item. **I would not use a token alone** if logging out has to be immediate, if permissions change during a session, or if the session data is large enough to matter on every request. The usual compromise is a short-lived token — five to fifteen minutes — with a refresh token that is checked against a store, so revocation takes effect within one token lifetime.

Statelessness costs a shared dependency

You have traded n independent partial failures for one shared total failure. **That is a real trade, not a free win.** The mitigation is that the shared store is a well-understood, replicated, failover-capable component, and it is one thing to make reliable rather than n things — but if it is a single un-replicated Redis, you have made availability worse, and you should say so before someone else does.

In-process caching: allowed, with a rule

Keeping a cache in each server's memory is faster than any shared store and it is legitimate. The rule is **a miss must produce a slow answer, never a wrong one**, and the corollary is that per-machine caches can disagree with each other for as long as their TTL — so anything requiring all users to see the same value at the same time cannot live there.

When stateful is genuinely right

- **Databases.** They are stateful by definition, and the whole point of pushing state into them is that they are built for it.
- **WebSocket and long-lived connections.** The connection is pinned; keep the meaning elsewhere.
- **Stream processing with local state** — a windowed aggregation over a Kafka partition keeps state on the machine deliberately, and recovers it from a changelog if the machine dies. That is stateful by design, with a recovery story.
- **Very large per-user working sets**, where fetching from a shared store on every request would cost more than the request itself. Then you shard users to machines deliberately — which is stickiness, but chosen and managed rather than inherited.

Naming these matters, because “everything should be stateless” is as unthinking as “everything should be sticky”.

Where this design breaks

- **The shared store becomes the bottleneck** long before the stateless tier does. Your app servers scale to a hundred; Redis is still one logical thing.
- **Statelessness does not survive careless code.** One module-level dictionary added in a hurry, and the property is quietly gone with no error. The only reliable check is the experiment: kill an instance and watch.
- **Sequential ids from a per-process counter** collide across machines silently — two servers both issue order number 1041. This is one of the nastiest examples because it corrupts data rather than degrading service.

8 In the interview

How it gets asked

- The direct one: *“Why should a web server be stateless?”*
- The follow-up, always: *“So where does the session live?”*
- The trap: *“Could you use sticky sessions instead?”*
- The probe: *“What in your design is holding state right now?”*
- The extension: *“What about WebSockets? They are inherently stateful.”*

What to say out loud, in the first ninety seconds

1. **Define it precisely.** “Stateless means any instance can handle any request, and killing an instance loses nothing that mattered. It does not mean there is no state — the state exists, it just does not live in the server.”
2. **Give the four consequences, not a slogan.** “Without it: I cannot add a machine without logging some users out, cannot remove one without losing their sessions, cannot deploy without disrupting people, and a crash is user-visible to exactly the fraction of users who were on that box.”
3. **Say where the state goes.** “Session in Redis, keyed by a session id in the cookie. That is one lookup of about half a millisecond against a request that takes fifty to two hundred — under one percent.”
4. **Pre-empt the sticky-session question.** “Sticky sessions are not a scaling technique; they are a symptom of state in the wrong place. They give back the uneven load, the visible crashes, the inability to drain, and auto-scaling.”
5. **Name the state people forget.** “The obvious one is the session. The ones that bite are rate-limit counters — with ten servers each counting separately, a hundred-per-minute limit is really a thousand — and scheduled jobs, which run once per machine unless something elects a leader.”
6. **Admit the cost.** “The trade is that every request now depends on the shared store, so I have swapped ten partial failures for one total one. That store has to be replicated with automatic failover, and I should decide what happens to a request when it is unavailable.”

The follow-ups

“So where does the session live?” “Default is Redis, keyed by a session id in the cookie. One `GET` of about half a millisecond in the same data centre, against a request that takes fifty to two hundred — so under one percent of the request. A million active sessions at about a kilobyte each is one gigabyte, which is nothing. The alternative is a signed token — a JWT — where the state travels in the cookie and there is no store or

lookup at all. That is genuinely attractive for a multi-region service, and it costs three things: **you cannot revoke it**, so ‘log out everywhere’ does not work until it expires; the server cannot update it, so a permission change does not take effect until the next login; and it is sent on every request, so eight hundred bytes times a thousand requests a second is about sixty-nine gigabytes a day of extra upload. The usual compromise is a short-lived token for identity plus a refresh token checked against a store, so revocation takes effect within one token lifetime.”

“Could you use sticky sessions instead?” “You can, and it works, and I would describe it as a temporary measure rather than a design — often the only option for an application you cannot change. What it costs is most of what the load balancer was for. Load becomes uneven, because users are not identical and sessions last half an hour; in practice the spread between the busiest and quietest server is two to three times, and you have to provision every server for the busiest one, so you are paying thirty to fifty percent extra continuously. A crash becomes user-visible again: ten servers, one dies, ten percent of logged-in users are logged out — the exact failure the balancer existed to hide. You cannot drain a server for deployment without disrupting the users pinned to it. And auto-scaling stops working, because a new machine only attracts new users and stays cold while the old ones stay hot.”

“What in your design is holding state right now?” “Let me go through it deliberately, because the obvious one is not the dangerous one. The session — moved to Redis. **Rate-limit counters** — these must be shared, because with ten servers each counting locally, a limit of a hundred a minute is really a thousand, and nothing in the logs tells you. **Scheduled jobs** — a nightly billing run in every process runs once per machine, so that needs a leader or a scheduler service. **File uploads** — writing to local disk means the retry lands on a different machine and cannot find the partial file, so those go to object storage. **Generated ids** — a per-process counter collides across machines and issues the same order number twice, so UUIDs or a shared allocator. And an **in-process cache** is fine, on one condition: a miss must make the answer slow, never wrong.”

“What about WebSockets? They are inherently stateful.” “The connection is, and there is no way around that — the socket exists on one machine. What I can do is make sure the machine holds **only the connection and none of the meaning**. The conversation history, the presence state, and undelivered messages all live in a shared store; the machine holds a socket and a user id. Then when a machine dies, its clients drop, reconnect — possibly to a different machine — and nothing is lost but a second. Delivering a message to a user on another machine needs a pub/sub layer: the sender’s

machine publishes to a channel for that user, and whichever machine holds their connection is subscribed and pushes it down. So: connections are stateful, the application does not have to be.”

“What does statelessness cost you?” “A shared dependency on the request path. Every request now needs the session store, so I have traded ten independent partial failures for one shared total one — and if that store is a single un-replicated instance, I have made availability worse rather than better. So it has to be replicated with automatic failover, and I should decide explicitly what a request does when it is unavailable: fail closed, fall back to a signed token, or serve logged-out content. The other cost is that the property is fragile against careless code — one module-level dictionary added in a hurry and it is silently gone. The only reliable check is the experiment: terminate an instance during a load test and see whether the error rate is a blip or a fixed fraction of users.”

“How would you verify a service is actually stateless?” “Kill an instance under load and look at the shape of the errors. A blip that recovers on retry means stateless. A sustained error rate that is roughly one over the number of instances, affecting the same users repeatedly, means something was in there. That experiment is exactly what Netflix’s Chaos Monkey institutionalises — terminating production instances at random during working hours, which only makes sense as a practice if the services genuinely are stateless. I would also grep for module-level mutable variables, local file writes and timers as a static check, but the experiment is the one that finds what the grep misses.”

A model answer

Asked: *why should a web server be stateless?*

“Let me start with what the word means, because it is used loosely. **Stateless means any instance can handle any request, and killing an instance mid-life loses nothing that mattered.** It does not mean there is no state — the session, the cart, the counters all still exist. They have just moved out of the server’s memory into somewhere every server can reach.

The reason is not really ‘it scales better’. It is four specific things that stop working without it.

I cannot add a machine. A new instance has no sessions, so once the balancer starts sending it traffic, a share of users are logged out. Adding capacity causes an incident, which is the wrong way round.

I cannot remove one. Whatever it held is gone, so scaling in is destructive — which means you stop doing it and pay for peak capacity at three in the morning.

I cannot deploy. Every deploy restarts every process and drops what they were holding, so deployment becomes a maintenance window. That makes you deploy rarely, which makes each deploy larger and riskier.

And a crash becomes user-visible. With ten servers, one dying logs out ten percent of users — the exact failure the load balancer was supposed to hide.

So the session goes into Redis, keyed by a session id in the cookie. That is one lookup of roughly half a millisecond against a request that takes fifty to two hundred — under one percent — and a million active sessions at a kilobyte each is one gigabyte. Neither number is an obstacle.

The state people forget is more interesting than the session. **Rate-limit counters** kept in memory mean ten servers each allow the full limit, so ‘a hundred a minute’ is really a thousand and nothing in the logs says so — that is a correctness bug, not a performance one. **Scheduled jobs** run once per machine unless something elects a leader. **File uploads** to local disk break when the retry lands elsewhere. **Sequential ids from a per-process counter** collide silently across machines. An **in-process cache** is fine, on one condition: a miss must make the answer slow, never wrong.

On sticky sessions, which is usually the next question: they work, and they are not a scaling technique — they are a symptom of state in the wrong place. They give back the even load, the invisible crashes, the ability to drain for a deploy, and auto-scaling. In steady state they typically leave a two to three times spread between the busiest and quietest server, and you have to size every server for the busiest.

And I would name the cost honestly: every request now depends on the session store, so I have swapped ten partial failures for one shared total one. That store needs replication and automatic failover, and I need an explicit answer for what a request does when it is unavailable.”

9 Recall card

- **Stateless means any instance can serve any request, and killing one loses nothing that mattered.** It does **not** mean there is no state — the state moved out of the server into a shared store. The test: *if this machine lost its memory now, would the answer be wrong or just slow?* Slow is fine.
- **Four things break without it, and the list is the answer:** you cannot **add** a machine (new users logged out), cannot **remove** one (their sessions gone), cannot **deploy** without disruption, and a **crash becomes user-visible** to exactly $1/n$ of users.
- **Session in Redis by default** — one lookup of ~ 0.5 ms against a 50–200 ms request, **under 1%**, and $1\text{M sessions} \times 1\text{ KB} = 1\text{ GB}$. A **signed token** removes the store but **cannot be revoked**, cannot be updated server-side, and costs ~ 800 bytes on **every** request ($\sim 69\text{ GB/day}$ at $1,000\text{ req/s}$). Usual answer: short-lived token plus a refresh token that is checked.
- **The state people forget: rate-limit counters** (10 servers each allowing 100 = an effective limit of **1,000**, silently — a correctness bug), **scheduled jobs** (one run per machine), local **file uploads**, and **per-process id counters** (two servers issue the same order number). An in-process cache is allowed if a miss is only slow.
- **Sticky sessions are not a scaling technique — they are a symptom of state in the wrong place.** They cost a $2\text{--}3 \times$ **spread** between busiest and quietest server (so 30–50% wasted capacity), user-visible crashes, no draining, and broken auto-scaling. And the honest cost of going stateless: **you swap n partial failures for one shared dependency**, so the store must be replicated with failover. **WebSocket connections are pinned — keep the connection on the machine and the meaning in the store.**

PRACTICE

Practice — Depth-first traversal: preorder, inorder, postorder

DSA topic: Depth-first traversal: preorder, inorder, postorder **System design topic:** Stateless services and why they scale

Code these, in this order

One rule for the whole set: **do not write three functions**. Write the skeleton — handle `None`, recurse left, recurse right — and then decide which of the three lines the visit goes on, by asking who needs what. Then do each one iteratively.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Binary Tree Preorder Traversal	LeetCode 144 (Easy)	The iterative version, and pushing right before left.
2	Binary Tree Inorder Traversal	LeetCode 94 (Easy)	The go-left loop shape, and <code>while stack or node</code> .
3	Binary Tree Postorder Traversal	LeetCode 145 (Easy)	The reversal trick, then the honest version.
4	Kth Smallest Element in a BST	LeetCode 230 (Medium)	Why inorder is the answer, and why iterative beats recursive here.

On all three of 1 to 3, do the recursion first and time yourself

Each should take under a minute. If any takes longer, you are recalling rather than deriving — write the skeleton and move the line.

On problem 1, get the push order wrong on purpose

Push left before right, run it on `[1,2,3]`, and record the output. Say why it looks plausible.

On problem 2, remove `or node` from the condition

Run it and record the result. Then say why the loop never starts.

On problem 3, write both versions

The reversal trick first, then the `last_visited` version. For the second, say in one sentence what the condition `peeked.right is None or last_visited is peeked.right` actually means.

On problem 4, do it iteratively and say why

Recursive inorder wants to finish the whole tree. Say what the iterative version costs in terms of `height` and `k`, and why that is better than `O(n)`.

The one-function drill

1. Write the skeleton with the three positions marked.
2. Produce all three orders by moving one line, without looking.
3. State each order as “where the node goes”.
4. Say what is identical across all three.

The choose-the-order drill

For each, say which traversal and why — the reason, not the name:

1. Copy a tree.
2. Compute the height of every node.
3. Print a BST’s values in ascending order.
4. Free every node in C++.
5. Record the path from the root to every leaf.
6. Compute the sum of every subtree.
7. Serialise a tree so it can be rebuilt.

Then state the one-line test that decides all seven.

The by-hand drill

1. Draw a seven-node tree.
2. Draw the anticlockwise loop and mark where each node is passed the first, second and third time.
3. Read off all three orders from the drawing.
4. Check them against code.

The iterative-preorder drill

1. Write it from memory.
2. Say why right is pushed before left.
3. Run the wrong push order on a three-node tree and record the output.
4. Say what order the wrong version actually produces.

The iterative-inorder drill

1. Write the five-line shape from memory.
2. Say the shape as one English sentence.
3. Trace it by hand on a four-node tree, recording the stack at every step.
4. Say what `while stack` alone returns, and why.

The iterative-postorder drill

1. Write the reversal version and say why reversing node-right-left gives postorder.
2. Write the `last_visited` version.
3. Say what the condition means in words.
4. Say which you would write under time pressure and which you would describe.

The Morris drill

1. Say what it achieves and at what cost.
2. Say what it does to the tree while it runs.
3. Say the two situations in which you could not use it.
4. Say whether you would write it in an interview, and what you would say instead.

The cost drill

1. State the time for all six implementations.
2. State the space for recursive, iterative and Morris.
3. Give the stack depth for a balanced and a degenerate million-node tree.
4. Compare depth-first against breadth-first on a perfect million-node tree, both directions.
5. Say what constraint bound should make you write the iterative version.

The break-it drill

Trigger each and record the exact output or error:

1. Left pushed before right in iterative preorder.
2. `while stack` instead of `while stack or node`.
3. A recursive traversal on a chain of 10,000 nodes.
4. `out.append(trail)` instead of `trail[:]` in the paths function.
5. Computing a height in preorder position.
6. Claiming a tree can be rebuilt from its preorder alone — construct two trees with the same preorder.

The definition drill

1. State the definition of stateless in one sentence, using “any instance” and “loses nothing”.
2. Say what it does *not* mean.
3. State the one-question test for whether something counts as state.
4. Say how you would verify statelessness experimentally.

The four-consequences drill

1. Name all four things that break without statelessness.
2. For each, say what a user actually experiences.
3. Say which of the four you live with every day, and which is dramatic but rare.
4. Say why “it scales better” is a worse answer than the list.

The hidden-state drill

For each, say why it is state and where it should go:

1. Rate-limit counters.
2. Scheduled jobs.
3. A file being uploaded.
4. Sequential id generation.
5. An in-process cache.
6. A WebSocket connection.

Then compute the effective rate limit for 10 servers each enforcing 100 per minute.

The session-store drill

1. Give the three places a session can live.
2. For each, state the per-request cost and the storage cost.
3. Say what a signed token cannot do, and give two situations where that matters.
4. Compute the daily extra upload for an 800-byte token at 1,000 req/s.
5. State the usual compromise and why it works.

The stickiness drill

1. Say what sticky sessions are and how they are implemented.
2. Name the four things they cost.
3. Give the typical load spread and the wasted capacity that follows.

4. Compute the fraction of users affected when one of ten servers dies, both ways.
5. State the one-sentence verdict.

The honest-cost drill

1. Say what you traded away by moving state into a shared store.
2. Say what that store now requires.
3. Give three possible answers to “what happens to a request when it is down”.
4. Say why statelessness is fragile against ordinary code changes.

The stateful-is-right drill

Name four situations where stateful is the correct design, and for each say what makes it acceptable.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Print the tree in inorder. Now do it iteratively.* One function with the visit in three places, why inorder matters only for a BST, the recursion, the complexity in both parts with the recursion-limit warning, then the go-left shape with the loop condition detail called out.
2. *Why should a web server be stateless?* The precise definition, the four consequences rather than a slogan, where the session goes with the under-one-percent number, sticky sessions pre-empted as a symptom, the state people forget with the rate-limiter example, and the honest cost of the shared dependency.
3. *Which traversal would you use, and why?* The three-way test — parent acts first, answer comes from the children, BST sorted order — with an example of each and the deletion case named.

Before you move on

- [] I can produce all three orders by moving one line, from a blank screen.
- [] I choose the order by reason and can say the reason, not the name.
- [] I can read all three orders off a drawing using the loop trick.
- [] I can write iterative preorder and say why right is pushed first.
- [] I can write iterative inorder as a shape, including `while stack or node`.

- [] I can write postorder by reversal and explain why the reversal works.
- [] I can describe the `last_visited` version in words.
- [] I can name Morris traversal, its cost, and its catch.
- [] I can state time and space for every version, and the depth-versus-width contrast.
- [] I know which constraint bound should push me to the iterative version.
- [] I know that one traversal never determines the tree, and what two do.
- [] I can define stateless precisely and say what it does not mean.
- [] I can list all four things that break without it.
- [] I can say where the session goes and give the under-one-percent number.
- [] I can name four kinds of state people forget, including the rate-limiter bug in numbers.
- [] I can give the four costs of sticky sessions and the one-sentence verdict.
- [] I can state the honest cost of the shared store and what it requires.
- [] I can name four cases where stateful is correct.
- [] I answered all three questions above out loud.

Breadth-first traversal: level order

DATA STRUCTURES AND ALGORITHMS

Breadth-first traversal: level order

You can process a tree level by level and know exactly where the level boundary is.

SYSTEM DESIGN

Caching: the single biggest win

You can place a cache at four different layers and say what each one saves.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Print the tree level by level. Now print each level as its own list.*
- *Where would you put a cache in this system?*

1 What this is, and why they ask it

Breadth-first traversal visits every node at depth 0, then every node at depth 1, then depth 2, and so on. It is the other way of walking a tree — [yesterday's](#) three orders all went deep first, and this one goes wide.

Three sentences. It uses a **queue** where depth-first used a stack, and that single swap is the whole difference. The version that prints the values in level order is six lines and everybody gets it; the version that returns **each level as its own list** requires knowing where one level ends and the next begins, and there is exactly one clean way to do that. And breadth-first is not an alternative to depth-first — **it is the right answer for a specific family of questions**, the ones about levels, about the shallowest anything, and about shortest paths.

They ask it because the follow-up — “*now group them by level*” — separates people. The answer is one line: **capture the length of the queue before you start draining it**. That number is exactly one level, because everything you add during the loop belongs to the next one. Candidates who have not seen the trick invent something with sentinel values or a dictionary keyed by depth, both of which work and both of which are three times the code.

2 The story

The tanker came on Tuesdays and Fridays at about six in the morning, and by half past five there were already thirty people standing with pots.

Selvam had been given the job of managing it after the argument in April, which had been about exactly one thing: some families were getting three pots and some were getting none.

What had been happening was obvious once you watched it. A man would fill his pot, walk twenty steps to his house, put it down, and come straight back and join the end of the queue with a second pot. The queue never got shorter. People who had been standing there since half past five were still standing there at seven, because there was always somebody rejoining the back.

Selvam's rule was very simple and he announced it every time before he started.

He would look at the line and count it. Not roughly — he counted heads, out loud, and then he said the number. Forty-one. And then he said: forty-one people, one pot each, and when I have done forty-one I stop and count again.

Anyone who joined after he had counted was not part of that forty-one. They were part of the next count. It did not matter that they were standing in the same line — the number had already been fixed.

So it went in rounds. Forty-one, then he counted again and it was twenty-six, then nine, then four. Everybody in a round got one pot before anybody got a second one.

The part he was strict about was the counting. He would not let himself start filling until he had said the number out loud, because the one time he had tried to just work down the line and stop “when it seemed right”, it had gone exactly the way it did in April.

There was a boy who asked him once why he did not just tell people not to rejoin the line.

Selvam said he had tried that and it had made everybody angry, because a family of eight genuinely does need three pots. The point was never to stop them coming back. The point was to make sure the second pot came after everybody else’s first one.

3 The idea in plain English

Selvam is doing a breadth-first traversal, and the head count before each round is the one line that matters.

- The queue of people is a **queue** in the code — first in, first out.
- One round is one **level** of the tree.
- Counting heads before starting the round is **capturing the level size**, and it is the whole trick.
- People rejoining the back during a round are the **children** you add while draining the current level; they belong to the next level, not this one.

Why a queue, and why that changes everything

depth-first	uses a STACK	→ last in, first out	→ you go deeper
breadth-first	uses a QUEUE	→ first in, first out	→ you go wider

That is the only structural difference. Take yesterday's iterative preorder, change the stack to a queue and `pop()` to `popleft()`, and you have a level-order traversal. Nothing else changes.

```
queue = deque([root])
while queue:
    node = queue.popleft()          # popleft, not pop
    visit(node)
    if node.left: queue.append(node.left)
    if node.right: queue.append(node.right)
```

Six lines, and it visits every node in level order: 1, 2, 3, 4, 5, 6. **But it does not tell you where one level ends.** The output is flat.

The level boundary, which is the actual lesson

The question is almost always “*return each level as its own list*” — `[[1], [2,3], [4,5,6]]`. And the answer is Selvam's head count:

```
while queue:
    level_size = len(queue)          # ← COUNT FIRST, before draining
    level = []
    for _ in range(level_size):
        node = queue.popleft()
        level.append(node.val)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    result.append(level)
```

Why it works, in one sentence: at the top of the outer loop the queue contains exactly one complete level, so its length is that level's size, and everything appended during the inner loop belongs to the next level.

That invariant is worth stating out loud in an interview, because it is the thing being tested. The `level_size` variable is not an optimisation — **without capturing it, `len(queue)` changes underneath you as you append**, and the inner loop never ends the way it should.

The other two ways, and why they are worse

A sentinel. Put a `None` in the queue as a level marker; when you pop it, the level is over, and you push another `None` if the queue is not empty.

Works, and it needs careful handling of the final marker to avoid an infinite loop. More state, more edge cases.

A dictionary keyed by depth. Do a depth-first walk carrying the depth down, and append each value to `levels[depth]` .

Also works, and it is genuinely useful — it is the shortest way to answer “give me every level” when you already have a recursive walk. Two things to know: it visits in depth-first order, so within a level the values still come out left to right (because the left subtree is always recursed first), and it uses `O(height)` stack instead of `O(width)` queue.

Know all three; write the `level_size` one.

Which problems need breadth-first

The rule: **anything about levels, anything about the shallowest, and anything about shortest paths.**

PROBLEM	WHY BFS
Level order traversal	the levels are the answer
Right side view	the last node of each level
Level averages, level maximums	one level at a time
Minimum depth	stops at the first leaf; DFS explores the whole tree
Zigzag / spiral order	levels, alternately reversed
Connect each node to its next right	the level is the linked list
Shortest path in an unweighted graph	the first time you reach a node is the shortest way

Minimum depth is the one to remember, because it is where BFS is not merely tidier but genuinely faster. A tree whose left branch is 10,000 deep and whose right child is a leaf: depth-first walks the whole left branch first and then finds the answer is 1. Breadth-first finds it in three steps.

And the counterpart: **use depth-first for anything computed from the children** — height, sums, diameter, balance — because those are naturally postorder and breadth-first has no convenient way to combine a node’s answer from its children’s.

Depth-first or breadth-first: the memory question

depth-first	<code>O(height)</code> — one frame per level of the current path
breadth-first	<code>O(width)</code> — the widest level, held all at once

For a **perfect** tree the widest level is the bottom one, holding about `n/2` nodes:

```
perfect tree, n = 1,000,000
depth-first    ~20 frames
breadth-first  ~500,000 nodes in the queue
```

For a **degenerate** tree it reverses completely:

```
chain, n = 10,000
depth-first    10,000 frames → RecursionError
breadth-first  1 node in the queue at any time
```

Neither is universally cheaper, and which one is safe depends entirely on the shape. That contrast is the answer to “what is the space complexity”, and giving both halves of it is what makes the answer sound considered.

The `deque`, and why not a list

```
from collections import deque
queue = deque([root])
node = queue.popleft()           #  $O(1)$ 
```

```
queue = [root]
node = queue.pop(0)              #  $O(n)$  – every remaining element shifts
```

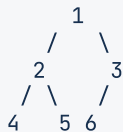
`list.pop(0)` moves every other element down by one, so a level-order traversal built on a list is $O(n^2)$ rather than $O(n)$.

```
n = 100,000 nodes
deque:  ~100,000 operations
list:   ~5,000,000,000 element moves    – minutes instead of milliseconds
```

`from collections import deque` is the first line of every BFS you write. It is the same `deque` from [day 073](#).

4 The picture

The queue over time, with the level boundaries marked. This is the diagram to be able to draw.



outer	queue at the TOP of the loop	level_size	level produced
1	[1]	1	[1]
2	[2, 3]	2	[2, 3]
3	[4, 5, 6]	3	[4, 5, 6]
4	[]	-	stop

result: [[1], [2, 3], [4, 5, 6]]

the invariant, stated:

AT THE TOP OF THE OUTER LOOP, THE QUEUE HOLDS EXACTLY ONE COMPLETE LEVEL.
So len(queue) is that level's size, and everything appended during the inner loop belongs to the NEXT level.

Selvam's rounds, drawn the same way:

```

6:00  line: 41 people      count = 41  → serve exactly 41
      (during those 41, 26 more join the back)
6:35  line: 26 people      count = 26  → serve exactly 26
      (during those 26, 9 more join)
6:52  line: 9              count = 9
6:58  line: 4              count = 4
7:01  line: 0              → done

```

the man who rejoins the queue during round 1 is served in round 2.
that is EXACTLY a node's children being appended during its own level.

Why capturing the size is not optional:

WITHOUT capturing:

```

while queue:
    for _ in range(len(queue)):
        node = queue.popleft()
        queue.append(node.left)
    ...

```

range(len(queue)) is evaluated ONCE in Python, so this happens to work – but the moment it becomes a while loop testing len(queue), the level never ends because appends keep topping it up.

WITH capturing:

```

while queue:
    n = len(queue)
    for _ in range(n):
        node = queue.popleft()
        queue.append(node.left)

```

n is fixed before any appends
→ correct, and obviously correct

Depth-first against breadth-first, on the two extreme shapes:

PERFECT TREE (n = 15)



DFS stack: 4 frames

BFS queue: 8 nodes (the bottom level)

→ DFS wins on memory

DEGENERATE TREE (n = 5)

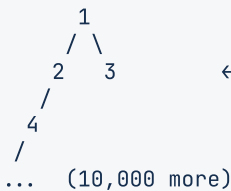


DFS stack: 5 frames

BFS queue: 1 node

→ BFS wins on memory

Minimum depth, which is the case where the choice changes the running time:



← 3 is a leaf at depth 1: the ANSWER

DFS: walks the entire left branch (10,000 nodes) before it ever sees node 3
BFS: level 0 → level 1 → finds a leaf immediately. 3 nodes visited.

5 The code, built step by step

Step 1 — `deque`, always

```
from collections import deque
queue = deque([root])
```

Never a list with `pop(0)`. It turns an $O(n)$ traversal into $O(n^2)$ with no error to tell you.

Step 2 — guard the empty tree first

```
if root is None:
    return []
```

A `deque([None])` will happily give you a `None` to call `.val` on. Handle it before the loop rather than checking inside it.

Step 3 – capture the size, and say why

```
level_size = len(queue)          # exactly one level, fixed before we append
```

Say the invariant out loud as you write it: “*at this point the queue holds exactly one complete level.*” That sentence is what the question is testing.

Step 4 – drain exactly that many

```
for _ in range(level_size):
    node = queue.popleft()
    level.append(node.val)
    if node.left: queue.append(node.left)
    if node.right: queue.append(node.right)
```

Check for `None` before appending, not after popping. Pushing `None`s and filtering later works and makes every subsequent line have to think about it.

Step 5 – the variants are all one line each

Once the skeleton is right, the family falls out:

```
result.append(level)                # level order
result.append(level[-1])             # right side view
result.append(sum(level) / len(level)) # level averages
result.append(max(level))            # level maximums
result.append(level if depth % 2 == 0 else level[::-1]) # zigzag
```

That is why the skeleton is worth memorising rather than the problems. Five interview questions, one loop, one differing line.

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

def level_order_flat(root: TreeNode | None) → list[int]:
    """Breadth-first, but the output is flat – no level boundaries.

    This is yesterday's iterative preorder with a QUEUE instead of a STACK.
    That single swap is the entire difference between the two families.
    """
    if root is None:
        return []
    out: list[int] = []
    queue = deque([root])
    while queue:
        node = queue.popleft()          # popleft, not pop
        out.append(node.val)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return out

def level_order(root: TreeNode | None) → list[list[int]]:
    """LeetCode 102. Each level as its own list.

    THE LINE THAT MATTERS: level_size = len(queue), captured BEFORE draining.

    Invariant: at the top of the outer loop the queue holds exactly one
    complete level. So its length is that level's size, and everything
    appended during the inner loop belongs to the NEXT level.
    """
    if root is None:
        return []
    result: list[list[int]] = []
    queue = deque([root])

    while queue:
        level_size = len(queue)          # ← count the heads first
        level: list[int] = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
```

```

return result

def right_side_view(root: TreeNode | None) → list[int]:
    """LeetCode 199. What you see standing to the right of the tree:
    the LAST node of each level. One line different from level_order."""
    if root is None:
        return []
    view: list[int] = []
    queue = deque([root])
    while queue:
        level_size = len(queue)
        for i in range(level_size):
            node = queue.popleft()
            if i == level_size - 1:          # the last one in this level
                view.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return view

def level_averages(root: TreeNode | None) → list[float]:
    """LeetCode 637. Same skeleton, one differing line."""
    if root is None:
        return []
    out: list[float] = []
    queue = deque([root])
    while queue:
        level_size = len(queue)
        total = 0
        for _ in range(level_size):
            node = queue.popleft()
            total += node.val
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        out.append(total / level_size)
    return out

def zigzag_level_order(root: TreeNode | None) → list[list[int]]:
    """LeetCode 103. Left to right, then right to left, alternating.

    Reverse the LIST, not the traversal. Trying to alternate the queue
    order itself is a classic way to lose an afternoon.
    """
    if root is None:
        return []
    result: list[list[int]] = []
    queue = deque([root])
    left_to_right = True

    while queue:
        level_size = len(queue)
        level: list[int] = []

```

```

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level if left_to_right else level[::-1])
        left_to_right = not left_to_right

    return result

```

```

def min_depth(root: TreeNode | None) → int:
    """LeetCode 111. THE case where BFS is genuinely faster than DFS.

    BFS stops at the first leaf it meets. DFS would walk an entire deep
    branch before discovering the answer was 1.

    Note "leaf" means BOTH children are None – a node with one child is not
    a leaf, and `1 + min(left, right)` gets that wrong.
    """
    if root is None:
        return 0
    queue = deque([(root, 1)])
    while queue:
        node, depth = queue.popleft()
        if node.left is None and node.right is None:
            return depth # first leaf found: shallowest
        if node.left:
            queue.append((node.left, depth + 1))
        if node.right:
            queue.append((node.right, depth + 1))
    return 0

```

```

def max_width(root: TreeNode | None) → int:
    """The widest level – which is also the peak size of the BFS queue,
    so this function measures its own space complexity."""
    if root is None:
        return 0
    widest = 0
    queue = deque([root])
    while queue:
        level_size = len(queue)
        widest = max(widest, level_size)
        for _ in range(level_size):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return widest

```

```

def levels_by_dfs(root: TreeNode | None) → list[list[int]]:
    """The third way: depth-first, carrying the depth DOWN as an argument.

```

Same answer, and left-to-right within each level comes free because the

left subtree is always recursed first. Uses $O(\text{height})$ stack instead of $O(\text{width})$ queue – which is the better choice on a WIDE tree and the worse one on a DEEP tree.

```
"""
```

```
result: list[list[int]] = []
```

```
def walk(node: TreeNode | None, depth: int) → None:
```

```
    if node is None:
```

```
        return
```

```
    if depth == len(result):
```

```
        result.append([])
```

```
        # first node seen at this depth
```

```
    result[depth].append(node.val)
```

```
    walk(node.left, depth + 1)
```

```
    walk(node.right, depth + 1)
```

```
walk(root, 0)
```

```
return result
```

```
def connect_next_right(root: TreeNode | None) → TreeNode | None:
```

```
    """LeetCode 116/117: link every node to the next one on its level.
```

The level IS the linked list, so this is level order with one extra line – join consecutive nodes as you drain each level.

```
"""
```

```
if root is None:
```

```
    return None
```

```
queue = deque([root])
```

```
while queue:
```

```
    level_size = len(queue)
```

```
    previous: TreeNode | None = None
```

```
    for _ in range(level_size):
```

```
        node = queue.popleft()
```

```
        if previous is not None:
```

```
            previous.next = node
```

```
            # type: ignore[attr-defined]
```

```
        previous = node
```

```
        if node.left:
```

```
            queue.append(node.left)
```

```
        if node.right:
```

```
            queue.append(node.right)
```

```
    previous.next = None
```

```
    # type: ignore[attr-defined]
```

```
return root
```

```
def level_order_slow(root: TreeNode | None) → list[int]:
```

```
    """The trap, written out so you can time it: a list with pop(0).
```

Every pop(0) shifts every remaining element down by one, so this is $O(n^2)$. On 100,000 nodes that is ~5 billion element moves.

```
"""
```

```
if root is None:
```

```
    return []
```

```
out: list[int] = []
```

```
queue = [root]
```

```
while queue:
```

```
    node = queue.pop(0)
```

```
    #  $O(n)$ , not  $O(1)$ 
```

```
    out.append(node.val)
```

```
    if node.left:
```

```
        queue.append(node.left)
```

```

        if node.right:
            queue.append(node.right)
    return out

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

if __name__ == "__main__":
    root = from_list([1, 2, 3, 4, 5, 6])

    print(level_order_flat(root))      # [1, 2, 3, 4, 5, 6]
    print(level_order(root))           # [[1], [2, 3], [4, 5, 6]]
    print(right_side_view(root))       # [1, 3, 6]
    print(level_averages(root))        # [1.0, 2.5, 5.0]
    print(zigzag_level_order(root))    # [[1], [3, 2], [4, 5, 6]]
    print(max_width(root))             # 3
    print(levels_by_dfs(root) == level_order(root)) # True

    # minimum depth: the case where BFS is genuinely faster
    lopsided = TreeNode(1, TreeNode(2, TreeNode(4, TreeNode(8))), TreeNode(3))
    print(min_depth(lopsided))         # 2 ← via node 3, found immediately
    print(level_order(lopsided))       # [[1], [2, 3], [4], [8]]

    print(level_order(None), min_depth(None)) # [] 0
    print(level_order(TreeNode(7)))        # [[7]]

    # the two shapes, and what each traversal costs
    chain = from_list([1, 2, None, 3, None, 4])
    print(max_width(chain))             # 1 ← BFS queue never exceeds one node

```

6 What it costs

Time

every node is enqueued exactly once and dequeued exactly once
 → $O(n)$

Identical to depth-first. Neither family is faster in the general case; they differ in *which node they reach first*, which is why minimum depth is faster with BFS and height is more natural with DFS.

Space — the number that actually differs

```
breadth-first:  O(width)    – the widest level, held in the queue at once
depth-first:    O(height)   – one frame per level of the current path
```

For the two extreme shapes:

```
PERFECT tree, n = 1,000,000
  width of the bottom level  ~500,000 nodes
  height                    ~20
  → BFS holds 500,000 references (~4 MB of pointers)
  → DFS holds 20 frames

DEGENERATE tree (a chain), n = 10,000
  width                    1
  height                  10,000
  → BFS holds 1 node
  → DFS: RecursionError: maximum recursion depth exceeded
```

A factor of twenty-five thousand, in opposite directions. The honest answer to “which is more memory-efficient” is “*it depends on the shape, and here are the two extremes*”.

For a random or roughly balanced tree, the widest level is still around $n/2$, so **BFS is usually the more memory-hungry of the two** — and it is the one that is *safe* from stack overflow.

The `deque` versus `list` difference, measured

```
deque.popleft()  O(1)
list.pop(0)      O(n) – every remaining element shifts down one slot

n = 1,000        deque ~1,000 ops          list ~500,000 element moves
n = 100,000      deque ~100,000 ops         list ~5,000,000,000
                                           → minutes, not milliseconds
```

This is a silent $O(n^2)$. The code is correct and the tests on a ten-node tree pass instantly.

Minimum depth: where the choice changes the complexity

tree with a leaf at depth 1 and a left branch 10,000 deep

BFS: visits 3 nodes $O(1)$ in practice

DFS: visits all 10,002 nodes $O(n)$

More generally, for the shallowest leaf at depth `d` in a tree with branching factor `b`:

BFS visits $O(b^d)$ nodes – everything above the answer, and no more

DFS visits $O(n)$ – potentially the whole tree

When the answer is shallow and the tree is deep, BFS is not tidier, it is faster.

Queue memory, concretely

a deque entry is one pointer	8 bytes
plus deque block overhead	~10-20%
500,000 entries	~4-5 MB

Small in absolute terms — the point is not that BFS runs out of memory on a million-node tree, it is that **the growth is in width**, so it is **unbounded in a way $O(\text{height})$ is not** for shapes like a wide-and-shallow tree.

7 The traps

Trap 1 — a list instead of a deque

```
queue = [root]
node = queue.pop(0)
```

Correct output, $O(n^2)$ running time, no error. On a hundred thousand nodes it is the difference between milliseconds and minutes. `from collections import deque` **first, every time.**

Trap 2 — not capturing the level size

```
while queue:
    level = []
    while queue:
        node = queue.popleft()
        level.append(node.val)
        queue.append(node.left)
    ...
    # WRONG: appends keep topping it up
```

The inner loop drains the queue *and everything added to it*, so the first “level” is the entire tree and the result is `[[1, 2, 3, 4, 5, 6]]`. One level, all the nodes. **Selvam’s April.**

Trap 3 — appending `None` children

```
queue.append(node.left)           # might be None
queue.append(node.right)
```

```
AttributeError: 'NoneType' object has no attribute 'val'
```

on the next `popleft`. Check before appending. And note that a `None` in the queue also corrupts `len(queue)`, so the level sizes are wrong even if you filter later.

Trap 4 — forgetting the empty-tree guard

```
queue = deque([root])             # root is None
```

Same `AttributeError`, on the very first iteration. `if root is None: return []` before the loop.

Trap 5 — `min` for minimum depth

```
def min_depth(node):
    if node is None:
        return 0
    return 1 + min(min_depth(node.left), min_depth(node.right))
```

```
min_depth(TreeNode(1, None, TreeNode(2))) → 1      correct answer: 2
```

A node with one child is **not a leaf**, but `min` treats the missing child as a depth-0 branch and returns 1. This is the most-failed easy tree problem, and BFS avoids it entirely by checking “both children are `None`” explicitly.

Trap 6 — trying to zigzag by reversing the traversal

```
if not left_to_right:
    queue.appendleft(node.right)    # trying to alternate the queue itself
```

It can be made to work with two stacks, and doing it by manipulating one queue is a reliable way to lose twenty minutes. **Collect the level normally and reverse the list.** `level[::-1]` is `O(width)` and costs nothing that matters.

Trap 7 — assuming BFS gives you a parent’s answer from its children

```
# trying to compute height with BFS
```

You can count levels, and that gives the height — but anything that *combines* children’s results (sums, diameter, balance) has no natural place to do the combining in a level-order loop, because a parent is long gone by the time its children are processed. **Those are depth-first, postorder, problems.**

Trap 8 — quoting $O(n)$ space for BFS without saying why

$O(n)$ is technically true — the queue never exceeds n — but it hides the real answer. **The space is $O(\text{width})$** , which is $n/2$ for a perfect tree and 1 for a chain. Saying $O(\text{width})$ and then giving both extremes is the answer that sounds like you have thought about it.

8 In the interview

How it gets asked

- The pair: “Print the tree level by level. Now print each level as its own list.” LeetCode 102.
- The variants, which are the same loop: right side view (199), level averages (637), zigzag (103), connect next pointers (116/117).
- The one where it matters: “Find the minimum depth.” LeetCode 111.
- The comparison: “When would you use BFS instead of DFS?”
- The space probe: “What is the space complexity, and how does it compare to DFS?”

What to say out loud, in the first ninety seconds

1. **Name the one structural difference.** “Breadth-first is the same walk as iterative preorder with a queue instead of a stack. That single swap is what makes it go wide instead of deep.”
2. **State the invariant before writing the loop.** “At the top of each outer iteration, the queue holds exactly one complete level. So I capture its length first — that number is the level size — and everything I append while draining belongs to the next level.”
3. **Use a deque, and say why.** “A deque, not a list — `list.pop(0)` shifts every element and turns this into $O(n^2)$ with no error to tell you.”

4. **Give the space as $O(\text{width})$, with both extremes.** “ $O(n)$ time, and $O(\text{width})$ space — about half the nodes for a perfect tree, and one node for a chain. Depth-first is the opposite: $O(\text{height})$, so twenty frames on a balanced million-node tree and a million on a chain.”
5. **Say when BFS is the right tool.** “Levels, the shallowest anything, and shortest paths. Minimum depth is the case where it is genuinely faster, not just tidier.”
6. **Say when it is not.** “Anything computed from the children — height, sums, diameter, balance — is depth-first postorder, because BFS has nowhere natural to combine a parent’s answer from its children’s.”

The follow-ups

“**Now group them by level.**” “One line: capture `len(queue)` into a variable at the top of the outer loop, before draining anything. The reason it works is the invariant — at that moment the queue contains exactly one complete level, so its length is the level size, and every child appended during the inner loop belongs to the next level. Without the capture, the inner loop keeps draining a queue that is being topped up, so the first ‘level’ is the whole tree. There are two other ways: a `None` sentinel pushed as a level marker, which works and needs care to avoid an infinite loop at the end; and a depth-first walk carrying the depth down and appending into `levels[depth]`, which is genuinely neat and uses $O(\text{height})$ stack instead of $O(\text{width})$ queue. I would write the `level_size` version because it is the shortest and the invariant is easy to state.”

“**What is the space complexity, and how does it compare to DFS?**” “BFS is $O(\text{width})$ — the widest level, held in the queue at once. DFS is $O(\text{height})$. Those are opposite and neither wins in general. On a **perfect** million-node tree, the bottom level holds about five hundred thousand nodes, so BFS holds half a million references while DFS holds about twenty frames. On a **chain** of ten thousand nodes it reverses completely: BFS holds one node and DFS overflows the stack — Python’s default limit is a thousand. So the practical rule is that BFS is usually the heavier of the two on memory and the safer of the two against stack overflow. I would say $O(\text{width})$ rather than $O(n)$, because $O(n)$ is true and hides all of that.”

“**When would you use BFS instead of DFS?**” “Three families. Anything where **the levels are the answer** — level order, right side view, level averages, zigzag, connecting each node to its next right neighbour. Anything asking for **the shallowest** something, where BFS stops the moment it finds it. And **shortest paths in an unweighted graph**, where the first time you reach a node is guaranteed to be by the shortest route — that is the same algorithm on a different structure. Minimum depth is the clearest example of the second family: if the left branch is ten thousand deep and the right child is a leaf, DFS walks the entire left branch before discovering the answer is 2, and

BFS finds it in three nodes. Conversely, I would use DFS for anything computed from the children — height, subtree sums, diameter, balance — because those are postorder and BFS has no natural place to do the combining.”

“Find the minimum depth.” “BFS, and I would say why rather than just doing it. Walk level by level carrying the depth, and return as soon as I meet a node with **both** children `None`. That is the first leaf in level order, which is the shallowest leaf by definition, and I stop there rather than exploring the rest. The recursive version has a trap that catches almost everybody: `1 + min(left, right)` is wrong, because a node with exactly one child is **not a leaf**, and `min` treats the missing side as a zero-depth branch. On a root with only a right child it returns 1 instead of 2. You have to special-case the one-child node, and BFS avoids the whole thing because ‘both children are None’ is an explicit test.”

“Do it with `O(1)` extra space.” “Not possible with a queue — the queue is the algorithm. There is one special case: if the nodes have a `next` pointer, as in the ‘connect each node to its next right node’ problem, then once one level is linked you can walk it as a linked list and build the next level’s links, which gives level-by-level processing in `O(1)` extra space. That is a genuinely clever trick and it only works because the tree has been augmented. For a plain binary tree, level order needs the queue.”

“How would you do zigzag order?” “Same skeleton, and I would **reverse the list, not the traversal**. Collect each level left to right as usual, and append it reversed on alternate levels. Trying to alternate the queue’s direction is possible with two stacks and is a reliable way to spend twenty minutes on something that is one `[::-1]`. The reversal is `O(width)` per level, so the total is still `O(n)`.”

A model answer

Asked: *print the tree level by level — and then print each level as its own list.*

“Level order is breadth-first, and the useful way to see it is that it is **the iterative preorder from yesterday with a queue instead of a stack**. A stack is last-in-first-out so you go deeper; a queue is first-in-first-out so you go wider. That one swap is the entire difference between the two families.

The flat version is six lines: put the root in a queue, and while the queue is not empty, take from the front, record it, and push its children on the back.

The grouped version is what the question is really about, and it comes down to one line. **At the top of each outer iteration, the queue holds exactly one complete level.** So I capture its length into a variable *before* draining anything — that number is the size of this level — and then I take exactly that many nodes. Everything I append while doing so is a child, so it belongs to the *next* level, and it is safely sitting behind the ones I am still to process.

That capture is not an optimisation; it is the correctness. Without it, the inner loop drains a queue that is being topped up as it goes, and the first ‘level’ turns out to be the entire tree.

Two implementation details I would say aloud. I use a `deque`, because `list.pop(0)` shifts every remaining element and quietly turns this into $O(n^2)$ — on a hundred thousand nodes that is billions of element moves and no error to tell you. And I check each child for `None` **before** appending, rather than filtering after popping, because a `None` in the queue also corrupts the level sizes.

Complexity: $O(n)$ time, since every node is enqueued and dequeued exactly once — the same as depth-first. Space is $O(\text{width})$, which I would give rather than $O(n)$ because the contrast is the interesting part. On a perfect million-node tree, the bottom level holds about five hundred thousand nodes, so BFS holds half a million while a depth-first walk holds about twenty stack frames. On a chain it reverses entirely: BFS holds one node and DFS overflows the stack. Neither is universally cheaper.

Once this skeleton is right, a family of interview questions is one differing line each: append the last element of each level for the right side view, the average for level averages, the level reversed on alternate rows for zigzag. And the one where the choice genuinely matters is minimum depth — BFS returns at the first leaf it meets, where a depth-first walk might explore a ten-thousand-node branch before discovering the answer was 2.”

9 Recall card

- **Breadth-first is iterative preorder with a QUEUE instead of a STACK.** That one swap is the whole difference: last-in-first-out goes deep, first-in-first-out goes wide. Use `collections.deque` — `list.pop(0)` is $O(n)$ and silently makes the traversal $O(n^2)$ (~5 billion moves at $n = 100,000$).
- **The one line that matters:** `level_size = len(queue)`, captured BEFORE draining. The invariant: *at the top of the outer loop the queue holds exactly one complete level*, so its length is that level's size and everything appended inside belongs to the next. Without it, the first "level" is the entire tree.
- $O(n)$ time — same as DFS. Space is $O(\text{width})$, not $O(n)$ — ~500,000 nodes for a perfect million-node tree against ~20 DFS frames, and 1 node for a chain where DFS raises `RecursionError`. Opposite directions; neither wins in general.
- **Use BFS for: levels are the answer** (level order, right side view, averages, zigzag, next-right pointers), **the shallowest anything**, and **shortest paths in an unweighted graph**. Minimum depth is where it is genuinely faster — 3 nodes against 10,002. **Use DFS for anything computed from the children** (height, sums, diameter, balance): BFS has nowhere to combine them.
- **Check children for None before appending** (a `None` in the queue corrupts the level sizes too), guard the empty tree before the loop, and **zigzag by reversing the LIST, not the traversal**. And the recursive `min_depth` trap: `1 + min(left, right)` is wrong — a node with one child is not a leaf, and it returns 1 instead of 2.

1 What this is, and why they ask it

A **cache** is a small, fast copy of data you have already fetched or computed, kept somewhere closer so that the next request for it does not have to do the work again.

Three sentences. The reason it works is not cleverness — it is the **latency table**: memory is about a thousand times faster than a disk and about a million times faster than a request across the world, so keeping a copy one layer up is an enormous win for almost no effort. The reason it works *in practice* is that real access patterns are extremely uneven: a few percent of your data serves most of your requests, which is why a small cache achieves a 90 percent hit rate. And the reason it is dangerous is that a cache is a **second copy of the truth**, which can disagree with the first.

They ask it because it is the highest-leverage change in most systems, and because “add a cache” is the answer candidates reach for without being able to say **where**, **what the hit rate would be**, or **what happens when it is wrong**. The good answer names a layer, gives a hit rate, does the arithmetic for what reaches the database afterwards, and states the staleness the design is accepting.

2 The story

The land records office had one clerk at the counter and one storeroom at the back, and the storeroom was up a half-flight of stairs and along a corridor.

Bhaskar had been at the counter for nine years. A person came, gave a survey number, and he went and got the file. Forty seconds there, some time looking, forty seconds back. Call it two minutes if the file was where it should be, and considerably longer if it was not.

Somewhere around his third year he noticed something that changed how he worked.

It was the same files. Not roughly the same — the same. There were about eleven thousand files in that room and on any given day he fetched maybe sixty, and forty-five of those sixty were from a set of about twenty files that came up again and again. Two large layouts near the highway that were being subdivided and sold in pieces, a temple trust property with a dispute running, and a handful of others.

So he stopped putting those back.

He kept them in the drawer of his own desk. Twenty files, and when somebody asked for one of them he had it open before they had finished saying the number.

The effect was much larger than he expected. He had thought he would save some walking. What actually happened was that the queue stopped forming. Three out of four people were now dealt with in fifteen seconds instead of two minutes, so the one person who did need a walk to the storeroom did not have six people building up behind him while Bhaskar was gone.

There were two problems and he had to solve both.

The drawer only held about twenty-five files, so when a new one became busy he had to take something out. He used the obvious rule: whatever he had not been asked for in the longest time went back.

And the other problem was the serious one. In his fourth year, a correction was made to one of the highway files — a subdivision registered, the entry updated in the storeroom copy. His drawer copy said the old thing. He gave out the old thing for eleven days, to four different people, and one of those people acted on it.

After that he had a rule. Any file that got touched, he took out of the drawer that same day, whether or not the drawer was full. He would rather walk to the storeroom than be confidently wrong.

3 The idea in plain English

Bhaskar has built a cache, and he met both of its problems in the right order: eviction, and then the one that actually hurts.

- The storeroom is the **origin** — the source of truth. Slow, complete, correct.
- The desk drawer is the **cache**. Small, fast, and a *copy*.
- Finding the file in the drawer is a **cache hit**. Having to walk is a **cache miss**.
- Three out of four is a **hit rate** of 75 percent.
- “Whatever I have not been asked for in the longest time goes back” is **LRU eviction** — least recently used, from [day 076](#).
- Removing a file when it changes is **invalidation**, and the eleven days is **staleness**.

Why it works: the two facts underneath

Fact one — the layers are orders of magnitude apart.

in-process memory	~100 nanoseconds	
a cache on another machine (Redis)	~0.5 milliseconds	5,000× slower
a database query hitting disk	~10 milliseconds	20× slower again
a request across the world	~150 milliseconds	15× slower again

Fact two — access is extremely uneven. Real traffic follows something close to a power law: a small fraction of the items get most of the requests. That is why a cache holding one percent of the data can serve ninety percent of the requests, and it is the fact that makes the whole technique viable.

typical consumer product		
top 1% of items	→	~50% of requests
top 10% of items	→	~90% of requests

If access were uniform, caching would barely work. Say that when you justify a hit rate — it shows you know the assumption you are relying on.

The hit-rate arithmetic, which is the number to produce

$$\text{effective latency} = \text{hit_rate} \times \text{cache_time} + (1 - \text{hit_rate}) \times \text{origin_time}$$

With a 0.5 ms cache and a 20 ms database:

hit rate 0%	$0.0 \times 0.5 + 1.00 \times 20$	= 20.0 ms
hit rate 50%	$0.5 \times 0.5 + 0.50 \times 20$	= 10.3 ms
hit rate 90%	$0.9 \times 0.5 + 0.10 \times 20$	= 2.5 ms
hit rate 95%	$0.95 \times 0.5 + 0.05 \times 20$	= 1.5 ms
hit rate 99%	$0.99 \times 0.5 + 0.01 \times 20$	= 0.7 ms

Going from 90 to 95 percent halves the latency, because what remains is almost entirely the misses. That is the non-obvious shape of the curve and it is worth saying: **at high hit rates, the misses are the whole cost**, so a small improvement in hit rate is a large improvement in latency.

And the load side, which matters more:

6,000 reads/second, hit rate 90%	
→ 600/second reach the database	a 10× reduction
hit rate 95%	
→ 300/second	a 20× reduction

One number, one order of magnitude off the database. That is why “add a cache” is the first answer to almost every read-heavy scaling problem.

The four layers, and what each one saves

This is the answer to “where would you put a cache”, and there are four right answers, not one.

LAYER	WHERE IT LIVES	WHAT IT SAVES	TYPICAL TTL
Browser / client	the user’s device	the entire network round trip	minutes to a year, for static assets
CDN / edge	~100 locations worldwide	the distance to your servers	minutes to days
Application	in the server process, or Redis	the database query and the computation	seconds to minutes
Database	the buffer pool	the disk read	managed automatically

Name all four, then say which one this system needs. Candidates who say “I’d put Redis in front of the database” have given one of four answers and usually not the biggest one — for a media-heavy product the CDN saves far more.

- Within the application layer there are two sub-choices, and the difference is real:
 - In-process cache** — a dictionary in the server’s own memory. Roughly 100 nano-seconds, no network at all. But every server has its own, so they can disagree, and each one warms up separately after a deploy. **Good for small, rarely-changing data:** feature flags, configuration, a currency table.
 - Shared cache (Redis, Memcached)** — one logical cache all servers share. About half a millisecond, one copy of the truth, survives a deploy. **The default for anything user-specific or frequently written.**

What to cache, and what not to

Cache it if it is read far more than written, expensive to produce, and tolerable slightly stale.

Do not cache anything where a stale answer is a wrong answer with consequences: a bank balance at the moment of a transfer, an inventory count at the moment of purchase, a permission check. [Yesterday’s](#) rule generalises: **a cache miss must make the answer slow, never wrong.**

The read-to-write ratio is the number that decides it:

100:1 or higher	cache aggressively
10:1	cache with a short TTL
1:1	a cache mostly adds work – every write invalidates
write-heavy	do not cache the data; cache derived views if anything

The key is a design decision

user:12345:profile	one user's profile
feed:12345:page:1	one page of one user's feed
product:987:price:INR	price, per currency

Three rules that come up:

- **Include everything the value depends on.** A price cached without the currency, or a page cached without the page number, serves the wrong thing to somebody.
- **Do not include anything it does not depend on.** A key with a timestamp in it never hits.
- **Namespace it,** so you can find and clear a family of keys, and so two features cannot collide.

The three problems every cache has

Staleness. The copy disagrees with the truth. You control it with a TTL and with invalidation, and [tomorrow](#) is entirely about that.

Eviction. The cache is smaller than the data, so something must go. LRU by default — [day 076](#).

The stampede. A popular key expires and a thousand requests all miss at the same instant and all hit the database at once. This is the failure that turns a cache into an outage, and it has a name and a fix:

```
popular key expires at t=0
→ 1,000 in-flight requests all miss
→ 1,000 identical database queries
→ the database, sized for 600/s, gets 1,600 in one second
```

The fix is a lock per key: the first request to miss takes a short lock and does the work; the rest either wait for it or briefly serve the stale value. That, plus **jittered TTLs** so a thousand keys written at the same moment do not all expire at the same moment. You met this as *cache stampede* on [day 090](#).

4 The picture

The four layers, and what each one removes from the path.

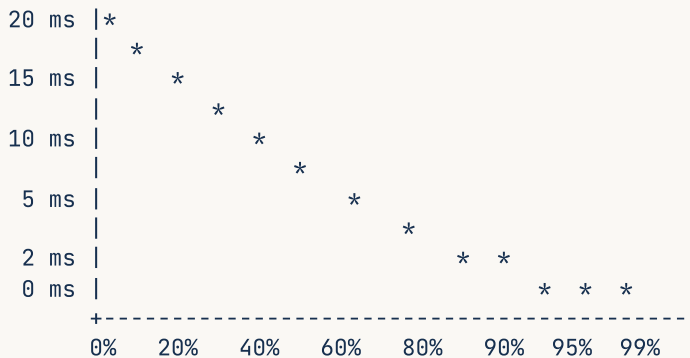


FIGURE 101.1

What to notice: **each layer removes everything to its right**. A browser hit costs nothing at all; a CDN hit costs 20 ms instead of 150; a Redis hit costs 0.5 ms instead of 20. The further left the hit, the more of the path disappears.

The hit-rate curve, drawn, because its shape is the surprising part:

effective latency, 0.5 ms cache, 20 ms origin



the curve is nearly flat until ~70%, then falls off a cliff.

0 → 50% saves 9.7 ms

90 → 95% saves 1.0 ms (but HALVES the remaining latency)

90 → 95% also HALVES the database load: 600/s → 300/s

^^^ usually the point

Bhaskar's drawer, as the same arithmetic:

storeroom trip ~120 seconds
 drawer ~15 seconds
 hit rate 45 of 60 = 75%

before: 60 × 120 s

= 7,200 s of walking per day

after: 45 × 15 s + 15 × 120 s

= 675 + 1,800 = 2,475 s

^ a 66% reduction

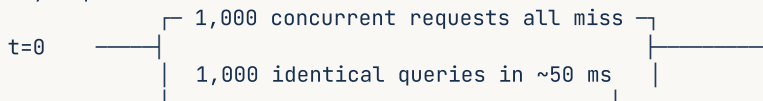
and the effect he did NOT predict: the queue.

with 15 fetches a day instead of 60, the one person who needs a walk no longer has six people waiting behind him.

The stampede, drawn, because it is the failure mode:

normal: _____ 600 req/s to the DB (10% of 6,000)

key expires at t=0:



^ the database sized for 600/s
receives 20,000/s for a moment

fix 1: per-key lock — the first miss fetches, the rest wait (or serve stale)
fix 2: jittered TTL — expiry $\pm 10\%$, so a thousand keys do not expire together
fix 3: refresh before expiry — at 90% of the TTL, refresh in the background

5 How it actually works

The read path, in two patterns

Cache-aside (lazy loading) — the application does the work. This is the default and what to describe unless asked otherwise.

```
value = cache.get(key)
if value is None:                # miss
    value = database.query(...)
    cache.set(key, value, ttl=300)
return value
```

Three properties worth stating: **only requested data is ever cached**, so the cache stays relevant; the application controls the key and the TTL; and **a cache failure degrades to slow rather than broken**, because the code path still works with the cache returning nothing.

Read-through — the cache itself fetches on a miss. Cleaner application code, and it makes the cache a mandatory component: if it is down, nothing works.

The distinction to say out loud: with cache-aside, the cache being down means the system is slow. With read-through, it means the system is down.

The write path, which is where designs differ

Three options, and interviewers ask you to choose.

Write-through — write to the cache and the database together, synchronously.

- + the cache is never stale
- every write pays both costs
- caches data that may never be read

Write-around — write to the database only, and invalidate the cache entry.

- + no wasted cache space for write-once data
- the next read is a miss
- the usual default for most systems

Write-back (write-behind) — write to the cache and flush to the database later.

- + very fast writes, and repeated writes to one key collapse into one
- DATA LOSS if the cache dies before the flush
- only for data you can afford to lose: counters, view counts, last-seen timestamps

Tomorrow is this in full. For today: **write-around by default, write-through when reads follow writes immediately, write-back only for data whose loss is acceptable.**

Sizing a cache

The question “how big should it be?” has a real answer:

1. what is the working set? — the data actually requested in a window
2. what hit rate do you want? — 90% typically needs the top ~10% of items
3. size = (items × bytes per item) × the fraction you need

Worked:

10,000,000 user profiles × 2 KB	= 20 GB total
90% of reads hit the top 10%	= 1,000,000 profiles
cache size for a 90% hit rate	= 1,000,000 × 2 KB = 2 GB

Two gigabytes for a 90 percent hit rate on twenty gigabytes of data. That number — cache a tenth of the data, serve nine-tenths of the traffic — is the shape to carry, and it comes straight from the unevenness of access.

What real systems use

- **Redis** and **Memcached** for the shared application cache. Redis has data structures, persistence, and replication; Memcached is simpler and purely a cache. **Redis is the default answer** and it is fair to say so.

- **CDNs** — Cloudflare, Akamai, CloudFront, Fastly — for anything static and anything media. **This is usually the biggest single win in a consumer product** and it is the layer candidates most often skip.
- **Database buffer pools** — PostgreSQL’s shared buffers, MySQL’s InnoDB buffer pool — cache disk pages in memory automatically. Sizing this correctly is often worth more than adding Redis, and it is free.
- **HTTP caching headers** — `Cache-Control`, `ETag`, `Last-Modified` — are how you use the browser and CDN layers at all. Getting these right is the cheapest performance work available.
- **Facebook’s memcached deployment** is the canonical published example of application caching at scale, and the paper is where much of the standard vocabulary — leases for stampede protection, cold cluster warm-up — comes from.

6 The numbers

The layers, in time

in-process dict lookup	~100 ns	
Redis, same data centre	~0.5 ms	5,000× the dict
database, index hit in memory	~1 ms	
database, hitting disk	~10 ms	
CDN edge hit	~20 ms	
origin across a continent	~150 ms	

Load reduction, which is the real argument

6,000 read QPS at the application		
hit rate	reaches the database	reduction
-----	-----	-----
0%	6,000/s	1×
50%	3,000/s	2×
80%	1,200/s	5×
90%	600/s	10×
95%	300/s	20×
99%	60/s	100×

At 90 percent you have turned “we need to shard the database” into “one database is fine”. That is the sentence that connects caching to [day 098](#).

Cost

a managed Redis instance, ~13 GB	~₹20,000 - 30,000 / month
a database instance able to serve 6,000 QPS instead of 600	several times that, plus replicas
CDN bandwidth	~₹1 - 7 per GB, and it REPLACES your own egress at a similar or higher rate

The cache is almost always cheaper than the capacity it removes. Worth saying, because it makes the decision a business argument rather than a preference.

Memory sizing, worked twice

SESSIONS	
1,000,000 active × 1 KB	= 1 GB
USER PROFILES	
10,000,000 × 2 KB total	= 20 GB
cache the hot 10%	= 2 GB for a ~90% hit rate
RENDERED FEED PAGES	
1,000,000 active users × 20 KB	= 20 GB
→ too big; cache the ID LIST instead:	
1,000,000 × 20 ids × 8 B	= 160 MB
→ then hydrate the items from a second, much smaller cache	

That third one is a real design move: cache identifiers rather than rendered content, because the ids are two orders of magnitude smaller and the items are shared between many users’ feeds.

TTL, and what it is really choosing

TTL	max staleness	refreshes/day for one key	DB load per key
-----	-----	-----	-----
10 s	10 s	8,640	high
60 s	1 min	1,440	
5 min	5 min	288	
1 hour	1 hour	24	
1 day	1 day	1	negligible

The TTL is a dial between staleness and database load, and nothing else. Choosing it is choosing how wrong you are willing to be, in seconds — and saying it that way is much better than “I’d use five minutes”.

The stampede, in numbers

```
one popular key, 1,000 requests in flight when it expires
  without protection: 1,000 identical queries in ~50 ms = 20,000 QPS burst
  with a per-key lock: 1 query; 999 wait ~20 ms or serve stale

1,000 keys all written at the same moment with a 300 s TTL
  without jitter: all 1,000 expire in the same second
  with ±10% jitter: spread over 60 seconds
```

7 The trade-offs

Every cache is a second copy of the truth

That is the whole trade, and everything else follows from it. You are buying latency and load reduction with **correctness risk**, and the currency is seconds of staleness.

I would not cache a value where being briefly wrong causes an irreversible action: a balance being debited, the last unit of stock being sold, a permission being checked. For those, the cache can hold the *display*, never the *decision* — the same distinction as [day 095's](#) auction price.

In-process or shared?

In-process is a thousand times faster and needs no network. It costs you consistency between servers — ten servers can hold ten different values for up to the TTL — and a cold start after every deploy. **Use it for small, slow-changing, non-user-specific data:** feature flags, configuration, a country list.

Shared (Redis) is one copy, survives deploys, and can be invalidated centrally. It costs half a millisecond and it is a dependency on the request path. **Use it for everything user-specific.**

Many systems run both — a small in-process cache in front of Redis — which multiplies the layers and also multiplies the invalidation problem.

A high hit rate is not automatically good

A 99 percent hit rate on data that changes every second means you are serving stale answers 99 percent of the time. **Hit rate measures how often you avoided work, not how often you were right.** Report it alongside the staleness the TTL permits.

Caching makes failures stranger

- **A cache that is down** turns into a load spike on the origin at exactly the moment you least want one. That is why the fallback matters: cache-aside degrades to slow, read-through degrades to broken.
- **A cache that is up but empty** — after a restart or a deploy — is the same spike. Large deployments warm caches before taking traffic for exactly this reason.
- **A poisoned cache** serves a wrong value fast, to everybody, until the TTL expires. The debugging experience is memorable: the database is right, the code is right, and the answer is wrong.

Where this design breaks

- **Write-heavy data.** If every write invalidates, the cache spends its time being cleared, and you have added latency for nothing.
 - **Personalised content.** A cache key per user means the hit rate collapses, because each user has their own entries and each is read rarely. The fix is to cache the shared parts and assemble per user.
 - **Unbounded key space.** Caching search results by query string means most keys are never read twice — a hit rate close to zero and a cache full of rubbish. Cache the popular queries only, or nothing.
 - **The stampede** is the failure that converts a cache from a benefit into an outage, and it needs an explicit answer.
-

8 In the interview

How it gets asked

- The open one: *“Where would you put a cache in this system?”*
- The number one: *“What hit rate would you expect, and what does the database see afterwards?”*
- The correctness one: *“What happens when the underlying data changes?”* — which is tomorrow.
- The failure one: *“Your cache goes down. What happens?”*
- The nasty one: *“A very popular item’s cache entry expires. What happens?”*

What to say out loud, in the first ninety seconds

1. **Name all four layers, then choose.** “There are four places: the browser, the CDN, the application — in-process or Redis — and the database’s own buffer pool. For a media-heavy product the CDN is the biggest win; for this one I think the application layer is.”
2. **Justify it with the read-to-write ratio.** “This is read-heavy, roughly fifty reads per write, so a cache is the first thing I would add.”
3. **Give a hit rate with a reason.** “I would expect around ninety percent, because access is very uneven — the top ten percent of items typically serve ninety percent of requests. If access were uniform, caching would barely help, and I would want to check that assumption.”
4. **Do the load arithmetic.** “At six thousand reads a second and a ninety percent hit rate, six hundred reach the database — a tenfold reduction, which is the difference between needing to shard and not.”
5. **Say the TTL as a staleness decision.** “A sixty-second TTL means I am accepting up to a minute of staleness for this data. The TTL is a dial between how wrong I am willing to be and how much load the database takes.”
6. **Pre-empt the stampede.** “One thing I would build in: a per-key lock on a miss, and jittered TTLs. Otherwise a popular key expiring sends every in-flight request to the database at the same instant.”

The follow-ups

“**What hit rate would you expect, and why?**” “Around ninety percent for something like user profiles or product pages, and the reason is the shape of the access pattern rather than the size of the cache. Real traffic is very uneven — the top one percent of items typically serves about half the requests and the top ten percent serves about ninety. So caching a tenth of the data gets me nine-tenths of the traffic: twenty gigabytes of profiles, two gigabytes of cache, ninety percent hits. The assumption I am relying on is that unevenness, and I would say so — if access were uniform, a cache holding ten percent of the data would give a ten percent hit rate and would not be worth having. The load effect is the part I care about: at six thousand reads a second, ninety percent means six hundred reach the database, and going to ninety-five percent halves that again. At high hit rates the misses are essentially the entire cost, which is why the last few percent are worth chasing.”

“**Your cache goes down. What happens?**” “That depends on which pattern I used, and it is worth being explicit. With **cache-aside** — the application checks the cache, and on a miss queries the database and populates it — a cache failure looks exactly

like a hundred percent miss rate. The system still works and it is slow. With **read-through**, where the cache fetches on my behalf, a cache failure means the system is down. So I would use cache-aside. But 'still works' has a catch: the database was sized for six hundred queries a second and is suddenly getting six thousand, so it may not survive. The mitigations are a circuit breaker in front of the database, request coalescing so identical concurrent queries become one, and — for the important paths — a small in-process cache as a second line, which cannot be taken out by the same failure. The same spike happens after a cache restart with an empty cache, which is why large systems warm caches before taking traffic.”

“**A very popular item’s cache entry expires. What happens?**” “That is the **cache stampede**, and it is the failure that turns a cache into an outage. At the instant it expires, every in-flight request for that key misses, and they all issue the same database query. A thousand concurrent requests becomes a thousand identical queries in about fifty milliseconds — a twenty-thousand-per-second burst against a database sized for six hundred. Three fixes and I would use the first two by default. **A per-key lock**: the first request to miss takes a short lock and does the fetch, and the others either wait for it or serve the slightly stale value. **Jittered TTLs**: add ten percent randomness, so a thousand keys written in the same second do not all expire in the same second. And optionally **refresh ahead**: at ninety percent of the TTL, refresh in the background so the entry never actually expires under load.”

“**Where exactly would you put it — you said four layers.**” “I would work from the outside in, because the outer layers save more. **Browser and CDN** first, for anything static or media — images, scripts, video. That removes the request entirely rather than making it faster, and for a media-heavy product it is by far the biggest win and the one people skip. **Application cache** next, for query results and computed values; Redis if the data is user-specific or changes, in-process if it is small and slow-changing like feature flags. **Database buffer pool** last, and it is free — sizing it so the working set fits in memory is often worth more than adding Redis and costs nothing but a configuration change. The mistake I want to avoid is saying ‘put Redis in front of the database’ as though that is the only option, when the CDN might remove ten times as much traffic.”

“**What would you not cache?**” “Anything where a stale answer is a wrong answer with consequences. A bank balance at the moment a transfer is authorised. Stock level at the moment of purchase. A permission check. The rule I use is that **a cache miss must make the answer slow, never wrong** — and the corollary is that a cache can hold the *display* value while the *decision* reads the truth. Showing ‘in stock’ from a cache is fine; deciding whether to accept the order is not. Beyond correctness, I would

not cache write-heavy data, because every write invalidates and the cache spends its life being cleared; and I would not cache things with an unbounded key space, like search queries, because most keys are read once and the hit rate is close to zero.”

“How would you choose the TTL?” “By deciding how stale I am willing to be, and then reading the load off that — the TTL is a dial between those two and nothing else. A product’s price might tolerate five minutes; a user’s own profile after they edit it tolerates zero, so that one I would invalidate on write rather than time out. A ten-second TTL on one key means 8,640 refreshes a day; an hour means twenty-four. I would also say what I would measure: hit rate and staleness together, because a ninety-nine percent hit rate on data that changes every second means I am confidently wrong almost all the time. Hit rate measures avoided work, not correctness.”

A model answer

Asked: *where would you put a cache in this system?*

“There are four places, and I want to name all of them before choosing, because they save very different amounts.

The browser, for static assets — that removes the request entirely rather than making it faster. **A CDN**, which puts a copy at about a hundred locations world-wide; for anything media-heavy this is usually the single biggest win, because it saves the distance, and no server optimisation can fix distance. **The application layer** — either in the server process or in a shared Redis — which saves the database query and any computation. And **the database’s own buffer pool**, which caches disk pages in memory and is free: sizing it so the working set fits is often worth more than adding Redis.

For this system, which is read-heavy at roughly fifty reads per write, I would put the main cache at the application layer, in Redis, using **cache-aside**: check the cache, and on a miss query the database and populate it. I choose cache-aside over read-through deliberately, because with cache-aside a cache failure makes the system **slow**, and with read-through it makes it **down**.

On the numbers. I would expect about a ninety percent hit rate, and the reason is the access pattern rather than the cache size: real traffic is very uneven, with the top ten percent of items typically serving ninety percent of requests. So twenty gigabytes of profiles needs about two gigabytes of cache for ninety percent hits. That assumption is worth stating — if access were uniform, this would not work at all.

The effect I care about is on load rather than latency. At six thousand reads a second, ninety percent means **six hundred reach the database** — a tenfold reduction, and that is the difference between needing to shard and one database being comfortable. Going to ninety-five percent halves it again, and that is the shape worth knowing: at high hit rates the misses are essentially the entire cost.

The TTL I would treat as a staleness decision rather than a performance one. Sixty seconds means I am accepting up to a minute of being wrong on this data. For anything a user changes about themselves I would invalidate on write instead, because ‘I saved it and it did not change’ is the worst possible version of staleness.

Two things I would build in from the start. **Per-key locking on a miss**, so that when a popular key expires, one request does the fetch and the others wait or serve stale — otherwise a thousand in-flight requests become a thousand identical queries in fifty milliseconds and take the database down. And **jittered TTLs**, so a batch of keys written in the same second does not expire in the same second.

And one thing I would not cache: anything where being briefly wrong causes an irreversible action. Stock levels can be cached for *display* and must be read from the truth for the *decision*.”

9 Recall card

- **A cache works because of two facts: the layers are orders of magnitude apart** (memory ~ 100 ns, Redis ~ 0.5 ms, DB ~ 20 ms, cross-world ~ 150 ms) **and access is extremely uneven** — the top 10% of items serve $\sim 90\%$ of requests. **If access were uniform, caching would barely work** — say that assumption aloud.
- **Four layers, not one: browser · CDN · application (in-process or Redis) · database buffer pool.** Work outside in; for media-heavy products the **CDN is the biggest win** and the one candidates skip. The buffer pool is **free**.
- **Do the load arithmetic, not just latency:** 6,000 reads/s at a 90% hit rate leaves 600/s on the database — $10\times$, the difference between sharding and not. 95% halves it again, because **at high hit rates the misses are the entire cost**. Sizing: cache $\sim 10\%$ of the data for $\sim 90\%$ hits (20 GB $\rightarrow$ 2 GB).
- **Use cache-aside:** on a miss, query and populate. Then **a cache failure makes the system slow, not broken** — read-through makes it broken. Write path: **write-around by default**, write-through when reads follow writes, **write-back only for data you can afford to lose**.
- **The TTL is a dial between staleness and load, and nothing else.** And the **stampede is the failure that turns a cache into an outage** — a popular key expires, 1,000 in-flight requests issue 1,000 identical queries in 50 ms. Fix with a **per-key lock** and **jittered TTLs**. Finally: **a cache miss must make the answer slow, never wrong** — cache the *display*, never the *decision*.

DSA topic: Breadth-first traversal: level order **System design topic:** Caching: the single biggest win

Code these, in this order

One rule for the whole set: **say the invariant out loud before writing the loop** — *at the top of the outer loop, the queue holds exactly one complete level*. Everything else in these four problems is one differing line.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Binary Tree Level Order Traversal	LeetCode 102 (Medium)	Capturing <code>len(queue)</code> before draining.
2	Binary Tree Right Side View	LeetCode 199 (Medium)	The same loop; the last node of each level.
3	Minimum Depth of Binary Tree	LeetCode 111 (Easy)	Where BFS is genuinely faster, and the <code>min</code> trap.
4	Binary Tree Zigzag Level Order Traversal	LeetCode 103 (Medium)	Reversing the list, not the traversal.

On problem 1, break the boundary on purpose

Replace the inner `for _ in range(level_size)` with `while queue`. Run it and record the output. Say in one sentence why the first level turned out to be the whole tree.

On problem 2, do it two ways

Once with BFS taking the last of each level, once with a DFS that visits right before left and records the first node seen at each depth. Say which one you would write and why the DFS version needs no queue.

On problem 3, write the wrong recursion first

Write `1 + min(left, right)`, run it on a root with only a right child, and record the answer. Then say what “leaf” actually means and fix it. Then write the BFS version and say what it costs on a tree whose left branch is 10,000 deep.

On problem 4, try the wrong approach for two minutes only

Attempt to alternate the queue's direction. Stop after two minutes, then do it with `level[::-1]`. The point of the exercise is to feel how much cheaper the right approach is.

The one-swap drill

1. Write iterative preorder from yesterday.
2. Change two things to make it level order, and name them.
3. Say why a stack goes deep and a queue goes wide.
4. Say what the output is missing without the level boundary.

The invariant drill

1. State the invariant in one sentence.
2. Write the outer and inner loops from memory.
3. Say what `level_size` is, and why it must be captured before any append.
4. Trace the queue at the top of every outer iteration for a six-node tree.

The three-ways drill

1. Write the `level_size` version.
2. Describe the sentinel version and name its edge case.
3. Write the DFS-with-depth version.
4. Say what each one costs in space, and when you would pick the third.

The variants drill

Write each as one differing line on the same skeleton:

1. Level order. 2. Right side view. 3. Level averages. 4. Level maximums. 5. Zigzag.
2. Maximum width.

Then say why memorising the skeleton beats memorising six problems.

The when-BFS drill

1. Name the three families of problems that want BFS.
2. Name the family that wants DFS instead, and say why BFS cannot do it.
3. Construct the tree that makes minimum depth much faster with BFS, and count nodes visited both ways.
4. Say what BFS on an unweighted graph guarantees.

The space drill

1. State the space of BFS and of DFS in terms of the tree's shape.
2. Compute both for a perfect million-node tree.
3. Compute both for a ten-thousand-node chain.
4. Say which one raises `RecursionError` and on which shape.
5. Say why `O(n)` is a worse answer than `O(width)`.

The deque drill

1. Say what `list.pop(0)` costs and why.
2. Compute the element moves for a level-order traversal of 100,000 nodes with a list.
3. Say what error you get when the list version is too slow.
4. Write the first line of every BFS you will ever write.

The break-it drill

Trigger each and record the exact output or error:

1. `while queue` as the inner loop.
 2. Appending `None` children.
 3. No empty-tree guard.
 4. A list with `pop(0)` on a large tree, timed.
 5. `1 + min(left, right)` on a one-sided tree.
 6. Trying to compute subtree sums with a level-order loop — say why you cannot.
-

The why-it-works drill

1. Name the two facts that make caching work.
2. Give the latency figures for the four layers.
3. State the access-pattern assumption and say what happens to caching without it.
4. Give the top-1% and top-10% request shares.

The arithmetic drill

Compute each, showing the working:

1. Effective latency at hit rates of 0, 50, 90, 95 and 99 percent, with a 0.5 ms cache and a 20 ms origin.
2. Database load at 6,000 read QPS for the same five hit rates.
3. Say what shape the latency curve has and why the last few percent matter.

4. Cache size for a 90 percent hit rate on 10M profiles of 2 KB each.
5. Refreshes per day per key for TTLs of 10 s, 60 s, 5 min and 1 day.

The four-layers drill

1. Name all four layers and what each one saves.
2. Say which is usually the biggest win for a media-heavy product.
3. Say which one is free and what “configuring it” means.
4. Give the two sub-choices inside the application layer and when to use each.

The patterns drill

1. Write the cache-aside read path in five lines.
2. Say what cache-aside gives you that read-through does not, in one sentence about failure.
3. Name the three write patterns and give a use case for each.
4. Say which write pattern can lose data, and what kind of data that is acceptable for.

The key-design drill

1. Give the three rules for cache keys.
2. Write keys for: a user’s profile, page 3 of a user’s feed, a product price in two currencies.
3. Give an example of a key that will never hit.
4. Say why namespacing matters.

The stampede drill

1. Describe the stampede in two sentences.
2. Compute the burst for 1,000 in-flight requests on one expiring key.
3. Name three fixes and say which two you would use by default.
4. Say what jitter is preventing, with the thousand-keys example.

The what-not-to-cache drill

1. State the rule about slow versus wrong.
2. Give three values you would never cache for a decision.
3. Say what “cache the display, not the decision” means with an example.
4. Name three data shapes where a cache adds work rather than removing it.

The failure drill

For each, say what happens and what you would add:

1. Redis goes down.
2. Redis restarts empty during peak traffic.
3. A wrong value is written into the cache.
4. A user edits their profile and immediately reloads the page.
5. Every request has a different cache key.
6. The hit rate is 99 percent on data that changes every second.

Two of the six are not availability problems. Name them and say what they are.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Print the tree level by level. Now print each level as its own list.* The one swap from a stack to a queue, the invariant stated before the code, `level_size` with the reason it is correctness rather than optimisation, `deque` with the `O(n2)` warning, and `O(width)` space with both extremes.
 2. *Where would you put a cache in this system?* All four layers named before choosing, the read:write ratio as the justification, a hit rate with the unevenness assumption stated, the load arithmetic, the TTL framed as a staleness decision, and the stampede pre-empted.
 3. *A very popular item's cache entry expires. What happens?* The stampede described with the burst arithmetic, per-key locking and jittered TTLs as the fixes, and refresh-ahead as the optional third.
-

Before you move on

- [] I can turn iterative preorder into level order by changing two things.
- [] I can state the queue invariant in one sentence.
- [] I can write the `level_size` loop from memory and say why the capture is correctness.
- [] I ran the version without the capture and can explain the output.
- [] I can write five variants as one differing line each.

- [] I can name the three families that want BFS and the one that wants DFS.
- [] I can construct the tree where BFS is genuinely faster for minimum depth.
- [] I know the `1 + min(left, right)` trap and what “leaf” really means.
- [] I can state BFS space as `O(width)` and give both extremes.
- [] I always write `from collections import deque` first.
- [] I can name the two facts that make caching work.
- [] I can name all four cache layers and what each saves.
- [] I can compute effective latency and database load for any hit rate.
- [] I can size a cache from the data size and a target hit rate.
- [] I can write cache-aside and say what it gives me on failure.
- [] I can name the three write patterns and which one loses data.
- [] I can describe the stampede with numbers and name two fixes.
- [] I can state the slow-versus-wrong rule and give something I would never cache.
- [] I answered all three questions above out loud.

Height, depth, and diameter

DATA STRUCTURES AND ALGORITHMS

Height, depth, and diameter

You can compute the diameter in one pass, using the height as a by-product.

SYSTEM DESIGN

Cache invalidation and eviction policies

You can pick write-through, write-back or write-around and defend it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Find the diameter of the binary tree in $O(n)$.
- How do you keep the cache consistent with the database?

1 What this is, and why they ask it

The **diameter** of a tree is the length of the longest path between any two nodes in it. The path does not have to pass through the root, and it does not have to end at a leaf on both sides — though in practice it always does.

Three sentences. The obvious solution computes, at every node, the height of its left subtree plus the height of its right subtree, and takes the largest — which is correct and is $O(n^2)$, because computing a height is itself a walk of a whole subtree. The fix is the technique this whole phase turns on: **a function that returns one thing to its parent while recording a different thing in an outer variable**. Here it returns the height and records the diameter. And once you have seen that trick, five more problems become one-pass instead of two-pass.

They ask it because the $O(n^2)$ version looks completely reasonable and passes small tests, so the question is really “*can you see that height is being recomputed?*” — and then “*can you restructure the recursion so that the thing you need is already being calculated?*” The follow-ups are the same trick applied elsewhere: balanced-tree checking, maximum path sum, and the largest subtree satisfying some property.

2 The story

The house had been added to four times since it was built, and by the time Zubeida inherited it, it sprawled.

There was the original bit at the front with three rooms. Somebody had built two rooms off the side in the sixties. Her father had added a kitchen and a store behind that, and then the long room at the back for her brother’s family, which had its own two small rooms off it.

She wanted an intercom put in, the kind with handsets in each room, and the man who came to do it stood in the front doorway and asked her one question that took the whole afternoon to answer: **what is the longest run of wire I will need, from any one room to any other?**

Her first thought was that it must be from the front room to the back room, because those were the two ends of the house. But it was not, and she worked out why while walking around.

The front rooms were all close together. The long run was actually from the small room off her brother's end, all the way through the middle, and out to the second room of the sixties extension — and neither of those was the front or the back of the house.

The man had a method that she liked, and it was the reason the afternoon ended at four rather than at midnight.

He did not measure between pairs of rooms. There were fourteen rooms, and measuring every pair would have been ninety-one measurements.

Instead he stood at each junction — each place where the house split into two or more directions — and asked only one thing: **going that way, how far is the furthest room? And going that way?** Then he added the two answers together, and that was the longest run passing through that junction.

He wrote each one down as he went, in his phone, and at the end he took the biggest of them.

And there was one economy in it that Zubeida did not spot until he explained it. When he stood at a junction and worked out “the furthest room in that direction”, he was really asking the same question he would ask standing at the next junction along, plus a few steps. So he never measured a corridor twice. He started at the far ends and worked back towards the front door, and each junction's answer was built out of the answers he already had.

He said the men who do it the other way — measuring from every room to every room — are still there at nine at night, and they get the same number.

3 The idea in plain English

The electrician has computed a diameter, and his economy — “each junction's answer is built out of the answers I already have” — is the whole technique.

- Each room and junction is a node.
- “How far is the furthest room going that way” is the **height** of that subtree.
- The longest run **through** a junction is `left height + right height`.
- The answer for the whole house is the largest of those, over every junction.

The definition, and its two conventions

Diameter = the number of edges on the longest path between any two nodes.



the longest path: $4 \rightarrow 2 \rightarrow 5$ (2 edges)
or $4 \rightarrow 2 \rightarrow 1 \rightarrow 3$ (3 edges) ← longer

diameter = 3 edges = 4 nodes

LeetCode 543 counts edges, so a single node has diameter 0 and the example above is 3. Some textbooks count nodes, giving 4. **Say which you are using**, exactly as with height on [day 098](#). The relationship is always `nodes = edges + 1`.

The path does not have to go through the root, and that is the point of the problem. A tree with a deep left subtree and nothing on the right has its diameter entirely inside the left subtree.

The key observation

For any node, the longest path **whose highest point is that node** is:

```
height(left subtree) + height(right subtree) + 2      (in edges, where an empty subtree is -1)
```

Every path has exactly one highest point — one node on it that is closer to the root than all the others. So if you compute that quantity at every node and take the maximum, you have considered every possible path exactly once.

That is the whole algorithm, and stating it in that form — *every path has a unique highest node, so I evaluate every node as the top of a path* — is what makes the solution sound derived rather than recalled.

Why the obvious version is $O(n^2)$

```
def diameter(node):
    if node is None:
        return 0
    through = height(node.left) + height(node.right) + 2
    return max(through, diameter(node.left), diameter(node.right))
```

Correct. And `height` walks an entire subtree, and it is called at every node, so the total work is the sum of all the subtree sizes:

```
balanced tree:    O(n log n)
degenerate tree:  O(n^2)
```

```
n = 10,000, a chain
O(n^2) → ~50,000,000 operations  - seconds
O(n)   → ~10,000                  - instant
```

Zubeida's electrician measuring every corridor once per junction instead of once, ever.

The trick: return one thing, record another

The insight is that `height` **already visits every node once** and already knows both children's heights at the moment it needs them. So the diameter can be computed *inside* the height function, and stored somewhere outside.

```
def diameter_of_binary_tree(root):
    best = 0                                # the OUTER variable

    def height(node):
        nonlocal best
        if node is None:
            return -1
        left = height(node.left)
        right = height(node.right)
        best = max(best, left + right + 2)    # RECORD the diameter here
        return 1 + max(left, right)         # RETURN the height

    height(root)
    return best
```

Two different quantities, and they must not be confused:

- **Returned to the parent:** the height. This is what the parent needs — “how far down does your side go”.
- **Recorded in `best`:** the diameter through this node. The parent has no use for this, because a path bending at a child cannot be extended upward.

That is the sentence to say out loud in the interview. *“The value I return is not the value I am looking for. I return the height because that is what my parent needs, and I record the diameter on the side, because a path that bends here cannot be part of a longer path through my parent.”*

Once you have that sentence, this whole family of problems is the same code with two lines changed.

The family

PROBLEM	RETURN TO THE PARENT	RECORD ON THE SIDE
Diameter (543)	height	<code>left + right + 2</code>
Balanced tree (110)	height, or <code>-2</code> meaning “already unbalanced”	— (short-circuit instead)
Maximum path sum (124)	best downward sum, floored at 0	<code>left + right + node.val</code>
Longest univalue path (687)	longest same-value arm	<code>left + right</code>
Largest BST subtree (333)	<code>(min, max, size, is_bst)</code>	<code>size</code> when <code>is_bst</code>

Five interview problems, one shape. Day 104 does maximum path sum properly; today is the shape.

Balanced, in one pass, as the second example

The naive check calls `height` at every node — the same $O(n^2)$. The one-pass version returns the height normally, and returns a **sentinel** when it has already found an imbalance:

```
if left == -2 or right == -2 or abs(left - right) > 1:
    return -2 # -2 means "unbalanced somewhere below"
return 1 + max(left, right)
```

The sentinel propagates all the way up without any further work being done, which is what makes it a single pass. `-2` is chosen because a real height is never less than `-1`.

An alternative that some prefer is returning a tuple `(height, is_balanced)`. It is clearer and it allocates a tuple per node. **Either is fine; say which you are doing and why.**

The relationship between height and diameter

Two facts worth having, because interviewers probe them:

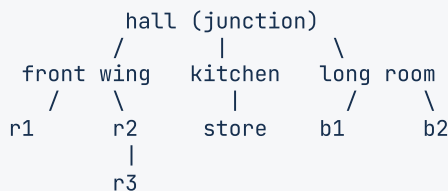
```
diameter ≥ height      always
diameter ≤ 2 × height  always
```

The first because the path from the root to the deepest leaf is itself a path. The second because any path is two downward paths glued at their highest node, and each is at most the height.

And the case people get wrong: **the diameter can be entirely inside one subtree and not touch the root at all.**

4 The picture

Zubeida's house, and why the answer is not front-to-back.



heights (edges to the deepest room below):

front wing: 2 (via $r2 \rightarrow r3$)

kitchen: 1

long room: 1

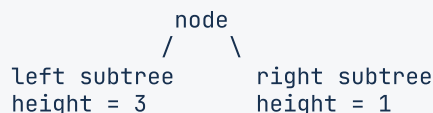
longest path THROUGH the hall = $2 + 1 + 2 = 5$ edges

$\quad \quad \quad \wedge \quad \wedge \quad \wedge$
 front wing's height + long room's height + the two edges

path: $r3 \rightarrow r2 \rightarrow \text{front wing} \rightarrow \text{hall} \rightarrow \text{long room} \rightarrow b1$

NOT front-to-back: $r1 \rightarrow \dots \rightarrow b1$ is only 4 edges.

The two quantities, at one node:



RETURNED to the parent: $1 + \max(3, 1) = 4$
 "my side goes 4 deep"

RECORDED in `best`: $3 + 1 + 2 = 6$
 "the longest path bending here is 6 edges"

the parent CANNOT use the 6. A path that bends here has already used both of my children, so it cannot also go up through me.

The $O(n^2)$ version, drawn as wasted work:

```

a chain of 5 nodes: 1 - 2 - 3 - 4 - 5

height() is called at every node by diameter():
  at node 1: walks 5 nodes
  at node 2: walks 4
  at node 3: walks 3
  at node 4: walks 2
  at node 5: walks 1
  ---
  15 node visits for 5 nodes

n = 10,000 → ~50,000,000 visits
one pass → 10,000

```

The one-pass version, traced:



call order (postorder – children first):

```

height(4): left=-1 right=-1 best = max(0, -1+-1+2)=0 returns 0
height(5): left=-1 right=-1 best = max(0, 0) = 0 returns 0
height(2): left=0 right=0 best = max(0, 0+0+2) = 2 returns 1
height(3): left=-1 right=-1 best = max(2, 0) = 2 returns 0
height(1): left=1 right=0 best = max(2, 1+0+2) = 3 returns 2

```

answer: 3 (the path 4 → 2 → 1 → 3)

note `best` only ever goes UP, and each node is visited exactly once.

5 The code, built step by step

Step 1 – say the key observation before writing anything

“Every path has exactly one highest point — one node on the path that is nearest the root. So if I evaluate every node as the top of a path, I have considered every path exactly once. At a node, the longest path bending there is the left subtree’s height plus the right subtree’s height plus two.”

That sentence is the solution. Everything after it is bookkeeping.

Step 2 — write the naive version and then criticise it

```
through = height(node.left) + height(node.right) + 2
return max(through, diameter(node.left), diameter(node.right))
```

“That is correct, and it is $O(n^2)$ on a skewed tree, because `height` walks a whole subtree and I am calling it at every node. On a ten-thousand-node chain that is fifty million operations instead of ten thousand.”

Writing the wrong version deliberately and naming its cost is a strong move, because it shows you found the improvement rather than remembered it.

Step 3 — notice what the height function already knows

“`height` visits every node once and, at the moment it computes a node’s height, it already has both children’s heights. So the diameter through that node is available for free, right there. I do not need a second traversal — I need a place to put the answer.”

Step 4 — the outer variable

```
best = 0
def height(node):
    nonlocal best
```

`nonlocal` is required to assign to a variable from the enclosing function. Without it, Python treats `best` as a new local and the outer one never changes — **a silent zero**, which is trap 2 below.

The alternatives are a one-element list (`best = [0]` , mutated as `best[0] = ...`) or a class attribute. `nonlocal` is cleanest in Python 3.

Step 5 — get the base case right for the convention

```
if node is None:
    return -1                                # EDGE convention
```

Then a leaf returns `1 + max(-1, -1) = 0` , and `best` at a leaf is `-1 + -1 + 2 = 0` . Both correct.

If you return `0` for `None` instead, the formula becomes `left + right` with no `+ 2` . Both work, and mixing them gives an answer that is off by two, which is exactly the kind of bug that survives small tests.

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

def diameter_naive(root: TreeNode | None) → int:
    """The version to write, criticise, and then replace.

    Correct, and  $O(n^2)$  on a skewed tree: height() walks an entire subtree
    and is called at every node. On a 10,000-node chain that is ~50 million
    operations instead of 10,000.
    """
    def height(node: TreeNode | None) → int:
        if node is None:
            return -1
        return 1 + max(height(node.left), height(node.right))

    def walk(node: TreeNode | None) → int:
        if node is None:
            return 0
        through = height(node.left) + height(node.right) + 2
        return max(through, walk(node.left), walk(node.right))

    return walk(root)

def diameter(root: TreeNode | None) → int:
    """LeetCode 543, in one pass.

    THE IDEA: every path has exactly one highest node, so evaluating every
    node as the top of a path considers every path exactly once.

    THE TRICK: the function RETURNS the height (what the parent needs) and
    RECORDS the diameter in an outer variable (what the parent cannot use,
    because a path bending here has already spent both of my children).

    Edge convention: empty tree is -1, a leaf has height 0, so the path
    through a node is left + right + 2.

    Time  $O(n)$  – one visit per node. Space  $O(\text{height})$  for the stack.
    """
    best = 0

    def height(node: TreeNode | None) → int:
        nonlocal best
        if node is None:
            return -1
        left = height(node.left)
        right = height(node.right)
        best = max(best, left + right + 2) # RECORD: the path bending here
        return 1 + max(left, right)      # RETURN: what my parent needs

    height(root)
    return best
```

```

def is_balanced_naive(root: TreeNode | None) → bool:
    """Same  $O(n^2)$  mistake, same fix available."""
    def height(node: TreeNode | None) → int:
        if node is None:
            return -1
        return 1 + max(height(node.left), height(node.right))

    def check(node: TreeNode | None) → bool:
        if node is None:
            return True
        if abs(height(node.left) - height(node.right)) > 1:
            return False
        return check(node.left) and check(node.right)

    return check(root)

def is_balanced(root: TreeNode | None) → bool:
    """LeetCode 110, in one pass, using a SENTINEL.

    -2 means "unbalanced somewhere below me". It propagates all the way up
    without any further work, which is what makes this a single pass.
    -2 is safe because a genuine height is never below -1.
    """
    UNBALANCED = -2

    def height(node: TreeNode | None) → int:
        if node is None:
            return -1
        left = height(node.left)
        if left == UNBALANCED:
            return UNBALANCED # short-circuit: stop working
        right = height(node.right)
        if right == UNBALANCED or abs(left - right) > 1:
            return UNBALANCED
        return 1 + max(left, right)

    return height(root) != UNBALANCED

def is_balanced_tuple(root: TreeNode | None) → bool:
    """The same, returning (height, is_balanced) instead of a sentinel.

    Clearer, and it allocates a tuple per node. Either is fine – say which
    you are doing and why.
    """
    def check(node: TreeNode | None) → tuple[int, bool]:
        if node is None:
            return -1, True
        lh, lb = check(node.left)
        if not lb:
            return 0, False
        rh, rb = check(node.right)
        if not rb or abs(lh - rh) > 1:
            return 0, False
        return 1 + max(lh, rh), True

    return check(root)[1]

def diameter_with_path(root: TreeNode | None) → tuple[int, list[int]]:
    """The follow-up: return the path itself, not just its length.

    Same shape, but now the recursion returns (height, the deepest downward

```

```

path from here) so the winning pair can be glued together at the node
where the best diameter was found.
"""
best = 0
best_path: list[int] = []

def walk(node: TreeNode | None) → tuple[int, list[int]]:
    nonlocal best, best_path
    if node is None:
        return -1, []
    lh, lpath = walk(node.left)
    rh, rpath = walk(node.right)
    if lh + rh + 2 > best:
        best = lh + rh + 2
        best_path = lpath[::-1] + [node.val] + rpath
    if lh ≥ rh:
        return lh + 1, lpath + [node.val]
    return rh + 1, rpath + [node.val]

walk(root)
return best, best_path

def longest_univalue_path(root: TreeNode | None) → int:
    """LeetCode 687. The same shape with one condition added: an arm only
    counts if the child's value equals mine."""
    best = 0

    def arm(node: TreeNode | None) → int:
        nonlocal best
        if node is None:
            return 0
        left = arm(node.left)
        right = arm(node.right)
        left = left + 1 if node.left and node.left.val == node.val else 0
        right = right + 1 if node.right and node.right.val == node.val else 0
        best = max(best, left + right)    # RECORD
        return max(left, right)         # RETURN

    arm(root)
    return best

def height(node: TreeNode | None) → int:
    """Plain height, for comparison. Edge convention."""
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))

def diameter_broken_no_nonlocal(root: TreeNode | None) → int:
    """The trap, written out. Without `nonlocal`, the assignment creates a
    NEW local `best` inside height(), and the outer one is never touched."""
    best = 0

    def h(node: TreeNode | None) → int:
        if node is None:
            return -1
        left = h(node.left)
        right = h(node.right)
        # best = max(best, left + right + 2)    ← would raise UnboundLocalError
        return 1 + max(left, right)

    h(root)
    return best
# always 0

```

```

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

if __name__ == "__main__":
    root = from_list([1, 2, 3, 4, 5])
    print(diameter(root), diameter_naive(root))          # 3 3
    print(height(root))                                  # 2
    print(diameter_with_path(root))                      # (3, [4, 2, 1, 3])

    # the diameter does NOT have to pass through the root
    lopsided = from_list([1, 2, None, 3, 4, 5, 6])
    print(diameter(lopsided))                            # 4 ← 5→3→2→4→6
    print(height(lopsided))                              # 3
    print(diameter(lopsided) ≥ height(lopsided))         # True, always
    print(diameter(lopsided) ≤ 2 * height(lopsided))     # True, always

    print(diameter(None), diameter(TreeNode(1)))         # 0 0
    print(diameter(TreeNode(1, TreeNode(2))))            # 1

    print(is_balanced(root), is_balanced_tuple(root))   # True True
    chain = from_list([1, 2, None, 3, None, 4])
    print(is_balanced(chain))                            # False

    print(longest_univalue_path(from_list([5, 4, 5, 1, 1, None, 5]))) # 2

    print(diameter_broken_no_nonlocal(root))            # 0 ← silently wrong

    # the cost difference, on a chain
    import time
    long_chain = TreeNode(0)
    node = long_chain
    for i in range(1, 2000):
        node.right = TreeNode(i)
        node = node.right
    t = time.perf_counter(); diameter(long_chain); fast = time.perf_counter() - t
    t = time.perf_counter(); diameter_naive(long_chain); slow = time.perf_counter() - t
    print(f"one pass {fast:.4f}s vs naive {slow:.4f}s ({slow / fast:.0f}x)")

```

6 What it costs

The two versions

```
one pass:  each node visited exactly once          O(n)
naive:     height() called at every node, and each
           call walks that node's whole subtree
           = the sum of all subtree sizes
```

```
balanced tree:  sum of subtree sizes = O(n log n)
degenerate tree: n + (n-1) + (n-2) + ... = n(n+1)/2 = O(n^2)
```

n = 1,000	one pass 1,000	naive ~500,000	500×
n = 10,000	one pass 10,000	naive ~50,000,000	5,000×
n = 100,000	one pass 100,000	naive ~5×10 ⁹	50,000× – will not finish

The measurement is in the code above. Run it rather than trusting the table; at 2,000 nodes the difference is already a few hundred times.

Space

```
recursion stack    O(height)
outer variable     O(1)
-----
extra space        O(height)
```

balanced, n = 1,000,000	~20 frames
degenerate, n = 10,000	10,000 frames → RecursionError

Same warning as every tree problem. A **skewed tree is exactly the worst case for both the naive time and the recursive space**, which is worth noticing: the input that makes the naive version $O(n^2)$ is the same input that overflows the stack.

The `diameter_with_path` variant is different: it builds lists as it goes, so its space is $O(n)$ in the worst case, not $O(\text{height})$. **Returning a path costs more than returning a number**, and saying so is the right answer if asked to produce the path.

The bounds

```
height    ≤    diameter    ≤    2 × height
```

a chain of n nodes:	height = n-1,	diameter = n-1	(diameter = height)
a perfect tree, h=3:	height = 3,	diameter = 6	(diameter = 2 × height)

Both bounds are tight, and being able to give the shape that achieves each is a good answer to “how do those two relate?”.

Why `+ 2` and not `+ 1`

```
edge convention, empty subtree = -1
```

```
leaf:                left = -1, right = -1 → -1 + -1 + 2 = 0    correct
node with 1 child: left = 0, right = -1 → 0 + -1 + 2 = 1    correct
node with 2 leaves: left = 0, right = 0 → 0 + 0 + 2 = 2    correct
```

The `+ 2` is **the two edges from the node down into each subtree**. With the node convention (empty = 0) the formula is `left + right` with no addition, and mixing the two gives an answer off by exactly two — which passes on a single node and fails on everything else.

7 The traps

Trap 1 — assuming the diameter passes through the root

```
return height(root.left) + height(root.right) + 2    # WRONG
```

```
diameter(from_list([1, 2, None, 3, 4, 5, 6]))
this version → 3
correct      → 4    (the path 5 → 3 → 2 → 4 → 6, which never touches the root)
```

The path bends at some node; there is no reason it is the root. This is the single most common wrong answer, and it passes on symmetric test trees.

Trap 2 — forgetting `nonlocal`

```
best = max(best, left + right + 2)
```

```
UnboundLocalError: cannot access local variable 'best' where it is not associated with a value
```

Python sees an assignment to `best` and makes it a local, so the read on the right-hand side fails. With a read-only reference it would not error — it would silently use the outer value and never update it, returning 0. **Declare `nonlocal best` at the top of the inner function.**

Trap 3 — returning the diameter instead of the height

```
return max(left, right, left + right + 2) # WRONG
```

This returns something the parent cannot use. A path that bends at this node has already used both children, so it cannot continue upward — **the parent needs a single downward arm, not a bent path.** The result is an answer that is too large and grows nonsensically with depth.

Say the sentence: “what I return and what I record are different quantities.”

Trap 4 — mixing the two conventions

```
if node is None:
    return 0 # node convention
best = max(best, left + right + 2) # edge convention's formula
```

Off by two on every non-trivial tree, and correct on a single node. **Pick edges or nodes, and make the base case and the formula agree.**

Trap 5 — `is_balanced` calling `height` at every node

```
if abs(height(node.left) - height(node.right)) > 1:
```

The same $O(n^2)$, in a different problem. The interviewer will accept it and then ask for one pass. Use the sentinel, or the tuple.

Trap 6 — a sentinel value that a real height could take

```
return -1 # as "unbalanced"
```

`-1` is the height of an empty tree, so this is indistinguishable from a legitimate answer and the check silently reports unbalanced trees as balanced. **`-2` is safe because a real height is never below `-1`**; with the node convention, `-1` is safe because a real height is never below `0`.

Trap 7 — recursion on a deep tree

```
diameter(chain_of_10000_nodes)
```

```
RecursionError: maximum recursion depth exceeded
```

The $O(n)$ version fixes the time and not the space. If n can be 10^5 with no balance guarantee, either convert to an explicit stack with a postorder walk, or say the risk out loud.

Trap 8 — reporting the number of nodes when asked for edges

```
diameter(from_list([1,2,3,4,5]))
edges: 3
nodes: 4
```

Both are defensible answers to “the diameter”; only one matches the tests. **Ask, or state which you are giving.**

8 In the interview

How it gets asked

- The base: “Find the diameter of the binary tree.” LeetCode 543.
- The constraint: “Now do it in $O(n)$.” — which is the real question.
- The sibling: “Is this tree height-balanced?” LeetCode 110, and the same one-pass fix.
- The escalation: “Maximum path sum.” LeetCode 124, which is [day 104](#).
- The extension: “Return the path, not just its length.”

What to say out loud, in the first ninety seconds

1. **Define it, and flag the two conventions.** “The diameter is the longest path between any two nodes. I will count edges, so a single node is 0 — tell me if you want the node count and I will add one.”
2. **Say the thing people get wrong, before you get it wrong.** “It does not have to pass through the root. The path bends at some node, and that node can be anywhere.”
3. **State the key observation.** “Every path has exactly one highest node. So if I evaluate every node as the top of a path, I consider every path exactly once. At a node, the bent path is $\text{left height} + \text{right height} + 2$.”
4. **Write the naive version and name its cost.** “The direct version calls `height` at every node, and `height` walks a whole subtree — so it is $O(n \log n)$ on a balanced tree and $O(n^2)$ on a chain.”
5. **State the trick precisely.** “The improvement is that `height` already visits every node once and already has both children’s heights when it needs them. So I compute the diameter inside the height function and record it in an outer variable. **The**

value I return and the value I am looking for are different: I return the height because that is what my parent needs, and I record the diameter, because a path bending here cannot be extended upward through my parent.”

6. **Give both complexities.** “ $O(n)$ time, one visit per node. $O(\text{height})$ space for the stack — twenty frames on a balanced million-node tree, and a million on a chain.”

The follow-ups

“**Why is the naive version $O(n^2)$?**” “Because `height` is itself a full walk of a subtree, and I am calling it once per node. The total work is the sum of every subtree’s size. On a balanced tree that sums to $O(n \log n)$; on a skewed tree it is $n + (n-1) + (n-2) + \dots$, which is $O(n^2)$. Concretely, on a ten-thousand-node chain that is about fifty million operations against ten thousand for the one-pass version. And it is worth noticing that the input that makes it quadratic — a chain — is the same input that overflows the recursion stack, so a skewed tree is the worst case for both.”

“**How does the one-pass version work?**” “The observation is that I am already computing every height, and at the moment I compute a node’s height I have both children’s heights sitting in local variables. The diameter through that node is exactly those two added together, plus two edges. So there is nothing more to compute — I just need somewhere to put it. I keep an outer variable and update it at every node. The part worth stating explicitly is that **the returned value and the recorded value are different quantities**: I return the height, because that is what my parent needs to know about my side; I record the bent path, because my parent cannot use it — a path that bends at me has already consumed both of my children, so it cannot also go up through me.”

“**Now check whether the tree is balanced, in one pass.**” “Same shape. The naive version calls `height` at every node and is $O(n^2)$ for the same reason. In one pass, the function returns the height, but returns a **sentinel** as soon as it discovers an imbalance — I use `-2`, because a genuine height with the edge convention is never below `-1`. Once a `-2` appears it propagates straight to the top with no further work, which is what makes it a single pass. The alternative is returning a tuple of `(height, is_balanced)`, which is clearer and allocates a tuple per node. I would use the sentinel and say why the specific value is safe — using `-1` there would be indistinguishable from the empty-tree height and would silently report unbalanced trees as balanced.”

“**Return the actual path, not just its length.**” “Same recursion with a bigger return value: instead of returning just the height, return the height and the deepest downward path from this node. Then when a node beats the best diameter, the winning path is the left arm reversed, plus this node, plus the right arm. The cost changes

though, and I would say so: building those lists makes the space $O(n)$ in the worst case rather than $O(\text{height})$, because every frame is holding a path. If that mattered, the alternative is to record only the *node* where the best diameter was found in the first pass, then do a second short walk from there to reconstruct the two arms — two passes, but $O(\text{height})$ space.”

“What is the relationship between the height and the diameter?” “ $\text{height} \leq \text{diameter} \leq 2 \times \text{height}$, and both bounds are tight. The lower bound because the root-to-deepest-leaf path is itself a path, so the diameter is at least the height. The upper bound because every path is two downward arms glued at its highest node, and each arm is at most the height. A chain achieves the lower bound — its diameter equals its height. A perfect tree achieves the upper bound — the path between two opposite leaves is exactly twice the height.”

“What if the tree is very deep?” “Then the $O(n)$ fix does not help me, because it fixed the time and not the space — the recursion is still $O(\text{height})$, and a chain of ten thousand nodes exceeds Python’s default limit of a thousand. Two options. Raise the limit, which is honest but fragile. Or do the postorder walk with an explicit stack: I push each node with a flag, and on the second visit I have both children’s heights available from a dictionary or a value stack, and I do exactly the same two operations. That is more code and it is what I would write if the constraint said $n \leq 10^5$ with no balance guarantee.”

A model answer

Asked: *find the diameter of the binary tree in $O(n)$.*

“The diameter is the longest path between any two nodes. Two things to pin down first. I will count **edges**, so a single node has diameter 0 — say if you want the node count and it is one more. And the path **does not have to pass through the root**; it bends at some node, and that node can be anywhere. A tree with a deep left subtree and one node on the right has its diameter entirely inside the left side.

The observation that makes this tractable: **every path has exactly one highest node** — one node on it that is closer to the root than all the others. So if I evaluate every node as the top of a path, I have considered every possible path exactly once, with no duplication. And at a given node, the longest path bending there is simply `height of the left subtree + height of the right subtree + 2`.

The direct implementation of that is to write a `height` function and call it at every node. It is correct, and it is $O(n^2)$ on a skewed tree, because `height` is itself a walk of a whole subtree and I am doing it once per node — the total is the sum of all subtree sizes. On a ten-thousand-node chain that is around fifty million operations against ten thousand.

The fix is to notice that `height` **already** visits every node exactly once, and at the moment it computes a node’s height it already has both children’s heights sitting in local variables. So the diameter through that node is available right there, for free. There is nothing extra to compute — I only need somewhere to put it. So I keep a variable outside the recursion and update it at every node.

The part I want to say precisely, because it is the whole technique: **the value I return and the value I am looking for are different**. I return the `height`, because that is what my parent needs — ‘how far down does your side go’. I record the **bent path** on the side, because my parent has no use for it: a path that bends at me has already consumed both of my children, so it cannot also travel upward through me. Confusing those two — returning the bent path — gives an answer that is too large and grows with depth.

One Python detail: the inner function needs `nonlocal` to assign to that outer variable, otherwise Python makes it a new local and you get an `UnboundLocalError` — or, if you only read it, a silent zero.

Complexity is $O(n)$ time, one visit per node, and $O(\text{height})$ space for the call stack — about twenty frames on a balanced million-node tree and a million on a chain, so if the input can be skewed and large I would either raise the recursion limit or do the postorder walk with an explicit stack.

And the same shape solves several neighbouring problems: balanced-tree checking with a sentinel instead of a recorded maximum, maximum path sum with the arms floored at zero, and longest same-value path with one extra condition on each arm. It is one technique, not four.”

9 Recall card

- **Diameter = the longest path between any two nodes, and it does NOT have to pass through the root** — that is the most common wrong answer and it passes on symmetric tests. Count **edges** by default (LeetCode 543): a single node is 0, and `nodes = edges + 1`.
- **The key observation: every path has exactly one highest node**, so evaluating every node as the top of a path considers every path exactly once. At a node the bent path is `left height + right height + 2` (edge convention, empty = `-1`).
- **The naive version calls `height` at every node** → `O(n log n)` **balanced**, `O(n2)` **on a chain** — 50 million operations at `n = 10,000` against 10,000. **The same skewed input is also the stack-overflow case.**
- **The trick, and it is the whole phase: RETURN one quantity, RECORD another.** Return the **height** (what the parent needs); record the **bent path** in an outer variable (the parent cannot use it — a path bending here has already spent both children). Returning the bent path instead gives an answer that grows with depth. Python needs `nonlocal`, or you get `UnboundLocalError` or a silent 0.
- **The same shape solves five problems: diameter, balanced in one pass** (return a `-2` **sentinel** — `-1` is a real height and would silently pass), maximum path sum, longest univalue path, largest BST subtree. `O(n)` time, `O(height)` **space** — and returning the *path* rather than the length costs `O(n)` space instead. Bounds: `height ≤ diameter ≤ 2 × height`, both tight.

1 What this is, and why they ask it

Invalidation is how a cached copy stops being served once the real value has changed. **Eviction** is how a cache decides what to throw away when it is full. They are different problems and they are almost always confused.

Three sentences. Invalidation is about **correctness** — the cache holds something that is now wrong — and eviction is about **capacity**, where the cache holds something that is still right but which you cannot afford to keep. Invalidation has three real mechanisms, and choosing between them is choosing how stale you are willing to be. And there is one specific race between a read and a write that produces a permanently wrong cache entry, which is the thing interviewers are actually testing.

They ask it because [yesterday's](#) lesson made the cache sound free, and this is the bill. The famous line — *there are only two hard things in computer science: cache invalidation and naming things* — is a joke about a real problem: the moment there are two copies of a value, something has to guarantee they agree, and nothing in the cache itself does. Say “I would add a cache” without saying how it is invalidated and you have described half a design.

2 The story

The board outside Chandran's shop had the prices on it in chalk, and the board was the whole reason people stopped there instead of walking another hundred metres to the market.

Tomato 40. Onion 30. Beans 60. Written big, readable from the road, so nobody had to get off their scooter to ask.

The prices came from the wholesale market at four in the morning, and they moved. Not every day, but often enough — a lorry did not arrive, or it rained in Kolar, and onions were 45 instead of 30.

There were three ways of handling that, and over about six years Chandran tried all three.

The first was to change the board the moment he unloaded. Four fifteen in the morning, before opening, with a wet cloth and cold hands. That was correct and it was miserable, and on the days when nothing had changed it was fifteen minutes of work for nothing.

The second was to not touch the board at all in the morning, and instead **wipe off any line whose price had moved**. The board then said tomato and beans, and nothing about onions, and the first person to ask for onions was told the price by Chandran and he chalked it back up then. Less work, and for the first hour of the day some people had to actually ask.

The third was what he did on Sundays, when he was tired. He wiped the whole board at closing time and wrote it fresh the next morning from scratch.

The thing that convinced him to settle on the second way happened in his fourth year and it was not about work at all.

His nephew was helping. A customer asked for beans, the boy shouted the question inside, Chandran said sixty, and the boy went to write it up. But in between the question and the chalk, Chandran had come out and changed beans to seventy, because a new crate had come in at a higher rate.

The boy, who had been holding the old number in his head the whole time, then wrote sixty on the board. Over the top of the seventy.

And there it stayed. Nobody wiped it, because nothing had changed since — the board was simply wrong, and it stayed wrong until somebody happened to notice at about four in the afternoon.

Chandran's rule after that was that his nephew was allowed to **wipe** a line, and was never allowed to **write** one from a number he had been holding for a while. If you have been carrying an old number around, do not put it on the board. Rub it out and let the next person ask.

3 The idea in plain English

Chandran's three methods are the three cache-write strategies, and his nephew's mistake is the race condition every interviewer asks about.

- The board is the **cache**; the crates and the wholesale rate are the **origin** — the database.
- Changing the board at four fifteen is **write-through**: update the cache when you update the truth.

- Wiping a line when the price moves is **write-around with invalidation**: update the truth, delete the cache entry, and let the next reader repopulate it.
- Wiping the whole board at closing time is a **TTL** — everything expires and is rebuilt.
- The boy writing a stale sixty over a fresh seventy is the **read-modify race**, and it produces a cache entry that is wrong *and not going to be corrected*.

Invalidation: the three mechanisms

One — TTL (time to live). Every entry expires after `n` seconds. Nothing has to be tracked or notified.

- + trivially simple, and self-healing: every bug in your invalidation logic is fixed within one TTL
- + no coordination between the writer and the cache at all
- guaranteed staleness of up to the TTL

A TTL is the seatbelt. Even with perfect explicit invalidation, put one on every key — it bounds how long any mistake can last.

Two — explicit invalidation on write. When the data changes, delete the key.

- + staleness is near zero
- the writer must know every key derived from the data it just changed, which is the hard part

The difficulty is not the deleting. It is that one write can affect many keys: changing a product's price invalidates the product page, the category listing, the search result, the user's cart total, and the homepage banner. **Finding all of them is the actual work**, and the standard answer is to publish an event and let each cache owner decide what to drop.

Three — versioned keys. Put a version in the key, and bump the version instead of deleting anything.

```
product:987:v41    →    product:987:v42
```

- + no delete, no race — a new version is simply a different key
- + old entries fall out by themselves under eviction
- you must store the current version somewhere, and read it
- the old entries occupy memory until evicted

This is genuinely useful when one change invalidates thousands of keys: bump a *namespace* version and every key underneath it becomes unreachable in one operation.

The write path: three strategies

	WHAT HAPPENS ON A WRITE	STALENESS	COST
Write-through	write DB and cache, synchronously	none	every write pays both; caches data nobody may read
Write-around	write DB, delete the cache key	one miss	the next read is slow
Write-back	write cache, flush to DB later	none to readers	data loss if the cache dies

Write-around is the default. Say it that way: *“I write to the database and delete the cache entry. The next read repopulates it.”*

Write-through is right when a write is almost always followed immediately by a read of the same key — a user editing their own profile, then being shown it. It wastes cache space on data nobody reads.

Write-back is fast and it can lose data, so it is only for values whose loss is tolerable: view counts, last-seen timestamps, non-critical counters. Its real advantage is **collapsing writes** — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

Delete, do not update

This is the rule Chandran arrived at, and it is the single most useful thing in this lesson.

On a write, delete the cache key. Do not write the new value into the cache.

Why: writing the new value opens a window where a *concurrent reader* can overwrite it with an older value. Deleting does not, because a deleted key produces a miss, and a miss re-reads the truth.

Here is the race, precisely. It is the interview question.

A is a READER (cache miss). B is a WRITER.

```
t1  A: cache GET x → miss
t2  A: DB read x   → gets the OLD value, 60
t3      B: DB write x = 70
t4      B: cache DELETE x           (or SET x = 70)
t5  A: cache SET x = 60             ← A writes its stale value LAST
```

result: the cache holds 60. The database holds 70.
Nothing will correct it – there is no further write.
It is wrong until the TTL expires.

A has been holding an old number in its head, and it writes it on the board last. Exactly the nephew.

- The mitigations, in the order you would state them:
- 1. **A TTL on every key.** Bounds the damage to the TTL. Always do this.
 - 2. **Delete rather than set on write.** It does not eliminate the race above but it eliminates the much more common writer-versus-writer version of it.
 - 3. **Delete twice** (“delayed double delete”): delete, write the database, then delete again after a short delay, so a reader that populated the cache in between is cleared.
 - 4. **A per-key lock** on the miss path, so only one reader is ever fetching-and-setting a key.
 - 5. **Compare-and-set with a version**, so a stale write cannot land on top of a newer one — the same mechanism as [day 095’s](#) auction.

Say the race, then say the TTL bounds it, then say the stronger options exist. That sequence is the answer.

Eviction: a different problem

Invalidation removes entries that are **wrong**. Eviction removes entries that are **right but do not fit**.

POLICY	THROWS OUT	GOOD FOR	BAD AT
LRU	least recently used	almost everything	one big scan wipes the cache
LFU	least frequently used	stable popularity	new items never build a count
FIFO	oldest inserted	nothing much	ignores usage entirely
Random	any entry	when memory is tight	unpredictable, but surprisingly decent
TTL only	expired entries	time-bounded data	does not respond to memory pressure

LRU is the default and it is right most of the time, because recency predicts reuse. You implemented it on [day 076](#).

Its famous weakness is worth being able to name: **a single sequential scan of everything** — a nightly report, an export, a crawler — touches every key once, evicts the entire working set, and leaves you with a cache full of items nobody will ask for again. The hit rate collapses and stays collapsed until the working set is faulted back in.

The fixes: **admission control** (do not put an item in the cache until it has been requested twice), or **segmented LRU** (a small probationary area that new items must earn their way out of). Modern caches use **W-TinyLFU**, which is an admission filter in front of an LRU — it is what Caffeine and several CDNs use, and naming it is a strong signal.

Redis’s policies, which you will be asked about by name

noeviction	refuse writes when full ← the DEFAULT, and it surprises people
allkeys-lru	evict any key, least recently used
allkeys-lfu	evict any key, least frequently used
allkeys-random	
volatile-lru	evict only keys that have a TTL set
volatile-ttl	evict the key expiring soonest
volatile-random	

noeviction is the default, which means a Redis used as a cache without configuration will start returning errors on writes when it fills up rather than making room. **If Redis is a cache, set `allkeys-lru`** . **If Redis is a database, leave `noeviction`** . That one sentence is a genuinely useful thing to say.

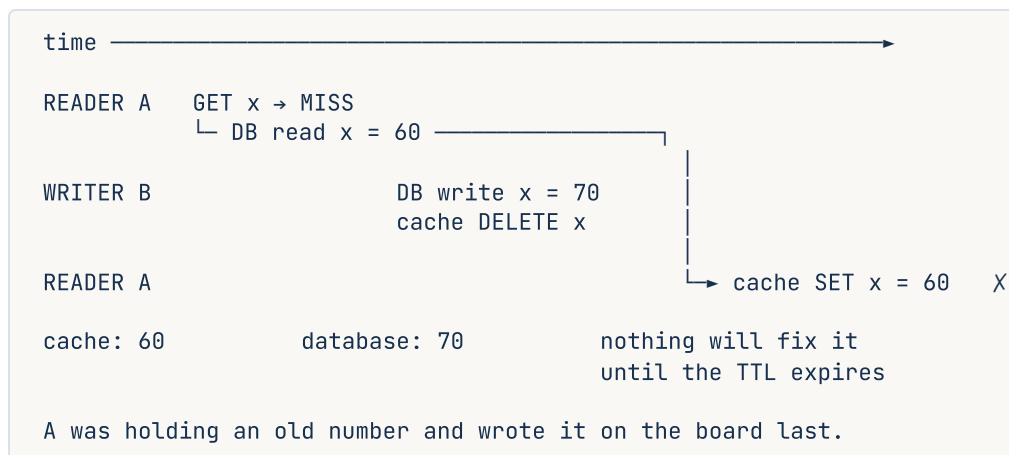
Also: Redis’s LRU is **approximate** — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access. Five samples gets you close to true LRU; the sample count is tunable.

4 The picture

The two problems, which are genuinely separate.

INVALIDATION	EVICTION
the entry is WRONG	the entry is RIGHT but does not fit
caused by: a write to the origin	caused by: memory pressure
fixed by: TTL, delete on write, versioned keys	fixed by: LRU, LFU, admission control
failure: serving stale data (a correctness bug)	failure: a collapsed hit rate (a performance bug)

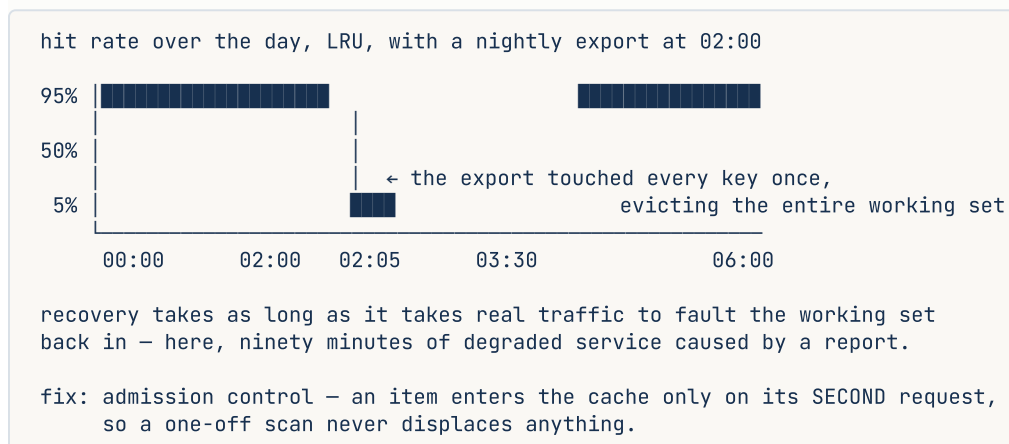
The race, drawn on a timeline. This is the diagram to be able to produce.



The three write strategies, side by side:



Eviction under a scan, which is the failure worth drawing:



5 How it actually works

Choosing an invalidation strategy, in order

Step 1 — put a TTL on everything. Even if you invalidate explicitly. It is the bound on every bug you have not found yet, and it costs nothing.

Step 2 — decide the tolerable staleness per kind of data. This is a product question, not an engineering one:

a user's own profile after they edit it	0 seconds	→ invalidate on write
another user's profile	60 seconds	→ TTL
a product price	5 minutes	→ TTL, plus invalidate on change
a category listing	10 minutes	→ TTL
a homepage banner	1 hour	→ TTL
exchange rates	1 minute	→ TTL

“I saved it and it did not change” is the worst possible staleness, which is why the user’s own writes always get explicit invalidation even when everything else is on a TTL.

Step 3 — for explicit invalidation, publish an event rather than deleting directly. The writer does not know every derived key. Publishing `product.updated` and letting each cache owner subscribe keeps that knowledge where it belongs.

Step 4 — for one-to-many invalidation, version a namespace.

```
GET category:electronics:version → 17
GET category:electronics:v17:page:1

to invalidate the whole category:
INCR category:electronics:version → 18
→ every v17 key is now unreachable, in one operation
→ they age out under eviction on their own
```

One `INCR` invalidates thousands of keys with no scan and no key list. This is the trick to reach for when someone says “but that change affects a hundred cached pages”.

The cache-aside code, written properly

```
def get_product(product_id: int) → Product:
    key = f"product:{product_id}"
    cached = cache.get(key)
    if cached is not None:
        return deserialize(cached)

    product = db.query("SELECT ... WHERE id = ?", product_id)
    cache.set(key, serialise(product), ttl=300)    # ALWAYS a TTL
    return product

def update_product(product_id: int, changes: dict) → None:
    db.update("UPDATE products SET ... WHERE id = ?", product_id, changes)
    cache.delete(f"product:{product_id}")        # DELETE, not SET
    events.publish("product.updated", product_id) # others invalidate their own
```

Three things to point at while writing it: the TTL is not optional; the write **deletes** rather than sets; and the database write happens **before** the delete, not after.

That last one matters:

```
delete first, then write DB:
    t1 delete cache
    t2 a reader misses, reads the OLD value from the DB, caches it
    t3 the write lands in the DB
    → the cache holds the old value with a fresh TTL

write DB first, then delete:
    the window is much smaller – only a reader who read before t1
    and writes after the delete
```

Database first, then invalidate. Both orders have a race; this one's window is far smaller.

Sizing eviction

The question “is my cache big enough” has a measurable answer:

```
watch two numbers:
    hit rate           - is the cache doing its job?
    eviction rate      - how many keys per second are being pushed out?

low hit rate + high eviction rate    → too small; the working set does not fit
low hit rate + low eviction rate     → wrong keys, or access is uniform;
                                     a bigger cache will not help
high hit rate + high eviction rate   → fine; a long tail is churning harmlessly
```

That two-by-two is the answer to “how would you tune the cache”, and it is much better than “I would make it bigger”.

What real systems do

- **Redis** with `maxmemory` and `maxmemory-policy allkeys-lru` is the standard application cache. Its LRU is **approximate** — sampled, not exact — and `maxmemory-samples` tunes how close it gets.
- **Memcached** uses a segmented LRU with slab allocation, which is why it can suffer *slab calcification*: memory assigned to one object size cannot easily be reused for another.
- **CDNs** invalidate by **purge** — an API call that tells every edge location to drop a URL — and it is slow and rate-limited, which is why the standard practice is **cache-busting URLs** instead: `app.a3f9c2.js` rather than `app.js`. **A new name is a new key, so nothing needs invalidating.** That is the versioned-key idea, applied at the edge.
- **Facebook's memcached paper** introduced **leases**: on a miss, the cache hands the reader a token, and a `set` is only accepted if the token is still valid — which closes exactly the race above. It is the most rigorous published answer to this problem.
- **Caffeine** (Java) and several CDNs use **W-TinyLFU**: a small frequency sketch decides whether a new item is worth admitting at all, which is what makes them resistant to scans.

6 The numbers

What a TTL is actually choosing

TTL	max staleness	refreshes/day/key	DB load for 1M cached keys
10 s	10 s	8,640	100,000 reads/s
60 s	1 min	1,440	17,000 reads/s
5 min	5 min	288	3,300 reads/s
1 hour	1 hour	24	280 reads/s
1 day	1 day	1	12 reads/s

The TTL is a dial between staleness and origin load, and the relationship is linear. Going from 60 seconds to 5 minutes cuts the refresh load by a factor of five and increases the worst-case staleness by the same factor. There is no cleverness in it — that is the entire trade.

The cost of the race, if you never fix it

```
6,000 reads/s, 100 writes/s
race window (DB read to cache set)      ~2 ms
probability a given write overlaps a
  concurrent read of the SAME key      depends on key popularity

for a hot key read 100 times/second:
  P(a write lands inside a 2 ms window) 100 × 0.002 = 20% per write
→ a hot key that is written once a minute is stale
  for the remainder of its TTL, roughly one time in five
```

With a 300-second TTL, that is a wrong value served for up to five minutes, several times an hour, on your most-read key. That number is why the race is worth taking seriously rather than dismissing as unlikely.

Eviction and hit rate

```
working set 2 GB, cache 2 GB      hit rate ~90%,  eviction rate low
working set 2 GB, cache 1 GB      hit rate ~70%,  eviction rate high
working set 2 GB, cache 500 MB    hit rate ~45%,  eviction rate very high

and the load consequence at 6,000 reads/s:
90% →    600 reads/s reach the DB
70% →   1,800 reads/s
45% →   3,300 reads/s
```

Halving the cache more than doubled the database load. Cache memory is usually the cheapest capacity you can buy, and this table is the argument.

The scan disaster, measured

```
before the nightly export  hit rate 95%,   600 DB reads/s
during and after           hit rate 5%,   5,700 DB reads/s      9.5×
recovery time              as long as real traffic takes to refill the working set
                           – commonly 30-90 minutes

with admission control (enter on the second request):
  the export's keys are never admitted at all
  hit rate during the export: unchanged
```

A nightly report causing a ninety-minute morning outage is a real and common incident, and “an admission filter, or run the export against a replica with its own cache” is the answer.

Write-back's collapse ratio

```
a video watched 10,000 times in a minute
write-through: 10,000 database writes
write-back:      1 database write per flush interval

collapse ratio for a hot counter: 100x - 10,000x
cost of a crash: up to one flush interval of counts, lost
```

That ratio is why view counters are the canonical write-back use case — and why they are also the canonical example of data whose loss is acceptable.

7 The trade-offs

TTL against explicit invalidation

A TTL is simple, needs no coordination, and is self-healing — every invalidation bug you have not found is bounded by it. It guarantees staleness.

Explicit invalidation gives near-zero staleness and requires the writer to know every derived key, which is where the bugs live. Miss one, and it is stale until the TTL saves you — which is why you want both.

Use both, always. Explicit invalidation for the paths that matter, and a TTL underneath as the backstop.

Delete or update on write?

Delete. Two reasons. It closes the writer-versus-writer race entirely, because there is nothing to overwrite. And it avoids caching values that nobody subsequently reads — a write-heavy key that is rarely read should not be occupying cache space.

I would update instead if the value is expensive to recompute and is certain to be read immediately — a user's own profile right after they save it, where the alternative is a guaranteed miss on the very next request. That is write-through, and it is a deliberate exception.

Write-back, and what it is really for

I would not use write-back for anything I could not lose. The failure is not subtle: the cache dies and everything not yet flushed is gone, with no trace.

But for counters it is the right answer and the alternative is worse — ten thousand database writes a minute for a view count is real load, spent on a number nobody needs to be exact. The honest framing is: *this data is approximate by nature, so I am trad-*

ing exactness I do not need for load I cannot afford.

LRU, and where it fails

LRU by default. It is one line of configuration, it matches how most data is accessed, and everyone understands it.

I would not use plain LRU if the workload contains periodic full scans — reports, exports, crawlers — because a single scan evicts the entire working set. Then: admission control, a separate cache for the scanning workload, or run the scan against a replica.

LFU is right when popularity is stable and long-lived — a catalogue where the same items are popular for months. Its weakness is the mirror image: a genuinely new item can never accumulate a count and so is evicted immediately, which is why practical implementations decay the counts over time.

The thing that is genuinely hard

One write invalidating many keys. A price change touches the product page, the category listing, the search index, the cached cart totals, and the homepage. There is no mechanism that finds those for you. The three answers are: **events**, so each cache owner drops its own keys; **namespace versioning**, so one `INCR` invalidates a whole family; or **short TTLs**, and accept the staleness.

Anyone who says invalidation is easy has only ever invalidated one key at a time.

Where this breaks entirely

- **Multiple caches with different TTLs.** The browser has one copy, the CDN another, Redis a third. A user who “hard refreshes” and sees a different answer from a normal refresh is seeing exactly this, and invalidating all layers is genuinely difficult. **This is why static assets get content-hashed names instead.**
- **Read-your-own-writes across layers.** A user saves and immediately reads, and the read goes through a CDN with a five-minute TTL. Invalidating Redis was not enough.
- **Cache warm-up after a deploy.** An empty cache at peak is the same spike as an outage, which is why large systems pre-warm before taking traffic.

8 In the interview

How it gets asked

- The direct one: *“How do you keep the cache consistent with the database?”*

- The race: “A write and a read happen at the same time. Walk me through it.”
- The choice: “Write-through, write-back or write-around?”
- The eviction one: “The cache is full. What do you throw away?”
- The nasty one: “A nightly report runs and the morning is slow. Why?”
- The scope one: “Changing this one product invalidates a hundred cached pages. Now what?”

What to say out loud, in the first ninety seconds

1. **Separate the two problems immediately.** “Invalidation is about correctness — the entry is wrong. Eviction is about capacity — the entry is right but does not fit. They have different mechanisms and different failure modes.”
2. **Give the default and the backstop together.** “Write to the database, then **delete** the cache key — write-around. And a TTL on every key regardless, as the bound on every invalidation bug I have not found.”
3. **Say delete, not update, and why.** “I delete rather than write the new value, because writing opens a window where a concurrent reader holding an older value can overwrite mine.”
4. **Name the staleness you are accepting, per data type.** “A user’s own profile after they edit it gets explicit invalidation, because ‘I saved it and it did not change’ is the worst kind of staleness. Another user’s profile can sit on a sixty-second TTL.”
5. **Pre-empt the eviction question.** “LRU, and `allkeys-lru` specifically in Redis, because the default is `noeviction` which returns errors on write when full.”
6. **Flag the scan weakness.** “The failure mode of LRU is a sequential scan — a nightly export touches every key once and evicts the entire working set.”

The follow-ups

“**Walk me through the race between a read and a write.**” “A reader misses the cache and goes to the database, and gets the old value — say 60. Before it writes that into the cache, a writer updates the database to 70 and deletes the cache key. Then the reader finishes and sets the cache to 60. The cache now holds 60, the database holds 70, and **nothing will correct it**, because there are no further writes — it is wrong until the TTL expires. The reader was holding an old number and wrote it last. Four mitigations, in the order I would apply them. A **TTL on every key**, which bounds the damage — that is non-negotiable regardless. **Delete rather than set on write**, which eliminates the much more common writer-versus-writer version. A **delayed double delete**: delete, write, then delete again after a short delay to clear anything a reader

populated in between. And if it genuinely matters, a **per-key lock on the miss path**, or Facebook's **lease** mechanism, where the cache hands out a token on a miss and only accepts a `set` if that token is still valid."

"**Write-through, write-back or write-around?**" "**Write-around by default:** write the database, delete the cache key, let the next read repopulate. It wastes nothing on data that is written and never read, and deleting is safer than setting. **Write-through** when a write is almost always followed by a read of the same key — a user editing their own profile and then being shown it — because otherwise you have engineered a guaranteed miss on the very next request. **Write-back** only for data I can afford to lose, and the canonical case is counters: a video watched ten thousand times in a minute is ten thousand database writes with write-through and one with write-back. That collapse ratio is the point, and the cost is that a crash loses up to one flush interval of counts — which for a view counter is acceptable and for anything financial is not."

"**The cache is full. What do you throw away?**" "LRU, because recency predicts reuse and it is one configuration line. In Redis specifically I would set `allkeys-lru`, and I would point out that the **default is noeviction**, which means an unconfigured Redis used as a cache starts returning errors on writes when it fills rather than making room. Redis's LRU is also **approximate** — it samples a few keys and evicts the least recently used among them, because maintaining exact ordering would cost a list update on every read. LRU's weakness is **scans**: one nightly export touches every key once and evicts the whole working set, and the hit rate does not recover until real traffic faults it back in. The fix is **admission control** — only admit an item on its second request — which is what W-TinyLFU does, and it is why Caffeine and several CDNs use it."

"**A nightly report runs and the morning is slow. Why?**" "That is the LRU scan problem, and it is a real incident pattern rather than a theoretical one. The report reads every row once. Every one of those reads populates the cache, and every insertion evicts something from the actual working set. By the time it finishes, the cache is full of items nobody will request again, the hit rate has gone from ninety-five percent to single digits, and the database is taking nine or ten times its normal read load at exactly the hour traffic picks up. Recovery is however long real traffic takes to refill the working set — commonly half an hour to ninety minutes. Three fixes: **do not admit scan traffic** into the cache at all, which is admission control or simply a flag on those queries; run the report against a **read replica** with its own cache; or run it against a snapshot. The one thing that does not work is making the cache bigger."

“Changing one product invalidates a hundred cached pages. Now what?”

“Three options and I would pick based on how many keys and how well I know them. If the set is knowable, **publish an event** — `product.updated` — and let each cache owner delete its own keys, because the writer should not have to know that the search service caches something derived from this. If the set is large or unknown, **version a namespace**: put a version number in the key prefix, and invalidate by incrementing that one counter — a single `INCR` makes thousands of keys unreachable, and they age out under eviction on their own. And if neither is worth the complexity, **short TTLs and accept the staleness**, which is a legitimate answer if the data can be a minute old. The one thing I would not do is try to enumerate and delete a hundred keys from the writer, because that list is wrong within a month.”

“Does the order matter — delete the cache before or after writing the database?” “Yes, and both orders have a race, but the windows are very different. If I **delete first and then write**, a reader can miss, read the old value from the database, and cache it with a fresh TTL — all before my write lands. That window is as long as my database write takes, which is milliseconds, and the result is stale for a full TTL. If I **write first and then delete**, the only reader that can hurt me is one that read the database *before* my write and sets the cache *after* my delete — a much smaller window. So: **database first, then invalidate**. And I would still have the TTL underneath, because neither order is airtight.”

A model answer

Asked: *how do you keep the cache consistent with the database?*

“First I want to separate two things that get run together. **Invalidation** is about correctness — the cached entry is wrong because the underlying value changed. **Eviction** is about capacity — the entry is still correct but there is no room for it. Different mechanisms, different failure modes: bad invalidation serves wrong data, bad eviction just makes things slow.

For invalidation, my default is **write-around**: write the database, then **delete** the cache key, and let the next reader repopulate it. I delete rather than write the new value, and that choice matters — writing opens a window where a concurrent reader that is holding an older value can land on top of mine, whereas a deleted key simply produces a miss and a fresh read of the truth.

And regardless of everything else, **a TTL on every key**. Not because I expect to need it, but because it bounds every invalidation bug I have not found yet. Explicit invalidation is where the mistakes live — one write often affects many derived keys, and the writer rarely knows all of them.

The race I would want to talk about is the read-write one. A reader misses, goes to the database, gets 60. Before it caches that, a writer sets the database to 70 and deletes the key. Then the reader completes and caches 60. **The cache now holds 60, the database holds 70, and nothing is going to fix it** — there are no more writes, so it stays wrong until the TTL expires. On a hot key that is read a hundred times a second, a write has roughly a one-in-five chance of hitting that window. So: the TTL bounds it; deleting rather than setting removes the more common writer-versus-writer variant; a delayed second delete clears anything a reader populated in between; and if it genuinely matters, a per-key lock on the miss path, or leases, where the cache issues a token on a miss and only accepts a `set` if the token is still valid.

On ordering: **write the database first, then invalidate**. Both orders race, but deleting first leaves a window as long as the database write during which a reader can cache the old value with a fresh TTL, which is much worse.

For the staleness budget I would go data type by data type, because it is a product decision. A user’s own profile after they edit it gets explicit invalidation — ‘I saved it and it did not change’ is the worst possible staleness. Another user’s profile is fine on a sixty-second TTL. A homepage banner can be an hour.

On eviction: **LRU**, and in Redis specifically `allkeys-lru`, because the default is `noeviction` and an unconfigured Redis will start refusing writes rather than making room. The weakness worth naming is scans — a nightly export touches

every key once and evicts the entire working set, taking the hit rate from ninety-five percent to nearly nothing and roughly ten-times-ing the database load right as the morning starts. The fix is admission control, not a bigger cache.

And if you tell me one write invalidates a hundred cached pages, my answer is namespace versioning: put a version in the key prefix and increment it once, which makes every key underneath unreachable in a single operation and lets them age out under eviction.”

9 Recall card

- **Invalidation is correctness (the entry is wrong); eviction is capacity (the entry is right but does not fit).** Different mechanisms, different failures. Default: **write-around** — write the DB, then **DELETE** the key — plus a **TTL on every key** as the bound on every invalidation bug you have not found.
- **Delete, do not update, on a write, and write the database FIRST, then invalidate** — deleting first leaves a window as long as the DB write in which a reader caches the old value with a fresh TTL.
- **The race to be able to draw:** reader misses → reads 60 from the DB → writer sets 70 and deletes → reader sets 60. **Cache 60, DB 70, and nothing will correct it** until the TTL. On a key read $100 \times /s$, a write has $\sim 20\%$ chance of hitting the window. Fixes in order: **TTL · delete-not-set · delayed double delete · per-key lock or leases.**
- **Write-through** when a read always follows the write (a user’s own profile). **Write-around** by default. **Write-back only for data you can lose** — a counter watched 10,000 times a minute becomes **one** DB write instead of 10,000.
- **LRU by default, and `allkeys-lru` in Redis — the default is `noeviction`, which returns errors when full.** Redis’s LRU is **approximate** (sampled). Its weakness is **scans**: a nightly export evicts the whole working set, hit rate $95\% \rightarrow 5\%$, DB load $\sim 10 \times$, recovery 30–90 minutes — fixed by **admission control**, not a bigger cache. And for **one write invalidating many keys**: publish an **event**, or **version a namespace** so one `INCR` invalidates thousands.

DSA topic: Height, depth, and diameter **System design topic:** Cache invalidation and eviction policies

Code these, in this order

One rule for the whole set: **before writing the one-pass version, write the $O(n^2)$ one and say its cost out loud.** Finding the improvement is what is being tested, and you cannot demonstrate finding it if you start at the answer.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Maximum Depth of Binary Tree	LeetCode 104 (Easy)	The base case, and which convention you are using.
2	Balanced Binary Tree	LeetCode 110 (Easy)	Spotting the repeated <code>height</code> call, and the sentinel.
3	Diameter of Binary Tree	LeetCode 543 (Easy)	Return one thing, record another.
4	Longest Univalue Path	LeetCode 687 (Medium)	The same shape with a condition on each arm.

On problem 2, write the naive one and time it

Build a chain of 2,000 nodes and time both versions. Record the ratio. Then say why a chain is the worst case for the naive version *and* for the recursion depth.

On problem 3, break it three ways on purpose

Run each and record the output: assume the path goes through the root; return the bent path instead of the height; omit `nonlocal`. Say which of the three raises and which two are silent.

On problem 4, notice what changed

Compare it line by line with the diameter solution. Say exactly which lines differ, and say what stayed the same.

After all four, write the two-line summary

For each problem: what is returned to the parent, and what is recorded on the side. If you can fill that table in from memory, you have the technique.

The definition drill

1. Define diameter in one sentence.
2. Give the edge answer and the node answer for a five-node example.
3. Construct a tree whose diameter does not touch the root, and give both numbers.
4. State the two bounds relating height and diameter, and give a shape that achieves each.

The key-observation drill

1. State the observation about every path having one highest node.
2. Say why that means evaluating every node considers every path exactly once.
3. Write the formula for the path bending at a node, in the edge convention.
4. Say why it is `+ 2` and what the formula becomes in the node convention.

The naive-cost drill

1. Write the naive version.
2. Say why it is not `O(n)`, in terms of what `height` does.
3. Compute the total work for a balanced tree and for a chain.
4. Give the operation counts at $n = 1,000, 10,000$ and $100,000$.
5. Say what else goes wrong on the same input that makes it quadratic.

The return-versus-record drill

1. State the technique in one sentence.
2. For the diameter, say what is returned and what is recorded.
3. Say why the parent cannot use the recorded value.
4. Fill in the table for all five problems in the family.
5. Return the bent path instead and describe how the answer misbehaves.

The `nonlocal` drill

1. Write the inner function without `nonlocal` and run it.
2. Quote the error, and say why Python produced it.
3. Say what happens if you only *read* the outer variable instead of assigning.
4. Name two alternatives to `nonlocal` and say which you prefer.

The sentinel drill

1. Write the one-pass balanced check with a sentinel.
2. Say why `-2` is safe and `-1` is not, under the edge convention.

3. Say what the safe sentinel would be under the node convention.
4. Write the tuple version and say what it costs.
5. Say what “short-circuit” means here and where the saving comes from.

The convention drill

1. Write the base case for both conventions.
2. Write the matching bending formula for each.
3. Mix them and compute the answer for a five-node tree; say by how much it is wrong.
4. Say why the mixed version is right on a single node.

The break-it drill

Trigger each and record the exact output or error:

1. Assuming the diameter passes through the root, on an asymmetric tree.
2. Omitting `nonlocal`.
3. Returning `max(left, right, left + right + 2)`.
4. Base case `0` with the `+ 2` formula.
5. `-1` as the unbalanced sentinel.
6. A recursive diameter on a 10,000-node chain.

The extension drill

1. Modify the solution to return the path, not the length.
 2. Say what the space complexity becomes and why.
 3. Describe the two-pass alternative that keeps `O(height)` space.
 4. Say which you would offer first in an interview.
-

The two-problems drill

1. Define invalidation and eviction, and say what each is about.
2. Give the cause and the fix for each.
3. Say what a failure of each one looks like to a user.
4. Say which one is a correctness bug and which is a performance bug.

The mechanisms drill

1. Name the three invalidation mechanisms.
2. For each, give one advantage and one cost.

3. Say why a TTL belongs on every key even with perfect explicit invalidation.
4. Say what makes explicit invalidation hard, and give an example with five affected keys.

The race drill

1. Draw the read-write race on a timeline, labelling both actors.
2. Say why nothing corrects it afterwards.
3. Compute the probability for a key read 100 times a second and a 2 ms window.
4. Name four mitigations in the order you would apply them.
5. Say which of them fully closes the race and which only bound it.

The ordering drill

1. State both possible orders of the database write and the cache delete.
2. For each, describe the race and the size of its window.
3. Say which order you would use and why.
4. Say what still protects you when the chosen order loses the race anyway.

The write-strategy drill

1. Name the three strategies and describe each in one line.
2. Give a use case for each, and the specific data type for write-back.
3. Compute the collapse ratio for a counter written 10,000 times a minute.
4. Say what a crash costs under write-back.
5. Say why you delete rather than set, in two reasons.

The eviction drill

1. Name five eviction policies and what each throws away.
2. Say which is the default choice and why.
3. Name LRU's failure mode and describe the incident it causes, with numbers.
4. Name the fix, and the algorithm that implements it.
5. State Redis's default policy and say why that surprises people.

The tuning drill

Fill in the two-by-two: for each combination of hit rate and eviction rate, say what it means and what you would do.

1. Low hit rate, high eviction rate.
2. Low hit rate, low eviction rate.

3. High hit rate, high eviction rate.

Then compute the database load at 6,000 reads/second for hit rates of 90, 70 and 45 percent.

The one-to-many drill

1. Say what makes “one write invalidates a hundred keys” hard.
2. Give the three answers and say when you would choose each.
3. Write the namespace-versioning keys and the single command that invalidates them.
4. Say what happens to the old entries afterwards.
5. Say how CDNs solve the same problem for static assets.

The failure drill

For each, say what happens and what you would add:

1. A nightly export runs at 02:00.
2. Redis is deployed with no `maxmemory-policy`.
3. A user saves their profile and immediately reloads the page.
4. A reader caches a value it read before a concurrent write.
5. The same value is cached in the browser, the CDN and Redis with different TTLs.
6. The cache is emptied by a restart at peak traffic.

Two of the six are not fixed by changing the cache configuration. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the diameter of the binary tree in $O(n)$.* The definition with the convention declared, the not-through-the-root warning given before you can get it wrong, the every-path-has-one-highest-node observation, the naive version priced, then the return-versus-record sentence said precisely, and both complexities.
2. *How do you keep the cache consistent with the database?* Invalidation and eviction separated, write-around plus a TTL as the default, delete-not-update with the reason, the read-write race drawn with what happens afterwards, the write ordering, and the staleness budget per data type.

3. *The cache is full. What do you throw away?* LRU with `allkeys-lru`, Redis's surprising default, the approximate-LRU detail, the scan failure with its numbers, and admission control as the fix.

Before you move on

- [] I can define diameter and give both conventions.
- [] I always say “it does not have to pass through the root” before writing code.
- [] I can state the every-path-has-one-highest-node observation.
- [] I can write the naive version and price it on a chain.
- [] I can say the return-versus-record sentence precisely.
- [] I know why the parent cannot use the recorded value.
- [] I can fill in the family table for all five problems.
- [] I know what `nonlocal` is for and what happens without it.
- [] I can write the one-pass balanced check and justify the sentinel value.
- [] I can give both bounds relating height and diameter, with achieving shapes.
- [] I know what returning the path costs, and the two-pass alternative.
- [] I can separate invalidation from eviction in one sentence each.
- [] I can name the three invalidation mechanisms and their costs.
- [] I can draw the read-write race and say why nothing fixes it.
- [] I know which order to write the database and the cache in, and why.
- [] I can name the three write strategies and the data type for write-back.
- [] I know Redis's default eviction policy and why it matters.
- [] I can describe the LRU scan incident with numbers and name the fix.
- [] I can invalidate a hundred keys with one command.
- [] I answered all three questions above out loud.

Same tree, symmetric tree, and subtree

DATA STRUCTURES AND ALGORITHMS

Same tree, symmetric tree, and subtree

You can compare two trees structurally and handle the null cases without crashing.

SYSTEM DESIGN

Content delivery networks

You can say what a CDN caches, where it sits, and what it cannot help with.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Are these two trees identical? Is this tree a subtree of that one?*
- *Would a CDN help this system? What would it not help with?*

Same tree, symmetric tree, and subtree

1 What this is, and why they ask it

Three problems that share one idea: a recursion that walks **two trees at once**, in step, comparing as it goes.

Three sentences. Everything so far has recursed on one tree; today the function takes two nodes and asks whether the subtrees below them match. That change makes the **base cases** the whole difficulty — there are three of them, not two, and the one people forget is “*exactly one of them is `None`*”. And once you have the two-tree walk, symmetry is the same function with the arguments crossed over, and “is this a subtree” is that function called at every node of the bigger tree.

They ask it because it is the cleanest test of null-handling in the whole topic. Almost every wrong answer here is a crash, not a wrong result — and the crash is always the same one, on the same line. It is also where candidates first meet a problem whose obvious $O(n \times m)$ solution has a much better answer if you are willing to change representation, which is the serialisation trick at the end.

2 The story

Kamala had been embroidering for thirty-one years and the shop sent her the work that needed checking, because she was quick and she did not miss things.

Three jobs came on the same Thursday and they were three different kinds of looking.

The first was a pair of pillow covers that were supposed to be identical. Two pieces, same design, one for each side of the bed. She did not look at them as pictures — she put them next to each other and went through them together, stitch group by stitch group, in the same order on both. Flower here, flower there, same colour, same size. Move on. The moment two things did not match she stopped, because there was no point continuing.

The second was a table runner with a border that was meant to be a mirror. Whatever was on the left of the middle was supposed to be reflected on the right. This one she did differently, and she was particular about it. She did not compare the left

half with the left half. She put her left thumb at the outermost point of the left side and her right thumb at the outermost point of the right side, and walked them **towards each other** at the same speed. The far left against the far right. Then the next one in from each side. That is what mirror means, she said — it is not the same order, it is the opposite order.

The third took her most of the afternoon. A large bedspread had come back from a customer who said a particular motif on it was wrong. She had the correct motif on a small square in her hand, about the size of a postcard, and she had to find whether that exact thing appeared anywhere in a bedspread the size of a door.

There was no clever way. She went across the bedspread, and at every place a motif started, she held the small square against it and checked the whole thing, stitch by stitch. Most of them failed on the first or second stitch and she moved on in a second. Two of them looked promising and she had to go all the way through before finding a difference.

She said afterwards that the third job was the same as the first job, done many times over.

And she made one point about the third that mattered. When she found a place where the motif started correctly, she had to check that it matched **completely** — all of it, right down to the last stitch, with nothing extra hanging off the bottom. A motif that matched for most of the way and then continued into something else was not the motif.

3 The idea in plain English

Kamala's three jobs are LeetCode 100, 101 and 572, and her method for each is the algorithm.

Same tree: walk two trees in step

The recursion takes **two nodes** instead of one:

```
def is_same(a, b):
    if a is None and b is None:
        return True                # both ran out together
    if a is None or b is None:
        return False               # one ran out first
    return (a.val == b.val
            and is_same(a.left, b.left)
            and is_same(a.right, b.right))
```

Three base cases, and the middle one is the whole lesson.

both None	→ True	two empty subtrees are the same
one None	→ False	different SHAPE, so they cannot match
neither None	→	compare the values, then both pairs of children

The `or` case is the one people leave out, and leaving it out does not give a wrong answer — it gives:

```
AttributeError: 'NoneType' object has no attribute 'val'
```

because the next line reads `a.val` when `a` is `None`.

Order matters in the `and` chain. Python short-circuits, so `a.val == b.val` is only evaluated after both null checks have passed, and the recursive calls are only made if the values match. That is Kamala stopping the moment two things did not match.

Symmetric tree: the same function, with the arguments crossed

A tree is **symmetric** if its left subtree is a mirror image of its right subtree. The trap is that “mirror” is not “same”.

```
def is_mirror(a, b):
    if a is None and b is None:
        return True
    if a is None or b is None:
        return False
    return (a.val == b.val
            and is_mirror(a.left, b.right)      # LEFT against RIGHT
            and is_mirror(a.right, b.left))     # RIGHT against LEFT
```

One line different from `is_same` : the children are crossed. `a.left` is compared with `b.right`, not with `b.left`.

That is Kamala’s thumbs walking towards each other. The outermost on the left matches the outermost on the right, not the innermost.

The common wrong answer:

```
return is_same(root.left, root.right)      # WRONG
```

This says the two halves are *identical*, not mirrored — which is a different property. On this tree:



`is_same(left, right)` → True (both halves are identical)
`is_symmetric` → False (a mirror would need 3 on the OUTSIDE of one side)

Identical halves are a translation; symmetric halves are a reflection. They coincide only when each half is itself symmetric.

Subtree: the same function, called everywhere

```

def is_subtree(big, small):
    if small is None:
        return True
    if big is None:
        return False
    return (is_same(big, small)
            or is_subtree(big.left, small)
            or is_subtree(big.right, small))
  
```

At every node of the big tree, ask “does the small tree match starting here?” — Kamala holding the square against every place a motif starts.

The word “subtree” is stricter than it sounds, and this is the trap. A subtree means **a node and everything below it** — you cannot stop midway.

big: 1 small: 1
 / \ /
 2 3 2

`is_subtree` → FALSE

The `1` in the big tree has a right child; the `1` in the small tree does not. `is_same` correctly returns `False` at the `or` base case. This is “nothing extra hanging off the bottom”.

The $O(n \times m)$ problem, and the trick

`is_subtree` calls `is_same` at every node, and each call can walk the whole small tree, so the worst case is $O(n \times m)$.

The trick: serialise both trees to strings and ask whether one string contains the other. Then `str.find` does the work in $O(n + m)$.

Two details make it correct, and both are the point of the exercise:

```

1. include the NULLS, or different trees serialise the same way
   preorder of 1→2 (left child) is "1,2"
   preorder of 1→2 (right child) is "1,2"    ← same string, different trees
   with nulls: "1,2,#,#,#" vs "1,#,2,#,#"    ← now distinct

2. put a DELIMITER before every value, or numbers merge
   tree containing 12 serialises to "...12,..."
   tree containing 2 serialises to "...2,..."
   "2" is a substring of "12" → a FALSE MATCH
   fix: use ",12," and ",2," – the commas make the boundary explicit

```

Both of those are silent wrong answers, not crashes, which is why they are worth knowing.

The honest complexity: `str.find` in CPython is not a naive scan, but the guaranteed-linear version needs a proper string-matching algorithm. Say $O(n + m)$ with KMP, and note that the built-in is usually fine in practice.

The shape all three share

```

def compare(a, b):
    if a is None and b is None: return True
    if a is None or b is None:  return False
    return a.val == b.val and compare(...) and compare(...)

```

Write those three lines first, every time. Then decide what goes in the two recursive calls: straight for `is_same`, crossed for `is_mirror`. That is the whole family.

4 The picture

Same against mirror, which is the distinction the day turns on.

IS_SAME(a, b) compares:

```

      a      b
     /\    /\
    L  R    L' R'

L with L'   (left with left)
R with R'   (right with right)

```

```

  ||
  ||
  ||
  ||

```

parallel

IS_MIRROR(a, b) compares:

```

      a      b
     /\    /\
    L  R    L' R'

L with R'   (left with RIGHT)
R with L'   (right with LEFT)

```

```

  /\
  /\
  /\
  /\

```

crossed

Kamala's two methods, drawn:

PILLOW COVERS (same)

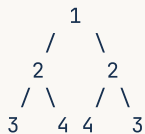
piece 1: A B C D
 piece 2: A B C D
 ↓ ↓ ↓ ↓
 same order, both

TABLE RUNNER BORDER (mirror)

left half: A B C | C B A :right half
 → → | ← ←
 thumbs walk TOWARDS each other
 outermost against outermost

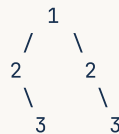
A tree that is symmetric, and one that is not but has identical halves:

SYMMETRIC



left half: 2(3,4)
 right half: 2(4,3)
 → mirrored ✓
 is_symmetric → True

IDENTICAL HALVES, NOT SYMMETRIC



left half: 2(_,3)
 right half: 2(_,3)
 → identical, but a mirror would need
 the right half to be 2(3,_)
 is_same(left, right) → True ← the trap
 is_symmetric → False

Why “subtree” is strict:

BIG



SMALL



2

is_subtree?

NO
 the 1 in BIG has a right child;
 the 1 in SMALL does not.
 A subtree is a node AND EVERYTHING
 below it – nothing extra allowed.

YES
 node 2 and everything below it
 (which is nothing) matches.

The serialisation traps, both of them:

TRAP 1 – no null markers

```
tree A:  1          tree B:  1
        /            \
        2              2
```

preorder without nulls: "1,2" and "1,2" ← identical strings!
preorder with nulls: "1,2,#,#,#" vs "1,#,2,#,#" ← distinct ✓

TRAP 2 – no delimiters

```
big contains ... 12 ... → serialises with "12"
small is      2         → serialises with "2"
"2" in "12" → TRUE     ← a match that does not exist

fix: emit ",12," and ",2," → ",2," is not in ",12," ✓
```

5 The code, built step by step

Step 1 – write the three base cases before anything else

```
if a is None and b is None:
    return True
if a is None or b is None:
    return False
```

Two lines, always, in that order. The `and` case first, then the `or` case. After those two lines both `a` and `b` are guaranteed non-`None`, so every line below can dereference freely.

Say it out loud: *“both empty is a match; one empty is a mismatch of shape; after that both exist.”*

Step 2 – compare the value, then recurse

```
return (a.val == b.val
        and is_same(a.left, b.left)
        and is_same(a.right, b.right))
```

The `and` short-circuits, so a value mismatch stops immediately and neither subtree is walked. That is not an optimisation you add — it is what `and` already does, and it is why this is fast on the common case.

Step 3 — symmetry is the same function with the arguments crossed

```
return (a.val == b.val
        and is_mirror(a.left, b.right)
        and is_mirror(a.right, b.left))
```

Write `is_same` first, then copy it and cross the arguments. Do not try to write `is_mirror` from scratch; the whole point is that it is one change.

And the entry point:

```
def is_symmetric(root):
    return root is None or is_mirror(root.left, root.right)
```

An empty tree is symmetric. A single node is symmetric, because `is_mirror(None, None)` is `True`.

Step 4 — subtree is `is_same` called at every node

```
return (is_same(big, small)
        or is_subtree(big.left, small)
        or is_subtree(big.right, small))
```

Note the base cases here are **not** the same as `is_same`'s:

```
if small is None:
    return True                # an empty tree is a subtree of anything
if big is None:
    return False               # ran out of big tree, small is not empty
```

The asymmetry is deliberate, and getting it backwards is trap 5 below.

Step 5 — the serialisation version, with both fixes

```
return f",{node.val}," if leaf else ...
```

Emit a delimiter around every value, and a marker for every `None`. Then:

```
return serialise(small) in serialise(big)
```

Say both fixes out loud as you write them: “nulls, so different shapes give different strings; delimiters, so 2 does not match inside 12.”

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

def is_same(a: TreeNode | None, b: TreeNode | None) → bool:
    """LeetCode 100. Two trees, walked in step.

    THREE base cases, and the middle one is what everyone forgets:
        both None      → True    (ran out together)
        exactly one    → False   (different shape)
        neither        → compare values, then both pairs of children

    Omitting the `or` line does not give a wrong answer – it gives
    AttributeError: 'NoneType' object has no attribute 'val'.

    Time O(min(n, m)) – `and` short-circuits at the first difference.
    Space O(min(height))
    """
    if a is None and b is None:
        return True
    if a is None or b is None:
        return False
    return (a.val == b.val
            and is_same(a.left, b.left)
            and is_same(a.right, b.right))

def is_mirror(a: TreeNode | None, b: TreeNode | None) → bool:
    """is_same with the arguments CROSSED. That is the only difference.

    left against RIGHT, right against LEFT – the outermost of one side
    matched with the outermost of the other.
    """
    if a is None and b is None:
        return True
    if a is None or b is None:
        return False
    return (a.val == b.val
            and is_mirror(a.left, b.right)    # crossed
            and is_mirror(a.right, b.left))    # crossed

def is_symmetric(root: TreeNode | None) → bool:
    """LeetCode 101. An empty tree is symmetric; so is a single node.

    NOT is_same(root.left, root.right) – that tests whether the halves are
    IDENTICAL, which is a translation, not a reflection.
    """
    return root is None or is_mirror(root.left, root.right)
```

```

def is_symmetric_iterative(root: TreeNode | None) → bool:
    """The same, with a queue holding PAIRS. Useful when the tree may be
    deep enough to overflow the stack."""
    if root is None:
        return True
    queue = deque([(root.left, root.right)])
    while queue:
        a, b = queue.popleft()
        if a is None and b is None:
            continue
        if a is None or b is None:
            return False
        if a.val != b.val:
            return False
        queue.append((a.left, b.right))    # crossed
        queue.append((a.right, b.left))    # crossed
    return True

def is_subtree(big: TreeNode | None, small: TreeNode | None) → bool:
    """LeetCode 572. is_same, tried at every node of the big tree.

    NOTE the base cases differ from is_same's, and deliberately:
        small is None → True    (an empty tree is a subtree of anything)
        big is None   → False   (ran out of tree, and small is not empty)

    "Subtree" means a node AND EVERYTHING below it. A match that continues
    into extra nodes is not a match.

    Time O(n × m) worst case. See is_subtree_by_string for O(n + m).
    """
    if small is None:
        return True
    if big is None:
        return False
    return (is_same(big, small)
            or is_subtree(big.left, small)
            or is_subtree(big.right, small))

def serialise(node: TreeNode | None) → str:
    """Preorder with TWO essential details.

    1. NULL MARKERS, or different trees serialise identically:
        1 with a left child → "1,2"    without markers
        1 with a right child → "1,2"    ... the same string.
    2. DELIMITERS around every value, or numbers merge:
        a tree containing 12 gives "...12..."
        searching for a tree containing 2 finds "2" inside "12"
        → a false match. ",2," is not inside ",12,".
    """
    if node is None:
        return ",#"
    return f",{node.val}" + serialise(node.left) + serialise(node.right)

def is_subtree_by_string(big: TreeNode | None, small: TreeNode | None) → bool:
    """O(n + m) with a linear substring search, instead of O(n × m).

```

```

CPython's `in` is not a naive scan, but the guaranteed-linear version
needs KMP. Say "O(n + m) with KMP" rather than claiming it for `in`.
"""
if small is None:
    return True
return serialise(small) in serialise(big)

def invert(node: TreeNode | None) → TreeNode | None:
    """LeetCode 226. The mirror OF a tree, rather than a test.

    is_symmetric(t) is exactly is_same(t, invert(copy_of(t))) – but doing it
    that way allocates a whole second tree, where is_mirror allocates
    nothing.
    """
    if node is None:
        return None
    node.left, node.right = invert(node.right), invert(node.left)
    return node

def count_matching_subtrees(big: TreeNode | None, small: TreeNode | None) → int:
    """The follow-up: not "is it there" but "how many times".
    Cannot short-circuit, so it is the full O(n × m)."""
    if big is None:
        return 0
    return (int(is_same(big, small))
            + count_matching_subtrees(big.left, small)
            + count_matching_subtrees(big.right, small))

def is_same_iterative(a: TreeNode | None, b: TreeNode | None) → bool:
    """Two trees, one stack of pairs. No recursion depth risk."""
    stack = [(a, b)]
    while stack:
        x, y = stack.pop()
        if x is None and y is None:
            continue
        if x is None or y is None:
            return False
        if x.val ≠ y.val:
            return False
        stack.append((x.left, y.left))
        stack.append((x.right, y.right))
    return True

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:

```



```

        node.right = TreeNode(values[i]); queue.append(node.right)
    i += 1
    return root

if __name__ == "__main__":
    a = from_list([1, 2, 3])
    b = from_list([1, 2, 3])
    c = from_list([1, 2, None, 3])

    print(is_same(a, b))                # True
    print(is_same(a, c))                # False
    print(is_same(None, None), is_same(a, None)) # True False
    print(is_same_iterative(a, b))      # True

    print(is_symmetric(from_list([1, 2, 2, 3, 4, 4, 3]))) # True
    print(is_symmetric(from_list([1, 2, 2, None, 3, None, 3]))) # False
    print(is_symmetric(None), is_symmetric(TreeNode(1))) # True True
    print(is_symmetric_iterative(from_list([1, 2, 2, 3, 4, 4, 3]))) # True

    # the trap: identical halves are NOT symmetry
    trap = from_list([1, 2, 2, None, 3, None, 3])
    print(is_same(trap.left, trap.right)) # True ← the wrong test
    print(is_symmetric(trap))            # False ← the right answer

    big = from_list([3, 4, 5, 1, 2])
    small = from_list([4, 1, 2])
    print(is_subtree(big, small))        # True
    print(is_subtree_by_string(big, small)) # True

    # "subtree" is strict: nothing extra below
    print(is_subtree(from_list([1, 2, 3]), from_list([1, 2]))) # False

    # the delimiter trap, demonstrated
    print(serialise(TreeNode(12)))        # ,12,#,#
    print(serialise(TreeNode(2)))         # ,2,#,#
    print(serialise(TreeNode(2)) in serialise(TreeNode(12))) # False – correct
    # without the leading comma it would be "2,#,#" in "12,#,#" → True, wrong

    print(count_matching_subtrees(from_list([1, 1, 1]), TreeNode(1))) # 2

```

6 What it costs

Same tree

```

time    O(min(n, m)) – `and` stops at the first mismatch
space   O(min(h1, h2)) – the recursion stack

```

min, **not** **max**, and it is worth saying: the walk stops as soon as one tree runs out or a value differs, so it can never visit more nodes than the smaller tree has.

In practice, the common case is much better than the bound:

```
two random trees of 1,000 nodes: usually a mismatch within the first few nodes
two identical trees of 1,000:    all 1,000 visited
```

Symmetric tree

```
time    O(n)    - every node visited once, in pairs
space   O(h)
```

Each node is compared exactly once, against its mirror partner.

Subtree — the interesting one

```
naive:      O(n × m)
n = nodes in the big tree, m = nodes in the small tree
is_same is called at each of n nodes and can walk m nodes
```

```
n = 1,000, m = 100    → up to 100,000 comparisons
n = 10,000, m = 1,000 → up to 10,000,000
```

In practice it is far better, because most `is_same` calls fail on the root value immediately — Kamala moving on after one stitch. The worst case needs an adversarial input:

```
big:    a chain of 10,000 nodes all valued 1
small:  a chain of 100 nodes all valued 1, then a 2 at the end
→ every position matches for 100 steps before failing
→ ~1,000,000 comparisons, and this is a real LeetCode test case
```

The string version:

```
serialise both      O(n + m)
substring search    O(n + m) with KMP
-----
total               O(n + m) time, O(n + m) space
```

The trade is explicit: linear time, at the cost of linear extra memory for the two strings. On a million-node tree the serialisation is several megabytes of string, where the naive version uses only the stack.

Why serialisation needs both fixes, in numbers

```
without null markers:
  number of distinct trees with n nodes:  the Catalan number,  $\sim 4^n / n^{1.5}$ 
  number of distinct null-free preorders:  $n!$  / (repeats)
  → many trees collide. The 2-node example collides immediately.

without delimiters:
  any small tree whose values are suffixes of the big tree's values
  can produce a false match: 2 inside 12, 3 inside 13 or 23, and so on
  → on values up to  $10^4$ , collisions are common rather than exotic
```

Recursion depth

```
all three problems:  O(height) frames
degenerate trees:   RecursionError past ~1,000
```

The iterative versions with a stack or queue of **pairs** are in the code above, and they are the answer if the constraint allows a chain.

7 The traps

Trap 1 — forgetting the “exactly one is `None`” case

```
def is_same(a, b):
    if a is None and b is None:
        return True
    return a.val == b.val and ...      # a might still be None
```

```
AttributeError: 'NoneType' object has no attribute 'val'
```

The most common crash in this topic. Write both base-case lines together as a unit, before anything else.

Trap 2 — writing symmetry as `is_same(root.left, root.right)`

```
return is_same(root.left, root.right)      # WRONG
```

```
is_symmetric(from_list([1, 2, 2, None, 3, None, 3]))
this version → True
correct      → False
```

Identical halves are a **translation**; symmetry is a **reflection**. They coincide only when each half is itself symmetric, which is why the wrong version passes on so many test cases.

Trap 3 — crossing only one of the two recursive calls

```
return (a.val == b.val
        and is_mirror(a.left, b.right)
        and is_mirror(a.right, b.right))    # should be b.left
```

Half-mirrored, and it produces `True` for some asymmetric trees and `False` for some symmetric ones. **Write the two crossed calls as a pair and read them back:** left-right, right-left.

Trap 4 — comparing values without comparing shape

```
return inorder(a) == inorder(b)            # WRONG
```

Two different trees can share a traversal:

```
      1           1
       \         /
        2       2

preorder without nulls: [1, 2] and [1, 2] → "equal"
```

Same list, different trees. **One traversal never determines a tree** — the same fact as [day 100](#), and it is why the serialisation trick needs null markers.

Trap 5 — the subtree base cases backwards

```
if big is None:
    return True                # WRONG
if small is None:
    return False               # WRONG
```

An empty small tree is a subtree of anything; an empty big tree contains nothing but the empty tree. Getting these round the wrong way makes `is_subtree` return `True` for everything, which passes the first test case and nothing else.

Trap 6 — thinking “subtree” means “appears somewhere”

```
big:      1          small: 1
         / \         /
        2   3        2

many people expect True; the answer is FALSE
```

A subtree is a node **and everything below it**. The 1 in the big tree has a right child that the small tree does not, so it is not the same subtree. If the question genuinely means “does this pattern appear partway down”, that is a different and harder problem — ask.

Trap 7 — serialising without null markers

```
def serialise(node):
    if node is None:
        return ""
    return f",{node.val}" + serialise(node.left) + serialise(node.right)
```

```
serialise(1 with left child 2) → ",1,2"
serialise(1 with right child 2) → ",1,2" ← identical
```

Different trees, same string, silent wrong answer.

Trap 8 — serialising without delimiters

```
return f"{node.val}" + ...
```

```
big contains 12, small is 2
"2#,#" in "12#,#" → True ← a match that does not exist
```

A leading comma before every value is the fix, and it costs one character. This is the classic “my solution passes 130 of 182 test cases” bug.

8 In the interview

How it gets asked

- The warm-up: “Are these two trees identical?” LeetCode 100.
- The twist: “Is this tree symmetric about its centre?” LeetCode 101.
- The composition: “Is this tree a subtree of that one?” LeetCode 572.

- The optimisation: “Your subtree solution is $O(n \times m)$. Can you do better?”
- The related: “Invert a binary tree.” LeetCode 226 — famously.

What to say out loud, in the first ninety seconds

1. **Name the shape.** “This is a recursion over **two** trees at once — the function takes a node from each and asks whether the subtrees below them match.”
2. **State the three base cases before writing them.** “Both `None` is a match. Exactly one `None` is a mismatch, because the shapes differ. After those two lines, both are guaranteed to exist and I can dereference freely.”
3. **Say what happens if you skip the middle one.** “Leaving out the ‘exactly one’ case does not give a wrong answer, it gives an `AttributeError` on the value comparison.”
4. **For symmetry, say the difference explicitly.** “Symmetry is the same function with the arguments crossed — left against right and right against left. It is *not* checking that the two halves are identical; identical halves are a translation and symmetry is a reflection.”
5. **For subtree, define the word.** “A subtree means a node **and everything below it**, so a match that continues into extra nodes is not a match.”
6. **Give the complexity and flag the improvement.** “Same tree is $O(\min(n, m))$ because the `and` short-circuits. Subtree is $O(n \times m)$ naively, and there is a linear alternative via serialisation.”

The follow-ups

“What are the base cases, and why three?” “Because two nodes can be `None` in three combinations, and they mean different things. **Both** `None` — two empty subtrees, which match, so `True`. **Exactly one** `None` — the shapes differ, so `False`, and this is the one people leave out. **Neither** `None` — now I can safely compare values and recurse. The reason the middle one matters so much is that omitting it does not produce a wrong answer; it produces an `AttributeError` when the next line reads `.val` on a `None`. I write both base-case lines as a unit before anything else, so that everything below them is guaranteed safe.”

“Is symmetry just comparing the two halves?” “No, and this is the trap. Comparing the halves with `is_same` asks whether they are **identical**, which is a translation. Symmetry is a **reflection**: the outermost node on the left must match the outermost node on the right, so the recursion crosses the arguments — my left child against your right child, my right against your left. The counter-example is a tree where both halves are `2` with a right child `3`: the halves are identical, so the naive test says symmetric,

but a mirror would need the 3 on the outside of one side and the inside of the other. The two tests agree only when each half is itself symmetric, which is why the wrong version passes a lot of test cases.”

“Your subtree solution is $O(n \times m)$. Can you do better?” “Yes — serialise both trees to strings and ask whether one contains the other, which is $O(n + m)$ with a linear string search. Two details make it correct and both are silent if you get them wrong. **Null markers:** without them, a node with only a left child and a node with only a right child serialise to the same string, so different trees compare equal. **Delimiters around every value:** without them, a tree containing the value 2 matches inside a tree containing 12, because “2” is a substring of “12” — so I emit a comma before every value and “,2,” is not inside “,12,”. The trade is explicit: linear time for linear extra memory, since I am now holding two strings proportional to the trees. And I would say $O(n + m)$ with KMP rather than claiming it for Python’s `in`, which is fast but not guaranteed linear.”

“What is the complexity of the naive version really?” “ $O(n \times m)$ as a bound, and much better in practice, because almost every `is_same` call fails on the root value comparison and returns after one step. The worst case needs an adversarial input — a big tree that is a chain of ten thousand identical values and a small tree that is a hundred of the same value followed by a difference. Then every position matches for a hundred steps before failing, and that is about a million comparisons. That input exists in the LeetCode test set, which is why the string version is worth knowing.”

“Could you compare the trees by their traversals instead?” “Only if the traversal records the nulls. A single traversal without nulls does not determine a tree — a node with one left child and a node with one right child both give the preorder `[1, 2]`. With null markers, a preorder does determine the tree uniquely, which is exactly what makes the serialisation trick work. The other way is two traversals, one of which must be inorder, which is the reconstruction problem rather than this one.”

“How would you handle a very deep tree?” “Both problems recurse to the height, so a ten-thousand-node chain exceeds Python’s default limit. The iterative version is straightforward here because the state is just a pair of nodes: a stack — or a queue, which is what I would use for the symmetric version — holding **pairs**, popping a pair, applying the same three base cases, and pushing the two child pairs. For symmetry the pushes are crossed exactly as in the recursion. It is barely longer than the recursive version and removes the depth risk entirely.”

A model answer

Asked: *are these two trees identical? And is this tree a subtree of that one?*

“Both are the same idea: a recursion that walks **two trees at once**, taking a node from each and asking whether what hangs below them matches.

The whole difficulty is in the base cases, and there are **three**, because two nodes can be `None` in three different combinations. **Both** `None` means two empty subtrees, which match. **Exactly one** `None` means the shapes differ, so they cannot match — and this is the case people leave out. Leaving it out does not give a wrong answer; it gives an `AttributeError`, because the next line reads `.val` on a `None`. **Neither** `None` means I can safely compare the values and recurse on both pairs of children. I write the first two lines as a unit before anything else, so everything below is guaranteed safe.

The comparison itself is `a.val == b.val` and `same(left, left)` and `same(right, right)`, and the `and` short-circuits, so a mismatch stops the walk immediately. That is why the complexity is $O(\min(n, m))$ rather than $O(n)$ — it can never visit more nodes than the smaller tree has.

If you then ask about symmetry, it is the same function with **the arguments crossed**: my left against your right, my right against your left. I would flag the trap explicitly, because it is the most common wrong answer here — checking that the two halves are *identical* is not the same thing. Identical halves are a translation, symmetry is a reflection, and they only coincide when each half is itself symmetric.

For subtree, it is `is_same` tried at every node of the bigger tree. Two things to pin down. The base cases are **not** the same as `is_same`’s: an empty small tree is a subtree of anything, and an empty big tree contains nothing. And ‘subtree’ is stricter than people expect — it means a node **and everything below it**, so if the small tree is a `1` with a left child and the big tree’s `1` also has a right child, that is not a match. Nothing extra is allowed to hang off the bottom.

That is $O(n \times m)$ in the worst case, and much better in practice because most attempts fail on the first value. If you want a guaranteed improvement, I would serialise both trees and ask whether one string contains the other — $O(n + m)$ with a linear string search. Two details are essential and both fail silently: **null markers**, or a left-only child and a right-only child produce the same string; and **a delimiter before every value**, or a tree containing 2 matches inside one containing 12. The cost is holding two strings proportional to the trees, where the naive version uses only the stack.”

9 Recall card

- All three problems are one recursion over TWO trees, and the base cases are the whole difficulty. THREE of them: both `None` → `True` · exactly one `None` → `False` · neither → compare values then both child pairs. Omitting the middle line gives `AttributeError: 'NoneType' object has no attribute 'val'`, not a wrong answer. Write both base lines as a unit first.
- Symmetry is `is_same` with the arguments CROSSED — left against right, right against left. It is `not is_same(root.left, root.right)`: identical halves are a *translation*, symmetry is a *reflection*, and the wrong version returns `True` on `[1,2,2,null,3,null,3]`.
- “Subtree” means a node AND EVERYTHING below it — nothing extra may hang off the bottom, so a `1` with a left child is not a subtree of a `1` with both children. And its base cases are the opposite way round: **empty small** → `True`, **empty big** → `False`.
- Same tree is `O(min(n, m))` because `and` short-circuits. Subtree is `O(n × m)` — bad in the worst case (a chain of identical values), fine in practice because most attempts fail on the first value.
- The `O(n + m)` alternative: serialise both trees and search for one string inside the other — and it needs BOTH fixes, each silent if missed. Null markers, or a left-only and a right-only child give the same string; a **delimiter before every value**, or `"2"` matches inside `"12"`. Say `O(n + m)` with KMP, and note the cost is `O(n + m)` extra memory.

1 What this is, and why they ask it

A **content delivery network** is a set of caches placed in hundreds of cities around the world, in front of your servers. A user's request goes to the nearest one, and if it has the content, that is the end of the journey — your servers never hear about it.

Three sentences. It exists because of one number from [day 097](#): **a round trip from Mumbai to Virginia is about 200 milliseconds of pure distance**, and no amount of faster code removes it. It works because most of the bytes a typical site sends are the same for everybody — images, video, scripts, stylesheets — so one copy near the user serves thousands of people. And its limits are exactly the mirror of that: **anything personal, anything that changes on every request, and anything that writes** has to come from your servers.

They ask it because it is usually the single biggest performance win available, and because candidates who have only read about caching put Redis in front of the database and stop there. For a media-heavy product the CDN removes **more than ninety percent of the bytes** before Redis is even consulted. The follow-up — *“what would it not help with?”* — is the one that separates people who have used one from people who have heard of one.

2 The story

The paper had been printed in Chennai since 1953, and for Madurai that meant the lorry.

It left the press at about half past one in the morning and if everything went well it was in Madurai by half past six. The bundles were dropped at the agent's, the boys sorted them, and the first houses got their paper around eight. In Chennai people had been reading it since six.

There were other problems besides the time. If it rained badly on the Trichy road the paper did not arrive at all. Twice a year the lorry broke down and an entire district got nothing, and the phone at the office rang all morning.

In 1994 they set up a press in Madurai, and one in Coimbatore the year after.

The thing that people outside the business assumed was that the Madurai press wrote its own paper. It did not. The paper was still written and laid out in Chennai. What went down the wire at midnight was the **pages** — the finished layout — and the Madurai press printed from that.

So the writing happened once, in one place, and the printing happened in three places, near the people reading it. Madurai got its paper at half past five in the morning instead of eight, and a lorry breaking down stopped mattering.

Two things did not get solved by it, and the editor was clear about both.

The first was that the local pages — the Madurai city section, with the local advertisements and the school results — could not be prepared in Chennai. Those were genuinely different for each city, so each press had to have somebody doing them. You cannot print the same thing everywhere if the thing is not the same everywhere.

The second was worse, and it happened in 1997. A court order came at four in the morning saying a particular story had to be pulled. Chennai stopped its press in about ten minutes. Getting Coimbatore and Madurai to stop took nearly an hour of phone calls, and by then a few thousand copies were already on bicycles.

After that they had a rule about anything legally sensitive: hold it for one edition. Not because the presses were slow, but because **once a thing has been sent out to three cities, calling it back is a different and much harder job than never sending it.**

3 The idea in plain English

The regional presses are a CDN, and the editor's two problems are exactly the two limits of one.

- Chennai is the **origin** — your servers, the source of truth.
- The Madurai and Coimbatore presses are **edge locations**, also called **PoPs** (points of presence).
- Sending the finished pages down the wire is **origin pull**: the edge fetches from the origin on demand.
- The paper arriving at 5:30 instead of 8 is the **latency** saved.
- The lorry breaking down no longer mattering is the **availability** and **origin offload** benefit.
- The local city pages are **personalised content**, which a shared cache cannot serve.

- The court order at four in the morning is a **purge**, and its difficulty is the reason for **cache-busting URLs**.

Why it exists: distance is not an engineering problem

```
Mumbai to Virginia          ~13,000 km
light in fibre               ~200,000 km/s (about 2/3 of light in vacuum)
one way                      ~65 ms
round trip                   ~130 ms
real routes are not straight, and there are hops: ~180-220 ms in practice
```

No server optimisation touches that number. A page that makes four sequential requests pays it four times — nearly a second before any computation. The only fix is to be closer, and being closer means having a copy somewhere else.

```
same city / same country edge ~5-20 ms
same continent                ~30-50 ms
across the world              ~150-220 ms
```

What actually goes over the wire

The reason a CDN is such a large win is the **byte** split, not the request split:

```
a typical page load
HTML (personal, changes)      ~30 KB      2%
JavaScript, CSS (identical for all) ~500 KB    25%
images (identical for all)    ~1,400 KB  70%
API responses (personal)      ~50 KB      3%
-----
total                          ~2,000 KB

CDN-servable                   ~1,900 KB  95% of the bytes
must come from origin          ~100 KB      5%
```

Ninety-five percent of the bytes and typically seventy to eighty percent of the requests. That is why this layer is the biggest win in the [four-layer picture](#), and why “put Redis in front of the database” is answering a smaller question.

How content gets to the edge

Pull (origin pull) — the default and what to describe. The first user in a city requests something, the edge does not have it, so it fetches from the origin, keeps a copy, and serves everybody after that.

- + nothing to manage; content appears where it is needed
- + you never push things nobody wants
- the FIRST user in each city pays the full origin latency (a "cold miss")
- with ~300 locations, up to 300 cold misses per object

Push — you upload content to the CDN ahead of time. Used for large predictable objects: a game patch, a film release, a software update where the first-user penalty would be unacceptable.

Tiered caching / origin shield — the fix for the cold-miss problem. Edges do not go to the origin directly; they go to a regional parent cache, and only that parent talks to the origin.

```
without a shield: 300 edges × 1 cold miss each = 300 origin requests per object
with a shield:   300 edges → ~10 regional parents → 1 origin request
```

This is the detail that shows you have run one. Without it, a popular new object causes a 300-fold burst at the origin — a stampede at a different scale.

How the edge knows when to let go

The rules travel with the response, as HTTP headers.

```
Cache-Control: public, max-age=31536000, immutable
    public      any shared cache may store it
    max-age     seconds the BROWSER keeps it
    immutable   never revalidate; the content will never change at this URL

Cache-Control: private, no-store
    private     only the user's browser, never a shared cache
    no-store    do not keep it at all

Cache-Control: public, s-maxage=300, max-age=0
    s-maxage    the CDN keeps it 5 minutes; the browser keeps it 0
    → the CDN absorbs the load, the user always revalidates
```

s-maxage is the one worth knowing by name — it lets you cache hard at the edge while keeping the browser honest, which is exactly what you want for HTML that changes.

And revalidation, so that an unchanged object costs almost nothing:

```
ETag: "a3f9c2"           a fingerprint of the content
client: If-None-Match: "a3f9c2"
server: 304 Not Modified  ~200 bytes instead of the whole file
```

The two things it cannot do — and the answer to the follow-up

One: personalised content. If the response differs per user, a shared cache has nothing to share. The edge would need one copy per user, which is not a cache — it is storage with extra steps.

The standard fix is **to split the page**: serve a cacheable shell from the edge and fetch the personal parts separately. That is why the modern shape is a static HTML shell plus an API call, rather than a server-rendered page with the user's name baked in.

Two: writes. A `POST` cannot be cached, by definition — it has an effect. Every write travels the full distance to the origin.

That leads to the asymmetry worth stating: **a CDN makes a read-heavy product feel local and does nothing for a write-heavy one.** A photo-sharing app is transformed; a collaborative editor is not.

There is a real modern exception, and mentioning it is a good signal: **edge compute** — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin. It does not change the physics for writes, which still need the origin, but it moves the “assemble the personal bits” step much closer.

Cache keys, and the mistake everyone makes

The edge keys on the URL, **including the query string**. So:

<code>/image.jpg</code>	one cached object
<code>/image.jpg?utm_source=twitter</code>	a DIFFERENT cached object, same bytes
<code>/image.jpg?utm_source=facebook</code>	a third
<code>/image.jpg?t=1717171717</code>	a fourth, and it will never be requested again

Marketing parameters silently destroy hit rates, because every share link creates a new cache key for identical content. Every CDN lets you strip or allow-list query parameters, and doing so is often the single largest hit-rate improvement available on a real site.

The same applies to `Vary: Vary: Accept-Encoding` is normal and doubles the entries. `Vary: User-Agent` multiplies them by thousands and effectively disables caching.

Invalidation: purge, or rename

From [yesterday](#): explicit invalidation is hard. At the edge it is harder, because there are hundreds of copies.

```
PURGE    an API call telling every location to drop a URL
          takes seconds to a minute to propagate
          rate-limited by every provider
          → the court order at four in the morning
```

The standard practice is not to purge at all. Instead, put a fingerprint in the file name:

```
app.js           → app.a3f9c2.js
style.css        → style.7b21e4.css
```

A new build produces a new name, which is a new cache key, which needs no invalidation — and lets you set `max-age` to a year. **A new name is a new object. Nothing has to be recalled.** That is the versioned-key idea from yesterday, applied at the edge, and it is why every modern build tool hashes filenames.

4 The picture

Where it sits, and what never reaches you.

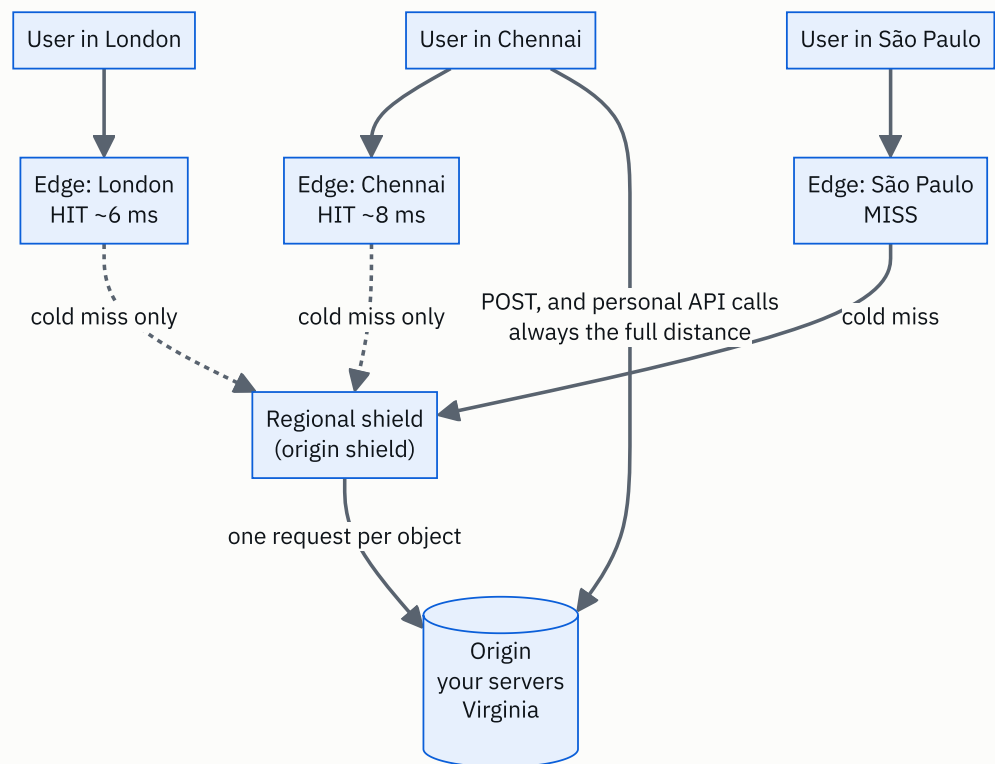
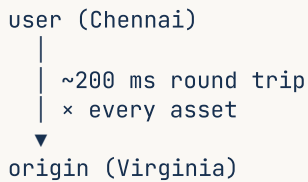


FIGURE 103.1

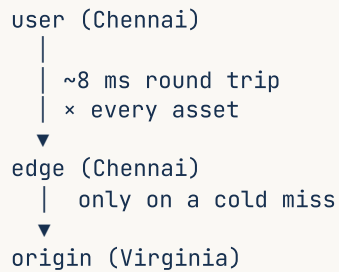
What to notice: **the dotted arrows are rare and the solid ones are common.** And the bottom arrow — writes and personal data — bypasses the whole thing and pays the full 200 ms, every time.

The latency, drawn:

WITHOUT a CDN



WITH a CDN



a page with 4 sequential round trips:

without: $4 \times 200 \text{ ms} = 800 \text{ ms}$ before anything renders

with: $4 \times 8 \text{ ms} = 32 \text{ ms}$

The byte split, which is the real argument:

a 2 MB page load



origin bandwidth without a CDN: $2,000 \text{ KB} \times \text{every user}$

origin bandwidth with a CDN: $100 \text{ KB} \times \text{every user}$ a 20× reduction

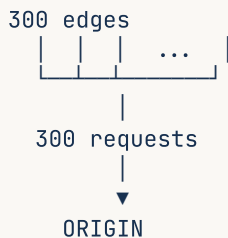
at 100,000 page loads/hour:

without: 200 GB/hour of egress from your servers

with: 10 GB/hour

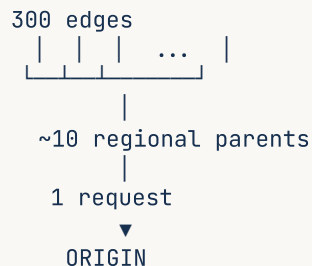
Cold misses, with and without a shield:

WITHOUT AN ORIGIN SHIELD



a newly published object causes
a 300× burst at the origin

WITH AN ORIGIN SHIELD



The query-string problem:

the same 400 KB image, shared four ways:

/photo.jpg	key 1	← the only one anybody needed
/photo.jpg?utm_source=twitter	key 2	
/photo.jpg?utm_source=whatsapp	key 3	
/photo.jpg?fbclid=IwAR2x...	key 4	← unique per SHARE. Never hits.

hit rate on this object: ~25% instead of ~99%
origin egress: 4× what it should be

fix: configure the CDN to ignore utm_*, fbclid and friends in the cache key

5 How it actually works

One request, end to end

1. DNS lookup for cdn.example.com
the CDN's DNS returns an address near the user,
or ANYCAST is used: the same IP is announced from every location
and the internet routes to the nearest one
2. TCP + TLS handshake to the EDGE
this is the second big win: the handshake is ~8 ms away
instead of ~200 ms, and a handshake is 2-3 round trips
3. the edge looks up the cache key (URL + configured query params + Vary)
HIT → serve, done. Origin never involved.
MISS → go to the regional shield, then the origin
4. on a miss, the edge stores the response according to Cache-Control
and serves it
5. subsequent requests in that city: hits

Step 2 is worth calling out, because people count only the response. A TLS handshake is two or three round trips, so terminating it at the edge saves 400 to 600 milliseconds on a cold connection from India to Virginia — often more than the content transfer itself.

What to put behind it

```
always:      images, video, fonts, CSS, JS, downloads, anything static
usually:     API responses that are the same for everyone
              (a product catalogue, a public feed, exchange rates)
with care:   HTML – use s-maxage so the edge caches and the browser does not
never:       anything with a user's data in it
              anything with Set-Cookie
              POST, PUT, DELETE
```

The **Set-Cookie** rule is worth its own line. Caching a response that contains a **Set-Cookie** header serves one user's session cookie to everybody who requests that URL afterwards. It is the most serious mistake available at this layer, and every CDN has a safeguard against it — which you can accidentally override.

Video, which is a different problem

Video is not one file. It is **segmented**: split into two-to-ten-second chunks, at several bitrates, with a manifest listing them (HLS or DASH).

```
movie.m3u8          the manifest (small, changes rarely)
720p/segment_0001.ts ~2 MB
720p/segment_0002.ts
1080p/segment_0001.ts ~5 MB
...
```

Two consequences: each segment is a normal cacheable object, so a CDN serves video with no special machinery; and **the player switches bitrate between segments**, which is why streaming adapts to a bad connection rather than stalling. This is why video was the original driver for CDNs and remains most of their traffic.

Real products

- **Cloudflare** — ~300+ cities, anycast, and a free tier, which is why it appears in so many architectures. **Workers** is its edge-compute product.
- **Akamai** — the oldest and the largest by locations, historically ~4,000 points of presence, deep inside carrier networks.
- **CloudFront** (AWS), **Fastly** (known for very fast purges — seconds rather than minutes, which changes what you can cache), **Google Cloud CDN**.
- **Netflix Open Connect** is the interesting one: Netflix ships its own hardware to internet providers and pre-loads the catalogue overnight, so the film you watch is often served from a box inside your own ISP. That is push distribution taken to its conclusion, and it is why Netflix traffic barely touches the public internet backbone.

6 The numbers

Latency

origin round trip, India to US-East	~200 ms
edge round trip, same city	~8 ms
saving per round trip	~192 ms
a page needing 4 sequential round trips	
without a CDN	~800 ms
with	~32 ms
plus the TLS handshake (2-3 round trips)	
without	~400-600 ms
with	~16-24 ms

The handshake saving is often larger than the content saving, and it is the part candidates miss.

Offload, which is the number to quote

100,000 page loads/hour, 2 MB each		
	origin bytes/hour	origin requests/hour
no CDN	200 GB	~3,000,000
CDN, 95% byte offload	10 GB	~700,000
		(personal API calls only)

origin bandwidth at ₹7/GB	
without:	$200 \text{ GB} \times 24 \times 30 \times ₹7 = ₹10,08,000 \text{ / month}$
with:	$10 \text{ GB} \times 24 \times 30 \times ₹7 = ₹50,400 \text{ / month}$
CDN cost:	$190 \text{ GB} \times 24 \times 30 \times ₹1.5 \approx ₹2,05,000 \text{ / month}$

saving	$\approx ₹7,50,000 \text{ / month}$

A CDN is usually cheaper than the egress it replaces, because CDN bandwidth is bought in bulk and priced well below cloud egress. That is a genuine argument, not a hand-wave — cloud providers charge a premium for data leaving their network.

Hit rate

well-configured static assets	95 - 99%
images with fingerprinted names	99%+
HTML with s-maxage=60	60 - 90%
anything with utm_* in the key	25 - 60% ← the query-string problem
personalised API responses	~0%

The gap between 99% and 60% on the same content is usually cache-key configuration, not content.

Cold misses and the shield

```
300 edge locations, one new object
  no shield:      up to 300 origin requests, in a burst
  with a shield:  ~10 regional parents → 1-10 origin requests

a 200 MB game patch, 300 edges, no shield:
  60 GB pulled from the origin in the first minutes
```

Purge, and why renaming wins

```
purge propagation      seconds to ~60 s, provider-dependent
purge rate limits      commonly a few thousand URLs per day on standard plans
fingerprinted rename   0 ms, unlimited, and lets max-age be a year

a deploy changing 400 asset files:
  purge:  400 API calls, rate-limited, and a window where old and new mix
  rename:  0 calls, and old and new coexist safely for anyone mid-session
```

“Old and new coexist safely” is the underrated part. A user who loaded the page before the deploy can still fetch the old JavaScript it referenced, instead of getting a mismatched new file.

Video

```
1080p stream              ~5 Mbit/s
1,000 concurrent viewers   5 Gbit/s
→ ~5 machines' worth of network link, from ONE origin

with a CDN: the origin serves each segment once per region
1,000 viewers of the same live stream → a handful of origin fetches
```

This is why video without a CDN is not a cost problem, it is an impossibility problem — you cannot buy that much network out of one location.

7 The trade-offs

What you give up

Control over invalidation. Content is now in hundreds of places you do not operate. Purges are slow and rate-limited. **This is why the industry answer is renaming rather than purging** — you avoid the problem instead of solving it.

A dependency you cannot debug. When a CDN misbehaves — a bad configuration, a regional outage, a stale object in one city — you are looking at a system you cannot log into. Every large CDN has had an outage that took a visible fraction of the internet down with it.

Cache-key complexity. Query strings, `Vary`, cookies and device detection all multiply the number of stored objects. A misconfiguration does not error; it quietly halves your hit rate.

Cost, at low volume. Below a certain traffic level the CDN's minimum charges exceed the egress saved. It is a win at scale and neutral or negative for a small internal tool.

When it does not help

Write-heavy products. A collaborative editor sends a stream of small writes, all of which must reach the origin. The CDN can serve the application shell and nothing else.

Genuinely personal pages. A banking dashboard is different for every user on every request. Split it: static shell from the edge, data from the origin.

Very low-latency interaction. A CDN adds a hop on a miss. For an API where every response is dynamic, you are adding a small amount of latency to every request in exchange for nothing — although most CDNs still help here through **connection reuse and better routing**, which is a genuine but smaller effect.

Single-region audiences. If all your users and your servers are in one city, the distance saving is near zero. You still get offload and DDoS absorption, and those may be reason enough.

The security angle, which is half the reason people buy one

- **DDoS absorption.** A CDN has vastly more capacity than your origin, and it absorbs volumetric attacks before they reach you. For many companies this, not speed, is the purchase justification.
- **TLS termination and certificate management** at the edge.
- **Hiding the origin.** If nobody knows your servers' addresses, they cannot be attacked directly — provided you actually firewall them to accept only CDN traffic, which people forget.

Where it breaks badly

- **Caching a personalised response.** One user's data served to everybody. Usually caused by a `Cache-Control: public` set where it should have been `private`, or by ignoring cookies in the cache key. **This is a data-breach class of bug, not a**

performance bug.

- **Caching a `Set-Cookie`**. The same, with session cookies.
 - **A too-long `max-age` on something that changes**. A one-year cache on `app.js` without a fingerprint means users run last year's code and there is no way to reach them.
 - **A cold cache during a launch**. The moment you most need the CDN is the moment it has nothing, which is exactly what pre-warming and push distribution are for.
-

8 In the interview

How it gets asked

- The direct one: *"Would a CDN help this system? What would it not help with?"*
- Inside a design: *"How do you serve images to users worldwide?"*
- The invalidation probe: *"You deploy a new version of your JavaScript. How do users get it?"*
- The limits probe: *"Can you cache the HTML?"*
- The numbers probe: *"How much traffic does this actually save you?"*

What to say out loud, in the first ninety seconds

1. **Give the reason in terms of physics.** "A round trip from India to a US data centre is about 200 milliseconds of pure distance, and no code change touches that. A CDN answers it by having a copy near the user."
2. **Say what it serves, by bytes.** "Images, video, CSS and JavaScript are identical for every user and are typically ninety-five percent of a page load's bytes. So the CDN removes almost all of my bandwidth and most of my requests before any other cache is consulted."
3. **Describe pull, and then the shield.** "Origin pull by default — the first user in a city causes a fetch, everybody after that gets a hit. With hundreds of locations that is hundreds of cold misses per object, so I would enable an origin shield: edges go to a regional parent and only the parent talks to my origin."
4. **Answer the second half of the question before it is asked.** "What it cannot help with: anything personalised, and anything that writes. A shared cache has nothing to share if the response differs per user."

5. **Say how you invalidate, and that you avoid it.** “I do not purge. I put a content hash in the file name, so a new build is a new URL and needs no invalidation — which also lets me set a one-year `max-age`.”
6. **Mention the security benefit.** “For many companies the DDoS absorption and TLS termination are half the reason to have one.”

The follow-ups

“**What would it not help with?**” “Two things, and they are the same thing seen from two sides. **Personalised content** — if the response differs per user, a shared cache has nothing to share, and storing one copy per user is not caching, it is storage with an extra hop. The fix is to split the page: a cacheable shell from the edge, and the personal parts fetched separately, which is why the modern shape is a static shell plus an API call. And **writes** — a `POST` has an effect, so it cannot be cached and always travels the full distance. The consequence worth stating: a CDN transforms a read-heavy product and does almost nothing for a write-heavy one. A photo app becomes local; a collaborative editor does not. The partial modern exception is edge compute — running your code at the edge lets some personalisation happen there — but it does not change the physics for writes.”

“**You deploy new JavaScript. How do users get it?**” “I would not purge, and that is the interesting part of the answer. Purging is slow — seconds to a minute to propagate across hundreds of locations — and it is rate-limited, and a deploy might change four hundred files. Instead I put a content hash in the file name: `app.a3f9c2.js`. A new build produces a new name, which is a new cache key, so there is nothing to invalidate and I can set `max-age` to a year with `immutable`. Two extra benefits: the old and new files **coexist safely**, so a user who loaded the page before the deploy can still fetch the old script it references rather than getting a mismatched new one; and the HTML that references it is the only thing that needs a short TTL. That is the versioned-key idea applied at the edge — avoid invalidation rather than solve it.”

“**Can you cache the HTML?**” “Sometimes, and it depends entirely on whether it is personalised. If the HTML contains the user’s name or anything from their session, then no — and I would be careful, because caching a personalised response at the edge is a data-breach class of bug rather than a performance one. One user’s page served to everybody, and the usual causes are a `Cache-Control: public` where it should be `private`, or a cache key that ignores cookies. If the HTML is the same for everyone, then yes, and I would use `s-maxage` — which sets the shared-cache lifetime separately from the browser’s — so the edge absorbs the load while the browser still revalidates. The pattern I would default to is a fully static shell cached hard, with the personal data fetched by a separate API call that is never cached.”

“How much does it actually save?” “Two numbers. **Latency:** a round trip drops from about 200 milliseconds to about 8 for a user in India served by a local edge. And it is not just the response — the TLS handshake is two or three round trips, so terminating it at the edge saves another 400 to 600 milliseconds on a new connection, which is frequently larger than the content transfer. **Offload:** on a typical 2 MB page, roughly 95 percent of the bytes are shared assets, so at a hundred thousand page loads an hour my origin goes from 200 GB an hour to about 10. In money, that is the difference between about ten lakh a month of cloud egress and about two lakh of CDN bandwidth plus fifty thousand of residual egress — CDN bandwidth is bought in bulk and priced well under cloud egress, so it is genuinely cheaper, not just faster.”

“What would you check if the hit rate is only 60 percent?” “The cache key, before anything else. The edge keys on the full URL including the query string, so every `utm_source` and `fbclid` on a shared link creates a separate cached object for identical bytes — a single image shared four ways becomes four objects and a 25 percent hit rate. Every CDN lets you strip or allow-list query parameters, and that is usually the single largest improvement available. Next I would look at `Vary: Vary: Accept-Encoding` is normal and roughly doubles the entries, but `Vary: User-Agent` multiplies them by thousands and effectively turns the cache off. Then cookies — if the origin sets a cookie on a static asset response, most CDNs will refuse to cache it. And finally TTLs that are simply too short for content that never changes.”

“What are the risks of putting one in?” “Three. **Invalidation control** — content is in hundreds of places I do not operate, purges are slow and rate-limited, which is why I would design around renaming instead. **An opaque dependency** — when the CDN misbehaves I cannot log into it, and every major CDN has had an outage that took a visible fraction of the internet with it, so I would want to know whether I can fail back to serving directly from the origin. And **misconfiguration**, which fails silently in both directions: too aggressive and I serve one user’s data to another, too conservative and I quietly get a 60 percent hit rate. I would also make sure the origin is firewalled to accept only CDN traffic — otherwise hiding the origin behind a CDN provides no protection at all, because the servers are still directly reachable by anyone who finds the address.”

A model answer

Asked: would a CDN help this system, and what would it not help with?

“Yes, and I would start with why, because the reason is not really about caching — it is about distance. A round trip from a user in India to a data centre in Virginia is about two hundred milliseconds, and that is pure physics: thirteen thousand kilometres at roughly two-thirds the speed of light, in each direction, before any computer does anything. **No amount of faster code touches it.** The only fix is to be closer, and a CDN is a few hundred copies of your static content placed in cities near your users.

The reason it is the biggest single win is the byte split. On a typical two-megabyte page, the images, JavaScript, CSS and fonts are identical for every user and are about ninety-five percent of the bytes. So the CDN removes almost all of my bandwidth and most of my requests **before** Redis or the database is consulted at all. At a hundred thousand page loads an hour, my origin goes from two hundred gigabytes an hour to about ten.

I would use **origin pull**: the first user in a city misses, the edge fetches from my origin, and everyone after that in that city gets a hit. With a few hundred locations that means a few hundred cold misses per new object, so I would enable an **origin shield** — edges go to a regional parent cache and only the parent talks to my origin. Otherwise publishing a popular new file produces a three-hundred-fold burst at my servers.

The saving is also bigger than people count, because the **TLS handshake** terminates at the edge. A handshake is two or three round trips, so on a new connection that is another four to six hundred milliseconds saved, often more than the content transfer itself.

Now the second half of your question, which is the more important one. **A CDN cannot help with anything personalised, and it cannot help with writes.** If a response differs per user then a shared cache has nothing to share — storing one copy per user is not a cache. And a `POST` has an effect, so it travels the full two hundred milliseconds every time. The consequence is an asymmetry worth stating: a CDN transforms a read-heavy product and does very little for a write-heavy one.

The design that follows is to **split the page**: a fully static shell cached hard at the edge, and the personal data fetched by a separate API call that is never cached. And I would be careful here, because accidentally caching a personalised response — a `Cache-Control: public` where it should be `private`, or a cache key that ignores cookies — serves one user’s data to everybody. That is a security bug, not a performance bug.

For invalidation I would avoid the problem rather than solve it: **content hashes in file names**, so `app.js` becomes `app.a3f9c2.js`. A new build is a new URL, so nothing needs purging, I can set a one-year `max-age`, and old and new coexist safely for anyone mid-session. Purging exists, and it is slow and rate-limited, and I would keep it for emergencies.

And one thing that is not about performance at all: for many companies the **DDoS absorption** is half the reason to buy one, since the CDN has far more capacity than the origin and soaks up volumetric attacks before they arrive — provided the origin is firewalled to accept only CDN traffic.”

9 Recall card

- **A CDN exists because distance is physics, not engineering.** India ↔ Virginia is ~200 ms round trip and no code change removes it. Edges put a copy ~8 ms away. The **TLS handshake** (2–3 round trips) terminates at the edge too — often a **bigger saving** than the content itself.
- **Quote the byte split: ~95% of a page's bytes are shared assets** (images, JS, CSS, video), so a CDN removes almost all origin bandwidth **before Redis is consulted** — 200 GB/hour → 10 GB/hour at 100k page loads. And CDN bandwidth is **cheaper than cloud egress**, so it usually saves money too.
- **Origin pull by default, plus an ORIGIN SHIELD** — without it, 300 edges each cold-miss and you get a **300 × burst** at the origin per new object. Push distribution for large predictable objects (game patches, film releases; Netflix Open Connect ships boxes into ISPs).
- **What it cannot do: personalised content and writes.** A shared cache has nothing to share if the response differs per user; a `POST` always travels the full distance. **Split the page** — static shell at the edge, personal data via an uncached API call. **Caching a personalised response or a Set-Cookie is a data-breach bug, not a performance bug.**
- **Do not purge — rename.** Content hashes (`app.a3f9c2.js`) make a new build a new key: no invalidation, `max-age` of a year, and **old and new coexist safely** mid-session. And when the hit rate is bad, **check the cache key first**: `utm_*` and `fbclid` create a new object per share link (99% → 25%), and `Vary: User-Agent` effectively disables caching.

DSA topic: Same tree, symmetric tree, and subtree **System design topic:** Content delivery networks

Code these, in this order

One rule for the whole set: **write the two base-case lines before anything else, as a unit.** Both `None` is `True`; exactly one `None` is `False`. Every crash in this topic is on the line that comes after those two, when they are not both there.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Same Tree	LeetCode 100 (Easy)	The three base cases, and the <code>and</code> short-circuit.
2	Symmetric Tree	LeetCode 101 (Easy)	Crossed arguments — mirror, not identical.
3	Subtree of Another Tree	LeetCode 572 (Easy)	Different base cases, and what “subtree” strictly means.
4	Invert Binary Tree	LeetCode 226 (Easy)	Producing a mirror rather than testing for one.

On problem 1, delete the middle base case on purpose

Run it on two trees of different shapes and record the exact error and the line it happens on. Then say in one sentence why the missing case is a crash rather than a wrong answer.

On problem 2, write the wrong version first

Write `is_same(root.left, root.right)`, run it on `[1,2,2,null,3,null,3]`, and record what it says. Then write the crossed version and say what “mirror” means as opposed to “identical”.

On problem 3, test the strictness

Run `is_subtree([1,2,3], [1,2])` and record the answer. Say why it is `False` and what would have to be true for it to be `True`.

On problem 4, then connect it back

After inverting, check that `is_symmetric(t)` equals `is_same(t, invert(copy(t)))`. Say why you would still write `is_mirror` instead, in terms of allocation.

The base-case drill

1. Write the three base cases from memory.
2. Say what each one means about the two trees.
3. Say what error the missing middle case produces, and on which expression.
4. Say why everything after those two lines can dereference safely.
5. Say what the `and` short-circuit buys you, and what complexity that gives.

The mirror drill

1. Write `is_same` and then derive `is_mirror` from it by changing one thing.
2. Say the change in words.
3. Construct a tree where identical halves are not symmetric.
4. Cross only one of the two calls and find an input where it is wrong.
5. Write the entry point and say why an empty tree and a single node are both symmetric.

The subtree drill

1. Write the base cases for `is_subtree` and say why they differ from `is_same`'s.
2. Swap them and say what the function now returns for everything.
3. Define “subtree” precisely, in terms of what may hang below.
4. Give an input where a partial match is not a subtree.
5. Say what question you would ask the interviewer if they meant partial matching.

The complexity drill

1. State the time for same tree, and say why it is `min` rather than `max`.
2. State the naive subtree complexity and give the worst-case input that achieves it.
3. Say why it is usually much better than the bound in practice.
4. Give the operation counts for $n = 10,000$ and $m = 1,000$.

The serialisation drill

1. Write `serialise` with both fixes.
2. Construct two different trees that collide without null markers.

3. Construct a false match that occurs without delimiters.
4. State the complexity, and say what you are trading for it.
5. Say how you would phrase the complexity honestly with respect to `str.find`.

The break-it drill

Trigger each and record the exact output or error:

1. `is_same` without the “exactly one is None” case.
2. `is_same(root.left, root.right)` as a symmetry test.
3. One of the two mirror calls uncrossed.
4. `is_subtree` with the base cases swapped.
5. Serialisation without null markers, comparing a left-only and a right-only tree.
6. Serialisation without delimiters, searching for a tree containing 2 inside one containing 12.
7. A recursive comparison on two chains of 10,000 nodes.

The iterative drill

1. Write `is_same` with an explicit stack of pairs.
 2. Write `is_symmetric` with a queue of pairs.
 3. Say what the crossed pushes look like in the iterative version.
 4. Say when you would bother writing these.
-

The why-it-exists drill

1. Compute the round trip from India to Virginia, showing the working.
2. Say why no server optimisation touches it.
3. Give the edge, same-continent and cross-world figures.
4. Say what a page with four sequential round trips costs, both ways.
5. Say why the TLS handshake saving is often bigger than the content saving.

The byte-split drill

1. Give the byte breakdown of a typical 2 MB page.
2. State the percentage a CDN can serve, in bytes and in requests.
3. Compute origin bandwidth per hour at 100,000 page loads, both ways.
4. Compute the monthly cost difference, including the CDN’s own bandwidth charge.
5. Say why this makes the CDN the biggest of the four cache layers.

The distribution drill

1. Define origin pull and say what a cold miss is.
2. Compute the cold misses for one new object across 300 edges.
3. Describe an origin shield and recompute.
4. Say when you would use push instead, with two examples.
5. Name the company that ships hardware into ISPs and say why.

The headers drill

1. Write the `Cache-Control` header for a fingerprinted asset.
2. Write one for a personalised page.
3. Write one that caches at the edge but not in the browser, and name the directive.
4. Explain `ETag` and what a 304 saves.
5. Say what `Vary: User-Agent` does to the cache.

The limits drill

1. Name the two things a CDN cannot help with, and say why they are the same thing.
2. Describe the page-splitting design that works around the first.
3. Say what edge compute changes and what it does not.
4. Say which kinds of product are transformed and which are barely helped, with an example of each.

The invalidation drill

1. Say how long a purge takes and what limits it.
2. Describe the renaming approach and what it makes possible.
3. Say what “old and new coexist safely” prevents.
4. Compute the work for a deploy changing 400 files, both ways.

The hit-rate drill

1. Give the expected hit rates for five kinds of content.
2. Say what a shared link with `utm_source` does to the cache key.
3. Compute the hit rate for one image shared four ways.
4. Name the three things you would check first when the hit rate is 60 percent.

The failure drill

For each, say what happens and what you would add:

1. A personalised page is served with `Cache-Control: public`.

2. A response carrying `Set-Cookie` is cached.
3. `app.js` is cached for a year with no fingerprint.
4. A launch happens with a completely cold cache.
5. The CDN has a regional outage.
6. The origin is not firewalled to CDN traffic only.

Two of the six are security problems rather than performance problems. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Are these two trees identical? Is this tree a subtree of that one?* The two-tree recursion named, the three base cases stated before writing, what the missing one costs, the short-circuit complexity, then the strict meaning of subtree with its different base cases and the $O(n \times m)$ bound with the serialisation alternative and both of its silent traps.
 2. *Would a CDN help this system? What would it not help with?* Distance as physics with the 200 ms number, the 95 percent byte split, origin pull plus a shield with the $300 \times$ burst, then the two limits — personalisation and writes — the page-splitting fix, and renaming instead of purging.
 3. *Is this tree symmetric?* The crossed arguments, the counter-example where identical halves are not symmetric, and why an empty tree and a single node both qualify.
-

Before you move on

- [] I write the two base-case lines as a unit before anything else.
- [] I can say what error the missing middle case produces.
- [] I can derive `is_mirror` from `is_same` by naming one change.
- [] I have a tree where identical halves are not symmetric.
- [] I know why `is_subtree`'s base cases are the opposite way round.
- [] I can state what "subtree" strictly means and give a failing partial match.
- [] I can state the same-tree complexity and say why it is `min`.
- [] I know the adversarial input that makes the naive subtree quadratic.
- [] I can write `serialise` with both fixes and explain each.

- [] I can construct the collision and the false match that each fix prevents.
- [] I can write both iterative versions with pairs.
- [] I can compute the India–Virginia round trip from first principles.
- [] I can give the byte split and the offload arithmetic.
- [] I can explain a cold miss and why an origin shield matters, with the $300 \times$ number.
- [] I can write three different `Cache-Control` headers and say what each is for.
- [] I can name both CDN limits and the page-splitting fix.
- [] I can explain why renaming beats purging, including the coexistence benefit.
- [] I know the first three things to check when a hit rate is low.
- [] I answered all three questions above out loud.

Path problems, and the return-value trick

DATA STRUCTURES AND ALGORITHMS

Path problems, and the return-value trick

You can return one thing to the parent while tracking another in an outer variable.

SYSTEM DESIGN

Database replication

You can draw leader-follower replication and say what happens when the leader dies.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the maximum path sum in the binary tree.*
- *How do you make the database survive a machine failure?*

Path problems, and the return-value trick

1 What this is, and why they ask it

A **path** in a tree is a sequence of nodes where each one is connected to the next. Path problems ask you to find one — the longest, the heaviest, the one that adds to a target — and they are the hardest family in the topic because **the answer you want and the answer your parent needs are different things**.

Three sentences. You met the shape on [day 102](#) with diameter: return the height, record the bend. Today it does real work, because the values can be **negative**, and that introduces a decision at every node — *is this branch worth taking at all?* And there are two different definitions of “path” in circulation, so the first thing to do is establish which one is meant.

They ask maximum path sum because it is one of the hardest problems that still fits in fifteen minutes, and it fails in three distinct ways: returning the bent path to the parent, forgetting that a negative branch should be dropped, and forgetting that the answer itself can be negative. Get those three right and the code is nine lines. Get any of them wrong and it is confidently, silently wrong on exactly the inputs the tests use.

2 The story

Devaraj collected for the temple fund and his territory was the seven villages off the Kalyanpur road.

It was not a straight road. It forked twice, and the branches forked again, so what he was really working was a set of lanes spreading out from the junction near the water tank.

Every village was worth something to him, and the figure he had in his book was not the donation — it was the donation **minus** what it cost him to get there and back. The two big villages were plus four hundred and plus three hundred. But the one at the top of the north lane was minus sixty, because it was eleven kilometres of bad road for a family that gave twenty rupees, and the little settlement past the quarry was minus a hundred and ten.

He could not do all of them in a day. What he did was pick a single run — go up one lane, come back down through a junction, and out along another — and he wanted the best run he could get.

The way he worked it out was the thing his nephew found strange.

He would stand at a junction and ask, for each lane going out of it, one question only: **is there anything worth having down there, and how much?** Not “what is down there” — how much, as a single number, and never less than zero. If the best he could do down the north lane was minus sixty, he wrote zero, because he was not obliged to go down it at all. Zero was always available.

Then, standing at that junction, the best run **through** it was the north lane’s number plus the south lane’s number plus whatever the junction village itself was worth.

But — and this is the part his nephew kept getting wrong — when he then walked back to the *previous* junction and it asked him “how much is down your lane?”, he could not give that number. Because a run that goes up the north lane and down the south lane has already used the junction in the middle. He cannot then also carry on backwards out of it. From the previous junction’s point of view, all he can offer is **one** arm: the junction village plus the better of its two lanes.

So there were two numbers at every junction. The best run that turned around there — which he wrote down in his book and never used again except to compare. And the best single arm going down — which was the only thing he could report upwards.

His nephew, who did this for a season, kept reporting the first number, and kept coming home with routes that doubled back on themselves and could not actually be walked.

3 The idea in plain English

Devaraj has solved maximum path sum, and both of the things his nephew got wrong are the traps.

- Each village is a node; its value is the node’s value, and it can be **negative**.
- “Is there anything worth having down there, and never less than zero” is `max(0, arm)` — the floor.
- The best run **turning around** at a junction is `left + right + node.val`, and it is **recorded**.
- The best **single arm** to offer upwards is `node.val + max(left, right)`, and it is **returned**.

- Reporting the turning-around number upwards is the nephew’s mistake, and it produces paths that cannot exist.

First: which definition of “path”?

Two are in use and they need different code.

ROOT-TO-LEAF PATH	starts at the root, ends at a leaf, always goes down LeetCode 112, 113, 257
ANY-TO-ANY PATH	starts anywhere, ends anywhere, may bend at one node LeetCode 124, 543, 687

Ask which one, or state your assumption. They are different problems: root-to-leaf is a preorder walk carrying state down, and any-to-any is the postorder return-value trick.

There is a third variant that catches people: **downward but not necessarily root-to-leaf** — LeetCode 437, path sum III, where a path can start and end anywhere as long as it goes downwards. That one has a completely different best solution, and it is at the end of this lesson.

Root-to-leaf: information goes down

```
def walk(node, remaining):
    if node is None:
        return False
    remaining -= node.val
    if node.left is None and node.right is None:    # a LEAF
        return remaining == 0
    return walk(node.left, remaining) or walk(node.right, remaining)
```

The state is carried down as an argument, so there is nothing to undo — the [day 094](#) rule. And the leaf test is `both children are None`, never `node is None`, which is trap 1.

For “collect all such paths” you also carry the path itself, and that *does* need choose-recurse-un-choose plus the copy:

```
trail.append(node.val)        # choose
...
trail.pop()                   # un-choose
```

Any-to-any: the return-value trick, doing real work

Every path has exactly one **highest node** — the point where it turns around. So evaluate every node as that turning point, and you have considered every path exactly once. Same observation as [day 102](#).

At a node, two different quantities exist and **they must not be confused**:

```
RECORD (the answer):  left_arm + right_arm + node.val
                      the best path that TURNS AROUND here

RETURN (to the parent): node.val + max(left_arm, right_arm)
                      the best SINGLE ARM going down from here
```

The parent cannot use the recorded value, because a path that turns around at me has already used both of my children — it cannot also continue upward through me. That is the sentence to say.

The floor at zero, which is what negatives add

```
left = max(0, arm(node.left))
right = max(0, arm(node.right))
```

`max(0, ...)` means “I am not obliged to take this branch”. If the best the left subtree can offer is negative, taking nothing is better, and taking nothing contributes zero.

Without the floor:

```
tree:
  -10
 /  \
9    20
    /  \
  15   7
```

```
with max(0, ...):  15 + 7 + 20 = 42      correct
without:          the -10 root drags everything through it, and the
                  answer comes out wrong on any tree with negatives
```

This is exactly Kadane’s algorithm from the array world — *drop the prefix when it turns negative* — applied to two branches instead of one.

And the trap the floor creates

```
best = 0          # WRONG
```

If every node is negative, the answer is negative — a single node, the least bad one. Initialising `best` to `0` returns `0` for `[-3]`, which is not a path.

```
best = float("-inf") # correct
```

The floor applies to the arms, never to the answer. A path must contain at least one node, so the answer is at worst the largest single value. Saying that distinction out loud is what separates people who understand the floor from people who copied it.

The family, one table

PROBLEM	RETURN TO PARENT	RECORD ON THE SIDE	FLOOR?
Diameter (543)	height	<code>left + right + 2</code>	no
Max path sum (124)	<code>val + max(left, right)</code>	<code>left + right + val</code>	yes, <code>max(0, arm)</code>
Longest univalue path (687)	longest matching arm	<code>left + right</code>	no
Max sum root-to-leaf	—	carried down	no
Largest BST subtree (333)	<code>(min, max, size, ok)</code>	<code>size</code> when <code>ok</code>	no

One shape, five problems. If you can fill this table in, you have the technique.

Path sum III: the one that is not this shape at all

“Count the paths summing to a target, where a path goes downwards but need not start at the root.”

The obvious solution runs the root-to-leaf walk starting from every node: `O(n2)`, or `O(n log n)` on a balanced tree.

The good solution is **prefix sums with a hash map**, exactly as on [day 038](#), applied to the current root-to-node path:

```
running += node.val
count += seen[running - target] # how many earlier prefixes make a valid path
seen[running] += 1             # choose
...recurse...
seen[running] -= 1             # UN-CHOOSE – this is essential
```

The `seen[running] -= 1` on the way out is the whole trick. The map must contain only the prefixes on the *current* root-to-node path — a prefix from a sibling branch is not reachable from here. It is [day 094’s](#) un-choose, applied to a dictionary instead of a list, and forgetting it counts paths that bend, which is not what was asked.

That takes it from $O(n^2)$ to $O(n)$, and it is the strongest single thing you can show in this family.

4 The picture

Devaraj's junction, and the two numbers.

```

              junction (value 20)
             /           \
    north lane       south lane
    best arm: -60      best arm: 15
    → max(0, -60) = 0  → max(0, 15) = 15

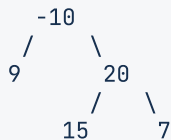
RECORD (turns around here): 0 + 15 + 20 = 35
RETURN (one arm, upwards): 20 + max(0, 15) = 35
                        (they coincide here only because one arm is 0)

with both arms positive:
              junction (value 20)
             /           \
    best arm: 15       best arm: 7

RECORD: 15 + 7 + 20 = 42 ← the answer candidate
RETURN: 20 + max(15, 7) = 35 ← what the parent may use

the parent CANNOT have 42: a path bending here has spent BOTH children.
```

The classic tree, traced:



```

arm(9):  left=max(0,-inf...)=0, right=0
         RECORD 0 + 0 + 9 = 9          best = 9
         RETURN 9 + max(0,0) = 9

arm(15): RECORD 15, RETURN 15          best = 15
arm(7):  RECORD 7,  RETURN 7          best = 15

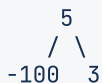
arm(20): left = max(0, 15) = 15
         right = max(0, 7) = 7
         RECORD 15 + 7 + 20 = 42      best = 42      ← the answer
         RETURN 20 + max(15, 7) = 35

arm(-10): left = max(0, 9) = 9
          right = max(0, 35) = 35
          RECORD 9 + 35 + (-10) = 34  best stays 42
          RETURN -10 + 35 = 25

answer: 42  (the path 15 → 20 → 7, which never touches the root)

```

What the floor does, drawn:



WITHOUT the floor:	WITH the floor:
left arm = -100	left arm = max(0, -100) = 0
right arm = 3	right arm = max(0, 3) = 3
RECORD -100 + 3 + 5 = -92	RECORD 0 + 3 + 5 = 8
RETURN 5 + max(-100, 3) = 8	RETURN 5 + max(0, 3) = 8

the recorded value is wrong without the floor: the real best path through 5 is [5, 3] = 8, not [-100, 5, 3] = -92.
Nothing forces you to walk down a lane that loses money.

The two definitions of “path”, side by side:

ROOT-TO-LEAF (112, 113, 257)



valid: 1-2, 1-3

invalid: 2-1-3

(does not start
at the root)

ANY-TO-ANY (124, 543, 687)



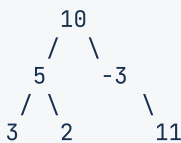
valid: 2-1-3, 2, 1-2, 3, ...

invalid: nothing that
revisits a node

state goes DOWN as an argument
preorder
no undo needed (values are passed)

state comes UP as a return value
postorder
the RECORD/RETURN split

Path sum III's map, and why the un-choose matters:



walking down 10 → 5 → 3, `seen` holds the prefixes {0:1, 10:1, 15:1}
walking back up to 5 and down to 2, `seen` must NO LONGER contain 18
(the prefix from the 3 branch) – 3 is not on the path to 2.

without seen[running] -= 1 on the way out:
the map accumulates prefixes from sibling branches
→ counts "paths" that bend, which are not downward paths
→ an over-count with no error

5 The code, built step by step

Step 1 – establish which “path” is meant

“Is the path root-to-leaf, or can it start and end anywhere? And can it bend at a node, or must it go strictly downwards?” **Three different problems, and asking costs ten seconds.**

Step 2 – for root-to-leaf, carry the state down

```
remaining -= node.val
if node.left is None and node.right is None:
    return remaining == 0
```

A leaf is a node whose two children are both `None`. Not `node is None` — that is trap 1 and it makes a one-sided node look like a leaf.

Step 3 — for any-to-any, write the two lines and label them

```
best = max(best, left + right + node.val)    # RECORD  
return node.val + max(left, right)          # RETURN
```

Write them adjacent, with those two words in your head. **The recorded line is the answer; the returned line is what the parent can use.** Say the reason: a path bending here has already spent both children.

Step 4 — the floor on the arms

```
left = max(0, arm(node.left))  
right = max(0, arm(node.right))
```

Say what it means: *“I am not obliged to take a branch. If the best it offers is negative, I take nothing, which contributes zero.”*

Step 5 — initialise the answer to negative infinity, not zero

```
best = float("-inf")
```

Because the answer itself can be negative — a tree of all-negative values has a best path of one node. **The floor applies to the arms, not to the answer.**

The complete solution

```
from collections import defaultdict, deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

# -----
# ROOT-TO-LEAF: state travels DOWN as an argument. Preorder. No undo needed.
# -----

def has_path_sum(root: TreeNode | None, target: int) → bool:
    """LeetCode 112. Does any root-to-leaf path add to `target`?

    A LEAF is a node with BOTH children None. Using `node is None` as the
    base case makes a one-sided node look like a leaf and gives wrong
    answers on trees with single children.
    """
    if root is None:
        return False
    remaining = target - root.val
    if root.left is None and root.right is None:
        return remaining == 0
    return (has_path_sum(root.left, remaining)
            or has_path_sum(root.right, remaining))

def all_path_sums(root: TreeNode | None, target: int) → list[list[int]]:
    """LeetCode 113. Every root-to-leaf path adding to `target`.

    Now the path itself is state, so this IS choose-recurse-un-choose, and
    the recorded path must be a COPY.
    """
    result: list[list[int]] = []
    trail: list[int] = []

    def walk(node: TreeNode | None, remaining: int) → None:
        if node is None:
            return
        trail.append(node.val)                # choose
        remaining -= node.val
        if node.left is None and node.right is None and remaining == 0:
            result.append(trail[:])           # COPY
        else:
            walk(node.left, remaining)
            walk(node.right, remaining)
        trail.pop()                           # un-choose

    walk(root, target)
    return result

def binary_tree_paths(root: TreeNode | None) → list[str]:
    """LeetCode 257. Every root-to-leaf path, as strings.

    Note this version passes the path as a STRING argument, so there is
    nothing to undo – the day-094 rule that values passed as arguments
    cannot need restoring.
    """
    paths: list[str] = []
```

```

def walk(node: TreeNode | None, so_far: str) → None:
    if node is None:
        return
    so_far = f"{so_far}→{node.val}" if so_far else str(node.val)
    if node.left is None and node.right is None:
        paths.append(so_far)
        return
    walk(node.left, so_far)
    walk(node.right, so_far)

walk(root, "")
return paths

# -----
# ANY-TO-ANY: the RECORD / RETURN split. Postorder.
# -----

def max_path_sum(root: TreeNode | None) → int:
    """LeetCode 124. The hardest of the family, in nine lines.

    THREE things must be right, and each fails silently:

    1. RECORD `left + right + val`, RETURN `val + max(left, right)`.
       The parent cannot use the recorded value – a path bending here has
       already spent both children, so it cannot continue upward.

    2. FLOOR each arm at 0: max(0, arm). You are never obliged to take a
       branch, so a negative arm contributes nothing. This is Kadane's
       "drop the prefix when it goes negative", on two branches.

    3. Initialise `best` to -inf, NOT 0. The floor applies to the ARMS, not
       to the ANSWER – an all-negative tree has a best path of one node.

    Time O(n). Space O(height).
    """
    best = float("-inf")

    def arm(node: TreeNode | None) → int:
        nonlocal best
        if node is None:
            return 0
        left = max(0, arm(node.left))      # never obliged to take a branch
        right = max(0, arm(node.right))
        best = max(best, left + right + node.val)  # RECORD: bends here
        return node.val + max(left, right)        # RETURN: one arm

    arm(root)
    return int(best)

def max_path_sum_with_path(root: TreeNode | None) → tuple[int, list[int]]:
    """The follow-up: return the path, not just the sum.

    Same shape, bigger return value. Costs O(n) space instead of O(height),
    because every frame now holds a list.
    """
    best = float("-inf")
    best_path: list[int] = []

    def arm(node: TreeNode | None) → tuple[int, list[int]]:
        nonlocal best, best_path
        if node is None:
            return 0, []
        ls, lp = arm(node.left)
        rs, rp = arm(node.right)
        if ls < 0:
            ls, lp = 0, []
        if rs < 0:

```

```

        rs, rp = 0, []
        if ls + rs + node.val > best:
            best = ls + rs + node.val
            best_path = lp + [node.val] + rp[::-1]
        if ls ≥ rs:
            return node.val + ls, [node.val] + lp
        return node.val + rs, [node.val] + rp

    arm(root)
    return int(best), best_path

def max_root_to_leaf(root: TreeNode | None) → int:
    """For contrast: root-to-leaf only. NO floor, because you MUST reach a
    leaf – you cannot decline a branch when there is only one."""
    if root is None:
        return 0
    if root.left is None:
        return root.val + max_root_to_leaf(root.right)
    if root.right is None:
        return root.val + max_root_to_leaf(root.left)
    return root.val + max(max_root_to_leaf(root.left),
                          max_root_to_leaf(root.right))

# -----
# DOWNWARD, ANYWHERE TO ANYWHERE: prefix sums on a tree.
# -----

def path_sum_iii_naive(root: TreeNode | None, target: int) → int:
    """LeetCode 437, the obvious way: start a downward walk at every node.
    O(n^2) worst case, O(n log n) on a balanced tree."""
    def down_from(node: TreeNode | None, remaining: int) → int:
        if node is None:
            return 0
        remaining -= node.val
        return ((1 if remaining == 0 else 0)
                + down_from(node.left, remaining)
                + down_from(node.right, remaining))

    if root is None:
        return 0
    return (down_from(root, target)
            + path_sum_iii_naive(root.left, target)
            + path_sum_iii_naive(root.right, target))

def path_sum_iii(root: TreeNode | None, target: int) → int:
    """LeetCode 437 in O(n), with prefix sums and a hash map.

    Exactly day 038's subarray-sum-equals-k, applied to the current
    root-to-node path.

    THE CRITICAL LINE is `seen[running] -= 1` on the way out. The map must
    hold ONLY the prefixes on the current path – a prefix from a sibling
    branch is not reachable from here. Without it you count "paths" that
    bend, which the question did not ask for, and there is no error.
    """
    seen: dict[int, int] = defaultdict(int)
    seen[0] = 1 # the empty prefix
    count = 0

    def walk(node: TreeNode | None, running: int) → None:
        nonlocal count
        if node is None:
            return
        running += node.val
        count += seen[running - target] # earlier prefixes that complete a path
        seen[running] += 1 # choose

```

```

        walk(node.left, running)
        walk(node.right, running)
        seen[running] -= 1                                # UN-CHOOSE – the whole trick

    walk(root, 0)
    return count

def sum_root_to_leaf_numbers(root: TreeNode | None) → int:
    """LeetCode 129. Each root-to-leaf path spells a number; sum them.
    State carried down, so nothing to undo."""
    def walk(node: TreeNode | None, so_far: int) → int:
        if node is None:
            return 0
        so_far = so_far * 10 + node.val
        if node.left is None and node.right is None:
            return so_far
        return walk(node.left, so_far) + walk(node.right, so_far)

    return walk(root, 0)

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
            i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
            i += 1
    return root

if __name__ == "__main__":
    classic = from_list([-10, 9, 20, None, None, 15, 7])
    print(max_path_sum(classic))                # 42    (15 → 20 → 7)
    print(max_path_sum_with_path(classic))      # (42, [15, 20, 7])

    # the floor matters
    print(max_path_sum(from_list([5, -100, 3]))) # 8      (5 → 3), not -92

    # the answer can be negative: -inf, not 0
    print(max_path_sum(from_list([-3])))        # -3
    print(max_path_sum(from_list([-2, -1])))    # -1

    print(max_path_sum(from_list([1, 2, 3])))   # 6

    # root-to-leaf family
    t = from_list([5, 4, 8, 11, None, 13, 4, 7, 2, None, None, 5, 1])
    print(has_path_sum(t, 22))                  # True
    print(all_path_sums(t, 22))                 # [[5, 4, 11, 2]]
    print(binary_tree_paths(from_list([1, 2, 3, None, 5])))
    # ['1→2→5', '1→3']
    print(sum_root_to_leaf_numbers(from_list([1, 2, 3]))) # 25    (12 + 13)

    # the leaf trap: a one-sided node is NOT a leaf
    one_sided = from_list([1, 2])
    print(has_path_sum(one_sided, 1))           # False – 1 alone is not a path to a leaf
    print(has_path_sum(one_sided, 3))           # True

    # path sum III

```

```
p = from_list([10, 5, -3, 3, 2, None, 11, 3, -2, None, 1])
print(path_sum_iii(p, 8), path_sum_iii_naive(p, 8))      # 3 3
print(path_sum_iii(from_list([1, None, 2, None, 3]), 3)) # 2
```

6 What it costs

Maximum path sum

```
time      0(n)      every node visited once
space     0(height) the recursion stack
```

One pass, one visit per node, because the arm value each node needs from its children is computed exactly once and returned upward. There is no recomputation to eliminate — unlike the naive diameter, this problem has no obvious quadratic version to fall into.

Root-to-leaf

```
has_path_sum      0(n) time, 0(height) space
all_path_sums     0(n × height) time in the worst case, because each recorded
                  path is copied and a path is up to `height` long
                  0(n × height) space for the output
```

```
a perfect tree of 1,000,000 nodes
leaves           500,000
path length      20
output           500,000 × 20 = 10,000,000 integers ≈ 400 MB in Python
```

Collecting all paths is $0(n \times \text{height})$, not $0(n)$, and it is worth saying, because the output alone can be enormous.

`binary_tree_paths` passing a string down is worse than it looks: each concatenation builds a new string, so it is $0(n \times \text{height})$ in copying too. Passing a list and joining once at a leaf is the faster version, at the cost of an un-choose.

Path sum III: the improvement worth showing

```
naive:  a downward walk started at every node
        balanced tree:  0(n log n)
        degenerate:     0(n^2)

prefix sums with a map:  0(n) time, 0(height) extra space for the map
```


n = 1,000	naive ~10,000	prefix ~1,000
n = 100,000	naive ~1,700,000	prefix ~100,000
n = 100,000 (a chain)	naive ~5,000,000,000	– will not finish

The map holds at most `height` entries at any moment, because entries are removed on the way out. That is worth stating: it is `O(height)`, not `O(n)`, and the reason is the un-choose.

The floor, and why it is not just an optimisation

```
a tree of 1,000 nodes, values uniformly in [-100, 100]
with the floor:      the answer is a genuinely positive path
without:             the answer is dragged down by forced negative branches
                     and is simply WRONG, not slow
```

`max(0, arm)` changes the answer, not the speed. It is not an optimisation and calling it one in an interview is a tell.

Space, and the deep-tree warning

```
max_path_sum          O(height)
max_path_sum_with_path O(n)      every frame holds a list
all_path_sums         O(n × height) for the output
```

```
a chain of 10,000 nodes → RecursionError: maximum recursion depth exceeded
```

Same as every tree problem. If the constraint allows a chain, either convert to an explicit postorder stack or say the risk.

7 The traps

Trap 1 – `node is None` as the leaf test

```
if node is None:
    return remaining == 0          # WRONG
```

A node with one child has a `None` on the other side, so this reports a “path” that stops halfway.

```
has_path_sum(from_list([1, 2]), 1)
this version → True      (it "found" the path [1], stopping at the missing right child)
correct      → False     (1 is not a leaf)
```

A leaf is a node with both children `None`. Test it on the node, not on the recursion.

Trap 2 — returning the bent path to the parent

```
return max(left + right + node.val, node.val + max(left, right))    # WRONG
```

Devaraj's nephew. The parent may then build on a path that has already used both of my children, giving routes that double back. The result is too large and grows with depth — no error, and it passes on trees where one child is always missing.

Return one arm. Record the bend. Say the reason.

Trap 3 — no floor on the arms

```
left = arm(node.left)
right = arm(node.right)
```

```
max_path_sum(from_list([5, -100, 3]))
without the floor → -92
correct          → 8
```

A negative branch is dragged into every path through the node. **You are never obliged to take a branch.**

Trap 4 — flooring the answer instead of the arms

```
best = 0    # WRONG
```

```
max_path_sum(from_list([-3]))
with best = 0 → 0    - but 0 is not a path; there is no empty path
correct      → -3
```

The floor says “an *arm* may contribute nothing”. It does not say “the answer may be nothing”, because a path must contain at least one node. `float("-inf")`.

Trap 5 — the wrong definition of “path”

Solving root-to-leaf when they meant any-to-any, or the reverse, produces a completely different answer with confident code. **Ask, or state the assumption in your first sentence.**

Trap 6 — forgetting the un-choose in path sum III

```
seen[running] += 1
walk(node.left, running)
walk(node.right, running)

# missing: seen[running] -= 1
```

The map accumulates prefixes from sibling branches, so it counts paths that bend — which is not what “downward path” means.

```
path_sum_iii(from_list([1, -1, 1]), 1)
without the un-choose: an over-count
with:                  correct
```

No error, just a number that is too large. [Day 094's](#) rule applied to a dictionary.

Trap 7 — forgetting `seen[0] = 1`

```
seen = defaultdict(int)

# missing seen[0] = 1
```

Every path that starts at the root is missed, because there is no empty prefix to subtract. Exactly the same sentinel as [day 037's](#) `prefix[0] = 0`.

Trap 8 — string concatenation down a deep tree

```
so_far = so_far + "→" + str(node.val)
```

Each concatenation copies the whole string, so on a chain of n nodes this is $O(n^2)$ in copying alone. Correct, and slow enough to time out on a large skewed tree. Pass a list and join once at the leaf.

8 In the interview

How it gets asked

- The hard one: “Find the maximum path sum. A path can start and end at any node.” LeetCode 124.
- The easier opener: “Does any root-to-leaf path sum to this target?” LeetCode 112, then 113 for all of them.
- The one with a trick: “Count the paths summing to a target, going downwards.” LeetCode 437.

- The follow-up: “Return the path, not just the sum.”
- The probe: “What does your function return, and why is that not the answer?”

What to say out loud, in the first ninety seconds

1. **Pin down “path” first.** “Can the path start and end anywhere, or is it root-to-leaf? And can it bend at a node? Those are three different problems.”
2. **State the key observation.** “Every path has exactly one highest node — the point where it turns around. So I evaluate every node as that turning point and I have considered every path exactly once.”
3. **Name the two quantities, and say they differ.** “At each node there are two numbers. The best path that *bends* here, which is `left + right + value` — that is the answer candidate, and I record it. And the best single *arm* going down, which is `value + max(left, right)` — that is what I return, because my parent can only extend one arm.”
4. **Say why the parent cannot use the recorded value.** “A path that bends at me has already used both my children, so it cannot also continue upward through me.”
5. **Introduce the floor with its reason.** “I floor each arm at zero, because I am never obliged to take a branch — if the best it offers is negative, taking nothing is better and contributes zero. That is Kadane’s rule on two branches.”
6. **Flag the initialisation.** “And `best` starts at negative infinity, not zero — the floor applies to the arms, not to the answer. An all-negative tree has a best path of one node.”

The follow-ups

“What does your function return, and why is that not the answer?” “It returns the best single downward arm from this node — the node’s value plus the better of its two children’s arms. That is not the answer, because the answer is the best path that *bends* somewhere, and a bent path uses both children. My parent can only extend a path that comes up through me on one side, so a bent path is useless to it. So I keep the two apart: I **record** `left + right + value` in an outer variable as a candidate answer, and I **return** `value + max(left, right)` for my parent to build on. Merging them — returning the larger of the two — is the classic bug: it lets a parent extend a path that has already turned around, producing routes that revisit nodes, and the answer comes out too large with no error at all.”

“Why the `max(0, ...)` ?” “Because a branch is optional. If the best the left subtree can offer is negative, I do not take it — the path simply does not go that way, and ‘not going that way’ contributes zero. Without the floor, a single very negative node pois-

ons every path through its parent: on `[5, -100, 3]` the answer comes out as `-92` instead of `8`. It is exactly Kadane's algorithm from the array world — drop the running sum when it goes negative — applied to two branches instead of one. And the important distinction is that the floor applies to the **arms**, not to the **answer**: `best` must start at negative infinity, because a path has to contain at least one node, so an all-negative tree's answer is the least-negative single value, not zero.”

“Count the downward paths summing to k.” “The obvious solution starts a downward walk at every node — `O(n log n)` on a balanced tree and `O(n2)` on a chain. The better one is prefix sums with a hash map, which is exactly the subarray-sum-equals-k technique on the current root-to-node path: keep a running sum, and at each node add the number of earlier prefixes equal to `running - target`. The critical detail is that I **remove the prefix on the way back out** — `seen[running] -= 1`. The map must contain only the prefixes on the current path, because a prefix from a sibling branch is not reachable going downward from here. Without that line, the count includes paths that bend, which is not what was asked, and there is no error — just a number that is too big. It also means the map holds at most `height` entries, so the extra space is `O(height)` rather than `O(n)`. And I need `seen[0] = 1` for the empty prefix, or every path starting at the root is missed.”

“Return the path itself.” “Same shape with a bigger return value: each call returns both the arm sum and the list of nodes on that arm, and when a node beats the best, the winning path is the left arm plus this node plus the right arm reversed. The cost changes and I would say so — every frame now holds a list, so the space goes from `O(height)` to `O(n)`. If that mattered, the alternative is to record only the *node* where the best bend occurred in the first pass, then do a second short walk from there to reconstruct both arms — two passes, `O(height)` space.”

“What is the complexity?” “`O(n)` time — every node visited exactly once, and each node's arm value is computed once and returned upward, so there is nothing to recompute. `O(height)` space for the recursion stack, which is about twenty frames on a balanced million-node tree and a million on a chain — and Python's default recursion limit is a thousand, so if the input can be skewed and large I would either write it with an explicit postorder stack or say the risk out loud.”

“How is this different from the root-to-leaf version?” “The direction the information travels, and therefore the traversal. Root-to-leaf carries the remaining target **down** as an argument — preorder, and nothing needs undoing because a value passed as an argument cannot be modified by the callee. Any-to-any brings the arm sums **up** as return values — postorder, with the record/return split. There is one more difference

that catches people: root-to-leaf has **no floor**, because you are obliged to reach a leaf and cannot decline the only branch available. The floor exists precisely because an any-to-any path may stop anywhere.”

A model answer

Asked: *find the maximum path sum in the binary tree, where a path may start and end at any node.*

“First, let me pin down ‘path’, because there are three versions of this question. Root-to-leaf, which must start at the root and end at a leaf. Strictly downward but starting anywhere. And any-to-any, where the path may **bend** at one node. You have said any-to-any, so the path goes up one side, turns around at some node, and comes down the other.

The observation that makes it tractable: **every path has exactly one highest node** — the point where it turns. So if I evaluate every node as that turning point, I have considered every possible path exactly once, with no duplication.

At each node there are **two different numbers**, and keeping them apart is the whole problem. The best path that **bends** here is `left arm + right arm + my value` — that is a candidate for the answer, so I **record** it in a variable outside the recursion. The best single **arm** going down from here is `my value + max(left arm, right arm)` — and that is what I **return**, because my parent can only extend a path that arrives through me on one side. My parent cannot use the bent number: a path that turns around at me has already spent both of my children, so it cannot also continue upward.

Confusing those two is the classic failure — it lets a parent build on a path that has already turned around, so you get routes that revisit nodes, and the answer is too large with no error.

Then the part that negative values force. I **floor each arm at zero**: `max(0, arm)`. I am never obliged to take a branch, so if the best the left subtree offers is negative, I take nothing and that contributes zero. Without it, one very negative node poisons every path through its parent — on `[5, -100, 3]` you get `-92` instead of `8`. It is Kadane’s rule, on two branches instead of one.

And the trap that the floor creates: `best` must start at **negative infinity**, not zero. The floor says an *arm* may contribute nothing; it does not say the *answer* may be nothing, because a path must contain at least one node. On a tree of all-negative values the answer is the least-negative single node, and initialising to zero returns zero, which is not a path.

That is nine lines. `O(n)` time — one visit per node, and each arm computed once — and `O(height)` space for the stack, so about twenty frames on a balanced million-node tree and a million on a chain, which is the case where I would want an explicit stack instead.”

9 Recall card

- Pin down “path” first: root-to-leaf (state goes DOWN as an argument, pre-order, no undo) or any-to-any (arms come UP as return values, postorder, the record/return split). They are different problems, and root-to-leaf has **no floor** because you cannot decline the only branch.
- **Every path has exactly one highest node**, so evaluate every node as the turning point. Then keep two quantities apart: **RECORD** `left + right + val` (the bend — the answer candidate) and **RETURN** `val + max(left, right)` (one arm — all the parent can extend). Merging them lets a parent build on a path that has already turned around: too large, no error.
- **Floor each arm:** `max(0, arm)` — you are never obliged to take a branch, so a negative arm contributes nothing. Without it, `[5, -100, 3]` gives **−92 instead of 8**. This is Kadane’s rule on two branches.
- **But initialise** `best` **to** `-inf`, **not 0** — the floor applies to the **arms**, not the **answer**. A path must contain a node, so `[-3]` answers **−3**, and starting at 0 answers 0, which is not a path.
- **A leaf is a node with BOTH children** `None` — `node is None` makes a one-sided node look like a leaf. And **path sum III is not this shape**: it is **prefix sums with a hash map** on the current root-to-node path (`seen[0] = 1`, `count += seen[running - target]`), with `seen[running] -= 1` **on the way out** — which keeps it `O(n)` time, `O(height)` space, and without it you silently count paths that bend.

1 What this is, and why they ask it

Replication means keeping a complete copy of your data on more than one machine, and keeping the copies in step as writes arrive.

Three sentences. The standard arrangement is **leader-follower**: one machine accepts all the writes, and sends a stream of changes to the others, which apply them in the same order. It exists for two reasons that are worth separating — **surviving a machine failure**, which is what today is about, and **serving more reads**, which is [tomorrow](#). And the entire difficulty is concentrated in one moment: **the leader dies, and something has to decide which follower takes over, using incomplete information**.

They ask it because “add a replica” sounds like a checkbox and is not. The interesting questions are: does the leader wait for the follower before confirming a write, and what does the user lose if it does not? How long is the system unable to accept writes while a new leader is chosen? And what happens if the old leader comes back and still thinks it is in charge? A candidate who can answer those three has actually thought about it.

2 The story

The two brothers ran the same business from two towns, and it had worked for nineteen years.

Kuppusamy was in Erode and he kept the book. Every purchase, every sale, every advance — all of it went into the ledger on his desk, in order, numbered.

Selvaraj was in Salem. He did not keep his own book. What he had was a copy, and the way it stayed a copy was that Kuppusamy read the new entries down the phone every evening at about eight, and Selvaraj wrote them into his own ledger in exactly the same order with exactly the same numbers.

That was the whole system. One book that things were written into, and one book that only ever received.

The rule about the order was strict, and Selvaraj had learned why the hard way in his second year. He had once written entry 4,112 before 4,111 because 4,111 was long and he wanted to get the short one down first. Two months later there was a dispute about which of two payments had come first, and his book said one thing and Erode's said another. After that he wrote them in order or not at all.

There was one thing they argued about for years and never fully settled.

When a customer stood in front of Kuppusamy in Erode with cash, did he have to ring Salem and wait for Selvaraj to write it down before giving the receipt?

If he did, the customer waited — sometimes two minutes, sometimes twenty if Selvaraj was out. If he did not, the customer was served immediately and there was a window, up to a day long, in which Erode knew about a payment that Salem did not.

They settled on doing it the fast way for ordinary sales and the slow way for anything over fifty thousand rupees. Wait for the confirmation when the amount is large enough that losing it would matter.

In 2011 Kuppusamy had a heart attack on a Tuesday morning.

Selvaraj had to take over the book, and he discovered what the fast way had cost. The last call had been Monday evening. Everything Kuppusamy had written on Tuesday morning — nine entries — was in a ledger in a locked shop in Erode, and Selvaraj had no idea what was in it. He had to start writing from where his own copy ended, and reconcile the nine entries later from receipts and memory.

And then the part nobody had planned for. Kuppusamy came home from hospital eleven days later, sat down at his desk, and carried on writing in his book — because as far as he was concerned it was his book. For two days there were two people writing entries with the same numbers into two different ledgers, and untangling that took a month.

The rule they wrote down afterwards, and taped inside both ledgers, was one line: **there is only ever one book being written into, and the way you know it is yours is that the other person has said so out loud.**

3 The idea in plain English

The brothers have built leader-follower replication, and every hard part of it happened to them.

- Kuppusamy's ledger is the **leader** (also *primary*, or historically *master*). It is the only place writes go.

- Selvaraj's copy is a **follower** (*replica, standby, historically slave*).
- The evening phone call is the **replication stream**.
- Writing entries in exactly the same order is the fundamental requirement: **replication is an ordered log of changes**, not a copy of the data.
- Waiting for Salem before giving the receipt is **synchronous replication**; not waiting is **asynchronous**.
- The nine lost Tuesday entries are **data loss on failover**, measured as **RPO**.
- Two people writing into two ledgers is **split-brain**, and it is the worst failure in the subject.

What actually gets replicated

Not the data — **the changes, in order**. Three ways, and the differences matter:

Statement replication. Send the SQL: `UPDATE accounts SET balance = balance - 100 WHERE id = 7.`

Compact, and **dangerous**: any statement that is not deterministic gives a different result on the follower. `NOW()`, `RAND()`, an auto-increment race — each one silently makes the copy diverge. MySQL supported this and moved away from it.

Row-based replication. Send the resulting rows: *“row 7's balance changed from 500 to 400”*.

Bigger on the wire, and safe, because there is nothing to re-evaluate. **This is the modern default.**

Write-ahead-log shipping. Send the database's own internal change log, byte for byte. This is what PostgreSQL streaming replication does. Extremely efficient and tightly coupled: the follower must run the same version of the same engine, which makes zero-downtime upgrades harder.

The point to make: replication ships an ordered stream of changes, and the order is not negotiable. Selvaraj writing 4,112 before 4,111.

Synchronous, asynchronous, and the one in between

This is the decision, and it is a genuine trade rather than a best practice.

ASYNCHRONOUS

- leader writes, confirms to the client immediately, sends to followers whenever
- + fast writes; a slow or dead follower never affects the client
- a leader crash LOSES whatever had not been sent yet

SYNCHRONOUS

- leader writes, waits for the follower to confirm, then confirms to the client
- + no data loss on failover – the follower has everything
- every write pays the round trip; and if the follower is down, WRITES STOP

SEMI-SYNCHRONOUS

- wait for ONE follower out of several; the rest are asynchronous
- the usual real answer

Fully synchronous replication to all followers is almost never used, and it is worth saying why: it converts every follower into a single point of failure for writes. One slow replica and the whole system stops accepting writes. **Semi-synchronous — wait for one, let the others lag — keeps the durability guarantee and keeps one bad machine from stopping everything.**

The brothers' compromise is exactly semi-synchronous applied selectively: fast for ordinary sales, wait for confirmation above fifty thousand rupees.

Failover: the hard part

The leader dies. Five things must happen, and each has a way of going wrong.

1. Detect it. Usually a timeout: no heartbeat for `n` seconds. Too short and a slow moment triggers an unnecessary failover; too long and you are down for that whole period.

2. Choose a new leader. Pick the follower with the most complete log. This needs agreement between the surviving machines — a consensus problem, which is [day 119](#).

3. Promote it. It stops following and starts accepting writes.

4. Redirect the clients. They were configured to write somewhere; that address must now mean the new machine. Usually a virtual address, a DNS change, or a proxy layer.

5. Prevent the old leader from writing. This is the one people forget, and it is the one that causes real disasters.

Split-brain, and fencing

Kuppusamy coming home and carrying on writing.

If the old leader is not actually dead — just unreachable, or paused, or on the far side of a network problem — it may still be accepting writes from clients that can still see it. Now there are two leaders, both accepting writes, both assigning the same identifiers. **Every write to the loser is lost, and the data cannot be automatically reconciled.**

The defences:

- **Fencing** (also called STONITH — “shoot the other node in the head”): the new leader forcibly disables the old one — cutting power, revoking its storage lease, or blocking it at the network level — before accepting a single write.
- **A quorum**: a leader may only accept writes while it can see a majority of the cluster. A partitioned old leader cannot see a majority, so it stops on its own. This is why odd cluster sizes matter.
- **Epoch or term numbers**: every leadership period has an increasing number, and followers reject anything stamped with an older one. The old leader’s writes are simply refused.

“There is only ever one book being written into, and you know it is yours because the others said so.” That is a quorum, in one sentence.

The two numbers that describe a failure

```
RPO - Recovery Point Objective
    how much data you are willing to lose, in time
    = the replication lag at the moment of the crash
    synchronous: 0.  asynchronous: seconds, or whatever the lag was.

RTO - Recovery Time Objective
    how long you are willing to be unavailable
    = detection + election + promotion + client redirection
    typically 10-60 seconds automated; minutes to hours manual.
```

Give both numbers when you describe a failover design. “RPO of a few seconds, RTO of about thirty” is a complete answer; “we have a replica” is not.

Other topologies, briefly

Multi-leader. Two or more machines accept writes, usually one per region. Removes the write bottleneck and the cross-region latency — and introduces **write conflicts**, because two leaders can modify the same row at the same time and there is no natural order. You then need conflict resolution: last-write-wins (which loses data), application-defined merges, or CRDTs. **Only worth it for multi-region writes or offline-capable clients.**

Leaderless. No leader at all; clients write to several nodes and read from several. This is Dynamo, Cassandra and Riak, and it replaces “who is the leader” with quorum arithmetic — $R + W > N$ — which is [day 117](#).

Say the default is leader-follower and name when you would leave it.

4 The picture

The standard arrangement.

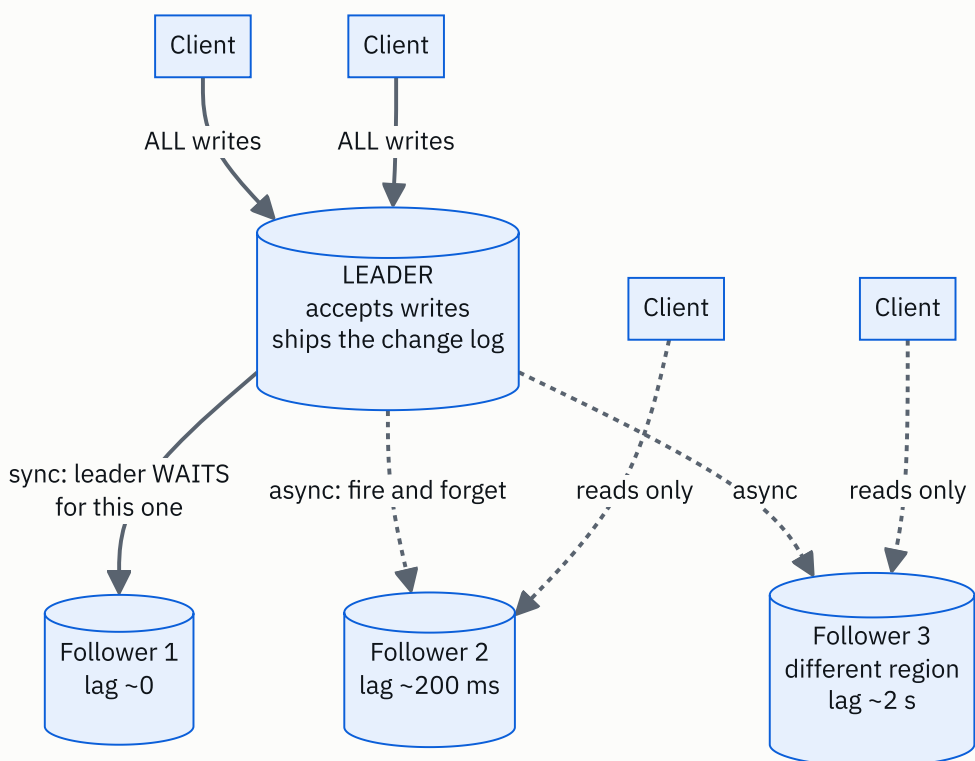


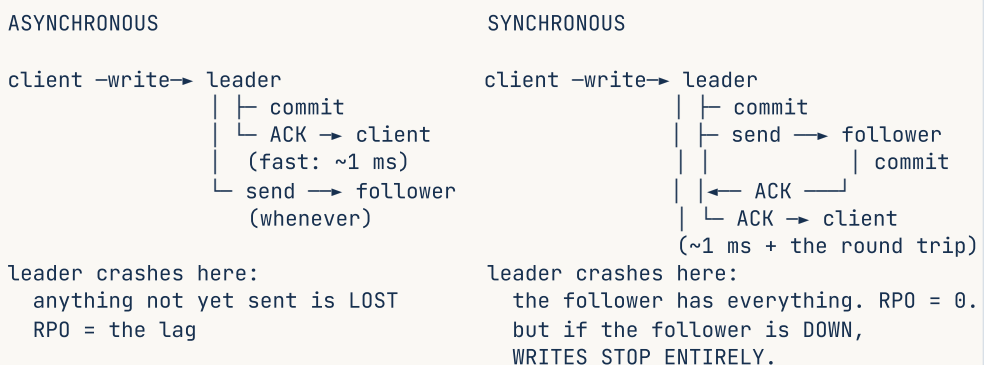
FIGURE 104.1

What to notice: **only one machine has an inbound write arrow.** That is the definition. And exactly one follower is synchronous — semi-synchronous — so a durability guarantee exists without any single follower being able to stop writes.

The three replication formats:

STATEMENT	"UPDATE t SET ts = NOW() WHERE id = 7" small on the wire DANGEROUS: NOW(), RAND(), auto-increment races → the follower computes a DIFFERENT value. Silent divergence.
ROW-BASED	"row id=7: ts 10:00:01 → 10:00:04" bigger on the wire SAFE: nothing is re-evaluated → the modern default
WAL / LOG SHIPPING	the engine's own byte-level change log smallest and fastest tightly coupled: same engine, same version → PostgreSQL streaming replication

Synchronous against asynchronous, on the clock:



The failover, with the split-brain danger drawn in:

```

t=0    leader is healthy, follower lag 200 ms
t=10   ** leader stops responding ** (crashed? or just unreachable?)
t=15   no heartbeat for 5 s → failure suspected
t=17   surviving nodes agree on the follower with the most complete log
t=18   ** FENCE the old leader ** ← the step people skip
t=19   promote: the follower starts accepting writes
t=21   clients redirected (virtual IP / DNS / proxy)
        |----- RT0: ~21 s of no writes -----|
        RPO: the 200 ms of writes that never reached the follower

```

WITHOUT the fence at t=18:

the old leader was only unreachable, not dead.

Some clients can still see it. It keeps accepting writes.

→ TWO leaders, both assigning the same ids, for as long as nobody notices.

→ every write to the loser is lost, and reconciliation is manual.

5 How it actually works

Setting up a follower

1. take a consistent snapshot of the leader, noting the exact log position
2. copy that snapshot to the new machine
3. the follower connects and asks for everything since that position
4. it applies the backlog until it catches up
5. from then on it applies changes as they arrive

Step 1 is the interesting one: the snapshot must be tied to a precise position in the change stream, or the follower does not know where to resume, and applying the wrong range either loses changes or applies them twice. Every database has a mechanism for this (PostgreSQL's `pg_basebackup` with a replication slot, MySQL's binlog coordinates).

Step 4 is the one people forget in an interview: a new follower for a 2 TB database is not available in seconds. Copying takes hours, and then it must catch up on everything that happened during the copy.

How the follower applies changes

The follower has a single-threaded “apply” process in the simplest designs, and that is a common source of lag: the leader wrote with fifty concurrent connections, and the follower replays it with one.

Modern versions parallelise this — replaying non-conflicting transactions concurrently — but **the ordering guarantee limits how much parallelism is safe**, and a write-heavy leader can produce a stream a single follower cannot keep up with.

Failover, automated versus manual

Manual — a human decides. Slow (minutes to hours), and it cannot split-brain, because a person checks. Still used for small systems where an hour of downtime is survivable.

Automated — the cluster decides. Fast (tens of seconds), and it can be wrong: a network hiccup that makes a healthy leader unreachable triggers an unnecessary failover, and if fencing is imperfect you get two leaders.

The honest trade to state: automatic failover trades a rare catastrophic failure for a frequent minor one. Most systems take that trade and add fencing.

What real systems do

- **PostgreSQL** — streaming replication of the write-ahead log; `synchronous_commit` chooses the durability level; **Patroni** plus **etcd** is the standard automated-failover stack, using etcd's consensus for leader election and fencing.
- **MySQL** — binary-log replication, now row-based by default; **Group Replication** and **Orchestrator** handle failover; **semi-synchronous** is a first-class setting.
- **Amazon RDS Multi-AZ** — a synchronous standby in another availability zone, with automatic failover in typically 60–120 seconds. **Aurora** replicates at the storage layer instead, so failover is much faster and the replicas share one storage volume.
- **MongoDB** — replica sets with an explicit election protocol and a majority requirement, which is precisely the quorum defence against split-brain.
- **Redis** — asynchronous replication and **Sentinel** for failover. Redis replication is asynchronous even in the failover path, which is why Redis is not a safe place for data you cannot lose.

6 The numbers

Lag, in practice

same data centre, moderate write load	1 - 50 ms
same data centre, heavy write load	100 ms - a few seconds
cross-region (India to US)	200 ms + apply time
during a bulk load or schema change	seconds to MINUTES

The number that matters is not the average, it is the maximum during a bad moment. A batch job that writes ten million rows can push a follower minutes behind, and that is exactly when a failover would be most expensive.

RPO: what asynchronous replication costs

write rate	1,000 writes/second
replication lag	200 ms
→ writes at risk	$1,000 \times 0.2 = 200$ writes
lag of 5 seconds during a bulk job	
→ writes at risk	5,000

Two hundred lost writes on a normal day; five thousand during a batch job. That is the concrete answer to “what does asynchronous cost”, and it is far more useful than “you might lose some data”.

RTO: what a failover costs

detection (heartbeat timeout)	5 - 30 s
election / agreement	1 - 5 s
fencing the old leader	1 - 5 s
promotion	1 - 10 s
client redirection (DNS TTL!)	1 - 60 s

total, automated	10 - 90 s
total, manual	5 - 60 minutes

Client redirection is the sneaky one. If clients resolve the leader through DNS with a 60-second TTL, your RTO is at least 60 seconds however fast the database part was. That is why production systems use a proxy or a virtual IP rather than DNS for this.

The cost of synchronous replication

write to the leader, same AZ, no replication	~1 ms	
+ synchronous follower in the same AZ	~1.5 ms	(+50%)
+ synchronous follower in another AZ	~3 ms	(+200%)
+ synchronous follower in another REGION	~200 ms	(+20,000%)

Cross-region synchronous replication is essentially never done, and that number is why: every write would take a fifth of a second. The standard arrangement is synchronous within a region and asynchronous across regions, which gives zero data loss for a machine failure and some loss for a whole-region failure.

Rebuilding a follower

database size	2 TB
copy over a 1 Gbit/s link	$2 \text{ TB} \div 125 \text{ MB/s} \approx 4.4 \text{ hours}$
plus catch-up on ~4.4 hours of accumulated changes	
→ a new follower is available in ~5-6 hours, not minutes	

Say this if someone suggests “just add a replica” during an incident. It is not a fix for a problem you are having now.

How many followers

1 leader + 1 sync follower	survives one machine failure, RPO 0
1 leader + 1 sync + 1 async	adds a read replica or a cross-region copy
1 leader + 2 sync	survives two failures; writes wait for BOTH
majority quorum of 3 or 5	survives 1 or 2 failures with automatic election

Odd numbers, because a majority of an even number is the same as a majority of the odd number below it — four nodes tolerate the same single failure that three do, and cost more.

7 The trade-offs

Synchronous or asynchronous?

Asynchronous is fast and loses data on failover — the lag, in writes. **Synchronous** loses nothing and makes every write wait, and turns a follower into a single point of failure for writes.

Take semi-synchronous: one synchronous follower, the rest asynchronous. You get the durability guarantee from the one, and no single follower can stop writes because the others do not block.

I would use fully asynchronous if the data is genuinely tolerant of loss — analytics events, caches, session data — and the write rate is high enough that the round trip matters. **I would use synchronous across all followers essentially never**, and I would say why.

Automatic or manual failover?

Automatic gives an RTO of tens of seconds and occasionally fails over when it should not — a network blip, a garbage-collection pause, a slow disk. Each unnecessary failover is a small outage and a risk of split-brain.

Manual cannot split-brain and takes minutes to hours.

Take automatic, with fencing. And say the honest framing: **you are trading a rare catastrophe for a frequent inconvenience**, which is usually the right trade but is a trade.

Replication is not a backup

This deserves its own line, because it is a real and expensive misunderstanding.

Replication copies mistakes faithfully and instantly. A `DELETE` without a `WHERE` clause is replicated to every follower in milliseconds. So is a bad migration, and so is application-level corruption.

You need both: replication for machine failure, and point-in-time backups for human failure. A follower with a deliberate delay — an hour behind — is a cheap partial defence, because it gives you an hour to notice and stop it.

Where this design breaks

- **The leader is still one machine for writes.** Replication does nothing for write throughput. When one machine cannot take the write rate, the answer is sharding, which is [day 106](#).
- **Followers can fall behind without limit.** A write burst, a long transaction, a schema change — and now failover means losing minutes rather than milliseconds. Monitor lag as a first-class metric and alert on it.
- **Cross-region replication cannot be synchronous.** Physics, not engineering — the same [day 097](#) number. Accept regional data loss, or accept 200 ms writes.
- **Schema changes are the classic replication outage.** A migration that locks a table on the leader blocks the replication stream, followers fall minutes behind, and now you cannot fail over either.

8 In the interview

How it gets asked

- The direct one: *“How do you make the database survive a machine failure?”*
- The trade: *“Synchronous or asynchronous replication? Why?”*
- The hard one: *“The leader dies. Walk me through exactly what happens.”*
- The nasty one: *“The old leader comes back. Then what?”*
- The trap: *“So you don’t need backups, right?”*

What to say out loud, in the first ninety seconds

1. **Name the topology and the invariant.** “Leader-follower: one machine takes all the writes and ships an ordered log of changes to the others. The invariant is that there is exactly one machine accepting writes at any moment.”
2. **Say what is replicated.** “The changes, in order — not the data. Row-based by default, because statement-based replication breaks on anything non-deterministic like `NOW()` or `RAND()`.”
3. **Make the durability choice explicit and pick the middle.** “Semi-synchronous: the leader waits for one follower before confirming, and the rest are asynchronous. That gives an RPO of zero for a single machine failure without letting any one follower block writes.”
4. **Give both failure numbers.** “RPO — how much data I lose — is the replication lag, so a few hundred writes at a thousand a second and 200 milliseconds of lag. RTO — how long I cannot write — is detection plus election plus promotion plus

client redirection, so ten to ninety seconds automated.”

5. **Go straight to fencing.** “The step people skip is fencing the old leader before promoting. If it was only unreachable rather than dead, you get two leaders, both accepting writes, and that data cannot be automatically reconciled.”
6. **Separate replication from backup.** “Replication is not a backup. A `DELETE` with no `WHERE` is copied to every follower in milliseconds.”

The follow-ups

“**Synchronous or asynchronous?**” “Semi-synchronous, which is the answer that is actually used. **Asynchronous** means the leader confirms immediately and ships changes whenever — writes stay fast and a slow follower never affects the client, but a leader crash loses everything not yet sent. At a thousand writes a second with two hundred milliseconds of lag, that is about two hundred writes, and during a bulk job when the lag is five seconds it is five thousand. **Fully synchronous** loses nothing and costs a round trip on every write — and, worse, it makes every follower a single point of failure for writes: one slow replica and the system stops accepting writes entirely. So: **wait for one follower, let the others lag.** Zero data loss for a single machine failure, and no individual follower can stop the system. And across regions I would never do it synchronously — a 200-millisecond round trip on every write is not a database, it is a queue.”

“**The leader dies. Walk me through it.**” “Five steps, and the interesting failures are in the last two. **Detect** — no heartbeat for some timeout, and that timeout is a real trade: too short and a slow moment causes an unnecessary failover, too long and that is dead time. **Elect** — pick the follower with the most complete log, which requires the surviving nodes to agree, so it is a consensus problem. **Fence the old leader** — forcibly stop it from writing, whether by cutting its power, revoking its storage lease, or blocking it at the network. **Promote** the chosen follower. **Redirect the clients**, which is usually a virtual IP or a proxy — and I would avoid DNS here, because a sixty-second TTL means an RTO of at least sixty seconds no matter how fast everything else was. All in, ten to ninety seconds automated. The data lost is whatever had not reached the new leader — the lag at the moment of the crash, which is zero if it was the synchronous follower.”

“**The old leader comes back. Then what?**” “That is split-brain, and it is the worst failure in this topic because it cannot be automatically repaired. If the old leader was only *unreachable* rather than dead — a network partition, a long garbage-collection pause, a hung disk — it may still have clients that can see it, and it will happily keep accepting writes. Now two machines are assigning the same identifiers to different data, and when they reconnect there is no correct merge; somebody has to decide by

hand which writes to discard. Three defences and I would use at least two. **Fencing** — the new leader forcibly disables the old one before accepting a single write. **A quorum** — a leader may only accept writes while it can see a majority of the cluster, so a partitioned old leader stops on its own, which is why cluster sizes are odd. And **epoch numbers** — every leadership term has an increasing number and followers reject anything stamped with an older one, so the old leader's writes are simply refused."

"Does this help with read load?" "Yes, and that is the second reason to have replicas — but it is a separate conversation with its own problem, which is replication lag. If reads go to a follower, a user who writes and immediately reads may not see their own write, because the follower is a few hundred milliseconds behind. That is a very real product bug — 'I posted a comment and it is not there' — and the fixes are things like routing a user's reads to the leader for a short window after they write. I would keep that separate from today's question, which is about surviving a failure rather than serving more reads."

"So you don't need backups?" "No — and this is worth being blunt about, because it is an expensive misunderstanding. **Replication copies mistakes faithfully and instantly.** A **DELETE** without a **WHERE** clause is on every follower in milliseconds. So is a bad migration, and so is application-level corruption that writes wrong values. Replication protects against **machine** failure; backups protect against **human and software** failure, and they are different threats. I would want point-in-time recovery — a base backup plus the change log, so I can restore to a moment just before the mistake. A cheap partial defence worth mentioning is a **deliberately delayed follower**, kept an hour behind, which gives you an hour to notice a catastrophic statement and stop it before it is applied."

"How long does it take to add a new follower?" "Not minutes, which is worth knowing before you suggest it during an incident. You take a consistent snapshot of the leader tied to an exact position in the change log, copy it, and then replay everything since that position. For a two-terabyte database over a gigabit link, the copy alone is about four and a half hours, and then it has to catch up on the four and a half hours of writes that happened meanwhile. So five to six hours before it is useful. Which means replicas are capacity you provision in advance, not a lever you pull under load."

A model answer

Asked: how do you make the database survive a machine failure?

“Replication, in a leader-follower arrangement: one machine accepts all writes, and ships an ordered stream of changes to one or more followers that apply them in the same order. The invariant that makes the whole thing work is that **exactly one machine is accepting writes at any moment** — and most of the difficulty in this topic is in enforcing that during a failure.

What travels is the **changes, in order** — not the data. I would use row-based replication, because statement-based replication ships the SQL and anything non-deterministic — `NOW()`, `RAND()`, an auto-increment race — evaluates differently on the follower and silently diverges. PostgreSQL ships its write-ahead log, which is the same idea at a lower level.

The decision to make explicit is **durability**. Asynchronous means the leader confirms to the client immediately and ships changes whenever — fast, and a crash loses whatever had not been sent. At a thousand writes a second with two hundred milliseconds of lag, that is roughly two hundred writes gone, and during a bulk job when lag hits five seconds it is five thousand. Fully synchronous loses nothing and makes every write wait for the follower — and it turns each follower into a single point of failure for writes, so one slow replica stops the system. So I would use **semi-synchronous**: wait for one follower, let the rest lag. Zero data loss for a single machine failure, and no individual follower can block writes.

When the leader dies, five things happen: detect it by heartbeat timeout; agree on the follower with the most complete log; **fence the old leader**; promote the new one; and redirect the clients. Two of those are worth dwelling on. **Client redirection** is often the slowest part — if clients find the leader through DNS with a sixty-second TTL, the RTO is at least sixty seconds regardless of how fast the database was, so I would use a proxy or a virtual IP. And **fencing** is the step people skip, and it is the one that causes disasters: if the old leader was merely unreachable rather than dead, it keeps accepting writes from clients that can still see it, and you have two leaders assigning the same identifiers to different data with no automatic way to reconcile them. The defences are fencing, a quorum requirement so a partitioned leader stops on its own, and epoch numbers so followers reject writes from a stale term.

I would quote two numbers for any failover design: **RPO**, how much data is lost, which is the replication lag at the moment of the crash; and **RTO**, how long writes are unavailable, which is ten to ninety seconds automated and minutes to hours manual.

Two limits worth stating up front. This does **nothing for write throughput** — there is still one machine taking writes, and when that is the constraint the answer is sharding. And **replication is not a backup**: a `DELETE` with no `WHERE` reaches every follower in milliseconds. Machine failure is replication's job; human error is what point-in-time backups are for, and a deliberately delayed follower an hour behind is a cheap partial defence against exactly that.”

9 Recall card

- **Leader-follower: exactly ONE machine accepts writes and ships an ORDERED log of changes.** Order is not negotiable. **Row-based** replication by default — **statement-based breaks on anything non-deterministic** (`NOW()`, `RAND()`, auto-increment races) and diverges silently. PostgreSQL ships the WAL.
- **Semi-synchronous is the real answer: wait for ONE follower, let the rest lag.** Asynchronous loses the lag on a crash (~200 writes at 1,000/s and 200 ms lag; 5,000 during a bulk job). Fully synchronous loses nothing but makes **every follower a single point of failure for writes**. Cross-region synchronous is ~200 ms per write — never.
- **Failover is five steps: detect · elect · FENCE · promote · redirect.** Quote **RPO** (data lost = the lag) and **RTO** (10–90 s automated, minutes to hours manual). **Client redirection through DNS makes RTO ≥ the TTL** — use a proxy or virtual IP.
- **Split-brain is the worst failure and it cannot be auto-repaired:** an old leader that was only *unreachable* keeps accepting writes, and two leaders assign the same ids to different data. Defend with **fencing**, a **quorum** (leader must see a majority — hence odd cluster sizes), and **epoch numbers** so followers reject a stale term.
- **Replication is NOT a backup** — a `DELETE` with no `WHERE` reaches every follower in milliseconds. Machine failure → replication; human error → **point-in-time backups**, plus a **deliberately delayed follower** as a cheap partial defence. And replication does **nothing for write throughput** — that is sharding. Adding a follower to a 2 TB database takes ~5–6 hours, so it is not a lever you pull during an incident.

Practice — Path problems, and the return-value trick

DSA topic: Path problems, and the return-value trick **System design topic:** Database replication

Code these, in this order

One rule for the whole set: **before writing anything, say which kind of path is meant** — root-to-leaf, strictly downward, or any-to-any with a bend. Three different problems, three different shapes, and the question rarely says which.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Path Sum	LeetCode 112 (Easy)	The leaf test, and state carried down as an argument.
2	Binary Tree Paths	LeetCode 257 (Easy)	Collecting paths, and why passing a string needs no undo.
3	Path Sum III	LeetCode 437 (Medium)	Prefix sums on a tree, and the un-choose on the map.
4	Binary Tree Maximum Path Sum	LeetCode 124 (Hard)	Record versus return, the floor, and the <code>-inf</code> start.

On problem 1, use the wrong leaf test first

Write the base case as `if node is None: return remaining == 0`. Run it on `[1, 2]` with target 1 and record the answer. Then say what a leaf actually is.

On problem 3, write both versions and time them

Write the naive start-from-every-node version, then the prefix-sum one. Time both on a chain of 2,000 nodes and record the ratio. Then delete `seen[running] -= 1` and say what the count does and why.

On problem 4, break it three ways

Run each and record: return the bent value to the parent; drop the `max(0, ...)`; start `best` at 0. Say which inputs expose each one, and whether any of them raises.

After all four, fill in the table

For each problem: what is returned to the parent, what is recorded on the side, and whether there is a floor. If you can produce that from memory, you have the technique.

The definition drill

1. Name the three meanings of “path” and give a LeetCode number for each.
2. Say which traversal each one uses and which direction information travels.
3. Say which one has no floor, and why not.
4. Write the question you would ask the interviewer, in one sentence.

The leaf drill

1. Write the correct leaf test.
2. Say what `node is None` reports on a one-sided node.
3. Give the input and target where the wrong test differs, with both answers.
4. Say why this trap is specific to root-to-leaf problems.

The record-versus-return drill

1. Write the two lines and label them.
2. Say the sentence explaining why the parent cannot use the recorded value.
3. Merge them into one return and describe how the answer misbehaves.
4. Fill in the family table for five problems.

The floor drill

1. Write the floor and say what it means in one sentence.
2. Run `[5, -100, 3]` with and without it and record both answers.
3. Say which array algorithm this is the same idea as.
4. Say why root-to-leaf problems must not have it.

The `-inf` drill

1. Say what the floor applies to and what it does not.
2. Run `[-3]` with `best = 0` and record the answer.
3. Say why 0 is not a valid answer for that tree.
4. State the rule in one line.

The prefix-sum drill

1. Write the path-sum-III solution from memory.
2. Say what `seen[0] = 1` is for, and which earlier day it comes from.
3. Say what the un-choose keeps out of the map, and why that matters.
4. Say what the map's size is bounded by, and why it is not `O(n)`.
5. Give the complexity of both versions on a balanced tree and on a chain.

The break-it drill

Trigger each and record the exact output:

1. `node is None` as the leaf test, on a one-sided tree.
2. Returning the bent value to the parent.
3. No floor, on `[5, -100, 3]`.
4. `best = 0`, on an all-negative tree.
5. Missing `seen[running] -= 1`.
6. Missing `seen[0] = 1`.
7. String concatenation down a 5,000-node chain, timed.

The extension drill

1. Modify max path sum to return the path.
 2. State what the space becomes and why.
 3. Describe the two-pass alternative that keeps `O(height)`.
 4. Say what `all_path_sums` costs in output size for a perfect million-node tree.
-

The topology drill

1. Draw leader-follower and state the invariant in one sentence.
2. Say what travels between them, and in what property.
3. Name the three replication formats and give the danger of the first.
4. Name the two other topologies and say when each is worth it.

The durability drill

1. Define asynchronous, synchronous and semi-synchronous.
2. Say what each one loses and what each one costs.
3. Say why fully synchronous makes followers a liability.
4. Compute the writes at risk at 1,000 writes/s with 200 ms and 5 s of lag.

5. Give the latency figures for a synchronous follower in the same AZ, another AZ, another region.

The failover drill

1. Name the five steps in order.
2. Say which step people skip and what it causes.
3. Say which step is often the slowest, and what to use instead.
4. Compute a realistic RTO from the components.
5. Define RPO and RTO in one sentence each, and say what each depends on.

The split-brain drill

1. Describe the failure in two sentences.
2. Say why it cannot be automatically repaired.
3. Name the three defences and say what each one does.
4. Say why cluster sizes are odd.
5. Give the one-sentence rule from the brothers' ledger.

The not-a-backup drill

1. Say what replication protects against and what it does not.
2. Give the specific statement that replication faithfully copies.
3. Name what you need instead, and what it consists of.
4. Describe the delayed-follower trick and what window it buys.

The numbers drill

Compute or state each:

1. Typical lag in the same data centre, cross-region, and during a bulk load.
2. Writes at risk for two different lag figures.
3. A realistic automated RTO, broken down by component.
4. Time to build a new follower for a 2 TB database over 1 Gbit/s.
5. Why four nodes tolerate no more failures than three.

The failure drill

For each, say what happens and what you would add:

1. The synchronous follower goes down.
2. A schema migration locks a table on the leader.

3. Clients find the leader through DNS with a 60-second TTL.
4. A batch job writes ten million rows.
5. Someone runs `DELETE FROM orders` with no `WHERE`.
6. Someone suggests adding a replica to fix an ongoing overload.

Two of the six are not solved by replication at all. Name them and say what solves them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the maximum path sum in the binary tree.* The definition of path pinned down first, the one-highest-node observation, record versus return said precisely with the reason the parent cannot use the bend, the floor with its Kadane connection, the `-inf` initialisation with the reason, and both complexities.
2. *How do you make the database survive a machine failure?* Leader-follower with the one-writer invariant, an ordered change log in row format, semi-synchronous with the writes-at-risk arithmetic, the five failover steps with fencing named, RPO and RTO given as numbers, and replication distinguished from backup.
3. *The old leader comes back. Then what?* Split-brain described, why it cannot be auto-repaired, and the three defences with what each one does.

Before you move on

- ☐ I ask which kind of path is meant before writing anything.
- ☐ I can write the correct leaf test and say what the wrong one does.
- ☐ I can say the record-versus-return sentence precisely.
- ☐ I know why the parent cannot use the recorded value.
- ☐ I can write the floor and say what it means in plain words.
- ☐ I know that the floor applies to the arms and not the answer.
- ☐ I ran `[-3]` with `best = 0` and know what it returns.
- ☐ I can write path sum III with prefix sums from memory.
- ☐ I know what the un-choose on the map keeps out, and the bound it gives.
- ☐ I can fill in the family table for five problems.

- [] I can state the leader-follower invariant in one sentence.
- [] I can name the three replication formats and the danger of statement-based.
- [] I can define all three durability levels and say which one is actually used.
- [] I can compute writes at risk from a write rate and a lag.
- [] I can name the five failover steps and say which one people skip.
- [] I can define RPO and RTO and give realistic numbers for both.
- [] I can describe split-brain and name three defences.
- [] I can say why replication is not a backup, with the specific example.
- [] I answered all three questions above out loud.

Lowest common ancestor

DATA STRUCTURES AND ALGORITHMS

Lowest common ancestor

You can find the LCA in a general binary tree and in a BST, and know why they differ.

SYSTEM DESIGN

Read replicas and replication lag

You can explain why a user sometimes does not see their own write.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the lowest common ancestor of these two nodes.*
- *The user posted a comment and cannot see it. What happened?*

1 What this is, and why they ask it

The **lowest common ancestor** of two nodes is the deepest node that has both of them somewhere beneath it. “Deepest” is the whole point — the root is always a common ancestor, and it is almost never the answer.

Three sentences. In a **general binary tree** you have to search, because nothing about the values tells you where anything is, and the solution is a postorder walk that returns a node upward and lets the answer emerge at the place where two different children both report success. In a **binary search tree** you do not search at all — the ordering tells you which way to go, so it is a walk down from the root with no recursion and no extra space. And there is a hidden assumption in the standard problem that changes the code completely when it is removed: **both nodes are guaranteed to be in the tree.**

They ask it because the general-tree solution is six lines that are genuinely hard to derive and trivial to recognise, so it separates people who have understood the post-order return pattern from people who have memorised a snippet. The BST version then checks whether you notice that the extra structure makes the search unnecessary — and “*why is your BST answer different?*” is the follow-up that decides the round.

2 The story

The engagement was in the morning and the lunch went on until four, and by two o’clock the two young men sitting near the fan had established that they were somehow related and could not work out how.

One was from the Salem side. The other had come with the bride’s people from Pollachi. They had worked out that both of their families had once been in the same village, and then they had got stuck, because neither of them knew more than three generations back.

Somebody sent them to Ponnamma, who was eighty-six and sitting in the good chair by the window, and who had been asked this question at every function for forty years.

She did not do it the way they had been trying. They had been listing names at each other — my father, his father, his father — and hoping to hit a match, which works but takes a long time when neither person is sure of the spellings.

What she did was ask about the branches.

She named her husband's four brothers, and asked, one at a time: is this boy from your line? Is that boy from your line?

For the eldest brother's line, the answer was no to both. Nothing there — stop, do not go further down it.

For the second brother, one of them was in it. Just one.

For the third brother, the other one was in it. Also just one.

And at that point she stopped and said: then it is my husband's father. Because one of you comes down through one son and the other comes down through a different son, and the first place you are both underneath is the man those two sons belonged to.

Somebody asked what if they had both been under the second brother.

She said then the answer would not be her husband's father at all, it would be somewhere down inside that line, and you would have to go and ask the same question again down there. You keep going down as long as they are both on the same side. The moment they split, you have found it.

And she added the other case, which took her one sentence. If she had asked the four brothers and one of them had said “this boy is my grandson, and the other boy is *me*” — then the answer is that brother himself. A person can be their own ancestor for this purpose. Otherwise you would go looking past him and find nothing.

3 The idea in plain English

Ponnamma has just described the general-tree algorithm exactly, including the case people forget.

- Each brother's line is a **subtree**.
- “Is either of these two in your line?” is the recursive call.
- **Two different children both say yes → this node is the answer.**
- **Only one child says yes → the answer is inside that child**, so pass its answer upward unchanged.

- “The other boy is me” → **a node can be its own ancestor**, and that is the base case people omit.

The general binary tree

```
def lca(node, p, q):
    if node is None or node is p or node is q:
        return node                # found one, or ran out
    left = lca(node.left, p, q)
    right = lca(node.right, p, q)
    if left and right:
        return node                # they SPLIT here – this is the answer
    return left or right          # both on one side, or neither
```

Six lines. Read them as Ponnammma’s questions:

`node is p or node is q`	"the other boy is me"	→ report myself upward
left and right both non-None	"they came down two lines"	→ I am the answer
only one non-None	"both are in that one line"	→ pass its answer up
both None	"neither is under me"	→ report nothing

The subtle part is what the function returns. It is not “the LCA of this subtree”. It is:

“If I have found the answer below me, the answer. Otherwise, whichever of `p` or `q` I have found, if any. Otherwise `None`.”

Three different meanings from one return value. That overloading is why the code is short and why it is hard to derive. **Say the three meanings out loud in the interview** — it is the thing being tested.

Why it is correct

Consider where `p` and `q` sit relative to a node:

both in the left subtree	→ left is non-None, right is None → return left (the answer is deeper)
both in the right subtree	→ mirror image
one in each	→ both non-None → THIS node is the answer
p is this node, q below it	→ return this node immediately; the parent sees one non-None child
neither below	→ None

The fourth case is the ancestor case, and the early return handles it: if `node is p`, we stop and report `p` upward without looking for `q` at all. That is correct precisely *because* both nodes are guaranteed to exist — if `q` is anywhere, it must be below `p`, so `p` is the answer.

The assumption that changes everything

"Both p and q exist in the tree."

Every standard solution relies on it. Remove it and the code above is wrong:

```
tree: 1 → 2 (left child)
lca(tree, node2, node_not_in_tree) → returns node 2

but node 2 is NOT the LCA of anything, because the other node does not exist.
The correct answer is None.
```

The fix is to **count what you actually found**, and check at the end:

```
found = 0
...
if node is p or node is q:
    found += 1
...
return answer if found == 2 else None
```

LeetCode 1644 is exactly this variant, and asking “*can I assume both nodes are present?*” before you start is one of the cheapest good impressions available in this problem.

The binary search tree: no search at all

In a BST everything left of a node is smaller and everything right is larger. So you never have to look for anything — the values tell you where they are.

```
def lca_bst(node, p, q):
    while node:
        if p.val < node.val and q.val < node.val:
            node = node.left          # both smaller: the answer is left
        elif p.val > node.val and q.val > node.val:
            node = node.right         # both larger: the answer is right
        else:
            return node               # they SPLIT here (or one IS here)
```

The split point is the answer. Walk down while both are on the same side; the first node where they diverge — or where one of them is the node — is the LCA.

That `else` branch quietly covers three cases at once: `p < node < q`, `q < node < p`, and `node` being `p` or `q` itself. It is worth pointing out, because it looks like a case has been missed.

```
general tree: 0(n) time, 0(height) space, must visit every node in the worst case
BST:          0(height) time, 0(1) space, iterative, no recursion at all
```

On a balanced BST of a million nodes that is twenty comparisons against a million. The reason for the difference, stated properly: *in a general tree you must search because nothing tells you where the nodes are; in a BST the ordering is a map.*

With parent pointers, it is a different problem entirely

If each node knows its parent, you can walk **upwards** from both nodes, and the problem becomes finding where two paths merge — which is the intersection of two linked lists.

```
method 1: put p's ancestors in a set, walk up from q until you hit one    0(height) time and space
method 2: measure both depths, advance the deeper one, then move together 0(height) time, 0(1) space
method 3: the two-pointer swap — walk up from p, and when you run out,
          restart from q, and vice versa. They meet at the LCA.           0(height) time, 0(1) space
```

Method 3 is the same trick as the linked-list intersection problem from [day 083](#), and recognising that is a strong answer.

If you have to answer many queries

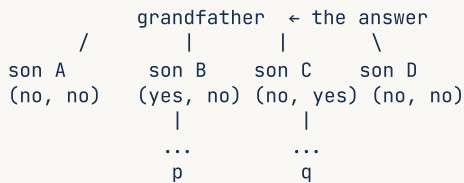
One LCA is $O(n)$. A thousand LCA queries on the same tree is $O(1000n)$, and there is a much better answer, worth naming rather than implementing:

binary lifting	preprocess $O(n \log n)$, each query $O(\log n)$
Euler tour + sparse table (RMQ)	store, for every node, its 2^k -th ancestor preprocess $O(n \log n)$, each query $O(1)$
Tarjan's offline	$O(n \alpha(n))$ for all queries at once, using union-find

“For a single query, the postorder walk. For many queries on a static tree, binary lifting.” That sentence is the complete answer to the follow-up.

4 The picture

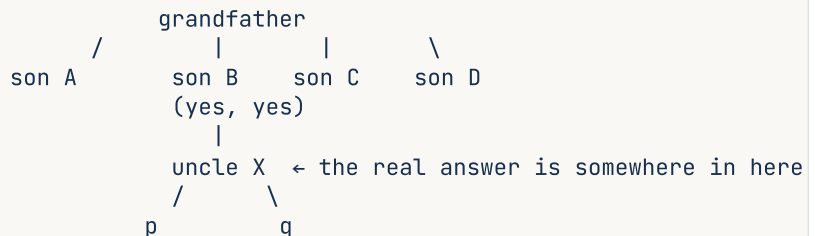
Ponnamma's four questions, drawn.



son A: neither → None
son B: found p → reports p upward
son C: found q → reports q upward
son D: neither → None

at the grandfather: TWO children returned something → the grandfather is the LCA

And the case where the answer is deeper:



only ONE child returned something → pass it upward unchanged,
because the split happened further down, inside son B's line.

The three meanings of the return value:

return value	means
-----	-----
None	neither p nor q is anywhere below me
p (or q)	I found exactly one of them, and no split below me
some other node	that node is the LCA, found below me – pass it up

ONE return value, THREE meanings. That overloading is why the code is six lines and why it is hard to derive from scratch.

The general tree traced, on the standard example:



`lca(3, p=5, q=1):`

`lca(5, ...) → node IS p → return 5` (stop; do not look inside)
`lca(1, ...) → node IS q → return 1`
 both non-None → return 3 **ANSWER: 3**

`lca(3, p=5, q=4):`

`lca(5, ...) → node IS p → return 5` (4 is below, but we stop – correct BECAUSE both are guaranteed to exist)
`lca(1, ...) → None`
 only left non-None → return 5 **ANSWER: 5**

The BST version, which needs no search:



`lca_bst(p=2, q=8):`

at 6: $2 < 6$ and $8 > 6$ → they SPLIT → answer is 6

`lca_bst(p=2, q=4):`

at 6: both < 6 → go left
 at 2: p IS this node → answer is 2

`lca_bst(p=3, q=5):`

at 6: both < 6 → go left
 at 2: both > 2 → go right
 at 4: $3 < 4$ and $5 > 4$ → SPLIT → answer is 4

three comparisons, no recursion, $O(1)$ space.

5 The code, built step by step

Step 1 — ask about the assumption

“Are both nodes guaranteed to be in the tree? And is it a plain binary tree or a search tree? Those change the solution completely.”

Ten seconds, and both answers change the code.

Step 2 — the base case, which is two things at once

```
if node is None or node is p or node is q:  
    return node
```

`None` means “ran out”. `node is p` means “found one — report it upward”. **Combining them into one line is correct and it hides that there are two ideas**, so say both out loud as you write it.

Note `is`, not `=`. The problem gives you node *objects*, and values may repeat.

Step 3 — recurse both sides unconditionally

```
left = lca(node.left, p, q)  
right = lca(node.right, p, q)
```

Both, always. You cannot short-circuit on the left result, because “found something on the left” is exactly the case where you still need to know about the right.

Step 4 — the two-line decision

```
if left and right:  
    return node  
return left or right
```

a split: I am the answer
one side, or neither

Say what each line means. `left and right` is the whole algorithm; the second line is just passing the message up.

Step 5 — for the BST, throw the recursion away

```
while node:  
    if p.val < node.val and q.val < node.val:  
        node = node.left  
    elif p.val > node.val and q.val > node.val:  
        node = node.right  
    else:  
        return node
```

Say the reason for the difference: “in a general tree I have to search, because nothing tells me where the nodes are. In a BST the ordering tells me, so I walk down — $O(\text{height})$ time and $O(1)$ space.”

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

def lca(node: TreeNode | None, p: TreeNode, q: TreeNode) → TreeNode | None:
    """LeetCode 236. General binary tree. Assumes BOTH p and q exist.

    The return value has THREE meanings, and that overloading is the whole
    trick:
        None           neither is below me
        p or q         I found exactly one, with no split below me
        other node     that is the LCA, found below me – pass it upward

    `node is p` returns immediately without looking for q. That is correct
    ONLY because both are guaranteed present: if q exists at all, it must be
    below p, so p is the answer.

    Time O(n), space O(height).
    """
    if node is None or node is p or node is q:
        return node # ran out, or found one
    left = lca(node.left, p, q)
    right = lca(node.right, p, q)
    if left and right:
        return node # they SPLIT here
    return left or right # one side, or neither

def lca_may_not_exist(root: TreeNode | None, p: TreeNode, q: TreeNode
                      ) → TreeNode | None:
    """LeetCode 1644. WITHOUT the guarantee that both nodes are present.

    The plain version returns p when q is absent, which is wrong. So count
    what was actually found and check at the end. Note the count must happen
    AFTER recursing, or an early return skips the other node's tally.
    """
    found = 0

    def walk(node: TreeNode | None) → TreeNode | None:
        nonlocal found
        if node is None:
            return None
        left = walk(node.left)
        right = walk(node.right)
        if node is p or node is q:
            found += 1
            return node # still report myself upward
        if left and right:
            return node
        return left or right

    answer = walk(root)
    return answer if found == 2 else None

def lca_bst(root: TreeNode | None, p: TreeNode, q: TreeNode) → TreeNode | None:
```

"""LeetCode 235. Binary SEARCH tree – no searching required.

Walk down while both targets are on the same side. The first node where they diverge, or where one of them IS the node, is the answer.

The `else` covers three cases at once: $p < \text{node} < q$, $q < \text{node} < p$, and node being p or q itself.

Time $O(\text{height})$ – 20 comparisons on a balanced million-node tree, against $O(n)$ for the general version. Space $O(1)$: no recursion at all.

"""

```
node = root
while node:
    if p.val < node.val and q.val < node.val:
        node = node.left
    elif p.val > node.val and q.val > node.val:
        node = node.right
    else:
        return node                # the split point
return None
```

```
def lca_bst_recursive(node: TreeNode | None, p: TreeNode, q: TreeNode
    ) → TreeNode | None:
    """The same, recursively. Shorter to read,  $O(\text{height})$  space instead of  $O(1)$ ."""
    if node is None:
        return None
    if p.val < node.val and q.val < node.val:
        return lca_bst_recursive(node.left, p, q)
    if p.val > node.val and q.val > node.val:
        return lca_bst_recursive(node.right, p, q)
    return node
```

```
def lca_by_paths(root: TreeNode | None, p: TreeNode, q: TreeNode
    ) → TreeNode | None:
    """The version people invent first: find both root-to-node paths, then
    walk them together until they diverge.

    Correct, easy to explain, and it costs  $O(n)$  EXTRA space for the two
    paths where the postorder version costs only the stack. Worth mentioning
    as the intuition and then replacing.
```

"""

```
def path_to(node: TreeNode | None, target: TreeNode,
    trail: list[TreeNode]) → bool:
    if node is None:
        return False
    trail.append(node)                # choose
    if node is target:
        return True
    if path_to(node.left, target, trail) or path_to(node.right, target, trail):
        return True
    trail.pop()                      # un-choose
    return False
```

```
path_p: list[TreeNode] = []
path_q: list[TreeNode] = []
if not path_to(root, p, path_p) or not path_to(root, q, path_q):
    return None                    # handles absence for free
```

```
answer = None
for a, b in zip(path_p, path_q):
    if a is not b:
        break
    answer = a
return answer
```

```
def lca_with_parents(p: "ParentNode", q: "ParentNode") → "ParentNode | None":
    """LeetCode 1650: nodes carry a parent pointer.
```

This is the LINKED LIST INTERSECTION problem: walk up from p, and when you run out, continue from q – and vice versa. Both pointers travel exactly (depth_p + depth_q) steps, so they arrive together at the merge point.

0(height) time, 0(1) space, and no set.
"""

```
a, b = p, q
while a is not b:
    a = a.parent if a.parent else q
    b = b.parent if b.parent else p
return a
```

```
class ParentNode:
    def __init__(self, val: int) → None:
        self.val = val
        self.left: "ParentNode | None" = None
        self.right: "ParentNode | None" = None
        self.parent: "ParentNode | None" = None
```

```
def lca_deepest_leaves(root: TreeNode | None) → TreeNode | None:
    """LeetCode 1123: the LCA of all the deepest leaves.
```

A different use of the same postorder shape: return (depth, lca) and combine – if the two sides are equally deep, this node is the answer.
"""

```
def walk(node: TreeNode | None) → tuple[int, TreeNode | None]:
    if node is None:
        return 0, None
    ld, la = walk(node.left)
    rd, ra = walk(node.right)
    if ld == rd:
        return ld + 1, node          # equally deep: I am the LCA
    if ld > rd:
        return ld + 1, la
    return rd + 1, ra

return walk(root)[1]
```

```
def find(node: TreeNode | None, value: int) → TreeNode | None:
    if node is None:
        return None
    if node.val == value:
        return node
    return find(node.left, value) or find(node.right, value)
```

```
def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
            i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
            i += 1
    return root
```

```
if __name__ == "__main__":
    tree = from_list([3, 5, 1, 6, 2, 0, 8, None, None, 7, 4])
```

```

p, q = find(tree, 5), find(tree, 1)
print(lca(tree, p, q))                    # TreeNode(3)

p, q = find(tree, 5), find(tree, 4)
print(lca(tree, p, q))                    # TreeNode(5) — a node is its own ancestor

p, q = find(tree, 7), find(tree, 4)
print(lca(tree, p, q))                    # TreeNode(2)

print(lca_by_paths(tree, find(tree, 7), find(tree, 8))) # TreeNode(3)

# the assumption, and what breaks without it
stray = TreeNode(99)
print(lca(tree, find(tree, 5), stray))    # TreeNode(5) ← WRONG
print(lca_may_not_exist(tree, find(tree, 5), stray)) # None ← correct
print(lca_may_not_exist(tree, find(tree, 5), find(tree, 4))) # TreeNode(5)

# the BST version
bst = from_list([6, 2, 8, 0, 4, 7, 9, None, None, 3, 5])
print(lca_bst(bst, find(bst, 2), find(bst, 8)))    # TreeNode(6)
print(lca_bst(bst, find(bst, 2), find(bst, 4)))    # TreeNode(2)
print(lca_bst(bst, find(bst, 3), find(bst, 5)))    # TreeNode(4)
print(lca_bst_recursive(bst, find(bst, 3), find(bst, 5))) # TreeNode(4)

print(lca_deepest_leaves(from_list([3, 5, 1, 6, 2, 0, 8, None, None, 7, 4])))
# TreeNode(2)

# parent pointers: the linked-list intersection trick
a = ParentNode(1); b = ParentNode(2); c = ParentNode(3)
d = ParentNode(4); e = ParentNode(5)
a.left, a.right = b, c
b.parent = c.parent = a
b.left, b.right = d, e
d.parent = e.parent = b
print(lca_with_parents(d, c).val)            # 1
print(lca_with_parents(d, e).val)            # 2

```

6 What it costs

General binary tree

time	$O(n)$	every node may have to be visited
space	$O(\text{height})$	the recursion stack

$O(n)$ and there is no better single-query answer for a general tree, and the reason is worth saying: you cannot rule out any subtree without looking in it, because nothing about a node tells you what is below it.

The best case is much better — if both nodes are near the root, the early return stops immediately — but the worst case is a full traversal.

Binary search tree

```
time    O(height)  → O(log n) balanced, O(n) degenerate
space   O(1)       iterative, no stack
```

```
n = 1,000,000, balanced
  general tree version: 1,000,000 node visits, ~20 stack frames
  BST version:          20 comparisons,    0 extra space

a factor of 50,000 in time and unbounded in space
```

That contrast is the answer to “why is the BST version different?” The ordering is a map; without it you are searching blind.

The path-based version

```
time    O(n)       two searches
space   O(n)       two paths, each up to `height`, plus the stack
```

Correct and easy to explain, and it uses `O(height)` extra space for the paths on top of the stack. **On a degenerate tree that is `O(n)` extra**, where the postorder version uses only the stack. It also handles the “node may not exist” case for free, which is a genuine advantage worth mentioning.

With parent pointers

```
set-based      O(height) time, O(height) space
depth-align    O(height) time, O(1) space
two-pointer swap O(height) time, O(1) space
```

The swap version: each pointer travels `depth_p + depth_q` steps in total, so they arrive at the merge point together. **Same as the linked-list intersection trick**, and it needs no depth computation at all.

Many queries

```
k queries, one at a time          O(k · n)
binary lifting                    O(n log n) preprocess, O(log n) per query
Euler tour + sparse table         O(n log n) preprocess, O(1) per query
Tarjan's offline (union-find)     O((n + k) α(n)) for all of them together
```

```
n = 100,000, k = 100,000
naive:                            10^10 operations          – will not finish
binary lifting:                    ~1.7M preprocess + 1.7M queries = 3.4M – instant
```

Know the names and the trade; you will almost never be asked to implement them.

The absence check

<code>lca</code> (assumes presence)	$O(n)$, one pass
<code>lca_may_not_exist</code>	$O(n)$, one pass, plus a counter – same complexity
<code>lca_by_paths</code>	$O(n)$, and absence detected for free

Handling absence costs nothing asymptotically, which is worth saying: there is no reason to avoid the safer version other than that the question usually promises presence.

7 The traps

Trap 1 — assuming both nodes are present when they are not

```
tree: 1 with a left child 2
lca(tree, node_2, node_not_in_tree) → TreeNode(2)
```

The standard six-line solution returns the node it found, because it stopped as soon as it saw it. There is no error and the answer looks plausible. **Ask about the guarantee, and if it is not there, count what you found.**

Trap 2 — comparing values instead of identity

```
if node.val == p.val:                # WRONG if values can repeat
```

The problem hands you node *objects*. If two nodes share a value, `==` matches the wrong one and the answer is silently wrong. Use `is`. And if the interviewer says values are unique, say that you are relying on it.

Trap 3 — short-circuiting the second recursive call

```
left = lca(node.left, p, q)
if left:
    return left                # WRONG – never looks right
return lca(node.right, p, q)
```

This returns the first node found rather than the ancestor. On the standard tree with `p = 5` and `q = 1` it returns 5 instead of 3. **Both sides must be evaluated**, because “found something on the left” is exactly the case where the right side still matters.

Trap 4 — forgetting that a node is its own ancestor

```
if node is None:
    return None                    # missing the `node is p or node is q` case
```

Then `lca(tree, 5, 4)` where 4 is below 5 returns 4 or `None` rather than 5. **The problem statements say so explicitly** — “a node can be a descendant of itself” — and it is the case people skip when deriving the solution.

Trap 5 — using the BST version on a tree that is not a BST

```
lca_bst(general_tree, p, q)
```

It walks down comparing values, takes a wrong turn immediately, and returns `None` or a wrong node. **No error.** Always confirm which kind of tree you have; it is the difference between `O(log n)` and `O(n)` and between right and wrong.

Trap 6 — the BST version with `<=` instead of `<`

```
if p.val ≤ node.val and q.val ≤ node.val:
    node = node.left                # WRONG when p IS the node
```

If `p` is the current node, this walks past it into the left subtree and loses the answer. The condition must be strict, so that “one of them is this node” falls into the `else`.

Trap 7 — recursion depth

```
lca(chain_of_10000_nodes, p, q)
RecursionError: maximum recursion depth exceeded
```

The general version is `O(height)` deep. The BST version is iterative and immune, which is another point in its favour.

Trap 8 — counting `found` before recursing

```
if node is p or node is q:
    found += 1
    return node                    # returns BEFORE recursing
```

In the absence-checking version, returning early means you never look below this node — so if the *other* target is a descendant, it is never counted, and `found` stays at 1. **Recurse first, then tally**, which is why `lca_may_not_exist` above looks different from the plain version.

8 In the interview

How it gets asked

- The general one: *“Find the lowest common ancestor of two nodes in a binary tree.”* LeetCode 236.
- The BST one, usually first: *“Same question, but it is a binary search tree.”* LeetCode 235.
- The comparison: *“Why is your BST answer different?”*
- The removal of the guarantee: *“What if one of the nodes might not be in the tree?”* LeetCode 1644.
- The variation: *“Each node has a parent pointer.”* LeetCode 1650.
- The scaling one: *“Now answer a hundred thousand LCA queries on the same tree.”*

What to say out loud, in the first ninety seconds

1. **Ask the two questions.** “Is it a plain binary tree or a search tree? And are both nodes guaranteed to be in it? Both change the solution.”
2. **State the definition precisely.** “The deepest node with both of them beneath it — and a node counts as its own descendant, so if one is an ancestor of the other, that one is the answer.”
3. **Describe the general algorithm as a question asked of each side.** “At each node I ask both subtrees: is either target below you? If two different children come back with something, the paths split here, so this node is the answer. If only one does, the answer is inside it, so I pass it up unchanged.”
4. **Name the overloaded return value.** “The return value means three things: `None` for nothing found, one of the targets if I found exactly one, or the answer itself. That overloading is why it is six lines.”
5. **For the BST, say why it is different.** “In a general tree I must search, because nothing tells me where the nodes are. In a BST the ordering is a map — I walk down while both are on the same side, and the first node where they split is the answer. `O(height)` time, `O(1)` space, no recursion.”
6. **Give both complexities.** “General: `O(n)` time, `O(height)` space. BST: `O(height)` time, `O(1)` space — twenty comparisons instead of a million node visits on a balanced million-node tree.”

The follow-ups

“Why is the BST version different?” “Because a BST’s ordering tells you where things are, and a general binary tree’s values tell you nothing. In a general tree, to know whether `p` is in the left subtree I have to look in the left subtree — there is no short-cut, so the worst case is visiting every node. In a BST, if both targets are smaller than the current node, they are both in the left subtree, definitively, without looking. So I walk down from the root while both are on the same side, and the first node where they diverge — or where one of them is the node — is the answer. That makes it $O(\text{height})$ instead of $O(n)$, and $O(1)$ space instead of $O(\text{height})$, because it is a loop rather than a recursion. On a balanced million-node tree that is about twenty comparisons against a million visits.”

“What if one of the nodes might not be in the tree?” “Then the standard solution is wrong, and it is wrong silently. It returns as soon as it sees either target, so if I ask for the LCA of a node in the tree and one that is not, it returns the first one — a plausible-looking node that is not an ancestor of anything relevant. The fix is to **count what I actually found** and check at the end that it was two. One detail matters there: the count has to happen *after* recursing into both children, not before returning early — otherwise a target that is a descendant of the other is never counted and the tally stops at one. The path-based version handles this for free, because if the search for a node fails there is no path, so that is another reason to mention it.”

“Each node has a parent pointer. Does that change anything?” “Completely — it becomes the linked-list intersection problem. Walking up from each node gives two paths that merge at the LCA, so I want the first common node of two converging lists. Three ways. Put all of `p`’s ancestors in a set and walk up from `q` until I hit one, which is $O(\text{height})$ time and space. Compute both depths, advance the deeper one until they are level, then move together, which is $O(1)$ space. Or the two-pointer swap: walk up from `p`, and when it runs out, continue from `q`; do the mirror for the other pointer. Each travels `depth_p + depth_q` steps in total, so they arrive at the merge point together — $O(1)$ space and no depth computation. That last one is the same trick as finding where two linked lists intersect.”

“Now answer a hundred thousand queries on the same tree.” “Then a per-query $O(n)$ walk is 10^{10} operations and will not finish, so I would preprocess. The standard answer is **binary lifting**: for every node, store its 2^0 -th, 2^1 -th, 2^2 -th ancestor and so on — $O(n \log n)$ to build. Then a query lifts the deeper node up to the other’s depth, and lifts both together in decreasing powers of two until they are one step below the answer, which is $O(\log n)$ per query. For a hundred thousand nodes and queries that is a few million operations rather than ten billion. If I need $O(1)$ per

query there is the Euler tour plus a sparse table for range-minimum, and if all the queries are known in advance there is Tarjan's offline algorithm with union-find. I would name these rather than write one out unless you want it — the important part is recognising that the answer changes from 'walk' to 'preprocess'."

"What does your function actually return?" "Three different things, which is what makes it short and hard to derive. `None` means neither target is below me. A target node means I found exactly one of them and no split has happened below me — so I am reporting it upward for an ancestor to use. Any other node means the answer was found below me and I am passing it up unchanged. The whole algorithm is then two lines: if both children returned something, the paths split here so I am the answer; otherwise pass along whichever one returned something. And the early return when the node is a target is correct only because both nodes are guaranteed present — if the other one exists at all it must be below this one, so this one is the answer."

"Could you do it by finding both paths from the root?" "Yes, and it is the version most people invent first, so it is worth offering as the intuition. Find the root-to-node path for each target, then walk the two lists together and take the last node they agree on. It is `O(n)` time, easy to explain, and it detects a missing node for free. The cost is `O(height)` extra space for the two paths, on top of the recursion stack — so on a skewed tree that is `O(n)` extra memory where the postorder version uses only the stack. I would describe this one, then write the postorder version, and say the trade."

A model answer

Asked: *find the lowest common ancestor of these two nodes.*

“Two questions first, because both change the answer. **Is this a plain binary tree or a binary search tree?** And **are both nodes guaranteed to be in the tree?**

Taking the general binary tree with both nodes present. The definition is the deepest node that has both of them beneath it — and importantly, a node counts as a descendant of itself, so if one target is an ancestor of the other, that one is the answer.

The way I would describe the algorithm before writing it: at every node, I ask each subtree one question — *is either target somewhere below you?* If **two different children** both come back with something, then the two targets are on opposite sides, which means the paths split exactly here, so **this node is the answer**. If only **one** child comes back with something, both targets are inside that child, so the answer is deeper and I pass what it gave me straight up. If neither does, I report nothing.

That is six lines, and the reason it is hard to derive is that **the return value means three different things**: `None` for ‘nothing below me’; one of the targets for ‘I found exactly one and there was no split’; and any other node for ‘that is the answer, found below me’. One value, three meanings.

The base case combines two ideas: if the node is `None` I have run out, and if the node **is** one of the targets I stop and report it upward without looking further. That early return is correct **only because both nodes are guaranteed present** — if the other one exists anywhere, it must be below this one, so this one is the answer. If you remove that guarantee, this solution silently returns a node that is not an ancestor of anything, and the fix is to count what I actually found and return `None` unless it was two.

One implementation detail: I compare with `is`, not `=`, because the problem hands me node objects and values may repeat.

That is `O(n)` time — I may have to visit every node, because in a general tree nothing tells me where anything is — and `O(height)` space for the stack.

If it is a binary search tree, I would not search at all, and this is the difference worth stating. The ordering is a map: if both targets are smaller than the current node, they are both in the left subtree, definitively, without looking. So I walk down from the root while they are on the same side, and the first node where they diverge — or where one of them is the node — is the answer. That is

$O(\text{height})$ time and $O(1)$ space with no recursion at all: about twenty comparisons on a balanced million-node tree, against a million node visits for the general version.”

9 Recall card

- **LCA = the deepest node with both targets beneath it, and a node counts as its own descendant** — so if one is an ancestor of the other, that one is the answer. Ask two questions first: **plain tree or BST?** and **are both nodes guaranteed present?**
- **General tree, six lines: base case** `node is None` or `node is p` or `node is q` → **return it; recurse BOTH sides; if left and right: return node** (they split here); **else return left or right**. The return value carries **three meanings**: nothing found · exactly one target found · the answer itself. Use `is`, not `=` — values may repeat.
- **The early return on `node is p` is correct ONLY because both nodes are guaranteed present.** Without that guarantee it silently returns a non-ancestor: **count what you found and require 2**, tallying *after* recursing, not before an early return.
- **In a BST there is no search: walk down while both targets are on the same side, and the first split — or a node that IS a target — is the answer.** $O(\text{height})$ time, $O(1)$ space, **no recursion** — about **20 comparisons against 1,000,000 node visits** on a balanced million-node tree. Use strict `<` and `>`, or a node that *is* a target gets walked past.
- **With parent pointers it becomes linked-list intersection** — the two-pointer swap (walk up from `p`, then continue from `q`, and vice versa) meets at the LCA in $O(1)$ space. And for **many queries**, switch from walking to preprocessing: **binary lifting**, $O(n \log n)$ build and $O(\log n)$ per query — 10^{10} operations becomes a few million.

1 What this is, and why they ask it

A **read replica** is a follower from [yesterday](#) that you also send queries to. Writes still go to the leader; reads are spread across the copies.

Three sentences. It is the cheapest way to multiply read capacity, and it works because most systems are read-heavy — fifty to a thousand reads per write. The price is **replication lag**: a follower is always a little behind, so a read from it can return data that is already out of date. And the specific failure that lag causes is the one users notice and report: **they do something, then immediately look at it, and it is not there.**

They ask it because “add read replicas” is the standard answer to a read-heavy system and most candidates stop there. The interesting part is the bug it introduces. *“The user posted a comment and cannot see it”* is a real support ticket, it is not a code defect, and being able to name it — **read-your-own-writes violation** — and give three fixes with their costs is what the question is for.

2 The story

The electricity board office had four counters and Rajan had been sent to sort out his mother’s bill, which the board said was unpaid and which she was certain she had paid in March.

He paid it again at counter two rather than argue — eleven hundred and forty rupees, receipt, stamp, the man wrote it in his register and typed it into the machine.

Then he went to counter four, three metres away, to ask for a no-dues letter, because that was what he had actually come for.

The woman at counter four looked at her screen and said the bill was unpaid.

He said he had paid it four minutes ago. He put the receipt on the counter.

She said the receipt was fine, she could see it was genuine, but her screen said unpaid, and she could not write a no-dues letter against a screen that said unpaid.

He asked her to look again. She looked again. Still unpaid.

The man at counter two, when Rajan went back, was not surprised at all. He said the machines in the hall did not all talk to the same place. His counter wrote into the main system directly, because he took money. The other counters read from a copy that got updated every so often — usually within a few minutes, sometimes twenty if it was busy, and once, on a day in December when they were doing something in the back office, not until the next morning.

He said the usual advice was to come back after lunch.

Rajan said that was a strange way to run an office, and the man agreed, and then told him the thing that actually solved it. He said: do not go to counter four. Come back to me. My counter reads from the same place it writes to, so I can see it immediately. It is slower because there is a queue, and that is why they send people to the other counters — but if you have just paid, only my counter will know.

Rajan queued again at counter two, and the same man printed the no-dues letter in about forty seconds.

3 The idea in plain English

The hall is a leader with three read replicas, and Rajan has just experienced the exact failure they cause — and the exact fix.

- Counter two is the **leader**: writes go there, and reads from it are always current.
- Counters one, three and four are **read replicas**.
- “Updated every so often” is **replication lag**.
- Not seeing his own payment is a **read-your-own-writes violation**.
- “Come back to me, my counter reads from where it writes” is **routing a user’s reads to the leader for a period after they write** — the standard fix.

Why read replicas work at all

```
read:write ratio, typical products
social feed / news      100:1 to 1000:1
e-commerce browsing    200:1
messaging              ~1:1
analytics ingestion    write-heavy
```

If a system does 6,000 reads and 120 writes a second, then **the leader’s write load is trivial and its read load is everything**. Move the reads elsewhere and one leader comfortably handles the writes for a long time.

1 leader	6,000 reads/s + 120 writes/s	→ struggling
1 leader + 3 replicas	120 writes/s on the leader 2,000 reads/s on each replica	→ comfortable

This is the cheapest scaling move available for a read-heavy system, and it needs no application changes except deciding where each query goes.

What lag actually is

The follower is applying a stream of changes it received a moment ago. **Lag is how far behind it is, measured in time.**

same data centre, normal load	1 - 50 ms
same data centre, heavy writes	100 ms - a few seconds
cross-region	200 ms + apply time
during a bulk load or migration	seconds to MINUTES

The average is not the number that matters. The maximum during a bad moment is. A follower that is usually 20 milliseconds behind and occasionally four minutes behind will produce four minutes of inexplicable support tickets.

The three guarantees users actually expect

Users do not have a consistency model in their heads, but they have three expectations, and each has a name.

Read-your-own-writes. *If I do something, I see it.* Rajan's payment.

This is the one that generates tickets, because the user knows for certain that they did the thing.

Monotonic reads. *Time does not go backwards.* If I refresh and see my comment, then refresh again and it is gone, something is badly wrong.

This happens when two consecutive reads land on **different replicas** with different lag. It is worse than plain staleness, because it looks like data loss.

Consistent prefix reads. *Cause comes before effect.* You should not see the answer to a question before the question.

This one is mostly a sharding problem rather than a replica problem, but it is worth naming: if two related writes are replicated through different channels, they can arrive out of order.

The fixes for read-your-own-writes

Four, in increasing order of sophistication. **Know all four; the first is usually the answer.**

One — read from the leader after writing. For a short window (say thirty seconds), send that user's reads to the leader. Simple, effective, and it puts some read load back on the leader.

Two ways to decide the window: **time-based** (thirty seconds after any write) or **object-based** (reads of anything this user recently modified go to the leader). Object-based is more precise and needs you to track what they touched.

Two — read from the leader for data the user owns. A user's own profile always comes from the leader; everybody else's comes from a replica. Coarse, simple, and it covers most of the real cases because the things you edit are usually things you own.

Three — a consistency token. After a write, the leader returns its current log position, and the client sends it back with subsequent reads. A replica that has not reached that position either waits or forwards to the leader.

Precise and correct, and it requires the client to carry the token and the database to expose the position. **Real systems do this:** DynamoDB's strongly consistent reads and Aurora's session tokens are versions of it.

Four — just wait. The application delays a moment after writing before reading back. **This is not a fix**, it is a race with better odds, and it is worth naming so you can dismiss it.

The fix for monotonic reads

Pin each user to one replica — by hashing their user id — so their reads always come from the same copy. They may see stale data, but they never see time run backwards.

The cost: load balance becomes worse, and if that replica dies, those users jump to a different one and may go backwards once.

Where the lag actually comes from

Worth knowing, because “reduce the lag” is a real answer and it needs a cause:

- **A single-threaded apply process** on the follower replaying a leader that had fifty concurrent writers. Modern databases parallelise this, but the ordering guarantee limits how much.

- **Long transactions.** A transaction is replicated when it commits, so a five-minute transaction produces five minutes of lag in one lump.
- **Bulk operations:** a migration, a backfill, a `DELETE` of a million rows.
- **The follower's own read load.** A replica busy serving queries applies changes more slowly, which is a genuinely unpleasant feedback loop: more read traffic causes more lag causes more stale answers.
- **Locks on the follower:** a long analytical query can block the apply process outright.

When replicas stop helping

This is the limit worth stating, because it is not obvious:

Every replica applies every write.

A replica does not reduce write work — it duplicates it. So as the write rate rises, each replica spends more of its capacity on replication and less on serving reads, and adding more replicas does not help.

```
write load per machine, 10 replicas at 1,000 writes/s
→ every one of the 11 machines is doing 1,000 writes/s
→ if a machine tops out at 5,000 writes/s, the whole design tops out there,
   no matter how many replicas you add
```

At that point the answer is sharding, which is [tomorrow](#).

4 The picture

The arrangement, and the failure.

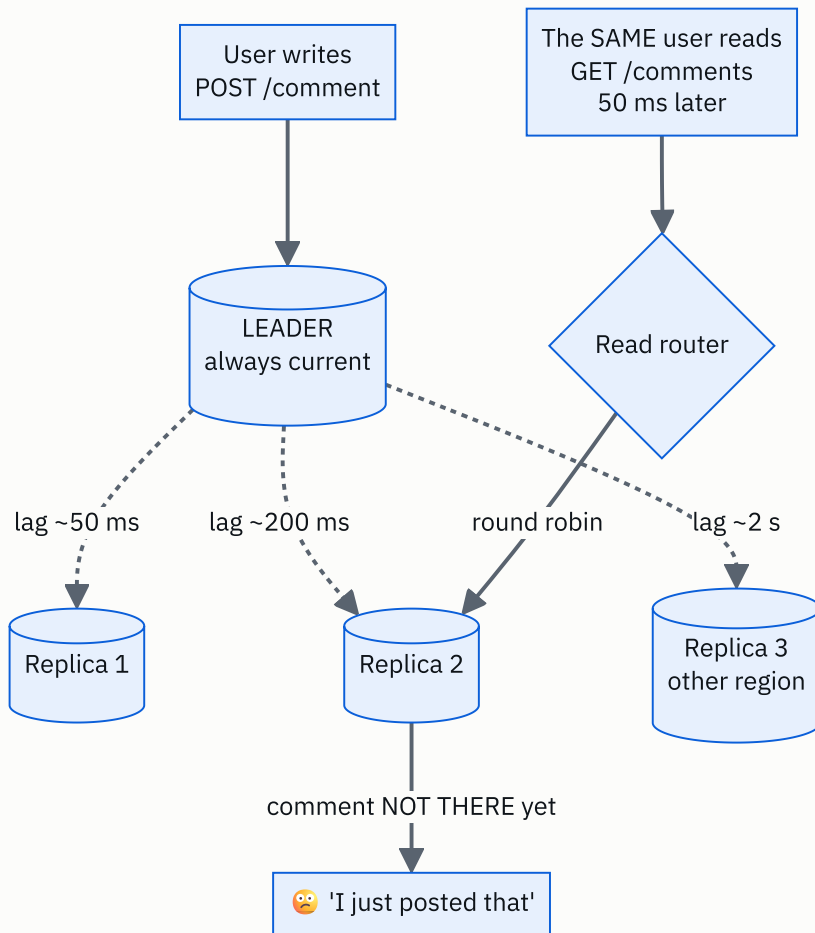


FIGURE 105.1

What to notice: **the write and the read are correct in isolation.** Nothing is broken. The user simply asked a machine that has not been told yet.

The timeline, which is what to draw when explaining it:

```

t=0      user POSTs a comment      → leader commits it.  Leader: [c1]
t=0      leader ACKs               → the UI says "posted"
t=30ms   the change starts shipping to replicas
t=50ms   user's browser GETs the comment list
          the read router picks Replica 2, which is 200 ms behind
          Replica 2: []            → "no comments"
t=200ms  Replica 2 applies the change.  Replica 2: [c1]
t=8s     user refreshes in confusion → now it is there

```

NOTHING IS BROKEN. The write succeeded, the read was accurate for the machine it asked. The user experience is a bug all the same.

Monotonic reads, which is the worse-looking failure:

```

read 1 → Replica 1 (lag 20 ms)  sees the comment  ✓
read 2 → Replica 3 (lag 2 s)    comment GONE        XX

```

the user watched their comment DISAPPEAR.
Plain staleness looks like a delay; this looks like data loss.

fix: pin the user to ONE replica, by hashing their user id.
They may be stale, but time never runs backwards.

The four fixes, compared:

FIX	precision	cost	used by
read leader for 30 s after any write	coarse	some read load returns to the leader	most systems
read leader for data the user owns	medium	needs an ownership rule	social products
consistency token (log position)	exact	client must carry it; DB must expose the position	DynamoDB, Aurora
sleep before reading	none	it is a race, not a fix	nobody, on purpose

Where replicas stop helping:

writes/s	leader	each replica	replica capacity left for reads
100	100 w/s	100 w/s	~98% plenty
1,000	1,000	1,000	~80% fine
4,000	4,000	4,000	~20% struggling
5,000	5,000	5,000	0% SATURATED – adding replicas does nothing at all

EVERY REPLICA APPLIES EVERY WRITE. Replication multiplies read capacity and does NOT reduce write work. Past the write ceiling, the answer is sharding.

5 How it actually works

Routing reads

Three places the decision can live, and each is used in practice.

In the application. Two connection pools — one to the leader, one to the replicas — and the code chooses. Explicit and simple; every developer has to remember the rule, which is where the bugs come from.

In a proxy. ProxySQL, PgBouncer with routing, or a managed endpoint. The application sees one address; the proxy inspects the query and sends `SELECT` to a replica and everything else to the leader. **The catch: a `SELECT` inside a transaction that also writes must go to the leader**, and a proxy that routes purely on the verb gets that wrong.

In the driver. Some clients understand a replica set and route themselves — MongoDB's read preferences are the clearest example, with settings like `primary`, `primaryPreferred`, `secondaryPreferred` and `nearest`.

Measuring lag

Every database exposes it, and it should be a first-class alert:

PostgreSQL	<code>pg_stat_replication.replay_lag</code>	(a time interval)
MySQL	<code>Seconds_Behind_Master</code>	(approximate; it lies during stalls)
MongoDB	<code>rs.printSecondaryReplicationInfo()</code>	
Managed	CloudWatch <code>ReplicaLag</code> , or the equivalent	

A more honest measure: write a heartbeat row on the leader every second containing the current time, and on the follower compute *now minus the timestamp of the latest heartbeat you have applied*. That catches stalls that the built-in counters can miss, and it is what most serious deployments do.

Then act on it: if a replica's lag exceeds a threshold, take it out of the read pool automatically. A replica that is four minutes behind is worse than one fewer replica.

The consistency token, concretely

1. client writes; the leader returns its current log position (an LSN)
2. client stores it — in a cookie, a header, or the session
3. client reads, sending the token
4. the router picks a replica and compares the token with the replica's applied position
 - replica has caught up → serve from the replica
 - replica is behind → wait briefly, or fall back to the leader

This is the only fix that is exactly right, and its cost is that the token has to survive the round trip through a client you may not control, and the position has to be comparable across machines.

Replicas for things other than read scaling

Worth mentioning, because it changes how many you want:

- **A dedicated analytics replica**, so a twenty-minute report never touches the machines serving users. Often deliberately allowed to lag.
- **A backup replica**, so taking a backup does not slow the leader.
- **A delayed replica**, an hour behind on purpose, as protection against a destructive statement — [yesterday's](#) human-error defence.
- **A cross-region replica**, for disaster recovery and for local reads.

Different replicas for different jobs, with different lag tolerances, is a more sophisticated answer than “three replicas”.

What real systems do

- **Amazon RDS read replicas** are asynchronous with lag typically in the tens of milliseconds, and `ReplicaLag` is the metric everyone alerts on. **Aurora** replicates at the storage layer, so its replica lag is usually single-digit milliseconds — a genuinely different architecture.
- **MongoDB** exposes read preference per query, and `readConcern: "majority"` plus causal consistency sessions provide read-your-own-writes without going to the primary.
- **DynamoDB** makes it explicit and per-request: eventually consistent reads are the default and cost half as much; strongly consistent reads cost double and are served from the leader replica. **Pricing the consistency is the clearest expression of this trade anywhere.**
- **Facebook** published a design where a write sets a short-lived marker for that user, and reads consult the marker to decide whether to go to the primary region — object-based routing at scale.

6 The numbers

What replicas buy

```
6,000 reads/s + 120 writes/s
```

```
one leader alone:      6,120 operations/s on one machine
```

```
leader + 3 replicas:   120 writes/s on the leader
```

```
2,000 reads/s on each replica
```

```
→ the leader is now at 2% of its previous load
```

Read capacity scales linearly with replicas; write capacity does not scale at all.

The lag window, as a probability

```
write, then read 50 ms later
replica lag ~200 ms
→ the read lands inside the lag window → STALE, guaranteed
```

```
write, then read 3 seconds later (a human clicking)
replica lag ~200 ms
→ stale only if the lag spiked → rare
```

```
how often does a user hit this?
  writes per day per active user          ~5
  reads within 1 second of a write        ~4 of those 5 (the UI reloads)
  P(replica lag > 1 s)                    ~2% under normal load
  → ~0.4 stale reads per user per day, and MUCH worse during a lag spike
```

Under one percent, and it is the one percent that files support tickets, because the user knows for certain that they did the thing.

The cost of the leader-read window

```
30-second window after any write
5 writes per user per day, ~15 reads inside those windows
as a share of a user's ~100 daily reads: ~15%

→ the leader takes back ~15% of the read load
→ at 6,000 reads/s that is 900 reads/s on the leader
```

Worth knowing before you promise the fix is free. Object-based routing — only reads of things the user just touched — cuts this to a few percent.

The write ceiling

```
a machine that sustains 5,000 writes/s
```

write rate	replica capacity used by replication	left for reads
-----	-----	-----
500 w/s	10%	90%
2,000 w/s	40%	60%
4,000 w/s	80%	20%
5,000 w/s	100%	0%

At 5,000 writes a second, a replica can serve no reads at all, and adding an eleventh replica adds nothing. That is the number that says “the next step is sharding, not more replicas”.

Cross-region

replica in the same region	lag ~10-50 ms,	read latency ~1 ms
replica in another region	lag ~200 ms+,	read latency ~1 ms locally but 200 ms if you read across
a user in India reading from a US replica: ~200 ms		
a user in India reading from an India replica: ~2 ms, and up to 200+ ms stale		

The trade is explicit: a local replica is a hundred times faster and a little more wrong.

Lag alerting thresholds

warn at	1 second	something is starting to go wrong
page at	10 seconds	read-your-own-writes is broken for everybody
evict at	30 seconds	take it out of the read pool automatically

Automatic eviction is the part people forget. A replica that is minutes behind is actively harmful, and serving from four healthy replicas is better than five with one lying.

7 The trade-offs

Read replicas buy read capacity and cost correctness

That is the whole trade. You get near-linear read scaling for the price of **a small, unpredictable window in which reads are wrong**. Every fix trades some of the capacity back.

I would not add read replicas if the read:write ratio is near 1:1 — messaging, for example — because there is little read load to move and every replica still pays the full write cost.

Which fix for read-your-own-writes?

Read from the leader after a write is the default. It is simple, needs no client cooperation, and costs ten to fifteen percent of the read load. **Take this unless there is a reason not to.**

Object-based routing — the leader only for things the user recently touched — is more precise and cheaper, and it needs you to track what they touched, which is real bookkeeping.

Consistency tokens are exactly right and require the client to carry a token and the database to expose a comparable position. **I would use them when reads and writes cross service boundaries**, where “thirty seconds after their write” is not something a downstream service knows.

Sleeping before reading is not a fix. Say so.

Stale reads are sometimes fine, and saying which is a strong answer

Not every read needs to be current. Sorting them is more valuable than making everything consistent:

must be current	the user's own data after they change it anything a decision is made on (balance, stock at checkout) a permission check
can be seconds stale	another user's profile, a feed, a comment count, search results, a dashboard
can be minutes stale	analytics, reports, trending lists

That triage is the answer to “how would you decide what goes to a replica?” — and the same rule as [day 101's](#) cache: the display may be stale; the decision may not.

Pinning users for monotonic reads

Pinning gives you monotonic reads and gives up even load balancing, because users are not uniform. It also means a replica failure moves that group of users to a new copy, where they may go backwards once.

Worth it when users can see a value change back, which is anything with a visible timeline or count.

Where this design breaks

- **The write ceiling.** Every replica applies every write, so past a certain write rate replicas stop helping entirely. That is a sharding conversation.
- **Lag spikes are correlated with load.** The moment you most need replicas — a traffic peak — is when the write rate is highest and lag is worst. The failure arrives exactly when it is least convenient.
- **A slow replica poisons results silently.** Without automatic eviction on lag, one machine serves stale answers to a fraction of users indefinitely, and it looks like an intermittent application bug.
- **Analytical queries on a serving replica** cause lag for everyone using it. Give reports their own replica.

- **Failover promotes a replica**, and if it was behind, you lose the difference — the RPO from [yesterday](#). Read-replica lag and failover data loss are the same number wearing two hats.

8 In the interview

How it gets asked

- The support ticket: *“The user posted a comment and cannot see it. What happened?”*
- The design: *“This system is read-heavy. How do you scale the database?”*
- The follow-up: *“How do you fix read-your-own-writes?”*
- The worse one: *“A user refreshes and their comment disappears. What is that?”*
- The limit: *“You have ten read replicas and it is still slow. Now what?”*

What to say out loud, in the first ninety seconds

1. **Name the failure precisely.** “That is a **read-your-own-writes violation** — replication lag. The write went to the leader, the read went to a replica that had not applied it yet. Nothing is broken; both operations were correct.”
2. **Give the shape of the numbers.** “Lag is usually tens of milliseconds and occasionally seconds, so a read within a second of a write is at real risk — and the UI almost always reloads immediately after a write, which is exactly the worst case.”
3. **Give the default fix and its cost.** “Route that user’s reads to the leader for a short window after they write — thirty seconds. That costs maybe ten to fifteen percent of the read load going back to the leader, and it fixes the case users actually notice.”
4. **Name the sharper alternatives.** “More precisely: route only reads of things the user just modified; or a consistency token, where the write returns a log position and the read waits for a replica that has reached it.”
5. **Mention the second, uglier failure.** “There is a related one — **monotonic reads**. Two consecutive reads landing on replicas with different lag can make a comment appear and then disappear, which looks like data loss. The fix is to pin a user to one replica.”
6. **State the limit before being asked.** “Replicas multiply read capacity and do nothing for writes — every replica applies every write — so past a certain write rate they stop helping and the answer becomes sharding.”

The follow-ups

“**How do you fix read-your-own-writes?**” “Four options and I would default to the first. **Route the user’s reads to the leader for a window after they write** — say thirty seconds. Simple, needs no client cooperation, and it costs roughly ten to fifteen percent of the read load coming back to the leader, because the reads right after a write are the ones the UI does automatically. **Object-based routing** is the sharper version: only reads of things this user recently modified go to the leader, which is a few percent instead of fifteen, at the cost of tracking what they touched. **A consistency token** is exactly right: the write returns the leader’s log position, the client sends it with subsequent reads, and a replica that has not reached that position either waits or forwards to the leader — that is what DynamoDB and Aurora do, and I would reach for it when reads and writes cross service boundaries, where ‘thirty seconds after their write’ is not something a downstream service knows. And the fourth is sleeping before the read, which is not a fix — it is a race with better odds.”

“**A user refreshes and their comment disappears. What is that?**” “That is a **monotonic reads** violation, and it is worse than plain staleness because it looks like data loss. The first read landed on a replica with 20 milliseconds of lag and saw the comment; the second read landed on a different replica with two seconds of lag and did not. Both reads were accurate for the machine they asked. The fix is to **pin each user to one replica**, usually by hashing their user id, so their reads always come from the same copy. They may see stale data, but time never runs backwards. The cost is that load balance gets worse, because users are not uniform, and if that replica dies the pinned users jump to another and may go backwards once at that moment.”

“**You have ten read replicas and it is still slow. Now what?**” “Then I would check what is actually saturated, because more replicas may be doing nothing. **Every replica applies every write**, so replication is not free capacity — if the write rate is 4,000 a second and a machine tops out at 5,000, every replica is spending eighty percent of itself on replication and has only twenty percent left for reads. At that point adding an eleventh replica adds almost nothing. The answers, in order of what I would try: **cache**, because a ninety percent hit rate removes ten times more read load than another replica; **check the lag**, because a replica minutes behind may be serving garbage while still counting as capacity; and if the write rate is genuinely the ceiling, **shard**, which is the only thing that scales writes. I would also look at whether analytical queries are running on serving replicas, since one twenty-minute report can stall the apply process for everybody on that machine.”

“How do you decide which reads go to a replica?” “By triaging what can be stale, which is the same rule as caching: **the display may be stale, the decision may not**. Reads that must be current: the user’s own data right after they change it, and anything a decision is made on — a balance being debited, stock at checkout, a permission check. Reads that can be seconds stale: other people’s profiles, feeds, comment counts, search results. Reads that can be minutes stale: analytics and reports, which I would give their own dedicated replica so a twenty-minute query never touches the machines serving users. Making that classification explicit is more valuable than making everything consistent, because everything-consistent means everything goes to the leader and the replicas were pointless.”

“How would you detect a replica falling behind?” “The built-in metrics — `replay_lag` in PostgreSQL, `Seconds_Behind_Master` in MySQL — and I would not trust them alone, because they can under-report during a stall. The more honest measure is a **heartbeat**: write a row on the leader every second containing the current time, and on the follower compute now minus the timestamp of the latest heartbeat it has applied. That catches a stalled apply process that the counter misses. Then the important part is acting on it automatically: **evict a replica from the read pool when its lag exceeds a threshold**, say thirty seconds. A replica that is minutes behind is actively harmful — four healthy replicas serve users better than five with one lying — and without automatic eviction it looks like an intermittent application bug for as long as nobody investigates.”

“Does this help with write load at all?” “Not at all, and it slightly hurts. Every write is applied on every replica, so replication multiplies the write work rather than dividing it — a leader plus ten replicas means eleven machines each doing the full write rate. Read replicas scale **reads** near-linearly and scale writes by exactly zero. When the write rate is the constraint the only real answer is sharding: splitting the data so each machine takes a fraction of the writes. That is a much larger change, because you lose transactions and joins across shards.”

A model answer

Asked: *the user posted a comment and cannot see it. What happened?*

“That is a **read-your-own-writes violation** caused by replication lag, and the first thing to say is that **nothing is broken**. The write went to the leader and succeeded. The read went to a read replica which had not yet applied that change, and returned exactly what it had. Both operations were correct; the combination is wrong.

The mechanism: the leader takes all the writes and ships an ordered stream of changes to the replicas, which apply them a moment later. That moment is usually tens of milliseconds and occasionally seconds — and during a bulk load or a long transaction it can be minutes. Meanwhile, the read that follows a write is almost always **immediate**, because the interface reloads the list the instant the post succeeds. So the read lands squarely inside the lag window, which is why this is the failure users actually encounter rather than a theoretical one.

The standard fix is to **route that user’s reads to the leader for a short window after they write** — thirty seconds is typical. It needs no client cooperation, and the cost is real and worth quoting: the reads immediately after a write are perhaps ten to fifteen percent of a user’s reads, so that much of the read load comes back to the leader. A sharper version routes to the leader only for **objects the user recently modified**, which is a few percent instead of fifteen but requires tracking what they touched. And the exact version is a **consistency token**: the write returns the leader’s log position, the client sends it back with reads, and a replica that has not reached that position either waits or forwards to the leader. That is what DynamoDB and Aurora expose, and I would use it when reads and writes happen in different services, where one cannot know about the other’s recent activity.

I would also mention the related failure, because it is worse and it has a different fix. **Monotonic reads**: if two consecutive reads land on replicas with different lag, the comment can appear and then disappear. Plain staleness looks like a delay; that looks like data loss. The fix is to **pin a user to one replica** by hashing their id — they may be stale, but time never runs backwards.

Operationally, the thing I would insist on is **measuring lag with a heartbeat** rather than trusting the built-in counter, and **automatically evicting a replica from the read pool when it exceeds a threshold**. A replica that is four minutes behind is worse than not having it, and without eviction it produces exactly this ticket, intermittently, for one user in five, until somebody investigates.

And the limit worth stating unprompted: replicas scale **reads** and do nothing for writes, because every replica applies every write. Past a certain write rate they stop helping altogether, and the next step is sharding rather than more copies.”

9 Recall card

- **A read replica is a follower you also query. Reads scale near-linearly; writes do not scale at all — every replica applies every write**, so at 4,000 w/s on a 5,000 w/s machine each replica has only 20% left for reads, and the eleventh replica adds nothing. Past that, the answer is **sharding**.
- **The failure users report is a read-your-own-writes violation, and nothing is broken** — the write went to the leader, the read went to a replica that had not applied it. The UI reloads immediately after a write, so the read lands **inside** the lag window by design.
- **Four fixes: read from the leader for ~30 s after a write** (default; costs ~10–15% of reads back on the leader) · **object-based routing** (only what the user just touched; a few percent) · **a consistency token** — the write returns a log position, the read waits for a replica that has reached it (DynamoDB, Aurora) · **sleeping, which is not a fix**.
- **Monotonic reads is the uglier sibling**: two reads on replicas with different lag make a comment appear and then vanish — it looks like data loss. Fix by **pinning a user to one replica** (hash their id); the cost is uneven load and one backwards jump if that replica dies.
- **Measure lag with a heartbeat row, not just `Seconds_Behind_Master`, and evict a replica from the read pool automatically past a threshold** — a replica minutes behind is worse than one fewer replica. And triage reads like a cache: **the display may be stale, the decision may not** — balances, stock at checkout and permission checks always come from the leader.

DSA topic: Lowest common ancestor **System design topic:** Read replicas and replication lag

Code these, in this order

One rule for the whole set: **ask the two questions before writing** — plain tree or search tree, and are both nodes guaranteed to be present. They change the code, and asking takes ten seconds.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	LCA of a Binary Search Tree	LeetCode 235 (Medium)	Noticing that the ordering removes the search entirely.
2	LCA of a Binary Tree	LeetCode 236 (Medium)	The overloaded return value, and recursing both sides.
3	LCA of a Binary Tree II	LeetCode 1644 (Medium)	What breaks when the nodes may be absent.
4	LCA of Deepest Leaves	LeetCode 1123 (Medium)	The same postorder shape returning a pair.

On problem 1, write it iteratively

No recursion. Then say the time and space, and compare both against the general version on a balanced million-node tree.

On problem 2, say the three meanings before you code

Write the three meanings of the return value down first. Then write the six lines. If the code comes before the sentence, you are recalling rather than deriving.

On problem 3, break problem 2 first

Take your working solution from problem 2 and call it with a node that is not in the tree. Record what it returns. Then fix it, and say why the count must happen after recursing.

On problem 4, notice the shape

Compare it with day 102's diameter and day 104's max path sum. Say what all three share, and what the return value carries in each.

The two-questions drill

1. State both questions and say what each one changes.
2. Say what the standard solution returns when a node is absent.
3. Say what changes if it is a BST.
4. Say what changes if nodes carry parent pointers.

The definition drill

1. Define LCA in one sentence.
2. Say why the root is almost never the answer.
3. State the self-ancestor rule and give an example where it decides the answer.
4. Say what happens to the algorithm if you omit that case.

The three-meanings drill

1. Write the six lines from memory.
2. State the three meanings of the return value.
3. Say what `left and right` means in words.
4. Say what `left or right` is doing, and to whom.
5. Say why both recursive calls must always run.

The assumption drill

1. Construct a tree and a call where the plain solution is wrong.
2. Say why it stops early, and why that is correct under the guarantee.
3. Write the counting version.
4. Say why the count must come after the recursion, with the input that proves it.
5. Name a version of the solution that handles absence for free.

The BST drill

1. Write the iterative version from memory.
2. Say what the `else` branch covers — all three cases.
3. Change `<` to `≤` and find the input it breaks.
4. Run the BST version on a non-BST and say what happens and why there is no error.
5. Give both complexities against the general version, with numbers at $n = 1,000,000$.

The parent-pointer drill

1. Name the three methods and their costs.

2. Write the two-pointer swap.
3. Say why both pointers travel the same total distance.
4. Name the linked-list problem this is the same as.

The many-queries drill

1. Compute the cost of 100,000 naive queries on a 100,000-node tree.
2. Name three preprocessing approaches and their build and query costs.
3. Say which one you would name first and what it stores.
4. Say the one-sentence rule for when to switch from walking to preprocessing.

The break-it drill

Trigger each and record the exact output:

1. Comparing `.val` instead of using `is`, on a tree with duplicate values.
 2. Short-circuiting after the left recursive call.
 3. Omitting the `node is p or node is q` case.
 4. The BST version on a non-BST.
 5. `≤` in the BST comparison, where `p` is the current node.
 6. The plain solution with an absent node.
 7. A recursive LCA on a 10,000-node chain.
-

The mechanism drill

1. Say what a read replica is and what still goes to the leader.
2. Give typical read:write ratios for four kinds of product.
3. Compute the leader's load before and after adding three replicas at 6,000 reads and 120 writes.
4. Say which kind of product replicas do not help.

The lag drill

1. Give typical lag figures for four situations.
2. Say why the average is the wrong number to quote.
3. Name five causes of lag.
4. Describe the feedback loop between read traffic and lag.

The three-guarantees drill

1. Name all three guarantees users expect and give the everyday phrasing of each.

2. Say which one produces support tickets and why.
3. Say which one looks like data loss.
4. Say which one is mostly a sharding problem.

The fixes drill

1. Name the four fixes for read-your-own-writes in order.
2. For each, say its precision and its cost.
3. Compute the share of read load that returns to the leader under the default fix.
4. Say which fix real systems expose as a per-request option, and name two.
5. Say which one is not a fix, and why you would mention it anyway.

The monotonic drill

1. Describe the failure in two sentences.
2. Say why it is worse than plain staleness.
3. Give the fix and the two costs it carries.
4. Say what happens to pinned users when their replica dies.

The ceiling drill

1. State the sentence about every replica applying every write.
2. Fill in the table of write rate against replica capacity left for reads.
3. Say what the answer is once the ceiling is reached.
4. Say what you would try before that, in order.

The operations drill

1. Name the lag metric for two databases.
2. Describe the heartbeat method and say what it catches that the metric misses.
3. Give the three alert thresholds and the action at each.
4. Say why automatic eviction matters, in terms of what the failure looks like.
5. Name four jobs a replica can have besides serving reads.

The failure drill

For each, say what happens and what you would add:

1. A user writes and reads 50 ms later.
2. Two consecutive reads land on replicas with different lag.
3. A twenty-minute report runs on a serving replica.

4. A replica is four minutes behind and still in the read pool.
5. The write rate reaches the machine's ceiling.
6. A leader fails over to a replica that was behind.

Two of the six are the same underlying number wearing different names. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the lowest common ancestor of these two nodes.* The two questions asked first, the definition with the self-ancestor rule, the algorithm described as a question asked of each subtree, the three meanings of the return value, why the early return is correct under the guarantee, and then the BST version with the reason it is different.
 2. *The user posted a comment and cannot see it. What happened?* The failure named precisely, “nothing is broken” said explicitly, the timing that makes it likely rather than rare, the default fix with its cost as a percentage, the sharper alternatives, and the monotonic sibling failure.
 3. *Why is your BST answer different from your binary tree answer?* Search versus map, the walk-down rule, `0(height)` and `0(1)` against `0(n)` and `0(height)`, and the numbers at a million nodes.
-

Before you move on

- [] I ask both questions before writing any LCA code.
- [] I can state the three meanings of the return value.
- [] I can write the six lines from memory and explain each.
- [] I know why both recursive calls must run.
- [] I can construct the input where the standard solution is wrong.
- [] I know why the count must come after the recursion.
- [] I can write the BST version iteratively and say what the `else` covers.
- [] I know what happens when the BST version meets a non-BST.
- [] I can give both complexities with numbers at a million nodes.
- [] I can write the parent-pointer two-pointer swap and name the problem it matches.

- ☐ I can name three preprocessing approaches for many queries.
- ☐ I can name the failure when a user cannot see their own write.
- ☐ I can say “nothing is broken” and explain why both operations were correct.
- ☐ I can name all four fixes with their precision and cost.
- ☐ I can compute the read load that returns to the leader under the default fix.
- ☐ I can describe monotonic reads and say why it looks like data loss.
- ☐ I can state the every-replica-applies-every-write ceiling with the table.
- ☐ I know how to measure lag honestly and what to do when it exceeds a threshold.
- ☐ I answered all three questions above out loud.

Binary search trees: the ordering property

DATA STRUCTURES AND ALGORITHMS

Binary search trees: the ordering property

You can state the BST invariant precisely, including the whole-subtree part.

SYSTEM DESIGN

Sharding, part one: choosing the key

You can pick a shard key and predict which queries it will make expensive.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What makes a tree a binary search tree?*
- *How would you shard this table? What breaks after you do?*

Binary search trees: the ordering property

1 What this is, and why they ask it

A **binary search tree** is a binary tree with one extra rule about the values: at every node, everything in the left subtree is smaller and everything in the right subtree is larger.

Three sentences. That single rule turns a structure where finding a value costs $O(n)$ into one where it costs $O(\text{height})$, because one comparison eliminates an entire subtree. **The rule is about whole subtrees, not about the immediate children**, and confusing those two is the single most common error in the topic — it produces a tree that looks correct locally and is broken globally. And the guarantee is conditional: $O(\log n)$ only holds while the tree is **balanced**, which nothing so far forces.

They ask “*what makes a tree a binary search tree?*” as a warm-up and they are listening for one specific thing: whether you say “**all of the left subtree**” or “**the left child**”. The second answer is wrong, it is the answer most people give, and [day 108](#) exists entirely to punish it. Everything else in the phase — insert, delete, validate, balance, k-th smallest — depends on stating the invariant correctly first.

2 The story

The three of them played the number game on every long bus journey, and Anitha, who was eleven, had made up a rule that ruined it for her brother.

The game was that one person thought of a number between one and a hundred and the others guessed, and after each guess they were told higher or lower.

Anitha was very good at it. She always guessed fifty first, and then twenty-five or seventy-five, and she usually had it in six or seven guesses while her brother was still working upwards from four.

Her brother, who was eight, could not see why she did that, so on the way back from Trichy she made him be the one thinking of the number, and she explained what she was doing as she went.

She said: when you tell me higher than fifty, I am not just learning something about fifty. I am throwing away half of everything. One to fifty is gone. It is not coming back. It cannot be the number.

Then when you say lower than seventy-five, I have thrown away seventy-six to a hundred as well, so the number is somewhere between fifty and seventy-five, and I only have twenty-five things left.

And this is the part he got wrong when it was his turn to answer.

He had thought of sixty. She guessed fifty; he said higher. She guessed seventy-five; he said lower. She guessed sixty-two; he said lower — correct. She guessed fifty-five; and he said **higher**, which was true of fifty-five and sixty, so he thought he was being honest.

But he had already said lower than sixty-two, and now he was saying higher than fifty-five, and Anitha did the arithmetic and said fifty-eight.

He said no, sixty.

She said you told me lower than sixty-two, and you told me higher than fifty-five, and sixty is inside that, so how is fifty-eight wrong.

And it took them ten minutes to work out what had happened: on the guess before, he had said **lower** when the guess was fifty-one, because fifty-one was lower than sixty-two, which was the last thing he had been thinking about. He had been answering each guess against the guess before it instead of against his actual number.

Every single answer he gave was consistent with the one before it. Every one of them, checked against its neighbour, was fine. And the answers taken together described a number that did not exist.

Anitha's rule after that was one sentence, and she made him repeat it: **every answer has to be true about the number, not about the last thing you said.**

3 The idea in plain English

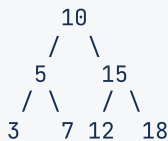
Anitha's brother has committed exactly the classic BST error, and her rule is the invariant.

The invariant, stated correctly

For every node: every value in its left subtree is less than the node, and every value in its right subtree is greater than the node.

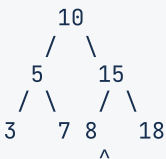
“Every value in the subtree”, not “the child”. That word is the entire lesson.

CORRECT



every node's left subtree is entirely smaller, every right subtree entirely larger

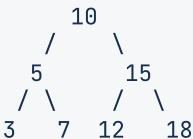
WRONG BUT LOOKS FINE



8 < 15, so the LOCAL rule holds at node 15 – but 8 is in the RIGHT subtree of 10 and 8 < 10. NOT a BST.

The 8 there is the brother's fifty-one: correct relative to its immediate neighbour, and wrong relative to everything it is really underneath.

Every node lives inside a range, inherited from every ancestor above it — and that is the precise way to say the invariant:



range: (-inf, +inf)

5 in (-inf, 10)

15 in (10, +inf)

7 in (5, 10)

12 in (10, 15)

a node's range is NARROWED by every ancestor, not just its parent.
going left → the upper bound becomes the parent's value
going right → the lower bound becomes the parent's value

“Every answer has to be true about the number, not about the last thing you said.”

What the rule buys

One comparison eliminates a whole subtree.

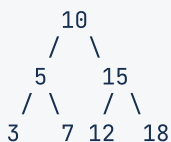
```
def search(node, target):
    while node:
        if target == node.val:
            return node
        node = node.left if target < node.val else node.right
```

Four lines, and no recursion. At each step you discard everything on one side. That is [binary search](#) from day 42, on a structure that supports insertion and deletion.

plain binary tree, find a value:	$O(n)$	– you must look in both subtrees
BST, find a value:	$O(\text{height})$	
balanced BST:	$O(\log n)$	– 20 steps in a million nodes
degenerate BST:	$O(n)$	– a chain, and you are back where you started

Inorder is sorted, and it is the definition in another form

Walk left, node, right — from [day 100](#) — and on a BST the values come out in ascending order.



inorder: 3, 5, 7, 10, 12, 15, 18 ← sorted

This is not a coincidence; it is the invariant restated. **“Everything left is smaller” is “the left subtree comes first in sorted order”**. And it gives you a second, very clean way to check whether a tree is a BST: walk it inorder and confirm the values increase. That is [day 108](#).

Duplicates: a decision you must state

The invariant as written says “less than” and “greater than”, which leaves equal values undefined. Four conventions exist and **you must say which one you are using**:

- | | |
|------------------------|--|
| 1. NOT ALLOWED | the simplest, and what most interview problems assume |
| 2. equal goes LEFT | left subtree is $\leq$ node |
| 3. equal goes RIGHT | right subtree is $\geq$ node |
| 4. a COUNT on the node | the tree holds distinct values, each with a multiplicity |

Option 4 is the best real answer — it keeps the tree smaller and makes deletion of one occurrence trivial — and option 1 is what the problems assume. What you must not do is allow duplicates on both sides, because then search cannot know which way to go and the structure loses its point.

Why a BST rather than the alternatives

The comparison table is the reason anyone uses one, and it is the answer to “why not just use a hash map?”

	search	insert	delete	sorted order	min/max
sorted array	$O(\log n)$	$O(n)$	$O(n)$	free	$O(1)$
linked list	$O(n)$	$O(1)*$	$O(1)*$	no	$O(n)$
hash map	$O(1)$	$O(1)$	$O(1)$	NO	$O(n)$
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	free	$O(\log n)$

AAAAAAAAAAAAA

(* given the position)

A hash map beats a BST on every single-key operation and cannot do ordered ones at all. That is the whole argument:

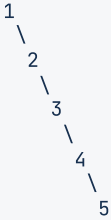
- “Give me everything between 20 and 40” — a BST does it in $O(\log n + k)$; a hash map must scan everything.
- “What is the smallest value?” — a BST walks left; a hash map scans everything.
- “Give me them in order” — a BST walks inorder; a hash map has no order.
- “What is the next value above 37, if 37 is not present?” — a BST answers it; a hash map cannot.

Say it as one sentence: use a hash map unless you need order, and then use a BST. That sentence is worth more than any implementation detail.

The catch: balance is not guaranteed

Nothing in the definition says a BST is balanced. Insert sorted data and you get a chain:

insert 1, 2, 3, 4, 5 in order:



height = n - 1. Every operation is $O(n)$. This is a linked list with extra fields.

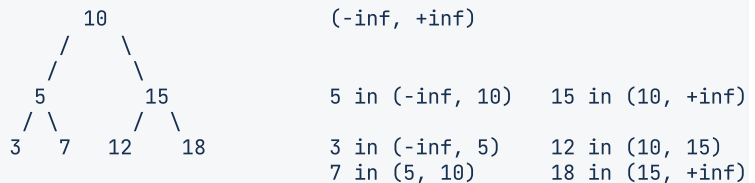
And sorted input is not an unusual case — it is the most common one, because data arrives sorted by id, by timestamp, or by name far more often than randomly. That is why real systems use **self-balancing** BSTs — AVL trees, red-black trees — which rotate to keep the height at $O(\log n)$, and it is [day 109](#).

Where you have already met them

- `sortedcontainers.SortedList` in Python and `std::map` / `std::set` in C++ are ordered structures with exactly these guarantees. Python's own `dict` and `set` are hash-based and unordered by value.
- **Database indexes** are B-trees, which are BSTs with a high branching factor so that each node fills a disk page. Same invariant, tuned for disk.
- `bisect` on a sorted list gives you $O(\log n)$ search with $O(n)$ insertion — the right choice when the data barely changes, and the wrong one when it does.

4 The picture

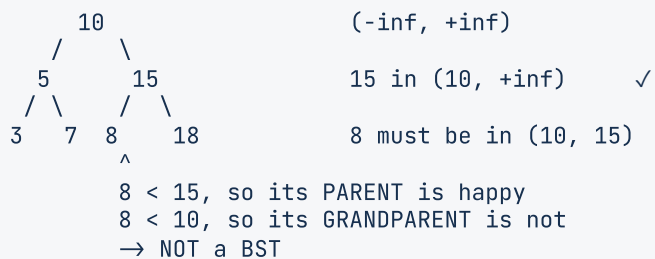
The invariant, drawn as ranges rather than as comparisons. **This is the diagram to be able to produce**, because it makes both the rule and the classic error obvious.



going LEFT narrows the UPPER bound to the parent's value
going RIGHT narrows the LOWER bound to the parent's value

the bounds accumulate from EVERY ancestor, not just the parent.

The classic error, drawn the same way:



search(8) at the root: $8 < 10$, so go LEFT — and 8 is on the right.
The value is in the tree and the search cannot find it.

That last line is why it matters. A locally-correct, globally-broken BST does not raise anything — it just fails to find values that are present.

Anitha's game, as the same picture:

```

guess 50, "higher"    → the number is in (50, 100]
guess 75, "lower"     → the number is in (50, 75)
guess 62, "lower"     → the number is in (50, 62)
guess 55, "higher"    → the number is in (55, 62)      → 56..61

```

the brother's error: he answered 51 against 62 rather than against 60.
Each answer was consistent with its NEIGHBOUR and the set of answers described no number at all.

What one comparison buys:

searching for 12 in a balanced BST of 1,000,000 nodes

```

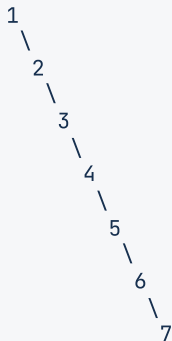
step 1: compare with the root    → discard 500,000 nodes
step 2: compare                  → discard 250,000
step 3:                          → discard 125,000
...
step 20:                        → found

```

20 comparisons. In a plain binary tree of the same size: up to 1,000,000.

And the degenerate case, which is not exotic:

inserting 1..7 in order



height 6, search = 7 steps

inserting 4,2,6,1,3,5,7



height 2, search = 3 steps

SAME VALUES, SAME RULE
the ORDER OF INSERTION decides everything

5 The code, built step by step

Step 1 — state the invariant before writing anything

“At every node, **all** values in the left subtree are smaller and **all** values in the right subtree are larger. Not just the immediate children — the whole subtree. Equivalently, every node lives inside a range inherited from all of its ancestors.”

Say “all” and say “subtree”. That sentence is what the question is checking.

Step 2 — search, which is the whole point of the structure

```
while node:
    if target == node.val:
        return node
    node = node.left if target < node.val else node.right
```

Iterative, four lines, $O(1)$ space. **There is no reason to write this recursively** — there is nothing to combine on the way back up, so the recursion would only cost stack frames.

Step 3 — min and max are walks, not searches

```
while node.left:
    node = node.left          # the minimum is the leftmost node
```

The smallest value is as far left as you can go; the largest is as far right. $O(\text{height})$, and this is one of the ordered operations a hash map simply cannot do.

Step 4 — say the duplicates policy out loud

“I will assume values are distinct, which is what the problem usually means. If duplicates are possible I would keep a count on each node rather than allowing equal values on both sides, because equal values on both sides means a search cannot know which way to go.”

Step 5 — flag the balance assumption

“All of this is $O(\text{height})$. That is $O(\log n)$ only if the tree is balanced, and nothing so far guarantees that — inserting sorted data gives a chain and every operation becomes $O(n)$. Real implementations self-balance.”

Saying this before being asked is the difference between quoting a fact and understanding it.

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

def search(root: TreeNode | None, target: int) → TreeNode | None:
    """LeetCode 700. One comparison discards a whole subtree.

    Iterative on purpose: there is nothing to combine on the way back up,
    so recursion would only cost stack frames. O(height) time, O(1) space.
    """
    node = root
    while node:
        if target == node.val:
            return node
        node = node.left if target < node.val else node.right
    return None

def contains(root: TreeNode | None, target: int) → bool:
    return search(root, target) is not None

def minimum(root: TreeNode | None) → TreeNode | None:
    """The leftmost node. A hash map cannot answer this without a full scan."""
    if root is None:
        return None
    node = root
    while node.left:
        node = node.left
    return node

def maximum(root: TreeNode | None) → TreeNode | None:
    if root is None:
        return None
    node = root
    while node.right:
        node = node.right
    return node

def insert(root: TreeNode | None, value: int) → TreeNode:
    """LeetCode 701. Walk to where the value BELONGS and put a leaf there.

    Note this does no balancing at all, so inserting sorted data builds a
    chain – and sorted input is the COMMON case, not an exotic one.
    """
    if root is None:
        return TreeNode(value)
    node = root
    while True:
        if value < node.val:
            if node.left is None:
                node.left = TreeNode(value)
            else:
                node = node.left
        else:
            if node.right is None:
                node.right = TreeNode(value)
            else:
                node = node.right
```

```

        return root
        node = node.left
    elif value > node.val:
        if node.right is None:
            node.right = TreeNode(value)
            return root
        node = node.right
    else:
        return root # duplicate: this version ignores it

def inorder(root: TreeNode | None) → list[int]:
    """On a BST this is SORTED ORDER – the invariant restated."""
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        walk(node.left)
        out.append(node.val)
        walk(node.right)

    walk(root)
    return out

def range_query(root: TreeNode | None, low: int, high: int) → list[int]:
    """Everything in [low, high], in order. O(height + k).

    THE operation a hash map cannot do: whole subtrees are skipped without
    being visited, because the ordering says they cannot contain anything
    in range.
    """
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        if node.val > low:
            walk(node.left) # only if something left could qualify
        if low ≤ node.val ≤ high:
            out.append(node.val)
        if node.val < high:
            walk(node.right)

    walk(root)
    return out

def ceiling(root: TreeNode | None, target: int) → int | None:
    """The smallest value ≥ target. Another hash-map impossibility.

    Walk down remembering the best candidate seen while going left.
    """
    best: int | None = None
    node = root
    while node:
        if node.val == target:
            return node.val
        if node.val > target:
            best = node.val # a candidate; something smaller may exist
            node = node.left
        else:
            node = node.right
    return best

def floor_value(root: TreeNode | None, target: int) → int | None:

```

```

"""The largest value ≤ target. The mirror image."""
best: int | None = None
node = root
while node:
    if node.val == target:
        return node.val
    if node.val < target:
        best = node.val
        node = node.right
    else:
        node = node.left
return best

def kth_smallest(root: TreeNode | None, k: int) → int | None:
    """LeetCode 230. Inorder, stopping after k nodes.

    Iterative so it can stop: a recursive inorder naturally wants to finish
    the whole tree. O(height + k), not O(n).
    """
    stack: list[TreeNode] = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right
    return None

def height(node: TreeNode | None) → int:
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))

def build_from_sorted(values: list[int]) → TreeNode | None:
    """LeetCode 108. Build a BALANCED BST from sorted values.

    Take the middle as the root, then recurse on each half. This is the
    answer to "your tree degenerated" when you control the insertion order.
    O(n) time, height exactly floor(log2(n)).
    """
    def build(lo: int, hi: int) → TreeNode | None:
        if lo > hi:
            return None
        mid = (lo + hi) // 2
        return TreeNode(values[mid], build(lo, mid - 1), build(mid + 1, hi))

    return build(0, len(values) - 1)

class CountingBST:
    """The right way to handle duplicates: distinct values, each with a count.

    Keeps the tree smaller, makes "remove one occurrence" trivial, and never
    leaves a search unsure which way to go – which is what happens if equal
    values are allowed on both sides.
    """

    class Node:
        def __init__(self, val: int) → None:
            self.val = val
            self.count = 1

```

```

        self.left: "CountingBST.Node | None" = None
        self.right: "CountingBST.Node | None" = None

def __init__(self) → None:
    self.root: "CountingBST.Node | None" = None

def add(self, value: int) → None:
    if self.root is None:
        self.root = self.Node(value)
        return
    node = self.root
    while True:
        if value == node.val:
            node.count += 1
            return
        if value < node.val:
            if node.left is None:
                node.left = self.Node(value)
                return
            node = node.left
        else:
            if node.right is None:
                node.right = self.Node(value)
                return
            node = node.right

def count_of(self, value: int) → int:
    node = self.root
    while node:
        if value == node.val:
            return node.count
        node = node.left if value < node.val else node.right
    return 0

def find_in_plain_tree(node: TreeNode | None, target: int) → TreeNode | None:
    """For contrast: without the ordering, BOTH subtrees must be searched.
    O(n), and this is what the BST rule buys you."""
    if node is None:
        return None
    if node.val == target:
        return node
    return (find_in_plain_tree(node.left, target)
            or find_in_plain_tree(node.right, target))

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

if __name__ == "__main__":
    bst = from_list([10, 5, 15, 3, 7, 12, 18])

    print(inorder(bst))          # [3, 5, 7, 10, 12, 15, 18] - sorted
    print(search(bst, 7), search(bst, 99)) # TreeNode(7) None

```



```

print(minimum(bst).val, maximum(bst).val)           # 3 18
print(range_query(bst, 6, 13))                      # [7, 10, 12]
print(ceiling(bst, 11), floor_value(bst, 11))       # 12 10
print(ceiling(bst, 19), floor_value(bst, 2))       # None None
print(kth_smallest(bst, 3))                         # 7

# THE CLASSIC ERROR: locally fine, globally broken
broken = from_list([10, 5, 15, 3, 7, 8, 18])
print(inorder(broken))                             # [3, 5, 7, 10, 8, 15, 18] - NOT sorted
print(search(broken, 8))                           # None ← 8 IS in the tree
print(find_in_plain_tree(broken, 8))               # TreeNode(8) ← the exhaustive search finds it

# insertion order decides the shape
sorted_inserts = None
for v in range(1, 8):
    sorted_inserts = insert(sorted_inserts, v)
print(height(sorted_inserts))                     # 6 ← a chain

balanced = build_from_sorted([1, 2, 3, 4, 5, 6, 7])
print(height(balanced), inorder(balanced))         # 2 [1..7]

print(height(sorted_inserts), height(balanced))     # 6 2 - same values

# duplicates, done properly
counting = CountingBST()
for v in (5, 3, 5, 8, 5, 3):
    counting.add(v)
print(counting.count_of(5), counting.count_of(3), counting.count_of(9)) # 3 2 0

# the comparison that justifies the structure
import bisect
values = list(range(0, 1000, 2))
print(bisect.bisect_left(values, 500)) # a sorted list: O(log n) search, O(n) insert

```

6 What it costs

Everything is $O(\text{height})$

```

search      O(height)
insert      O(height)
delete      O(height)      - day 107
min / max   O(height)
successor   O(height)
ceiling/floor O(height)
range query O(height + k)  - k results
inorder     O(n)

```

And **height** is the whole question.

```

balanced      height = log2(n)
n = 1,000      10
n = 1,000,000  20
n = 1,000,000,000  30

```

```

degenerate    height = n - 1
n = 1,000,000  999,999

```

```

n = 1,000,000
balanced BST search:  ~20 comparisons
degenerate BST search: ~1,000,000 comparisons
ratio:                50,000x

```

Say $O(\text{height})$ and then say what makes the height what it is. Saying $O(\log n)$ without the condition is the answer that gets challenged.

Against the alternatives

	search	insert	delete	ordered ops
sorted array	$O(\log n)$	$O(n)$	$O(n)$	excellent
hash map	$O(1)$	$O(1)$	$O(1)$	IMPOSSIBLE
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	excellent

```

n = 1,000,000, one insertion
sorted array:  up to 1,000,000 element moves
balanced BST:  ~20 comparisons and one pointer change

```

That row is why BSTs exist: a sorted array matches them on search and is catastrophic on insertion.

And the ordered operations, which is why hash maps do not replace them:

```

"everything between 20 and 40"
BST:      O(log n + k)  ~20 steps plus the results
hash map: O(n)         scan all 1,000,000 keys

"the smallest key"
BST:      O(log n)      walk left
hash map: O(n)         scan everything

"the next key above 37"
BST:      O(log n)
hash map: cannot be done at all without scanning

```

Space

```
per node in Python: ~120 bytes (value + two references + object overhead)
1,000,000 nodes:    ~120 MB
```

```
the same values in a sorted Python list of ints: ~40 MB
the same values in a set:                        ~60 MB
```

A BST costs about three times a sorted array in memory — the price of the pointers. That is the trade: memory and pointer-chasing, in exchange for insertion staying logarithmic.

Insertion order, measured

```
inserting 1,000 values
```

random order	expected height $\sim 2 \times \log_2(n) \approx 20$
sorted order	height 999
sorted then reversed	height 999
alternating outward	height $\sim \log_2(n)$

```
random insertion is FINE – the expected height is about  $2 \log n$ .
the danger is not randomness, it is ORDER, and real data arrives ordered.
```

That is the point worth making: a randomly built BST is fine without any balancing. The reason self-balancing trees exist is that real input is sorted by id, by timestamp, or by name far more often than it is random.

7 The traps

Trap 1 — stating the invariant about children instead of subtrees

```
"the left child is smaller and the right child is larger"
```

Wrong, and it is the most common answer. It permits the tree from the picture, where 8 sits in the right subtree of 10. Every local comparison passes; the structure is broken.

```
search(broken_tree, 8) → None
the value IS in the tree, and the search cannot reach it.
```

No exception, no error — just a lookup that fails for a value that is present.

Trap 2 — assuming $O(\log n)$

```
# "BST search is  $O(\log n)$ "
```

Only if balanced. A BST built by inserting sorted values is a chain, and sorted insertion is the common case, not a corner case.

```
for i in range(100000): insert(tree, i)
→ height 99,999
→ search is  $O(n)$ 
→ and a recursive traversal raises RecursionError
```

Say $O(\text{height})$, then say what determines the height.

Trap 3 — duplicates allowed on both sides

```
if value ≤ node.val:
    go left
else:
    go right
```

...in one function, and the reverse somewhere else. Now equal values exist on both sides and a search does not know which way to go, so it finds some of them and not others.

Pick one convention and apply it everywhere, or keep a count on the node.

Trap 4 — comparing types that do not order

```
insert(tree, "banana")    # into a tree of integers
```

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

A BST requires a **total order**. Anything comparable works — integers, strings, tuples — and anything mixed does not. Worth a sentence when the values are not numbers.

Trap 5 — using a BST when a hash map would do

If the only operations are “insert”, “delete” and “is it there”, a **hash map is $O(1)$ and simpler**. Choosing a BST there is a worse answer, and an interviewer may be checking exactly that.

The BST is justified by the ordered operations: range, min, max, successor, sorted iteration, k-th smallest. If none of those appear in the requirements, say so and use a hash map.

Trap 6 — assuming inorder sorted means it is a BST

It does, actually — and that is a valid check. But the reverse mistake is common: **checking only that each node's children are correctly placed does not prove sortedness.** The inorder check is $O(n)$ and correct; the local check is $O(n)$ and wrong.

Trap 7 — forgetting that the shape depends on insertion order

```
insert 4,2,6,1,3,5,7 → a perfect tree, height 2
insert 1,2,3,4,5,6,7 → a chain, height 6
```

Same values, same rule, same final set. **The tree is not determined by its contents** — a fact that surprises people and matters for the next several days.

Trap 8 — searching for a value using `==` on objects

If nodes hold objects rather than numbers, `<` must be defined and `=` must agree with it. A class with a custom `__lt__` and a default `__eq__` produces a tree where a search descends correctly and then fails the equality test. **Comparison and equality must be consistent**, which is what `functools.total_ordering` is for.

8 In the interview

How it gets asked

- The definitional one, usually first: “What makes a tree a binary search tree?”
- The operation: “Search for a value in this BST.” LeetCode 700.
- The comparison: “Why would you use a BST instead of a hash map?”
- The catch: “What is the worst case for a BST?”
- The construction: “Build a balanced BST from this sorted array.” LeetCode 108.

What to say out loud, in the first ninety seconds

1. **State the invariant with the word “all”.** “At every node, **all** the values in the left subtree are smaller and **all** the values in the right subtree are larger. Not just the immediate children — that is the distinction that matters, and it is what makes validating a BST a real problem.”

2. **Give the equivalent formulation.** “Equivalently, every node lives inside a range inherited from all of its ancestors: going left narrows the upper bound, going right narrows the lower bound.”
3. **Say what it buys.** “One comparison at each node discards an entire subtree, so search is $O(\text{height})$ instead of $O(n)$.”
4. **Immediately qualify it.** “ $O(\text{height})$, not $O(\log n)$ — that only holds if the tree is balanced, and nothing in the definition forces that. Inserting sorted data gives a chain.”
5. **Give the third formulation.** “An inorder traversal of a BST produces sorted order, which is the same rule stated differently and is the cleanest way to check one.”
6. **Say when you would not use one.** “If I only need insert, delete and lookup, a hash map is $O(1)$ and simpler. A BST earns its place when I need order — ranges, minimum, successor, sorted iteration.”

The follow-ups

“Why would you use a BST instead of a hash map?” “For the ordered operations, and only for those. A hash map beats a BST on every single-key operation — $O(1)$ against $O(\log n)$ — so if all I need is insert, delete and lookup, the hash map is both faster and simpler and I would say so. What it cannot do at all is anything involving order. ‘Give me everything between 20 and 40’ is $O(\log n + k)$ in a BST and a full scan in a hash map. ‘What is the smallest key’ is a walk down the left spine against a full scan. ‘What is the next key above 37, given 37 is not present’ is answerable in a BST and simply not answerable in a hash map without looking at everything. And sorted iteration is free from a BST and impossible from a hash map. So: **hash map unless I need order, then BST.**”

“What is the worst case?” “A chain — height $n - 1$ — and every operation becomes $O(n)$, which is a linked list with extra fields. The important part is that this is not an exotic input: it happens whenever values are inserted in sorted order, and real data arrives sorted by id, by timestamp or by name far more often than randomly. It is worth saying that a **randomly** built BST is fine — the expected height is about $2 \log n$ — so the enemy is not adversarial data, it is ordinary ordered data. That is why production implementations self-balance: AVL and red-black trees rotate on insertion to keep the height at $O(\log n)$, and B-trees do the same idea on disk for database indexes.”

“How do you handle duplicates?” “That is a decision I would state rather than assume, because the invariant as usually written says strictly less and strictly greater and leaves equality undefined. Four options: disallow them, which is what most inter-

view problems assume; send equal values consistently left; send them consistently right; or — the one I would actually build — **keep a count on each node**, so the tree holds distinct values with multiplicities. The count version keeps the tree smaller, makes ‘remove one occurrence’ trivial, and avoids the real failure mode, which is allowing equal values on both sides. If that happens, a search cannot know which way to go, and it finds some occurrences and not others.”

“Build a balanced BST from a sorted array.” “Take the middle element as the root, then recursively build the left subtree from the left half and the right from the right half. $O(n)$ time, and the height is exactly $\lceil \log_2 n \rceil$, so it is optimally balanced. The insight worth stating is *why* this is the answer to the degenerate-tree problem: inserting a sorted array one value at a time gives a chain, because each new value is larger than everything already there. Taking the middle first is precisely what stops that. It also shows that the shape of a BST is decided by the **insertion order**, not by the set of values — the same seven numbers can give a chain of height 6 or a perfect tree of height 2.”

“Prove your tree is actually a BST.” “Two correct ways and one wrong one. The wrong one, which is what most people write, is checking at each node that the left child is smaller and the right child is larger — that is local and it passes on trees that are globally broken, where a value sits in the right subtree of an ancestor it should be left of. The first correct way is to pass a **range** down: every node must lie strictly inside $(low, high)$, where going left sets **high** to the parent’s value and going right sets **low** to it. The second is to do an **inorder traversal and check that the values strictly increase**, which works because inorder on a BST is sorted order by definition. Both are $O(n)$; the inorder one is easier to get right and the range one uses $O(height)$ space instead of $O(n)$.”

“What does a broken BST actually do?” “It silently fails to find values that are present, and that is worth demonstrating rather than describing. If an 8 ends up in the right subtree of a 10, then a search for 8 compares with 10, goes left because 8 is smaller, and never looks at the branch where the 8 actually is. It returns ‘not found’ for a value that is in the structure. No exception, no error — and an exhaustive traversal would find it, which makes the bug look impossible when you are debugging.”

A model answer

Asked: *what makes a tree a binary search tree?*

“The ordering rule, and the precise wording matters more here than in almost any other definition in the subject.

At every node, all the values in its left subtree are smaller than it, and all the values in its right subtree are larger. Not the left *child* — the whole left *subtree*. That distinction is the entire content of the definition, and getting it wrong gives you a tree that satisfies every local comparison and is globally broken.

Another way to say the same thing, which I find clearer: **every node lives inside a range inherited from all of its ancestors.** The root’s range is unbounded. Going left narrows the upper bound to the parent’s value; going right narrows the lower bound. So a node in the right subtree of the root and the left subtree of that node has both bounds set by two different ancestors, and it must satisfy both. The classic failure is a node that satisfies its parent and violates its grandparent.

And a third equivalent statement: **an inorder traversal of a BST produces the values in sorted order.** That is not a consequence, it is the same rule restated, and it is the cleanest way to verify one.

What the rule buys is that a single comparison eliminates an entire subtree. Searching is a walk down from the root — if the target is smaller, everything to the right is irrelevant and you never look at it. That makes search $O(\text{height})$ instead of the $O(n)$ you would pay in a plain binary tree, where nothing about the values tells you which way to go.

I would immediately qualify that, though, because it is the thing people quote wrongly: it is $O(\text{height})$, **not** $O(\log n)$. Nothing in the definition says the tree is balanced. If you insert values in sorted order, every new value is larger than everything already there, so it goes to the far right and you build a chain of height $n - 1$ — a linked list with extra fields, where every operation is $O(n)$. And sorted insertion is not an unusual case; real data arrives sorted by id or timestamp far more often than randomly. A **randomly** built tree is fine — its expected height is about $2 \log n$ — so the problem is order, not adversaries. That is why production implementations self-balance with rotations, and why database indexes are B-trees.

Finally, when I would use one. A hash map beats a BST on every single-key operation — $O(1)$ against $O(\log n)$ — so if all I need is insert, delete and lookup, I would use the hash map. The BST earns its place when I need **order**: range quer-

ies, minimum and maximum, the next value above a key that is not present, sorted iteration, k-th smallest. Those are $O(\log n)$ in a BST and impossible in a hash map without scanning everything.”

9 Recall card

- **The invariant, with the word that matters:** at every node, **ALL values in the LEFT SUBTREE are smaller and ALL values in the RIGHT SUBTREE are larger.** Not the immediate children — that answer permits a globally broken tree in which `search` returns `None` for a value that is present, with no error.
- **Equivalent formulations:** every node lies inside a **RANGE** inherited from **ALL its ancestors** (left narrows the upper bound, right narrows the lower), and an **INORDER traversal gives sorted order**. Both are the same rule and both give correct validations.
- **One comparison discards a whole subtree** $\rightarrow O(\text{height})$. Say $O(\text{height})$, not $O(\log n)$ — that needs **balance**, which nothing guarantees. **Sorted insertion builds a chain** (height $n-1$, every operation $O(n)$), and sorted input is the *common* case. A **randomly** built BST is fine: expected height $\sim 2 \log n$. **The shape depends on insertion order, not on the set of values** — the same 7 values give height 6 or height 2.
- **Use a hash map unless you need order.** A hash map wins every single-key operation ($O(1)$) and cannot do **ranges, min/max, successor/ceiling, sorted iteration or k-th smallest** at all. Those are what a BST is for. Against a sorted array: same $O(\log n)$ search, but insertion is $O(\log n)$ **versus up to a million element moves**.
- **State a duplicates policy:** disallow, always-left, always-right, or — best — **a count on each node**. Equal values on both sides means a search cannot know which way to go and finds some occurrences and not others. And **build a balanced BST from sorted data by taking the middle as the root** — $O(n)$, height exactly $\lceil \log_2 n \rceil$.

Sharding, part one: choosing the key

1 What this is, and why they ask it

Sharding means splitting one logical table across several machines, so that each machine holds a subset of the rows and takes the reads and writes for that subset.

Three sentences. It is the only technique that scales **writes**, which is why it exists — **replication** copies every write to every machine and **read replicas** hit a hard ceiling because of that. The whole design reduces to one decision, the **shard key**, and that decision is very expensive to change later. And the reason it is expensive is that sharding takes things away permanently: **no joins across shards, no transactions across shards, and no globally unique constraint except on the key itself**.

They ask it because the answer separates people who have read that sharding scales writes from people who know what it costs. The good answer names a key, says which query it makes fast, says which query it makes terrible, and says what the design does about that second query. “*What breaks after you do?*” is the question — the first half is setup.

2 The story

The sub-registrar’s office had one records room and it had run out of floor.

Eighty-odd years of documents in one long room with racks up to the ceiling, and it had reached the point where two people looking for two different things could not get past each other in the aisle.

They were given three more rooms down the corridor, and the question of how to divide the documents took a committee three meetings.

The first proposal was by surname. A to F in room one, G to L in room two, and so on. It was easy to explain and everybody could remember it, and it was tried for about five months.

It did not work, and the reason was obvious in hindsight to everybody except the person who had proposed it. The room with the S names had nearly half the documents. There were four rooms and one of them had a permanent queue outside it while another was used twice a day. Splitting the alphabet evenly does not split the *people* evenly, because names are not spread evenly.

The second proposal came from the youngest clerk and was resisted because it sounded stupid. He said: take the document number, divide it by four, and use the remainder. Nought goes in room one, one in room two, two in room three, three in room four.

Nobody could see the sense of it, because it put documents from the same street, the same family and the same week in four different rooms. But it split the work perfectly evenly, and after two weeks nobody was queuing anywhere.

What it cost showed up in the third month.

A lawyer came in wanting every transaction on one particular survey number over twenty years. Under the old system, or even under the surname system, that had been one visit to one place. Now it meant walking to four rooms, asking four people, and combining four answers, and if one of those rooms was locked because the man had gone for lunch, you had nothing.

And the second thing, which was worse and took a year to surface. Documents had always been numbered in one sequence, and each room now issued its own numbers. Two rooms issued the same number in the same week, to two different documents, and nobody noticed until the two turned up in the same court case.

The head clerk's rule afterwards, which was written on a board and stayed there, was two lines.

Decide what you are splitting by before you split, because moving it afterwards means moving everything.

And write down, in advance, which questions will now need all four rooms — because those are the ones that will be asked.

3 The idea in plain English

The records office has been sharded, and it hit every consequence in the right order: the hotspot, the cross-shard query, and the identifier collision.

- Each room is a **shard** — a machine holding a subset of the rows.
- The rule for deciding which room a document goes in is the **shard key** and the **sharding strategy**.
- The S room with half the documents is a **hotspot**.
- The lawyer's twenty-year search is a **cross-shard query**, also called a **scatter-gather**.

- Two rooms issuing the same number is the **unique identifier** problem.

What sharding is, and what it is not

REPLICATION	every machine holds ALL the data scales READS, not writes; every replica applies every write
SHARDING	every machine holds SOME of the data scales WRITES and STORAGE; each shard sees only its own traffic

They are orthogonal and real systems use both: shard the data across `N` groups, and replicate each shard within its group. Say that early — a candidate who treats them as alternatives has misunderstood both.

The four strategies

Range sharding. Rows go to a shard by where the key falls in an ordered range.

```
user_id 1 - 1,000,000 → shard 1
        1,000,001 - 2,000,000 → shard 2
```

- + range queries stay on one shard: "users 500 to 900" is one machine
- + easy to split a shard: cut a range in two
- HOTSPOTS, and they are almost guaranteed with any sequential key

Every new user has the highest id, so every new user goes to the last shard. With a timestamp key it is worse: 100 percent of writes go to one shard while the others are idle. This is the single most common sharding mistake, and it has a name — **the append hotspot**.

Hash sharding. Apply a hash function to the key and take the remainder.

```
shard = hash(user_id) % 4
```

- + even distribution, essentially for free
- + no hotspot from sequential keys, because the hash destroys the ordering
- range queries are DEAD: adjacent keys land on different shards
- adding a shard moves almost everything — unless you use consistent hashing

The youngest clerk's proposal. **Even distribution and no locality**, and those are the same property seen from two sides.

Directory sharding. Keep an explicit lookup table: this key lives on that shard.

- + total flexibility; move a single tenant without moving anything else
- + you can give a big customer their own shard
- the directory is a lookup on every request, and a single point of failure

This is what multi-tenant systems actually do, because customers are wildly different sizes and you need to be able to move one.

Geographic sharding. By region — European users in Europe, Indian users in India.

- + low latency for local users; often a legal requirement (data residency)
- a user who travels is far from their data
- regions are not equal sizes

Choosing the key: four requirements

One — it must appear in almost every query. If the key is `user_id` and a query filters by `email`, that query has no idea which shard to ask, so it asks all of them. **The key must match your dominant access pattern**, and knowing that pattern is a prerequisite you should ask about out loud.

Two — high cardinality. The number of distinct values bounds the number of shards you can ever have. Sharding by `country` gives you about 200 buckets — and one of them is enormous. Sharding by `gender` gives you two. **The key must have far more distinct values than you will ever have shards.**

Three — even distribution. Not just many values: **evenly used** values. Surnames have high cardinality and a very skewed distribution — that was room three's problem.

Four — stability. The key must not change. Changing a row's shard key means deleting it from one shard and inserting it into another, which is a distributed transaction, which is the thing you gave up. **Shard by `user_id`, never by `email` or `city`.**

What breaks — the second half of the question

Cross-shard joins. `SELECT ... FROM orders JOIN users ON ...` where the two tables are sharded differently. The database cannot do it. Your options are to denormalise (copy the user's name onto the order), to query both and join in the application, or to shard both tables by the same key so related rows land together — which is **co-location**, and it is the good answer.

Cross-shard transactions. "Move money from A to B" is trivial on one machine and requires two-phase commit or a saga across two — [days 120 and 121](#). Most systems avoid the problem by choosing a key that keeps the things that change together on one

shard.

Unique constraints. A shard can only enforce uniqueness within itself. **A globally unique email address cannot be enforced by the shards** — you need a separate service, or a second table sharded by email whose job is to own that uniqueness.

Auto-increment ids. Two rooms issuing the same number. The fixes: UUIDs (large and unordered), a central allocator (a bottleneck), per-shard ranges (shard 1 gets 1–1M), or **Snowflake ids** — a timestamp, a machine id and a counter packed into 64 bits, which are unique, roughly ordered, and need no coordination. **Snowflake is the standard answer** and worth naming.

Aggregations. `COUNT(*)`, `SUM`, `ORDER BY ... LIMIT 10` across all shards must query every shard and combine. A top-10 needs the top 10 from each shard, merged — which is [merging k sorted lists](#).

Rebalancing. Adding a shard with plain `hash % N` remaps almost every key. That is [day 107](#) and [day 108](#).

The question that chooses the key

What is the single most common query, and what does it filter by?

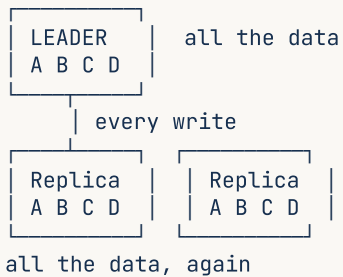
For a social product it is “give me this user’s timeline” → shard by `user_id`. For a chat app it is “give me this conversation’s messages” → shard by `conversation_id`. For a multi-tenant SaaS it is “everything for this customer” → shard by `tenant_id`, and that one is the cleanest of all, because tenants never share data.

Then write down the queries the key makes expensive, before you shard, because those are exactly the ones somebody will ask for in month three.

4 The picture

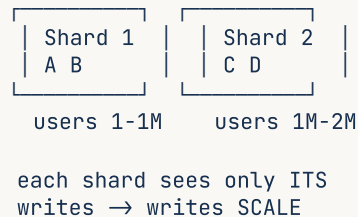
Replication and sharding, which do different things.

REPLICATION (day 104)



scales READS
every machine does every write

SHARDING (today)



scales WRITES and STORAGE
each machine does 1/N of the writes

REAL SYSTEMS DO BOTH: shard into N groups, replicate within each group.

The strategies, on the same data:

users 1..1,000,000, four shards

RANGE

shard 1: ids 1 - 250,000
shard 2: ids 250,001 - 500,000
shard 3: ids 500,001 - 750,000
shard 4: ids 750,001 - 1,000,000

"users 1000 to 2000"
→ ONE shard

new signups (id 1,000,001+)
→ ALL go to shard 4
→ 100% of writes on 25% of
the hardware ← THE APPEND HOTSPOT

HASH

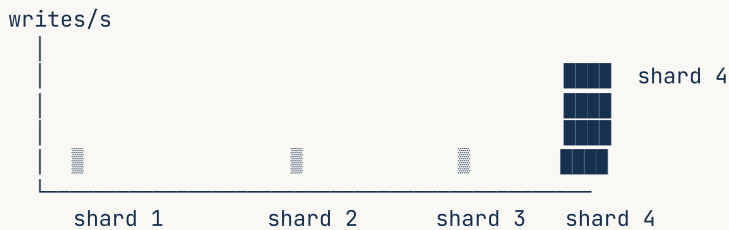
shard 1: $\text{hash}(\text{id}) \% 4 = 0$
shard 2: $\text{hash}(\text{id}) \% 4 = 1$
shard 3: $\text{hash}(\text{id}) \% 4 = 2$
shard 4: $\text{hash}(\text{id}) \% 4 = 3$

"users 1000 to 2000"
→ ALL FOUR shards, and 1,000
separate lookups

new signups
→ spread evenly
across all four

The hotspot, drawn as load:

RANGE-SHARDED BY TIMESTAMP OR SEQUENTIAL ID



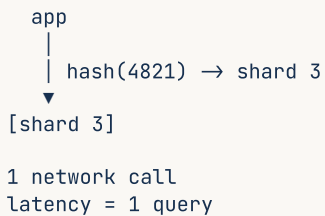
25% of the hardware is doing 100% of the writes.
The other three machines are paid for and idle.

HASH-SHARDED

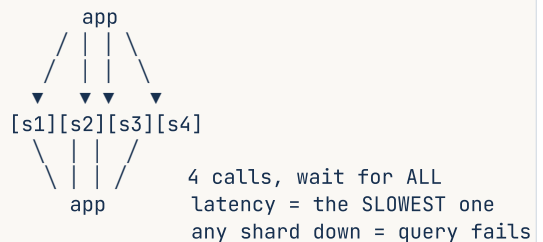


The scatter-gather, which is the cost of hashing:

QUERY ON THE SHARD KEY
"orders for user 4821"



QUERY ON ANYTHING ELSE
"orders placed yesterday"



That last line is the part people miss: a scatter-gather's latency is the maximum of its parts, not the average, and its availability is the product of all of them. Four shards at 99.9 percent each give a scatter-gather query 99.6 percent availability.

5 How it actually works

Where the routing decision lives

IN THE APPLICATION	the code computes the shard and picks a connection simple, and every service must implement it identically
IN A PROXY	Vitess, Citus, ProxySQL: the app sees one database and the proxy routes, and can even fan out and merge → the usual answer at scale
IN THE DATABASE	MongoDB, Cassandra, DynamoDB, CockroachDB shard natively → the routing is invisible; the CONSTRAINTS are not

Even when the database shards for you, the constraints remain: MongoDB still cannot join across shards efficiently, and DynamoDB still makes you choose a partition key that decides everything.

A worked choice: an e-commerce order table

candidate keys	what it makes fast	what it kills
order_id (hash)	"fetch order 8412"	"my orders" → all shards
user_id (hash)	"my orders" → ONE shard	"orders today" → all shards
created_at (range)	"orders today" → one shard	APPEND HOTSPOT: every new order hits the same shard
seller_id (hash)	"seller's orders" → one shard	buyer queries → all shards

user_id wins, and the reasoning is the answer: the dominant query is *"show me my orders"*, users are numerous and roughly evenly active, and a user's id never changes. Then write down what it costs: *"orders placed today across all users"* becomes a scatter-gather, so analytics goes to a separate system fed by a change stream rather than querying the shards.

And co-locate. Shard `order_items` by the same `user_id`, not by `order_id`, so an order and its items are on one machine and the join stays local. That single decision removes most cross-shard joins from the system.

The composite-key trick for hotspots

If you must range-shard on something sequential, prefix the key with something that spreads:

bad:	shard by timestamp → every write goes to the newest shard
good:	shard by (user_id, timestamp) → writes spread across users; one user's history is still contiguous and range-queryable within its shard

This is exactly what Cassandra's partition key plus clustering key does, and it is the standard fix for the append hotspot: **partition by something that spreads, order by something that is sequential.**

Handling the queries the key makes slow

Three answers, and a good candidate names all three:

A global secondary index — a second table sharded by the other field, mapping it back to the primary key. Costs a second write on every insert, and it is eventually consistent unless you pay for a distributed transaction.

Denormalise — store the data twice, sharded two ways. Same trade: writes cost more, reads get cheaper.

A separate system — push changes to a search index (Elasticsearch) or an analytics warehouse via a change stream. **This is what real systems do for “orders today across all users”**, because those queries are analytical and do not belong on the serving path at all.

What real systems do

- **Vitess** shards MySQL and runs YouTube; it puts the routing in a proxy layer and supports resharding without downtime.
 - **Citus** does the same for PostgreSQL, with an explicit distribution column and co-location groups.
 - **MongoDB** requires a shard key at collection creation and — famously — historically could not change it, which taught a generation of teams that the decision is close to permanent.
 - **DynamoDB** makes it unavoidable: you choose a **partition key** and a **sort key**, and every access pattern must be designed around them up front. Its per-partition throughput limit means a hot partition key is throttled even when the table has capacity — the hotspot problem, priced.
 - **Cassandra** is the same model with the composite key trick built into its data model.
 - **Instagram** sharded PostgreSQL by user id early, and published their scheme: ids embed the shard, so any id tells you where its row lives. That is a genuinely good trick worth mentioning.
-

6 The numbers

When you actually need it

```
one machine sustains      ~5,000 writes/second (relational, indexed)
one machine holds         a few TB comfortably
```

```
you need sharding when EITHER:
  write rate > ~5,000/s and vertical scaling is exhausted
  data size > the largest machine's disk or working set
```

```
10,000 writes/s, 4 shards → 2,500 writes/s each   comfortable
10,000 writes/s, 1 machine → impossible
```

And not before. The cost of sharding is permanent; the cost of one more vertical step is a restart.

Hotspot arithmetic

```
range-sharded by timestamp, 4 shards, 10,000 writes/s

shard 4 (the newest range): 10,000 writes/s → over capacity, failing
shards 1-3:                0 writes/s → paid for, idle

effective capacity: 5,000 writes/s from 4 machines
→ you have quadrupled the cost and gained NOTHING
```

That is the number that makes the append hotspot real. Hash sharding on the same hardware gives 2,500 per shard and works.

Skew, measured

```
sharding a social product by user_id, hashed

the average user      ~100 followers, ~5 posts/day
the top 0.01% user    ~50,000,000 followers, and their writes fan out

→ the SHARD holding a celebrity does not have more rows, it has more TRAFFIC
→ observed skew in real systems: 5-20x between the busiest and quietest shard
```

Even a perfect hash does not fix skew in activity. The fix is not a better hash — it is special-casing the outliers, which for a social product means handling celebrity accounts differently.

Scatter-gather cost

```
a query on the shard key          1 network call, ~1 ms
a scatter-gather over 16 shards    16 calls in parallel
                                   latency = the SLOWEST of 16

if each shard's p99 is 10 ms:
  P(at least one is slow) = 1 - 0.99^16 = 15%
  → the query's p50 is now close to the shards' p99
```

Fanning out turns your p99 into your p50. That is the tail-latency amplification argument and it is worth stating precisely.

Availability too:

```
16 shards, each 99.9% available
a query needing ALL of them: 0.999^16 = 98.4%
→ from 8.8 hours of downtime a year to 140 hours
```

Storage and capacity per shard

1 billion rows × 1 KB	= 1 TB	
16 shards	= 62.5 GB each	comfortable
4 shards	= 250 GB each	fine
1 machine	= 1 TB	possible, but the working set will not fit in memory

The cost of getting the key wrong

```
changing the shard key = rewriting every row into a new arrangement

1 TB, 16 shards, moving everything
at 100 MB/s per shard, in parallel    ~10 GB per shard ÷ 100 MB/s ≈ 100 s of pure copy
in practice, with dual writes,
verification and a cutover             WEEKS of engineering, months of calendar
```

“Weeks of engineering” is the honest number, and it is why the decision deserves the meeting.

7 The trade-offs

Range or hash?

Range keeps related keys together, so range queries are local and splitting a shard is easy. It almost guarantees a hotspot on any sequential key, and sequential keys are the normal case.

Hash distributes perfectly and destroys locality — those are the same property. Range queries become scatter-gathers, and adding a shard remaps everything unless you use consistent hashing.

Take hash by default; take range when range queries dominate and the key is not sequential. And when you need both, use the composite trick: **partition by something that spreads, order by something sequential.**

Sharding costs you three things permanently

Joins. Only within a shard. Fixed by co-locating related tables on the same key, or by denormalising.

Transactions. Only within a shard. Fixed by choosing a key that keeps things that change together together — which is the real reason `tenant_id` is such a good key.

Unique constraints. Only on the shard key. A globally unique email needs a separate owner.

Say all three when asked what breaks. They are the answer.

Fewer big shards or more small ones?

Fewer means less scatter-gather fan-out and less operational surface. **More** means finer rebalancing and smaller blast radius when one fails.

The usual answer is **more logical shards than physical machines** — say, 1,024 logical shards mapped onto 8 machines. Then adding a machine moves logical shards rather than rehashing keys, and you never re-shard again. **That indirection is worth mentioning; it is what makes rebalancing tractable**, and it is [tomorrow's](#) subject.

When not to shard

I would not shard if vertical scaling has room, if the read load is the problem (that is caching and replicas), or if the data is under a terabyte and the write rate is under a few thousand a second.

And I would not shard a table that is small but joined against everything. Reference tables — countries, currencies, product categories — should be replicated to every shard, not split.

Where this breaks

- **The unbounded partition.** One tenant, one celebrity, one popular conversation grows without limit and its shard cannot be split, because a shard key value cannot be divided. **This is the failure that has no clean fix** — you must design the key so

that no single value can grow indefinitely, which usually means a composite key with a bucket.

- **Cross-shard analytics** are permanently awkward. Send changes to a warehouse and stop trying.
 - **Schema changes** now happen `N` times, and a partially-applied migration leaves shards disagreeing.
 - **The routing layer** becomes a critical dependency with its own availability.
-

8 In the interview

How it gets asked

- The pair: *“How would you shard this table? What breaks after you do?”*
- The choice: *“Range or hash? Why?”*
- The trap: *“You’ve sharded by timestamp. What happens?”*
- The consequence: *“How do you get a globally unique order id now?”*
- The limit: *“When would you not shard?”*

What to say out loud, in the first ninety seconds

1. **Separate it from replication.** “Replication copies all the data everywhere and scales reads. Sharding splits the data and scales writes and storage. They are orthogonal — I would shard into `N` groups and replicate within each group.”
2. **Ask the question that chooses the key.** “What is the dominant query, and what does it filter by? The shard key has to be the thing almost every query already knows.”
3. **Name the key and give all four reasons.** “`user_id`, hashed: it is in the dominant query, it has high cardinality, users are roughly evenly active, and a user’s id never changes — which matters, because changing a row’s shard key means moving it between machines.”
4. **Say hash and say why not range.** “Hash, because a range on any sequential key gives an append hotspot — every new row goes to the newest shard, so a quarter of the hardware does all the writes.”
5. **Answer the second half before it is asked.** “What breaks: no joins across shards, no transactions across shards, and no unique constraint except on the key. Plus auto-increment ids, which now collide between shards.”

6. **Name the query you have made expensive.** “All orders placed today’ now needs every shard, so I would not serve it from here at all — that goes to a warehouse fed by a change stream.”

The follow-ups

“**You’ve sharded by timestamp. What happens?**” “An **append hotspot**, and it is the worst outcome available because it costs money and delivers nothing. Every new row has the newest timestamp, so every write goes to the shard holding the newest range. With four shards and ten thousand writes a second, one machine is receiving all ten thousand and failing while the other three are paid for and idle — so I have quadrupled the cost and my effective capacity is still one machine’s. Reads are just as bad, because recent data is the most-read data, so the hot shard is hot in both directions. The fix, if I genuinely need time-ordered locality, is the composite key: **partition by something that spreads, and order by time within the partition.** (`user_id, timestamp`) spreads the writes across users while keeping one user’s history contiguous and range-queryable. That is exactly Cassandra’s partition-key-plus-clustering-key model.”

“**What breaks after you shard?**” “Four things, and three of them are permanent. **Joins** only work within a shard, so I would co-locate related tables on the same key — `order_items` sharded by `user_id` rather than `order_id`, so an order and its items live together — and denormalise where I cannot. **Transactions** only work within a shard, so ‘move money from A to B’ becomes two-phase commit or a saga, and the better answer is to choose a key that keeps the things that change together on one machine. **Unique constraints** only work on the shard key, so a globally unique email needs a separate table sharded by email whose job is to own that uniqueness. And **auto-increment ids** collide, because each shard has its own counter — the standard fix is Snowflake-style ids: a timestamp, a machine identifier and a counter packed into 64 bits, which are unique and roughly ordered and need no coordination.”

“**Range or hash?**” “**Hash by default.** It distributes evenly for free, and evenness is the thing you are actually buying — a shard scheme that is uneven has cost you money and given you nothing. The price is that it destroys locality, so range queries become scatter-gathers, and adding a shard remaps almost every key unless I use consistent hashing. **Range** is right when range queries dominate *and* the key is not sequential — sharding by something like a geographic region or a product category, where the ranges are stable and the new data does not all land at one end. The honest summary is that hash gives you distribution and takes away locality, and those are the same property viewed from either side.”

“How expensive is a query that does not use the shard key?” “It becomes a **scatter-gather**: ask every shard, wait for all of them, merge. Two costs that people usually only count one of. **Latency** — the query takes as long as the *slowest* shard, not the average. If each shard has a p99 of 10 ms and I fan out to sixteen, there is about a fifteen percent chance at least one is slow, so my query’s median is now close to the shards’ p99. That is tail-latency amplification. And **availability** — the query needs all sixteen to answer, so at 99.9 percent each, the query is 98.4 percent available, which is 140 hours of failure a year instead of nine. So I would not serve such queries from the shards at all: either a **global secondary index** sharded by the other field, or push changes into a search index or a warehouse and query there.”

“When would you not shard?” “Whenever something cheaper still has room, and usually something does. If the **reads** are the problem, that is caching and read replicas, and a ninety percent cache hit rate removes ten times more load than sharding would. If **one machine still has headroom**, scale up — up to about 32 to 64 vCPU the price per unit of power is flat, and it is one command and a restart. Sharding is for when the **write rate** exceeds what one machine can do, around five thousand writes a second for a relational database, or the **data no longer fits** the largest machine. The reason to be strict about this is that sharding’s cost is not the migration — it is permanent: every feature built afterwards has to live within the shard key I chose today, and changing that key later is weeks of engineering with dual writes and a cutover. I would also not shard reference tables — countries, currencies, categories. Those get replicated to every shard, not split.”

“How do you pick the key concretely?” “I ask what the single most common query is and what it filters by, because the key has to be something almost every query already knows. Then I check four things. **Is it in the dominant query** — otherwise every request is a fan-out. **Cardinality** — the number of distinct values bounds how many shards I can ever have, so `country` gives me 200 buckets with one enormous one, and `gender` gives me two. **Even distribution** — not just many values, but evenly used ones; surnames have high cardinality and terrible skew. And **stability** — the value must never change, because changing it means deleting from one shard and inserting into another, which is precisely the distributed transaction I gave up. For most consumer products that lands on `user_id`; for chat it is `conversation_id`; and for multi-tenant SaaS it is `tenant_id`, which is the cleanest of all because tenants never share rows.”

A model answer

Asked: *how would you shard this table, and what breaks after you do?*

“First, what sharding is *for*, because it is often confused with replication. Replication puts all the data on every machine and scales **reads** — and it has a hard ceiling, because every replica applies every write. Sharding splits the data so each machine holds a subset, which is the only thing that scales **writes** and **storage**. They are orthogonal, and a real deployment does both: shard into `N` groups, and replicate within each group.

Before choosing a key I would ask one question: **what is the dominant query, and what does it filter by?** The shard key has to be something almost every request already knows, because a query that does not include it has to ask every shard.

For an orders table I would take `user_id`, **hashed**, and I would give four reasons. It is in the dominant query — ‘show me my orders’. It has high cardinality, so it does not limit how many shards I can have. Users are roughly evenly active, so the distribution is reasonable. And a user’s id never changes, which matters more than it sounds: changing a row’s shard key means deleting it from one machine and inserting it on another, which is a distributed transaction — the exact thing sharding takes away.

Hash rather than range, because a range on anything sequential produces an **append hotspot**: every new row has the newest key, so every write lands on the newest shard. With four shards and ten thousand writes a second, one machine takes all ten thousand and fails while three sit idle — I would have quadrupled the cost for no capacity at all. If I needed time locality I would use a composite key: partition by something that spreads, order by time inside it.

I would also **co-locate**: shard `order_items` by `user_id` too, not by `order_id`, so an order and its items live on the same machine and that join stays local. That single choice removes most of the cross-shard joins from the system before they exist.

Now what breaks, which is the more important half. **Joins** work only within a shard. **Transactions** work only within a shard, so anything spanning two users needs two-phase commit or a saga. **Unique constraints** work only on the shard key, so a globally unique email needs its own table sharded by email to own that uniqueness. And **auto-increment ids** collide, because every shard has its own counter — I would use Snowflake-style ids, a timestamp plus a machine id plus a counter in 64 bits, which are unique and roughly sorted and need no coordination.

And I would name the query I have just made expensive: **‘all orders placed today, across all users’** now touches every shard. A scatter-gather costs the *slowest* shard’s latency, not the average, and its availability is the product of all of them — sixteen shards at 99.9 percent gives that query 98.4 percent. So I would not serve it from here: analytics goes to a warehouse fed by a change stream, and if I need a user-facing lookup by a non-key field, that is a global secondary index sharded by that field.

Finally, the thing I would say before any of this: **I would not shard unless I had to**. If reads are the problem, that is caching and replicas. If one machine still has headroom, scale up — it is one command. Sharding is for a write rate past what one machine can take, or data past what one machine can hold, and its cost is not the migration, it is that every feature afterwards lives inside the key I chose today.”

9 Recall card

- Replication copies all the data everywhere and scales READS; sharding splits the data and scales WRITES and STORAGE. They are orthogonal — real systems shard into N groups and replicate within each.
- The question that chooses the key: what is the dominant query, and what does it filter by? Then four requirements: in the dominant query · high cardinality · evenly distributed · never changes. That is usually `user_id`, `conversation_id`, or — cleanest of all — `tenant_id`.
- Hash by default; range only when range queries dominate AND the key is not sequential. Range on anything sequential gives an **append hotspot**: at 10,000 writes/s over 4 shards, **one machine takes all 10,000 and three sit idle** — quadruple the cost, no capacity. The fix is a **composite key: partition by something that spreads, order by time within it**.
- What breaks, permanently: no cross-shard joins · no cross-shard transactions · no unique constraint except on the key. Plus auto-increment ids collide — use **Snowflake ids** (timestamp + machine + counter in 64 bits). **Co-locate related tables on the same key**, and replicate small reference tables to every shard instead of splitting them.
- A query without the shard key is a scatter-gather, and it costs twice: latency is the **SLOWEST shard** (16 shards with a 10 ms p99 → ~15% chance one is slow, so your p50 becomes their p99) and **availability is the product** ($16 \times 99.9\% =$

98.4%, i.e. 140 hours a year). **Do not shard until a single machine is out of write capacity (~5,000 writes/s) or disk** — the cost is not the migration, it is that every future feature lives inside today's key.

PRACTICE

Practice — Binary search trees: the ordering property

DSA topic: Binary search trees: the ordering property **System design topic:** Sharding, part one: choosing the key

Code these, in this order

One rule for the whole set: **say the invariant out loud before each problem, using the word “subtree”**. If you catch yourself saying “the left child”, stop and say it again properly. That habit is what day 108 is going to test.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Search in a Binary Search Tree	LeetCode 700 (Easy)	One comparison discarding a subtree; write it iteratively.
2	Convert Sorted Array to BST	LeetCode 108 (Easy)	Why taking the middle prevents the degenerate case.
3	Range Sum of BST	LeetCode 938 (Easy)	Skipping whole subtrees using the ordering.
4	Kth Smallest Element in a BST	LeetCode 230 (Medium)	Inorder as sorted order, and stopping early.

On problem 1, write it both ways and pick

Recursive and iterative. Say why the iterative version is preferable here, in terms of what happens on the way back up.

On problem 2, then build the bad tree

After building the balanced one, insert the same sorted array one value at a time into an empty BST. Print both heights. Record the two numbers.

On problem 3, prove the pruning

Add a counter for nodes visited. Compare against a version that visits everything and filters. Run it on a 1,000-node BST with a narrow range and record both counts.

On problem 4, do it iteratively and say why

Say what the recursive version would do that you do not want, and give the complexity in terms of `height` and `k`.

The invariant drill

1. State the invariant using the word “subtree”.
2. State it again as a range inherited from ancestors.
3. State it a third time in terms of inorder traversal.
4. Say why all three are the same statement.

The classic-error drill

1. Build the tree with an 8 in the right subtree of 10.
2. Print its inorder traversal and say what is wrong with it.
3. Run `search` for 8 and record the result.
4. Run an exhaustive search for 8 and record the result.
5. Say in one sentence why this bug is hard to debug.

The ranges drill

1. Draw a seven-node BST and label every node with its permitted range.
2. Say what happens to the range when you go left, and when you go right.
3. Say which ancestor sets each bound for a node two levels down.
4. Say why “check the parent” is not enough.

The what-it-buys drill

1. Write `search` in four lines.
2. Say what a single comparison eliminates.
3. Give the comparison count for a balanced million-node BST.
4. Give it for the same values in a plain binary tree, and for a chain.

The balance drill

1. Insert 1 to 7 in order and give the height.
2. Insert them middle-first and give the height.
3. Say what this proves about what determines a tree’s shape.
4. Say what the expected height of a randomly built BST is.
5. Say why sorted input is the common case, not the exotic one.

The against-alternatives drill

1. Fill in the comparison table for sorted array, hash map and balanced BST.
2. Give four operations a hash map cannot do.
3. State the one-sentence rule for choosing between them.

4. Compute an insertion into a million-element sorted array against a balanced BST.
5. Give the memory ratio between a BST and a sorted list of the same values.

The duplicates drill

1. Name the four conventions.
2. Say which one most problems assume and which one you would build.
3. Say what goes wrong if equal values are allowed on both sides.
4. Write the counting node and the `add` method.

The break-it drill

Trigger each and record the exact output or error:

1. `search` on the locally-correct, globally-broken tree.
 2. Inserting a string into a tree of integers.
 3. Inserting 1 to 10,000 in order, then recursing over the result.
 4. Allowing duplicates left in one function and right in another, then searching.
 5. Using a BST where a hash map would do — state what you lost.
-

The two-techniques drill

1. Define replication and sharding in one line each.
2. Say which scales reads and which scales writes.
3. Say why they are orthogonal and what a real deployment does.
4. Say why replication has a hard ceiling.

The choosing drill

1. State the one question that chooses the shard key.
2. Name the four requirements a key must satisfy.
3. For each requirement, give a key that fails it and say how.
4. Choose a key for: a social feed, a chat app, multi-tenant SaaS, an orders table.

The strategies drill

1. Name all four strategies.
2. For each, give one thing it makes fast and one it makes terrible.
3. Say which one multi-tenant systems use and why.
4. Say which one is sometimes a legal requirement.

The hotspot drill

1. Describe the append hotspot in two sentences.
2. Compute the load distribution for 10,000 writes/s over 4 range shards on a timestamp.
3. Say what the effective capacity is, and what it cost.
4. Give the composite-key fix and name the database whose data model is built on it.
5. Say why hashing does not fix skew in *activity*, and give the social-product example.

The what-breaks drill

1. Name the three permanent losses.
2. For each, give the workaround and its cost.
3. Say what happens to auto-increment ids, and name the standard fix.
4. Say what co-location means and give a concrete example.
5. Say what you do with small reference tables.

The scatter-gather drill

1. Describe a scatter-gather.
2. Say why its latency is the maximum rather than the average.
3. Compute the probability that at least one of 16 shards is slow, given a 10 ms p99.
4. Compute the availability of a 16-shard fan-out at 99.9% each.
5. Name three ways to avoid serving such a query from the shards.

The when-not-to drill

1. Give three situations where you would not shard, and what you would do instead.
2. State the write rate and data size that justify it.
3. Say what the real cost of sharding is, and why it is not the migration.
4. Estimate the calendar cost of changing a shard key later.

The failure drill

For each, say what happens and what you would add:

1. The table is sharded by `created_at`.
2. Someone asks for “all orders placed today”.
3. Two shards issue the same order number.
4. A user changes the field you sharded by.
5. One tenant grows to ten times every other tenant.

6. A migration is applied to 15 of 16 shards.

One of the six has no clean fix. Name it and say what you must do at design time instead.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What makes a tree a binary search tree?* The invariant with the word “subtree”, the range formulation, inorder as the third statement, what one comparison buys, the immediate qualification that it is $O(\text{height})$ not $O(\log n)$, and when you would use a hash map instead.
 2. *How would you shard this table? What breaks after you do?* Sharding separated from replication, the question that chooses the key, the key with all four reasons, hash with the append-hotspot justification, co-location, then the three permanent losses plus ids, and the query you have just made expensive.
 3. *What is the worst case for a BST, and how do real systems avoid it?* The chain from sorted insertion, why sorted input is normal, the expected height of a random tree, and self-balancing plus B-trees.
-

Before you move on

- [] I state the invariant using the word “subtree”, every time.
- [] I can give all three equivalent formulations.
- [] I built the broken tree and saw `search` fail to find a value that is present.
- [] I can label a drawn BST with every node’s permitted range.
- [] I say $O(\text{height})$ and then say what determines the height.
- [] I know the expected height of a randomly built BST.
- [] I can fill in the comparison table against sorted arrays and hash maps.
- [] I can name four operations a hash map cannot do.
- [] I can state a duplicates policy and say what breaks without one.
- [] I know why the shape depends on insertion order, with the two heights to prove it.
- [] I can separate replication from sharding in one line each.
- [] I can state the question that chooses the shard key.

- [] I can name the four requirements and a key that fails each.
- [] I can describe the append hotspot with the load arithmetic.
- [] I know the composite-key fix and the database built on it.
- [] I can name the three permanent losses and the id problem.
- [] I can compute scatter-gather latency and availability for 16 shards.
- [] I can say when I would not shard, and what I would do instead.
- [] I answered all three questions above out loud.

BST insert, search, and delete

DATA STRUCTURES AND ALGORITHMS

BST insert, search, and delete

You can delete a node with two children and explain the successor swap.

SYSTEM DESIGN

Sharding, part two: rebalancing and hot spots

You can describe resharding without downtime and identify a hot shard.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Delete this node from the BST. It has two children.*
- *One shard is taking 60 percent of the traffic. Now what?*

1 What this is, and why they ask it

Three operations on a binary search tree. **Search** walks down comparing. **Insert** walks down to where the value belongs and puts a leaf there. **Delete** is the hard one, and it is the whole lesson.

Three sentences. Search and insert are four lines each and nobody gets them wrong. Delete has **three cases** — no children, one child, two children — and the third one cannot simply remove the node, because something has to occupy that position and almost nothing may. And the value that may is **the node's inorder neighbour**: the smallest value in its right subtree, or the largest in its left. There is no other choice, and being able to say *why* there is no other choice is what the question is for.

They ask it because it is the first tree operation that **modifies structure**, and modifying a pointer-based structure with recursion has a specific idiom — `node.left = delete(node.left, key)` — that people either know or fight against for ten minutes. And because “delete a node with two children” has a one-sentence answer that most candidates arrive at by trial and error rather than by reasoning.

2 The story

The photographs were on the long shelf in the front room and there were nineteen of them, in order, from the oldest at the left end to the newest at the right.

Sarojini had put them in that order in 1998 and had been maintaining it ever since. It was not decorative — it was how she found things. If somebody asked which year her brother-in-law's daughter got married, she went to roughly the middle and worked outwards, and she never had to look at more than four or five.

In March, one of them — the wedding group from 1974, in the heavy black frame — had to go to the man in the market to have the glass replaced.

That left a gap in the middle of the shelf, and Sarojini's daughter-in-law, trying to be helpful, closed the gap by sliding everything after it one place to the left.

Sarojini was not pleased, and her objection was not about tidiness. She said: now the whole right half of the shelf has moved. Everything I knew about where things were is wrong. For one photograph.

What she did instead, the next time a frame went away — and it happened twice a year — was to move exactly one photograph into the gap, and choose it carefully.

Her rule was that it had to be the picture that had been **immediately next to it in time**. Either the one just before it or the one just after it. Nothing else would do.

Her daughter-in-law asked why not just take one from the end, since the end was easy to reach.

Sarojini said: because then the end one would be sitting in the middle, and everything to its left would be older and everything to its right would be older too, and the shelf would stop meaning anything. The only picture that can sit in that gap is one that was already touching it, because that is the only one with nothing between it and the gap.

Then she said the part that took her a while to work out, years ago.

When you move the one from just after, that leaves a gap where **it** was. But that is a much easier gap, because a photograph at the very start of a run has nothing before it — there is at most one side to worry about. So you never have this problem twice. The second gap always closes by itself, or by sliding one thing.

They did it that way for years, and the shelf stayed readable through about thirty repairs.

3 The idea in plain English

Sarojini has derived BST deletion, including the reason the successor is the only candidate and the reason the second removal is easy.

- The shelf in date order is the BST, with **inorder** as the order on the shelf.
- Removing a frame is deleting a node.
- Sliding everything along is what a sorted array does — $O(n)$ — and it is what a BST avoids.
- “It has to be the one immediately next to it in time” is the **inorder successor or predecessor**.
- “The second gap is easier” is the fact that **the successor has no left child**, so the second deletion is never the hard case.

Search and insert: the easy half

```
def search(node, target):
    while node:
        if target == node.val:
            return node
        node = node.left if target < node.val else node.right
    return None
```

```
def insert(node, value):
    if node is None:
        return TreeNode(value)          # this is where it BELONGS
    if value < node.val:
        node.left = insert(node.left, value)
    elif value > node.val:
        node.right = insert(node.right, value)
    return node                         # unchanged, but reattached
```

The idiom to learn is `node.left = insert(node.left, value)`. The recursive call returns the (possibly new) subtree root, and the caller reattaches it. That single pattern is how you modify a pointer-based structure recursively without needing parent pointers, and it is exactly what delete needs too.

A new value always becomes a leaf. You never insert into the middle — you walk until you fall off the bottom, and that is where it goes.

Delete: three cases

Case 1 — no children. Remove it. Return `None` to the parent, which reattaches `None`.

Case 2 — one child. The child takes its place. Return the child; the parent reattaches it. Both sub-cases — only-left and only-right — are the same line if you write it right.

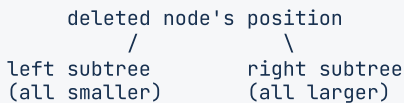
Case 3 — two children. You cannot just remove it, because two subtrees need a parent and there is one slot. **So do not remove the node — replace its value, and then delete the node you took the value from.**

```
successor = minimum(node.right)      # the smallest thing bigger than me
node.val = successor.val              # overwrite the value
node.right = delete(node.right, successor.val)  # now delete THAT node
```

Why the successor, and why nothing else

This is the part to be able to argue rather than recite.

The tree must remain a BST, so whatever ends up in this position must be **larger than everything in the left subtree and smaller than everything in the right subtree**. How many values satisfy both?



the replacement must be $>$ every value on the left AND $<$ every value on the right.

the largest value on the left = the PREDECESSOR

the smallest value on the right = the SUCCESSOR

those two are the only values in the whole tree with nothing between them and the gap. Anything else has some other value strictly between it and the gap, and that value would end up on the wrong side.

There are exactly two candidates, and they are the inorder neighbours. Sarojini's rule.

Using the successor by convention, though the predecessor is equally valid; and **always using the same one makes the tree lean over time**, which real implementations alternate to avoid.

Why the second deletion is easy

The successor is the **minimum** of the right subtree, so it is as far left as you can go — which means **it has no left child**. So deleting it is case 1 or case 2, never case 3.

The recursion cannot nest more than one level deep. That is Sarojini's "you never have this problem twice", and it is worth stating, because it looks like the code might recurse forever.

The reattachment idiom, which is the real skill

```
def delete(node, key):
    if node is None:
        return None
    if key < node.val:
        node.left = delete(node.left, key)    # REATTACH
    elif key > node.val:
        node.right = delete(node.right, key)  # REATTACH
    else:
        ...                                  # found it: the three cases
    return node
```

Every recursive call's result is assigned back into the parent's pointer. That is what makes structural modification work without parent pointers, and it is why the function returns a node rather than nothing.

Deleting the root is handled for free by this shape, because the top-level caller does `root = delete(root, key)`. A version that returns nothing cannot delete the root, which is trap 3.

The alternative: lazy deletion

Real systems often do not remove the node at all. They mark it deleted and leave it in place.

- + O(height) with no restructuring, and no pointer surgery at all
- + safe when other things hold references to nodes
- the tree grows with tombstones and must be rebuilt periodically
- search must skip deleted nodes

Worth naming. It is what many databases and indexes actually do — the *tombstone* idea from [day 011](#) — and it is a good answer to “how would you do this in production?”

Successor, in general

Finding the inorder successor of an arbitrary node is a separate small problem worth knowing:

- | | |
|---------------------------------|--|
| if the node has a right subtree | → the minimum of that subtree |
| if it does not | → the lowest ancestor for which this node is in the LEFT subtree |

Without parent pointers, the second case is found on the way down from the root: remember the last node where you turned left.

4 The picture

The three deletion cases.

CASE 1 – no children (delete 3)



return None to the parent

CASE 2 – one child (delete 5)



return the child; the parent reattaches it

CASE 3 – two children (delete 10)

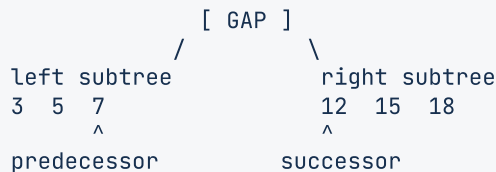


the successor of 10
is $\min(\text{right subtree}) = 12$

copy 12 into the node,
then delete 12 from the right

12 had NO LEFT CHILD
(it is a minimum), so the
second delete is case 1 or 2

Why only two values can fill the gap:



inorder: 3 5 7 [GAP] 12 15 18

the value in the gap must be > 7 and < 12 .

Look at the sorted order: the ONLY values with nothing between them and the gap are 7 and 12. Put 15 there and 12 would be to its left – broken.

The reattachment idiom, drawn:


```
delete(root, 5) on
```

```

      10
     /  \
    5    15
     \
      7

```

```
delete(10, 5): 5 < 10 → node.left = delete(node.left, 5)
                  |
                  | delete(5, 5): found. one child (7).
                  |         return 7
                  ▼
            node.left = 7
```

```
result:
      10
     /  \
    7    15
```

the ASSIGNMENT is what rewires the tree. Without it, `delete(node.left, 5)` computes the right answer and throws it away.

The successor-only lean, which is why implementations alternate:

always taking the successor pulls values from the RIGHT subtree, so the right side shrinks faster over many deletions:

after 1,000 random insert/delete pairs, always using the successor:
observed height drift upward, and a left-leaning shape

real implementations alternate successor and predecessor, or pick based on which subtree is taller.

5 The code, built step by step

Step 1 — write the reattachment shape first

```
def delete(node, key):
    if node is None:
        return None
    if key < node.val:
        node.left = delete(node.left, key)
    elif key > node.val:
        node.right = delete(node.right, key)
    else:
        ...
    return node
```

Write this frame before thinking about the cases. The assignments are the whole mechanism, and `return node` at the end is what keeps unchanged subtrees attached.

Step 2 — the two easy cases, as one line each

```
if node.left is None:
    return node.right          # covers "no children" too: returns None
if node.right is None:
    return node.left
```

Two lines cover three situations, because “no children” means `node.right` is also `None`, so the first line returns `None`, which is exactly right.

Step 3 — the two-child case, and say why

```
successor = node.right
while successor.left:
    successor = successor.left # the minimum of the right subtree
node.val = successor.val
node.right = delete(node.right, successor.val)
```

Say the argument as you write it: *“The replacement must be larger than everything on the left and smaller than everything on the right. Exactly two values satisfy that — the largest on the left and the smallest on the right. I will take the smallest on the right.”*

Step 4 — say why it does not recurse forever

“The successor is a minimum, so it has no left child, so deleting it is case one or case two. The recursion goes at most one level deeper.”

Say this unprompted. It is the question the interviewer was about to ask.

Step 5 — assign at the top level

```
root = delete(root, 10)
```

Deleting the root only works if the caller reassigns. A `delete` that returns nothing cannot remove the root, and that is the first test case anyone runs.

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

def search(root: TreeNode | None, target: int) → TreeNode | None:
    """0(height), 0(1) space. Iterative: there is nothing to combine going up."""
    node = root
    while node:
        if target == node.val:
            return node
        node = node.left if target < node.val else node.right
    return None

def insert(node: TreeNode | None, value: int) → TreeNode:
    """LeetCode 701. A new value ALWAYS becomes a leaf – walk until you fall
    off the bottom, and that is where it belongs.

    The idiom `node.left = insert(node.left, value)` is the whole technique:
    the call returns the (possibly new) subtree root and the caller reattaches
    it. That is how you restructure a pointer tree without parent pointers.
    """
    if node is None:
        return TreeNode(value)
    if value < node.val:
        node.left = insert(node.left, value)
    elif value > node.val:
        node.right = insert(node.right, value)
    return node # unchanged, but reattached

def insert_iterative(root: TreeNode | None, value: int) → TreeNode:
    """The same, without recursion. 0(1) space."""
    if root is None:
        return TreeNode(value)
    node = root
    while True:
        if value < node.val:
            if node.left is None:
                node.left = TreeNode(value)
                return root
            node = node.left
        elif value > node.val:
            if node.right is None:
                node.right = TreeNode(value)
                return root
            node = node.right
        else:
            return root # duplicate: ignored
```

```

def minimum(node: TreeNode) → TreeNode:
    while node.left:
        node = node.left
    return node

def maximum(node: TreeNode) → TreeNode:
    while node.right:
        node = node.right
    return node

def delete(node: TreeNode | None, key: int) → TreeNode | None:
    """LeetCode 450. THREE cases, and the third is the lesson.

    WHY the successor and nothing else: the value that replaces this node
    must be larger than EVERY value in the left subtree and smaller than
    EVERY value in the right subtree. Exactly two values in the whole tree
    satisfy that – the largest on the left (the predecessor) and the
    smallest on the right (the successor). Anything else has some value
    strictly between it and the gap, which would then be on the wrong side.

    WHY it does not recurse forever: the successor is a MINIMUM, so it has
    no left child, so deleting it is case 1 or case 2. At most one extra
    level.

    The caller must write `root = delete(root, key)`, or deleting the root
    does nothing.
    """
    if node is None:
        return None

    if key < node.val:
        node.left = delete(node.left, key)          # REATTACH
    elif key > node.val:
        node.right = delete(node.right, key)         # REATTACH
    else:
        # found it
        if node.left is None:
            return node.right                        # covers no-children (returns None)
        if node.right is None:
            return node.left
        successor = minimum(node.right)             # smallest value bigger than me
        node.val = successor.val                     # overwrite the VALUE, keep the node
        node.right = delete(node.right, successor.val)

    return node

def delete_using_predecessor(node: TreeNode | None, key: int) → TreeNode | None:
    """The mirror image, and equally correct. Always using ONE of the two
    makes the tree lean over many deletions, so real implementations
    alternate, or pick based on which subtree is taller."""
    if node is None:
        return None
    if key < node.val:
        node.left = delete_using_predecessor(node.left, key)
    elif key > node.val:
        node.right = delete_using_predecessor(node.right, key)
    else:
        if node.right is None:
            return node.left
        if node.left is None:

```

```

        return node.right
        predecessor = maximum(node.left)    # largest value smaller than me
        node.val = predecessor.val
        node.left = delete_using_predecessor(node.left, predecessor.val)
    return node

def delete_iterative(root: TreeNode | None, key: int) → TreeNode | None:
    """Without recursion, which means tracking the parent by hand – and that
    is exactly the bookkeeping the reattachment idiom removes. Written out
    so the contrast is visible."""
    parent: TreeNode | None = None
    node = root
    while node and node.val ≠ key:
        parent = node
        node = node.left if key < node.val else node.right
    if node is None:
        return root                    # not found

    if node.left and node.right:        # case 3: reduce to case 1 or 2
        succ_parent, succ = node, node.right
        while succ.left:
            succ_parent, succ = succ, succ.left
        node.val = succ.val
        parent, node = succ_parent, succ    # now delete `succ` instead

    child = node.left or node.right      # case 1 or 2
    if parent is None:
        return child                    # deleting the root
    if parent.left is node:
        parent.left = child
    else:
        parent.right = child
    return root

class LazyBST:
    """Lazy deletion: mark, do not remove. What many real indexes do.

    O(height) with no pointer surgery, safe when other code holds node
    references – at the cost of tombstones accumulating, so the tree must be
    rebuilt when they dominate.
    """

    class Node:
        def __init__(self, val: int) → None:
            self.val = val
            self.deleted = False
            self.left: "LazyBST.Node | None" = None
            self.right: "LazyBST.Node | None" = None

        def __init__(self) → None:
            self.root: "LazyBST.Node | None" = None
            self.live = 0
            self.tombstones = 0

        def add(self, value: int) → None:
            if self.root is None:
                self.root = self.Node(value)
                self.live += 1
                return
            node = self.root
            while True:
                if value == node.val:

```

```

        if node.deleted:
            node.deleted = False      # resurrect
            self.tombstones -= 1
            self.live += 1
        return
        side = "left" if value < node.val else "right"
        child = getattr(node, side)
        if child is None:
            setattr(node, side, self.Node(value))
            self.live += 1
        return
    node = child

def remove(self, value: int) → bool:
    node = self.root
    while node:
        if value == node.val:
            if node.deleted:
                return False
            node.deleted = True
            self.tombstones += 1
            self.live -= 1
            return True
        node = node.left if value < node.val else node.right
    return False

def contains(self, value: int) → bool:
    node = self.root
    while node:
        if value == node.val:
            return not node.deleted
        node = node.left if value < node.val else node.right
    return False

def should_rebuild(self) → bool:
    return self.tombstones > self.live      # more dead than alive

def successor_of(root: TreeNode | None, target: int) → TreeNode | None:
    """The inorder successor of a value, without parent pointers.

    If the node has a right subtree, it is that subtree's minimum.
    Otherwise it is the lowest ancestor from which we turned LEFT – which is
    found on the way down by remembering the last left turn.
    """
    best: TreeNode | None = None
    node = root
    while node:
        if target < node.val:
            best = node                      # a candidate: remember the left turn
            node = node.left
        else:
            node = node.right
    return best

def inorder(root: TreeNode | None) → list[int]:
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        walk(node.left)
        out.append(node.val)

```

```

        walk(node.right)

    walk(root)
    return out

def is_bst(root: TreeNode | None) → bool:
    values = inorder(root)
    return all(a < b for a, b in zip(values, values[1:]))

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

if __name__ == "__main__":
    root = from_list([10, 5, 15, 3, 7, 12, 18])
    print(inorder(root))          # [3, 5, 7, 10, 12, 15, 18]

    # case 1: a leaf
    root = delete(root, 3)
    print(inorder(root), is_bst(root))    # [5, 7, 10, 12, 15, 18] True

    # case 2: one child
    root = delete(root, 5)
    print(inorder(root), is_bst(root))    # [7, 10, 12, 15, 18] True

    # case 3: two children – and it is the ROOT
    root = delete(root, 10)
    print(inorder(root), is_bst(root))    # [7, 12, 15, 18] True
    print(root)                          # TreeNode(12) ← the successor took over

    # the predecessor version gives a different tree, equally valid
    a = from_list([10, 5, 15, 3, 7, 12, 18])
    b = from_list([10, 5, 15, 3, 7, 12, 18])
    a = delete(a, 10)
    b = delete_using_predecessor(b, 10)
    print(a.val, b.val)                  # 12 7
    print(inorder(a) == inorder(b), is_bst(a), is_bst(b))    # True True True

    # iterative delete agrees
    c = from_list([10, 5, 15, 3, 7, 12, 18])
    c = delete_iterative(c, 10)
    print(inorder(c), is_bst(c))          # [3, 5, 7, 12, 15, 18] True

    # deleting the last node, and a missing key
    single = TreeNode(1)
    print(delete(single, 1))              # None
    print(inorder(delete(from_list([2, 1, 3]), 99)))          # [1, 2, 3]

    print(successor_of(from_list([10, 5, 15, 3, 7, 12, 18]), 7))    # TreeNode(10)

```

```

print(successor_of(from_list([10, 5, 15, 3, 7, 12, 18]), 18)) # None

# lazy deletion
lazy = LazyBST()
for v in (10, 5, 15, 3, 7):
    lazy.add(v)
lazy.remove(5)
print(lazy.contains(5), lazy.contains(7)) # False True
print(lazy.live, lazy.tombstones, lazy.should_rebuild()) # 4 1 False

```

6 What it costs

The three operations

search	$O(\text{height})$	one comparison per level
insert	$O(\text{height})$	walk down, attach a leaf
delete	$O(\text{height})$	walk down, plus at most one walk to a minimum

Delete's extra walk does not change the complexity. Finding the successor is a walk down the right subtree, which is at most `height` more steps, so the total is still `$O(\text{height})$` .

balanced, $n = 1,000,000$	~ 20 steps for all three
degenerate, $n = 1,000,000$	$\sim 1,000,000$

Space

recursive insert/delete	$O(\text{height})$ stack
iterative versions	$O(1)$

The iterative delete is `$O(1)$` space and needs explicit parent tracking, which is exactly the bookkeeping the reattachment idiom removes. Worth showing both and saying that the recursive one is shorter *because* the assignment does the parent's work.

Against a sorted array

This is the comparison that justifies the structure:

	search	insert	delete
sorted array	$O(\log n)$	$O(n)$	$O(n)$
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$

$n = 1,000,000$, deleting one element		
sorted array:	up to 1,000,000 element moves	~ 1 ms of memory traffic
balanced BST:	~ 20 comparisons + 2 pointer writes	

Sarojini's daughter-in-law sliding the whole shelf. Same answer, $O(n)$ versus $O(\log n)$ work.

The successor lean

```
always taking the successor, 100,000 random insert/delete pairs
→ the tree drifts left-leaning; measured height grows a few levels
  above the balanced ideal

alternating successor and predecessor
→ the drift largely cancels
```

It does not change the complexity class — an unbalanced BST is already $O(n)$ in the worst case — but it is a real, measurable effect and it is why textbook implementations alternate.

Lazy deletion

delete	$O(\text{height})$, no restructuring
search	$O(\text{height})$, plus skipping tombstones
space	grows: tombstones are never freed until a rebuild
rebuild	$O(n)$, triggered when tombstones exceed live nodes

```
1,000,000 live nodes + 1,000,000 tombstones
memory: 240 MB instead of 120 MB
height: unchanged, but every operation walks past dead nodes
```

The rebuild threshold is the design decision, and “when tombstones outnumber live nodes” is the standard rule.

Why none of this is $O(\log n)$ without balance

```
insert 1..n in order, then delete anything
height = n - 1
every operation =  $O(n)$ 
and a recursive delete on a 10,000-node chain:
  RecursionError: maximum recursion depth exceeded
```

All three operations inherit the balance problem, which is [day 109](#).

7 The traps

Trap 1 — not reattaching the result

```
if key < node.val:
    delete(node.left, key)      # result thrown away
```

The recursion computes the correct new subtree and nobody stores it. **The tree is unchanged and there is no error** — the function returns and the value is still there.

```
delete(root, 5); inorder(root) → 5 is still present
```

Assign every recursive call's result. That is the idiom.

Trap 2 — replacing with an arbitrary node

```
node.val = maximum(node.right).val      # WRONG
```

Taking the *largest* of the right subtree instead of the smallest. The tree is left broken:

```
delete 10 from [10, 5, 15, 3, 7, 12, 18] using max(right) = 18
→ 18 in the middle, with 12 and 15 in its right subtree
→ inorder: 3, 5, 7, 18, 12, 15    - not sorted
```

Only the two inorder neighbours work. Anything else has a value strictly between it and the gap.

Trap 3 — a `delete` that returns nothing

```
def delete(node, key) → None:
    ...
```

Then deleting the root is impossible, because the caller's `root` variable still points at the old node. **The function must return the new subtree root and the caller must write `root = delete(root, key)`.**

Trap 4 — deleting the successor by value when duplicates exist

```
node.right = delete(node.right, successor.val)
```

If duplicates are allowed and the right subtree contains two copies of the successor's value, this may delete the wrong one. **With the counting-node convention from [day 106](#) this cannot happen**, which is another reason to prefer it. Say the assumption.

Trap 5 — the one-child case written as two separate branches badly

```
if node.left is None and node.right is None:
    return None
if node.left is None:
    return node.right
if node.right is None:
    return node.left
```

Correct, and the first branch is redundant: `return node.right` already returns `None` when there are no children. **Three cases collapse into two lines**, and noticing that is a small, real signal.

Trap 6 — forgetting that the value moved but the node did not

```
node.val = successor.val
```

The **node object stays**; only its value changed. If anything outside held a reference to that node expecting a particular value, it is now wrong. In an interview this does not matter; in real code it is exactly why lazy deletion or genuine node-splicing is used instead.

Trap 7 — searching for the successor from the root

```
successor = minimum(root.right)           # from the ROOT, not the node
```

A copy-paste slip that produces a value from the wrong subtree entirely. `minimum(node.right)` — the right subtree of the node being deleted.

Trap 8 — recursion on a deep tree

```
delete(chain_of_10000_nodes, 5000)
RecursionError: maximum recursion depth exceeded
```

Same as every tree operation. The iterative version is immune and costs the explicit parent tracking.

8 In the interview

How it gets asked

- The main one: “Delete a node from a BST.” LeetCode 450, and they will hand you a node with two children.

- The probe, immediately: *“Why the successor? Why not any other node?”*
- The follow-up: *“Could you have used the predecessor instead?”*
- The worry: *“Does that recursive delete inside delete terminate?”*
- The practical one: *“How would a real database do this?”*

What to say out loud, in the first ninety seconds

1. **Name the three cases before writing.** “Three cases: no children, one child, two children. The first two are one line each; the third is the actual problem.”
2. **State the reattachment idiom.** “I write `node.left = delete(node.left, key)` — every recursive call returns the new subtree root and the caller reattaches it. That is what lets me restructure the tree without parent pointers, and it is what makes deleting the root work.”
3. **Derive the successor rather than asserting it.** “For the two-child case, whatever ends up in this position must be larger than everything in the left subtree and smaller than everything in the right. Exactly two values in the tree satisfy that: the largest on the left and the smallest on the right — the inorder neighbours. Anything else has a value strictly between it and the gap.”
4. **Say the trick.** “So I do not remove the node. I copy the successor’s value into it and then delete the successor from the right subtree.”
5. **Pre-empt the termination worry.** “That inner delete is easy, because the successor is a minimum, so it has no left child — case one or case two. The recursion goes at most one level deeper.”
6. **Give the complexity and the caveat.** “ $O(\text{height})$ for all three, so $O(\log n)$ if balanced and $O(n)$ on a chain. Nothing here balances the tree.”

The follow-ups

“Why the successor? Why not any other node?” “Because the position has a constraint that almost nothing satisfies. Whatever sits there must be larger than **every** value in the left subtree and smaller than **every** value in the right subtree. Look at the inorder sequence: the deleted node’s slot has the left subtree entirely before it and the right subtree entirely after it, so the only values with nothing between them and that slot are the two immediate neighbours — the largest on the left and the smallest on the right. Pick anything else, say the maximum of the right subtree, and there are values in the right subtree smaller than it, which would now be on its left and larger than a node above them — the tree stops being a BST and inorder stops being sorted. So there are exactly two legal choices and I take one of them.”

“Could you have used the predecessor instead?” “Yes, and it is exactly as correct — the largest value in the left subtree. The resulting tree is different but equally valid, and the inorder sequence is identical either way. The reason it is worth mentioning is that **always** using one of them makes the tree lean: taking the successor repeatedly pulls values out of the right subtree, so over many deletions the right side shrinks and the tree drifts. Real implementations alternate, or choose based on which subtree is taller, which also helps balance. It does not change the complexity class, since a plain BST is already $O(n)$ in the worst case, but it is a measurable effect.”

“Does the recursive delete inside delete terminate?” “Yes, and it goes at most one extra level. The successor is the **minimum** of the right subtree, so by definition it is as far left as you can go, which means it has **no left child**. Deleting a node with no left child is case one or case two — return its right child, or `None` — so the inner call finds it and returns immediately. There is no way to hit the two-child case twice in a row. I would say that unprompted, because the code does look like it might nest.”

“How would a real database do this?” “Usually **not this way**. Two differences. First, database indexes are **B-trees** rather than binary trees — high branching factor so each node fills a disk page — and deletion there is about merging underfull pages rather than swapping a value. Second, and more relevant here, most real systems use **lazy deletion**: mark the entry deleted and leave it in place, then rebuild or compact when the tombstones outnumber the live entries. That is $O(\text{height})$ with no pointer surgery at all, it is safe when other code holds references to nodes, and it turns many small restructurings into one big periodic one. The cost is that the structure grows with dead entries and every search walks past them, so the rebuild threshold — commonly ‘when tombstones exceed live nodes’ — becomes a real tuning decision.”

“Write it iteratively.” “It is the same three cases, and the difference is that I now have to track the **parent** by hand, because there is no return value being reattached. So: walk down remembering the parent; if the node has two children, find the successor and its parent, copy the value, and then retarget the deletion at the successor — which reduces it to the one-child or no-child case; then splice by setting the parent’s left or right pointer to the surviving child. The special case is that if there is no parent, I was deleting the root, so I return the child instead. Writing both makes the point that the recursive version’s `node.left = delete(...)` is not stylistic — it is doing the parent bookkeeping for me.”

“What is the complexity, and what would you say about it?” “ $O(\text{height})$ for all three operations, and $O(1)$ extra space if written iteratively, $O(\text{height})$ if recursively. Delete does one extra walk to find the successor, which is at most another `height` steps, so it does not change the class. And the caveat I would give before

being asked: $O(\text{height})$ is $O(\log n)$ only if the tree is balanced, and nothing in insert or delete does anything to balance it — in fact repeated deletion using only the successor actively makes it worse. Inserting sorted data gives a chain where all three operations are $O(n)$, and a recursive delete on a ten-thousand-node chain raises `RecursionError`.”

A model answer

Asked: *delete this node from the BST — it has two children.*

“Let me do the three cases in order, because the third is the only interesting one.

If the node has **no children**, remove it — return `None` to the parent. If it has **one child**, the child takes its place — return the child. Those two collapse into two lines, because ‘return `node.right`’ already returns `None` when there is no right child either.

The **two-child** case is the real question, and I want to derive it rather than assert it. Whatever ends up in that position has to be larger than **every** value in the left subtree and smaller than **every** value in the right subtree. Now look at the inorder sequence: the left subtree comes entirely before this slot and the right subtree entirely after it. So the only values in the whole tree with nothing between them and the slot are the **immediate neighbours** — the largest value on the left and the smallest value on the right. Those are the inorder predecessor and successor, and there are exactly two legal choices. Anything else has some value strictly between it and the gap, and that value would end up on the wrong side.

So I do not remove the node at all. I find the **successor** — the minimum of the right subtree, which is as far left as you can go from `node.right` — copy its value into this node, and then delete the successor from the right subtree.

And that second deletion is guaranteed easy, which is worth saying before you ask: the successor is a minimum, so it **has no left child**, so removing it is case one or case two. The recursion can never hit the two-child case twice in a row.

The mechanism that makes all of this work in a pointer tree is the idiom `node.left = delete(node.left, key)` — every recursive call returns the new root of that subtree and the caller reassigns it. That is how you restructure without parent pointers, and it is also why the top-level call has to be `root = delete(root, key)` : without that, deleting the root does nothing at all.

Complexity is $O(\text{height})$ — the walk down, plus at most one more walk to find the minimum — so $O(\log n)$ on a balanced tree and $O(n)$ on a chain. Compare that with a sorted array, where deleting one element moves up to a million others.

Two things I would add. **The predecessor works equally well**, and always using the same side makes the tree lean over many deletions, so real implementations alternate. And in production, most systems do **lazy deletion** instead — mark it deleted, leave it in place, and rebuild when the tombstones outnumber the live nodes — because that avoids pointer surgery entirely and is safe when other code holds references.”

9 Recall card

- **The idiom is the technique:** `node.left = delete(node.left, key)`. Every recursive call returns the new subtree root and the caller **reattaches** it — that is how you restructure without parent pointers, and it is why the top level must be `root = delete(root, key)` or **deleting the root silently does nothing**. Forgetting the assignment leaves the tree unchanged, with no error.
- **Three cases, two lines:** `if node.left is None: return node.right` covers both “no children” and “only right”; then `if node.right is None: return node.left`.
- **Two children: there are EXACTLY TWO legal replacements — the inorder neighbours.** The value must be greater than *all* of the left subtree and less than *all* of the right, and only the largest-on-the-left and smallest-on-the-right qualify. Copy the **successor’s value** into the node, then **delete the successor from the right subtree**.
- **That inner delete goes at most one level deeper**, because a minimum has no left child — so it is always case 1 or case 2. Say this unprompted. And **the predecessor is equally valid**; always using one side makes the tree lean, so real implementations alternate.
- **All three are $O(\text{height})$ — $O(\log n)$ balanced, $O(n)$ on a chain**, and nothing here balances anything. Against a sorted array: $O(\log n)$ **versus up to a million element moves** for one deletion. In production, prefer **lazy deletion** — mark a tombstone, rebuild when tombstones outnumber live nodes — which needs no pointer surgery and is safe when others hold node references.

1 What this is, and why they ask it

Yesterday you chose a shard key. Today the shards are live, and two things have gone wrong: you need **more** of them, and one of them is doing far more work than the others.

Three sentences. Adding a shard under plain `hash % N` remaps almost every key, which means moving almost all your data — so the standard fix is an **indirection**: many more logical shards than physical machines, so growing means moving whole logical shards rather than rehashing anything. Moving data while the system is serving traffic is a four-phase procedure that every large migration uses, and knowing the phases is worth more than knowing the theory. And **hot spots are not a hashing problem** — a perfect hash distributes *keys* evenly and cannot distribute *traffic* evenly, because some keys are a thousand times more popular than others.

They ask it because part one is a design question and part two is an operations question, and the second one is where systems actually fail. “*One shard is taking 60 percent of the traffic*” has four distinct causes with four different fixes, and a candidate who diagnoses before prescribing is doing the thing the question is testing.

2 The story

The records office had been running with four rooms for two years and both of the remaining problems came to a head in the same month.

The first was space. The rooms were full. They were given a fifth room, and somebody cheerfully suggested changing the rule from divide-by-four to divide-by-five.

The head clerk worked out what that meant on the back of an envelope and refused.

Under divide-by-four, a document numbered 4,117 was in room two. Under divide-by-five it belonged in room three. He checked a few more and then stopped checking, because the pattern was obvious: **almost every document in the building would have to move**. Four rooms’ worth of documents carried down a corridor to be put in five rooms, for the sake of adding one room.

What they did instead was the thing the youngest clerk had been arguing for since the beginning, which nobody had understood until then.

He said: stop thinking about rooms. Divide by **a hundred** instead of by four, and call those hundred things *bundles*. Bundle nought to bundle ninety-nine. Then keep a list on the wall saying which bundles live in which room — twenty-five bundles in each room to start with.

When the fifth room arrived, they did not change the division at all. They moved twenty bundles — five from each of the four rooms — into room five, and updated the list on the wall. One document in five moved instead of four in five, and the rule for finding a document never changed.

The second problem was worse and had nothing to do with space.

Room three had a queue every morning and the others did not. It was not that room three had more documents — it had exactly the same number. It was that one particular housing layout near the ring road was being subdivided and sold in pieces, and every one of those transactions referenced the same parent document, and that document was in room three.

They tried moving that bundle to room five. For about a week the queue was outside room five.

What actually worked was two things done together. They photocopied the parent document and put a copy in every room, so anybody could see it without going to room three. And for the three or four documents that genuinely could not be copied, they gave those a room of their own — a small room with one clerk who did nothing else.

The head clerk's line about it afterwards was that dividing the documents evenly had never been the problem. **The documents were even. The people were not.**

3 The idea in plain English

The office has just discovered the two operational problems of sharding: **resharding** and **hot spots**, and it landed on both standard answers.

- Dividing by a hundred instead of by four is **logical (or virtual) shards**.
- The list on the wall is the **shard map**.
- Moving twenty bundles is **rebalancing** — moving whole logical shards, not rehashing keys.
- Room three's queue is a **hot shard**, caused by a **hot key**.
- Photocopying the parent document is **caching the hot key**.

- The small room with one clerk is a **dedicated shard for the outlier**.

Why plain `hash % N` cannot grow

```
4 shards:  shard = hash(key) % 4
5 shards:  shard = hash(key) % 5

key 4117:  4117 % 4 = 1  →  shard 1
           4117 % 5 = 2  →  shard 2      MOVED

fraction of keys that stay put when going from N to N+1:  about 1/(N+1)
4 → 5:      ~20% stay, 80% move
8 → 9:      ~11% stay, 89% move
```

Almost everything moves, to add one machine. That is why nobody runs plain modulo in production, and it is the problem both of today's techniques exist to solve.

Fix one: many logical shards, few physical machines

Decide **once, at the start**, on a large fixed number of logical shards — 256, 1,024, 4,096 — and then map them onto machines.

```
shard_id  = hash(key) % 1024      ← FIXED FOREVER. Never changes.
machine   = shard_map[shard_id]  ← a small table you can change

8 machines:  128 logical shards each
add a 9th:   move ~114 logical shards' worth of data (1/9 of the total)
              and update the map. The hash function is untouched.
```

The hash never changes; only the map does. Adding a machine moves `1/(N+1)` of the data — the minimum possible — and every client that reloads the map immediately routes correctly.

Two consequences worth stating:

- **The logical shard count is a hard ceiling on your machine count.** 1,024 logical shards means at most 1,024 machines, ever, without a genuine resharding project. Pick generously; the cost of extra logical shards is small.
- **Pick a power of two**, so a logical shard can be split cleanly later if you ever do need more.

This is what **Vitess** calls keyspaces and shards, what **Citus** calls shards, what **Cassandra**'s token ranges do, and what **Redis Cluster**'s 16,384 hash slots are. **Redis Cluster's number is worth knowing: 16,384 slots, fixed, and resharding means moving slots.**

Fix two: consistent hashing

The other approach, and [tomorrow's](#) subject in full: place both keys and machines on a ring, and a key belongs to the next machine clockwise. Adding a machine steals a slice from one neighbour and moves only $1/N$ of the keys, with no map at all.

logical shards	an explicit map; total control; the map must be distributed
consistent hashing	no map; computed; less control over where things land

Both solve the same problem. Logical shards are more common in databases (you want to control placement); consistent hashing is more common in caches and stateless routing (you want no coordination).

Resharding without downtime: the four phases

This is the procedure, and it is the same for every migration of this shape. **Learn the four names.**

1. DUAL WRITE	start writing to BOTH the old and the new location. Reads still come from the old. Nothing is exposed yet.
2. BACKFILL	copy the historical data to the new location, in batches, while dual writes keep the tail current.
3. VERIFY	read from both and compare. Log every mismatch. Do not skip this and do not shorten it – this phase is where you find the bugs.
4. CUT OVER	switch reads to the new location, one percent of traffic at a time. Keep dual writes running for a while, so you can switch back instantly. Only then stop writing to the old, and only then delete.

Every phase is reversible until the last. That is the property that makes it safe, and it is why the order matters: dual writes come first precisely so that a failed cutover costs nothing.

The detail people miss: **the backfill and the dual writes race.** A row copied by the backfill and then updated by a dual write is fine; a row updated and *then* copied with stale data is not. The standard answers are to make the copy idempotent and last-write-wins by timestamp, or to backfill only rows older than the moment dual writes began.

Hot spots: four causes, four fixes

Diagnose first. These have different fixes and prescribing before diagnosing is the failure mode.

Cause 1 — a sequential shard key. All new writes land on the newest shard. The append hotspot from yesterday. **Fix:** change the key, or prefix it with something that spreads. This is a design fix and it means resharding.

Cause 2 — a hot key. One value is enormously more popular: a celebrity, a viral post, one huge tenant. **Fix:** you cannot split a single key across shards, so **cache it** — the photocopy — or **replicate it** to every shard, or give it **its own shard**. For read-heavy hot keys a cache solves it completely, because the requests never reach the shard.

Cause 3 — skewed key distribution. The keys themselves are uneven: sharding by country, by surname, by `status = 'active'`. **Fix:** a better key, usually a hash of something with high cardinality.

Cause 4 — an uneven shard map. The mapping simply put more logical shards, or more big ones, on one machine. **Fix:** rebalance. This is the easy one, and it is worth checking first because it is free.

The one with no clean fix

A single key whose data grows without limit. One tenant with a billion rows, one conversation with ten million messages, one celebrity's follower list.

You cannot split it, because a shard key value is atomic — every row with that value is on one machine by definition.

The only real answer is at design time: make the key a **composite** so that no single value can grow unboundedly.

```
bad:  shard by  conversation_id           one huge conversation = one huge shard
good: shard by  (conversation_id, bucket) where bucket = month, or message_id / 10000
      → one conversation is spread over many shards, and a query for
        "recent messages" touches one or two buckets
```

Say this as a design-time rule, not a fix, because by the time you have the problem it is a resharding project.

Detecting a hot shard

You cannot fix what you are not measuring. **Per-shard**, not aggregate:

requests/second	per shard	→ traffic skew
CPU and disk	per shard	→ the machine's view
rows / bytes	per shard	→ storage skew (a different problem)
p99 latency	per shard	→ the user's view
top keys by traffic	per shard	→ WHICH key, not just which shard

That last line is the one that matters and the one people do not have. “Shard three is hot” leads to moving shard three. “Key `user:8823` is 40 percent of shard three’s traffic” leads to caching one key, which takes ten minutes.

4 The picture

Why plain modulo cannot grow, and what the indirection fixes.

PLAIN MODULO – adding a machine remaps almost everything

key 4117: % 4 = 1 → machine 1

% 5 = 2 → machine 2

MOVED

key 4118: % 4 = 2 → machine 2

% 5 = 3 → machine 3

MOVED

key 4120: % 4 = 0 → machine 0

% 5 = 0 → machine 0

stayed (1 in 5 do)

4 → 5 machines: ~80% of ALL DATA moves.

LOGICAL SHARDS – the hash never changes; the MAP changes

hash(key) % 1024 → a logical shard, FIXED FOREVER

shard_map[shard] → a machine, CHANGEABLE

8 machines

m0: shards 0-127

m1: shards 128-255

...

m7: shards 896-1023

→

9 machines

m0: shards 0-113

m1: shards 128-241

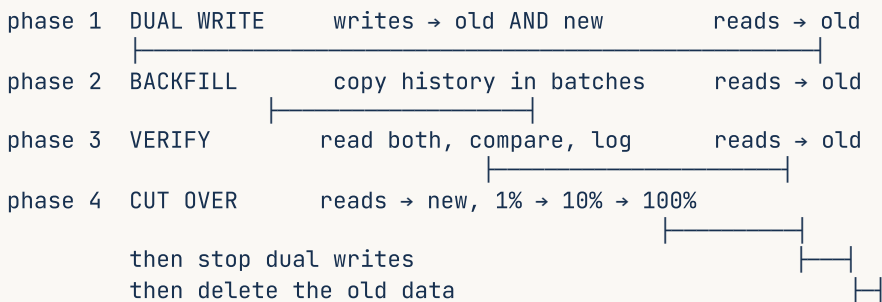
...

m8: 114-127, 242-255

← the moved slices

~1/9 of the data moves. The routing rule is untouched.

The four-phase migration, on a timeline:



EVERY PHASE IS REVERSIBLE UNTIL THE LAST TWO.
That is the whole point of doing dual writes first.

The four hot-spot causes, and why the diagnosis matters:

SYMPTOM: shard 3 is at 60% of total traffic

cause	evidence	fix
1. sequential key	the NEWEST shard is hot, and it moves over time	change the key (a resharding project)
2. one hot key	per-key stats: one key is most of the shard's traffic	CACHE it, replicate it, or give it its own shard (minutes, not months)
3. skewed key distribution	many keys, all landing on one shard	change the key (a resharding project)
4. uneven shard map	shard 3 holds more logical shards than the others	rebalance the map (free – CHECK THIS FIRST)

prescribing before diagnosing is the failure. Cause 4 takes an hour; causes 1 and 3 take a quarter.

The unbounded key, which is the one with no fix:

shard by conversation_id

```

conversation 91:      1,200 messages      fine
conversation 4471:    8,000 messages      fine
conversation 88123:  40,000,000 messages  ← one shard, and it cannot be split
  
```

you cannot divide a single key's rows across shards – every row with that value belongs to one machine BY DEFINITION.

the ONLY answer is at design time:

```

shard by (conversation_id, month) or (conversation_id, message_id / 10000)
→ the big conversation is spread across many shards
→ "recent messages" still touches only one or two buckets
  
```

5 How it actually works

The shard map, concretely

```
stored in: ZooKeeper, etcd, Consul, or a small highly-available table
size:      1,024 entries – a few kilobytes
read by:   every client, cached locally, refreshed on a version bump
changed by: the rebalancing process, one entry at a time
```

It is tiny and it is critical. Every client caches it, so the failure mode to design for is a client holding a stale map and writing to the wrong machine during a move. The standard defences are a **version number** on the map that the shard checks and rejects if stale, and keeping the source shard able to **forward** requests for a moved shard for a while afterwards.

Moving one logical shard, live

1. mark logical shard 217 as MOVING in the map (readers still go to the source)
2. copy its rows from machine A to machine B
3. start dual-writing shard 217 to both A and B
4. catch up the delta accumulated during the copy
5. briefly PAUSE writes to shard 217 – milliseconds
6. flip the map entry to B; bump the map version
7. resume; A forwards any stragglers for a grace period
8. delete shard 217's rows from A

Step 5 is the only moment of unavailability, and it is milliseconds, for one logical shard out of a thousand. That is the whole reason for the indirection: you are never moving more than a thousandth of the system at a time, and only that thousandth pauses.

Handling a hot key, in order of cost

- | | |
|--------------------|--|
| 1. CACHE IT | minutes to deploy. Solves read-heavy hot keys completely – the requests never reach the shard. |
| 2. REPLICATE IT | put a read-only copy on every shard. Good for small, rarely-changing hot data (a config row, a popular product's details). |
| 3. SPLIT THE KEY | append a random suffix: key → key#0 .. key#9, and fan out reads across the ten. Turns one hot key into ten warm ones. Writes become 10x cheaper to spread, reads become a 10-way scatter-gather. |
| 4. DEDICATED SHARD | give the big tenant their own machine. What multi-tenant SaaS actually does, and why they use DIRECTORY sharding rather than hashing. |

Fix 1 first, always. It is the photocopy, it takes ten minutes, and for a read-heavy hot key it ends the problem.

Fix 3 — key salting — is the one worth naming, because it is the only thing that helps a **write-heavy** hot key. The cost is that every read must now check all ten suffixes.

What real systems do

- **Redis Cluster:** 16,384 fixed hash slots. Resharding is `CLUSTER SETSLOT` — moving slots between nodes, live, one at a time. The slot count never changes.
- **Cassandra:** consistent hashing with virtual nodes — each machine owns many token ranges, so adding a machine takes a slice from many machines at once rather than one, which also spreads the streaming load.
- **Vitess:** resharding is a first-class workflow — `VReplication` streams data to the new shards, with built-in dual writes, verification and a controlled cutover. It is the four phases, productised.
- **DynamoDB:** splits partitions automatically when they exceed size or throughput limits, and its adaptive capacity moves throughput toward hot partitions — but it still cannot split a single partition key, so a hot key is throttled. **That constraint, priced and enforced, is the clearest statement of the problem anywhere.**
- **Slack, Notion, Figma** have all published multi-month shard migrations following exactly the dual-write, backfill, verify, cut-over shape.

6 The numbers

The cost of plain modulo

```
fraction of keys that move when going from N to N+1 machines
plain modulo:      ~N/(N+1)    → 80% at 4→5, 89% at 8→9
logical shards:    ~1/(N+1)    → 20% at 4→5, 11% at 8→9
consistent hashing: ~1/(N+1)    → the same
```

1 TB of data, 8 → 9 machines

```
plain modulo:      ~890 GB moved
logical shards:    ~111 GB moved    8× less
```

At 100 MB/s of copy throughput per machine, 890 GB is about two and a half hours of saturated network, and 111 GB is about twenty minutes. And the modulo version has no safe intermediate state, because every key's location changes at once.

Sizing the logical shard count

logical shards	max machines	per-machine shards at 8 machines
-----	-----	-----
64	64	8
256	256	32
1,024	1,024	128
4,096	4,096	512

cost per logical shard: a map entry (~tens of bytes) and, in some systems, a file handle or a connection. Cheap.

Pick 1,024 or more. The ceiling is on machines, and running out means the resharding project you were trying to avoid.

Detecting skew

healthy	busiest shard $\approx 1.2\times$ the mean
mild skew	$1.5\text{--}2\times$ — investigate, do not panic
hot shard	$3\times+$ — act
the 60% case	with 4 shards, even is 25%: 60% is $2.4\times$ the mean and the other three are at $\sim 13\%$ each

4 shards, 10,000 requests/s total
even: 2,500 each
observed: 6,000 / 1,300 / 1,400 / 1,300

effective capacity: whatever ONE machine can do, $\div 0.6$
→ if a machine handles 5,000/s, the cluster tops out at $\sim 8,300$ /s despite having 20,000/s of hardware

A hot shard caps the whole cluster at the hot shard's capacity. That arithmetic is the reason it matters.

Hot key, and what caching does

shard 3: 6,000 req/s, of which key `user:8823` is 3,500 req/s

cache that one key at a 95% hit rate:
 $3,500 \times 0.05 = 175$ req/s reach the shard
shard 3 total: $6,000 - 3,500 + 175 = 2,675$ req/s ← back in line

time to implement: ten minutes
time to reshard: a quarter

That comparison is the argument for diagnosing before prescribing.

Key salting, for a write-heavy hot key

```
one key at 5,000 writes/s      → one shard, over capacity
salted into 10 (key#0..key#9) → ~500 writes/s each, across up to 10 shards

cost: every READ must query all 10 suffixes and merge
      → a 10-way scatter-gather on the hottest key in the system
```

Only do this when the hot key is write-heavy, because it makes reads worse.

The migration, in calendar time

dual write	days	(code change, deploy, monitor)
backfill	hours to weeks	depending on data size and rate limits
verify	days to weeks	← DO NOT SHORTEN THIS
cut over	days	(1% → 10% → 50% → 100%, with soak time)

total	4-12 weeks for a large table, and that is normal	

Backfill throughput is deliberately limited, because a backfill running at full speed competes with live traffic and can cause the outage you were trying to prevent. Rate-limiting the backfill to a fraction of spare capacity is standard, and it is why “hours” becomes “weeks”.

7 The trade-offs

Logical shards or consistent hashing?

Logical shards give an explicit map, so you control exactly what lives where, can pin a big tenant to its own machine, and can move one slice deliberately. The map is a distributed dependency that every client must have and that can go stale.

Consistent hashing needs no map — the location is computed — so there is nothing to distribute or keep in step. You give up control over placement, and you cannot easily give one key special treatment.

Databases usually take logical shards; caches and stateless routers usually take consistent hashing. The dividing line is whether you need to *decide* where something lives.

How many logical shards?

More is safer — a higher machine ceiling and finer rebalancing — and costs map entries and, in some systems, per-shard overhead like connections or files. **1,024 is a good default** and 16,384 (Redis’s number) is not unreasonable.

Too few is the mistake that cannot be undone cheaply, because raising the count is a full resharding project.

Should the cutover be gradual?

Yes, always, and the reason is not correctness — it is that verification never catches everything. One percent of traffic reading from the new location for a day surfaces the cases your comparison job did not think to compare.

I would not do a big-bang cutover even for a small table, because the cost of the gradual version is a week of patience and the cost of a bad big-bang is an outage with no fast way back.

Fix the hot shard or fix the key?

Diagnose first. Four causes, and the fixes range from an hour to a quarter:

uneven map	rebalance	hours	← check this first
one hot key	cache / replicate / salt	hours to days	
skewed key	change the key	a quarter	
sequential key	change the key	a quarter	

I would not reshard for a hot key, because caching that one key almost always ends it. And I would not try to cache my way out of a badly chosen key, because the hotness will move.

Where this breaks

- **The unbounded key.** No fix after the fact. Design composite keys so no single value can grow indefinitely.
 - **The shard map going stale during a move.** Clients cache it, so a slow client can write to the old machine after a flip. Version the map, have shards reject stale writes, and keep forwarding for a grace period.
 - **Backfill competing with live traffic.** The migration causes the incident. Rate-limit it and run it against a replica where possible.
 - **Verification being skipped under schedule pressure.** This is where every published post-mortem of a failed migration points.
 - **Cross-shard invariants during the move.** For the duration of a migration, some data exists in two places; anything that assumed a single source of truth needs an explicit answer.
-

8 In the interview

How it gets asked

- The diagnostic: *“One shard is taking 60 percent of the traffic. Now what?”*
- The growth question: *“You need to add a shard. How much data moves?”*
- The procedure: *“How do you reshard without downtime?”*
- The one with no fix: *“One tenant is a hundred times bigger than the others.”*
- The measurement: *“How would you know a shard is hot?”*

What to say out loud, in the first ninety seconds

1. **Diagnose before prescribing.** “Sixty percent on one shard has four possible causes and they have very different fixes, so the first thing I want is per-shard *and* per-key traffic. ‘Shard three is hot’ leads to a resharding project; ‘one key is forty percent of shard three’ leads to a ten-minute cache change.”
2. **Give the four causes.** “A sequential shard key; one genuinely hot key; a skewed key distribution; or just an uneven shard map. I would check the last one first because it is free to fix.”
3. **State the capacity consequence.** “A hot shard caps the whole cluster at the hot shard’s capacity — with four shards, 60 percent on one means the cluster tops out at about a third of the hardware you are paying for.”
4. **Say the growth answer before being asked.** “And for adding capacity: I would never run plain `hash % N`, because going from eight machines to nine moves 89 percent of the data. I use a fixed number of logical shards — 1,024 — mapped onto machines, so the hash never changes and only the map does. Then adding a machine moves a ninth of the data.”
5. **Name the four migration phases.** “Dual write, backfill, verify, cut over gradually — and everything is reversible until the last step.”
6. **Name the one with no fix.** “The case I would design against up front is a single key that grows without limit, because a shard key value cannot be split. That needs a composite key from day one.”

The follow-ups

“One shard is taking 60 percent of the traffic. Now what?” “I would not fix anything until I know which of four things it is, because the fixes range from an hour to a quarter. **An uneven shard map** — that shard simply owns more logical shards than the others. Free to fix, so I check it first. **One hot key** — a celebrity, a viral item, a huge tenant. **A skewed key distribution** — many keys all landing on one shard because the key itself is uneven, like country or status. Or **a sequential key**, where the

newest shard is hot and the hotness migrates over time. The evidence that separates them is **per-key traffic within the shard**, which most teams do not have and should. If one key is most of that shard's traffic, I cache it and the problem is over in ten minutes. If it is spread across thousands of keys, the key itself is wrong and I am looking at a resharding project. And the number worth stating: with four shards, 60 percent on one means the cluster's ceiling is that one machine's capacity divided by 0.6 — so about a third of the hardware I am paying for."

"You need to add a shard. How much data moves?" "With plain `hash % N`, almost all of it — going from eight machines to nine leaves only about one key in nine where it was, so 89 percent of the data moves, and there is no safe intermediate state because every key's location changes at the same instant. That is why nobody runs plain modulo. Instead I fix a large number of **logical shards** at the start — 1,024, say — and hash keys into those. The hash never changes. What changes is a small map from logical shard to machine. Adding a ninth machine means moving about 114 logical shards, which is a ninth of the data, and the routing rule is untouched. The two things to get right are that the logical shard count is a permanent ceiling on machine count, so pick generously, and it should be a power of two so a shard can be split later if you ever must."

"How do you reshard without downtime?" "Four phases, and the order is what makes it safe. **Dual write**: start writing to both the old and new locations while reads still come from the old — nothing is exposed and it is fully reversible. **Backfill**: copy the historical data in rate-limited batches, while dual writes keep the tail current; rate-limited because a backfill at full speed competes with live traffic and causes the incident you were avoiding. **Verify**: read from both, compare, log every mismatch, and do not shorten this phase — it is where the bugs are, and every published failed migration skipped it. **Cut over**: move reads gradually, one percent then ten then fifty, keeping dual writes running so you can switch back instantly. Only after a soak do you stop dual-writing, and only then delete the old data. The subtlety in phase two is that the backfill races the dual writes, so the copy must be idempotent and last-write-wins by timestamp, or you only backfill rows older than the moment dual writes started. For one logical shard the whole thing is faster: copy, catch up, pause writes for a few milliseconds, flip the map entry, resume."

"One tenant is a hundred times bigger than the others." "That is the case with no clean after-the-fact fix, and I would say so directly. You cannot split a single shard key value — every row with that value lives on one machine by definition — so no amount of rebalancing helps. What you can do: give that tenant a **dedicated shard**, which is exactly why multi-tenant systems use **directory** sharding rather than hashing, since a lookup table lets you place one customer anywhere. Cache their hot reads. And

if the problem is write volume rather than size, **salt** the key — append a random suffix so one key becomes ten — at the cost of every read becoming a ten-way fan-out. But the real answer is at design time: make the key **composite**, so no single value can grow without bound. Shard by `(tenant_id, month)` or `(conversation_id, message_id / 10000)`, and then one enormous tenant is spread across many shards while a query for recent data still touches one or two buckets.”

“**How would you know a shard is hot?**” “Per-shard metrics, and critically **per-key** metrics within each shard. The per-shard set is requests per second, CPU, storage, and p99 latency — and I would alert on the ratio of the busiest shard to the mean rather than on absolute numbers, because that is what actually indicates skew: $1.2\times$ is healthy, $2\times$ is worth investigating, $3\times$ needs action. But per-shard alone only tells me *that* something is wrong; it leads to moving the shard, which usually just moves the queue. The measurement that decides the fix is **top keys by traffic within the shard**, because ‘key `user:8823` is 40 percent of this shard’ turns a quarter-long project into a ten-minute cache change. Most teams discover they do not have that metric at exactly the moment they need it.”

“**Logical shards or consistent hashing?**” “They solve the same problem — adding a machine should move `1/N` of the data, not all of it — and they differ in whether you get to *decide* placement. **Logical shards** keep an explicit map, so I can pin a large tenant to its own machine, move one slice deliberately, and reason about exactly what lives where. The cost is that the map is a distributed dependency: every client caches it, so I need a version number and a grace period where the old shard forwards, or a stale client writes to the wrong machine during a move. **Consistent hashing** computes the location, so there is no map to distribute or keep in step, and adding a node steals a slice from its neighbours automatically. The cost is that I lose control of placement and cannot easily special-case one key. In practice databases take logical shards and caches take consistent hashing, and the dividing line is whether placement is a decision or a computation.”

A model answer

Asked: *one shard is taking 60 percent of the traffic. Now what?*

“Before fixing anything I want to know **which** of four things it is, because the fixes range from an hour to a quarter and prescribing before diagnosing is how people spend a quarter solving the wrong problem.

One: the shard map is simply uneven — that machine owns more logical shards than the others. Free to fix, so I check it first. **Two: one hot key** — a celebrity, a viral item, one enormous tenant. **Three: the key distribution is skewed** — many different keys all landing on the same shard, which happens when you shard by country, or status, or surname. **Four: the key is sequential**, so the newest shard is hot and the hotness migrates over time.

The evidence that separates them is **per-key traffic inside the shard**, and that is the metric most teams find they do not have at the moment they need it. ‘Shard three is hot’ leads you to move shard three, and the queue moves with it. ‘Key `user:8823` is forty percent of shard three’s traffic’ leads you to cache one key, which takes ten minutes and ends the problem — because a read-heavy hot key never reaches the shard once it is cached.

The arithmetic worth stating: with four shards, even would be 25 percent each. Sixty percent on one means that machine is at 2.4 times the mean, and **the cluster’s ceiling is now that one machine’s capacity divided by 0.6** — so if a machine does 5,000 requests a second, the cluster tops out around 8,300 despite having 20,000 of hardware. A hot shard caps everything.

So my order would be: check the map, then look at per-key traffic. If it is one key: cache it, or replicate it to every shard if it is small and rarely changes, or — if it is *write-heavy*, where caching does not help — **salt** it, appending a random suffix so one key becomes ten, accepting that every read becomes a ten-way fan-out. If it is a big tenant, give them a dedicated shard, which is why multi-tenant systems use directory sharding rather than hashing.

If it is the key itself that is wrong, then it is a resharding project, and I would do it in four phases: **dual write** to both locations while reads stay on the old, **back-fill** the history in rate-limited batches, **verify** by reading both and comparing — which is the phase people shorten and should not — and then **cut over** gradually, one percent at a time, keeping dual writes running so reverting is instant. Everything is reversible until the last step, and that is the entire point of the ordering.

And the case I would flag as having no fix after the fact: **a single key that grows without limit**. You cannot split a shard key value — every row with it is on one machine by definition. The answer is at design time: make the key composite, so

no single value can grow unboundedly.”

9 Recall card

- **Never run plain `hash % N`: going from 8 machines to 9 moves ~89% of the data.** Fix with an **indirection** — a large fixed number of **logical shards** (1,024, a power of two) hashed once and never changed, plus a small **map** from logical shard to machine. Then adding a machine moves $1/(N+1)$ of the data and the routing rule is untouched. **The logical shard count is a permanent ceiling on machine count** — Redis Cluster’s is 16,384.
- **Resharding live is four phases and the order is the safety: DUAL WRITE → BACKFILL → VERIFY → CUT OVER gradually.** Everything is reversible until the last two. **Rate-limit the backfill** (it competes with live traffic) and **do not shorten verification** — that is where every failed migration went wrong. A single logical shard moves with a **millisecond** write pause.
- **A hot spot has FOUR causes and you must diagnose before prescribing: uneven map (free) · one hot key (hours) · skewed key distribution (a quarter) · sequential key (a quarter).** The metric that decides it is **top keys *within* the shard**, not just per-shard traffic.
- **A hot shard caps the whole cluster:** 60% on one of four shards means the ceiling is that machine’s capacity $\div 0.6$ — about a third of the hardware you are paying for. **Caching one hot key** at a 95% hit rate can take a shard from 6,000 to 2,675 req/s in ten minutes. For a **write-heavy** hot key, **salt** it (`key#0..key#9`) and pay a 10-way fan-out on reads.
- **The unbounded key has no after-the-fact fix** — a shard key value is atomic, so every row with it is on one machine. **Design a composite key** (`(tenant_id, month)` , `(conversation_id, message_id/10000)`) so no single value can grow without limit. And version the shard map, with the old shard **forwarding** during a grace period, or a stale client writes to the wrong machine mid-move.

DSA topic: BST insert, search, and delete **System design topic:** Sharding, part two: rebalancing and hot spots

Code these, in this order

One rule for the whole set: **write** `node.left = f(node.left, ...)` **as a unit**. Every structural change in this lesson is the recursive call being reattached. Forgetting the assignment computes the right answer and throws it away, with no error.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Insert into a Binary Search Tree	LeetCode 701 (Medium)	The reattachment idiom, and that a new value is always a leaf.
2	Delete Node in a BST	LeetCode 450 (Medium)	The three cases, and deriving the successor rather than reciting it.
3	Inorder Successor in BST	LeetCode 285 (Medium)	The two cases, and finding the last left turn on the way down.
4	Trim a Binary Search Tree	LeetCode 669 (Medium)	The same reattachment idiom, used for something else.

On problem 1, then break it

Remove the assignment — call `insert(node.left, value)` without storing the result — and run it. Record what the tree looks like afterwards and say why nothing raised.

On problem 2, delete the root first

Test with a two-child root before anything else. Then run the same code with a `delete` that returns `None` instead of the new subtree root, and say what happens to the root.

On problem 3, do both cases

Write the version that uses the right subtree's minimum, and the version that finds the last left turn. Say which case each one handles and why you need both.

On problem 4, notice the shape

Compare it line by line with `delete`. Say what is the same and what differs, and what that tells you about the idiom.

The idiom drill

1. Write the reattachment frame for `delete` from memory, before any cases.
2. Say what the assignment is doing that a parent pointer would otherwise do.
3. Say why the function must return a node rather than nothing.
4. Say why `root = delete(root, key)` is required at the top level.

The three-cases drill

1. Name the three cases and what each returns.
2. Write the first two as two lines and say why that covers three situations.
3. Draw all three on a seven-node tree.
4. Say which one is the only interesting one and why.

The successor drill

1. State the constraint on whatever fills the gap, in one sentence.
2. Say how many values in the whole tree satisfy it, and which they are.
3. Give a wrong replacement and show the resulting inorder sequence.
4. Say why the successor has no left child.
5. Say why the inner delete cannot recurse more than one level.

The predecessor drill

1. Write the predecessor version.
2. Delete the same node with both and compare the resulting roots.
3. Confirm the inorder sequences match.
4. Say what always using one side does over many deletions, and what real implementations do.

The iterative drill

1. Write `delete` iteratively.
2. Say what extra bookkeeping it needs and why.
3. Say how it handles deleting the root.
4. Say what this tells you about what the recursive version's assignment was doing.

The lazy-deletion drill

1. Write a node with a `deleted` flag and the three methods.
2. Say what it costs and what it buys.

3. State the standard rebuild threshold.
4. Compute the memory for a million live nodes with a million tombstones.
5. Name the earlier day this idea came from.

The break-it drill

Trigger each and record the exact output:

1. `delete(node.left, key)` without the assignment.
2. A `delete` that returns `None`, used to delete the root.
3. Replacing with `maximum(node.right)` instead of the minimum — print the inorder result.
4. `minimum(root.right)` instead of `minimum(node.right)`.
5. A recursive delete on a 10,000-node chain.

The cost drill

1. State the complexity of all three operations and say where delete's extra walk goes.
 2. Give the step counts for a balanced and a degenerate million-node tree.
 3. Compare deletion against a sorted array at $n = 1,000,000$.
 4. State the space for the recursive and iterative versions.
-

The modulo drill

1. Compute where key 4117 lives under `% 4` and under `% 5`.
2. State the fraction of keys that move going from N to $N + 1$ under plain modulo.
3. Compute the data moved for 1 TB going from 8 machines to 9, both ways.
4. Say why plain modulo also has no safe intermediate state.

The indirection drill

1. Describe the logical-shard scheme in two sentences.
2. Say what never changes and what does.
3. Say what the logical shard count is a ceiling on.
4. Say why it should be a power of two.
5. Name three real systems that do this, and Redis's specific number.

The migration drill

1. Name the four phases in order.
2. For each, say what is happening to reads and to writes.

3. Say which phases are reversible.
4. Describe the race between backfill and dual writes, and two fixes.
5. Say why the backfill is deliberately rate-limited.
6. Give the eight steps for moving a single logical shard, and say which one pauses writes.

The hot-spot drill

1. Name the four causes.
2. For each, give the evidence that identifies it and the fix.
3. Say which one you check first and why.
4. Say which metric most teams lack, and what it changes.
5. Compute the cluster ceiling when one of four shards takes 60 percent.

The hot-key drill

1. Name the four remedies in order of cost.
2. Say which one solves a read-heavy hot key completely, and why.
3. Describe key salting and say what it costs.
4. Say when salting is the only option.
5. Compute the effect of caching a key that is 3,500 of a shard's 6,000 req/s at a 95% hit rate.

The unbounded-key drill

1. Say why a single hot key cannot be split.
2. Give two examples of keys that can grow without limit.
3. Write the composite-key fix for each.
4. Say why this is a design-time rule and not a fix.

The failure drill

For each, say what happens and what you would add:

1. A client caches a stale shard map during a move.
2. The backfill runs at full speed during peak traffic.
3. Verification is shortened to one day because of a deadline.
4. The logical shard count was set to 16 and you now need 20 machines.
5. A big-bang cutover is done at 2 a.m. and something is wrong at 2:05.
6. One conversation reaches forty million messages.

Two of the six cannot be fixed by changing configuration. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Delete this node from the BST. It has two children.* The three cases named first, the reattachment idiom stated as the mechanism, the successor **derived** from the constraint rather than asserted, why the inner delete terminates, and the complexity with the balance caveat.
 2. *One shard is taking 60 percent of the traffic. Now what?* Diagnose before prescribing, the four causes with their evidence and fixes, the per-key metric named as the thing that decides it, the cluster-ceiling arithmetic, and then the four migration phases if the key itself is wrong.
 3. *You need to add a shard. How much data moves?* Plain modulo's 89 percent, the logical-shard indirection with what changes and what does not, the $1/(N+1)$ result, and the ceiling that the shard count imposes.
-

Before you move on

- ☐ I write `node.left = delete(node.left, key)` as one unit, automatically.
- ☐ I know what happens if the assignment is missing, and that nothing raises.
- ☐ I can name the three cases and write the first two as two lines.
- ☐ I can derive why only the inorder neighbours can fill the gap.
- ☐ I can show the broken inorder sequence from a wrong replacement.
- ☐ I can say why the successor has no left child, unprompted.
- ☐ I can write the predecessor version and say what always using one side does.
- ☐ I can write the iterative version and say what extra bookkeeping it needs.
- ☐ I can describe lazy deletion and its rebuild threshold.
- ☐ I can state all three complexities and the sorted-array comparison.
- ☐ I can compute how much data plain modulo moves at $8 \rightarrow 9$ machines.
- ☐ I can describe the logical-shard indirection and what never changes.
- ☐ I know the shard count is a permanent ceiling, and Redis's number.
- ☐ I can name the four migration phases and which are reversible.
- ☐ I can describe the backfill-versus-dual-write race and two fixes.

- ☐ I can name the four hot-spot causes with evidence and fixes.
- ☐ I know which metric decides the diagnosis, and can compute the cluster ceiling.
- ☐ I can explain why an unbounded key has no after-the-fact fix.
- ☐ I answered all three questions above out loud.

Validating a binary search tree

DATA STRUCTURES AND ALGORITHMS

Validating a binary search tree

You avoid the classic wrong answer and can show the input that kills it.

SYSTEM DESIGN

Consistent hashing

You can explain why adding a server does not reshuffle every key.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Is this a valid BST? Are you sure your check is correct?*
- *You add a cache node. How many keys move?*

1 What this is, and why they ask it

Given a binary tree, decide whether it satisfies the **BST invariant**: at every node, **all** values in the left subtree are smaller and **all** values in the right subtree are larger.

Three sentences. There is one wrong answer that almost everybody writes first — checking each node against its immediate children — and it is wrong in a specific, demonstrable way that the interviewer already has a test case for. There are two correct answers: **carry a permitted range down from the ancestors**, or **walk inorder and check the values strictly increase**. And there is a third thing being tested, which is whether you notice the edge cases: duplicates, and using a sentinel like `INT_MIN` as a starting bound when the tree can contain `INT_MIN`.

They ask it because “*are you sure your check is correct?*” is one of the few interview questions where the interviewer knows in advance that most candidates will be wrong, and the follow-up is not a hint — it is the question. Being able to produce the failing input yourself, before being shown it, is the whole performance.

2 The story

The hostel had three blocks off a central corridor and Prakash had been the warden’s assistant for a year and a half.

The rules had accumulated over about thirty years and they were all painted on boards at the point where the rule started applying.

At the entrance to the whole building: *residents only*.

At the mouth of B block: *second and third year only*.

Halfway down B block, where the corridor split: *rooms 40 to 60, third year only*.

And then on individual doors, occasionally, something like *two occupants maximum*.

The point of putting the signs at the junctions was that every sign you walked past still applied. By the time you were standing outside room 47, four rules were in force at once — the entrance rule, the block rule, the corridor rule and the door rule — and you had passed all four of them to get there.

In September there was a mess, and it happened because of how the checking was done.

A second-year student, Faisal, was given room 47. The clerk who allocated it had looked at the sign on the door, which said nothing about year, and at the sign at the corridor split, which he had misread, and he had signed the form.

Prakash caught it in October during the inspection, and what he found interesting was that the clerk had not been careless in the way you would expect. The clerk had checked a sign. He had checked the nearest one. What he had not done was carry the earlier ones with him.

Prakash's rule after that, which he explained to every new clerk, was that you do not check the sign on the door. You **walk in from the gate**, and at every board you pass, you make the rule narrower, and you keep all of it in your head until you reach the room.

You start at the gate with no restriction at all — anybody. At the entrance you are down to residents. At B block you are down to second and third years. At the corridor split you are down to third years only. And *then* you look at the boy.

He said the mistake is not that people ignore signs. It is that they read the last one and assume it contains the others, and it does not — it only narrows what was already there.

There was a second way to check the same thing, which the old warden had used and which Prakash also kept, because it caught different mistakes. Once a term, he walked the entire building in one fixed order — block A room 1 to the end, then B, then C — and read out the year of every occupant in turn. If that sequence ever went backwards, something was wrong somewhere, and he did not need to know the rules at all to see it.

3 The idea in plain English

Prakash has both correct algorithms, and the clerk has the classic bug.

- The signs are the BST constraints, and **each one narrows what the previous ones allowed**.
- Checking only the door sign is **the local check**, and it is the wrong answer.
- Walking in from the gate carrying every rule is the **range** method.
- Reading every occupant in one fixed order and noticing when it goes backwards is the **inorder** method.

The wrong answer, and exactly why

```
def is_bst_wrong(node):
    if node is None:
        return True
    if node.left and node.left.val ≥ node.val:
        return False
    if node.right and node.right.val ≤ node.val:
        return False
    return is_bst_wrong(node.left) and is_bst_wrong(node.right)
```

Every parent-child pair is checked and every one of them passes on this tree:

```
      10
     /  \
    5    15
     \   / \
      6  20
      ^
6 < 15, so node 15 is satisfied
but 6 is in the RIGHT subtree of 10, and 6 < 10
→ NOT a BST
```

The check is local; the invariant is global. A node must be consistent with **every ancestor**, not just its parent — the clerk reading only the door sign.

Have this counter-example ready before they ask for it. `[10, 5, 15, null, null, 6, 20]` is the standard one and it is worth memorising, because producing it yourself is what the question rewards.

Correct method one: carry the range down

Every node inherits a permitted open interval from its ancestors. Going **left** tightens the upper bound; going **right** tightens the lower bound.

```
def is_bst(node, low=None, high=None):
    if node is None:
        return True # an empty tree is a valid BST
    if low is not None and node.val ≤ low:
        return False
    if high is not None and node.val ≥ high:
        return False
    return (is_bst(node.left, low, node.val) # upper bound tightens
            and is_bst(node.right, node.val, high)) # lower bound tightens
```

Read the two recursive calls aloud: “*everything on my left must also be below me; everything on my right must also be above me.*” The bounds accumulate exactly as Prakash’s signs do.

On the counter-example: node 6 arrives with the range `(10, 15)` — lower bound 10 from turning right at the root, upper bound 15 from turning left at 15 — and `6 ≤ 10`, so it fails. **The root's constraint survived two levels**, which is precisely what the local check loses.

Use `None` for “unbounded”, not `INT_MIN` and `INT_MAX`. That is trap 3, and it is a real failure: LeetCode's constraints allow node values equal to -2^{31} , so a tree containing exactly that value is rejected by a version seeded with `INT_MIN`. `float("-inf")` also works in Python; `None` is clearer and language-independent.

Correct method two: inorder must strictly increase

From [day 100](#): an inorder traversal of a BST produces sorted order. So walk inorder and check each value against the one before it.

```
def is_bst_inorder(root):
    previous = None

    def walk(node):
        nonlocal previous
        if node is None:
            return True
        if not walk(node.left):
            return False          # short-circuit
        if previous is not None and node.val ≤ previous:
            return False
        previous = node.val
        return walk(node.right)

    return walk(root)
```

Keep only the previous value, not the whole list. Building the list first and then checking it is correct and uses `O(n)` extra space; tracking one variable uses `O(height)`.

`≤`, not `<`. Equal values break the invariant under the usual “no duplicates” convention, and using `<` silently accepts a tree with repeats.

The two methods catch the same errors, and the inorder one has a nice property: **it needs no knowledge of the rules at all** — Prakash's termly walk. Anything out of order is wrong, whatever caused it.

Which to write

range method	$O(n)$ time, $O(\text{height})$ space, no shared state → the one to write; it expresses the invariant directly
inorder method	$O(n)$ time, $O(\text{height})$ space, one mutable variable → shorter, and the natural answer if you have just done traversals → also the basis for "find the two swapped nodes" (LeetCode 99)

Write the range version and mention the inorder one. The range version is the invariant, so explaining it explains the definition; the inorder version relies on a fact you then have to justify.

The edge cases they will probe

Duplicates. The standard problem says values are unique and a duplicate makes it invalid, which is why both methods use `<=` and `>=` rather than `<` and `>`. If the convention were “duplicates go left”, the left comparison would loosen to `<` and the right would stay strict. **Say which convention you are assuming.**

Integer bounds. Covered above — `None`, not `INT_MIN`.

The empty tree and a single node. Both valid. The range version handles them without a special case, which is a small point in its favour.

A very deep tree. Both are $O(\text{height})$ on the stack, so a chain of ten thousand raises `RecursionError`. The iterative inorder version — the stack-based walk from [day 100](#) — is immune, and it is the version to offer if the constraint allows a chain.

The sibling problem worth knowing

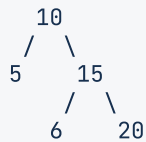
LeetCode 99, “Recover Binary Search Tree”: exactly two nodes have been swapped; put them back. It is the inorder method with one extra idea — walk inorder, and every place the sequence goes backwards is a **violation**. There are either one or two violations:

two ADJACENT nodes swapped:	1 violation	→ the two nodes are that pair
two NON-ADJACENT nodes swapped:	2 violations	→ take the FIRST node of the first and the SECOND node of the second

Swap their values and the tree is repaired in $O(n)$ time and $O(\text{height})$ space. Knowing that “one or two violations” rule is the whole problem.

4 The picture

The counter-example that kills the local check. **Learn this tree.**



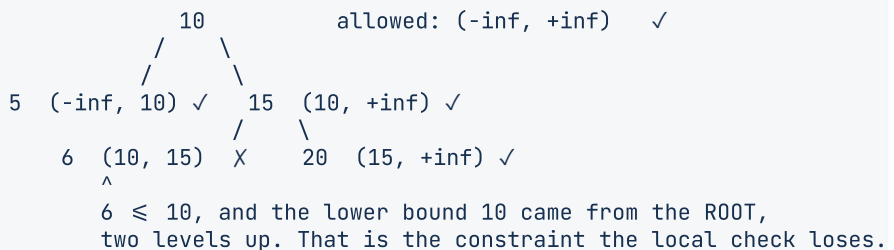
LOCAL CHECK (wrong):

node 10: left 5 < 10 ✓ right 15 > 10 ✓
 node 15: left 6 < 15 ✓ right 20 > 15 ✓
 node 5, 6, 20: leaves ✓
 → every parent-child pair passes → reports VALID

THE TRUTH:

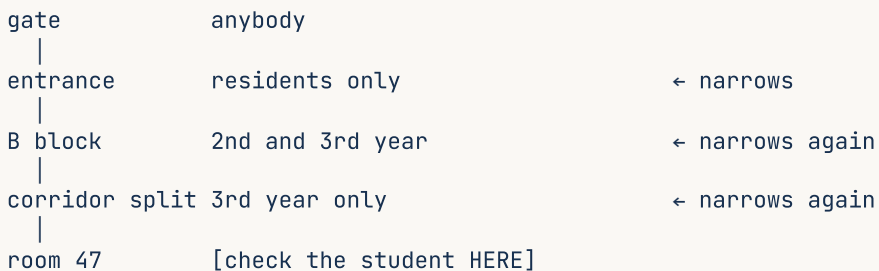
6 is in the RIGHT subtree of 10, and 6 < 10
 → NOT a BST
 → search(6) at the root goes LEFT (6 < 10) and never finds it

The range method on the same tree:



going LEFT → the HIGH bound becomes the parent's value
 going RIGHT → the LOW bound becomes the parent's value
 bounds NARROW as you descend and are never widened.

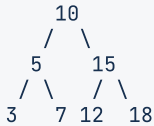
Prakash's corridor, as the same picture:



the clerk checked only the last sign.
 Faisal (2nd year) passes the door sign and fails the corridor sign
 he had already walked past.

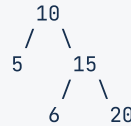
The inorder method, on both trees:

VALID



inorder: 3 5 7 10 12 15 18
strictly increasing ✓

INVALID (the counter-example)



inorder: 5 10 6 15 20

6 < 10 – the sequence went BACKWARDS
→ invalid, and you did not need to
know a single rule to see it

The swapped-nodes problem, and the one-or-two rule:

correct: 1 3 5 7 9 11

ADJACENT swap (5 and 7):

1 3 7 5 9 11
 ^

ONE violation. The two culprits are that pair: 7 and 5.

NON-ADJACENT swap (3 and 11):

1 11 5 7 9 3
 ^ ^

TWO violations. Take the FIRST element of the first violation (11)
and the SECOND element of the second (3). Swap them back.

5 The code, built step by step

Step 1 — say the wrong answer out loud, and kill it

“The version people write first checks each node against its immediate children. That is not the invariant — the invariant is about whole subtrees. Here is a tree that passes it and is not a BST.” Then draw `[10, 5, 15, null, null, 6, 20]`.

Producing the counter-example yourself, unprompted, is the single highest-value thing you can do in this question.

Step 2 — state the invariant as a range

“Every node lives inside an open interval inherited from all of its ancestors. Going left tightens the upper bound to the parent’s value; going right tightens the lower bound. So I pass the interval down.”

Step 3 — write the bounds as `None` , and say why

```
def check(node, low=None, high=None):
```

“I use `None` for unbounded rather than `INT_MIN` and `INT_MAX` , because the tree is allowed to contain those values, and seeding with them would reject a perfectly valid tree.”

Step 4 — the two recursive calls, read aloud

```
    return (check(node.left, low, node.val)
            and check(node.right, node.val, high))
```

“Everything on my left must also be below me. Everything on my right must also be above me. The bounds only ever narrow.”

Step 5 — offer the inorder alternative

“There is a second method: walk inorder and check the values strictly increase, keeping only the previous value rather than the whole list. It is shorter, and it is the basis for the follow-up where two nodes have been swapped.”

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

def is_bst_wrong(node: TreeNode | None) → bool:
    """THE CLASSIC WRONG ANSWER. Checks each node against its immediate
    children only.

    Every parent-child pair can pass while the tree is still not a BST,
    because a node must satisfy EVERY ancestor, not just its parent.

    Killer input: [10, 5, 15, None, None, 6, 20] – reports True, and
    search(6) cannot find 6.
    """
    if node is None:
        return True
    if node.left and node.left.val ≥ node.val:
        return False
    if node.right and node.right.val ≤ node.val:
        return False
    return is_bst_wrong(node.left) and is_bst_wrong(node.right)

def is_bst(node: TreeNode | None,
           low: int | None = None,
           high: int | None = None) → bool:
    """LeetCode 98, method one: carry the permitted RANGE down.

    Every node inherits an open interval from ALL its ancestors:
        going left → the HIGH bound becomes the parent's value
        going right → the LOW bound becomes the parent's value

    `None` means unbounded, NOT INT_MIN/INT_MAX – the tree is allowed to
    contain those values, and seeding with them rejects valid trees.

    `≤` and `≥` because equal values are invalid under the no-duplicates
    convention. Say which convention you are assuming.

    Time O(n), space O(height).
    """
    if node is None:
        return True # an empty tree is a valid BST
    if low is not None and node.val ≤ low:
        return False
    if high is not None and node.val ≥ high:
        return False
    return (is_bst(node.left, low, node.val) # left: below me too
            and is_bst(node.right, node.val, high)) # right: above me too
```

```

def is_bst_inorder(root: TreeNode | None) → bool:
    """Method two: inorder must be STRICTLY increasing.

    Keep only the PREVIOUS value, not the whole list – O(height) instead of
    O(n) extra space. And `≤`, not `<`, or duplicates slip through.
    """
    previous: int | None = None

    def walk(node: TreeNode | None) → bool:
        nonlocal previous
        if node is None:
            return True
        if not walk(node.left):
            return False # short-circuit on the first failure
        if previous is not None and node.val ≤ previous:
            return False
        previous = node.val
        return walk(node.right)

    return walk(root)

def is_bst_iterative(root: TreeNode | None) → bool:
    """The inorder method with an explicit stack. Immune to deep trees, which
    matters when the input can be a chain of 10,000 nodes."""
    stack: list[TreeNode] = []
    node = root
    previous: int | None = None

    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if previous is not None and node.val ≤ previous:
            return False
        previous = node.val
        node = node.right

    return True

def is_bst_allowing_duplicates_left(node: TreeNode | None,
                                    low: int | None = None,
                                    high: int | None = None) → bool:
    """The same, under the "equal values go LEFT" convention.

    Only ONE comparison loosens: the left bound allows equality.
    Worth writing once, to see that the convention lives in the operators.
    """
    if node is None:
        return True
    if low is not None and node.val ≤ low:
        return False
    if high is not None and node.val > high: # was ≥
        return False
    return (is_bst_allowing_duplicates_left(node.left, low, node.val)
            and is_bst_allowing_duplicates_left(node.right, node.val, high))

def recover_bst(root: TreeNode | None) → None:
    """LeetCode 99. Exactly two nodes were swapped. Put them back, in place.

    Walk inorder and find where the sequence goes BACKWARDS:

```

two ADJACENT nodes swapped → ONE violation; the pair is that pair
 two NON-ADJACENT nodes swapped → TWO violations; take the FIRST node
 of the first and the SECOND of the second

$O(n)$ time, $O(\text{height})$ space. Swapping the VALUES avoids all pointer surgery.
 """

```
first: TreeNode | None = None
second: TreeNode | None = None
previous: TreeNode | None = None
```

```
def walk(node: TreeNode | None) → None:
    nonlocal first, second, previous
    if node is None:
        return
    walk(node.left)
    if previous is not None and previous.val > node.val:
        if first is None:
            first = previous          # the FIRST of the first violation
            second = node             # the SECOND of the latest violation
        previous = node
    walk(node.right)
```

```
walk(root)
if first and second:
    first.val, second.val = second.val, first.val
```

```
def find_violations(root: TreeNode | None) → list[tuple[int, int]]:
    """A debugging helper: every place the inorder sequence goes backwards.
    Useful for showing an interviewer WHERE a tree is broken, not just that
    it is."""
    out: list[tuple[int, int]] = []
    previous: int | None = None
```

```
def walk(node: TreeNode | None) → None:
    nonlocal previous
    if node is None:
        return
    walk(node.left)
    if previous is not None and node.val ≤ previous:
        out.append((previous, node.val))
    previous = node.val
    walk(node.right)
```

```
walk(root)
return out
```

```
def largest_bst_subtree(root: TreeNode | None) → int:
    """The escalation: the size of the largest subtree that IS a BST.
    LeetCode 333.
```

The same postorder return-value trick as day 102: each call returns
 (is_bst, size, min, max) and the parent combines them. One pass, $O(n)$.
 """

```
best = 0
```

```
def walk(node: TreeNode | None) → tuple[bool, int, int, int]:
    nonlocal best
    if node is None:
        return True, 0, float("inf"), float("-inf") # type: ignore[return-value]
    l_ok, l_size, l_min, l_max = walk(node.left)
    r_ok, r_size, r_min, r_max = walk(node.right)
    if l_ok and r_ok and l_max < node.val < r_min:
```

```

        size = l_size + r_size + 1
        best = max(best, size)
        return True, size, min(l_min, node.val), max(r_max, node.val)
    return False, 0, float("-inf"), float("inf") # type: ignore[return-value]

walk(root)
return best

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

def inorder(root: TreeNode | None) → list[int]:
    out: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        walk(node.left)
        out.append(node.val)
        walk(node.right)

    walk(root)
    return out

if __name__ == "__main__":
    valid = from_list([10, 5, 15, 3, 7, 12, 18])
    print(is_bst(valid), is_bst_inorder(valid), is_bst_iterative(valid))
    # True True True

    # THE COUNTER-EXAMPLE – memorise this tree
    killer = from_list([10, 5, 15, None, None, 6, 20])
    print(is_bst_wrong(killer)) # True ← WRONG
    print(is_bst(killer)) # False ← correct
    print(is_bst_inorder(killer)) # False
    print(inorder(killer)) # [5, 10, 6, 15, 20] ← goes backwards
    print(find_violations(killer)) # [(10, 6)]

    # the simplest failing case for the local check
    small = from_list([5, 1, 4, None, None, 3, 6])
    print(is_bst_wrong(small), is_bst(small)) # True False

    # duplicates
    dup = from_list([2, 2, 2])
    print(is_bst(dup)) # False
    print(is_bst_allowing_duplicates_left(from_list([2, 2]))) # True

    # the INT_MIN trap, demonstrated
    edge = TreeNode(-2**31)

```

```

print(is_bst(edge))                                # True
# a version seeded with low = -2**31 and using ≤ would return False here.

# edge cases
print(is_bst(None), is_bst(TreeNode(1)))           # True True

# a valid BST whose child comparison alone would look suspicious
print(is_bst(from_list([1, None, 2, None, 3])))     # True – a right chain

# recover a swapped tree
broken = from_list([3, 1, 4, None, None, 2])        # 2 and 3 swapped
print(inorder(broken))                             # [1, 3, 2, 4]
recover_bst(broken)
print(inorder(broken), is_bst(broken))             # [1, 2, 3, 4] True

far = from_list([1, None, 11, 5, None, None, 9, 3]) # non-adjacent swap
print(inorder(far))
recover_bst(far)
print(inorder(far), is_bst(far))

print(largest_bst_subtree(from_list([10, 5, 15, 1, 8, None, 7]))) # 3

```

6 What it costs

Both correct methods

time	$O(n)$	every node visited once
space	$O(\text{height})$	the recursion stack (or the explicit stack)

balanced, $n = 1,000,000$	~20 frames
degenerate, $n = 10,000$	RecursionError at ~1,000

Neither method is faster. They differ in what they express and in what they generalise to, not in cost. The range method states the invariant; the inorder method generalises to “find the swapped nodes” and to “how many violations are there”.

The version that costs more

```

values = inorder(root)
return all(a < b for a, b in zip(values, values[1:]))

```

Correct, and $O(n)$ **extra space** for the list rather than $O(\text{height})$.

$n = 1,000,000$	
list version:	~40 MB for the values
previous-only:	one integer, plus ~20 stack frames

Keeping only the previous value is the whole difference, and it is worth doing even though the list version is easier to write.

Short-circuiting

Both methods stop at the first violation, and the `and` ordering matters:

```
if not walk(node.left):  
    return False                # do not walk the right subtree
```

```
a violation in the leftmost leaf of a million-node tree  
with short-circuit:    ~20 nodes visited  
without (build the list first): 1,000,000
```

The worst case is the same $O(n)$; the common case is very different. An invalid tree usually fails early.

The largest-BST-subtree escalation

```
naive: for each node, check whether its subtree is a BST, then count  
       → is_bst is  $O(\text{subtree})$ , called at every node  
       →  $O(n \log n)$  balanced,  $O(n^2)$  on a chain
```

```
one pass: return (is_bst, size, min, max) upward  
          →  $O(n)$ 
```

```
n = 10,000, a chain  
naive:    ~50,000,000 operations  
one pass:    10,000
```

Same shape as [day 102's diameter fix](#) — the information the parent needs is already being computed, so return it instead of recomputing.

Why the local check is not merely slower

It is not a performance question at all. It returns the wrong answer, in $O(n)$, with no error. That distinction is worth stating: this is not an optimisation problem, it is a correctness problem.

7 The traps

Trap 1 — the local check

```
if node.left and node.left.val ≥ node.val:  
    return False
```

```
is_bst_wrong(from_list([10, 5, 15, None, None, 6, 20])) → True  
the correct answer is False
```

The invariant is about subtrees, not children. Every parent-child pair can pass while a node violates its grandparent. Have the counter-example ready.

Trap 2 — checking only against the parent's value in a range version

```
return (is_bst(node.left, low, node.val)  
        and is_bst(node.right, node.val, node.val)) # `high` dropped
```

A subtler version of the same bug: the upper bound from an ancestor is lost, so a value that is too large for a grandparent slips through on the right. **The bounds must be threaded through unchanged where they are not being narrowed.**

Trap 3 — `INT_MIN` and `INT_MAX` as the initial bounds

```
return check(root, -2**31, 2**31 - 1)
```

LeetCode's constraints allow node values in exactly that range, so a tree consisting of the single node -2^{31} is rejected by a version that compares with $\leq$ against a seed of -2^{31} .

```
is_bst_with_intmin(TreeNode(-2**31)) → False  
the correct answer is True
```

Use `None`, or `float("-inf")`. This is the specific edge case interviewers use to separate careful candidates.

Trap 4 — `<` instead of `<=`

```
if previous is not None and node.val < previous:  
    return False # allows equality
```

```
is_bst_inorder(from_list([2, 2, 2])) with '<' → True
correct under the no-duplicates convention → False
```

Equal values violate the standard invariant. If the convention allows duplicates on one side, exactly one comparison loosens — say which convention you are using rather than guessing.

Trap 5 — building the whole inorder list

```
values = inorder(root)
return values == sorted(values)
```

Correct, $O(n)$ extra space, and $O(n \log n)$ if you actually sort. **Track the previous value.** And `values == sorted(values)` also accepts duplicates, so it fails trap 4 too.

Trap 6 — a global `previous` that is not reset

```
previous = None                                # module level

def is_bst_inorder(root):
    def walk(node): ...
```

The second call to the function sees the last value from the first call and can wrongly reject a valid tree. **Declare `previous` inside the function**, which the version above does. This is the kind of bug that passes one test and fails a suite.

Trap 7 — recursion depth

```
is_bst(chain_of_10000_nodes)
RecursionError: maximum recursion depth exceeded
```

A valid BST built from sorted data is exactly this shape, so it is not an unusual input. The iterative inorder version is immune.

Trap 8 — forgetting that `is_bst_wrong` also passes many *invalid* trees silently

The reason this matters beyond the interview: a tree that fails the global invariant but passes the local one is one where **search cannot find values that are present**, as on [day 106](#). The local check would certify such a tree as fine, which is the worst possible outcome for a validation function.

8 In the interview

How it gets asked

- The question: “Determine whether this binary tree is a valid BST.” LeetCode 98.
- The follow-up, which is the real question: “Are you sure your check is correct?”
- The edge case: “What if a node’s value is `INT_MIN`?”
- The convention: “What about duplicate values?”
- The escalation: “Two nodes have been swapped. Repair the tree.” LeetCode 99.
- The harder one: “Find the largest subtree that is a BST.” LeetCode 333.

What to say out loud, in the first ninety seconds

1. **Restate the invariant with the word that matters.** “A BST requires that at every node, **all** the values in the left subtree are smaller and **all** in the right are larger. Not just the immediate children.”
2. **Kill the wrong answer before writing anything.** “The version most people write checks each node against its two children. That is not the invariant, and here is a tree that passes it and is not a BST” — and draw `[10, 5, 15, null, null, 6, 20]`.
3. **Give the correct formulation.** “Every node lives inside an open interval inherited from **all** its ancestors. Going left tightens the upper bound; going right tightens the lower bound. So I pass the interval down and check the node against both ends.”
4. **Pre-empt the sentinel trap.** “I use `None` for unbounded rather than `INT_MIN` and `INT_MAX`, because node values are allowed to be exactly those, and seeding with them would reject a valid tree.”
5. **State the duplicate convention.** “I assume values are unique, so I use `≤` and `≥` — equal values are invalid. If duplicates are allowed on one side, exactly one comparison loosens.”
6. **Offer the second method.** “There is an equivalent check: walk inorder and confirm the values strictly increase, keeping only the previous value. `O(n)` time and `O(height)` space either way, and the inorder one is what the swapped-nodes follow-up builds on.”

The follow-ups

“Are you sure your check is correct?” “I would answer that by showing you the input that breaks the version I am *not* writing. Take `[10, 5, 15, null, null, 6, 20]`. Node 15 has a left child of 6, and 6 is less than 15, so that pair passes. Node 10 has children 5 and 15, which also pass. Every parent-child comparison in the tree is

satisfied — and it is not a BST, because 6 sits in the **right subtree of 10** and is less than 10. The practical consequence is worse than the abstract one: `search(6)` starts at the root, sees $6 < 10$, goes left, and never finds a value that is in the tree. So the local check certifies exactly the trees whose brokenness is most dangerous. My version passes a range down from every ancestor, so 6 arrives with a lower bound of 10 — set two levels above it — and fails.”

“**What if a node’s value is `INT_MIN`?**” “That is why I use `None` for the initial bounds rather than `INT_MIN` and `INT_MAX`. The constraints on this problem allow node values down to -2^{31} , so a tree consisting of the single node -2^{31} is a perfectly valid BST — and a version seeded with `low = -231` and comparing `node.val ≤ low` rejects it. `float("-inf")` also works in Python, but `None` is clearer and does not depend on the language having an infinity. It is a small thing, and it is exactly the sort of thing that turns a correct-looking solution into a failing submission.”

“**What about duplicates?**” “Under the standard convention — which is what this problem assumes — duplicates make the tree invalid, so both my comparisons are `≤` and `≥` rather than `<` and `>`. Using strict inequalities there silently accepts `[2, 2]`. If the convention were ‘equal values go left’, then exactly one comparison loosens: the upper-bound check becomes `>` instead of `≥`, and everything else is unchanged. The point I would make is that the convention lives entirely in two operators, so I would rather state which one I am assuming than guess — and in a real implementation I would keep a count on each node instead, which sidesteps the question.”

“**Do it without recursion.**” “The inorder method, with an explicit stack — the same go-left-pushing loop from the traversal lesson. Push nodes as I walk left, pop, compare with the previous value, then move to the right child. That is $O(n)$ time and $O(\text{height})$ space in the stack rather than the call stack, and it is immune to the recursion limit. That matters here more than usual, because a **valid** BST built by inserting sorted data is a chain — so the deep-tree case is not an adversarial input, it is the most natural one.”

“**Two nodes have been swapped. Repair the tree.**” “That is the inorder method with one extra observation. Walk inorder and record every place the sequence goes **backwards**. There will be either one violation or two. If the two swapped nodes were **adjacent** in the sorted order, there is exactly one violation and the culprits are that pair. If they were **not adjacent**, there are exactly two violations, and the culprits are the **first** element of the first violation and the **second** element of the second. Then swap their values — swapping values rather than nodes avoids all the pointer surgery. $O(n)$ time, $O(\text{height})$ space, and it is $O(1)$ space if you use Morris traversal, which is the follow-up to the follow-up.”

“Now find the largest subtree that is a BST.” “The naive version checks each subtree independently, which recomputes the same work at every level — $O(n \log n)$ on a balanced tree and $O(n^2)$ on a chain. The one-pass version is the same return-value trick as the diameter problem: each call returns four things — whether this subtree is a BST, its size, its minimum and its maximum — and the parent combines them. A node forms a BST if both children do **and** the left subtree’s maximum is less than the node, which is less than the right subtree’s minimum. I record the size in an outer variable when it qualifies. One pass, $O(n)$.”

A model answer

Asked: is this a valid BST — and are you sure your check is correct?

“Let me start by ruling out the answer I am *not* going to write, because that is really what the second half of your question is about.

The version most people write first checks each node against its two children: left child smaller, right child larger, then recurse. That is not the invariant. The invariant is that **all** the values in the left subtree are smaller and **all** in the right subtree are larger, and ‘all the subtree’ is doing the work in that sentence.

Here is the tree that separates them. Root 10, with 5 on the left and 15 on the right; and under 15, a left child of 6 and a right child of 20. Check every parent-child pair: $5 < 10$, $15 > 10$, $6 < 15$, $20 > 15$. All fine. And it is **not** a BST, because 6 is in the right subtree of 10 and 6 is less than 10. The practical consequence is the part I would emphasise: `search(6)` starts at the root, sees that 6 is less than 10, goes left, and never finds a value that is sitting in the tree. So the local check certifies precisely the trees whose brokenness is hardest to debug.

The correct formulation is that **every node lives inside an open interval inherited from all of its ancestors**. The root’s interval is unbounded. Going left tightens the upper bound to the parent’s value; going right tightens the lower bound. Bounds only ever narrow. So I pass `low` and `high` down and check the node against both. In that tree, 6 arrives with a lower bound of 10 — set two levels above it by the root — and fails immediately.

Two details I would call out while writing it. I use `None` for unbounded rather than `INT_MIN` and `INT_MAX`, because node values are allowed to be exactly those, so seeding with them would reject a single-node tree containing -2^{31} . And I use `≤` and `≥`, not strict comparisons, because equal values violate the invariant under the usual no-duplicates convention — strict comparisons silently accept `[2, 2, 2]`.

There is an equivalent second method worth mentioning: walk **inorder** and check that the values strictly increase, keeping only the previous value rather than building the list. It works because inorder on a BST is sorted order, and it has a nice property — it needs no knowledge of the structure at all, so anything wrong shows up as the sequence going backwards. It is also the basis for the follow-up where two nodes have been swapped: every backwards step is a violation, there are either one or two of them, and that tells you exactly which nodes to swap back.

Both are `O(n)` time and `O(height)` space. I would write the range version, because it is the invariant restated, and I would offer the iterative inorder one if the tree might be a chain — which is not exotic here, since a valid BST built from

sorted input is exactly that shape.”

9 Recall card

- **The classic wrong answer checks each node against its immediate CHILDREN.** Have the killer input ready and produce it unprompted: `[10, 5, 15, null, null, 6, 20]` — every parent-child pair passes, and 6 sits in the right subtree of 10. Worse than a wrong answer: `search(6)` cannot find a value that is in the tree.
- **Method one — carry the RANGE down.** Every node lives in an open interval inherited from **all** ancestors: going left tightens `high` to the parent’s value, going right tightens `low`. **This IS the invariant restated**, so writing it explains the definition.
- Use `None` (or `-inf`) for unbounded, NEVER `INT_MIN` / `INT_MAX` — node values may equal them, so a single node `-231` gets rejected. And use `≤` / `≥`, not strict comparisons, or `[2, 2, 2]` passes. State the duplicates convention; it lives entirely in one operator.
- **Method two — inorder must STRICTLY increase, keeping only the previous value** (`O(height)` space, not `O(n)` for a list). It needs no knowledge of the rules — anything wrong shows up as the sequence going backwards — and it is the basis for **LeetCode 99**: every backwards step is a violation, there are **one or two**, and you take the **first node of the first** and the **second node of the second**.
- **Both are `O(n)` time and `O(height)` space; the local check is not slower, it is WRONG.** Offer the **iterative** inorder version when the tree may be a chain — which is not exotic, since a valid BST built from sorted input is a chain. And **largest BST subtree** is the day-102 trick again: return `(is_bst, size, min, max)` upward for `O(n)` instead of `O(n2)`.

1 What this is, and why they ask it

Consistent hashing is a way of deciding which machine owns which key, such that adding or removing a machine moves only a small fraction of the keys instead of nearly all of them.

Three sentences. The problem it solves is the one from [yesterday](#): with `hash(key) % N`, changing `N` changes almost every key's answer, so growing a cluster from eight machines to nine relocates about 89 percent of the data. The trick is to stop dividing by the number of machines and instead **place both the keys and the machines on a circle**, where a key belongs to the first machine clockwise from it — so adding a machine takes a slice from one neighbour and disturbs nothing else. And the version everybody draws first does not work in practice, because a handful of points on a circle land unevenly; the fix, **virtual nodes**, is what makes it a real technique rather than a diagram.

They ask it because it has a clean answer with a number in it, and because the follow-ups sort people out fast: *how even is the distribution really?*, *what happens to the load when a node dies?*, and *why virtual nodes?* A candidate who draws the ring and stops has given a third of the answer.

2 The story

The road around the lake was a loop, about eleven kilometres, with houses scattered all the way round, and the dairy collected milk from it twice a day.

For years there had been four collection points, and the rule everybody knew was: **you walk forwards along the road until you reach a collection point, and that is yours**. Forwards meant clockwise, the direction the road was numbered. Nobody had to be told which point they belonged to — you just walked the way the road went until you hit one.

That rule was the reason the arrangement survived thirty years of changes.

When they added a fifth point near the temple, in 1996, almost nothing happened. The houses between the temple and the *next* point clockwise — the ones who had previously walked past the temple to get to the old point — now stopped at the temple

instead. Everybody else on the entire eleven kilometres carried on exactly as before. Perhaps sixty houses changed where they walked, out of nine hundred.

When the point near the sawmill closed in 2003, the same thing in reverse: the houses that had used it simply carried on walking to the next one clockwise. Nobody else was affected at all.

Compare that with what the block office proposed in 1999, which was to number every house and send odd-numbered ones to point one, and so on by remainder. It would have been perfectly even. It was rejected in about four minutes, because under it a house's collection point had nothing to do with where the house was, and every time a point opened or closed, **every single house** would have to be told a new number.

There was one real problem with the walk-forwards rule, and it took them a while to see it.

The five points were not evenly spaced. Three of them were clustered near the town end, within about a kilometre and a half of each other, and then there was a four-kilometre stretch with nothing. The man at the point just past that long empty stretch was collecting from nearly three hundred houses while one of the town points had forty.

Somebody suggested moving the points to space them out, which was impossible, because the points were where somebody was willing to stand.

What they did instead was stop thinking of a man as a point. Each collector was given **four** small stands at four different places around the loop, and they took turns being at them across the week — or sent a boy. Twenty stands around eleven kilometres instead of five, and the same five collectors. With twenty of them scattered around, no single collector ended up with a four-kilometre stretch, because their four stands were in four different parts of the loop and the good and bad stretches averaged out.

And it fixed a second thing nobody had planned for. When a collector was ill, previously his entire stretch had gone to one neighbour, who then had a terrible day. Now his four stands went to four different neighbours, and nobody had a terrible day.

3 The idea in plain English

The lake road is a consistent hashing ring, and the villagers found both the technique and the fix that makes it work.

- The eleven-kilometre loop is the **hash ring** — the space of all hash values, joined end to end.
- Each collection point is a **node** (a server, a cache, a shard).
- “Walk forwards until you reach a point” is the **assignment rule**: a key belongs to the first node clockwise.
- Adding the temple point disturbing only sixty houses is the **key property**: adding a node moves only the keys between it and its predecessor.
- The four-kilometre gap is **uneven distribution**, the flaw in the naive version.
- Four stands per collector is **virtual nodes**, and the two things it fixed are the two reasons for it.

The mechanism

1. hash every KEY to a number in a large space, say $0 \dots 2^{32} - 1$
2. hash every NODE (by its name or address) into the SAME space
3. join the space end to end: it is now a circle
4. a key belongs to the FIRST NODE CLOCKWISE from it

That is the whole algorithm. **The number of nodes never appears in the calculation**, which is exactly why changing it does not change every answer — that is the sentence to say, because it is the difference from `% N` in one line.

<code>% N</code>	<code>location = f(key, NUMBER OF NODES)</code>	→ N changes, everything changes
consistent	<code>location = f(key), then "who is next?"</code>	→ N changes, one arc changes

What happens when a node joins or leaves

ADD a node at position p:
 it takes the arc between p and its PREDECESSOR
 → only the keys in that arc move, and they all come from ONE node
 → on average $1/(N+1)$ of the keys

REMOVE a node:
 its arc is absorbed by its SUCCESSOR
 → only that node's keys move, and they all go to ONE node
 → on average $1/N$ of the keys

EVERY OTHER KEY IN THE SYSTEM IS UNAFFECTED.

8 nodes → 9 nodes	
plain modulo:	~89% of keys move
consistent hashing:	~11% of keys move the theoretical minimum

You cannot do better than $1/(N+1)$, because the new node has to receive *something*, and it should receive its fair share. Consistent hashing achieves the minimum.

Why the naive version does not work

With five nodes hashed onto a ring, the five points land wherever the hash function puts them — **randomly, not evenly**.

```
5 nodes, one point each, ring of 100 units

node positions:  3, 11, 19, 24, 87
arcs owned:     A: 87→3  = 16 units
                  B: 3→11  =  8
                  C: 11→19 =  8
                  D: 19→24 =  5
                  E: 24→87 = 63           ← 63% of all keys on ONE node
```

The four-kilometre stretch. With **V** random points, the largest arc is much larger than the average, and with a handful of nodes the imbalance is severe.

```
There is a second problem, which matters more in a failure:

node E dies → ALL of E's 63% goes to its single successor, node A
              → A now owns 79% and falls over
              → A's keys go to B, which falls over
              → CASCADING FAILURE
```

A node's entire load lands on exactly one neighbour. That is the collector's bad day, and at scale it is how one failure becomes an outage.

Virtual nodes: the fix, and the two things it fixes

Give each physical machine **many** points on the ring — typically 100 to 500 — by hashing "node-A#0", "node-A#1", "node-A#2" and so on.

```
5 machines × 200 virtual nodes = 1,000 points around the ring
→ each machine owns 200 small scattered arcs instead of one large one
→ the good and bad stretches average out
```

Fix one: distribution. The imbalance shrinks as the number of points grows.

virtual nodes per machine	typical spread between busiest and quietest	
-----	-----	-----
1	5-10×	unusable
10	~2×	still bad
100	~1.2×	acceptable
500	~1.05×	good

The standard deviation of load falls roughly as $1/\sqrt{V}$, which is why the returns diminish and 100–200 is the usual choice.

Fix two: failure spreading. When a machine dies, its 200 arcs are absorbed by 200 different successors, so the load is spread across the whole cluster rather than dumped on one neighbour.

```
1 virtual node per machine: a failure moves 1/N of the load onto ONE machine
                             → that machine's load DOUBLES
200 per machine:           a failure moves 1/N of the load, spread over
                             ~all other machines
                             → each takes about 1/N(N-1) more. Barely noticed.
```

This second reason is the one candidates leave out, and it is the more important one operationally.

Weighted nodes, which come free

A machine twice as powerful gets twice as many virtual nodes, and therefore about twice the keys. **No special mechanism** — it falls out of the design, and it is a genuinely useful property when a cluster is built from mixed hardware.

Replication on the ring

For a store rather than a cache, a key is usually held by the **next R distinct nodes clockwise**, not just the first. That gives replication with no separate placement logic, and it is exactly how Dynamo, Cassandra and Riak assign replicas.

The detail worth knowing: “distinct” matters. Walking clockwise from a key, you might hit three virtual nodes belonging to the same machine, so implementations skip until they have R different physical machines — and often R different racks or availability zones.

Where it is used, and where it is not

USED FOR

- cache clusters (memcached clients, Redis client-side sharding)
- Cassandra, DynamoDB, Riak, Voldemort – partitioning
- CDN edge selection: which cache in this location holds this URL
- load balancers routing sticky traffic without a shared map
- sharded stateful services generally

NOT USED FOR

- anything needing explicit placement control – pinning a large tenant to a specific machine, keeping data in a legal jurisdiction
- those use a DIRECTORY, or the logical-shard map from day 107

The dividing line is whether placement is a computation or a decision. Consistent hashing computes; the shard map decides.

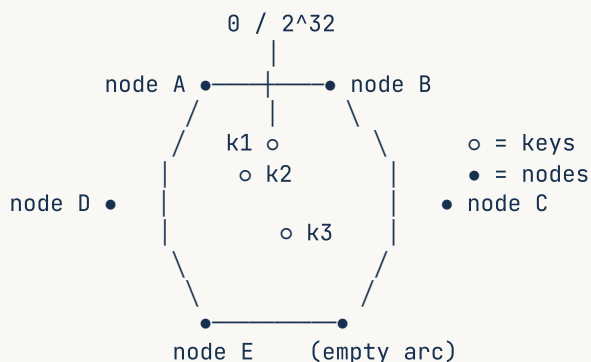
The alternatives, worth naming

- **Rendezvous hashing** (highest random weight): for each key, compute `hash(key, node)` for every node and pick the highest. Same minimal-movement property, no ring, no virtual nodes, and perfectly even — at the cost of $O(N)$ work per lookup instead of $O(\log N)$. **Better for small clusters.**
- **Jump consistent hash:** a few lines of arithmetic, no memory, perfectly even — but nodes must be numbered `0..N-1` and you can only remove the *last* one. **Great when the cluster only grows.**
- **Maglev hashing** (Google's load balancer): builds a lookup table for $O(1)$ routing with very even distribution and small disruption. **Used where lookups must be extremely fast.**

Naming one alternative and its trade-off is a strong finish to this question.

4 The picture

The ring, and what adding a node actually disturbs.



RULE: a key belongs to the first NODE CLOCKWISE from it.

k1 → B k2 → B k3 → C

ADD node F between B and C:

the keys in the arc (B, F] now go to F

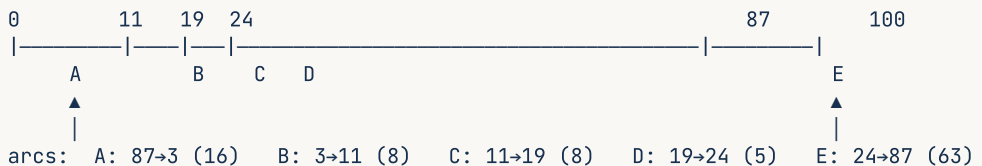
→ k3 may move to F

→ k1, k2 and EVERY OTHER KEY are untouched

→ and everything F receives comes from exactly ONE node: C

The naive version's problem, drawn to scale:

5 nodes, ONE point each, ring of 100



E owns 63% of the keyspace. This is the four-kilometre stretch.

AND IF E DIES: all 63% goes to A, its single successor.
A now has 79%. A falls over. Then B. Cascade.

Virtual nodes, and both fixes at once:

SAME 5 MACHINES, 200 VIRTUAL NODES EACH – 1,000 points on the ring

|aAcBdEbCaDeAbBcCdDeE...(1,000 small arcs, interleaved)...|

FIX 1 – distribution:

each machine owns ~200 small scattered arcs
busiest / quietest $\approx 1.2\times$ instead of $8\times$

FIX 2 – failure spreading:

machine E dies $\rightarrow$ its 200 arcs are absorbed by ~200 DIFFERENT successors
 $\rightarrow$ the load spreads across the whole cluster
 $\rightarrow$ each survivor takes $\sim 1/N(N-1)$ more, instead of one machine doubling

The comparison that answers the question:

8 nodes $\rightarrow$ 9 nodes, 1 TB of data

PLAIN MODULO

keys that move: $\sim 89\%$
data moved: ~ 890 GB
source machines: ALL of them
safe partial state: NO

CONSISTENT HASHING

keys that move: $\sim 11\%$
data moved: ~ 111 GB
source machines: spread over the ring
safe partial state: yes, arc by arc

and the reason, in one line:

modulo: location = $f(\text{key}, N)$ N changes $\rightarrow$ everything changes
consistent: location = $f(\text{key})$, then "who is next clockwise?"
 N changes $\rightarrow$ one arc changes

5 How it actually works

The implementation, in outline

```
ring = {}                                # hash position → physical node
sorted_positions = []                    # kept sorted, for binary search

def add_node(name, vnodes=200):
    for i in range(vnodes):
        pos = hash_32(f"{name}#{i}")
        ring[pos] = name
        insort(sorted_positions, pos)

def get_node(key):
    pos = hash_32(key)
    i = bisect_right(sorted_positions, pos)  # the first position > pos
    if i == len(sorted_positions):
        i = 0                               # wrap around the circle
    return ring[sorted_positions[i]]
```

Three details that decide whether it works:

- **bisect on a sorted list of positions** makes a lookup $O(\log(N \times V))$ — about 11 comparisons for five machines with 200 virtual nodes each. This is the [binary search](#) from day 42 doing real work.
- **The wrap-around** — if a key hashes past the last node, it belongs to the first. Forgetting it is the classic implementation bug, and it silently sends a slice of keys to the wrong place.
- **The hash function must be well-distributed but need not be cryptographic.** MD5 was traditional; **MurmurHash** or **xxHash** are the modern choices, being much faster and well-spread. Do **not** use Python's built-in `hash()` for anything shared between processes — it is randomised per process by default, so two machines would disagree about where a key lives.

Migrating the keys when a node joins

For a **cache**, you usually do nothing: the new node starts empty, its keys miss, and they are repopulated from the origin. The cost is a brief dip in hit rate for $1/N$ of the keys.

For a **store**, the arc's data must actually move:

1. the new node joins the ring in a "joining" state — it receives WRITES but reads still go to the old owner
2. the old owner streams the arc's data to it
3. once caught up, the new node starts serving reads for that arc
4. the old owner deletes the arc's data

That is the same four-phase shape as [yesterday's](#) resharding, which is worth noting: dual write, backfill, cut over, clean up.

Bounded-load consistent hashing

A refinement worth naming, because it addresses the honest weakness. Plain consistent hashing balances **keys**, not **traffic** — a hot key still overloads its owner.

The bounded-load variant caps each node at, say, 1.25 times the average, and when a key hashes to a node that is already at its cap, it walks on to the next one clockwise. **Used by Google's load balancers and by Vimeo, whose engineering write-up is the standard reference.**

What real systems do

- **Amazon Dynamo** (the 2007 paper) introduced virtual nodes for exactly the two reasons above, and it is where the vocabulary comes from.
 - **Cassandra** calls them **vnodes** and uses 256 per machine by default. Its ring is the partitioner (Murmur3 by default), and replication walks clockwise skipping same-rack nodes.
 - **DynamoDB** partitions this way internally, and splits a partition when it gets too large or too hot — though it still cannot split a single partition key, which is [yesterday's](#) unbounded-key problem.
 - **memcached clients** ([ketama](#), originally from Last.fm) do consistent hashing entirely client-side, with no coordination between clients at all — which is the purest demonstration of the property: every client computes the same answer independently.
 - **Akamai** — the technique was invented at MIT in 1997 for exactly this problem, assigning web objects to a changing set of caches, and Akamai was founded on it.
-

6 The numbers

The headline

keys that move when the cluster changes size

	plain modulo	consistent hashing
4 → 5	~80%	~20%
8 → 9	~89%	~11%
16 → 17	~94%	~6%
100 → 101	~99%	~1%

consistent hashing moves $1/(N+1)$, which is the theoretical minimum: the new node must receive its fair share, and nothing else moves.

1 TB across 8 nodes, adding a 9th

modulo:	~890 GB moved	≈ 2.5 hours of saturated 1 Gbit/s copying
consistent:	~111 GB moved	≈ 20 minutes

Virtual nodes and evenness

V per machine	spread (busiest / mean)	std. dev. of load
-----	-----	-----
1	5-10×	~100%
10	~2×	~32%
100	~1.2×	~10%
200	~1.15×	~7%
500	~1.05×	~4%

the deviation falls roughly as $1/\sqrt{V}$, so returns diminish quickly. 100-256 is the standard choice; Cassandra defaults to 256.

Cost of virtual nodes:

5 machines × 200 vnodes	=	1,000 ring entries	~50 KB
100 machines × 256	=	25,600 entries	~1 MB
lookup cost	=	$O(\log(N \times V))$	
	=	~15 comparisons at 25,600 entries	

Negligible memory, negligible lookup cost. There is no reason to skimp.

Failure spreading, quantified

10 machines, one dies

V = 1: its 10% of keys go to ONE successor
→ that machine goes from 10% to 20% of total load: it DOUBLES
→ if it was at 60% capacity, it is now at 120%. It fails. Cascade.

V = 200: its 10% is spread over ~9 other machines
→ each goes from 10% to ~11.1% of total load
→ a 1.1% increase each. Nothing happens.

That contrast is the strongest argument for virtual nodes, and it is about availability rather than evenness.

Cache hit rate during a change

8 cache nodes, adding a 9th, no data migration
keys that relocate: ~11%
those keys miss once, then repopulate

at a 95% baseline hit rate and 10,000 reads/s:
during the repopulation window, hit rate dips to ~85%
origin load goes from 500/s to ~1,500/s for a few minutes

with plain modulo:
89% of keys relocate
hit rate falls to ~10%
origin load goes from 500/s to ~9,000/s → the origin falls over

“Adding a cache node takes down the database” is a real incident, and it is what consistent hashing prevents.

Lookup cost against the alternatives

consistent hashing (ring)	$O(\log(N \times V))$	~15 comparisons
rendezvous hashing	$O(N)$	N hashes per lookup – fine for $N < 50$
jump consistent hash	$O(\log N)$	a few arithmetic operations, NO memory
maglev	$O(1)$	a table lookup, ~65,000 entries per cluster

7 The trade-offs

What it costs you

Placement control. You cannot choose where a key lives; the hash decides. So you cannot pin a large tenant to a specific machine, keep data in a jurisdiction, or move one noisy key. **That is the reason databases with explicit shard maps exist**, and the

reason multi-tenant systems use directory sharding instead.

It balances keys, not traffic. A perfect ring with a hot key still has a hot node — yesterday's point, and it is the honest weakness. Bounded-load consistent hashing addresses it partially; caching the hot key addresses it completely.

Range queries are gone. The hash destroys ordering, so “all keys between X and Y” is a scan of every node. If you need ranges, use ordered partitioning with an explicit map.

Ring or logical shard map?

The ring needs no shared state — every client computes the same answer from the node list alone, which is why memcached clients use it with no coordination whatsoever. **The map** is explicit, controllable and must be distributed and kept in step.

Take the ring for caches and stateless routing; take the map for databases, where you want to decide placement, and where you already have a coordination service.

How many virtual nodes?

100 to 256. Below about 100 the distribution is visibly uneven; above about 500 the improvement is marginal and the ring gets large.

I would use more for a small cluster — with three machines, the spread with few virtual nodes is severe — **and fewer** for a very large one, where `N` itself already smooths things and the ring size starts to matter.

Consistent hashing or rendezvous hashing?

Rendezvous is simpler and more even: for each key, hash it with every node and take the highest score. No ring, no virtual nodes, no sorted structure, and it handles weights cleanly. Its cost is $O(N)$ per lookup.

Take rendezvous for small clusters — under about fifty nodes, `N` hashes per lookup is genuinely fine and the code is ten lines. **Take consistent hashing for large ones,** where $O(\log(N \times V))$ matters.

Being able to say “rendezvous hashing is simpler and I would use it under fifty nodes” is a strong, uncommon answer.

Where it breaks

- **A hot key.** The ring cannot help; only caching, replication or key splitting can.

- **Client disagreement.** Every client must have the same view of the node list. If one client thinks a dead node is alive, it routes there. A shared membership service — or a gossip protocol, as Cassandra uses — is required, and it is a real dependency.
- **A non-deterministic hash.** Python's `hash()` on strings is randomised per process by default, so two processes disagree about every key. **Use an explicit hash function.**
- **The wrap-around bug.** Keys hashing past the last node must go to the first. Forget it and a slice of the keyspace silently goes nowhere or to the wrong node.
- **Flapping nodes.** A machine that repeatedly leaves and rejoins causes its arc to migrate back and forth. Membership needs hysteresis, exactly like the health-check thresholds on [day 099](#).

8 In the interview

How it gets asked

- The direct one: *“You add a cache node. How many keys move?”*
- The setup: *“Why not just use `hash(key) % N`?”*
- The one that separates people: *“Why virtual nodes?”*
- The failure probe: *“A node dies. What happens to its load?”*
- The limits: *“Does this solve hot keys?”*

What to say out loud, in the first ninety seconds

1. **State the problem first.** “With `hash % N`, the number of nodes is *inside* the calculation, so changing it changes almost every key’s answer — going from eight nodes to nine relocates about 89 percent of the keys.”
2. **Give the mechanism in one sentence.** “Hash the keys and the nodes into the same space, treat it as a circle, and a key belongs to the first node clockwise. The node count never appears in the calculation.”
3. **Give the number.** “Adding a node moves about `1/(N+1)` of the keys — 11 percent at eight-to-nine — and they all come from one neighbour. Everything else is untouched. That is the theoretical minimum, because the new node has to receive its fair share.”
4. **Pre-empt the flaw.** “The version people draw first — one point per node — does not work: a handful of random points on a circle land very unevenly, so one node can easily own 60 percent of the ring.”

5. **Give both reasons for virtual nodes.** “So each machine gets 100 to 200 points. That evens the distribution to about 1.2 times the mean — but the more important reason is failure: with one point per node, a machine’s entire load lands on a single successor and doubles it, which cascades. With 200 points, its load spreads across the whole cluster.”
6. **Name the honest limit.** “It balances *keys*, not *traffic*. A hot key still overloads its owner, and that is a caching problem, not a hashing problem.”

The follow-ups

“**Why not just use** `hash(key) % N`?” “Because `N` is inside the calculation, so every key’s location is a function of the cluster size. Going from eight nodes to nine, only about one key in nine stays put — 89 percent relocate. For a cache that means the hit rate collapses from 95 percent to near zero and the origin takes the entire read load: at 10,000 reads a second, the database goes from 500 to 9,000 requests a second, which is how ‘we added a cache node’ becomes ‘the database fell over’. For a store it means moving 890 gigabytes out of a terabyte to add one machine, and there is no safe intermediate state because every key changes owner at the same instant. Consistent hashing removes `N` from the calculation entirely — the key’s position on the ring never changes; only the answer to ‘who is the next node clockwise’ does.”

“**Why virtual nodes?**” “Two reasons, and the second is the one people miss. **Distribution:** with one point per machine, the points land wherever the hash puts them, which is random rather than even. With five machines it is common for one to own well over half the ring — I have seen the textbook example where one node owns 63 percent. Giving each machine 100 to 200 points makes it own many small scattered arcs, and the good and bad stretches average out, taking the spread from something like eight times the mean down to about 1.2 times. **Failure spreading**, which matters more: with one point each, when a machine dies its *entire* load goes to a single successor — that machine’s load doubles, and if it was at 60 percent capacity it is now at 120 and it fails too. That is a cascade. With 200 points, the dead machine’s arcs are absorbed by 200 different successors, so in a ten-node cluster each survivor takes about one percent more and nobody notices.”

“**A node dies. What happens to its load?**” “Its arcs are absorbed by whichever nodes are next clockwise from each of them. With virtual nodes that is many different machines, so in a ten-machine cluster each survivor’s share goes from 10 percent to about 11.1 percent — a one percent increase, invisible. Without virtual nodes it all lands on one machine and doubles it. For a **cache**, that is the end of the story: the keys miss and repopulate from the origin, and the hit rate dips for `1/N` of the keys for a few minutes. For a **store**, the data has to already be somewhere else, which is why a

ring-based store keeps each key on the next R **distinct** nodes clockwise — replication falls straight out of the ring with no separate placement logic. ‘Distinct’ is the detail: walking clockwise you may hit several virtual nodes of the same machine, so you skip until you have R different physical machines, and usually R different racks.”

“**Does this solve hot keys?**” “No, and I would say that plainly, because it is the honest limitation. Consistent hashing distributes **keys** evenly; it cannot distribute **traffic** evenly, because one key can be a thousand times more popular than another and it still lives on exactly one node. A perfectly balanced ring with a celebrity’s key on it has a hot node. The fixes are the ones from yesterday: cache that key so the requests never reach the node, replicate it to every node if it is small and rarely changes, or salt it into ten keys if the traffic is writes. There is also a refinement called **bounded-load consistent hashing**, which caps each node at, say, 1.25 times the average and walks a key on to the next node when its owner is full — Google’s load balancers and Vimeo use it — but it manages the symptom rather than removing it.”

“**How would you implement the lookup?**” “Keep the virtual node positions in a sorted array, and for a key, binary-search for the first position greater than the key’s hash — with a wrap-around to index zero if the key hashes past the last node. That is $O(\log(N \times V))$, so about fifteen comparisons for a hundred machines with 256 virtual nodes each, and the ring itself is under a megabyte. Two implementation details that are actual bugs rather than preferences. The **wrap-around** must be there, or a slice of the keyspace goes to the wrong node silently. And the **hash must be deterministic across processes** — Python’s built-in `hash()` on strings is randomised per process by default, so two clients would disagree about where every key lives. I would use MurmurHash or xxHash; MD5 works and is slower, and a cryptographic hash is not needed.”

“**Is there anything simpler?**” “**Rendezvous hashing**, and I would genuinely prefer it for a small cluster. For each key you compute `hash(key, node)` for every node and pick the highest score. It has the same minimal-movement property, it needs no ring, no sorted structure and no virtual nodes, it is perfectly even, and weighting a node is a one-line change. The cost is $O(N)$ hashes per lookup instead of $O(\log(N \times V))$ — which for under about fifty nodes is nothing, and for a thousand nodes is too much. There is also **jump consistent hash**, which is a few lines of arithmetic with no memory at all, but it requires nodes to be numbered zero to $N-1$ and you can only remove the last one — so it is excellent for a cluster that only grows and useless otherwise. And **Maglev**, Google’s, which builds a lookup table for $O(1)$ routing.”

A model answer

Asked: *you add a cache node. How many keys move?*

“With consistent hashing, about $1/(N+1)$ — so going from eight nodes to nine, roughly 11 percent, and they all come from one neighbour. Everything else in the cluster is untouched. That is the theoretical minimum, because the new node has to receive its fair share and nothing more.

The reason that matters is what the alternative does. With `hash(key) % N`, the node count is *inside* the calculation, so changing it changes almost every key’s answer — eight to nine relocates about **89 percent** of the keys. For a cache that is not a migration inconvenience, it is an outage: the hit rate falls from 95 percent to near zero, and at ten thousand reads a second the origin goes from about 500 requests a second to nine thousand. ‘We added a cache node and the database fell over’ is a real incident, and this is its cause.

The mechanism is simple. Hash every **key** into a large space — say 32 bits — and hash every **node** by its name into the *same* space. Treat that space as a circle by joining the ends. A key belongs to the **first node clockwise** from it. The number of nodes never appears anywhere in that calculation, which is exactly why changing it does not change every answer: a new node inserts itself at one point on the circle and takes the arc between itself and its predecessor. One arc changes; the rest of the ring does not know anything happened.

The version people draw first, with one point per node, does not work in practice, and I want to say why before you ask. A handful of points hashed onto a circle land **randomly, not evenly** — with five nodes it is entirely normal for one to own more than half the ring. So each physical machine gets many points, typically **100 to 256**, by hashing `node-A#0`, `node-A#1` and so on. That is virtual nodes, and it fixes two separate things.

The obvious one is **distribution**: many small scattered arcs per machine, so the busiest is about 1.2 times the mean instead of eight times. The one that matters more is **failure**. With one point per machine, when a machine dies its entire share lands on a single successor and that machine’s load doubles — and if it was at 60 percent capacity it is now over 100 and it fails too, which is a cascade. With 200 points, the dead machine’s arcs are absorbed by 200 different successors, so in a ten-node cluster each survivor takes about one percent more and nothing happens.

Two limits I would state. It balances **keys**, not **traffic** — a hot key still overloads its owner, and that is a caching problem rather than a hashing one. And it gives up **placement control**: you cannot pin a big tenant to a chosen machine or keep data in a jurisdiction, which is why databases often use an explicit shard map

instead, and caches and stateless routers use the ring — because the ring needs no shared state at all. Every memcached client computes the same answer independently, with no coordination whatsoever, and that is the property that makes it worth the complexity.”

9 Recall card

- **The problem:** `hash % N` puts the node count **INSIDE** the calculation, so $8 \rightarrow 9$ nodes relocates **~89%** of keys — for a cache, the hit rate collapses and the origin takes **9,000 req/s instead of 500**. **The fix:** hash keys **AND** nodes into one space, join it into a circle, and a key belongs to the first node **CLOCKWISE**. The node count never appears.
- **Adding a node moves** $1/(N+1)$ of the keys — **~11% at $8 \rightarrow 9$** — and they all come from **ONE neighbour**. That is the theoretical minimum. Removing a node sends its arc to its successor and disturbs nothing else.
- **Virtual nodes (100–256 per machine, Cassandra defaults to 256) exist for TWO reasons, and the second is the one people miss.** Distribution: one point per node lands randomly, so one node can own **63% of the ring**; 200 points brings the spread to $\sim 1.2 \times$ the mean. **Failure spreading:** with one point, a dead machine’s entire load lands on one successor and **doubles** it $\rightarrow$ cascade; with 200, it spreads across the cluster and each survivor takes **~1% more**.
- **Implementation:** sorted positions + binary search, $O(\log(N \times V)) \approx 15$ comparisons; the ring is under a megabyte. Two real bugs: the **wrap-around** (a key past the last node belongs to the first) and a **non-deterministic hash** — Python’s `hash()` is randomised per process, so use MurmurHash or xxHash. Replication falls out for free: the next **R** distinct nodes clockwise.
- It balances **KEYS, not TRAFFIC** — it does not solve hot keys, and it gives up placement control (hence directory sharding for big tenants and jurisdictions). Rendezvous hashing is simpler and more even at $O(N)$ per lookup — genuinely better under **~50 nodes**; jump hash is memory-free but only allows removing the last node; Maglev gives $O(1)$.

DSA topic: Validating a binary search tree **System design topic:** Consistent hashing

Code these, in this order

One rule for the whole set: **write the wrong version first, run it on the killer input, and record the result.** You cannot claim to have avoided a trap you have never seen fire.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Validate Binary Search Tree	LeetCode 98 (Medium)	The range method, and the counter-example to the local check.
2	Minimum Absolute Difference in BST	LeetCode 530 (Easy)	Inorder plus a previous value — the same one-variable idea.
3	Recover Binary Search Tree	LeetCode 99 (Medium)	One or two violations, and which nodes to take from each.
4	Largest BST Subtree	LeetCode 333 (Medium)	The postorder return-value trick returning four things.

On problem 1, run the wrong version first

Write the child-comparison version. Run it on `[10, 5, 15, null, null, 6, 20]` and record what it says. Then run `search(6)` on that tree and record that too. Then write the correct one.

On problem 1 again, break the bounds

Write a version seeded with `-2**31` and `2**31 - 1`, and run it on a single node whose value is `-2**31`. Record the answer and say why it is wrong.

On problem 3, do both swap shapes

Build a tree with two adjacent values swapped, and another with two non-adjacent values swapped. Print the inorder sequence for each and count the violations. Say which nodes you take in each case.

On problem 4, write the naive version first and time it

Check each subtree independently, then do the one-pass version. Time both on a 2,000-node chain and record the ratio.

The wrong-answer drill

1. Write the child-comparison version from memory.
2. Draw the killer input and say why every pair passes.
3. Say what `search` does on that tree, and why that is the worst kind of wrong.
4. Produce a second, smaller failing input.

The range drill

1. Write the range version from memory.
2. Say what happens to `low` and `high` on each branch.
3. Label every node of a seven-node tree with its permitted interval.
4. Trace the killer input and say which bound catches the 6, and where that bound came from.

The sentinel drill

1. Say why `None` is used rather than `INT_MIN`.
2. Give the exact input that breaks the `INT_MIN` version.
3. Name one other value that works, and one reason to prefer `None`.

The inorder drill

1. Write the inorder version keeping only the previous value.
2. Say why `≤` rather than `<`.
3. Say what the list-building version costs and why you would not write it.
4. Say what the short-circuit saves, with an example.
5. Write the iterative version and say when you would use it.

The convention drill

1. State the no-duplicates convention and the two comparisons it implies.
2. Change to “duplicates go left” and say which single comparison changes.
3. Run `[2, 2, 2]` through both.
4. Say what you would do in a real implementation instead.

The recover drill

1. State the one-or-two-violations rule.
2. For an adjacent swap, say which nodes are the culprits.
3. For a non-adjacent swap, say which node you take from each violation.

4. Say why you swap values rather than nodes.
5. State the space complexity and name the $O(1)$ variant.

The break-it drill

Trigger each and record the exact output:

1. The child-comparison version on the killer input.
2. A range version that drops `high` when recursing right.
3. `INT_MIN` as the seed, on a node valued `-2**31`.
4. `<` instead of `≤`, on `[2, 2, 2]`.
5. A module-level `previous` variable, called twice.
6. A recursive validation on a 10,000-node chain.

The escalation drill

1. Write the naive largest-BST-subtree and say its complexity on a chain.
 2. Write the one-pass version and say what the four returned values are.
 3. Say what condition makes a node's subtree a BST, in terms of the children's returns.
 4. Name the earlier day this shape came from.
-

The modulo drill

1. Say why `hash % N` breaks when `N` changes, in one sentence about where `N` appears.
2. Give the fraction of keys that move for $4 \rightarrow 5$, $8 \rightarrow 9$ and $100 \rightarrow 101$.
3. Compute the origin load when a cache node is added under plain modulo, at 10,000 reads/s and a 95% baseline hit rate.
4. Say why there is no safe intermediate state.

The ring drill

1. State the four steps of the mechanism.
2. Say what happens to the keys when a node is added, and where they come from.
3. Say what happens when a node is removed.
4. Give the fraction that moves and say why it is the minimum.
5. Draw a ring with four nodes and three keys and assign each key.

The virtual-nodes drill

1. Give both reasons for virtual nodes.

2. Say which one candidates usually miss.
3. Fill in the table of virtual nodes against spread.
4. Compute what happens to one survivor when a machine dies, with 1 and with 200 virtual nodes, in a ten-machine cluster.
5. State the memory and lookup cost of 100 machines at 256 virtual nodes.

The implementation drill

1. Write `add_node` and `get_node` in outline.
2. Say what data structure holds the positions and what the lookup costs.
3. Name the two implementation details that are bugs rather than preferences.
4. Say why Python's built-in `hash()` must not be used, and what to use instead.
5. Say how replication falls out of the ring, and what "distinct" means there.

The limits drill

1. Say whether consistent hashing solves hot keys, and why.
2. Name the refinement that partially addresses it and who uses it.
3. Say what placement control you give up, and which systems therefore do not use it.
4. Say what happens to range queries.

The alternatives drill

1. Describe rendezvous hashing in two sentences.
2. Give its cost and say at what cluster size you would prefer it.
3. Describe jump consistent hash and its restriction.
4. Name the `0(1)` alternative and who built it.

The failure drill

For each, say what happens and what you would add:

1. Two clients disagree about which nodes are alive.
2. A key hashes past the last node on the ring.
3. A node repeatedly leaves and rejoins.
4. One key receives 40 percent of all traffic.
5. Someone asks for all keys between X and Y.
6. A large tenant must be kept on a specific machine in a specific country.

Two of the six are not solved by consistent hashing at all. Name them and say what is.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Is this a valid BST? Are you sure your check is correct?* The invariant with “all the subtree”, the wrong answer killed with a counter-example you produce yourself, the range formulation with bounds narrowing, the `None` -not- `INT_MIN` detail, the duplicates convention, and the inorder alternative offered.
2. *You add a cache node. How many keys move?* The $1/(N+1)$ number, why modulo is catastrophic with the origin-load arithmetic, the four-step mechanism with the node count absent from it, both reasons for virtual nodes, and the honest limits.
3. *Why virtual nodes?* Distribution with the 63-percent example, then failure spreading with the doubling-and-cascade arithmetic, and the standard count.

Before you move on

- ☐ I can produce the killer input from memory, before being asked.
- ☐ I can say what `search` does on that tree, and why that is worse than a wrong answer.
- ☐ I can write the range version and label a tree with its intervals.
- ☐ I use `None` for unbounded and can give the input that breaks `INT_MIN`.
- ☐ I use `≤` and can say what `<` accepts.
- ☐ I can state the duplicates convention and which comparison it changes.
- ☐ I can write the inorder version with one variable, and the iterative one.
- ☐ I can state the one-or-two-violations rule and which nodes to take.
- ☐ I can write the one-pass largest-BST-subtree and name what it returns.
- ☐ I can say why `hash % N` breaks, in terms of where `N` appears.
- ☐ I can give the movement fractions for modulo and for the ring.
- ☐ I can compute the origin load spike from a naive cache resize.
- ☐ I can state the four steps of the ring mechanism.
- ☐ I can give both reasons for virtual nodes and say which is usually missed.
- ☐ I can do the failure arithmetic for 1 versus 200 virtual nodes.
- ☐ I can name the wrap-around bug and the hash-determinism bug.
- ☐ I can say what consistent hashing does not solve, and what does.
- ☐ I can name one alternative and the cluster size where I would prefer it.

- [] I answered all three questions above out loud.

Balanced trees, and why balance matters

DATA STRUCTURES AND ALGORITHMS

Balanced trees, and why balance matters

You can show the input that turns a BST into a linked list, and name the fix.

SYSTEM DESIGN

Back-of-the-envelope estimation

You can produce QPS, storage and bandwidth numbers out loud in three minutes.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is the worst case for a BST? How do real databases avoid it?*
- *Estimate the storage for five years of this system's data.*

Balanced trees, and why balance matters

1 What this is, and why they ask it

A tree is **balanced** when its height stays close to $\log n$ instead of growing towards n . Every $O(\log n)$ claim about a binary search tree depends on it, and nothing in **insert** or **delete** does anything to make it true.

Three sentences. The failing input is not exotic — **sorted data**, which is how data usually arrives — and it turns a million-node tree into a chain where every operation is a million steps. The fix is a **rotation**: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order. And the two families of self-balancing tree — AVL and red-black — differ in how strictly they enforce balance, which is a straight trade between read speed and write cost.

They ask it because the previous three days all quietly assumed balance and this is where the assumption gets paid for. The expected answer has three parts: **the input that breaks it**, **what a rotation does**, and **which structure you would actually use** — and for the last part the correct answer includes “I would not implement a red-black tree in an interview, and here is what I would do instead.”

2 The story

Ilango's cart was a two-wheeled hand cart with the wheels in the middle, and he sold fruit from it on the road outside the college.

The way it worked was that the load sat over the axle and he lifted the handles to move it. If the weight was spread evenly on both sides of the wheels, the handles were light — he could lift them with one hand and push the cart with the other on his phone.

If everything was piled at the far end, the handles wanted to fly up, and he had to hang his whole weight on them to keep the cart level. Twenty minutes of that and his arms were finished.

The problem was that a morning's selling did it to him automatically.

He loaded properly at four in the morning — front and back matched. Then customers bought, and they did not buy evenly. Everybody wanted the sweet limes at the front, so by ten the front was empty and there were still forty kilos of oranges over the back wheel, and the cart was unusable.

The obvious fix, which he did for the first year, was to stop, take everything off, and repack the whole cart. That took twenty minutes and he could not sell during it.

What he learned from the man who sold him the cart was much smaller.

You do not repack. You move **one crate**. You pick the crate nearest the wheels on the heavy side and you lift it across to the other side. Ten seconds. The cart is level again, nothing else has been touched, and the fruit is still in the same order — sweet limes at the front, oranges at the back, because you only moved something that was already in the middle.

He said the trick is that it has to be a crate near the middle. If you take one from the far end and carry it all the way across, you have changed the arrangement and now you cannot find anything. Take the one at the pivot; that one can go either way without disturbing the order.

Ilango did that maybe six or seven times a day, ten seconds each. Total, about a minute. Against twenty minutes of repacking, twice.

And he said one more thing about it that took him a season to understand. He did not wait until the cart was unusable. He checked after every few sales, and if it was starting to lean, he moved a crate then. If you wait, one crate is not enough and you are back to repacking.

3 The idea in plain English

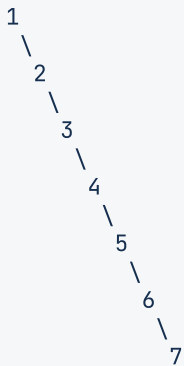
Ilango's crate is a rotation, and the two things he learned — move something near the pivot, and correct early — are the two things that make self-balancing trees work.

- Weight all at one end is a **degenerate tree**.
- Repacking the whole cart is **rebuilding the tree** — $O(n)$.
- Moving one crate near the wheels is a **rotation** — $O(1)$, and it changes nothing else.
- “The fruit is still in the same order” is the crucial property: **a rotation preserves the inorder sequence**.
- Checking after every few sales is **rebalancing on every insertion**, rather than when it is already broken.

The failing input

```
for value in [1, 2, 3, 4, 5, 6, 7]:  
    insert(tree, value)
```

Every value is larger than everything already there, so it goes to the far right, every time.



height = $n - 1$
every operation = $O(n)$
this is a linked list with an unused pointer per node

Sorted input is the normal case, not an adversarial one. Data arrives sorted by id, by timestamp, by name, by creation order — far more often than randomly. And reverse-sorted does the same thing to the left.

```
n = 1,000,000  
balanced:    ~20 comparisons per lookup  
degenerate:  1,000,000  
ratio:       50,000×
```

Also: a recursive traversal of a chain of 10,000 nodes raises `RecursionError`. So the failure is not only slow, it can be fatal.

The one piece of good news, worth saying: **a randomly built BST is fine**. Its expected height is about $2 \log n$ — around 40 for a million nodes — so the enemy is *order*, not adversarial data.

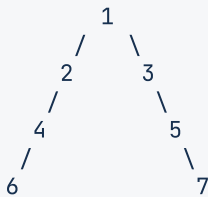
What “balanced” means

The usual definition is **height-balanced**:

At **every** node, the heights of the left and right subtrees differ by at most 1.

“Every node”, not just the root — the same whole-subtree distinction as the [BST invariant](#), and it fails in the same way if you check only the top.

a tree whose ROOT is balanced but which is not balanced:



root: left height 2, right height 2 → difference 0 ✓
node 2: left height 1, right height -1 → difference 2 ✗

Checking this naively calls `height` at every node, which is $O(n^2)$ — the [day 102](#) mistake. The one-pass version returns the height, or a sentinel meaning “already unbalanced”, and short-circuits.

Rotations

A **rotation** rearranges three or four pointers around one node so that a child moves up and the node moves down. It reduces the height on one side by one, and — this is the property everything depends on — **it does not change the inorder sequence**, so the tree is still a valid BST afterwards.

RIGHT ROTATION about y



LEFT ROTATION about x



inorder before: A x B y C
inorder after: A x B y C

IDENTICAL. That is why it is legal.

The code is four lines and worth knowing by heart:

```
def rotate_right(y):
    x = y.left
    y.left = x.right      # B moves across
    x.right = y           # y goes down
    return x              # x is the new subtree root
```

```
return x matters: the caller must reattach, exactly as with deletion. node.left = rotate_right(node.left) .
```

Ilango’s crate: **only the thing at the pivot moves**, and the order is preserved.

The four cases

After inserting, one node becomes unbalanced. There are four shapes and two of them need two rotations.

LEFT-LEFT	inserted into the left child's LEFT subtree	→ one RIGHT rotation
RIGHT-RIGHT	inserted into the right child's RIGHT subtree	→ one LEFT rotation
LEFT-RIGHT	inserted into the left child's RIGHT subtree	→ LEFT on the child, then RIGHT
RIGHT-LEFT	inserted into the right child's LEFT subtree	→ RIGHT on the child, then LEFT

The zig-zag cases need two rotations, and the reason is worth one sentence: a single rotation on a zig-zag shape just produces the mirror-image zig-zag. The first rotation straightens it into a left-left or right-right, and the second fixes it.

AVL and red-black: the same idea, two settings

Both are BSTs that rotate on insertion and deletion. They differ in **how strict** the balance rule is.

	AVL	RED-BLACK
Rule	heights differ by ≤ 1 at every node	no red node has a red child; every root-to-leaf path has the same number of black nodes
Height bound	$\leq 1.44 \log n$	$\leq 2 \log n$
Lookups	faster (shorter tree)	slightly slower
Insert/delete	more rotations	fewer rotations
Used by	in-memory indexes, read-heavy structures	<code>std::map</code> , Java’s <code>TreeMap</code> , the Linux kernel scheduler

Red-black wins in practice because most workloads mix reads and writes, and its looser rule means far less rotation on write. **AVL wins when reads dominate heavily.**

Do not implement either in an interview. The honest answer is: *“I would use the language’s ordered map, which is a red-black tree. If I had to hand-build something with these guarantees I would write a treap or a skip list, because they are twenty lines instead of two hundred.”*

The alternatives that are actually writable

Treap. A BST by key and a heap by a random priority. Insert normally, then rotate up while the priority is out of order. Because the priorities are random, the shape is a randomly built BST, whose expected height is $O(\log n)$. **About twenty-five lines**, and it is the answer to “implement a balanced tree” in a real interview.

Skip list. Not a tree at all — a linked list with random express lanes. Same $O(\log n)$ expected bounds, much simpler code, and it is what **Redis uses for sorted sets**.

Both are randomised: they guarantee $O(\log n)$ *expected*, not worst case. **Say that;** it is the trade you are making for the simplicity.

And what databases actually do

Not binary trees at all. A database index is a **B-tree** (or B+ tree), which is the same idea with a much higher branching factor: each node holds hundreds of keys and fills one disk page.

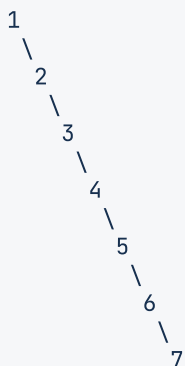
```
binary tree, 1,000,000 keys: height ~20 → 20 disk reads
B-tree, 500 keys per node:    height ~3  → 3 disk reads
```

A disk read is about 10 ms and a memory read is 100 ns, so the height is the cost. Reducing 20 reads to 3 is the entire reason B-trees exist, and it is the answer to “how do real databases avoid it?”

4 The picture

The failing input, drawn against what it should be.

insert 1..7 IN ORDER

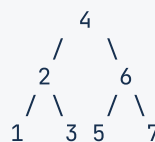


height 6
search: up to 7 steps

SAME SEVEN VALUES.
SAME RULE.

Only the ORDER OF INSERTION differs.

insert 4,2,6,1,3,5,7



height 2
search: up to 3 steps

A rotation, with the inorder sequence written underneath — which is the point.

BEFORE (right-heavy)

x

/ \

A y

/ \

B C

inorder: A x B y C

^^^^^^^^^^^^^^

AFTER a LEFT rotation about x

y

/ \

x C

/ \

A B

inorder: A x B y C

^^^^^^^^^^^^^^

IDENTICAL – which is why the tree is still a valid BST.

height on the right: reduced by one.

nodes touched: three pointers. Everything else in the tree is untouched.

The four cases:

LEFT-LEFT

z

/

y

/

x

rotate RIGHT(z)

RIGHT-RIGHT

z

\

y

\

x

rotate LEFT(z)

LEFT-RIGHT

z

/

y

\

x

step 1: LEFT(y)

step 2: RIGHT(z)

RIGHT-LEFT

z

\

y

/

x

step 1: RIGHT(y)

step 2: LEFT(z)

the zig-zag cases need TWO rotations, because ONE rotation on a zig-zag produces the mirror-image zig-zag – it has to be straightened first.

Ilango's cart, as the same picture:

BEFORE

[][][o][o][o]

▲

wheels

heavy behind the axle

handles fly up

AFTER moving ONE crate at the pivot

[o][][o][o][]

▲

wheels

level, and the ORDER of the crates along the cart is unchanged

what he did NOT do: unload and repack (O(n))

what he did: move the crate AT the pivot (O(1))

Where the height goes:

```
n = 1,000,000
```

structure	height	lookup
-----	-----	-----
degenerate BST	999,999	1,000,000 comparisons
random BST	~40	~40
red-black tree	≤ 40	≤ 40
AVL tree	≤ 29	≤ 29
B-tree (500/node)	3	3 DISK READS ← the one that matters

5 The code, built step by step

Step 1 — show the failing input first

“The worst case is sorted input, and it is not adversarial — data arrives sorted by id or timestamp far more often than randomly. Insert one to a million in order and you have a chain: every operation is a million steps instead of twenty, and a recursive traversal raises `RecursionError`.”

Demonstrate it rather than describe it. Building the chain takes three lines.

Step 2 — define balanced with “every node”

```
abs(height(node.left) - height(node.right)) ≤ 1
```

...at **every** node, not just the root. And write the check in one pass, with a sentinel — the [day 102](#) technique — because the naive version is $O(n^2)$.

Step 3 — the rotation, and the property that makes it legal

```
def rotate_right(y):
    x = y.left
    y.left = x.right
    x.right = y
    return x
```

Say the property as you write it: “This does not change the inorder sequence, so the tree is still a valid BST. It just moves one node up and one down.” That is the sentence the whole topic rests on.

Step 4 — name the four cases and why two need two rotations

“Left-left and right-right are one rotation. Left-right and right-left are zig-zags, and a single rotation on a zig-zag gives you the mirror-image zig-zag — so you straighten it first with a rotation on the child, then fix it with a rotation on the node.”

Step 5 — say what you would actually use

“I would not hand-write a red-black tree. In production I would use the language’s ordered map. If I had to implement something with these guarantees, I would write a **treap** — a BST by key and a heap by a random priority — which is about twenty-five lines and gives $O(\log n)$ expected, or a **skip list**, which is what Redis uses for sorted sets.”

The complete solution

```
import random
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

def height(node: TreeNode | None) → int:
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))

def is_balanced(root: TreeNode | None) → bool:
    """Height-balanced at EVERY node, in one pass.

    -2 is the sentinel for "already unbalanced". Safe because a real height
    is never below -1. The naive version calls height() at every node and is
    O(n^2) on a chain – the day 102 mistake.
    """
    UNBALANCED = -2

    def check(node: TreeNode | None) → int:
        if node is None:
            return -1
        left = check(node.left)
        if left == UNBALANCED:
            return UNBALANCED          # short-circuit
        right = check(node.right)
        if right == UNBALANCED or abs(left - right) > 1:
            return UNBALANCED
        return 1 + max(left, right)

    return check(root) != UNBALANCED

# -----
# ROTATIONS – the whole technique, in eight lines.
# -----

def rotate_right(y: TreeNode) → TreeNode:
    """y goes down, its left child x comes up. Returns the new subtree root.

    THE PROPERTY THAT MAKES IT LEGAL: the inorder sequence is unchanged.
    before:  A x B y C
    after:   A x B y C
    So the tree is still a valid BST. Only the SHAPE changed.

    O(1): three pointers. Nothing outside this subtree is touched.
    """
    x = y.left
    assert x is not None
    y.left = x.right          # B moves across
```

```

x.right = y
return x

# y goes down
# the caller must REATTACH this

def rotate_left(x: TreeNode) → TreeNode:
    """The mirror image."""
    y = x.right
    assert y is not None
    x.right = y.left
    y.left = x
    return y

# -----
# AVL – strict balance. Written out once so the four cases are concrete.
# -----

class AVLNode:
    __slots__ = ("val", "left", "right", "height")

    def __init__(self, val: int) → None:
        self.val = val
        self.left: "AVLNode | None" = None
        self.right: "AVLNode | None" = None
        self.height = 0
        # a leaf has height 0

    def _h(node: "AVLNode | None") → int:
        return -1 if node is None else node.height

    def _update(node: AVLNode) → None:
        node.height = 1 + max(_h(node.left), _h(node.right))

    def _balance_factor(node: AVLNode) → int:
        return _h(node.left) - _h(node.right)

    def _rotate_right(y: AVLNode) → AVLNode:
        x = y.left
        assert x is not None
        y.left = x.right
        x.right = y
        _update(y)
        _update(x)
        return x
        # y first – it is now BELOW x

    def _rotate_left(x: AVLNode) → AVLNode:
        y = x.right
        assert y is not None
        x.right = y.left
        y.left = x
        _update(x)
        _update(y)
        return y

def avl_insert(node: "AVLNode | None", value: int) → AVLNode:
    """Insert, then rebalance on the way back up.

    THE FOUR CASES:
    LL left-heavy, inserted left of the left child → rotate RIGHT
    RR right-heavy, inserted right of the right child → rotate LEFT

```


LR left-heavy, inserted RIGHT of the left child → LEFT(child), RIGHT(node)
 RL right-heavy, inserted LEFT of the right child → RIGHT(child), LEFT(node)

The zig-zag cases need TWO rotations because one rotation on a zig-zag just produces the mirror-image zig-zag.

"""

```

if node is None:
    return AVLNode(value)
if value < node.val:
    node.left = avl_insert(node.left, value)
elif value > node.val:
    node.right = avl_insert(node.right, value)
else:
    return node # duplicate: ignored

_update(node)
balance = _balance_factor(node)

if balance > 1: # left-heavy
    assert node.left is not None
    if value > node.left.val: # LEFT-RIGHT
        node.left = _rotate_left(node.left)
    return _rotate_right(node) # LEFT-LEFT

if balance < -1: # right-heavy
    assert node.right is not None
    if value < node.right.val: # RIGHT-LEFT
        node.right = _rotate_right(node.right)
    return _rotate_left(node) # RIGHT-RIGHT

return node

```

```

def avl_height(node: "AVLNode | None") → int:
    return _h(node)

```

```

def avl_inorder(node: "AVLNode | None", out: list[int] | None = None) → list[int]:
    out = [] if out is None else out
    if node is None:
        return out
    avl_inorder(node.left, out)
    out.append(node.val)
    avl_inorder(node.right, out)
    return out

```

```

# -----
# TREAP – the one to actually write in an interview. ~25 lines.
# -----

```

```

class TreapNode:
    __slots__ = ("val", "priority", "left", "right")

    def __init__(self, val: int) → None:
        self.val = val
        self.priority = random.random() # RANDOM: this is what balances it
        self.left: "TreapNode | None" = None
        self.right: "TreapNode | None" = None

```

```

def treap_insert(node: "TreapNode | None", value: int) → TreapNode:
    """A BST by VALUE and a max-heap by PRIORITY.

```

Insert as a normal BST leaf, then rotate it upward while its random

priority is higher than its parent's. Because the priorities are random, the resulting shape is exactly a RANDOMLY BUILT BST – expected height $O(\log n)$ – regardless of the insertion order.

$O(\log n)$ EXPECTED, not worst case. Say that; it is the trade for twenty-five lines instead of two hundred.

"""

```
if node is None:
    return TreapNode(value)
if value < node.val:
    node.left = treap_insert(node.left, value)
    if node.left.priority > node.priority:
        left = node.left
        node.left = left.right
        left.right = node
        node = left
        # rotate right
elif value > node.val:
    node.right = treap_insert(node.right, value)
    if node.right.priority > node.priority:
        right = node.right
        node.right = right.left
        right.left = node
        node = right
        # rotate left
return node
```

```
def treap_height(node: "TreapNode | None") → int:
    if node is None:
        return -1
    return 1 + max(treap_height(node.left), treap_height(node.right))
```

```
def treap_inorder(node: "TreapNode | None", out: list[int] | None = None) → list[int]:
    out = [] if out is None else out
    if node is None:
        return out
    treap_inorder(node.left, out)
    out.append(node.val)
    treap_inorder(node.right, out)
    return out
```

```
# -----
# The plain BST, for comparison, and the rebuild fix.
# -----
```

```
def bst_insert(node: TreeNode | None, value: int) → TreeNode:
    if node is None:
        return TreeNode(value)
    if value < node.val:
        node.left = bst_insert(node.left, value)
    elif value > node.val:
        node.right = bst_insert(node.right, value)
    return node
```

```
def rebuild_balanced(root: TreeNode | None) → TreeNode | None:
    """The  $O(n)$  alternative to rotations: flatten to sorted order, then
    rebuild from the middle. Ilango repacking the whole cart.
```

Correct, simple, and $O(n)$ – fine as an occasional maintenance operation, useless as a per-insertion strategy.

"""

```
values: list[int] = []
```

```

def flatten(node: TreeNode | None) → None:
    if node is None:
        return
    flatten(node.left)
    values.append(node.val)
    flatten(node.right)

flatten(root)

def build(lo: int, hi: int) → TreeNode | None:
    if lo > hi:
        return None
    mid = (lo + hi) // 2
    return TreeNode(values[mid], build(lo, mid - 1), build(mid + 1, hi))

return build(0, len(values) - 1)

if __name__ == "__main__":
    # THE FAILING INPUT – three lines, and it is the whole worst case
    chain: TreeNode | None = None
    for v in range(1, 8):
        chain = bst_insert(chain, v)
    print(height(chain), is_balanced(chain))           # 6 False

    # the same values, balanced
    good: TreeNode | None = None
    for v in (4, 2, 6, 1, 3, 5, 7):
        good = bst_insert(good, v)
    print(height(good), is_balanced(good))           # 2 True

    # rotations preserve inorder
    y = TreeNode(5, TreeNode(3, TreeNode(1), TreeNode(4)), TreeNode(8))

    def inorder(n: TreeNode | None, out=None):
        out = [] if out is None else out
        if n:
            inorder(n.left, out); out.append(n.val); inorder(n.right, out)
        return out

    before = inorder(y)
    y = rotate_right(y)
    print(before, inorder(y), before == inorder(y))   # same list, True
    print(y.val)                                       # 3 – the new root

    # AVL survives sorted input
    avl: AVLNode | None = None
    for v in range(1, 8):
        avl = avl_insert(avl, v)
    print(avl_height(avl), avl_inorder(avl))          # 2 [1..7]

    avl_big: AVLNode | None = None
    for v in range(1, 1001):
        avl_big = avl_insert(avl_big, v)
    print(avl_height(avl_big))                         # 9    (1.44 log2(1000) ≈ 14)

    # a plain BST with the same input
    plain: TreeNode | None = None
    for v in range(1, 1001):
        plain = bst_insert(plain, v)
    print(height(plain))                               # 999 ← the disaster

    # a treap does not care about insertion order
    random.seed(7)
    treap: TreapNode | None = None

```

```

for v in range(1, 1001):
    treap = treap_insert(treap, v)
    print(treap_height(treap))                # ~20, from SORTED input
    print(treap_inorder(treap)[:5])          # [1, 2, 3, 4, 5]

# and the O(n) rebuild
fixed = rebuild_balanced(chain(
    print(height(fixed), is_balanced(fixed))    # 2 True

# a randomly built BST is fine – the enemy is ORDER, not randomness
values = list(range(1, 1001))
random.shuffle(values)
rnd: TreeNode | None = None
for v in values:
    rnd = bst_insert(rnd, v)
    print(height(rnd))                        # ~20, roughly 2*log2(1000)

```

6 What it costs

The height, which is everything

structure	worst-case height	lookup at n = 1,000,000
-----	-----	-----
degenerate BST	n - 1	1,000,000
random BST	$\sim 2 \log n$ (expected)	~ 40
red-black tree	$\leq 2 \log n$	≤ 40
AVL tree	$\leq 1.44 \log n$	≤ 29
perfect	$\log n$	20

The AVL bound of $1.44 \log n$ is worth knowing — it comes from the Fibonacci-shaped worst case, the sparsest tree that still satisfies the AVL rule.

Rotations, and the write cost

AVL insert:	up to 2 rotations, then the height updates propagate up
AVL delete:	up to $O(\log n)$ rotations – this is the AVL weakness
red-black insert:	at most 2 rotations
red-black delete:	at most 3 rotations

1,000,000 sequential insertions	
AVL:	$\sim 1,000,000$ rotations (roughly one per insert on sorted input)
red-black:	$\sim 600,000$
plain BST:	0 rotations, and a chain

That is the trade in one table: AVL gives shorter trees and does more work on writes.

The measurement that decides it

read-heavy (95% reads)	AVL wins: shorter tree, rotations rarely paid
write-heavy (50%+ writes)	red-black wins: far fewer rotations
general purpose	red-black – which is why <code>std::map</code> , Java's <code>TreeMap</code> and the Linux scheduler all use it

The rebuild alternative

rotation: $O(1)$ per fix, applied on every insertion
rebuild: $O(n)$, applied occasionally

rebuilding a 1,000,000-node tree: ~1,000,000 operations, and the tree is unusable during it

Rebuilding is right as maintenance and wrong as a strategy — Ilango unloading the cart. There is a middle ground, **scapegoat trees**, which rebuild only the subtree that went out of balance and achieve $O(\log n)$ amortised with no per-node book-keeping.

Treap and skip list

treap	$O(\log n)$ EXPECTED, ~25 lines, one random number per node
skip list	$O(\log n)$ EXPECTED, ~40 lines, no rotations at all
AVL	$O(\log n)$ WORST CASE, ~150 lines
red-black	$O(\log n)$ WORST CASE, ~250 lines and famously fiddly deletion

probability a treap of 1,000,000 nodes exceeds height 100: vanishingly small (the expected height is ~40, and the tail falls off exponentially)

“Expected” is a real caveat and it almost never bites. Say it, then say you would still use the treap.

B-trees, and why databases are different

the cost model changes completely once the data is on disk:

memory read	~100 ns	
SSD read	~100 μ s	1,000×
disk seek	~10 ms	100,000×

so the number of NODE VISITS is the cost, and reducing height is everything.

```
1,000,000 keys
binary tree:      height ~20  → 20 reads  → 200 μs on SSD
B-tree, 500 keys/node: height 3  → 3 reads  → 30 μs
B-tree, 1,000 keys/node: height 2  → 2 reads
```

A B-tree node is sized to one disk page — typically 4 KB or 16 KB — so it holds hundreds of keys, and the tree is three levels deep for a billion rows. That is the answer to “how do real databases avoid the worst case”, and it is not “they use AVL trees”.

7 The traps

Trap 1 — assuming balance without saying so

```
# "BST operations are  $O(\log n)$ "
```

Only if balanced. Say $O(\text{height})$ and then say what makes the height what it is. An interviewer who hears an unqualified $O(\log n)$ will produce the sorted-input case.

Trap 2 — checking balance only at the root

```
return abs(height(root.left) - height(root.right)) ≤ 1
```

Balance is required at **every** node. The tree in the picture has a balanced root and is not balanced. Same whole-subtree distinction as the BST invariant itself.

Trap 3 — the $O(n^2)$ balance check

```
if abs(height(node.left) - height(node.right)) > 1:
    return False
return is_balanced(node.left) and is_balanced(node.right)
```

Correct, and `height` walks a whole subtree at every node — $O(n \log n)$ balanced, $O(n^2)$ on a chain. [Day 102's fix](#): return the height, or a sentinel.

Trap 4 — a rotation that loses a subtree

```
def rotate_right(y):  
    x = y.left  
    x.right = y          # WRONG ORDER — B is lost  
    y.left = x.right  
    return x
```

Setting `x.right = y` before rescuing `x.right` throws away the middle subtree. **The order of the two assignments is the whole rotation**, and getting it wrong silently deletes nodes.

Write it as: “*rescue B, then swing y down.*”

Trap 5 — forgetting to reattach the rotation’s result

```
rotate_right(node.left)          # result discarded
```

Same trap as [deletion](#). The rotation returns the new subtree root and the caller must assign it. Without that, the tree is unchanged and there is no error.

Trap 6 — updating heights in the wrong order

```
_update(x)                      # x first  
_update(y)
```

After a right rotation, `y` is **below** `x`, so `y`’s height must be recomputed first. Updating `x` first uses `y`’s stale height and the whole tree’s heights drift, which silently breaks the balance logic without breaking the BST property.

Trap 7 — using one rotation on a zig-zag

```
if balance > 1:  
    return _rotate_right(node)    # no LR check
```

On a left-right shape, a single right rotation produces the mirror-image right-left shape — still unbalanced. **Check the direction of the insertion relative to the child**, and do the child rotation first.

Trap 8 — offering to implement a red-black tree

You will not finish it, and deletion has cases that are genuinely hard to get right under time pressure. **Say what you would use instead.** “I would use the language’s ordered map; if I had to write one, a treap or a skip list” is a better answer than a half-finished

red-black tree.

8 In the interview

How it gets asked

- The pair: “What is the worst case for a BST? How do real databases avoid it?”
- The demonstration: “Show me an input that makes it degenerate.”
- The mechanism: “What is a rotation, and why is it allowed?”
- The choice: “AVL or red-black?”
- The challenge: “Implement a self-balancing tree.”

What to say out loud, in the first ninety seconds

1. **Give the failing input immediately, and say it is ordinary.** “Insert sorted data and every value goes to the far right — you get a chain of height $n - 1$. And sorted input is the *normal* case: data arrives ordered by id or timestamp far more often than randomly.”
2. **Give the number.** “At a million nodes that is a million comparisons instead of twenty — a factor of fifty thousand — and a recursive traversal raises `RecursionError` past about a thousand.”
3. **Say the one piece of good news.** “A *randomly* built BST is fine — expected height about $2 \log n$. So the enemy is order, not adversaries.”
4. **Define balanced with “every node”.** “Height-balanced means the two subtree heights differ by at most one at **every** node, not just the root.”
5. **Explain a rotation by its property.** “A rotation moves one node up and one down by rewiring three pointers, and the reason it is legal is that **it does not change the inorder sequence** — so the tree is still a valid BST. It is $O(1)$ and it touches nothing outside that subtree.”
6. **Say what you would use.** “In production, the language’s ordered map, which is a red-black tree. If I had to write one here, a treap — twenty-five lines, $O(\log n)$ expected.”

The follow-ups

“Show me the input.” “`for v in range(1, n): insert(tree, v)`. Every value is larger than everything already in the tree, so it goes to the far right every time, and the result is a chain with height $n - 1$ — a linked list with an unused pointer per node. Reverse-sorted does the same thing to the left. And I would stress that this is not

an adversarial input: data arrives sorted by primary key, by timestamp, by name or by creation order far more often than it arrives shuffled, so a plain BST built from a database dump degenerates by default. The one reassuring fact is that a genuinely *random* insertion order gives an expected height of about $2 \log n$, so the problem is order rather than malice.”

“**What is a rotation and why is it allowed?**” “It is a local rearrangement of three pointers that moves a child up and its parent down. Right-rotating `y` with left child `x`: `x`’s right subtree becomes `y`’s left, `y` becomes `x`’s right, and `x` is the new subtree root. The reason it is legal is the property worth stating explicitly: **the inorder sequence is identical before and after**. Before it reads A, x, B, y, C; afterwards it reads A, x, B, y, C. So the BST invariant still holds — only the shape changed. It is $O(1)$, it touches nothing outside the subtree, and the caller must reattach the returned node, exactly like the deletion idiom. Two ways to get it wrong: assigning in the wrong order, which silently discards the middle subtree; and updating cached heights in the wrong order, since after the rotation the old parent is now the child.”

“**AVL or red-black?**” “They are the same idea with different strictness. **AVL** requires the subtree heights to differ by at most one at every node, which bounds the height at $1.44 \log n$ — a shorter tree, so faster lookups — and it pays for that with more rotations on writes, up to $O(\log n)$ of them on a delete. **Red-black** uses a looser colouring rule that bounds the height at $2 \log n$, so lookups are slightly slower, but insertion needs at most two rotations and deletion at most three. For a mixed or write-heavy workload red-black wins, which is why `std::map`, Java’s `TreeMap` and the Linux kernel’s scheduler all use it. AVL wins when reads dominate heavily — an index that is built once and read constantly. And I would say I would not hand-write either: red-black deletion in particular has cases that are hard to get right under time pressure.”

“**Implement a self-balancing tree.**” “I would write a **treap**, and say why. It is a BST by key and a max-heap by a **random** priority: insert as a normal leaf, then rotate it upward while its priority exceeds its parent’s. Because the priorities are random, the resulting shape is statistically identical to a randomly built BST — expected height $O(\log n)$ — **regardless of the insertion order**, which is exactly the property I need. That is about twenty-five lines against two hundred and fifty for red-black. The trade I would state honestly is that it is $O(\log n)$ **expected**, not worst case — though for a million nodes the probability of exceeding, say, height 100 is vanishingly small. The other twenty-line option is a **skip list**, which is not a tree at all — a linked list with random express lanes — and it is what Redis uses for sorted sets.”

“How do real databases avoid this?” “They do not use binary trees at all. A database index is a **B-tree**, which is the same balancing idea with a much higher branching factor: each node is sized to one disk page — typically 4 or 16 kilobytes — so it holds hundreds of keys rather than one. The reason is the cost model: on disk, the number of **node visits** is the cost, because a read is ten milliseconds against a hundred nano-seconds in memory. A binary tree over a million keys has height twenty, so twenty reads. A B-tree with five hundred keys per node has height three. For a billion rows it is still only four. That reduction in height is the entire point, and it is why ‘use an AVL tree’ is the wrong answer to a database question.”

“Is there an alternative to rotating on every insert?” “Two. **Rebuild** — flatten the tree to sorted order and rebuild from the middle, which is $O(n)$ and correct and completely wrong as a per-insertion strategy, though perfectly reasonable as occasional maintenance on a mostly-read structure. And the middle ground, **scapegoat trees**, which do no bookkeeping at all until a subtree goes too far out of balance, then rebuild just that subtree — $O(\log n)$ amortised, no colours or heights stored per node. Worth naming because it shows the design space is not just ‘rotate always or never’.”

A model answer

Asked: what is the worst case for a BST, and how do real databases avoid it?

“The worst case is a **chain**, and the input that produces it is sorted data.

Insert one through a million in order into a plain BST. Every value is larger than everything already there, so it goes to the far right, every time, and you end up with a tree of height $n - 1$ — a linked list with an unused pointer per node. Every operation that was supposed to be twenty comparisons is now a million, a factor of fifty thousand, and a recursive traversal raises `RecursionError` at about a thousand nodes.

The thing I would emphasise is that this is not an adversarial input. **Data arrives sorted** — by primary key, by timestamp, by name, by creation order — far more often than it arrives shuffled. A tree built from a database dump degenerates by default. The one reassuring fact is that a genuinely *random* insertion order is fine: expected height about $2 \log n$. So the enemy is order, not malice.

The fix inside the tree is a **rotation**. It rewires three pointers so that a child moves up and its parent moves down, and the reason it is legal is the property everything rests on: **it does not change the inorder sequence**. Before and after, the tree reads the same left to right, so it is still a valid BST — only the shape has changed. It is $O(1)$ and touches nothing outside that subtree. There are four cases: two straight ones needing a single rotation, and two zig-zags needing two, because one rotation on a zig-zag just gives you the mirror-image zig-zag.

Self-balancing trees rotate on every insertion so the tree never gets far out of shape. **AVL** enforces a height difference of at most one at every node, bounding the height at $1.44 \log n$ — shorter tree, faster reads, more rotations on writes. **Red-black** uses a looser rule bounding it at $2 \log n$, with at most two rotations per insert — which is why it is what `std::map`, Java’s `TreeMap` and the Linux scheduler actually use.

But for the database half of your question: **they do not use binary trees**. An index is a **B-tree**, which is the same balancing idea with a branching factor of hundreds — each node sized to one disk page. The reason is the cost model. On disk, the cost is the number of node visits, because a read is ten milliseconds against a hundred nanoseconds in memory, so the height is the running time. A binary tree over a million keys is twenty levels deep, so twenty reads; a B-tree with five hundred keys per node is three levels, so three reads. Over a billion rows it is four. That is the whole reason B-trees exist.

And if you asked me to implement something balanced right now, I would not write a red-black tree — deletion has cases that are genuinely hard under time pressure. I would write a **treap**: a BST by key and a heap by a random priority,

about twenty-five lines, which gives the shape of a randomly built tree regardless of insertion order. $O(\log n)$ expected rather than worst case, and I would say so.”

9 Recall card

- **The worst case is a CHAIN, and the input is SORTED DATA — which is the normal case**, since data arrives ordered by id or timestamp far more often than randomly. At $n = 1,000,000$ that is **1,000,000 comparisons instead of ~ 20** , plus `RecursionError` past $\sim 1,000$. **A randomly built BST is fine** (expected height $\sim 2 \log n$): the enemy is order, not adversaries.
- **Balanced means the subtree heights differ by ≤ 1 at EVERY node, not just the root** — and check it in **one pass with a sentinel**, because calling `height` at every node is $O(n^2)$ on a chain.
- **A rotation rewires three pointers to move a child up and its parent down, and it is legal because IT DOES NOT CHANGE THE INORDER SEQUENCE.** $O(1)$, touches nothing outside the subtree, and the caller must **reattach** the returned root. Four cases: **LL and RR need one rotation; LR and RL are zig-zags and need two**, because one rotation on a zig-zag gives the mirror-image zig-zag.
- **AVL: height $\leq 1.44 \log n$, faster reads, more rotations on writes. Red-black: height $\leq 2 \log n$, ≤ 2 rotations per insert** — which is why `std::map`, Java's `TreeMap` and the Linux scheduler use it. **Do not hand-write either.** Write a **treap** (BST by key, heap by a *random* priority, ~ 25 lines, $O(\log n)$ expected, immune to insertion order) or a **skip list** (what Redis uses for sorted sets).
- **Databases do not use binary trees — they use B-trees**, because on disk the cost is the number of node visits (**10 ms a read against 100 ns in memory**). A node is sized to one page and holds hundreds of keys: **1,000,000 keys is height 20 in a binary tree and height 3 in a B-tree**; a billion rows is four levels.

1 What this is, and why they ask it

Day 097 gave you the conversions. Today is the **performance**: doing the whole thing out loud, for a system you have never seen, in about three minutes, while somebody watches.

Three sentences. The skill is not arithmetic — it is a **fixed routine** that always produces the same five numbers, so that you are never deciding what to work out next. It depends on a small set of figures held in memory, because looking things up is not available and hesitating is what makes an estimate sound like a guess. And it depends on **rounding aggressively and saying that you are**, because an estimate whose purpose is to choose an architecture does not need to be right to two significant figures.

They ask it because it is the moment a design conversation becomes concrete, and because it is very easy to tell the difference between someone doing it and someone performing it. The tell is not accuracy. It is whether the candidate **states assumptions before using them, sanity-checks the answer against something known, and says what the number means for the design** — because a number with no consequence attached was not worth computing.

2 The story

Muthu had been building walls for twenty-six years and he did not carry a measuring tape to a first visit.

A man wanted a compound wall around a plot near the water tank and asked how much it would come to. Muthu walked the boundary, counting, with his hands behind his back.

Twenty-eight paces on the road side. Then the long side, forty-one. Then twenty-eight again, then forty-one. His pace was about two and a half feet, and he had checked that against a tape often enough to trust it.

He said, out loud, so the man could follow: about a hundred and thirty-eight paces, call it a hundred and forty, times two and a half — three hundred and fifty feet. Six feet high. Two thousand one hundred square feet of wall.

The man asked how many bricks.

Muthu said a nine-inch wall takes about ten bricks a square foot. So twenty-one thousand. Add five per cent for breakage and cutting — call it twenty-two thousand.

Then he did the thing that his nephew, who had been with him four years, had learned to watch for.

He stopped and said: the last one I did like this was the school ground, and that was about four hundred feet, and we used twenty-six thousand bricks. This is a bit shorter, so twenty-two is about right.

The nephew asked once why he bothered, since he had just worked it out.

Muthu said that working it out tells you a number and does not tell you whether you have made a mistake. You can drop a nought and never notice. Comparing against something you have actually built tells you whether the number is the right *size*. If he had come out with two hundred thousand bricks, the arithmetic would have looked fine on the way through and the answer would have been nonsense, and the only thing that catches it is remembering the school ground.

He was also, deliberately, not precise. He said three fifty feet when it was three forty-five, and ten bricks a square foot when it was nine and a half. He rounded up every time, and he said so to the customer, because the point of the number was to decide whether the man could afford the wall — and for that, twenty-two thousand and twenty thousand mean exactly the same thing.

The number he was careful about was the one that came at the end: the lorry. A lorry carries about four thousand bricks. Twenty-two thousand means six lorries, and six is not five, and that changes the day.

3 The idea in plain English

Muthu is doing back-of-the-envelope estimation, and his three habits are the three things being marked.

- Counting paces and multiplying is the **routine** — the same sequence every time, said out loud.
- “The school ground was four hundred feet and twenty-six thousand bricks” is the **sanity check**.
- Rounding up and saying so is **precision discipline**.
- The lorries are the **consequence** — the number that changes a decision, which is the only number that actually mattered.

The routine: six steps, in this order

Never deviate. The value is that you are not deciding what to do next.

- | | |
|--------------|---|
| 1. USERS | daily active users. State the assumption. |
| 2. ACTIONS | actions per user per day. REASON about it, do not guess. |
| 3. QPS | requests/day ÷ 100,000, then × 3 for peak. |
| 4. SPLIT | read:write ratio. Ask, or assume and say so. |
| 5. STORAGE | writes/day × bytes per record × days × replication. |
| 6. BANDWIDTH | requests/s × bytes per response. Only if media is involved. |

Then the seventh step, which is the one that counts: say what each number *means*.

“Two thousand requests a second, so six to eight application servers.” “Five terabytes a year, so this is one database with archiving, not a sharding problem.”

A number with no consequence attached was not worth saying.

The figures to hold in memory

You cannot look anything up. This is the whole list, and it is short.

TIME

seconds per day	86,400	→ round to 100,000
seconds per month	2.6 M	→ round to 2.5 M
seconds per year	31.5 M	→ round to 30 M

CONVERSIONS

1 million/day	≈	10 QPS
100 million/day	≈	1,000 QPS
1 billion/day	≈	10,000 QPS

SIZES

1 KB × 1 million	=	1 GB
1 KB × 1 billion	=	1 TB
1 MB × 1 million	=	1 TB
1 MB × 1 billion	=	1 PB

TYPICAL RECORDS

short post / tweet	~300 B
database row, 10 columns	~1 KB
chat message	~200 B
log line	~500 B
JSON API response	~5 KB
web page, total	~2 MB
compressed photo	~2 MB
1080p video, per minute	~50 MB

MACHINE CAPACITY

app server	~1,000 QPS of real work
relational DB, reads	~10,000/s indexed
relational DB, writes	~5,000/s
Redis	~100,000 ops/s
one network link	~1 GB/s
one SSD	~500 MB/s, ~50,000 IOPS

LATENCY

memory	~100 ns
SSD	~100 µs
same-DC round trip	~0.5 ms
cross-continent	~30 ms
cross-world	~150 ms

MULTIPLIERS

peak factor	×3	(×100+ for a scheduled event)
replication	×3	
DAU from registered	~10-20%	
cache hit rate	~90%	

That fits on one page and it is the entire toolkit. Everything else is multiplication.

Rounding discipline

Round to one significant figure and say that you are doing it.

86,400	→	100,000	makes the answer ~15% low
365	→	400 (or 300)	depending on which way you want to err
1,024	→	1,000	
0.15	→	0.2	

Two rules that make rounding safe rather than sloppy:

Round in the direction that makes you safe. For capacity, round *up*: over-provisioning is a cost, under-provisioning is an outage. For savings claims, round *down*.

Say which direction and why. “I am rounding 86,400 up to 100,000, so my QPS numbers are about fifteen percent low — which does not change the architecture.” That single sentence converts sloppiness into judgement.

The sanity check, which is the thing candidates skip

After every estimate, compare against something known. Muthu’s school ground.

```
"I got 50 TB/day of media."
Is that plausible? Instagram handles roughly 100 million uploads a day.
→ if my system is a tenth of Instagram, 50 TB/day is far too high.
→ re-check: did I use 2 MB or 20 MB per photo?

"I got 200 QPS."
Twitter's average was ~6,000 tweets/second at its peak of fame.
→ 200 QPS for a global product is suspiciously small.
→ re-check: is my DAU number registered users rather than daily?

"I got 40 servers."
Stack Overflow served the whole site on about a dozen machines.
→ 40 for something simpler suggests I have over-counted somewhere.
```

Two or three reference points are enough, and they catch the error that actually happens: a factor of ten, from a dropped zero or from confusing per-day with per-second.

The three sanity anchors worth memorising

a small product	100,000 DAU	→ ~10-20 QPS average
a large product	10 million DAU	→ ~2,000 QPS average
a giant	1 billion DAU	→ ~200,000 QPS average

If your answer is not near one of those for the size of product described, something is wrong.

Reasoning about actions per user

This is the only step that is not arithmetic, and it is where candidates guess when they should reason.

Do not say “let us assume twenty”. Say *why* twenty:

“A user opens the app three or four times a day. Each session loads a feed — that is one request for the feed plus a handful for images, though images go to a CDN so they do not count against my servers. They scroll a couple of pages and open one or two items. So roughly five server requests per session, four sessions — twenty a day.”

The reasoning is the answer. The number is a by-product, and the interviewer is listening to the derivation.

What to do when you do not know something

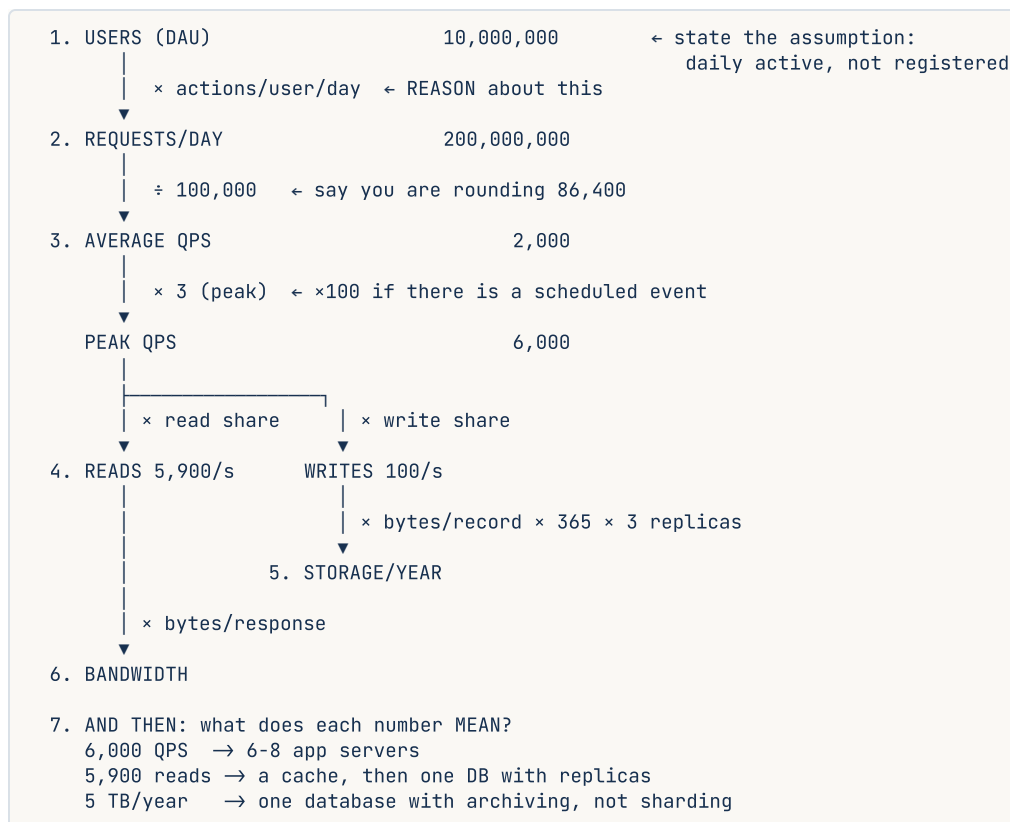
Ask. “What is the read-to-write ratio?” is a legitimate question and asking it is better than assuming.

If they will not tell you, assume and label it. “I will assume a hundred reads per write, which is typical for a feed product. If it is closer to one-to-one, like messaging, the answer changes and I would want to know.”

Never silently invent a number. A stated assumption that turns out wrong is a good answer with a wrong input; an unstated one is a guess.

4 The picture

The routine, as a ladder you climb the same way every time.



Rounding, drawn as a decision:

EXACT	ROUNDED	error	does it change the answer?
86,400 s/day	100,000	-15%	no
365 days	400	+10%	no
1,024 MB	1,000	-2%	no
2,314 QPS	2,000-2,500	±10%	no
2,314 QPS	23,000	10×	YES – and that is the error a sanity check catches

the arithmetic is allowed to be 20% wrong.
it is NOT allowed to be 10x wrong, and the only defence is comparing against something you already know.

Muthu's wall, as the same routine:

1. paces 138 → round to 140
2. × 2.5 ft/pace = 350 ft
3. × 6 ft high = 2,100 sq ft
4. × 10 bricks/sq ft = 21,000
5. + 5% breakage = 22,000
6. SANITY CHECK school ground: 400 ft → 26,000 bricks.
This is shorter. 22,000 is the right SIZE. ✓
7. CONSEQUENCE 4,000 bricks per lorry → SIX lorries, not five.
^^^ the only number that changed a decision

5 How it actually works

Worked example one: a chat application

“Design WhatsApp. Give me numbers.”

1. USERS
2,000,000,000 registered, and I will assume 50% daily active
→ 1,000,000,000 DAU (state it: "if that is wrong, everything scales")
2. ACTIONS – reasoned
an active user sends ~40 messages a day and receives ~40
sends are the writes; receives are fan-out, not separate client requests
→ 40,000,000,000 messages sent per day
3. QPS
40 billion ÷ 100,000 = 400,000 writes/second average
× 3 peak = 1,200,000 writes/second peak
4. SPLIT
messaging is roughly 1:1 – every message sent is read about once
→ reads ≈ writes. THIS IS THE UNUSUAL PART and worth saying:
"unlike a feed product, caching and read replicas will not save me here"
5. STORAGE
200 bytes per message × 40 billion = 8 TB/day
× 365 = 2.9 PB/year
× 3 replicas = 8.8 PB/year
→ WhatsApp does not store delivered messages, which is why it can exist.
If we did retain them, this is the number that forces that decision.
6. CONNECTIONS – the real constraint here
1 billion users with a persistent connection
÷ ~1,000,000 connections per tuned server
= ~1,000 servers just holding connections open
7. WHAT IT MEANS
- 1.2M writes/s is far past one database: this must be sharded, by conversation_id, from day one
- reads ≈ writes means caching does NOT rescue this
- the binding constraint is CONNECTIONS, not CPU

That last line is the point of the whole exercise. The estimate found the constraint, and the constraint is not the one you would have guessed.

Worked example two: a video service

“Design YouTube. How much storage?”

1. UPLOADS
500 hours uploaded per minute (a published figure, but derive it too:
 $2 \text{ billion users} \times 0.0002 \text{ uploads/day} \approx 400,000 \text{ uploads/day}$)
2. RAW SIZE
 $500 \text{ hours/minute} \times 60 = 30,000 \text{ hours/hour}$
 $\times 24 = 720,000 \text{ hours/day}$
1 hour of 1080p $\approx 3 \text{ GB}$
 $\rightarrow 2.2 \text{ PB/day of raw uploads}$
3. TRANSCODING MULTIPLIER $\leftarrow$ the step people forget
every video is stored at ~ 5 resolutions
plus audio-only, plus thumbnails
the lower resolutions are much smaller: total $\approx 1.5\times$ the original
 $\rightarrow \sim 3.3 \text{ PB/day}$
4. PER YEAR
 $3.3 \text{ PB} \times 365 \approx 1,200 \text{ PB} = 1.2 \text{ EB/year}$
5. REPLICATION
 $\times 2$ (not $\times 3$ – erasure coding is used for cold video, $\sim 1.4\times$ overhead)
 $\rightarrow \sim 1.7 \text{ EB/year}$
6. BANDWIDTH – the number that dwarfs storage
1 billion hours watched per day
at 5 Mbit/s average = 2.25 GB/hour
 $1 \text{ billion} \times 2.25 \text{ GB} = 2,250 \text{ PB/day of egress}$
 $\div 86,400 \text{ s} \approx 26 \text{ TB/second}$
 $\rightarrow$ this is why Netflix and YouTube ship hardware into ISPs
7. WHAT IT MEANS
 - storage is object storage with erasure coding, not a database
 - the DOMINANT cost is bandwidth, by an order of magnitude
 - therefore the architecture is a CDN problem, not a database problem

Notice step 3. The transcoding multiplier is the step candidates omit, and omitting it makes the answer wrong by a factor that matters.

Worked example three: a URL shortener

“How much storage in five years?”

1. WRITES
100,000,000 new URLs per day (state it as the assumption)
2. RECORD SIZE
short code (7 chars) 7 B
long URL ~100 B
user id, timestamps ~30 B
row overhead + index ~100 B
→ ~250 B, call it 500 B with the index
3. PER YEAR
100M × 500 B = 50 GB/day
× 365 = 18 TB/year
× 5 years = 90 TB
× 3 replicas = 270 TB
4. SANITY CHECK
90 TB of pure text over five years. Is that plausible?
100M/day × 5 years = 180 billion URLs at ~500 B – yes, arithmetic holds.
And 180 billion is more than the entire indexed web, which suggests
100M/day is a generous assumption. Worth saying.
5. READS
read:write ≈ 10:1 → 1 billion redirects/day = 10,000 QPS average
= 30,000 peak
6. WHAT IT MEANS
 - 90 TB does not fit one machine: this is sharded, by the short code
 - 30,000 QPS of reads on tiny immutable records: a cache at 95%+ is easy, so ~1,500/s reach the store
 - records are immutable, so caching has no invalidation problem at all

Step 4 is the one to imitate. The sanity check did not just confirm the arithmetic — it questioned the *input*, which is where estimates actually go wrong.

6 The numbers

The multiplication table for this skill

DAU × actions/day	= requests/day
requests/day ÷ 100,000	= average QPS
average QPS × 3	= peak QPS
writes/day × bytes	= storage/day
storage/day × 365 × 3	= storage/year, replicated
QPS × bytes/response	= bandwidth
peak QPS ÷ 1,000	= app servers
reads/s × (1 - hit rate)	= reads reaching the database

Eight lines. That is the entire subject.

Where estimates go wrong, in order of frequency

- | | |
|---|-----------------------------|
| 1. registered users used as daily active | 10× too high |
| 2. per-day confused with per-second | 86,400× wrong |
| 3. forgetting the peak factor | 3× too low |
| 4. forgetting replication | 3× too low |
| 5. forgetting a fan-out or transcoding multiplier | 2-100× too low |
| 6. forgetting that media goes to a CDN | 10× too high on server load |

Every one of those is a factor, not a percentage — which is exactly why the arithmetic being 20 percent off does not matter and the sanity check does.

Sanity anchors

Twitter	~6,000 tweets/second average, ~143,000 peak (a 2013 spike)
Instagram	~100 million photos/day
WhatsApp	~100 billion messages/day
YouTube	~500 hours uploaded per minute
Google search	~100,000 searches/second
Stack Overflow	~2 billion page views/year on ~a dozen servers

The Stack Overflow one is the most useful, because it is the counterweight: it is the reminder that most systems do not need a thousand machines, and that a candidate whose estimate produces four hundred servers for a modest product has made an error.

How precise is precise enough?

answer differs by	does the design change?
-----	-----
±20%	no
×2	rarely
×10	YES — different number of machines, maybe different DB
×100	YES — a different architecture entirely

So the target is the right order of magnitude, and one significant figure is plenty. Say that; it gives you permission to round and it tells the interviewer you know what the number is for.

The consequence table

The last step, and the one that turns arithmetic into design:

peak QPS	what it means
< 100	one server. Genuinely. Do not over-build.
1,000	a few servers, one database
10,000	load balancer, cache, read replicas, one DB still fine
100,000	sharding, and the design conversation changes
1,000,000+	a specialised architecture; expect published papers
storage	what it means
< 100 GB	one machine, in memory if you like
1 TB	one database, comfortable
10 TB	one big machine, or start planning to shard
100 TB	sharded, definitely
1 PB+	object storage, not a database
write rate	what it means
< 1,000/s	one database
5,000/s	one large database, at its limit
> 10,000/s	sharded. No other option.

Memorise the left column of those three tables. That is what makes the seventh step automatic.

7 The trade-offs

Estimation is for choosing, not for planning

The purpose is to decide the **shape** of the design — one database or twenty, cache or no cache, CDN or not. It is not a capacity plan, and treating it as one is the wrong use.

So: round hard, move fast, and spend the saved time on what the numbers imply. A candidate who spends eight minutes getting to 2,314 QPS has spent six minutes badly.

When the estimate is the whole answer

Occasionally the numbers settle the design entirely and there is nothing else to say. “You want to store every GPS ping from a million vehicles every second” — that is 86 billion points a day, and the answer is a time-series store with downsampling, and no amount of discussion changes it.

Recognising that early is a strength, not a shortcut.

When the estimate misleads

- **Averages hide the peak.** The system that fails is the one sized for average QPS.
- **Fan-out is invisible in user-facing numbers.** One celebrity post is one write to the API and fifty million writes to the feed system. The user-facing rate says nothing about it, and it is the single biggest trap in estimating a social product.
- **Connections are not requests.** A chat app with a million idle users has almost no QPS and a million open sockets. Memory per connection, not CPU per request, is the constraint.
- **Storage accumulates; QPS does not.** Always quote storage per year, and always ask about retention.
- **Media dwarfs everything and belongs to a CDN.** Counting image bytes against your servers overstates the load by ten times or more.

Being wrong out loud

Recovering from a mistake well is worth more than not making one. If a sanity check shows you are a factor of ten out, say so: *“That is too big — Instagram is about a hundred million photos a day and I have just produced a billion. Let me check my per-user rate.”*

An interviewer who sees you catch your own error learns more about you than one who sees a clean answer.

8 In the interview

How it gets asked

- The direct one: *“Estimate the storage for five years of this system’s data.”*
- As the second thing in any design: *“Before we go further, give me some numbers.”*
- The specific probe: *“How many servers?”* / *“What is the peak QPS?”*
- The check: *“Does that number seem right to you?”*
- The consequence: *“So what does that tell you about the design?”*

What to say out loud, in the first ninety seconds

1. **Announce the routine.** “Let me work up from users: daily actives, actions per day, requests per second, peak, then the read-write split, then storage.”
2. **State every assumption as you use it.** “I will assume that ten million is daily active rather than registered. If it is registered, daily active is usually ten to twenty percent, and everything below divides by ten.”

3. **Reason about actions per user; do not guess.** “Three or four sessions a day, five server requests each — images go to a CDN so they do not count — so about twenty.”
4. **Round out loud.** “There are 86,400 seconds in a day and I am rounding to a hundred thousand, so my QPS is about fifteen percent low. That does not change anything.”
5. **Sanity-check against something known.** “That gives two thousand QPS average. For a product with ten million daily users, that is the right order — Twitter was around six thousand tweets a second at its peak of fame.”
6. **Say what it means.** “Six thousand peak QPS is six to eight application servers and one database with a cache. Five terabytes a year is one database with archiving, not a sharding problem.”

The follow-ups

“**Estimate the storage for five years.**” “Let me build it up. First the write rate: ten million daily active users, and I will reason about writes rather than guess — this is a feed product, so a user posts perhaps once every couple of days, which is five million posts a day. Then the record size: a post row with an id, a user id, text, timestamps and indexes is about one kilobyte, and I will say a kilobyte rather than five hundred bytes because indexes usually double it. So five gigabytes a day, which is about **1.8 terabytes a year**, or **9 terabytes over five years** — and with three-way replication, roughly **27 terabytes on disk**. Media is a separate number and much larger: if ten percent of posts have a photo at two megabytes plus thumbnails, that is another terabyte a day, so **400 terabytes a year**, and that goes to object storage rather than a database. Sanity check: nine terabytes of text over five years for a ten-million-user product feels right — it is the sort of number one large machine holds. What it means for the design: the metadata is one database with archiving of old rows, and the media is S3 behind a CDN.”

“**Does that number seem right to you?**” “Let me check it against something I know rather than re-doing the arithmetic, because re-doing it repeats the same mistake. Instagram handles roughly a hundred million photo uploads a day. My system is about a hundredth of Instagram’s size, so I would expect something around a million uploads a day — and I got half a million, which is the right order. If I had produced fifty million, I would go back and look for a dropped factor of ten, most likely from using registered users where I meant daily active, or from confusing per-day with per-second. Those two are the errors that actually happen, and they are factors of ten and eighty-six thousand respectively, which is why the sanity check matters more than the precision.”

“How many servers?” “Peak QPS divided by what one server does. I have six thousand peak, and an application server doing real work handles around a thousand requests a second — so six machines at a hundred percent utilisation, which is not a thing you run, so **nine or ten at a sane sixty to seventy percent**. That is the compute side. I would then check the database separately: five thousand nine hundred reads a second is borderline for one relational database, but with a cache at ninety percent only about six hundred reach it, which is very comfortable. So the shape is: a load balancer, ten app servers, Redis, one primary with a couple of read replicas. And I would say the caveat — those per-server numbers depend enormously on what a request does, so if each request runs a complex query, one server might handle two hundred rather than a thousand and I would want to measure rather than assume.”

“What is the read-to-write ratio, and does it matter?” “It matters more than almost any other number, because it decides whether the problem is caching or sharding. If it is high — a hundred to one, which is typical for a feed or a catalogue — then reads dominate, a cache removes ninety percent of them, and read replicas handle the rest; the write path is almost free. If it is close to one to one, which is messaging, then caching does very little and I am looking at sharding the write path much sooner. So I would ask, and if you would not tell me I would assume a hundred to one for a feed product and say clearly that the answer changes if it is closer to even.”

“So what does that tell you about the design?” “That is the step that makes the arithmetic worth doing, so let me go through each number. **Six thousand peak QPS** means six to ten application servers — a normal deployment, nothing exotic. **A hundred writes a second** means one database takes the writes comfortably; I am nowhere near needing to shard, which would cost me joins and transactions. **Five thousand nine hundred reads a second** means a cache is the highest-leverage thing I can add, because at ninety percent it removes ten times more load than another replica. **Five terabytes a year** means one database with archiving rather than a distributed store. And if any one of those numbers had been ten times larger, the answer would be different — at a hundred thousand QPS or ten thousand writes a second, I would be sharding from day one and the whole conversation changes.”

“What if you do not know how many actions a user takes?” “I ask, because it is a legitimate question and asking is better than inventing. If you will not tell me, I reason it out loud rather than guessing a number: how many times does someone open this app in a day, what does a session actually do, and how many of those requests reach my servers rather than a CDN. For a feed product that lands around twenty; for a messaging app it is closer to eighty; for e-commerce it is five page views and a fraction

of an order. Then I label it as an assumption and carry on, so that if it is wrong you can correct one input rather than the whole estimate. The thing I would not do is state a number with no derivation — the reasoning is what you are actually asking for.”

A model answer

Asked: *estimate the storage for five years of this system's data.*

“Let me work up from users, and I will say every assumption as I use it so you can correct any of them.

Users. You said ten million. I am going to take that as **daily active**, not registered — and that matters, because if it is registered then daily active is usually ten to twenty percent of it and every number I give divides by ten.

Writes. I need writes per day, and rather than guess a number I will reason: this is a feed product, so a typical user posts something perhaps once every two days. That gives **five million posts a day**.

Record size. A post row — id, user id, text, a couple of timestamps, some flags — is a few hundred bytes, and indexes typically double that, so I will call it **one kilobyte**. Rounding up, deliberately, because under-estimating storage is the expensive direction.

So **5 GB a day**, times 365 is about **1.8 TB a year**, times five years is **9 TB**, and with the standard three-way replication that is roughly **27 TB on disk**.

Media is separate and much bigger. If ten percent of those posts carry a photo, at about two megabytes plus a few thumbnails, that is another **1 TB a day — 400 TB a year**. That does not belong in a database; it goes into object storage behind a CDN, with only the key stored in the row.

Let me sanity-check the shape of that before I go on, because the error that actually happens in these estimates is a factor of ten, not twenty percent. Instagram handles around a hundred million photo uploads a day. My system is roughly a hundredth of that size and I have produced half a million uploads a day, which is the right order. If I had come out with fifty million I would go back and look for registered-versus-daily-active, or a per-day-versus-per-second slip.

And then the part that makes the arithmetic worth doing — **what it means**. Nine terabytes of metadata over five years is comfortably one database with old rows archived; it is not a sharding problem, which matters because sharding would cost me joins and transactions permanently. Four hundred terabytes a year of media is object storage and a CDN, and the CDN is what stops that bandwidth touching my servers at all. The only number I would want to firm up before committing is the posts-per-user rate, because everything above scales linearly with it.”

9 Recall card

- **Six steps, in the same order, every time:** users $\rightarrow$ actions/day $\rightarrow$ QPS ($\div 100,000$, $\times 3$ for peak) $\rightarrow$ read:write split $\rightarrow$ storage ($\times$ bytes $\times 365 \times 3$) $\rightarrow$ bandwidth. Then the **seventh and most important:** say what each number *means* for the design. A number with no consequence attached was not worth computing.
- **Round to one significant figure and SAY that you are** — “86,400 rounded to 100,000, so I am $\sim 15\%$ low, which changes nothing”. Round **up** for capacity, **down** for savings. The target is the right **order of magnitude**: $\pm 20\%$ changes nothing, $\times 10$ changes the design.
- **Sanity-check against something known, every time.** Twitter $\sim 6,000$ tweets/s; Instagram $\sim 100\text{M}$ photos/day; WhatsApp $\sim 100\text{B}$ messages/day; YouTube ~ 500 hours uploaded/minute; Stack Overflow ran on $\sim$ a dozen servers. The errors that actually happen are **factors**: registered-as-daily-active ($10\times$), per-day-as-per-second ($86,400\times$), forgetting peak ($3\times$), replication ($3\times$), fan-out or transcoding ($2\text{--}100\times$).
- **Reason about actions per user; never guess a number.** “Three or four sessions, five server requests each, images go to the CDN — so twenty.” **The derivation is the answer.** And when you do not know, **ask**; if refused, **assume and label it**.
- **Memorise the consequence table.** Peak QPS: $< 100 \rightarrow$ one server $\cdot 1,000 \rightarrow$ a few + one DB $\cdot 10,000 \rightarrow$ LB + cache + replicas $\cdot 100,000 \rightarrow$ sharding. Storage: 1 TB $\rightarrow$ one DB $\cdot 100$ TB $\rightarrow$ sharded $\cdot 1$ PB $\rightarrow$ object storage. Writes: $> 10,000/\text{s} \rightarrow$ sharded, no other option. And watch the three invisible constraints: **fan-out**, **connections** (not requests), and **storage accumulating** while QPS does not.

Practice — Balanced trees, and why balance matters

DSA topic: Balanced trees, and why balance matters **System design topic:** Back-of-the-envelope estimation

Code these, in this order

One rule for the whole set: **build the degenerate tree first, with your own hands, and measure it.** Three lines produce it. Every claim in this lesson is about a number you can print.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Balanced Binary Tree	LeetCode 110 (Easy)	Balance at <i>every</i> node, in one pass with a sentinel.
2	Convert Sorted Array to BST	LeetCode 108 (Easy)	The $O(n)$ rebuild, and why the middle is the root.
3	Balance a Binary Search Tree	LeetCode 1382 (Medium)	Flatten to sorted order, then rebuild — the whole cart repacked.
4	Insert into a Binary Search Tree	LeetCode 701 (Medium)	Then extend it into an AVL insert with the four cases.

Before problem 1, build the disaster

Insert 1 to 1,000 in order into a plain BST. Print the height. Then shuffle the same values and do it again. Record both numbers and the ratio.

On problem 2, say why the middle

Explain in one sentence why taking the middle element as the root prevents the chain, and give the exact height it produces.

On problem 3, then compare against rotations

You have now written the $O(n)$ rebuild. Say what a rotation costs instead, and when each is the right tool.

On problem 4, extend it

Add height tracking and the four rebalancing cases. Insert 1 to 1,000 in order and print the height. Then write a treap and do the same, and compare all three heights.

The failing-input drill

1. Write the three lines that produce a degenerate tree.
2. Say why sorted input is the normal case rather than an adversarial one.
3. Give the comparison count for a balanced and a degenerate million-node tree.
4. Say what else fails on a chain besides speed, and quote the error.
5. State the expected height of a randomly built BST.

The definition drill

1. Define height-balanced, using the word “every”.
2. Draw a tree whose root is balanced and which is not balanced.
3. Write the one-pass check with a sentinel and say why `-2` is safe.
4. Say what the naive check costs on a chain.

The rotation drill

1. Write `rotate_right` in four lines from memory.
2. Write the inorder sequence before and after and say what that proves.
3. Say what happens if the two assignments are swapped.
4. Say what the caller must do with the return value.
5. Say what `0(1)` means here in terms of pointers touched.

The four-cases drill

1. Name all four cases and the rotations each needs.
2. Draw each one.
3. Say why the zig-zag cases need two rotations, in one sentence.
4. Apply a single rotation to a left-right shape and draw what you get.

The AVL drill

1. Write `avl_insert` with the four cases.
2. Say in which order the heights must be updated after a rotation, and why.
3. Insert 1 to 1,000 in order and print the height.
4. Compare against the plain BST and the theoretical bound.

The comparison drill

1. Fill in the AVL versus red-black table from memory.
2. Say which is used by `std::map`, Java and the Linux scheduler, and why.

3. Say which you would pick for a read-heavy in-memory index.
4. Say what you would actually say if asked to implement one.

The treap drill

1. Write a treap insert from memory.
2. Say what makes it balanced, in one sentence about randomness.
3. Insert 1 to 1,000 in sorted order and print the height.
4. State the guarantee precisely, including the word “expected”.
5. Name the other twenty-line option and the product that uses it.

The B-tree drill

1. Say why databases do not use binary trees.
2. Give the latency figures that make height the cost.
3. Compute the height of a B-tree over a million keys with 500 keys per node.
4. Do the same for a billion rows.
5. Say what decides the number of keys per node.

The break-it drill

Trigger each and record the exact output or error:

1. A recursive traversal of a 10,000-node chain.
 2. A rotation with the two assignments in the wrong order.
 3. A rotation whose result is not reattached.
 4. Height updates applied in the wrong order after a rotation.
 5. A single rotation applied to a left-right shape.
 6. The naive balance check on a 2,000-node chain, timed.
-

The routine drill

1. Name the six steps in order, then the seventh.
2. Say what the seventh step is for, in one sentence.
3. Do all seven for a system with 5 million DAU and 30 actions a day.
4. Say which step is reasoning rather than arithmetic.

The memory drill

Write from memory, then check:

1. Seconds per day, month and year, with their rounded forms.
2. The three requests-per-day to QPS conversions.
3. The four size conversions.
4. Six typical record sizes.
5. Five machine capacities.
6. Five latency figures.
7. Four standard multipliers.

The rounding drill

1. State the rule for which direction to round, and why it differs for capacity and savings.
2. Say the sentence you would use out loud when rounding 86,400.
3. Give the error percentage and say why it does not matter.
4. Say what magnitude of error *does* matter, and what catches it.

The sanity-check drill

1. Name five reference points from memory.
2. Say which one is the counterweight, and what it counterweighs.
3. For each of these, say whether it is plausible and why: 50 TB/day of photos; 200 QPS for a global product; 40 servers for a modest site.
4. List the six common errors in order of frequency, with the factor each introduces.

The worked-example drill

Do each end to end, out loud, under three minutes:

1. A chat application with a billion daily users.
2. A video service — storage and bandwidth.
3. A URL shortener over five years.

For each, say what the *binding constraint* turned out to be, and whether it was the one you expected.

The consequence drill

From memory, give what each of these means for the design:

1. Peak QPS of 100, 1,000, 10,000, 100,000.
2. Storage of 1 TB, 100 TB, 1 PB.
3. A write rate of 1,000/s, 5,000/s, 10,000/s.

Then say why the third table is the one that ends a discussion.

The misleading drill

For each, say what the naive estimate misses:

1. A celebrity with fifty million followers posts.
 2. A chat app with a million idle users.
 3. A system that has been running for four years.
 4. A page that loads 1.8 MB of images.
 5. An estimate quoted as an average.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is the worst case for a BST, and how do real databases avoid it?* The sorted input demonstrated rather than described, the fifty-thousand-times number, the good news about random insertion, what a rotation is with the inorder property as the justification, AVL against red-black, and then B-trees with the disk-read arithmetic.
 2. *Estimate the storage for five years of this system's data.* The routine announced, every assumption stated as used, actions reasoned rather than guessed, the rounding admitted out loud, a sanity check against something known, and then what the number means for the design.
 3. *Implement a self-balancing tree.* Why not red-black, what a treap is in one sentence, why randomness gives balance regardless of insertion order, and the “expected not worst case” caveat given honestly.
-

Before you move on

- [] I can produce the degenerate tree in three lines and print its height.
- [] I can say why sorted input is normal, not adversarial.
- [] I know the expected height of a randomly built BST.
- [] I define balanced with the word “every” and can draw the counter-example.
- [] I can write `rotate_right` from memory and state the inorder property.
- [] I know what happens if the two assignments are swapped.
- [] I can name the four cases and say why zig-zags need two rotations.

- [] I know in which order heights must be updated after a rotation.
- [] I can fill in the AVL versus red-black table and name who uses which.
- [] I can write a treap and state its guarantee with the word “expected”.
- [] I can compute B-tree heights and give the latency figures that justify them.
- [] I can recite the six-step routine and the seventh step.
- [] I can write the whole memory list without checking.
- [] I round out loud with the direction and the reason.
- [] I sanity-check every estimate against a named reference point.
- [] I can list the six common errors and the factor each introduces.
- [] I have done all three worked examples under a timer.
- [] I can give the consequence tables from memory.
- [] I answered all three questions above out loud.

Building a tree from its traversals

DATA STRUCTURES AND ALGORITHMS

Building a tree from its traversals

You can reconstruct a tree from preorder plus inorder, and say why postorder plus preorder fails.

SYSTEM DESIGN

Capacity planning: QPS, storage, bandwidth

You can turn a product requirement into a server count.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Build the tree from its preorder and inorder traversals.*
- *How many servers does this system need?*

1 What this is, and why they ask it

Given two lists of the same values — the tree read in two different orders — rebuild the tree that produced them.

Three sentences. **One traversal is never enough:** several different trees share the same preorder, so the question always gives you two. The pair works because each list answers a different question — **preorder tells you which node is the root, and inorder tells you where the split between the two subtrees is** — and neither alone answers both. And the pair that does *not* work is preorder plus postorder, because both of them say “root first” or “root last” and neither says where the boundary falls.

They ask it because the idea is three lines and the implementation is bookkeeping, so it separates people who can hold index ranges in their head under pressure. And because the obvious version is $O(n^2)$ — it scans the inorder list to find the root every time — and the fix is a hash map built once, which is the same “stop recomputing what you already know” move as [day 102](#).

2 The story

At the seventieth-birthday function there was a photograph of about forty relatives on a long stage, and the argument afterwards was about who was who.

Half the people in it were dead and the rest were guessing. Somebody’s phone had the picture and nobody could agree on the man third from the left in the back row.

Old Kamalam, who was the only person there over eighty, settled it, and the way she did it was what Sridhar remembered.

She did not look at the photograph much. She said she had two things in her head from that day and either one alone was useless, and the two together were enough.

The first was the order the photographer had called people up. He had done it by family, starting from the eldest: he called a man, and then that man’s people, and then went back and called the next man and his people. So her first list was a chain of names in the order they were shouted.

The second was the order they were standing, left to right, which is what the photograph itself shows.

She said the calling order tells you who is senior — the first name shouted in any group is the head of that group. And the standing order tells you where the group splits, because a man's people stood around him, some to his left and some to his right, and everybody to his left in the picture belongs to one side of his family and everybody to his right belongs to the other.

So she did it like this. The first name in the calling list was Ranganathan — so Ranganathan is the senior man. Then she found Ranganathan in the standing order. Eleven people were standing to his left. So those eleven are his one branch, and everybody after him is the other branch.

And then she said, and this is the part that made it work: those eleven people appear in the calling list too, immediately after Ranganathan — the next eleven names. Because the photographer called a man and then finished his people before moving on.

So she split both lists at the same time. Eleven names from each side, and she did the same thing again inside them.

Sridhar's cousin asked whether she could do it from the calling order alone.

She said no — the calling order tells you the seniors but not where anybody stood, and there are dozens of arrangements that would produce the same shouting. And the standing order alone tells you the arrangement but not who is senior. Either one on its own is nothing. It is the two together.

3 The idea in plain English

Kamalam has just reconstructed a binary tree from preorder and inorder, and her explanation of why one list is useless is the reason the problem gives you two.

- The calling order is **preorder**: root, then left subtree, then right subtree.
- The standing order is **inorder**: left subtree, then root, then right subtree.
- “The first name shouted is the head” is: **preorder [0] is the root.**
- “Everybody to his left in the picture is one branch” is: **the root's position in the inorder list splits it into the left and right subtrees.**
- “Those eleven appear next in the calling list” is the index bookkeeping, and it is the fiddly part.

Why one traversal is not enough

```
      1          1
     /          /
    2          2

preorder: [1, 2] and [1, 2] identical
inorder:  [1, 2] and [2, 1] different ← this is what distinguishes them
postorder: [2, 1] and [2, 1] identical
```

Two different trees, the same preorder. So a single traversal cannot determine a tree, and any solution that claims otherwise is wrong. (The exception: a traversal that also records the `None`s does determine it, which is [serialisation](#).)

What each traversal actually tells you

PREORDER	root, left, right	→ the FIRST element is the root
POSTORDER	left, right, root	→ the LAST element is the root
INORDER	left, root, right	→ finds the BOUNDARY between the subtrees

Inorder is the one that locates the split, and the other two identify the root. That is why every working pair contains inorder, and it is the sentence that answers the follow-up.

The algorithm

1. the root is `preorder[0]`
2. find the root in the inorder list; say it is at position `k`
3. everything before `k` in inorder = the LEFT subtree (`k` elements)
everything after `k` in inorder = the RIGHT subtree
4. in preorder, the next `k` elements are the left subtree,
and everything after them is the right subtree
5. recurse on both halves

Step 4 is the one to say out loud, because it is not obvious and it is what makes the split possible on both lists at once: preorder visits a node then finishes its *entire* left subtree before starting the right, so the left subtree occupies a contiguous run immediately after the root.

Kamalam: “*those eleven appear in the calling list immediately after him.*”

Worked, on the standard example

```
preorder = [3, 9, 20, 15, 7]
inorder  = [9, 3, 15, 20, 7]

root = preorder[0] = 3
find 3 in inorder → position 1
  → left subtree has 1 element:  inorder[0:1] = [9]
  → right subtree has 3 elements: inorder[2:5] = [15, 20, 7]
  → in preorder: left is the next 1 element = [9]
                    right is the rest       = [20, 15, 7]

recurse left: preorder [9],      inorder [9]      → leaf 9
recurse right: preorder [20,15,7], inorder [15,20,7]
  root = 20, found at inorder position 1 (within that slice)
  left = [15], right = [7]

result:
      3
     / \
    9  20
       / \
      15  7
```

The naive version, and why it is $O(n^2)$

```
k = inorder.index(root_value)           # a LINEAR SCAN, at every node
```

`list.index` walks the list, so finding the root costs $O(n)$ and it happens n times.

```
balanced tree:   $O(n \log n)$ 
degenerate tree:  $O(n^2)$ 
n = 10,000, a chain: ~50,000,000 operations
```

The fix is one line: build a dictionary from value to index, once, before starting.

```
position = {value: i for i, value in enumerate(inorder)}
```

Then each lookup is $O(1)$ and the whole thing is $O(n)$. **This requires the values to be unique**, which the problem guarantees and which is worth stating.

The bookkeeping, which is the actual difficulty

Slicing the lists is easy to write and allocates a new list at every node — $O(n^2)$ memory and time again. **Pass index ranges instead.**

```
def build(pre_start, in_start, in_end):
    ...
```

Only three parameters are needed, not four: **the preorder end is implied**, because the size of the subtree is `in_end - in_start`.

And the one line people get wrong:

```
left_size = k - in_start
right_pre_start = pre_start + 1 + left_size
```

`pre_start + 1 + left_size`: skip the root, then skip the entire left subtree. Getting this wrong does not raise — it builds a wrong tree.

The cleanest version avoids the arithmetic entirely by walking preorder with a moving pointer:

```
index = 0                                # a cursor into preorder

def build(in_start, in_end):
    nonlocal index
    if in_start > in_end:
        return None
    root = TreeNode(preorder[index])
    index += 1                            # consume one preorder element
    k = position[root.val]
    root.left = build(in_start, k - 1)    # LEFT FIRST — this is essential
    root.right = build(k + 1, in_end)
    return root
```

The left recursion must come first, because it consumes exactly the preorder elements belonging to the left subtree, leaving the cursor at the start of the right subtree. Swap the two lines and the tree is built from the wrong values with no error at all.

Postorder plus inorder

Same idea, mirrored:

```
the root is postorder[-1] (the LAST element)
walk postorder BACKWARDS, and build the RIGHT subtree FIRST
```

Right first, for exactly the same reason: walking backwards through postorder, the right subtree's elements come immediately before the root.

Why preorder plus postorder fails

```
      1          1
       \        /
        2      2

preorder: [1, 2] and [1, 2]    identical
postorder: [2, 1] and [2, 1]    identical
```

Two different trees, identical in both lists. Neither traversal says where the boundary is: preorder says “root first” and postorder says “root last”, and with a single child there is no way to tell which side it is on.

The exception is worth knowing because interviewers ask it: **if every node has zero or two children — a full binary tree — then preorder plus postorder does determine the tree**, because the ambiguous single-child case cannot occur. LeetCode 889 is exactly that problem.

The BST special case

For a **binary search tree** you only need **one** traversal, because inorder is implied — it is the sorted order of the values.

```
preorder of a BST → sort it to get the inorder → reconstruct
or, better: use the BST range property directly, in O(n)
```

LeetCode 1008 is this, and the good solution does not sort at all: walk the preorder with a permitted range, exactly as in [validation](#), and take elements while they fit.

4 The picture

The split, drawn on both lists at once. **This is the diagram to reproduce.**

```

preorder = [ 3 | 9 | 20 15 7 ]
             ^   ^   ^^^^^^^^^^^
             root | right subtree
                  left subtree (1 element)

```

```

inorder = [ 9 | 3 | 15 20 7 ]
            ^   ^   ^^^^^^^^^^^
            |   root right subtree
            left subtree

```

STEP 1: the root is preorder[0] = 3
 STEP 2: find 3 in inorder → position 1
 STEP 3: inorder splits: 1 element left, 3 elements right
 STEP 4: preorder splits at the SAME SIZES: 1 element then 3
 → because preorder finishes the ENTIRE left subtree
 before starting the right one

both lists are now cut into matching pieces. Recurse.

Kamalam's two lists:

```

CALLING ORDER (preorder)
  Ranganathan, [his 11 people ...], [the rest ...]
  ^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^
  the head     one branch, all      the other branch
                together

```

```

STANDING ORDER (inorder)
  [11 people ...], Ranganathan, [the rest ...]
  ^^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^
  to his left        him              to his right

```

the SAME eleven people, in both lists, in different places.
 Finding him in the standing order tells you the number ELEVEN,
 and that number cuts the calling list too.

Why one list is not enough, and why preorder plus postorder is not either:

	1 \ 2	1 / 2	
preorder	[1, 2]	[1, 2]	SAME
inorder	[1, 2]	[2, 1]	DIFFERENT ← only this distinguishes
postorder	[2, 1]	[2, 1]	SAME
pre + in	→ works	(inorder disagrees)	
post + in	→ works	(inorder disagrees)	
pre + post	→ FAILS	(both agree, and the trees differ)	

the reason in one line: INORDER is the only traversal that says where the BOUNDARY between the two subtrees falls. The other two only say where the root is.

The cursor version, traced:

```
preorder = [3, 9, 20, 15, 7], index = 0
inorder  = [9, 3, 15, 20, 7], position = {9:0, 3:1, 15:2, 20:3, 7:4}

build(0, 4): root = pre[0] = 3, index → 1, k = 1
  build(0, 0): root = pre[1] = 9, index → 2, k = 0
    build(0, -1) → None
    build(1, 0) → None
    returns 9
  build(2, 4): root = pre[2] = 20, index → 3, k = 3
    build(2, 2): root = pre[3] = 15, index → 4 → leaf 15
    build(4, 4): root = pre[4] = 7, index → 5 → leaf 7
    returns 20
  returns 3

the CURSOR only moves forward, and the LEFT call must run first –
it consumes exactly the left subtree's preorder elements, leaving the
cursor at the start of the right subtree.
```

5 The code, built step by step

Step 1 — say why two traversals

“One traversal never determines a tree — a node with only a left child and a node with only a right child have the same preorder. So the question gives two, and they do different jobs: preorder identifies the root, and inorder locates the boundary between the subtrees.”

Step 2 — the recursive idea, before any indices

“The root is the first preorder element. Find it in inorder; everything to its left is the left subtree and everything to its right is the right. And because preorder finishes the entire left subtree before starting the right, the same sizes cut the preorder list too.”

Step 3 — kill the $O(n^2)$ before writing it

```
position = {value: i for i, value in enumerate(inorder)}
```

“Searching inorder for the root with `.index` is a linear scan at every node — $O(n^2)$ on a chain. One dictionary, built once, makes each lookup $O(1)$ and the whole thing $O(n)$. This needs the values to be unique, which the problem guarantees.”

Step 4 — pass indices, not slices

```
def build(in_start, in_end):
```

“Slicing allocates a new list at every node, which puts the $O(n^2)$ back in memory. I pass index ranges instead.”

Step 5 — the cursor, and the ordering rule

```
root = TreeNode(preorder[index])
index += 1
root.left = build(in_start, k - 1)      # LEFT FIRST
root.right = build(k + 1, in_end)
```

“**The left call must come first.**” It consumes exactly the preorder elements of the left subtree, leaving the cursor pointing at the start of the right subtree. Swapping the two lines builds a wrong tree silently.

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

def build_from_pre_in(preorder: list[int], inorder: list[int]) → TreeNode | None:
    """LeetCode 105. Preorder + inorder → the unique tree.

    WHY TWO LISTS: preorder says which node is the ROOT; inorder says where
    the BOUNDARY between the subtrees is. Neither alone answers both, and a
    single traversal never determines a tree.

    THE KEY STEP: preorder finishes the ENTIRE left subtree before starting
    the right, so the size found in inorder cuts the preorder list too.

    O(n) time and O(n) space, thanks to the position map. Without it,
    `inorder.index(...)` makes it O(n^2) on a skewed tree.

    Requires UNIQUE values – say so.
    """
    position = {value: i for i, value in enumerate(inorder)}
    index = 0 # a cursor into preorder

    def build(in_start: int, in_end: int) → TreeNode | None:
        nonlocal index
        if in_start > in_end:
            return None
        root = TreeNode(preorder[index])
        index += 1 # consume one preorder element
        k = position[root.val]
        root.left = build(in_start, k - 1) # LEFT FIRST – essential
        root.right = build(k + 1, in_end)
        return root

    return build(0, len(inorder) - 1)

def build_from_pre_in_explicit(preorder: list[int],
                               inorder: list[int]) → TreeNode | None:
    """The same, with the preorder index computed rather than carried.

    The line people get wrong is `pre_start + 1 + left_size`: skip the root,
    then skip the WHOLE left subtree. Getting it wrong builds a wrong tree
    with no error.
    """
    position = {value: i for i, value in enumerate(inorder)}

    def build(pre_start: int, in_start: int, in_end: int) → TreeNode | None:
        if in_start > in_end:
            return None
        root = TreeNode(preorder[pre_start])
        k = position[root.val]
        left_size = k - in_start
        root.left = build(pre_start + 1, in_start, k - 1)
```

```

        root.right = build(pre_start + 1 + left_size, k + 1, in_end)
        return root

    return build(0, 0, len(inorder) - 1)

def build_from_post_in(inorder: list[int], postorder: list[int]) → TreeNode | None:
    """LeetCode 106. Postorder + inorder. The mirror image.

    The root is the LAST postorder element, so walk postorder BACKWARDS –
    and build the RIGHT subtree FIRST, because going backwards the right
    subtree's elements come immediately before the root.
    """
    position = {value: i for i, value in enumerate(inorder)}
    index = len(postorder) - 1          # a cursor, moving BACKWARDS

    def build(in_start: int, in_end: int) → TreeNode | None:
        nonlocal index
        if in_start > in_end:
            return None
        root = TreeNode(postorder[index])
        index -= 1
        k = position[root.val]
        root.right = build(k + 1, in_end)  # RIGHT FIRST – the mirror of above
        root.left = build(in_start, k - 1)
        return root

    return build(0, len(inorder) - 1)

def build_from_pre_post(preorder: list[int],
                        postorder: list[int]) → TreeNode | None:
    """LeetCode 889. Preorder + postorder – which is AMBIGUOUS in general.

    It works ONLY for a FULL binary tree (every node has 0 or 2 children),
    because the ambiguous single-child case cannot occur. For any other tree
    this returns ONE valid answer among several.

    The trick: preorder[index + 1] is the root of the left subtree; find it
    in postorder to learn the left subtree's size.
    """
    position = {value: i for i, value in enumerate(postorder)}
    index = 0

    def build(post_start: int, post_end: int) → TreeNode | None:
        nonlocal index
        if post_start > post_end:
            return None
        root = TreeNode(preorder[index])
        index += 1
        if post_start == post_end:
            return root          # a leaf
        left_root_pos = position[preorder[index]]
        root.left = build(post_start, left_root_pos)
        root.right = build(left_root_pos + 1, post_end - 1)
        return root

    return build(0, len(postorder) - 1)

def build_bst_from_preorder(preorder: list[int]) → TreeNode | None:
    """LeetCode 1008. For a BST, ONE traversal is enough.

    Inorder is implied – it is the sorted order – so no second list is
    needed. And you do not have to sort: walk the preorder with a permitted
    RANGE, exactly like validation, taking elements while they fit.

```



```

0(n) time, 0(height) space. Each element is consumed exactly once.
"""
index = 0

def build(low: float, high: float) → TreeNode | None:
    nonlocal index
    if index ≥ len(preorder):
        return None
    value = preorder[index]
    if not (low < value < high):
        return None # belongs to an ancestor's other side
    index += 1
    node = TreeNode(value)
    node.left = build(low, value)
    node.right = build(value, high)
    return node

return build(float("-inf"), float("inf"))

def build_naive(preorder: list[int], inorder: list[int]) → TreeNode | None:
    """The O(n^2) version, written out so you can time it.

    `inorder.index(...)` is a linear scan, done once per node, and slicing
    allocates a new pair of lists at every node.
    """
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    k = inorder.index(preorder[0]) # O(n) scan, at EVERY node
    root.left = build_naive(preorder[1:k + 1], inorder[:k])
    root.right = build_naive(preorder[k + 1:], inorder[k + 1:])
    return root

def preorder_of(node: TreeNode | None, out: list[int] | None = None) → list[int]:
    out = [] if out is None else out
    if node:
        out.append(node.val); preorder_of(node.left, out); preorder_of(node.right, out)
    return out

def inorder_of(node: TreeNode | None, out: list[int] | None = None) → list[int]:
    out = [] if out is None else out
    if node:
        inorder_of(node.left, out); out.append(node.val); inorder_of(node.right, out)
    return out

def postorder_of(node: TreeNode | None, out: list[int] | None = None) → list[int]:
    out = [] if out is None else out
    if node:
        postorder_of(node.left, out); postorder_of(node.right, out); out.append(node.val)
    return out

def to_list(root: TreeNode | None) → list[int | None]:
    if root is None:
        return []
    out: list[int | None] = []
    queue: deque[TreeNode | None] = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            out.append(None); continue
        out.append(node.val); queue.append(node.left); queue.append(node.right)
    while out and out[-1] is None:

```

```

        out.pop()
    return out

if __name__ == "__main__":
    pre = [3, 9, 20, 15, 7]
    ino = [9, 3, 15, 20, 7]
    post = [9, 15, 7, 20, 3]

    tree = build_from_pre_in(pre, ino)
    print(to_list(tree)) # [3, 9, 20, None, None, 15, 7]
    print(preorder_of(tree) == pre and inorder_of(tree) == ino) # True

    print(to_list(build_from_pre_in_explicit(pre, ino)) == to_list(tree)) # True
    print(to_list(build_from_post_in(ino, post)) == to_list(tree)) # True

    # WHY ONE TRAVERSAL IS NOT ENOUGH
    left_only = TreeNode(1, TreeNode(2))
    right_only = TreeNode(1, None, TreeNode(2))
    print(preorder_of(left_only), preorder_of(right_only)) # [1,2] [1,2] - same
    print(inorder_of(left_only), inorder_of(right_only)) # [2,1] [1,2] - DIFFER
    print(postorder_of(left_only), postorder_of(right_only)) # [2,1] [2,1] - same

    # so pre + post is ambiguous, and the two trees are indistinguishable
    print(preorder_of(left_only) == preorder_of(right_only)
          and postorder_of(left_only) == postorder_of(right_only)) # True

    # ...but it works for a FULL binary tree
    full = build_from_pre_post([1, 2, 4, 5, 3, 6, 7], [4, 5, 2, 6, 7, 3, 1])
    print(to_list(full)) # [1, 2, 3, 4, 5, 6, 7]

    # a BST needs only ONE traversal
    bst = build_bst_from_preorder([8, 5, 1, 7, 10, 12])
    print(inorder_of(bst)) # [1, 5, 7, 8, 10, 12] - sorted
    print(preorder_of(bst)) # [8, 5, 1, 7, 10, 12] - round trips

    # edge cases
    print(to_list(build_from_pre_in([], []))) # []
    print(to_list(build_from_pre_in([1], [1]))) # [1]

    # a skewed tree - the  $O(n^2)$  case
    n = 2000
    skew_pre = list(range(n))
    skew_in = list(range(n))
    import time
    t = time.perf_counter(); build_from_pre_in(skew_pre, skew_in)
    fast = time.perf_counter() - t
    t = time.perf_counter(); build_naive(skew_pre, skew_in)
    slow = time.perf_counter() - t
    print(f"O(n) {fast:.4f}s vs naive {slow:.4f}s ({slow / fast:.0f}x)")

```

6 What it costs

The two implementations

```
with a position map, index ranges:
    time    0(n)      each node created once, each lookup 0(1)
    space   0(n)      the map, plus 0(height) stack

naive (index + slicing):
    time    0(n^2)    an 0(n) scan and 0(n) slicing, at every node
    space   0(n^2)    new lists allocated at every node
```

```
n = 2,000, a skewed tree
0(n):    ~2,000 operations      instant
naive:   ~4,000,000 + allocations noticeably slow

n = 10,000, skewed
naive:   ~100,000,000           seconds
```

The measurement is in the code above — run it rather than taking the ratio on trust.

Where each cost comes from

```
inorder.index(root)  0(n) per node → 0(n^2) total on a chain
preorder[1:k+1]      0(k) per node → 0(n^2) total, and the same again in memory
position map         0(n) once    → 0(1) per node
index ranges         0(1) per node → no allocation at all
```

Two separate $O(n^2)$ s, and both are removed by the same two changes.

Space, honestly

```
the position map      n entries      ~50 bytes each in Python → ~500 KB at n = 10,000
the recursion stack   0(height)
the output tree        n nodes        unavoidable
```

The map is $O(n)$ extra space that the naive version does not use — so this is a genuine time-for-space trade, and it is overwhelmingly worth it. **Say it as a trade rather than as a free win.**

The BST version

```
preorder of a BST, range-based: 0(n) time, 0(height) space, no map at all
sort-then-reconstruct:          0(n log n) time
```

Half the work and no map, because the ordering supplies the inorder for free. Worth knowing that the sort version exists and is worse.

Recursion depth

```
a skewed tree of 10,000 nodes:  
RecursionError: maximum recursion depth exceeded
```

Same as every tree problem, and this one is particularly exposed because the input that makes it slow — a chain — is the same input that overflows.

7 The traps

Trap 1 — claiming one traversal is enough

```
def build(preorder): ...
```

A node with only a left child and a node with only a right child both give preorder `[1, 2]`. **No single traversal determines a tree**, unless it also records the `None`s.

Trap 2 — recursing right before left with the cursor version

```
root.right = build(k + 1, in_end)    # WRONG ORDER  
root.left = build(in_start, k - 1)
```

The cursor consumes preorder elements in order, so the right call takes the elements that belong to the left subtree. **The tree is built from the wrong values and nothing raises** — the output is a valid tree with the values scrambled.

Trap 3 — `pre_start + left_size` instead of `pre_start + 1 + left_size`

```
root.right = build(pre_start + left_size, k + 1, in_end)    # missing the +1
```

Off by one: the right subtree starts one element too early, so it re-uses the last element of the left subtree. **A wrong tree, no error**. Say the formula in words as you write it: *skip the root, then skip the whole left subtree*.

Trap 4 — the `O(n2)` scan

```
k = inorder.index(preorder[0])
```

Correct, and quadratic on a skewed tree. **One dictionary comprehension removes it**. This is what the interviewer means by “can you do better”.

Trap 5 — slicing instead of passing indices

```
build(preorder[1:k + 1], inorder[:k])
```

Correct, and it allocates two new lists at every node — $O(n^2)$ time *and* memory. Even with the position map, slicing keeps the quadratic behaviour, so **both fixes are needed**.

Trap 6 — assuming values are unique when they might not be

```
position = {value: i for i, value in enumerate(inorder)}
```

With duplicates, the map keeps only the last occurrence and the reconstruction picks the wrong split point. **More fundamentally, duplicates make the answer ambiguous** — there may be several valid trees. LeetCode guarantees uniqueness; **say that you are relying on it**.

Trap 7 — trying preorder plus postorder in general

```
build_from_pre_post([1, 2], [2, 1])
```

Returns a valid tree, and there are two. **It is only unique for a full binary tree**, where every node has zero or two children. Presenting it as a general solution is wrong.

Trap 8 — not validating that the inputs are consistent

If the two lists are not actually traversals of the same tree — different lengths, or a value in one and not the other — the code either raises a `KeyError` from the position map or builds nonsense. In an interview it is enough to **state the assumption**; in real code you would check the lengths and the multiset of values first.

8 In the interview

How it gets asked

- The main one: “Build the tree from its preorder and inorder traversals.” LeetCode 105.
- The mirror: “Now from inorder and postorder.” LeetCode 106.
- The trap question: “Could you do it from preorder and postorder?”
- The optimisation: “Your solution is $O(n^2)$. Can you make it linear?”
- The special case: “What if it is a BST? How many traversals do you need?”

What to say out loud, in the first ninety seconds

1. **Say why there are two lists.** “One traversal never determines a tree — a node with only a left child and one with only a right child have the same preorder. So the two lists do different jobs.”
2. **Say what each one does.** “**Preorder identifies the root** — it is the first element. **Inorder locates the boundary** — the root’s position in it splits the remaining values into the left and right subtrees.”
3. **Give the step that is not obvious.** “And because preorder finishes the *entire* left subtree before starting the right, the size I learn from inorder cuts the preorder list in the same place.”
4. **Kill the quadratic version before writing it.** “The obvious implementation searches inorder for the root at every node, which is $O(n^2)$ on a skewed tree. I build a value-to-index map once, so each lookup is $O(1)$.”
5. **Say you will pass indices.** “And I pass index ranges rather than slicing, because slicing allocates a new pair of lists at every node and puts the quadratic behaviour back.”
6. **Flag the ordering rule.** “I keep a cursor into the preorder list, and the left recursion must happen before the right — it consumes exactly the left subtree’s elements and leaves the cursor at the start of the right.”

The follow-ups

“**Could you do it from preorder and postorder?**” “Not in general, and the counterexample is small: a root with only a left child and a root with only a right child have preorder `[1, 2]` and postorder `[2, 1]` — **both lists identical, two different trees**. The reason is structural rather than incidental: preorder says the root is first, postorder says it is last, and **neither says where the boundary between the subtrees falls**. Inorder is the only traversal that locates that boundary, which is why every working pair contains it. There is one exception worth knowing: if the tree is **full** — every node has zero or two children — then the ambiguous single-child case cannot arise and preorder plus postorder does determine the tree. That is LeetCode 889, and the trick is that the element right after the root in preorder is the left subtree’s root, so finding it in postorder gives you the left subtree’s size.”

“**Your solution is $O(n^2)$. Make it linear.**” “There are two separate quadratic costs and both need fixing. The first is **finding the root in the inorder list** — `inorder.index(...)` is a linear scan done once per node, so $O(n^2)$ on a chain. I build a dictionary from value to index once, before starting, and every lookup becomes $O(1)$. The second is **slicing**: passing `preorder[1:k+1]` allocates a new list at every

node, which is quadratic in time and memory even with the map. So I pass **index ranges** instead and never copy anything. Together those make it $O(n)$ time and $O(n)$ space for the map. The trade to state honestly: the map is $O(n)$ extra memory the naive version does not use, and it is obviously worth it. And the map requires the values to be **unique**, which the problem guarantees.”

“**What if it is a binary search tree?**” “Then **one traversal is enough**, because inorder is implied — it is just the sorted order of the values. So given the preorder of a BST I could sort it to obtain the inorder and run the same algorithm, which is $O(n \log n)$. But there is a better way that does not sort at all: walk the preorder list once with a **permitted range**, exactly like validating a BST. The current element becomes a node only if it falls inside the range; otherwise it belongs to some ancestor’s other side and I return. That consumes each element exactly once, so it is $O(n)$ time and $O(\text{height})$ space with no map. That is LeetCode 1008.”

“**Do it from inorder and postorder.**” “The same idea mirrored. The root is now the **last** postorder element rather than the first preorder one, so I walk postorder **backwards** with a cursor moving down. And critically I build the **right subtree first**, because going backwards through postorder the right subtree’s elements are the ones immediately before the root. Everything else — the position map, the index ranges, the split at the root’s inorder position — is identical. If I built left first, the cursor would hand the right subtree’s values to the left, and the result would be a valid-looking tree with the values in the wrong places and no error.”

“**What breaks if the values are not unique?**” “Two things, and the second is more fundamental. The **implementation** breaks, because the position map keeps only one index per value, so finding ‘the’ root in the inorder list picks an arbitrary occurrence and splits in the wrong place. But even with a perfect implementation, **the answer becomes ambiguous** — with repeated values there can be several distinct trees producing the same pair of traversals, so there is no unique tree to return. That is why the problem guarantees distinct values, and it is worth stating the assumption rather than silently relying on it.”

“**How would you verify your answer?**” “Round-trip it: traverse the tree I just built in preorder and in inorder, and check that both match the inputs exactly. That is $O(n)$ and it catches every class of bug here — the swapped recursion order, the off-by-one in the preorder index, and a wrong split point — because all of them produce a tree whose traversals differ from the inputs. I would write that check first, before the reconstruction, because without it a wrong tree is very hard to spot by eye.”

A model answer

Asked: *build the tree from its preorder and inorder traversals.*

“First, why the question gives me two lists. **One traversal never determines a tree.** A root with only a left child and a root with only a right child both have the preorder `[1, 2]` — same list, different trees. So a single traversal is not enough, and the two I am given do different jobs.

Preorder tells me which node is the root: it is the first element, because preorder visits the root before anything else. **Inorder tells me where the boundary is:** once I know the root, its position in the inorder list splits everything else into the values that are in the left subtree and the values that are in the right.

And then the step that is not obvious, which is what makes both lists usable at once: **preorder finishes the entire left subtree before it starts the right one.** So if inorder tells me the left subtree has eleven values, then in preorder the eleven elements immediately after the root are exactly those eleven, and everything after them is the right subtree. One number cuts both lists.

That is the algorithm: take the root from preorder, find it in inorder, split both lists at the matching sizes, recurse.

The straightforward implementation of that is `O(n2)`, and I would rather say so than have you point it out. Two costs. Searching the inorder list for the root is a linear scan done once per node — quadratic on a skewed tree. And slicing the lists allocates a new pair at every node, which is quadratic again, in memory as well as time. So: I build a **value-to-index dictionary** once at the start, which makes each lookup constant, and I **pass index ranges** rather than slices, so nothing is copied. That is `O(n)` time and `O(n)` space for the map — a real trade, and clearly the right one.

The version I would write keeps a **cursor** into the preorder list rather than computing preorder offsets by arithmetic, because the arithmetic — skip the root, then skip the whole left subtree — is where the off-by-one lives. With a cursor, the rule to be careful about is that the **left recursion must run before the right:** it consumes exactly the left subtree’s preorder elements and leaves the cursor sitting at the start of the right subtree. Swap those two lines and you get a perfectly valid tree built from the wrong values, with nothing raised.

Two things I would add. This assumes the **values are unique** — with duplicates the map is wrong and, more importantly, the answer is genuinely ambiguous. And if you asked whether I could do it from **preorder and postorder** instead, the

answer is no in general: both of those say where the root is and neither says where the boundary falls, so those same two one-child trees are indistinguishable. It works only if the tree is full.”

9 Recall card

- **One traversal NEVER determines a tree** — a left-only and a right-only child both give preorder `[1, 2]`. The two lists do different jobs: **preorder identifies the ROOT (first element)**; **inorder locates the BOUNDARY between the subtrees**.
- **The non-obvious step: preorder finishes the ENTIRE left subtree before starting the right**, so the size learned from inorder cuts the preorder list at the same place. One number splits both lists.
- **Two separate $O(n^2)$ s, and both must be fixed:** `inorder.index()` (a scan per node) → a **value-to-index map built once**; and **SLICING (a new list per node)** → **pass INDEX RANGES**. Together: `O(n)` time, `O(n)` space. Requires **unique values** — say so.
- **With a cursor into preorder, the LEFT recursion must run first** — it consumes exactly the left subtree’s elements. Swapping the lines builds a valid tree from the wrong values, silently. For **postorder + inorder**, mirror it: root is the **last** element, walk **backwards**, and build the **RIGHT** subtree first.
- **Preorder + postorder is AMBIGUOUS** — both say where the root is, neither says where the boundary is — **except for a full binary tree** (every node has 0 or 2 children). And **a BST needs only ONE traversal**, because inorder is implied by the ordering: walk the preorder with a permitted **range**, `O(n)` and no map. Verify any answer by **round-tripping** both traversals.

1 What this is, and why they ask it

Day 109 produced numbers. Today turns them into a **fleet**: how many machines, of what size, with how much room to spare.

Three sentences. The naive answer — peak load divided by what one machine does — is wrong by roughly a factor of two, and the reason is the most useful idea in this lesson: **a queue's waiting time explodes as utilisation approaches one**, so a machine at 90 percent busy is not “90 percent used”, it is unusable. On top of that you need headroom for a **machine failing**, for **growth between now and the next purchase**, and for the fact that **autoscaling takes minutes** while a traffic spike takes seconds. And the three resources — compute, storage, bandwidth — run out at different times, so the binding one is rarely the one you were watching.

They ask “*how many servers?*” because it is the shortest question with a genuinely layered answer. Dividing two numbers takes ten seconds; explaining why you then multiply by 1.5 for utilisation, add one for redundancy, and add fifty percent for growth is what the question is for.

2 The story

Dr Vasanthi's clinic opened at nine and she saw patients until about two, and for eleven years the waiting room had been a problem that everybody blamed on the wrong thing.

She could see a patient in about ten minutes. Six an hour, five hours, so thirty patients in a morning — and the receptionist, sensibly, booked thirty.

The waiting room was chaos. People booked for eleven were being seen at half past twelve. Everybody assumed she was slow, or that patients were arriving late, and both were slightly true and neither was the reason.

Her nephew, who was doing something with statistics at college, sat in the waiting room for three mornings and then explained it to her in a way she remembered.

He said: you can do thirty. So you booked thirty. But patients do not arrive spread out evenly, and they do not all take ten minutes. Some take four and some take twenty-five. When you have booked exactly as many as you can do, **there is no gap**

anywhere in the morning to absorb a long one. One patient who takes twenty-five minutes pushes everybody after him back by fifteen, and nothing in the rest of the morning ever catches that up, because every slot after it is already full.

He said the queue does not grow slowly as you book more. It grows slowly, and slowly, and then at the end it goes vertical.

He gave her the numbers he had counted. At twenty patients — two thirds of what she could do — the average wait was about six minutes. At twenty-four, about twelve. At twenty-seven, about half an hour. At thirty, an hour and a half and getting worse every day.

She dropped it to twenty-four and the waiting room emptied out. She was seeing six fewer people, and she was finishing earlier, and nobody was angry.

Two other things came out of those three mornings.

He pointed out that on the two days a month when her assistant was away, she did everything herself and the whole thing collapsed — so the twenty-four was really only true when both of them were there. If she wanted the clinic to work on the assistant's day off, she had to book for what **one** person could do, not two.

And he said the thing about the following year. The colony behind the clinic was being built out and there would be more people, so whatever number she chose would be wrong within eighteen months, and she should decide now whether she was planning for today or for the day she would have to hire somebody.

3 The idea in plain English

The clinic is a capacity plan, and all three of the nephew's points are the three things people leave out of the server count.

- Ten minutes a patient is **service time**; thirty a morning is **capacity**.
- Booking thirty is planning at **100 percent utilisation**, and the waiting room is the **queue**.
- “It goes vertical at the end” is the fact that **latency explodes near saturation** — the single most useful idea here.
- The assistant's day off is **N + 1 redundancy**: size so that the system still works with one unit gone.
- The new colony is **growth headroom**.

The naive answer, and why it is wrong

```
servers = peak QPS ÷ QPS per server  
         = 6,000 ÷ 1,000  
         = 6
```

Six machines is the answer to a different question: how many machines are needed for the work to be theoretically possible. It is not how many you run.

Three multipliers are missing.

Multiplier one: utilisation, and why latency explodes

The essential fact. For a queue where work arrives unevenly, the average waiting time behaves roughly like:

```
wait ∝ utilisation / (1 - utilisation)
```

That denominator is the whole story. As utilisation approaches 1, the wait approaches infinity.

utilisation	relative queueing delay
-----	-----
50%	1× (the baseline)
70%	2.3×
80%	4×
90%	9×
95%	19×
99%	99×

At 90 percent utilisation, requests wait nine times longer than at 50 percent. Not nine percent — nine times. And the machine is not broken; it is doing exactly what it was asked to do.

So you size for 60 to 70 percent, not 100. That is a multiplier of about 1.5.

```
6 machines at 100% → 9-10 machines at 65%
```

Dr Vasanthi booking twenty-four instead of thirty.

The reason it is not a straight line, in one sentence: **work does not arrive evenly**. If exactly one request arrived every millisecond and each took exactly one millisecond, 100 percent utilisation would be fine. Real arrivals are bursty, and a burst arriving at a busy machine has nowhere to go but the queue.

Multiplier two: redundancy

A machine will fail, be patched, or be restarted for a deployment. **The fleet must still work when one is gone.**

N+1	survives one failure	→ add 1 machine
N+2	survives two	→ add 2
2N	survives a whole zone	→ double it

9 machines at 65% utilisation	
lose one → the remaining 8 go to 73%	still fine
lose two → 7 go to 84%	degraded, and latency is 4x

The smaller the fleet, the more expensive redundancy is, proportionally:

fleet of 3, lose one	→ the survivors take 50% more each
fleet of 30, lose one	→ 3.4% more each

That is a real argument for more, smaller machines rather than fewer, larger ones — **the opposite of the [day 098 vertical-scaling instinct](#)**, and worth being able to reconcile: scale up until the cost curve bends, then out, and past that point smaller units make failure cheaper.

Multiplier three: growth

Capacity you provision today has to last until you next provision. **Ask over what horizon.**

20% growth per year, planning 12 months ahead	→ ×1.2
100% growth per year (a doubling), 12 months	→ ×2
a launch, growth unknown	→ autoscale, and set the ceiling high

On cloud infrastructure this multiplier is much smaller than it used to be, because adding a machine takes minutes rather than a purchase order. Say that: the growth multiplier is a function of how long it takes you to add capacity.

Putting the three together

raw need	peak QPS ÷ per-server QPS	6,000 ÷ 1,000	= 6
× utilisation	÷ 0.65		= 9.2 → 10
+ redundancy	N+1		= 11
× growth	×1.2 for a year		= 13.2 → 14

ANSWER: about 14 machines, and here is the reasoning for each step.

Six to fourteen. The multipliers matter more than the division.

Autoscaling does not remove the problem

Autoscaling changes *when* you buy capacity, not whether you need headroom.

```
spike arrives                t = 0
metric window                +60 s      (you scale on a 1-minute average)
scaling decision + API call  +10 s
instance boots               +60 s
application starts, warms up +60 s
passes health checks, gets traffic +30 s
-----
total                        ~3-4 minutes
```

For three to four minutes, the existing fleet takes the entire spike alone. So the steady-state fleet must survive the spike unaided for that long — which means autoscaling saves money on the *trough*, not on the peak.

This is the most common mistake in a design interview involving autoscaling, and stating it is a strong signal.

The three resources run out at different times

Compute, storage and bandwidth are separate plans, and the binding one is often not the obvious one.

```
COMPUTE    peak QPS ÷ per-server QPS, then the multipliers
            → scales with TRAFFIC, and is elastic

STORAGE     bytes/day × retention × replication
            → scales with TIME, and only ever grows
            → you cannot autoscale it down

BANDWIDTH   QPS × bytes per response
            → often the first to bind, for anything with media
            → and it is metered, so it is a bill rather than a wall
```

Ask which one binds first. For a text-heavy API it is compute; for a photo product it is bandwidth; for a logging system it is storage. Getting that right early changes the whole architecture.

Per-machine numbers, and the honest caveat

```
app server, simple request    1,000 - 5,000 QPS
app server, real work         200 - 1,000 QPS
relational DB, indexed reads  5,000 - 15,000 QPS
relational DB, writes         500 - 5,000 QPS
Redis                        100,000+ ops/s
one 10 Gbit/s network link    ~1 GB/s
one SSD                       ~500 MB/s, ~50,000 IOPS
```

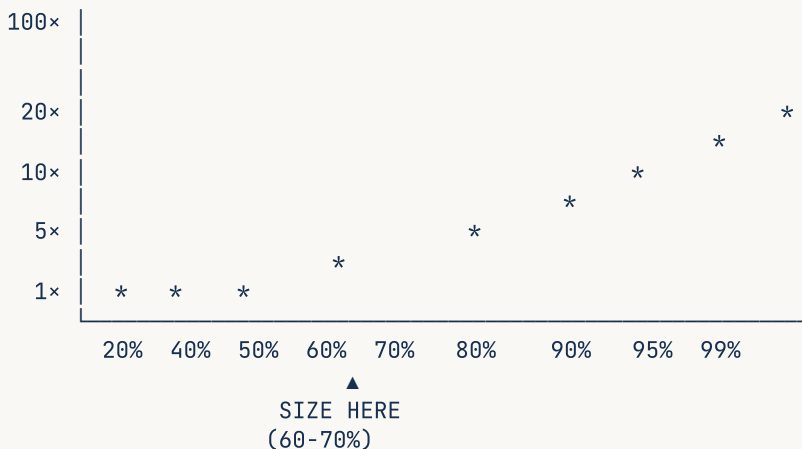
These are order-of-magnitude figures and they depend enormously on what a request does. Say so: “I am assuming about a thousand requests a second per application server, and that depends completely on the work per request — if each one runs a complex query it might be two hundred. I would measure rather than assume, and this is the number I would want from a load test first.”

Naming the number you would go and measure is better than pretending to know it.

4 The picture

The utilisation curve. **This is the diagram that makes the whole lesson.**

queueing delay (relative)

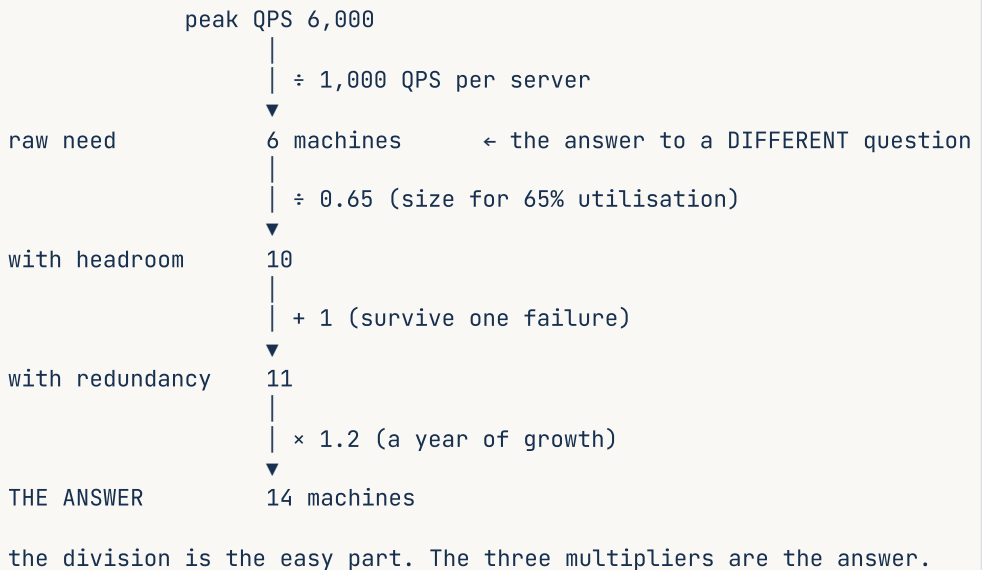


the curve is nearly flat to 60% and then goes vertical.

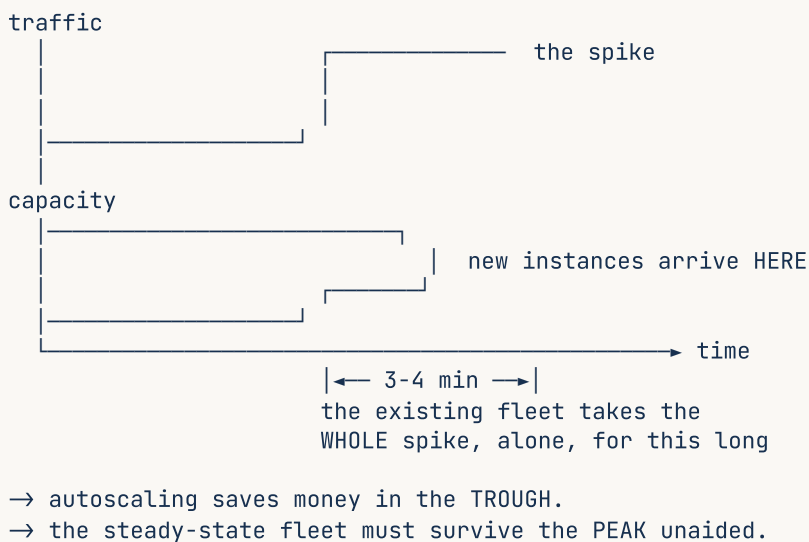
90% utilisation is NOT "90% used" – it is 9x the waiting time.

Dr Vasanthi booking 30 patients when she could do 30.

The three multipliers, stacked:



Why autoscaling does not remove the headroom:



Redundancy is proportionally cheaper in a bigger fleet:

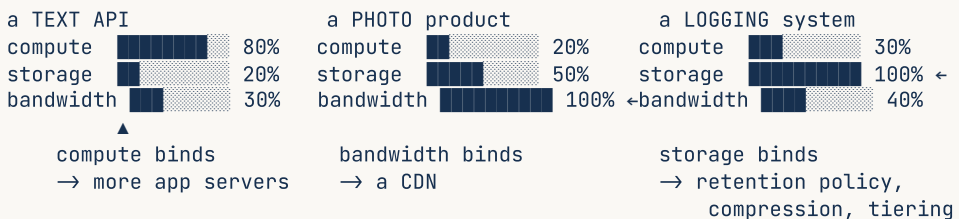
fleet size	lose one	survivors' load increases by
-----	-----	-----
2	1 left	+100% ← catastrophic
3	2 left	+50%
5	4 left	+25%
10	9 left	+11%
30	29 left	+3.4%

many small machines make failure cheap.

few large machines make failure expensive.

(and this pulls AGAINST the vertical-scaling argument from day 098 – scale up until the cost curve bends, then out.)

The three resources, and which binds:



5 How it actually works

The procedure

- | | |
|------------------------------|--|
| 1. peak QPS | from the estimate (day 109), including the peak factor |
| 2. per-server capacity | from a load test if possible; an assumption if not |
| 3. raw machine count | $1 \div 2$ |
| 4. $\div$ target utilisation | 0.6-0.7 |
| 5. + redundancy | N+1, or N+2, or 2N for zone failure |
| 6. $\times$ growth | over your provisioning horizon |
| 7. repeat for storage | bytes/day $\times$ retention $\times$ replication |
| 8. repeat for bandwidth | QPS $\times$ bytes/response |
| 9. say which one binds | and what you would change if it were different |

Steps 4 to 6 are the ones that distinguish an answer from a division.

Choosing the utilisation target

It is not always 65 percent, and knowing when it moves is worth stating:

latency-sensitive, user-facing	50-60%	the queueing curve is the constraint
general web tier	60-70%	the default
batch / async workers	80-90%	a queue in front absorbs the burst, and nobody is waiting
anything with a hard SLA at p99	40-50%	tail latency is what saturation destroys

Batch work is the interesting case: because a queue absorbs arrival variance and nobody is watching a spinner, you can run those machines hot. **That is the single biggest lever for cost**, and it is why moving work off the request path is worth so much.

Sizing storage, which is a different kind of plan

```
bytes per record × records per day × retention days × replication factor
```

5,000,000 writes/day × 1 KB	=	5 GB/day
× 365 (one year retention)	=	1.8 TB
× 3 (replication)	=	5.5 TB
+ 30% for indexes	=	7.2 TB
+ 20% free space headroom	=	8.6 TB

Two additions people forget:

- **Indexes.** A well-indexed table is commonly 1.3 to 2 times the size of its data. Say “plus thirty percent for indexes” and it sounds like you have run a database.
- **Free space.** A disk at 95 percent full is a problem for a database long before it is full — compaction, vacuum and temporary sort space all need room.

And storage cannot be scaled down, so the retention policy is the capacity plan. “Ninety days hot, then archive” is a capacity decision disguised as a product decision.

Sizing bandwidth

```
peak QPS × bytes per response = bytes per second
```

6,000 QPS × 5 KB JSON	=	30 MB/s	one machine's link, easily
6,000 QPS × 500 KB images	=	3 GB/s	THREE machines' worth of network, for the images alone

Which is why images go to a CDN — and the number above is the reason, stated as a number rather than as a principle.

Also worth separating: **egress is metered and charged**, so bandwidth is usually a bill rather than a wall. Compute and storage are walls.

What real practice looks like

- **Load test to find the per-machine number.** Every figure in the table above is a starting assumption; the real one comes from ramping load against one instance until p99 latency degrades, and taking the QPS just below that knee. **The knee is**

the capacity, not the point where it falls over.

- **Autoscaling policies** are usually on CPU utilisation with a target around 50 to 60 percent — which looks low until you remember the queueing curve, and that the metric lags.
- **Google's SRE practice** is to run services at a utilisation that keeps error budgets intact, and to treat headroom as a deliberate, costed decision rather than a leftover.
- **Reserved or committed capacity** is 30 to 60 percent cheaper than on-demand, so the usual pattern is to reserve the steady-state fleet and autoscale the peak on-demand.

6 The numbers

The utilisation multiplier

target utilisation	machines needed for 6,000 QPS at 1,000 QPS each	
-----	-----	-----
100%	6	(and unusable: latency is unbounded)
90%	6.7 → 7	queueing delay 9x
80%	7.5 → 8	queueing delay 4x
70%	8.6 → 9	queueing delay 2.3x
65%	9.2 → 10	the usual target
50%	12	for a hard tail-latency SLA

The cost of going from 90 to 65 percent is three machines. The benefit is a four-fold reduction in queueing delay. Stated that way, it is obviously worth it.

The full worked count

GIVEN: 6,000 peak QPS, ~1,000 QPS per app server

raw	6
÷ 0.65 utilisation	9.2 → 10
+ 1 redundancy (N+1)	11
× 1.2 growth (12 months)	13.2 → 14

and check the failure case:

14 machines, one dies → 13 carry 6,000 QPS → 462 QPS each → 46% utilisation ✓
14 machines, a whole AZ (a third) dies → 9 carry 6,000 → 667 each → 67% ✓

That last check is the part that impresses: sizing is not finished until you have verified the failure case.

Storage, fully loaded

5 GB/day of rows
× 365 days
× 3 replicas
× 1.3 for indexes
÷ 0.8 for free-space headroom

= 1.8 TB
= 5.5 TB
= 7.2 TB
= 9.0 TB

→ "about 9 TB provisioned for 1.8 TB of actual data"
a 5x multiplier, and every step of it is real.

People quote 1.8 TB and provision 2. The 5× is the answer.

Bandwidth, and when it binds

responses	peak QPS	bytes/response	bandwidth	machines' worth of link
-----	-----	-----	-----	-----
JSON API	6,000	5 KB	30 MB/s	0.03
HTML pages	6,000	50 KB	300 MB/s	0.3
thumbnails	6,000	200 KB	1.2 GB/s	1.2
full images	6,000	2 MB	12 GB/s	12
video	1,000	5 Mbit/s each	625 MB/s	0.6 (but continuous)

Anything above about 1 GB/s means the CDN is not optional, and this table says at what point.

Autoscaling lag, and what it costs

reaction time ~3-4 minutes
a spike of 3x, arriving in 10 s

fleet sized at 65% for normal load:
during the spike it is at 195% → requests queue, latency explodes,
health checks start failing, and the autoscaler may even remove instances

fleet sized to absorb a 3x spike unaided:
normal utilisation ~22%, which is expensive

the usual compromise:
size for the LARGEST spike you can predict (evening peak) as steady state,
autoscale for the unpredictable part,
and put a QUEUE in front of anything that can be made asynchronous

Cost, which is what the number is for

14 app servers × ₹12,000/month	=	₹168,000	
reserved instead of on-demand (-40%)	=	₹101,000	
database: 1 primary + 2 replicas	=	₹150,000	
Redis	=	₹25,000	
storage 9 TB block	=	₹72,000	
bandwidth 30 MB/s ≈ 78 TB/month @ ₹7/GB	=	₹546,000	← the biggest line

~₹894,000/month			

Bandwidth being the largest line is the common surprise, and it is the arithmetic that justifies a CDN in a budget meeting rather than in a design review.

7 The trade-offs

Utilisation against cost

Running at 40 percent is safe and costs 60 percent more than running at 65. Running at 90 saves money and gives you nine times the queueing delay and no room for a failure.

Sixty to seventy percent is the default for user-facing work; eighty to ninety is right for batch work behind a queue. The difference is whether a human is waiting.

Few large machines, or many small?

Many small makes failure cheap — losing one of thirty costs 3.4 percent — and makes autoscaling granular. **Few large** is simpler to operate, uses less total overhead, and makes each failure expensive: losing one of three costs the survivors 50 percent each.

And this pulls against day 098's advice to scale up first. The reconciliation: scale up while the price per unit of power is flat, then out — and once you are out, prefer enough units that any single loss is absorbable.

Autoscale, or provision statically?

Autoscaling saves real money on a workload with a large daily trough, and it cannot react to a fast spike — three to four minutes is an eternity for a 10-second surge.

Static provisioning is predictable, and you pay for the peak at three in the morning.

Take both: reserve the steady state, autoscale the predictable daily variation, and put a queue in front of anything that can absorb a spike asynchronously.

Where the plan is wrong

- **Per-machine capacity is a guess until you load-test it.** Everything downstream is proportional to it, so it is the number worth measuring first.
 - **The peak factor is the input most often wrong,** and it is a multiplier on everything.
 - **Scaling the application does not scale the database.** Fourteen app servers with twenty connections each is 280 database connections, which many databases will not accept — so a connection pooler becomes a required component, not an optimisation.
 - **Storage is a ratchet.** It only goes up, it cannot be autoscaled down, and the retention policy is the only real lever.
 - **A dependency you do not control has its own limits.** A third-party API with a 1,000-requests-per-second quota is your capacity ceiling regardless of how many machines you run.
-

8 In the interview

How it gets asked

- The direct one: *“How many servers does this system need?”*
- The probe: *“Why not six? You said each one does a thousand.”*
- The failure question: *“What happens when one of them dies?”*
- The autoscaling question: *“Would you just autoscale?”*
- The binding-constraint question: *“What runs out first?”*

What to say out loud, in the first ninety seconds

1. **Do the division, and immediately say it is not the answer.** “Six thousand peak QPS at about a thousand per server is six machines — but six is the answer to ‘what is theoretically possible’, not ‘what would I run’.”
2. **Give the utilisation reason, with the number.** “I would size for sixty-five percent utilisation, not a hundred, because queueing delay grows as utilisation over one-minus-utilisation. At ninety percent, requests wait about nine times longer than at fifty. So that is ten machines, not six.”
3. **Add redundancy explicitly.** “Then $N + 1$, because a machine will fail or be patched and the fleet has to work without it. Eleven.”

4. **Add growth, and qualify it.** “Then growth over my provisioning horizon — twenty percent for a year, so fourteen. On cloud that multiplier is small, because adding a machine takes minutes.”
5. **Check the failure case.** “And I would verify: with fourteen, losing one leaves thirteen at forty-six percent, and losing a whole availability zone leaves nine at sixty-seven percent. Both fine.”
6. **Say which resource binds.** “That is compute. I would size storage and bandwidth separately, because they run out at different times — and for anything with images, bandwidth binds first.”

The follow-ups

“Why not six? You said each one does a thousand.” “Because a machine at a hundred percent utilisation is not a machine that is fully used — it is a machine with an unbounded queue. Waiting time in a queue grows roughly as utilisation divided by one minus utilisation, so the curve is nearly flat up to about sixty percent and then goes vertical: at eighty percent requests wait four times longer than at fifty, at ninety percent nine times, at ninety-nine percent about a hundred. The machine is not broken and nothing has failed — that is simply what queues do when work arrives unevenly, and real traffic is bursty. So I size for sixty-five percent, which turns six machines into ten. The cost of that is four machines; the benefit is a fourfold reduction in queueing delay and somewhere for a burst to go. For a batch workload behind a queue I would happily run at eighty or ninety, because nobody is waiting and the queue absorbs the variance — that difference is one of the biggest cost levers there is.”

“What happens when one of them dies?” “That is the check I would do before giving you a final number, and it is why I add $N + 1$. With fourteen machines carrying six thousand QPS, losing one leaves thirteen at about four hundred and sixty each, which is forty-six percent utilisation — comfortable. Losing an entire availability zone, which is a third of them, leaves nine carrying six hundred and sixty-seven each, about sixty-seven percent — still inside my target. If the answer to either of those had been over eighty percent, the fleet is too small, because a failure would then push it into the steep part of the queueing curve and the latency damage would be far worse than the lost capacity. It is also worth saying that redundancy is proportionally much cheaper in a large fleet: losing one of thirty costs the survivors three percent each, and losing one of three costs them fifty percent.”

“Would you just autoscale?” “Autoscaling is worth having and it does not remove the need for headroom, because it is slow relative to a spike. The chain is: the metric window is typically a minute, then the scaling decision and API call, then the instance boots, then the application starts and warms up, then it has to pass health checks

before it receives traffic — three to four minutes end to end. A traffic spike arrives in ten seconds. So for those three or four minutes the **existing fleet takes the entire spike alone**, which means the steady-state fleet still has to survive the peak unaided. What autoscaling actually saves is money in the **trough** — at three in the morning — not capacity at the peak. So my pattern would be: reserve the steady-state fleet, which is also thirty to sixty percent cheaper than on-demand, autoscale the predictable daily variation, and put a queue in front of anything that can be processed asynchronously, because a queue is the only thing that genuinely absorbs a spike instantly.”

“**What runs out first?**” “Depends on the product, and I would size all three separately rather than assume. **Compute** scales with traffic and is elastic. **Storage** scales with time and only ever grows — you cannot autoscale it down, so the retention policy is the capacity plan. **Bandwidth** scales with response size, and for anything media-heavy it binds first by a wide margin. Concretely: six thousand requests a second of five-kilobyte JSON is thirty megabytes a second, which is three percent of one machine’s network link. The same six thousand serving two-megabyte images is twelve gigabytes a second — twelve machines’ worth of network for the images alone, which is why that content goes to a CDN. So for a text API compute binds, for a photo product bandwidth binds, and for a logging system storage binds — and the architecture follows from which one it is.”

“**How confident are you in the thousand-QPS-per-server figure?**” “Not very, and I would say so rather than pretend. It is an order-of-magnitude assumption and it depends entirely on what a request actually does — a simple cached lookup might be five thousand a second and something running a complex query might be two hundred. Since every number downstream is proportional to it, it is the first thing I would measure: ramp load against a single instance until p99 latency starts to degrade, and take the QPS just **below that knee**, not the point where it falls over. That knee is the real capacity. I would rather tell you ‘here is the shape of the calculation and here is the one number I would go and measure’ than give you a confident figure built on a guess.”

“**How would you size storage?**” “Bytes per record times records per day times retention times replication — and then two things people leave out. **Indexes**, which typically add thirty percent to a hundred percent on a well-indexed table. And **free-space headroom**, because a database on a disk that is ninety-five percent full is in trouble long before it is full: compaction, vacuum and temporary sort space all need room, so I would provision to about eighty percent target usage. Concretely: five gigabytes a day of rows, times a year, is 1.8 terabytes; times three replicas is 5.5; plus thirty percent for indexes is 7.2; divided by 0.8 for headroom is about nine terabytes provisioned for 1.8 terabytes of actual data. That five-times multiplier is the thing

people miss when they quote the raw figure. And the real lever is **retention** — storage is a ratchet, so ‘ninety days hot then archive’ is a capacity decision wearing a product decision’s clothes.”

A model answer

Asked: *how many servers does this system need?*

“Let me do the division first and then explain why it is not the answer.

Six thousand requests a second at peak, and roughly a thousand a second per application server, gives **six machines**. That is the answer to ‘how many are theoretically required’. It is not what I would run, and there are three multipliers between that and a real number.

Utilisation. A machine at a hundred percent busy is not fully used — it is a machine with an unbounded queue. Waiting time grows roughly as utilisation over one minus utilisation, so the curve is almost flat up to sixty percent and then goes vertical: at eighty percent requests wait four times longer than at fifty, and at ninety percent nine times. That is not a failure; it is what queues do when arrivals are bursty, and real traffic is bursty. So I size for about **sixty-five percent**, which turns six into **ten**.

Redundancy. A machine will fail, or be patched, or be restarted during a deploy, and the fleet has to work without it. $N + 1$ takes me to **eleven**.

Growth. Capacity has to last until I next provision. Twenty percent for a year takes me to about **fourteen**. On cloud infrastructure that multiplier is small, because adding a machine is minutes rather than a purchase order.

Then I would **check the failure case**, because the sizing is not finished until I have. Fourteen machines, one dies: thirteen carry six thousand, so four hundred and sixty each, forty-six percent — fine. A whole availability zone dies, so a third of them: nine carry six thousand, six hundred and sixty-seven each, sixty-seven percent — still inside target. If either had come out above eighty percent I would add machines, because a failure would then push the fleet into the steep part of the queueing curve and the latency damage would be worse than the lost capacity.

Two things I would add unprompted. **Autoscaling does not remove this headroom** — the chain from metric window through boot to passing health checks is three to four minutes, and a spike arrives in seconds, so the existing fleet takes the whole spike alone for that long. Autoscaling saves money in the trough, not capacity at the peak.

And **compute is only one of three plans**. Storage scales with time and only ever grows, so retention is the real lever — and the fully-loaded number is about five times the raw data once you count replication, indexes and free-space headroom. Bandwidth scales with response size, and it is the one that binds first for anything with media: six thousand requests a second of JSON is thirty megabytes a second,

and the same six thousand serving two-megabyte images is twelve gigabytes a second, which is twelve machines' worth of network link. That number is the reason for the CDN.

The one figure I would want to replace with a measurement is the thousand QPS per server. Everything above is proportional to it, and the way to get it is a load test that ramps until p99 degrades — the capacity is the QPS just below that knee, not where it falls over.”

9 Recall card

- **Peak QPS ÷ per-server QPS is NOT the answer** — it is the answer to “what is theoretically possible”. Three multipliers follow: ÷ **utilisation (0.65)** · + **redundancy (N + 1)** · × **growth**. 6 machines becomes **14**.
- **The one idea that matters: queueing delay grows as $u / (1 - u)$** . Flat to ~60%, then vertical: **80% = 4 × the wait**, **90% = 9 ×**, **99% = 99 ×**. So **size for 60–70% for user-facing work, 80–90% for batch behind a queue** — that difference is the biggest cost lever available.
- **Verify the failure case before answering**. 14 machines, one dies → 13 at 46%; a whole AZ dies → 9 at 67%. And **redundancy is proportionally cheaper in a big fleet**: losing 1 of 30 costs the survivors 3.4%, losing 1 of 3 costs them 50%.
- **Autoscaling does not remove headroom**. Metric window + decision + boot + warm-up + health checks = **3–4 minutes**, and a spike arrives in seconds — so **the existing fleet takes the whole spike alone**. Autoscaling saves money in the **trough**; a **queue** is the only thing that absorbs a spike instantly.
- **Three separate plans, and the binding one is rarely the obvious one**. Compute scales with traffic; **storage scales with time and is a ratchet** (retention is the plan — and fully loaded it is ~5 × the raw data after replication, + **30% indexes** and ÷ **0.8 free space**); **bandwidth binds first for media** — 6,000 QPS × 5 KB is 30 MB/s, but × 2 MB is **12 GB/s**, twelve machines' worth of link. And the per-server figure is an assumption: **load-test to the knee where p99 degrades**, and take the QPS just below it.

DSA topic: Building a tree from its traversals **System design topic:** Capacity planning: QPS, storage, bandwidth

Code these, in this order

One rule for the whole set: **round-trip every answer**. Build the tree, then traverse it in both orders and compare against the inputs. Every bug in this lesson produces a valid-looking tree with the values in the wrong places, and the round trip is the only thing that catches it by eye.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Construct Binary Tree from Pre-order and Inorder	LeetCode 105 (Medium)	The split, the position map, and the cursor ordering.
2	Construct Binary Tree from Inorder and Postorder	LeetCode 106 (Medium)	The mirror: last element, backwards cursor, right subtree first.
3	Construct BST from Preorder	LeetCode 1008 (Medium)	Why one traversal suffices, and the range walk that avoids sorting.
4	Construct Binary Tree from Pre-order and Postorder	LeetCode 889 (Medium)	Why this is ambiguous in general, and what makes it work here.

On problem 1, write the naive version first and time it

Use `inorder.index(...)` and slicing. Time it on a 2,000-node chain against the map-and-indices version. Record the ratio and say which of the two changes removed which cost.

On problem 1 again, swap the recursion order

Put `root.right = ...` before `root.left = ...` with the cursor version. Run it on the standard example and record the tree. Say why nothing raised.

On problem 2, get the mirror right by reasoning

Do not copy problem 1. Derive it: which element is the root, which way does the cursor move, which subtree must be built first, and why.

On problem 4, construct the ambiguity

Before solving it, build the two trees with the same preorder and postorder and print both. Then say what extra property the problem guarantees that makes the answer unique.

The why-two drill

1. Build the two trees that share a preorder and print all three traversals of each.
2. Say which traversal distinguishes them and which two do not.
3. State what each of the three traversals tells you, in one line each.
4. Say why every working pair contains inorder.

The split drill

1. Write out the two lists for the standard example and draw the split.
2. Say why the size learned from inorder also cuts the preorder list.
3. Do the split by hand for a seven-node tree, two levels deep.
4. Say what would go wrong if preorder interleaved the two subtrees.

The complexity drill

1. Name the two separate $O(n^2)$ costs and where each comes from.
2. Say which change fixes each one.
3. Give the operation counts at $n = 2,000$ and $n = 10,000$ for a chain.
4. State the space cost of the fix and say why it is worth it.
5. Say what assumption the position map depends on.

The cursor drill

1. Write the cursor version from memory.
2. Say why the left recursion must run first.
3. Trace the cursor's value at every call for the standard example.
4. Write the explicit-index version and say the formula for the right subtree's start, in words.

The mirror drill

1. Write the postorder-plus-inorder version.
2. State the three differences from the preorder version.
3. Say what happens if you build the left subtree first.

The ambiguity drill

1. State why preorder plus postorder fails, in one sentence about boundaries.
2. Give the two-node counter-example.
3. State the condition under which it does work.
4. Say how the algorithm finds the left subtree's size in that case.

The BST drill

1. Say why one traversal is enough for a BST.
2. Give the $O(n \log n)$ approach and the $O(n)$ approach.
3. Write the range-based version.
4. Say what it has in common with day 108's validation.

The break-it drill

Trigger each and record the exact output:

1. The recursion order swapped in the cursor version.
 2. `pre_start + left_size` without the `+ 1`.
 3. Duplicated values in the inputs.
 4. Slicing instead of index ranges, on a 5,000-node chain, timed.
 5. Preorder plus postorder on a non-full tree — print both valid answers.
 6. A reconstruction of a 10,000-node chain.
-

The naive-answer drill

1. Do the division for 6,000 peak QPS at 1,000 per server.
2. Say why that number is the answer to a different question.
3. Name all three multipliers and apply them in order.
4. Give the final number and the reasoning for each step.

The queueing drill

1. Write the relationship between utilisation and waiting time.
2. Fill in the delay multiples for 50, 70, 80, 90, 95 and 99 percent.
3. Say why the curve is not a straight line, in one sentence about arrivals.
4. Give the utilisation target for user-facing work, for batch work, and for a hard p99 SLA.
5. Say why batch work can run hot, and why that is the biggest cost lever.

The redundancy drill

1. State what $N + 1$, $N + 2$ and $2N$ mean.
2. For a fleet of 14 at 65 percent, compute the utilisation after losing one machine and after losing a third.
3. Fill in the table of fleet size against the survivors' load increase.
4. Say how this argument pulls against the day 098 vertical-scaling advice, and reconcile them.

The autoscaling drill

1. List the five stages of autoscaling lag with their durations.
2. Say what the fleet must therefore be able to do unaided.
3. Say what autoscaling actually saves, and where.
4. Give the three-part pattern you would recommend instead of pure autoscaling.

The three-resources drill

1. Say what each of compute, storage and bandwidth scales with.
2. Say which one cannot be scaled down and what the only lever is.
3. For a text API, a photo product and a logging system, say which binds.
4. Compute bandwidth for 6,000 QPS at 5 KB, 200 KB and 2 MB per response.
5. Say at what bandwidth figure a CDN stops being optional.

The storage drill

Compute the fully-loaded figure, showing every step:

1. 5 GB/day of rows, one year retention, three replicas.
2. Add indexes.
3. Add free-space headroom.
4. State the total multiplier over the raw data, and name the two steps people omit.
5. Say what the real lever is and why.

The measurement drill

1. Say how you would obtain the per-server QPS figure.
2. Define the knee and say why capacity is below it rather than at the failure point.
3. Say why this number matters more than any other in the plan.
4. Say what you would tell an interviewer instead of quoting a confident figure.

The failure drill

For each, say what happens and what you would add:

1. The fleet is sized at 90 percent utilisation and one machine fails.
2. A $3\times$ spike arrives in ten seconds.
3. Fourteen app servers each open twenty database connections.
4. The disk reaches 95 percent full.
5. A third-party API has a 1,000 requests per second quota.
6. The peak factor used in the estimate was 2 and the real one is 6.

Two of the six are not fixed by adding machines. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Build the tree from its preorder and inorder traversals.* Why one traversal is never enough with the counter-example, what each list contributes, the non-obvious step about preorder finishing the left subtree, both $O(n^2)$ costs killed before writing, and the cursor-ordering rule.
 2. *How many servers does this system need?* The division done and immediately disowned, the queueing curve with real multiples, $N+1$ with the failure case verified, growth qualified by provisioning speed, the autoscaling-lag caveat, and which of the three resources binds.
 3. *Could you build the tree from preorder and postorder?* No in general, the two-node counter-example, the reason stated in terms of boundaries, and the full-tree exception with how the size is found.
-

Before you move on

- [] I can build the two trees that share a preorder and show which traversal separates them.
- [] I can say what each of the three traversals contributes.
- [] I can explain why the size from inorder also cuts the preorder list.
- [] I can name both $O(n^2)$ costs and the change that fixes each.
- [] I know the position map requires unique values, and I say so.

- [] I can write the cursor version and say why left must come first.
- [] I can state the explicit-index formula in words.
- [] I can derive the postorder version rather than copying it.
- [] I can give the preorder-plus-postorder counter-example and the exception.
- [] I can write the BST range version and say why no second list is needed.
- [] I round-trip every reconstruction before trusting it.
- [] I never give peak-QPS-over-per-server as a final answer.
- [] I can write the queueing relationship and the delay multiples.
- [] I can say why batch work can run at 90 percent and user-facing work cannot.
- [] I verify the failure case before quoting a fleet size.
- [] I can list the five stages of autoscaling lag and say what it really saves.
- [] I can say what each of the three resources scales with, and which is a ratchet.
- [] I can produce the fully-loaded storage figure and name the two omitted steps.
- [] I answered all three questions above out loud.

Serialising and deserialising a tree

DATA STRUCTURES AND ALGORITHMS

Serialising and deserialising a tree

You can turn a tree into a string and back, preserving structure including nulls.

SYSTEM DESIGN

Single points of failure

You can look at an architecture diagram and circle every box that kills the system.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Serialise this tree, then rebuild it from the string.*
- *Which component here takes the whole system down if it fails?*

1 What this is, and why they ask it

Serialising means turning a structure in memory into a flat sequence of characters — something you could send over a network or write to a file. **Deserialising** is rebuilding the original from that sequence.

Three sentences. [Yesterday](#) established that one traversal does not determine a tree, and today is the exception that proves it: **a single traversal *does* determine a tree if it records where the missing children are.** The whole problem is therefore about what you write down for “nothing”, and getting that right makes the reconstruction a straight walk with no index arithmetic at all. And there is a second decision — the **delimiter** — which is invisible until the values are more than one digit long, at which point it silently corrupts everything.

They ask it because it is a design question dressed as an implementation question. There is no clever algorithm; there are four choices — traversal order, null marker, delimiter, and how you consume the string on the way back — and a candidate who makes those four choices deliberately, out loud, is doing exactly what the question is for.

2 The story

Ponnusamy was eighty-four and his brother’s grandson was in Canada and had asked, twice, for the family tree.

He could not write much any more, so he did it as a voice recording on his phone, and it took him three attempts across two weeks.

The first attempt was useless and he knew it before he finished. He had gone across: all four of his grandfather’s sons first, then all of their children, then all of theirs. The boy in Canada wrote back saying he had ninety names and no idea who belonged to whom.

The second attempt went down instead of across, which was better. He named a man, then that man’s eldest son, then that son’s eldest, all the way to the bottom, then came back up and did the next one. It felt right while he was recording it.

It was still wrong, and the reason took him a while to see.

The boy wrote back with the specific problem. He said: you say “Ranganathan, then Muthu, then Selvi, then Anand” — and I do not know whether Selvi is Muthu’s daughter or Ranganathan’s second child. I cannot hear you climbing back up. When you finish a line and go back to do the next one, nothing in your voice tells me.

So the third attempt had one extra thing in it, and that is the one that worked.

Whenever a man had no more children to name, Ponnusamy said “**no one**” — out loud, deliberately, twice if there had been two possible places. He felt ridiculous doing it. There were long stretches of the recording that were just his voice saying “no one, no one” into the phone.

But it meant the boy could follow him. Every time he heard “no one”, he knew a line had ended and the old man was about to climb back up. He could rebuild the whole thing on the first pass, top to bottom, without ever going back to check anything.

There was one other correction between the second and third attempt, and it was small and it mattered.

In the second recording Ponnusamy had run names together — “Ranganathan Muthu Selvi” — because that is how people talk. The boy misheard “Ram Kumar” as one name in one place and two names in another. So on the third attempt the old man paused firmly between every single name, and said so at the start of the recording: *I am going to stop after each one, so you know where each name ends.*

The recording is eleven minutes long and about a third of it is the word “no one” and the pauses.

3 The idea in plain English

Ponnusamy’s third recording is a preorder serialisation with null markers and a delimiter, and both of his corrections are the two decisions the problem is really about.

- Going across was **level order**; going down was **preorder**.
- “No one” is the **null marker**, and it is what lets the reader know when to climb back up.
- Pausing between names is the **delimiter**.
- The boy rebuilding it in one pass, without going back, is what makes deserialisation $O(n)$ with no index arithmetic.

Why the markers are the whole problem

```
      1           1
       \         /
        2       2

preorder without markers:  "1,2"    and    "1,2"    IDENTICAL
preorder with markers:    "1,#,2,#,#" and "1,2,#,#,#" distinct ✓
```

One traversal plus the nulls determines the tree completely. That is the exception to [yesterday's](#) rule, and it is worth stating explicitly, because the two facts sound contradictory until you see why: without markers you cannot tell a missing left child from a missing right one, and the marker is exactly that information.

The count is worth knowing: **a tree with n nodes has $n + 1$ null positions**, so the serialised form has $2n + 1$ tokens. Every node has two child slots, $n - 1$ of them are filled by other nodes, so $2n - (n - 1) = n + 1$ are empty.

Preorder, and why it makes deserialisation trivial

```
def serialise(node):
    if node is None:
        return "#"
    return f"{node.val},{serialise(node.left)},{serialise(node.right)}"
```

```
def deserialise(tokens):
    value = next(tokens)
    if value == "#":
        return None
    node = TreeNode(int(value))
    node.left = deserialise(tokens)
    node.right = deserialise(tokens)
    return node
```

tokens is an ITERATOR

Six lines each, and no index arithmetic anywhere. That is the reason to choose pre-order.

The mechanism is worth saying precisely: **the reader consumes tokens in exactly the order the writer produced them**, so an iterator (or a single moving index) is all the bookkeeping required. The recursion itself remembers where it was — the boy following the recording without notes.

Use an iterator, not a list with `pop(0)`. `list.pop(0)` shifts every remaining element, turning an $O(n)$ deserialisation into $O(n^2)$. `iter()` plus `next()` is $O(1)$ per token.

The delimiter, which is invisible until it is not

```
without a delimiter:  "12##2##"
                      is that the value 12, or a 1 and a 2?

with a delimiter:    "12,##,2,##"
                      unambiguous
```

Any value longer than one character makes this a real bug, and so does any negative value: `-1` without a delimiter is indistinguishable from a `-` and a `1`.

Two rules:

- **Pick a delimiter that cannot appear in a value.** A comma is fine for integers. If values were arbitrary strings, you would need escaping or a length prefix.
- **Pick a null marker that cannot be a value.** `#` or `null` is safe; `-1` is not, because `-1` might be in the tree. This is the same class of mistake as using `INT_MIN` as a sentinel on [day 108](#).

Level order: the other correct answer

LeetCode displays trees in level order with nulls, and serialising that way is equally valid.

```
      1
     /\
    2  3
     /\  \
    4  5

preorder:  1,2,##,3,4,##,5,##
level order: 1,2,3,##,4,5
```

Level order is often shorter, because trailing nulls can be trimmed and because a wide shallow tree has its nulls clustered at the end. **Preorder is shorter for a deep narrow tree**. Neither dominates.

The trade to state:

PREORDER	recursion, an iterator, six lines each way deserialisation needs no queue and no index arithmetic
LEVEL ORDER	a queue both ways, and the format humans and LeetCode use deserialisation must pair each node with its two waiting values

Write preorder; mention that level order is what the platform displays.

The BST special case: no markers at all

For a binary search tree you can serialise the preorder **without any nulls** and still rebuild it uniquely, because the ordering supplies the missing information.

```
preorder of a BST: 8,5,1,7,10,12          no markers, no delimiter problems
rebuild: walk the list with a permitted RANGE, exactly as on day 110
```

```
general tree: 2n + 1 tokens
BST:          n tokens
→ roughly HALF the size
```

That is a real compression and a good answer to “can you make it smaller?” It is LeetCode 449, and the reason it works is that a BST’s inorder is implied by sorting, so one traversal is already two.

Making it genuinely compact

If asked for a compact format rather than a readable one:

- | | |
|---|-------------------------------|
| 1. use the BST trick where applicable | ~50% saving |
| 2. binary encoding: 4 bytes per int, 1 bit per null | vs ~7 bytes per token as text |
| 3. structure and values as separate streams | nulls compress extremely well |
| 4. succinct encoding: 2 bits per node for the shape | $2n + o(n)$ BITS total |

Point 4 is the theoretical answer and worth naming: the number of distinct binary trees with n nodes is the Catalan number, which is about 4^n , so $2n$ bits is information-theoretically necessary and sufficient for the shape. **You would not implement it;** naming it shows you know where the floor is.

The design decisions, as a list

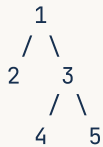
This is what the question is actually asking you to do:

- | | |
|--------------------|---|
| 1. TRAVERSAL ORDER | preorder (simple recursion) or level order (platform format) |
| 2. NULL MARKER | a token that cannot be a value – “#”, not “-1” |
| 3. DELIMITER | a character that cannot appear in a value – “,” for integers |
| 4. CONSUMPTION | an iterator or a moving index, NEVER <code>list.pop(0)</code> |
| 5. VALIDATION | what do you do with malformed input? |

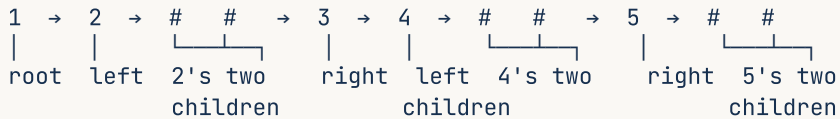
Say all five out loud as choices, with the reason for each. That is the performance.

4 The picture

The same tree, both formats, with the nulls visible.



PREORDER with markers, read as the recursion runs:



"1,2,#,#,3,4,#,#,5,#,#"

11 tokens = $2n + 1$, with $n = 5$

LEVEL ORDER with markers:

level 0: 1
 level 1: 2 3
 level 2: # # 4 5 (2's two missing children, then 3's two)

"1,2,3,#,#,4,5"

7 tokens after trimming trailing nulls

Why the markers are load-bearing:



WITHOUT markers

preorder: 1,2 preorder: 1,2 ← identical, two different trees

WITH markers

preorder: 1,#,2,#,# preorder: 1,2,#,#,# ← distinct

^
 "no left child"

^
 "a left child follows"

the marker is EXACTLY the information that distinguishes them.

A tree of n nodes has $n+1$ empty slots, so the string has $2n+1$ tokens.

Deserialisation as a single forward walk — no arithmetic, no going back:

```

tokens: 1 , 2 , # , # , 3 , 4 , # , # , 5 , # , #
      |
next() → 1   make node 1
              build 1.left:
next() → 2   make node 2
              build 2.left:
next() → #   None                                ← "no one"
              build 2.right:
next() → #   None                                ← "no one"
              return 2
              build 1.right:
next() → 3   make node 3
              build 3.left:
next() → 4   make node 4
next() → #   None
next() → #   None
              return 4
              build 3.right:
next() → 5   make node 5
next() → #   None
next() → #   None
              return 5
              return 3
              return 1

```

the CURSOR only ever moves forward. The recursion remembers the position. Ponnusamy's nephew, writing it down on the first pass.

The delimiter problem:

```

tree containing 12 and 2

WITHOUT a delimiter: "12##2##"
  read one character at a time: 1, 2, #, #, 2, #, # → a THREE-node tree, wrong
  read greedily:                12, #, #, 2, #, # → right, but how would
                                          you know where to stop?

WITH a delimiter:      "12,#,#,2,#,#" → unambiguous

and negatives make it worse: "-1" without a delimiter is a "-" and a "1".

```

5 The code, built step by step

Step 1 — announce the four decisions

“There is no clever algorithm here — there are four choices. Traversal order, the null marker, the delimiter, and how I consume the string coming back. Let me make each one deliberately.”

That framing is most of the answer.

Step 2 — choose preorder, and say why

“Preorder, because the reader can consume the tokens in exactly the order the writer produced them — so deserialisation is a single forward walk with no index arithmetic. Level order also works and is what the platform displays, but it needs a queue on both sides.”

Step 3 — the markers, and the count

```
if node is None:  
    return "#"
```

“Every missing child gets an explicit marker. That is what makes one traversal sufficient — without it, a node with only a left child and one with only a right child produce the same string. A tree of n nodes has $n + 1$ empty slots, so the output is $2n + 1$ tokens.”

Step 4 — the delimiter, with the reason

“A comma between every token. It is invisible with single-digit values and it is a real bug the moment a value has two digits or a minus sign — `12` and `-1` are both unreadable without it.”

Step 5 — an iterator, never `pop(0)`

```
tokens = iter(data.split(","))
```

“I deserialise from an iterator. Using a list and `pop(0)` shifts every remaining element on each call, which turns an $O(n)$ reconstruction into $O(n^2)$ — correct output, and a hundred times slower on a hundred-thousand-node tree.”

The complete solution

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

class Codec:
    """LeetCode 297. Preorder with explicit null markers.

    THE FOUR DECISIONS:
    1. traversal    preorder – the reader consumes tokens in the order the
                       writer produced them, so no index arithmetic is needed
    2. null marker  "#" – a token that CANNOT be a value. Not -1.
    3. delimiter    ",", – a character that cannot appear in a value
    4. consumption an ITERATOR, never list.pop(0), which is O(n) per call

    A tree of n nodes has n+1 empty child slots, so the output is 2n+1 tokens.
    O(n) time and O(n) space both ways.
    """

    NULL = "#"
    SEP = ","

    def serialise(self, root: TreeNode | None) → str:
        parts: list[str] = []

        def walk(node: TreeNode | None) → None:
            if node is None:
                parts.append(self.NULL)      # THE marker – without it, ambiguous
                return
            parts.append(str(node.val))
            walk(node.left)
            walk(node.right)

        walk(root)
        return self.SEP.join(parts)          # join once: O(n), not O(n^2)

    def deserialise(self, data: str) → TreeNode | None:
        tokens = iter(data.split(self.SEP))  # an ITERATOR, not a list

        def build() → TreeNode | None:
            value = next(tokens)
            if value == self.NULL:
                return None
            node = TreeNode(int(value))
            node.left = build()              # consumes exactly the left subtree
            node.right = build()
            return node

        return build()

class LevelOrderCodec:
    """The other correct answer, and the format LeetCode displays.
```

Needs a QUEUE on both sides. Often shorter for a wide shallow tree (trailing nulls can be trimmed); longer for a deep narrow one.

```
NULL = "#"
SEP = ","
```

```
def serialise(self, root: TreeNode | None) → str:
    if root is None:
        return ""
    parts: list[str] = []
    queue: deque[TreeNode | None] = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            parts.append(self.NULL)
            continue
        parts.append(str(node.val))
        queue.append(node.left)
        queue.append(node.right)
    while parts and parts[-1] == self.NULL:    # trim trailing nulls
        parts.pop()
    return self.SEP.join(parts)
```

```
def deserialise(self, data: str) → TreeNode | None:
    if not data:
        return None
    values = data.split(self.SEP)
    root = TreeNode(int(values[0]))
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] != self.NULL:
            node.left = TreeNode(int(values[i]))
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] != self.NULL:
            node.right = TreeNode(int(values[i]))
            queue.append(node.right)
        i += 1
    return root
```

```
class BSTCodec:
```

```
    """LeetCode 449. A binary SEARCH tree needs NO null markers at all.
```

The ordering supplies the missing information: walk the preorder with a permitted RANGE and take elements while they fit – exactly the day 108 validation idea, and the day 110 reconstruction.

```
n tokens instead of 2n+1: roughly HALF the size. That is the answer to
"can you make it more compact?"
"""
```

```
def serialise(self, root: TreeNode | None) → str:
    parts: list[str] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return    # NO marker
        parts.append(str(node.val))
        walk(node.left)
        walk(node.right)
```

```

        walk(root)
        return ",".join(parts)

def deserialise(self, data: str) → TreeNode | None:
    if not data:
        return None
    values = [int(v) for v in data.split(",")]
    index = 0

    def build(low: float, high: float) → TreeNode | None:
        nonlocal index
        if index ≥ len(values):
            return None
        value = values[index]
        if not (low < value < high):
            return None # belongs to an ancestor's other side
        index += 1
        node = TreeNode(value)
        node.left = build(low, value)
        node.right = build(value, high)
        return node

    return build(float("-inf"), float("inf"))

class IterativeCodec:
    """Preorder serialisation without recursion, for very deep trees.

    A recursive codec on a 10,000-node chain raises RecursionError, and a
    chain is exactly what a BST built from sorted data looks like.
    """

    NULL = "#"

    def serialise(self, root: TreeNode | None) → str:
        parts: list[str] = []
        stack: list[TreeNode | None] = [root]
        while stack:
            node = stack.pop()
            if node is None:
                parts.append(self.NULL)
                continue
            parts.append(str(node.val))
            stack.append(node.right) # RIGHT first: a stack reverses
            stack.append(node.left)
        return ",".join(parts)

    def deserialise(self, data: str) → TreeNode | None:
        tokens = data.split(",")
        if not tokens or tokens[0] == self.NULL:
            return None
        root = TreeNode(int(tokens[0]))
        # stack of (node, which_child_is_next)
        stack: list[list] = [[root, "left"]]
        i = 1
        while stack and i < len(tokens):
            node, side = stack[-1]
            token = tokens[i]
            i += 1
            child = None if token == self.NULL else TreeNode(int(token))
            if side == "left":
                node.left = child
                stack[-1][1] = "right"
            else:
                node.right = child
                stack.pop()

```

```

        if child is not None:
            stack.append([child, "left"])
        return root

def serialise_broken_no_markers(node: TreeNode | None) → str:
    """The trap, written out: no markers, so different trees collide."""
    if node is None:
        return ""
    parts = [str(node.val)]
    left = serialise_broken_no_markers(node.left)
    right = serialise_broken_no_markers(node.right)
    return ",".join(p for p in (parts[0], left, right) if p)

def serialise_slow(node: TreeNode | None) → str:
    """The other trap: building the string by concatenation.

    Each += copies the whole string, so this is  $O(n^2)$  in copying. Collect
    into a list and join ONCE.
    """
    if node is None:
        return "#,"
    return str(node.val) + "," + serialise_slow(node.left) + serialise_slow(node.right)

def count_nodes(node: TreeNode | None) → int:
    return 0 if node is None else 1 + count_nodes(node.left) + count_nodes(node.right)

def same(a: TreeNode | None, b: TreeNode | None) → bool:
    if a is None and b is None:
        return True
    if a is None or b is None:
        return False
    return a.val == b.val and same(a.left, b.left) and same(a.right, b.right)

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

if __name__ == "__main__":
    codec = Codec()
    tree = from_list([1, 2, 3, None, None, 4, 5])

    s = codec.serialise(tree)
    print(s)
    print(len(s.split(",")), 2 * count_nodes(tree) + 1)
    print(same(codec.deserialise(s), tree))

    # WHY THE MARKERS MATTER
    left_only = TreeNode(1, TreeNode(2))

```

```

right_only = TreeNode(1, None, TreeNode(2))
print(codec.serialize(left_only))           # 1,2,##,##
print(codec.serialize(right_only))          # 1,##,2,##,##
print(serialize_broken_no_markers(left_only)) # 1,2
print(serialize_broken_no_markers(right_only)) # 1,2 ← COLLIDES

# level order
lo = LevelOrderCodec()
print(lo.serialize(tree))                   # 1,2,3,##,##,4,5
print(same(lo.deserialize(lo.serialize(tree)), tree)) # True

# the BST version needs no markers and is half the size
bst = from_list([8, 5, 10, 1, 7, None, 12])
bc = BSTCodec()
print(bc.serialize(bst))                   # 8,5,1,7,10,12
print(codec.serialize(bst))                 # 8,5,1,##,##,7,##,##,10,##,12,##,##
print(len(bc.serialize(bst).split(",")),
      len(codec.serialize(bst).split(","))) # 6 13
print(same(bc.deserialize(bc.serialize(bst)), bst)) # True

# iterative, for deep trees
it = IterativeCodec()
print(same(it.deserialize(it.serialize(tree)), tree)) # True

chain = None
for v in range(2000, 0, -1):
    chain = TreeNode(v, None, chain)
print(len(it.serialize(chain).split(","))) # 4001 = 2n+1
print(same(it.deserialize(it.serialize(chain)), chain)) # True

# edge cases
print(repr(codec.serialize(None)))          # ''
print(codec.deserialize("#"))              # None
print(codec.serialize(TreeNode(-1)))        # -1,##,##
print(same(codec.deserialize("-1,##,##"), TreeNode(-1))) # True

# negative values and multi-digit values need the delimiter
tricky = from_list([12, -1, 250])
print(codec.serialize(tricky))              # 12,-1,##,##,250,##,##
print(same(codec.deserialize(codec.serialize(tricky)), tricky)) # True

```

6 What it costs

Both directions

serialize	$O(n)$ time – each node visited once $O(n)$ space for the output, plus $O(\text{height})$ stack
deserialize	$O(n)$ time – each token consumed once $O(n)$ space for the token list, plus $O(\text{height})$ stack

Both linear, and there is nothing better: the output must contain every value, so $O(n)$ is the floor.

Size

```
general tree, preorder with markers: 2n + 1 tokens
BST, no markers:                      n tokens
```

```
n = 1,000,000 integers averaging 6 characters plus a comma:
general: 2,000,001 tokens ≈ 10 MB of text
BST:     1,000,000 tokens ≈ 7 MB
binary (4 bytes per int + 1 bit per null): ~4.3 MB
succinct (2 bits per node for the shape + values): ~4 MB
```

The BST trick roughly halves it, and that is the practical answer to “make it smaller”. The succinct bound — $2n$ bits for the shape — is the theoretical floor, because there are about 4^n distinct binary trees with n nodes.

The two $O(n^2)$ traps

```
string concatenation instead of join:
  each += copies the whole string so far
  n = 100,000 tokens of ~7 chars → ~35 GB of copying
  with join:                      ~700 KB, once

list.pop(0) instead of an iterator:
  each pop shifts every remaining element
  n = 100,000 tokens → ~5,000,000,000 element moves
  with an iterator:      100,000
```

Both are correct and both are catastrophically slow, and neither raises anything. They are the two performance bugs this problem is designed to surface.

Recursion depth

```
a chain of 10,000 nodes:
  recursive serialise  RecursionError: maximum recursion depth exceeded
  iterative serialise  fine
```

And a chain is exactly what a BST built from sorted data looks like, so this is not an exotic input. The iterative version is in the code above.

Preorder against level order, by size

shape	preorder tokens	level-order tokens (trimmed)
-----	-----	-----
perfect, n = 7	15	7
chain of 7 (all right)	15	13
the example above	11	7

Level order wins on wide shallow trees; preorder wins on deep narrow ones, and preorder wins on code simplicity in both cases. Neither dominates, which is why the answer is “either, and here is why I chose this one”.

7 The traps

Trap 1 — no null markers

```
if node is None:
    return ""
```

```
serialise(TreeNode(1, TreeNode(2)))    → "1,2"
serialise(TreeNode(1, None, TreeNode(2))) → "1,2"    ← IDENTICAL
```

Two different trees, one string. **The markers are not padding — they are the information that makes a single traversal sufficient.**

Trap 2 — no delimiter

```
return f"{node.val}{serialise(node.left)}{serialise(node.right)}"
```

Works perfectly on single-digit values and fails on everything else:

```
a tree containing 12 → "12##" → reads as 1, 2, #, #: a two-node tree
a tree containing -1 → "-1##" → reads as -, 1, ...
```

Silent corruption, and it passes every small test.

Trap 3 — a null marker that could be a value

```
NULL = "-1"
```

The moment the tree contains `-1`, deserialisation stops there and truncates the tree. **Same class of mistake as `INT_MIN` as a sentinel on [day 108](#).** Use a token that cannot be parsed as a value at all.

Trap 4 — `list.pop(0)` when deserialising

```
values = data.split(",")
value = values.pop(0)                                # O(n) per call
```

Correct output, $O(n^2)$ running time, no error. **Five billion element moves on a hundred thousand tokens.** Use `iter()` and `next()`, or a `deque`, or an index.

Trap 5 — string concatenation when serialising

```
result += str(node.val) + ","
```

Each `+=` builds a new string. $O(n^2)$ in copying. **Collect into a list and `join` once.**

Trap 6 — recursing right before left on the way back

```
node.right = build()
node.left = build()                # WRONG ORDER
```

The tokens are consumed in the order they were written, so the right call takes the left subtree's tokens. **A valid-looking tree with everything mirrored, and no error.** Exactly the [day 110](#) cursor trap.

Trap 7 — forgetting the empty tree

```
codec.deserialize("")              # ValueError or StopIteration
```

`serialise(None)` must produce something that `deserialize` accepts. With preorder that is naturally `"#"`, which round-trips cleanly. With level order you have to decide, and the empty string needs an explicit check.

Trap 8 — assuming values are small non-negative integers

Negative numbers, multi-digit numbers, and — if the values were strings — values containing the delimiter itself. **Say the assumption:** *“I am assuming integer values, so a comma is a safe delimiter. If values could be arbitrary strings I would need escaping or a length prefix.”*

8 In the interview

How it gets asked

- The main one: *“Serialise and deserialize a binary tree.”* LeetCode 297.
- The BST version: *“Now it is a BST. Can you do better?”* LeetCode 449.
- The size probe: *“How big is your output? Can you make it smaller?”*
- The format probe: *“Why preorder and not level order?”*

- The robustness probe: “What if the input is malformed?”

What to say out loud, in the first ninety seconds

1. **Frame it as design, not algorithm.** “There is no clever algorithm here — there are four decisions: the traversal order, the null marker, the delimiter, and how I consume the string coming back. Let me make each one deliberately.”
2. **State the key fact.** “A single traversal does not determine a tree — but a single traversal **plus explicit markers for the missing children** does. That is the whole idea, and the markers are what carry the structure.”
3. **Choose preorder with the reason.** “Preorder, because the reader consumes tokens in exactly the order the writer produced them, so deserialisation is one forward walk with no index arithmetic and no queue.”
4. **Say the marker constraint.** “The null token has to be something that cannot be a value — `#`, not `-1`, because `-1` might be in the tree.”
5. **Say the delimiter constraint.** “And a delimiter that cannot appear in a value. That is invisible with single digits and it is a real bug the moment there is a two-digit or negative value.”
6. **Give the size.** “A tree of `n` nodes has `n + 1` empty child slots, so the output is `2n + 1` tokens. Both directions are `O(n)`.”

The follow-ups

“**Why do you need the null markers?**” “Because without them a single traversal is ambiguous, and I can show the smallest case: a root with only a left child and a root with only a right child both have preorder `[1, 2]`. Same string, two different trees. The marker is exactly the information that distinguishes them — it says *which* child is missing, not just that one is. That is also the resolution of something that sounds contradictory: reconstructing from traversals needs **two** lists, but serialisation needs only **one**. The difference is the nulls. A tree with `n` nodes has `n + 1` empty child slots — every node has two, and `n - 1` of them are occupied by other nodes — so the output is `2n + 1` tokens, and roughly half of them are markers.”

“**Why preorder rather than level order?**” “Mainly because deserialisation becomes trivial. In preorder the reader consumes tokens in exactly the order the writer produced them, so a single forward iterator is all the state required — the recursion itself remembers where it is, and there is no index arithmetic and no queue. Level order is equally correct and it is the format LeetCode displays, but it needs a queue on both sides and the deserialiser has to pair each dequeued node with the next two tokens. On size neither dominates: level order is shorter for a wide shallow tree,

because trailing nulls can be trimmed, and preorder is shorter for a deep narrow one. So I would choose preorder for the code and mention that level order is the human-readable format.”

“**Now it is a BST. Can you do better?**” “Yes — for a BST I can drop the markers entirely and serialise just the preorder values, which is `n` tokens instead of `2n + 1`, so roughly half the size. It works because the ordering supplies the information the markers were carrying: on the way back I walk the values with a permitted **range**, exactly like validating a BST. Each value becomes a node only if it falls inside the current range; otherwise it belongs to some ancestor’s other side and I return. Each value is consumed exactly once, so it is still `O(n)`, with `O(height)` extra space and no marker tokens at all. That is LeetCode 449.”

“**How would you make it more compact still?**” “Three steps, in increasing order of effort. **Binary encoding** instead of text: four bytes per integer and one bit per null, against roughly seven characters per token as text — that is about a 2.5 times saving on its own. **Separate the structure from the values** into two streams, because the structure stream is a bit sequence that compresses extremely well while the values may not. And the theoretical answer, which I would name rather than implement: a **succinct encoding** uses `2n` bits for the shape, and that is the information-theoretic floor, because the number of distinct binary trees with `n` nodes is the Catalan number, which grows like `4n` — so you need about `2n` bits and no fewer. Knowing where the floor is tells you when to stop optimising.”

“**What are the performance traps here?**” “Two, and both are silent — correct output, catastrophically slow. **String concatenation** when serialising: each `+=` copies the entire string built so far, so a hundred thousand tokens is tens of gigabytes of copying. I collect into a list and `join` once. And `list.pop(0)` when deserialising: each pop shifts every remaining element, so a hundred thousand tokens is around five billion element moves. I use an iterator with `next()`, which is constant per token. Neither of those raises anything, and both pass a ten-node test instantly, which is exactly why they are worth calling out before writing the code.”

“**What if the tree is very deep?**” “Then the recursive version raises `RecursionError` at about a thousand nodes, and this is not an exotic input — a BST built from sorted data is a chain, which is precisely the shape. The iterative serialiser is straightforward: a stack, pushing right before left because a stack reverses. The iterative deserialiser is fiddlier, because you have to remember for each node on the stack whether the next token is its left or its right child — so the stack holds pairs of node and which-side-is-next. I would write the recursive version first, say the risk out loud, and offer the iterative one.”

A model answer

Asked: *serialise this tree, then rebuild it from the string.*

“Let me frame this as what it actually is, which is a design question rather than an algorithm question. There is no clever trick — there are **four decisions**: the traversal order, what I write for a missing child, the delimiter between tokens, and how I consume the string on the way back. Let me make each one deliberately, because that is where all the bugs live.

First, the fact the whole thing rests on. A single traversal does **not** determine a tree — a root with only a left child and a root with only a right child both have preorder `[1, 2]`. But a single traversal **plus an explicit marker for every missing child** does. The marker is exactly the information that distinguishes those two trees, and once it is there, one list is enough. A tree with `n` nodes has `n + 1` empty child slots, so the output is `2n + 1` tokens and about half of them are markers.

I will use **preorder**, and the reason is what it does to the reconstruction: the reader consumes tokens in exactly the order the writer produced them, so deserialisation is a single forward walk — take a token; if it is the marker return nothing; otherwise make a node, build its left subtree, then its right. No index arithmetic, no queue, and the recursion itself keeps track of the position. Level order is equally valid and is the format the platform displays, but it needs a queue on both sides.

Two constraints on the tokens, and both are invisible until they bite. The **null marker must be something that cannot be a value** — I will use `#`, and specifically not `-1`, because `-1` might be in the tree and deserialisation would stop dead there. And there must be a **delimiter that cannot appear in a value** — a comma is fine for integers. Without one, a tree containing `12` serialises to a string that reads back as a `1` and a `2`, and a negative number reads as a minus sign and a digit. That passes every single-digit test and corrupts everything else.

Two implementation details I would say while writing. I **collect into a list and join once** rather than concatenating, because each `+=` copies the whole string and makes it quadratic. And I deserialise from an **iterator**, not a list with `pop(0)`, because each `pop` shifts every remaining element — five billion moves on a hundred-thousand-token string, with no error to tell you.

Both directions are $O(n)$ time and $O(n)$ space, which is the floor, since the output has to contain every value.

And if you tell me it is a **binary search tree**, I can drop the markers entirely and emit just the values — `n` tokens instead of `2n + 1`, so about half the size — because the ordering carries the structure. On the way back I walk the values

with a permitted range and take each one only if it fits, which is the same idea as validating a BST.”

9 Recall card

- It is a **DESIGN** question with four decisions: **traversal order** · **null marker** · **delimiter** · **how you consume the string**. Say all four as choices, with reasons — that is the performance.
- A **single traversal does not determine a tree**; a **single traversal PLUS explicit null markers does**. The marker says *which* child is missing, which is exactly what distinguishes `1→left 2` from `1→right 2`. `n` nodes have `n+1` empty slots, so the output is `2n + 1` tokens.
- Choose **preorder** because the reader consumes tokens in the order the writer wrote them — one forward walk, no index arithmetic, no queue. Level order is equally valid and is the platform’s display format; it is shorter for wide shallow trees and longer for deep narrow ones.
- The **null marker must not be parseable as a value** (`#`, never `-1`) and the **delimiter must not appear in a value**. Without a delimiter, `12` reads back as `1`, `2` and `-1` as `-`, `1` — silent corruption that passes every single-digit test.
- Two silent `O(n2)` traps: `string +=` instead of `join` (~35 GB of copying at 100k tokens) and `list.pop(0)` instead of an iterator (~5 billion element moves). Both directions are `O(n)`. And a **BST needs NO markers** — `n` tokens instead of `2n+1`, **roughly half** — rebuilt by walking the values with a permitted range. The theoretical floor is `2n` bits for the shape, since there are $\sim 4^n$ binary trees.

1 What this is, and why they ask it

A **single point of failure** — an SPOF — is any component whose failure stops the whole system. Not degrades it. Stops it.

Three sentences. Finding them is a mechanical procedure — **point at each box in turn and ask “what if this is gone?”** — and doing it deliberately catches more than intuition does. The ones people find are the obvious boxes: the database, the load balancer. The ones that cause real outages are the **hidden** ones: a dependency that several “independent” components secretly share, a configuration service nobody drew, and a failure that takes out two things at once because they were never really independent.

They ask it because it is the fastest way to find out whether a candidate has operated a system or only designed one. The follow-up is always the same shape — “*you have two of those, so what?*” — and the good answer distinguishes **redundancy** (there is another one) from **failover** (something notices and switches) from **graceful degradation** (the feature is gone but the system is not), because having two of something with no mechanism to use the second one is not redundancy at all.

2 The story

Kottai was on the far side of the river and everything the village needed came across the bridge.

It was not a small bridge and it had been there since before anybody living could remember, and the village had been organised around it for four generations. The school bus, the milk lorry, the ambulance, the man who came on Fridays with vegetables — all of it, over that bridge.

In August 2015 a lorry carrying rods went through the parapet and the bridge was closed for eleven days.

What the eleven days taught them was not that they needed the bridge. Everybody already knew that. It was the *list* of things that turned out to depend on it, which was much longer than anyone had guessed. The water tanker could not come. The mobile

phone tower's diesel could not come, so after four days the tower went off and the village had no telephone either. Two people who were on regular medicines ran out.

The panchayat met afterwards and agreed to fix it, and the fixing took two rounds, which is the interesting part.

The first round was to build a second crossing — a smaller bridge two kilometres downstream, near the temple. That took three years and everybody was satisfied.

The second round happened because of a man called Devan, who had spent his working life on the railways and who came to the meeting where they were celebrating the new bridge and asked one question.

He asked where the diesel came from.

There was a pause and then somebody said: from the bunk on the main road.

And he said: yes, and how does the lorry from the bunk get to either bridge?

It came down the same approach road. Both bridges were fed by one road from the highway, and that road crossed a culvert about four kilometres out that flooded in a heavy monsoon. So the village had two bridges and still one way in, and nobody had noticed because the two bridges were drawn on the map as two separate things and the culvert was not drawn at all.

Devan's point, which the panchayat wrote into their notes, was that you do not find these by looking at the map. You find them by naming a thing the village needs — water, medicine, telephone — and following it backwards, one step at a time, until you are outside the village. Anything that shows up in two of those chains is the real problem, and it is usually something nobody thinks of as infrastructure at all.

3 The idea in plain English

The village has just done a single-point-of-failure analysis, and it found the two kinds: the obvious one and the shared one.

- The bridge is an **SPOF** — one component, and everything depends on it.
- The eleven days revealing water, diesel and medicine is **blast radius**: what actually stops.
- The second bridge is **redundancy**.
- The shared approach road is a **hidden SPOF** — two components that are not independent.

- The culvert not being on the map is why **you cannot find these by looking at the diagram alone.**
- Devan’s method — pick a thing the system must do, follow it backwards — is **dependency tracing**, and it is the technique.

The procedure for finding them

Method one: point at every box. Go through the diagram component by component and ask *“if this is gone right now, what still works?”* Mechanical, and it catches everything that is drawn.

Method two: trace a critical path backwards. Pick something the system must do — “a user logs in”, “an order is placed” — and list every component it touches, including the ones nobody drew. **Anything appearing in two different critical paths is where the real risk is.** Devan’s question.

Method three: ask what is not on the diagram. This is where the outages come from:

DNS	everything resolves names, and nobody draws it
the configuration service	every service reads it at startup; if it is down, nothing new can start
the secrets store	same, and worse: certificate expiry is a scheduled outage
the deployment pipeline	you cannot ship a fix during an incident
the identity provider	every internal tool and every login
the monitoring	you are blind, which turns a small outage into a long one
the certificate	expires at a fixed time, for everything at once
the shared library	a bad version deploys to every service
the ONE person	who knows how the failover works, on holiday

Naming three of those unprompted is the answer to “what did I miss?”

Series and parallel: the arithmetic

Components **in series** — every one is needed — multiply their availabilities, so **the total is always worse than the worst one.**

load balancer	99.99%	
app tier	99.95%	
database	99.95%	
cache	99.9%	(if the system fails without it)

combined	$0.9999 \times 0.9995 \times 0.9995 \times 0.999 = 99.79\%$ $\approx 18 \text{ hours down per year}$	

Four components that are each better than 99.9 percent give a system that is worse than any of them. That is the single most useful piece of arithmetic in this lesson, and it is why adding components has a cost even when each one is reliable.

Components **in parallel** — any one will do — multiply their *failure* probabilities, which is a completely different shape.

one machine at 99.9%	→ 8.8 hours down per year
two, either will do	→ $1 - 0.001^2 = 99.9999\%$ → 32 seconds
three	→ effectively never, from this cause

One extra machine turns nine hours into thirty seconds. The catch is the assumption of **independence**, and that is where the village's approach road comes in.

Correlated failure, which breaks the arithmetic

The parallel formula assumes the two components fail for unrelated reasons. Very often they do not:

two servers in the same rack	one power strip
two servers in the same zone	one datacentre, one network
two replicas of the same version	one bad deploy
two services using one library	one bug
two databases on one storage array	one array
everything, at once	one expired certificate
everything, at once	one bad configuration push

A bad deploy or a bad config push is the correlated failure that redundancy does not help with at all, because it reaches every replica simultaneously and on purpose. That is why deployment practice — canaries, staged rollouts, fast rollback — is an availability technique and not just a process preference.

Redundancy, failover, and degradation are three different things

This distinction is what the follow-up is testing.

Redundancy — there is more than one. On its own it does nothing.

Failover — something detects the failure and switches. This is the part that has to work, and it is the part that is usually untested: the detection delay, the switching mechanism, and whether the standby is actually current. **An untested failover is not redundancy; it is a second thing that might also be broken.**

Graceful degradation — the component is gone and the system continues with less. Often better value than redundancy, and much cheaper.

```
the recommendation service is down
  with redundancy:  a second copy serves recommendations
  with degradation: show a default list. Nobody notices.

the search service is down
  with degradation: show browse categories instead of a search box
```

“Which failures should degrade rather than fail?” is a design question worth raising unprompted. It turns a total outage into a smaller product.

Bulkheads and blast radius

A **bulkhead** is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

SHARED	PARTITIONED
one thread pool for all callers	a pool per caller
→ one slow dependency exhausts it and blocks everybody	→ one slow dependency exhausts only its own pool
one database for all tenants	a shard group per tier
→ one tenant's heavy query slows everyone	→ one tenant affects only their group

The related idea is the circuit breaker: after `n` consecutive failures, stop calling a dependency for a while and fail fast. That converts a slow dependency — which fills your thread pool and takes *you* down — into a fast error you can degrade around. **A slow dependency is more dangerous than a dead one**, and that sentence is worth saying.

Reducing blast radius, structurally

CELLS	partition users into independent stacks; a failure affects one cell
SHUFFLE	assign each customer a random subset of servers, so no two
SHARDING	customers share exactly the same set — one bad customer cannot take down everyone
REGIONS	a whole-region failure is survivable, at the cost of cross-region data problems
STATIC FALLBACK	serve a cached or default version when the dynamic path fails

Cell-based architecture is the strongest structural answer, and it is what AWS and several large services use: the question stops being “will it fail” and becomes “how many users does one failure reach”.

4 The picture

The same architecture, with every SPOF circled.

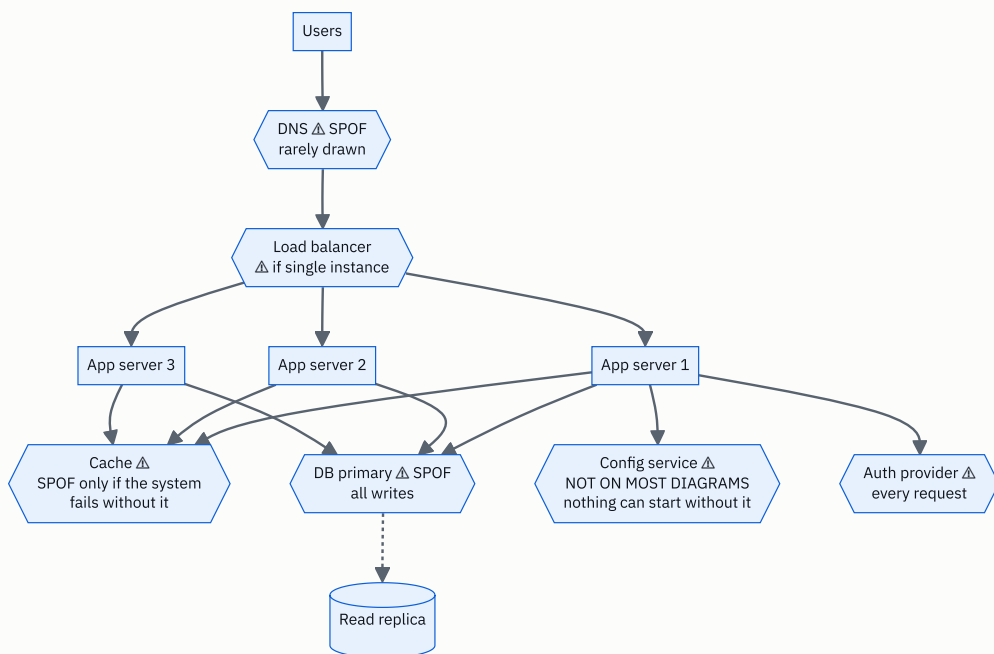
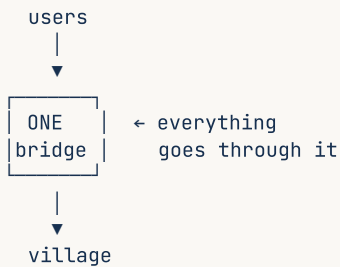


FIGURE 111.1

What to notice: **three of the five marked components are things people do not draw**. The app tier — the one with three boxes — is the only part most candidates protect.

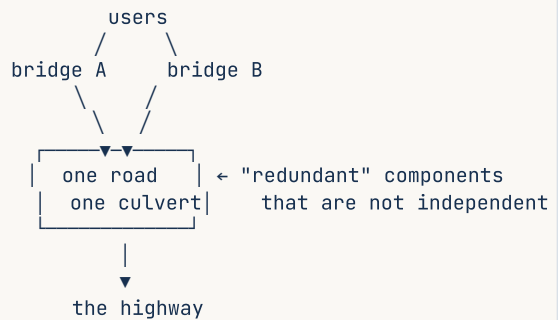
The two kinds of SPOF:

THE OBVIOUS ONE



found by: looking at the diagram

THE HIDDEN ONE



found by: tracing a dependency backwards until you leave the system

The availability arithmetic, both directions:

IN SERIES (all needed)
multiply the AVAILABILITIES

$99.99 \times 99.95 \times 99.95 \times 99.9$
 $= 99.79\%$
 $= \sim 18 \text{ HOURS down per year}$

adding a component makes it WORSE
 ...and the total is worse than the
 worst single component

IN PARALLEL (any one will do)
multiply the FAILURE RATES

$1 - (0.001 \times 0.001)$
 $= 99.9999\%$
 $= \sim 32 \text{ SECONDS per year}$

adding a copy makes it MUCH better
 ...IF the failures are independent,
 which is the whole question

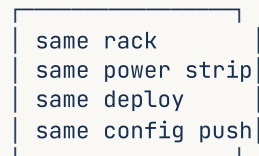
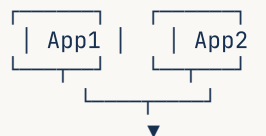
Correlated failure, drawn:

what the diagram says



independent, 99.9% each
 $\rightarrow$ combined 99.9999%

what is actually true



$\rightarrow$ combined $\approx 99.9\%$, not 99.9999%

the redundancy multiplied the COST and not the AVAILABILITY.

The three responses, and when each is right:

component fails

REDUNDANCY	another one exists (does nothing by itself)	costs money
FAILOVER	something DETECTS and SWITCHES	costs complexity, and must be TESTED
DEGRADATION	the feature is gone, the system is not	costs a product decision – often the best value

recommendations down → DEGRADE to a default list. Nobody notices.
the database down → FAILOVER. You cannot degrade away your data.
one app server down → REDUNDANCY. The load balancer handles it.

5 How it actually works

The audit, in order

1. draw the request path for ONE critical operation, end to end
2. list every component it touches – including DNS, config, auth, certificates
3. for each, ask: if this is gone, what still works?
4. classify: total outage / degraded / no effect
5. for the total-outage ones, choose redundancy, failover or degradation
6. for each redundant pair, ask: DO THEY FAIL INDEPENDENTLY?
7. write down how you would TEST each failover

Step 6 is Devan's question and step 7 is the one that is skipped.

Testing the failover

An untested failover has roughly a fifty percent chance of working, and the reasons are always the same:

the standby was never receiving replication and is hours behind
the standby's disk filled up months ago and nobody was watching
the failover script references a hostname that changed
the DNS TTL is 300 seconds, so "instant" failover takes five minutes
the application caches the database address at startup and never re-resolves
the certificate on the standby expired

So: game days and chaos engineering. Netflix's **Chaos Monkey** terminates production instances at random during working hours, and the point is not the termination — it is that a failover you exercise daily is a failover that works. **This only makes sense if services are [stateless](#)**, which is the connection worth drawing.

The dependencies nobody draws, in detail

DNS. Every name resolution. And its TTL sets a floor on how fast any DNS-based fail-over can be — a 300-second TTL means a five-minute recovery no matter how quickly you change the record.

Configuration and secrets. Services read them at startup, so a config service being down does not break running instances — it breaks **every restart, every deploy and every autoscaling event**, which means you cannot grow or recover during the incident.

Certificates. These expire at a specific instant, for everything using them, simultaneously — the purest correlated failure there is, and it has caused outages at almost every large company.

The deployment pipeline. If it is down you cannot ship the fix. This turns a ten-minute incident into a three-hour one.

Monitoring. If it is down you are blind. The outage is not longer because monitoring failed; it is longer because nobody knows what is wrong.

A shared library or sidecar. A bad version rolls out to every service at once. Redundancy is no defence, because the failure is deployed deliberately to all of it.

What real systems do

- **AWS availability zones** exist precisely to make failures independent: separate power, cooling and network, close enough for synchronous replication. **Multi-AZ is the standard first answer**, and multi-region is the expensive second one.
 - **AWS's cell-based architecture** partitions customers into independent stacks, so a failure affects one cell rather than a region. Their published availability writing is the standard reference for blast radius.
 - **Netflix's Chaos Monkey** and the wider Simian Army institutionalise failure injection.
 - **Hystrix**, and its successors, popularised the **circuit breaker** and **bulkhead** patterns — separate thread pools per dependency so one slow service cannot exhaust the whole application.
 - The most-cited real outages are almost all **correlated**, not single-component: a bad configuration push, an expired certificate, a BGP mistake, a bad deploy. **Redundancy did not help in any of them**, which is the argument for staged rollouts and fast rollback being availability work.
-

6 The numbers

The nines

availability	downtime per year	per month	per day
-----	-----	-----	-----
99%	3.65 days	7.3 hours	14.4 min
99.9%	8.77 hours	43.8 min	1.44 min
99.95%	4.38 hours	21.9 min	43 s
99.99%	52.6 minutes	4.4 min	8.6 s
99.999%	5.26 minutes	26 s	0.86 s

Being able to convert nines to minutes on the spot is a small, reliable signal.

Series: why adding components hurts

component	availability
-----	-----
DNS	99.99%
load balancer	99.99%
app tier	99.95%
cache	99.9%
database	99.95%
auth service	99.95%
-----	-----
product	99.73% → ~23.6 hours per year

Six components, each at least 99.9 percent, giving a system at 99.73 percent.

Two consequences worth stating:

- **Every dependency you add costs availability**, so removing a dependency from the critical path is an availability improvement.
- **If a component can be made optional — degrade instead of fail — it leaves the product entirely.** Making the cache optional above takes the system from 99.73 to 99.83 percent, which is nine hours a year, for no new hardware.

Parallel: what redundancy buys

copies	each at 99.9%	combined	downtime/year
-----	-----	-----	-----
1	99.9%	99.9%	8.77 hours
2	99.9%	99.9999%	31.6 seconds
3	99.9%	99.9999999%	~0.03 seconds

And the same table if the failures are correlated at ten percent — meaning one time in ten, whatever kills one kills both:

2 copies, 10% correlated → effective availability ≈ 99.99%
~53 minutes per year, not 32 seconds

A hundred times worse than the naive figure. That gap is why “are they independent?” is the question, and why two machines in the same rack are not two machines.

Blast radius, quantified

architecture	users affected by one failure
-----	-----
single stack	100%
2 regions, active-active	50%
10 cells	10%
100 cells	1%
shuffle sharding, 100 servers, 5 per customer	overlap between any two customers is tiny: one bad customer affects ~0.1% of the others

Shuffle sharding is the surprising one. With a hundred servers and five per customer, two customers share all five servers with probability about one in seventy-five million — so one abusive customer takes down almost nobody.

Failover time, and where it actually goes

detection (health check × threshold)	5 - 30 s	
election / decision	1 - 5 s	
fencing the old primary	1 - 5 s	
promotion	1 - 10 s	
client redirection:		
via a proxy or virtual IP	1 - 5 s	
via DNS with a 300 s TTL	up to 300 s	← usually the largest term
-----	-----	
total, proxy-based	10 - 60 s	
total, DNS-based	5+ minutes	

The DNS TTL is frequently the dominant term and it is entirely a configuration choice. Saying that is worth more than any amount of discussion about detection thresholds.

The cost of the last nine

99.9%	one machine per tier, backups	baseline
99.99%	redundancy per tier, multi-AZ, automated failover	~2x
99.999%	multi-region active-active, extensive automation, continuous failure testing, a team on call	~5-10x

Each additional nine costs roughly a multiple, not a percentage. The right question is therefore not “how do we get more nines” but “**what is an hour of downtime worth?**” — and for most products the answer does not justify five nines.

7 The trade-offs

Redundancy costs money and complexity

Doubling a tier doubles its cost, and the failover machinery is itself something that can fail — a bad health check can cause an unnecessary failover, which is an outage caused by the availability mechanism.

I would not add redundancy where degradation is cheaper. A second copy of the recommendation service costs money forever; a default list costs one afternoon.

Graceful degradation is usually the best value

It removes a component from the availability product entirely. A cache that the system requires is a multiplier; a cache the system can survive without is not in the calculation at all.

The cost is a product decision: someone has to agree that showing stale or default content is acceptable, and someone has to build and — crucially — **test** the fallback path, which otherwise rots quietly.

Multi-region: the expensive one

Multi-AZ is nearly free — synchronous replication within a region, automated failover, and a substantial availability gain.

Multi-region is a different order of problem: the latency makes synchronous replication impossible, so you accept data loss on a region failure, or you go active-active and accept write conflicts.

I would not go multi-region for availability alone unless the business genuinely requires it. The common reasons that *do* justify it are latency for a global user base and data-residency law — and availability comes along as a benefit.

Where the analysis goes wrong

- **Assuming independence.** The most common error, and the one that makes the arithmetic lie by a factor of a hundred.
- **Untested failover.** Roughly a coin flip whether it works. If it is not exercised, it is not real.

- **Forgetting the failure paths that are not the request path** — deployment, configuration, secrets, monitoring. You cannot fix an incident without them.
- **A slow dependency is worse than a dead one.** A dead one fails fast; a slow one fills your connection pool and takes you down with it. **Timeouts and circuit breakers are the defence**, and a missing timeout is one of the most common causes of a cascading outage.
- **The availability mechanism becoming the failure.** An aggressive health check that removes healthy servers under load makes the load worse on the rest, and cascades.
- **A person as an SPOF.** If one engineer knows how the failover works, that is a real dependency and it goes on the list.

8 In the interview

How it gets asked

- The direct one: *“Which component here takes the whole system down if it fails?”*
- The follow-up: *“You have two of those. So what?”*
- The one that separates people: *“What is not on this diagram?”*
- The arithmetic: *“Each component is 99.9 percent available. What is the system?”*
- The realistic one: *“Your database has a standby. Are you sure it works?”*

What to say out loud, in the first ninety seconds

1. **Give the method, not a list.** “I would go component by component and ask what still works if it is gone — and then, separately, trace one critical path backwards, because that finds the dependencies that are not on the diagram.”
2. **Name the obvious ones, then move past them.** “The database primary takes all writes; a single load balancer instance; the cache, if the system cannot serve without it.”
3. **Go straight to the undrawn ones.** “The ones that cause real outages are usually not on the picture: DNS, the configuration service, the secrets store, certificates, the deployment pipeline, and the identity provider.”
4. **Do the series arithmetic.** “And these compound: six components each at 99.9 percent or better give a system at about 99.7 percent — roughly a day a year. Every dependency on the critical path costs availability.”

5. **Distinguish the three responses.** “For each one: redundancy, failover, or degradation. They are not the same — having two of something with no mechanism to switch is not redundancy.”
6. **Ask the independence question.** “And for every redundant pair, do they actually fail independently? Two machines in one rack, two replicas of the same bad deploy, or two services behind one expired certificate are one component wearing two boxes.”

The follow-ups

“**You have two of those. So what?**” “Two is only useful if three things are true. First, something **detects** the failure — a health check with a threshold, and there is a real delay there: with a five-second interval and a threshold of two, detection takes ten to twelve seconds, during which the dead one is still receiving its share of traffic. Second, something **switches** — and clients have to learn about the switch, which is where DNS TTLs bite: a 300-second TTL means five minutes of recovery no matter how fast everything else was. Third, the second one has to be **current and healthy**, which is the part that is usually untested — the standby that is hours behind on replication, or whose disk filled up in March. So my answer is: redundancy without tested failover is not redundancy, it is a second thing that might also be broken. I would want to know when the failover was last exercised, and I would want it exercised on a schedule rather than during an incident.”

“**What is not on this diagram?**” “That is where the outages are. **DNS** — everything resolves names and nobody draws it, and its TTL is a floor on recovery time. **Configuration and secrets** — services read them at startup, so if that store is down, running instances are fine but nothing can restart, deploy or autoscale, which means you cannot recover or grow during the incident. **Certificates** — they expire at a fixed instant for everything using them at once, which is the purest correlated failure there is. **The deployment pipeline** — if it is down you cannot ship the fix, which turns a ten-minute incident into a three-hour one. **Monitoring** — if it is down you are blind, and the outage is longer because nobody knows what is wrong. And **a shared library or sidecar**, where a bad version rolls out to every service simultaneously and redundancy is no defence at all, because the failure was deployed deliberately to all of it.”

“**Each component is 99.9 percent available. What is the system?**” “Worse than any of them, and that is the point. Components **in series** — where every one is needed — multiply their availabilities. Six at 99.9 percent gives about 99.4 percent, which is roughly two days a year. So **every dependency you put on the critical path costs availability**, and the corollary is useful: removing a dependency, or making it optional so the system degrades instead of failing, is an availability improvement with no new

hardware. Making one cache optional in that chain is worth several hours a year. Components **in parallel** are the opposite — you multiply the failure rates, so two machines at 99.9 percent give 99.9999, which is nine hours down becoming about thirty seconds. But that formula assumes **independence**, and if the two fail together even ten percent of the time, the real figure is closer to 99.99 percent — about fifty minutes rather than thirty seconds. A hundred times worse than the naive number.”

“Your database has a standby. Are you sure it works?” “No, and I would not claim to be until it had been tested — an untested failover is close to a coin flip, and the reasons are always the same handful. The standby was not actually receiving replication and is hours behind. Its disk filled up and nobody was alerting on it. The failover script refers to a hostname that changed last year. The application caches the database address at startup and never re-resolves, so it keeps talking to the dead one. Or the certificate on the standby expired. So the thing I would want is not a better design but a **schedule**: fail over deliberately, in business hours, on a regular cadence, and measure how long it takes. That is what game days and chaos engineering are for — Netflix’s Chaos Monkey kills production instances during working hours, and the value is not the killing, it is that a failover exercised daily is one that works. And that only makes sense if the services are stateless, which ties back to why statelessness matters.”

“How would you reduce the blast radius?” “Four levers, in increasing order of effort. **Graceful degradation** — decide which features can be missing rather than fatal, so a failure costs a feature instead of the system; that is usually the best value and it removes the component from the availability product entirely. **Bulkheads** — a separate connection or thread pool per dependency, so one slow service cannot exhaust the whole application, plus circuit breakers so a slow dependency becomes a fast error rather than a queue that takes you down. Worth saying plainly: **a slow dependency is more dangerous than a dead one**, because a dead one fails fast. **Cells** — partition users into independent stacks so a failure reaches ten percent of users instead of a hundred. And **shuffle sharding**, which is the elegant one: give each customer a random subset of, say, five servers out of a hundred, and then two customers almost never share the same five — so one abusive customer degrades a fraction of a percent of the others rather than everybody.”

“Is there anything on the list that is not a machine?” “Yes, and it belongs on the list. **A person** — if one engineer is the only one who knows how the failover works or where the runbook is, that is a genuine single point of failure and it fails on holidays. **A process** — if every deploy needs one approval from one team, that team is on the critical path during an incident. And **a vendor** — a third-party payment provider or identity provider is a component you do not control, cannot make redundant, and

often cannot even monitor properly. For that last one the honest answer is usually degradation: decide in advance what the product does when the provider is down, rather than discovering it at the time.”

A model answer

Asked: *which component here takes the whole system down if it fails?*

“Let me answer with a method rather than a list, because the components I can see are the easy half.

The method is two passes. First, **point at every box and ask what still works if it is gone**. That catches everything drawn. Second — and this is the pass that matters — **pick one critical operation and trace it backwards**, naming every component it touches, including the ones nobody put on the picture. Anything that turns up in two different critical paths is where the real risk is.

From the diagram: the **database primary**, because all writes go through it. The **load balancer**, if it is a single instance. And the **cache** — but only if the system genuinely cannot serve without it, which is a design choice I would want to make deliberately rather than inherit.

The ones that cause actual outages are usually not drawn. **DNS**, which everything depends on and whose TTL is a hard floor on any DNS-based recovery. The **configuration and secrets store**, which does not break running instances but breaks every restart, deploy and autoscaling event — so you cannot recover or grow during the incident. **Certificates**, which expire at a fixed instant for everything at once. The **deployment pipeline**, without which you cannot ship the fix. And **monitoring**, without which you are blind and the outage is long for reasons unrelated to its cause.

Two pieces of arithmetic I would put on the board. Components **in series** multiply, so six components each at 99.9 percent give a system at about 99.4 — two days a year. Every dependency on the critical path costs availability, and making one of them **optional** — degrade rather than fail — takes it out of the product entirely, which is an availability gain with no new hardware. Components **in parallel** multiply their failure rates instead: two machines at 99.9 percent give 99.9999, so nine hours becomes thirty seconds. **But that assumes independence**, and if they fail together even ten percent of the time the real figure is nearer 99.99 — fifty minutes, not thirty seconds. That gap is the whole reason to ask whether two things are genuinely independent, and two servers in one rack, or two replicas of the same bad deploy, are one component wearing two boxes.

For each SPOF I would then choose between three different responses, which people run together. **Redundancy** means another one exists, and on its own it does nothing. **Failover** means something detects and switches — that is the part that has to work and is usually the part nobody has tested. **Degradation** means

the feature is gone and the system is not, and it is often the cheapest and best answer: the database needs failover because you cannot degrade away your data, but recommendations can just show a default list and nobody notices.

And the last thing I would say unprompted: **a slow dependency is more dangerous than a dead one**, because a dead one fails fast while a slow one fills your connection pool and takes you down with it. So timeouts and circuit breakers on every outbound call, and separate pools per dependency, are part of the answer to this question and not a separate topic.”

9 Recall card

- **An SPOF is any component whose failure stops the system.** Find them two ways: **point at every box and ask what still works**, and **trace one critical path backwards until you leave the system** — the second pass finds what the diagram does not show.
- **The ones that cause real outages are undrawn: DNS · config and secrets · certificates · the deploy pipeline · monitoring · a shared library · one person.** Config being down does not break running instances — it breaks every **restart, deploy and autoscale**, so you cannot recover during the incident.
- **Series multiply availabilities, parallel multiply failure rates.** Six components at 99.9% give **~99.4%** — **two days a year**, so every dependency on the critical path costs availability and **making one optional removes it from the product entirely**. Two parallel copies at 99.9% give **99.9999%** — **nine hours becomes 32 seconds**.
- **That parallel figure assumes INDEPENDENCE.** Same rack, same zone, same deploy, same config push, same certificate — and it collapses: **10% correlation turns 32 seconds into ~53 minutes**. A **bad deploy or config push is the correlated failure redundancy cannot help with**, which makes canaries and fast roll-back availability work.
- **Redundancy ≠ failover ≠ degradation.** Two of something with no tested switching is **a second thing that might also be broken** — and DNS TTL is often the largest term in failover time. **Degradation is usually the best value**. And **a slow dependency is more dangerous than a dead one**: use timeouts, **circuit breakers** and **bulkheads**, then **cells** and **shuffle sharding** to shrink blast radius from 100% of users to a fraction of a percent.

DSA topic: Serialising and deserialising a tree **System design topic:** Single points of failure

Code these, in this order

One rule for the whole set: **say the four decisions out loud before writing** — traversal order, null marker, delimiter, and how you consume the string. Every bug in this problem is one of those four made carelessly.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Serialize and Deserialize Binary Tree	LeetCode 297 (Hard)	Null markers, the delimiter, and consuming with an iterator.
2	Serialize and Deserialize BST	LeetCode 449 (Medium)	Why a BST needs no markers, and the range walk.
3	Find Duplicate Subtrees	LeetCode 652 (Medium)	Serialisation as a <i>key</i> — where the delimiter bug bites hardest.
4	Construct String from Binary Tree	LeetCode 606 (Medium)	A different format, and why its brackets are load-bearing.

On problem 1, break it three ways

Remove the markers and serialise both one-child trees — record the collision. Remove the delimiter and serialise a tree containing 12 — record the string. Swap the left and right recursion in the deserialiser — record the tree.

On problem 1 again, time both traps

Build a 100,000-node tree. Serialise with `+=` and with `join`. Deserialise with `list.pop(0)` and with an iterator. Record all four times.

On problem 2, compare the sizes

Serialise the same BST with both codecs and count the tokens. State the ratio and say where the saving comes from.

On problem 3, notice why it is the same problem

The subtree signature must be unique per shape *and* values. Say which of the four decisions this problem punishes hardest, and construct an input that breaks a careless implementation.

The four-decisions drill

1. Name all four decisions.
2. For each, state the constraint and one failing choice.
3. Say which of the four is invisible on single-digit test data.
4. Add a fifth decision about malformed input and say what you would do.

The markers drill

1. Build both one-child trees and serialise them with and without markers.
2. Say what the marker actually encodes, in one sentence.
3. Compute the number of markers for a tree of `n` nodes, and prove it.
4. Reconcile this with day 110's "one traversal is never enough".

The delimiter drill

1. Serialise a tree containing 12 without a delimiter and read the result back by hand.
2. Do the same for a tree containing `-1`.
3. State the rule for a valid delimiter and for a valid null marker.
4. Say what you would do if the values were arbitrary strings.

The consumption drill

1. Write the deserialiser with an iterator.
2. Rewrite it with `list.pop(0)` and time both at 100,000 tokens.
3. Say why the slow one raises nothing.
4. Name two other correct approaches.

The order drill

1. Write the preorder codec and the level-order codec.
2. Serialise a perfect tree of 7 and a right-chain of 7 with both, and count tokens.
3. Say which format wins on which shape.
4. Say which one you would write, and why.

The BST drill

1. Write the BST codec without markers.
2. Say why the markers are unnecessary, in one sentence about ordering.
3. Give the token counts for both codecs on the same BST.
4. Say what the range walk has in common with day 108.

The compactness drill

1. Give three ways to make the output smaller, in increasing order of effort.
2. State the information-theoretic floor for the shape and where it comes from.
3. Compute the text size for a million integers and compare with a binary encoding.
4. Say why knowing the floor is useful.

The break-it drill

Trigger each and record the exact output or error:

1. No null markers, on the two one-child trees.
 2. No delimiter, on a tree containing 12.
 3. `-1` used as the null marker, on a tree containing `-1`.
 4. `list.pop(0)` on a 100,000-token string, timed.
 5. String `+=` on a 100,000-node tree, timed.
 6. Right recursion before left in the deserialiser.
 7. `deserialise("")` and `serialise(None)`.
 8. A recursive codec on a 10,000-node chain.
-

The method drill

1. State the two passes for finding SPOFs.
2. Say what the second pass finds that the first cannot.
3. Do both passes on a diagram with a load balancer, three app servers, a cache and a database.
4. Say what question you ask about every redundant pair.

The undrawn drill

1. Name seven dependencies that are usually not on a diagram.
2. For each, say what specifically breaks when it fails.
3. Say which one does not affect running instances but stops recovery.

4. Say which one is the purest correlated failure and why.

The arithmetic drill

Compute each:

1. Downtime per year at 99%, 99.9%, 99.95%, 99.99% and 99.999%.
2. The product of six components at 99.99, 99.99, 99.95, 99.9, 99.95 and 99.95.
3. The combined availability of two and three parallel copies at 99.9%.
4. The same with 10% correlation, and the ratio to the naive figure.
5. The availability gain from making one 99.9% component optional.

The three-responses drill

1. Define redundancy, failover and degradation.
2. Say what redundancy alone achieves.
3. For each of these, choose one and justify it: recommendations service, database primary, one app server, search, the payment provider.
4. Say what makes a failover real rather than nominal.

The correlation drill

1. Name six ways two “independent” components can fail together.
2. Say which one redundancy cannot help with at all, and why.
3. Say what practice therefore counts as availability work.
4. Explain the village’s approach road in one sentence of system terms.

The blast-radius drill

1. Name the four levers in increasing order of effort.
2. Give the percentage of users affected for a single stack, two regions, ten cells and a hundred cells.
3. Explain shuffle sharding and compute the overlap for 5 servers out of 100.
4. Say what a bulkhead is and what a circuit breaker converts one failure into.
5. Say why a slow dependency is more dangerous than a dead one.

The failover-time drill

1. List the five components of failover time.
2. Say which is usually the largest, and that it is a configuration choice.
3. Compute a proxy-based and a DNS-based failover time.
4. Name six reasons an untested failover does not work.

The failure drill

For each, say what happens and what you would add:

1. A certificate used by every service expires at midnight.
2. A bad configuration is pushed to all replicas.
3. A downstream dependency slows from 50 ms to 5 seconds.
4. The config service is unavailable during an autoscaling event.
5. One engineer who knows the failover procedure is unreachable.
6. Two “redundant” servers are in the same rack.

Three of the six are not fixed by adding a redundant copy. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Serialise this tree, then rebuild it from the string.* The four decisions framed as design, why markers make one traversal sufficient with the collision shown, preorder chosen for what it does to deserialisation, the marker and delimiter constraints, the two $O(n^2)$ traps named before writing, and the BST halving offered.
 2. *Which component here takes the whole system down if it fails?* The two-pass method rather than a list, the drawn ones quickly, the undrawn ones at length, the series arithmetic with the two-days-a-year figure, the three distinct responses, and the independence question for every redundant pair.
 3. *You have two of those. So what?* Detection, switching and currency — with the DNS TTL as the usual largest term, and the six reasons an untested failover fails.
-

Before you move on

- [] I state the four serialisation decisions before writing any code.
- [] I can show the collision that markers prevent.
- [] I can compute the number of markers for `n` nodes and explain it.
- [] I can reconcile “one traversal is never enough” with “one traversal plus markers is”.
- [] I know which marker and delimiter choices are unsafe, and why.
- [] I use an iterator and can quote the cost of `pop(0)`.

- [] I join once and can quote the cost of `+=` .
- [] I can write the BST codec and say why it needs no markers.
- [] I know the token counts for both codecs and the ratio.
- [] I can name the information-theoretic floor for the shape.
- [] I have an iterative codec for deep trees.
- [] I can state the two-pass method for finding SPOFs.
- [] I can name seven undrawn dependencies and what each one breaks.
- [] I can do the series and parallel arithmetic in both directions.
- [] I can compute what 10% correlation does to a redundant pair.
- [] I can distinguish redundancy, failover and degradation with examples.
- [] I ask whether every redundant pair fails independently.
- [] I can name the four blast-radius levers and explain shuffle sharding.
- [] I can say why a slow dependency is worse than a dead one.
- [] I answered all three questions above out loud.

Trees revision and mock round

DATA STRUCTURES AND ALGORITHMS

Trees revision and mock round

You can solve two unseen tree problems cold.

SYSTEM DESIGN

Scaling revision and interview questions

You can scale an unseen read-heavy system from one box to many, out loud.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *This service is falling over at 10000 QPS. Walk me through fixing it.*

1 What this is, and why they ask it

Fourteen days ago you did not know what a leaf was. Since then: [vocabulary](#), [nodes in code](#), [the three depth-first orders](#), [level order](#), [diameter](#), [comparisons](#), [paths](#), [lowest common ancestor](#), [the BST invariant](#), [insert and delete](#), [validation](#), [balance](#), [reconstruction](#) and [serialisation](#).

Three sentences. Almost every tree problem is **one of six shapes**, and the entire skill is recognising which one in the first thirty seconds — because once you have named the shape, the code is eight lines you have already written. The recognition is not about the words in the problem; it is about **which way the information travels**: up from the children, down from the parent, across a level, or between two trees. And the five recurring bugs are the same five every time, so knowing them is worth more than knowing any individual problem.

They run a mock round because in a real interview nobody says “this is a postorder problem”. They say *“find the maximum sum of any path”*, and your first minute decides the next thirty.

2 The story

The workshop was two rooms behind a paint shop and Chandrasekhar had been making furniture in it since 1979.

His apprentice that year was a boy from the polytechnic who had been taught to draw and had never held a chisel, and the thing he could not get over was how fast the old man decided things.

A woman brought in a broken chair. The old man turned it over, looked at it for about four seconds, and said which joint had failed and which one he would put in instead. The boy had been staring at the same chair for two minutes.

He asked how.

Chandrasekhar said there were five joints. That is all. Everything he had made in forty years was one of five, or two of them together.

The boy said there must be more than five.

The old man said there are hundreds of names, and five joints. The names are for the catalogue. What you actually have to decide is one thing: **which way does the load go?**

If the load pushes two pieces apart along their length, you need one kind. If it twists them, you need a different one. If the piece has to come apart again for moving, that is a third. If it is a corner where two flat pieces meet, a fourth. If it is a leg taking weight straight down, the fifth.

He said the mistake everybody makes is to look at the shape of the wood. The shape tells you nothing. Two pieces that look identical need different joints depending on where the weight goes, and two pieces that look nothing alike need the same joint if the load is the same.

Then he said the part the boy wrote down and kept.

He said: when I look at a chair for four seconds, I am not thinking about the joint. I am watching the load. The joint is whatever the load tells me. If you learn the five joints you have learned nothing — you have to learn to see where the weight is going, and then there are only five answers and it does not matter which one it is.

The boy got quicker at it over about eight months. Not by learning more joints.

3 The idea in plain English

Chandrasekhar's question — *which way does the load go?* — is the tree question. The load is the **information**, and the direction it travels picks the shape.

The one question

Which way does the information travel?

Four answers, and they give the six shapes.

INFORMATION TRAVELS	SHAPE	TRAVERSAL	EXAMPLES
Up from the children, combined	simple recursion	postorder	height, count, sum, invert, mirror
Up , but you need <i>two</i> different things	the return-value trick	postorder	diameter, max path sum, balanced, largest BST subtree
Down from the parent, as an argument	carry state down	preorder	root-to-leaf paths, depth, path sum, serialisation
Across one level at a time	BFS with a queue	level order	level lists, right side view, minimum depth, zigzag
Between two trees in step	two-node recursion	any	same tree, symmetric, subtree
Guided by value ordering	walk down, do not search	—	BST search, validate, LCA, k-th smallest

That table is the fourteen days. Everything else is a detail on one of those six rows.

The universal skeleton

```
def solve(node):
    if node is None:
        return <the answer for an empty tree>
    left = solve(node.left)
    right = solve(node.right)
    return <combine left, right and node.val>
```

Write those five lines before deciding anything. Then fill in two blanks: what an empty tree returns, and how the pieces combine. Four of the six shapes are exactly this with different blanks.

Shape by shape, in one line each

1 — Simple recursion. The answer for a node is a function of the answers for its children.

```
return 1 + max(height(node.left), height(node.right))
```

Base case is the identity for the combining operation: `-1` for max-of-heights (edge convention), `0` for sums and counts, `True` for “all of them satisfy”.

2 — The return-value trick. You need one thing for the parent and a different thing for the answer.

```
best = max(best, left + right + node.val) # RECORD: the answer
return node.val + max(left, right)      # RETURN: what the parent can use
```

Say the sentence: *the value I return is not the value I am looking for.* A path that bends at me has spent both my children, so my parent cannot extend it. This is [day 102](#) and [day 104](#).

3 — Carry state down. The node needs something only an ancestor knows: a depth, a running path, a permitted range.

```
walk(node.left, depth + 1)
walk(node.left, low, node.val)      # a narrowing range — BST validation
```

Anything passed as an argument needs no undo. Anything mutated does — append and pop, the [day 094](#) rule.

4 — Level order. The levels themselves are the answer, or you want the shallowest something.

```
level_size = len(queue)              # capture BEFORE draining
```

5 — Two trees at once. Three base cases, and the middle one is the crash.

```
if a is None and b is None: return True
if a is None or b is None:  return False
```

6 — BST ordering. Do not search — walk down, and let the comparison discard a whole subtree.

```
node = node.left if target < node.val else node.right
```

The recognition rules

Reading a problem, these are the tells:

"the maximum/longest/best ... in the tree"	→ return-value trick (shape 2)
"root to leaf"	→ carry state down (shape 3)
"level" / "shallowest" / "each row"	→ BFS (shape 4)
"are these two ..." / "is this a subtree of"	→ two-tree walk (shape 5)
"binary search tree"	→ shape 6; do not search
"height / count / sum / invert"	→ simple recursion (shape 1)
"reconstruct / serialise"	→ markers or two traversals

And one negative rule: if the problem says *binary tree* and you catch yourself writing `if target < node.val`, stop. **A plain binary tree gives you no direction** — search is $O(n)$ and you must look in both subtrees.

The five bugs, which are the same five every time

1 — The missing `None` case. `AttributeError: 'NoneType' object has no attribute 'val'`. The base case is not defensive coding; it is the recursion.

2 — Local instead of global. Checking a node against its children rather than against all its ancestors. Both the **BST invariant** and **balance** fail this way, and both are silent.

3 — Returning the wrong thing to the parent. Returning the bent path instead of one arm. Silent, and the answer grows with depth.

4 — Recomputing what you already have. Calling `height` inside a walk that is already computing heights — $O(n^2)$ on a chain. The fix is always the return-value trick.

5 — Reference instead of copy. `result.append(current)` instead of `current[:]`. Every entry is the same empty list.

None of the five raises, except the first. That is why they are worth memorising as a checklist.

The complexity table

operation	time	space
-----	-----	-----
any full traversal	$O(n)$	$O(\text{height})$ DFS / $O(\text{width})$ BFS
BST search/insert/delete	$O(\text{height})$	$O(1)$ iterative
diameter, max path sum	$O(n)$	$O(\text{height})$
LCA, general tree	$O(n)$	$O(\text{height})$
LCA, BST	$O(\text{height})$	$O(1)$
serialise / deserialise	$O(n)$	$O(n)$
reconstruct from traversals	$O(n)$	$O(n)$ for the map
validate BST	$O(n)$	$O(\text{height})$
is_balanced, one pass	$O(n)$	$O(\text{height})$
is_balanced, naive	$O(n^2)$	$O(\text{height})$

Two sentences carry most of it. *Time is the whole tree; space is the deepest path.* And $O(\text{height})$ is $O(\log n)$ only if balanced, and nothing guarantees that.

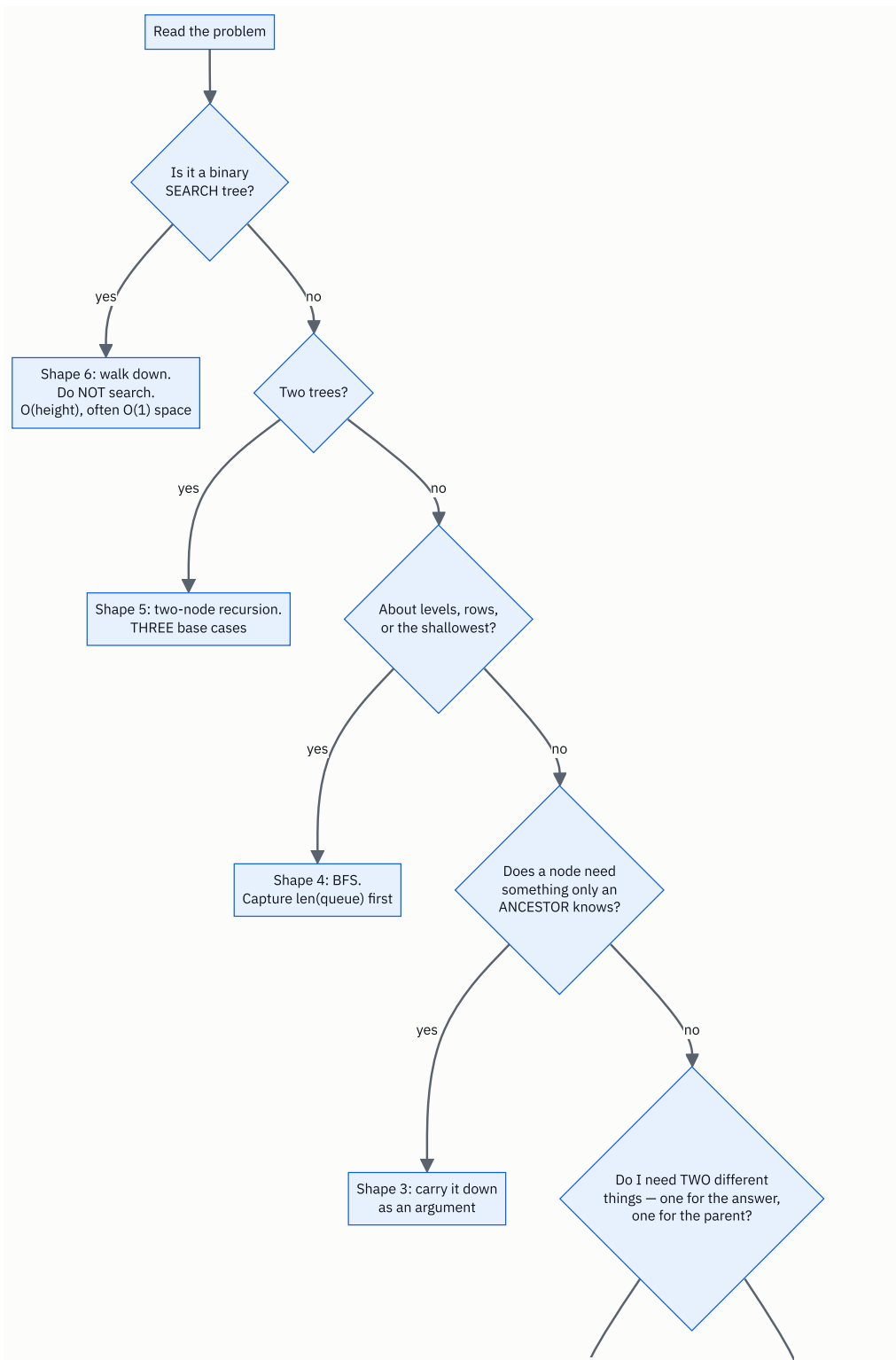
The heights that matter

n = 1,000,000			
balanced	~20	DFS stack: 20 frames.	BFS queue: ~500,000 nodes
degenerate	999,999	DFS: RecursionError.	BFS queue: 1 node
random BST	~40		

A chain is the normal case for a BST built from sorted data, and it breaks both the complexity and the recursion limit. Say the risk when the constraints allow it.

4 The picture

The decision, as thirty seconds of thinking.



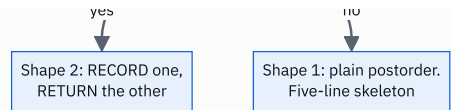


FIGURE 112.1

The six shapes, side by side, as code:

1 SIMPLE <pre>def f(node): if not node: return IDENTITY l = f(node.left) r = f(node.right) return COMBINE(l, r, node)</pre>	2 RETURN-VALUE TRICK <pre>best = -inf def f(node): if not node: return 0 l = max(0, f(node.left)) r = max(0, f(node.right)) best = max(best, l + r + node.val) ← RECORD return node.val + max(l, r) ← RETURN</pre>
3 CARRY DOWN <pre>def f(node, state): if not node: return state = narrow(state, node) f(node.left, state) f(node.right, state)</pre>	4 LEVEL ORDER <pre>while queue: n = len(queue) ← capture FIRST for _ in range(n): node = queue.popleft() ...append children...</pre>
5 TWO TREES <pre>def f(a, b): if not a and not b: return True if not a or not b: return False return a.val == b.val and ...</pre>	6 BST ORDERING <pre>while node: if target == node.val: return node node = node.left if target < node.val \ else node.right</pre>

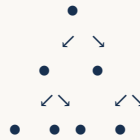
Where the information goes, drawn:

SHAPE 1 & 2
(up)



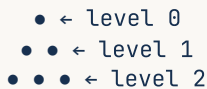
children answer first;
the parent combines
→ POSTORDER

SHAPE 3
(down)



the parent tells the child
what it needs to know
→ PREORDER, and no undo if
passed as an argument

SHAPE 4
(across)



a queue, one row at a time
→ O(width) space

SHAPE 5
(in step)



two nodes at once,
three base cases

5 The code, built step by step

Step 1 — the thirty seconds, out loud

“Is it a search tree? Are there two trees? Is it about levels? Does a node need something an ancestor knows? Do I need two different values — one to return and one to record?” **Five questions, and the answers pick the shape.**

Step 2 — write the skeleton before deciding the details

```
def solve(node):
    if node is None:
        return ...
    left = solve(node.left)
    right = solve(node.right)
    return ...
```

Then fill the two blanks. Deciding the base case *after* writing the combine is much easier than the reverse, because the base case is the identity of the combining operation.

Step 3 — state the complexity as you write, not afterwards

“ $O(n)$ time — one visit per node. $O(\text{height})$ space for the stack, so twenty frames on a balanced million-node tree and a million on a chain.”

Step 4 — run the five-bug checklist before saying you are done

1. is the None case handled, and does it return the right identity?
2. am I checking against ancestors, or only against children?
3. is what I RETURN the same as what I want? (usually it should not be)
4. am I recomputing something I already have?
5. if I record a list, am I recording a COPY?

Ten seconds, and it catches four bugs that do not raise.

Mock problem one — “Count the nodes whose value is greater than every value on the path from the root to it.”

Talk it through:

“Good nodes, LeetCode 1448. A node needs to know the maximum on the path from the root — that is information only an **ancestor** has, so this is **shape 3**: carry it down. I pass the maximum so far as an argument, and because it is a value rather than a mutation, there is nothing to undo. $O(n)$ time, $O(\text{height})$ space.”

```
def good_nodes(root: "TreeNode | None") → int:
    def walk(node, best_so_far: int) → int:
        if node is None:
            return 0
        count = 1 if node.val ≥ best_so_far else 0
        best_so_far = max(best_so_far, node.val) # NARROW, then descend
        return count + walk(node.left, best_so_far) + walk(node.right, best_so_far)

    return walk(root, float("-inf"))
```

Mock problem two — “Find the largest subtree sum in the tree.”

“I need the sum of every subtree, and the answer is the largest of them. The sum of a subtree is a function of its children’s sums — that is **up** — but the answer is not what I return. So it is **shape 2**: return the sum, record the maximum. Values can be negative, so I initialise the best to negative infinity rather than zero, and I do **not** floor the children at zero here, because a subtree sum includes everything below it whether I like it or not — unlike a *path*, which may stop.”

```

def largest_subtree_sum(root: "TreeNode | None") → int:
    best = float("-inf")

    def total(node) → int:
        nonlocal best
        if node is None:
            return 0
        s = node.val + total(node.left) + total(node.right)
        best = max(best, s)           # RECORD
        return s                     # RETURN
    total(root)
    return int(best)

```

The distinction from max path sum is worth stating: a subtree is not optional, so there is no `max(0, ...)`. Noticing that is the whole problem.

The complete revision file

```
from collections import deque

class TreeNode:
    def __init__(self, val: int = 0,
                  left: "TreeNode | None" = None,
                  right: "TreeNode | None" = None) → None:
        self.val = val
        self.left = left
        self.right = right

    def __repr__(self) → str:
        return f"TreeNode({self.val})"

# ---- SHAPE 1: simple recursion – the answer combines the children's -----

def height(node: TreeNode | None) → int:
    """Base case is the IDENTITY of the combining operation.
    -1 for max-of-heights, 0 for sums, True for 'all'."""
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))

def count_nodes(node: TreeNode | None) → int:
    if node is None:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)

def invert(node: TreeNode | None) → TreeNode | None:
    if node is None:
        return None
    node.left, node.right = invert(node.right), invert(node.left)
    return node

# ---- SHAPE 2: return one thing, record another -----

def diameter(root: TreeNode | None) → int:
    """RETURN the height (what the parent needs).
    RECORD the bend (the answer the parent cannot use)."""
    best = 0

    def h(node: TreeNode | None) → int:
        nonlocal best
        if node is None:
            return -1
        left, right = h(node.left), h(node.right)
        best = max(best, left + right + 2)    # RECORD
        return 1 + max(left, right)         # RETURN

    h(root)
    return best

def max_path_sum(root: TreeNode | None) → int:
    """Same shape, with the floor that negatives force.
```

```

max(0, arm) because a PATH may decline a branch.
best starts at -inf because a path must contain a node."""
best = float("-inf")

def arm(node: TreeNode | None) → int:
    nonlocal best
    if node is None:
        return 0
    left = max(0, arm(node.left))
    right = max(0, arm(node.right))
    best = max(best, left + right + node.val)
    return node.val + max(left, right)

arm(root)
return int(best)

def is_balanced(root: TreeNode | None) → bool:
    """Same shape with a SENTINEL instead of a recorded maximum.
    -2 is safe because a real height is never below -1."""
    def h(node: TreeNode | None) → int:
        if node is None:
            return -1
        left = h(node.left)
        if left == -2:
            return -2
        right = h(node.right)
        if right == -2 or abs(left - right) > 1:
            return -2
        return 1 + max(left, right)

    return h(root) != -2

# ---- SHAPE 3: carry state down as an argument -----

def good_nodes(root: TreeNode | None) → int:
    """A node needs the maximum on the path from the ROOT – only an ancestor
    knows it. Passed as a VALUE, so there is nothing to undo."""
    def walk(node: TreeNode | None, best_so_far: float) → int:
        if node is None:
            return 0
        count = 1 if node.val ≥ best_so_far else 0
        best_so_far = max(best_so_far, node.val)
        return count + walk(node.left, best_so_far) + walk(node.right, best_so_far)

    return walk(root, float("-inf"))

def root_to_leaf_paths(root: TreeNode | None) → list[list[int]]:
    """State that is MUTATED needs choose-recurse-un-choose, and a COPY."""
    out: list[list[int]] = []
    trail: list[int] = []

    def walk(node: TreeNode | None) → None:
        if node is None:
            return
        trail.append(node.val) # choose
        if node.left is None and node.right is None:
            out.append(trail[:]) # COPY
        else:
            walk(node.left)

```

```

        walk(node.right)
        trail.pop()                                # un-choose

    walk(root)
    return out

def is_bst(node: TreeNode | None,
          low: float = float("-inf"),
          high: float = float("inf")) → bool:
    """The range is state carried down. Narrows on every step, never widens."""
    if node is None:
        return True
    if not (low < node.val < high):
        return False
    return (is_bst(node.left, low, node.val)
            and is_bst(node.right, node.val, high))

# ---- SHAPE 4: level order -----
def level_order(root: TreeNode | None) → List[List[int]]:
    """len(queue) captured BEFORE draining is the whole trick."""
    if root is None:
        return []
    out: List[List[int]] = []
    queue = deque([root])
    while queue:
        size = len(queue)                                # capture FIRST
        level = []
        for _ in range(size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        out.append(level)
    return out

def min_depth(root: TreeNode | None) → int:
    """BFS is genuinely FASTER here: it stops at the first leaf."""
    if root is None:
        return 0
    queue = deque([(root, 1)])
    while queue:
        node, depth = queue.popleft()
        if node.left is None and node.right is None:
            return depth
        if node.left:
            queue.append((node.left, depth + 1))
        if node.right:
            queue.append((node.right, depth + 1))
    return 0

# ---- SHAPE 5: two trees in step -----
def is_same(a: TreeNode | None, b: TreeNode | None) → bool:
    """THREE base cases. The middle one is the crash if omitted."""
    if a is None and b is None:

```

```

        return True
    if a is None or b is None:
        return False
    return (a.val == b.val
            and is_same(a.left, b.left)
            and is_same(a.right, b.right))

def is_symmetric(root: TreeNode | None) → bool:
    """is_same with the arguments CROSSED. Not is_same(left, right)."""
    def mirror(a: TreeNode | None, b: TreeNode | None) → bool:
        if a is None and b is None:
            return True
        if a is None or b is None:
            return False
        return (a.val == b.val
                and mirror(a.left, b.right)
                and mirror(a.right, b.left))

    return root is None or mirror(root.left, root.right)

# ---- SHAPE 6: BST ordering – walk down, do not search -----

def bst_search(root: TreeNode | None, target: int) → TreeNode | None:
    node = root
    while node:
        if target == node.val:
            return node
        node = node.left if target < node.val else node.right
    return None

def lca_bst(root: TreeNode | None, p: int, q: int) → TreeNode | None:
    """The first SPLIT point is the answer. O(height), O(1)."""
    node = root
    while node:
        if p < node.val and q < node.val:
            node = node.left
        elif p > node.val and q > node.val:
            node = node.right
        else:
            return node
    return None

def kth_smallest(root: TreeNode | None, k: int) → int | None:
    """Inorder on a BST is sorted order. Iterative so it can STOP early:
    O(height + k), not O(n)."""
    stack: list[TreeNode] = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right
    return None

```



```

# ---- the two mock problems, worked -----

def largest_subtree_sum(root: TreeNode | None) → int:
    """Shape 2 – but with NO floor, because a subtree is not optional the
    way a path is. Noticing that is the whole problem."""
    best = float("-inf")

    def total(node: TreeNode | None) → int:
        nonlocal best
        if node is None:
            return 0
        s = node.val + total(node.left) + total(node.right)
        best = max(best, s)
        return s

    total(root)
    return int(best)

def from_list(values: list[int | None]) → TreeNode | None:
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); queue.append(node.right)
        i += 1
    return root

if __name__ == "__main__":
    t = from_list([3, 9, 20, None, None, 15, 7])

    print(height(t), count_nodes(t))           # 2 5
    print(diameter(t))                         # 3
    print(max_path_sum(from_list([-10, 9, 20, None, None, 15, 7]))) # 42
    print(is_balanced(t))                     # True
    print(good_nodes(from_list([3, 1, 4, 3, None, 1, 5]))) # 4
    print(root_to_leaf_paths(from_list([1, 2, 3]))) # [[1, 2], [1, 3]]
    print(level_order(t))                     # [[3], [9, 20], [15, 7]]
    print(min_depth(from_list([2, None, 3, None, 4]))) # 4
    print(is_same(t, from_list([3, 9, 20, None, None, 15, 7]))) # True
    print(is_symmetric(from_list([1, 2, 2, 3, 4, 4, 3]))) # True

    bst = from_list([6, 2, 8, 0, 4, 7, 9])
    print(is_bst(bst), bst_search(bst, 4))    # True TreeNode(4)
    print(lca_bst(bst, 2, 8), kth_smallest(bst, 3)) # TreeNode(6) 4

    print(largest_subtree_sum(from_list([1, -2, 3]))) # 3
    print(largest_subtree_sum(from_list([-1, -2, -3]))) # -1 ← not 0

    # the shapes that break things
    chain = None

```

```

for v in range(1, 3001):
    chain = TreeNode(v, chain)
    print(height(chain))
# 2999 – and a chain

```

6 What it costs

The whole phase in one table

problem family	time	extra space
any full traversal (DFS)	$O(n)$	$O(\text{height})$
any full traversal (BFS)	$O(n)$	$O(\text{width})$
return-value trick	$O(n)$	$O(\text{height})$
carry state down	$O(n)$	$O(\text{height})$
collect all paths	$O(n \cdot \text{height})$	$O(n \cdot \text{height})$ output
two-tree comparison	$O(\min(n, m))$	$O(\min \text{ height})$
subtree search, naive	$O(n \cdot m)$	$O(\text{height})$
subtree via serialisation	$O(n+m)$	$O(n+m)$
BST search/insert/delete	$O(\text{height})$	$O(1)$ iterative
BST k-th smallest	$O(\text{height}+k)$	$O(\text{height})$
validate BST	$O(n)$	$O(\text{height})$
balanced, one pass	$O(n)$	$O(\text{height})$
balanced, naive	$O(n^2)$	$O(\text{height})$
diameter, naive	$O(n^2)$	$O(\text{height})$
serialise/deserialise	$O(n)$	$O(n)$
reconstruct, with a map	$O(n)$	$O(n)$
reconstruct, naive	$O(n^2)$	$O(n^2)$

Three of those rows are the same mistake: naive balanced, naive diameter and naive reconstruction all recompute something already available. **The fix in all three cases is to return it instead.**

Depth-first against breadth-first

```

perfect tree, n = 1,000,000
  DFS stack    ~20 frames
  BFS queue    ~500,000 nodes      25,000× more

chain, n = 10,000
  DFS stack    10,000 → RecursionError
  BFS queue    1 node

```

Neither wins. DFS costs $O(\text{height})$, BFS costs $O(\text{width})$, and which is safe depends entirely on the shape. Say both halves.

The heights

n	balanced	random BST	degenerate
-----	-----	-----	-----
1,000	10	~20	999
1,000,000	20	~40	999,999
1,000,000,000	30	~60	—

Python’s recursion limit is 1,000, so any tree deeper than that breaks a recursive solution — and a BST built from sorted data is exactly that shape.

The three $O(n^2)$ s worth recognising on sight

naive is_balanced	height() called at every node
naive diameter	height() called at every node
naive reconstruction	inorder.index() plus slicing at every node
naive is_subtree	is_same() called at every node (unavoidable without serialising)
list.pop(0) anywhere	every pop shifts the whole list
string += in a loop	every concatenation copies the whole string

All six are correct and silent. Recognising the shape — *am I recomputing something I already have?* — is faster than remembering the list.

7 The traps

The five that account for almost every wrong answer in fourteen days.

Trap 1 — the missing `None` case

```
AttributeError: 'NoneType' object has no attribute 'val'
```

The only one that raises, which makes it the easiest. The base case is not defensive coding — it is the bottom of the recursion, and `None` means both “empty tree” and “no child here”.

Trap 2 — checking locally when the property is global

```
if node.left.val < node.val and node.right.val > node.val:    # BST — WRONG
if abs(height(root.left) - height(root.right)) ≤ 1:        # balance — WRONG
```

Both check one node against its children when the property must hold against **every ancestor** or at **every node**. `[10, 5, 15, null, null, 6, 20]` is the BST counter-example; keep it ready.

Trap 3 — returning the answer instead of what the parent needs

```
return max(left + right + node.val, node.val + max(left, right)) # WRONG
```

The parent cannot use a bent path. **What I return and what I record are different quantities**, and merging them gives an answer that grows with depth, silently.

Trap 4 — recomputing

Calling `height` inside something that is already computing heights. $O(n^2)$ on a chain, and it looks perfectly reasonable. **Ask: is the thing I need already being calculated?**

Trap 5 — reference instead of copy

```
out.append(trail) # → [[], [], []]
```

`trail[:]`. Every path problem that collects results has this trap.

Two more that are worth carrying:

The convention slip. Height by edges or by nodes; diameter by edges or nodes; the empty tree as `-1` or `0`. **Say which you are using before writing the base case**, and never mix them.

Assuming balance. $O(\text{height})$, not $O(\log n)$. Sorted insertion gives a chain, and sorted input is the common case.

8 In the interview

How it gets asked

- The mock: *“Two problems, no hints, talk as you go.”*
- The recognition test: a problem phrased so the shape is not obvious — *“count the nodes visible from the root”, “find the deepest leaves’ common ancestor”*.
- The efficiency probe: *“That is $O(n^2)$. Can you do it in one pass?”*
- The space probe: *“What is the space complexity, and how does it depend on the tree?”*
- The vocabulary check: *“What is the height of a single node?”*

What to say out loud, in the first ninety seconds

Whatever the problem:

1. **Ask the five questions.** “Is it a search tree? Two trees? About levels? Does a node need something only an ancestor knows? Do I need two different values — one to return and one to record?”
2. **Name the shape and the traversal together.** “The answer combines the children’s answers, so this is postorder — the five-line skeleton.”
3. **Say the base case as an identity.** “An empty tree returns zero, because I am summing” — or `-1` for heights, or `True` for a universal check.
4. **State both complexities immediately.** “`O(n)` time, one visit per node. `O(height)` space for the stack — about twenty frames on a balanced million-node tree, and a million on a chain.”
5. **Flag the depth risk if the constraints allow it.** “If the tree can be a chain and `n` is 10^5 , the recursion exceeds Python’s limit, so I would go iterative or say so.”
6. **Declare conventions.** “I will count edges, so a leaf has height zero — tell me if you want nodes.”

The follow-ups

“How do you decide the approach so quickly?” “I ask one question: **which way does the information travel?** If a node’s answer is built from its children’s, it is post-order — a five-line skeleton with two blanks. If a node needs something only an ancestor knows, like a depth or a permitted range, I carry it down as an argument, which also means there is nothing to undo. If the levels themselves are the answer, or I want the shallowest anything, it is breadth-first with a queue. If there are two trees, it is a two-node recursion with three base cases. And if it is a search tree, I do not search at all — I walk down and let each comparison discard a subtree. The sixth case is the one worth naming separately: when I need **two different things**, one for the answer and one for my parent, that is the return-value trick, and it is diameter, maximum path sum, balance and largest BST subtree all at once.”

“That is `O(n2)`. Can you do it in one pass?” “Yes, and the fix is the same every time: **I am recomputing something that is already being calculated.** The naive balance check calls `height` at every node, and `height` is itself a full walk of a subtree — so the total is the sum of all subtree sizes, which is `O(n log n)` balanced and `O(n2)` on a chain. But the walk already *has* both children’s heights at the moment it needs them. So instead of calling a separate function, I return the height up the recursion and either record the answer in an outer variable or return a sentinel meaning ‘already failed’. That single move fixes balance, diameter, maximum path sum and largest BST subtree — it is one technique, not four.”

“What is the space complexity?” “ $O(\text{height})$ for a depth-first solution and $O(\text{width})$ for a breadth-first one, and I would give both because they are opposite. On a **perfect** million-node tree, the recursion holds about twenty frames and a level-order queue holds half a million nodes — so depth-first is twenty-five thousand times cheaper. On a **chain**, it reverses completely: the queue holds one node and the recursion holds a million, which exceeds Python’s default limit of a thousand. So neither is universally better, and the honest answer names the shape it depends on. If the constraints allow a skewed tree with n up to 10^5 , I would either write it iteratively with an explicit stack or say the risk out loud rather than let it surprise us.”

“What is the height of a single node?” “Zero if I count edges, one if I count nodes, and I would state which before writing any code because mixing them is where the off-by-one bugs come from. The textbook convention is edges, so an empty tree is -1 and a leaf is 0 . LeetCode’s *Maximum Depth of Binary Tree* counts nodes, so an empty tree is 0 and a leaf is 1 . The code is identical apart from the base case. The same applies to diameter — edges or nodes, differing by one — and to whether `is_balanced` is checked at the root or at every node.”

“Give me a problem where breadth-first is genuinely better.” “Minimum depth. Depth-first has to explore a whole branch before it knows how deep it went, so on a tree whose left branch is ten thousand deep and whose right child is a leaf, it walks ten thousand nodes to discover the answer was two. Breadth-first returns at the first leaf it meets, which is three nodes. It is not just tidier — it is a different complexity in practice. The same argument applies to any ‘shortest’ or ‘shallowest’ question, and it generalises to shortest paths in an unweighted graph, where the first time you reach a node is guaranteed to be the shortest way. Conversely I would never use breadth-first for anything computed **from** the children, like height or subtree sums, because a level-order loop has nowhere natural to do the combining.”

“Which of these fourteen days do you actually use most?” “The return-value trick, by a distance. It is the answer to diameter, maximum path sum, balance checking, largest BST subtree and longest univalue path — five separate problems, one shape — and it is the one that turns a plausible $O(n^2)$ into $O(n)$. After that, carrying state down as an argument, because it covers every root-to-leaf question and BST validation and it removes the need for any undo. And the BST ordering rule, because recognising that you should walk rather than search is the difference between twenty comparisons and a million.”

A model answer

Asked, cold: *find the largest subtree sum in this tree.*

“Let me place it before I write anything. Five questions: is it a search tree — no, you said binary tree. Two trees — no. About levels — no. Does a node need something only an ancestor knows — no, a subtree’s sum depends entirely on what is below it. Do I need two different values, one to return and one to record — **yes**, and that is the shape.

Here is why. The **sum of a subtree** is exactly what my parent needs from me: their sum is their value plus my sum plus my sibling’s. So that is what I return. But the **answer** is the largest subtree sum anywhere, which is not something my parent can build from — it is a maximum over all nodes. So I record it separately, in a variable outside the recursion. That is the same shape as diameter and maximum path sum: **the value I return is not the value I am looking for**.

One thing I want to get right, because it is the difference between this and maximum path sum. In maximum path sum I floor each arm at zero — `max(0, arm)` — because a *path* is allowed to stop, so I am never obliged to include a negative branch. **A subtree is not optional**. If I am summing the subtree rooted at a node, everything below it is in the sum whether it helps or not. So there is no floor here, and putting one in would give the wrong answer on any tree with negative values.

The related detail: `best` starts at negative infinity, not zero. If every value in the tree is negative, the answer is the least-negative subtree, and starting at zero would return zero — which is not the sum of any subtree. Same reasoning as maximum path sum, different cause.

So: an empty tree returns zero — that is the identity for addition, which is why it is the right base case. Otherwise the sum is my value plus both children’s sums; I record it against the running maximum and return it.

`O(n)` time, one visit per node. `O(height)` space for the recursion — twenty frames on a balanced million-node tree, a million on a chain, so if this could be skewed and large I would go iterative.

Before I say I am done, my checklist: the `None` case returns the identity, yes. I am not checking anything locally that should be global, not applicable here. What I return differs from what I record, yes, deliberately. Am I recomputing anything — no, each sum is computed once and passed up. And I am not collecting lists, so there is no copy to worry about.”

9 Recall card

- **One question picks the shape: which way does the information travel?** Up, combined → simple postorder (the five-line skeleton). Up, but you need **TWO things** → the **return-value trick** (diameter, max path sum, balanced, largest BST subtree). **Down from an ancestor** → carry it as an argument (paths, depths, BST ranges — and no undo needed). **Across** → BFS with `len(queue)` captured first. **Between two trees** → three base cases. **A BST** → walk down, do not search.
- **The universal skeleton: handle `None`, recurse both children, combine.** The base case is the **identity** of the combining operation — `0` for sums, `-1` for max-of-heights, `True` for “all”.
- **The five bugs, and only the first one raises.** Missing `None` case · **local check where the property is global** (BST and balance) · returning the answer instead of what the parent needs · **recomputing what you already have** ($O(n^2)$ in balance, diameter and reconstruction) · **reference instead of copy**.
- **Time is the whole tree; space is the deepest path — but DFS is $O(\text{height})$ and BFS is $O(\text{width})$, in opposite directions.** Perfect million-node tree: 20 frames against 500,000 queued. Chain: 1 queued against `RecursionError`. $O(\text{height})$ is $O(\log n)$ only if balanced, and sorted input builds a chain.
- **Say the convention before the base case** (edges or nodes; empty tree `-1` or `0`), state both complexities as you write, and **run the five-bug checklist before declaring done** — four of the five are silent.

1 What this is, and why they ask it

Fifteen days ago “scale” was a word. Since then: [what scale means in numbers](#), [vertical against horizontal](#), [load balancers](#), [statelessness](#), [caching](#), [invalidation and eviction](#), [CDNs](#), [replication](#), [read replicas](#) and [lag](#), [sharding](#), [rebalancing](#) and [hot spots](#), [consistent hashing](#), [estimation](#), [capacity planning](#) and [single points of failure](#).

Three sentences. All of it collapses into **one ordered ladder** — a sequence of moves, cheapest and most reversible first — and the skill being tested is applying them **in order** rather than reaching for the most impressive one. Before any of it comes a **diagnosis**, because “falling over” has four different causes and three of the moves are useless for three of them. And every move has a price, so the answer that lands is the one that names the price at the same time as the move.

They ask “*walk me through fixing it*” because it is the only question in the phase with a wrong answer that sounds impressive: “*I’d shard the database.*” Reaching for the most expensive, least reversible move first — before caching, before replicas, before checking whether one machine still has headroom — is the specific failure the question is built to catch.

2 The story

The tap in the upstairs bathroom had gone to a trickle and Fareed’s landlord had already had two people look at it.

The first one had been there four hours and had taken up part of the floor to get at the pipe, found nothing wrong, and put the floor back badly. The second had wanted to replace the pump on the roof, which was eighteen thousand rupees, and had said the pressure was low.

The third man was older and did none of that. He stood in the bathroom for about a minute and asked three questions.

Is it only this tap, or all of them? Only this one.

Is it bad all day, or only in the morning? All day.

Was it sudden, or has it got worse slowly? Slowly, over about a year.

Then he unscrewed the little mesh at the end of the tap with his fingers, held it up to the light, and showed Fareed a disc of grey-white scale with about a fifth of the holes still open.

He cleaned it under the other tap with a pin, screwed it back on, and the water came out properly. Eleven minutes.

Fareed asked how he had known.

The man said he had not known. He had **ordered** it. There were about six things it could be, and he always did them in the same order: cheapest and quickest first, most expensive and most destructive last. The mesh takes one minute and costs nothing. The tap washer takes ten minutes and costs twenty rupees. The stopcock takes half an hour. The pipe means taking up the floor. The pump means eighteen thousand rupees.

He said the two men before him had both started at the wrong end, and the second one might even have been right that the pressure was low — but you do not find that out by buying a pump, you find it out by checking the cheap things first and seeing them all pass.

And he made a point of the three questions. He said “only this tap” ruled out four of the six before he touched anything. If it had been every tap in the flat, he would not have looked at the mesh at all.

3 The idea in plain English

The plumber has a diagnostic followed by an ordered ladder, and that is the whole answer to “*walk me through fixing it*”.

- The three questions are the **diagnosis**, and they eliminate most of the ladder before you start.
- “Cheapest and quickest first, most expensive and most destructive last” is the **order**.
- Buying the pump first is **sharding first**, and it is the wrong answer even when it is eventually needed.

Step zero: diagnose

Never start fixing before you know which of these it is. They have different answers and three of them are not helped by more machines at all.

1. IS ONE REQUEST SLOW, or are there TOO MANY?
 - one request slow → profiling, indexing, N+1 queries, a missing cache
 - MORE MACHINES DO NOTHING
 - too many → the ladder below
2. WHICH RESOURCE is saturated?
 - CPU → more app servers, or cheaper code
 - memory → bigger machines, or a leak
 - disk I/O → indexes, or a bigger buffer pool, or SSDs
 - network → a CDN, smaller responses
 - connections → a pooler, not more servers
 - a LOCK → none of the above; it is a contention problem
3. IS IT READS or WRITES?
 - reads → cache, CDN, replicas
 - writes → only sharding scales writes
4. IS IT STEADY or a SPIKE?
 - steady → capacity
 - spike → a queue, and autoscaling will not react in time

Say “throughput scales horizontally; latency does not” — that one sentence disposes of case 1, which is the most common misdiagnosis.

The ladder, in order

Each rung is cheaper, faster and more reversible than the one below it. **Go down only when the one above is exhausted.**

0. MEASURE	find the actual bottleneck	minutes	free
1. FIX THE OBVIOUS	an index, an N+1 query, a missing timeout	hours	free
2. SCALE UP	a bigger machine	minutes	linear cost
3. CACHE	90% hit rate = 10x fewer reads reach the database	hours	cheap
4. CDN	removes ~95% of the BYTES	hours	cheap
5. STATELESS + LOAD BALANCER	many app servers	days	linear
6. READ REPLICAS	scales reads; costs lag	days	linear
7. QUEUE / ASYNC	move work off the request path	days	cheap
8. SHARD	the ONLY thing that scales writes	months	permanent tax
9. MULTI-REGION	latency and disaster recovery	months	very expensive

Rungs 3 and 4 are where the leverage is. A ninety percent cache hit rate removes ten times more load than an extra replica, and it takes an afternoon.

Rung 8 is the pump. It is the only thing that scales writes, and its cost is not the migration — it is that every feature built afterwards lives inside the shard key you chose.

What each rung actually buys

CACHE (90% hit)	6,000 reads/s → 600 reach the DB	10x
CACHE (95%)	6,000 → 300	20x
CDN	removes ~95% of BYTES, ~75% of requests	
REPLICAS	reads scale linearly; writes not at all	
STATELESS + LB	app tier scales linearly; availability too	
QUEUE	absorbs a spike INSTANTLY; autoscaling cannot	
SHARD	writes scale linearly; joins and transactions gone	

And the ceiling on rung 6, which is the moment you must go to 8: **every replica applies every write**, so at 4,000 writes a second on a 5,000-writes-a-second machine, each replica has twenty percent of itself left for reads, and the eleventh replica adds nothing.

The price of every move

Naming the price at the same time as the move is what makes the answer sound experienced.

MOVE	PRICE
Scale up	a ceiling, downtime to resize, and still one machine — no availability gain
Cache	staleness , invalidation, and a stampede when a hot key expires
CDN	purges are slow; nothing personalised ; cache-key explosion from query strings
Stateless	a shared session store becomes a hot dependency on every request
Replicas	lag — a user may not see their own write
Queue	the caller no longer knows the result; at-least-once delivery
Shard	no cross-shard joins, transactions or unique constraints — permanently
Multi-region	writes cannot be synchronous; conflicts or data loss

The five numbers to have ready

86,400 s/day → round to 100,000	1 M/day ≈ 10 QPS, 1 B/day ≈ 10,000 QPS
one app server	~1,000 QPS of real work
one relational DB	~10,000 reads/s, ~5,000 writes/s
Redis	~100,000 ops/s
cache hit rate	~90% on a well-chosen key
peak factor	×3 (×100 for a scheduled event)
replication	×3 for storage
utilisation target	65% — because queueing delay is $u/(1-u)$

Those eight lines answer most numeric follow-ups.

The five sentences that carry the phase

1. **“Throughput scales horizontally; latency does not.”** — the diagnosis.
2. **“Every replica applies every write.”** — why replicas have a ceiling and sharding is the only answer for writes.
3. **“A cache miss must make the answer slow, never wrong.”** — what may and may not be cached.
4. **“Redundancy without tested failover is a second thing that might also be broken.”** — availability.
5. **“Sharding’s cost is not the migration; it is every feature afterwards.”** — why it is last.

The mock: *“falling over at 10,000 QPS”*

The structure of a good answer:

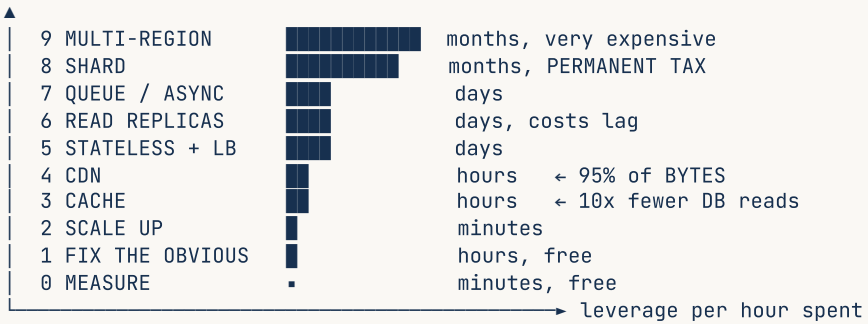
- | | |
|-------------------|--|
| 1. DIAGNOSE | "Slow requests or too many? Which resource? Reads or writes? Steady or a spike?"
— 30 seconds |
| 2. MEASURE | "What is the read-to-write ratio and where is the time going?" |
| 3. QUICK WINS | "Any missing index or N+1 query? Any missing timeout?" |
| 4. CACHE | "90% hit rate takes 9,500 reads/s down to 950." |
| 5. CDN | "If there is media, that removes most of the bytes before anything else runs." |
| 6. SCALE THE TIER | "Stateless app servers behind a load balancer: 10,000 peak at ~1,000 each and 65% utilisation is about 15 machines." |
| 7. REPLICAS | "For the residual reads. Cost: lag, and read-your-own-writes." |
| 8. ASYNC | "Move anything not needed for the response into a queue." |
| 9. ONLY THEN | "If writes are the ceiling – over ~5,000/s – shard, and here is the key and what it costs." |

Steps 1 to 3 before any architecture. That order is the answer.

4 The picture

The ladder, with the leverage marked.

cost / irreversibility



GO DOWN ONLY WHEN THE RUNG ABOVE IS EXHAUSTED.

"I'd shard it" as a first answer is buying the pump before checking the mesh.

The diagnosis, which comes before the ladder:

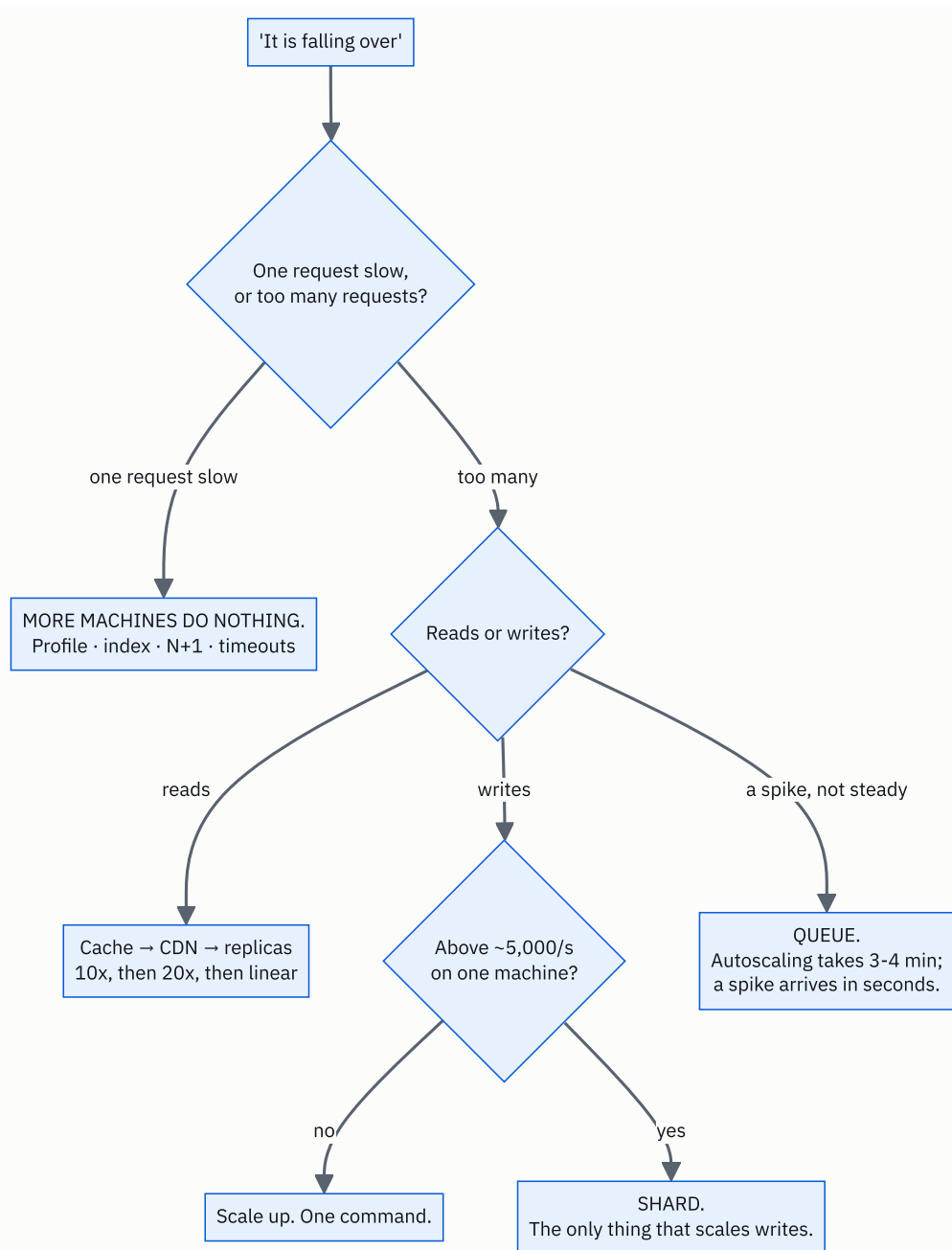
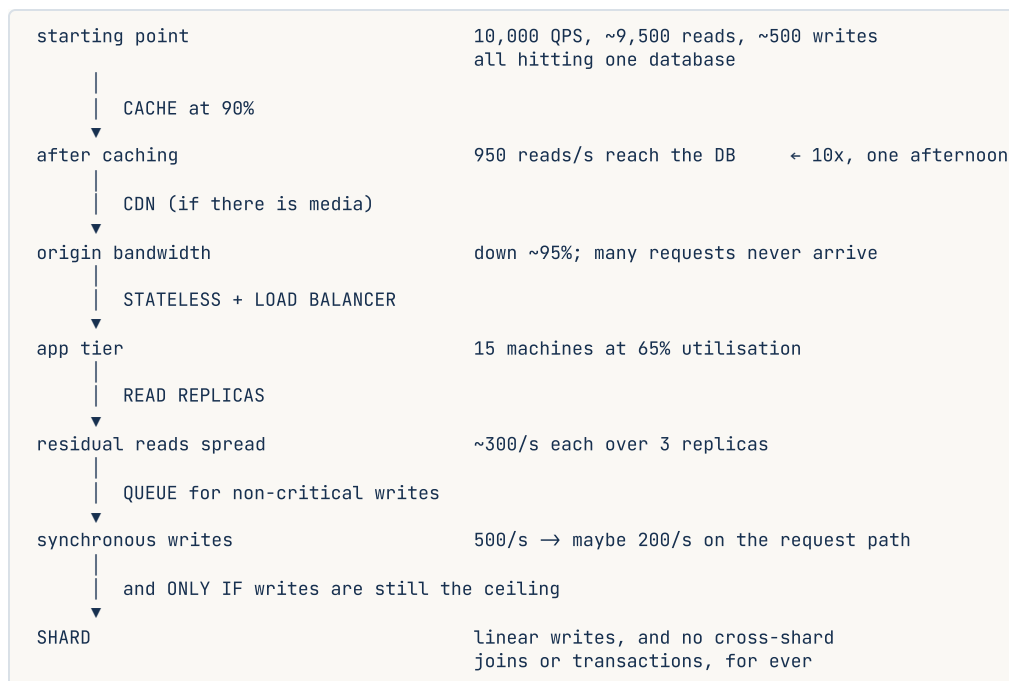


FIGURE 112.2

What each rung removes, on the same 10,000 QPS:



The plumber's order, as the same picture:

the mesh	1 min	free	← check FIRST
the washer	10 min	₹20	
the stopcock	30 min	₹200	
the pipe	4 hours	the floor comes up	
the pump	1 day	₹18,000	← the second plumber started HERE

and the three questions eliminated four rungs before he touched anything.

5 How it actually works

The full worked mock

“This service is falling over at 10,000 QPS. Walk me through fixing it.”

Minute 0–1: diagnose.

“Four questions before I change anything. **Is one request slow, or are there too many?** If a single query takes eight seconds, adding machines gives me ten eight-second queries — throughput scales horizontally, latency does not. **Which resource is saturated** — CPU, memory, disk, network, connections, or a lock? A lock is a contention problem and none of the usual moves help. **Reads or writes?** And **is it steady or a spike?**

I will assume: too many requests, CPU and database-bound, read-heavy at about twenty to one, and steady with an evening peak.”

Minute 1–2: measure and take the free wins.

“Before architecture: is there a missing index, or an N + 1 query pattern, or a call with no timeout? Those are hours of work and they routinely give more than a machine would. A missing timeout in particular is how a slow dependency takes the whole service down.”

Minute 2–4: the read moves.

“Ten thousand QPS at twenty-to-one is about 9,500 reads and 500 writes. **Cache first** — a well-chosen key gets around ninety percent, which takes 9,500 reads down to 950 reaching the database, a tenfold reduction for an afternoon’s work. I would use cache-aside so that a cache failure makes the system slow rather than broken, a TTL on every key as a backstop, and per-key locking plus jittered TTLs so a hot key expiring does not send a thousand identical queries at once.

Then the CDN, if any of this is media or static. That removes roughly ninety-five percent of the bytes and a large share of the requests before they reach my servers at all — for a media-heavy product it is the single biggest move and people skip it.”

Minute 4–6: the tier.

“**Make the app tier stateless** — sessions into Redis, not process memory — and put a load balancer in front with health checks. Then the count: 10,000 peak at roughly 1,000 QPS per server is ten machines at full utilisation, which is not a thing you run, so about fifteen at sixty-five percent, plus one for N+1. Sixty-five because queueing delay grows as $u/(1-u)$ — at ninety percent utilisation requests wait nine times longer than at fifty.

Read replicas for the residual reads, with the price stated: replication lag means a user may not see their own write, so I route a user’s reads to the primary for thirty seconds after they write.”

Minute 6–8: writes and asynchrony.

“Five hundred writes a second is comfortable for one database, so I am **not** sharding. What I would do is move anything not needed for the response — notifications, analytics, search indexing, thumbnails — onto a **queue**. That shrinks the request path and gives me something that absorbs a spike instantly, which auto-scaling cannot: the metric window plus boot plus warm-up plus health checks is three to four minutes, and a spike arrives in ten seconds.”

Minute 8–10: what would change my answer.

“If the write rate were 8,000 rather than 500, none of that helps and I would be sharding — by `user_id`, hashed, and the cost is no cross-shard joins, no cross-shard transactions and no global unique constraints, for ever. If it were a spike rather than steady load, I would go straight to the queue. And if one *request* were slow rather than there being many, I would be profiling and none of this ladder applies.”

That is the shape. Diagnosis, free wins, reads, tier, writes, and an explicit statement of what would change the answer.

The four systems worth being able to scale

Interviewers reuse a small set. **Know the shape of each.**

A READ-HEAVY FEED	ratio 100:1	cache + CDN + replicas. Shard late. Watch: celebrity fan-out.
A CHAT SYSTEM	ratio ~1:1	caching barely helps. Shard by conversation_id early. The constraint is CONNECTIONS, not CPU.
AN E-COMMERCE CHECKOUT	ratio 200:1	browse is cacheable; checkout is not. Inventory needs strong consistency: cache the DISPLAY, never the DECISION.
AN ANALYTICS PIPELINE	write-heavy	queue + batch + a columnar store. Do not try to serve this from the OLTP DB.

Naming which of these the prompt is, in the first minute, does a lot of the work.

The four things that break at each stage

1 → many app servers	connections (need a pooler) · in-process state · observability
no cache → cache	staleness · stampede on expiry · the read-write race
one DB → replicas	read-your-own-writes · monotonic reads · lag spikes
one DB → shards	joins · transactions · unique constraints · hot keys · rebalancing
one region → many	synchronous replication is impossible · conflicts

Each of those lists is a follow-up waiting to be asked.

6 The numbers

The conversions

86,400 s/day	→ round to 100,000, and say you are rounding
1 million/day	≈ 10 QPS
100 million/day	≈ 1,000 QPS
1 billion/day	≈ 10,000 QPS
1 KB × 1 billion	= 1 TB
1 MB × 1 billion	= 1 PB

Machine capacities

app server, real work	~1,000 QPS
relational DB, reads	~10,000/s indexed
relational DB, writes	~5,000/s
Redis	~100,000 ops/s
network link	~1 GB/s
SSD	~500 MB/s, ~50,000 IOPS
connections per app server	~20 to the DB (so 50 servers = 1,000 connections)

What each move buys, quantified

move	before	after	factor
-----	-----	-----	-----
cache 90%	6,000 reads/s	600	10x
cache 95%	6,000	300	20x
CDN	200 GB/hr	10 GB/hr	20x on bytes
3 read replicas	6,000/s on 1	2,000/s each	3x
shard × 8	10,000 w/s	1,250/s each	8x

The ceilings

vertical scaling	cost curve bends at ~32-64 vCPU; hard stop at ~4 TB RAM or ~50,000 IOPS
read replicas	every replica applies every write: at 4,000 w/s on a 5,000 w/s machine, 20% left for reads
one database	~5,000 writes/s
one app server	~1,000 QPS
autoscaling	3-4 minutes to react
DNS failover	bounded below by the TTL

Availability

99%	3.65 days/year	99.99%	52.6 minutes
99.9%	8.77 hours	99.999%	5.26 minutes
in series: six components at ~99.9% each → ~99.4% (~2 days/year)			
in parallel: two at 99.9% → 99.9999% (~32 s) – IF independent			
with 10% correlation → ~99.99% (~53 min)			

Utilisation

utilisation	queueing delay	so, for 10,000 QPS at 1,000/server
-----	-----	-----
50%	1×	20 machines
65%	1.9×	15 machines ← the target
80%	4×	13
90%	9×	11
100%	unbounded	10 (and unusable)

Sharding thresholds

shard when	write rate > ~5,000/s and vertical scaling is exhausted OR the working set exceeds the largest machine (~4 TB)
do NOT shard for read load	→ that is caching and replicas
8 → 9 machines	→ plain modulo moves ~89%, logical shards or consistent hashing ~11%

7 The trade-offs

The order is the answer, and it is defensible

Cheapest and most reversible first. Not because the expensive moves are wrong, but because:

- The cheap ones are often **enough** — a ninety percent cache hit rate removes ten times more load than another replica.
- They are **reversible**. A cache can be turned off; a shard key cannot be un-chosen.
- They **buy time to learn**. You will know much more about the workload in three months, and the shard key you pick then will be better.

The one exception: if the diagnosis says writes are the ceiling, the read moves genuinely do not help and going down the ladder in order would be theatre. **Say the diagnosis, then jump.**

Every move trades consistency for scale

cache	stale reads
CDN	stale content, slow purges
replicas	read-your-own-writes violations
queue	the caller does not know the result; at-least-once delivery
shard	no cross-shard transactions
multi-region	no synchronous writes

The pattern is one sentence: every rung buys throughput with correctness. The design question is which correctness you can afford to give up — and the rule from caching generalises: **the display may be stale; the decision may not.**

Where candidates go wrong

- **Sharding first.** The signature mistake, and it is why the question exists.
- **Not diagnosing.** Adding machines to a latency problem.
- **Forgetting the CDN.** For a media product it is the largest single move and it is invisible on most diagrams.
- **Treating autoscaling as headroom.** Three to four minutes of lag against a ten-second spike.
- **Not naming the price.** “Add read replicas” without “and a user may not see their own write” is half an answer.
- **Ignoring availability.** Every component added in series *reduces* availability, and a scaling answer that has quietly made the system less reliable is not a good answer.

Where the whole ladder stops working

- **Write-heavy at genuine scale.** Sharding is the last rung and it has a floor: eventually the coordination costs more than the parallelism buys.
 - **Strong consistency across regions.** The speed of light says no. You choose availability or consistency, which is [day 114](#).
 - **A single hot key.** No amount of sharding splits one key. Cache it, replicate it, or design a composite key so it cannot grow.
 - **Fan-out.** One celebrity post is one write to you and fifty million to the feed system. It is invisible in user-facing numbers and it is the biggest trap in estimating a social product.
-

8 In the interview

How it gets asked

- The mock: *“This service is falling over at 10,000 QPS. Walk me through fixing it.”*
- The trap version: *“Would you shard?”* — asked early, to see whether you say yes.
- The diagnosis probe: *“How would you know what to fix?”*
- The price probe: *“What does that cost you?”*
- The limit probe: *“When does that stop working?”*

What to say out loud, in the first ninety seconds

1. **Refuse to fix before diagnosing.** “Four questions first: is one request slow or are there too many; which resource is saturated; reads or writes; steady or a spike. Three of the four change the answer completely.”
2. **Dispose of the latency case.** “If a single request is slow, more machines do nothing — throughput scales horizontally, latency does not. That is profiling, indexing and timeouts.”
3. **Take the free wins before architecture.** “A missing index or an $N+1$ query is hours of work and often worth more than a machine.”
4. **Cache, with the number.** “At twenty-to-one reads, a ninety percent hit rate takes 9,500 reads a second down to 950 — tenfold, in an afternoon.”
5. **Give the tier count properly.** “Stateless behind a load balancer: 10,000 at a thousand each is ten machines at full utilisation, so about fifteen at sixty-five percent, because queueing delay grows as $\frac{u}{(1-u)}$.”

6. **Say when you would shard, and refuse until then.** “Five hundred writes a second is comfortable for one database, so I am not sharding. I would shard above about five thousand writes a second — and the cost is no cross-shard joins or transactions, permanently.”

The follow-ups

“**Would you shard?**” “Not yet, and I would want to say why rather than just no. Sharding is the only thing that scales **writes**, so the question is whether writes are my ceiling. At 10,000 QPS with a twenty-to-one read ratio that is about five hundred writes a second, and one relational database handles several thousand — so writes are nowhere near the limit and sharding would solve a problem I do not have while creating several I would keep. Its cost is not the migration; it is that **every feature built afterwards lives inside the shard key I choose today**: no joins across shards, no transactions across shards, no global unique constraints, and auto-increment ids that collide. I would shard when the write rate passes about five thousand a second and vertical scaling is exhausted, or when the working set no longer fits the largest machine. Until then the read moves — cache, CDN, replicas — do the work, and they are reversible.”

“**How would you know what to fix?**” “By diagnosing before prescribing, in four questions. **One request slow, or too many?** — because horizontal scaling only answers the second; if a single query takes eight seconds, ten machines give me ten eight-second queries. **Which resource is saturated?** — CPU means more servers or cheaper code, memory means bigger machines or a leak, disk means indexes or a bigger buffer pool, network means a CDN, connections mean a pooler rather than more servers, and a **lock** means none of those, because it is a contention problem. **Reads or writes?** — because reads have three cheap answers and writes have one expensive one. And **steady or a spike?** — because a spike needs a queue, and autoscaling takes three to four minutes while a spike arrives in ten seconds. Those four questions eliminate most of the ladder before I touch anything.”

“**What does that cost you?**” “Every rung buys throughput with correctness, and I would name the price with the move. **Caching** costs staleness, an invalidation problem, and a stampede when a hot key expires — so a TTL on everything, delete rather than update on write, and per-key locking. **A CDN** costs slow purges and cannot serve anything personalised. **Statelessness** turns the session store into a hot dependency on every request. **Read replicas** cost lag, so a user may not see their own write — I route their reads to the primary for thirty seconds afterwards. **A queue** means the caller no longer knows the result and delivery is at-least-once, so consumers must be idempotent. **Sharding** costs joins, transactions and unique constraints permanently. And

there is a cost people forget: every component added **in series** reduces availability — six at 99.9 percent gives about 99.4, which is two days a year — so a scaling change can quietly make the system less reliable.”

“**When does that stop working?**” “Each rung has a specific ceiling. **Vertical scaling**: the price per unit of power bends at around thirty-two to sixty-four vCPU, and there is a hard stop around four terabytes of memory. **Read replicas**: every replica applies every write, so at four thousand writes a second on a five-thousand machine each replica has twenty percent of itself left for reads and the eleventh adds nothing — that is the moment sharding becomes the only option. **Caching**: it stops helping when the working set does not fit or when access is uniform, and it cannot help a write-heavy workload at all. **Autoscaling**: three to four minutes of lag, so it saves money in the trough and not capacity at the peak. And **sharding** itself has a floor — a single hot key cannot be split, because every row with that key value is on one machine by definition, so that has to be designed away with a composite key rather than fixed later.”

“**Walk me through it for a chat system instead.**” “That changes the answer substantially, and it is worth saying why in one sentence: **messaging is roughly one-to-one reads to writes**, so caching and read replicas — the two cheapest rungs — barely help. The write rate is the load. So I would shard early, by `conversation_id`, so that a conversation’s messages live together and the common query stays on one machine. And the binding constraint is usually not CPU at all — it is **connections**: a billion users with a persistent connection at around a million per tuned server is a thousand machines just holding sockets open. That is memory per connection, not requests per second, and it means the design conversation is about connection handling and a pub/sub layer for delivery, not about caching.”

“**You have added all of that. Is the system more available or less?**” “Less, unless I have been deliberate about it, and I am glad you asked because it is the thing a scaling answer usually breaks. Components **in series** multiply their availabilities, so every dependency I add to the critical path costs me — six at 99.9 percent gives about 99.4, which is two days a year rather than nine hours. The mitigations are specific. Make the cache **optional** so a cache failure is slow rather than fatal, which removes it from the product entirely. Run at least two of every stateless thing, behind health checks, in different zones. And for each new dependency ask whether the system can **degrade** past it instead of failing — recommendations can show a default list, search can fall back to browse. That distinction, degradation rather than redundancy, is usually cheaper and it is what keeps the availability arithmetic from going backwards while I make the system faster.”

A model answer

Asked: *this service is falling over at 10,000 QPS. Walk me through fixing it.*

“I would not start fixing until I have diagnosed, because three of the four possible causes are not helped by the things people reach for.

Four questions. Is **one request slow**, or are there **too many requests**? Because throughput scales horizontally and latency does not — if a single query takes eight seconds, ten machines give me ten eight-second queries. **Which resource is saturated** — CPU, memory, disk, network, connections, or a lock? A lock is contention and none of the usual moves touch it. **Reads or writes?** And **steady load or a spike?**

Let me assume the common case: too many requests, read-heavy at about twenty to one, steady with an evening peak. So roughly 9,500 reads and 500 writes a second.

Before any architecture, the free wins: a missing index, an $N + 1$ query pattern, a call with no timeout. Those take hours and routinely beat a machine — and a missing timeout is how a slow dependency takes the whole service down with it.

Then cache, because it has the most leverage per hour spent. A well-chosen key gets around ninety percent, which takes 9,500 reads a second down to **950** reaching the database — tenfold, in an afternoon. Cache-aside, so a cache failure makes the system slow rather than broken; a TTL on every key as the backstop for invalidation bugs; and per-key locking with jittered TTLs, so a popular key expiring does not send a thousand identical queries at once.

Then the CDN, if any of this is media or static assets. That is roughly ninety-five percent of the bytes gone before they reach me at all, and for a media-heavy product it is the biggest single move — and the one people skip.

Then the tier. Make it stateless — sessions in Redis, not process memory — behind a load balancer with health checks. The count: 10,000 at roughly a thousand QPS per server is ten machines at full utilisation, which is not a thing you run, because queueing delay grows as utilisation over one minus utilisation — at ninety percent, requests wait nine times longer than at fifty. So about **fifteen machines at sixty-five percent**, plus one for $N + 1$, and I would check the failure case: losing one leaves the rest at seventy percent, which is fine.

Read replicas for the residual reads, and I would name the price: replication lag means a user may not see their own write, so I route a user's reads to the primary for thirty seconds after they write.

A **queue** for anything not needed in the response — notifications, analytics, search indexing. That shortens the request path and gives me the only thing that absorbs a spike instantly; autoscaling takes three to four minutes and a spike arrives in ten seconds.

And I would not shard. Five hundred writes a second is comfortable for one database. I would shard when writes pass about five thousand a second, or when the working set exceeds the largest machine — and I would say plainly that its cost is not the migration but the permanent one: no cross-shard joins, no cross-shard transactions, no global unique constraints, and every future feature living inside the shard key I pick today.

One last thing I would check, because scaling answers usually break it: every component I have added sits **in series** on the critical path, and availabilities multiply. Six components at 99.9 percent give about 99.4 — two days a year. So the cache must be **optional** rather than required, and for each new dependency I would ask whether the system can degrade past it rather than fail.”

9 Recall card

- **Diagnose before prescribing, in four questions: one slow request or too many · which resource is saturated · reads or writes · steady or a spike.** *“Throughput scales horizontally; latency does not”* disposes of the most common misdiagnosis, and a **lock** is contention that none of the moves fix.
- **The ladder, cheapest and most reversible first: measure → fix the obvious → scale up → CACHE → CDN → stateless + LB → read replicas → queue → SHARD → multi-region.** Rungs 3 and 4 hold the leverage: a **90% hit rate is 10×** and a **CDN removes ~95% of the bytes**, both in an afternoon.
- **“I’d shard it” as a first answer is the failure the question exists to catch.** Sharding is the only thing that scales **writes**, so shard only above **~5,000 writes/s** or when the working set exceeds the biggest machine — and its cost is **not the migration but every feature afterwards**.
- **Name the price with every move:** cache → staleness and stampedes · CDN → slow purges, nothing personalised · stateless → a hot shared store · replicas → **lag and read-your-own-writes** · queue → at-least-once, caller blind · shard → **no joins, transactions or unique constraints, permanently**.

- The numbers: 1 billion/day $\approx$ 10,000 QPS · app server $\sim$ 1,000 QPS · DB $\sim$ 10,000 reads and $\sim$ 5,000 writes · Redis $\sim$ 100,000 ops/s · peak $\times 3$ · size for 65% utilisation because delay is $u/(1-u)$ · every replica applies every write. And check the direction you moved availability: **components in series multiply**, so six at 99.9% is $\sim$ 99.4% — make the cache **optional**, and prefer **degradation** over redundancy.

DSA topic: Trees revision and mock round **System design topic:** Scaling revision and interview questions

Code these, in this order

This is a mock round. **Twenty-five minutes each, out loud, no references.** Before writing anything, ask the five questions and name the shape. If naming the shape takes more than thirty seconds, that — not the code — is what to practise.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Count Good Nodes in Binary Tree	LeetCode 1448 (Medium)	Recognising “carry state down” from an unfamiliar phrasing.
2	Maximum Level Sum of a Binary Tree	LeetCode 1161 (Medium)	Recognising a level problem, and the boundary capture.
3	Sum Root to Leaf Numbers	LeetCode 129 (Medium)	State down again, and the leaf test.
4	Binary Tree Maximum Path Sum	LeetCode 124 (Hard)	The return-value trick, cold, with the floor and the <code>-inf</code> .

Before each problem, say the five questions

Search tree? Two trees? Levels? Does a node need something an ancestor knows? Do I need two different values? Time yourself. Forty seconds or less.

On problem 4, run the checklist before declaring done

`None` case with the right identity · local versus global · return differs from record · am I recomputing · is it a copy. Ten seconds, and it catches four silent bugs.

After all four, write the shape table from memory

Six rows: information direction, shape name, traversal, two examples each. If any row is missing, that is the day to re-read.

The one-question drill

1. State the question that picks the shape.
2. Give the four directions information can travel and the six shapes they produce.

3. For each shape, give the traversal and two problems.
4. Say which shape is the most used, and why.

The skeleton drill

1. Write the five-line skeleton from memory.
2. Say what determines the base case, in one word.
3. Give the base case for: height, count, sum, “do all nodes satisfy P”, “does any node satisfy P”.
4. Say which of the six shapes are exactly this skeleton with different blanks.

The recognition drill

For each phrasing, name the shape in under ten seconds:

1. “the longest path between any two nodes”
2. “every root-to-leaf path summing to X”
3. “the value of each level’s largest node”
4. “is this tree a mirror of itself”
5. “the k-th smallest value”
6. “the deepest leaves’ common ancestor”
7. “count nodes greater than everything above them”
8. “the shallowest leaf”

The return-versus-record drill

1. State the technique in one sentence.
2. Fill in the table for diameter, max path sum, balanced, longest univalue path and largest BST subtree.
3. Say why the parent cannot use the recorded value.
4. Say what happens if you return the recorded value instead.

The five-bugs drill

1. Name all five bugs.
2. Say which one raises and which four are silent.
3. For each, give the specific wrong output it produces.
4. Run the checklist on a solution you have already written.

The complexity drill

1. Fill in the complexity table for all fourteen problem families.

2. Identify the three rows that are the same underlying mistake.
3. State the DFS and BFS space costs and the two shapes where each is disastrous.
4. Give the heights for balanced, random and degenerate trees at a million nodes.

The convention drill

1. Give both height conventions and their base cases.
2. Do the same for diameter.
3. Say the sentence you would use in an interview to settle it.
4. Mix them deliberately on a five-node tree and record how wrong the answer is.

The mock drill

Do two of these cold, timed, out loud, with no notes:

- Path Sum III (LeetCode 437)
- All Nodes Distance K in Binary Tree (LeetCode 863)
- Vertical Order Traversal (LeetCode 987)
- Delete Nodes and Return Forest (LeetCode 1110)
- Maximum Difference Between Node and Ancestor (LeetCode 1026)

Afterwards write down: how long the first minute took, whether you named the shape before coding, and whether you stated both complexities without being asked.

The diagnosis drill

1. Name the four diagnostic questions in order.
2. For each, say what the answer eliminates.
3. Say the sentence that disposes of the latency case.
4. Name the resource whose saturation none of the ladder fixes.

The ladder drill

1. List all ten rungs in order.
2. For each, give the time cost and the leverage.
3. Say which two rungs hold most of the leverage, and why.
4. Say the rule for when to go down a rung.

The price drill

For each move, name the price without looking:

cache · CDN · statelessness · read replicas · queue · sharding · multi-region · scaling up

Then say which single sentence describes the pattern across all of them.

The ceiling drill

State the ceiling for each:

1. Vertical scaling — two numbers.
2. Read replicas — and the sentence that explains it.
3. One relational database, reads and writes.
4. One app server.
5. Autoscaling reaction time.
6. DNS-based failover.

The numbers drill

From memory:

1. The three requests-per-day to QPS conversions.
2. Six machine capacities.
3. The utilisation table and the target, with the reason.
4. Downtime per year at five availability levels.
5. Series and parallel availability for six and two components.
6. What plain modulo costs at $8 \rightarrow 9$ machines, and what the fix costs.

The four-systems drill

For each, give the read:write ratio, the binding constraint, and the first two moves:

1. A read-heavy feed.
2. A chat system.
3. An e-commerce checkout.
4. An analytics pipeline.

The mock drill

Do the full ten-minute walkthrough out loud for each, with a timer:

1. *“This service is falling over at 10,000 QPS.”*
2. *“Same, but it is a chat system.”*
3. *“Same, but the load is a spike at 10 a.m. every day.”*
4. *“Same, but writes are 8,000 a second.”*

For each, say at which rung you stop and why.

The availability drill

1. Compute the availability of six components in series at 99.9% each.
 2. Say what making one of them optional is worth, in hours per year.
 3. Compute two parallel copies at 99.9%, and again with 10% correlation.
 4. Say why a scaling change can make a system less available, and what to do about it.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* For each: the five questions asked, the shape named before any code, the base case justified as an identity, both complexities stated unprompted, the depth risk flagged if the constraints allow it, and the five-bug checklist run before declaring done.
 2. *This service is falling over at 10,000 QPS. Walk me through fixing it.* The four diagnostic questions before anything, the free wins before architecture, cache with the tenfold number, the CDN if there is media, the tier count at 65 percent with the queueing reason, replicas with the lag price, a queue for the asynchronous work, and an explicit refusal to shard with the threshold that would change it.
 3. *Would you shard?* Not yet, and why — writes are the ceiling that matters, the threshold, and the permanent cost stated in three specific losses.
-

Before you move on

- [] I can state the one question that picks the tree shape.
- [] I can write the six-row shape table from memory.
- [] I can write the five-line skeleton and justify the base case as an identity.
- [] I can name the shape for eight unfamiliar phrasings in under ten seconds each.
- [] I can state the return-versus-record sentence and fill in its table.
- [] I can name the five bugs and say which four are silent.
- [] I run the checklist before saying a solution is done.
- [] I can fill in the complexity table and spot the three rows that share a mistake.
- [] I state DFS and BFS space costs together, with the shapes where each fails.

- [] I declare the height convention before writing a base case.
- [] I did two unseen problems cold and named the shape in the first thirty seconds.
- [] I can name the four diagnostic questions and what each eliminates.
- [] I can list all ten rungs of the ladder in order.
- [] I can name the price of every move without looking.
- [] I can state the ceiling of every rung.
- [] I know why replicas stop helping, in one sentence.
- [] I can produce all the numbers from memory.
- [] I can walk through the 10,000 QPS mock in ten minutes.
- [] I refuse to shard first, and can say exactly when I would.
- [] I check whether my scaling changes made availability worse.
- [] I answered all three questions above out loud.

PART XIII

Heaps and priority queues

Days 113 to 119 · 7 chapters

Alongside, on the system design track:

Distributed systems core

113 The heap: a tree stored in an array

114 Push, pop, and heapify

115 Python's heapq, and the min-heap-only problem

116 Top K problems

117 Merging K sorted lists

118 Two heaps: the running median

119 Heaps revision and mock round

The heap: a tree stored in an array

DATA STRUCTURES AND ALGORITHMS

The heap: a tree stored in an array

You can compute parent and child indices and explain the heap property.

SYSTEM DESIGN

Why distributed systems are hard

You can name the three things that stop being true once there are two machines.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is a heap? Where is the parent of index i ?*
- *What changes when you go from one server to two?*

1 What this is, and why they ask it

A **heap** is a binary tree with one rule about values — **every parent is smaller than both its children** (for a min-heap) — and one rule about shape: **every level is full except the last, which fills left to right**.

Three sentences. That value rule is much weaker than the **BST rule** and it is deliberately so: a heap does not know which of two siblings is smaller, so it cannot search — but it can tell you **the single smallest item instantly**, which is all some problems ever need. The shape rule is what earns its keep: a tree with no gaps can live in a **plain array with no pointers at all**, where a node at index i has its children at $2i + 1$ and $2i + 2$. And those two facts together give you a structure that costs a fifteenth of the memory of a pointer tree and answers “what is next?” in constant time.

They ask “*where is the parent of index i ?*” because it is a five-second question that reveals whether you understand why a heap is an array. And they ask “*what is a heap?*” because most candidates answer with the value rule and forget the shape rule — and the shape rule is the entire reason the structure exists.

2 The story

The clinic at the bus stand had one doctor, thirty-two plastic chairs bolted into four rows, and a compounder called Muniyandi who decided the order.

The chairs were numbered, one to thirty-two, painted on the backs. Everybody knew that chair one was seen next.

What Muniyandi did not do — and this is what the young doctor could not get used to in her first month — was put everybody in order of how ill they were.

She asked him once who was third-worst in the room, and he said he had no idea, and she thought he was being difficult.

He explained it that evening. He said: I only ever need one person. When the door opens I need to know who goes in, and that is chair one. Nobody has ever asked me for third-worst. If I tried to keep all thirty-two in exact order, every time somebody new walked in I would be shuffling half the room, and people would be moving chairs all afternoon for a question nobody asks.

What he did keep was a much smaller rule, and he was strict about it: **each chair had two chairs it was responsible for, and the person in a chair was never worse than either of those two.**

Chair one was responsible for two and three. Chair two for four and five. Chair three for six and seven. Always the same two, worked out from the number, so nobody had to remember anything.

When somebody new arrived he put them in the highest empty chair — the rows filled left to right, no gaps — and then walked them up. If they were worse than the person in the chair above, the two swapped, and he checked again. Usually one or two swaps. Never more than five, because there were only five rows' worth of steps from the bottom to the top.

And when the door opened, chair one went in, the person from the last occupied chair moved to chair one, and then walked back down the same way — swapping with the worse of their two responsibilities until they were in a chair where both of their two were less ill than them.

The young doctor's actual objection came later, and it was a fair one. She said: a man with chest pain is in chair two, and a man with a headache is in chair three, and there is somebody in chair six who is worse than the headache. Your rule does not notice.

Muniyandi said it does not, and it does not have to. Chair six is below chair three, and chair three is below chair one, and all I have ever promised is that **chair one is the worst person in the room**. I have not promised anything about how two and three compare, and I never will, because keeping that promise costs more than it is worth.

3 The idea in plain English

Muniyandi has built a min-heap, and both of his rules — the weak ordering and the no-gaps filling — are the two halves of the definition.

- “Never worse than the two you are responsible for” is the **heap property**.
- “No idea who is third-worst” is the crucial weakness, and it is deliberate.
- “Fill the highest empty chair, left to right” is the **complete tree** shape.
- “Worked out from the number” is the **array indexing**, which is what the shape rule buys.

- Walking a new arrival up is **sift-up**; walking the replacement down is **sift-down** — tomorrow.

The two rules, and why both are needed

The heap property. In a **min-heap**, every node is less than or equal to both of its children. In a **max-heap**, greater than or equal.



every parent $\leq$ both children

NOTE: 3 and 2 are NOT in order. Neither are 7 and 4 against 5. The heap says NOTHING about siblings, or about any two nodes not on the same root-to-leaf path.

This is much weaker than a BST, and the weakness is the point. A BST maintains a total order and pays $O(\log n)$ on every insertion to do it. A heap maintains only “the minimum is at the top” and pays $O(\log n)$ too — but for a far cheaper promise, which is why the constants are small and the memory is tiny.

heap:	the minimum is at the root.	Finding 5 costs $O(n)$.
BST:	every value has a known place.	Finding 5 costs $O(\text{height})$.

Say that comparison out loud. “A heap is not a sorted structure — it is a structure that knows its minimum.”

The shape property. A heap is a **complete** binary tree: every level is completely full except possibly the last, and the last fills from the left with no gaps.

That rule has nothing to do with values and everything to do with storage.

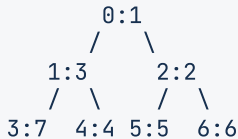
The array, which is the actual idea

Because there are no gaps, you can number the nodes level by level, left to right, and store them in a plain array. Then the tree structure is **arithmetic**:

```
for a node at index i (0-based):

    left child  = 2i + 1
    right child = 2i + 2
    parent     = (i - 1) // 2
```

```
array: [ 1, 3, 2, 7, 4, 5, 6 ]
index:  0 1 2 3 4 5 6
```



```
children of index 1 → 3 and 4  ✓
parent of index 5  → (5-1)//2 = 2  ✓
```

No pointers. No node objects. No allocation. Muniyandi’s numbered chairs — the relationships are computed from the number, so nobody has to remember them.

And this is **only possible because the tree is complete**. A tree with gaps would need a slot for every possible position, which is $2^{(h+1)} - 1$:

complete tree, 1,000 nodes	→ ~1,024 array slots	99% used
degenerate tree, 40 nodes	→ $\sim 10^{12}$ slots	impossible

That is why “complete” is half the definition and not a detail.

The 1-based variant, and why you might see it

Some texts index from 1, which makes the arithmetic prettier:

```
0-based: left = 2i+1, right = 2i+2, parent = (i-1)//2
1-based: left = 2i, right = 2i+1, parent = i//2
```

Python’s `heapq` is 0-based. Know both, use the 0-based one, and say which you are using — mixing them silently corrupts the structure.

What a heap is for

peek the minimum	$O(1)$	it is index 0
insert	$O(\log n)$	place at the end, sift UP
extract the minimum	$O(\log n)$	swap in the last, sift DOWN
build from n items	$O(n)$	not $O(n \log n)$ – day 114
find an arbitrary item	$O(n)$	THE weakness
delete an arbitrary	$O(n)$	because you must find it first

$O(1)$ peek is the reason the structure exists. Everything that repeatedly asks “*what is the next most urgent thing?*” wants a heap:

- **task schedulers and event loops** — the next event by time

- **Dijkstra's algorithm and A*** — the nearest unvisited node
- **top-k problems** — [day 116](#)
- **merging k sorted lists** — [day 117](#)
- **the running median** — [day 118](#), with two heaps
- **heapsort** — build a heap, extract everything

What a heap is not for

Searching. There is no ordering between siblings, so an arbitrary value could be anywhere. $O(n)$, and it is the answer that catches people out.

Sorted iteration. A heap's array is *not* sorted, and printing it is not printing sorted order. You must extract n times, at $O(\log n)$ each — which is $O(n \log n)$ and is exactly heapsort.

The k-th smallest, for arbitrary k. $O(k \log n)$ by extracting k times, which is fine for small k and terrible for $k = n/2$.

Say the weakness unprompted. “A heap gives me the minimum in constant time and nothing else in less than linear time.”

Heap against the alternatives

	peek min	insert	extract min	search	sorted order
sorted array	$O(1)$	$O(n)$	$O(1)$	$O(\log n)$	free
unsorted array	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n \log n)$
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	free
HEAP	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n \log n)$

The heap wins nothing outright and loses nothing badly, and that is the point: it is the cheapest structure that does insert-and-extract-minimum well, and its constants and memory are far better than a BST's because there are no pointers and no allocation.

1,000,000 integers		
pointer-based BST	~120 MB	
heap in a Python list	~8 MB	15x smaller
heap in a C array	4 MB	

That memory difference is a real reason to choose it, and it is worth quoting.

heapq, and its one surprise

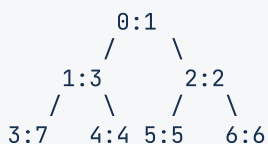
```
import heapq
h = []
heapq.heappush(h, 5)
heapq.heappush(h, 1)
heapq.heappop(h)           # 1
h[0]                       # peek, 0(1)
heapq.heapify(lst)         # 0(n), in place
```

Python's `heapq` is a **MIN-heap only**. There is no `max=True`. For a max-heap you push negated values — [day 115](#) — and that is the single most common practical gotcha.

Also: `heapq` **operates on a plain list**. There is no heap object; the list is the heap, and if you mutate it directly you break the invariant with no error.

4 The picture

The tree and its array, side by side. **This is the diagram to be able to draw.**



index: 0 1 2 3 4 5 6
array: [1, 3, 2, 7, 4, 5, 6]

left(i) = 2i + 1
right(i) = 2i + 2
parent(i) = (i-1) // 2

check: children of 1 → 3, 4 → values 7 and 4. $3 \leq \text{both}$ ✓
parent of 5 → $(5-1)//2 = 2$ → value 2. $2 \leq 5$ ✓

THE SAME THING, TWICE. The tree is a drawing; the array is the data.

Why “complete” is what makes the array possible:

COMPLETE – every level full except the last, which fills LEFT to right



index 0

1, 2

3, 4, 5 ← no gaps, so no wasted slots
array: 6 entries for 6 nodes

NOT COMPLETE



index 0

1, 2

3, ... 6 ← indices 4 and 5 are EMPTY
array: 7 slots for 5 nodes, and it gets
exponentially worse with depth

a degenerate tree of 40 nodes would need $\sim 10^{12}$ slots.
That is why the shape rule is half the definition.

The heap property is weak, and that is deliberate:



is 3 < 2? The heap does not care and does not know.
is 7 < 5? Same.

GUARANTEED: every node $\leq$ its own descendants

NOT GUARANTEED: anything between two nodes on different branches

CONSEQUENCE: the minimum is at index 0, instantly.
finding the value 5 means scanning: $O(n)$.
a BST would find it in $O(\text{height})$ – but a BST pays for
a total order it does not need here.

Muniyandi's clinic, as the same picture:

```

chairs  1  2  3  4  5  6  7      (his numbering is 1-based)
        [A] [B] [C] [D] [E] [F] [G]

chair 1's responsibilities:  2, 3      = 2i, 2i+1
chair 3's responsibilities:  6, 7
chair 6's boss:             3         = i // 2

he promises:  A is worse than B and C
               B is worse than D and E
he promises NOTHING about B against C.

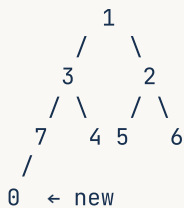
"Nobody has ever asked me for third-worst."

```

Insert and extract, at a glance (the mechanics are tomorrow):

INSERT 0

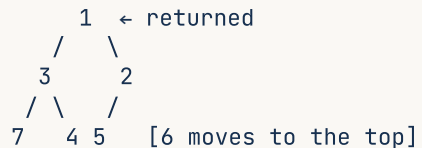
put it in the next free slot
then walk it UP while it is
smaller than its parent



$0 < 7$, swap. $0 < 3$, swap. $0 < 1$, swap.
at most `height` swaps = $O(\log n)$

EXTRACT MIN

take index 0 (the answer)
move the LAST element to index 0
then walk it DOWN, swapping with
the SMALLER child



at most `height` swaps either way
= $O(\log n)$

5 The code, built step by step

Step 1 — say both halves of the definition

“A heap has two rules. The **value** rule: every parent is less than or equal to both children — which is much weaker than a BST, because siblings are unordered. And the **shape** rule: it is a complete tree, every level full except the last which fills left to right. The shape rule is the one people forget, and it is the reason the whole thing works.”

Step 2 — derive the index arithmetic rather than reciting it

```

level 0:  index 0          1 node
level 1:  indices 1, 2     2 nodes
level 2:  indices 3, 4, 5, 6 4 nodes

```

“Numbering level by level, left to right, node i ’s children land at $2i + 1$ and $2i + 2$, and the parent is $(i - 1) // 2$. It works only because there are no gaps — that is what ‘complete’ buys.”

Step 3 — the peek, which is the whole value proposition

```
def peek(self):  
    return self.data[0]                #  $O(1)$ 
```

One line, constant time. Say why it matters: everything that repeatedly asks “what is next?” wants this and nothing else.

Step 4 — insert: place at the end, walk up

```
self.data.append(value)  
i = len(self.data) - 1  
while i > 0 and self.data[i] < self.data[(i - 1) // 2]:  
    parent = (i - 1) // 2  
    self.data[i], self.data[parent] = self.data[parent], self.data[i]  
    i = parent
```

Appending keeps the tree complete automatically, which is why the shape rule survives every operation for free.

Step 5 — say the weakness before being asked

“Finding an arbitrary value is $O(n)$, because siblings are unordered so it could be anywhere. A heap knows its minimum and nothing else.”

The complete solution

```
import heapq

class MinHeap:
    """A binary min-heap in a plain list. No nodes, no pointers, no allocation.

    TWO RULES:
    VALUE every parent  $\leq$  both children. Siblings are UNORDERED.
    SHAPE a COMPLETE tree – every level full except the last, which fills
          left to right. This is what allows array storage.

    INDEX ARITHMETIC (0-based; Python's heapq is 0-based too):
    left(i)  = 2i + 1
    right(i) = 2i + 2
    parent(i) = (i - 1) // 2
    """

    def __init__(self, items: list[int] | None = None) → None:
        self.data: list[int] = list(items or [])
        if self.data:
            self._heapify()

    # -- the arithmetic, named so the code reads as the tree -----

    @staticmethod
    def _parent(i: int) → int:
        return (i - 1) // 2

    @staticmethod
    def _left(i: int) → int:
        return 2 * i + 1

    @staticmethod
    def _right(i: int) → int:
        return 2 * i + 2

    # -- the operations -----

    def peek(self) → int:
        """THE reason the structure exists: O(1)."""
        if not self.data:
            raise IndexError("peek from an empty heap")
        return self.data[0]

    def push(self, value: int) → None:
        """Append (which keeps the tree complete for free), then sift UP.
        At most `height` swaps, so O(log n)."""
        self.data.append(value)
        self._sift_up(len(self.data) - 1)

    def pop(self) → int:
        """Take index 0, move the LAST element there, then sift DOWN.

        Moving the last element is what preserves completeness – removing
        from the middle would leave a gap and break the array layout.
        """
        if not self.data:
```

```

        raise IndexError("pop from an empty heap")
        smallest = self.data[0]
        last = self.data.pop()
        if self.data:
            self.data[0] = last
            self._sift_down(0)
        return smallest

def _sift_up(self, i: int) → None:
    while i > 0:
        parent = self._parent(i)
        if self.data[i] ≥ self.data[parent]:
            break
        self.data[i], self.data[parent] = self.data[parent], self.data[i]
        i = parent

def _sift_down(self, i: int) → None:
    n = len(self.data)
    while True:
        smallest = i
        left, right = self._left(i), self._right(i)
        if left < n and self.data[left] < self.data[smallest]:
            smallest = left
        if right < n and self.data[right] < self.data[smallest]:
            smallest = right
        if smallest == i:
            return
        self.data[i], self.data[smallest] = self.data[smallest], self.data[i]
        i = smallest

def _heapify(self) → None:
    """Build in O(n), not O(n log n). Day 114 explains why.
    Start at the last PARENT – everything after it is a leaf."""
    for i in range(len(self.data) // 2 - 1, -1, -1):
        self._sift_down(i)

def __len__(self) → int:
    return len(self.data)

def is_valid(self) → bool:
    """Check the property directly: every parent ≤ both children."""
    return all(
        self.data[self._parent(i)] ≤ self.data[i]
        for i in range(1, len(self.data))
    )

def find(self, value: int) → int:
    """THE WEAKNESS: O(n). Siblings are unordered, so the value could be
    anywhere. This is what a heap cannot do, and it is why a heap is not
    a substitute for a BST."""
    for i, v in enumerate(self.data):
        if v == value:
            return i
    return -1

def index_arithmetic_demo(n: int) → None:
    """Print the tree relationships for a heap of n items, to make the
    arithmetic concrete."""
    for i in range(n):

```

```

        left, right = 2 * i + 1, 2 * i + 2
        parent = (i - 1) // 2 if i > 0 else None
        print(f"index {i}: parent {parent}, children "
              f"{left if left < n else '-'}, {right if right < n else '-'}")

def is_heap(data: list[int]) → bool:
    """Is this array a valid min-heap? Checks the VALUE rule; the shape rule
    is automatic for any array, which is the point."""
    return all(data[(i - 1) // 2] ≤ data[i] for i in range(1, len(data)))

def heapsort(items: list[int]) → list[int]:
    """Build a heap, then extract everything. O(n log n), and it shows that
    a heap's array is NOT sorted – you have to extract to get order."""
    h = MinHeap(items)
    return [h.pop() for _ in range(len(h))]

def kth_smallest(items: list[int], k: int) → int:
    """O(n + k log n): build in O(n), then k extractions.
    Good for small k; for k = n/2 a sort is better."""
    h = MinHeap(items)
    for _ in range(k - 1):
        h.pop()
    return h.pop()

class Task:
    """Heaps hold anything ORDERABLE, not just numbers. A tuple is the usual
    trick: (priority, tie_breaker, payload) – see day 115 for why the
    tie-breaker matters."""

    def __init__(self, priority: int, name: str) → None:
        self.priority = priority
        self.name = name

    def __lt__(self, other: "Task") → bool:
        return self.priority < other.priority

    def __repr__(self) → str:
        return f"Task({self.priority}, {self.name!r})"

if __name__ == "__main__":
    h = MinHeap()
    for v in (5, 3, 8, 1, 9, 2, 7):
        h.push(v)

    print(h.data)                # [1, 3, 2, 5, 9, 8, 7]
    print(h.peek(), len(h))      # 1 7
    print(h.is_valid())          # True

    # THE ARRAY IS NOT SORTED – that is the point
    print(h.data == sorted(h.data)) # False
    print(heapsort([5, 3, 8, 1, 9, 2, 7])) # [1, 2, 3, 5, 7, 8, 9]

    index_arithmetic_demo(7)
    # index 0: parent None, children 1, 2
    # index 1: parent 0, children 3, 4

```



```

# index 3: parent 1, children -, -

# extraction gives sorted order, one at a time
print([h.pop() for _ in range(3)]) # [1, 2, 3]
print(h.is_valid()) # True

# building is O(n), not O(n log n)
big = MinHeap(list(range(1000, 0, -1)))
print(big.peek(), big.is_valid()) # 1 True

# THE WEAKNESS
print(big.find(500) != -1) # True, in O(n)

# heapq: the same thing, in C, and MIN-ONLY
lst = [5, 3, 8, 1]
heapq.heapify(lst) # O(n), in place
heapq.heappush(lst, 0)
print(lst[0], heapq.heappop(lst)) # 0 0
print(heapq.nsmallest(3, [5, 1, 8, 3, 9])) # [1, 3, 5]

# a max-heap by negating – day 115
maxh: list[int] = []
for v in (5, 3, 8, 1):
    heapq.heappush(maxh, -v)
print(-maxh[0]) # 8

# heaps hold anything orderable
tasks: list[Task] = []
for t in (Task(3, "email"), Task(1, "page"), Task(2, "log")):
    heapq.heappush(tasks, t)
print(heapq.heappop(tasks)) # Task(1, 'page')

# memory: a heap is an array, not a pointer structure
import sys
arr = list(range(100_000))
print(f"{sys.getsizeof(arr) / 1024:.0f} KB for 100,000 ints in a list")

```

6 What it costs

The operations

peek the minimum	$O(1)$	index 0
push	$O(\log n)$	append, then at most `height` swaps up
pop the minimum	$O(\log n)$	swap in the last, then at most `height` down
build from n items	$O(n)$	NOT $O(n \log n)$ – day 114
heapsort	$O(n \log n)$	n extractions
find an arbitrary value	$O(n)$	THE weakness
delete an arbitrary	$O(n)$	find is the expensive part
merge two heaps	$O(n)$	rebuild; a binary heap cannot merge cheaply

Height is $\lceil \log_2 n \rceil$ and it is guaranteed, not merely expected — because the tree is complete by construction. That is a real advantage over a BST, which needs balancing machinery to promise the same thing.

```
n = 1,000           height 9
n = 1,000,000       height 19
n = 1,000,000,000   height 29
```

Memory, which is the underrated advantage

1,000,000 integers

pointer-based BST (Python)	~120 bytes/node	→	~120 MB	
heap in a Python list	~8 bytes/slot + int objects	→	~8 MB of list,	
			~28 MB with the ints	
heap in a C/NumPy array	4 bytes/int	→	4 MB	

Fifteen to thirty times smaller than a pointer tree, because there are no node objects, no pointers, and no allocation per element. And the array is **contiguous**, so it is far friendlier to the CPU cache: a sift-down walks `2i+1`, `4i+3`, and so on, which spreads out — but the top levels of the heap stay resident in cache and those are where most comparisons happen.

Why `find` is $O(n)$

```
the value 5 could be at ANY index below the first index whose value exceeds 5
→ in the worst case, everywhere
→ n/2 of the nodes are leaves, and any of them could hold it
```

No pruning is possible, because siblings are unordered. This is the single sentence that explains why a heap does not replace a BST.

Against the alternatives, with numbers

task: insert 1,000,000 items, then extract the smallest 1,000

sorted array (kept sorted)	1,000,000 insertions × up to 1,000,000 shifts	– no
sort once, then read	$O(n \log n) = \sim 20,000,000$ comparisons	
balanced BST	$1,000,000 \times 20 = 20,000,000$, plus 120 MB	
HEAP	build $O(n) = 1,000,000$	
	+ 1,000 pops × 20 = 20,000	
	≈ 1,020,000 operations, and 8 MB	

Twenty times fewer operations and fifteen times less memory for this shape of problem, which is exactly the top-k shape of [day 116](#).

`i // 2` is the 1-based formula. For 0-based it is `(i - 1) // 2`. Mixing the conventions produces a structure that looks like a heap and is not, with no error.

```
0-based: left = 2i+1, right = 2i+2, parent = (i-1)//2
1-based: left = 2i,    right = 2i+1, parent = i//2
```

Trap 5 — removing the wrong element on pop

```
self.data[0] = self.data[-1]
self.data.pop()                # order matters
```

You must move the **last** element to the root and shorten the array — never remove from the middle, which would leave a gap and destroy the completeness the array layout depends on.

And the subtle case: **when there is exactly one element**, popping the last and then assigning it to index 0 would resurrect it. The version in the code checks `if self.data:` after the pop.

Trap 6 — sifting down against the wrong child

```
if self.data[left] < self.data[i]:
    swap with left                # WRONG when right is smaller
```

You must swap with the **smaller** of the two children. Swapping with the left one when the right is smaller leaves the new parent greater than its right child — the property is violated and nothing raises.

Trap 7 — mutating the list behind `heapq`'s back

```
lst[3] = 0                        # the invariant is now broken
heapq.heappop(lst)                # returns the wrong element
```

There is no heap object in Python — the list is the heap. Changing an element's value, sorting it, appending with `.append()` instead of `heappush`, all silently break it. If you must change a priority, push a new entry and mark the old one dead — [day 115](#).

Trap 8 — expecting a max-heap from `heapq`

```
heapq.heappush(h, x, reverse=True)    # TypeError — no such thing
```

`heapq` is min-only. Negate the values, or wrap them in a class with an inverted `__lt__`. This is the most common practical gotcha in Python and it is [tomorrow's neighbour](#).

8 In the interview

How it gets asked

- The definition: “What is a heap?”
- The five-second one: “Where is the parent of index i ?”
- The comparison: “How is it different from a BST?” / “Why not use a sorted array?”
- The weakness: “How would you find an arbitrary element?”
- The application: “Which problems does this actually solve?”

What to say out loud, in the first ninety seconds

1. **Give both rules, and flag which one people forget.** “Two rules. Values: every parent is less than or equal to both children — much weaker than a BST, because siblings are unordered. And shape: it is a **complete** tree, every level full except the last, filling left to right. The shape rule is the one people leave out, and it is the reason the structure exists.”
2. **Say what the shape buys.** “Because there are no gaps, it lives in a plain array — node i ’s children are at $2i + 1$ and $2i + 2$, and the parent is $(i - 1) // 2$. No pointers, no node objects, no allocation.”
3. **Give the value proposition in one line.** “ $O(1)$ to see the minimum, $O(\log n)$ to insert or remove it. That is all it promises, and it is exactly what anything asking ‘what is next?’ needs.”
4. **State the weakness before being asked.** “It cannot search: an arbitrary value could be anywhere, because siblings are unordered, so finding one is $O(n)$. A heap knows its minimum and nothing else.”
5. **Compare with a BST honestly.** “A BST maintains a total order and can find anything in $O(\text{height})$. A heap maintains a much weaker promise for the same $O(\log n)$ cost — which buys far smaller constants and about fifteen times less memory, because it is an array.”
6. **Name the height guarantee.** “And the height is $\log n$ **guaranteed**, not expected, because completeness is maintained by construction — no rotations, no balancing machinery.”

The follow-ups

“Where is the parent of index i ?” “ $(i - 1) // 2$, zero-based — and the children are at $2i + 1$ and $2i + 2$. I would say which convention I am using, because the one-based version is $i // 2$, $2i$ and $2i + 1$, and mixing them produces something that looks like a heap and is not, with no error. The reason the arithmetic works at all is the shape rule: the tree is **complete**, so numbering it level by level, left to right, leaves no gaps, and the positions are computable rather than stored. If the tree could have gaps you would need a slot for every possible position — $2^{(h+1)} - 1$ — and a forty-node chain would need about a trillion slots.”

“How is it different from a BST?” “The promise is much weaker, deliberately. A BST orders **every** pair of values, so it can find anything in $O(\text{height})$. A heap only guarantees that each node is smaller than its own descendants — it says nothing about siblings, or about any two nodes on different branches. So it gives you the minimum in constant time and cannot find anything else in less than linear time. The trade is worth it when you only ever need the extreme: you get the same $O(\log n)$ insertion for a cheaper promise, which means much smaller constants, a **guaranteed** $\log n$ height with no balancing machinery, and about fifteen times less memory because it is an array with no pointers or node objects. A million integers is roughly 8 MB as a heap against 120 MB as a pointer tree.”

“How would you find an arbitrary element?” “Linear scan, $O(n)$, and there is no way around it inside the heap itself — no pruning is possible, because siblings are unordered, so the value could be under either child of every node. If a problem needs both ‘give me the minimum’ and ‘find and update this specific element’, which Dijkstra’s algorithm does, the standard answer is an **indexed heap**: the heap plus a hash map from element to its current index, and every swap updates the map. That gets find and decrease-key down to $O(\log n)$, at the cost of maintaining the map on every operation. In Python the usual shortcut is lazier — push a new entry with the new priority and skip stale entries when you pop them.”

“Why not use a sorted array?” “A sorted array beats a heap on everything except the operation that matters here. Peeking the minimum is $O(1)$ in both. Searching is $O(\log n)$ in the array and $O(n)$ in the heap. Sorted iteration is free in the array. But **insertion** is $O(n)$ in a sorted array because everything after the insertion point shifts — a million-element array means up to a million moves per insert. The heap is $O(\log n)$. So: if the data is built once and only read, use a sorted array; if items arrive and leave continuously and you only ever need the extreme, use a heap. Concretely, for ‘insert a million items and take the smallest thousand’, a heap is about a million operations and a sorted array is a million insertions each shifting up to a million elements.”

“What is it actually used for?” “Anything that repeatedly asks ‘what is the next most urgent thing?’. **Task schedulers and event loops** — the next event by timestamp. **Dijkstra and A*** — the nearest unvisited node, which is the classic. **Top k** — keep a heap of size k and you get the k largest in $O(n \log k)$ instead of sorting everything. **Merging k sorted lists** — a heap of the k current heads. **The running median** — two heaps facing each other. And **heapsort**, which is build-then-extract-everything and is the one algorithm that sorts in $O(n \log n)$ with $O(1)$ extra space. The unifying property is that all of them need only the extreme element and never need to search.”

“Can you merge two heaps?” “Not cheaply with a binary heap — the honest answer is that you concatenate the arrays and rebuild, which is $O(n + m)$. Merging is the operation binary heaps are bad at, and it is why other heap variants exist: a **binomial heap** merges in $O(\log n)$, and a **Fibonacci heap** merges in $O(1)$ amortised and also does decrease-key in $O(1)$, which is what makes Dijkstra’s theoretical bound better. In practice nobody uses Fibonacci heaps because the constants are terrible — but naming them shows you know that the binary heap is a point on a curve rather than the only option.”

A model answer

Asked: *what is a heap, and where is the parent of index i?*

“A heap has **two** rules, and the second one is the one people leave out.

The **value** rule: every parent is less than or equal to both of its children, for a min-heap. That is a much weaker promise than a binary search tree makes — it says nothing at all about how two siblings compare, or about any two nodes on different branches. All it guarantees is that **the minimum is at the root**.

The **shape** rule: it is a **complete** binary tree. Every level is completely full except possibly the last, and the last fills from the left with no gaps.

That second rule is why the structure exists. Because there are no gaps, you can number the nodes level by level, left to right, and store them in a **plain array** — the tree structure becomes arithmetic rather than pointers. Node i ’s children are at $2i + 1$ and $2i + 2$, and **the parent of i is $(i - 1) // 2$** , zero-based. I would say the convention, because one-based indexing gives the prettier $2i$, $2i + 1$ and $i // 2$, and mixing them silently corrupts the structure.

The payoff is that there are no node objects, no pointers and no allocation per element: a million integers is about eight megabytes as a heap against a hundred and twenty as a pointer tree. And the height is $\log n$ **guaranteed** rather than expected, because completeness is preserved by construction — you append at the end and sift up, or move the last element to the root and sift down, and both keep the shape automatically. No rotations, no balancing machinery.

What it promises is narrow and worth being explicit about: $O(1)$ **to see the minimum**, $O(\log n)$ **to insert or extract it**, and $O(n)$ **for absolutely everything else**. Finding an arbitrary value is a linear scan, because siblings are unordered so no pruning is possible. A heap is not a sorted structure — its array is not in order, and getting sorted order costs n extractions.

Which makes it exactly the right tool for anything that repeatedly asks ‘what is the next most urgent thing?’ — schedulers, Dijkstra, top-k, merging sorted streams, a running median — and the wrong tool for anything that needs to look things up.”

9 Recall card

- **TWO** rules, and the second is the one people forget. **VALUE**: every parent $\leq$ both children — much weaker than a BST, because siblings are **UNORDERED**. **SHAPE**: a **COMPLETE** tree, every level full except the last, filling left to right. The shape rule is why the structure exists.

- **Completeness means no gaps, so the tree lives in a plain ARRAY:** `left = 2i + 1`, `right = 2i + 2`, `parent = (i - 1) // 2` (0-based; the 1-based forms are `2i`, `2i+1`, `i // 2` — **never mix them**). No pointers, no node objects, and the height is `log n` **guaranteed**, not expected.
- **The promise is narrow:** `O(1)` **peek**, `O(log n)` **push and pop**, `O(n)` **build** — and `O(n)` **for everything else**. Finding an arbitrary value is a **linear scan**, because no pruning is possible when siblings are unordered. **A heap knows its minimum and nothing else**, and its array is **not sorted**.
- **Memory is the underrated win:** 1,000,000 integers $\approx$ 8 MB as a heap against $\sim$ 120 MB as a **pointer tree**. For “insert a million, take the smallest thousand”, a heap is $\sim$ 1 million operations against 20 million for a sort.
- **Use it whenever the question is “what is next?”** — schedulers, **Dijkstra**, top-k, merging k sorted lists, running median, heapsort. **Do not use it to search**. If you need find-and-update too, that is an **indexed heap** (heap plus a value $\rightarrow$ index map). And Python’s `heapq` is **min-only**, operates on a **plain list**, and **breaks silently** if you mutate that list directly.

1 What this is, and why they ask it

A **distributed system** is one where the work is spread across machines that can only talk to each other over a network. You have been building them for fifteen days — load balancers, replicas, shards, caches — without naming the thing they all have in common.

Three sentences. On one machine, a function call either succeeds or throws, the clock is the same everywhere, and memory is shared — and **all three of those stop being true the moment there are two machines**. The one that matters most is **partial failure**: a call over a network has a third outcome that does not exist locally, “*I do not know*”, and almost every difficult problem in the rest of this phase is a consequence of it. And the honest conclusion is not a technique but a limit: there are things you would like to guarantee that are provably impossible, so the work is choosing which guarantee to give up.

They ask “*what changes with two machines?*” because it is the frame for everything after it — [CAP](#), [consistency models](#), [quorums](#), [leader election](#), [consensus](#). A candidate who can say “*the third outcome is ‘unknown’, so every operation must be safe to repeat*” has the key that unlocks all of them.

2 The story

The auction at the cattle market was on the second Tuesday and the two brothers had decided, finally, that they would buy the pair of bullocks together.

Selvam was in Palani. Murugan was in Dindigul, four hours away. Neither could get to the other’s town before the auction, and the rule they had agreed was that they would only bid if **both** of them turned up — one of them alone could not carry the money or handle the animals.

The phone line between those two exchanges was bad. Not dead — bad. Calls dropped in the middle, and you could not always tell whether the other person had heard the last thing you said.

Selvam rang on the Sunday and said: Tuesday, eleven o’clock, at the north gate.

The line cut.

He did not know whether Murugan had heard “eleven o’clock” or whether the call had dropped before that. So he rang again. Murugan answered and said yes, I heard, eleven, north gate.

And then Murugan put the phone down and immediately had the same problem in reverse. Selvam did not know that Murugan had heard him — unless Murugan’s confirmation had got through. And Murugan did not know that his confirmation had got through, so from where he sat, Selvam might still be sitting in Palani thinking the plan had failed.

So Murugan rang back and said: did you get my confirmation?

Selvam said yes.

And then Selvam thought about it while walking home and realised that Murugan now did not know whether *that* had got through either.

He described it to his wife that evening, who was less interested than he had hoped, and he put it like this: every message needs a reply to be sure it arrived, and the reply needs a reply, and there is no last one. You can go on all night and there is always one message at the end that somebody is not sure about.

What they actually did on the Tuesday was not solve it. They changed the rule.

Selvam said: I will be at the north gate at eleven, and I will wait until half past twelve, and if you are not there I will go home. It does not matter whether you got this message or not — if you are there, we buy; if you are not, I lose a morning.

Both of them turned up at ten to eleven.

Murugan said afterwards that the trick was not agreeing better. It was building a plan where **not agreeing did not cost very much**.

3 The idea in plain English

The brothers have discovered the Two Generals problem, and their solution is the one real systems use: stop trying to guarantee agreement, and make disagreement cheap.

- The bad phone line is an **unreliable network**.
- “I do not know whether he heard me” is **partial failure** — the third outcome.
- The endless chain of confirmations is the **Two Generals problem**, and it is **provably unsolvable**.
- “Wait until half past twelve and then go home” is a **timeout**.

- “It does not matter whether you got this message” is designing for **idempotency and independent progress**.

The three things that stop being true

One: an operation has three outcomes, not two.

ONE MACHINE		TWO MACHINES	
the call returns	✓	the call returns	✓
the call throws	X	the call throws	X
		THE CALL TIMES OUT	???
		and you cannot tell whether it happened	

This is the whole subject. A timeout does not mean failure. It means: the request may have been lost on the way, or executed and the reply lost on the way back, or it is still running and will complete in a minute. You cannot distinguish those from the outside, and **no amount of engineering makes that third outcome go away**.

Everything downstream follows: **if you cannot tell whether it happened, you have to be able to do it again safely**. That is why idempotency appears in every design in this course.

Two: there is no shared clock.

Two machines’ clocks drift, and NTP corrects them in jumps — so time can appear to go **backwards** on a machine.

```
machine A says the write happened at 10:00:00.100
machine B says its write happened at 10:00:00.050
→ is B's write earlier? YOU CANNOT TELL. Their clocks differ by more than that.
```

Consequences that come up constantly: “last write wins” is not well defined; you cannot order events across machines by timestamp; and a lease that expires “in 30 seconds” means 30 seconds *on somebody’s clock*.

The replacement is **logical time** — Lamport clocks and vector clocks — which order events by *causality* rather than by wall time. **What you can know is “A happened before B” when there is a chain of messages from A to B**. Anything else is genuinely concurrent, and calling it ordered is a fiction.

Three: the network is not reliable, and it is not fast.

```
a local function call    ~1 nanosecond
a call to another machine ~500,000 nanoseconds (0.5 ms) in the same DC
                        ~150,000,000 ns (150 ms) across the world
```

Five hundred thousand times slower in the best case. So a design that made sense as ten function calls becomes a design that makes no sense as ten network calls, and “just extract it into a service” is a performance decision, not only an organisational one.

The fallacies, which are the checklist

L. Peter Deutsch’s list from 1994 is still the standard, and each one is an assumption that is false and that people keep making:

- | | |
|-------------------------------|---|
| 1. the network is reliable | → messages are lost; design for retries |
| 2. latency is zero | → a call is ~500,000x a function call |
| 3. bandwidth is infinite | → the response size matters |
| 4. the network is secure | → encrypt, authenticate, authorise |
| 5. topology does not change | → machines come and go constantly |
| 6. there is one administrator | → nobody knows the whole system |
| 7. transport cost is zero | → serialisation and egress are real bills |
| 8. the network is homogeneous | → different clouds, versions, protocols |

Name three of these and say which one your design is currently assuming. That is a very strong move.

Partial failure, in detail

On one machine, if the process dies, *everything* stops — and that is a much easier world than it sounds. In a distributed system, **some parts are running and some are not, and the running parts do not know which is which.**

SCENARIO: service A calls service B to charge a card. A times out.

possibilities:

- | | |
|-----------------------------------|-----------------------------|
| 1. B never received it | → retrying is correct |
| 2. B charged the card, reply lost | → retrying charges TWICE |
| 3. B is still processing | → retrying may charge twice |
| 4. B is dead | → retrying will fail again |

A cannot tell these apart. Ever.

The only workable answer is to make the operation idempotent — give it a key, so that a repeat is recognised and returns the first result. That is why every serious API takes an idempotency key, and it is the single most reusable idea in this phase.

Exactly-once is not available

AT MOST ONCE	send it, do not retry.	May be lost.
AT LEAST ONCE	retry until acknowledged.	May be duplicated.
EXACTLY ONCE	...impossible over an unreliable network.	

You cannot have **exactly-once delivery**. What you can have is **at-least-once delivery plus idempotent processing**, which is *observationally* exactly-once — the effect happens once, even though the message may arrive several times.

Say it that way: “*exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key.*” Systems that advertise exactly-once are doing precisely this.

The impossibility results worth naming

Two, and naming them is worth more than deriving them.

Two Generals. Two parties over an unreliable network can never be **certain** they have agreed. Every message needs an acknowledgement, and the acknowledgement needs one. The brothers’ evening.

FLP (Fischer, Lynch, Paterson, 1985). In an asynchronous system where even one process may fail, there is **no algorithm that guarantees consensus** — because you cannot distinguish a crashed machine from a slow one.

What real systems do about them: stop demanding certainty, and use **timeouts** to make progress. A timeout is a *guess* that a machine is dead. It may be wrong — that is the price — and the whole of [Raft and Paxos](#) is machinery for being safe when the guess is wrong.

The one-line summary: you cannot detect failure, so you assume it after a timeout and design for being wrong.

What you get in exchange

The costs above are real, and so is the reason anybody accepts them:

SCALE	past what one machine can do	(day 098, 106)
AVAILABILITY	survive a machine failure	(day 104, 111)
LATENCY	be near your users	(day 103)
ISOLATION	one team's failure is not everyone's	(bulkheads, day 111)

Distribution is not a goal. It is a price you pay for one of those four, and the right first question about any distributed design is *which of the four am I buying?*

4 The picture

The third outcome, which is the whole subject.

ONE MACHINE

caller → function
← returns

TWO outcomes:

returned a value ✓
raised X

TWO MACHINES

caller → network → service
← ??? ←

THREE outcomes:

returned a value ✓
returned an error X
TIMED OUT ???

├ never arrived
├ ran, reply lost
├ still running
└ the machine is dead

AND YOU CANNOT TELL WHICH

EVERYTHING ELSE IN THIS PHASE IS A CONSEQUENCE OF THAT THIRD BOX.

The brothers' endless acknowledgements:

Selvam → "Tuesday, 11, north gate" → Murugan
← "yes, I heard" ←
→ "did you get my reply?" →
← "yes I did" ←
→ "and did you get THAT?" →
...

there is always ONE message at the end that somebody is unsure about.
No number of round trips removes it. That is the Two Generals result.

WHAT THEY DID INSTEAD:

"I will be there at 11 and wait until 12:30, whatever happens."
→ a TIMEOUT, plus a plan that does not require agreement.

The three broken assumptions, side by side:

ASSUMPTION	ON ONE MACHINE	ACROSS MACHINES
-----	-----	-----
a call succeeds or fails	two outcomes	THREE – and "unknown" is the one that matters
time is shared	one clock	clocks drift; NTP jumps; time can go BACKWARDS → use LOGICAL time
memory is shared and instant	one address space ~1 ns	messages, which are slow (~0.5 ms local, 150 ms global) and can be lost

Delivery guarantees, and where “exactly once” actually lives:

Circuit breakers. After `n` consecutive failures, stop calling and fail fast for a while. That protects *you* from a slow dependency and protects *it* from a thundering retry storm during recovery.

Bulkheads. A separate connection pool per dependency, so one slow service cannot exhaust everything.

Deadlines that propagate. Pass the remaining time budget down the call chain, so a service does not start work its caller has already given up on. gRPC does this natively and it is the mature version of timeouts.

Ordering without a clock

Lamport timestamps. Every process keeps a counter, increments it on each event, and sends it with every message. On receipt, `counter = max(own, received) + 1`.

gives you: if A happened-before B, then $L(A) < L(B)$
does NOT give you: the reverse. $L(A) < L(B)$ does not mean A caused B.

Vector clocks. A counter per process instead of one. Now you can distinguish “A happened before B” from “A and B are concurrent” — which is exactly what a conflict detector needs. The cost is a vector that grows with the number of processes.

Hybrid logical clocks combine physical time with a logical counter, so timestamps are both causally correct and roughly comparable to wall time. **CockroachDB and MongoDB use them**, and they are the modern answer.

And Google’s TrueTime takes the other route entirely: put atomic clocks and GPS receivers in every datacentre, bound the uncertainty to a few milliseconds, and then **deliberately wait out the uncertainty** before committing. Spanner is the only widely known system that buys its way past this problem with hardware.

Failure detection, honestly

heartbeat every 1 s, declare dead after 3 misses	→ ~3 s to detect
	higher false-positive rate
heartbeat every 5 s, 3 misses	→ ~15 s to detect
	fewer false positives

You are trading detection speed against false positives, and there is no setting that avoids both. A false positive means you fail over a healthy machine — which, if fencing is imperfect, gives you two leaders.

Phi-accrual failure detectors — used by Cassandra and Akka — output a *suspicion level* rather than a boolean, so the application can decide how much suspicion warrants action.

What real systems do

- **Stripe, AWS and most payment APIs require an idempotency key** on every write. It is the industry's standard answer to partial failure and it is worth naming specifically.
- **Kafka's "exactly-once semantics"** is at-least-once delivery plus idempotent producers plus transactional writes — the effect is exactly-once; the delivery is not.
- **gRPC deadlines** propagate a remaining time budget through the whole call chain.
- **Amazon Dynamo** used vector clocks for conflict detection and pushed resolution to the application — the shopping cart that merges rather than picks a winner.
- **Spanner's TrueTime** is the hardware answer, and its commit-wait is the clearest demonstration that clock uncertainty is a real cost and not a theoretical one.
- **Jepsen** is the testing practice worth naming: deliberately partition a real cluster and check whether it violates the guarantees it claims. It has found violations in most major distributed databases.

6 The numbers

The cost of leaving one machine

in-process function call	~1 ns	
same-machine IPC	~10 μ s	10,000x
same-datacentre RPC	~0.5 ms	500,000x
same-continent RPC	~30 ms	30,000,000x
cross-world RPC	~150 ms	150,000,000x

A network call is roughly half a million times a function call, so a design of ten chained calls costs five milliseconds before any work happens — and across regions, one and a half seconds.

Clock drift

a typical quartz clock drifts	~10-100 ppm = ~1-9 seconds per day
NTP over the internet	~1-50 ms accuracy, and it corrects in JUMPS
NTP on a local network	~0.1-1 ms
GPS / atomic (Spanner TrueTime)	~1-7 ms of BOUNDED uncertainty

```
two events 5 ms apart on different machines with 50 ms clock skew:  
→ the timestamps may report them in the WRONG ORDER  
→ "last write wins" silently picks the older write
```

That is the concrete cost of having no shared clock, and it is why last-write-wins loses data.

Failure probability at scale

```
one machine, 99.9% available in a year    →  ~8.8 hours down  
  
in a fleet of 1,000 machines with a 3-year MTBF each:  
  expected failures per year    1,000 / 3    ≈ 333  
  expected failures per day     ≈ 1
```

At a thousand machines, something is failing every day. So failure is not an exceptional path to be handled defensively — it is the normal operating condition, which is a genuine change in mindset from single-machine programming.

Timeout and retry arithmetic

```
timeout 1 s, 3 retries with exponential backoff:  
  attempt 1 at t=0          fails at t=1  
  attempt 2 at t=2          fails at t=3    (1 s backoff)  
  attempt 3 at t=5          fails at t=6    (2 s backoff)  
  total worst case ≈ 6 s  
  
if the CALLER's timeout is 3 s, attempts 2 and 3 are pure waste –  
the caller gave up at t=3.  
→ which is why DEADLINES must propagate, not just timeouts.
```

The retry storm

```
a dependency is down for 30 s; 1,000 clients each retry  
  without backoff: 1,000 retries/s continuously, and the moment it  
                   recovers it receives 1,000 simultaneous requests  
  with backoff + jitter: spread over ~30 s, ~30/s
```

Retries without backoff turn a 30-second outage into a much longer one, because the recovering service is immediately overwhelmed.

Duplicate probability

```
1,000,000 payment requests/day, 0.1% of which time out ambiguously  
→ 1,000 requests/day where the caller does not know whether it happened  
→ WITHOUT idempotency keys: up to 1,000 double charges a day  
→ WITH: 0
```

That number is the business case for idempotency keys, and it is far more persuasive than the theory.

7 The trade-offs

Do not distribute unless you are buying something

Every machine you add costs you: a network hop, a new failure mode, a component in series reducing availability, and an operational burden. **The only reasons that justify it are scale, availability, latency and isolation** — and if none of them applies, one machine is faster, simpler and more reliable.

A monolith on one large machine is a completely respectable answer, and saying so at the right moment is a mark of judgement rather than naivety.

Timeouts: too short or too long

Too short turns a healthy-but-slow service into a failed one, triggers unnecessary retries, and can cascade. **Too long** ties up your resources while a dead dependency does nothing, which is how a slow dependency takes down its callers.

The rule: a timeout must be shorter than your caller's timeout, and the whole chain must fit inside the user's patience. **Propagating deadlines is the mature form of this.**

At-least-once plus idempotency, or at-most-once?

At-least-once with idempotency is the default and it costs you a key, storage for seen keys, and discipline: every write path must honour it.

At-most-once is right when duplication is worse than loss and the operation genuinely cannot be made idempotent — some notifications, some metrics. **Say which you are choosing and why**, because silently assuming exactly-once is the most common conceptual error in this space.

Where the honesty matters

- **“Exactly-once” is a claim about the effect, never about delivery.** Anyone claiming exactly-once delivery has either not thought about it or is describing idempotent processing.
- **A timeout is a guess about death.** Every mechanism after this — fencing, quorums, epochs, leases — is machinery for being safe when the guess is wrong.
- **You cannot order events across machines by timestamp.** Last-write-wins loses data silently, and the amount lost is bounded by the clock skew, not by anything you control.
- **Testing is genuinely hard,** because the interesting failures are timing-dependent and rare. Fault injection and Jepsen-style testing exist because ordinary tests do not find these.

The thing that does not get easier

Debugging. On one machine you have a stack trace. Across twenty services you have twenty log streams, clocks that disagree, and a request that touched eight of them. **Distributed tracing with a correlation id is not a nice-to-have; it is the only way to understand what happened,** and it has to be designed in from the first day rather than added during an incident.

8 In the interview

How it gets asked

- The frame: *“What changes when you go from one server to two?”*
- The concrete one: *“Your service calls another and the call times out. What do you do?”*
- The trap: *“Can you guarantee exactly-once delivery?”*
- The clock one: *“How do you know which of two writes happened first?”*
- The judgement one: *“When would you not distribute?”*

What to say out loud, in the first ninety seconds

1. **Name the three broken assumptions.** “Three things stop being true. A call no longer either succeeds or fails — there is a third outcome. There is no shared clock. And memory is replaced by messages that are slow and can be lost.”
2. **Say which one matters most.** “The third outcome is the whole subject. A timeout means the request may never have arrived, or may have executed with the reply lost, or may still be running — and you cannot tell which.”

3. **Draw the consequence immediately.** “So if I cannot tell whether it happened, I have to be able to do it again safely. That is why idempotency keys appear in every design here.”
4. **Kill exactly-once before it is offered.** “Which is also why exactly-once *delivery* is impossible. What is achievable is at-least-once delivery plus idempotent processing, giving an exactly-once *effect*.”
5. **Give the clock consequence.** “And with no shared clock, you cannot order events across machines by timestamp — clock skew is often larger than the interval you are trying to order, so last-write-wins silently discards data. You order by causality instead.”
6. **Say why anyone accepts all this.** “Distribution is not a goal. It is the price of scale, availability, latency or isolation — and if I am not buying one of those, one machine is faster and more reliable.”

The follow-ups

“**Your call times out. What do you do?**” “First, name the problem: a timeout is not a failure, it is an **unknown**. Four things may have happened — the request never arrived, it executed and the reply was lost, it is still running, or the machine is dead — and from the outside they are indistinguishable. So the question is not ‘do I retry’, it is ‘is retrying safe’. If the operation is **idempotent** — a read, a set-to-a-value, a delete — I retry with exponential backoff and jitter, backoff so a brief outage does not become a retry storm and jitter so a thousand clients do not come back in lockstep. If it is **not** idempotent, like charging a card, retrying blindly may charge twice, so I make it idempotent: the caller generates a key per logical operation and sends it with every attempt, and the server records the key and returns the stored result on a repeat. That is what turns at-least-once delivery into an exactly-once effect. I would also cap the retries, and make sure my timeout is shorter than my caller’s — otherwise I am doing work nobody is waiting for any more.”

“**Can you guarantee exactly-once delivery?**” “No, and it is worth being precise about why rather than just saying no. Over an unreliable network, the sender cannot know whether a message arrived without an acknowledgement, and the acknowledgement can be lost, and an acknowledgement of the acknowledgement can be lost — that is the Two Generals result, and there is no protocol that removes the final uncertain message. So the choice is at-most-once, which may lose messages, or at-least-once, which may duplicate them. What **is** achievable is an **exactly-once effect**: at-least-once delivery plus idempotent processing, where a duplicate is recognised and discarded.

That is what systems advertising exactly-once are actually doing — Kafka’s exactly-once semantics is idempotent producers plus transactional writes on top of at-least-once delivery. The distinction between delivery and effect is the whole answer.”

“**How do you know which of two writes happened first?**” “Usually you cannot, and that is more surprising than it sounds. Two machines’ clocks drift — a typical quartz oscillator is tens of parts per million, which is seconds per day — and NTP corrects in jumps, so a machine’s clock can go **backwards**. If the skew is fifty milliseconds and the two writes are five milliseconds apart, the timestamps may report them in the wrong order, and ‘last write wins’ then silently discards the newer write. So you order by **causality** instead of by wall time. **Lamport timestamps** give you: if A happened before B, then A’s stamp is lower — though not the converse. **Vector clocks** go further and let you distinguish ‘A before B’ from ‘A and B are genuinely concurrent’, which is what a conflict detector needs, at the cost of a vector per process. The modern practical answer is **hybrid logical clocks**, which combine physical time with a logical counter so stamps are causally correct and still comparable to wall time — CockroachDB and MongoDB use them. And Google’s Spanner buys its way out with **True-Time**: atomic clocks and GPS bound the uncertainty to a few milliseconds, and then it deliberately **waits out** that uncertainty before committing.”

“**When would you not distribute?**” “Whenever I am not buying one of four things: **scale** past a single machine, **availability** through redundancy, **latency** by being near users, or **isolation** so one team’s failure is not everyone’s. If none of those applies, one machine is faster, simpler, and more available than two — because every added component sits in series and availabilities multiply. A monolith on a large machine is a completely respectable architecture, and going distributed for organisational reasons alone means paying the network tax on every call: a remote call is about half a million times a local one, so what was ten function calls becomes five milliseconds of pure latency, and across regions one and a half seconds. Plus you have acquired partial failure, clock skew, and a debugging problem that needs distributed tracing designed in from day one.”

“**What is the hardest part in practice?**” “Debugging, and it is not close. On one machine a failure gives you a stack trace. Across twenty services you have twenty log streams, machines whose clocks disagree by more than the events you are trying to order, and a single user request that touched eight of them — and the interesting failures are timing-dependent, so they do not reproduce. That is why a **correlation id** propagated through every call and distributed tracing are not optional extras; they are the only way to reconstruct what happened, and they have to be there before the incident. The related practice worth naming is deliberate fault injection — Jepsen-style

testing, where you partition a real cluster and check whether it violates the guarantees it claims. It has found violations in most major distributed databases, which tells you something about how hard this is to get right by reasoning alone.”

“**So how do real systems cope with the impossibility results?**” “They stop demanding certainty and use **timeouts** to make progress — and then build machinery to be safe when the timeout’s guess is wrong. FLP says you cannot guarantee consensus in an asynchronous system where a process may fail, because a crashed machine and a slow machine are indistinguishable. So a system **guesses** after a timeout: it assumes the machine is dead and elects a new leader. If the guess is wrong you have two leaders — and that is what **fencing**, **quorums** and **epoch numbers** exist for: the old leader cannot see a majority, or its writes are stamped with a stale term and rejected. The pattern is: you cannot detect failure, so you assume it and design so that being wrong is survivable. Every mechanism in the rest of this phase is a variation on that.”

A model answer

Asked: *what changes when you go from one server to two?*

“Three things stop being true, and all of the difficulty comes from them.

First, an operation no longer has two outcomes. On one machine a function call either returns or throws. Across a network there is a third: it **times out**, and you cannot tell whether the request never arrived, or executed and the reply was lost, or is still running, or the machine is dead. Those are indistinguishable from the outside, and no amount of engineering removes the third box.

That single fact generates most of the rest. **If I cannot tell whether something happened, I have to be able to do it again safely** — which is why every serious write API takes an **idempotency key**, and why **exactly-once delivery is impossible** while an exactly-once *effect* is achievable: at-least-once delivery plus idempotent processing. That distinction between delivery and effect is worth being precise about, because systems advertising exactly-once are all doing the second thing.

Second, there is no shared clock. Clocks drift by seconds a day and NTP corrects them in jumps, so time can run backwards on a machine. If the skew between two machines is fifty milliseconds and two writes are five milliseconds apart, their timestamps can report them in the wrong order — so ‘last write wins’ silently discards the newer value. You order events by **causality** instead: Lamport timestamps for happened-before, vector clocks if you also need to detect genuine concurrency, or hybrid logical clocks in practice. Spanner is the exception that proves the rule — it puts atomic clocks in every datacentre and then deliberately waits out the remaining uncertainty before committing.

Third, memory becomes messages. A local call is about a nanosecond; a call to another machine in the same datacentre is half a millisecond, and across the world a hundred and fifty. Half a million times slower in the best case. So a design that was ten function calls is a very different design as ten network calls.

The practical consequences I would build in from the start: **a timeout on every remote call**, because a slow dependency is more dangerous than a dead one; **retries with exponential backoff and jitter**, so a thirty-second outage does not become a retry storm on recovery; **idempotency keys** on every non-read operation; **circuit breakers**, so a failing dependency fails fast rather than filling my connection pool; and **a correlation id on every request**, because without distributed tracing an incident across twenty services is unreadable.

And the thing I would say before any of it: **distribution is not a goal**. It is the price of scale, availability, latency, or isolation. If I am not buying one of those four, one machine is faster, simpler and more available — because every compon-

ent I add sits in series, and availabilities multiply.”

9 Recall card

- **Three things stop being true with two machines: an operation has a THIRD outcome (“unknown”), there is NO shared clock, and memory becomes slow, lossy messages (~1 ns local against ~0.5 ms in a datacentre and ~150 ms across the world — half a million times).**
- **The third outcome is the whole subject.** A timeout may mean: never arrived · executed with the reply lost · still running · dead — **indistinguishable from outside**. Therefore **if you cannot tell whether it happened, it must be safe to repeat** — an **idempotency key** on every write.
- **Exactly-once DELIVERY is impossible** (Two Generals: every ack needs an ack).
**At-least-once delivery
- **idempotent processing = exactly-once EFFECT — which is what Kafka and everyone else actually mean. At 1M requests/day with 0.1% ambiguous timeouts, that is 1,000 potential double charges a day**.**
- **You cannot order events across machines by timestamp** — clock skew (tens of ms) usually exceeds the interval you are ordering, so **last-write-wins silently discards data**. Use **causality**: Lamport, vector clocks, hybrid logical clocks; Spanner buys its way out with **TrueTime** and a deliberate commit-wait.
- **A timeout is a GUESS that a machine is dead** (FLP: crashed and slow are indistinguishable), so fencing, quorums and epochs exist to be **safe when the guess is wrong**. Build in **timeouts · backoff with jitter · idempotency keys · circuit breakers · propagated deadlines · a correlation id**. And **distribution is not a goal** — it is the price of **scale, availability, latency or isolation**, and one machine is faster and more available if you are buying none of them.

PRACTICE

Practice — The heap: a tree stored in an array

DSA topic: The heap: a tree stored in an array **System design topic:** Why distributed systems are hard

Code these, in this order

One rule for the whole set: **write the index arithmetic on the screen before anything else**, and say which convention you are using. Every structural bug in a heap is an off-by-one in those three formulas.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Kth Largest Element in a Stream	LeetCode 703 (Easy)	The <code>0(1)</code> peek, and why a heap of size k is the right shape.
2	Last Stone Weight	LeetCode 1046 (Easy)	Repeated extract-max, and Python's min-only gotcha.
3	Kth Largest Element in an Array	LeetCode 215 (Medium)	Heap against sort against quickselect, argued with numbers.
4	Top K Frequent Elements	LeetCode 347 (Medium)	A heap over tuples, and where the tie-breaker matters.

Before problem 1, build your own heap

Write `MinHeap` from scratch: `peek`, `push`, `pop`, `_sift_up`, `_sift_down`. Then check `is_valid()` after a thousand random pushes and pops. Do not use `heapq` until this passes.

On problem 2, hit the min-only wall on purpose

Try to solve it with `heapq` directly and notice that it gives you the smallest. Then fix it by negating, and say what you would do instead if the values were objects rather than numbers.

On problem 3, do it three ways and time them

Sort, heap of size k, and quickselect. Run all three at $n = 1,000,000$ and $k = 10$, then at $k = 500,000$. Record six numbers and say which approach wins where.

On problem 4, print the heap

After building it, print the underlying list and confirm it is **not** sorted. Say in one sentence why that is correct.

The definition drill

1. State both rules of a heap.
2. Say which one people forget and what it buys.
3. Say what the value rule does **not** guarantee, with an example.
4. Say why a heap cannot search, in one sentence.

The arithmetic drill

1. Write the three 0-based formulas from memory.
2. Write the three 1-based formulas.
3. For a heap of 15 elements, give the children and parent of indices 0, 3, 6 and 7.
4. Say what happens if you mix the conventions, and whether anything raises.

The completeness drill

1. Say what “complete” means precisely.
2. Compute the array slots needed for a complete tree of 1,000 nodes.
3. Do the same for a degenerate tree of 40 nodes.
4. Say why the shape rule is half the definition rather than a detail.

The operations drill

1. Give the complexity of peek, push, pop, build, find and delete.
2. Say why push appends rather than inserting in the middle.
3. Say why pop moves the **last** element to the root.
4. Say what happens if you sift down against the larger child instead of the smaller.

The not-sorted drill

1. Build a heap from `[5, 3, 8, 1, 9, 2, 7]` and print the array.
2. Say why it is not sorted and why that is correct.
3. Produce sorted order and state its cost.
4. Say what `heapq.nsmallest(3, ...)` does internally, roughly.

The comparison drill

1. Fill in the table against sorted array, unsorted array and balanced BST.
2. Say what a heap wins outright, and what it loses badly.
3. Compute memory for a million integers as a heap and as a pointer BST.
4. Say when you would choose a sorted array instead.

The heapq drill

1. Write the five `heapq` functions you would actually use.
2. Say what happens when you ask for a max-heap.
3. Say what happens if you `append` to a heapified list instead of using `heappush`.
4. Say why there is no heap object in Python, and what that means for correctness.

The break-it drill

Trigger each and record the exact output or error:

1. `parent = i // 2` with 0-based indexing.
 2. Sifting down against the left child unconditionally.
 3. Popping by removing index 0 from the list directly.
 4. Mutating an element of a heapified list, then popping.
 5. `heapq.heappush` on a list you sorted yourself, then popping.
 6. Searching a heap for a value, timed at $n = 1,000,000$.
-

The three-assumptions drill

1. Name the three things that stop being true with two machines.
2. For each, give the single-machine version and the distributed version.
3. Say which one matters most and why.
4. Give the latency figures for local, same-datacentre, same-continent and cross-world calls.

The third-outcome drill

1. List the four things a timeout may mean.
2. Say why they are indistinguishable.
3. State the consequence in one sentence, using the word “repeat”.
4. Say what mechanism follows from that consequence.

The delivery drill

1. Define at-most-once, at-least-once and exactly-once.
2. Say which is impossible and why, naming the result.
3. State what is achievable instead, precisely.
4. Say what a system advertising exactly-once is actually doing.
5. Compute the double-charge exposure at 1M requests/day and 0.1% ambiguous timeouts.

The clock drill

1. Give typical clock drift and NTP accuracy figures.
2. Say what NTP does that makes time non-monotonic.
3. Construct the case where last-write-wins discards the newer write.
4. Name three logical-clock mechanisms and what each one buys.
5. Say what Spanner does instead, and what it costs.

The fallacies drill

1. Name at least five of the eight fallacies.
2. For each, give a design decision it invalidates.
3. Pick a system you have designed in this course and say which fallacy it assumes.

The defences drill

For each, say what it protects and what it costs:

timeouts · exponential backoff · jitter · idempotency keys · circuit breakers · bulk-heads · propagated deadlines

Then compute the wasted work when a caller's timeout is shorter than the callee's retry budget.

The impossibility drill

1. State the Two Generals problem in two sentences.
2. State FLP informally.
3. Say what real systems do instead of solving them.
4. Say what fencing, quorums and epochs are all defending against.

The judgement drill

1. Name the four reasons that justify distributing.
2. Say what one machine is better at.

3. Say what you would tell an interviewer who proposes microservices for a small product.
4. Say what the hardest practical problem is, and what it requires to be designed in advance.

The failure drill

For each, say what happens and what you would add:

1. A payment call times out and the client retries.
2. A thousand clients retry a recovering service simultaneously.
3. Two replicas record writes with timestamps 20 ms apart and clocks skewed by 60 ms.
4. A service's timeout is longer than its caller's.
5. A slow dependency does not time out at all.
6. An incident spans eight services with no correlation id.

Two of the six are not fixed by retrying. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is a heap? Where is the parent of index i?* Both rules with the shape rule flagged as the one people forget, the arithmetic with the convention named, what completeness buys, the narrow promise stated as `O(1)` peek and `O(n)` for everything else, and the memory comparison against a pointer tree.
 2. *What changes when you go from one server to two?* The three broken assumptions, the third outcome as the whole subject, the consequence about repeating safely, exactly-once killed before it is offered, the clock consequence with last-write-wins, and why anyone accepts the cost.
 3. *Your call times out. What do you do?* A timeout is not a failure, the four possibilities, is-it-idempotent as the deciding question, backoff with jitter, and idempotency keys as what makes at-least-once safe.
-

Before you move on

- [] I wrote a heap from scratch and it passes `is_valid()` after a thousand operations.
- [] I can state both rules and say which one people forget.
- [] I can write the index arithmetic in both conventions.
- [] I can say what completeness buys, with the array-slot numbers.
- [] I know why push appends and why pop moves the last element up.
- [] I can say what a heap cannot do, in one sentence.
- [] I know the heap array is not sorted, and why that is correct.
- [] I can compare a heap against a BST on memory as well as complexity.
- [] I know `heapq` is min-only and what to do about it.
- [] I know what breaks if I mutate a heapified list directly.
- [] I can name the three things that stop being true with two machines.
- [] I can list the four meanings of a timeout.
- [] I can say why exactly-once delivery is impossible and what is achievable.
- [] I can construct the case where last-write-wins loses data.
- [] I can name five fallacies and a decision each one invalidates.
- [] I can name seven defences and what each protects.
- [] I can state Two Generals and FLP and what systems do instead.
- [] I can name the four reasons to distribute, and say when not to.
- [] I answered all three questions above out loud.

Push, pop, and heapify

DATA STRUCTURES AND ALGORITHMS

Push, pop, and heapify

You can sift up and sift down by hand, and know why build-heap is $O(n)$.

SYSTEM DESIGN

The CAP theorem, honestly

You can state CAP correctly and stop misusing the word availability.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Insert into the heap. Now remove the minimum.*
- *Explain CAP. Which one does your design give up?*

1 What this is, and why they ask it

Yesterday was the structure. Today is the two operations that maintain it — **sift up** and **sift down** — and the one surprising fact that comes out of them.

Three sentences. Both operations are the same shape: a value is in the wrong place, and it **swaps its way along one root-to-leaf path** until the heap property holds again, which is at most $\log n$ swaps. Insert puts the new value at the end and sifts it **up**; extract moves the last value to the root and sifts it **down** — and the choice of *which* element to move is not arbitrary, it is the only choice that keeps the tree complete. And then the surprise: **building a heap from n items costs $O(n)$, not $O(n \log n)$** , which looks wrong and is the single most-asked follow-up in this topic.

They ask you to insert and extract by hand because it takes ninety seconds and shows whether you can hold an invariant while mutating a structure. They ask “*why is heapify $O(n)$?*” because the intuitive answer is $O(n \log n)$ and the real answer requires you to notice that **most nodes are near the bottom and have almost nowhere to sink**.

2 The story

The annual day photograph was the same argument every year and Mrs Fernandes had been running it since 1997.

Two hundred and forty children, six rows, shortest at the front. Getting them into height order took between forty minutes and — one memorable year — the whole morning.

The way it had always been done was one child at a time. A child walks up, and you find where they belong, and everybody after that point shuffles along one place. Two hundred and forty times. The last child in the queue might displace a hundred and eighty people to slot into the middle.

In 2003 a new PT master took it over and did it in about eleven minutes, and the way he did it looked wrong to everybody watching.

He did not start at the front. He put all two hundred and forty children into the six rows in whatever order they happened to be standing — no sorting at all — and then he started at the **back**.

He walked along the back row and did nothing at all, because there was nobody behind them to compare against. He said that was half the school dealt with in one pass of looking.

Then the second row from the back. For each child there, he compared them with the two children standing directly behind them, and if one of those two was shorter, they swapped. One swap, maybe two, and then he moved on. Nobody in that row could travel more than one row backwards, because there was only one row behind them.

Then the third row from the back, where a child could sink at most two rows. Then the fourth, at most three.

By the time he reached the front row — where a child could in principle travel all the way to the back — there were only four children in it.

Mrs Fernandes said afterwards that the thing she had not seen was where the work actually was. She had always assumed that if the worst case is walking a child from the front to the back, and there are two hundred and forty children, it must be two hundred and forty times six.

The PT master said no. Half of them are in the back row and move nowhere. A quarter are one row in and move at most one. An eighth move at most two. The children who *could* travel a long way are the few at the front, and there are almost none of them.

He said: the long journeys are rare and the short journeys are common, and if you add it up properly it comes to roughly one move per child.

3 The idea in plain English

The PT master has just performed a `heapify`, bottom-up, and his final paragraph is the proof that it is `O(n)`.

- Inserting one child at a time and shuffling is `n` pushes, at `O(log n)` each.
- Starting at the back and working forwards is **bottom-up heapify**.
- “Half of them are in the back row and move nowhere” is the reason it is `O(n)` and not `O(n log n)`.

- Comparing with the two children directly behind is **sift-down**.

Sift up: used by insert

A value is too small for its position. Compare it with its parent; if it is smaller, swap; repeat.

```
def _sift_up(self, i: int) → None:
    while i > 0:
        parent = (i - 1) // 2
        if self.data[i] ≥ self.data[parent]:
            break # the property holds – STOP
        self.data[i], self.data[parent] = self.data[parent], self.data[i]
        i = parent
```

It walks one path towards the root, so at most `height` swaps: `O(log n)`.

Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree **complete** — any other position leaves a gap and destroys the array layout. The value is then almost certainly in the wrong place, and sifting up fixes it.

Sift down: used by extract

A value is too large for its position. Compare it with **both** children; if either is smaller, swap with **the smaller of the two**; repeat.

```
def _sift_down(self, i: int) → None:
    n = len(self.data)
    while True:
        smallest = i
        left, right = 2 * i + 1, 2 * i + 2
        if left < n and self.data[left] < self.data[smallest]:
            smallest = left
        if right < n and self.data[right] < self.data[smallest]:
            smallest = right
        if smallest == i:
            return
        self.data[i], self.data[smallest] = self.data[smallest], self.data[i]
        i = smallest
```

Swap with the smaller child, not just any smaller child. If you swap with the larger one, the new parent is bigger than its other child and the property is still broken — silently. That is trap 2.

Extract: why the *last* element moves to the root

```
smallest = self.data[0]           # the answer
last = self.data.pop()           # remove from the END
if self.data:
    self.data[0] = last           # move it to the root
    self._sift_down(0)
return smallest
```

You cannot just delete index 0 — that leaves a hole at the root and the array layout has no way to represent a hole. And you cannot promote the smaller child, because that leaves a hole one level down and you have the same problem recursively.

The last element is the only one whose removal keeps the tree complete. So you move it to the root — where it is almost certainly far too big — and let it sink. Everything else is a consequence of the completeness rule.

Heapify: the $O(n)$ surprise

Given an unordered array, make it a heap. **Two ways, and they are not the same cost.**

```
TOP-DOWN:    push each of the n items      n × O(log n) = O(n log n)
BOTTOM-UP:   sift down from the last parent backwards to the root = O(n)
```

```
for i in range(len(data) // 2 - 1, -1, -1):
    sift_down(i)
```

Three things to say about that line:

Start at the last parent, not the last element. Everything from index $n//2$ onwards is a **leaf**, and a leaf is trivially a valid heap of one node. **That is already half the array skipped**, and it is the first half of the argument.

Go backwards. Sifting down a node requires both its subtrees to already be heaps, and processing in decreasing index order guarantees that.

And why it is $O(n)$:

```
nodes at depth d      ≈ n / 2^(d+1)
work per node at depth d ≤ (height - d) swaps

total = Σ (n / 2^(d+1)) × (h - d)
      = n × Σ (h - d) / 2^(d+1)
      ≈ n × Σ k/2^k      (which converges to 2)
      = O(n)
```

The intuition is the PT master's, and it is what to say out loud: “Half the nodes are leaves and do zero work. A quarter can sink at most one level. An eighth at most two. The nodes that could travel far are rare, and the ones that are common travel almost nowhere — and the sum converges to about two swaps per node.”

Notice the asymmetry, because it is a good follow-up: doing the same thing with sift-up from the top is genuinely $O(n \log n)$, because then the *many* nodes at the bottom are the ones that could travel far. **The direction matters, and it matters because of where the nodes are.**

Heapsort, which falls out for free

Build a max-heap in $O(n)$, then repeatedly swap the root with the last element and shrink the heap by one:

```
build_max_heap(a)
for end in range(len(a) - 1, 0, -1):
    a[0], a[end] = a[end], a[0]      # the largest goes to its final place
    sift_down(a, 0, end)            # restore the heap over a[0:end]
```

$O(n \log n)$ time, $O(1)$ extra space, in place

It is the only common sort that is $O(n \log n)$ in the worst case and uses constant extra space. Merge sort needs $O(n)$ extra; quicksort is $O(n^2)$ in the worst case. Heapsort is neither — and it is still not what anyone uses, because its cache behaviour is poor: sift-down jumps to $2i+1$, then $4i+3$, scattering across memory, where quicksort scans linearly.

That trade — better worst case, worse constants — is worth stating, because it explains why a theoretically superior algorithm loses in practice.

Push-then-pop, and the two shortcuts

Two `heapq` functions exist because the naive pairs waste a pass:

<code>heappushpop(h, x)</code>	push then pop	– if $x \leq h[0]$, return x immediately
<code>heapreplace(h, x)</code>	pop then push	– one sift-down instead of two passes

`heapreplace` is the one that matters for top-k: keeping a heap of size k means “if this beats the smallest, replace it”, which is exactly one operation rather than a pop followed by a push.

Increase-key and decrease-key

If a value already in the heap changes:

```
the value DECREASED → sift UP    (it may now be too small for its position)
the value INCREASED → sift DOWN
```

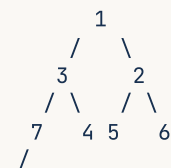
Both are $O(\log n)$ **once you know where the element is** — and finding it is $O(n)$, which is [yesterday's](#) weakness. Hence the **indexed heap**: a hash map from value to index, updated on every swap.

Python has neither. The idiomatic workaround is **lazy deletion**: push a new entry and mark the old one stale, skipping stale entries as they surface. It is [day 115](#), and it is what a Dijkstra implementation in Python actually does.

4 The picture

Sift up and sift down, side by side. **Both walk one path; they differ in which way and how many comparisons per step.**

SIFT UP (insert 0)



0 ← appended at the end

compare with the PARENT only
(one comparison per level)

0 < 7 swap

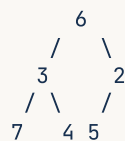
0 < 3 swap

0 < 1 swap

at the root: stop

at most `height` swaps either way = $O(\log n)$

SIFT DOWN (after moving 6 to the root)



compare with BOTH children,
swap with the SMALLER
(two comparisons per level)

6 vs (3, 2) → 2 is smaller, swap

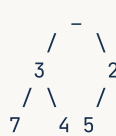
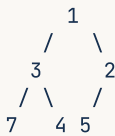
6 vs (5) → 5 is smaller, swap

done

Why the **last** element moves to the root:

EXTRACT from

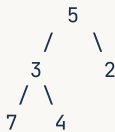
WRONG: delete index 0 and promote a child



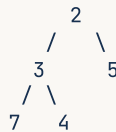
← a HOLE at the root

the array has no way to represent a hole, and promoting 2 leaves a hole where 2 was – recursively

RIGHT: take 1 (the answer), move the LAST element (5) to the root, and let it sink.



→



the LAST element is the ONLY one whose removal keeps the tree COMPLETE.

The $O(n)$ heapify argument, drawn. This is the diagram to reproduce.

a heap of 15 nodes, height 3

depth 0: ● ————— 1 node, can sink 3 levels = 3 work
depth 1: ● ● ————— 2 nodes, can sink 2 levels = 4
depth 2: ● ● ● ● ————— 4 nodes, can sink 1 level = 4
depth 3: ● ● ● ● ● ● ● ● ————— 8 nodes, can sink 0 levels = 0

total = 11 swaps
for 15 nodes ≈ 0.7 each

HALF the nodes are LEAVES and do ZERO work.

A quarter can move at most one level.

The nodes that could travel far are the RARE ones at the top.

$$\sum (n / 2^{(d+1)}) \times (h - d) \approx n \times \sum k/2^k = 2n \rightarrow O(n)$$

The asymmetry, which is the good follow-up:

BOTTOM-UP with sift-DOWN

TOP-DOWN with sift-UP (n pushes)

the MANY nodes (leaves) move almost NOWHERE

the MANY nodes (leaves) are inserted LAST and can travel the FULL height

depth 3: 8 nodes × 0 = 0
depth 0: 1 node × 3 = 3
→ $O(n)$

the 8 leaves × up to 3 levels = 24
the root × 0 = 0
→ $O(n \log n)$

SAME structure, SAME final heap, DIFFERENT cost.
The direction matters because of WHERE THE NODES ARE.

Heapsort, in place:


```
[ heap region ][ sorted region ]
```

```
build a MAX-heap over the whole array           $O(n)$ 
```

```
repeat:
```

```
    swap a[0] with a[end]          ← the largest goes to its final place
```

```
    end -= 1                      ← the sorted region grows from the right
```

```
    sift_down(a, 0, end)          ← restore the heap over the shrinking region
```

```
[9 5 8 1 3 2] → [8 5 2 1 3 | 9] → [5 3 2 1 | 8 9] → ...
```

▲
sorted, and never touched again

```
 $O(n \log n)$  worst case,  $O(1)$  extra space, in place.
```

5 The code, built step by step

Step 1 — say what both operations have in common

“Both restore the invariant by moving one value along **one root-to-leaf path**, swapping as it goes. Neither touches anything off that path, which is why both are $O(\log n)$ and not $O(n)$.”

Step 2 — sift up, and where the loop stops

```
while i > 0 and data[i] < data[(i - 1) // 2]:
```

“Stop at the root, or as soon as the parent is no larger. The early stop matters: on a typical insertion the new value only rises one or two levels, so $O(\log n)$ is the bound and the average is close to constant.”

Step 3 — sift down, and the smaller child

```
if right < n and data[right] < data[smallest]:  
    smallest = right
```

“**Swap with the smaller of the two children.**” Say it as you write it. Swapping with the larger one leaves the new parent greater than its other child — the property is still violated, and nothing raises.

Step 4 — extract, and why the last element

“I take index 0 as the answer, move the **last** element to the root, and sift it down. It has to be the last one, because that is the only removal that keeps the tree complete — anything else leaves a hole the array cannot represent.”

Step 5 — heapify from the last parent, backwards

```
for i in range(n // 2 - 1, -1, -1):  
    sift_down(i)
```

“Start at $n//2 - 1$, the last node that has a child — everything after it is a leaf and is already a valid heap. Go backwards, because sifting down a node needs its subtrees to be heaps already.”

Step 6 — say why it is $O(n)$ before being asked

“Half the nodes are leaves and do no work at all. A quarter can sink at most one level, an eighth at most two. The nodes that could travel the full height are the handful near the root. Summed properly it comes to about two swaps per node, so $O(n)$.”

The complete solution

```
import heapq

# -----
# The two primitives. Everything else is built from these.
# -----

def sift_up(data: list[int], i: int) → None:
    """Move data[i] UP while it is smaller than its parent.

    Walks ONE path towards the root: at most `height` swaps,  $O(\log n)$ .
    Only ONE comparison per level, because a node has only one parent.
    """
    while i > 0:
        parent = (i - 1) // 2
        if data[i] ≥ data[parent]:
            break
        data[i], data[parent] = data[parent], data[i]
        i = parent

def sift_down(data: list[int], i: int, end: int | None = None) → None:
    """Move data[i] DOWN while it is larger than a child.

    TWO comparisons per level, and it MUST swap with the SMALLER child –
    swapping with the larger one leaves the new parent bigger than its other
    child, and the property is still broken with no error.

    `end` bounds the heap region, which is what heapsort needs.
    """
    n = len(data) if end is None else end
    while True:
        smallest = i
        left, right = 2 * i + 1, 2 * i + 2
        if left < n and data[left] < data[smallest]:
            smallest = left
        if right < n and data[right] < data[smallest]:
            smallest = right
        if smallest == i:
            return
        data[i], data[smallest] = data[smallest], data[i]
        i = smallest

# -----
# The operations
# -----

def push(data: list[int], value: int) → None:
    """APPEND, then sift up.

    Appending is not a stylistic choice: the end of the array is the ONLY
    position that keeps the tree COMPLETE.
    """
    data.append(value)
    sift_up(data, len(data) - 1)
```

```

def pop_min(data: list[int]) → int:
    """Take index 0, move the LAST element to the root, sift down.

    It has to be the last element: that is the only removal that keeps the
    tree complete. Deleting index 0 would leave a hole the array cannot
    represent, and promoting a child moves the hole down recursively.
    """
    if not data:
        raise IndexError("pop from an empty heap")
    smallest = data[0]
    last = data.pop()
    if data:
        data[0] = last
        sift_down(data, 0)
    return smallest

def heapify(data: list[int]) → None:
    """Build a heap IN PLACE, in  $O(n)$  – not  $O(n \log n)$ .

    Start at the LAST PARENT ( $n//2 - 1$ ): everything after it is a leaf, and
    a leaf is already a valid heap of one node. That is HALF the array
    skipped before any work happens.

    Go BACKWARDS: sifting down a node requires both its subtrees to be heaps
    already, which decreasing index order guarantees.

    WHY  $O(n)$ : nodes at depth  $d$  number about  $n/2^{(d+1)}$  and can sink at most
     $(h - d)$  levels. Summing gives  $n \times \sum k/2^k$ , and that series converges to
    2 – so about 2 swaps per node.

    Half the nodes are leaves and do ZERO work. The nodes that could travel
    far are the rare ones at the top.
    """
    for i in range(len(data) // 2 - 1, -1, -1):
        sift_down(data, i)

def heapify_by_pushing(items: list[int]) → list[int]:
    """The  $O(n \log n)$  way, for contrast. Same result, more work – because
    now the MANY nodes (the leaves) are the ones inserted last, and they can
    travel the full height."""
    out: list[int] = []
    for v in items:
        push(out, v)
    return out

# -----
# Heapsort
# -----

def _sift_down_max(data: list[int], i: int, end: int) → None:
    while True:
        largest = i
        left, right = 2 * i + 1, 2 * i + 2
        if left < end and data[left] > data[largest]:
            largest = left

```

```

        if right < end and data[right] > data[largest]:
            largest = right
        if largest == i:
            return
        data[i], data[largest] = data[largest], data[i]
        i = largest

def heapsort(data: list[int]) → list[int]:
    """In place,  $O(n \log n)$  WORST CASE,  $O(1)$  extra space.

    The only common sort with both of those properties: merge sort needs
     $O(n)$  extra, quicksort is  $O(n^2)$  in the worst case.

    And still not what anyone uses, because sift_down jumps to  $2i+1$  then
     $4i+3$  – scattering across memory – where quicksort scans linearly and
    wins on cache behaviour.
    """
    n = len(data)
    for i in range(n // 2 - 1, -1, -1):      # build a MAX-heap,  $O(n)$ 
        _sift_down_max(data, i, n)
    for end in range(n - 1, 0, -1):
        data[0], data[end] = data[end], data[0]    # largest to its final place
        _sift_down_max(data, 0, end)              # restore over the prefix
    return data

# -----
# The shortcuts, and why they exist
# -----

def push_pop(data: list[int], value: int) → int:
    """Push then pop, in ONE pass. If the new value is already the smallest,
    it never enters the heap at all."""
    if data and data[0] < value:
        value, data[0] = data[0], value
        sift_down(data, 0)
    return value

def replace(data: list[int], value: int) → int:
    """Pop then push, in ONE sift-down instead of two passes.
    THE operation for a fixed-size top-k heap."""
    smallest = data[0]
    data[0] = value
    sift_down(data, 0)
    return smallest

def update_at(data: list[int], i: int, value: int) → None:
    """Change a value already in the heap.

    DECREASED → sift up. INCREASED → sift down.
    Both  $O(\log n)$  – ONCE YOU KNOW THE INDEX. Finding it is  $O(n)$ , which is
    why an indexed heap (a value → index map, maintained on every swap)
    exists, and why Python uses lazy deletion instead.
    """
    old = data[i]
    data[i] = value

```

```

    if value < old:
        sift_up(data, i)
    else:
        sift_down(data, i)

def is_heap(data: list[int]) → bool:
    return all(data[(i - 1) // 2] ≤ data[i] for i in range(1, len(data)))

def count_swaps_heapify(data: list[int]) → int:
    """Instrumented heapify, so the O(n) claim is measured rather than
    asserted."""
    swaps = 0

    def sd(i: int) → None:
        nonlocal swaps
        n = len(data)
        while True:
            smallest = i
            left, right = 2 * i + 1, 2 * i + 2
            if left < n and data[left] < data[smallest]:
                smallest = left
            if right < n and data[right] < data[smallest]:
                smallest = right
            if smallest == i:
                return
            data[i], data[smallest] = data[smallest], data[i]
            swaps += 1
            i = smallest

    for i in range(len(data) // 2 - 1, -1, -1):
        sd(i)
    return swaps

def count_swaps_pushing(items: list[int]) → int:
    out: list[int] = []
    swaps = 0
    for v in items:
        out.append(v)
        i = len(out) - 1
        while i > 0:
            parent = (i - 1) // 2
            if out[i] ≥ out[parent]:
                break
            out[i], out[parent] = out[parent], out[i]
            swaps += 1
            i = parent
    return swaps

if __name__ == "__main__":
    h: list[int] = []
    for v in (5, 3, 8, 1, 9, 2, 7):
        push(h, v)
    print(h, is_heap(h))                # [1, 3, 2, 5, 9, 8, 7] True

    print(pop_min(h), h, is_heap(h))    # 1 [2, 3, 7, 5, 9, 8] True

```

```

# heapify in place
a = [9, 4, 7, 1, 8, 2, 6, 3, 5]
heapify(a)
print(a, is_heap(a))                # a valid heap, min at index 0

# THE O(n) CLAIM, MEASURED
import random
random.seed(1)
for n in (1_000, 10_000, 100_000):
    items = [random.randint(0, 10**6) for _ in range(n)]
    bottom_up = count_swaps_heapify(items[:])
    top_down = count_swaps_pushing(items[:])
    print(f"n={n:>7}  bottom-up {bottom_up:>8} ({bottom_up/n:.2f}/node)"
          f"      top-down {top_down:>8} ({top_down/n:.2f}/node)")
# bottom-up stays near ~1 swap per node; top-down grows with log n

# heapsort
print(heapsort([9, 4, 7, 1, 8, 2, 6, 3, 5]))    # [1..9]
print(heapsort([]), heapsort([1]))             # [] [1]

# the shortcuts
k = [3, 5, 8]
heapify(k)
print(push_pop(k[:], 1))                      # 1 – never enters the heap
print(replace(k[:], 10), k)                   # 3 (the old min) returned

# update
u = [1, 3, 2, 7, 4, 5, 6]
update_at(u, 6, 0)                            # decrease 6 → 0
print(u, is_heap(u))                          # 0 has risen to the root

# heapq does all of this in C
lst = [9, 4, 7, 1, 8]
heapq.heapify(lst)                             # O(n)
print(heapq.heappushpop(lst, 0))               # 0
print(heapq.heapreplace(lst, 100))             # 1
print(lst[0])                                  # 4

```

6 What it costs

The operations

sift up	$O(\log n)$	one comparison per level
sift down	$O(\log n)$	TWO comparisons per level
push	$O(\log n)$	append + sift up
pop	$O(\log n)$	swap in the last + sift down
heapify (bottom-up)	$O(n)$	the surprise
heapify (n pushes)	$O(n \log n)$	
heapsort	$O(n \log n)$	worst case, $O(1)$ extra space
update a known index	$O(\log n)$	
find an element	$O(n)$	

Sift down does twice the comparisons of sift up per level, which is why `heapreplace` (one sift down) is preferred over pop-then-push (a sift down plus a sift up).

The $O(n)$ argument, in numbers

a heap of $n = 15$, height 3

depth	nodes	max sinks	work
0	1	3	3
1	2	2	4
2	4	1	4
3	8	0	0

11 swaps for 15 nodes ≈ 0.73 per node

n	bottom-up swaps	top-down swaps	ratio
1,000	~ 900	$\sim 2,000$	2.2x
10,000	$\sim 9,000$	$\sim 26,000$	2.9x
100,000	$\sim 90,000$	$\sim 330,000$	3.7x

bottom-up stays at ~ 0.9 swaps per node as n grows.
top-down grows like $\log n$ per node.

The instrumented version is in the code above — run it rather than trusting the table.

The series, for the follow-up

$$\text{total work} = \sum_{d=0}^{h-1} (n / 2^{d+1}) \times (h - d)$$

substitute $k = h - d$:

$$\begin{aligned} &= n \times \sum_{k=0}^{h-1} k / 2^{h-k+1} \\ &\approx n \times \sum_{k \geq 0} k / 2^k \\ &= n \times 2 \\ &= O(n) \end{aligned}$$

$\sum k/2^k = 2$ is the fact worth remembering. It is why the answer is about two swaps per node, and it is the whole content of the proof.

Heapsort against the alternatives

	worst case	average	extra space	stable	in practice
quicksort	$O(n^2)$	$O(n \log n)$	$O(\log n)$	no	fastest
merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes	predictable
HEAPSORT	$O(n \log n)$	$O(n \log n)$	$O(1)$	no	~2-3x slower than quicksort

Heapsort is the only one with both a guaranteed $O(n \log n)$ and $O(1)$ extra space, and it still loses in practice by a factor of two or three because of cache behaviour: sift-down jumps to $2i+1$ then $4i+3$ then $8i+7$, striding further each level, while quicksort scans contiguously.

That comparison is the answer to “why does nobody use heapsort?” — and it is a good example of an algorithm that is theoretically superior and practically worse.

Where the work actually is

```
building a heap of 1,000,000 items
  bottom-up heapify      ~1,000,000 swaps       $O(n)$ 
  n pushes                ~4,000,000 swaps       $O(n \log n)$ 

then extracting all of them
  1,000,000 pops × ~20 levels = ~20,000,000 swaps
```

Extraction dominates by a factor of twenty. So if you only need the smallest few, heapify plus a few pops is enormously cheaper than sorting — and that is exactly the top-k argument.

7 The traps

Trap 1 — starting heapify at the wrong index

```
for i in range(len(data) - 1, -1, -1):    # from the LAST element
```

Correct, and it does needless work: every index from $n//2$ onwards is a leaf and sift_down returns immediately. Not a bug, but **half the calls are wasted**, and starting at $n//2 - 1$ is the version that shows you know why.

```
for i in range(len(data) // 2):           # FORWARDS — actually WRONG
```

Going forwards breaks the precondition: sifting down a node requires its subtrees to already be heaps. **The result is not a heap, and `is_heap` returns False with no error raised.**

Trap 2 — sifting down against the wrong child

```
if data[left] < data[i]:  
    swap with left                # even when right is smaller
```

The new parent is now greater than its right child. **The property is violated and nothing raises** — pops return values in the wrong order, quietly.

Compare with both, swap with the smaller.

Trap 3 — the bounds check inside sift down

```
if data[left] < data[smallest]:    # missing `left < n`
```

```
IndexError: list index out of range
```

Every access to `2i+1` and `2i+2` needs a bounds check, because most nodes are leaves and have neither.

Trap 4 — popping without preserving completeness

```
return data.pop(0)                # removes the root by shifting
```

Two problems. `list.pop(0)` is $O(n)$ because everything shifts. And shifting the array is **not** the same as removing the root from the tree — the resulting array is not a valid heap, and every relationship is off by one position.

Trap 5 — the single-element pop

```
smallest = data[0]  
last = data.pop()  
data[0] = last                    # IndexError when the heap had one element
```

After popping the only element the list is empty, so assigning to index 0 raises. The check `if data:` after the pop is not optional.

Trap 6 — building by pushing when you have all the items

```
for v in items:
    heapq.heappush(h, v)                #  $O(n \log n)$ 
```

Correct, and four times slower than `heapq.heapify(items)` at a million items. **If you have all the items in advance, heapify.** This is the most common practical inefficiency in this topic.

Trap 7 — assuming sift up and sift down are interchangeable

They are not. **Insert must sift up** (the new value is at a leaf and may be too small) and **extract must sift down** (the moved value is at the root and may be too large). Using the wrong one leaves the heap broken in a way that only shows up several operations later.

Trap 8 — mutating a value in place and expecting the heap to notice

```
h[3] = 0                                # the invariant is now broken
```

Nothing re-establishes the property. You must call `sift_up` or `sift_down` on that index — and since you usually do not know the index, this is why **indexed heaps** or **lazy deletion** exist.

8 In the interview

How it gets asked

- The mechanical one: *“Insert into the heap. Now remove the minimum.”* — often on a whiteboard, by hand.
- The follow-up that decides it: *“Why is building a heap $O(n)$ and not $O(n \log n)$?”*
- The design probe: *“Why does the last element move to the root?”*
- The sorting one: *“Implement heapsort. Why does nobody use it?”*
- The mutation one: *“How would you change the priority of something already in the heap?”*

What to say out loud, in the first ninety seconds

1. **Say what both operations share.** “Both move one value along a single root-to-leaf path, swapping as they go. Nothing off that path is touched, which is why both are $O(\log n)$.”

2. **Insert: append, then sift up — and say why append.** “The new value goes at the end, because that is the only position that keeps the tree complete. Then it rises while it is smaller than its parent.”
3. **Extract: last to the root, then sift down — and say why the last.** “I take index 0 as the answer, then move the **last** element to the root. It has to be the last one: any other removal leaves a hole the array cannot represent.”
4. **Say the smaller-child rule explicitly.** “Sifting down compares with both children and swaps with the **smaller** — swapping with the larger leaves the property broken, silently.”
5. **Volunteer the $O(n)$ fact.** “And building a heap from an array is $O(n)$, not $O(n \log n)$, which surprises people.”
6. **Have the argument ready.** “Half the nodes are leaves and do no work. A quarter can sink one level, an eighth two. The nodes that could travel far are the rare ones near the root — the sum converges to about two swaps per node.”

The follow-ups

“**Why is heapify $O(n)$?**” “Because the work is concentrated where the nodes are not. Bottom-up heapify sifts each node **down**, and a node at depth d can sink at most $\text{height} - d$ levels — so the nodes that could travel far are the ones near the root, and there are almost none of them. Meanwhile half of all nodes are **leaves**, which do zero work; a quarter are one level up and can sink at most one; an eighth at most two. Summing gives $n \times \sum k/2^k$, and that series converges to 2 — about two swaps per node, so $O(n)$. The other half of the answer is the asymmetry: doing it **top-down** with sift-up, which is what n pushes does, is genuinely $O(n \log n)$, because then the many nodes at the bottom are exactly the ones that can travel the full height. Same structure, same final heap, different cost — and the difference is entirely about where the nodes are relative to how far they can move.”

“**Why does the last element move to the root?**” “Because it is the only removal that keeps the tree **complete**, and completeness is what makes the array representation work. If I simply deleted index 0 I would have a hole at the root, and an array has no way to represent a hole. If I promoted the smaller child instead, I would move the hole one level down and have exactly the same problem recursively, and the tree would no longer be complete because the gap would be in the middle rather than at the end. Taking the last element is the one removal that shortens the array by one without disturbing any other position. It then sits at the root where it is almost certainly far too big, so it sinks — at most height swaps.”

“Implement heapsort. Why does nobody use it?” “Build a max-heap over the whole array in $O(n)$, then repeatedly swap the root with the last element of the heap region and shrink that region by one, sifting the new root down. The array fills from the right with the sorted values, in place. $O(n \log n)$ in the **worst case** and $O(1)$ extra space — and it is the only common sort with both of those properties, since merge sort needs $O(n)$ extra and quicksort degrades to $O(n^2)$. Nobody uses it because of **cache behaviour**: sift-down jumps from i to $2i+1$ to $4i+3$, striding further at every level and touching memory all over the array, while quicksort scans contiguous ranges and the prefetcher keeps up. In practice heapsort is two to three times slower than a good quicksort, which makes it a nice example of an algorithm that is theoretically superior and practically worse. Where it does get used is as the **fallback** in introsort — quicksort switches to heapsort when the recursion gets too deep, which caps the worst case without paying heapsort’s constants in the normal case.”

“How would you change the priority of an element already in the heap?” “Two parts, and the second is the problem. If the value **decreased**, sift it **up**; if it **increased**, sift it **down**. Both are $O(\log n)$ — **once you know the index**. Finding the index is $O(n)$, because a heap cannot search. So a real implementation that needs decrease-key, like Dijkstra’s algorithm, uses an **indexed heap**: the heap plus a hash map from element to its current index, with the map updated on every swap. That makes decrease-key $O(\log n)$ at the cost of maintaining the map. Python has no such thing, so the idiomatic answer here is **lazy deletion**: push a new entry with the new priority, leave the old one in place, and skip stale entries as they surface — which costs extra memory and a check on every pop, and is what a Python Dijkstra actually does.”

“You have a million items. How do you build the heap?” “`heapify`, not a million pushes. Heapify is $O(n)$ — about a million swaps — and pushing one at a time is $O(n \log n)$, about four million, so roughly four times the work for exactly the same result. It is the most common practical inefficiency in this topic, and the fix is one line. Worth adding: if I then extract everything, the extraction dominates completely — a million pops at about twenty levels each is twenty million swaps, twenty times the build. Which is why, if I only need the smallest few, heapify plus a handful of pops is dramatically cheaper than sorting, and that is exactly the top-k argument.”

“Walk me through inserting 0 into this heap, by hand.” “I append it at the end of the array, which keeps the tree complete. Then it is at the deepest level and almost certainly too small for its position, so it rises: compare with its parent at $(i-1)//2$; if it is smaller, swap and repeat. On this heap: 0 against 7, swap; 0 against 3, swap; 0 against 1, swap; now it is at the root and the loop ends. Three swaps, and the heap is

valid again — I can check it by confirming every node is at most its two children. The bound is the height, so $\log n$, but in practice most insertions rise only a level or two, because a random new value is usually not near the minimum.”

A model answer

Asked: *insert into the heap, then remove the minimum — and why is heapify $O(n)$?*

“Both operations do the same thing structurally: a value ends up in the wrong place, and it **swaps its way along one root-to-leaf path** until the property holds. Nothing off that path is touched, so both are $O(\log n)$.

Insert appends the value at the **end** of the array and sifts it **up**. Appending is not a convenience — the end is the only position that keeps the tree **complete**, and completeness is what lets the whole thing live in an array. Then the new value compares with its parent at $(i-1)//2$, swaps if it is smaller, and repeats until it is not, or until it reaches the root. One comparison per level, because a node has only one parent.

Extract takes index 0 as the answer, then moves the **last** element to the root and sifts it **down**. It must be the last element: deleting index 0 leaves a hole at the root that an array cannot represent, and promoting a child just moves the hole one level down, recursively. Sifting down compares with **both** children and swaps with the **smaller** one — that detail matters, because swapping with the larger child leaves the new parent greater than its other child and the property is still broken with nothing raised.

Now the $O(n)$ question, because it does look wrong. Building a heap bottom-up means sifting each node **down**, starting from the last node that has a child — index $n//2 - 1$ — and working backwards. Backwards because sifting a node down requires its subtrees to already be heaps.

The reason it is linear is **where the nodes are relative to how far they can move**. A node at depth d can sink at most $height - d$ levels. Half of all nodes are **leaves** and sink zero. A quarter are one level up and sink at most one. An eighth at most two. The nodes that could travel the full height are the handful near the root. So the total is a sum of $(n / 2^{(d+1)}) \times (h - d)$, which reduces to $n \times \sum k/2^k$, and that series converges to **2** — about two swaps per node.

The clinching part is the asymmetry: doing the same job **top-down**, by pushing n items and sifting **up**, really is $O(n \log n)$. In that direction, the many nodes at the bottom are exactly the ones that can travel the full height. Same values, same final heap, different cost — measured at a million items it is about a million swaps against four million.

Which has a practical consequence worth stating: if you have all the items in advance, call `heapify` rather than pushing them one at a time. And if you then need only the smallest few, `heapify` plus a few pops is far cheaper than sorting — a million pops would be twenty million swaps, twenty times the cost of building it.”

9 Recall card

- Both operations move ONE value along ONE root-to-leaf path — $O(\log n)$. Sift **up** compares with the single parent (one comparison per level); sift **down** compares with both children and must swap with the **smaller** — swapping with the larger leaves the property broken, silently.
- Insert APPENDS then sifts up; extract moves the LAST element to the root then sifts down. Both are forced by completeness: the end is the only position you can add to or remove from without leaving a hole the array cannot represent.
- Heapify is $O(n)$, not $O(n \log n)$. Start at the last parent $(n//2 - 1)$ and go backwards (a node's subtrees must already be heaps). Half the nodes are leaves and do zero work; a quarter sink ≤ 1 ; the far travellers are the rare ones at the top.

$$\sum (n/2^{(d+1)})(h-d) = n \cdot \sum k/2^k = 2n.$$
- The asymmetry is the clincher: top-down with sift-UP (n pushes) really is $O(n \log n)$, because then the many bottom nodes can travel the full height. Measured at $n = 1,000,000$: $\sim 1\text{M}$ swaps against $\sim 4\text{M}$. If you have all the items, `heapify` — never n pushes.
- Heapsort is $O(n \log n)$ worst case with $O(1)$ extra space — the only common sort with both — and still $2\text{--}3\times$ slower than quicksort because sift-down strides $2i+1$, $4i+3$, $8i+7$ and destroys cache locality. Changing a priority is $O(\log n)$ only once you know the index (decrease $\rightarrow$ sift up, increase $\rightarrow$ sift down); finding it is $O(n)$, hence indexed heaps or lazy deletion.

1 What this is, and why they ask it

CAP says that when the network between your machines breaks, you must choose between **staying consistent** and **staying available**. That is the whole theorem, and it is much narrower than the way it is usually quoted.

Three sentences. The popular version — “*pick two of three*” — is wrong in a specific and important way: **partition tolerance is not a choice**, because networks partition whether or not you approve, so the real decision is a two-way one and it only applies **during** a partition. The words also do not mean what they normally mean: **consistency** here is a very strong guarantee about a single value, and **availability** is a very strong claim that *every* non-failing node answers. And the useful extension — **PACELC** — points out that the interesting trade-off happens far more often when there is *no* partition at all, between **latency** and **consistency**.

They ask it because it is the most misquoted result in the field, so it is a fast way to separate people who have read a summary from people who have thought about it. The tell is what happens when they ask “*which one do you give up?*” — the good answer is “*neither, until there is a partition; and then it depends on the operation, and different operations in the same system can choose differently.*”

2 The story

The cloth shop had two branches, one on the main road in Erode and one in Perundurai eleven kilometres away, and between them they shared one stock book.

The book lived in Erode. When Perundurai sold something, the boy there rang and told them, and the entry went into the book. When somebody in Perundurai asked whether a particular silk sari was still available — there was often only one of each — the boy rang and asked.

It worked for years.

Then the exchange had a fault and the two shops could not reach each other. Not the phones being engaged — completely down, and nobody could say for how long. It happened three or four times a year and it sometimes lasted two hours.

The manager in Perundurai had exactly two options when a customer stood in front of him with a sari in her hand and cash in the other, and he had to pick one before he could say anything.

He could **sell it**. Take the money, write it on a slip, and tell Erode later. Fast, and the customer leaves happy. But if Erode had sold the same sari twenty minutes ago, he has just sold something that does not exist, and somebody will be telephoned in the evening and told their sari is not coming.

Or he could **refuse**. Say: I cannot confirm this is still here, please come back after five o'clock. Correct — he will never sell something twice — and the customer is standing there with money and is being turned away over a telephone fault.

He could not do both. There was no third option, and no amount of cleverness produced one, because the only thing that would have helped was the thing that was broken.

What he actually did, and what took him a few years to arrive at, was to stop treating it as one decision.

For ordinary cotton — where there were forty of them in the back and one extra sale made no difference — he sold. For the expensive silks, where there was one of each and a double sale meant a very unpleasant phone call, he refused and asked people to come back.

And there was a third category he was proud of. For things where he could not check the stock but could check the *price*, he took an advance and wrote the order, promising to confirm by evening. The customer went home having done something, and nothing was promised that might not be true.

He told his son once that the mistake people make is arguing about which of the two is right. Both are wrong. The question is which one is wrong **less**, and that depends entirely on what is being sold.

3 The idea in plain English

The two shops are a distributed system, the telephone fault is a network partition, and the manager's final position is the correct understanding of CAP.

- The two branches are **nodes**; the shared stock book is the **data**.
- The exchange fault is a **network partition** — the nodes are alive but cannot talk.
- **Selling anyway** is choosing **availability** (AP).

- **Refusing until the line is back** is choosing **consistency** (CP).
- “Cotton one way, silk the other” is the real answer: **the choice is per operation, not per system.**

The theorem, stated correctly

When a network partition occurs, a distributed system must choose between remaining consistent and remaining available. It cannot be both.

Note what that sentence does not say. It says nothing about normal operation. When the network is healthy, a system can be both consistent and available, and almost all of them are.

The three words, with their real meanings — **and none of them means what it means in ordinary engineering conversation:**

C — Consistency. Specifically **linearizability**: every read sees the most recent write, and the system behaves as if there were one copy of the data. **This is not the C of ACID**, which is about integrity constraints. Two different meanings, same letter, endless confusion.

A — Availability. Every request to a **non-failing** node returns a non-error response. This is an extremely strong definition — it means no node may say “I am not sure, ask later”. **It is not “99.99% uptime”**, and this is where almost all the misuse comes from.

P — Partition tolerance. The system continues to operate when messages between nodes are lost.

Why “pick two of three” is wrong

Because P is not optional. Networks partition: a cable is cut, a switch fails, a rack loses power, a misconfigured firewall drops packets. You do not get to choose whether it happens; you only choose what your system does when it does.

“CA” – consistent and available, no partition tolerance
→ means: a single machine, or a cluster that stops entirely when the network breaks
→ a SINGLE-NODE database is genuinely CA, and that is the only honest example
→ any distributed system claiming CA is claiming partitions never happen

So the real theorem is a two-way choice made during a partition, and the systems are labelled CP or AP by what they do then.

CP and AP, with real examples

CP — refuse rather than be wrong. During a partition, the minority side stops serving. Requests fail or hang.

ZooKeeper, etcd, Consul	a minority partition refuses all writes
HBase	regions become unavailable
MongoDB (default)	the minority side cannot elect a primary, so no writes
Spanner	stops rather than serve stale data

Use CP for anything where a **wrong answer is worse than no answer**: configuration, leader election, locks, account balances, inventory at the point of sale.

AP — answer anyway and reconcile later. During a partition, every side keeps serving from what it has.

Cassandra, DynamoDB, Riak	both sides accept reads and writes
DNS	serves cached records long after the source is unreachable
CDNs	serve stale content rather than nothing
shopping carts (Dynamo)	merge the divergent versions afterwards

Use AP for anything where a **slightly stale answer is much better than an error**: feeds, product catalogues, recommendations, view counts, session data.

The manager's point: the choice is per operation

This is the sentence that makes the answer sound experienced.

the same e-commerce system:	
browsing the catalogue	AP — a stale price for a few seconds is fine
adding to a cart	AP — merge divergent carts, Dynamo's own example
checking out	CP — you cannot sell the last item twice
the payment itself	CP — obviously
the order confirmation email	AP — send it late rather than not at all

A system is not CP or AP. Individual operations are. Cotton and silk.

And there is a third position, which is the manager's advance-order: **degrade rather than refuse**. Do something useful that promises less — accept the order and confirm later, show cached data and label it as such, queue the write for when connectivity returns.

PACELC: the part CAP leaves out

CAP describes what happens during a partition, which is rare. **PACELC** adds the far more common case:

If there is a Partition, choose Availability or Consistency. Else, choose Latency or Consistency.

The “else” half is the one you live with every day. Even with a perfectly healthy network, **making a read strongly consistent means asking more than one machine, and that costs latency.**

read from the nearest replica	~1 ms, possibly stale
read from the leader	~5 ms, current
read with a quorum across regions	~150 ms, current

system	partition	no partition
-----	-----	-----
DynamoDB	PA	EL (eventual by default, strong costs 2x and is slower)
Cassandra	PA	EL (tunable per query)
MongoDB	PC	EC (reads from the primary by default)
Spanner	PC	EC (and pays commit-wait for it)
PostgreSQL (single)	–	EC

PACELC is the more useful framework in practice, because the “else” branch applies always and the “if” branch applies for a few minutes a year. Naming it is one of the strongest things you can say about CAP.

What CAP does not say, and the misuses to avoid

It is not about uptime. “We are AP so we have high availability” is a category error. A CP system can have five nines of uptime; it just refuses service on the minority side of a partition, which is a rare event.

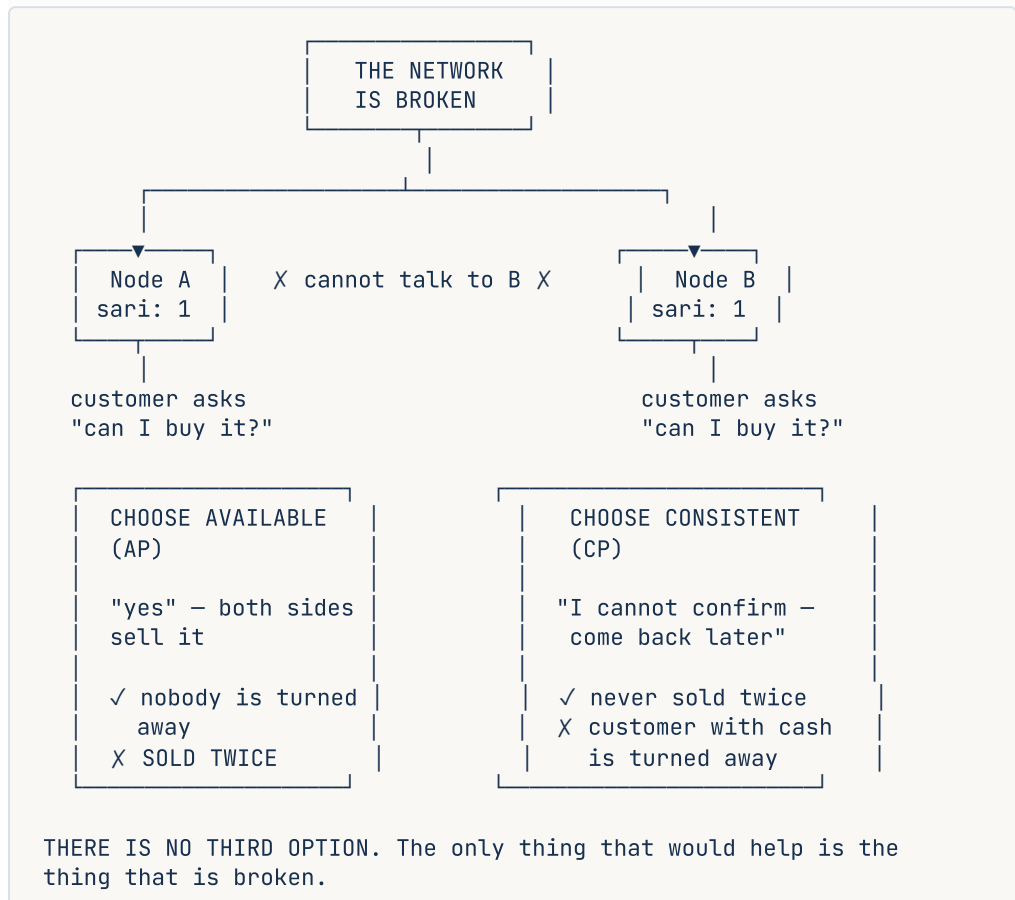
It is not a spectrum. The theorem is a hard impossibility about linearizability, not a dial. The *dial* exists — it is the [consistency models](#) of tomorrow, and the [quorum settings](#) of day 117 — but that is a different framework.

It says nothing about latency, throughput, or partial failures within a node. Most real outages are not partitions at all — they are overload, bad deploys and expired certificates.

And “eventually consistent” is not the same as AP. Eventual consistency is a guarantee about convergence over time; AP is a choice made during a partition. They usually travel together and they are different claims.

4 The picture

The partition, and the two choices.



Why “pick two of three” is the wrong picture:

THE POPULAR (WRONG) VERSION



"pick 2"

suggests CA is a real choice for a distributed system. It is not.

P IS NOT OPTIONAL — networks partition whether you like it or not.

THE HONEST VERSION

is there a partition?

NO

you can have BOTH C and A

(and PACELC says you still trade LATENCY vs CONSISTENCY here)

YES

choose ONE: C or A

The word "availability" means something unusual here:

CAP AVAILABILITY

every non-failing node returns a non-error response

a CP system in a partition:
the minority side returns errors
→ NOT "available" by CAP

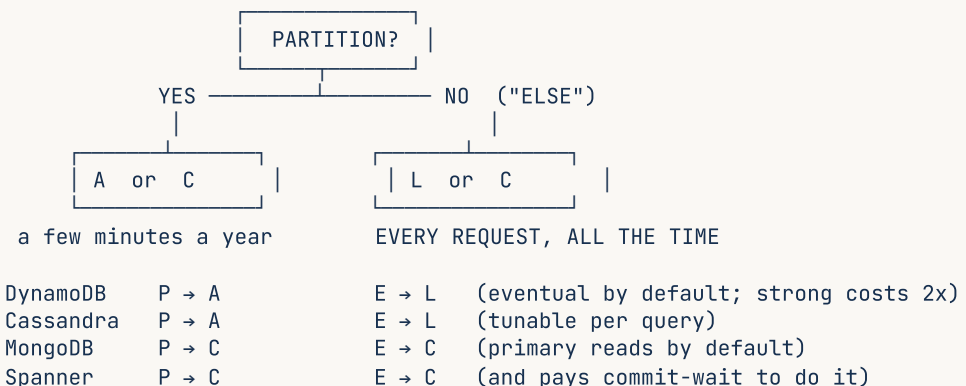
ORDINARY AVAILABILITY

"the system is up"
measured in nines

a CP system over a year:
partitions are rare
→ uptime can be 99.99%

SO: "we chose CP" does NOT mean "our system has poor uptime".
Confusing the two is the single most common misuse.

PACELC — the branch you actually live in:



the "else" branch applies ALWAYS. That is why PACELC is more useful.

The manager's real answer — per operation, not per system:

one e-commerce system

browse catalogue		AP	stale price for 2 s: fine
add to cart		AP	merge divergent carts
check stock		CP	cannot sell the last one twice
take payment		CP	obviously
send confirmation		AP	late is better than never

A SYSTEM IS NOT CP OR AP. OPERATIONS ARE.
Cotton one way, silk the other.

5 How it actually works

What a partition actually looks like

Not usually a cut cable. In practice:

an asymmetric partition	A can reach B, but B cannot reach A
a partial partition	A and B are fine; both lost the third node
a slow network	not partitioned, just 30 s round trips – which is INDISTINGUISHABLE from a partition from the outside (this is the day 113 point)
a GC pause	a node is frozen for 20 s and rejoins believing it is still the leader

That last one is why partitions are more common than people think: a long garbage-collection pause or a machine swapping is *observationally* a partition, and a system must survive it identically.

How a CP system behaves

Require a quorum. A write, and often a read, needs a majority of nodes to agree.

5 nodes, partitioned 3 2	
the side with 3 has a majority	→ keeps serving
the side with 2 does not	→ refuses

Why odd numbers. A 4-node cluster split 2|2 has no majority on either side and stops entirely — so four nodes tolerate exactly the same single failure that three do, at a higher cost.

Why a majority. Two majorities cannot exist simultaneously, so there can never be two sides both believing they may proceed. That single property is what prevents [split-brain](#).

How an AP system behaves

Accept writes on every side, and reconcile afterwards. Which means you need a conflict-resolution strategy, chosen deliberately:

LAST WRITE WINS	simplest, and it LOSES DATA – and with clock skew (day 113) it may keep the OLDER write
VERSION VECTORS	detect concurrency, then hand the conflict to the application – Dynamo's shopping cart
CRDTs	data types that merge automatically and correctly: counters, sets, registers. No conflicts by construction.
APPLICATION MERGE	e.g. union the two carts, which over-keeps rather than losing – Amazon's actual choice

Amazon's shopping-cart example is the canonical one and worth naming: they chose to union divergent carts, so an item you deleted might reappear. **Deliberately** — a resurrected item is a much smaller problem than a lost sale.

Tunable consistency: the dial that CAP does not describe

Cassandra and DynamoDB let you choose **per query**:

ONE	respond as soon as one replica answers	fastest, weakest
QUORUM	a majority must answer	$R + W > N$
ALL	every replica must answer	strongest, brittle

N = 3 replicas		
W=1, R=1	fast, may read stale	(1 + 1 = 2, not > 3)
W=2, R=2	strongly consistent	(2 + 2 = 4 > 3) ✓
W=3, R=1	fast reads, brittle writes	(3 + 1 = 4 > 3) ✓

$R + W > N$ is the condition that guarantees an overlap between the nodes written and the nodes read, so at least one node returns the newest value. That is [day 117](#), and it is the actual dial people mean when they say “we tuned our consistency”.

What real systems do

- **ZooKeeper, etcd, Consul** are CP by design, because they hold configuration and leadership, where a wrong answer is far worse than no answer. **Kubernetes stops scheduling if etcd loses quorum** — and that is correct behaviour, not a bug.
- **Cassandra and DynamoDB** are AP with tunable consistency, so the same cluster can serve a strongly consistent read and an eventually consistent one depending on the query.
- **MongoDB** is CP: the minority side of a partition cannot elect a primary, so it accepts no writes.

- **Spanner** is CP and famously “beats CAP” only in the sense that Google’s private network makes partitions extremely rare — it still chooses consistency when one occurs, and it pays a **commit-wait** of a few milliseconds on every transaction to do it.
- **DNS** is the everyday AP system: it serves cached records for the TTL regardless of whether the authoritative server is reachable, and nobody would want it any other way.

6 The numbers

How often does a partition actually happen?

a single-datacentre cluster	rare – a few events a year
cross-datacentre links	more common; several a year is normal
cross-region / over the internet	routine
"partition-like" events (GC pauses, overload, swap)	MUCH more common than real ones

Google’s Spanner papers report that most of their incidents were not network partitions at all, which is the honest framing: CAP describes the rare case, and PACELC’s “else” branch describes the normal one.

The cost of consistency when there is no partition

read from the local replica	~1 ms
read from the leader, same region	~5 ms
quorum read, same region	~5-10 ms
quorum read, cross-region	~150 ms+
Spanner's commit-wait	~5-10 ms on EVERY transaction

DynamoDB: a strongly consistent read costs 2x the capacity units of an eventually consistent one, and is slower.
→ the trade is priced, explicitly, per request

That last line is the clearest statement of PACELC anywhere: a vendor charging double for consistency is telling you exactly what it costs.

Quorum arithmetic

nodes	majority	tolerates	
-----	-----	-----	
3	2	1 failure	
4	3	1 failure	← no better than 3, and costs more
5	3	2 failures	
6	4	2 failures	← no better than 5
7	4	3 failures	

Even numbers buy nothing. And a 4-node cluster split 2|2 stops entirely, which is worse than a 3-node cluster split 2|1.

The cost of AP: reconciliation

- a 2-hour partition, 1,000 writes/second on each side
 - 7.2 million writes on each side to reconcile
 - with last-write-wins: an unknown number silently discarded
 - with version vectors: conflicts surfaced to the application

Last-write-wins is not free, and the amount of data it loses is proportional to the length of the partition and the write rate. **State that as a number**, because “we use last write wins” sounds harmless until you multiply it out.

Availability arithmetic, to separate CAP-A from uptime

- a CP system, partitions 4 times a year, 10 minutes each, affecting the minority third of nodes:
 - downtime for those clients 40 minutes/year → 99.992%
 - a CP system with FOUR NINES of uptime

"we chose CP" and "we have poor availability" are different claims.

7 The trade-offs

CP costs you availability exactly when the network is worst

The minority side stops serving. For a config store, that is correct — a wrong configuration is far worse than none. For a shopping catalogue it is absurd.

I would not choose CP for anything where a slightly stale answer is acceptable, because you are paying real refusals for a guarantee nobody needed.

AP costs you a reconciliation problem you must actually solve

Accepting writes on both sides means they will diverge, and **“we will sort it out later” is not a strategy** — you need a chosen mechanism, and every mechanism has a cost:

- **Last-write-wins** silently discards data, and with clock skew it may keep the *older* write.
- **Version vectors** are correct and push the decision to the application, which now has to have an answer.
- **CRDTs** merge automatically and correctly, and restrict you to data types that can do so.

I would not choose AP without naming the resolution strategy, because that is where the real design work is.

The choice is per operation, and saying so is the answer

Cotton and silk. **The strongest version of this answer is to give a table of operations in the system you are designing and mark each one**, rather than labelling the whole system.

Where CAP misleads

- **It describes a rare case.** Partitions are minutes a year; the latency-versus-consistency trade is every request. **PACELC is the more useful framework**, and saying so is a strong move.
- **The words are unusual.** CAP-availability is not uptime; CAP-consistency is not ACID's C. Most arguments about CAP are two people using the same word for different things.
- **“Eventually consistent” is not a synonym for AP.** One is a convergence guarantee, the other is a partition-time choice.
- **It ignores the failures that actually cause outages** — overload, bad deploys, expired certificates, configuration pushes. **None of those are partitions**, and a CAP-perfect system fails to all of them.

Where the theorem genuinely bites

- **Multi-region writes.** You cannot have synchronous cross-region replication and low-latency writes; physics forbids it. Choose consistency and pay 150 ms, or choose availability and reconcile.
- **Leader election.** By definition it needs agreement, so it is CP — which is why etcd stops rather than risk two leaders.

- **Inventory and money.** Where a double-spend is unacceptable, CP is not a preference.

8 In the interview

How it gets asked

- The direct one: *“Explain CAP. Which one does your design give up?”*
- The correction test: *“So you pick two of three?”*
- The applied one: *“Would you use Cassandra or PostgreSQL for this?”*
- The precision test: *“What does availability mean there — is that the same as uptime?”*
- The modern one: *“Have you heard of PACELC?”*

What to say out loud, in the first ninety seconds

1. **State it correctly, and narrowly.** “When a network partition occurs, you must choose between staying consistent and staying available. That is all it says — it is silent about normal operation.”
2. **Correct “two of three” before it is offered.** “The popular ‘pick two of three’ framing is misleading, because **partition tolerance is not a choice** — networks partition whether you like it or not. The only genuine CA system is a single node.”
3. **Define the words, because they are unusual.** “Consistency here means **linearizability** — every read sees the latest write — not ACID’s C. And availability means **every non-failing node returns a non-error response**, which is much stronger than ‘uptime’. A CP system can have four nines and still be ‘unavailable’ by CAP’s definition during a partition.”
4. **Make the choice per operation.** “And a system is not CP or AP — **operations are**. In one e-commerce system, browsing is AP and checkout is CP, because a stale price for two seconds is fine and selling the last item twice is not.”
5. **Name the third option.** “There is often a middle answer that is better than either: **degrade**. Serve cached data and label it, or accept the order and confirm later — do something useful that promises less.”
6. **Bring in PACELC.** “And the more useful framework is PACELC: if partitioned, availability or consistency — **else, latency or consistency**. The else branch applies to every request, all the time, and the partition branch applies for a few minutes a year.”

The follow-ups

“So you pick two of three?” “No, and that framing is the main reason CAP is misunderstood. **Partition tolerance is not something you choose** — cables get cut, switches fail, and a long garbage-collection pause looks exactly like a partition from outside. So ‘CA’ is not an option for a distributed system; the only honest CA system is a single machine, which trivially never partitions. The real theorem is a **two-way choice, made only during a partition**: when the network breaks, do the isolated nodes keep answering with possibly-stale data, or do they refuse? When the network is healthy you can have both consistency and availability, and almost every system does. So the labels CP and AP describe what a system does in a rare situation, not a permanent property.”

“What does availability mean there — is it uptime?” “No, and this is where most of the misuse comes from. CAP’s availability is a very strong, formal claim: **every request to every non-failing node returns a non-error response**. No node is allowed to say ‘I am not sure, try later’. Ordinary availability is a statistic about uptime, measured in nines. They come apart immediately: a CP system refuses service on the minority side of a partition, so it is not CAP-available — but if partitions happen four times a year for ten minutes and affect a third of clients, that is forty minutes a year, which is over four nines of ordinary uptime. So ‘we chose CP’ and ‘we have poor availability’ are completely different claims, and conflating them is the single most common CAP error.”

“Cassandra or PostgreSQL for this?” “That depends on the operation rather than the system, which is the answer I would actually want to give. Cassandra is AP with tunable consistency: during a partition every side keeps accepting reads and writes, and you resolve conflicts afterwards — and per query you can ask for ONE, QUORUM or ALL, so you can get a strongly consistent read when you need one by satisfying $R + W > N$. PostgreSQL with replication is effectively CP: writes go to one primary, and if that primary is on the wrong side of a partition, writes stop. So: if the workload is a feed, a catalogue, event ingestion, anything where a two-second-stale answer is fine and refusing is expensive, Cassandra. If it is money, inventory at the point of sale, or anything with real invariants across rows, PostgreSQL — and I would want the transactions and joins anyway. And most real products need both, for different parts.”

“Have you heard of PACELC?” “Yes, and I think it is the more useful framework, because it covers the case you actually live in. PACELC says: **if there is a Partition, choose Availability or Consistency; Else, choose Latency or Consistency**. The ‘else’ half is the point — even with a perfectly healthy network, making a read strongly consistent means consulting more than one machine, and that costs time. Reading from

the nearest replica is about a millisecond and may be stale; a quorum read is five to ten; a cross-region quorum is a hundred and fifty. DynamoDB prices this explicitly: a strongly consistent read costs twice the capacity of an eventually consistent one and is slower. So DynamoDB is PA/EL, MongoDB is PC/EC, Spanner is PC/EC and pays a commit-wait of several milliseconds on every transaction to be so. CAP describes a few minutes a year; PACELC's else branch describes every request."

"If you choose AP, what happens to the conflicting writes?" "That is the real design work, and 'we will reconcile later' is not an answer — you have to name the mechanism. **Last-write-wins** is simplest and it silently discards data, and worse, with clock skew it may keep the *older* write, since two machines' clocks can differ by more than the interval between the writes. **Version vectors** detect that two writes were genuinely concurrent rather than ordered, and hand the conflict to the application to resolve — which means the application must have a rule. **CRDTs** are data types that merge automatically and correctly — counters, sets, registers — so conflicts cannot occur by construction, at the cost of restricting what you can store. And **application-level merge** is what Amazon actually did for the shopping cart: union the divergent versions, so a deleted item may reappear. That is a deliberate choice — a resurrected item annoys a customer, a lost sale costs money. I would also put a number on it: a two-hour partition at a thousand writes a second on each side is seven million writes to reconcile, so last-write-wins is discarding an unknown but large quantity."

"Is there anything better than choosing?" "Often, yes, and it is the answer the theorem does not offer you: **degrade**. The theorem forces a choice between a correct answer and any answer, but you can frequently do something useful that promises less. Serve cached data and label it as possibly stale. Accept the order and confirm by email rather than confirming instantly. Queue the write locally and apply it when connectivity returns. Show the catalogue but disable checkout. Each of those keeps the user making progress without asserting something that might be false — and in product terms it is nearly always better than either a wrong answer or a blank error page. So my answer to 'which do you give up' is usually: neither, until there is a partition; and then it depends on the operation, and for several of them the right answer is to give up **completeness** rather than either consistency or availability."

A model answer

Asked: *explain CAP — which one does your design give up?*

“Let me state it precisely first, because the usual phrasing gets it wrong in a way that matters.

CAP says: **when a network partition occurs, a distributed system must choose between remaining consistent and remaining available.** That is the whole theorem. It says nothing about normal operation — when the network is healthy you can have both, and nearly every system does.

The popular version, ‘pick two of three’, is misleading because **partition tolerance is not a choice**. Cables get cut, switches fail, and a twenty-second garbage-collection pause is indistinguishable from a partition from the outside. You do not choose whether partitions happen; you only choose what you do when one does. The only genuine ‘CA’ system is a single machine, which never partitions because there is nothing to partition.

The words also do not mean what they normally mean, and most CAP arguments are really vocabulary arguments. **Consistency** here is **linearizability** — every read sees the most recent write, as if there were one copy — not the C of ACID, which is about integrity constraints. And **availability** is a very strong formal claim: **every** request to **every** non-failing node returns a non-error response. That is not uptime. A CP system refuses on the minority side of a partition, so it is not CAP-available — and if partitions cost forty minutes a year, it still has better than four nines of ordinary uptime.

So, to your question: **my design gives up neither, until there is a partition.** And then the honest answer is that it depends on the operation, because **a system is not CP or AP — operations are.** In an e-commerce system I would make browsing the catalogue AP, because a price that is two seconds stale is fine and refusing to show the shop is absurd. I would make the cart AP and merge divergent versions — which is what Amazon actually did, deliberately over-keeping items rather than losing a sale. And I would make the stock check and the payment CP, because selling the last item twice is a real cost that a refusal is not.

If I choose AP anywhere, I would name the reconciliation mechanism rather than saying ‘we will sort it out’: last-write-wins silently loses data and, with clock skew, can keep the *older* write; version vectors detect true concurrency and hand it to the application; CRDTs merge correctly by construction.

Two things I would add. There is often a **third option** better than either — degrade: serve cached data and label it, take the order and confirm later, disable checkout but keep the catalogue up. And the more useful framework is **PACELC**: if partitioned, availability or consistency; **else, latency or consistency**. That

‘else’ branch applies to every single request, all the time — DynamoDB charges twice as much for a strongly consistent read as an eventual one — whereas CAP’s branch applies for a few minutes a year.”

9 Recall card

- **The correct statement: WHEN A PARTITION OCCURS, choose between consistency and availability.** It says nothing about normal operation. “**Pick two of three**” is wrong because **P is not a choice** — cables cut, and a 20-second GC pause is indistinguishable from a partition. **The only genuine CA system is a single node.**
- **The words are unusual.** C = **linearizability** (every read sees the latest write) — not ACID’s C. A = **every non-failing node returns a non-error response** — not **uptime**. A CP system can refuse during partitions and still have **four nines** of ordinary availability. Conflating those is the most common misuse.
- **A system is not CP or AP — OPERATIONS are.** In one shop: browse AP, cart AP (merge), stock check CP, payment CP. **CP: etcd, ZooKeeper, MongoDB, Spanner, HBase. AP: Cassandra, DynamoDB, Riak, DNS, CDNs.** And there is a **third option the theorem does not offer: degrade** — serve labelled stale data, accept and confirm later, keep the catalogue and disable checkout.
- **If AP, name the reconciliation mechanism.** Last-write-wins **silently loses data** and with clock skew may keep the **older** write; version vectors detect true concurrency; CRDTs merge by construction; Amazon’s cart **unions** divergent versions on purpose. A 2-hour partition at 1,000 writes/s per side is **7 million writes** to reconcile.
- **PACELC is the more useful framework: if Partitioned, A or C; ELSE, Latency or Consistency.** The else branch applies to **every request** — local replica ~1 ms and stale, quorum ~5–10 ms, cross-region quorum ~150 ms — and **DynamoDB charges 2× for a strongly consistent read**, which is the trade, priced. Quorums need **odd** cluster sizes: 4 nodes tolerate no more failures than 3, and split 2|2 they stop entirely.

DSA topic: Push, pop, and heapify **System design topic:** The CAP theorem, honestly

Code these, in this order

One rule for the whole set: **after every operation, assert the heap property.** One line — `all(data[(i-1)//2] ≤ data[i] for i in range(1, len(data)))` — and it catches every structural bug in this lesson, all of which are otherwise silent.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Build your own <code>MinHeap</code>	—	<code>sift_up</code> , <code>sift_down</code> , <code>push</code> , <code>pop</code> , and the invariant check.
2	Sort an Array	LeetCode 912 (Medium)	Heapsort in place, and why it loses to quicksort anyway.
3	Kth Largest Element in an Array	LeetCode 215 (Medium)	<code>heapify</code> + k pops against a full sort, argued with numbers.
4	Find K Pairs with Smallest Sums	LeetCode 373 (Medium)	<code>heapreplace</code> on a fixed-size heap, rather than push-then-pop.

On problem 1, instrument it

Count swaps. Build a heap of 100,000 random values two ways — bottom-up `heapify` and `n` pushes — and record swaps per node for both, at `n = 1,000, 10,000` and `100,000`.

On problem 2, then compare against `sorted()`

Time your heapsort against Python's built-in on a million integers. Record the ratio and say what explains it, in one sentence about memory access.

On problem 3, do it three ways

Full sort; `heapify` plus k pops; a fixed-size heap of k. Time all three at `k = 10` and `k = 500,000` on a million elements. Say which wins where and why.

On problem 4, use `heapreplace`

Write it once with `heappush` followed by `heappop`, and once with `heapreplace`. Say what the difference is in passes over the heap.

The two-primitives drill

1. Write `sift_up` and `sift_down` from memory.
2. Say how many comparisons per level each one does, and why they differ.
3. Say which one insert uses and which one extract uses, and why not the reverse.
4. Say where each one stops.

The completeness drill

1. Say why insert appends rather than inserting anywhere else.
2. Say why extract moves the **last** element to the root.
3. Draw what happens if you delete index 0 directly.
4. Draw what happens if you promote the smaller child instead.

The smaller-child drill

1. Write the sift-down comparison correctly.
2. Change it to swap with the left child unconditionally and run a hundred random operations.
3. Say what the invariant check reports, and whether anything raised.
4. Say what a user of the heap would observe.

The $O(n)$ drill

1. State the two ways to build a heap and their complexities.
2. Say where `heapify` starts and in which direction, and justify both.
3. Give the depth-by-depth work table for a 15-node heap.
4. Write the sum and say what $\sum k/2^k$ converges to.
5. Explain the asymmetry — why sift-**up** top-down is genuinely $O(n \log n)$.
6. Report your measured swaps per node for both methods at three sizes.

The heapsort drill

1. Write heapsort in place.
2. Say why it builds a **max**-heap for an ascending sort.
3. State its three properties and say which common sort shares all three.
4. Say why nobody uses it, in one sentence about cache.
5. Name where it is used, and why.

The shortcuts drill

1. Say what `heappushpop` and `heapreplace` each do.

2. Say which one a fixed-size top-k heap wants, and why.
3. Say what `heappushpop` does when the new value is already the smallest.
4. Count the passes over the heap for each against the naive pair.

The mutation drill

1. Say what to do when a value in the heap decreases, and when it increases.
2. State the complexity, and the caveat.
3. Describe an indexed heap and what it costs.
4. Describe lazy deletion and say which language forces you into it.

The break-it drill

Trigger each and record the output or error:

1. `heapify` running forwards instead of backwards.
 2. `heapify` starting from the last element rather than the last parent — count the wasted calls.
 3. Sifting down without the `left < n` bounds check.
 4. `data.pop(0)` used to extract the minimum.
 5. Popping the only element without the `if data:` guard.
 6. Building a million-item heap by pushing, timed against `heapify`.
 7. Mutating `h[3]` directly and then popping.
-

The statement drill

1. State CAP correctly, in one sentence.
2. Say what it does **not** say.
3. Say why “pick two of three” is wrong.
4. Name the only genuine CA system.

The vocabulary drill

1. Define CAP's C precisely, and say what it is not.
2. Define CAP's A precisely, and say what it is not.
3. Compute the ordinary uptime of a CP system with four ten-minute partitions a year affecting a third of clients.
4. Say why that number matters for how you talk about CP.

The per-operation drill

1. Say the sentence about systems versus operations.
2. Mark each of these AP or CP with a reason: catalogue browsing, cart, stock check, payment, confirmation email, recommendations, session data, leader election.
3. Name the third option that is neither, and give three concrete forms of it.

The systems drill

1. Name four CP systems and four AP systems.
2. For each of Cassandra, MongoDB, DynamoDB and Spanner, give its PACELC classification.
3. Say what Kubernetes does when etcd loses quorum, and whether that is a bug.
4. Say why DNS is the everyday AP system.

The PACELC drill

1. State PACELC in one sentence.
2. Say which branch applies more often, and why that makes it more useful.
3. Give latency figures for a local read, a leader read, a same-region quorum and a cross-region quorum.
4. Say how DynamoDB prices the trade, and what that tells you.

The reconciliation drill

1. Name four conflict-resolution strategies.
2. For each, say what it costs.
3. Say what last-write-wins does when clocks are skewed.
4. Compute the writes to reconcile after a two-hour partition at 1,000 writes/s per side.
5. Describe Amazon's shopping-cart choice and say why it is deliberate.

The quorum drill

1. Fill in the nodes-to-tolerated-failures table for 3 through 7.
2. Say why even numbers buy nothing.
3. Say what a 4-node cluster does when split 2|2.
4. State the $R + W > N$ condition and say what it guarantees.

The failure drill

For each, say what happens and what you would choose:

1. A 20-second GC pause on one node.
2. An asymmetric partition where A can reach B but not the reverse.
3. A cross-region link fails during a sale.
4. A CP config store loses quorum.
5. Two sides of an AP system both accept a write to the same key.
6. A system claims exactly-once and strong consistency across regions with low latency.

The last one is a claim you should be suspicious of. Say why.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Insert into the heap. Now remove the minimum.* What both operations share, insert appending with the completeness reason, extract moving the last element with the hole argument, swap-with-the-smaller-child said explicitly, and both complexities.
 2. *Explain CAP. Which one does your design give up?* The correct narrow statement, “pick two of three” corrected with P not being optional, both words defined against what they are not, the per-operation answer with a concrete table, degradation as the third option, and PACELC as the more useful frame.
 3. *Why is building a heap $O(n)$?* Half the nodes are leaves, the depth-by-depth table, the series converging to 2, and the asymmetry that makes the top-down version genuinely $O(n \log n)$.
-

Before you move on

- [] I can write both sift operations from memory.
- [] I know how many comparisons each does per level, and why.
- [] I can say why insert appends and why extract moves the last element.
- [] I always swap with the smaller child, and know what happens if I do not.
- [] I know where `heapify` starts, which direction it runs, and why both.
- [] I can give the depth-by-depth work table and the series that converges to 2.
- [] I can explain the asymmetry between sift-up and sift-down builds.
- [] I measured swaps per node for both build methods.

- [] I can write heapsort and say why nobody uses it, and where it is used.
- [] I know what `heapreplace` saves and when to reach for it.
- [] I know what to do when a heap value changes, and why finding it is the hard part.
- [] I can state CAP in one sentence and say what it does not say.
- [] I can say why “pick two of three” is wrong.
- [] I can define CAP’s C and A against what they are not.
- [] I can compute a CP system’s ordinary uptime and explain why that matters.
- [] I can classify eight operations as AP or CP with reasons.
- [] I can name the third option and three concrete forms of it.
- [] I can state PACELC and say which branch applies more often.
- [] I can name four reconciliation strategies and what each costs.
- [] I answered all three questions above out loud.

Python's heapq, and the min-heap-only problem

DATA STRUCTURES AND ALGORITHMS

Python's heapq, and the min-heap-only problem

You can simulate a max-heap and store tuples with tie-breakers safely.

SYSTEM DESIGN

Consistency models

You can order strong, causal and eventual consistency and give an example of each.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Python only has a min-heap. How do you get the maximum?*
- *What consistency does this feature need? Justify it.*

Python's heapq, and the min-heap-only problem

1 What this is, and why they ask it

`heapq` is Python's heap, and it has three properties that shape every solution you will write with it: it is **min-only**, it operates on a **plain list** rather than an object, and it compares items with `<`.

Three sentences. Getting a max-heap means **inverting the ordering**, and the standard trick — negate the values — works for numbers and fails for everything else, so you need the second technique too. Storing anything richer than a number means pushing **tuples**, and the moment two tuples have equal first elements, Python compares the *second* element — which is why a tie-breaker is not optional but a correctness requirement. And because a heap cannot find or update an element, changing a priority means **lazy deletion**: push a new entry and skip the stale one when it surfaces.

They ask “*Python only has a min-heap, how do you get the maximum?*” because it takes five seconds to answer and the follow-up separates people: “*and what if the items are objects rather than numbers?*” The negation trick is common knowledge; knowing that it does not generalise, and knowing what to do instead, is not.

2 The story

The token machine at the sub-registrar's office was installed in 2011 and it did exactly one thing: it called the **lowest** number that had not yet been served.

For ordinary work that was correct and nobody thought about it. You took a token, and when your number came up you went in.

The problem arrived when the department decided that senior citizens and people with a disability should be seen ahead of everybody else. The machine had no setting for it. The man who had installed it had left the company, and the department that would have to approve a replacement met twice a year.

The clerk, Anjaneyulu, worked out the fix in about a day and a half, and it was not clever, it was sideways.

The machine called the lowest number. So he issued priority tokens with **low** numbers and ordinary tokens with **high** ones. Priority tokens started at one; ordinary tokens started at ten thousand. Nothing about the machine changed. It went on calling the lowest number available, and because of how the numbers were handed out, that was always the person who ought to go first.

He said afterwards that he had spent a morning trying to make the machine call the highest number, and about ten minutes on realising that he did not have to.

Two things went wrong with it, and both took him longer to work out than the original idea.

The first was that within the priority group he wanted the person who had arrived first to go first. So he started printing two things on the token — the priority band, and a running count of how many tokens had been issued that morning. The machine compared the band first, and when two bands were equal it compared the count, and the count was never equal to anything.

The second happened on a Thursday when he had briefly tried printing the person's **name** as the second thing instead of a count. Two people in the priority band, and the machine had to decide between them, and it compared the names — which worked, in a sense, until a man whose name was written in Telugu and a woman whose name was written in English came at the same time, and the machine stopped and would not continue until somebody cleared it.

He went back to the count. He said the second thing on the token has one job, which is to never be equal and always be comparable, and a name is neither.

3 The idea in plain English

Anjaneyulu has solved both of the practical problems with `heapq`, and his diagnosis of the second one is exactly right.

- The machine calling the lowest number is `heapq` being **min-only**.
- Issuing priority tokens with low numbers is **inverting the ordering** rather than changing the machine.
- The running count is a **tie-breaker**, and it exists so that the comparison never reaches the payload.
- The names jamming the machine is `TypeError: '<' not supported between instances of ...`.

The API, in full

```
import heapq

heapq.heapify(lst)           # O(n), in place – NOT n pushes
heapq.heappush(h, item)     # O(log n)
heapq.heappop(h)            # O(log n), returns the smallest
h[0]                         # peek, O(1) – there is no heappeek
heapq.heappushpop(h, item)  # push then pop, ONE pass
heapq.heapreplace(h, item)  # pop then push, ONE sift-down
heapq.nsmallest(k, iterable) # O(n log k)
heapq.nlargest(k, iterable)  # O(n log k)
heapq.merge(*iterables)     # a lazy merge of sorted inputs
```

There is no heap object. The list *is* the heap, which means:

```
h.append(5)                 # BREAKS the invariant, silently
h.sort()                    # technically still a valid heap, by luck
h[3] = 0                    # BREAKS it, silently
len(h)                      # fine – a heap is a list
for x in h: ...             # fine, and NOT in sorted order
```

Nothing raises. The list stays a list and the heap property is quietly gone, so subsequent pops return the wrong element. **Never touch the list except through `heapq` functions.**

Making a max-heap: negate the values

```
for v in values:
    heapq.heappush(h, -v)
largest = -heapq.heappop(h)
```

Negate on the way in, negate on the way out. The min of the negatives is the max of the originals.

Three caveats worth stating:

- **It only works for numbers.** Strings, tuples of strings and objects cannot be negated.
- **-x for floats is fine**, including `-0.0`, but be careful with `float('nan')`, which compares false with everything and will corrupt the heap silently.
- **The integer edge case:** in a fixed-width language, negating the most negative integer overflows. Python's integers are arbitrary precision so this does not bite here — **but say it if the interview is in C++ or Java**, where `-INT_MIN` is undefined behaviour.

Making a max-heap: invert the comparison

For anything that is not a number, wrap it in a class whose `__lt__` is reversed:

```
from dataclasses import dataclass, field

@dataclass(order=True)
class MaxItem:
    key: int
    payload: object = field(compare=False)

    def __lt__(self, other):
        return self.key > other.key           # INVERTED
```

This is the general answer, and it is what to give when they ask “*and if the items are objects?*” It also works for a compound ordering — largest by score, then earliest by time — where negation cannot express the mixture.

A third option worth knowing: `heapq` has private `_heapify_max`, `_heappop_max` and `_siftup_max` functions. **Do not use them** — they are undocumented, incomplete (there is no `_heappush_max`), and they change between versions. Naming them and rejecting them is a better answer than using them.

Tuples: the standard way to carry a payload

```
heapq.heappush(h, (priority, tie_breaker, payload))
```

Tuples compare element by element, so this orders by `priority` first and then by `tie_breaker`.

And that is exactly the danger. If two priorities are equal, Python moves on to the next element — and if that is your payload, it must be comparable.

```
heapq.heappush(h, (1, {"name": "a"}))
heapq.heappush(h, (1, {"name": "b"}))
```

```
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

The tie-breaker is not for tidiness. It is what stops the comparison ever reaching the payload. Anjaneyulu’s count.

The standard tie-breaker is a monotonically increasing counter:

```
counter = itertools.count()
heapq.heappush(h, (priority, next(counter), payload))
```

Two properties, and it needs both: it is **always comparable** (integers), and it is **never equal** (strictly increasing). A timestamp fails the second on a fast machine; a name fails the first when the types differ.

It also gives you FIFO within a priority for free, which is usually what you want and is worth mentioning.

Changing a priority: lazy deletion

A heap cannot find an element, so it cannot update one — [day 113's](#) weakness. Python offers nothing for this, and the idiomatic answer is:

```
heapq.heappush(h, (new_priority, next(counter), item))    # push the new one
dead.add(old_entry_id)                                  # mark the old one

while h:
    priority, seq, item = heapq.heappop(h)
    if seq in dead:
        continue                                         # skip the stale entry
    return item
```

Push the new entry, leave the old one, and skip it when it surfaces.

cost: the heap grows with stale entries — up to $O(\text{total pushes})$ rather than $O(\text{live items})$
benefit: no index map to maintain, and no code at all in the hot path

This is what a Python implementation of Dijkstra's algorithm actually does, and saying so — rather than describing an indexed heap you would not write — is the practical answer.

`nlargest` and `nsmallest`

```
heapq.nlargest(3, data)                                #  $O(n \log k)$ 
heapq.nlargest(3, data, key=lambda x: x.score)
```

They keep a heap of size `k`, so the memory is $O(k)$ rather than $O(n)$ and there is no sorting of the whole input. And the `key` argument means **no tuples and no tie-breaker needed** — the comparison never touches your objects.

But know where they stop being the right tool:

<code>k = 1</code>	use <code>max()</code> / <code>min()</code> — $O(n)$, no heap at all
<code>k</code> small (<code>< n/10</code>)	<code>nlargest</code> / <code>nsmallest</code> — $O(n \log k)$
<code>k</code> close to <code>n</code>	<code>sorted()</code> — $O(n \log n)$ and much better constants

CPython's implementation actually does this switching internally for the degenerate cases, which is a nice detail to know but not to rely on.

merge

```
heapq.merge(list_a, list_b, list_c)           # a lazy iterator
heapq.merge(*lists, key=..., reverse=...)
```

It is **lazy**, so it merges streams larger than memory — which is exactly the external-sort use case, and it is [day 117](#) done for you.

The comparison rules, in one place

```
heapq compares with < only. It never uses ==, >, or __gt__.
→ you only need __lt__ to make a class heap-compatible
→ @dataclass(order=True) generates it
→ functools.total_ordering also works and is more than needed

tuples compare LEFT TO RIGHT, moving on only when elements are EQUAL
→ so the payload is compared ONLY when everything before it ties
→ which is why the tie-breaker must never be equal
```

4 The picture

The min-only problem, and the two ways round it.

THE PROBLEM

```
values: 3 9 1 7
heap:    [1, 7, 3, 9]
pop → 1    ...but you wanted 9
```

heapq always gives you the SMALLEST

FIX 1 – NEGATE (numbers only)

```
push -3, -9, -1, -7
heap: [-9, -7, -3, -1]
pop → -9
negate again → 9 ✓

✓ trivial for ints and floats
X strings, tuples, objects
```

FIX 2 – INVERT __lt__ (anything)

```
class MaxItem:
    def __lt__(self, other):
        return self.key > other.key

heap holds MaxItem objects
✓ works for ANY type
✓ works for COMPOUND orderings
  (largest score, then earliest time)
```

Anjaneyulu's token machine, as the same picture:

```
the machine calls the LOWEST number. It cannot be changed.

priority tokens:  1, 2, 3, ...          ← issued LOW
ordinary tokens:  10001, 10002, ...     ← issued HIGH

the machine's behaviour is unchanged.
The ORDERING was changed, not the machine.
```

```
the machine calls the LOWEST number. It cannot be changed.

priority tokens:  1, 2, 3, ...          ← issued LOW
ordinary tokens:  10001, 10002, ...     ← issued HIGH

the machine's behaviour is unchanged.
The ORDERING was changed, not the machine.
```

```
the machine calls the LOWEST number. It cannot be changed.

priority tokens:  1, 2, 3, ...          ← issued LOW
ordinary tokens:  10001, 10002, ...     ← issued HIGH

the machine's behaviour is unchanged.
The ORDERING was changed, not the machine.
```

The tie-breaker, and what it prevents:

```
tuples compare LEFT TO RIGHT and stop at the first difference

(1, 5, obj_a)  vs  (1, 9, obj_b)
 ▲      ▲      ▲      ▲
|      |      |      |
├── 5 < 9 → DECIDED here. obj_a and obj_b are never touched.
├── equal, so move on
```

```
tuples compare LEFT TO RIGHT and stop at the first difference

(1, 5, obj_a)  vs  (1, 9, obj_b)
 ▲      ▲      ▲      ▲
|      |      |      |
├── 5 < 9 → DECIDED here. obj_a and obj_b are never touched.
├── equal, so move on
```

WITHOUT a tie-breaker:

```
(1, obj_a) vs (1, obj_b)
```

Python compares `obj_a < obj_b`
equal, so move on

`TypeError: '<' not supported between instances of 'dict' and 'dict'`

```
(1, obj_a) vs (1, obj_b)
```

Python compares `obj_a < obj_b`
equal, so move on

`TypeError: '<' not supported between instances of 'dict' and 'dict'`

```
THE TIE-BREAKER'S ONLY JOB:
  1. always comparable    (an int - a NAME fails this across scripts)
  2. never equal          (a strictly increasing counter - a TIMESTAMP
                           fails this on a fast machine)
```

- ```
THE TIE-BREAKER'S ONLY JOB:
 1. always comparable (an int - a NAME fails this across scripts)
 2. never equal (a strictly increasing counter - a TIMESTAMP
 fails this on a fast machine)
```

Lazy deletion, which is how a priority changes:

```

the heap the "dead" set

(5, 12, taskA) ← stale {12}
(3, 47, taskA) ← current
(8, 13, taskB)

pop → (3, 47, taskA) seq 47 not dead → USE IT
pop → (5, 12, taskA) seq 12 IS dead → SKIP, pop again
pop → (8, 13, taskB) fine

cost: the heap holds every version ever pushed, not just the live ones
benefit: no index map, no bookkeeping on every swap
→ and this is exactly what a Python Dijkstra does

```

```

the heap the "dead" set

(5, 12, taskA) ← stale {12}
(3, 47, taskA) ← current
(8, 13, taskB)

pop → (3, 47, taskA) seq 47 not dead → USE IT
pop → (5, 12, taskA) seq 12 IS dead → SKIP, pop again
pop → (8, 13, taskB) fine

cost: the heap holds every version ever pushed, not just the live ones
benefit: no index map, no bookkeeping on every swap
→ and this is exactly what a Python Dijkstra does

```

```

the heap the "dead" set

(5, 12, taskA) ← stale {12}
(3, 47, taskA) ← current
(8, 13, taskB)

pop → (3, 47, taskA) seq 47 not dead → USE IT
pop → (5, 12, taskA) seq 12 IS dead → SKIP, pop again
pop → (8, 13, taskB) fine

cost: the heap holds every version ever pushed, not just the live ones
benefit: no index map, no bookkeeping on every swap
→ and this is exactly what a Python Dijkstra does

```

```

the heap the "dead" set

(5, 12, taskA) ← stale {12}
(3, 47, taskA) ← current
(8, 13, taskB)

pop → (3, 47, taskA) seq 47 not dead → USE IT
pop → (5, 12, taskA) seq 12 IS dead → SKIP, pop again
pop → (8, 13, taskB) fine

cost: the heap holds every version ever pushed, not just the live ones
benefit: no index map, no bookkeeping on every swap
→ and this is exactly what a Python Dijkstra does

```

The `heapq` decision table:

| what you want                      | what to use                                            |
|------------------------------------|--------------------------------------------------------|
| the single largest                 | <code>max(data)</code> — $O(n)$ , no heap              |
| the $k$ largest, $k$ small         | <code>heapq.nlargest(k, data)</code> — $O(n \log k)$   |
| the $k$ largest, $k$ close to $n$  | <code>sorted(data)[-k:]</code> — better constants      |
| a running "next smallest"          | <code>heappush / heappop</code>                        |
| a running "next largest"           | negate, or invert <code>__lt__</code>                  |
| keep only the best $k$ so far      | a size- $k$ heap + <code>heapreplace</code>            |
| merge sorted streams               | <code>heapq.merge()</code> — LAZY                      |
| change a priority                  | lazy deletion (push new, skip stale)                   |
| build from a list you already have | <code>heapq.heapify()</code> — $O(n)$ , NOT $n$ pushes |

## 5 The code, built step by step

### Step 1 — answer the min-only question two ways

“For numbers, negate on the way in and on the way out — the minimum of the negatives is the maximum of the originals. For anything else, wrap the item in a class with an inverted `__lt__`, because you cannot negate a string or an object.”

**Give both. The second is what the follow-up is for.**

### Step 2 — push tuples, and say why the middle element exists

```
heapq.heappush(h, (priority, next(counter), payload))
```

“Tuples compare left to right, so if two priorities tie, Python compares the next element. If that is my payload, it has to be orderable — and a dictionary is not, so it raises. The counter guarantees the comparison never gets that far, and it gives me FIFO within a priority for free.”

### Step 3 — `heapify`, never $n$ pushes

```
heapq.heapify(items) # $O(n)$
```

“If I already have all the items, `heapify` is  $O(n)$  and pushing them one at a time is  $O(n \log n)$  — about four times the work at a million items, for the same result.”

### Step 4 — `heapreplace` for a fixed-size heap

```
if value > h[0]:
 heapq.heapreplace(h, value)
```



“For ‘keep the best k’, I compare against the smallest first and only then replace — one sift-down instead of a pop followed by a push.”

**Step 5 — say what you must never do to the list**

“There is no heap object in Python; the list is the heap. Appending to it, sorting it, or assigning to an element breaks the invariant silently — no exception, just wrong results from later pops.”

## The complete solution

```
import heapq
import itertools
from dataclasses import dataclass, field

1. The min-only problem: two answers

def k_largest_by_negation(values: list[int], k: int) → list[int]:
 """FIX 1: negate in, negate out. The min of the negatives is the max.

 Works for ints and floats. Does NOT work for strings, tuples of
 strings, or objects – which is why fix 2 exists.

 In C++/Java, note that -INT_MIN overflows. Python's ints are arbitrary
 precision, so it does not bite here, but say it if the language differs.
 """
 heap = [-v for v in values]
 heapq.heapify(heap) # O(n)
 return [-heapq.heappop(heap) for _ in range(k)]

@dataclass(order=True)
class MaxItem:
 """FIX 2: invert the comparison. Works for ANY type, and for compound
 orderings that negation cannot express.

 heapq only ever uses `<`, so __lt__ is the only method that must exist.
 `compare=False` keeps the payload out of the comparison entirely – which
 is the tie-breaker problem solved at the type level.
 """

 key: int
 payload: object = field(compare=False, default=None)

 def __lt__(self, other: "MaxItem") → bool:
 return self.key > other.key # INVERTED: bigger keys sort first

def k_largest_by_inversion(items: list[tuple[int, object]], k: int) → list[object]:
 heap = [MaxItem(key, payload) for key, payload in items]
 heapq.heapify(heap)
 return [heapq.heappop(heap).payload for _ in range(k)]

2. Tuples and the tie-breaker

class PriorityQueue:
 """A priority queue that will not raise on a tie.

 THE TIE-BREAKER has exactly two requirements and needs BOTH:
 1. always comparable – an int. A NAME fails this across scripts.
 2. never equal – strictly increasing. A TIMESTAMP fails this
 on a fast machine.

 Tuples compare left to right, so with a unique counter in position 2 the
 payload in position 3 is NEVER compared – which is the point.
```

```

Bonus: the counter gives FIFO within a priority for free.
"""

def __init__(self) → None:
 self._heap: list[tuple[int, int, object]] = []
 self._counter = itertools.count()

def push(self, priority: int, item: object) → None:
 heapq.heappush(self._heap, (priority, next(self._counter), item))

def pop(self) → object:
 return heapq.heappop(self._heap)[2]

def peek(self) → object:
 return self._heap[0][2] # 0(1)

def __len__(self) → int:
 return len(self._heap)

def demonstrate_tie_breaker_bug() → None:
 """Run this. It raises, and the error is worth seeing once."""
 h: list[tuple[int, dict]] = []
 heapq.heappush(h, (1, {"name": "a"}))
 try:
 heapq.heappush(h, (1, {"name": "b"}))
 except TypeError as exc:
 print(f"TypeError: {exc}")
 # '<' not supported between instances of 'dict' and 'dict'

3. Changing a priority: lazy deletion

class UpdatablePriorityQueue:
 """A heap cannot find or update an element, so: push the new version,
 mark the old one dead, and skip it when it surfaces.

 This is what a Python implementation of Dijkstra ACTUALLY does – an
 indexed heap (a value → index map maintained on every swap) is the
 textbook answer and nobody writes it in Python.

 Cost: the heap grows with every version ever pushed, not just the live
 entries. For Dijkstra that is O(E) rather than O(V), which is fine.
 """

 def __init__(self) → None:
 self._heap: list[tuple[int, int, object]] = []
 self._counter = itertools.count()
 self._entry: dict[object, int] = {} # item → its live sequence number

 def push(self, priority: int, item: object) → None:
 seq = next(self._counter)
 self._entry[item] = seq # any earlier seq is now stale
 heapq.heappush(self._heap, (priority, seq, item))

 def pop(self) → tuple[int, object] | None:
 while self._heap:
 priority, seq, item = heapq.heappop(self._heap)
 if self._entry.get(item) == seq: # is this the live version?
 del self._entry[item]

```

```

 return priority, item
 # otherwise it is stale – skip it
 return None

def __len__(self) → int:
 return len(self._entry) # LIVE entries, not heap size

def dijkstra(graph: dict[str, list[tuple[str, int]]], source: str) → dict[str, int]:
 """The canonical use of lazy deletion, written out.

 No decrease-key: push a better distance whenever we find one, and skip
 an entry whose distance is worse than the one already settled.
 """
 distance = {source: 0}
 heap = [(0, source)]
 settled: set[str] = set()

 while heap:
 d, node = heapq.heappop(heap)
 if node in settled:
 continue # a stale entry – skip it
 settled.add(node)
 for neighbour, weight in graph.get(node, []):
 nd = d + weight
 if nd < distance.get(neighbour, float("inf")):
 distance[neighbour] = nd
 heapq.heappush(heap, (nd, neighbour)) # push, do not update
 return distance

4. Fixed-size heaps, and the shortcuts

def k_largest_fixed_heap(values: list[int], k: int) → list[int]:
 """Keep a MIN-heap of size k. The smallest of the k largest is at the
 root, so a new value only matters if it beats it.

 O(n log k) time, O(k) space – better than sorting when k << n.
 heapreplace is ONE sift-down; heappop followed by heappush is two passes.
 """
 heap = values[:k]
 heapq.heapify(heap)
 for v in values[k:]:
 if v > heap[0]:
 heapq.heapreplace(heap, v) # NOT heappop + heappush
 return sorted(heap, reverse=True)

def top_k_by_key(items: list[dict], k: int, key: str) → list[dict]:
 """nlargest with a `key` avoids tuples and tie-breakers entirely –
 the comparison never touches your objects."""
 return heapq.nlargest(k, items, key=lambda d: d[key])

def running_median_hint() → None:
 """Two heaps facing each other: a MAX-heap of the lower half (negated)
 and a MIN-heap of the upper half. Day 118."""

5. The traps, written out so you can run them

```

```

def demonstrate_list_mutation() → None:
 """There is no heap object. Touching the list breaks it silently."""
 h = [1, 3, 2, 7, 4]
 heapq.heapify(h)
 print("valid:", all(h[(i - 1) // 2] ≤ h[i] for i in range(1, len(h))))
 h.append(0) # NOT heappush
 print("after append, valid:", all(h[(i - 1) // 2] ≤ h[i]
 for i in range(1, len(h))))
 print("pop returns:", heapq.heappop(h), "- not 0, which was the smallest")

def demonstrate_build_cost() → None:
 """heapify is O(n); n pushes is O(n log n). Same result."""
 import time
 data = list(range(200_000, 0, -1))

 t = time.perf_counter()
 a = data[:]
 heapq.heapify(a)
 fast = time.perf_counter() - t

 t = time.perf_counter()
 b: list[int] = []
 for v in data:
 heapq.heappush(b, v)
 slow = time.perf_counter() - t

 print(f"heapify {fast:.4f}s vs {len(data)} pushes {slow:.4f}s "
 f"({slow / fast:.1f}x)")

if __name__ == "__main__":
 values = [5, 1, 9, 3, 7, 2, 8]

 print(k_largest_by_negation(values, 3)) # [9, 8, 7]
 print(k_largest_by_inversion(
 [(5, "e"), (1, "a"), (9, "i"), (3, "c")], 2)) # ['i', 'e']
 print(k_largest_fixed_heap(values, 3)) # [9, 8, 7]

 # the tie-breaker
 demonstrate_tie_breaker_bug()
 # TypeError: '<' not supported between instances of 'dict' and 'dict'

 pq = PriorityQueue()
 pq.push(2, {"task": "email"})
 pq.push(1, {"task": "page"})
 pq.push(1, {"task": "alert"}) # SAME priority — no raise
 print(pq.pop(), pq.pop(), pq.pop())
 # {'task': 'page'} {'task': 'alert'} {'task': 'email'} FIFO within priority

 # lazy deletion
 upq = UpdatablePriorityQueue()
 upq.push(5, "taskA")
 upq.push(8, "taskB")
 upq.push(3, "taskA") # taskA's priority IMPROVED
 print(upq.pop(), upq.pop(), upq.pop())
 # (3, 'taskA') (8, 'taskB') None the stale (5, 'taskA') was skipped

 graph = {
 "a": [("b", 1), ("c", 4)],
 "b": [("c", 2), ("d", 5)],
 }

```

```

 "c": [("d", 1)],
 "d": [],
}
print(dijkstra(graph, "a")) # {'a': 0, 'b': 1, 'c': 3, 'd': 4}

nlargest with a key – no tuples, no tie-breaker
people = [{"n": "a", "age": 30}, {"n": "b", "age": 25}, {"n": "c", "age": 40}]
print(top_k_by_key(people, 2, "age")) # the two oldest

merge is LAZY
merged = heapq.merge([1, 4, 7], [2, 5, 8], [3, 6, 9])
print(list(merged)) # [1..9]

the traps
demonstrate_list_mutation()
demonstrate_build_cost()

peek is O(1) and there is no heappeek
h = [5, 1, 9]
heapq.heapify(h)
print(h[0]) # 1

```

## 6 What it costs

The `heapq` functions

|                           |               |                             |
|---------------------------|---------------|-----------------------------|
| <code>heapify</code>      | $O(n)$        |                             |
| <code>heappush</code>     | $O(\log n)$   |                             |
| <code>heappop</code>      | $O(\log n)$   |                             |
| <code>h[0]</code>         | $O(1)$        |                             |
| <code>heappushpop</code>  | $O(\log n)$   | one pass, not two           |
| <code>heapreplace</code>  | $O(\log n)$   | one sift-down               |
| <code>nsmallest(k)</code> | $O(n \log k)$ |                             |
| <code>nlargest(k)</code>  | $O(n \log k)$ |                             |
| <code>merge</code>        | $O(n \log k)$ | total, LAZY – $O(k)$ memory |

`heapify` against `n` pushes is the one that matters in practice:

|                                                  |               |              |
|--------------------------------------------------|---------------|--------------|
| 200,000 items, worst-case input (reverse sorted) |               |              |
| <code>heapify:</code>                            | $\sim 0.01$ s |              |
| <code>n pushes:</code>                           | $\sim 0.05$ s | roughly 4-5x |

The measurement is in the code above. Same result, four times the work, one line to fix.

### heapreplace against pop-then-push

```
heappop + heappush two operations: a sift-down and a sift-up
heapreplace ONE sift-down
```

```
in a top-k loop over 1,000,000 items with k = 100:
 the comparison `if v > h[0]` rejects ~99.99% before touching the heap
 the survivors cost one sift-down each instead of two passes
```

**In a hot loop that is a real constant-factor win**, and it is why the fixed-size-heap idiom is written that way.

### Top-k, three ways

```
n = 1,000,000

k = 10
sorted(data)[-10:] O(n log n) ~20,000,000 comparisons
heapq.nlargest(10, data) O(n log k) ~3,300,000 ← 6x fewer
size-k heap + replace O(n log k) ~3,300,000, and O(k) memory

k = 500,000
sorted(data)[-k:] O(n log n) ~20,000,000, excellent constants
heapq.nlargest(k, data) O(n log k) ~19,000,000, worse constants
→ SORT WINS. nlargest is not always the answer.
```

**The crossover is around  $k \approx n/10$** , and knowing that there is a crossover is the point.

```
k = 1 → max(data), O(n), no heap at all.
```

### Lazy deletion's memory cost

```
Dijkstra on a graph with V = 100,000 and E = 500,000
```

```
indexed heap: the heap holds at most V = 100,000 entries
lazy deletion: the heap holds up to E = 500,000 entries
 → 5x the memory, and ~5x the pops (most skipped)
```

**Still  $O(E \log V)$  overall**, which is the same asymptotic bound, and the constant is worth paying to avoid maintaining an index map on every swap.

## Tuple overhead

|                                |                                  |
|--------------------------------|----------------------------------|
| a heap of 1,000,000 plain ints | ~8 MB list + the int objects     |
| a heap of 1,000,000 3-tuples   | ~8 MB list + ~72 bytes per tuple |
|                                | ≈ 80 MB                          |

**Ten times the memory for the tuple wrapper.** For a large heap, `nlargest` with a `key` — which keeps only `k` items — or a class with `__slots__` is worth considering.

## Comparison count

```
heapq uses ONLY '<'.
→ a class needs only __lt__
→ @dataclass(order=True) generates all six, which is more than needed
→ functools.total_ordering also works and is heavier
```

**Each sift-down does two comparisons per level**, so a pop from a million-element heap is about forty comparisons. If `__lt__` is expensive — a database lookup, a string collation — that is where the time goes, not in the heap machinery.

## 7 The traps

### Trap 1 — expecting a max-heap parameter

```
heapq.heappush(h, x, reverse=True)
```

```
TypeError: heappush() takes 2 positional arguments but 3 were given
```

**There is no such option.** Negate, or invert `__lt__`.

### Trap 2 — the missing tie-breaker

```
heapq.heappush(h, (1, {"a": 1}))
heapq.heappush(h, (1, {"b": 2}))
```

```
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

**It only fails when priorities tie**, which means it passes in testing and raises in production on the first collision. Always include a counter.



### Trap 3 — a tie-breaker that can repeat

```
heapq.heappush(h, (priority, time.time(), payload))
```

`time.time()` has limited resolution, so two pushes in the same microsecond produce equal timestamps and Python falls through to the payload. Use `itertools.count()` — strictly increasing, always comparable, and it gives FIFO ordering for free.

### Trap 4 — touching the list directly

```
h.append(0)
heapq.heappop(h) # does NOT return 0
```

The list is the heap and appending does not sift. **The invariant is broken silently**, and later pops return wrong values. Same for `h[i] = x` and for sorting the list yourself.

### Trap 5 — building by pushing

```
for v in items:
 heapq.heappush(h, v) # $O(n \log n)$
```

Four to five times slower than `heapq.heapify(items)` for the same result. **If you have the items, use `heapify`.**

### Trap 6 — negating something that cannot be negated

```
heapq.heappush(h, -"banana")
```

```
TypeError: bad operand type for unary -: 'str'
```

And worse, the *silent* version: negating only the first element of a tuple gives an ordering that is inverted on the first field and normal on the rest, which is very rarely what anyone wants.

### Trap 7 — `nlargest` when `k` is close to `n`

```
heapq.nlargest(500_000, million_items)
```

Correct and slower than `sorted()`.  $O(n \log k)$  with  $k \approx n$  is  $O(n \log n)$  with worse constants and more memory. **The crossover is around  $k \approx n/10$ .**

#### Trap 8 — mutating an item that is inside the heap

```
task.priority = 1 # the heap does not notice
```

The heap ordered the item when it was pushed and has no way to observe a change. **Push a new entry and skip the stale one** — lazy deletion.

#### Trap 9 — `nan` in a heap of floats

```
heapq.heappush(h, float("nan"))
```

`nan < x` and `x < nan` are both `False`, so the sift comparisons behave arbitrarily and the heap property is quietly meaningless. **No exception.** Filter `nan` out before it goes in.

### 8 In the interview

#### How it gets asked

- The opener: “Python only has a min-heap. How do you get the maximum?”
- The follow-up that decides it: “And if the items are objects rather than numbers?”
- The tuple probe: “You are pushing tuples — what happens when two priorities are equal?”
- The practical one: “How do you change the priority of something already in the queue?”
- The efficiency one: “You have all million items already. How do you build the heap?”

#### What to say out loud, in the first ninety seconds

1. **Give the quick answer and immediately its limit.** “Negate on the way in and on the way out — the minimum of the negatives is the maximum. That works for numbers and only for numbers.”
2. **Give the general answer.** “For anything else, wrap the item in a class with an inverted `__lt__`. `heapq` only ever uses `<`, so that is the only method it needs — and it also handles compound orderings that negation cannot express.”
3. **Reject the private functions by name.** “There are undocumented `_heapify_max` and `_heappop_max` functions. I would not use them — there is no `_heappush_max`, and they change between versions.”
4. **Bring up the tie-breaker before being asked.** “If I am pushing tuples, I always include a counter as the second element. Tuples compare left to right, so if two priorities tie, Python compares whatever comes next — and if that is a dict or a cus-

tom object, it raises.”

5. **Say the tie-breaker’s two requirements.** “It has to be always comparable and never equal. A timestamp fails the second on a fast machine; a name fails the first. `itertools.count()` does both, and gives FIFO within a priority for free.”
6. **Say what you must not do to the list.** “And there is no heap object — the list is the heap. Appending to it or assigning to an element breaks the invariant with no exception.”

#### The follow-ups

“**And if the items are objects rather than numbers?**” “Then negation is not available, and this is where most people stop. The general answer is to **invert the comparison** rather than the values: wrap the item in a small class whose `__lt__` returns `self.key > other.key`. `heapq` only ever uses `<` — it never calls `>`, `=` or `__gt__` — so `__lt__` is the only method that must exist, and a `@dataclass(order=True)` with `compare=False` on the payload generates it and keeps the payload out of comparisons entirely. That approach has two advantages over negation beyond just working: it handles **compound orderings** — largest score first, then earliest timestamp — which negation cannot express, and it solves the tie-breaker problem at the type level, because the payload is never compared at all.”

“**What happens when two priorities are equal?**” “Python moves on to the next tuple element, and that is the bug. If my tuple is `(priority, payload)` and two priorities tie, it compares the payloads — and a dict or a custom object without `__lt__` raises `TypeError: '<' not supported between instances of 'dict' and 'dict'`. The nasty part is that it **only fails on a collision**, so it passes every test with distinct priorities and raises in production the first time two things are equally urgent. So I always push `(priority, counter, payload)` with a counter from `itertools.count()`. The tie-breaker has exactly two requirements and needs both: **always comparable** — an integer, where a name fails across scripts — and **never equal** — strictly increasing, where a timestamp fails on a fast machine. And it gives FIFO ordering within a priority for free, which is usually what you actually want.”

“**How do you change the priority of something already in the queue?**” “A heap cannot find an element, so it cannot update one — searching is  $O(n)$ . The textbook answer is an **indexed heap**: the heap plus a hash map from item to its current index, maintained on every swap, which makes decrease-key  $O(\log n)$ . Nobody writes that in Python. The idiomatic answer is **lazy deletion**: push a new entry with the new priority, leave the old one in place, and skip it when it surfaces — I keep a dict from item to its live sequence number and discard any popped entry whose sequence does not match. The cost is that the heap holds every version ever pushed rather than just

the live ones — for Dijkstra that is  $O(E)$  entries instead of  $O(V)$  — but it stays  $O(E \log V)$  overall and it keeps the hot path free of bookkeeping. That is exactly what a Python implementation of Dijkstra does, and it is why you see `if node in settled: continue` at the top of the loop.”

**“You already have a million items. How do you build the heap?”** `heapq.heapify(items)`, in place, which is  $O(n)$ . Pushing them one at a time is  $O(n \log n)$  — measured, about four to five times slower for exactly the same result — and it is the most common avoidable inefficiency in this topic. The reason `heapify` is linear is that it sifts **down** from the last parent backwards: half the nodes are leaves and do no work at all, a quarter can sink at most one level, and the nodes that could travel far are the rare ones near the root. Pushing sifts **up**, so the many nodes at the bottom are exactly the ones that can travel the full height, and that really is  $O(n \log n)$ .”

**“When would you not use a heap for top-k?”** “Two cases. When **k is 1**, `max()` is  $O(n)$  with no structure at all. And when **k is close to n**, sorting wins: `nlargest` is  $O(n \log k)$ , which at  $k \approx n$  is  $O(n \log n)$  with worse constants and more memory than a straight `sorted()`. The crossover is roughly  $k = n/10$  — at a million items and  $k = 10$  the heap does about three million comparisons against twenty million for a sort, a six-fold win; at  $k = 500,000$  the sort wins. So the answer is ‘a heap when k is small relative to n’, and knowing that there is a crossover at all is more useful than the exact figure. I would also mention `nlargest` with a `key=` argument, because it avoids tuples and tie-breakers entirely — the comparison never touches your objects.”

**“What can silently break a `heapq` heap?”** “Four things, and none of them raises. **Touching the list directly** — `append`, an index assignment, sorting it yourself — because there is no heap object and the list is the heap, so nothing re-establishes the invariant. **Mutating an item that is already inside**, because the heap ordered it at push time and cannot observe a change. **A missing tie-breaker**, which does raise but only on a collision, so it is silent until it is not. And `float('nan')`, which compares `False` against everything in both directions, so the sift comparisons behave arbitrarily and the heap property becomes meaningless with no error at all. My habit is to assert the invariant in tests — one line, `all(h[(i-1)//2] ≤ h[i] for i in range(1, len(h)))` — because every one of these is otherwise invisible.”

**A model answer**

Asked: *Python only has a min-heap. How do you get the maximum?*

“Two ways, and I would give both, because the first one is the common answer and it does not generalise.

**For numbers: negate.** Push `-v` and pop `-x`. The minimum of the negated values is the maximum of the originals, and nothing about the heap changes — I have inverted the *ordering*, not the structure. Two lines, and it is what I would write for a list of integers. One caveat worth stating if the language were C++ or Java: negating the most negative integer overflows. Python’s integers are arbitrary precision, so it does not bite here.

**For anything else: invert the comparison.** You cannot negate a string or an object, so instead I wrap the item in a small class whose `__lt__` returns `self.key > other.key`. `heapq` only ever uses `<` — it never calls `>` or `=` — so `__lt__` is the only method that has to exist. That version is strictly more general: it works for any type, and it handles **compound orderings** like ‘highest score, then earliest arrival’ that negation simply cannot express.

There are undocumented `_heapify_max` and `_heappop_max` functions in the module. I would name them and not use them — there is no `_heappush_max`, so the set is incomplete, and they are private and change between versions.

While I am here, the two things that actually bite people when using `heapq` for real work.

**Tuples and the tie-breaker.** To carry a payload you push `(priority, payload)` — and tuples compare left to right, so when two priorities are equal Python compares the *payloads*. If those are dicts or custom objects, that raises `TypeError: '<' not supported`. And it only happens **on a collision**, so it passes every test with distinct priorities and fails in production the first time two things are equally urgent. So I always push `(priority, counter, payload)` with `itertools.count()`. The tie-breaker needs two properties and both matter: **always comparable** — an integer, where a name fails across scripts — and **never equal** — strictly increasing, where a timestamp fails on a fast machine. It also gives FIFO within a priority for free.

**And changing a priority.** A heap cannot find an element, so there is no update. The textbook answer is an indexed heap with a value-to-index map maintained on every swap; the Python answer is **lazy deletion** — push the new version, leave the old one, and skip it when it surfaces by checking a live sequence number. That is what a Python Dijkstra actually does, and it costs memory proportional to the number of pushes rather than the number of live items.

Last thing: there is no heap object. The list is the heap, so `append`, an index assignment or sorting it yourself breaks the invariant with no exception at all.”

## 9 Recall card

- `heapq` is MIN-ONLY. Two fixes: NEGATE (numbers only — `push(-v)`, `pop` and negate back) or INVERT `__lt__` (any type, and the only way to express compound orderings). `heapq` uses **only** `<`, so `__lt__` is the only method a class needs. The private `_heapify_max` family is incomplete — name it and reject it.
- **Always push** `(priority, counter, payload)`. Tuples compare **left to right**, so equal priorities make Python compare the **payload** → `TypeError: '<' not supported between instances of 'dict' and 'dict'` — **and only on a collision**, so it passes tests and fails in production. The tie-breaker needs **both**: always comparable (an int — a *name* fails) and never equal (`itertools.count()` — a *timestamp* fails). Bonus: FIFO within a priority.
- **A heap cannot find or update — so changing a priority is LAZY DELETION**: push the new version, leave the old, skip it when it surfaces via a live sequence number. Costs `O(pushes)` entries instead of `O(live)` — and it is exactly what a Python Dijkstra does.
- **There is no heap object — the list IS the heap.** `append`, `h[i] = x`, or sorting it yourself breaks the invariant **with no exception**; so does mutating an item already inside; so does `float('nan')`, which compares `False` both ways. Assert `all(h[(i-1)//2] ≤ h[i] ...)` in tests.
- `heapify` is `O(n)`; `n` pushes is `O(n log n)` — measured  $\sim 4\text{--}5\times$  slower for the same result. Use `heapreplace` (one sift-down) rather than pop-then-push in a fixed-size top-k loop. And know where heaps stop winning: `k = 1` → `max()`; `k ≤ n/10` → `nlargest ( O(n log k) )`; `k` near `n` → `sorted()`. `nlargest(..., key=)` avoids tuples and tie-breakers entirely, and `heapq.merge` is lazy.

## 1 What this is, and why they ask it

A **consistency model** is a promise about what a read may return when there are several copies of the data. **CAP** said you must sometimes give up consistency; today is about the fact that consistency is not one thing you either have or lose — **it is a ladder**, and you choose a rung.

Three sentences. At the top is **linearizability**: the system behaves as if there were one copy and every operation happened at a single instant, which is what a single machine gives you for free and what a distributed system pays dearly for. At the bottom is **eventual consistency**: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile. And the interesting rungs are in the middle, particularly **causal consistency** and the four **session guarantees**, because they are what users actually notice and they are much cheaper than the top.

They ask “*what consistency does this feature need?*” because the answer is almost never “strong” and almost never “eventual” for a whole system — it is **per operation**, and the good answer names the specific anomaly it is preventing. A candidate who says “we’ll use eventual consistency” without being able to say *which* anomaly a user would see has not chosen anything.

## 2 The story

The news that Kuppan’s daughter had got the government job reached the village on a Tuesday, and Sarala spent the rest of the week reconstructing how.

It had started with Kuppan telling Ratnam at the milk booth at six in the morning. By eight, four people knew. By noon, most of the main street. By the evening, the far end near the burial ground had it, and by Thursday the outlying farms.

Three things about how it travelled struck her as interesting, and she pointed them out to her daughter-in-law, who was less interested.

The first was that at any given moment, different people knew different things, and none of them was wrong. The woman at the far end who had not heard yet was not mistaken — she simply had older information. And if you had stopped everybody talk-

ing on Tuesday evening, by Thursday everyone would have had the same news, without anybody doing anything special. It sorted itself out.

The second was about order, and this was the part she thought mattered. On Wednesday there was a second piece of news: that the family was arranging a small function on Sunday. Now — **nobody in the village heard about the function before they heard about the job**. Not one person. Because the function only made sense as a consequence of the job, so whoever passed it on always said both, in that order. The news travelled at different speeds to different people, but it never arrived out of order, because each piece was carried by somebody who already had the earlier piece.

The third was the one that had actually caused trouble, twice.

Sarala's own sister lived two streets away, and on Wednesday morning Sarala told her about the function. In the afternoon her sister came back from the temple and said, quite crossly, that she had asked three people and nobody had said anything about a function, and had Sarala made it up.

She had not. Her sister had simply asked people who had not been told yet. And what upset her was not being behind — it was that **she had heard it, and then heard nothing, and the second thing felt like it had taken the first one away**.

Sarala said the rule she had learned over sixty years was that you could not make news travel faster. What you could do was be careful about **who you sent somebody to ask**. If she wanted her sister to hear something confirmed, she sent her to the person she had told herself.

---

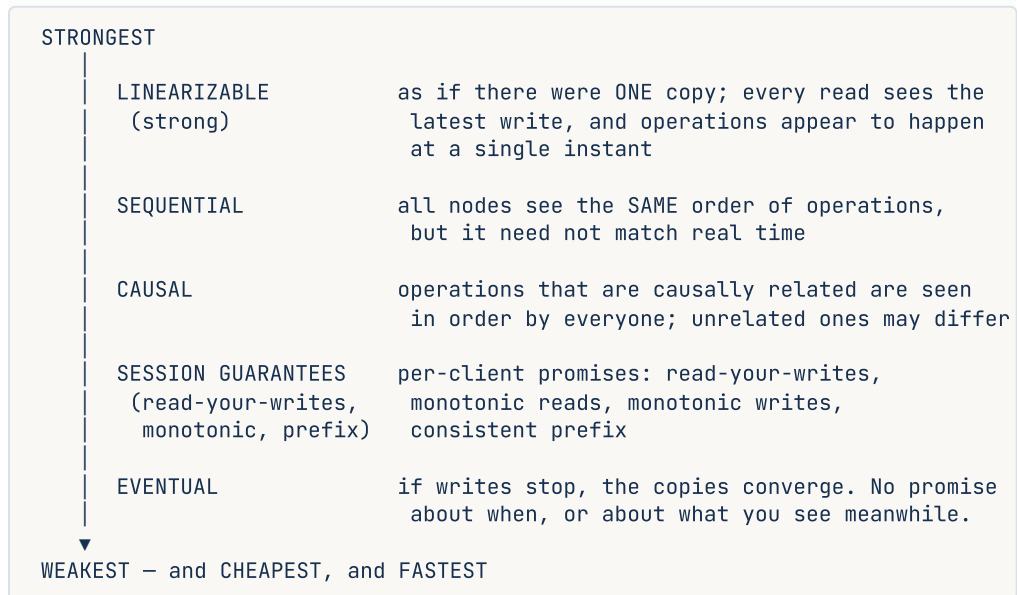
### 3 The idea in plain English

The village is an eventually consistent system, and Sarala has identified the three things that matter: convergence, causal order, and the session guarantees.

- Different people knowing different things is **replica divergence** — and none of them is wrong.
- “By Thursday everyone had it” is **eventual consistency**: convergence if writes stop.
- The function never arriving before the job is **causal consistency**.
- Her sister hearing it and then not hearing it is a **monotonic reads** violation.
- “Send her to the person I told” is **session guarantees** — routing to preserve what one person has seen.



## The ladder



**Going down the ladder buys latency and availability and costs anomalies.** The engineering question is always *which anomaly can this feature tolerate?* — not *how consistent can we be?*

**Linearizable:** as if there were one copy

**The definition to give:** every operation appears to take effect at a single instant between its start and its completion, and every read returns the most recently completed write.

The consequence that makes it expensive: **it is a real-time guarantee.** If I write and then telephone you, and you read, you must see my write — even though our two operations touched different machines and nothing connects them but the phone call.

what it costs:  
a read must consult a majority, or come from the leader  
→ same region: ~5-10 ms instead of ~1 ms  
→ cross-region: ~150 ms  
→ and during a partition, the minority side must REFUSE (CAP)

**Use it for:** locks, leader election, unique-constraint checks, account balances at the moment of a transfer, inventory at the point of sale, anything where two clients can race for the same thing.

**Sequential: everyone agrees on an order, just not the real one**

All nodes observe the same sequence of operations, and each client's own operations appear in the order it issued them — but the global order need not match wall-clock time.

```
A writes x=1 at 10:00:00
B writes x=2 at 10:00:01
→ a sequentially consistent system may order them (x=2, x=1)
 as long as EVERY node sees that same order
```

**In practice this rung is rarely offered explicitly**, but it is worth knowing because it is exactly the gap between “everyone agrees” and “it matches real time” — and that gap is what makes linearizability expensive.

**Causal: the important middle rung**

**If one operation could have influenced another, everyone sees them in that order. Operations that could not have influenced each other may be seen in any order.**

The village's news. Nobody heard about the function before the job, because the function was only ever passed on by somebody who already had the job news.

|                  |                                                                                                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------|
| causally related | A posts a photo, then A comments on it<br>→ nobody sees the comment before the photo                                 |
| concurrent       | A posts a photo, B posts an unrelated photo<br>→ different viewers may see them in either order,<br>and that is FINE |

**Why it matters:** almost every anomaly users actually complain about is a *causality* violation, not a staleness one.

|                                                |                             |
|------------------------------------------------|-----------------------------|
| "I replied to a message that isn't there"      | causal violation – REAL bug |
| "the like count says 41 and my friend sees 42" | staleness – nobody cares    |

**Causal consistency is available during a partition**, which linearizability is not — so it is the strongest model that is still AP. That single fact is why it is the most interesting rung on the ladder, and it is implemented with the vector clocks and version vectors from [day 113](#).

**The four session guarantees**

These are per-client promises, cheap to implement, and they cover most of what users notice.

**Read-your-writes.** *If I do something, I see it.* The most-reported bug in the phase — [day 105](#). Fix: route a user's reads to the leader for a window after they write, or carry a version token.

**Monotonic reads.** *Time does not go backwards.* Sarala's sister. If I see a value, a later read must not show an older one. Fix: **pin a client to one replica**.

**Monotonic writes.** *My own writes are applied in the order I made them.* If I set my name and then set my photo, the system must not apply them in the other order. Fix: route a client's writes through one path, or number them.

**Consistent prefix reads.** *Cause before effect.* You must not see the answer to a question before the question. Mostly a **sharding** problem — two related writes travelling through different partitions can arrive out of order.

**These four are worth memorising as a set**, because “which of the four does this feature need?” is a much better answer than “strong or eventual”.

Eventual: convergence, and nothing else

**If writes stop, all replicas will converge to the same value.** That is the entire promise. It says nothing about how long, and nothing about what a read returns meanwhile.

**Which means it permits things that surprise people:**

|                                                                              |                       |
|------------------------------------------------------------------------------|-----------------------|
| a read may return a value older than one you already saw                     | (no monotonic reads)  |
| you may not see your own write                                               | (no read-your-writes) |
| two reads in a row may disagree                                              | (no ordering at all)  |
| a value may go $5 \rightarrow 7 \rightarrow 5 \rightarrow 7$ before settling |                       |

**“Eventually” has no bound.** In practice convergence is milliseconds; during a partition it is the length of the partition; and if two replicas never reconcile, it is never.

**Use it for:** view counts, like counts, recommendations, search indexes, analytics, anything where the user cannot tell and would not care.

**And the honest addition:** eventual consistency requires a **conflict resolution** strategy, which is where the real design work is — last-write-wins (which loses data), version vectors, CRDTs, or an application merge.

CRDTs, briefly, because they change the question

A **conflict-free replicated data type** is a structure whose merge function is designed so that any two replicas that have seen the same set of updates end up identical, **regardless of the order they arrived in**.

|                 |                                                  |
|-----------------|--------------------------------------------------|
| a G-counter     | each node counts its own increments; merge = sum |
| a G-set         | add-only; merge = union                          |
| an OR-set       | add and remove with tags; merge is well defined  |
| an LWW-register | last write wins, with a tie-break rule           |

**The point is that conflicts become impossible by construction**, not that they are resolved well. The price is that you can only store what a CRDT can express — and a bank balance with a minimum of zero is not one of them, because “do not go below zero” is a global invariant and CRDTs deliberately have none.

**Redis, Riak and collaborative editors use them.** Naming CRDTs and saying what they cannot do is a strong finish.

How to choose, in one question

**What would a user actually notice, and how bad is it?**

|                                              |                             |
|----------------------------------------------|-----------------------------|
| the user does something and cannot see it    | → read-your-writes. Fix it. |
| the user sees it, then it vanishes           | → monotonic reads. Fix it.  |
| a reply appears before the message           | → causal. Fix it.           |
| two people can take the same seat            | → LINEARIZABLE. No choice.  |
| the like count is off by one for two seconds | → eventual. Leave it.       |

**Name the anomaly, not the model.** That is the whole skill.

#### 4 The picture

The ladder, with the cost on one side and the anomalies on the other.

|                       | cost                                                                                           |                                                          | what it prevents                                                 |
|-----------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------------|
| LINEARIZABLE          | <br>~5-150 ms | leader or quorum<br>reads; refuses during<br>a partition | everything<br>(and it is the only<br>one that prevents races)    |
| SEQUENTIAL            | <br>~1-5 ms   | global agreement on<br>order, not on time                | disagreement about order                                         |
| CAUSAL                | <br>~1 ms     | version vectors;<br>AVAILABLE during a<br>partition      | effect-before-cause<br>← the strongest model<br>that is still AP |
| SESSION<br>GUARANTEES | <br>~1 ms     | per-client routing<br>or a token                         | "I can't see my own post"<br>"it appeared then vanished"         |
| EVENTUAL              | <br>~1 ms     | read the nearest<br>replica                              | nothing – only convergence<br>(if writes stop)                   |

going DOWN buys latency and availability, and costs anomalies.  
the question is never "how consistent can we be" – it is  
"WHICH ANOMALY CAN THIS FEATURE TOLERATE?"

The village, as the ladder:

|                 |                                                                                                                                                                            |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EVENTUAL        | by Thursday everyone knew – with no one doing anything special<br>"different people knew different things, and none was wrong"                                             |
| CAUSAL          | NOBODY heard about the function before the job,<br>because whoever passed on the function already had the job news<br>→ causally related news never arrived out of order   |
| MONOTONIC READS | Sarala's sister heard it, then asked three people who had not<br>been told, and it FELT like the news had been taken away<br>→ the violation users actually complain about |
| THE FIX         | "send her to the person I told" – route her to a replica<br>that has already seen it                                                                                       |

The four session guarantees, with the user's own words:

|                   |                                                                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| READ-YOUR-WRITES  | "I posted it and it's not there"<br>write → leader. read → a replica that hasn't caught up.<br>FIX: read from the leader for ~30 s after a write.                                      |
| MONOTONIC READS   | "it was there, now it's gone"<br>read 1 → replica A (lag 20 ms). read 2 → replica B (lag 2 s).<br>FIX: pin the client to ONE replica.                                                  |
| MONOTONIC WRITES  | "I changed my name then my photo, and the name reverted"<br>write 1 and write 2 took different paths and arrived out of order.<br>FIX: one write path per client, or sequence numbers. |
| CONSISTENT PREFIX | "the reply is above the message"<br>two related writes on DIFFERENT SHARDS arrive out of order.<br>FIX: co-locate related writes, or a causal token.                                   |

Which rung for which feature:

| feature                                                | model            | why                              |
|--------------------------------------------------------|------------------|----------------------------------|
| seat booking                                           | LINEARIZABLE     | two people, one seat             |
| account balance at transfer                            | LINEARIZABLE     | a race is money                  |
| leader election / a lock                               | LINEARIZABLE     | two leaders is a disaster        |
| a user's own profile edit                              | read-your-writes | "I saved it and nothing changed" |
| a chat thread                                          | CAUSAL           | a reply before its message       |
| a social feed                                          | CAUSAL           | a comment before its post        |
| a like count                                           | EVENTUAL         | off by one for 2 s: invisible    |
| a recommendation list                                  | EVENTUAL         | nobody can tell                  |
| an analytics dashboard                                 | EVENTUAL         | minutes stale is expected        |
| A SYSTEM IS NOT "STRONGLY CONSISTENT". OPERATIONS ARE. |                  |                                  |

## 5 How it actually works

### Achieving each rung

**Linearizable:** all reads and writes go through a single leader, **or** a quorum with  $R + W > N$  plus a protocol that prevents a stale leader answering. **The subtlety is that quorum reads alone are not enough** — you also need to defend against an old leader that has not yet noticed it was replaced, which is what fencing and epoch numbers do.

**Causal:** attach a **version vector** to each write recording what the writer had seen, and a replica delays applying a write until it has applied everything that write depends on. That is why causal consistency is available during a partition: a replica can keep serving from what it has, and it just holds back writes whose dependencies have not arrived.

**Session guarantees:** mostly routing.

|                   |                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------|
| read-your-writes  | route to the leader for N seconds after a write,<br>or send a version token with the read |
| monotonic reads   | pin the client to one replica (hash the user id)                                          |
| monotonic writes  | one write path per client, or sequence numbers                                            |
| consistent prefix | co-locate causally related data on one shard                                              |

**Eventual:** read the nearest replica, write anywhere, reconcile in the background — with anti-entropy (replicas comparing and exchanging what they lack) and read-repair (fixing a stale replica when a read notices).

#### Tunable consistency, per query

Cassandra and DynamoDB let you choose per request, which makes the per-operation argument concrete:

```
N = 3 replicas

W=1, R=1 fastest; 1+1 = 2, NOT > 3 → may read stale
W=2, R=2 2+2 = 4 > 3 → strongly consistent, both slower
W=3, R=1 fast reads, brittle writes → a single replica down blocks writes
W=1, R=3 fast writes, slow reads
```

**$R + W > N$**  guarantees the read set and the write set overlap, so at least one replica returns the newest value. That is [day 117](#), and it is the dial people mean when they say they “tuned consistency”.

#### What real systems offer

- **PostgreSQL / MySQL on one primary** — linearizable for writes; reads from replicas are eventually consistent unless you route them to the primary.
- **DynamoDB** — eventually consistent reads by default, strongly consistent reads on request at **twice the capacity cost** and higher latency. The trade, priced per request.
- **Cassandra** — tunable per query, from ONE to QUORUM to ALL, plus **lightweight transactions** using Paxos when you genuinely need linearizability for one key.
- **MongoDB** — read and write concerns per operation, plus **causally consistent sessions**, which is one of the clearest implementations of that rung.
- **Spanner** — externally consistent, which is linearizability plus a real-time ordering guarantee across the whole database, bought with **TrueTime** and a commit-wait of a few milliseconds on every transaction.
- **Redis** — asynchronous replication, so a failover can lose recent writes: not linearizable, and the documentation says so.

- **Collaborative editors** (Google Docs, Figma) — CRDTs or operational transformation, because the requirement is that everyone converges to the same document and nobody's keystroke is lost, which is a merge problem rather than an ordering one.

### Measuring it

You cannot reason your way to correctness here, and the practice worth naming is **Jepsen**: run a real cluster, partition it deliberately, record every operation, and check afterwards whether the history is consistent with the model the vendor claims. **It has found violations in most major distributed databases**, including several that advertised linearizability.

“**I would want to see the Jepsen report**” is a genuinely strong thing to say about any distributed store.

## 6 The numbers

### The cost of each rung

| model             | typical read latency                         | availability during a partition    |
|-------------------|----------------------------------------------|------------------------------------|
| eventual          | ~1 ms (nearest replica)                      | fully available                    |
| session/monotonic | ~1 ms (pinned replica)                       | available                          |
| causal            | ~1-5 ms                                      | AVAILABLE – the strongest AP model |
| linearizable      | ~5-10 ms same region<br>~150 ms cross-region | minority side REFUSES              |

DynamoDB, per request:

|                            |                         |               |
|----------------------------|-------------------------|---------------|
| eventually consistent read | 0.5 read capacity units |               |
| strongly consistent read   | 1.0 units               | → 2x the cost |

**A vendor charging double is the clearest statement of the trade there is.**

### Staleness in practice

|                                 |                        |
|---------------------------------|------------------------|
| same datacentre, healthy        | 1-50 ms behind         |
| same datacentre, heavy writes   | 100 ms - a few seconds |
| cross-region                    | 200 ms+                |
| during a bulk load or migration | seconds to MINUTES     |

$P(\text{a user hits a stale read}) = P(\text{they read within the lag window})$

a UI that reloads immediately after a write:

read at ~50 ms after the write, replica lag ~200 ms  
→ stale ESSENTIALLY ALWAYS



**That is why read-your-writes is the anomaly users report**, and staleness in general is not: the read that follows a write is not a random sample, it is the worst possible sample.

#### What each fix costs

```
read from the leader for 30 s after a write
 reads inside those windows ~10-15% of a user's reads
 → that much read load returns to the leader

pin a client to one replica
 load spread between busiest and quietest 2-3x
 → ~30% wasted capacity

causal consistency with version vectors
 metadata per write one counter per replica
 → at 5 replicas, ~40 bytes per record; at 100, it stops being free

quorum reads (R=2, W=2, N=3)
 latency: the SECOND-slowest replica, not the fastest
 → typically 2-3x the single-replica read latency
```

#### The convergence window

```
anti-entropy interval typically seconds to minutes
read-repair on the next read of that key
hinted handoff replay when the failed node returns

so "eventually" in practice:
 milliseconds, if the key is read often
 minutes, if it is not
 never, if the replicas never exchange and no read touches it
```

**“Eventually” has no bound in the model and a practical distribution in reality** — and a key that is written and never read may stay divergent for a very long time.

## 7 The trade-offs

**Strong consistency costs latency always and availability sometimes**

**Always**, because a linearizable read must consult more than one machine — the “else” branch of [PACELC](#). **Sometimes**, because during a partition the minority must refuse.

**I would not use strong consistency for a read that a user cannot verify.** Nobody can tell whether a like count is two seconds old, and paying quorum latency for it on every request is buying a guarantee no one will ever observe.

Eventual consistency's cost is not staleness, it is the conflict problem

Staleness is usually invisible. What is not invisible is **what happens when two replicas disagree**, and “eventual” says nothing about that. You must choose:

- **Last-write-wins** — silently discards data, and with clock skew may keep the *older* write.
- **Version vectors** — correct detection, and the application must then have a resolution rule.
- **CRDTs** — no conflicts by construction, and you can only store what they can express.
- **Application merge** — Amazon's cart union: over-keep rather than lose.

**Choosing “eventual” without choosing one of those is not a design.**

Causal is the underrated middle

**It prevents the anomalies users actually complain about** — effect before cause — and it remains available during a partition, so it is the strongest model that does not force a CAP choice.

Its cost is **metadata**: a version vector per write, which grows with the number of replicas, and a replica that must hold back writes whose dependencies have not arrived.

**I would reach for causal (or the session guarantees) far more often than for linearizable**, and saying that is a considered position rather than a default.

Session guarantees are the cheapest real win

Read-your-writes and monotonic reads cost a routing rule, and between them they remove the two complaints that generate support tickets. **They are the highest ratio of user-visible improvement to engineering cost anywhere in this phase.**

Where the reasoning breaks down

- **Claiming a model without testing it.** Jepsen has found violations in most major systems, including ones that advertised linearizability. The correct posture is scepticism.
- **“Eventually” with no bound.** A key that is written and never read can stay divergent indefinitely, and anti-entropy intervals of minutes are normal.
- **Mixing models within one user action.** If checkout reads inventory from a replica and writes the order to the leader, the guarantee of the whole action is the weakest link.

- **Assuming a transaction gives you distributed consistency.** ACID's C is about integrity constraints within one database. It says nothing about several machines.

## 8 In the interview

### How it gets asked

- The direct one: *"What consistency does this feature need? Justify it."*
- The ladder question: *"What is the difference between strong and eventual consistency?"*
- The one that separates people: *"Is there anything in between?"*
- The applied one: *"A user posts a comment and cannot see it. Which guarantee is missing?"*
- The sceptical one: *"How would you know your database actually provides what it claims?"*

### What to say out loud, in the first ninety seconds

1. **Reject the binary.** "Consistency is not strong-or-eventual — it is a ladder, and the useful rungs are in the middle."
2. **Define the top precisely.** "Linearizable means the system behaves as if there were one copy: every read sees the latest completed write, in real time. That is what a single machine gives you free and what a distributed system pays for."
3. **Define the bottom precisely, including what it does not promise.** "Eventual means: if writes stop, the replicas converge. It promises nothing about when, and nothing about what you see meanwhile — a read can even return something older than one you already saw."
4. **Name the middle, and why it matters.** "In between is **causal** — anything that could have influenced anything else is seen in that order — and the **session guarantees**: read-your-writes, monotonic reads, monotonic writes, consistent prefix. Causal is the strongest model that is still available during a partition."
5. **Make it per operation.** "And it is chosen per operation. In one system: seat booking is linearizable, a chat thread is causal, and a like count is eventual."
6. **Choose by the anomaly.** "The question I actually ask is: what would a user notice, and how bad is it? 'I posted and cannot see it' is read-your-writes and I fix it. 'The count is off by one for two seconds' is eventual and I leave it."

## The follow-ups

**“Is there anything in between strong and eventual?”** “Yes, and it is where most real design happens. The rung that matters most is **causal consistency**: if one operation could have influenced another, everyone sees them in that order; operations that could not have influenced each other may be seen in any order. That is what prevents the anomalies users actually complain about — a reply appearing above the message it replies to, a comment before the post. And the crucial property is that **causal consistency remains available during a partition**, which linearizability does not — so it is the strongest model that does not force a CAP choice. Below it are the four **session guarantees**, which are per-client and very cheap: read-your-writes, monotonic reads, monotonic writes and consistent prefix. Those four cover almost everything a user would report, and they cost routing rules rather than quorums.”

**“A user posts a comment and cannot see it. Which guarantee is missing?”** **“Read-your-writes**, and it is the most-reported anomaly in this whole area — because the read that follows a write is not a random sample, it is the worst possible one. The write went to the leader; the read went to a replica a few hundred milliseconds behind; and the interface reloads immediately, so it lands squarely inside the lag window essentially every time. Three fixes, in increasing precision. **Route that user’s reads to the leader for a window after they write** — thirty seconds, simple, and it costs maybe ten to fifteen percent of the read load coming back to the leader. **Object-based routing** — only reads of things they recently touched, which is a few percent. Or a **version token**: the write returns the leader’s log position, the client sends it back with reads, and a replica that has not reached it either waits or forwards. And I would mention its sibling: **monotonic reads**, where the comment appears and then disappears because two consecutive reads hit replicas with different lag — that one looks like data loss, and the fix is to pin the client to one replica.”

**“What consistency does this feature need?”** “That depends on what a user would notice and how bad it is, and I would go feature by feature rather than declaring a system-wide policy. Anything where **two clients can race for the same thing** must be linearizable — seat booking, taking the last item of stock, a lock, a leader election, a balance at the moment of a transfer — because there the anomaly is not staleness, it is two people getting the same thing, and no amount of eventual convergence undoes it. Anything with **causal structure** wants causal consistency: chat threads, comments on posts, anything where B only makes sense given A. Anything a user **does to their own data** needs at least read-your-writes. And anything the user **cannot verify** — a like count, a recommendation list, a search index, an analytics figure — should be eventual, because paying quorum latency for a guarantee nobody can observe is pure cost.”

**“What exactly does eventual consistency promise?”** “Very little, and it is worth being precise because people assume more. It promises **only that if writes stop, all replicas will converge to the same value**. It does not say how long that takes. It does not promise you see your own writes. It does not promise that two consecutive reads agree — you can read a value and then read an older one. And ‘eventually’ has no bound in the model: in practice it is milliseconds for a hot key that gets read-repaired constantly, minutes for a cold one that waits for an anti-entropy pass, and indefinite for a key that is written once and never read. The other thing eventual consistency does not give you is a **conflict resolution** strategy — and that is where the real work is. Choosing ‘eventual’ without also choosing last-write-wins, version vectors, CRDTs or an application merge is not a design, it is a gap. And I would note that last-write-wins silently loses data, and with clock skew it can keep the older write.”

**“How would you know your database actually provides what it claims?”** “I would look for a **Jepsen** report, and I would be sceptical without one. Jepsen is the practice of running a real cluster, deliberately partitioning it, recording every operation with its start and end time, and then checking whether the resulting history is consistent with the model the vendor advertises. It has found violations in most major distributed databases — including several that claimed linearizability — which tells you two things: that these guarantees are genuinely hard to implement, and that documentation is not evidence. The related point for my own systems is that **you cannot reason your way to this**; the failures are timing-dependent and rare, so they need deliberate fault injection rather than ordinary tests.”

**“What about CRDTs?”** “They change the question rather than answering it. A conflict-free replicated data type is a structure whose merge is designed so that any two replicas that have seen the same set of updates end up identical **regardless of the order the updates arrived in** — a counter that sums per-node counts, a grow-only set that unions, an observed-remove set with tags. So conflicts become impossible by construction, not resolved well after the fact, and you get strong eventual consistency without coordination. The price is that **you can only store what a CRDT can express**, and the things they cannot express are exactly the global invariants — a balance that must not go below zero cannot be a CRDT, because that constraint is about the whole system and CRDTs deliberately have no global view. They are excellent for collaborative editing, presence, counters and sets, and they are not a general replacement for a transactional store.”

**A model answer**

*Asked: what consistency does this feature need? Justify it.*

“Before I answer for a feature, I want to reject the binary, because the useful part is the middle. Consistency is a **ladder**, not a switch.

At the top is **linearizable**: the system behaves as if there were exactly one copy, and every read returns the most recently completed write — a **real-time** guarantee, so if I write, then telephone you, and you read, you must see my write. That is what one machine gives you free and what a distributed system pays for: a read must consult the leader or a quorum, which is five to ten milliseconds in a region and about a hundred and fifty across regions, and during a partition the minority side must refuse.

At the bottom is **eventual**: if writes stop, the replicas converge. That is the entire promise. It does not say when, it does not promise I see my own writes, and it permits a read returning something older than one I already saw.

In between are the rungs that matter. **Causal consistency** — anything that could have influenced anything else is seen in that order by everyone, while genuinely concurrent operations may be seen in either order. That prevents the anomalies users actually report: a reply above its message, a comment before its post. And critically it **remains available during a partition**, so it is the strongest model that does not force a CAP choice. Below that are the four **session guarantees** — read-your-writes, monotonic reads, monotonic writes, consistent prefix — which are per-client, cost routing rules rather than quorums, and cover almost everything a user would complain about.

So my answer for a feature is chosen by asking **what would a user notice, and how bad is it?**

If two clients can **race for the same thing** — a seat, the last item in stock, a lock, a balance at the moment of a transfer — it must be **linearizable**, because the anomaly is not staleness, it is two people getting the same thing, and convergence afterwards does not undo it.

If the data has **causal structure** — a chat thread, comments on a post — **causal**, because effect before cause is a real bug and staleness is not.

If a user is looking at **their own data just after changing it**, at least **read-your-writes**, which I would implement by routing their reads to the leader for thirty seconds afterwards.

And if the user **cannot verify it** — a like count, recommendations, a search index, a dashboard — **eventual**, because paying quorum latency for a guarantee nobody can observe is pure cost.

One thing I would add: choosing eventual is not finished until I have chosen a **conflict resolution** strategy. Last-write-wins silently loses data and, with clock skew, can keep the older write. Version vectors detect true concurrency and hand it to the application. CRDTs make conflicts impossible by construction, at the price of only storing what they can express. And I would want to see a **Jepsen** report before believing any vendor's claim about which rung they are on."

## 9 Recall card

- **Consistency is a LADDER, not a switch: linearizable → sequential → CAUSAL → session guarantees → eventual.** Going down buys **latency and availability** and costs **anomalies**. The question is never "how consistent can we be" but "**which anomaly can this feature tolerate?**"
- **Linearizable = as if there were ONE copy, in real time.** Needs the leader or a quorum (~5–10 ms in region, ~150 ms across) and **refuses on the minority side of a partition**. Required whenever **two clients can race for the same thing**: seats, stock, locks, balances, leader election.
- **Causal is the underrated middle: anything that could have influenced anything else is seen in that order — and it stays AVAILABLE during a partition**, so it is the strongest model that is still AP. It prevents the anomalies users actually report (a reply above its message); staleness is not one of them.
- **The four session guarantees, by the user's own words:** "*I posted it and it's not there*" = **read-your-writes** (route to the leader for ~30 s, ~10–15% of reads); "*it was there, now it's gone*" = **monotonic reads** (pin to one replica); "*my two edits applied backwards*" = **monotonic writes**; "*the reply is above the message*" = **consistent prefix** (a sharding problem). **Cheapest real win in the phase.**
- **Eventual promises ONLY convergence if writes stop** — not your own writes, not monotonic reads, and with **no bound** on "eventually". Choosing it is not a design until you choose a **conflict resolution** strategy: last-write-wins (**loses data**, and with clock skew keeps the *older* write) · version vectors · **CRDTs** (no conflicts by construction, but cannot express global invariants like "never below zero") · application merge. **DynamoDB charges 2× for a strongly consistent read**, and **Jepsen** has found violations in most major systems — documentation is not evidence.

## PRACTICE

## Practice — Python's heapq, and the min-heap-only problem

**DSA topic:** Python's heapq, and the min-heap-only problem **System design topic:** Consistency models

Code these, in this order

One rule for the whole set: **every tuple you push has three elements** — priority, counter, payload — even when you think you do not need the counter. The bug it prevents only appears on a tie, which means it passes your tests and fails in production.

| # | PROBLEM                         | SOURCE                | WHAT IT IS REALLY TESTING                                                    |
|---|---------------------------------|-----------------------|------------------------------------------------------------------------------|
| 1 | Kth Largest Element in an Array | LeetCode 215 (Medium) | The max-heap workaround, and heap-versus-sort with numbers.                  |
| 2 | Top K Frequent Elements         | LeetCode 347 (Medium) | Tuples, the tie-breaker, and <code>nlargest</code> with a <code>key</code> . |
| 3 | Task Scheduler                  | LeetCode 621 (Medium) | A max-heap of counts, and re-pushing after a cooldown.                       |
| 4 | Network Delay Time              | LeetCode 743 (Medium) | Dijkstra with lazy deletion, written the Python way.                         |

On problem 1, do it four ways

`sorted()[-k]`, `heapq.nlargest`, a size-k heap with `heapreplace`, and `quickselect`. Time all four at  $k = 10$  and  $k = 500,000$  on a million elements. Record eight numbers.

On problem 2, break it on purpose

Push `(count, item_dict)` with two items of equal count and record the exact `TypeError`. Then fix it with a counter, and then rewrite the whole thing with `nlargest(..., key=...)` and say which you prefer.

On problem 3, use the right shortcut

Write the inner loop with `heappop` followed by `heappush`, then with `heapreplace`. Say how many passes over the heap each does.



### On problem 4, do not write an indexed heap

Write it with lazy deletion — push a better distance, and skip an entry whose node is already settled. Say how many entries the heap can hold in terms of  $V$  and  $E$ .

### The min-only drill

1. Give both ways to get a max-heap.
2. Say exactly where negation stops working, with three examples.
3. Write the inverted-`__lt__` class from memory.
4. Say which methods `heapq` actually calls, and what that means for a custom class.
5. Name the private max functions and say why you would not use them.

### The tuple drill

1. Say how tuples compare, and where the comparison stops.
2. Construct the `TypeError` and quote it exactly.
3. Say why the bug is invisible in testing.
4. State the two requirements of a tie-breaker and give something that fails each.
5. Say what the counter gives you for free.

### The API drill

Write from memory what each of these does and its complexity:

`heapify` · `heappush` · `heappop` · `h[0]` · `heappushpop` · `heapreplace` · `nsmallest` · `nlargest` · `merge`

Then say which one has no function at all and how you do it instead.

### The list drill

1. Say what `heapq` operates on, and what that means.
2. Name four things you can do to that object that break it silently.
3. Write the one-line invariant check.
4. Run `append` on a heapified list and show that a later pop is wrong.

### The lazy-deletion drill

1. Say why a heap cannot update a priority.
2. Describe the textbook fix and say why nobody writes it in Python.
3. Write lazy deletion with a live-sequence dict.
4. State the memory cost in terms of pushes versus live items.

5. Say what the equivalent line looks like inside a Dijkstra loop.

#### The top-k drill

1. Give three approaches and their complexities.
2. Say what to use when  $k = 1$ .
3. Say roughly where the crossover to sorting is.
4. Compute the comparison counts at  $n = 1,000,000$  for  $k = 10$  and  $k = 500,000$ .
5. Say what `key=` buys you beyond convenience.

#### The break-it drill

Trigger each and record the exact output or error:

1. `heapq.heappush(h, x, reverse=True)`.
  2. Two equal-priority tuples whose payloads are dicts.
  3. `time.time()` as a tie-breaker, pushed twice in a tight loop.
  4. `h.append(0)` on a heapified list, then `heappop`.
  5. Building a 200,000-item heap by pushing, timed against `heapify`.
  6. `- "banana"`.
  7. `float('nan')` pushed into a heap of floats, then several pops.
  8. `nlargest(500_000, million_items)` timed against `sorted()`.
- 

#### The ladder drill

1. Name all five rungs in order.
2. For each, say what it costs and what it prevents.
3. Say which rung is the strongest that is still available during a partition.
4. Say what going down the ladder buys.

#### The definitions drill

1. Define linearizable precisely, including the real-time part.
2. Define eventual precisely, including three things it does **not** promise.
3. Define causal in one sentence, and give a related and an unrelated pair.
4. Say what sequential adds and what it lacks.

#### The session-guarantees drill

1. Name all four.
2. For each, give the user's own words for the complaint.

3. For each, give the fix and its cost.
4. Say which one is mostly a sharding problem and why.

#### **The choose-a-model drill**

For each, name the model and justify in one sentence:

seat booking · a chat thread · a like count · a user's own profile edit · leader election ·  
a recommendation list · stock at checkout · an analytics dashboard · a search index

Then say the sentence about systems versus operations.

#### **The anomaly drill**

For each user complaint, name the missing guarantee:

1. "I posted it and it's not there."
2. "It was there, and now it's gone."
3. "The reply is above the message."
4. "Two of us booked the same seat."
5. "I changed my name and then my photo, and the name reverted."
6. "The count says 41 and my friend sees 42."

Say which one of the six you would not fix, and why.

#### **The conflict drill**

1. Name four conflict-resolution strategies.
2. For each, say what it costs.
3. Say what last-write-wins does under clock skew.
4. Explain CRDTs in two sentences and say what they cannot express.
5. Say why "we'll use eventual consistency" is not yet a design.

#### **The numbers drill**

1. Give read latency for each rung.
2. Give DynamoDB's price ratio for strong versus eventual reads.
3. Give typical replication lag in four situations.
4. Explain why the read following a write is the worst possible sample.
5. Give the cost of the read-your-writes fix, as a share of reads.

#### **The scepticism drill**

1. Name the practice for verifying a consistency claim.

2. Say what it does, mechanically.
  3. Say what it has found.
  4. Say what you would ask for before believing a vendor's claim.
- 

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Python only has a min-heap. How do you get the maximum?* Negation with its limits, inverted `__lt__` as the general answer, the private functions named and rejected, then the tie-breaker problem raised unprompted with its two requirements, and what you must never do to the list.
  2. *What consistency does this feature need? Justify it.* The ladder rather than a binary, linearizable and eventual both defined precisely, causal named as the strongest AP model, the four session guarantees, per-operation choices with concrete features, and the conflict-resolution requirement.
  3. *A user posts a comment and cannot see it. Which guarantee is missing?* Read-your-writes, why it happens essentially always rather than rarely, three fixes with their costs, and the monotonic-reads sibling.
- 

### Before you move on

- ☐ I can give both max-heap workarounds and say where each applies.
- ☐ I know `heapq` only calls `<`, and what that means for a custom class.
- ☐ I push three-element tuples by habit.
- ☐ I can quote the `TypeError` and say why it is invisible in testing.
- ☐ I can state the tie-breaker's two requirements and something that fails each.
- ☐ I know the whole `heapq` API and which function does not exist.
- ☐ I know four ways to break a heap silently.
- ☐ I can write lazy deletion and say its memory cost.
- ☐ I can write Dijkstra the Python way without an indexed heap.
- ☐ I know where the top-k crossover to sorting is.
- ☐ I always use `heapify` when I already have the items.
- ☐ I can name all five rungs of the consistency ladder in order.
- ☐ I can define linearizable and eventual precisely, including what eventual omits.

- [ ] I can say why causal is the strongest model that is still AP.
- [ ] I can name the four session guarantees and the complaint each one answers.
- [ ] I choose a model per operation and can justify nine features.
- [ ] I can name four conflict-resolution strategies and their costs.
- [ ] I know what CRDTs cannot express, and why.
- [ ] I know what Jepsen is and what it has found.
- [ ] I answered all three questions above out loud.

# Top K problems

## DATA STRUCTURES AND ALGORITHMS

### Top K problems

You can choose between a heap, a sort and quickselect and defend the choice with  $n$  and  $k$ .

## SYSTEM DESIGN

### Eventual consistency in practice

You can design a feature that stays usable while the data is briefly stale.

## THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the  $k$  largest elements. Which approach, and why?*
- *The like count is wrong for two seconds. Is that acceptable?*

## 1 What this is, and why they ask it

“Top k” is a family: the k largest values, the k most frequent words, the k closest points, the k-th largest element. They all reduce to the same question — **you need a few extremes out of many items, and you do not need the rest in any order.**

Three sentences. There are exactly **three** approaches — sort everything, keep a heap of size  $k$ , or partition with **quickselect** — and each one wins in a different region of  $n$  and  $k$ , so the answer is never a single technique. The heap version has one counter-intuitive detail that is the whole problem: **to find the k largest, you keep a MIN-heap**, because the thing you constantly need is the *worst* of your current best, so you can throw it away. And the choice is also constrained by the *shape* of the input: if the data arrives as a stream and does not fit in memory, sorting and quickselect are both unavailable and the heap is the only option.

They ask “*which approach, and why?*” because the question is explicitly about the trade-off, not about the code. A candidate who writes a heap solution and stops has answered half; a candidate who says “*at  $k = 10$  out of a million, a heap does about three million comparisons against twenty million for a sort — but at  $k = 500,000$  the sort wins on constants*” has answered it.

## 2 The story

The mango crates came off the lorry at four in the morning and by six Chellappa had to have twenty picked out for the hotel order.

There were about a thousand mangoes in the load, and the hotel wanted the twenty best — big, unmarked, and at exactly the right ripeness. The rest went to the market at whatever they fetched.

His nephew, in his first week, tried to be systematic about it and started arranging the entire load in order of quality along the ground. He had about two hundred laid out by half past four and was already in trouble, because every new mango meant walking along the line to find where it belonged.

Chellappa did it differently and finished in about forty minutes.

He kept a shallow basket at his feet with twenty mangoes in it. That was it. He picked up each mango from the crate, looked at it for perhaps a second, and did one of two things.

If it was worse than the worst one in his basket, he threw it in the market pile without another thought. That was most of them — maybe nine hundred out of a thousand.

If it was better, he took the worst one out of the basket, threw *that* in the market pile, and put the new one in.

The thing his nephew could not work out was how he knew, instantly, which one in the basket was the worst. He was not searching through twenty mangoes each time.

Chellappa showed him. He kept the basket tilted, and the worst one always sat at the low corner. When he put a new one in, he gave the basket a small shake and the poor one worked its way down to the corner again. It took a moment. He never had to look at the other nineteen.

The nephew said: but at the end you still do not know which of the twenty is the best.

Chellappa said the hotel had not asked which was best. They had asked for twenty good ones. Knowing the order inside the basket would have cost him the whole morning and nobody had asked for it.

Then he said the thing that stuck. He said if they had wanted five hundred out of the thousand, he would not have done it this way at all — he would have gone through and split the load roughly into good and bad and taken the good half, because at that point carrying half the crate around in a basket is madness.

---

### 3 The idea in plain English

Chellappa's basket is a size- $k$  heap, the tilt is the heap property, and his last remark is the reason there are three approaches rather than one.

- The basket of twenty is a **fixed-size heap of size  $k$** .
- “The worst one sits at the low corner” is why it must be a **min-heap** when you want the largest.
- Throwing away anything worse than the worst is the **0(1) rejection** that makes it fast.



- The nephew laying out the whole load is **sorting everything** — correct, and far more work than asked for.
- “If they wanted five hundred, I would split the load instead” is **quickselect**.

The counter-intuitive part

To find the **k LARGEST**, keep a **MIN-heap** of size **k**.

**Say it out loud, because it feels backwards.** The reasoning: your basket holds the best **k** so far. The only question you ever ask is “*is this new item better than the worst thing I am keeping?*” — so the item you need instant access to is the **minimum** of your basket. A min-heap puts exactly that at the root.

```

heap = values[:k]
heapq.heapify(heap) # O(k)
for v in values[k:]:
 if v > heap[0]: # better than my worst? O(1) check
 heapq.heapreplace(heap, v) # ONE sift-down
return heap # the k largest, in no order

```

**The mirror rule:** to find the **k smallest**, keep a **max-heap** of size **k** — in Python, a min-heap of negated values.

**And the `heap[0]` comparison is where the speed comes from.** Most items fail it and are rejected in one comparison with no heap operation at all — Chellappa’s nine hundred mangoes.

The three approaches

1. **SORT EVERYTHING** `sorted(data)[-k:]`  
 $O(n \log n)$  time,  $O(n)$  space (or  $O(1)$  if in place)  
 → simplest; excellent constants; returns them in order for free
2. **HEAP OF SIZE k** `the basket`  
 $O(n \log k)$  time,  $O(k)$  space  
 → the only option for a **STREAM** or data too big for memory  
 → wins when **k** is small relative to **n**
3. **QUICKSELECT** `partition, recurse one side only`  
 $O(n)$  AVERAGE,  $O(n^2)$  worst case,  $O(1)$  extra space  
 → fastest when all the data is in memory and you can reorder it  
 → gives you the **k-th element** and an **UNORDERED** set of the top **k**

**Say all three and then choose.** That is the question.

## Which one wins, and where

|                               |                                                 |
|-------------------------------|-------------------------------------------------|
| k = 1                         | max(data) – $O(n)$ , no structure at all        |
| k very small ( $< n/100$ )    | heap: $O(n \log k)$ with a cheap rejection test |
| k moderate                    | quickselect if in memory: $O(n)$ average        |
| k close to n                  | SORT – $O(n \log n)$ with far better constants  |
| streaming / unbounded n       | HEAP – the only one that works                  |
| need them in order            | sort, or heap then sort the k results           |
| many queries on the same data | sort ONCE, then every query is $O(k)$           |

The crossover between heap and sort is around  $k \approx n/10$ , and the exact point depends on constants rather than asymptotics — which is worth saying, because it explains why the theoretically better  $O(n \log k)$  loses to  $O(n \log n)$  in practice at large k.

## Quickselect

The same partition step as [quicksort](#), but **you only recurse into one side**.

```
def select(lo, hi, k_index):
 p = partition(lo, hi) # everything left of p is smaller
 if p == k_index:
 return data[p]
 if p < k_index:
 return select(p + 1, hi, k_index) # ONE side only
 return select(lo, p - 1, k_index)
```

Why it is  $O(n)$  on average: the first partition costs  $n$ , the next costs  $n/2$ , then  $n/4$ , and  $n + n/2 + n/4 + \dots = 2n$ . The geometric series is the whole argument, and it is the same shape as the [heapify](#) sum.

Why it is  $O(n^2)$  in the worst case: if every pivot is the smallest or largest element, each partition removes one item. **Sorted input with a first-element pivot does exactly that**, and it is not a rare input.

## The fixes, in order of practicality:

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| random pivot      | $O(n)$ expected; adversary-proof in practice; ONE line           |
| median of three   | cheap, helps on partly-sorted data, still $O(n^2)$ adversarially |
| median of medians | $O(n)$ GUARANTEED worst case, and constants so bad it is         |
|                   | slower in practice – know the name, do not write it              |
| introsselect      | quickselect that switches to median-of-medians after             |
|                   | too many bad partitions – what real libraries do                 |

**Always randomise the pivot**, and say why: sorted input is the common case, not an adversarial one.

**And the side effect people forget: quickselect reorders the input array.** If the caller needs the original order, you must copy first, which costs the  $O(1)$  space advantage.

The variants, and what changes

**k most frequent** — count first, then top-k on the counts.

```
counts = Counter(words) # O(n)
return heapq.nlargest(k, counts, key=counts.get) # O(m log k), m distinct
```

**Note the two different sizes.** The heap is over the number of **distinct** items, not the number of items — which matters enormously when a million words have only five thousand distinct values.

**And bucket sort beats everything here when the counts are bounded:** a frequency can be at most  $n$ , so you can bucket by count and read down from the top for  $O(n)$  overall with no heap at all. **That is the answer to “can you do better than  $O(n \log k)$  ?” for this specific problem.**

**k closest points** — the same shape with a computed key.

```
heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)
```

**Do not take the square root.** It is a monotonic function, so it does not change the ordering, and it costs a transcendental operation per point. Saying that is a cheap, real optimisation.

**k-th largest specifically** — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

**Top k in a stream** — the heap, and only the heap. You cannot sort what you cannot hold.

The output-order question, which is worth asking

```
sort: returns them in order
size-k heap: returns them UNORDERED – the heap array is not sorted
quickselect: returns them UNORDERED
```

**If the answer must be sorted, add  $O(k \log k)$  to sort the  $k$  results** — which is negligible when  $k$  is small and is exactly the cost that makes sorting competitive when  $k$  is large.

**Ask whether the order matters.** It is a real requirement and it changes the comparison.

**4 The picture**

The basket, which is the algorithm.

1,000 mangoes, basket of 20

for each mango:

is it better than the WORST  
one in the basket?

no (~900)

yes (~100)

market pile  
(rejected in  
one comparison)

swap out the worst,  
shake the basket  
(heapreplace,  $O(\log k)$ )

← ONE comparison,  $O(1)$

THE BASKET IS A MIN-HEAP:  
you want the  $k$  LARGEST, so the thing you constantly need is  
the SMALLEST of what you are keeping – a min-heap puts it at `heap[0]`.

at the end: the basket holds the top 20, IN NO ORDER.  
The hotel did not ask for an order.

The three approaches, side by side:

| <u>SORT EVERYTHING</u>                                                                           | <u>HEAP OF SIZE <math>k</math></u>                                                                    | <u>QUICKSELECT</u>                                                                                                                                   |
|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>[ ][ ][ ][ ][ ][ ]<br/>↓ sort<br/>[ ][ ][ ][ ][ ][ ]<br/>take the last <math>k</math></div> | <div>basket<br/>stream [ <math>k=3</math> ]</div>                                                     | <div>[ ?  <math>p</math>  ? ]<br/>↓ partition<br/>[ &lt; <math>p</math>  <math>p</math>  &gt; <math>p</math> ]<br/>↓<br/>RECURSE ONE SIDE ONLY</div> |
| <div><math>O(n \log n)</math>, <math>O(n)</math><br/>in order, free<br/>works on any input</div> | <div><math>O(n \log k)</math>, <math>O(k)</math><br/>NOT in order<br/>THE ONLY streaming option</div> | <div><math>O(n)</math> avg, <math>O(1)</math> space<br/>NOT in order<br/>reorders the input</div>                                                    |

Where each one wins:

$n = 1,000,000$

| k       | sort           | size-k heap       | quickselect                        |
|---------|----------------|-------------------|------------------------------------|
| -----   | -----          | -----             | -----                              |
| 1       | 20,000,000 cmp | n/a – use max()   | n/a – use max()                    |
| 10      | 20,000,000     | ~3,300,000 ← ✓    | ~2,000,000 ← ✓                     |
| 1,000   | 20,000,000     | ~10,000,000       | ~2,000,000 ← ✓                     |
| 100,000 | 20,000,000 ← ✓ | ~17,000,000       | ~2,000,000 ← ✓                     |
| 500,000 | 20,000,000 ← ✓ | ~19,000,000       | ~2,000,000 ← ✓                     |
|         |                | (worse constants) | (but reorders, and $O(n^2)$ worst) |

the crossover between HEAP and SORT is around  $k \approx n/10$ ,  
and it is decided by CONSTANTS, not by the asymptotics.

Why quickselect is  $O(n)$  :

|                           |                                   |
|---------------------------|-----------------------------------|
| partition the whole array | n comparisons                     |
| recurse into ONE half     | $n/2$                             |
|                           | $n/4$                             |
|                           | $n/8$                             |
|                           | ...                               |
|                           | -----                             |
|                           | $2n \quad \rightarrow \quad O(n)$ |

compare with QUICKSORT, which recurses into BOTH halves:

|                                               |
|-----------------------------------------------|
| n                                             |
| $n/2 + n/2 = n$                               |
| $n/4 \times 4 = n$                            |
| ... $\log n$ levels $\rightarrow O(n \log n)$ |

ONE side instead of two turns  $\log n$  levels of full work into  
a geometric series that sums to 2.

The worst case, and why it is not exotic:

sorted input  $[1,2,3,\dots,n]$ , pivot = first element

partition  $\rightarrow$  pivot is the smallest, so ONE element is removed  
 $n + (n-1) + (n-2) + \dots = n^2/2 \rightarrow O(n^2)$

THE FIX: choose the pivot at RANDOM. One line.

$\rightarrow O(n)$  EXPECTED, and an adversary cannot construct the bad case

median-of-medians gives  $O(n)$  GUARANTEED and is slower in practice –  
know the name, do not write it.

## 5 The code, built step by step

### Step 1 — ask the three questions that decide the approach

“How big is  $k$  relative to  $n$ ? Does the data fit in memory, or is it a stream? And do you need the results in order?” **Those three answers choose the algorithm**, and asking them is the answer to the question.

### Step 2 — say the counter-intuitive rule as you write it

```
heap = values[:k]
heapq.heapify(heap)
```

**“For the  $k$  largest I keep a min-heap, because the thing I constantly need is the worst of what I am keeping, so I can throw it away.”** That sentence is what the problem is testing.

### Step 3 — the $O(1)$ rejection, which is where the speed is

```
if v > heap[0]:
```

“Most items fail this in one comparison and never touch the heap. On random data only about  $k \log(n/k)$  items ever get inserted, which is why the practical cost is far below the  $O(n \log k)$  bound.”

### Step 4 — `heapreplace`, not pop-then-push

```
heapq.heapreplace(heap, v)
```

“One sift-down instead of a sift-down and a sift-up. In a hot loop over a million items that is a real constant-factor win.”

### Step 5 — offer quickselect with its caveats

“If everything is in memory and I am allowed to reorder it, quickselect is  $O(n)$  average — better than both. Two caveats: it is  $O(n^2)$  if I pick pivots badly, so I randomise; and it **mutates the input**, so if the caller needs the original order I have to copy first.”

## The complete solution

```
import heapq
import random
from collections import Counter

1. The size-k heap – the basket

def k_largest_heap(values: list[int], k: int) → list[int]:
 """The k largest, via a MIN-heap of size k.

 THE COUNTER-INTUITIVE PART: for the k LARGEST you keep a MIN-heap,
 because the thing you constantly need is the WORST of what you are
 currently keeping – so you can throw it out. A min-heap puts exactly
 that at heap[0].

 The `v > heap[0]` test is where the speed lives: most items fail it in
 ONE comparison and never touch the heap at all.

 O(n log k) time, O(k) space. Returns them UNORDERED.
 THE ONLY option for a stream or for data larger than memory.
 """
 if k ≤ 0:
 return []
 if k ≥ len(values):
 return list(values)

 heap = values[:k]
 heapq.heapify(heap)
 for v in values[k:]:
 if v > heap[0]:
 heapq.heapreplace(heap, v)
 return heap

0(k)
0(1) rejection – most fail here
ONE sift-down, not pop+push
unordered

def k_smallest_heap(values: list[int], k: int) → list[int]:
 """The mirror: for the k SMALLEST, a MAX-heap of size k.
 In Python that is a min-heap of negated values."""
 if k ≤ 0:
 return []
 heap = [-v for v in values[:k]]
 heapq.heapify(heap)
 for v in values[k:]:
 if -v > heap[0]:
 heapq.heapreplace(heap, -v)
 return [-v for v in heap]

2. Sorting – simplest, and the right answer more often than people think

def k_largest_sort(values: list[int], k: int) → list[int]:
 """O(n log n), excellent constants, and they come back IN ORDER.

 Beats the heap once k is a large fraction of n, and beats everything if
 you will run many queries against the same data – sort once, then each
 query is O(k).
 """
 return sorted(values, reverse=True)[:k]
```

```

3. Quickselect – $O(n)$ average, when the data is in memory

def quickselect(values: list[int], k: int) → int:
 """The k-th LARGEST element (1-indexed), in $O(n)$ average time.

 Quicksort's partition, but recursing into ONE side only:
 $n + n/2 + n/4 + \dots = 2n \rightarrow O(n)$

 $O(n^2)$ worst case if the pivots are always extreme – and SORTED INPUT
 with a fixed pivot does exactly that, which is not a rare input. So the
 pivot is RANDOM: one line, and it makes the bad case unconstructible.

 NOTE: this MUTATES `values`. Copy first if the caller needs the original
 order – which costs the $O(1)$ -space advantage.
 """
 if not 1 ≤ k ≤ len(values):
 raise ValueError("k out of range")

 target = len(values) - k # the k-th largest is at this index
 lo, hi = 0, len(values) - 1

 while lo < hi:
 p = _partition(values, lo, hi)
 if p == target:
 return values[p]
 if p < target:
 lo = p + 1 # ONE side
 else:
 hi = p - 1
 return values[lo]

def _partition(values: list[int], lo: int, hi: int) → int:
 """Lomuto partition with a RANDOM pivot."""
 pivot_index = random.randint(lo, hi) # THE line that avoids $O(n^2)$
 values[pivot_index], values[hi] = values[hi], values[pivot_index]
 pivot = values[hi]

 boundary = lo
 for i in range(lo, hi):
 if values[i] < pivot:
 values[boundary], values[i] = values[i], values[boundary]
 boundary += 1
 values[boundary], values[hi] = values[hi], values[boundary]
 return boundary

def k_largest_quickselect(values: list[int], k: int) → list[int]:
 """The SET of the k largest, unordered, in $O(n)$ average.
 Partition until the boundary is exactly at n-k, then take the tail."""
 if k ≤ 0:
 return []
 if k ≥ len(values):
 return list(values)
 data = list(values) # copy: quickselect mutates
 quickselect(data, k)
 return data[len(data) - k:]

4. The variants

```



```

def top_k_frequent(items: list[str], k: int) → list[str]:
 """LeetCode 347. Count first, then top-k over the DISTINCT items.

 Note the two different sizes: the heap is over m distinct items, not n
 total. A million words with 5,000 distinct values makes the heap step
 almost free.

 O(n) to count + O(m log k) for the heap.
 """
 counts = Counter(items)
 return heapq.nlargest(k, counts, key=counts.get)

def top_k_frequent_buckets(items: list[str], k: int) → list[str]:
 """The O(n) answer for this SPECIFIC problem: a frequency can be at most
 n, so bucket by count and read down from the top. No heap at all.

 This is the answer to "can you do better than O(n log k)?" here – and it
 only works because the KEY IS BOUNDED, which is the general lesson.
 """
 counts = Counter(items)
 buckets: list[list[str]] = [[] for _ in range(len(items) + 1)]
 for item, count in counts.items():
 buckets[count].append(item)

 out: list[str] = []
 for count in range(len(buckets) - 1, 0, -1):
 for item in buckets[count]:
 out.append(item)
 if len(out) == k:
 return out
 return out

def k_closest_points(points: list[tuple[int, int]], k: int) → list[tuple[int, int]]:
 """LeetCode 973. The same shape with a computed key.

 DO NOT take the square root: it is monotonic, so it does not change the
 ordering, and it costs a transcendental operation per point.
 """
 return heapq.nsmallest(k, points, key=lambda p: p[0] ** 2 + p[1] ** 2)

class KthLargestStream:
 """LeetCode 703. A stream – the ONLY case where the heap is forced.

 You cannot sort what you cannot hold, and quickselect needs random
 access to the whole array.
 """

 def __init__(self, k: int, initial: list[int]) → None:
 self.k = k
 self.heap = list(initial)
 heapq.heapify(self.heap)
 while len(self.heap) > k:
 heapq.heappop(self.heap) # keep only the k largest

 def add(self, value: int) → int:
 if len(self.heap) < self.k:
 heapq.heappush(self.heap, value)
 elif value > self.heap[0]:
 heapq.heapreplace(self.heap, value)
 return self.heap[0] # the k-th largest, O(1)

```

```

def k_largest_across_sorted_lists(lists: list[list[int]], k: int) → list[int]:
 """When the inputs are already sorted, neither a full sort nor
 quickselect is needed – merge lazily and take k. Day 117."""
 return list(heapq.merge(*lists, reverse=True))[:k]

5. Measuring the choice

def compare_approaches(n: int, k: int) → None:
 """Run this rather than trusting the table."""
 import time
 data = [random.randint(0, 10 ** 9) for _ in range(n)]

 t = time.perf_counter(); a = k_largest_sort(data, k)
 sort_time = time.perf_counter() - t

 t = time.perf_counter(); b = k_largest_heap(data, k)
 heap_time = time.perf_counter() - t

 t = time.perf_counter(); c = k_largest_quickselect(data, k)
 qs_time = time.perf_counter() - t

 assert sorted(a) == sorted(b) == sorted(c)
 print(f"n={n:>9} k={k:>7} sort {sort_time:.4f}s "
 f"heap {heap_time:.4f}s quickselect {qs_time:.4f}s")

if __name__ == "__main__":
 random.seed(0)
 values = [5, 1, 9, 3, 7, 2, 8, 6, 4]

 print(sorted(k_largest_heap(values, 3), reverse=True)) # [9, 8, 7]
 print(sorted(k_smallest_heap(values, 3))) # [1, 2, 3]
 print(k_largest_sort(values, 3)) # [9, 8, 7]
 print(quickselect(values[:], 3)) # 7
 print(sorted(k_largest_quickselect(values, 3), reverse=True)) # [9, 8, 7]

 # the heap result is NOT sorted – the hotel did not ask for an order
 print(k_largest_heap(values, 4)) # e.g. [6, 7, 9, 8]

 # variants
 words = "the quick brown fox the lazy dog the fox".split()
 print(top_k_frequent(words, 2)) # ['the', 'fox']
 print(top_k_frequent_buckets(words, 2)) # ['the', 'fox']
 print(k_closest_points([(1, 3), (-2, 2), (5, 8), (0, 1)], 2)) # [(0,1), (-2,2)]

 stream = KthLargestStream(3, [4, 5, 8, 2])
 print([stream.add(v) for v in (3, 5, 10, 9, 4)]) # [4, 5, 5, 8, 8]

 # edge cases
 print(k_largest_heap([], 3), k_largest_heap([1, 2], 5)) # [] [1, 2]
 print(k_largest_heap([1, 1, 1], 2)) # [1, 1]

 # WHERE EACH ONE WINS – run it, do not trust the table
 compare_approaches(1_000_000, 10)
 compare_approaches(1_000_000, 100_000)
 compare_approaches(1_000_000, 500_000)

```

## 6 What it costs

### The three approaches

| approach    | time                             | space  | in order? | streaming?             |
|-------------|----------------------------------|--------|-----------|------------------------|
| -----       | -----                            | -----  | -----     | -----                  |
| sort        | $O(n \log n)$                    | $O(n)$ | YES       | no                     |
| size-k heap | $O(n \log k)$                    | $O(k)$ | no        | YES                    |
| quickselect | $O(n)$ average<br>$O(n^2)$ worst | $O(1)$ | no        | no<br>(and it MUTATES) |

### Comparison counts at $n = 1,000,000$

| k       | sort       | heap        | quickselect |
|---------|------------|-------------|-------------|
| -----   | -----      | -----       | -----       |
| 10      | 20,000,000 | ~3,300,000  | ~2,000,000  |
| 1,000   | 20,000,000 | ~10,000,000 | ~2,000,000  |
| 100,000 | 20,000,000 | ~17,000,000 | ~2,000,000  |
| 500,000 | 20,000,000 | ~19,000,000 | ~2,000,000  |

**Asymptotically quickselect always wins. In practice it often does not**, because `sorted()` in CPython is a highly tuned C implementation and quickselect written in Python is interpreted. **That is worth saying:** the constants dominate at these sizes, which is why the measured comparison in the code matters more than the table.

### The rejection rate, which is the heap's real advantage

|                                                                                                                      |                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| the number of items that ever ENTER a size-k heap, on random data:<br>expected insertions $\approx k \cdot \ln(n/k)$ |                                                                                                 |
| $n = 1,000,000, k = 10$                                                                                              | $\rightarrow$ ~115 insertions out of 1,000,000 items<br>999,885 rejected in ONE comparison each |

**So the practical cost is  $n$  cheap comparisons plus about a hundred heap operations** — far below the  $O(n \log k)$  bound, which assumes every item is inserted. **That is why the heap is so much better than its bound suggests on random data**, and it is a genuinely good thing to know.

**And the worst case:** ascending input, where every item beats the current worst.

|                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------|
| sorted ascending, $n = 1,000,000, k = 10$ : 1,000,000 insertions<br>$\rightarrow$ the full $O(n \log k)$ , and no rejections at all |
|-------------------------------------------------------------------------------------------------------------------------------------|

## Space, which often decides it

```
n = 1,000,000,000 items (does not fit in memory)

sort impossible without external sort
quickselect impossible – needs random access to everything
size-k heap $O(k)$ = a few hundred bytes. THE ONLY OPTION.
```

That is the case where the choice is not a trade-off at all.

## The frequency variant

```
n = 1,000,000 words, m = 5,000 distinct

counting $O(n)$ = 1,000,000
heap over DISTINCT $O(m \log k)$ = $5,000 \times \log(10) \approx 17,000$
→ the heap step is 0.2% of the work; the COUNT dominates

bucket sort $O(n)$ – no heap at all, and it is exact
```

Noticing that **m** and **n** are different is worth stating, and the bucket-sort answer for bounded keys is the strongest finish available on that problem.

## Quickselect's recursion

```
expected depth $O(\log n)$
worst-case depth $O(n)$ → RecursionError if written recursively

→ write it ITERATIVELY, as the code above does, with a while loop
 adjusting lo and hi. One less thing to worry about.
```

## 7 The traps

### Trap 1 – using a max-heap for the k largest

```
heap = [-v for v in values] # a max-heap of EVERYTHING
heapq.heapify(heap)
return [-heapq.heappop(heap) for _ in range(k)]
```

Correct, and it is  $O(n + k \log n)$  with  $O(n)$  space — it holds the entire input. The whole point of the size-k heap is that it holds **k** items, and to do that it must be a **min**-heap so the worst is at the root.

**On a stream this version is not merely slower, it is impossible.**

#### Trap 2 — pop-then-push instead of `heapreplace`

```
heapq.heappop(heap)
heapq.heappush(heap, v)
```

Two passes over the heap instead of one sift-down. Correct, and a measurable constant-factor loss in a loop over a million items.

#### Trap 3 — forgetting that the results are unordered

```
return k_largest_heap(values, 3) # e.g. [6, 7, 9, 8]
```

The heap's array is not sorted, and neither is quickselect's tail. **If the answer must be in order, add  $O(k \log k)$**  — and say so, because a test comparing against a sorted expectation will fail.

#### Trap 4 — a fixed pivot in quickselect

```
pivot = values[hi] # always the last element
```

On sorted or reverse-sorted input, every partition removes one element:  $O(n^2)$ . **Sorted input is the common case, not adversarial.** One line — `random.randint(lo, hi)` — fixes it.

#### Trap 5 — forgetting that quickselect mutates

```
kth = quickselect(user_data, 5) # user_data is now reordered
```

The caller's array is scrambled. If the original order matters, copy first — which costs the  $O(1)$  space advantage that made quickselect attractive.

#### Trap 6 — `k > len(values)` and other boundaries

```
heap = values[:k] # fine
for v in values[k:]: # empty, fine
 return heap # returns everything – correct
```

Handled by luck here, but `quickselect` with `k > n` must raise, and `k ≤ 0` must return empty. **Check the boundaries explicitly**; every one of these problems has a `k = 0`, `k = n` and empty-input test case.

### Trap 7 — the square root in k-closest-points

```
key=lambda p: math.sqrt(p[0]**2 + p[1]**2)
```

Correct, and it computes a square root a million times for no reason: the ordering is identical without it because `sqrt` is monotonic on non-negative values. **A free optimisation, and interviewers notice.**

### Trap 8 — heap size over the wrong `n` in the frequency problem

```
heapq.nlargest(k, items) # over ALL items, not the counts
```

You want the top `k` **distinct** items by frequency, so the heap is over the `m` distinct keys. Running it over all `n` items is both wrong and much slower.

## 8 In the interview

### How it gets asked

- The open one: “Find the  $k$  largest elements. Which approach, and why?”
- The specific: “Find the  $k$ -th largest element.” LeetCode 215.
- The variant: “Top  $k$  frequent elements.” LeetCode 347 — and then “can you do better than  $O(n \log k)$ ?”
- The constraint: “The data is a stream and does not fit in memory.”
- The efficiency probe: “Can you do it in  $O(n)$ ?”

### What to say out loud, in the first ninety seconds

1. **Ask the three questions that decide it.** “How big is `k` relative to `n`? Does everything fit in memory, or is it a stream? And do you need the results in order?”
2. **Name all three approaches before choosing.** “Sort everything —  $O(n \log n)$ , simplest, and they come back ordered. A heap of size `k` —  $O(n \log k)$ ,  $O(k)$  space, and the only option for a stream. Quickselect —  $O(n)$  average, but it mutates the input.”
3. **Give the counter-intuitive rule explicitly.** “For the  $k$  largest I keep a min-heap of size  $k$ , because the thing I constantly need is the **worst** of what I am keeping, so I can throw it away — and a min-heap puts exactly that at the root.”
4. **Point at where the speed is.** “The comparison against `heap[0]` rejects most items in one comparison without touching the heap. On random data only about  $k \log(n/k)$  items ever get inserted — for a million items and  $k = 10$ , about a hun-

dred insertions out of a million.”

5. **Give a concrete comparison.** “At  $n$  a million and  $k$  ten, the heap does roughly three million comparisons against twenty million for a sort. At  $k = 500,000$ , the sort wins on constants.”
6. **Say the space consequence.** “And if the data does not fit in memory, it is not a trade-off — sorting and quickselect are both unavailable and the heap is the only option.”

#### The follow-ups

“**Why a min-heap for the largest? That feels backwards.**” “It does, and the reason it is right is that the heap is not holding the answer to the question you are asking — it is holding the **candidates**, and the only question you ever ask of it is *‘is this new item better than the worst thing I am currently keeping?’* So the element I need instant access to is the **minimum** of my current best  $k$ , and a min-heap puts exactly that at index zero. When a new item beats it, I evict the minimum and insert the new one. At the end, the heap contains the  $k$  largest — unordered, because nobody asked for an order. The mirror rule is the same logic: for the  $k$  **smallest**, keep a **max**-heap, which in Python means negating.”

“**Can you do it in  $O(n)$  ?**” “Yes, with **quickselect**, if the data is in memory and I am allowed to reorder it. It is quicksort’s partition step, but I only recurse into the side containing the  $k$ -th position — so the cost is  $n + n/2 + n/4 + \dots$ , a geometric series summing to  $2n$ . Two caveats I would state rather than let you find. It is  $O(n^2)$  in the worst case, when every pivot is extreme — and **sorted input with a fixed pivot does exactly that**, which is a common input, not an adversarial one, so the pivot must be **random**. And it **mutates the array**, so if the caller needs the original order I have to copy first, which costs the  $O(1)$ -space advantage. If I needed a guaranteed  $O(n)$  I would name **median-of-medians**, and then say that its constants are bad enough that it is slower in practice — which is why real libraries use **introselect**: quickselect that falls back to it after too many bad partitions.”

“**Top  $k$  frequent — can you do better than  $O(n \log k)$  ?**” “Yes, for this specific problem, because the key is **bounded**. A frequency can be at most  $n$ , so instead of a heap I can bucket by count: an array of  $n + 1$  lists, put each distinct item in the bucket for its count, then read down from the highest bucket until I have  $k$ . That is  $O(n)$  overall with no heap at all, and it is exact rather than approximate. The general lesson is the one worth stating: **when the sort key is bounded and small, bucketing beats comparison-based approaches**, which is the same reason counting sort exists. I

would also point out something about the heap version that people miss — the heap is over the number of **distinct** items, not the total. A million words with five thousand distinct values makes the heap step about 0.2% of the work; the counting dominates.”

“**The data is a stream and does not fit in memory.**” “Then it stops being a trade-off. Sorting needs to hold everything, and quickselect needs random access to the whole array, so **both are unavailable** — the size- $k$  heap is the only option. I keep  $k$  items and  $O(1)$  state, process each element once, and the answer is available at any moment. That is also why the ‘ $k$ -th largest in a stream’ problem exists as a separate question: it is the one shape where the choice is forced. If  $k$  itself were too large to hold, I would be into approximate territory — count-min sketch for frequencies, or reservoir sampling — and I would say so rather than pretend the exact answer is available.”

“**Which would you actually write?**” “For an interview, the **size- $k$  heap**, and I would say why: it is six lines, it handles the streaming case for free, its space is  $O(k)$  rather than  $O(n)$ , and its practical cost is far below its bound because most items are rejected in one comparison. For production with everything in memory and  $k$  a large fraction of  $n$ , I would just **sort** —  $O(n \log n)$  with excellent library constants beats a hand-written  $O(n \log k)$  in Python, and it returns the results ordered. And if I were running many top- $k$  queries against the same unchanging data, I would sort **once** and then every query is  $O(k)$ . Quickselect I would offer as the  $O(n)$  answer and mention that in practice, in Python, `sorted()` in C often beats it — the asymptotics say one thing and the constants say another, and it is worth measuring rather than assuming.”

“**What are the boundary cases?**” “Four, and each problem in the family has all of them.  $k = 0$  — return empty rather than raising.  $k \geq n$  — return everything, and note that the heap version handles it naturally while quickselect must check. **An empty input.** And **duplicates** — ‘the three largest’ of `[5, 5, 5]` is `[5, 5, 5]`, not one element, which matters because a solution built on a set would get it wrong. I would also ask whether the result needs to be **in order**, because the heap and quickselect both return it unordered, and adding  $O(k \log k)$  to sort the answer is the difference between passing and failing a test that compares against a sorted expectation.”

#### A model answer

Asked: *find the  $k$  largest elements — which approach, and why?*



“Three questions first, because they decide the answer: **how big is  $k$  relative to  $n$** , **does everything fit in memory or is this a stream**, and **do the results need to be in order?**

There are three approaches and each wins somewhere.

**Sort everything** and take the last  $k$ .  $O(n \log n)$ , the simplest possible code, excellent library constants, and the results come back ordered for free.

**A heap of size  $k$** .  $O(n \log k)$  time and — the important part —  $O(k)$  space. This is the one I would write by default, and it has a detail that sounds backwards: **for the  $k$  largest, you keep a MIN-heap**. The heap holds my current best  $k$ , and the only question I ever ask is ‘is this new item better than the *worst* one I am keeping?’ — so the element I need instantly is the minimum of my candidates, and a min-heap puts that at index zero. If the new item beats it, I evict and insert; otherwise I discard it after one comparison.

That rejection is where the real speed is. On random data only about  $k \cdot \ln(n/k)$  items ever enter the heap — for a million items with  $k$  of ten, that is roughly a **hundred** insertions out of a million, and the other 999,900 cost a single comparison each. So the practical cost is far below the  $O(n \log k)$  bound, which assumes every item is inserted.

**Quickselect** is the third. Quicksort’s partition, recursing into only the side that contains the  $k$ -th position, so the cost is  $n + n/2 + n/4 + \dots$ , which sums to  $2n$  —  $O(n)$  average. Two caveats I would raise myself: it is  $O(n^2)$  when pivots are extreme, and **sorted input with a fixed pivot does exactly that**, so the pivot must be random; and it **reorders the caller’s array**, so preserving the original costs a copy and with it the  $O(1)$ -space advantage.

Choosing between them: at a million items and  $k$  of ten, the heap is about three million comparisons against twenty million for a sort — a clear win. At  $k$  of five hundred thousand, the sort wins on constants, and the crossover is around  $k \approx n/10$ . Quickselect is asymptotically best and in Python is often beaten by `sorted()` in C, so I would measure rather than assume.

And there is one case where it is not a trade-off at all: if the data is a **stream** or larger than memory, sorting and quickselect are both impossible, and the size- $k$  heap is the only option.

One thing I would confirm before writing: the heap and quickselect both return the results **unordered**. If you need them sorted, that is an extra  $O(k \log k)$ .”

## 9 Recall card

- Three approaches, and the answer is which one and why: sort ( $O(n \log n)$ , ordered, simplest) · a size- $k$  heap ( $O(n \log k)$ ,  $O(k)$  space, the ONLY streaming option) · quickselect ( $O(n)$  average,  $O(1)$  space, but it mutates the input). Ask: how big is  $k$  relative to  $n$  · in memory or a stream · do you need them in order?
- For the  $k$  LARGEST, keep a MIN-heap of size  $k$  — the heap holds candidates, and the only question you ask is “better than the worst I am keeping?”, so you need the **minimum** at the root. Mirror rule:  $k$  smallest  $\rightarrow$  a max-heap. Use `heapreplace`, not pop-then-push.
- The `v > heap[0]` test is where the speed is. On random data only  $\sim k \cdot \ln(n/k)$  items ever enter the heap —  $\sim 100$  insertions out of 1,000,000 at  $k = 10$  — so the practical cost is far below the bound. Worst case is **ascending input**, where every item is inserted.
- Quickselect is  $O(n)$  because it recurses into ONE side:  $n + n/2 + n/4 + \dots = 2n$ . It is  $O(n^2)$  when pivots are extreme, and **sorted input with a fixed pivot does exactly that** — so randomise the pivot, always. Name **median-of-medians** (guaranteed  $O(n)$ , bad constants) and **introselect** (what real libraries do).
- Crossover is around  $k \approx n/10$ , decided by constants.  $k = 1 \rightarrow \text{max}()$ . Both the heap and quickselect return results **UNORDERED** — add  $O(k \log k)$  if order matters. For **top- $k$  frequent**, the heap is over the **distinct** items, and **bucket sort by count** is  $O(n)$  because the key is bounded — that is the “can you do better?” answer.

## 1 What this is, and why they ask it

Yesterday was the ladder of consistency models. Today is the practical half: **how do you build a product on top of the weak end of it, so that users do not notice — or, when they do, do not mind?**

Three sentences. The answer is not a technique but a discipline: **decide, per piece of data, what a user would see and how bad it is**, and then choose from a small set of patterns — show the state honestly, predict it optimistically, or route around the staleness. The systems that do this well take an idea from outside software entirely: **banks, couriers and ticket offices have always been eventually consistent, and they cope by making the pending state visible** rather than pretending it does not exist. And the convergence machinery underneath — read repair, anti-entropy, hinted handoff — is worth knowing because “eventually” has a shape and you should be able to say what it is.

They ask “*is two seconds acceptable?*” because it is a product question wearing an engineering costume, and the right answer starts by asking **what happens in those two seconds**. A candidate who says “yes, eventual consistency is fine” has not asked; a candidate who says “for a like count yes, and here is how I would make the user’s *own* like feel instant anyway” has designed something.

## 2 The story

Mariappan had banked at the same branch for thirty-one years and he still went in on Fridays, and the thing he had never once complained about was the cheque.

He would hand over a cheque for, say, eleven thousand rupees. The clerk stamped it, entered it, and gave him a slip. And the slip said the amount, and it said — printed, in the same size as everything else — that it would be available on Tuesday.

His account, if he checked it that evening, showed two numbers. The balance, which had gone up by eleven thousand. And below it, in the same statement, an amount marked as not yet cleared.

His nephew, who had started using a phone application for everything, found this ridiculous. He said the money is either there or it is not.

Mariappan said that was exactly the point, and it was not there yet, and the bank was telling him so. The cheque had to go to the other bank, and the other bank had to agree, and that took until Tuesday. What would his nephew prefer — that they hide it, and let him spend money that might come back?

The nephew said they could just refuse to show it at all until Tuesday.

Mariappan said they had tried that, before his time, and people came in on Saturday convinced their cheque had been lost.

There were two other things about the arrangement that he pointed out, and both of them had taken the bank a long time to arrive at.

The first was that the branch let him **spend against it in small amounts** — up to two thousand rupees — even before Tuesday. Not the whole eleven. Because for two thousand they were confident enough, and if the cheque did bounce they could take it back out of the account without much trouble. Above that they made him wait.

The second was what happened when a cheque did bounce, which had happened to him twice in thirty-one years. The bank did not pretend it had never gone in. They **reversed it** — a separate line on the statement, with the date, saying the amount had been taken back out and why. Both entries stayed.

He said the whole system worked because nobody ever claimed to know something they did not know. The slip said Tuesday. The statement said uncleared. The reversal said reversed.

---

### 3 The idea in plain English

The bank is an eventually consistent system that has been running for two centuries, and every one of its conventions is a pattern you should copy.

- The cheque taking until Tuesday is **replication lag**, made explicit.
- “Available on Tuesday” on the slip is **showing the pending state** rather than hiding it.
- The uncleared line on the statement is **surfacing the uncertainty in the data model**.
- Spending up to two thousand is **bounded optimism** — act early where the downside is small.
- The reversal line is a **compensating action**, not a deletion.

- “Nobody ever claimed to know something they did not know” is the whole design principle.

The first question is always the same

**What does a user see during the window, and how bad is it?**

Not “is eventual consistency acceptable” in the abstract. The answer depends entirely on **which datum** and **which user**.

|                                                     |                     |
|-----------------------------------------------------|---------------------|
| a like count off by one for 2 s, seen by a stranger | invisible           |
| a like count that does not move when YOU tap it     | a BUG, and reported |
| a follower count off by three for 10 s              | invisible           |
| a bank balance off by ₹11,000 for 2 s               | terrifying          |
| a seat marked free that was sold 2 s ago            | a double booking    |

**The same two seconds is unnoticeable, annoying, or unacceptable depending on the datum.** So the question is never answered at the system level.

The rule that carries most of it

**Cache — and replicate — the DISPLAY. Never the DECISION.**

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| SHOWING "12 left in stock"           | stale is fine; it is information    |
| DECIDING whether to accept the order | must be current; it is a commitment |

**Every one of the patterns below is an application of that line.** It is the same rule as [caching](#), and it generalises to every weakly consistent system.

The four patterns

**1 — Optimistic UI: predict the result and show it immediately.**

The user taps *like*. You increment the number on their screen **before the server replies**, because you are almost certain what the answer will be.

```
tap → number changes instantly, locally
 → request sent in the background
 → success: nothing further happens, the user never knew
 → failure: revert, and say why
```

**This makes read-your-writes a client-side problem rather than a server-side one, which is enormously cheaper: no leader routing, no version tokens, no extra load. It is the single highest-value pattern here.**

Its limit is the failure case: **you must be able to revert visibly and honestly.** Optimistically showing a payment as complete is not acceptable, because there is no good way to un-show it.

## **2 — Show the pending state honestly.**

Mariappan's slip. When you cannot know yet, **say so in the interface** rather than showing a value that might change.

|                        |                                 |
|------------------------|---------------------------------|
| "Processing"           | a transfer that has not settled |
| "Uploading"            | a photo not yet replicated      |
| "Pending"              | an order not yet confirmed      |
| "Last updated 30s ago" | a dashboard                     |

**Users tolerate uncertainty they can see and are angry about uncertainty they discover.** That is the whole lesson from the bank, and it is a product decision as much as an engineering one.

## **3 — Route around the staleness for the affected user.**

From [day 105](#): send a user's reads to the leader for a window after they write, or only for the objects they touched, or carry a version token.

**Use this when the optimistic prediction is not safe** — when the server may legitimately reject or transform what the user did.

## **4 — Compensate rather than prevent.**

The bank's reversal. Instead of guaranteeing that a bad outcome cannot happen, **allow it and have a defined correction.**

|                                |                                      |
|--------------------------------|--------------------------------------|
| an airline oversells a flight  | → compensation at the gate           |
| two people order the last item | → one is refunded and apologised to  |
| a payment is double-charged    | → automatic reversal within 24 hours |

**This is how nearly all commerce works**, and it is often far cheaper than the coordination required to prevent the case. **The rule for choosing: compensate when the correction is cheaper than the coordination.** Overselling a hotel room is; over-selling a concert seat that costs you a lawsuit is not.

**Bounded optimism, which is the bank's cleverest move**

Mariappan spending two thousand of an eleven-thousand cheque. **You do not have to choose between fully trusting the stale value and fully refusing.**

```
the risk is small act now, reconcile later
the risk is large wait for confirmation

examples:
 a small purchase on a card with a stale balance → approve offline
 a large one → require authorisation
 a coupon redeemed twice in a race → allow it, absorb the loss
 the last ticket to a sold-out show → serialise it
```

**Card networks do exactly this:** terminals have an offline floor limit, below which they approve without contacting the bank at all. The stale answer is accepted because the exposure is bounded.

**How convergence actually happens**

“Eventually” is not magic, and there are three mechanisms. **Naming them is what turns a vague answer into a specific one.**

**Read repair.** When a read touches several replicas and they disagree, the coordinator returns the newest value **and writes it back** to the stale ones. Free convergence for anything that gets read.

```
→ hot keys converge in milliseconds
→ cold keys are never repaired this way
```

**Anti-entropy.** A background process where replicas compare and exchange what they lack. To avoid shipping everything, they compare **Merkle trees** — a hash tree over the key ranges, so two replicas can find exactly which ranges differ in  $O(\log n)$  exchanges instead of comparing every key.

```
→ runs every few minutes to hours
→ this is what converges the COLD keys
```

**Hinted handoff.** If a replica is down, another node accepts the write **on its behalf** and holds a hint; when the node returns, the hint is replayed.

```
→ keeps writes available during a short outage
→ and hints are usually kept for a bounded time only, after which
 anti-entropy has to do the work
```

**So the honest answer to “how long is eventually?”:**

|                                                                           |                                                                            |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------|
| a key read often                                                          | milliseconds (read repair)                                                 |
| a key rarely read                                                         | minutes to hours (anti-entropy interval)                                   |
| a replica that was down                                                   | until hinted handoff replays, or anti-entropy                              |
| a key never read, on a replica that missed the write and the hint expired | until the next anti-entropy pass – which is why anti-entropy exists at all |

## What to make eventual, and what not to

|                            |                                              |
|----------------------------|----------------------------------------------|
| <b>SAFE TO BE EVENTUAL</b> | <b>MUST BE STRONG</b>                        |
| like/view/follower counts  | account balances at the moment of a transfer |
| recommendations            | stock at the point of purchase               |
| search indexes             | seat and ticket allocation                   |
| analytics dashboards       | unique constraints (usernames, emails)       |
| "who is online"            | permission checks                            |
| feed ordering              | anything a user cannot undo                  |
| comment counts             | anything with legal or financial consequence |

THE TEST: if this is wrong for two seconds, can the mistake be corrected afterwards without anyone being harmed?

**Counters deserve a special note**, because they are the canonical eventual datum and they are almost always implemented wrong the first time: a read-modify-write of a shared counter is a **lost update** under concurrency. The right answers are an **atomic increment** in the store, or a **per-replica counter summed on read** — which is a CRDT and converges without coordination.

## 4 The picture

The bank statement, which is the design pattern.

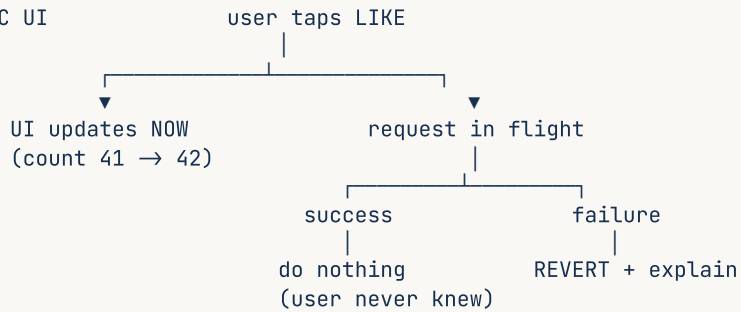
|                              |           |                                        |
|------------------------------|-----------|----------------------------------------|
| Balance                      | ₹ 47,300  | ← the optimistic number                |
| Of which uncleared           | ₹ 11,000  | ← the HONEST caveat                    |
| Available now                | ₹ 36,300  | ← what you can actually use            |
| 12 Sep Cheque 447812 +11,000 | UNCLEARED | ← the pending state, VISIBLE           |
| 15 Sep Cheque 447812 -11,000 | RETURNED  | ← a COMPENSATING entry, not a deletion |

THREE numbers instead of one, because three different questions have three different answers – and the system never claims to know something it does not know.

The four patterns, and when each applies:



## 1. OPTIMISTIC UI



→ makes read-your-writes a CLIENT problem. Cheapest fix there is.  
→ limit: you must be able to revert HONESTLY. Never for payments.

## 2. SHOW THE PENDING STATE "Processing..." "Uploading..." "Updated 30s ago"

→ users tolerate uncertainty they can SEE, and resent uncertainty they DISCOVER

## 3. ROUTE AROUND IT this user's reads → the leader, for 30 s

→ when optimism is not safe because the server may reject or transform

## 4. COMPENSATE allow the bad outcome, define the correction

→ airlines oversell; the correction is cheaper than the coordination

Bounded optimism — the middle position:



CARD NETWORKS DO EXACTLY THIS: an offline floor limit, below which the terminal approves without contacting the bank at all.

→ you do NOT have to choose between fully trusting stale data and fully refusing.

How convergence happens, and how long it takes:

WRITE arrives at 2 of 3 replicas

— replica C missed it

#### READ REPAIR

a read touches A, B, C, sees the disagreement, returns the newest, and WRITES IT BACK to C

→ milliseconds, for HOT keys

→ never happens for cold keys

#### ANTI-ENTROPY

a background pass compares MERKLE TREES to find which

RANGES differ in  $O(\log n)$

exchanges, then syncs them

→ minutes to hours

→ this is what fixes COLD keys

#### HINTED HANDOFF

C was DOWN, so B accepted the write on its behalf and kept a hint;

when C returns, B replays it

→ keeps writes available during a short outage

→ hints expire, after which anti-entropy has to do the work

SO "EVENTUALLY" IS:

milliseconds for a frequently read key

minutes-hours for a rarely read one

never for a key nobody reads on a replica that missed the write and whose hint expired – until the next anti-entropy pass

The decision, per datum:

"If this value is wrong for two seconds, what happens?"

nobody can tell (like counts, recommendations) → EVENTUAL. Do nothing.

the USER who changed it can tell (their own post, their own profile) → OPTIMISTIC UI, or route their reads to the leader for 30 s

two users can get the same thing → LINEARIZABLE. No choice. (seats, stock, usernames)

money or law is involved → LINEARIZABLE, or COMPENSATE with a defined, auditable correction

## 5 How it actually works

Designing a counter that is eventually consistent and correct

The canonical case, and the naive version is wrong:

```
count = db.get(key) # read
db.set(key, count + 1) # modify-write → LOST UPDATE
```

**Three correct options**, in increasing order of scale:

1. **ATOMIC INCREMENT** in the store      Redis INCR, DynamoDB ADD  
→ correct, and the counter is a single hot key
2. **PER-REPLICA COUNTERS**, summed on read (a G-counter CRDT)  
→ each node counts its own increments; the value is the sum  
→ converges with no coordination at all
3. **SHARDED** counters  
→ increment one of N sub-counters at random; sum on read  
→ spreads the write load; the read costs N lookups

**Option 2 is the one worth naming**, because it explains why counters are the textbook CRDT: addition is commutative and associative, so order does not matter and conflicts cannot occur.

**Making a write idempotent so retries are safe**

From [day 113](#): at-least-once delivery plus idempotent processing gives an exactly-once effect.

```
the client generates a key per logical operation
the server records the key with the result
a repeat returns the stored result instead of acting again
```

**In an eventually consistent store the idempotency check is itself eventually consistent**, which is a real subtlety: two replicas may not yet know about each other's record of the same key. **The usual answer is to make the idempotency store strongly consistent even when the data store is not** — it is small, it is written once, and the cost is bounded.

**Handling the failure of an optimistic update**

1. apply locally and show it
2. send the request
3. on failure:  
    revert the local change  
    TELL the user what happened, specifically  
    offer the action again
4. never leave the two states diverged silently

**The failure path is the whole design.** An optimistic UI without a considered revert is worse than no optimism, because the user believes something that is not true and finds out later.

Measuring convergence, so you can talk about it

```
write a sentinel value on the leader with a timestamp
read it from every replica on a schedule
record the delay until each one has it
alert on the p99 rather than the mean
```

“How stale is our data?” should be a graph, not an opinion — and it is the same heartbeat technique as measuring [replication lag](#).

What real systems do

- **DynamoDB and Cassandra** implement all three convergence mechanisms — read repair, anti-entropy with Merkle trees, and hinted handoff — and the Dynamo paper is where the vocabulary comes from.
  - **Amazon’s shopping cart** is the canonical worked example: divergent carts are **unioned**, so a deleted item can reappear. **Deliberately** — a resurrected item annoys; a lost item costs a sale.
  - **Twitter’s like and retweet counts** are eventually consistent and shown optimistically to the user who tapped, which is why *your* like always feels instant even though the global count lags.
  - **Airlines oversell** systematically, because compensation at the gate is far cheaper than the coordination required to guarantee it never happens.
  - **Card terminals** approve offline below a floor limit, which is bounded optimism in a system that predates computers.
  - **Google Docs and Figma** use CRDTs or operational transformation, because the requirement is that everyone converges and nobody’s keystroke is lost — a merge problem rather than an ordering one.
-

## 6 The numbers

### How long “eventually” actually is

| mechanism          | typical convergence                            |
|--------------------|------------------------------------------------|
| -----              | -----                                          |
| read repair        | milliseconds – but ONLY for keys that get read |
| hinted handoff     | seconds after the node returns                 |
| anti-entropy       | minutes to hours (the scheduled interval)      |
| no mechanism fires | indefinite                                     |

a hot key (read 1,000×/s) converges in milliseconds  
a cold key (read once a week) converges at the next anti-entropy pass

**So the distribution is bimodal**, which is worth saying: popular data is almost always fresh, and the long tail is exactly the data nobody is looking at.

### The window in which a user can notice

|                             |                    |
|-----------------------------|--------------------|
| replication lag, healthy    | 1-50 ms            |
| replication lag, heavy load | 100 ms - seconds   |
| during a bulk job           | seconds to MINUTES |

$P(\text{a user sees stale data}) = P(\text{they read within the lag window})$

a random read, lag 50 ms, one read per minute  
→ ~0.00008 probability. Invisible.

a read triggered BY the write (the UI reloading)  
→ lands inside the window essentially ALWAYS

**That asymmetry is the entire reason read-your-writes is the reported bug and general staleness is not.**

## What each fix costs

|                              |                                                                       |
|------------------------------|-----------------------------------------------------------------------|
| optimistic UI                | zero server cost; a client change<br>→ the highest-value pattern here |
| pending-state UI             | zero server cost; a data-model and design change                      |
| leader routing for 30 s      | ~10-15% of a user's reads return to the leader                        |
| leader routing, object-based | ~2-5%                                                                 |
| version tokens               | a token on every request; the store must expose a comparable position |
| strong consistency           | 2x cost in DynamoDB, and 5-150 ms of latency                          |

**The two cheapest patterns cost nothing on the server at all**, which is why they should be considered first.

## Compensation economics

|                                                                                  |                               |
|----------------------------------------------------------------------------------|-------------------------------|
| an airline: overselling by 3% and compensating                                   |                               |
| cost of compensation                                                             | ~₹15,000 per bumped passenger |
| frequency                                                                        | ~1 in 2,000 passengers        |
| expected cost                                                                    | ~₹7.50 per passenger          |
| revenue from the extra seats sold                                                | ~₹300 per passenger           |
| → compensating is ~40x cheaper than the empty seats that guaranteeing would cost |                               |

**That arithmetic is the argument**, and it generalises: **compensate when the expected cost of the correction is below the cost of the coordination.**

## Counter approaches

|                                            |                                                                                  |
|--------------------------------------------|----------------------------------------------------------------------------------|
| 1,000,000 increments/second on one counter |                                                                                  |
| read-modify-write                          | WRONG – lost updates under any concurrency                                       |
| atomic INCR (Redis)                        | correct; one hot key, ~100,000 ops/s per instance<br>→ needs sharding above that |
| sharded counter, 100                       | 10,000 ops/s per shard; a read costs 100 lookups                                 |
| per-replica CRDT counter                   | each node counts locally; the read sums N values<br>→ no coordination at all     |

## Merkle tree comparison

comparing 1,000,000 keys between two replicas

key by key                      1,000,000 exchanges

Merkle tree                    ~20 exchanges to locate a differing range,  
                                     then only that range is synced

→  $O(\log n)$  to find where they differ, instead of  $O(n)$

**That is why anti-entropy is affordable at all**, and it is the same idea as a hash of a hash of a hash.

## 7 The trade-offs

Optimistic UI trades honesty for latency, and needs a revert

**It is the cheapest and best pattern for the common case**, and it fails badly when the revert is awkward. Never optimistically confirm a payment, a booking, or anything with a receipt — because there is no graceful way to un-tell someone.

Showing the pending state costs product complexity

Three states — done, pending, failed — instead of one. Every screen and every notification has to handle all three, and somebody has to design what “pending” looks like.

**The bank's answer is that it is worth it**, because the alternative — hiding the uncertainty — produces users who discover it and lose trust. **I would show the pending state whenever the window is long enough for a user to act during it.**

Compensation trades a guarantee for an occasional apology

**Cheaper than coordination when the correction is cheap and the case is rare.** It stops being acceptable when the correction is expensive, when the harm cannot be undone, or when it is illegal.

**And it has a hidden cost people miss:** a compensating action needs a whole workflow — detection, notification, refund, audit — and building that properly is not free. It is often more engineering than the coordination it replaced, and worth it anyway because it degrades better.

## Eventual consistency without a conflict strategy is not a design

From [yesterday](#): last-write-wins silently loses data and, with clock skew, may keep the older write. **Choosing eventual means choosing one of last-write-wins, version vectors, CRDTs, or an application merge** — and saying which.

### Where this approach breaks

- **Anything a user cannot undo.** A sent message, a published post, a completed payment. Optimism there is a lie you cannot retract.
- **Anything with a legal or financial audit requirement**, where “we corrected it later” is not an acceptable answer.
- **Chains of dependent eventual operations.** One stale read feeding another compounds, and the window becomes unbounded.
- **Cold data.** Read repair never fires, so convergence waits on anti-entropy — and the data nobody looks at is exactly the data that stays wrong longest, which is a poor property when somebody finally looks.
- **The idempotency store itself being eventually consistent**, which quietly reintroduces duplicates.

## 8 In the interview

### How it gets asked

- The direct one: *“The like count is wrong for two seconds. Is that acceptable?”*
- The follow-up: *“How would you make the user’s own action feel instant?”*
- The design one: *“How do you build a feature that works while the data is stale?”*
- The mechanism one: *“How long is ‘eventually’, actually?”*
- The hard one: *“What would you never make eventually consistent?”*

### What to say out loud, in the first ninety seconds

1. **Turn it into the right question.** “It depends what a user sees in those two seconds, and for which user. A like count off by one, seen by a stranger, is invisible. The same count not moving when you tap it is a bug people report.”
2. **Give the rule.** “The rule I use is: the **display** may be stale, the **decision** may not. Showing ‘twelve in stock’ can be seconds old; deciding whether to accept the order cannot.”



3. **Name the cheapest fix first.** “For the user’s own action I would use an **optimistic UI** — update the number locally the moment they tap, send the request in the background, and revert visibly if it fails. That makes read-your-writes a client-side problem, which costs nothing on the server.”
4. **Name the honest fallback.** “Where optimism is not safe, I would **show the pending state** — the way a bank shows an uncleared cheque. Users tolerate uncertainty they can see and resent uncertainty they discover.”
5. **Say what “eventually” actually means.** “Convergence comes from read repair for hot keys — milliseconds — and anti-entropy for cold ones, which is minutes to hours. So the distribution is bimodal: popular data is nearly always fresh and the long tail is what nobody is looking at.”
6. **Draw the line.** “And I would not make anything eventual where two users can get the same thing, or where the mistake cannot be corrected afterwards without harming someone.”

#### The follow-ups

“**How would you make the user’s own action feel instant?**” “**Optimistic UI**, and it is the highest-value pattern in this whole area because it costs nothing on the server. When they tap like, I increment the number on their screen immediately — before the request even leaves — because I am almost certain what the answer will be. The request goes in the background; on success nothing further happens and they never knew there was a request; on failure I **revert visibly and say why**. The reason it matters so much is that it converts read-your-writes from a server problem into a client one: no leader routing, no version tokens, no extra database load, and it fixes the single most reported anomaly in this phase. Its limit is the failure path — you must be able to revert **honestly**, so I would never do it for a payment or a booking confirmation, where there is no graceful way to un-tell someone. And I would build the revert first, because an optimistic UI without a considered failure path is worse than none: the user believes something untrue and finds out later.”

“**How long is ‘eventually’, actually?**” “It has a shape, and there are three mechanisms with very different speeds. **Read repair**: when a read touches several replicas and they disagree, the coordinator returns the newest value and writes it back to the stale ones — milliseconds, and it only ever fires for keys that get **read**. **Hinted handoff**: if a replica was down when the write happened, another node accepted it on that node’s behalf and replays it when it returns — seconds after recovery, and hints expire. **Anti-entropy**: a scheduled background pass where replicas compare **Merkle trees** to find which key ranges differ —  $O(\log n)$  exchanges instead of comparing a million keys — and then sync only those ranges. That runs every few minutes to hours. So the dis-

tribution is **bimodal**: a hot key converges in milliseconds because read repair fires constantly, and a cold key waits for the next anti-entropy pass. Which has an uncomfortable corollary — the data that stays wrong longest is exactly the data nobody is looking at, right up until somebody does.”

“**What would you never make eventually consistent?**” “Anything where **two users can get the same thing**, and anything where **the mistake cannot be corrected afterwards without harming someone**. Concretely: seat and ticket allocation, stock at the moment of purchase, unique constraints like usernames and email addresses, permission checks, and account balances at the moment of a transfer. In all of those the failure is not staleness — it is two people holding the same seat, and no amount of later convergence undoes that. The test I actually apply is: *if this value is wrong for two seconds, can the mistake be corrected afterwards without anyone being harmed?* If yes, eventual is fine and probably cheaper. If no, it needs to be linearizable, and I would rather pay the latency than design an apology workflow.”

“**An airline oversells flights. Isn’t that just a bug?**” “No — it is the most instructive example in the whole topic, because it is a deliberate choice with arithmetic behind it. Guaranteeing a seat for every ticket means coordinating perfectly and flying with empty seats when people do not turn up. Overselling by around three percent and compensating the occasional bumped passenger costs perhaps fifteen thousand rupees each, at roughly one in two thousand passengers — call it seven or eight rupees per passenger expected — against a few hundred rupees per passenger of revenue from the seats they can now sell. **Compensation is around forty times cheaper than prevention**. The general rule is: **compensate when the expected cost of the correction is below the cost of the coordination** — and the same logic explains why a hotel oversells and a concert with legal liability does not. The engineering consequence is that a compensating action is a real workflow — detection, notification, refund, audit — and building it properly is often more work than the coordination it replaced. It is still the right choice, because it degrades better.”

“**How do you show a value you are not sure about?**” “Honestly, and with the uncertainty in the **data model** rather than only in the wording. The bank statement is the design: it shows the balance, and separately the uncleared amount, and separately what is actually available — three numbers, because three different questions have three different answers. In software that is ‘Processing’, ‘Uploading’, ‘Pending’, or ‘Last updated thirty seconds ago’ on a dashboard. The principle is that **users tolerate uncertainty they can see and resent uncertainty they discover** — and the failure mode of hiding it is worse than the failure mode of showing it, because a user who dis-

covers that a number was wrong stops trusting all your numbers. The cost is product complexity: three states instead of one, on every screen and in every notification. I would pay it whenever the window is long enough for a user to act during it.”

“**What about a counter — is that not trivially eventual?**” “The concept is, and the implementation catches people. The naive version — read the count, add one, write it back — is a **lost update** under any concurrency, and it is wrong even on a single machine. Three correct approaches. An **atomic increment** in the store, like Redis `INCR` or DynamoDB `ADD` — correct, and it makes the counter one hot key, so it caps out around a hundred thousand a second. A **sharded counter** — increment one of a hundred sub-counters at random and sum on read — which spreads the writes and makes the read cost a hundred lookups. Or **per-replica counters summed on read**, which is a G-counter CRDT: each node counts only its own increments, the value is the sum, and because addition is commutative and associative there is no order to get wrong and no conflict to resolve. That last one is why counters are the textbook CRDT, and it converges with **no coordination at all.**”

A model answer

Asked: *the like count is wrong for two seconds. Is that acceptable?*

“It depends on **who is looking and at what**, so let me split it, because there are really two questions hiding in one.

For a **stranger** looking at somebody else’s post, two seconds is completely invisible. Nobody knows what the count should be, nobody is comparing it against anything, and being off by one is unobservable. So: yes, eventual, and I would not spend anything to fix it.

For the **user who just tapped like**, two seconds is a bug and it gets reported. And it is not a rare event — it is essentially certain, because the read that follows a write is not a random sample, it is the worst possible one: the interface refreshes immediately, so it lands squarely inside the replication window every single time.

The rule I would apply throughout is: **the display may be stale, the decision may not**. Showing ‘twelve left in stock’ can be seconds old; deciding whether to accept the order cannot.

So for the person who tapped, my first choice is an **optimistic UI**: increment the number on their screen the instant they tap, before the request is even sent, because I am almost certain what the answer will be. Send it in the background; on success nothing further happens and they never knew there was a request; on failure revert visibly and say why. That is the highest-leverage pattern here because it costs **nothing on the server** — it converts read-your-writes from a routing problem into a client concern. The one thing I would build first is the failure path, because an optimistic UI with no considered revert is worse than none.

Where optimism is not safe — where the server may legitimately reject or transform the action — I would either route that user’s reads to the leader for a short window, which costs ten to fifteen percent of their reads, or **show the pending state honestly**. That is the bank-statement pattern: an uncleared cheque is displayed as uncleared, with the date it will clear. Users tolerate uncertainty they can see and resent uncertainty they discover.

I would also be able to say **how long ‘eventually’ is**, because it is not one number. Hot keys converge in milliseconds through read repair — a read that sees disagreement writes the newest value back. Cold keys wait for anti-entropy, which compares Merkle trees to find differing ranges and runs every few minutes to hours. So it is bimodal, and the long tail is exactly the data nobody is reading.

And I would draw the line: I would not make anything eventual where **two users can get the same thing**, or where the mistake cannot be corrected afterwards without harming somebody. Seats, stock at purchase, usernames, permissions,

balances at the moment of a transfer. For a like count, though — yes, two seconds is fine, and I would spend the effort on making the user's own tap feel instant instead.”

## 9 Recall card

- **The question is never “is eventual consistency acceptable” — it is “what does a user see in that window, and how bad is it?”** The same two seconds is invisible on a stranger's like count, a reported bug on your own, and a double booking on a seat. **The rule: the DISPLAY may be stale, the DECISION may not.**
- **Four patterns. OPTIMISTIC UI is the highest-value one** — update locally on tap, send in the background, **revert visibly on failure** — because it makes read-your-writes a *client* problem and costs nothing on the server. Never use it where the revert is not honest (payments, bookings). Then: **show the pending state** (the bank's uncleared cheque), **route around it** (leader reads for 30 s, ~10–15% of reads), and **compensate** rather than prevent.
- **Bounded optimism is the underrated move:** act on stale data where the exposure is small and wait where it is large — which is exactly a card terminal's **offline floor limit**. You do not have to choose between fully trusting stale data and fully refusing.
- **“Eventually” is bimodal and has three mechanisms:** read repair (milliseconds, but only for keys that get READ) · hinted handoff (seconds after a node returns; hints expire) · anti-entropy (minutes to hours, comparing MERKLE TREES to find differing ranges in  $O(\log n)$  instead of  $O(n)$ ). The data that stays wrong longest is the data nobody reads.
- **Never eventual: two users can get the same thing, or the mistake cannot be corrected without harm** — seats, stock at purchase, unique constraints, permissions, balances at transfer. **Compensate when the expected correction is cheaper than the coordination** (airlines:  $\sim 40\times$  cheaper). And a **counter** is not trivial: read-modify-write is a **lost update** — use an atomic increment, a sharded counter, or **per-replica counters summed on read**, which is a CRDT and needs no coordination at all.

**DSA topic:** Top K problems **System design topic:** Eventual consistency in practice

Code these, in this order

One rule for the whole set: **name all three approaches before writing any of them, then choose out loud with `n` and `k`**. The question is explicitly about the trade-off, so producing one solution without the comparison is answering half of it.

| # | PROBLEM                         | SOURCE                | WHAT IT IS REALLY TESTING                                |
|---|---------------------------------|-----------------------|----------------------------------------------------------|
| 1 | Kth Largest Element in a Stream | LeetCode 703 (Easy)   | The one case where the heap is forced, and why.          |
| 2 | Top K Frequent Elements         | LeetCode 347 (Medium) | Counting first, and the <code>O(n)</code> bucket answer. |
| 3 | K Closest Points to Origin      | LeetCode 973 (Medium) | A computed key, and not taking the square root.          |
| 4 | Kth Largest Element in an Array | LeetCode 215 (Medium) | Quickselect with a random pivot, written iteratively.    |

On problem 1, say why the heap is forced

Before coding, say in one sentence why sorting and quickselect are both unavailable here.

On problem 2, write both versions

The `nlargest` version and the bucket version. Say which is `O(n)` and exactly what makes it possible. Then say what the heap is actually sized over, and why that matters.

On problem 3, count the square roots

Write it with `math.sqrt` and without. Time both at a million points and record the difference. Say in one sentence why removing it is safe.

On problem 4, break it deliberately

Write quickselect with a fixed last-element pivot, run it on `list(range(100_000))`, and record what happens. Then add the random pivot and run it again.

### The three-approaches drill

1. Name all three and give time, space, ordering and streaming support for each.
2. Say which question each of  $n$ ,  $k$  and “in memory or a stream” answers.
3. Say roughly where the crossover between heap and sort is.
4. Say what to use when  $k = 1$ .

### The min-heap drill

1. State the counter-intuitive rule and justify it in one sentence.
2. State the mirror rule for the  $k$  smallest.
3. Write the size- $k$  heap loop from memory.
4. Say what `heap[0]` is doing in the comparison and why it is  $O(1)$ .
5. Say why `heapreplace` rather than pop-then-push.

### The rejection drill

1. Give the expected number of insertions for a size- $k$  heap on random data.
2. Compute it for  $n = 1,000,000$  and  $k = 10$ .
3. Say what that means for the real cost versus the stated bound.
4. Give the input that produces the worst case, and say why it is not exotic.

### The quickselect drill

1. Write it iteratively from memory, with a random pivot.
2. Write the sum that shows it is  $O(n)$ .
3. Say why quicksort is  $O(n \log n)$  and this is not, in one sentence.
4. Give the worst-case input and say why it is common.
5. Name two fixes beyond a random pivot, and say which one real libraries use.
6. Say what it does to the caller's array.

### The variants drill

1. For top- $k$  frequent, say what the heap is sized over.
2. Give the  $O(n)$  alternative and the property that makes it possible.
3. For  $k$ -closest, say what you would omit and why it is safe.
4. For a stream, say why the heap is the only option.
5. For many queries on unchanging data, say what you would do instead.

### The ordering drill

1. Say which approaches return results in order and which do not.

2. Say what it costs to fix.
3. Say why this matters for a test.

#### The break-it drill

Trigger each and record the exact output, error or timing:

1. A max-heap of everything instead of a size-k min-heap, on a stream.
2. `heappop` + `heappush` instead of `heapreplace`, timed at  $n = 1,000,000$ .
3. Quickselect with a fixed pivot on sorted input.
4. Quickselect on the caller's array, then printing the original.
5. `nlargest(500_000, million_items)` timed against `sorted()`.
6. Top-k-frequent with the heap over all items rather than the counts.
7. `k = 0`, `k = n`, `k > n`, and an empty input, on all three approaches.

#### The right-question drill

1. State the question you ask instead of "is eventual consistency acceptable?"
2. Give five data points and say how bad two seconds of staleness is for each.
3. State the display-versus-decision rule.
4. Apply it to a stock count on a product page and at checkout.

#### The four-patterns drill

1. Name all four patterns.
2. For each, say what it costs on the server.
3. Say which one converts a server problem into a client one, and how.
4. Say what must be built first for that one, and why.
5. Give two things you would never apply it to.

#### The pending-state drill

1. Describe the bank statement's three numbers and what each answers.
2. Give four software equivalents of a pending state.
3. State the principle about uncertainty in one sentence.
4. Say what showing it costs, and when it is worth it.

#### The bounded-optimism drill

1. Describe the cheque example precisely, with both amounts.
2. State the general rule.



3. Name the real-world system that does this and the term for its threshold.
4. Give two software examples.

#### **The convergence drill**

1. Name all three mechanisms.
2. For each, say what triggers it and how fast it is.
3. Say which keys each one converges, and which it never touches.
4. Explain Merkle trees in one sentence and give the exchange count for a million keys.
5. Answer “how long is eventually?” with a distribution rather than a number.

#### **The never-eventual drill**

1. State the test you apply.
2. Give six things you would never make eventually consistent.
3. For each, say what the failure actually is — and note that it is not staleness.
4. Say what you would do instead for one of them.

#### **The compensation drill**

1. Describe the airline arithmetic with numbers.
2. State the general rule for when to compensate.
3. Give two cases where compensation is not acceptable.
4. Name the hidden engineering cost of a compensating action.

#### **The counter drill**

1. Write the naive counter and say what is wrong with it.
2. Name three correct approaches and their limits.
3. Say which one is a CRDT and why it converges without coordination.
4. Compute the throughput of each at 1,000,000 increments a second.

#### **The failure drill**

For each, say what happens and what you would add:

1. A user likes a post and the count does not move.
2. A user’s optimistic action fails and the UI silently keeps the old value.
3. A key is written once and never read, on a replica that missed the write.
4. Two users buy the last item at the same moment.
5. An idempotency store is itself eventually consistent.

6. A dashboard shows a number with no indication of when it was computed.

Two of the six are not fixed by better replication. Name them.

---

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the  $k$  largest elements. Which approach, and why?* The three questions that decide it, all three approaches named with their properties, the min-heap rule with its justification, the rejection argument with numbers, quickselect with both caveats, and the streaming case where it is not a trade-off.
  2. *The like count is wrong for two seconds. Is that acceptable?* The question split by who is looking, the display-versus-decision rule, optimistic UI as the cheapest fix with its failure path, the pending state as the honest fallback, what “eventually” actually is, and where you draw the line.
  3. *Why a min-heap for the  $k$  largest?* The heap holds candidates, the only question you ask is about the worst of them, and the mirror rule.
- 

### Before you move on

- ☐ I name all three approaches before writing any of them.
- ☐ I can state the min-heap rule and justify it in one sentence.
- ☐ I know the mirror rule for the  $k$  smallest.
- ☐ I can give the expected insertion count for a size- $k$  heap.
- ☐ I know the worst-case input for the heap approach.
- ☐ I can write quickselect iteratively with a random pivot.
- ☐ I can write the sum that shows quickselect is  $O(n)$ .
- ☐ I know the worst-case input and why it is common.
- ☐ I remember that quickselect mutates the caller’s array.
- ☐ I know where the crossover to sorting is.
- ☐ I know the  $O(n)$  bucket answer for top- $k$ -frequent and why it works.
- ☐ I do not take the square root in  $k$ -closest-points.
- ☐ I always check  $k = 0$ ,  $k = n$  and empty input.
- ☐ I ask what a user sees rather than whether eventual is acceptable.

- [ ] I can state the display-versus-decision rule.
- [ ] I can name all four patterns and their server cost.
- [ ] I would build the revert path before the optimistic update.
- [ ] I can name all three convergence mechanisms and what each one reaches.
- [ ] I can answer “how long is eventually?” with a distribution.
- [ ] I can state the test for what must never be eventual.
- [ ] I answered all three questions above out loud.

# Merging K sorted lists

## DATA STRUCTURES AND ALGORITHMS

### Merging K sorted lists

You can merge  $k$  lists in  $O(n \log k)$  and explain why the heap holds only  $k$  items.

## SYSTEM DESIGN

### Quorums: why $R$ plus $W$ must exceed $N$

You can compute a quorum and show why the inequality guarantees an overlap.

## THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Merge  $k$  sorted lists efficiently.*
- *You have five replicas. What  $R$  and  $W$  would you pick?*

## 1 What this is, and why they ask it

You have  $k$  lists, each already sorted, holding  $n$  items between them. Produce one sorted list.

Three sentences. The whole problem reduces to a single repeated question — **which of the  $k$  current front items is smallest?** — and that is precisely what a heap answers in  $O(\log k)$ . The detail that carries the complexity, and the one the interviewer is checking, is that **the heap holds only  $k$  items, never  $n$** : one candidate per list, replaced as each is consumed. And there is a second  $O(n \log k)$  solution — merging the lists pairwise in a tournament — which is worth knowing because it uses no heap at all and beats the heap when comparisons are expensive.

They ask it because it is where the heap stops being a data structure and becomes a technique. It is also the shape of **external sorting** — how you sort a file larger than memory — so it is one of the few interview problems whose answer is literally how real systems work.

## 2 The story

The sorting office at the district headquarters took the bundles off the night bus at about half past four, and Basheer had been the man who combined them since 1988.

Six bundles arrived, one from each of the six taluk offices. Each bundle was already in order by pincode — the taluk clerks did that overnight — and Basheer's job was to turn the six into one sequence for the delivery vans.

The new man who came in 2004 did it by putting all six bundles on the table together, going through the entire pile, finding the lowest pincode, taking it, and starting again. It worked. It took him until nine o'clock.

Basheer's method took about seventy minutes, and it looked lazy.

He did not look at the bundles at all. He looked at **six letters** — the top one of each bundle — and nothing else. Six letters, laid out in front of him in a small arc.

He picked the lowest of those six, put it in the outgoing tray, and then **pulled the next letter off whichever bundle he had just taken from** and put it in the gap.

Six letters in front of him, always. Never seven, never five. Whatever was on the table was one candidate from each bundle, and the answer was always among them, because everything behind a letter in its own bundle had a higher pincode than it.

The new man watched for a while and asked how Basheer knew he was not missing something further down one of the bundles.

Basheer said: the bundles are already in order. If the top letter of the Erode bundle is 638001, then nothing else in that bundle is lower. So the lowest letter in the whole load has to be the lowest of the six on the table. There is nowhere else it could be hiding.

And he made one more point, which was the reason he could do it at four in the morning without thinking. He said he never had to compare six letters properly either. He kept them arranged so that the lowest was always at his left hand. When he took it and slid a new one in, he only had to nudge the new one along until it sat in the right place — usually one or two positions — and the lowest was at his left hand again.

He said the whole job was six letters and a small nudge, repeated four thousand times.

---

### 3 The idea in plain English

Basheer's six letters are a heap of size  $k$ , and his argument about the Erode bundle is the proof that the method is correct.

- The six bundles are the  $k$  sorted lists.
- The six letters on the table are the **heap**: one candidate per list.
- “Nothing else in that bundle is lower” is the **invariant** that makes it correct.
- “Pull the next one off the bundle I just took from” is what keeps the heap at exactly  $k$ .
- “Nudge it into place” is `heappush` —  $O(\log k)$ , not a scan of six.

The invariant, which is the whole correctness argument

**At every moment, the smallest remaining item across all lists is one of the  $k$  current front items.**

**Why:** each list is sorted, so within a list nothing behind the front is smaller than the front. The overall smallest must therefore be at the front of *some* list — and the heap holds all  $k$  fronts.

**That is the entire proof**, and being able to give it in one sentence is what separates a memorised solution from an understood one.

#### The algorithm

```
heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
heapq.heapify(heap) # $O(k)$

out = []
while heap:
 value, list_index, item_index = heapq.heappop(heap) # $O(\log k)$
 out.append(value)
 nxt = item_index + 1
 if nxt < len(lists[list_index]):
 heapq.heappush(heap, (lists[list_index][nxt], list_index, nxt))
return out
```

#### Three things to point at while writing it.

**The heap holds  $k$  items, never  $n$ .** One per list. Every pop is immediately followed by at most one push from the same list, so the size never grows. That is why the complexity is  $O(n \log k)$  and not  $O(n \log n)$  — and it is the thing the question is testing.

**The tuple carries where it came from.** You pop a value and must know which list to advance, so the entry is `(value, list_index, item_index)` rather than just the value.

**The list index doubles as the tie-breaker.** Two equal values would make Python compare the next tuple element, and the list index is always distinct — so the payload is never compared. That is [day 115's](#) rule, satisfied for free here.

**Why  $O(n \log k)$  and not  $O(n \log n)$**

|                               |               |                                |
|-------------------------------|---------------|--------------------------------|
| n items are popped, once each | n pops        |                                |
| each pop costs                | $O(\log k)$   | – the heap holds $k$ , not $n$ |
| -----                         |               |                                |
| total                         | $O(n \log k)$ |                                |

**The difference is real and worth quantifying:**

$n = 1,000,000$  items,  $k = 100$  lists

$O(n \log n) \rightarrow 1,000,000 \times 20 = 20,000,000$  comparisons

$O(n \log k) \rightarrow 1,000,000 \times 7 = 7,000,000$

$\rightarrow \sim 3\times$  fewer

$O(n \cdot k) \rightarrow 1,000,000 \times 100 = 100,000,000 \leftarrow$  the naive scan

**And the space difference is the bigger one:**  $O(k)$  instead of  $O(n)$ , which is what makes the technique work on data that does not fit in memory.

The naive approaches, and why they lose

**Scan all  $k$  fronts each time.** The new man's method, sort of.

$O(n \cdot k)$  time,  $O(k)$  space

Correct, and it re-compares the same  $k - 1$  items on every step. At  $k = 100$  it is fourteen times slower than the heap. **The heap's only job is to avoid re-finding the minimum you already knew.**

**Concatenate everything and sort.**

$O(n \log n)$  time,  $O(n)$  space

Correct, simple, and it **throws away the fact that the inputs are sorted** — which is the only interesting property of the problem. It is also  $O(n)$  space, so it cannot stream.

**Merge them one at a time, sequentially.**

merge list 1 with list 2, then with list 3, then with list 4 ...

$\rightarrow$  the accumulating result is re-walked every time

$\rightarrow 2n + 3n + \dots = O(n \cdot k)$

**This is the trap** — it looks like a divide-and-conquer solution and it is not. The accumulated list gets longer each time, so early elements are copied  $k$  times.

The other  $O(n \log k)$  : pairwise tournament merging

**Merge them in pairs, then merge the results in pairs, and so on** — a knockout tournament.



```
round 1: k lists → k/2 lists each element touched once: n
round 2: k/2 → k/4 n
...
log k rounds, n work per round → O(n log k)
```

**The same complexity as the heap, with no heap.** Its advantages:

- **No priority queue at all** — just the two-list merge you already know.
- **Better cache behaviour**, because each merge is two linear scans.
- **Trivially parallel**: the merges in a round are independent.

Its disadvantages:

- **$O(n)$  space** for the intermediate results (unless the lists are linked lists, where merging is in place — which is why LeetCode 23 uses linked lists).
- **Not streaming**: it needs all the data up front.

**Say both.** “Two solutions at  $O(n \log k)$ : a heap of the  $k$  fronts, or a pairwise tournament. The heap is  $O(k)$  space and streams; the tournament has better constants and parallelises.”

External sorting: the reason this matters

**How do you sort a hundred gigabytes with eight gigabytes of memory?**

```
1. read a chunk that fits in memory, sort it, write it to disk (a "run")
2. repeat until the input is consumed → ~13 sorted runs
3. MERGE the runs with a k-way merge → exactly today's problem
```

**Step 3 is this algorithm**, and the  $O(k)$  memory is what makes it possible: you hold one block from each run, not the whole file. **That is why merge is the sorting algorithm of the disk world** and quicksort is the one of memory — quicksort needs random access, and merging is sequential in every direction.

**It is also how a database merges sorted index scans**, and how log-structured storage engines compact their SSTables. Naming one of those is a strong finish.

Variants, which are the same shape

**Merge k sorted linked lists** — LeetCode 23. The heap holds nodes; the tuple is `(node.val, index, node)`; and you advance with `node.next`. **The tournament version merges in place with no extra space**, which is why the linked-list framing exists.



SCAN ALL  $k$  FRONTS EACH TIME  
for each of  $n$  outputs:  
scan  $k$  fronts  $\rightarrow O(n \cdot k)$

re-compares the same  $k-1$  items  
every single step

SEQUENTIAL PAIRWISE  $\leftarrow$  THE TRAP  
 $\text{merge}(1,2) \rightarrow \text{merge}(\cdot,3) \rightarrow \dots$   
the accumulator is re-walked  
each time:  $2n + 3n + \dots = O(n \cdot k)$   
LOOKS like divide and conquer;  
it is not.

TOURNAMENT PAIRWISE  $\leftarrow$  also  $O(n \log k)$   
round 1:  $k \rightarrow k/2$  ( $n$  work)  
round 2:  $k/2 \rightarrow k/4$  ( $n$  work)  
 $\dots \log k$  rounds  
 $O(n \log k)$ ,  $O(n)$  space, NO HEAP, better cache, parallelises

CONCATENATE AND SORT  
[all  $n$  items]  $\rightarrow$  sort

$O(n \log n)$ ,  $O(n)$  space  
THROWS AWAY the sortedness,  
which is the only interesting  
property of the input

HEAP OF  $k$  FRONTS  $\leftarrow$  the answer  
 one candidate per list  
 $O(n \log k)$  time,  $O(k)$  space  
STREAMS

The costs, at scale:

$n = 1,000,000$  items,  $k = 100$  lists

| approach            | comparisons       | space  |                             |
|---------------------|-------------------|--------|-----------------------------|
| scan $k$ fronts     | 100,000,000       | $O(k)$ |                             |
| concatenate + sort  | 20,000,000        | $O(n)$ |                             |
| sequential pairwise | $\sim 50,000,000$ | $O(n)$ |                             |
| HEAP of $k$ fronts  | 7,000,000         | $O(k)$ | $\leftarrow$ time AND space |
| tournament pairwise | 7,000,000         | $O(n)$ |                             |

External sorting, which is why this technique exists:

100 GB file, 8 GB of memory

PASS 1 – create sorted runs

read 8 GB, sort in memory, write to disk  
repeat

} ~13 sorted runs on disk

PASS 2 – k-way merge ← TODAY'S ALGORITHM

hold ONE BLOCK from each of the 13 runs in memory

|      |      |      |     |     |
|------|------|------|-----|-----|
| run1 | run2 | run3 | ... | r13 |
|------|------|------|-----|-----|

 ← the heap of k fronts

pop the smallest, write it out, refill from that run

$O(k)$  memory is the WHOLE POINT: you never hold the file, only  $k$  blocks.

→ this is why MERGE is the sorting algorithm of disks and QUICKSORT is the one of memory.

## 5 The code, built step by step

**Step 1 – state the invariant before writing anything**

“Each list is sorted, so nothing behind a list’s front element is smaller than that front. Therefore the overall smallest must be one of the  $k$  fronts — there is nowhere else it can be. So I keep exactly those  $k$  candidates in a heap.”

**That sentence is the correctness proof and it takes ten seconds.**

**Step 2 – say the size, because it is the point**

“The heap holds  $k$  items, not  $n$ . Every pop is followed by at most one push from the same list, so the size never grows. That is what makes it  $O(n \log k)$  rather than  $O(n \log n)$ , and  $O(k)$  space rather than  $O(n)$ .”

**Step 3 – the tuple, with all three fields**

```
(value, list_index, item_index)
```

“I need to know which list a popped value came from, so I can advance that list. And the list index is always distinct, so it doubles as a tie-breaker — two equal values never cause Python to compare the payload.”

#### Step 4 — heapify the initial fronts, do not push them

```
heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
heapq.heapify(heap) # O(k), not O(k log k)
```

**And skip the empty lists**, or the first pop crashes on an index that does not exist.

#### Step 5 — offer the second $O(n \log k)$ solution

“There is another solution at the same complexity with no heap at all: merge the lists in pairs, then merge those results in pairs — a tournament,  $\log k$  rounds of  $n$  work each. It has better cache behaviour and parallelises, and it needs  $O(n)$  space unless the lists are linked, which is why the classic version of this problem uses linked lists.”

## The complete solution

```
import heapq
from typing import Iterable, Iterator

1. The heap solution – $O(n \log k)$ time, $O(k)$ space, and it STREAMS

def merge_k_sorted(lists: list[list[int]]) → list[int]:
 """LeetCode 23, array form.

 THE INVARIANT: each list is sorted, so nothing behind a list's front is
 smaller than that front. Therefore the overall smallest is one of the k
 fronts – there is nowhere else it can hide. That is the whole proof.

 THE HEAP HOLDS k ITEMS, NEVER n : every pop is followed by at most one
 push from the SAME list, so the size is invariant. That is why this is
 $O(n \log k)$ and $O(k)$ space rather than $O(n \log n)$ and $O(n)$.

 The tuple is (value, list_index, item_index):
 - list_index says which list to advance after a pop
 - and it is always DISTINCT, so it doubles as the tie-breaker and the
 payload is never compared (day 115's rule, satisfied for free)
 """
 heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
 heapq.heapify(heap) # $O(k)$, not k pushes

 out: list[int] = []
 while heap:
 value, li, ii = heapq.heappop(heap) # $O(\log k)$
 out.append(value)
 nxt = ii + 1
 if nxt < len(lists[li]):
 heapq.heappush(heap, (lists[li][nxt], li, nxt))
 return out

def merge_k_sorted_lazy(lists: list[list[int]]) → Iterator[int]:
 """The same, as a generator – so the caller can stop early and the
 output never has to fit in memory. This is what makes it usable for
 external sorting."""
 heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
 heapq.heapify(heap)
 while heap:
 value, li, ii = heapq.heappop(heap)
 yield value
 nxt = ii + 1
 if nxt < len(lists[li]):
 heapq.heappush(heap, (lists[li][nxt], li, nxt))

2. Linked lists – the classic framing, because merging is IN PLACE

class ListNode:
 def __init__(self, val: int = 0, nxt: "ListNode | None" = None) → None:
 self.val = val
 self.next = nxt

def merge_k_linked_heap(heads: list["ListNode | None"]) → "ListNode | None":
```

```

"""The heap version for linked lists. The tuple carries the NODE, and the
index is the tie-breaker because ListNode has no __lt__."""
heap: list[tuple[int, int, ListNode]] = [
 (node.val, i, node) for i, node in enumerate(heads) if node
]
heapq.heapify(heap)

dummy = ListNode()
tail = dummy
while heap:
 _, i, node = heapq.heappop(heap)
 tail.next = node
 tail = node
 if node.next:
 heapq.heappush(heap, (node.next.val, i, node.next))
tail.next = None
return dummy.next

def merge_two(a: "ListNode | None", b: "ListNode | None") → "ListNode | None":
 """The two-list merge – the primitive the tournament is built from.
 IN PLACE for linked lists: no new nodes, only pointer changes."""
 dummy = ListNode()
 tail = dummy
 while a and b:
 if a.val ≤ b.val:
 tail.next, a = a, a.next
 else:
 tail.next, b = b, b.next
 tail = tail.next
 tail.next = a or b
 return dummy.next

def merge_k_linked_tournament(heads: list["ListNode | None"]) → "ListNode | None":
 """THE OTHER $O(n \log k)$ SOLUTION – no heap at all.

 Merge in pairs, then merge those results in pairs: $\log k$ rounds, each
 doing $O(n)$ work.

 Advantages over the heap: better cache behaviour (two linear scans per
 merge), trivially parallel (the merges in a round are independent), and
 for LINKED lists it is $O(1)$ extra space because merging is in place.

 Disadvantage: it needs all the data up front – it cannot stream.
 """
 if not heads:
 return None
 lists = [h for h in heads if h]
 if not lists:
 return None

 while len(lists) > 1:
 merged: list[ListNode] = []
 for i in range(0, len(lists) - 1, 2):
 merged.append(merge_two(lists[i], lists[i + 1])) # type: ignore[arg-type]
 if len(lists) % 2:
 merged.append(lists[-1])
 lists = merged # $k \rightarrow k/2$, one round done
 return lists[0]

3. The approaches that lose, written out so the difference is concrete

```

```

def merge_by_scanning(lists: list[list[int]]) → list[int]:
 """O(n·k): re-find the minimum from scratch every time.

 Correct, and it re-compares the same k-1 items on every single step.
 The heap's ONLY job is to avoid re-finding a minimum it already knew.
 """
 positions = [0] * len(lists)
 out: list[int] = []
 total = sum(len(lst) for lst in lists)

 for _ in range(total):
 best_value = None
 best_list = -1
 for i, lst in enumerate(lists):
 if positions[i] < len(lst):
 if best_value is None or lst[positions[i]] < best_value:
 best_value = lst[positions[i]]
 best_list = i
 out.append(best_value) # type: ignore[arg-type]
 positions[best_list] += 1
 return out

def merge_by_concatenating(lists: list[list[int]]) → list[int]:
 """O(n log n), O(n) space. Correct, simple, and it THROWS AWAY the
 sortedness – which is the only interesting property of the input.
 Also cannot stream."""
 return sorted(v for lst in lists for v in lst)

def merge_sequentially(lists: list[list[int]]) → list[int]:
 """THE TRAP: it looks like divide and conquer and it is not.

 The accumulator grows every round, so the early elements are walked
 k times: 2n + 3n + ... = O(n·k).
 """
 result: list[int] = []
 for lst in lists:
 merged: list[int] = []
 i = j = 0
 while i < len(result) and j < len(lst):
 if result[i] ≤ lst[j]:
 merged.append(result[i]); i += 1
 else:
 merged.append(lst[j]); j += 1
 merged.extend(result[i:]); merged.extend(lst[j:])
 result = merged # ← re-walked next round
 return result

4. The variants – the same heap, a small change

def smallest_range(lists: list[list[int]]) → tuple[int, int]:
 """LeetCode 632: the smallest range containing at least one element from
 each list.

 THE SAME heap of k fronts, plus ONE extra variable: the maximum of the
 current fronts. The range is [heap_min, current_max], and advancing the
 minimum is the only move that can shrink it.
 """
 heap = [(lst[0], i, 0) for i, lst in enumerate(lists)]
 heapq.heapify(heap)

```



```

current_max = max(lst[0] for lst in lists)

best = (heap[0][0], current_max)
while True:
 value, li, ii = heapq.heappop(heap)
 if current_max - value < best[1] - best[0]:
 best = (value, current_max)
 if ii + 1 == len(lists[li]):
 return best # one list exhausted: no more ranges
 nxt = lists[li][ii + 1]
 current_max = max(current_max, nxt)
 heapq.heappush(heap, (nxt, li, ii + 1))

def kth_smallest_in_matrix(matrix: list[list[int]], k: int) → int:
 """LeetCode 378: each row is a sorted list, so this is a k-way merge
 that STOPS after k pops. O(k log rows) rather than O(n log n)."""
 rows = len(matrix)
 heap = [(matrix[r][0], r, 0) for r in range(min(rows, k))]
 heapq.heapify(heap)
 value = 0
 for _ in range(k):
 value, r, c = heapq.heappop(heap)
 if c + 1 < len(matrix[r]):
 heapq.heappush(heap, (matrix[r][c + 1], r, c + 1))
 return value

def external_merge_sort(chunks: list[Iterable[int]]) → Iterator[int]:
 """The reason this technique matters: `heapq.merge` is a lazy k-way
 merge, so it works on runs far larger than memory.

 100 GB with 8 GB of RAM:
 pass 1 – sort chunks that fit, write ~13 runs to disk
 pass 2 – merge the runs, holding ONE BLOCK from each ← this line
 """
 return heapq.merge(*chunks)

5. Measuring the difference

def compare(n: int, k: int) → None:
 """Run this rather than trusting the table."""
 import random
 import time

 per = n // k
 lists = [sorted(random.randint(0, 10 ** 9) for _ in range(per))
 for _ in range(k)]

 for name, fn in (("heap", merge_k_sorted),
 ("concat+sort", merge_by_concatenating),
 ("sequential", merge_sequentially),
 ("scan k", merge_by_scanning)):
 t = time.perf_counter()
 fn(lists)
 print(f" {name} {time.perf_counter() - t:.4f}s")

def to_linked(values: list[int]) → "ListNode | None":
 head = None
 for v in reversed(values):
 head = ListNode(v, head)

```

```

return head

def from_linked(node: "ListNode | None") → list[int]:
 out = []
 while node:
 out.append(node.val)
 node = node.next
 return out

if __name__ == "__main__":
 lists = [[1, 4, 5], [1, 3, 4], [2, 6]]

 print(merge_k_sorted(lists)) # [1, 1, 2, 3, 4, 4, 5, 6]
 print(list(merge_k_sorted_lazy(lists))) # the same, lazily
 print(merge_by_scanning(lists) == merge_k_sorted(lists)) # True
 print(merge_by_concatenating(lists) == merge_k_sorted(lists)) # True
 print(merge_sequentially(lists) == merge_k_sorted(lists)) # True

 # linked lists – both solutions
 heads = [to_linked(l) for l in lists]
 print(from_linked(merge_k_linked_heap(heads))) # [1, 1, 2, 3, 4, 4, 5, 6]
 heads = [to_linked(l) for l in lists]
 print(from_linked(merge_k_linked_tournament(heads))) # same

 # edge cases
 print(merge_k_sorted([])) # []
 print(merge_k_sorted([[], [], []])) # []
 print(merge_k_sorted([[], [1, 2], []])) # [1, 2] ← empty lists skipped
 print(merge_k_sorted([[5]])) # [5]

 # equal values across lists – the tie-breaker earns its keep
 print(merge_k_sorted([[1, 1], [1, 1], [1]])) # [1, 1, 1, 1, 1]

 # the variants
 print(smallest_range([[4, 10, 15, 24, 26],
 [0, 9, 12, 20],
 [5, 18, 22, 30]])) # (20, 24)
 print(kth_smallest_in_matrix([[1, 5, 9], [10, 11, 13], [12, 13, 15]], 8)) # 13

 # heapq.merge is lazy and does this for you
 print(list(heapq.merge([1, 4, 7], [2, 5, 8], [3, 6, 9]))) # [1..9]

 # WHERE THE DIFFERENCE SHOWS – run it
 print("n=100,000 k=100")
 compare(100_000, 100)

```

## 6 What it costs

### The four approaches

| approach                                             | time           | space        | streams? | uses sortedness? |
|------------------------------------------------------|----------------|--------------|----------|------------------|
| -----                                                | -----          | -----        | -----    | -----            |
| scan k fronts                                        | $O(n \cdot k)$ | $O(k)$       | yes      | yes              |
| concatenate + sort                                   | $O(n \log n)$  | $O(n)$       | no       | NO               |
| sequential pairwise                                  | $O(n \cdot k)$ | $O(n)$       | no       | yes              |
| HEAP of k fronts                                     | $O(n \log k)$  | $O(k)$       | YES      | yes              |
| tournament pairwise                                  | $O(n \log k)$  | $O(n) \cdot$ | no       | yes              |
| * $O(1)$ for linked lists, where merging is in place |                |              |          |                  |

### At scale

|                                    |                         |                                                       |                            |  |
|------------------------------------|-------------------------|-------------------------------------------------------|----------------------------|--|
| n = 1,000,000 items, k = 100 lists |                         |                                                       |                            |  |
| scan k fronts                      | 100,000,000 comparisons |                                                       |                            |  |
| concatenate + sort                 | 20,000,000              |                                                       |                            |  |
| sequential pairwise                | ~50,000,000             |                                                       |                            |  |
| heap                               | 7,000,000               |                                                       | ← 14x better than scanning |  |
| tournament                         | 7,000,000               |                                                       |                            |  |
|                                    |                         |                                                       |                            |  |
| k = 2                              | log k = 1               | the heap gains nothing; just merge them               |                            |  |
| k = 10                             | log k = 3.3             | ~3x better than a full sort                           |                            |  |
| k = 100                            | log k = 6.6             | ~3x better than a full sort, 14x better than scanning |                            |  |
| k = 10,000                         | log k = 13.3            | approaching log n; the advantage narrows              |                            |  |

When **k** approaches **n** — many tiny lists — **log k** approaches **log n** and the heap’s advantage disappears. At that point concatenate and sort, because its constants are far better.

### Space, which is the more important difference

|                                                             |        |                           |  |  |
|-------------------------------------------------------------|--------|---------------------------|--|--|
| n = 100,000,000 items (does not fit in memory), k = 50 runs |        |                           |  |  |
| concatenate + sort                                          | $O(n)$ | → IMPOSSIBLE              |  |  |
| tournament                                                  | $O(n)$ | → IMPOSSIBLE (for arrays) |  |  |
| heap of k fronts                                            | $O(k)$ | → 50 entries. Trivial.    |  |  |

That is the whole reason external sorting works, and it is why “merge” is the disk-world sorting algorithm.

## The heap versus the tournament, honestly

same complexity:  $O(n \log k)$

### HEAP

$\log k$  comparisons per element  
random access into the heap  
→ poor cache locality  
 $O(k)$  space  
STREAMS  
sequential

### TOURNAMENT

$\log k$  comparisons per element  
two LINEAR scans per merge  
→ excellent cache locality  
 $O(n)$  space ( $O(1)$  for linked lists)  
needs everything up front  
the merges in a round are INDEPENDENT  
→ trivially parallel

**Measured, the tournament is often  $1.5\text{--}2\times$  faster on arrays in memory**, and the heap is the only one that works when the data does not fit. **Neither dominates**, which is why both are worth naming.

### Where the constants live

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| each heap pop:  | $\sim \log k$ comparisons, plus tuple allocation in Python |
| each tournament |                                                            |
| merge step:     | 1 comparison, and sequential memory access                 |

**In Python the tuple allocation per push is a real cost** — a heap of a million pushes allocates a million three-tuples at about 72 bytes each. `heapq.merge` avoids some of this internally, which is one reason to prefer it over a hand-rolled loop.

## 7 The traps

### Trap 1 — a heap of everything instead of a heap of $k$

```
heap = [v for lst in lists for v in lst] # ALL n items
heapq.heapify(heap)
```

Correct output, and it is  $O(n)$  space and  $O(n \log n)$  time — which is just “concatenate and sort” with extra steps. **The whole point is that the heap holds one candidate per list.**

**And on a stream it is not merely slower, it is impossible.**

### Trap 2 — not skipping empty lists

```
heap = [(lst[0], i, 0) for i, lst in enumerate(lists)]
```

```
IndexError: list index out of range
```

An empty input list has no `lst[0]`. The `if lst` guard is not defensive coding — it is required, and `[], [1, 2], []` is the first test case anyone runs.

Trap 3 — no tie-breaker with an unorderable payload

```
heapq.heappush(heap, (node.val, node)) # two nodes with val 5
```

```
TypeError: '<' not supported between instances of 'ListNode' and 'ListNode'
```

Day 115's trap, and it appears in exactly this problem because equal values across lists are common. The list index is the natural tie-breaker and it is always distinct, so `(value, index, node)` fixes it for free.

Trap 4 — sequential merging, believing it is divide and conquer

```
for lst in lists:
 result = merge_two(result, lst)
```

$O(n \cdot k)$ , because the accumulator is re-walked every round:  $2n + 3n + 4n + \dots$ . The tournament pairs them up so that each round touches every element exactly once — that one structural difference is  $O(n \cdot k)$  against  $O(n \log k)$ .

Trap 5 — advancing the wrong list

```
value, li, ii = heapq.heappop(heap)
heapq.heappush(heap, (lists[0][ii + 1], 0, ii + 1)) # always list 0
```

You must advance the list the popped value came from, which is why the tuple carries `list_index`. This produces plausible-looking but wrong output, not a crash.

Trap 6 — pushing before checking the bound

```
heapq.heappush(heap, (lists[li][ii + 1], li, ii + 1)) # no bound check
```

```
IndexError: list index out of range
```

when a list is exhausted. The heap shrinks as lists run out — that is normal and expected, and the `if nxt < len(...)` guard is what allows it.

### Trap 7 — `heapify` versus `k` pushes

```
for i, lst in enumerate(lists):
 heapq.heappush(heap, (lst[0], i, 0)) # $O(k \log k)$
```

Correct, and `heapify` is  $O(k)$ . Negligible when `k` is small, and it is the same [day 114](#) point, so it is worth getting right by habit.

### Trap 8 — building the output list when the caller wants a stream

```
return out # $O(n)$ memory for the result
```

If the point is that the data does not fit in memory, **returning a list defeats it**. Yield instead — which is what `heapq.merge` does, and it is the difference between a technique that works on a hundred gigabytes and one that does not.

## 8 In the interview

### How it gets asked

- The classic: “Merge *k* sorted linked lists.” LeetCode 23.
- The complexity probe: “Why  $O(n \log k)$  and not  $O(n \log n)$ ?”
- The alternative probe: “Can you do it without a heap?”
- The applied one: “How would you sort a file larger than memory?”
- The variant: “Find the smallest range covering at least one element from each list.” LeetCode 632.

### What to say out loud, in the first ninety seconds

1. **State the invariant, which is the proof.** “Each list is sorted, so nothing behind a list’s front is smaller than that front. So the overall smallest must be one of the `k` fronts — there is nowhere else it can be.”
2. **Say what the heap holds, because that is the answer.** “So I keep exactly those `k` candidates in a heap: one per list. Pop the smallest, output it, and push the next item from **that** list.”
3. **Give the complexity with the reason attached.** “`n` pops at  $O(\log k)$  each, because the heap holds `k` and not `n` — so  $O(n \log k)$  time and, more importantly,  $O(k)$  space.”

4. **Say what the tuple carries and why.** “The entry is  $(\text{value}, \text{list index}, \text{position})$ . I need the list index to know which list to advance — and it is always distinct, so it also serves as the tie-breaker and the payload is never compared.”
5. **Offer the second solution.** “There is another  $O(n \log k)$  with no heap: merge in pairs, then merge those in pairs —  $\log k$  rounds of  $n$  work. Better cache behaviour and it parallelises; it needs  $O(n)$  space unless the lists are linked.”
6. **Name the real use.** “This is exactly the merge phase of an external sort, which is how you sort a file larger than memory — and the  $O(k)$  space is the whole reason it works.”

#### The follow-ups

“**Why  $O(n \log k)$  and not  $O(n \log n)$ ?**” “Because of what the heap contains. It holds **one candidate per list** —  $k$  items — not all  $n$ . Every pop is followed by at most one push, and that push comes from the same list the popped item came from, so the size is invariant at  $k$  and only falls as lists are exhausted. So the cost is  $n$  pops at  $O(\log k)$  each. Concretely, at a million items across a hundred lists that is about seven million comparisons against twenty million for a full sort — three times fewer. But the difference I care about more is **space**:  $O(k)$  instead of  $O(n)$ . With a hundred million items across fifty runs, the heap holds fifty entries, which is what makes the technique work on data that does not fit in memory at all.”

“**Can you do it without a heap?**” “Yes, and at the same complexity. **Merge the lists in pairs**, then merge those results in pairs, and so on — a knockout tournament. There are  $\log k$  rounds, and each round touches every element exactly once, so it is  $O(n)$  per round and  $O(n \log k)$  overall. It has real advantages: no priority queue at all, just the two-list merge; **much better cache behaviour**, because each merge is two linear scans rather than random access into a heap; and the merges within a round are independent, so it **parallelises trivially**. Measured on arrays in memory it is often one and a half to two times faster than the heap. Its disadvantage is space:  $O(n)$  for the intermediate results — though for **linked lists** merging is in place, so it is  $O(1)$ , which is exactly why the classic version of this problem uses linked lists. And it cannot stream, which is where the heap wins outright.”

“**What is the trap here?**” “Merging them **sequentially** — merge list one with list two, then that result with list three, and so on. It looks like the same idea and it is  $O(n \cdot k)$ , because the accumulated result is re-walked on every round:  $2n + 3n + 4n + \dots$ . The elements of the first list get copied  $k$  times. The tournament version pairs them up so that each round touches every element exactly once, and that single struc-

tural difference is  $O(n \cdot k)$  against  $O(n \log k)$ . The other trap is putting **all**  $n$  items in the heap, which produces the right answer and is just ‘concatenate and sort’ with extra steps — and on a stream it is not slower, it is impossible.”

**“How would you sort a file larger than memory?”** “An external merge sort, and its second phase is exactly this problem. **Pass one:** read as much as fits in memory, sort it, write it out as a sorted run, and repeat — a hundred gigabytes with eight gigabytes of memory gives about thirteen runs. **Pass two:** do a  $k$ -way merge of those thirteen runs, holding one **block** from each in memory rather than the whole run. That is the heap of  $k$  fronts, and the  $O(k)$  memory is the entire reason it is possible. It also explains a broader fact: merge is the sorting algorithm of the disk world and quicksort is the one of memory, because quicksort needs random access while merging is sequential in every direction — which is exactly what disks and network streams are good at. The same machinery is what a database uses to merge sorted index scans, and what log-structured storage engines use to compact SSTables.”

**“Now find the smallest range covering an element from each list.”** “The same heap, with one extra variable. I keep the  $k$  fronts in a min-heap as before, and I also track the **maximum** of the current fronts. At any moment the range `[heap minimum, current maximum]` covers at least one element from every list, because the heap holds exactly one per list. To shrink it I must raise the minimum — nothing else can help — so I pop the smallest, record the range if it is the best so far, and push the next item from that list, updating the maximum. I stop as soon as one list is exhausted, because from then on no range can cover all  $k$ . It is  $O(n \log k)$  and the insight is that the heap is already doing the work; I just needed to notice what else its contents tell me.”

**“What are the edge cases?”** “Four, and they all appear in the tests. **An empty input list** among the others — `lst[0]` raises, so the initial heap build must skip empties; `[[], [1,2], []]` is the first case anyone runs. **All lists empty**, which should return empty rather than crash. **Equal values across lists**, which is common and is exactly where a missing tie-breaker raises `TypeError` — the list index solves it for free because it is always distinct. And **a list running out mid-merge**, which is normal: the heap simply shrinks, and the `if next_index < len(list)` guard is what allows that rather than an `IndexError`. I would also check `k = 1` and a single-element list, because both exercise the loop boundaries.”

**A model answer**

Asked: *merge  $k$  sorted lists efficiently.*



“Let me start with why this can be done better than sorting everything, because that argument is the solution.

Each list is already **sorted**. So within any list, nothing behind the front element is smaller than that front element. Which means the smallest item in the entire collection has to be at the **front of some list** — there is nowhere else it could be hiding. So at any moment I only ever need to consider  $k$  candidates: one per list.

That is exactly what a heap is for. I put the  $k$  front elements in a min-heap, pop the smallest, write it to the output, and then push the **next** element from the list I just took from. The heap goes from  $k$  to  $k-1$  and back to  $k$ , so its size never grows.

**That invariant is the whole complexity result:** the heap holds  $k$  items, not  $n$ . There are  $n$  pops and each costs  $O(\log k)$ , so it is  $O(n \log k)$  time — about seven million comparisons for a million items across a hundred lists, against twenty million for a full sort. And it is  $O(k)$  **space**, which matters more, because it means the technique works when the data does not fit in memory.

One implementation detail: the heap entry is `(value, list index, position)` rather than just the value. I need the list index to know which list to advance after a pop — and because it is always distinct, it also acts as the tie-breaker, so when two lists have equal values Python never tries to compare whatever comes after. That matters here because equal values across lists are common, and without it you get a `TypeError` on nodes or objects.

I would also mention the second  $O(n \log k)$  solution, because it is genuinely different: **merge the lists in pairs, then merge those results in pairs** — a tournament,  $\log k$  rounds each doing  $O(n)$  work. It uses no heap at all, has much better cache behaviour because every merge is two linear scans, and the merges within a round are independent so it parallelises. Its cost is  $O(n)$  space for the intermediates — unless the lists are **linked lists**, where merging is in place and it is  $O(1)$ , which is exactly why the classic version of this problem is posed with linked lists.

The trap I would avoid explicitly is merging them **sequentially** — first with second, then that with third — because the accumulator is re-walked every round and it degenerates to  $O(n \cdot k)$ . Pairing them up is what makes each round touch every element once.

And the reason this is worth knowing beyond the interview: it is the merge phase of an **external sort**. To sort a hundred gigabytes with eight gigabytes of memory, you write sorted runs and then do a k-way merge holding one block from each — the  $O(k)$  memory is the entire point.”

## 9 Recall card

- **The invariant IS the proof: each list is sorted, so nothing behind a list's front is smaller than that front — therefore the overall smallest is one of the  $k$  fronts, and there is nowhere else it can be.** Keep exactly those  $k$  candidates in a heap.
- **The heap holds  $k$ , never  $n$  —** every pop is followed by at most one push **from the same list**, so the size is invariant. That gives  $O(n \log k)$  time (7M comparisons against 20M for a sort at  $n = 1M, k = 100$ ) and — the part that matters more —  $O(k)$  space, which is what lets it work on data larger than memory.
- **The entry is  $(value, list\_index, position)$ .** The index says which list to advance, and because it is always **distinct** it doubles as the **tie-breaker** — equal values across lists are common, and without it you get `TypeError` on unorderable payloads.
- **There is a second  $O(n \log k)$  with no heap: a pairwise TOURNAMENT —  $\log k$  rounds,  $O(n)$  each.** Better cache locality, trivially parallel,  $O(n)$  space ( $O(1)$  for linked lists, which is why the classic problem uses them), but it cannot stream. **The trap is SEQUENTIAL pairwise merging**, which re-walks the accumulator and is  $O(n \cdot k)$ .
- **This is the merge phase of an EXTERNAL SORT:** sort runs that fit in memory, then k-way merge holding one block per run. It is why **merge is the disk world's sorting algorithm and quicksort is memory's**. Edge cases that bite: **skip empty lists** when building the heap (`IndexError` otherwise), bound-check before pushing, and expect the heap to **shrink** as lists are exhausted.

## 1 What this is, and why they ask it

A **quorum** is the number of replicas that must respond before an operation counts as done. Write to  $W$  of  $N$  replicas; read from  $R$  of them; and if you choose them correctly, **the read is guaranteed to see the write**.

Three sentences. The rule is one inequality —  $R + W > N$  — and the reason it works is the pigeonhole principle: if the set you wrote to and the set you read from together exceed the total, they must share at least one replica, and that replica has the newest value. It is the **dial** that **CAP** does not describe and that **consistency models** hinted at, because moving  $R$  and  $W$  slides you between fast-and-stale and slow-and-correct without changing anything else. And the interesting part is what happens when you deliberately break the inequality, which is what most large systems actually do.

They ask “five replicas — what  $R$  and  $W$ ?” because it is arithmetic with a reason behind it, and the follow-ups are unforgiving: *why not  $W = 5$ ?*, *what does the overlapping replica actually give you?*, and *is that really linearizable?* The last one has a surprising answer, and knowing it separates people who have read about quorums from people who have thought about them.

## 2 The story

The temple trust had eleven trustees and a rule book written in 1954, and the rule that mattered was about how a decision became a decision.

For anything involving money, six trustees had to sign. Not a majority of those present — six of the eleven, wherever they were.

Perumal, who kept the register, explained the other half of the rule to a new trustee in his first month, because the new man had asked a reasonable question: how do you know, six months later, what was actually decided?

The rule book said you asked six trustees.

The new man said that was strange. Six to decide and six to find out — why the same number?

Perumal said it is not a coincidence and it is not tradition. There are eleven of us. If six signed, and you go and ask six, then **at least one of the people you ask must have been one of the people who signed**. Six and six is twelve, and there are only eleven of us, so somebody is in both groups. There is no way to pick six who were all absent.

He said if the rule had been five to decide and five to ask, you could very easily talk to five people who had all been away that day, and every one of them would honestly tell you nothing had been decided. Five and five is ten, and eleven minus ten leaves room for a complete miss.

The new man asked why they did not just require all eleven to sign, which would make it certain.

Perumal said they had tried it in the sixties, and it had lasted about a year. One trustee was in Singapore for four months and the trust could not do anything at all. **The more people you need, the more certain you are and the more easily you are stopped**. Six is the smallest number that cannot be dodged.

And there was one more part of the rule that the new man found odd until it was explained. If you asked six and two of them said one thing and four said another, the register entry with the later date was the decision, and you were supposed to **tell the two who had it wrong**, so that next time they would be right.

Perumal said that was the part that kept the whole thing from drifting. Without it, the same two people would go on giving the same wrong answer for years, and every time you happened to ask them you would get it.

### 3 The idea in plain English

The trust is a quorum system, and Perumal's argument about six and six is the proof of

$$R + W > N.$$

- Eleven trustees are the  $N$  **replicas**.
- Six signatures is the **write quorum**,  $W$ .
- Asking six is the **read quorum**,  $R$ .
- "Somebody must be in both groups" is the **pigeonhole argument**, and it is the whole theorem.
- Requiring all eleven is  $W = N$ , and the trustee in Singapore is why nobody does it.
- Telling the two who had it wrong is **read repair**.

## The rule, and why it works

$R + W > N$  guarantees that the read set and the write set overlap.

**The proof is one sentence.** The write went to  $W$  replicas. The read asks  $R$  replicas. If  $R + W > N$ , then those two sets cannot be disjoint — there are not enough replicas for them to avoid each other — so at least one replica you read from received the write, and it has the newest value.

```
N = 5
W = 3, R = 3 → 3 + 3 = 6 > 5 ✓ they MUST share at least one
W = 2, R = 2 → 2 + 2 = 4 ≤ 5 ✗ you can read {A,B} after writing {D,E}

the overlap size is exactly R + W - N
W=3, R=3, N=5 → at least 1 replica in common
W=4, R=4, N=5 → at least 3 in common
```

**And the second half, which people forget: you must be able to tell which value is newest.** The overlapping replica has the new value, but the others may return an old one, so every record carries a **version** — a counter, a timestamp, or a vector clock — and the reader takes the highest. Without versioning, the overlap is useless because you cannot tell which of the returned values to believe.

**That is the second sentence of the answer**, and it is the one that shows understanding rather than recall.

### Choosing R and W

|               |                                                                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------|
| N=3, W=2, R=2 | the default. $2+2 = 4 > 3$ ✓<br>tolerates 1 failure for both reads and writes<br>balanced latency                                 |
| N=3, W=3, R=1 | fast reads (any replica), brittle writes<br>ONE replica down blocks ALL writes                                                    |
| N=3, W=1, R=3 | fast writes, slow reads<br>ONE replica down blocks ALL reads                                                                      |
| N=5, W=3, R=3 | $3+3 = 6 > 5$ ✓<br>tolerates 2 failures for both. The standard for<br>something important.                                        |
| N=5, W=1, R=1 | $1+1 = 2 \leq 5$ ✗ NO overlap guarantee<br>fastest possible, fully eventual<br>→ and this is what many large systems actually run |

**The choice is a latency-and-availability trade**, and the shape of it is worth saying:

- Increasing  $W$  makes writes slower and less available, and reads faster.
- Increasing  $R$  does the reverse.
- $W = 1, R = N$  and  $W = N, R = 1$  both satisfy the inequality and are opposite extremes.
- The balanced choice is  $W = R = \lceil (N+1)/2 \rceil$  — a majority both ways.

Why not  $W = N$ ?

Perumal's trustee in Singapore. **Requiring every replica means any single failure blocks every write.**

```

N = 3, W = 3
 each replica 99.9% available
 P(all three up) = 0.999^3 = 99.7% → ~26 hours/year of blocked writes

N = 3, W = 2
 P(at least 2 up) = 99.9997% → ~1.6 minutes/year

```

**Sixteen thousand times better availability, for one fewer replica. That arithmetic is the answer to “why not just write to all of them?”**

Latency: the quorum is as slow as its slowest member

This is the operational fact people miss.

```

W = 3 of N = 5
→ you wait for the THIRD FASTEST replica to acknowledge
→ not the average, not the fastest

replica latencies: 2 ms, 3 ms, 5 ms, 40 ms, 200 ms
W=1 → 2 ms
W=3 → 5 ms
W=5 → 200 ms ← one slow replica sets the whole latency

```

**Which is why  $W = N$  is bad for latency as well as availability:** you inherit the tail of every replica. A quorum of a majority lets you ignore the slowest  $N - W$  replicas, and **that is the main reason quorums exist at all** — not just fault tolerance, but tail-latency insulation.

**Sloppy quorums and hinted handoff**

Strict quorums require the  $W$  acknowledgements to come from the  $N$  replicas **that own the key**. A **sloppy quorum** relaxes that: if some of the owners are unreachable, write to  $W$  nodes *somewhere* in the cluster, and hand the data back when the owners return.

|        |                                                                                                                                                                                                                        |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| STRICT | W acks from the N designated replicas<br>→ if fewer than W owners are reachable, the write FAILS<br>→ $R + W > N$ still guarantees overlap                                                                             |
| SLOPPY | W acks from ANY W nodes, with hints for the absent owners<br>→ the write SUCCEEDS during a partition<br>→ $R + W > N$ NO LONGER guarantees overlap, because the writers and the readers may be different sets entirely |

**Say that trade explicitly.** Dynamo and Cassandra default to sloppy quorums because availability during a partition is worth more to them than the overlap guarantee — and the guarantee is restored when the hints are replayed. **It is a deliberate weakening, not a bug.**

The uncomfortable truth: quorums are not linearizable

$R + W > N$  guarantees you read a value at least as new as the last completed write. It does not give you **linearizability**. Three specific holes:

**A partial write.** A write reaches two of three replicas and then the client dies. It was never acknowledged, so it is neither committed nor rolled back — and subsequent reads may or may not see it, non-deterministically.

**Concurrent writes.** Two clients write different values at the same time, each reaching a quorum. There is no ordering between them, so which one “wins” depends on the conflict-resolution rule, and different readers may briefly see different answers.

**Read-after-read going backwards.** A read repairs some replicas and not others, so a subsequent read touching a different subset may return an older value — a **monotonic reads** violation.

**The fix for genuine linearizability is a consensus protocol** — Paxos or Raft — which is [day 119](#). Cassandra exposes this as **lightweight transactions**, which use Paxos for a single key and are roughly four times slower than a quorum write.

**Being able to say “a quorum is not linearizability, and here is why” is the strongest single thing you can say about this topic.**

Read repair, and how the system heals

When a read collects  $R$  responses and they disagree, the coordinator:

1. returns the value with the **highest version**;
2. **writes it back** to the replicas that were behind.

Perumal telling the two trustees who had it wrong. **Without it, a stale replica stays stale for every future read that happens to include it**, and the divergence persists indefinitely.

Read repair only fixes keys that are **read**. Cold keys are healed by **anti-entropy** with Merkle trees, and a replica that was down is caught up by **hinted handoff**.

#### 4 The picture

The pigeonhole argument, drawn. **This is the diagram.**

N = 5 replicas: A B C D E

WRITE to W=3: [A] [B] [C] D E

▲ ▲ ▲  
the write set

READ from R=3: A B [C] [D] [E]

▲ ▲ ▲  
the read set

|   |
|---|
| C |
|---|

 ← IN BOTH. Guaranteed.

$3 + 3 = 6 > 5$ , and there are only 5 replicas – so the two sets of three CANNOT be disjoint. There is nowhere for them both to hide.

overlap size =  $R + W - N = 1$

NOW BREAK IT: W = 2, R = 2

WRITE to: [A] [B] C D E

READ from: A B C [D] [E]

← NO OVERLAP. The read misses the write.

$2 + 2 = 4 \leq 5$ . There is room for the two sets to avoid each other entirely.

The trust, as the same picture:



11 trustees. Six sign. You ask six.

signed: ●●●●●●○○○○○

asked: ○○○○○●●●●●●



$6 + 6 = 12 > 11$ , so at least ONE person is in both groups.

if it were FIVE and FIVE:

signed: ●●●●●○○○○○

asked: ○○○○○●●●●●

$5 + 5 = 10 \leq 11$  – and  $11 - 10 = 1$  leaves room for a COMPLETE MISS.  
You could ask five people who were all away, and every one of them would honestly say nothing was decided.

Choosing R and W, and what each costs:

N = 5

| W | R | W+R | overlap | write latency | read latency | tolerates                      |
|---|---|-----|---------|---------------|--------------|--------------------------------|
| - | - | --- | -----   | -----         | -----        | -----                          |
| 1 | 5 | 6   | 1       | fastest       | slowest      | 0 replica failures for reads   |
| 2 | 4 | 6   | 1       | fast          | slow         | 1 write, 1 read failure        |
| 3 | 3 | 6   | 1       | balanced      | balanced     | 2 failures BOTH ways ← default |
| 4 | 2 | 6   | 1       | slow          | fast         | 1 write, 3 read failures       |
| 5 | 1 | 6   | 1       | SLOWEST       | fastest      | 0 failures for writes          |
| 1 | 1 | 2   | X       | fastest       | fastest      | NO guarantee – fully eventual  |

the OVERLAP is the same (1) for every valid row.

what changes is WHERE you pay: writes or reads, latency or availability.

Latency is the slowest member of the quorum:

five replicas respond in: 2 ms · 3 ms · 5 ms · 40 ms · 200 ms

W=1 | 2 ms  
W=3 | 5 ms ← ignores the slow two  
W=5 | 200 ms ← inherits the WORST tail

THIS is the main reason quorums exist: not only fault tolerance, but insulation from the tail latency of the slowest N-W replicas.

Why a quorum is not linearizability:

### 1. PARTIAL WRITE

client writes to A, B → then DIES before the ack  
the write is neither committed nor rolled back  
→ some later reads see it, some do not. Non-deterministic.

### 2. CONCURRENT WRITES

client 1 writes x=5 to {A,B,C}  
client 2 writes x=9 to {C,D,E} at the same moment  
→ no ordering exists between them; the "winner" depends on the conflict rule, and different readers may briefly disagree

### 3. READS GOING BACKWARDS

read 1 touches {A,B,C}, repairs A and B  
read 2 touches {C,D,E} ← D and E are still stale  
→ the second read returns an OLDER value. Monotonic reads violated.

$R + W > N$  gives you "at least as new as the last COMPLETED write".  
It does NOT give you a single global order. For that: Paxos or Raft.

## 5 How it actually works

### The write path

1. the coordinator sends the write to ALL N replicas (not just W)
2. it waits for W acknowledgements
3. it returns success to the client
4. the remaining  $N - W$  acks arrive later, or never

→ the write is SENT to all N; only the ACK count is W  
→ which is why the extra replicas still converge, and why W is about latency and durability rather than about how many copies exist

**That detail matters:**  $W = 2$  of  $N = 3$  does not mean only two copies exist. All three are written to; you simply do not wait for the third.

### The read path

1. the coordinator sends the read to R replicas (often to all N)
2. it collects R responses with their VERSIONS
3. it returns the highest version to the client
4. it WRITES THAT VALUE BACK to any replica that was behind ← read repair

**Step 3 is why versioning is not optional.** The overlap guarantees one replica has the new value; the version is how you recognise which one.

## The version, and what it can and cannot do

```
a simple counter or timestamp
→ tells you which is NEWER
→ cannot distinguish "newer" from "concurrent"
→ last-write-wins, and it SILENTLY LOSES the other write

a vector clock (one counter per replica)
→ distinguishes "A happened before B" from "A and B are CONCURRENT"
→ surfaces genuine conflicts to the application instead of hiding them
→ costs one counter per replica in every record
```

**Dynamo used vector clocks and handed conflicts to the application** — the shopping cart that unions divergent versions. **Cassandra uses last-write-wins timestamps** and accepts the data loss, which is simpler and is why clock skew matters there.

## Sloppy quorums, concretely

```
N = 3 for key K, owned by replicas A, B, C. W = 2.
B and C are unreachable.

STRICT: only A is available → fewer than W → the write FAILS

SLOPPY: write to A, and to D and E (which do NOT own K),
 with HINTS saying "this belongs to B and C"
 → the write SUCCEEDS
 → when B and C return, D and E hand the data over
 → until then, a strict read of {A,B,C} may MISS it
```

**So during the window,  $R + W > N$  does not hold in any meaningful sense** — the writers and readers are different sets. **That is the price of availability, paid knowingly.**

## What real systems do

- **Dynamo, Cassandra, Riak** expose  $N$ ,  $R$  and  $W$  directly, with **QUORUM** as a named level meaning  $\lceil N/2 \rceil + 1$ . Cassandra's default for a serious workload is  $N=3, R=QUORUM, W=QUORUM$ .
- **Cassandra's LOCAL\_QUORUM** is the practical multi-region setting: a majority **within one datacentre**, so you get the overlap guarantee locally without paying a cross-region round trip on every operation. **Naming this is a strong, specific detail.**
- **DynamoDB** hides the knobs and offers two levels: eventually consistent reads (one replica, half the cost) and strongly consistent reads (a quorum, full cost and higher latency).

- **Cassandra’s lightweight transactions** ( IF NOT EXISTS ) use Paxos for genuine linearizability on a single partition, at roughly four times the cost of a quorum write.
- **ZooKeeper, etcd and Raft-based systems** use a majority quorum for **consensus**, not just for overlap — which is the difference between  $R + W > N$  and an actual total order.

### 6 The numbers

The inequality, worked

| N | majority | W=R=majority | W+R   | overlap | tolerates                     |
|---|----------|--------------|-------|---------|-------------------------------|
| 3 | 2        | 2, 2         | 4 > 3 | 1       | 1 failure                     |
| 4 | 3        | 3, 3         | 6 > 4 | 2       | 1 failure ← no better than 3  |
| 5 | 3        | 3, 3         | 6 > 5 | 1       | 2 failures                    |
| 6 | 4        | 4, 4         | 8 > 6 | 2       | 2 failures ← no better than 5 |
| 7 | 4        | 4, 4         | 8 > 7 | 1       | 3 failures                    |

Even **N** buys nothing, exactly as with **leader election**: a majority of 4 is 3, the same as a majority of 3 requires you to tolerate only one failure — and you pay for an extra replica.

Availability, by configuration

|                                                    |                        |                                         |
|----------------------------------------------------|------------------------|-----------------------------------------|
| each replica 99.9% available (8.8 hours/year down) |                        |                                         |
| N=3, W=1                                           | P(≥1 up) = 99.9999999% | ~0.03 s/year                            |
| N=3, W=2                                           | P(≥2 up) = 99.9997%    | ~1.6 minutes/year                       |
| N=3, W=3                                           | P(3 up) = 99.7%        | ~26 hours/year ← WORSE than one machine |
| N=5, W=3                                           | P(≥3 up) = 99.99999%   | ~0.3 s/year                             |

**W = N is worse than a single machine.** That is the arithmetic behind “never require all replicas”, and it is worth stating as a number rather than a principle.

## Latency, by configuration

replica response times: 2, 3, 5, 40, 200 ms (N = 5)

W=1 2 ms the fastest replica

W=2 3 ms

W=3 5 ms ← ignores the slowest two entirely

W=4 40 ms

W=5 200 ms inherits the worst tail

p99 effect: with W=3 of 5, you need three of five to be fast, which is far more likely than all five being fast.

**This is the tail-latency argument for quorums**, and it is the half people forget: a majority quorum does not just tolerate failures, it **routinely ignores the slowest replicas**, which is worth more in day-to-day latency than the fault tolerance is.

## The cost of each configuration

6,000 reads/s, 600 writes/s, N=3

W=2, R=2 each write hits 3 replicas, waits for 2 → 1,800 replica writes/s  
each read hits 2 (or 3) replicas → 12,000-18,000 replica reads/s

W=2, R=1 reads hit 1 replica → 6,000 replica reads/s  
(but NO overlap guarantee: 2 + 1 = 3, not > 3)

**R = 1 halves the read load and gives up the guarantee** — which is exactly the eventual-versus-strong choice, priced in load rather than in latency.

## Cross-region

same-region quorum ~5 ms  
cross-region quorum ~150-300 ms (you wait for a remote replica)

N=6 across 2 regions, W=4 → every write waits for a remote ack: ~150 ms  
N=6, LOCAL\_QUORUM=2 → ~5 ms, with the overlap guarantee only  
WITHIN the region

**That is why LOCAL\_QUORUM exists**, and it is the standard multi-region configuration: local consistency, cross-region eventual.

## Overlap size

overlap = R + W - N

N=5, W=3, R=3 → 1 replica in common (the minimum that works)

N=5, W=4, R=4 → 3 in common

A bigger overlap buys nothing for correctness — one is enough — and costs latency and availability on both sides.  $R + W = N + 1$  is the efficient point.

## 7 The trade-offs

Where you pay: reads or writes

The inequality says the *sum* must exceed  $N$ ; it says nothing about how to split it. So the choice is which side pays.

|                      |                    |                                        |
|----------------------|--------------------|----------------------------------------|
| read-heavy workload  | → low R, high W    | (W=N, R=1: fast reads, brittle writes) |
| write-heavy workload | → low W, high R    |                                        |
| balanced / unknown   | → W = R = majority |                                        |

I would default to majority-majority unless the ratio is extreme, because it tolerates the most failures on both sides and its latency is symmetric.

Strict or sloppy?

**Strict** preserves the overlap guarantee and refuses writes when fewer than  $W$  owners are reachable — which is a CP choice.

**Sloppy** accepts the write anywhere and hands it back later, so writes stay available during a partition — an AP choice, and it **gives up the guarantee** for the duration.

**Dynamo-lineage systems default to sloppy**, and I would say so explicitly rather than let someone assume the inequality always holds. **It is a deliberate weakening.**

Quorum or consensus?

A **quorum gives you overlap**. It is cheap, it needs no leader, and every node can coordinate.

**Consensus gives you a total order**. It costs a leader, an election protocol, and roughly four times the latency for a single operation.

I would use a quorum for data and consensus for decisions — configuration, locks, leadership, unique constraints — which is exactly why Cassandra holds your data and etcd holds your cluster's mind.

## What versioning you choose matters more than R and W

```
timestamps (last-write-wins)
+ simple, one field
- silently loses one of two concurrent writes
- with clock skew, may keep the OLDER one

vector clocks
+ distinguishes concurrent from ordered
+ surfaces conflicts instead of hiding them
- one counter per replica in every record, and the app must resolve
```

“Which conflict-resolution strategy?” is a more consequential question than “which R and W?”, and it is the one candidates skip.

### Where quorums mislead

- **They are not linearizability.** Partial writes, concurrent writes and non-monotonic reads all remain possible.
- **$R + W > N$  says nothing about durability.**  $W = 2$  means two replicas acknowledged; if both are in the same rack and it loses power, the write is gone. **Placement matters as much as the count.**
- **Sloppy quorums quietly suspend the guarantee**, and the system does not tell you when.
- **Bigger is not better.**  $W = N$  is worse than one machine for availability and inherits the worst replica’s latency.

## 8 In the interview

### How it gets asked

- The arithmetic: “You have five replicas. What R and W would you pick?”
- The proof: “Why does  $R + W > N$  work?”
- The trap: “Why not write to all five?”
- The depth probe: “Is that linearizable?”
- The applied one: “How would you configure this across two regions?”

### What to say out loud, in the first ninety seconds

1. **Give the rule and the proof together.** “ $R + W > N$ , and the reason is pigeonhole: if the set I wrote to and the set I read from together exceed the total number of replicas, they must share at least one — and that one has the newest value.”

2. **Give the concrete answer.** “For  $N = 5$  I would take  $W = 3, R = 3$ . Six is greater than five, so there is always an overlap of at least one, and it tolerates two failures on both the read and the write path.”
3. **Say the second half immediately.** “And the overlap is only useful if I can tell which returned value is newest — so every record carries a version, and the reader takes the highest. Without versioning the guarantee is worthless.”
4. **Pre-empt “why not all five”.** “I would not use  $W = 5$ : any single replica being down blocks every write, and at 99.9 percent each that is twenty-six hours a year against about a minute and a half for  $W = 3$ .”
5. **Give the latency reason, which is the underrated one.** “A quorum also insulates me from tail latency — with  $W = 3$  of 5 I wait for the third fastest and ignore the slowest two, so one sick replica does not set my write latency.”
6. **Volunteer the limitation.** “One thing worth saying: a quorum is **not** linearizability. It guarantees I read something at least as new as the last completed write, not a single global order.”

#### The follow-ups

“**Why does  $R + W > N$  work?**” “Pigeonhole. The write went to  $W$  replicas; the read asks  $R$  replicas. If  $R + W > N$ , those two sets cannot be disjoint — there simply are not enough replicas for them to avoid each other — so at least one replica in my read set received the write. With  $N = 5, W = 3, R = 3$ , six exceeds five, so the overlap is at least one. Break it —  $W = 2, R = 2$ , which is four and not more than five — and I can write to A and B and then read from D and E and miss it entirely. And the second half matters just as much: the overlapping replica has the new value, but the others will return old ones, so **every record has to carry a version** and the reader takes the highest. Without versioning I have an overlap I cannot recognise, which is no guarantee at all.”

“**Why not write to all five?**” “Because availability collapses and latency inherits the worst replica. With each replica at 99.9 percent, requiring all three of  $N = 3$  gives about 99.7 percent — roughly twenty-six hours a year of blocked writes, which is **worse than a single machine**. Requiring two gives 99.9997 percent, about a minute and a half a year. That is a factor of sixteen thousand for one fewer acknowledgement. And separately, a quorum’s latency is the latency of its **slowest member**: if five replicas respond in 2, 3, 5, 40 and 200 milliseconds, waiting for three costs 5 milliseconds and waiting for all five costs 200. So a majority quorum is not only fault tolerance — it routinely lets me **ignore the slowest replicas**, which is worth more in day-to-day p99 than the failure tolerance is.”



**“Is that linearizable?”** “No, and this is the most important thing to know about quorums.  $R + W > N$  guarantees I read a value at least as new as the last **completed** write — it does not give a single global order, and there are three concrete holes. **Partial writes**: a client writes to two of three replicas and then dies before the acknowledgement, so the write is neither committed nor rolled back, and subsequent reads may or may not see it, non-deterministically. **Concurrent writes**: two clients each reach a quorum with different values at the same moment; there is no ordering between them, so which one wins depends on the conflict-resolution rule and different readers can briefly disagree. And **reads going backwards**: a read that repairs some replicas but not others means a later read touching a different subset can return an older value, which violates monotonic reads. For genuine linearizability you need a consensus protocol — Paxos or Raft — and Cassandra exposes exactly that as lightweight transactions, at roughly four times the cost of a quorum write.”

**“How would you configure this across two regions?”** “Not with a global quorum, because every write would wait for a cross-region acknowledgement — a hundred and fifty to three hundred milliseconds instead of five. The standard answer is **LOCAL\_QUORUM**: a majority **within one datacentre**, so I get the overlap guarantee locally at local latency, and cross-region replication is asynchronous and eventual. So with  $N = 6$  split three and three, I would use a local quorum of two rather than a global quorum of four. What that costs is honest and worth stating: a reader in the other region may not see a write for the replication lag, and if a whole region is lost, the writes that had not yet crossed are gone. That is the same trade as **synchronous versus asynchronous replication** — cross-region synchronous is physics-limited and essentially nobody does it.”

**“What is a sloppy quorum?”** “A strict quorum requires the  $W$  acknowledgements to come from the  $N$  replicas that actually **own** the key. If fewer than  $W$  of those owners are reachable, the write fails — a CP choice. A **sloppy** quorum relaxes it: write to any  $W$  nodes in the cluster, with **hints** recording which owners the data really belongs to, and hand it over when they return. The write succeeds during a partition, which is an AP choice, and the cost is that  $R + W > N$  **no longer guarantees an overlap** for the duration — the writers and the readers may be entirely different sets. Dynamo-lineage systems default to this because availability during a partition is worth more to them than the guarantee, and the guarantee is restored when hinted handoff replays. I would call it out explicitly, because it is easy to quote the inequality and not notice that the system you are using has suspended it.”

**“How does the system heal after a stale read?”** “Three mechanisms, and they cover different data. **Read repair**: when the coordinator collects  $R$  responses and they disagree, it returns the highest version **and writes it back** to the replicas that were

behind — so any key that gets read converges within milliseconds. That is the trust register telling the two trustees who had it wrong, and without it the same replicas would keep giving the same stale answer indefinitely. **Hinted handoff**: writes accepted on behalf of a node that was down are replayed when it returns. And **anti-entropy**: a background pass comparing Merkle trees to find which key ranges differ in  $O(\log n)$  exchanges, which is the only thing that fixes **cold** keys — the ones no read ever touches. So the convergence time is bimodal: milliseconds for hot keys, and the anti-entropy interval for everything else.”

#### A model answer

Asked: *you have five replicas — what R and W would you pick?*

“ $W = 3$  and  $R = 3$ ”, and let me give the rule and the reason together, because the reason is the whole answer.

The rule is  $R + W > N$ . Here that is three plus three is six, which is greater than five. And the reason it works is pigeonhole: the write went to three replicas, the read asks three replicas, and there are only five in total — so those two sets of three **cannot avoid each other**. At least one replica I read from received the write. If I dropped to  $W = 2$ ,  $R = 2$ , that is four, which is not more than five, and I could write to A and B and then read from D and E and miss the write entirely.

There is a second half that is just as necessary and that people leave out. The overlapping replica has the new value, but the other two will return old ones — so **every record must carry a version**, and the reader takes the highest. Without versioning I have an overlap I cannot identify, which is no guarantee at all. And what kind of version matters: a plain timestamp gives last-write-wins, which silently discards one of two concurrent writes and, with clock skew, may keep the older one. A vector clock distinguishes ‘newer’ from ‘concurrent’ and surfaces the conflict instead of hiding it, at the cost of a counter per replica in every record.

Three-and-three is also the balanced choice: it tolerates **two failures on both the read and the write path**, which is the most you can get from five, and its latency is symmetric.

I would not use  $W = 5$ , and the arithmetic is stark. With each replica at 99.9 percent available, requiring all of them gives you availability **worse than a single machine** — for  $N = 3$ ,  $W = 3$  that is about twenty-six hours a year of blocked writes against ninety seconds for  $W = 2$ . And there is a latency reason as well, which is the one people miss: a quorum’s latency is the latency of its **slowest member**. If five replicas respond in 2, 3, 5, 40 and 200 milliseconds,  $W = 3$  costs five milliseconds and  $W = 5$  costs two hundred. So a majority quorum is not only fault tolerance — it routinely lets me **ignore the slowest replicas**, which does more for my p99 than the fault tolerance does.

The last thing I would say unprompted: **this is not linearizability**.  $R + W > N$  gives me a value at least as new as the last completed write; it does not give a single global order. A partial write from a client that died is neither committed nor rolled back; two concurrent writes have no ordering between them; and successive reads can go backwards if one repaired some replicas and the next touches

others. If I need a genuine total order — for a lock, a unique username, or a leader — that is consensus, Paxos or Raft, and it costs several times a quorum write.”

## 9 Recall card

- $R + W > N$ , and the proof is **PIGEONHOLE**: the write set and the read set together exceed the number of replicas, so they **must share at least one** — and that replica has the newest value.  $N=5, W=3, R=3 \rightarrow 6 > 5 \checkmark$ .  $W=2, R=2 \rightarrow 4 \leq 5 \times$ , and you can write {A,B} then read {D,E} and miss it entirely.
- **The overlap is useless without VERSIONS** — every record carries one and the reader takes the highest. A timestamp gives last-write-wins (**silently loses a concurrent write**, and with clock skew keeps the *older*); a **vector clock** distinguishes concurrent from ordered and surfaces the conflict.
- **Never  $W = N$** . At 99.9% per replica,  $N=3, W=3$  gives **~26 hours/year** of blocked writes — *worse than one machine* — against **~1.6 minutes** for  $W=2$ . And a **quorum's latency is its SLOWEST member**: with responses of 2/3/5/40/200 ms,  $W=3$  costs 5 ms and  $W=5$  costs 200. **Tail-latency insulation is the underrated half of why quorums exist.**
- **A QUORUM IS NOT LINEARIZABILITY**. It guarantees a value at least as new as the last *completed* write, not a global order. Three holes: a **partial write** from a dead client is neither committed nor rolled back; **concurrent writes** have no ordering; and **reads can go backwards** after partial read repair. For a total order you need **consensus** (Paxos/Raft) — Cassandra's lightweight transactions,  $\sim 4 \times$  the cost.
- **A SLOPPY quorum accepts  $W$  acks from ANY nodes with hints for the absent owners** — the write survives a partition and the inequality's guarantee is **suspended** until hints replay. Dynamo-lineage systems default to this. Healing is **read repair** (hot keys, milliseconds) · **hinted handoff** (a returning node) · **anti-entropy with Merkle trees** (cold keys). Across regions use **LOCAL\_QUORUM**:  $\sim 5$  ms instead of  $\sim 150\text{--}300$  ms, with the guarantee holding only within the region. **Even  $N$  buys nothing.**

## Practice — Merging K sorted lists

**DSA topic:** Merging K sorted lists **System design topic:** Quorums: why R plus W must exceed N

Code these, in this order

One rule for the whole set: **say the invariant before writing** — *each list is sorted, so the overall smallest must be at the front of some list, and the heap holds all  $k$  fronts*. That sentence is the correctness proof and it takes ten seconds.

| # | PROBLEM                                       | SOURCE                | WHAT IT IS REALLY TESTING                                 |
|---|-----------------------------------------------|-----------------------|-----------------------------------------------------------|
| 1 | Merge Two Sorted Lists                        | LeetCode 21 (Easy)    | The primitive the tournament is built from.               |
| 2 | Merge k Sorted Lists                          | LeetCode 23 (Hard)    | The heap of $k$ fronts, and both $O(n \log k)$ solutions. |
| 3 | Kth Smallest Element in a Sorted Matrix       | LeetCode 378 (Medium) | A k-way merge that stops after $k$ pops.                  |
| 4 | Smallest Range Covering Elements from K Lists | LeetCode 632 (Hard)   | The same heap plus one extra variable.                    |

On problem 2, write all four approaches

Heap, tournament, sequential pairwise, and concatenate-and-sort. Time all four at  $n = 100,000$  with  $k = 100$ . Record four numbers and say which two are  $O(n \log k)$ .

On problem 2 again, break the tuple

Push `(node.val, node)` with two nodes of equal value and record the exact `TypeError`. Then say what the natural tie-breaker is here and why it is free.

On problem 3, notice what changes

Say what makes this a k-way merge, what  $k$  is, and why stopping after  $k$  pops changes the complexity from  $O(n \log \text{rows})$  to  $O(k \log \text{rows})$ .

On problem 4, say what you added

Compare it line by line with problem 2. Say exactly which variable is new and why advancing the minimum is the only move that can shrink the range.

### The invariant drill

1. State the invariant in one sentence.
2. Give the one-sentence proof.
3. Say what would break it if a list were not sorted.
4. Say why this makes the heap size  $k$  rather than  $n$ .

### The complexity drill

1. Derive  $O(n \log k)$  from the pops and the heap size.
2. Compute the comparison counts at  $n = 1,000,000$  for  $k = 2, 10, 100$  and  $10,000$ .
3. Say at what  $k$  the heap stops being worth it, and what to do instead.
4. State the space complexity and say why it is the more important number.

### The four-approaches drill

1. Name all four and give time, space, streaming and whether each uses the sorted-ness.
2. Identify the two at  $O(n \log k)$ .
3. Say which one is the trap and why it looks correct.
4. Say which one throws away the only interesting property of the input.

### The tournament drill

1. Describe it in two sentences.
2. Say why it is  $O(n \log k)$ , in terms of rounds and work per round.
3. Give three advantages over the heap.
4. Give its disadvantage, and the input type that removes it.
5. Say when you would choose it.

### The tuple drill

1. Write the heap entry from memory and name all three fields.
2. Say what each field is for.
3. Say which field doubles as the tie-breaker and why it is always safe here.
4. Construct the `TypeError` you get without it.

### The external-sort drill

1. Describe both passes of an external merge sort.
2. Say which pass is today's algorithm.
3. Say what is held in memory during that pass.

4. Say why merge is the disk world's sort and quicksort is memory's.
5. Name two production systems that do this.

#### The break-it drill

Trigger each and record the exact output or error:

1. A heap containing all  $n$  items instead of  $k$ .
2. No `if lst` guard when building the initial heap, with `[[], [1,2]]`.
3. `(node.val, node)` with two equal values.
4. Advancing list 0 instead of the list the value came from.
5. Pushing without the bound check when a list is exhausted.
6. Sequential pairwise merging, timed at  $k = 100$ .
7. Returning a list rather than yielding, on input larger than memory.

#### The inequality drill

1. State the rule and give the one-sentence pigeonhole proof.
2. For  $N = 5$ , list every  $(W, R)$  pair that satisfies it with  $W + R = N + 1$ .
3. Compute the overlap size formula and apply it to two configurations.
4. Construct a concrete miss for  $N = 5, W = 2, R = 2$ .

#### The versioning drill

1. Say why the overlap is useless without versions.
2. Compare timestamps against vector clocks on three axes.
3. Say what last-write-wins does under clock skew.
4. Say which question is more consequential than choosing  $R$  and  $W$ .

#### The configuration drill

1. Give the default for  $N = 3$  and for  $N = 5$ .
2. Give a read-heavy configuration and a write-heavy one.
3. Say what  $W = 1, R = 1$  gives you and what it gives up.
4. Say why even values of  $N$  buy nothing.

#### The availability drill

Compute, at 99.9% per replica:

1.  $N=3, W=1$ ,  $N=3, W=2$ ,  $N=3, W=3$ ,  $N=5, W=3$ .

2. The ratio between  $W=2$  and  $W=3$  for  $N=3$ .
3. Say which configuration is worse than a single machine, and by how much.

#### The latency drill

1. Given replica responses of 2, 3, 5, 40 and 200 ms, give the latency for  $W = 1$  through  $W = 5$ .
2. State the general rule in one sentence.
3. Say why this is the underrated reason quorums exist.
4. Compute the cross-region cost and name the setting that avoids it.

#### The not-linearizable drill

1. State what  $R + W > N$  actually guarantees.
2. Describe all three holes with a concrete scenario each.
3. Say what you need instead for a genuine total order.
4. Name the Cassandra feature that provides it and its cost.

#### The sloppy-quorum drill

1. Define strict and sloppy.
2. Say what sloppy gains and what it suspends.
3. Say which systems default to it and why.
4. Say when the guarantee is restored.

#### The healing drill

1. Name all three mechanisms.
2. For each, say what triggers it and which keys it reaches.
3. Say which one fixes cold keys and why nothing else does.
4. Say what the trust register's equivalent of read repair was.

#### The failure drill

For each, say what happens and what you would configure:

1. One of five replicas is permanently slow but not down.
2. A client writes to two of three replicas and then crashes.
3. Two clients write different values to overlapping quorums simultaneously.
4. A read repairs A and B; the next read touches C, D and E.
5. Two of three replicas acknowledging a write are in the same rack, which loses power.



6. A global quorum is configured across two regions.

Two of the six are not fixed by adjusting  $R$  and  $W$ . Name them.

---

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Merge  $k$  sorted lists efficiently.* The invariant stated as the proof, the heap holding  $k$  and not  $n$  with the complexity derived from it, the tuple's three fields with the free tie-breaker, the tournament offered as the second  $O(n \log k)$ , the sequential trap named, and external sorting as the real use.
  2. *You have five replicas. What  $R$  and  $W$  would you pick?* Three and three with the pigeonhole proof, versioning named as the necessary second half,  $W = N$  rejected with the availability arithmetic, the tail-latency argument, and the admission that a quorum is not linearizability.
  3. *Is a quorum linearizable?* No, what it actually guarantees, all three holes with a scenario each, and what you need instead.
- 

### Before you move on

- ☐ I can state the merge invariant and its one-sentence proof.
- ☐ I can derive  $O(n \log k)$  from the heap size.
- ☐ I know why  $O(k)$  space matters more than the time bound.
- ☐ I can name all four approaches and identify the trap.
- ☐ I can describe the tournament and say when I would choose it.
- ☐ I write the three-field tuple by habit and know why the index is a free tie-breaker.
- ☐ I skip empty lists when building the initial heap.
- ☐ I can describe both passes of an external merge sort.
- ☐ I can adapt the merge to smallest-range and to a sorted matrix.
- ☐ I can state  $R + W > N$  and give the pigeonhole proof.
- ☐ I can construct a concrete miss when the inequality fails.
- ☐ I always mention versioning as the necessary second half.

- [ ] I can compute availability for four configurations and spot the one worse than a single machine.
- [ ] I can give the tail-latency argument with numbers.
- [ ] I know that a quorum is not linearizability, and all three reasons.
- [ ] I can define a sloppy quorum and say what it suspends.
- [ ] I can name all three healing mechanisms and which keys each reaches.
- [ ] I know what `LOCAL_QUORUM` is for and what it costs.
- [ ] I answered all three questions above out loud.

# Two heaps: the running median

## DATA STRUCTURES AND ALGORITHMS

### Two heaps: the running median

You can keep a stream balanced across two heaps and read the median in  $O(1)$ .

## SYSTEM DESIGN

### Leader election

You can say why a system needs one leader and how it picks a new one.

## THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the median of a stream of numbers.*
- *The leader crashed. How does the cluster choose the next one?*

## 1 What this is, and why they ask it

Numbers arrive one at a time and after each one you must be able to say the **median** — the middle value if the numbers were sorted.

Three sentences. Sorting after every arrival is  $O(n \log n)$  per query and hopeless; the trick is to keep the numbers **already split into a lower half and an upper half**, so the median is at the boundary and costs nothing to read. Each half is a heap, and they face each other: a **max-heap** holding the lower half so its largest is at the top, and a **min-heap** holding the upper half so its smallest is at the top — **the two elements at the boundary are exactly the ones you need**. And the only real work is keeping the two halves the same size, which is two lines and is where every bug in this problem lives.

They ask it because it is the clearest example of a technique that is otherwise hard to motivate: **maintain an invariant incrementally rather than recomputing an answer**. It is also the shape of a small family — the sliding-window median, scheduling problems, “IPO”-style greedy selection — where two heaps facing each other answer a question neither could answer alone.

## 2 The story

Mrs Kurian took the fourth standard for PT and every year in November there was a photograph, and every year she needed the children in two lines of equal length, shorter children in one and taller in the other.

The complication was that they never all arrived at once. They came out of the classroom in ones and twos over about ten minutes, and the boy at the front of each line kept changing.

The way she had done it in her first year was to line everybody up in full height order and then cut the line in half. That worked and it meant re-sorting forty children every time one more came out of the door.

By her third year she did something else.

She kept **two lines from the start**. Left line for the shorter half, right line for the taller half. And she kept each line arranged so that the child nearest her was the extreme one — in the left line, the **tallest** of the short children stood at the front; in the right line, the **shortest** of the tall children stood at the front.

So the two children standing in front of her, one from each line, were always the two in the middle of the whole class.

When a new child came out, she did two things and never more.

She looked at the child against the front of the left line. Shorter or equal — join the left line. Taller — join the right line. Either way the child worked their way to the correct position within their own line, which took a few steps, not forty.

Then she counted. If one line was now two longer than the other, she took the front child of the longer line and moved them to the front of the shorter one. One child, every time, never two.

Her student teacher that year asked what happened if a child joined the left line but was actually taller than half the right line.

Mrs Kurian said it could not happen, because she had checked against the front of the left line first — and the front of the left line is the tallest short child. Anybody shorter than him belongs on the left. The check is against the boundary, not against the whole line.

And she said the counting was the part people got wrong. If you do not move a child across, the lines drift apart, and after twenty children one line has fifteen and the other has five, and the two children in front of you are not the middle of anything.

---

### 3 The idea in plain English

Mrs Kurian's two lines are two heaps facing each other, and both of her rules — check against the boundary, then rebalance by one — are the algorithm.

- The left line is a **max-heap** of the lower half: its **largest** is at the top.
- The right line is a **min-heap** of the upper half: its **smallest** is at the top.
- The two children in front of her are the two middle values, so the **median is at the boundary**.
- “Check against the front of the left line” is the routing rule.
- “Move one child across” is the **rebalance**, and it is exactly one move.

## The two invariants

Everything follows from these, and stating them is most of the answer.

1. ORDER    every value in the low heap  $\leq$  every value in the high heap
2. BALANCE    the two sizes differ by at most 1

Given both, the median is free:

```
odd total → the larger heap's top
even total → the average of the two tops
```

**O(1)**. No sorting, no scanning — the answer is sitting at the boundary because the invariants put it there.

The insert, in three steps

```
heapq.heappush(low, -value) # 1. always push here
heapq.heappush(high, -heapq.heappop(low)) # 2. move the top across
if len(high) > len(low): # 3. rebalance if needed
 heapq.heappush(low, -heapq.heappop(high))
```

Three lines, and they are worth reading carefully because they look strange.

**Step 1 — push into `low` unconditionally.** Not “compare and choose”. Pushing to `low` regardless is correct because step 2 fixes it.

**Step 2 — move `low`’s top to `high`.** This is what guarantees invariant 1. Whatever the new value was, after this move the largest thing in `low` is genuinely  $\leq$  the smallest thing in `high`, because the largest candidate was just handed over.

**Step 3 — restore the balance.** After steps 1 and 2, `low` has lost one and `high` has gained one, so `high` may now be one too big. One move back fixes it.

**This “push, shift, rebalance” form is worth memorising as a unit** — it avoids all the branching of the version that compares first, and there is no case analysis to get wrong.

The alternative version compares first:

```
if not low or value ≤ -low[0]:
 heapq.heappush(low, -value)
else:
 heapq.heappush(high, value)
then rebalance whichever side is too big
```

Both are correct. **The first is shorter and has no branches; the second is easier to explain.** Say which one you are writing and why.

The max-heap, which Python does not have

From [day 115](#): `heapq` is min-only, so the lower half is stored **negated**.

```
heapq.heappush(low, -value) # store the negation
-low[0] # read back with another negation
```

Every value that goes into `low` is negated, and every value that comes out is negated again. Getting one of the two negations wrong is the single most common bug here, and it produces a plausible-looking median that is quietly wrong.

Say the convention out loud as you write it: “`low` holds negated values, so `-low[0]` is the largest number in the lower half.”

Reading the median

```
if len(low) > len(high):
 return -low[0] # odd count: the extra one is in low
return (-low[0] + high[0]) / 2 # even count: average the two tops
```

Two conventions exist for where the extra element goes when the count is odd — `low` or `high` — and either is fine as long as the median function agrees with the rebalance rule. **Pick one and be consistent**; mixing them gives an answer that is right for even counts and wrong for odd ones.

Why not the obvious alternatives

|                                                     |                                         |                                |
|-----------------------------------------------------|-----------------------------------------|--------------------------------|
| sort after every insert                             | $O(n \log n)$ per query                 | → $O(n^2 \log n)$ overall      |
| keep a sorted list, insert with <code>bisect</code> | $O(\log n)$ to FIND, $O(n)$ to SHIFT    | → $O(n)$ per insert            |
| a balanced BST with subtree sizes                   | $O(\log n)$ insert AND $O(\log n)$ k-th | → more general, much more code |
| TWO HEAPS                                           | $O(\log n)$ insert, $O(1)$ median       |                                |

The sorted-list version is the one worth discussing, because it is what people reach for and it is  $O(n)$  per insert: `bisect` finds the position in  $O(\log n)$  and then `list.insert` shifts everything after it.

```
n = 100,000 insertions
sorted list: ~2,500,000,000 element moves
two heaps: ~1,700,000 comparisons
```

**And the BST answer is worth naming:** an order-statistic tree gives you the  $k$ -th element for any  $k$ , not just the middle. **Two heaps are better when you only ever want the median**, because they are twenty lines rather than two hundred.

The variants

**Sliding-window median** — LeetCode 480. The same two heaps, plus **removal**, which a heap cannot do. The standard answer is **lazy deletion** from [day 115](#): keep a dictionary of values scheduled for removal, and discard them when they surface at a top.

**The catch is the size accounting.** Lazily deleted elements are still physically in the heaps, so you must track the **logical** sizes separately from `len(heap)` — and that is where the bugs are.

**Any percentile, not just the median.** Change the balance rule: to track the 90th percentile, keep the heaps at a 9:1 ratio instead of 1:1. **Same structure, one different comparison.**

**“IPO” / maximise capital** — LeetCode 502. A min-heap of projects by cost and a max-heap of affordable projects by profit: move everything affordable from the first to the second, then take the best. **Two heaps facing each other again, with a different question.**

**Meeting rooms II, task scheduling** — a heap of end times against a stream of start times, which is the same “one heap holds candidates, another holds committed” shape.

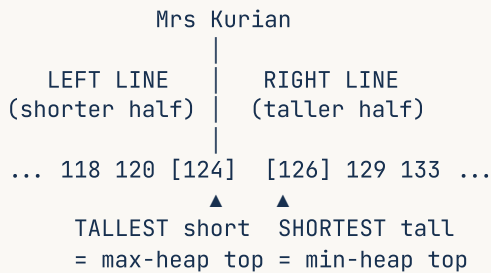
**The unifying idea worth stating:** *two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.*

---

#### 4 The picture

The two lines, which is the data structure.





THE TWO CHILDREN IN FRONT OF HER ARE THE TWO MIDDLE VALUES.

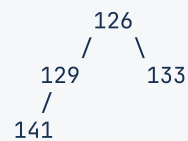
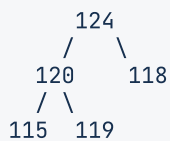
odd total → the front of whichever line is longer  
 even total → the average of the two fronts

$O(1)$ . The invariants put the answer at the boundary.

The two heaps, drawn as heaps:

LOW – a MAX-heap (stored negated in Python)

HIGH – a MIN-heap



top = 124 (the LARGEST of the lower half)

top = 126 (the SMALLEST of the upper half)

INVARIANT 1: everything in LOW  $\leq$  everything in HIGH

INVARIANT 2:  $|\text{len}(\text{LOW}) - \text{len}(\text{HIGH})| \leq 1$

median (7 values, odd) → 124, the top of the larger heap

median (8 values, even) →  $(124 + 126) / 2$

The insert, step by step:

inserting 122 into LOW={115,118,119,120,124} HIGH={126,129,133}

STEP 1 push into LOW unconditionally  
LOW = {115,118,119,120,122,124} top = 124  
HIGH = {126,129,133}  
→ sizes 6 and 3. Unbalanced, and that is fine for now.

STEP 2 move LOW's top to HIGH  
LOW = {115,118,119,120,122} top = 122  
HIGH = {124,126,129,133} top = 124  
→ INVARIANT 1 is now guaranteed: the largest candidate was handed over, so  $\max(\text{LOW}) \leq \min(\text{HIGH})$

STEP 3 rebalance if HIGH is bigger  
sizes 5 and 4 → no move needed

median → LOW is larger, so  $-\text{low}[0] = 122$

WHY PUSH TO LOW BLINDLY? Because step 2 repairs it. There is no case analysis and no branch to get wrong.

Why the rebalance is not optional:

WITHOUT step 3, inserting an increasing sequence:

|          |         |                |
|----------|---------|----------------|
| after 1: | LOW={1} | HIGH={}        |
| after 2: | LOW={1} | HIGH={2}       |
| after 3: | LOW={1} | HIGH={2,3}     |
| after 4: | LOW={1} | HIGH={2,3,4}   |
| after 5: | LOW={1} | HIGH={2,3,4,5} |

▲                      ▲  
the "median" reads as 1 or  $(1+2)/2$  – and the real median is 3

the boundary has DRIFTED away from the middle.

Mrs Kurian: "after twenty children one line has fifteen and the other five, and the two children in front of you are not the middle of anything."

The negation, which is where the bugs are:

Python has no max-heap, so LOW stores NEGATED values.

|       |                               |                                                      |
|-------|-------------------------------|------------------------------------------------------|
| push: | heappush(low, -value)         | store the negation                                   |
| peek: | -low[0]                       | negate again to read it                              |
| move: | heappush(high, -heappop(low)) | ONE negation, because it is leaving the negated heap |

TWO negations on the way in and out; ONE when moving across.  
Get any of them wrong and you get a plausible number that is WRONG.

The family, all the same shape:

|                                                                                                                                    |                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| RUNNING MEDIAN                                                                                                                     | LOW: max-heap of the small half<br>HIGH: min-heap of the large half<br>→ the boundary IS the answer                                             |
| SLIDING WINDOW<br>MEDIAN                                                                                                           | the same, plus LAZY DELETION for elements leaving the window – and LOGICAL sizes tracked separately from len(heap)                              |
| ANY PERCENTILE                                                                                                                     | the same, with the balance ratio changed (9:1 instead of 1:1 for the 90th)                                                                      |
| IPO / MAX CAPITAL                                                                                                                  | LOW: min-heap of projects by COST<br>HIGH: max-heap of affordable projects by PROFIT<br>→ move everything affordable across, then take the best |
| THE UNIFYING IDEA: two heaps facing each other maintain a BOUNDARY in a changing set, and the boundary is what you were asked for. |                                                                                                                                                 |

## 5 The code, built step by step

### Step 1 — state both invariants before writing anything

“I keep the numbers split into a lower half and an upper half. Two invariants: everything in the low half is at most everything in the high half, and their sizes differ by at most one. Given those, the median is at the boundary and costs nothing to read.”

### Step 2 — say which heap is which, and why

“The **low** half is a **max**-heap, so its largest — the biggest of the small numbers — is at the top. The **high** half is a **min**-heap, so its smallest is at the top. Those two are exactly the middle values.”

### Step 3 — the negation convention, said out loud

```
heapq.heappush(low, -value)
```

“Python’s `heapq` is min-only, so the low heap stores negated values. `-low[0]` is the largest number in the lower half. Two negations in and out; that is where the bugs are.”

### Step 4 — push, shift, rebalance — as one unit

```
heapq.heappush(low, -value)
heapq.heappush(high, -heapq.heappop(low))
if len(high) > len(low):
 heapq.heappush(low, -heapq.heappop(high))
```

“Push into low unconditionally, move low’s top across — which is what guarantees the ordering — then move one back if high has become too big. Three lines, no branches, no case analysis.”

#### Step 5 — the median, matching the convention

```
if len(low) > len(high):
 return float(-low[0])
return (-low[0] + high[0]) / 2
```

“The rebalance rule lets `low` be one larger, so on an odd count the extra element is in `low` and that is the median. The median function has to agree with the rebalance rule — if they disagree you get the right answer for even counts and the wrong one for odd.”

## The complete solution

```
import heapq
from collections import defaultdict

class MedianFinder:
 """LeetCode 295. The running median in $O(\log n)$ insert, $O(1)$ query.

 TWO INVARIANTS, and everything follows from them:
 1. ORDER every value in `low` $\leq$ every value in `high`
 2. BALANCE $|\text{len}(\text{low}) - \text{len}(\text{high})| \leq 1$, with `low` allowed to be
 the larger one

 `low` is a MAX-heap of the smaller half (negated, since heapq is min-only)
 `high` is a MIN-heap of the larger half

 So the two heap tops are the two middle values, and the median is free.
 """

 def __init__(self) → None:
 self.low: list[int] = [] # max-heap, values stored NEGATED
 self.high: list[int] = [] # min-heap, values stored as-is

 def add(self, value: int) → None:
 """PUSH, SHIFT, REBALANCE – three lines, no branches.

 Pushing into `low` unconditionally is safe because the shift repairs
 it: whatever the new value was, moving low's top to high guarantees
 $\text{max}(\text{low}) \leq \text{min}(\text{high})$, because the largest candidate was just handed
 over.
 """
 heapq.heappush(self.low, -value) # 1. push
 heapq.heappush(self.high, -heapq.heappop(self.low)) # 2. shift
 if len(self.high) > len(self.low): # 3. rebalance
 heapq.heappush(self.low, -heapq.heappop(self.high))

 def median(self) → float:
 """ $O(1)$. The invariants have already put the answer at the boundary.

 This MUST agree with the rebalance rule about which heap holds the
 extra element on an odd count – here, `low`.
 """
 if not self.low:
 raise ValueError("no values yet")
 if len(self.low) > len(self.high):
 return float(-self.low[0]) # odd: the extra is in low
 return (-self.low[0] + self.high[0]) / 2 # even: average the tops

 def __len__(self) → int:
 return len(self.low) + len(self.high)

 def check_invariants(self) → bool:
 """Assert both invariants directly – run this in tests, because every
 bug in this problem breaks one of them silently."""
 if abs(len(self.low) - len(self.high)) > 1:
 return False
 if self.low and self.high and -self.low[0] > self.high[0]:
 return False
 return True
```

```

class MedianFinderBranching:
 """The same thing, written with an explicit comparison instead of the
 shift. Correct, easier to explain, more branches to get wrong.

 Say which version you are writing and why."""

 def __init__(self) → None:
 self.low: list[int] = []
 self.high: list[int] = []

 def add(self, value: int) → None:
 if not self.low or value ≤ -self.low[0]:
 heapq.heappush(self.low, -value)
 else:
 heapq.heappush(self.high, value)

 # rebalance: low may be at most one larger
 if len(self.low) > len(self.high) + 1:
 heapq.heappush(self.high, -heapq.heappop(self.low))
 elif len(self.high) > len(self.low):
 heapq.heappush(self.low, -heapq.heappop(self.high))

 def median(self) → float:
 if len(self.low) > len(self.high):
 return float(-self.low[0])
 return (-self.low[0] + self.high[0]) / 2

class PercentileTracker:
 """The generalisation: change the BALANCE RATIO and you track any
 percentile, not just the median.

 p = 0.5 → the median (1:1). p = 0.9 → the 90th percentile (9:1).
 Same structure, one different comparison.
 """

 def __init__(self, p: float = 0.5) → None:
 self.p = p
 self.low: list[int] = [] # max-heap, negated
 self.high: list[int] = [] # min-heap

 def add(self, value: int) → None:
 heapq.heappush(self.low, -value)
 heapq.heappush(self.high, -heapq.heappop(self.low))
 # low should hold about p of the total
 total = len(self.low) + len(self.high)
 want_low = max(1, round(self.p * total))
 while len(self.low) < want_low:
 heapq.heappush(self.low, -heapq.heappop(self.high))
 while len(self.low) > want_low:
 heapq.heappush(self.high, -heapq.heappop(self.low))

 def value(self) → float:
 return float(-self.low[0])

class SlidingWindowMedian:
 """LeetCode 480. Two heaps plus LAZY DELETION, because a heap cannot
 remove an arbitrary element.

 THE HARD PART is the size accounting: lazily removed elements are still

```

physically in the heaps, so the LOGICAL sizes must be tracked separately from len(heap). Every bug in this problem is there.

```
"""
def __init__(self) → None:
 self.low: list[int] = [] # max-heap, negated
 self.high: list[int] = [] # min-heap
 self.pending: dict[int, int] = defaultdict(int) # value → count to drop
 self.low_size = 0 # LOGICAL sizes
 self.high_size = 0

def _prune(self, heap: list[int], is_low: bool) → None:
 """Discard scheduled-for-removal values sitting at the top."""
 while heap:
 value = -heap[0] if is_low else heap[0]
 if self.pending[value] > 0:
 self.pending[value] -= 1
 heapq.heappop(heap)
 else:
 break

def _rebalance(self) → None:
 if self.low_size > self.high_size + 1:
 self._prune(self.low, True)
 heapq.heappush(self.high, -heapq.heappop(self.low))
 self.low_size -= 1
 self.high_size += 1
 elif self.low_size < self.high_size:
 self._prune(self.high, False)
 heapq.heappush(self.low, -heapq.heappop(self.high))
 self.high_size -= 1
 self.low_size += 1
 self._prune(self.low, True)
 self._prune(self.high, False)

def add(self, value: int) → None:
 if not self.low or value ≤ -self.low[0]:
 heapq.heappush(self.low, -value)
 self.low_size += 1
 else:
 heapq.heappush(self.high, value)
 self.high_size += 1
 self._rebalance()

def remove(self, value: int) → None:
 self.pending[value] += 1 # LAZY: mark, do not search
 if self.low and value ≤ -self.low[0]:
 self.low_size -= 1
 if value == -self.low[0]:
 self._prune(self.low, True)
 else:
 self.high_size -= 1
 if self.high and value == self.high[0]:
 self._prune(self.high, False)
 self._rebalance()

def median(self) → float:
 if self.low_size > self.high_size:
 return float(-self.low[0])
 return (-self.low[0] + self.high[0]) / 2
```

```

def median_sliding_window(nums: list[int], k: int) → list[float]:
 """The window walked across the array."""
 window = SlidingWindowMedian()
 out: list[float] = []
 for i, value in enumerate(nums):
 window.add(value)
 if i ≥ k:
 window.remove(nums[i - k])
 if i ≥ k - 1:
 out.append(window.median())
 return out

def find_maximized_capital(k: int, capital: int,
 profits: list[int], costs: list[int]) → int:
 """LeetCode 502 – two heaps facing each other with a DIFFERENT question.

 LOW: a min-heap of projects by COST (what might become affordable)
 HIGH: a max-heap of affordable projects by PROFIT (what to do next)

 Each round: move everything now affordable from LOW to HIGH, then take
 HIGH's best. Same structure, different boundary.
 """
 by_cost = list(zip(costs, profits))
 heapq.heapify(by_cost) # min-heap on cost
 affordable: list[int] = [] # max-heap on profit (negated)

 for _ in range(k):
 while by_cost and by_cost[0][0] ≤ capital:
 cost, profit = heapq.heappop(by_cost)
 heapq.heappush(affordable, -profit)
 if not affordable:
 break
 capital += -heapq.heappop(affordable)
 return capital

class MedianBySortedList:
 """The alternative people reach for. $O(\log n)$ to FIND the position and
 $O(n)$ to SHIFT – so $O(n)$ per insert.

 n = 100,000 insertions: ~2.5 billion element moves against ~1.7 million
 heap comparisons.
 """

 def __init__(self) → None:
 self.data: list[int] = []

 def add(self, value: int) → None:
 import bisect
 bisect.insort(self.data, value) # $O(\log n)$ find, $O(n)$ shift

 def median(self) → float:
 n = len(self.data)
 if n % 2:
 return float(self.data[n // 2])
 return (self.data[n // 2 - 1] + self.data[n // 2]) / 2

if __name__ == "__main__":
 mf = MedianFinder()
 for v in (5, 15, 1, 3):

```



```

 mf.add(v)
 print(f"add {v:>3} → median {mf.median():>6.1f} "
 f"low={sorted(-x for x in mf.low)} high={sorted(mf.high)} "
 f"ok={mf.check_invariants()}")
add 5 → median 5.0
add 15 → median 10.0
add 1 → median 5.0
add 3 → median 4.0

both versions agree
a, b = MedianFinder(), MedianFinderBranching()
import random
random.seed(0)
for _ in range(1000):
 v = random.randint(-1000, 1000)
 a.add(v); b.add(v)
 assert abs(a.median() - b.median()) < 1e-9
print("both versions agree over 1,000 insertions")

and against the honest reference
ref = MedianBySortedList()
c = MedianFinder()
for _ in range(1000):
 v = random.randint(-1000, 1000)
 ref.add(v); c.add(v)
 assert abs(ref.median() - c.median()) < 1e-9
print("matches a sorted-list reference over 1,000 insertions")

WHY THE REBALANCE MATTERS – an increasing sequence is the worst case
drift = MedianFinder()
for v in range(1, 8):
 drift.add(v)
print(drift.median(), drift.check_invariants()) # 4.0 True

the variants
print(median_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
[1.0, -1.0, -1.0, 3.0, 5.0, 6.0]

p90 = PercentileTracker(0.9)
for v in range(1, 101):
 p90.add(v)
print(p90.value()) # ~90

print(find_maximized_capital(2, 0, [1, 2, 3], [0, 1, 1])) # 4

edge cases
single = MedianFinder(); single.add(7)
print(single.median()) # 7.0
two = MedianFinder(); two.add(1); two.add(2)
print(two.median()) # 1.5
dup = MedianFinder()
for v in (5, 5, 5, 5):
 dup.add(v)
print(dup.median()) # 5.0

```

## 6 What it costs

### The operations

|        |             |                                                      |
|--------|-------------|------------------------------------------------------|
| add    | $O(\log n)$ | at most three heap operations, each $O(\log n)$      |
| median | $O(1)$      | both answers are at the tops                         |
| space  | $O(n)$      | every value is stored once, in one heap or the other |

The  $O(1)$  median is the point. Everything else about the design exists to make that line free.

### Against the alternatives

| approach                                   | insert      | median        | space  |
|--------------------------------------------|-------------|---------------|--------|
| sort on every query                        | $O(1)$      | $O(n \log n)$ | $O(n)$ |
| sorted list ( <code>bisect.insort</code> ) | $O(n)$      | $O(1)$        | $O(n)$ |
| order-statistic BST                        | $O(\log n)$ | $O(\log n)$   | $O(n)$ |
| TWO HEAPS                                  | $O(\log n)$ | $O(1)$        | $O(n)$ |

$n = 100,000$  insertions

|             |                   |                                                      |
|-------------|-------------------|------------------------------------------------------|
| sorted list | $O(n)$ per insert | → ~2,500,000,000 element moves                       |
| two heaps   | $O(\log n)$       | → ~1,700,000 comparisons<br>~1,500x fewer operations |

The sorted-list version is the honest competitor and it is  $O(n)$  per insert because `list.insert` shifts. It is genuinely faster for small  $n$ , because the shift is a memory move in C while a heap operation is interpreted Python — the crossover is around a few thousand elements, which is worth knowing rather than claiming the heap always wins.

And the order-statistic BST is the more general answer: it gives the  $k$ -th element for any  $k$ , not just the middle, at the cost of  $O(\log n)$  for the query instead of  $O(1)$  and roughly ten times the code.

### The constant factor

|           |                                                         |
|-----------|---------------------------------------------------------|
| each add: | 1 push + 1 pop + 1 push (+1 pop +1 push if rebalancing) |
|           | = 3 to 5 heap operations, each $O(\log n)$              |

The push-shift-rebalance form always does at least three operations, where the branching version often does one. On random data the branching version is measurably faster; the shift version is shorter and has no case analysis. Say which trade you

are making.

#### The sliding window

```
window size k, array length n

add + remove per step O(log k)
total O(n log k)
space O(k) live values, plus lazily deleted ones
```

**The lazy deletion cost is the subtlety:** removed elements stay in the heaps until they reach a top, so the heaps can grow to  $O(n)$  in the worst case even though only  $k$  values are logically present.

**The mitigation** is that a removed element is pruned as soon as it surfaces, so in practice the heaps stay near  $O(k)$  — but the worst case is real and worth stating.

#### Where the median problem gets harder

|                                  |                                                                               |
|----------------------------------|-------------------------------------------------------------------------------|
| running median, insert only      | two heaps. $O(\log n)$ / $O(1)$ .                                             |
| sliding window median            | two heaps + lazy deletion. $O(\log k)$ .                                      |
| median with arbitrary deletion   | two heaps + lazy deletion + an index<br>→ or just use an order-statistic tree |
| any percentile                   | change the balance ratio                                                      |
| approximate percentiles at scale | t-digest or HdrHistogram, $O(1)$ space                                        |

**That last row is the production answer** and worth naming: for monitoring — p50, p95, p99 over millions of events — nobody keeps every value. **t-digest** and **HdrHistogram** give accurate percentiles in bounded memory, which is what your metrics system is actually doing.

## 7 The traps

### Trap 1 — forgetting the rebalance

```
heapq.heappush(self.low, -value)
heapq.heappush(self.high, -heapq.heappop(self.low))
missing step 3
```

On an increasing sequence, everything drifts into **high** and the “median” reads from a **low** that holds one element.

```
add 1..7 without rebalancing → median reads as 1.0
correct answer → 4.0
```

**No error, and the answer is plausible.** Mrs Kurian's fifteen-and-five.

#### Trap 2 — a negation missing

```
return float(self.low[0]) # forgot to negate
```

`low` stores negated values, so this returns the negative of the median. **Two negations on the way in and out, and one when moving across** — and the version that moves a value between heaps needs exactly one.

```
heapq.heappush(self.high, -heapq.heappop(self.low)) # correct
heapq.heappush(self.high, heapq.heappop(self.low)) # WRONG: no negation
```

#### Trap 3 — the median function disagreeing with the rebalance rule

```
rebalance lets LOW be the larger one
if len(self.high) > len(self.low): ...

but the median reads from HIGH on an odd count
if len(self.high) > len(self.low):
 return float(self.high[0])
```

**Right for even counts, wrong for odd.** The two rules must agree about which heap holds the extra element. Pick one convention and write both lines from it.

#### Trap 4 — comparing against the wrong boundary

```
if value ≤ self.high[0]: # comparing against HIGH's top
 heapq.heappush(self.low, -value)
```

In the branching version you must compare against `low`'s **top** — the largest of the small values. Comparing against `high`'s top puts values in the wrong heap and breaks invariant 1.

**Mrs Kurian: “the check is against the boundary, not against the whole line” — and specifically against the near end of the left line.**

#### Trap 5 — an empty heap on the first insert

```
if value ≤ -self.low[0]: # IndexError when low is empty
```

```
IndexError: list index out of range
```

The branching version needs `if not self.low or ...`. The **push-shift-rebalance version has no such problem**, which is one more argument for it.

#### Trap 6 — integer division for the even case

```
return (-self.low[0] + self.high[0]) // 2 # floor division
```

The median of 1 and 2 is 1.5, not 1. `/`, not `//` — and returning `float` even for the odd case keeps the type consistent.

#### Trap 7 — the sliding window's size accounting

```
if self.low_size > self.high_size + 1: # LOGICAL sizes
if len(self.low) > len(self.high) + 1: # PHYSICAL sizes – WRONG
```

Lazily deleted elements are still physically present, so `len(heap)` is not the logical size. **Every bug in the sliding-window version is this**, and it produces a median that is correct most of the time.

#### Trap 8 — assuming the heaps are sorted

```
print(self.low) # NOT the sorted lower half
```

A heap's array is not sorted — [day 113](#). Only the **tops** are meaningful, which is exactly what the design relies on and exactly what makes it cheap.

## 8 In the interview

### How it gets asked

- The classic: “Design a data structure that supports adding numbers and finding the median.” LeetCode 295.
- The follow-up they always ask: “What if all the numbers are in the range 0 to 100?”
- The harder one: “Now do it over a sliding window.” LeetCode 480.
- The generalisation: “What if I wanted the 90th percentile instead?”
- The comparison: “Why not just keep a sorted list?”

### What to say out loud, in the first ninety seconds

1. **State the shape before the structure.** “I keep the numbers split into a lower half and an upper half, so the median is at the boundary between them and costs nothing to read.”
2. **Say which heap is which, and why.** “The lower half is a **max**-heap, so its largest is on top; the upper half is a **min**-heap, so its smallest is on top. Those two are the middle values.”
3. **Give both invariants.** “Two invariants: everything in low is at most everything in high, and the sizes differ by at most one. Given those, the median is  $O(1)$ .”
4. **Describe the insert as one unit.** “Push into low, move low’s top to high — which is what guarantees the ordering — then move one back if high is now bigger. Three lines, no branches.”
5. **Flag the negation.** “Python’s `heapq` is min-only, so the low heap stores negated values — two negations in and out, and that is where the bugs are.”
6. **Give both complexities.** “ $O(\log n)$  to add,  $O(1)$  to read the median,  $O(n)$  space.”

### The follow-ups

“**Why not just keep a sorted list?**” “It is a fair alternative and it is  $O(n)$  per insert, which is the thing to say. `bisect` finds the position in  $O(\log n)$ , but `list.insert` then has to **shift** every element after it, so at a hundred thousand insertions that is around two and a half billion element moves against about one and a half million heap comparisons for the two-heap version. I would add one honest caveat: for small  $n$  the sorted list is actually **faster**, because the shift is a `memmove` in C while a heap operation is interpreted Python — the crossover is somewhere in the low thousands. So ‘use two heaps’ is the right answer at scale and not automatically the right answer at every size. The other alternative worth naming is an **order-statistic tree** — a balanced BST with subtree sizes — which gives the  $k$ -th element for **any**  $k$  in  $O(\log n)$ , not just the middle. Two heaps are better when the median is all you ever need, because they are twenty lines rather than two hundred.”

“**What if all the numbers are between 0 and 100?**” “Then I would not use heaps at all — I would keep a **count array** of 101 buckets. Adding is  $O(1)$ : just increment. Finding the median is a scan across at most 101 buckets accumulating counts until I pass half the total, so it is  $O(1)$  in terms of  $n$  — the bucket count is a constant. That is better than  $O(\log n)$  insert and it is exact, not approximate. The general lesson is the same one as counting sort and as the top- $k$  bucket trick: **when the key space is small and bounded, bucketing beats every comparison-based structure.** If the

range were bounded but large — say a million — I would use a **Fenwick tree** over the value range, which gives  $O(\log \text{ range})$  insert and  $O(\log \text{ range})$  for the  $k$ -th element, and is what you actually want for a large bounded domain.”

“**Now do it over a sliding window.**” “Same two heaps, plus the problem that a heap cannot remove an arbitrary element. So: **lazy deletion**. When a value leaves the window I record it in a dictionary of pending removals rather than searching for it, and I discard it when it surfaces at a heap top. The part that is genuinely fiddly, and where all the bugs are, is that lazily deleted elements are **still physically in the heaps** — so I have to track the **logical** sizes separately from `len(heap)`, and the rebalance must use the logical sizes. If you use `len(heap)` there, you get a median that is correct most of the time and wrong occasionally, which is the worst kind of bug. The complexity is  $O(n \log k)$  overall, and the heaps stay near  $O(k)$  in practice because removed elements get pruned as soon as they reach a top — though the worst case is  $O(n)$ .”

“**What if I wanted the 90th percentile?**” “Change the balance ratio and nothing else. The median keeps the heaps at one-to-one; for the 90th percentile I keep them at nine-to-one, so that ninety percent of the values are in the low heap and the answer is the top of the low heap. Same two heaps, same push-shift structure, one different comparison in the rebalance. Worth adding what happens at real scale, though: for monitoring — p50, p95, p99 over millions of events a minute — nobody stores every value, because that is  $O(n)$  memory forever. The production answer is an approximate structure like **t-digest** or **HdrHistogram**, which give accurate percentiles in bounded memory and are what your metrics system is doing under the hood. Two heaps are the exact answer for a stream you can afford to keep.”

“**Why push into the low heap unconditionally?**” “Because the next line repairs it, and that removes all the case analysis. If I push into `low` and then immediately move `low`’s top into `high`, then whatever the new value was, the largest candidate for the lower half has just been handed over — so `max(low) ≤ min(high)` is guaranteed afterwards. Then I only have to fix the sizes. The alternative is to compare the new value against `low`’s top and choose a heap, which is also correct and is easier to explain, but it has two branches plus an empty-heap check on the very first insert, and each of those is a chance to be wrong. I would write the three-line version and mention that the branching one exists — and note that the branching one is measurably faster on random data, because it usually does one heap operation rather than three.”

“**How would you test it?**” “Against an honest reference. I keep a sorted list alongside, insert into both, and assert the medians agree after every single insertion over a few thousand random values — that catches the negation errors and the odd/even con-

vention mismatch immediately, which are otherwise plausible-looking wrong numbers. And separately I assert the **invariants** directly after each add: the sizes differ by at most one, and `max(low) ≤ min(high)`. Those two assertions catch every structural bug in this problem, and each of them is one line. The specific cases I would include are a strictly increasing sequence — which is what a missing rebalance breaks — a single element, exactly two elements, and all-duplicates.”

#### A model answer

Asked: *find the median of a stream of numbers.*



“The naive approach is to sort after every insertion, which is  $O(n \log n)$  per query. The idea that fixes it is to **keep the numbers already split** into a lower half and an upper half, so the median sits at the boundary between them and reading it costs nothing.

Concretely: two heaps facing each other. The lower half is a **max-heap**, so the **largest** of the small numbers is at its top. The upper half is a **min-heap**, so the **smallest** of the large numbers is at its top. Those two values are exactly the two middle elements of the whole stream.

Two invariants make it work. **Order**: everything in the low heap is at most everything in the high heap. **Balance**: their sizes differ by at most one. Given both, the median is  $O(1)$  — on an odd count it is the top of the larger heap, and on an even count it is the average of the two tops.

The insert is three lines and I would write them as one unit. **Push** the new value into the low heap unconditionally. **Shift** the low heap’s top across into the high heap — that is the line that guarantees the ordering invariant, because whatever the new value was, the largest candidate for the lower half has just been handed over. Then **rebalance**: if the high heap is now larger, move one back. No branches and no case analysis.

One Python detail worth saying aloud: `heapq` is min-only, so the low heap stores **negated** values — two negations on the way in and out, and exactly one when moving a value across. That is where the bugs in this problem live, along with the median function disagreeing with the rebalance rule about which heap holds the extra element on an odd count.

$O(\log n)$  to add,  $O(1)$  to read,  $O(n)$  space.

Compared with the alternative people reach for — a sorted list with `bisect` — that is  $O(n)$  per insert, because insertion has to shift everything after it: about two and a half billion element moves at a hundred thousand insertions, against roughly a million and a half heap comparisons.

And if you told me the values were bounded — say zero to a hundred — I would throw all of this away and keep a **count array**, which is  $O(1)$  to insert and a constant scan to find the median. Same lesson as counting sort: when the key space is small and bounded, bucketing beats every comparison-based structure.”

## 9 Recall card

- **Keep the numbers already SPLIT so the median sits at the boundary.** `low` is a **max-heap** of the smaller half (its **largest** on top), `high` is a **min-heap** of the larger half (its **smallest** on top) — those two are the middle values.  $O(\log n)$  add,  $O(1)$  median.
- **Two invariants, and everything follows:**  $\text{ORDER}(\max(\text{low}) \leq \min(\text{high}))$  and **BALANCE** (sizes differ by  $\leq 1$ ). Assert both in tests — every bug here breaks one of them **silently**.
- **PUSH, SHIFT, REBALANCE, as one unit:** push into `low` unconditionally · move `low`'s top into `high` (this is what guarantees the ordering) · move one back if `high` is bigger. **No branches, no case analysis.** The branching version is also correct, faster on random data, and has an empty-heap trap on the first insert.
- **Python has no max-heap, so `low` stores NEGATED values — two negations in and out, one when moving across.** And **the median function must agree with the rebalance rule** about which heap holds the extra element, or you are right on even counts and wrong on odd. Use `/`, not `//`.
- **Forgetting the rebalance is the classic bug:** an increasing sequence drifts everything into one heap and the median reads plausibly and wrongly. Variants: **sliding window** = the same + **lazy deletion**, where you must track **logical** sizes separately from `len(heap)`; **any percentile** = change the balance ratio; **bounded values** → a **count array**,  $O(1)$ ; **at scale** → **t-digest** / **HdrHistogram**, which is what your metrics system really does.

## 1 What this is, and why they ask it

**Leader election** is how a group of machines agrees that exactly one of them is in charge, and how they agree on a replacement when that one stops responding.

Three sentences. A leader exists to make coordination cheap: with one machine deciding, ordering is free and there are no conflicts to resolve, which is why [replication](#), [sharding](#) and every distributed lock are built on one. Electing a replacement is hard for exactly the reason from [day 113](#) — **you cannot distinguish a crashed leader from a slow one** — so every election starts with a **guess** made by a timeout. And the entire body of machinery around it exists for one purpose: **being safe when that guess is wrong**, because a wrong guess means two leaders, which is the worst failure in the subject.

They ask it because it is where several earlier threads meet: partial failure, timeouts, quorums, fencing. The answer that lands is not the election algorithm — it is *“the election is the easy part; the hard part is making sure the old leader cannot still be writing.”*

---

## 2 The story

The four boats went out together from the same landing and had done for as long as anyone could remember, and the rule was that one man decided where they fished that day.

It was not seniority. It was whoever had the radio set that worked, more or less, and by 2009 that was Antony. He called the ground at about five in the morning, and the other three followed him out and stayed within sight.

The reason for having one man decide was not politics. It was that the fish moved, and if the four boats spread out and each guessed, they caught nothing. Four boats on the same shoal caught something. One decision, followed by everybody, beat four good decisions.

The rule about what happened when Antony did not call was written on the wall of the landing shed, and it had been rewritten twice.

The first version said: if you do not hear from him, go where you think best. That lasted one season. On the mornings when his radio was down but he had gone out anyway, the four boats ended up in four places and the day was wasted.

The second version said: if you do not hear from Antony by half past five, Selvaraj decides. That was better and it failed on a specific morning in 2011, when Antony's radio was working but the aerial on Selvaraj's boat was wet. Antony called at ten past five; Selvaraj did not hear it; at half past five Selvaraj started calling the ground; and two boats followed Antony while two followed Selvaraj.

The third version, which was still on the wall, was longer and it had three parts.

**One.** Nobody decides alone. Before you call a ground, you must first raise the other boats and have at least **three of the four** — including yourself — say they will follow you. Two boats cannot make a decision, because two other boats might be making a different one.

**Two.** Every decision carries a **number**, counting up, and the man calling it says it. *“This is the fourth call of the morning.”* If you hear a call with a lower number than one you have already followed, you ignore it — because it is somebody who has been out of contact and does not know yet.

**Three.** If you have agreed to follow somebody, you say so, and then **you do not agree to follow anybody else for that call number**. One vote per number.

The old fisherman who had written it said the second part was the one that had taken him longest to think of. He said the problem was never that they could not choose. The problem was that a man who had been out of contact would come back on the radio still believing he was in charge, and sound completely confident, because from where he sat nothing had changed.

The numbers meant nobody had to argue. You just checked whether his number was old.

---

### 3 The idea in plain English

The landing shed's third rule is Raft, essentially unchanged, and the fisherman's observation about the confident man is the reason for all of it.

- One boat deciding is the **leader**, and the reason is coordination, not hierarchy.
- “By half past five” is the **election timeout** — the guess that the leader is gone.
- “Three of the four must agree” is a **quorum** — the majority from [day 117](#).

- The counting number is the **term** (or epoch), and ignoring lower numbers is **fencing**.
- “One vote per number” is what stops two leaders being elected in the same term.
- The confident man returning is **split-brain**, and the numbers are what make it harmless.

Why have a leader at all

**Because coordination without one is expensive.** With a single decision-maker:

|             |                                                       |
|-------------|-------------------------------------------------------|
| ordering    | the leader's sequence IS the order. Nothing to agree. |
| conflicts   | impossible: only one writer.                          |
| correctness | invariants can be checked in one place.               |
| simplicity  | no consensus on every operation, only on who leads.   |

**Compare with leaderless** — Dynamo, Cassandra — where every node accepts writes:

- + no election, no failover gap, no single bottleneck
- conflicts are now YOUR problem: vector clocks, CRDTs, last-write-wins
- no global ordering, so no transactions across keys
- no place to enforce a uniqueness constraint

**The trade in one line: a leader turns a hard agreement problem into an easy one, in exchange for a failover gap and a bottleneck.** Say that, because “why not leaderless?” is the follow-up.

Where leaders appear

|                      |                                           |           |
|----------------------|-------------------------------------------|-----------|
| database replication | one primary takes writes                  | (day 104) |
| sharding             | one node owns each shard                  | (day 107) |
| distributed locks    | the lock holder is a leader of one        |           |
| job schedulers       | one instance runs the cron, not all fifty |           |
| stream processing    | one consumer owns each partition          |           |
| cluster management   | one controller reconciles state           |           |

**The scheduled-job case is the one people meet first:** deploy a nightly billing job to fifty machines and it runs fifty times. **Leader election is the fix, and it is worth naming as the everyday version.**

The three ways to elect one

**1 — Use a consensus protocol yourself.** Raft or Paxos, built into your system. Correct, and a substantial amount of code.

**2 — Use a lock service.** ZooKeeper, etcd or Consul already run consensus; you ask them for a lock or a key with a lease and whoever gets it is the leader. **This is what almost everybody does**, and it is the right answer in an interview: *“I would not implement Raft; I would use etcd.”*

**3 — A lease from a shared store.** A row in a database with an expiry, taken by a conditional write. Simple, and correct **only if** the store itself is linearizable and you use fencing tokens.

**Raft’s election, which is the fisherman’s rule**

**Terms.** Time is divided into numbered terms. Each term has at most one leader.

**The timeout.** Every follower has a **randomised** election timeout, typically 150–300 ms. If it hears nothing from the leader in that time, it becomes a **candidate**, increments the term, votes for itself, and asks the others for votes.

**The vote.** A node grants its vote if it has not already voted in that term **and** the candidate’s log is at least as up to date as its own. **One vote per term** — the fisherman’s rule three.

**The majority.** A candidate becomes leader on receiving votes from a majority. **Two candidates cannot both get a majority**, so at most one leader per term — which is the pigeonhole argument from [day 117](#) doing safety work rather than consistency work.

**The randomisation matters.** If every follower timed out at the same instant, they would all become candidates, split the vote, and nobody would win — then repeat. **A random timeout in a range makes one node time out first almost always**, which is why elections usually complete in a single round.

**Fencing: the part that actually matters**

**The election is the easy half.** The hard half is that the old leader may not know it has been replaced.

```
the old leader was PAUSED, not dead
 a long garbage-collection pause
 the machine was swapping
 a network partition that has now healed

→ it wakes up still believing it is the leader
→ and it is CONFIDENT, because from its point of view nothing happened
```

**The defence is a monotonically increasing number** — a **fencing token**, a term, an epoch — attached to every write:

```
leader in term 5 writes: "write X, term 5"
a new leader is elected: term 6
the storage layer has seen term 6, so it REJECTS anything stamped term 5
```

**The storage layer does the enforcing**, not the leader. That is the crucial detail: you cannot trust a node to know it has been deposed, so the thing being written to must refuse stale writes.

**The fisherman's numbers.** *"If you hear a call with a lower number than one you have already followed, ignore it."*

The other defences, worth naming:

- **STONITH** — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.
- **A quorum requirement for the leader itself:** a leader must be able to see a majority to act, so a partitioned old leader **stops on its own**.

**Leases, and why they need clocks carefully**

A **lease** is leadership with an expiry: you are the leader until time **T** unless you renew.

```
+ the old leader stops BY ITSELF when the lease expires – no fencing needed
 in the common case
- it depends on CLOCKS, and clocks drift and jump (day 113)
```

**The safe pattern:** the leader treats its lease as expiring **earlier** than the grantor does, so there is a gap in which nobody believes they are leader.

```
grantor: the lease is valid until T
holder: I will stop acting at T - delta

delta must exceed the maximum clock skew plus the maximum message delay
```

**Get that backwards and you have two leaders during the gap**, which is why leases are not a substitute for fencing tokens — they reduce how often you need them.

**The trade-off in the timeout**

|                        |                                                                        |
|------------------------|------------------------------------------------------------------------|
| short election timeout | fast failover, more FALSE elections<br>(a GC pause looks like a crash) |
| long election timeout  | fewer false elections, longer unavailability                           |

|                      |                                         |
|----------------------|-----------------------------------------|
| Raft's typical range | 150-300 ms, randomised                  |
| failover in practice | ~1 second, including client redirection |
| ZooKeeper session    | typically 5-30 seconds                  |

**Every false election is a small outage:** the old leader stops, a new one starts, in-flight work is retried, and caches are cold. **So the timeout is tuned by how often you can afford that**, and there is no setting that avoids both problems — the same trade as [health checks](#).

#### 4 The picture

The election, as a timeline.

```

t=0 leader (term 5) is sending heartbeats every 50 ms
 ────▼───▼───▼───
t=200 ** the leader stops responding ** (crashed? paused? partitioned?)

t=200 followers stop hearing heartbeats
t=380 follower B's RANDOM timeout fires first (180 ms)
 → becomes a CANDIDATE
 → term 5 → 6
 → votes for itself, asks the others

t=385 C votes for B (has not voted in term 6, B's log is up to date)
t=387 D votes for B
 → B has 3 of 5: a MAJORITY

t=390 B is leader for term 6, starts heartbeats
t=400+ clients redirected

— the RANDOM timeout is what stops all four becoming candidates at once,
 splitting the vote, and having to repeat. One node almost always fires
 first.
```

The dangerous case, and what saves it:



THE OLD LEADER WAS PAUSED, NOT DEAD

A (term 5) — 20-second GC pause —————> wakes up  
"I am the leader"  
...and it is CONFIDENT,  
because from where it  
sits, nothing happened

meanwhile: B elected leader in term 6, has been writing for 15 seconds

A now sends: "write X, term 5"

▲  
THE STORAGE LAYER has already seen term 6, so it REJECTS this.

— the enforcement is at the STORAGE, not at the leader. You cannot trust  
a node to know it has been deposed.

The three defences, and what each one catches:

|                                 |                                                                                                                                                                                |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FENCING TOKEN<br>(term / epoch) | every write carries a monotonically increasing term;<br>the storage rejects anything stamped older<br>→ catches the paused leader that woke up                                 |
| QUORUM REQUIREMENT              | a leader may only act while it can see a majority<br>→ a partitioned old leader STOPS ON ITS OWN<br>→ and it is why cluster sizes are ODD                                      |
| STONITH / FENCING<br>the node   | the new leader forcibly disables the old one before<br>accepting a single write<br>→ catches everything, and needs infrastructure<br>(power control, storage lease revocation) |

The fisherman's three rules, mapped:

|                                                              |                                                                                                  |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| "three of the four must agree"                               | → a MAJORITY QUORUM<br>two candidates cannot both get one,<br>so at most one leader per number   |
| "every call carries a number, and<br>you ignore a lower one" | → TERMS and FENCING<br>the confident man's calls are<br>recognised as stale without argument     |
| "one vote per number"                                        | → a node votes at most once per term<br>which is what makes the majority<br>argument work at all |

Leases and the safety gap:

grantor's view: |———— lease valid until T ————|  
holder's view: |—— I stop acting at T - delta ——|

▲  
the SAFETY GAP:  
nobody believes  
they are leader

delta must exceed (max clock skew) + (max message delay)

GET IT BACKWARDS — the holder thinking the lease lasts LONGER than the grantor does — and there is a window with TWO leaders.

## 5 How it actually works

Doing it with etcd or ZooKeeper — the practical answer

**Nobody implements Raft for an application.** They use something that already has it.

```
etcd: put a key with a LEASE, using a compare-and-swap that only
 succeeds if the key does not exist
 → the winner is the leader
 → it must KEEP RENEWING the lease (a keepalive)
 → if it stops, the key expires and someone else wins

ZooKeeper: create an EPHEMERAL SEQUENTIAL node
 → the lowest sequence number is the leader
 → "ephemeral" means it disappears when the session ends
 → each node watches only the one immediately below it
 (which avoids the herd effect of everyone watching the leader)
```

**The ZooKeeper detail is worth knowing:** if all `n` followers watch the leader's node, its disappearance wakes all of them at once — a **herd**. Watching only your immediate predecessor means one node wakes.

**And the fencing token comes from the same place**

```
etcd: the lease's revision number increases monotonically
ZooKeeper: the znode's zxid, or the sequence number

→ pass it with every write, and have the storage reject older ones
```

**A lock without a fencing token is not safe**, and this is the single most useful practical takeaway: any lock service can hand you a lock and then have you pause for thirty seconds, and only the token protects what you do next.

### What a leader actually does differently

|                                  |                                    |
|----------------------------------|------------------------------------|
| all writes go to it              | → ordering is free                 |
| it assigns sequence numbers      | → replicas apply in the same order |
| it holds the authoritative state | → reads from it are current        |
| it sends heartbeats              | → so followers know it is alive    |

**And the corollary:** the leader is a **bottleneck** for writes and a **single point of failure** for the failover window. Both are accepted deliberately in exchange for cheap coordination.

### The client's problem, which people forget

Electing a leader is useless if clients keep talking to the old one.

|                                                |                            |
|------------------------------------------------|----------------------------|
| discovery options:                             |                            |
| ask any node, and be redirected                | (Raft's usual answer)      |
| look it up in the same lock service            | (an extra dependency)      |
| a proxy in front that knows the current leader | (an extra hop)             |
| DNS                                            | → bounded below by the TTL |

**The DNS option is the trap** — a 60-second TTL means a 60-second failover no matter how fast the election was, which is the same [day 104](#) point.

### What real systems do

- **Raft** is the default modern answer because it was explicitly designed to be understandable, and it is what **etcd**, **Consul**, **CockroachDB**, **TiDB** and **RethinkDB** use.
  - **Paxos** is older, harder to describe, and underpins **Chubby** (Google's lock service) and **Spanner**.
  - **ZooKeeper** uses **ZAB**, which is Paxos-like, and is the classic answer for Hadoop-era systems and for **Kafka** before KRaft.
  - **Kafka** historically used ZooKeeper for controller election and now uses its own Raft implementation (**KRaft**) — a good example of a system that outgrew an external dependency.
  - **Kubernetes** elects a leader per controller using a **lease object in etcd**, which is exactly the lock-service pattern.
  - **Redis Sentinel** does leader election for failover, and Redis's own documentation is candid that Redis replication is asynchronous, so a failover can lose writes.
-

## 6 The numbers

### Election timing

|                           |                                            |
|---------------------------|--------------------------------------------|
| heartbeat interval        | 50 ms                                      |
| election timeout (random) | 150-300 ms                                 |
| detection                 | 150-300 ms after the last heartbeat        |
| election round            | 1-2 round trips → ~5-20 ms in a datacentre |
| becoming leader           | ~200-350 ms total                          |
| + client redirection      | ~100 ms to several seconds                 |
| -----                     |                                            |
| practical failover        | ~1 second, and DNS-based can be MINUTES    |

**The election itself is fast. The client redirection is usually the slow part**, which is why the proxy or virtual-IP approach beats DNS.

### The timeout trade

| election timeout                 | detection | false elections                           |
|----------------------------------|-----------|-------------------------------------------|
| -----                            | -----     | -----                                     |
| 50 ms                            | fast      | frequent – a 100 ms GC pause triggers one |
| 150-300 ms                       | ~0.3 s    | rare, and this is Raft's default range    |
| 5 s                              | slow      | essentially never                         |
| 30 s (ZooKeeper session default) | very slow | never, but 30 s of unavailability         |

a false election costs:  
the old leader stops accepting writes  
in-flight requests fail and are retried  
caches on the new leader are cold  
→ typically 1-5 seconds of degraded service, for nothing

**So the timeout is tuned by how often you can afford a small unnecessary outage**, which is exactly the health-check trade from [day 099](#).

### Cluster size

| nodes | majority | tolerates | election messages (a round) |
|-------|----------|-----------|-----------------------------|
| ----- | -----    | -----     | -----                       |
| 3     | 2        | 1         | 2                           |
| 5     | 3        | 2         | 4                           |
| 7     | 4        | 3         | 6                           |
| 9     | 5        | 4         | 8                           |

**Odd numbers only.** Four nodes tolerate the same single failure as three, and a 2-2 split leaves nobody with a majority so the cluster stops entirely.

**And bigger is not better:** more nodes means more messages per decision, so **latency rises with cluster size**. Three or five is almost always right, and etcd's own guidance is to stay at five or below.

#### The GC pause problem, quantified

```
a JVM full GC on a large heap 100 ms - several SECONDS
a Raft election timeout 150-300 ms

→ a full GC longer than the timeout looks EXACTLY like a crash
→ the node is deposed, wakes up, and believes it is still leader
→ WITHOUT fencing, it writes with stale authority
```

**This is not theoretical** — it is the most commonly reported cause of split-brain in production, and it is why fencing tokens exist rather than being an optional refinement.

#### Lease safety margin

```
lease duration 10 s
max clock skew (NTP) ~50 ms
max message delay ~200 ms
safety margin (delta) ≥ 250 ms, and in practice 1-2 s

holder stops acting at T - 2 s
grantor considers expired T
→ a 2-second window in which NOBODY is leader
→ which is the price of never having TWO
```

#### The leader as a bottleneck

```
all writes through one node
 a relational leader ~5,000 writes/s
 etcd ~10,000 writes/s (small keys, fsync-bound)

→ when the write rate exceeds one machine, a single leader is the ceiling
→ the answer is SHARDING: one leader PER SHARD, so leadership scales
 with the number of shards
```

**“One leader per shard” is the answer to “isn’t the leader a bottleneck?”**, and it is what every large system does.

## 7 The trade-offs

**A leader costs you a bottleneck and a gap**

**All writes go through one machine**, so its capacity is the system's write ceiling, and there is a window during every failover when nothing can be written.

**The mitigation is sharding** — one leader per shard — so leadership scales horizontally even though each individual leader does not.

**I would not use a single leader** if the write rate exceeds one machine and the data does not partition cleanly, or if writes must survive a whole-region failure without a gap — which pushes you towards leaderless.

**Leader or leaderless?**

**A leader** makes ordering free and conflicts impossible, at the cost of the gap and the bottleneck.

**Leaderless** (Dynamo, Cassandra) has no election, no failover gap, and no bottleneck — and it hands you the conflict problem: vector clocks, CRDTs or last-write-wins, plus no global ordering and no way to enforce a uniqueness constraint.

**Take a leader when you need ordering or invariants; take leaderless when you need availability and can resolve conflicts.** That is the same CP/AP choice from [day 114](#), arriving from a different direction.

**Roll your own, or use etcd?**

**Use etcd or ZooKeeper.** Implementing Raft correctly is months of work and the failure modes are subtle; using a service that already does it is a configuration file.

**The cost is a new dependency in the critical path** — if etcd loses quorum, nothing can elect a leader, which is why Kubernetes stops scheduling when etcd is unhealthy. **That is correct behaviour**, and worth saying so, because it looks like a bug.

**I would build my own** only if the system is itself a database that cannot take a dependency, which is why CockroachDB and Kafka eventually did.

**Fencing is not optional**

**A lock without a fencing token is not a lock.** Any lock service can grant you leadership and then let you pause for thirty seconds, and when you wake up the lock is somebody else's while you still believe it is yours.

**The token has to be enforced by the resource being protected**, not by the holder — because the holder is precisely the thing that cannot be trusted to know.

### Where it goes wrong

- **A GC pause longer than the election timeout.** The most common real cause of split-brain.
  - **Clock-based leases without a safety margin**, or with the margin the wrong way round.
  - **Even-numbered clusters**, which tolerate no more failures and stop entirely on an even split.
  - **Client redirection through DNS**, where the TTL sets the floor on failover time.
  - **Too many nodes**, where every decision needs more round trips and latency rises for no extra fault tolerance.
  - **Forgetting that the leader is also a cache-warmth problem** — the new leader starts cold, so failover costs more than the election time suggests.
- 

### 8 In the interview

#### How it gets asked

- The direct one: *“The leader crashed. How does the cluster choose the next one?”*
- The dangerous one: *“What if the old leader comes back?”*
- The practical one: *“Would you implement Raft?”*
- The design one: *“I have fifty servers and a nightly job. How do I run it once?”*
- The limits one: *“Isn’t the leader a bottleneck?”*

#### What to say out loud, in the first ninety seconds

1. **Say why there is a leader at all.** “A leader makes coordination cheap: one writer means ordering is free and there are no conflicts. That is why replication, sharding and locks are all built on one.”
2. **Start the election with the guess.** “Followers stop hearing heartbeats and, after a **randomised** timeout, one becomes a candidate, increments the term, and asks for votes. The randomisation is what stops everyone becoming a candidate at once and splitting the vote.”
3. **State the safety property.** “A candidate needs a **majority**, and a node votes at most once per term — so two candidates cannot both win. At most one leader per term.”
4. **Go straight to the hard part.** “The election is the easy half. The hard half is that the old leader may not know it was replaced — a long GC pause looks exactly like a crash, and the node wakes up confident.”

5. **Name fencing and say where it is enforced.** “So every write carries a monotonically increasing term, and **the storage layer rejects anything stamped with an older one**. The enforcement has to be at the resource, not at the leader, because the leader is precisely what cannot be trusted.”
6. **Say what you would actually build.** “And I would not implement Raft — I would take a lease from etcd or ZooKeeper, and use the revision number as the fencing token.”

#### The follow-ups

“**What if the old leader comes back?**” “That is the failure the whole design exists to prevent, and the reason it happens is that a paused leader is **indistinguishable from a crashed one**. A twenty-second garbage-collection pause, a machine swapping, or a network partition that heals — from the node’s own point of view nothing happened, so it wakes up confident that it is still in charge and starts writing. Three defences and I would want at least two. **Fencing tokens:** every write carries the term, and the storage rejects anything older than the highest term it has seen — the crucial part being that the **storage** enforces it, because the leader cannot be trusted to know it was deposed. **A quorum requirement for the leader itself:** it may only act while it can see a majority, so a partitioned old leader stops on its own — which is also why cluster sizes are odd. And **STONITH**, where the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting a single write. Without any of these you get two leaders writing simultaneously, and that data cannot be automatically reconciled.”

“**Would you implement Raft?**” “No, and I would say so directly. Implementing consensus correctly is months of work and the failure modes are subtle enough that most hand-rolled implementations are wrong in ways that only appear under partition. I would take a lease from **etcd** or **ZooKeeper**, which already run consensus: in etcd, put a key with a lease using a compare-and-swap that only succeeds if the key is absent, keep renewing it, and whoever holds it is the leader. In ZooKeeper, create an ephemeral sequential node and the lowest sequence number wins — and each node watches only the one immediately below it, which avoids waking every node at once when the leader disappears. Crucially, both give me a **monotonically increasing number** — etcd’s revision, ZooKeeper’s zxid — which I use as the fencing token. The cost of that choice is a new dependency on the critical path: if etcd loses quorum, nothing can elect a leader, which is exactly why Kubernetes stops scheduling when etcd is unhealthy. That is correct behaviour rather than a bug. I would build my own only if the system were itself a database that cannot take an external dependency, which is why CockroachDB and Kafka eventually did.”



**“I have fifty servers and a nightly job. How do I run it once?”** “That is leader election in its most everyday form, and the failure without it is that the job runs fifty times — fifty billing runs, fifty emails to every customer. The simplest correct answer is a **lease**: each instance tries to acquire a named lock in etcd, Redis or a database row with a compare-and-set, and only the winner runs the job. Three things I would get right. The lease must have an **expiry**, so a crashed holder does not block the job forever. The holder must **renew** it while the job runs, or a long job outlives its own lease and a second instance starts. And the job itself should be **idempotent** where possible, because if the lease does expire mid-run — a GC pause, again — two instances may overlap, and that is far cheaper to survive than to prevent. If the job is long, I would also make it **resumable** and checkpoint its progress, so an overlap costs duplicated work rather than duplicated side effects.”

**“Isn’t the leader a bottleneck?”** “Yes, in two ways, and both are accepted deliberately. **Throughput**: every write goes through one machine, so its capacity is the system’s write ceiling — a relational leader is around five thousand writes a second and etcd around ten thousand for small keys. **Availability**: there is a window during every failover when nothing can be written, typically around a second including client redirection. The standard answer to the throughput problem is **sharding**: one leader per shard, so leadership scales with the number of shards even though each individual leader does not. That is what every large system does — Kafka elects a leader per partition, and a sharded database elects one per shard. The alternative is going **leaderless**, which removes the election and the bottleneck and hands you conflict resolution instead: vector clocks or CRDTs, no global ordering, and no way to enforce a uniqueness constraint. That is the same CP-versus-AP choice arriving from a different direction.”

**“How long is the failover, and what sets it?”** “The election itself is fast: heartbeats every fifty milliseconds, a randomised election timeout of a hundred and fifty to three hundred, then one or two round trips to collect votes — so a new leader exists within about a third of a second. **The slow part is almost always telling the clients.** If they discover the leader through DNS with a sixty-second TTL, the failover is sixty seconds regardless of how fast the election was — which is why production systems use a proxy or a virtual IP, or have clients ask any node and be redirected. There is also a cost the timing does not show: the new leader starts with **cold caches**, so the first minute after a failover is slower even once everything is technically working. And the timeout itself is a trade — shorter means faster detection and more **false** elections, and every false election is a small outage in which the old leader stops, in-flight work is retried and caches go cold. A GC pause longer than the timeout triggers one, which is exactly the case that then needs fencing.”

**“Why is the election timeout randomised?”** “To avoid a **split vote**. If every follower used the same timeout, they would all stop hearing heartbeats at the same moment, all become candidates at the same instant, all vote for themselves, and none would reach a majority — and then they would all time out again together and repeat, potentially for a long time. Randomising the timeout over a range, typically a hundred and fifty to three hundred milliseconds, means one node almost always fires meaningfully earlier than the others, wins the votes before anyone else even becomes a candidate, and the election completes in a single round. It is a very small design decision that makes the difference between elections that reliably converge and elections that can livelock, and it is one of the reasons Raft is easier to reason about than the alternatives.”

**A model answer**

*Asked: the leader crashed. How does the cluster choose the next one?*

“Let me answer in two halves, because the second one is where the difficulty is.

**The election.** Followers expect heartbeats from the leader — say every fifty milliseconds. If a follower hears nothing for its **election timeout**, which is randomised between roughly a hundred and fifty and three hundred milliseconds, it becomes a **candidate**: it increments the **term** number, votes for itself, and asks the other nodes for their votes. A node grants its vote if it has not already voted in that term and the candidate’s log is at least as up to date as its own. A candidate that receives votes from a **majority** becomes the leader for that term.

Two details in that carry the safety. **A majority is required**, and a node votes at most once per term — so two candidates cannot both win, and there is at most one leader per term. That is the pigeonhole argument again. And the **timeout is randomised**, because if every follower timed out simultaneously they would all become candidates, split the vote, and have to repeat — randomising means one node almost always fires first and the election finishes in one round.

That whole process takes about a third of a second. **The slow part is usually telling the clients** — if they find the leader through DNS with a sixty-second TTL, the failover is sixty seconds however fast the election was.

**Now the hard half.** The election assumed the old leader was dead, and that was a **guess** — a timeout cannot distinguish a crashed node from a slow one. A twenty-second garbage-collection pause looks exactly like a crash, and the node wakes up **still believing it is the leader** and completely confident, because from where it sits nothing happened. That is split-brain, and two leaders writing simultaneously produces data that cannot be automatically reconciled.

So: **fencing**. Every write carries the term number, and the storage layer rejects anything stamped with a term older than the highest it has seen. The essential detail is that **the storage enforces it, not the leader** — because the leader is precisely the thing that cannot be trusted to know it has been deposed. On top of that I would require a leader to be able to see a **majority** in order to act at all, so a partitioned old leader stops on its own; and where the infrastructure allows it, have the new leader forcibly fence the old one before accepting a single write.

And in practice I would not implement any of this. I would take a lease from **etcd** — a compare-and-set on a key with an expiry, renewed by the holder — and use etcd’s revision number as the fencing token. The honest cost is a dependency on the critical path: if etcd loses quorum, no leader can be elected, which is exactly why Kubernetes stops scheduling when etcd is unhealthy.”

## 9 Recall card

- **A leader exists to make coordination cheap: one writer means ordering is free and conflicts are impossible.** That is why replication, sharding, locks and schedulers are built on one. The everyday case is a **nightly job on 50 machines running 50 times**.
- **The election: heartbeats stop → a RANDOMISED timeout (150–300 ms) fires → a candidate increments the TERM, votes for itself, and needs a MAJORITY.** One vote per term, so two candidates cannot both win — at most one leader per term. **Randomisation prevents split votes** and makes elections finish in one round.
- **The election is the EASY half. The hard half: a paused leader is indistinguishable from a crashed one** — a GC pause longer than the timeout is the most common real cause of split-brain — **and it wakes up confident**.
- **FENCING is the answer, and the STORAGE must enforce it, not the leader:** every write carries a monotonically increasing term/epoch, and anything older is rejected. Plus a **quorum requirement** so a partitioned leader stops on its own (hence **odd cluster sizes**), and **STONITH** where available. **A lock without a fencing token is not a lock.**
- **Do not implement Raft — take a lease from etcd or ZooKeeper** (ephemeral sequential nodes, each watching only its predecessor to avoid a herd) and **use their revision/zxid as the token**. The cost is a critical-path dependency — Kubernetes stopping when etcd loses quorum is correct, not a bug. And the leader is a **write bottleneck**, answered by **one leader per shard; client redirection, not the election**, usually dominates failover time.

## Practice — Two heaps: the running median

**DSA topic:** Two heaps: the running median **System design topic:** Leader election

### Code these, in this order

One rule for the whole set: **assert both invariants after every insertion**. Sizes differ by at most one, and `max(low) ≤ min(high)`. Two lines, and they catch every bug in this problem — all of which otherwise produce a plausible wrong number.

| # | PROBLEM                      | SOURCE                | WHAT IT IS REALLY TESTING                                             |
|---|------------------------------|-----------------------|-----------------------------------------------------------------------|
| 1 | Find Median from Data Stream | LeetCode 295 (Hard)   | The two invariants, the negation, and push-shift-rebalance.           |
| 2 | IPO                          | LeetCode 502 (Hard)   | Two heaps facing each other with a different question.                |
| 3 | Sliding Window Median        | LeetCode 480 (Hard)   | Lazy deletion, and logical sizes versus <code>len(heap)</code> .      |
| 4 | Meeting Rooms II             | LeetCode 253 (Medium) | The simpler cousin: one heap of end times against a stream of starts. |

### On problem 1, write both versions

Push-shift-rebalance, and the branching version. Run both over ten thousand random values and assert they agree. Say which you would write in an interview and why.

### On problem 1 again, break it three ways

Remove the rebalance and run 1 to 7. Drop one negation. Make the median function read from the wrong heap on odd counts. Record all three outputs and say which of them ever raises.

### On problem 3, track the sizes separately

Write it once using `len(heap)` for the rebalance and once using logical counters. Find an input where they differ and record it.

### On problem 4, notice the shape

Say what the single heap holds, what the boundary is, and why this problem does not need two.

### The invariants drill

1. State both invariants.
2. Say what each one buys.
3. Write the two-line assertion.
4. For each invariant, say what breaks if it is violated.

### The structure drill

1. Say which heap is a max-heap and which is a min-heap, and why each way round.
2. Say what the two tops represent.
3. Give the median rule for odd and even counts.
4. Say why the median is  $O(1)$ .

### The insert drill

1. Write push-shift-rebalance from memory.
2. Explain why pushing into `low` unconditionally is safe.
3. Say which line guarantees the ordering invariant.
4. Write the branching version and name its two extra risks.

### The negation drill

1. Say how many negations there are on the way in, on the way out, and when moving across.
2. Write the line that moves a value from `low` to `high`.
3. Drop a negation and record what the median becomes.

### The convention drill

1. Say which heap holds the extra element on an odd count in your implementation.
2. Say what must agree with that choice.
3. Change one and not the other, and record which counts are now wrong.

### The alternatives drill

1. Give insert and query complexity for four approaches.
2. Compute the element moves for a sorted list at  $n = 100,000$ .
3. Say where the crossover with a sorted list actually is, and why.
4. Say what an order-statistic tree gives you that two heaps do not.

### The variants drill

1. Say what changes for the sliding-window version.
2. Say what changes for the 90th percentile.
3. Describe the IPO shape in one sentence.
4. Name the production answer for percentiles at scale, and why.

### The break-it drill

Trigger each and record the exact output or error:

1. No rebalance, on 1 to 7 ascending.
  2. `float(self.low[0])` without the negation.
  3. The median reading from `high` on an odd count while the rebalance favours `low`.
  4. `//` instead of `/` for the even case.
  5. The branching version with no empty-heap check, on the first insert.
  6. Comparing against `high[0]` instead of `-low[0]` when routing.
  7. `len(heap)` instead of logical sizes in the sliding-window version.
- 

### The why-a-leader drill

1. Give four things a leader makes cheap.
2. Give four things leaderless gives up.
3. State the trade in one sentence.
4. Name the everyday case where a beginner meets this problem.

### The election drill

1. Describe the sequence from missed heartbeat to new leader.
2. Say what a term is and what changes it.
3. State the two rules that guarantee at most one leader per term.
4. Say why the timeout is randomised, and what happens without it.
5. Give typical figures for the heartbeat, the timeout and the whole failover.

### The split-brain drill

1. Say why a paused leader is indistinguishable from a crashed one.
2. Name three causes of a pause long enough to matter.
3. Describe what the old leader believes when it wakes.
4. Name the three defences and say what each one catches.

5. Say where fencing must be enforced, and why not at the leader.

#### **The practical drill**

1. Say whether you would implement Raft, and why.
2. Describe the etcd approach in three steps.
3. Describe the ZooKeeper approach, including the watch detail.
4. Say where the fencing token comes from in each.
5. State the cost of using an external service, with the Kubernetes example.

#### **The lease drill**

1. Define a lease.
2. Say what it buys over a plain lock.
3. Draw the safety gap and say which way round it must go.
4. Compute a safety margin from clock skew and message delay.
5. Say why a lease does not remove the need for fencing.

#### **The sizing drill**

1. Fill in the nodes-to-tolerated-failures table for 3 through 9.
2. Say why even numbers buy nothing.
3. Say why bigger is not better, in terms of latency.
4. Give the recommended cluster size and the source of that guidance.

#### **The bottleneck drill**

1. Name the two ways a leader is a bottleneck.
2. Give write-rate figures for a relational leader and for etcd.
3. State the standard answer, in four words.
4. Name two systems that do it that way.

#### **The failure drill**

For each, say what happens and what you would add:

1. A JVM full GC pauses the leader for four seconds.
2. All followers have identical election timeouts.
3. Clients find the leader through DNS with a 60-second TTL.
4. A four-node cluster splits two and two.
5. A lock is granted, the holder pauses, and the lock expires mid-operation.



6. A nightly job is deployed to fifty machines with no coordination.

Two of the six are not fixed by a faster election. Name them.

---

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the median of a stream of numbers.* The split-at-the-boundary idea before the structure, which heap is which and why, both invariants, push-shift-rebalance as one unit with the reason the blind push is safe, the negation convention, and the comparison against a sorted list.
  2. *The leader crashed. How does the cluster choose the next one?* The election in one paragraph with terms, majority and randomised timeouts, then the hard half — a paused leader is indistinguishable from a dead one — fencing with the enforcement at the storage, and what you would actually build.
  3. *What if the old leader comes back?* Why it happens, what it believes, the three defences, and where each one acts.
- 

### Before you move on

- ☐ I can state both median invariants and write the assertion.
- ☐ I know which heap is a max-heap and can say why each way round.
- ☐ I can write push-shift-rebalance from memory and justify the blind push.
- ☐ I know exactly how many negations go where.
- ☐ My median function agrees with my rebalance rule.
- ☐ I ran the version with no rebalance and saw the drift.
- ☐ I can compare against a sorted list with real numbers, and name the crossover.
- ☐ I know what an order-statistic tree adds.
- ☐ I can adapt it to a percentile and to a sliding window.
- ☐ I know the production answer for percentiles at scale.
- ☐ I can say why a leader exists, in terms of coordination.
- ☐ I can describe the election including terms, majority and randomised timeouts.
- ☐ I can say why the randomisation matters.
- ☐ I can explain why a paused leader is indistinguishable from a dead one.

- [ ] I can name three fencing defences and say where each acts.
- [ ] I know that the storage must enforce the token, not the leader.
- [ ] I would use etcd or ZooKeeper, and know where the token comes from.
- [ ] I can draw the lease safety gap the right way round.
- [ ] I know why cluster sizes are odd and why bigger is not better.
- [ ] I can say what usually dominates failover time.
- [ ] I answered all three questions above out loud.

# Heaps revision and mock round

## DATA STRUCTURES AND ALGORITHMS

Heaps revision and mock round

## SYSTEM DESIGN

Consensus, and Raft in plain English

## 1 What this is, and why they ask it

Six days ago you did not know what a heap was. Today you close the topic.

This is not a new idea. It is the day you turn six days of separate lessons into one thing you can use under pressure: a way of *recognising* that a problem wants a heap, before you have written a line.

There is exactly one question that does the recognising, and it is short:

**Do I repeatedly need the smallest or largest thing, while the collection keeps changing?**

If yes, heap. If the collection never changes, sort once and index — a heap buys you nothing. If you need the whole order, sort. If you need *one end, again and again, from a set that grows and shrinks* — that is the heap's exact job, and nothing else does it in  $O(\log n)$ .

Everything in this lesson hangs off that sentence.

By the end you will have:

- the recognition question, and the three ways problems phrase it so you do not notice
- the five shapes every heap problem in an interview takes
- the `heapq` checklist — the six things that actually go wrong in Python
- one table of costs you can recite
- two mock problems you have not seen, solved out loud, with the timing of a real round

This is a revision day. Read it with the previous six days closed.

## 2 The story

Meera has worked the admissions desk at a district hospital for eleven years. The desk is a narrow counter by the front door, and from eight in the morning people arrive at it without stopping.

New staff find the desk terrifying. There is a queue, and the queue does not behave. A man with a cut hand joins the back. Four minutes later a woman is carried in by two neighbours and everything reshuffles. A child who has been waiting an hour is still waiting. New staff try to hold the whole line in their head — who came when, who is where, what is fair — and by ten they are exhausted and making mistakes.

Meera does not hold the line in her head. She told a new girl why on her second day.

“Every single thing anyone asks me at this desk,” she said, “is one of five questions. Not six. Five. And before I answer any of them I ask myself one thing first: do they want the next one, or do they want the whole list?”

The girl said that could not be true.

“Try me,” said Meera. “Go on. Ask me anything you have been asked today.”

The girl said: who goes in next.

“Next one.”

Who are the five sickest people here.

“Next one, five times.”

How many people are waiting.

“Whole list. Count them. That is not my kind of question at all, that is a different job.”

The girl thought, and asked: which family is exactly in the middle of the waiting time — half have waited longer, half less.

“Next one,” said Meera, “but from both ends at once. I keep the long-waiters on my left and the short ones on my right and I watch the two people standing at the join. That is the only hard one, and it is still the same question.”

The girl was quiet for a while. Then she said the thing that made Meera nod.

“So you never actually sort the queue.”

“Sort it,” said Meera. “For what? By the time I finished sorting it, it would be wrong. Two people would have gone in and three arrived. I only ever need to know who is next. Everyone else can stand in any order they like and it costs me nothing.”

---

### 3 The idea in plain English

Meera's first question is the recognition question:

#### Do you want the next one, or the whole list?

*Next one, repeatedly, from a collection that keeps changing* is the heap. Everything else is a different tool. That is the whole of the recognition, and it is worth saying out loud until it is automatic, because interview problems do not say “use a heap”. They say one of three things instead.

**Phrasing one: the superlative.** “The  $k$  largest.” “The closest points.” “The most frequent words.” “The cheapest option available now.” Any superlative, asked more than once, is the recognition question wearing a hat.

**Phrasing two: the schedule.** “Process the events in order as they arrive.” “Which meeting room frees up first?” “Which task should the worker take next?” A schedule is a stream of *next-one* questions. The collection changes between every question, which is exactly the condition.

**Phrasing three: the boundary.** “The median.” “The 90th percentile.” A boundary is two *next-one* questions facing each other — the largest of the small half and the smallest of the large half — which is why the boundary problems take two heaps and not one.

Then there is the negative recognition, which matters just as much. **A heap is the wrong answer when:**

- The collection is fixed. Sort once, then index. Sorting is  $O(n \log n)$  once; a heap gives you the same and then makes you pop  $n$  times to get the order back.
- You need the whole order. Sorting is faster in practice and one line long.
- You need to find, update, or delete an arbitrary element by value. A heap cannot do this. Finding an element in a heap is  $O(n)$  — a linear scan, because a heap is only ordered along the paths from the top, not left to right. If a problem needs “increase the priority of task 47”, you need a heap with an index map, or a different structure entirely.
- $k$  is close to  $n$ .  $O(n \log k)$  and  $O(n \log n)$  are the same number when  $k = n$ , and the sort has a smaller constant and one line of code.

The last sentence of the story is the reason the whole structure exists. Meera never sorts the queue. A heap does not sort. It maintains just enough order to answer *who is next* and no more — one comparison per parent-child pair, along the paths to the top,

and nothing about left-to-right. That partial order is what makes push and pop  $O(\log n)$  instead of  $O(n)$ . You are buying exactly one fact, cheaply, over and over.

4 The picture

The recognition, as one decision

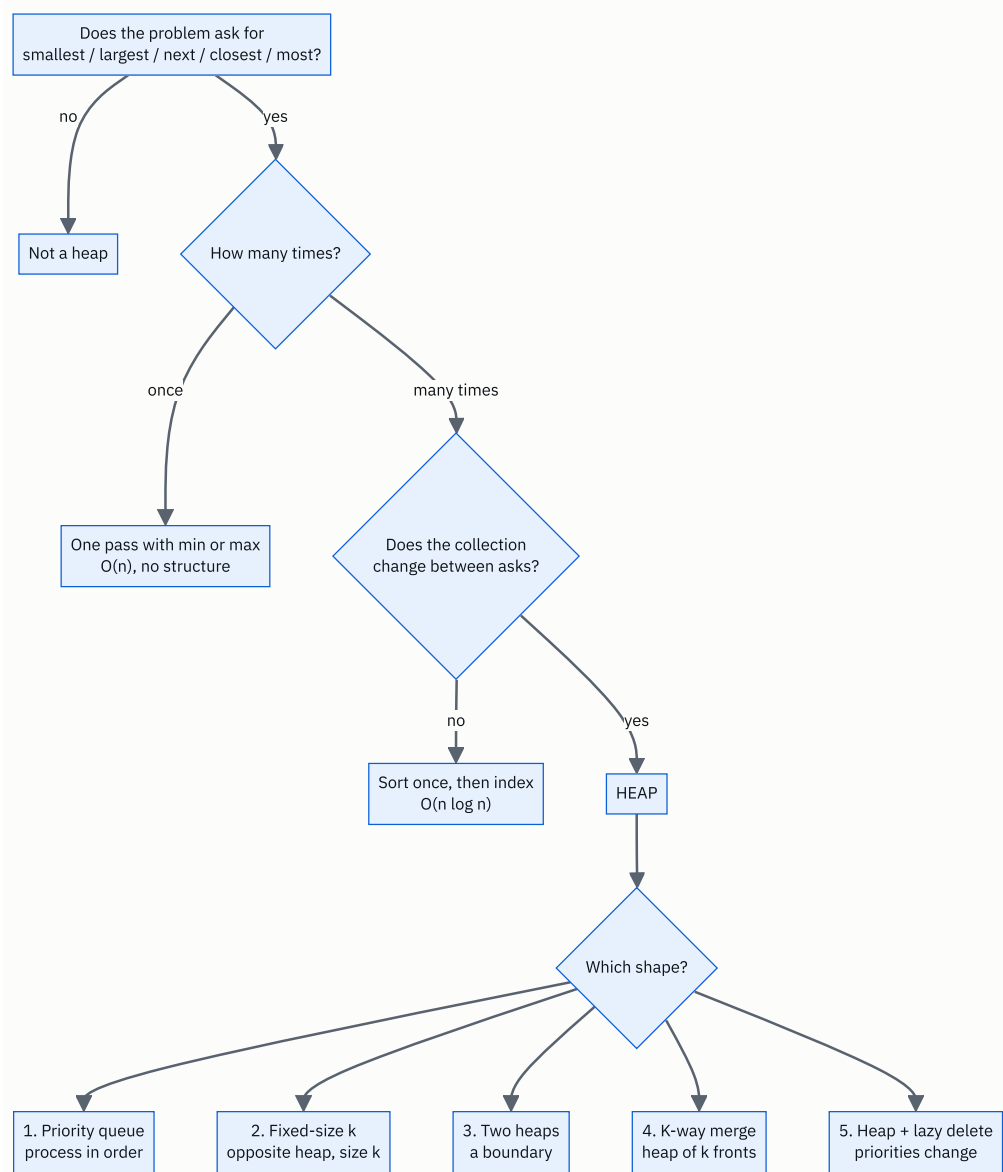


FIGURE 119.1

*Notice the two questions that send you away from the heap. “Once” and “does not change” are both very common, and both have simpler answers. The heap is the leaf you reach only after ruling those out.*

### The five shapes, side by side

SHAPE 1 – PRIORITY QUEUE  
"process in priority order"

heap holds: everything  
size:  $n$   
op: push all, pop all  
cost:  $O(n \log n)$   
ex: Dijkstra, task scheduler

SHAPE 3 – TWO HEAPS  
"the median, a percentile"

low = max-heap (small half)  
high = min-heap (large half)  
invariant:  $\max(\text{low}) \leq \min(\text{high})$   
            sizes differ by  $\leq 1$   
cost:  $O(\log n)$  insert,  $O(1)$  read  
ex: running median, sliding median

SHAPE 5 – HEAP + LAZY DELETION  
"priorities change, or items expire"

heap holds: stale entries too  
op: push the new version; on pop,  
    discard entries that are out of date  
cost:  $O(\log n)$  amortised,  $O(n)$  space worst case  
ex: Dijkstra without decrease-key, sliding-window median,  
    task queue with cancellation

TRAP:  $\text{len}(\text{heap})$  is NOT the logical size. Track that separately.

SHAPE 2 – FIXED-SIZE K  
"the k largest"

heap holds: exactly k  
size: k  
op: push, then pop if  $> k$   
cost:  $O(n \log k)$   
ex: top k frequent, k closest

TRAP: the heap is the OPPOSITE  
kind. k largest  $\rightarrow$  MIN-heap.

SHAPE 4 – K-WAY MERGE  
"merge k sorted lists"

heap holds: the k fronts  
size: k, never n  
op: pop, emit, push successor  
cost:  $O(n \log k)$   
ex: merge k lists, sorted matrix

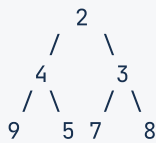
TRAP: push (value, index, ...) so  
ties never compare the payload.

*Notice that shapes 2, 4 and 5 all keep the heap small —  $k$ ,  $k$ , and “however many stale entries have piled up”. The size of the heap is the single most useful thing to say out loud when you start, because it is where the complexity comes from.*



## The array, one last time

tree view



array view

| index: | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|--------|---|---|---|---|---|---|---|
| value: | 2 | 4 | 3 | 9 | 5 | 7 | 8 |

parent(i) = (i - 1) // 2

left(i) = 2\*i + 1

right(i) = 2\*i + 2

RULE: every parent  $\leq$  both children.

NOT A RULE: left  $\leq$  right. Look at index 1 and 2: 4 then 3.

Look at 3 and 4: 9 then 5. Neither is sorted.

Notice the “not a rule” line. Half of all heap confusion comes from expecting the array to be sorted. It is not, it never will be, and it does not need to be — the only guaranteed fact is that index 0 holds the minimum.

### 5 The code, built step by step

This section is a set of templates. Learn these five and you can write any heap problem in an interview. Each one is short enough to write from memory, and that is the point — you should not be *designing* during a round, you should be recognising and then typing.

#### Shape 1 — the plain priority queue

Push everything, pop in order. This is the shape when the problem says “process these in priority order”.

```
import heapq

def process_in_order(tasks: list[tuple[int, str]]) → list[str]:
 """tasks are (priority, name). Lower number = higher priority."""
 heap = list(tasks)
 heapq.heapify(heap) # O(n), not n pushes
 done: list[str] = []
 while heap:
 priority, name = heapq.heappop(heap)
 done.append(name)
 return done
```

Two details worth having automatic. `heapify` is  $O(n)$  and  $n$  separate pushes are  $O(n \log n)$  — when you have all the data up front, always heapify. And the tuple orders by its first element, then its second, which is why `(priority, name)` sorts by priority

and breaks ties alphabetically for free.

### Shape 2 — the fixed-size k heap

The most common heap problem in interviews, and the one with the trap in it.

```
def k_largest(numbers: list[int], k: int) → list[int]:
 """The k largest values. Note: a MIN-heap."""
 heap: list[int] = []
 for value in numbers:
 heapq.heappush(heap, value)
 if len(heap) > k:
 heapq.heappop(heap) # evict the smallest
 return heap
```

Say the trap out loud every time, because it never stops feeling backwards: **for the k largest you keep a min-heap**. The top of a min-heap is the smallest thing you are keeping, which is exactly the thing to throw away when a better one arrives. The heap holds the survivors; its top is the weakest survivor.

The cost is  $O(n \log k)$ , and the space is  $O(k)$  — which is the real reason to do it this way. Sorting is  $O(n \log n)$  time and  $O(n)$  space. When  $n$  is a billion and  $k$  is ten, the space is what saves you.

### Shape 3 — two heaps

The boundary shape. `low` is a max-heap holding the small half; Python has no max-heap, so the values go in negated.

```
class Median:
 def __init__(self) → None:
 self.low: list[int] = [] # max-heap, values NEGATED
 self.high: list[int] = [] # min-heap

 def add(self, value: int) → None:
 heapq.heappush(self.low, -value) # 1. push
 heapq.heappush(self.high, -heapq.heappop(self.low)) # 2. shift
 if len(self.high) > len(self.low): # 3. rebalance
 heapq.heappush(self.low, -heapq.heappop(self.high))

 def median(self) → float:
 if len(self.low) > len(self.high):
 return float(-self.low[0])
 return (-self.low[0] + self.high[0]) / 2
```

Push-shift-rebalance is one unit — never write one of the three lines without the other two. The blind push into `low` is safe precisely *because* the shift follows it and moves `low`'s new top across, which is what restores  $\max(\text{low}) \leq \min(\text{high})$ .

#### Shape 4 — the k-way merge

```
def merge_k(lists: list[list[int]]) → list[int]:
 heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
 heapq.heapify(heap)
 out: list[int] = []
 while heap:
 value, list_index, position = heapq.heappop(heap)
 out.append(value)
 if position + 1 < len(lists[list_index]):
 nxt = lists[list_index][position + 1]
 heapq.heappush(heap, (nxt, list_index, position + 1))
 return out
```

Three fields, always: the value to order by, the list it came from, and where in that list. The middle field is a free tie-breaker — list indices are distinct integers, so two equal values never fall through to comparing the payload.

#### Shape 5 — the heap with lazy deletion

Used whenever an item's priority changes, or items expire, and you cannot reach into the heap to fix them.

```
def dijkstra(graph: dict[int, list[tuple[int, int]]], start: int) → dict[int, int]:
 best: dict[int, int] = {start: 0}
 heap = [(0, start)]
 while heap:
 distance, place = heapq.heappop(heap)
 if distance > best.get(place, float("inf")):
 continue # stale entry: skip it
 for neighbour, weight in graph[place]:
 candidate = distance + weight
 if candidate < best.get(neighbour, float("inf")):
 best[neighbour] = candidate
 heapq.heappush(heap, (candidate, neighbour))
 return best
```

The one line that matters is `if distance > best.get(...): continue`. You never remove the outdated entry. You leave it in the heap and ignore it when it surfaces. That is the whole technique, and it is why real Dijkstra implementations do not need a `decrease-key` operation.

#### The `heapq` checklist

Six facts. These are the things that actually go wrong.

```
import heapq

heapq.heapify(items) # O(n), in place, returns None
heapq.heappush(heap, item) # O(log n)
heapq.heappop(heap) # O(log n), returns the SMALLEST
heap[0] # O(1), peek, does not remove
heapq.heappushpop(heap, x) # push then pop, one sift, faster
heapq.heapreplace(heap, x) # pop then push, heap must be non-empty
heapq.nlargest(k, items) # O(n log k) when k << n
```

1. **Min-heap only.** For a max-heap, negate on the way in and on the way out. Two negations, never one.
2. **heapify returns None.** `heap = heapq.heapify(x)` gives you `None` and a confusing crash later.
3. **heap[0] is a peek, not a pop.** There is no `heapq.peak`.
4. **Tuples compare left to right.** Add a unique tie-breaker before any object you do not want compared.
5. **len(heap) is not the logical size** when you are deleting lazily. Track the real count yourself.
6. **You cannot delete an arbitrary element.** If you need to, delete it lazily or use a different structure.

The negation, written out once

For a max-heap of tuples, negate only the key you are ordering by, and remember there is one negation on the way in and one on the way out:

```
max_heap: list[tuple[int, str]] = []
heapq.heappush(max_heap, (-score, name)) # in: negate
top_score, top_name = max_heap[0]
top_score = -top_score # out: negate back
```

## The complete solution: one file with all five shapes

```
"""Day 119 – the five heap shapes, ready to run.

Each function is the smallest complete example of one shape.
"""

from __future__ import annotations

import heapq
from collections import Counter

----- Shape 1: priority queue -----

def schedule(tasks: list[tuple[int, str]]) → list[str]:
 """Return task names in priority order. Lower number goes first."""
 heap = list(tasks)
 heapq.heapify(heap)
 return [heapq.heappop(heap)[1] for _ in range(len(heap))]

----- Shape 2: fixed-size k -----

def top_k_frequent(words: list[str], k: int) → list[str]:
 """The k most frequent words. O(n log k) time, O(n) space for the counts."""
 counts = Counter(words)
 heap: list[tuple[int, str]] = []
 for word, count in counts.items():
 heapq.heappush(heap, (count, word))
 if len(heap) > k:
 heapq.heappop(heap)
 return [word for count, word in sorted(heap, reverse=True)]

def k_closest_points(points: list[tuple[int, int]], k: int) → list[tuple[int, int]]:
 """The k points closest to the origin. Max-heap of size k, via negation."""
 heap: list[tuple[int, int, int]] = []
 for x, y in points:
 distance = x * x + y * y # no square root needed
 heapq.heappush(heap, (-distance, x, y))
 if len(heap) > k:
 heapq.heappop(heap) # evict the farthest
 return [(x, y) for negative_distance, x, y in heap]

----- Shape 3: two heaps -----

class MedianFinder:
 """Running median in O(log n) per insert, O(1) per query."""

 def __init__(self) → None:
 self.low: list[int] = [] # max-heap, values negated
 self.high: list[int] = [] # min-heap

 def add(self, value: int) → None:
 heapq.heappush(self.low, -value)
 heapq.heappush(self.high, -heapq.heappop(self.low))
 if len(self.high) > len(self.low):
 heapq.heappush(self.low, -heapq.heappop(self.high))

 def median(self) → float:
 if not self.low:
 raise ValueError("no values yet")
```

```

 if len(self.low) > len(self.high):
 return float(-self.low[0])
 return (-self.low[0] + self.high[0]) / 2

----- Shape 4: k-way merge -----

def merge_sorted_lists(lists: list[list[int]]) → list[int]:
 """Merge k sorted lists. O(n log k) time, O(k) heap space."""
 heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
 heapq.heapify(heap)
 merged: list[int] = []
 while heap:
 value, list_index, position = heapq.heappop(heap)
 merged.append(value)
 following = position + 1
 if following < len(lists[list_index]):
 heapq.heappush(heap, (lists[list_index][following], list_index, following))
 return merged

----- Shape 5: heap with lazy deletion -----

def shortest_paths(
 graph: dict[str, list[tuple[str, int]]], start: str
) → dict[str, int]:
 """Dijkstra with lazy deletion instead of decrease-key."""
 best: dict[str, int] = {start: 0}
 heap: list[tuple[int, str]] = [(0, start)]
 while heap:
 distance, place = heapq.heappop(heap)
 if distance > best.get(place, 10**18):
 continue # stale: a better route won already
 for neighbour, weight in graph.get(place, []):
 candidate = distance + weight
 if candidate < best.get(neighbour, 10**18):
 best[neighbour] = candidate
 heapq.heappush(heap, (candidate, neighbour))
 return best

if __name__ == "__main__":
 print(schedule([(3, "post"), (1, "fire"), (2, "leak")]))
 # ['fire', 'leak', 'post']

 print(top_k_frequent(["a", "b", "a", "c", "b", "a"], 2))
 # ['a', 'b']

 print(k_closest_points([(1, 1), (5, 5), (0, 2), (9, 9)], 2))
 # two closest, order not guaranteed

 finder = MedianFinder()
 for number in [5, 15, 1, 3]:
 finder.add(number)
 print(number, "→", finder.median())
 # 5 → 5.0 / 15 → 10.0 / 1 → 5.0 / 3 → 4.0

 print(merge_sorted_lists([[1, 4, 9], [2, 3], [], [0, 10]]))
 # [0, 1, 2, 3, 4, 9, 10]

 roads = {
 "home": [("market", 4), ("park", 1)],
 "park": [("market", 2), ("school", 7)],
 "market": [("school", 3)],
 "school": [],
 }

```

```

}
print(shortest_paths(roads, "home"))
{'home': 0, 'market': 3, 'park': 1, 'school': 6}

```

Run it. Every printed line is in the comments so you can check without thinking.

## 6 What it costs

### The one table

| OPERATION                 | COST          | WHY                                   |
|---------------------------|---------------|---------------------------------------|
| heappush                  | $O(\log n)$   | one sift up, at most the height       |
| heappop                   | $O(\log n)$   | one sift down, at most the height     |
| heap[0] (peek)            | $O(1)$        | it is just index 0                    |
| heapify                   | $O(n)$        | not $O(n \log n)$ — see below         |
| build by n pushes         | $O(n \log n)$ | n separate sifts                      |
| find an arbitrary value   | $O(n)$        | the heap is not ordered left to right |
| delete an arbitrary value | $O(n)$        | find it first                         |
| heapq.nlargest(k, xs)     | $O(n \log k)$ | a size-k heap internally              |
| sort                      | $O(n \log n)$ | for comparison                        |

Why `heapify` is  $O(n)$  and not  $O(n \log n)$

This gets asked, so have the arithmetic ready. `heapify` sifts down from the last parent up to the root. Most nodes are near the bottom, and nodes near the bottom sift down almost no distance.

For `n = 1,000,000`:

| level  | nodes at level | max sift distance | work                |
|--------|----------------|-------------------|---------------------|
| -----  |                |                   |                     |
| bottom | 500,000        | 0                 | 0                   |
| next   | 250,000        | 1                 | 250,000             |
| next   | 125,000        | 2                 | 250,000             |
| next   | 62,500         | 3                 | 187,500             |
| next   | 31,250         | 4                 | 125,000             |
| ...    |                |                   |                     |
| root   | 1              | 19                | 19                  |
| -----  |                |                   |                     |
| total  |                |                   | $\approx 1,000,000$ |

Half the nodes do zero work. A quarter do one step. The sum  $n/2 \times 0 + n/4 \times 1 + n/8 \times 2 + \dots$  converges to  $n$ . Compare with a million pushes, which is  $1,000,000 \times 20 = 20,000,000$  steps. **Heapify is twenty times cheaper here, and it is one line.** Use it whenever you have the data up front.

The k versus n arithmetic

Where the fixed-size shape actually pays. Take  $n = 1,000,000$  :

| approach               | comparisons | memory held     |
|------------------------|-------------|-----------------|
| sort, take k           | 20,000,000  | 1,000,000 items |
| heap of size k=10      | 3,300,000   | 10 items        |
| heap of size k=100     | 6,600,000   | 100 items       |
| heap of size k=1,000   | 10,000,000  | 1,000 items     |
| heap of size k=100,000 | 17,000,000  | 100,000 items   |
| heap of size k=500,000 | 19,000,000  | 500,000 items   |

$\log_2(10) = 3.3$  ,  $\log_2(100) = 6.6$  ,  $\log_2(1,000,000) = 20$  . Two things fall out. The time advantage **shrinks as k grows** and is nearly gone by  $k = n/2$  . The memory advantage does not shrink — it is always  $k$  against  $n$  , and at  $k = 10$  that is a hundred thousand times less. **When you justify the heap in an interview, lead with the space.** It is the stronger argument and most candidates never make it.

The stream argument

The third and best reason, and it has no arithmetic — it has a constraint. Sorting needs all the data at once. If the values arrive one at a time and there are more of them than fit in memory, sorting is not slow, it is *impossible*. A size-k heap handles an infinite stream in  $O(k)$  space. Say this whenever the problem mentions a stream, a log, a feed, or “as they arrive”.

7 The traps

**Trap 1: the wrong heap for the k problem.** For the *k largest* you keep a *min*-heap. Every single person gets this backwards at first. The fix is to say what the top of the heap is *for*: it is the thing you throw away. For the *k largest*, you throw away the smallest survivor.

|            |                      |                               |            |
|------------|----------------------|-------------------------------|------------|
| k largest  | → MIN-heap of size k | → top is the weakest survivor | → evict it |
| k smallest | → MAX-heap of size k | → top is the worst survivor   | → evict it |



**Trap 2: forgetting a negation, or doing it once.** For a max-heap you negate going in and negate coming out. One negation gives you numbers that look almost right.

```
>>> heap = []
>>> heapq.heappush(heap, -50)
>>> heapq.heappush(heap, -20)
>>> heap[0] # forgot to negate back
-50
```

No error. Just a minus sign that flows into your answer.

**Trap 3: comparing objects that cannot be compared.**

```
>>> import heapq
>>> class Task: pass
>>> heap = []
>>> heapq.heappush(heap, (1, Task()))
>>> heapq.heappush(heap, (1, Task()))
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
TypeError: '<' not supported between instances of 'Task' and 'Task'
```

Note *when* it fails: only when the first fields tie. It passes your first test and fails in production. Always push a unique tie-breaker — a counter, an index — before the payload.

**Trap 4: assuming the array is sorted.**

```
>>> heap = [5, 3, 8, 1, 9, 2]
>>> heapq.heapify(heap)
>>> heap
[1, 3, 2, 5, 9, 8]
```

1, 3, 2 — index 1 holds 3 and index 2 holds 2. It is not sorted and never will be. Only `heap[0]` is guaranteed. If someone asks for the second-smallest, it is *not* `heap[1]`; it is `min(heap[1], heap[2])`.

**Trap 5: `heapify` returns `None`.**

```
>>> heap = heapq.heapify([5, 3, 8])
>>> heap[0]
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
TypeError: 'NoneType' object is not subscriptable
```

It sorts in place. Call it as a statement.

### Trap 6: popping from an empty heap.

```
>>> heapq.heappop([])
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
IndexError: index out of range
```

`while heap:` rather than a fixed count, or check the length.

**Trap 7: using `len(heap)` as the logical size with lazy deletion.** If half the entries are stale, `len(heap)` is roughly twice the truth. Every rebalance and every size check reads the wrong number. Keep your own counter.

**Trap 8: reaching for a heap when a sort is right there.** If the input is a fixed list and you need the whole thing ordered, `sorted(items)` is faster and one line. Using a heap there is a small signal that you are pattern-matching rather than thinking. Say the reason you chose it.

## 8 In the interview

### How it gets asked

Heap problems almost never say “heap”. They say:

- “Find the  $k$  most frequent elements in this list.”
- “Return the  $k$  closest points to the origin.”
- “Merge these sorted lists.”
- “Design a system that tracks the median of a stream of numbers.”
- “You have a stream of stock prices. Report the top 10 gainers at any moment.”
- “Given a list of meetings, find the minimum number of rooms needed.”
- “Find the shortest path in a weighted graph.” — Dijkstra is a heap problem in a coat.

And the ones that test whether you know when *not* to:

- “Sort these million numbers.” — not a heap. Say so.
- “Find the maximum value in this list.” — one pass, `O(n)`. Not a heap.

### The first ninety seconds

Say this, in this order, before any code:

“Let me check what kind of question this is. I need the largest values, and I need them from a collection that keeps changing as I scan — so this is repeated ‘give me the extreme’, which is a heap rather than a sort.

Specifically it is the fixed-size shape: I keep a heap of exactly  $k$ , and the top of it is the weakest thing I am keeping, so it is the one I evict when something better arrives. Since I want the  $k$  *largest*, the heap is a *min*-heap. That feels backwards and it is worth saying out loud.

That gives me  $O(n \log k)$  time and — this is the part I actually care about —  $O(k)$  space. Sorting is  $O(n \log n)$  and  $O(n)$ . If  $n$  is a billion and  $k$  is ten, the space is the difference between this working and not.

One more thing: because the heap only ever holds  $k$  items, this works on a stream. I never need all the data at once. Shall I write it?”

That is ninety seconds and it contains the recognition, the shape, the trap, both costs, and the streaming argument. Very few candidates give the space and streaming reasons at all.

#### The follow-ups

“What if  $k$  is close to  $n$ ?”

“Then the heap stops being worth it.  $O(n \log k)$  and  $O(n \log n)$  are the same thing when  $k = n$ , and sorting has a smaller constant factor and is one line. The crossover in practice is around  $k = n/10$ . Below that, heap; above it, I would sort. And if  $k$  is larger than  $n/2$ , there is a neat trick — find the  $n - k$  smallest instead and take the complement.”

“Can you do it without a heap?”

“Quickselect. It partitions around a pivot like quicksort but only recurses into the side containing the  $k$ th element, so the average is  $O(n)$  — better than the heap. Two catches. The worst case is  $O(n^2)$  with a bad pivot, though a random pivot makes that vanishingly unlikely. And it needs the whole array in memory and rearranges it, so it does not work on a stream. If the data fits and I want the  $k$  largest once, quickselect. If it is a stream or I need to keep answering, heap.”

“How do you change an item’s priority?”

“A heap cannot do it directly — finding the item is  $O(n)$  because the array is not ordered left to right. Two options. Keep a dictionary from item to its index in the array and update it on every swap, which gives  $O(\log n)$  decrease-key but is fiddly and easy to get wrong. Or delete lazily: push the new version, leave the old one, and skip entries when they surface that no longer match your best-known value. That is one extra line and it is what real Dijkstra implementations do. The cost is memory — the heap can hold up to one entry per push. I would do lazy deletion unless memory was the constraint.”

**“Why is heapify  $O(n)$  when  $n$  pushes are  $O(n \log n)$ ?”**

“Because most nodes are near the bottom and barely move. Half the nodes are leaves and sift down zero steps, a quarter sift one, an eighth sift two. The sum  $n/2 \times 0 + n/4 \times 1 + n/8 \times 2 + \dots$  converges to  $n$ . Pushing is the other direction — every push starts at the bottom and can travel the full height, so every one of them is potentially  $\log n$ . Practically: at a million elements heapify is about a million steps and  $n$  pushes is about twenty million.”

**The model answer**

*“Design something that returns the 10 most-viewed articles right now, over a stream of view events.”*

“Let me get the shape first. Views arrive continuously, and at any moment someone can ask for the top ten. So it is repeated ‘give me the extreme’ from a changing collection — a heap, not a sort. And  $k$  is ten while  $n$  is unbounded, so it is the fixed-size shape.

The naive version: a dictionary from article to view count, and a min-heap of size ten holding `(count, article)`. On each view I bump the count. If the article is already in the heap its count is now stale, and if it is not in the heap I compare against `heap[0]` and swap if it beats it.

Straight away there is a bug I should name. The counts inside the heap go stale, because I bump the dictionary but the heap holds a copy. So this is really the fifth shape — a heap with lazy deletion. When I pop, I check the entry’s count against the dictionary and discard it if they disagree. That is one line and it is the line most people miss.

Costs. The dictionary is  $O(\text{distinct articles})$  — say ten million articles at fifty bytes, that is 500 MB, which is a lot but survivable on one machine. The heap is  $O(k)$ , nothing. Each view is  $O(\log k)$  which at  $k = 10$  is about three comparisons — at a hundred thousand views a second that is trivial.

Now the honest part. At real scale you would not do this. The dictionary is the problem, not the heap — it grows with distinct articles and it lives on one machine. Production would use a sketch: count-min sketch or a Space-Saving structure, which gives approximate counts in bounded memory, a few megabytes for millions of items, with a small error you can bound. You trade exactness for memory, and for ‘top ten articles’ nobody cares if the tenth is occasionally the eleventh.

If they wanted exact counts and a real-time window, I would shard by article ID, keep a per-shard top- $k$ , and merge the shard results — which is a  $k$ -way merge, the fourth shape, so the same tool twice at different levels.

The one thing I would push back on: ‘right now’ needs defining. All-time, last hour, or last day? A time window changes the answer completely — I would need per-bucket counts and to expire old buckets, which is a different structure. I would ask before building.”

That answer names the shape, finds the staleness bug unprompted, gives the arithmetic, admits where the approach fails, and asks the clarifying question. That is what a strong round sounds like.

### Mock round: two problems, timed

Do these before reading the walkthroughs. Twenty minutes each. Talk out loud the whole time.

#### MOCK PROBLEM 1 – TASK SCHEDULER WITH COOLDOWN

You have a list of tasks, each labelled `A` to `Z`. Each unit of time you can run one task or stay idle. The same task cannot run twice within `n` units of time. Return the minimum time to finish everything.

Example: `["A","A","A","B","B","B"]`, `n = 2`  $\rightarrow$  `8`. One valid order is `A B idle A B idle A B`.

**Minutes 0-3, the recognition.** “Every step I need the task with the most work left that is not on cooldown. That is repeated ‘give me the largest’ from a changing set — heap. Counts change as I run tasks, so it is a plain priority queue over counts, with a side structure for the cooling-down ones.”

**Minutes 3-6, the picture.** A max-heap of remaining counts. A queue of `(count, time_it_becomes_ready)` for tasks on cooldown. Each tick: move anything whose ready-time has arrived from the queue back to the heap; pop the heap and run it; if it has work left, put it on the cooldown queue.

#### Minutes 6-15, the code.

```
import heapq
from collections import Counter, deque

def least_interval(tasks: list[str], n: int) -> int:
 counts = Counter(tasks)
 heap = [-count for count in counts.values()] # max-heap via negation
 heapq.heapify(heap)
 cooling: deque[tuple[int, int]] = deque() # (negative count, ready time)
 time = 0
 while heap or cooling:
 time += 1
 if cooling and cooling[0][1] == time:
 heapq.heappush(heap, cooling.popleft()[0])
 if heap:
 remaining = heapq.heappop(heap) + 1 # one less; less negative
 if remaining:
 cooling.append((remaining, time + n + 1))
 return time
```

**Minutes 15-20, the analysis.** “Time is  $O(\text{total tasks} \times \log 26)$ , and since there are only 26 labels the log is a constant — effectively  $O(\text{total})$ . Space is  $O(26)$ , constant. The negation is the trap: `+1` makes a negative number *less* negative, which is

the right direction, and `if remaining:` is true for any non-zero value including negatives, which is exactly what I want.”

If you finish early, the follow-up is always: *is there a formula?* There is — `max((most_frequent - 1) * (n + 1) + number_of_tasks_tied_for_most, len(tasks))`. Mention that you know it exists and that the simulation is the one you would trust under pressure.

#### MOCK PROBLEM 2 — THE K-TH SMALLEST SUM FROM TWO ARRAYS

Given two sorted arrays `a` and `b`, return the  $k$ -th smallest value of `a[i] + b[j]` over all pairs.

`a = [1, 7, 11]`, `b = [2, 4, 6]`, `k = 3` → 9 (the sums in order are 3, 5, 7, 9, ...).

**The recognition.** “There are `len(a) * len(b)` sums, which could be a trillion — I cannot build them all. But I need them in increasing order and I only need the first  $k$ . Repeated ‘give me the smallest’ from a changing set: heap.”

**The insight, and this is the whole problem.** Think of the sums as a grid where row  $i$  is `a[i] + b[0]`, `a[i] + b[1]`, ... . Every row is sorted, because `b` is sorted. So this is *merging `len(a)` sorted lists* — the fourth shape — and stopping after  $k$  pops.

**The code.**

```
def kth_smallest_sum(a: list[int], b: list[int], k: int) → int:
 heap = [(a[i] + b[0], i, 0) for i in range(min(len(a), k))]
 heapq.heapify(heap)
 for _ in range(k - 1):
 total, i, j = heapq.heappop(heap)
 if j + 1 < len(b):
 heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
 return heap[0][0]
```

**The analysis.** “The heap holds at most `min(len(a), k)` entries, so `O(k log k)` time and `O(k)` space, regardless of how enormous the full grid is. `min(len(a), k)` rather than `len(a)` matters: if `a` has a million entries and `k` is five, seeding a million-entry heap would be silly — I can never pop more than  $k$  of them.”

The point of this mock is the reframing. The problem does not look like a merge. Saying “*each row is sorted, so this is a  $k$ -way merge*” out loud is the entire solution, and it is the sentence the interviewer is listening for.

**9 Recall card**

**The recognition question:** do I repeatedly need the smallest or largest, from a collection that keeps changing? Yes → heap. Fixed collection → sort. Whole order → sort. Once → one pass.

**The three phrasings:** a superlative asked repeatedly; a schedule; a boundary.

**The five shapes:**

| # | SHAPE          | HEAP HOLDS               | COST                         | EXAMPLE                  |
|---|----------------|--------------------------|------------------------------|--------------------------|
| 1 | Priority queue | everything               | $O(n \log n)$                | scheduler, Dijkstra      |
| 2 | Fixed-size k   | exactly k, opposite kind | $O(n \log k)$ , $O(k)$ space | top k frequent           |
| 3 | Two heaps      | the two halves           | $O(\log n)$ in, $O(1)$ read  | running median           |
| 4 | K-way merge    | the k fronts             | $O(n \log k)$                | merge k lists            |
| 5 | Lazy deletion  | stale entries too        | $O(\log n)$ amortised        | Dijkstra, sliding median |

**The costs:** push  $O(\log n)$ , pop  $O(\log n)$ , peek  $O(1)$ , heapify  $O(n)$ , find anything  $O(n)$ .

**Heapify is  $O(n)$**  because half the nodes are leaves that move zero steps:  $n/2 \times 0 + n/4 \times 1 + n/8 \times 2 + \dots \rightarrow n$ .

**The k trap:** k *largest* → *min*-heap. The top is the weakest survivor, the one you evict.

**The `heapq` six:** min-heap only (negate twice); `heapify` returns `None`; `heap[0]` peeks; tuples compare left to right so add a tie-breaker; `len(heap)` lies when deleting lazily; no arbitrary delete.

**The array is not sorted.** Only `heap[0]` is guaranteed. Second-smallest is `min(heap[1], heap[2])`.

**Lead with space, not time.**  $O(k)$  versus  $O(n)$  is the strong argument, and it is the one that makes streams possible at all.

**Not a heap when:** the data is fixed, you need the whole order,  $k \approx n$ , or you must update items by value.

**Next:** [Day 120 — The trie: a tree of characters](#)

**Previous:** [Day 118 — Two heaps: the running median](#)



## 1 What this is, and why they ask it

Yesterday you elected a leader. Today you find out that electing a leader is a *special case* of a much bigger problem, and that the bigger problem has one good answer which almost every serious distributed system now uses.

The bigger problem is **consensus**: getting a group of machines to agree on something, and to *stay* agreed, even though some of them crash, some of them are slow, and messages between them get lost, delayed, or delivered out of order.

That sounds abstract. It is not. Every one of these is the same problem:

- Who is the leader? (yesterday)
- Which machine currently owns this lock?
- What is the current configuration — how many machines are in the group, and which?
- In what order did these writes happen?
- Did this transaction commit, or not?

The answer to all five is: **agree on an ordered list of decisions, and make sure every machine has the same list**. That is what a consensus algorithm does. Not one value — a *sequence*, forever.

Raft is the consensus algorithm you should be able to explain. It was designed in 2013 explicitly to be understandable, because the previous standard — Paxos — was correct but famously hard to follow and harder to implement. Raft is what runs inside etcd, Consul, CockroachDB, TiKV, MongoDB's replica sets, and the control plane of most Kubernetes clusters on earth.

By the end you will be able to say what consensus is, describe Raft in three parts, explain why the log is the real answer, give the numbers, and — the part that separates a good answer from a memorised one — say when you should *not* use consensus.

## 2 The story

The Pazhaya Bazaar shopkeepers' association has run the same way for forty years. Every Sunday evening, five committee members meet above the cloth shop and take decisions: who gets the corner stall for the festival, how much the sweepers are paid,

whether the water tank is repaired this month.

For thirty of those forty years the association was in a permanent, low-grade mess. Not because people were dishonest — because people were *absent*.

Somebody would miss a Sunday. The next week they would arrive with a completely reasonable idea, the meeting would run, and a decision would be taken that quietly contradicted one taken while they were away. Two members would each write notes in their own diaries and the diaries would not match. Once, memorably, two different men both believed they had been given the corner stall for the same festival, both had witnesses, and both were right about what they remembered.

Then Ismail became secretary and changed three things, and the mess simply stopped.

The first thing was one book. Not five diaries — one book, and copies of it. Every decision goes in the book as a numbered line. Never in the middle. Never out of order. Line 41 goes after line 40 and before line 42, always.

The second thing was a rule about what counts. A decision is not a decision because it was said in the room. A decision counts once **at least three of the five have written that exact line, at that exact number, in their copy of the book**. Three out of five. Until three copies carry it, the line is only a proposal, and Ismail will not act on it.

The third thing was the rule that made it actually work, and it took him a while to see it. Before you copy line 41 into your book, you must first show Ismail that your book already has lines 1 to 40, and that they match his. If your book stops at line 38, you do not get line 41. You get 39 and 40 first, in that order, and only then 41.

A young member once objected that this was slow. Why not just let everyone write down whatever they had heard and sort it out later?

“Because that is what we did for thirty years,” Ismail said. “Look — I do not need everyone to agree. I have never had everyone. Someone is always at a wedding. I need three, and I need the three to be agreeing about the *same numbered line*. Then when the fourth man comes back from the wedding, he does not argue. He copies. He was not there, so he does not get a vote on what happened. He gets a copy.”

The young member asked what happens if Ismail himself misses a Sunday.

Ismail smiled at that, because it was the right question. “Then the four of you pick someone to hold the book. And the first thing you check, before you pick anybody, is whose copy has the most lines in it. Not who is senior. Not who shouts. Whose book is longest. Give it to him, because everything anybody agreed to is somewhere in that book, and the man with the longest book cannot be missing it.”

### 3 The idea in plain English

Ismail rebuilt Raft from scratch, and the three things he changed are the three parts of it.

**Consensus, defined properly.** A group of machines must agree on a value, such that:

- **Agreement** — no two machines decide different values.
- **Validity** — the value decided was actually proposed by someone (nobody invents it).
- **Termination** — every non-failed machine eventually decides.

Getting agreement on *one* value is called single-decree consensus, and it is a solved but useless problem. Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around **a replicated log**: an ordered, numbered list of entries, identical on every machine.

**Why the log is the whole trick.** This is the idea worth carrying out of this lesson. If every machine starts in the same state and applies the same list of changes in the same order, every machine ends in the same state. That is it. It is called **state machine replication**, and it converts a hard problem — “keep five machines’ data identical” — into a slightly less hard problem: “make five machines agree on the order of a list.”

You have already seen why order matters. `set balance = 100` then `add 50` gives 150. Swap them and you get 100. Same operations, different order, different money. Agreeing on the *set* of operations is not enough; you must agree on the sequence.

**Raft in three parts.** That is genuinely all there is:

1. **Leader election.** One machine is the leader; the rest are followers. Yesterday’s lesson.
2. **Log replication.** The leader takes every client request, appends it to its own log, and sends it to the followers. Once a majority have written it down, the entry is **committed** and can be applied.
3. **Safety.** A set of restrictions that guarantee a newly elected leader cannot destroy a committed entry.

**Commitment is the word to be careful with.** An entry in a leader’s log is not yet real. It becomes real — *committed* — the moment a majority of the group have it stored. Ismail’s “three of the five have written it down”. Before that moment the leader

will not tell the client “done”, and will not apply it. After that moment, the entry is permanent: it will survive any sequence of failures the group is designed to survive, because any future majority necessarily overlaps this one.

That overlap is the same pigeonhole argument as the quorum rule from day 117. Any two majorities of five share at least one member. So the majority that elects the next leader always contains at least one machine that has every committed entry.

**Safety is the part people skip, and it is the interesting part.** Raft adds two restrictions:

- **The election restriction.** A machine only votes for a candidate whose log is *at least as up-to-date as its own*. Ismail’s “whose book is longest”. This is what makes it impossible to elect a leader that is missing a committed entry — because a candidate missing one cannot get votes from the majority that has it.
- **The current-term restriction.** A leader never commits an entry from a *previous* term just because it is now on a majority. It only counts an entry as committed once it has replicated an entry from its *own* term. This one is subtle and there is a famous diagram about it; the short version is that an entry sitting on a majority is not necessarily safe if it was written by an earlier, now-dead leader, because a different future leader could still legally overwrite it.

**The log-matching property**, Ismail’s third rule. When the leader sends entry 41, it also says “this should follow entry 40 from term 3”. A follower whose log does not match at that point *refuses*. The leader then walks backwards until it finds where the two logs agree, and overwrites everything after that. The result is a strong guarantee: **if two logs contain an entry with the same index and the same term, then the logs are identical in every entry up to that point.** One comparison verifies the entire history.

---

#### 4 The picture

State machine replication — the whole idea in one diagram

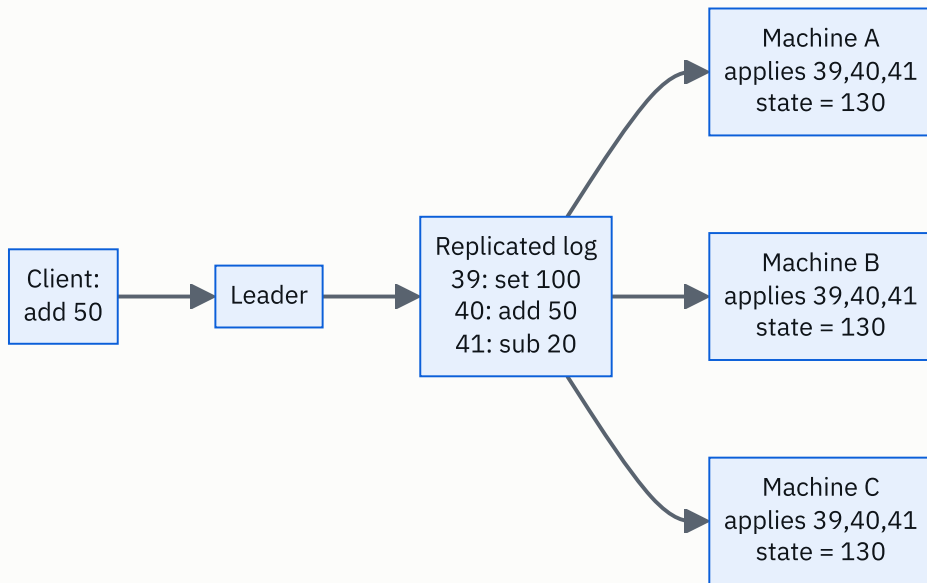


FIGURE 119.2

*Notice that the machines never compare their state with each other. They only agree on the list. Identical state falls out for free, which is why this is the standard way to build a replicated anything.*

### The commit rule

Five machines. The leader has appended entry 41 to its own log.

```
A (leader) [...38][39][40][41]
B [...38][39][40][41] ← acknowledged
C [...38][39][40][41] ← acknowledged
D [...38][39] (slow, or down)
E [...38] (down)
```

Count of machines holding 41: A, B, C = 3 of 5 = MAJORITY

→ entry 41 is COMMITTED

→ the leader applies it and answers the client

→ D and E will receive it later; they do not get a say

Why this is permanent:

any future leader needs 3 votes out of 5.

Any 3 of {A,B,C,D,E} must include at least one of {A,B,C}.

So any future leader hears from someone holding entry 41.

And the election restriction means it cannot win without having it.

*Notice that D and E being behind is not an error condition. It is the normal state of a healthy cluster. Somebody is always at a wedding.*

### The three parts, and what each one guarantees

| PART           | GUARANTEES                                                                                            | IF IT FAILS                         |
|----------------|-------------------------------------------------------------------------------------------------------|-------------------------------------|
| 1. Election    | at most one leader per term<br>(majority + one vote per term)                                         | split brain                         |
| 2. Replication | every follower's log becomes<br>identical to the leader's<br>(log matching + backtracking)            | divergent state                     |
| 3. Safety      | a new leader never loses a<br>committed entry<br>(election restriction +<br>current-term commit rule) | committed data<br>silently vanishes |

The first two make it WORK. The third makes it CORRECT.

Most explanations cover the first two and stop. Do not stop.

## Log repair, step by step

```
Leader (term 5) has: [1:a][2:b][3:c][4:d][5:e]
Follower F has: [1:a][2:b][3:x][4:y]
 ^^^^ diverged: entries from a dead leader

Leader sends entry 5, saying "previous is index 4, term 5".
F checks index 4: it has term 3 there, not 5. REJECT.

Leader steps back, sends entry 4, "previous is index 3, term 5".
F checks index 3: term 3, not 5. REJECT.

Leader steps back, sends entry 3, "previous is index 2, term 2".
F checks index 2: term 2. MATCH. ACCEPT.

F now truncates everything after index 2 and takes the leader's entries:
F becomes: [1:a][2:b][3:c][4:d][5:e]

The follower's uncommitted entries x and y are DESTROYED. That is allowed –
they were never committed, so no client was ever told they succeeded.
```

*Notice the last line, because it is the reassuring part. Raft does overwrite follower logs, but only entries that were never committed, which means no client was ever promised anything about them.*

## 5 How it actually works

### The three states

Every machine is in exactly one of three states, and there are only four transitions between them.

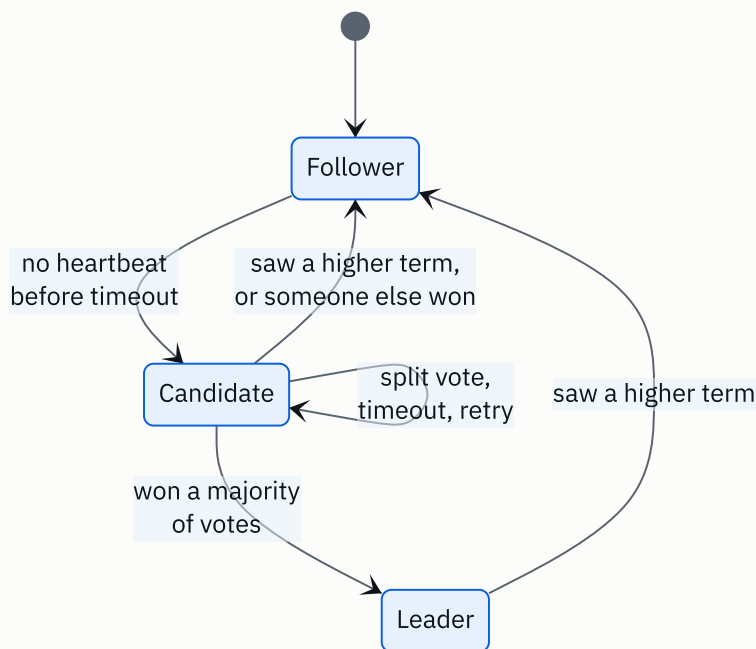


FIGURE 119.3

The transition that surprises people is **Leader → Follower**. A leader that sees a message carrying a term number higher than its own **immediately steps down**, without argument. It does not check whether it is still healthy or still has followers. A higher term means the world has moved on, and the leader's job is to notice and stop. This single rule is what stops a returning old leader from fighting.

### What a log entry contains

```

from dataclasses import dataclass
from typing import Any

@dataclass(frozen=True)
class LogEntry:
 term: int # the term of the leader that created this entry
 index: int # position in the log, starting at 1
 command: Any # what to apply, e.g. ("set", "balance", 100)

```

The `term` field is what makes log matching work. Index alone is not enough — two different leaders in two different terms can both write an entry at index 41, and they will be different entries. `(index, term)` identifies an entry uniquely across the whole life of the cluster.



## The leader's state

```
@dataclass
class LeaderState:
 current_term: int
 log: list[LogEntry]
 commit_index: int # highest entry known committed
 last_applied: int # highest entry applied to the state machine

 next_index: dict[str, int] # per follower: the next entry to send
 match_index: dict[str, int] # per follower: highest entry known replicated
```

`next_index` starts optimistic — the leader assumes each follower is fully caught up — and walks backwards on rejection. `match_index` is the pessimistic, confirmed truth, and it is what the commit calculation reads.

## The replication cycle

```
def append_command(self, command: Any) → None:
 """Client request arrives at the leader."""
 entry = LogEntry(term=self.current_term, index=len(self.log) + 1, command=command)
 self.log.append(entry)
 self.replicate_to_all_followers() # in parallel, not one by one
```

Note that the leader appends locally *first*, then sends. Its own log counts as one of the majority.

```
def on_append_response(self, follower: str, success: bool, their_index: int) → None:
 if success:
 self.match_index[follower] = their_index
 self.next_index[follower] = their_index + 1
 self.recompute_commit_index()
 else:
 self.next_index[follower] -= 1 # step back and retry
```

The naive backtracking is one entry per round trip, which is slow if a follower is a thousand entries behind. Real implementations have the follower return a hint — “my log for that term starts at index 200” — so the leader jumps straight there. Worth mentioning as a refinement.

## The commit calculation, with the restriction

This is the function that people get wrong, so read the last condition carefully.

```

def recompute_commit_index(self) → None:
 """Advance commit_index to the highest entry stored on a majority."""
 for candidate in range(len(self.log), self.commit_index, -1):
 holders = 1 + sum(1 for m in self.match_index.values() if m ≥ candidate)
 if holders ≤ len(self.cluster) // 2:
 continue # not a majority
 if self.log[candidate - 1].term ≠ self.current_term:
 continue # THE RESTRICTION
 self.commit_index = candidate
 break

```

The `1 +` is the leader counting itself. The last check is the current-term restriction: **a leader never declares an old-term entry committed on majority count alone**. It waits until an entry from its own term is committed, which — because commitment is cumulative — carries all the earlier entries with it. Without this check there is a real sequence of five failures that loses acknowledged data. It is the single most commonly omitted rule in hand-rolled implementations.

#### The vote, with the election restriction

```

def on_vote_request(self, term: int, candidate: str,
 their_last_index: int, their_last_term: int) → bool:
 if term < self.current_term:
 return False # stale candidate
 if term > self.current_term:
 self.current_term = term
 self.voted_for = None
 self.state = "follower" # step down

 if self.voted_for not in (None, candidate):
 return False # one vote per term

 my_last = self.log[-1] if self.log else LogEntry(0, 0, None)
 up_to_date = (their_last_term, their_last_index) ≥ (my_last.term, my_last.index)
 if not up_to_date:
 return False # THE RESTRICTION

 self.voted_for = candidate
 self.reset_election_timer()
 return True

```

The comparison is by **term first, then index** — a longer log with an older last term loses to a shorter log with a newer one. This is Ismail’s “whose book is longest”, made precise. A candidate whose log is missing a committed entry cannot be up-to-date relative to the majority that holds it, so it cannot win.

#### The client’s side, which is more awkward than people expect

Three problems the algorithm does not solve for you.

**Finding the leader.** The client contacts any machine; a follower replies “not me, try C”. Clients cache that and retry on failure. Getting this wrong is why real failover takes seconds when the election takes milliseconds.

**Duplicate commands.** The leader commits an entry, then crashes before answering. The client retries. Without protection, the command applies twice — and “add 50” applied twice is a real bug. The fix is a unique client-supplied ID per command, with the state machine keeping the last-seen ID per client and ignoring repeats. This makes commands idempotent. **Consensus gives you exactly-once ordering, not exactly-once execution** — you have to add that yourself.

**Stale reads.** A follower can serve a read from an out-of-date log. So can a *leader* that has been partitioned away and does not yet know it. Three options: route reads to the leader and have it confirm its leadership with a heartbeat round first (correct, costs a round trip); use a leader lease (correct if clocks are bounded, cheap); or read from any follower and accept staleness (fast, and fine for a lot of things).

#### The snapshot problem

The log grows forever, and you cannot keep it forever. So periodically each machine writes a **snapshot** of its state machine and discards the log entries the snapshot covers. A follower so far behind that the leader has already discarded the entries it needs is sent the whole snapshot instead. Every real implementation has this; it is often more code than the consensus itself.

## 6 The numbers

### Round trips, which is the cost that matters

One write, five-machine cluster, all in one data centre:

|                                |         |                          |
|--------------------------------|---------|--------------------------|
| client → leader                | 0.5 ms  |                          |
| leader appends locally + fsync | 1-5 ms  | ← often the largest term |
| leader → followers (parallel)  | 0.5 ms  |                          |
| follower appends + fsync       | 1-5 ms  |                          |
| follower → leader              | 0.5 ms  |                          |
| leader → client                | 0.5 ms  |                          |
| -----                          |         |                          |
| total                          | 4-12 ms |                          |

Note what dominates: the disk sync, not the network.  
Entries must be durable before they are acknowledged, or a crash loses a committed entry.

The disk sync is the honest answer to “why is consensus slow”. Most people say “the network round trips”. In one data centre, two fsync calls cost more than the whole network path.

Across regions it flips completely:

Three machines, Mumbai / Singapore / Frankfurt:

Mumbai → Singapore      ~35 ms  
Mumbai → Frankfurt      ~110 ms  
majority = leader + the nearer of the two = Singapore

write latency ≈ 2 × 35 = 70 ms + disk

The majority is reached by the *second-fastest* member, so with three machines your latency is the round trip to your nearest peer. Placing all three in one region gives millisecond writes and no regional survival. This is a real design decision, not a detail.

Throughput

etcd, three nodes, SSD, small values:      ~10,000-40,000 writes/sec  
etcd reads (from the leader):      ~100,000+/sec  
etcd reads (serialisable, any node):      much higher, may be stale

Batching is what makes this work: one round trip can carry hundreds of entries. Per-entry cost falls; per-batch latency does not.

Cluster size

| NODES | MAJORITY | FAILURES TOLERATED | RELATIVE WRITE LATENCY    |
|-------|----------|--------------------|---------------------------|
| 3     | 2        | 1                  | fastest                   |
| 5     | 3        | 2                  | slower (3rd-fastest peer) |
| 7     | 4        | 3                  | slower again              |
| 9     | 5        | 4                  | slowest                   |

Three for most things, five when the data is critical. Beyond five, latency climbs and the failure tolerance you gain is rarely the thing that actually kills you. Note the tail-latency effect: with five machines you wait for the *third* fastest, and the third-fastest of five is meaningfully slower than the second of three.

Failover, end to end

|                                          |                                |
|------------------------------------------|--------------------------------|
| leader crashes                           | t = 0                          |
| follower election timeout fires          | t = 150-300 ms (randomised)    |
| votes collected                          | t = +2 ms                      |
| new leader sends first heartbeat         | t = +2 ms                      |
| new leader commits a no-op from its term | t = +5 ms                      |
| clients discover the new leader          | t = +100 ms to several seconds |
| -----                                    |                                |
| algorithm:                               | ~200-300 ms                    |
| observed:                                | 1-10 seconds                   |

The gap between those last two numbers is where real incidents live. The election is fast; client rediscovery is not. Say this in an interview.

Data sizes — the thing consensus is *for*

|                            |                              |                                       |
|----------------------------|------------------------------|---------------------------------------|
| etcd default storage quota | 2 GB                         | (8 GB maximum, and that is a warning) |
| typical Kubernetes cluster | ~100 MB-1 GB of etcd data    |                                       |
| ZooKeeper practical limit  | a few GB, must fit in memory |                                       |

**Consensus systems hold configuration, not data.** A few gigabytes, total. If a question implies putting user data through Raft directly, that is the point to push back — unless it is one shard of a system like CockroachDB, which runs *thousands* of separate Raft groups, one per data range, precisely so that no single group is large or busy.

7 The trade-offs

Consensus versus a quorum system

You saw quorums on day 117 and they look similar. They are not the same thing.

|                    | QUORUM (DYNAMO-STYLE)             | CONSENSUS (RAFT)                 |
|--------------------|-----------------------------------|----------------------------------|
| Leader             | none                              | one, per group                   |
| Ordering           | none — conflicts happen           | total order, guaranteed          |
| Conflicts          | possible; you resolve them        | impossible                       |
| Write availability | any <code>W</code> replicas       | needs a majority                 |
| Latency            | fast, tunable                     | one extra round trip             |
| Guarantee          | eventual consistency              | linearizability                  |
| Use for            | shopping carts, sessions, metrics | config, locks, leadership, money |

The trade in one sentence: **consensus buys you a guaranteed order, and you pay for it with a round trip and with unavailability during a partition.**

#### The CAP position, honestly

Raft is a **CP** system. During a network partition, the minority side stops serving writes entirely. It does not serve stale data, and it does not accept writes it cannot order — it refuses. Three machines with one partitioned off: the two-machine side keeps working, the single machine returns errors until the partition heals.

That is the correct behaviour for a lock service and the wrong behaviour for a shopping cart. Match the tool to what a wrong answer costs.

#### What consensus does not give you

Be precise here, because candidates routinely over-claim.

- **Not exactly-once execution.** Ordering, yes. De-duplication, no — add client IDs.
- **Not fast reads by default.** A strictly correct read costs a round trip, unless you use a lease.
- **Not protection from bad data.** Raft replicates a wrong command perfectly to every machine.
- **Not tolerance of lying machines.** Raft assumes crash faults, not malicious ones. A compromised machine can corrupt the cluster. Byzantine fault tolerance is a different, much more expensive family.
- **Not scale.** One Raft group is one leader, one write path. Scaling means many groups.

#### Raft versus Paxos versus ZAB

“Multi-Paxos, Raft and ZAB all solve the same problem and all have the same fundamental cost — a majority and a round trip. Raft’s contribution is not performance, it is that it is described in a way people can implement correctly. It insists on a strong leader, a log with no gaps, and it separates the three parts cleanly. Paxos allows more concurrency in principle, but the number of subtly broken hand-rolled Paxos implementations is the practical argument for Raft. ZAB is ZooKeeper’s, very close to Raft in spirit, and predates it.”

That is the right length for this question. Do not go further unless pushed.

### When not to use consensus

The most valuable judgement in this lesson.

- **When you do not need ordering.** Metrics, logs, click events, caches. Consensus is pure cost.
- **When the data is large.** Gigabytes, not terabytes. Consensus coordinates; it does not store.
- **When writes are frequent and independent.** One leader, one write path. Shard instead.
- **When you can use a database that already does this.** Any managed relational database with synchronous replication has consensus inside it. Do not rebuild it.
- **When one machine would do.** A single machine with good backups is linearizable, cheap, simple, and has no split-brain. If the availability requirement genuinely permits it, take it.

The strongest thing you can say in a round is: *“I would not implement Raft. I would run etcd, or use a database that has consensus inside it, because the failure modes here are subtle and the value of a hand-rolled implementation is negative.”*

## 8 In the interview

### How it gets asked

- *“How do these five machines agree on anything?”*
- *“Explain Raft.”* — and the tell: how quickly you get to the log rather than the election.
- *“What is the difference between a quorum and consensus?”*
- *“How does etcd work? How does Kubernetes store its state?”*
- *“How would you build a distributed lock?”* — leads to consensus, leases and fencing.
- *“Why is your database’s failover ten seconds when the election is 200 milliseconds?”*
- *“When would you not use consensus?”* — the strongest signal of the lot.

## The first ninety seconds

“Consensus is getting a group of machines to agree on a value and stay agreed, despite crashes, delays and lost messages. In practice you never need one value — you need an ordered sequence of them, so every real system is built on a replicated log. That is the key idea: if every machine starts identical and applies the same commands in the same order, they stay identical. Hard problem becomes easier problem — agree on a list’s order.

Raft has three parts. **Election**: one leader per term, chosen by majority vote, terms increasing. **Replication**: the leader appends every command to its log and pushes it out; once a majority have it stored, it is committed and can be applied and acknowledged. **Safety**: two restrictions that stop a new leader losing committed data — you only vote for a candidate whose log is at least as up to date as yours, and a leader only counts entries from its own term when advancing the commit point.

The cost is a majority and a round trip per write — realistically four to twelve milliseconds in one data centre, dominated by the disk sync rather than the network. And it is CP: during a partition the minority side refuses writes rather than diverging.

Which part would you like me to go deeper on?”

That covers everything, in order of importance, and ends with a question.

## The follow-ups

“Why does the log matter more than the election?”

“Because the election is a means, not the end. The point of the whole system is a replicated state machine, and that only works if every machine applies the same commands in the same order. The leader exists to *assign* that order — that is its actual job. You could imagine other ways to pick a leader, but you cannot do without the ordered log. It is also the practical tell: candidates who have only read the summary talk about voting and stop there.”

“What stops a new leader from losing committed data?”



“Two rules working together. First, an entry is committed only when a majority holds it. Second, a candidate only wins with votes from a majority, and each voter refuses a candidate whose log is behind its own. Any two majorities of five overlap in at least one machine, so the electing majority always contains someone with every committed entry — and that machine will not vote for a candidate missing it.

There is a second, subtler rule: a leader does not commit an entry from a previous term just because it now sits on a majority. It waits until it has committed an entry from its own term. There is a specific five-step failure sequence in the Raft paper where skipping that loses acknowledged data. It is the rule most hand-rolled implementations miss.”

**“Why is my failover ten seconds if the election is 200 milliseconds?”**

“Because the election is not the slow part. Clients still have to *find* the new leader. If discovery is through DNS with a sixty-second TTL, that is your failover time. If it is through a load balancer with a ten-second health check, that is your failover time. If clients cache the old leader’s address and only retry after a thirty-second timeout, that is your failover time.

Then there are the tails: the new leader may need to catch a follower up, or read a snapshot; connection pools reconnect; a cold cache means the first thousand queries are slow. Fixing consensus tuning here would be optimising the wrong number entirely.”

**“Quorum or consensus for this?”**

“What does a wrong answer cost? If two people can hold the same lock, or money can be double-spent, or two machines can both believe they are the leader — consensus, and pay the round trip. If the worst case is a deleted item reappearing in a shopping cart, or a slightly stale view count — quorum, and take the availability and the speed. The real question is not which is technically stronger. It is what inconsistency costs in this specific place.”

**The model answer**

*“Design a distributed lock service. A few thousand clients need to coordinate access to shared resources.”*

“Let me be clear about what a lock actually has to guarantee, because it changes everything: at most one holder at a time, and that must survive machine failures. So a wrong answer here has a real cost, which puts me on the consensus side rather than the quorum side.

**What I would build.** Not a lock service. I would run etcd — three or five nodes — and put a thin client library in front of it. A lock is a key with a lease: acquire is a compare-and-set that succeeds only if the key does not exist, and it attaches a lease of, say, ten seconds. The holder renews the lease every three seconds in the background. If the holder dies, renewals stop, the lease expires, etcd deletes the key, and the next waiter’s watch fires. No lock is held forever by a dead machine, and I did not write a consensus algorithm.

**The hard part, which is not the locking.** A lock service cannot actually prevent two clients from acting at once. It can only prevent two from *holding the key*. Consider: client A acquires, then its process pauses for twelve seconds — a garbage collection pause, or the machine is descheduled. The lease expires. B acquires legitimately. A wakes up still believing it holds the lock and writes.

No amount of consensus fixes this, because A’s belief is local and stale, and A has no way to know time passed. The fix is **fencing**. Every acquisition returns a monotonically increasing number — etcd’s revision number is one, for free. Every write to the protected resource carries that number. The *resource* rejects any write carrying a number lower than the highest it has seen. So when A wakes up and writes with token 33, the storage has already seen 34 from B and refuses.

The important detail: the check happens **at the resource, not at the client**. You cannot trust a client to know it has been deposed — that is exactly what it does not know.

**Numbers.** Three etcd nodes in one region: acquire is a Raft write, four to twelve milliseconds, dominated by the disk sync. Renewals are cheap. Thousands of clients each renewing every three seconds: a few thousand operations a second, well inside etcd’s range. Watches are cheap — clients wait rather than poll. Storage: a lock key is tiny, and even a hundred thousand of them is a few megabytes against etcd’s two-gigabyte quota.

**What I would tell them honestly.** During a partition, the minority side cannot acquire or renew, which means it loses its locks and stops working. That is correct and it is the point — a lock service that stays available during a partition is not a

lock service. Second, lease length is a real trade: ten seconds means a crashed holder blocks others for up to ten seconds, but a shorter lease means an unlucky pause loses the lock. I would start at ten and tune it with real data.

**What I would ask before building.** How long is a lock held? If it is milliseconds, the lease machinery is overkill and I would look at whether the operation can be made idempotent and retried instead — that removes the need for a lock at all, and is very often the better answer. If it is minutes, the lease and fencing design is right. And what is being protected? If the resource cannot enforce a fencing token, this whole design is advisory only, and I would want to say that out loud rather than let anyone believe it is safe.”

That answer refuses to build the wrong thing, names the failure that the obvious design misses, puts the enforcement in the right place, has real numbers, and ends by questioning whether a lock is needed at all.

## 9 Recall card

**Consensus:** a group agrees on a value and stays agreed, despite crashes, delays and lost messages. Agreement, validity, termination.

**The real problem is a sequence, not a value.** Hence the **replicated log**: same start state + same commands + same order = same end state. That is **state machine replication**, and it is the whole idea.

**Raft has three parts:** 1. **Election** — one leader per term, majority vote, one vote per term. 2. **Replication** — leader appends, pushes out; a majority storing it makes it **committed**. 3. **Safety** — restrictions so a new leader cannot lose committed data.

**The two safety rules:** - **Election restriction** — vote only for a candidate whose log is at least as up to date as yours (compare by term first, then index). “Whose book is longest.” - **Current-term rule** — a leader only advances the commit point on an entry from its *own* term.

**Log matching:** same index + same term  $\Rightarrow$  identical logs up to there. One check verifies all history.

**Commit = a majority has it stored.** Permanent, because any future majority overlaps this one.

**A leader seeing a higher term steps down immediately.** No argument.

**Numbers:** one write = 4-12 ms in one region, **dominated by the disk sync, not the network**; ~10-40k writes/sec for etcd; heartbeat 50 ms; election timeout 150-300 ms randomised; algorithmic failover ~200-300 ms but **observed failover 1-10 s because clients must rediscover the leader**; cross-region latency = round trip to your second-nearest peer; etcd holds ~2 GB, config not data.

**Cluster sizes are odd.** 3 tolerates 1, 5 tolerates 2. Beyond 5, latency grows for little gain.

**Raft is CP.** The minority side refuses writes rather than diverging.

**It does not give you:** exactly-once execution (add client IDs), free fast reads (leases or a round trip), protection from bad commands, tolerance of malicious machines, or scale (shard into many groups).

**Quorum vs consensus:** no order and possible conflicts, versus total order and none. Choose by what a wrong answer costs.

**The best answer to “would you implement Raft?” is no.** Run etcd, or use a database that has it inside.

---

**Next:** [Day 120 — Distributed transactions and two-phase commit](#)

**Previous:** [Day 118 — Leader election](#)

**DSA topic:** Heaps revision and mock round **System design topic:** Consensus, and Raft in plain English

Code these, in this order

One rule for the whole set: **before writing a line, say which of the five shapes it is and how big the heap will be.** Out loud. If you cannot name the shape, you have not recognised the problem yet, and starting to type is the worst thing you can do.

| # | PROBLEM                         | SOURCE                | WHAT IT IS REALLY TESTING                       |
|---|---------------------------------|-----------------------|-------------------------------------------------|
| 1 | Kth Largest Element in an Array | LeetCode 215 (Medium) | Shape 2, and whether you reach for quickselect. |
| 2 | Task Scheduler                  | LeetCode 621 (Medium) | Shape 1 plus a cooldown queue. Today's mock 1.  |
| 3 | Find K Pairs with Smallest Sums | LeetCode 373 (Medium) | Shape 4 in disguise. Today's mock 2.            |
| 4 | Network Delay Time              | LeetCode 743 (Medium) | Shape 5 — Dijkstra with lazy deletion.          |
| 5 | Reorganize String               | LeetCode 767 (Medium) | Shape 1, and knowing when it is impossible.     |

Say the shape first, every time

Write the shape number and the heap size as a comment on line one, before any code. Five problems, five comments. Then check them against the lesson.

On problem 1, write three solutions

Sort, heap, quickselect. Time all three at  $n = 1,000,000$  with  $k = 10$  and again with  $k = 500,000$ . Six numbers. Say where the crossover is and why the ordering changes.

On problem 3, do the reframing out loud

Before coding, say the sentence that turns it into a merge. If you cannot produce that sentence, the code will not come either.

#### On problem 4, delete the stale check

Remove the `if distance > best[place]: continue` line. Record whether the answer changes and what happens to the running time. Say why the answer is what it is.

#### On problem 5, find the impossible case

Construct an input that cannot be reorganised. State the condition in one line of arithmetic.

---

#### The recognition drill

1. State the recognition question in one sentence.
2. Give the three phrasings problems use instead of saying “heap”.
3. Give four situations where a heap is the wrong tool.
4. For each of the five problems above, name the shape without looking.

#### The five shapes drill

1. Name all five from memory.
2. For each: what the heap holds, its size, the cost, one example.
3. Say which three keep the heap small, and why that is the important column.
4. Give a problem for each shape that is not in the lesson.

#### The k-trap drill

1. Say which kind of heap you keep for the k largest, and why.
2. Say it again for the k smallest.
3. Give the one-sentence reason that makes it stop feeling backwards.
4. Write both loops from memory.

#### The heapify drill

1. Give the cost of `heapify` and of n pushes.
2. Draw the level-by-level work table for  $n = 1,000,000$ .
3. Say why half the nodes contribute zero.
4. Give the ratio between the two approaches at a million elements.

#### The cost drill

1. Recite the cost table: push, pop, peek, heapify, find, delete.
2. Say why find is  $O(n)$  and not  $O(\log n)$ .
3. Compute comparisons for  $n = 1,000,000$  at  $k = 10, 1,000$  and  $500,000$ .

4. Say which argument you lead with in an interview, and why it is not time.

#### The heapq drill

1. List the six gotchas from memory.
2. Say what `heapq.heapify` returns.
3. Say how many negations a max-heap needs, and where.
4. Say when `len(heap)` is not the size.
5. Write the max-heap-of-tuples push and read, with both negations.

#### The alternatives drill

1. Compare heap and quickselect on four axes.
2. Say when quickselect wins and when it cannot be used at all.
3. Say what to do when  $k$  is larger than  $n/2$ .
4. Say what you would use if items' priorities change, and give both options.

#### The break-it drill

Trigger each and record the exact output or error:

1. `heap = heapq.heapify([3,1,2])` then `heap[0]`.
2. A max-heap read without negating back.
3. `(1, some_object)` pushed twice with equal first fields.
4. `heapq.heappop([])`.
5. Reading `heap[1]` expecting the second-smallest.
6. A max-heap of size  $k$  used for the  $k$  largest.
7. Dijkstra with the stale check removed, on a dense graph.

#### The mock drill

Redo both mock problems from the lesson, timed, twenty minutes each, talking the whole time. Then:

1. For mock 1, say what the negation does to `+1` and why `if remaining:` is correct.
  2. For mock 1, state the closed-form formula and say why you would still write the simulation.
  3. For mock 2, say why the heap is seeded with `min(len(a), k)` entries and not `len(a)`.
  4. For mock 2, say the reframing sentence in under ten words.
-

### The consensus drill

1. Define consensus and give its three properties.
2. Say why single-value consensus is not what real systems need.
3. Explain state machine replication in two sentences.
4. Give the example that shows why order matters, with numbers.
5. Name five different questions that are all the same consensus problem.

### The three-parts drill

1. Name Raft's three parts in order.
2. Say what each one guarantees and what breaks without it.
3. Say which two make it work and which one makes it correct.
4. Say which part most explanations skip.

### The commit drill

1. Define committed.
2. Draw the five-machine picture with three acknowledgements.
3. Give the overlap argument for why commitment is permanent.
4. Say what the leader does before answering the client.
5. Say why two slow followers are not an error condition.

### The safety drill

1. State the election restriction and the everyday phrasing for it.
2. Say how the up-to-date comparison is ordered, and why term comes first.
3. State the current-term commit restriction.
4. Say what goes wrong without it, in one sentence.
5. Say which of the two is more commonly omitted in hand-rolled implementations.

### The log-matching drill

1. State the log-matching property.
2. Walk through the repair example step by step.
3. Say what happens to the follower's diverged entries and why that is safe.
4. Say why naive backtracking is slow and what real implementations do instead.
5. Say why `(index, term)` and not index alone identifies an entry.

### The state-machine drill

1. Name the three states and the four transitions.



2. Say what triggers each transition.
3. Say what a leader does on seeing a higher term, and why it does not argue.
4. Say why that single rule stops a returning old leader from fighting.

#### **The client drill**

1. Name the three problems Raft does not solve for the client.
2. Say how a client finds the leader.
3. Say why a committed command can be applied twice and how to prevent it.
4. Give the three options for reads, with the cost of each.
5. Say why a partitioned leader can serve a stale read.

#### **The numbers drill**

1. Break a single write into its six steps with timings.
2. Say what dominates in one data centre, and what dominates across regions.
3. Compute cross-region write latency for a three-region cluster.
4. Give etcd's write and read throughput ranges.
5. Give the algorithmic failover time and the observed one, and explain the gap.
6. Give etcd's storage quota and say what that implies about what belongs in it.

#### **The trade-offs drill**

1. Fill in the quorum-versus-consensus table on seven rows.
2. State the trade in one sentence.
3. Say where Raft sits on CAP and what the minority side does.
4. Name five things consensus does not give you.
5. Give five situations where you would not use consensus.
6. Compare Raft, Paxos and ZAB in four sentences and stop.

#### **The failure drill**

For each, say what happens and what you would do:

1. A follower is a thousand entries behind and the leader has already snapshotted past them.
2. The leader commits an entry, then crashes before answering; the client retries.
3. A new leader finds an entry from an old term sitting on a majority.
4. A three-node cluster is partitioned one and two.
5. A client reads from a leader that was partitioned away four seconds ago.

6. Someone proposes putting two terabytes of user data behind one Raft group.

Two of the six are not consensus failures at all. Name them.

---

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the  $k$  most frequent elements in a stream.* The recognition question first, then the shape, then the min-heap trap named as a trap,  $O(n \log k)$  time with  $O(k)$  space and the space led with, and the streaming argument as the closing reason.
  2. *Explain Raft.* Consensus defined, the log as the real answer with state machine replication, the three parts in order, both safety restrictions, the cost with the disk sync named as the dominant term, and CP stated plainly.
  3. *When would you not use consensus?* Five situations, the quorum comparison, and “I would not implement Raft — I would run etcd” with the reason.
- 

### Before you move on

- ☐ I can state the recognition question in one sentence.
- ☐ I can name all five shapes and what each heap holds.
- ☐ I say the shape and the heap size before I write any code.
- ☐ I know that  $k$  largest means a min-heap, and why.
- ☐ I lead with space, not time, when justifying a heap.
- ☐ I can make the streaming argument.
- ☐ I can explain why heapify is  $O(n)$  with the level table.
- ☐ I know all six `heapq` gotchas without looking.
- ☐ I know a heap cannot find or update an arbitrary element, and both work-arounds.
- ☐ I can compare heap with quickselect and say when each wins.
- ☐ I know four situations where a heap is the wrong tool.
- ☐ I solved both mock problems in twenty minutes each, talking out loud.
- ☐ I can define consensus and its three properties.
- ☐ I can explain state machine replication in two sentences.
- ☐ I get to the log quickly, not just the election.

- [ ] I can name Raft's three parts and what each guarantees.
- [ ] I can define committed and give the overlap argument.
- [ ] I can state both safety restrictions and what each prevents.
- [ ] I can state the log-matching property and walk through a repair.
- [ ] I know a leader steps down on a higher term without arguing.
- [ ] I know consensus gives ordering, not exactly-once execution.
- [ ] I know the disk sync dominates a single-region write.
- [ ] I can explain why observed failover is seconds, not milliseconds.
- [ ] I know consensus holds configuration, not data, and the size limit.
- [ ] I can say when I would use a quorum instead.
- [ ] I would run etcd rather than implement Raft, and can say why.
- [ ] I answered all three questions above out loud.

## PART XIV

# Tries

---

Days 120 to 124 · 5 chapters

**Alongside, on the system design track:**

Distributed systems core

[120](#) The trie: a tree of characters

[121](#) Insert, search, and prefix search

[122](#) Autocomplete and word dictionaries

[123](#) Tries in interviews: word search II

[124](#) Tries revision and mock round

# The trie: a tree of characters

## DATA STRUCTURES AND ALGORITHMS

The trie: a tree of characters

## SYSTEM DESIGN

Distributed transactions and two-phase commit

## 1 What this is, and why they ask it

You already have a way to store a set of words. A hash set. `word in words` is  $O(1)$ , it is one line, and for most jobs it is the right answer. So this lesson has to start by justifying itself.

Here is the question a hash set cannot answer:

**How many of these words start with "pre" ?**

A hash set has no idea. Hashing destroys the relationship between `"prefix"` and `"pre"` on purpose — a good hash function scatters similar keys to unrelated places, which is exactly what makes it fast. To answer that question with a hash set you must look at every word you have ever stored.

The **trie** is the structure that answers it. A trie stores a set of words as a tree of characters, where the path from the root to a node spells out a prefix, and every word sharing that prefix hangs below it. To find everything starting with `"pre"` you walk three steps and look at what is underneath. The time depends on the length of `"pre"`, not on how many words you have stored. A million words, ten million, a billion — still three steps.

The name is from *retrieval*, and it is pronounced either “try” or “tree”. Nobody agrees and nobody minds. It is also called a **prefix tree**, which is the more honest name and the one worth using out loud, because it tells you what it is for.

By the end of this lesson you will know what a trie is, why the path is the key, what a node actually holds, how to build one, and — the part that gets skipped — what it costs in memory, which is the reason tries are not used more often than they are.

Tomorrow you will do the operations properly: insert, search, prefix search, delete, and the problems that come from them. Today is the structure itself.

## 2 The story

Bhaskar has looked after the post room at a boys' hostel for nine years. Four hundred boys, and on a good day two hundred letters.

When he started, the post room had one long table. Letters came in a sack at eleven, he tipped them on the table, and boys came and rummaged. It worked when there were eighty boys. At four hundred it was chaos — boys pushing, letters on the floor, and every single boy touching every single letter to find his own.

So Bhaskar put up a rack of twenty-six wooden boxes and painted a letter on each. A letter for Suresh goes in the S box. A boy looking for post opens one box and looks through what is in it, and that is all.

For a year this was wonderful. Then it stopped being wonderful, and the reason was the S box.

There were sixty-one boys with names starting with S. On a heavy day the S box held forty letters and a boy would stand there going through all forty. Meanwhile the X box had been empty since 2019 and the Q box had been used exactly once.

Bhaskar's fix was the thing that made the whole system work. He did not add more big boxes. He took the S box off the rack, and inside it he put a row of small dividers, each with a second letter painted on it: Sa, Sh, Su, Sw. Sixty-one boys, split across four dividers.

He did not do this to the X box. The X box has one boy in it. Splitting it would be silly.

Over the years the busy parts kept splitting. The Sh divider grew until he put three sub-dividers inside it — Sha, Shi, Shu — and Sha eventually got its own three inside that. The Q box is still one box with nothing in it. The rack is deep in some places and shallow in others, and it grew that way on its own, according to where the letters actually were.

A new boy asked him once how he finds anything.

"I do not find anything," said Bhaskar. "I spell. Watch — Shantanu." He touched the S box, then the Sh divider inside it, then Sha inside that. Three touches. "There. Everything for every boy whose name starts Sha is in my hand. Eleven boys. That is a small enough pile to look through."

The boy asked whether it took longer now there were four hundred boys than when there were eighty.

Bhaskar had clearly been asked this before. "It takes as long as the name is," he said. "S-h-a. Three touches. If eight hundred boys move in tomorrow, it is still three touches. The rack gets deeper where they crowd, but the walk is the same walk."

---

### 3 The idea in plain English

Bhaskar built a trie, one splitting box at a time, and everything about the structure is in what he said.

**The path is the key.** This is the sentence to remember. In a hash set, a node holds the word. In a trie, **no node holds a word at all** — the word is spelled out by the sequence of edges you followed to get there. The letter is on the edge (or, equivalently, in the key of the child dictionary), not in the node. The node at the end of the path `c → a → t` does not contain the string `"cat"`. It contains a flag saying “a word ends here”, and nothing else.

Beginners nearly always draw a trie with the letters inside the circles, which is fine as a picture and misleading as a model. What is really stored at each node is *a map from the next character to the next node*.

**Shared prefixes are stored once.** `"car"`, `"card"`, `"care"`, `"careful"` share `c-a-r`. In a hash set that is four separate strings, and the letters `c`, `a`, `r` are stored four times. In a trie there is one `c` node, one `a` below it, one `r` below that, and the words diverge only where they actually differ. This is why tries are natural for dictionaries, where words genuinely do share beginnings.

**Two flags decide everything, and they are different questions.** A node needs to answer:

- *Does a word end here?* — a boolean, usually called `is_end` or `is_word`.
- *Do any words continue past here?* — implied by whether it has children.

These are independent, and confusing them is the single most common trie bug. With `"car"` and `"card"` stored, the node at `c-a-r` has `is_end = True` **and** has a child `d`. With only `"card"` stored, that same node has `is_end = False` and still has the child. A node with `is_end = False` is not empty — it is a place words pass through.

**The depth is the word length, not the word count.** Bhaskar’s line. A trie over a million English words is at most about thirty levels deep, because the longest English word is about thirty letters. Adding another million words does not make it deeper. It makes it *wider* in the middle, which costs memory but not time. Every operation on a word of length `L` takes `O(L)` steps, full stop — no dependence on `n` at all.

That is a genuinely unusual property. Almost every structure you have met so far has `n` in its complexity somewhere. A trie does not.

**What it buys you over a hash set.** Exactly three things:



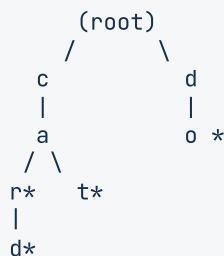
1. **Prefix queries.** “Does anything start with `pre` ?” and “give me everything starting with `pre`” — the whole reason the structure exists.
2. **Sorted order for free.** Walk the children in alphabetical order and you emit the words sorted, with no sorting step. A hash set gives you no order at all.
3. **No hashing and no collisions.** Lookup is  $O(L)$  guaranteed, not  $O(L)$  average. A hash set must read the whole key to hash it — which is also  $O(L)$  — and then may collide. In the worst case a trie is strictly better; in the average case they are close.

**What it costs.** Memory, and a lot of it. This is the honest part that most explanations bury, and it is covered properly in §6. A trie stores one node per distinct prefix, and each node carries a whole dictionary. For a set of unrelated random strings — which share almost no prefixes — a trie can use ten to fifty times the memory of the hash set holding the same words, and be slower besides. **A trie is only worth it when the keys genuinely share prefixes and you genuinely need prefix queries.**

#### 4 The picture

##### A trie holding four words

Words stored: "car", "card", "cat", "do"



\* = is\_end is True (a word finishes here)

Read the paths:

|                      |                                          |
|----------------------|------------------------------------------|
| root → c → a → r     | spells "car", is_end → yes, it is a word |
| root → c → a → r → d | spells "card", is_end → yes              |
| root → c → a → t     | spells "cat", is_end → yes               |
| root → d → o         | spells "do", is_end → yes                |

Notice what is NOT a word:

|              |                                              |
|--------------|----------------------------------------------|
| root → c     | spells "c" — a node exists, is_end is False  |
| root → c → a | spells "ca" — a node exists, is_end is False |

Nine nodes hold four words totalling twelve characters.  
The prefix "ca" is stored ONCE and serves three words.

Notice the two nodes with no star. `c` and `ca` exist and are not words. Existence and word-ness are separate facts, and every trie bug lives in the gap between them.

What a node actually holds

|                                                                                                                                                       |                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>The picture people draw</p> <p>-----</p> <pre>      ( a )      /   \     ( r )  ( t )</pre> <p>The letters look like they live in the circles.</p> | <p>What is actually stored</p> <p>-----</p> <pre>node at path "ca":   children = { 'r': &lt;node&gt;,                't': &lt;node&gt; }   is_end   = False</pre> <p>The letter is a KEY in the parent's dictionary. The node itself does not know which letter it is, and does not know its own prefix.</p> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Notice that a node cannot tell you what word it represents. It has no idea. Only the walk knows, which is why any operation that needs to report words must carry the prefix down with it.

The prefix query, which is the entire point

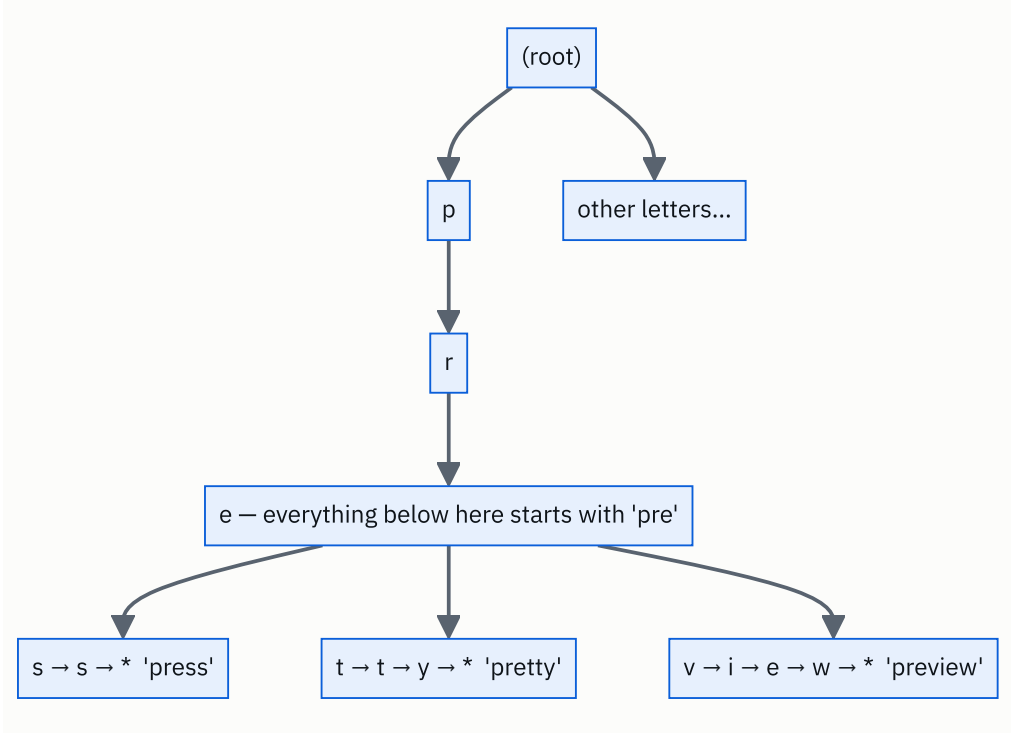


FIGURE 120.1

Notice that answering “what starts with `pre` ?” costs three steps to reach the node, and then a walk of only the subtree below it. The rest of the trie is never touched, however large it is.

### Where the memory goes

Storing “car”, “card”, “cat”, “do” – the good case:

```
characters in the words: 3 + 4 + 3 + 2 = 12
nodes actually created: 9
ratio: 0.75 nodes per character (prefixes shared)
```

Storing 4 random 10-character IDs – the bad case:

```
"x7k2m9p4qa"
"b3n8v1z5wc"
"h6j0t2r7ye"
"d9f4l3s8ug"
```

```
characters: 40
nodes created: 41 (nothing shares anything past the root)
ratio: ~1.0 nodes per character
```

And a node is not one byte. In Python a node with a dict is ~200+ bytes. So 41 nodes ≈ 8 KB to store 40 characters that a set would hold in about 300 bytes.

Notice the second case. When keys do not share prefixes, a trie is a very expensive way to store a string one character per object. This is the case that decides whether you should use one.

## 5 The code, built step by step

### The node

Everything starts here, and it is four lines.

```
class TrieNode:
 def __init__(self) → None:
 self.children: dict[str, "TrieNode"] = {}
 self.is_end: bool = False
```

That is the whole data structure. No letter, no word, no parent pointer. A map to children and one flag. Write this from memory; it is the part interviewers watch you write.

## The trie, and its root

```
class Trie:
 def __init__(self) → None:
 self.root = TrieNode()
```

The root is an ordinary node representing the empty prefix. It is never a word unless you deliberately insert the empty string. Every walk starts here.

## Insert

Walk down, creating nodes for characters that are missing, and mark the last one.

```
def insert(self, word: str) → None:
 node = self.root
 for character in word:
 if character not in node.children:
 node.children[character] = TrieNode()
 node = node.children[character]
 node.is_end = True
```

Five lines. The loop body is “make the child if it is not there, then step into it”. The single line after the loop is what makes this a word rather than just a path.

`setdefault` collapses the two lines into one, and is worth knowing:

```
for character in word:
 node = node.children.setdefault(character, TrieNode())
```

Use whichever you can write without hesitating. Clarity beats brevity in a round.

## The walk, factored out

Search and prefix search do the same walk and differ only in what they check at the end. Write the walk once.

```
def _walk(self, prefix: str) → TrieNode | None:
 """Return the node at the end of prefix, or None if the path breaks."""
 node = self.root
 for character in prefix:
 if character not in node.children:
 return None
 node = node.children[character]
 return node
```

### Search and starts\_with — one line each

```
def search(self, word: str) → bool:
 node = self._walk(word)
 return node is not None and node.is_end

def starts_with(self, prefix: str) → bool:
 return self._walk(prefix) is not None
```

**This pair is the lesson.** They walk identically. `search` demands `is_end`; `starts_with` only demands that the path exists. If you can explain why those two lines differ by exactly that one condition, you understand tries.

### Collecting every word under a prefix

The operation a hash set cannot do at all. Note that the prefix must be carried down, because nodes do not know their own path.

```
def words_with_prefix(self, prefix: str) → list[str]:
 start = self._walk(prefix)
 if start is None:
 return []
 found: list[str] = []
 self._collect(start, prefix, found)
 return found

def _collect(self, node: TrieNode, so_far: str, found: list[str]) → None:
 if node.is_end:
 found.append(so_far)
 for character in sorted(node.children):
 self._collect(node.children[character], so_far + character, found)
```

Two details worth saying out loud. `sorted(node.children)` makes the output alphabetical for free — that is the second advantage over a hash set. And `if node.is_end` is checked *before* recursing, not instead of it, because `"car"` being a word does not stop `"card"` from being one too.

### Counting words under a prefix, without listing them

If you only need the count, storing it on the way in is far better than counting on the way out.

```
class CountingTrieNode:
 def __init__(self) → None:
 self.children: dict[str, "CountingTrieNode"] = {}
 self.is_end: bool = False
 self.words_below: int = 0 # words in this subtree
```

```

def insert(self, word: str) → None:
 node = self.root
 node.words_below += 1
 for character in word:
 node = node.children.setdefault(character, CountingTrieNode())
 node.words_below += 1
 node.is_end = True

```

Now `count_with_prefix` is `O(L)` instead of `O(size of the subtree)`. This is the single most useful augmentation, and it is three extra lines.

### The array-of-26 variant

When the alphabet is fixed and small, an array is faster than a dictionary — no hashing, better memory locality — at the cost of always allocating 26 slots even for a node with one child.

```

class ArrayTrieNode:
 __slots__ = ("children", "is_end")

 def __init__(self) → None:
 self.children: list["ArrayTrieNode | None"] = [None] * 26
 self.is_end = False

 def index(self, character: str) → int:
 return ord(character) - ord("a")

```

In Python the dictionary version is usually better, because a Python list of 26 `None`s is not the compact thing it is in C. In C++ or Java, `TrieNode*[26]` is the standard choice. Know both and say which you would pick and why.

## The complete solution

```
"""Day 120 - a trie, complete and runnable."""

from __future__ import annotations

class TrieNode:
 """One node. Holds a map to its children and whether a word ends here."""

 __slots__ = ("children", "is_end", "words_below")

 def __init__(self) → None:
 self.children: dict[str, TrieNode] = {}
 self.is_end: bool = False
 self.words_below: int = 0

class Trie:
 """A prefix tree over lowercase words."""

 def __init__(self) → None:
 self.root = TrieNode()
 self.size = 0

 # ----- building -----

 def insert(self, word: str) → None:
 """Add a word. O(L) time. Inserting a duplicate is a no-op."""
 node = self.root
 path = [node]
 for character in word:
 node = node.children.setdefault(character, TrieNode())
 path.append(node)
 if node.is_end:
 return # already present
 node.is_end = True
 self.size += 1
 for visited in path:
 visited.words_below += 1

 # ----- the shared walk -----

 def _walk(self, prefix: str) → TrieNode | None:
 node = self.root
 for character in prefix:
 child = node.children.get(character)
 if child is None:
 return None
 node = child
 return node

 # ----- asking -----

 def search(self, word: str) → bool:
 """Is this exact word stored? O(L)."""
 node = self._walk(word)
 return node is not None and node.is_end

 def starts_with(self, prefix: str) → bool:
 """Does any stored word begin with this prefix? O(L)."""
 return self._walk(prefix) is not None

 def count_with_prefix(self, prefix: str) → int:
 """How many stored words begin with this prefix? O(L)."""
 node = self._walk(prefix)
 return node.words_below if node else 0
```

```

def words_with_prefix(self, prefix: str) → list[str]:
 """Every stored word beginning with this prefix, in alphabetical order."""
 start = self._walk(prefix)
 if start is None:
 return []
 found: list[str] = []
 self._collect(start, prefix, found)
 return found

def _collect(self, node: TrieNode, so_far: str, found: list[str]) → None:
 if node.is_end:
 found.append(so_far)
 for character in sorted(node.children):
 self._collect(node.children[character], so_far + character, found)

----- inspecting -----

def node_count(self) → int:
 """How many nodes exist. Useful for seeing the memory cost."""
 total = 0
 stack = [self.root]
 while stack:
 node = stack.pop()
 total += 1
 stack.extend(node.children.values())
 return total

def __len__(self) → int:
 return self.size

def __contains__(self, word: str) → bool:
 return self.search(word)

if __name__ == "__main__":
 trie = Trie()
 for word in ["car", "card", "care", "careful", "cat", "dog", "do"]:
 trie.insert(word)

 print(len(trie)) # 7
 print(trie.search("car")) # True
 print(trie.search("ca")) # False ← a path, not a word
 print(trie.starts_with("ca")) # True ← the difference
 print(trie.search("cars")) # False
 print(trie.count_with_prefix("car")) # 4 (car, card, care, careful)
 print(trie.count_with_prefix("do")) # 2 (do, dog)
 print(trie.words_with_prefix("car")) # ['car', 'card', 'care', 'careful']
 print(trie.words_with_prefix("z")) # []
 print("careful" in trie) # True

 # Where the memory goes.
 characters = sum(len(w) for w in ["car", "card", "care", "careful", "cat", "dog", "do"])
 print(characters, trie.node_count()) # 25 characters, 16 nodes

 # The same trie built from unrelated keys: nothing is shared.
 scattered = Trie()
 for word in ["qxyz", "mkbt", "rphn", "wfjd"]:
 scattered.insert(word)
 print(16, scattered.node_count()) # 16 characters, 17 nodes

```

Run it. The last two prints are the ones to sit with: sixteen nodes for twenty-five characters of real words, seventeen nodes for sixteen characters of random ones. That difference is the whole argument for when to use this structure.



## 6 What it costs

### Time

| OPERATION                        | COST                                      | NOTE                                    |
|----------------------------------|-------------------------------------------|-----------------------------------------|
| <code>insert(word)</code>        | $O(L)$                                    | $L$ = word length                       |
| <code>search(word)</code>        | $O(L)$                                    |                                         |
| <code>starts_with(prefix)</code> | $O(L)$                                    |                                         |
| <code>count_with_prefix</code>   | $O(L)$                                    | with the <code>words_below</code> count |
| <code>words_with_prefix</code>   | $O(L + \text{total characters returned})$ | you must build each string              |
| building from $n$ words          | $O(\text{total characters})$              |                                         |

**There is no  $n$  anywhere.** That is the headline. Searching a trie of ten words and a trie of ten billion words takes the same number of steps for the same word.

Compare with the alternatives for a prefix query on  $n$  words:

$n = 1,000,000$  words, prefix "pre" of length 3, 400 matches

|              |                                        |                        |
|--------------|----------------------------------------|------------------------|
| hash set:    | scan all 1,000,000 and test startswith | → 1,000,000 operations |
| sorted list: | binary search for the range boundaries | → ~20 + 400 operations |
| trie:        | 3 steps down, then walk 400 results    | → 3 + 400 operations   |

Note the middle row. **A sorted list plus binary search also answers prefix queries**, in  $O(\log n + k)$ , and uses far less memory. It is the right answer for a static word list. The trie wins on updates — adding a word to a sorted list is  $O(n)$  because everything shifts, while the trie is  $O(L)$  — and on more complex queries such as wild-cards.

### Memory, which is the real story

Count nodes, then count bytes per node.

**Nodes.** One node per distinct prefix in the whole set. For real English:

a dictionary of 100,000 English words  
average length 8, so 800,000 characters  
distinct prefixes (measured, roughly): ~250,000 nodes

ratio: about 0.3 nodes per character — heavy sharing near the top, almost none near the bottom.

**Bytes per node.** In Python this is brutal:

```

object header ~56 bytes
empty dict ~64 bytes (more once it grows)
bool ~28 bytes (shared, so ~8 in practice)
int (words_below) ~28 bytes

realistic total ~150-250 bytes per node

```

250,000 nodes × 200 bytes ≈ 50 MB

The same 100,000 words in a Python set:

100,000 strings × (49 + 8) bytes ≈ 5.7 MB, plus set overhead ≈ 10 MB

Trie: ~5× the memory. And that is the FAVOURABLE case, with heavy prefix sharing.

The array-of-26 version in C++ is worse on nodes and better per operation:

```

TrieNode { TrieNode* children[26]; bool is_end; }
= 26 × 8 bytes + 1 byte + padding = 216 bytes per node
250,000 nodes × 216 = 54 MB – and most of those pointers are null.

```

A node with one child wastes  $25 \times 8 = 200$  bytes.

### The random-key disaster

Where a trie should never be used:

1,000,000 UUIDs, 36 characters each

shared prefixes: essentially none past the first 3-4 characters

nodes ≈ 1,000,000 × 33 = 33,000,000

at 200 bytes each = 6.6 GB

The same UUIDs in a set: 1,000,000 × ~85 bytes ≈ 85 MB

Ratio: about 78×. This is not a trade-off. It is a mistake.

**Say this in an interview.** “A trie is only worth its memory when the keys share prefixes. For random identifiers it can be fifty to eighty times worse than a hash set, and slower too.”

### The fixes, briefly

Three, and knowing they exist is enough for today:

- **Compressed trie (radix tree / Patricia trie).** Any chain of single-child nodes is collapsed into one node holding the whole substring. "careful" after "care" becomes one node holding "ful" instead of three nodes. Typically cuts nodes by 60-80%. This is what real routing tables and etcd's key store use.

- **DAWG / DAFSA.** Also merge identical *suffixes*, turning the tree into a graph. A 100,000-word dictionary can drop to a few thousand nodes. Excellent for a fixed word list, painful to update.
- **Ternary search tree.** Three pointers per node instead of 26. Slower per step, dramatically less memory.

The honest comparison table

|                  | HASH SET                  | SORTED LIST                 | TRIE                           |
|------------------|---------------------------|-----------------------------|--------------------------------|
| exact lookup     | <code>O(L)</code> average | <code>O(L log n)</code>     | <code>O(L)</code> worst case   |
| prefix exists    | impossible                | <code>O(L log n)</code>     | <code>O(L)</code>              |
| all with prefix  | <code>O(n)</code> scan    | <code>O(L log n + k)</code> | <code>O(L + k)</code>          |
| insert           | <code>O(L)</code>         | <code>O(n)</code>           | <code>O(L)</code>              |
| sorted output    | no                        | free                        | free                           |
| memory           | smallest                  | small                       | $5\times$ to $80\times$ larger |
| wildcards, fuzzy | no                        | no                          | yes                            |

7 The traps

**Trap 1: confusing “the path exists” with “a word ends here”.** The defining trie bug.

```
>>> trie = Trie()
>>> trie.insert("card")
>>> trie.search("car")
False
>>> trie.starts_with("car")
True
```

No error, no crash — just a `False` where you expected `True`, or the reverse. If `search` returns `True` for prefixes of stored words, you forgot to check `is_end`. If autocomplete returns nothing, you probably checked `is_end` where you should not have.

**Trap 2: forgetting `is_end` entirely on insert.** The trie builds perfectly and every `search` returns `False`. The structure looks right when you print it. Check the last line of `insert` first.

**Trap 3: assuming a leaf means a word and a word means a leaf.** Neither holds.

```
"car" and "card" stored:
 the node at "car" is a word AND has a child → word, not a leaf
"card" stored alone:
 the node at "car" is a leaf's parent, not a word → not a word, has a child
```

There is one true statement: **every leaf must be a word**, because a path that leads nowhere and ends nothing should never have been created. If you find a non-word leaf, your delete is broken.

**Trap 4: expecting a node to know its own prefix.** It does not, and cannot.

```
>>> node = trie._walk("car")
>>> node.word
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
AttributeError: 'TrieNode' object has no attribute 'word'
```

Carry the prefix down as a parameter, or store the word on the end node — but then you are paying for it, and you should say so.

**Trap 5: the character set.** `ord(character) - ord('a')` on anything other than lowercase ASCII:

```
>>> ord('A') - ord('a')
-32
>>> children[-32]
```

No error — Python indexes from the end, so it silently writes into the wrong slot and corrupts the trie. `ord('é') - ord('a')` gives 136 and *does* raise `IndexError`. Normalise the input, or use a dictionary, which handles any character. Ask about the character set in the interview; it is a good question.

**Trap 6: building the result strings and forgetting they cost something.** `words_with_prefix("a")` on an English dictionary returns tens of thousands of strings, and `so_far + character` allocates a new string at every step of every path. If you only need a count, store `words_below`. If you only need the top ten, stop early. Returning everything under `""` returns the entire dictionary.

**Trap 7: using a trie for random keys.** No error. Just several gigabytes, and it will be slower than the set it replaced. See §6.

**Trap 8: recursion depth on long keys.** A recursive collect over keys thousands of characters long:

```
RecursionError: maximum recursion depth exceeded
```

Python's default limit is 1000. For words it never happens. For DNA sequences or file paths it will. Use an explicit stack if key length is unbounded.

**Trap 9: thinking a trie helps with substrings.** A trie finds *prefixes*. It cannot tell you which words contain `"art"` in the middle. That needs a suffix trie or suffix automaton — a different structure, much larger. If the question says “contains”, a plain trie is the wrong answer.

---

## 8 In the interview

### How it gets asked

- *“Implement a trie with insert, search and startsWith.”* — LeetCode 208, and the standard opener.
- *“Design autocomplete for a search box.”* — the system-flavoured version.
- *“Given a board of letters and a dictionary, find every word on the board.”* — word search II; the trie turns an impossible search into a feasible one.
- *“Design a phone contacts search that filters as you type.”*
- *“Find the longest common prefix of a set of strings.”*
- *“Store IP routing rules and find the longest matching prefix.”* — real routers use compressed tries.
- And the one that separates candidates: *“Why not just use a hash set?”*

## The first ninety seconds

“The reason to reach for a trie rather than a hash set is prefix queries. A hash set can tell me whether `'prefix'` is stored; it cannot tell me whether anything *starts with* `'pre'` without scanning everything, because hashing deliberately destroys the relationship between similar keys.

A trie stores the words as a tree of characters. The path from the root spells the prefix — and this is the key idea: **no node stores a word**. A node stores a map from the next character to the next node, plus one boolean saying whether a word ends here. The word only exists as the walk.

That gives me  $O(L)$  for insert, search and prefix search, where  $L$  is the length of the word — and notice there is no  $n$  in that at all. A million words or a billion, `'pre'` is three steps. I also get alphabetical order for free by visiting children in sorted order.

What it costs is memory, and I would want to be honest about that. One node per distinct prefix, and each node carries a dictionary — a couple of hundred bytes in Python. For English words, where prefixes are shared heavily, that is maybe five times a hash set. For random identifiers, where nothing is shared, it can be fifty times worse and slower as well. So the question I would ask before choosing this is: do the keys actually share prefixes, and do I actually need prefix queries?

Shall I write it?”

That is the recognition, the key idea, both costs, and the disqualifying question — in ninety seconds.

## The follow-ups

“Why not a hash set?”

“For exact lookup I would use the hash set — it is smaller, simpler, and just as fast, since hashing has to read the whole key anyway. The trie earns its place only when I need prefixes. Three things it gives me that a set cannot: prefix queries in  $O(L)$ , sorted iteration for free, and a guaranteed worst case with no collisions. If none of those are needed, the trie is pure cost.”

“Why not a sorted list with binary search?”

“That is the better question, and for a *static* word list a sorted array is often the right answer —  $O(L \log n)$  to find the prefix range, then walk it, and it uses a fraction of the memory with far better locality. The trie wins in two places. Updates: inserting into a sorted array is  $O(n)$  because everything shifts, against  $O(L)$  for the trie. And complex queries: a wildcard search like `c.t`, or searching a board of letters where I need to abandon a path the moment it stops being a valid prefix. A sorted array cannot do that pruning.”

### “How would you cut the memory?”

“Compress the chains. Any run of single-child nodes gets collapsed into one node holding the whole substring — that is a radix tree, and it typically removes sixty to eighty percent of the nodes, because the bottom of a trie is mostly long thin chains. If the word list is fixed I would go further and merge identical suffixes too, which makes it a DAWG — a graph rather than a tree — and can shrink a hundred thousand words to a few thousand nodes. The catch is that a DAWG is very hard to update, so it suits a shipped dictionary, not a live index.”

### “How would you return the top ten completions rather than all of them?”

“Storing every completion under `'a'` and then sorting is the wrong shape — that is most of the dictionary. Two better options. If I need them by frequency, I store at every node the best few completions in its subtree, precomputed at build time; then a lookup is  $O(L)$  and the answer is sitting at the node. That costs memory proportional to nodes times ten. Or I walk the subtree with a size-ten heap, which is yesterday’s shape two, and stop when I have enough. I would use the precomputed version for a search box, because the read path has to be fast and the write path is a nightly build.”

### The model answer

“Design the autocomplete for a search box. As the user types, show the ten most popular completions.”

“Let me set the shape first. This is a prefix query on every keystroke, so a hash set is out — it cannot answer it. A trie is the natural fit:  $O(L)$  per keystroke, no dependence on how many queries I have stored.

**The structure.** A trie over past search queries. At each node I store the ten best completions in that subtree, precomputed — each as (popularity, query text) . So 'pre' is three steps down and then I read a list of ten that is already sitting there. No subtree walk, no sorting, no heap at request time. The whole read path is: walk  $L$  nodes, return a list.

**The numbers.** Say ten million distinct queries, average twenty characters. That is 200 million characters, but queries share prefixes heavily, so maybe 60 million nodes. At 200 bytes a node in Python that is 12 GB — too much for one machine, and this is exactly why I would not build it in Python. In C++ with a compressed radix tree the node count drops by roughly seventy percent to around 18 million, and those nodes are smaller. Call it 2-4 GB, which fits comfortably in memory on one machine. The top-ten lists are the other cost: 60 million nodes  $\times$  10 entries is too much, so I would only store them at nodes above some depth — the first six characters, say — because nobody autocompletes a twenty-character prefix and if they do, walking the small subtree below it is cheap.

**Serving it.** The trie is read-only at request time and rebuilt nightly from query logs. That is the decision that makes the whole thing easy: no concurrent writes, no locks, no invalidation. I build a new trie, load it, and swap. It fits in memory, so I replicate it across every serving machine and any machine can answer any request — no sharding by prefix, no coordination. If it did not fit, I would shard by first character, but I would try very hard to avoid that, because it turns one lookup into a routed one.

**Latency.** Twenty-character prefix means at most twenty pointer hops in memory — under a microsecond for the trie itself. The real budget goes to the network and to the fact that the browser fires a request on every keystroke. I would debounce at about 150 milliseconds on the client and cancel in-flight requests, which cuts the request volume by more than half before it ever reaches me.

**What I would flag.** Freshness: a nightly rebuild means a query that becomes popular this morning does not appear until tomorrow. If that matters — breaking news, a new product launch — I would put a small in-memory layer of today's trending queries in front and merge the two result lists at read time. That is a second, much smaller structure rather than a change to the main one.



**What I would ask.** Personalised or global? Personalised completions change the design completely — I cannot precompute per user at that scale, so it becomes a global trie plus a small per-user history merged at read time. And is it multilingual? That kills the array-of-26 optimisation immediately and forces a map-based node, which I would probably do anyway.”

That answer picks the structure for a stated reason, moves the work to build time, gives real memory arithmetic including the point where Python fails, notices the client-side win, names the freshness weakness, and ends with the two questions that would change the design.

## 9 Recall card

**A trie is a tree of characters.** The **path spells the key**. No node stores a word.

**A node holds exactly two things:** `children: dict[str, TrieNode]` and `is_end: bool`.

```
class TrieNode:
 def __init__(self) → None:
 self.children: dict[str, TrieNode] = {}
 self.is_end: bool = False
```

**Insert:** walk, creating missing children, then set `is_end = True` on the last node.

**Search vs starts\_with — the whole lesson in two lines:**

```
search: node is not None and node.is_end
starts_with: node is not None
```

**Existence and word-ness are different facts.** With `"card"` stored, the node at `"car"` exists and is not a word. The only guarantee: **every leaf is a word**.

**Costs:** insert, search, prefix — all  $O(L)$ . **No `n` anywhere.** Collecting words costs  $O(L + \text{characters returned})$ .

**Three things a hash set cannot do:** prefix queries, sorted output for free, guaranteed worst case.

**The price is memory.** One node per distinct prefix, ~150-250 bytes each in Python. English words:  $\sim 5 \times$  a hash set. Random identifiers:  $50\text{--}80 \times$  worse and slower. **Only use a trie when the keys share prefixes and you need prefix queries.**

**The real competitor is a sorted array + binary search** —  $O(L \log n + k)$ , far less memory. The trie wins on frequent updates and on wildcard or pruned searches.

**Shrink it with:** a radix tree (collapse single-child chains, –60-80% nodes), a DAWG (also merge suffixes, static only), or a ternary search tree.

**Store** `words_below` **on every node** to answer “how many start with this?” in  $O(L)$ .

**A trie does prefixes, not substrings.** “Contains” needs a suffix structure.

---

**Next:** [Day 121 — Insert, search, and prefix search](#)

**Previous:** [Day 119 — Heaps revision and mock round](#)

## 1 What this is, and why they ask it

A transaction on one machine is a solved problem. You learned it on day 32: begin, do several things, commit — and if anything fails, none of it happened. All or nothing. The machine can guarantee that because one process owns all the data and one log records the decision.

Now split the data across two machines.

The money leaves the payments database. The order is written to the orders database. Both must happen, or neither.

Nothing in your toolkit does this. The payments machine can commit its half and the orders machine can fail a millisecond later, and now money has left an account for an order that does not exist. There is no shared log, no shared lock, no single process that owns both.

**Two-phase commit** — 2PC — is the classic answer. It has been in databases since the 1970s, it is what `XA` means, and it is the thing every interviewer expects you to be able to describe.

It is also, in most modern designs, the wrong answer, and knowing *why* is the more valuable half of this lesson. 2PC has a specific, well-known failure mode that blocks your system indefinitely, and a generation of engineers built architectures specifically to avoid it. Tomorrow's lesson — the saga pattern — is what they built instead.

So the goal today is: describe 2PC precisely, understand exactly where it breaks, know its variants and what they cost, and be able to say clearly when you would use it and when you would not.

## 2 The story

Farida is buying a flat. It is the third one she has tried to buy this year, and the first two fell through, which is why she now understands the problem better than her own lawyer.

There are four people who must all move on the same day. Farida hands over the money. The seller hands over the keys and signs the papers. Farida's bank releases the loan. The seller's bank closes the seller's old loan and releases the flat's papers from its vault. Four separate offices, four separate sets of paperwork.

The first sale collapsed because everyone acted in their own order. Farida's bank released the money on a Tuesday. The seller's bank, which had no idea, sat on the papers for eleven days because someone was on leave. For eleven days Farida had no money and no flat. She got the money back eventually. It took four months.

The second time, she used a woman named Kalpana, who does nothing else for a living.

Kalpana's method has two parts, and she explains it to every client the same way.

"First I ask everybody a question. Not 'do it' — a question. I ring each of the four and I say: *if I ask you on Friday morning, can you do your part, with certainty?* And they check. The bank checks the loan is sanctioned and the amount is sitting there. The seller checks the papers are actually in his hand and not still at his mother's house. Each one comes back to me with one word. Yes or no.

"And if they say yes, they are not just guessing. They are promising. From that moment they set their part aside and they do not touch it. The money is blocked. The papers are in an envelope with your name on it, in a drawer, not being used for anything else. They have given up the right to change their minds."

The second part is shorter.

"If all four say yes, I ring all four back and say one word: go. Nobody is allowed to ask me why, nobody is allowed to check anything, nobody is allowed to say 'actually'. They said yes. They do it.

"And if even one says no — one — I ring everybody and say: drop it. Nobody is out of pocket, because nobody has done anything yet. That is the point of asking first."

Farida asked the obvious question. What if you fall ill on Friday morning, after everyone has said yes?

Kalpana did not brush it aside. "Then you have a bad problem," she said. "They have all set their part aside. The bank is holding your money, blocked, doing nothing. The seller cannot sell to anybody else, because he has promised those papers to you. And none of them can decide for themselves what to do, because none of them knows what the others said. If the seller assumes it is off and sells elsewhere, and the bank assumes it is on and pays out, you have a catastrophe.

“So they wait. All of them. Blocked, doing nothing, until somebody finds me. That is the flaw in my method and I will not pretend it is not there. What I do about it is simple — my junior has a copy of every file, and she knows to open it if I do not appear. She is not there to be clever. She is there so that somebody alive knows what was decided.”

### 3 The idea in plain English

Kalpna is a transaction coordinator, and her method is two-phase commit exactly.

**The problem.** A transaction touches data on several machines. Each machine can guarantee all-or-nothing *locally*. Nobody can guarantee it *globally*, because each machine only sees its own half and each one can fail independently.

**The roles.** One **coordinator** — Kalpna — and several **participants**, the machines holding the data. The coordinator is not a database. It just runs the protocol and, crucially, keeps a durable record of what it decided.

**Phase one: prepare.** The coordinator asks every participant: *can you commit this, for certain?* Each participant does all the work — validates constraints, writes the changes to its log, takes the locks — and then stops just short of committing. It replies `yes` or `no`.

A `yes` is not an opinion. **It is an irrevocable promise.** Having voted yes, the participant has given up the right to abort on its own. It must be able to commit later even if it crashes and restarts in between, which means the vote and the prepared state must be written to durable storage before the vote is sent. And it holds its locks the entire time.

**Phase two: commit or abort.** If every vote is `yes`, the coordinator writes `COMMIT` to its own durable log — **that write is the moment the transaction becomes real** — and then tells everyone to commit. If any vote is `no`, or anyone fails to answer, it writes `ABORT` and tells everyone to roll back.

In phase two, participants have no say. They do what they are told, retrying forever if necessary.

**The single point that makes it correct.** The commit decision exists in exactly one place: the coordinator’s log. Not a majority, not a vote — one durable record on one machine. Everything else is derived from it. That is what makes 2PC simple to reason about, and it is also the source of its problem.

**The problem, named.** 2PC is a **blocking protocol**. If the coordinator fails after collecting all the yes votes but before broadcasting the decision, every participant is stuck. They cannot commit — maybe someone voted no and the decision was abort. They cannot abort — maybe everyone voted yes and the decision was commit. They have promised, so they cannot decide alone. They hold their locks and wait.

And they wait for a *human*, in practice. This is called the **in-doubt** or **uncertainty window**, and it is not theoretical: DBAs who have run XA transactions in production know the experience of finding a prepared transaction holding locks on a production table with no coordinator alive to resolve it.

**Why no clever protocol fixes it.** There is a theorem here, and it is worth being able to state: **no atomic commit protocol can be non-blocking in the presence of network partitions**. A participant that has voted yes and can hear nothing simply does not have the information to decide, and no amount of extra messages creates information out of silence. Three-phase commit reduces the window but only survives crashes, not partitions — which is why nobody uses it.

**The comparison that matters, and it comes up constantly.** 2PC and consensus are not the same thing and not competitors:

- **Consensus** (yesterday) makes several machines agree on **one value**, and survives a minority failing, because it uses a majority. It needs  $2f+1$  machines to tolerate  $f$  failures.
- **2PC** makes several machines agree on **whether one transaction happened**, and needs **unanimity** — every participant must vote yes. One participant down means abort. It does not tolerate failures at all; it just avoids inconsistency.

The modern combination is to use both: run 2PC, but make the *coordinator* a replicated state machine backed by Raft, so that the coordinator's log survives its own failure and a new coordinator can pick up in-doubt transactions. Kalpana's junior with the copy of every file. This is what Google's Spanner and CockroachDB do, and it is the strongest thing you can say about 2PC in an interview.

---

#### 4 The picture

##### The happy path

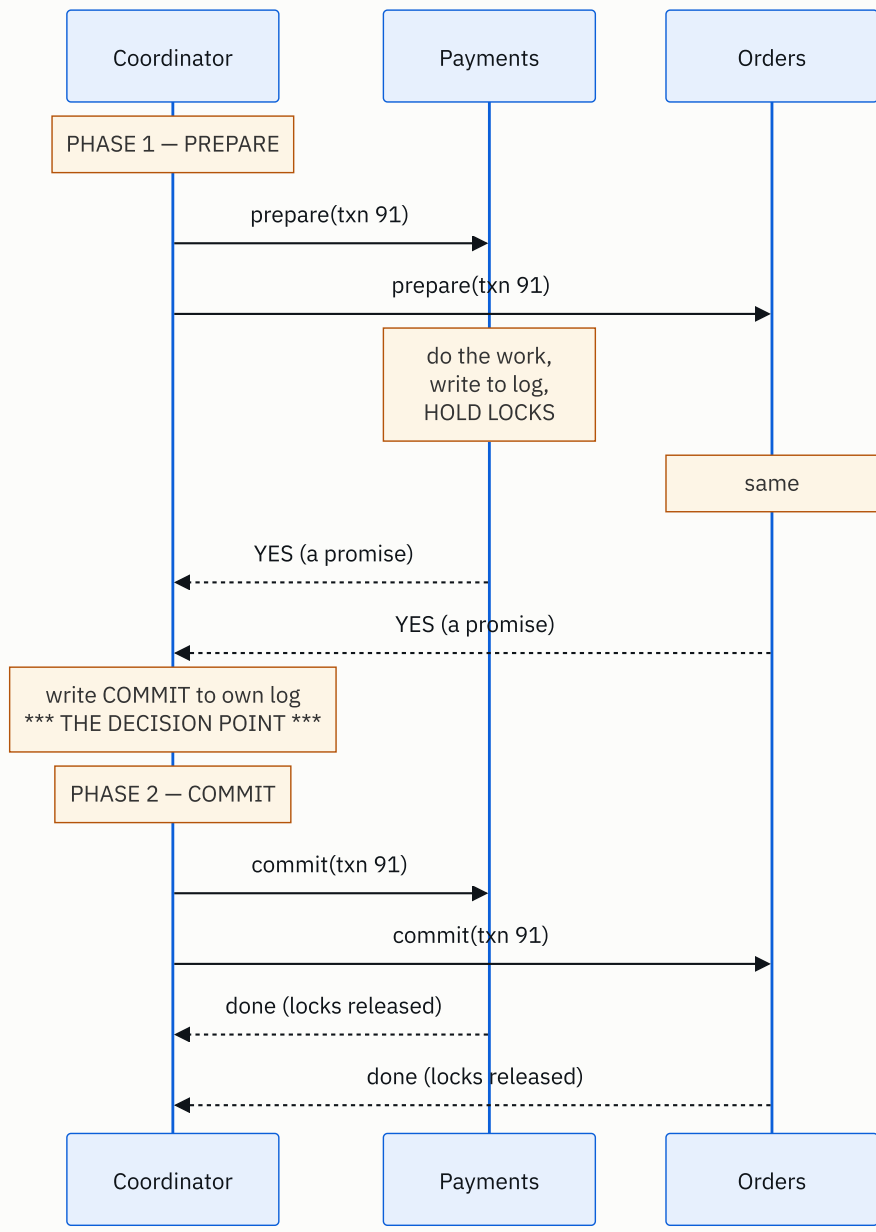


FIGURE 120.2

*Notice how long the locks are held: from the middle of phase one until the middle of phase two — two full network round trips plus two disk syncs. On one machine a transaction holds locks for microseconds. Here it is milliseconds, and if anything goes wrong, minutes.*

### The failure that defines the protocol

```
t=0 Coordinator sends prepare to P and O
t=1 P writes its log, locks rows, votes YES
t=2 O writes its log, locks rows, votes YES
t=3 Coordinator receives both votes
t=4 Coordinator's machine loses power
 *** before writing COMMIT, before sending anything ***

Now:
P: "I voted yes. I am holding locks on 4,000 rows.
 I cannot commit — maybe O voted no.
 I cannot abort — maybe the decision was commit.
 I must wait."
O: exactly the same.

P and O ask each other. It does not help:
 both are in the same state, and neither knows the decision,
 because the decision was never made.

The rows stay locked. Every other transaction touching them queues.
This continues until the coordinator comes back, or a human intervenes.
```

*Notice the last line. The resolution is not automatic. That is the whole objection to 2PC.*

### Where the decision lives

| CONSENSUS (Raft)                                                                                                                                                           | TWO-PHASE COMMIT                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| -----                                                                                                                                                                      | -----                                                    |
| decision = what a MAJORITY<br>agreed                                                                                                                                       | decision = what the COORDINATOR<br>wrote to its log      |
| survives $f$ failures out of $2f+1$                                                                                                                                        | survives ZERO participant failures<br>(one down = abort) |
| needs a majority to vote YES                                                                                                                                               | needs EVERYONE to vote YES                               |
| question: "what value?"                                                                                                                                                    | question: "did it happen?"                               |
| Combine them: run 2PC with a Raft-replicated coordinator.<br>Now the decision is durable even if the coordinator machine dies.<br>This is what Spanner and CockroachDB do. |                                                          |



## What each participant knows, and when

| participant state             | can it decide alone?     | holding locks? |
|-------------------------------|--------------------------|----------------|
| before prepare                | YES – just abort         | no             |
| voted NO                      | YES – already aborted    | no (released)  |
| voted YES,<br>no decision yet | NO – THE IN-DOUBT WINDOW | YES            |
| received decision             | YES – do as told         | until done     |

The third row is the entire problem.  
Everything written about 2PC is about shrinking that row.

## 5 How it actually works

### The participant's side

The important thing is that a `yes` vote must be durable *before* it is sent.

```
from enum import Enum

class Vote(Enum):
 YES = "yes"
 NO = "no"

def on_prepare(self, txn_id: str, changes: list[Change]) → Vote:
 try:
 self.acquire_locks(changes) # may block or time out
 self.validate(changes) # constraints, permissions
 self.write_to_log(txn_id, changes, state="PREPARED")
 self.fsync() # MUST be durable before voting
 except Exception:
 self.release_locks(txn_id)
 return Vote.NO # free to abort: nothing promised
 return Vote.YES # now committed to committing
```

If the machine dies immediately after the `fsync`, it restarts, reads `PREPARED` in its log, re-acquires the locks, and asks the coordinator what was decided. That is why the `fsync` cannot be skipped: a vote that does not survive a crash is a lie.

## The coordinator's side

```
def run(self, txn_id: str, participants: list[Participant]) → bool:
 # ---- phase 1 ----
 votes = []
 for participant in participants:
 try:
 votes.append(participant.prepare(txn_id, timeout=5.0))
 except TimeoutError:
 votes.append(Vote.NO) # silence counts as NO

 decision = "COMMIT" if all(v is Vote.YES for v in votes) else "ABORT"

 # ---- the decision point ----
 self.log.write(txn_id, decision)
 self.log.fsync() # AFTER this, it is real

 # ---- phase 2 ----
 for participant in participants:
 self.deliver_forever(participant, txn_id, decision)
 return decision == "COMMIT"
```

Three lines carry all the weight.

`except TimeoutError: votes.append(Vote.NO)` — **silence is a no**. In phase one, safety is easy: when in doubt, abort. Nobody has committed anything yet.

`self.log.fsync()` — the decision point. Before it, the transaction never happened. After it, it certainly did, and no failure can change it.

`deliver_forever` — in phase two, silence is *not* an option. The coordinator retries until every participant acknowledges, for hours if necessary, because the decision is already made and participants are holding locks waiting for it.

```
def deliver_forever(self, participant, txn_id: str, decision: str) → None:
 delay = 0.1
 while True:
 try:
 participant.finish(txn_id, decision)
 return
 except NetworkError:
 sleep(min(delay, 30.0))
 delay *= 2
```

## Recovery, which is most of the real code

**A participant restarts.** It scans its log. For each `PREPARED` transaction with no outcome recorded, it re-acquires the locks and asks the coordinator. It must not guess.

```
def on_restart(self) → None:
 for txn_id, state in self.log.scan():
 if state == "PREPARED":
 self.reacquire_locks(txn_id)
 decision = self.coordinator.ask_outcome(txn_id) # blocks
 self.apply(txn_id, decision)
```

**The coordinator restarts.** For every transaction with a decision written, it resumes delivering it. For every transaction that reached phase one with no decision written, it writes `ABORT` — safe, because if no decision was recorded, nobody was ever told to commit.

```
def on_restart(self) → None:
 for txn_id, record in self.log.scan():
 if record.decision is None:
 self.log.write(txn_id, "ABORT") # safe: nobody was told to commit
 self.log.fsync()
 self.resume_phase_two(txn_id)
```

**Note the asymmetry.** A participant in doubt must *ask*. A coordinator in doubt may *decide*, and always decides abort. That is not arbitrary — the coordinator is the only party that knows nothing was broadcast.

### The presumed-abort optimisation

Every real implementation does this. If a coordinator is asked about a transaction it has no record of, it answers `ABORT`. That means the coordinator never has to log anything for aborted transactions, and can forget a transaction the moment it is complete. It removes a large fraction of the logging in a system where most transactions are read-only or short.

### XA, which is what this is called in practice

`XA` is the standard interface, from 1991. Postgres, MySQL, Oracle, IBM MQ and most message brokers implement it. In Java it is `JTA` — `UserTransaction.begin()`, `commit()` — with an application server as the coordinator.

```
-- Postgres speaks 2PC directly.
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 7;
PREPARE TRANSACTION 'txn-91'; -- phase 1: durable, locks held, no going back

-- ... coordinator asks the other participants ...

COMMIT PREPARED 'txn-91'; -- phase 2
-- or
ROLLBACK PREPARED 'txn-91';
```

Try this and then look at what you have created:

```
SELECT gid, prepared, owner FROM pg_prepared_xacts;
```

A prepared transaction survives a server restart and holds its locks the whole time. In Postgres, `max_prepared_transactions` defaults to **zero** — the feature is off unless you deliberately enable it, which tells you what the maintainers think of it.

### Three-phase commit, and why you should know it and not use it

3PC inserts a `pre-commit` phase between prepare and commit, so participants learn that everyone voted yes before anyone commits. A participant that reaches pre-commit and then loses the coordinator can safely commit on a timeout.

It fails under network partitions. Two groups can time out on opposite sides and reach opposite decisions — which is worse than blocking, because blocking is at least safe. It also adds a third round trip. **Mention that you know it exists, that it only helps for crashes and not partitions, and that nobody deploys it.** That is the complete correct answer.

## 6 The numbers

### Latency

|                                                               |                               |
|---------------------------------------------------------------|-------------------------------|
| Single-machine transaction:                                   |                               |
| begin, write, fsync, commit                                   | ~1-5 ms (the fsync dominates) |
| 2PC across 3 participants, same data centre:                  |                               |
| phase 1: coordinator → participants                           | 0.5 ms                        |
| participant work + fsync                                      | 1-5 ms                        |
| votes back                                                    | 0.5 ms                        |
| coordinator log write + fsync                                 | 1-5 ms                        |
| phase 2: send decision                                        | 0.5 ms                        |
| participant commit + fsync                                    | 1-5 ms                        |
| acknowledgements                                              | 0.5 ms                        |
|                                                               | -----                         |
| total                                                         | 5-18 ms                       |
| So: 3-5× a local transaction, and FOUR fsyncs instead of one. |                               |

**The fsyncs dominate, not the network.** Same as consensus yesterday. Say it, because most people blame the network.

Across regions it becomes untenable:

```
Mumbai coordinator, participants in Mumbai / Singapore / Frankfurt:
phase 1 round trip = slowest participant = ~220 ms (Frankfurt)
phase 2 round trip = ~220 ms
total = ~450 ms + disk
```

And unlike consensus, you cannot wait for a majority.  
2PC needs EVERY participant. The slowest one sets your latency,  
every single time.

### The lock window, which is the real cost

```
local transaction: locks held for ~1 ms
2PC transaction: locks held for ~10 ms → 10× contention
2PC, coordinator crashes: minutes to hours
```

What that does to throughput on a hot row:

```
lock held 1 ms → at most ~1,000 transactions/sec on that row
lock held 10 ms → at most ~100 transactions/sec
lock held 60 s → everything touching that row is dead
```

**This is the argument that actually wins.** Not “2PC is slow” — “2PC multiplies lock contention by ten, and the hot rows in your system are exactly the ones a distributed transaction wants to touch.”

### Availability multiplies the wrong way

Because 2PC needs unanimity, availability is a *product*, not a majority:

each participant available 99.9% of the time

```
2 participants: 0.9992 = 99.80% → 17.5 hours down/year
3 participants: 0.9993 = 99.70% → 26.3 hours
5 participants: 0.9995 = 99.50% → 43.8 hours
10 participants: 0.99910 = 99.00% → 87.6 hours
```

Compare Raft with 5 nodes tolerating 2 failures:  
availability ≈ 99.9999% → about 30 seconds/year

Two structures, opposite shapes. **Consensus gets more reliable as you add machines. 2PC gets less reliable.** That single sentence is worth the whole section.

## The in-doubt window

|                                        |                       |
|----------------------------------------|-----------------------|
| best case (coordinator healthy):       | 1-10 ms               |
| coordinator crashes, fast restart:     | seconds to minutes    |
| coordinator crashes, no auto-recovery: | until a human notices |
| observed in production XA incidents:   | hours                 |

Meanwhile: locks held, queues building, unrelated transactions timing out because they touch one of the locked rows.

## What it does not cost

Fairness note: with a Raft-replicated coordinator, the in-doubt window collapses to a leader election — a few hundred milliseconds — and the availability problem largely goes away. Spanner reports commit latencies in the tens of milliseconds for cross-shard transactions. **2PC's reputation was earned by single-coordinator XA in the 1990s, not by the modern version.** Say so; it is a mark of someone who has read past the standard criticism.

## 7 The trade-offs

### The one-line summary

2PC buys you atomicity across machines, and you pay with blocking, lock contention, latency, and an availability that falls as you add participants.

### When 2PC is the right answer

- **Inside one database product, across its own shards.** Spanner, CockroachDB, TiDB, YugabyteDB all use 2PC internally with a replicated coordinator. It is invisible, tuned, and correct.
- **Few participants, one data centre, short transactions.** Two or three, milliseconds long.
- **When correctness is non-negotiable and compensation is impossible.** Some financial and regulatory operations genuinely cannot be undone after the fact.
- **When the alternative is worse.** Sometimes eventual consistency really is unacceptable and the complexity of a saga exceeds the cost of blocking.

### When it is not

- **Across services you do not own.** A payment gateway will not join your XA transaction.

- **Across a wide area.** Hundreds of milliseconds, with the slowest participant setting the pace.
- **Long-running work.** A transaction spanning a human approval would hold locks for hours.
- **Many participants.** Availability decays as a product.
- **Microservices in general.** This is the big one. 2PC requires participants to expose prepare/commit and hold locks on your behalf, which couples them tightly — the exact opposite of why the services were split up. **Say this out loud: the industry moved away from 2PC not mainly because it is slow, but because it reintroduces the coupling that services exist to remove.**

#### Against the alternatives

| APPROACH               | ATOMIC?       | BLOCKS?    | ISOLATION   | COMPLEXITY | USE WHEN                          |
|------------------------|---------------|------------|-------------|------------|-----------------------------------|
| Single database        | yes           | no         | full        | lowest     | data can live together            |
| 2PC / XA               | yes           | <b>yes</b> | full        | medium     | few participants, one DC, short   |
| 2PC + Raft coordinator | yes           | briefly    | full        | high       | you are building the database     |
| Saga (tomorrow)        | no — eventual | no         | <b>none</b> | high       | services, long-running, wide area |
| Outbox + events        | no — eventual | no         | none        | medium     | one write plus notifications      |
| Don't distribute       | yes           | no         | full        | lowest     | <b>check this first</b>           |

The last row is not a joke. **The best answer to a distributed transaction question is often to ask why the data is distributed.** If two tables are always written together, they may simply belong in one database, or in the same shard. Spanner and CockroachDB both let you co-locate related rows precisely so the transaction stays local. Eliminating the distributed transaction beats optimising it.

#### The isolation point people miss

2PC gives **atomicity** — all or nothing. It does not by itself give **isolation** across participants. Between phase one and phase two, a reader on participant A can see the change while a reader on participant B cannot. Getting a globally consistent *read* needs something extra — a global timestamp, snapshot isolation, Spanner's TrueTime. This is a good detail to raise unprompted; it shows you know the difference between the A and the I in ACID.

## 8 In the interview

### How it gets asked

- *“Money moves between two banks with separate databases. How do you keep it consistent?”*
- *“Your order service and inventory service have separate databases. How do you make them atomic?”*
- *“What is two-phase commit, and what is wrong with it?”* — the direct version.
- *“What happens if the coordinator dies?”* — the real question hiding inside every version.
- *“Why do microservices avoid distributed transactions?”*
- *“How does Spanner do cross-shard transactions?”* — the senior version.



## The first ninety seconds

“The problem is that each machine can guarantee all-or-nothing for its own data, but nobody can guarantee it across machines, because each one can fail independently.

Two-phase commit adds a coordinator. **Phase one, prepare:** it asks every participant ‘can you commit this, for certain?’ Each one does all the work, writes it durably to its log, takes its locks, and stops just short of committing. A yes here is not an opinion — it is an irrevocable promise, and it has to survive a crash, so it is fsynced before the vote is sent. **Phase two:** if every vote is yes, the coordinator writes COMMIT to its own log — that write is the moment the transaction becomes real — and tells everyone to go. Any no, or any silence, and it is abort.

Now the important part, which is where it breaks. If the coordinator dies after collecting all the yes votes but before broadcasting the decision, every participant is stuck. They cannot commit, because maybe someone voted no. They cannot abort, because maybe the decision was commit. They promised, so they cannot decide alone. They hold their locks and wait — often for a human. 2PC is a **blocking** protocol, and there is a theorem saying no commit protocol can avoid that under network partitions.

The cost that actually kills it in practice is not latency, it is the lock window. Locks are held across two network round trips instead of microseconds, so contention goes up roughly tenfold on exactly the hot rows you care about. And availability is a product, not a majority — every extra participant makes it worse, which is the opposite of consensus.

Which is why for something like an order and inventory service I would reach for a saga instead. Shall I go into that, or into how Spanner makes 2PC workable?”

## The follow-ups

“How do you fix the blocking?”

“You cannot remove it — the theorem says so under partitions. But you can shrink it a great deal. The modern answer is to stop making the coordinator a single machine and make it a replicated state machine backed by Raft. The commit decision goes into a replicated log rather than one machine’s disk, so if the coordinator dies a new one is elected in a few hundred milliseconds, reads the log, and finishes the job. The in-doubt window goes from ‘until a human notices’ to ‘one election’. That is what Spanner and CockroachDB do, and it is why 2PC is fine inside a modern distributed database and painful as XA between separate products.

People will also mention three-phase commit here. It reduces the window but only survives crashes, not partitions — under a partition two sides can reach opposite decisions, which is worse than blocking because blocking is at least safe. Nobody deploys it.”

### **“Why don’t microservices use it?”**

“Three reasons, and the third is the real one. Availability multiplies down — five services at three nines gives you 99.5%. Locks are held across service boundaries, so one slow service degrades everything that touches the same rows. And the coupling: to be a participant, a service has to expose prepare and commit and hold locks on someone else’s behalf, which is exactly the tight coupling the services were split up to remove. You end up with a distributed monolith with worse latency. The industry moved to sagas — a sequence of local transactions with compensating actions — trading atomicity for availability and accepting that there are windows where the system is visibly inconsistent.”

### **“What if a participant votes yes and then crashes?”**

“That case is fine, and it is exactly what the durable vote is for. The participant wrote PREPARED to its log and fsynced it before voting. On restart it scans its log, finds the prepared transaction, re-acquires the locks, and asks the coordinator what was decided. It must not guess in either direction. The coordinator, meanwhile, keeps retrying delivery — in phase two it never gives up, because the decision is already made and everyone else is waiting on it.

The reverse case is the asymmetry worth naming: a coordinator that restarts and finds a transaction with no decision logged writes ABORT immediately. That is safe, because if nothing was logged, nothing was broadcast. A participant in doubt must ask; a coordinator in doubt may decide, and always decides abort.”

**“Is 2PC the same as consensus?”**

“No, and they are not competitors. Consensus makes machines agree on a value and survives a minority failing, because it needs a majority — five machines tolerate two failures. 2PC decides whether one transaction happened and needs unanimity — one participant down means abort, so it tolerates no failures at all. Consensus gets more reliable as you add machines; 2PC gets less. The modern systems use both: 2PC for the atomicity across shards, with the coordinator’s log replicated by Raft so the decision itself survives failure.”

**The model answer**

*“An order service and an inventory service have separate databases. Placing an order must reserve stock. How do you make that atomic?”*

“Before I design anything, I want to ask why they are separate, because the cheapest fix is often to remove the problem. If orders and inventory are always written together and nothing else needs them apart, they may simply belong in one database, or in one shard of a database that supports co-location. That gives me a local transaction — full atomicity, full isolation, one fsync, no coordinator. I would push on this first, because every option below is worse than it.

Assuming they are genuinely separate services with separate teams and lifecycles:

**2PC would work and I would not choose it.** Both services would expose prepare and commit, the order service or a coordinator would drive it, and I would get true atomicity. The costs: inventory rows are the hottest rows in the system, and holding locks on them across two network round trips instead of microseconds cuts throughput on popular items by roughly ten times. Availability becomes the product of both services. And inventory now has to hold locks on behalf of orders, which couples the two services exactly as much as sharing a database would, without the benefits.

**What I would build is a saga.** Reserve the stock first, as a local transaction in inventory, with the reservation carrying an expiry. Then create the order, as a local transaction in orders. If the order creation fails, issue a compensating action: release the reservation. Each step is a local ACID transaction; the sequence is driven by events or by an orchestrator that keeps state.

**The consequences I would state honestly, because they are real.** There is a window where stock is reserved and no order exists — a customer might see an item as unavailable for a few hundred milliseconds when it is actually free. Sagas have no isolation, so a concurrent order can see intermediate state. And compensation is not rollback: if the reservation succeeded and the order failed and the compensation *also* failed, I have leaked a reservation. That is why the reservation must expire on its own — a background job reclaiming reservations older than fifteen minutes is what makes the design safe, and it is the part people forget.

**The transactional detail that matters most.** In the order service, writing the order row and publishing the ‘order created’ event must be atomic, or I get an order with no event or an event with no order. I would use the **outbox pattern**: write the event into an outbox table in the same local transaction as the order, and have a separate process read the outbox and publish. That turns a two-system problem into a one-database problem, and it is the single most useful trick in this whole area.

**Numbers.** Say a thousand orders a second at peak, three hundred milliseconds end to end for the saga including the event hop. With 2PC I would have maybe ten to eighteen milliseconds per transaction but serialised on hot inventory rows — a popular item during a sale would become the bottleneck outright. The saga has no cross-service locks at all, so hot items are limited only by inventory's own local transaction rate.

**What would change my mind.** If this were money rather than stock — if the failure mode were 'the customer was charged and has no order' — I would look much harder at either co-locating the data or using a database with built-in distributed transactions. Overselling a T-shirt is a business problem with a business answer, which is what makes the saga acceptable here. Losing a payment is not."

That answer questions the premise first, describes 2PC accurately, rejects it for named reasons with numbers, proposes the alternative, states its weaknesses honestly, includes the outbox detail, and says what would change the decision.

## 9 Recall card

**The problem:** each machine guarantees all-or-nothing locally; nobody guarantees it across machines.

**2PC has one coordinator and several participants.**

**Phase 1 — prepare:** "can you commit, for certain?" Participant does the work, writes it durably, **takes locks**, votes. **A yes is an irrevocable promise** and must survive a crash — fsync before voting.

**Phase 2 — commit/abort:** all yes → coordinator writes **COMMIT** to its log (**this write is the decision point**) → tells everyone. Any no or any silence → abort.

**Phase 1: silence = NO** (safe, nothing committed). **Phase 2: retry forever** (the decision exists).

**The flaw: 2PC blocks.** Coordinator dies after the votes, before the decision → participants cannot commit (maybe a no), cannot abort (maybe a commit), cannot decide alone (they promised). **Locks held until a human resolves it.** No protocol avoids this under partitions.

**The asymmetry:** participant in doubt must **ask**; coordinator in doubt **decides ABORT**.

**Numbers:** ~5-18 ms vs 1-5 ms locally, **four fsyncs**; locks held  $\sim 10\times$  longer  $\rightarrow \sim 10\times$  less throughput on hot rows; in-doubt window minutes to hours. Cross-region  $\approx 450$  ms, set by the **slowest** participant.

**Availability is a product:** 3 participants at 99.9%  $\rightarrow$  99.7%. **Consensus improves with more machines; 2PC gets worse.** Not the same problem: consensus = majority on a value; 2PC = unanimity on “did it happen”.

**The modern fix:** replicate the coordinator’s log with Raft. In-doubt window becomes one election. Spanner, CockroachDB, TiDB.

**3PC:** shrinks the window, fails under partitions, nobody uses it. Know it, do not propose it.

**XA** is the standard; `PREPARE TRANSACTION` in Postgres, where `max_prepared_transactions` defaults to **0**.

**2PC gives atomicity, not isolation** across participants — a global consistent read needs more.

**Avoid it for:** services you do not own, wide areas, long transactions, many participants, microservices. **The real objection is coupling**, not speed.

**First question to ask: why is the data distributed at all?** Co-locating beats optimising.

---

**Next:** [Day 121 — The saga pattern](#)

**Previous:** [Day 119 — Consensus, and Raft in plain English](#)

**DSA topic:** The trie: a tree of characters **System design topic:** Distributed transactions and two-phase commit

Code these, in this order

One rule for the whole set: **write the node class from memory before you look at anything.** Two fields. If you cannot produce those two lines cold, nothing else in this topic will come.

| # | PROBLEM                      | SOURCE                | WHAT IT IS REALLY TESTING                                                        |
|---|------------------------------|-----------------------|----------------------------------------------------------------------------------|
| 1 | Implement Trie (Prefix Tree) | LeetCode 208 (Medium) | The node, insert, and <code>search</code> vs <code>startsWith</code> .           |
| 2 | Longest Common Prefix        | LeetCode 14 (Easy)    | Whether you know a trie is <i>overkill</i> here.                                 |
| 3 | Longest Word in Dictionary   | LeetCode 720 (Medium) | Every prefix must also be a word — <code>is_end</code> used as a real condition. |
| 4 | Replace Words                | LeetCode 648 (Medium) | Stopping the walk at the first <code>is_end</code> .                             |
| 5 | Map Sum Pairs                | LeetCode 677 (Medium) | The <code>words_below</code> augmentation, with sums instead of counts.          |

On problem 1, write the two one-liners side by side

`search` and `startsWith`, adjacent, and say aloud the single condition that differs. Then insert `"card"` only and check `search("car")`, `startsWith("car")`. Record both.

On problem 2, solve it twice

Once with a trie, once with a plain vertical scan. Compare line counts and memory. Say which you would write in an interview and why. This problem is here to teach you when *not* to use today's structure.

On problem 3, notice what `is_end` is doing

The condition is that every prefix of the answer is also a word. Say why that makes this a trie problem rather than a sorting problem, and what would change if the condition were dropped.

### On problem 5, add the augmentation

Store a running value at every node on the insert path. Then handle the update case — inserting the same key twice with different values — and record what breaks if you only add rather than adjust by the difference.

### Measure the memory yourself

Build a trie over an English word list and over the same number of random 10-character strings. Print `node_count()` for both, and the character totals. Two ratios. Keep them; they are the argument you will make in the interview.

### The structure drill

1. Write `TrieNode` from memory. Two fields.
2. Say what a node does *not* contain, and why that matters.
3. Say where the letter actually lives.
4. Say what the root represents.

### The path drill

1. State “the path is the key” and explain it in two sentences.
2. Given a trie of `car`, `card`, `cat`, `do`, list every node and say which are words.
3. Say how many nodes exist and how many characters were stored.
4. Say why a node cannot report its own prefix.

### The two-flags drill

1. State the two independent questions a node answers.
2. For `card` stored alone, give `is_end` and children for the node at `car`.
3. For `car` and `card` both stored, give the same.
4. State the one thing that is always true about leaves.
5. Say which bug appears if you check `is_end` in `starts_with`, and which if you omit it in `search`.

### The complexity drill

1. Give the cost of insert, search, prefix-exists and collect.
2. Say what is missing from all four, and why that is unusual.
3. Compare a prefix query against a hash set and a sorted array at  $n = 1,000,000$ .
4. Say which of the three you would pick for a static word list, and why.



### The memory drill

1. Say how many nodes a trie creates, in terms of prefixes.
2. Give a realistic bytes-per-node figure for Python and say where it goes.
3. Compute the memory for 100,000 English words and compare with a set.
4. Compute it for 1,000,000 UUIDs and give the ratio.
5. State the rule for when a trie is worth its memory.

### The alternatives drill

1. Describe a radix tree and give the typical node reduction.
2. Describe a DAWG and say what it cannot do.
3. Say what a ternary search tree trades.
4. Say which one a router uses and why.

### The variants drill

1. Compare the dict node and the array-of-26 node on three axes.
2. Say which you would use in Python and which in C++ , with reasons.
3. Say what breaks in the array version with uppercase input, and what breaks with accented input.
4. Say what `words_below` buys and how many lines it costs.

### The break-it drill

Trigger each and record the exact output or error:

1. `insert` with the final `is_end = True` line removed, then any `search` .
  2. `search("car")` with only `"card"` stored.
  3. `starts_with` written with the `is_end` check included.
  4. `ord('A') - ord('a')` used as a list index.
  5. `ord('é') - ord('a')` used as a list index.
  6. A recursive collect over a key 2,000 characters long.
  7. `words_with_prefix("")` on a full dictionary.
  8. Accessing `node.word` on a plain `TrieNode` .
- 

### The problem drill

1. State what a distributed transaction is, in one sentence.
2. Say why a single-machine transaction cannot be extended to two machines.

3. Give the concrete two-database failure that motivates the whole lesson.

#### The protocol drill

1. Name the two roles.
2. Describe phase one, including what the participant does before voting.
3. Say what a `yes` vote commits the participant to.
4. Say why the vote must be fsynced before it is sent.
5. Describe phase two and say how much say participants have in it.
6. Name the exact moment the transaction becomes real.

#### The blocking drill

1. Describe the failure that defines 2PC, step by step with timings.
2. Say why a stuck participant cannot commit.
3. Say why it cannot abort.
4. Say why asking the other participants does not help.
5. State the theorem in one sentence.
6. Say what actually resolves it in production.

#### The asymmetry drill

1. Say what a participant in doubt must do.
2. Say what a coordinator in doubt does, and which way it decides.
3. Explain why that is safe and not arbitrary.
4. Say what `presumed abort` saves.

#### The comparison drill

1. Give four differences between 2PC and consensus.
2. Say which needs a majority and which needs unanimity.
3. Say which gets more reliable as machines are added, and which gets less.
4. Describe the modern combination of the two and name two systems that use it.

#### The numbers drill

1. Break a 2PC transaction into its steps with timings and count the fsyncs.
2. Say what dominates in one data centre.
3. Compute cross-region latency and say which participant sets it.
4. Compute availability for 2, 3, 5 and 10 participants at 99.9% each.
5. Compare with a five-node Raft cluster.

6. Give the lock window in both cases and the throughput consequence on a hot row.

#### The trade-offs drill

1. Give four situations where 2PC is the right answer.
2. Give five where it is not.
3. State the real objection to 2PC in microservices, and say why it is not speed.
4. Fill in the six-row alternatives table.
5. Say what the first question about any distributed transaction should be.
6. Say what 2PC does *not* give you, in terms of ACID letters.

#### The variants drill

1. Describe three-phase commit in two sentences.
2. Say what it fixes and what it does not.
3. Say why “worse than blocking” is the right criticism of it.
4. Say what `XA` is and give the Postgres commands.
5. Give the Postgres default for `max_prepared_transactions` and say what it signals.

#### The failure drill

For each, say what happens and what you would do:

1. A participant votes yes, then loses power.
2. The coordinator writes COMMIT, then loses power before sending anything.
3. The coordinator loses power after collecting votes but before writing the decision.
4. One of five participants is unreachable during phase one.
5. A prepared transaction is found holding locks on a production table with no coordinator alive.
6. A reservation is created, the follow-up write fails, and the compensating action also fails.

Two of the six are not fixed by anything inside 2PC. Name them.

---

#### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Implement a trie, and tell me why not just use a hash set.* The prefix query as the reason the structure exists, the path being the key with no word in any node, the two fields, `search` versus `startsWith` as one differing condition, `O(L)` with no `n`, and the memory cost stated honestly with the random-key case.
  2. *Two databases, one transaction. How?* The problem, both phases with the promise and the fsync, the decision point named, the blocking failure in full, the lock-window and availability arithmetic, and where you would go instead.
  3. *What happens if the coordinator dies?* The in-doubt window, why the participants cannot decide, the asymmetry between participant and coordinator recovery, and the Raft-replicated-coordinator fix.
- 

### Before you move on

- [ ] I can write `TrieNode` from memory, two fields.
- [ ] I can say what a node does not contain and where the letter lives.
- [ ] I can state “the path is the key” and explain it.
- [ ] I can write `insert` including the final `is_end` line.
- [ ] I can write `search` and `starts_with` and name the one differing condition.
- [ ] I know that existence and word-ness are separate facts.
- [ ] I know the one thing that is always true about leaves.
- [ ] I know all four operations are `O(L)` with no `n`.
- [ ] I can name the three things a hash set cannot do.
- [ ] I can give the memory arithmetic for English words and for random keys.
- [ ] I know the rule for when a trie is worth its memory.
- [ ] I know a sorted array is the real competitor, and where the trie beats it.
- [ ] I can describe a radix tree and a DAWG.
- [ ] I know `words_below` and what it buys.
- [ ] I know a trie does prefixes, not substrings.
- [ ] I can describe both phases of 2PC precisely.
- [ ] I know a `yes` vote is an irrevocable promise and must be fsynced first.
- [ ] I can name the exact moment the transaction becomes real.
- [ ] I can describe the blocking failure without notes.
- [ ] I can state why no protocol avoids it under partitions.
- [ ] I know the participant/coordinator recovery asymmetry.

- [ ] I can give the availability product and compare it with Raft.
- [ ] I can give the lock-window argument with throughput numbers.
- [ ] I know 2PC gives atomicity but not isolation.
- [ ] I know the real objection in microservices is coupling.
- [ ] I know 3PC exists, what it fixes, and why nobody uses it.
- [ ] I would ask why the data is distributed before designing anything.
- [ ] I answered all three questions above out loud.

# Insert, search, and prefix search

## DATA STRUCTURES AND ALGORITHMS

Insert, search, and prefix search

## SYSTEM DESIGN

The saga pattern

## 1 What this is, and why they ask it

Yesterday you built the structure. Today you make it work, and you meet the operation nobody expects.

Insert, search and prefix search are the three operations every trie question asks for, and all three are short — five lines, two lines, one line. If the interview stopped there, this would be a ten-minute lesson.

It does not stop there. The follow-up is almost always one of these two:

**“Now delete a word.”**

**“Now support a wildcard: `c.t` should match `cat` and `cut`.”**

Delete is the interesting one, because it is the only trie operation that is genuinely harder than it looks. Insert only ever adds. Search only ever reads. Delete has to *remove* nodes — and deciding which nodes may be removed is a real problem with four different cases, three of which look wrong the first time you see them.

Wildcards are the other one, and they matter because they break the single most comforting property of a trie. Every operation so far has followed one path. A wildcard forces you to follow *many*, which turns a walk into a search, and the cost changes shape completely.

By the end of this lesson you will be able to write all four operations from memory, state the exact rule for when a node may be pruned, handle wildcards with the right complexity analysis, and answer the two problems these operations exist for: LeetCode 208 and LeetCode 211.

## 2 The story

The municipal park in Ravindran’s town has four kilometres of gravel paths, and he has been the head gardener for twenty-two years.

The paths were never planned. They grew. Somebody put a bench under the gulmohar tree in 1994, so a path was laid to the bench. Later a drinking-water tap went in further along, so the path was extended past the bench to the tap. Then a small shrine

appeared off to one side, and a branch was laid from the middle of the path to the shrine.

This is how the whole park works. A path exists because something at the end of it needs reaching.

Last month the council removed the drinking-water tap. It had not worked in three years and the pipe was being re-laid.

A younger man on the crew asked whether they should now pull up the path to the tap, since it went nowhere. Ravindran said yes, and then said the thing that took him about five years to learn properly.

“Walk backwards from where the tap was,” he told him. “Not forwards. Backwards. And pull up the gravel as you come. But you stop — and listen to me, this is the whole job — you stop the moment one of two things happens.

“You stop if you reach a fork. If another path branches off here, then this bit of gravel is still being walked on by people going to the other place. It is not dead. Leave it.

“And you stop if you reach somewhere that is itself a place. The bench under the gulmohar. Nobody is going past the bench any more, but people still come to the bench. That path has to stay right up to the bench, and only the part beyond it comes up.”

The young man walked it and came back puzzled about something else. He had also been told to remove a bench near the gate — but the path to the gate bench carried on past it, down to the pond.

“Then you take away the bench and you touch nothing else,” said Ravindran. “Not one stone. The path is still needed. It is just not a destination any more, it is a way through. People walk it every day to reach the pond. If you pull that up because the bench is gone, you have cut off the pond.”

The young man said it was strange that removing a bench sometimes meant a hundred metres of digging and sometimes meant nothing at all.

“It is not strange,” said Ravindran. “It is the same rule both times. A piece of path stays if anybody is still using it — to get somewhere further, or to get to that spot itself. If neither, it goes. You are not deciding about the bench. You are deciding, one step at a time on the way back, about each piece of gravel.”

---



### 3 The idea in plain English

Ravindran stated trie deletion exactly, and his two stopping conditions are the two flags from yesterday.

**The three easy operations first**, so they are out of the way.

**Insert** walks the word, creating any child that is missing, and sets `is_end = True` on the final node. The only subtlety is that inserting a word twice must not corrupt anything — with a plain `is_end` it is naturally idempotent, but the moment you add a counter it is not, and you must check.

**Search** walks the word and returns `is_end` on the final node. If the walk falls off — a character with no child — the answer is `False`.

**Prefix search** walks the prefix and returns whether the walk survived. It does *not* look at `is_end`. This one line is the entire difference between the two operations, and it is what LeetCode 208 is testing.

All three share the same walk, which is why you write the walk once.

**Now delete, which is the real lesson.**

The naive idea — find the end node and set `is_end = False` — is not wrong, but it leaves rubbish behind. Delete `"careful"` from a trie holding only that word and you have seven dead nodes forming a path to nowhere. Do that a few thousand times and the trie is mostly corpses.

So deletion has two parts: unmark the word, then **walk back up and remove nodes that are no longer needed**.

**A node is no longer needed when both of these are true:**

1. It has **no children** — nothing continues past it.
2. Its `is_end` is **False** — no word ends here.

That is Ravindran's rule, and it is the same rule his two stopping conditions describe from the other side. A fork is "it has children". A bench is "`is_end` is True". If either holds, stop; the node is doing a job.

**The four cases, which is what you should actually say in an interview:**

| CASE                                   | EXAMPLE                        | WHAT HAPPENS                                                |
|----------------------------------------|--------------------------------|-------------------------------------------------------------|
| The word is not there                  | delete "cap" from {car}        | Do nothing. Return False.                                   |
| The word is a <b>prefix</b> of another | delete "car" from {car, card}  | Clear is_end only. <b>Remove no nodes.</b>                  |
| The word <b>extends</b> another        | delete "card" from {car, card} | Remove the d node. Stop at r, because is_end is True there. |
| The word is alone on its branch        | delete "dog" from {dog, car}   | Remove g, o, d. Stop at the root.                           |

Cases two and three are the mirror image of each other and both catch people. Case two is Ravindran’s bench by the gate — remove the destination, keep every stone. Case three is the tap — remove the tail, stop at the bench.

**The direction matters.** You cannot decide from the top down, because whether a node may be removed depends on what is *below* it, and on the way down you have not looked yet. So deletion is naturally recursive: go all the way to the end, then answer the question on the way back up. Each level asks its child “may I remove you?” and the child answers only after it has removed what it could.

**And then wildcards, which change the shape of everything.**

Support . matching any single character. search("c.t") should find "cat" and "cut".

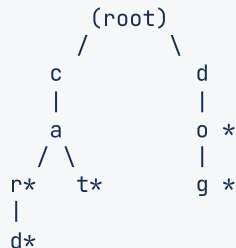
Every operation so far followed one path. At a . you cannot know which child to enter, so **you try them all**. That makes it a depth-first search over the trie rather than a walk, and it is why this operation is recursive while the others are loops.

The cost changes completely. A normal search is  $O(L)$ . A search with  $w$  wildcards is up to  $O(26^w \times L)$  in the worst case — though in practice the trie prunes hard, because most of those 26 branches do not exist. A leading . is the expensive case: it forks 26 ways immediately with nothing narrowing it down.

#### 4 The picture

Delete, all four cases on one trie

Starting trie: {car, card, cat, do, dog}



CASE 1 – delete "cap" (not present)  
walk c → a → p ... no child 'p'. Return False. Change nothing.

CASE 2 – delete "car" (a prefix of "card")  
reach node r. Set is\_end = False.  
May I remove r? It has child 'd'. NO. ← a fork  
Stop. Zero nodes removed.  
Result: "car" no longer found, "card" still found.

CASE 3 – delete "card" ("car" is still a word)  
reach node d. Set is\_end = False.  
May I remove d? No children, not a word. YES. Remove it.  
May I remove r? Now no children, but is\_end is True. NO. ← a bench  
Stop. One node removed.

CASE 4 – delete "dog" (nothing else below d-o-g; "do" is a word)  
reach node g. Set is\_end = False.  
May I remove g? No children, not a word. YES.  
May I remove o? Now no children, but is\_end is True (that is "do"). NO.  
Stop. One node removed.

CASE 4b – delete "do" as well, after "dog" is gone  
reach node o. Set is\_end = False.  
May I remove o? No children, not a word. YES.  
May I remove d? Now no children, not a word. YES.  
Stop at root (never removed).  
Two nodes removed; the whole 'd' branch is gone.

*Notice that the same operation removed zero, one and two nodes depending only on what else was in the trie. The rule never changed.*

## The rule, stated once

```
+-----+
| A node may be removed when BOTH hold:
|
| len(node.children) == 0
| node.is_end is False
|
| Otherwise it is still doing a job:
| children → words pass through it
| is_end → a word finishes at it
+-----+
```

And it must be checked BOTTOM-UP, because "has children" is only correct after the children have been pruned.

## Wildcard search branches

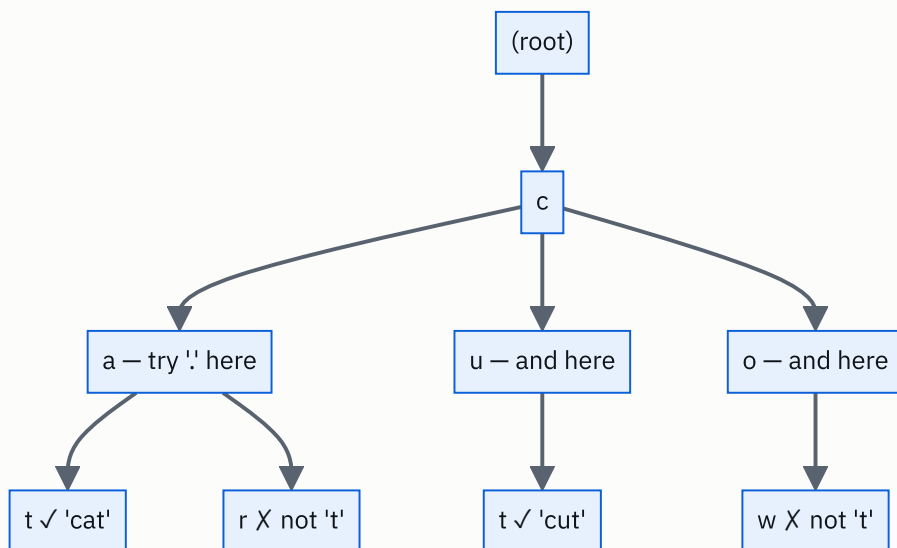


FIGURE 121.1

Notice that the `.` opens every existing child, not all 26 letters. The trie itself does the pruning — if no word starts `cz`, that branch is never explored. This is why the `26^w` worst case almost never happens on real words.

## The cost of a leading wildcard

|                              |                                                                                                                                               |           |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <code>search("cat")</code>   | 1 path, 3 steps.                                                                                                                              | cheap     |
| <code>search("c.t")</code>   | at 'c': 1 child path<br>at '.': every child of 'c' – maybe 8<br>at 't': check one child in each<br>≈ 8 short paths.                           | fine      |
| <code>search("..t")</code>   | at '.': every child of root – up to 26<br>at 't': every child of each of those – up to 26 each<br>≈ 676 paths before the 't' filters anything | expensive |
| <code>search(".....")</code> | seven wildcards, no filtering at all:<br>this visits every node at depth 7.<br>On an English dictionary that is most of the trie.             |           |

*Notice the pattern: wildcards early are expensive, wildcards late are cheap, because a real character earlier in the pattern narrows the search before the fork happens.*

## 5 The code, built step by step

### The node and the shared walk

```
class TrieNode:
 __slots__ = ("children", "is_end")

 def __init__(self) → None:
 self.children: dict[str, "TrieNode"] = {}
 self.is_end: bool = False

 def _walk(self, prefix: str) → TrieNode | None:
 node = self.root
 for character in prefix:
 child = node.children.get(character)
 if child is None:
 return None
 node = child
 return node
```

Every operation except delete and wildcard search is one line on top of this.

### Insert, search, starts\_with

```
def insert(self, word: str) → None:
 node = self.root
 for character in word:
 node = node.children.setdefault(character, TrieNode())
 node.is_end = True

def search(self, word: str) → bool:
 node = self._walk(word)
 return node is not None and node.is_end

def starts_with(self, prefix: str) → bool:
 return self._walk(prefix) is not None
```

That is LeetCode 208, complete. Write it in ninety seconds and spend the rest of the round on the follow-ups, which is where the marks are.

### Delete, recursively

The recursion is the natural shape because the decision is made on the way back up. The return value means “**I have been removed; the caller may consider removing itself**”.

```
def delete(self, word: str) → bool:
 """Remove a word. Returns False if it was not there."""
 if not self.search(word):
 return False # case 1
 self._delete(self.root, word, 0)
 return True

def _delete(self, node: TrieNode, word: str, depth: int) → bool:
 if depth == len(word):
 node.is_end = False # unmark the word
 return not node.children # prunable only if childless

 character = word[depth]
 child = node.children[character] # safe: search() confirmed the path
 if self._delete(child, word, depth + 1):
 del node.children[character]
 return not node.children and not node.is_end

 return False # a descendant said stop
```

Four lines carry the meaning.

`node.is_end = False` is the actual deletion. Everything after it is cleanup.

`return not node.children` at the base case is the `is_end` half of the rule — we have just set `is_end` to `False`, so only the children matter here.

`return not node.children and not node.is_end` is the rule in full, checked *after* the child was removed, which is why the order of those two statements cannot be swapped.

`return False` when the child said no: once anyone below says stop, nothing above may be removed either. The recursion unwinds without touching anything else.

### Delete, iteratively

Worth knowing, because interviewers sometimes ask for it and because it makes the bottom-up direction visible. Record the path down, then walk it backwards.

```
def delete_iterative(self, word: str) → bool:
 node = self.root
 path: list[tuple[TrieNode, str]] = []
 for character in word:
 child = node.children.get(character)
 if child is None:
 return False
 path.append((node, character))
 node = child

 if not node.is_end:
 return False
 node.is_end = False

 for parent, character in reversed(path):
 child = parent.children[character]
 if child.children or child.is_end:
 break
 del parent.children[character]
 return True
```

`if child.children or child.is_end: break` is the fork and the bench, in one line.

### Wildcard search

The first operation that is a search rather than a walk.

```

def search_pattern(self, pattern: str) → bool:
 """ '.' matches any single character. """
 return self._match(self.root, pattern, 0)

def _match(self, node: TrieNode, pattern: str, index: int) → bool:
 if index == len(pattern):
 return node.is_end

 character = pattern[index]
 if character == ".":
 for child in node.children.values():
 if self._match(child, pattern, index + 1):
 return True
 return False

 child = node.children.get(character)
 return child is not None and self._match(child, pattern, index + 1)

```

That is LeetCode 211. Note the two things that make it correct and fast: the base case still checks `is_end` — a pattern must match a whole word, not a prefix — and the loop returns `True` on the first hit rather than exploring the rest.

One optimisation worth mentioning: if the pattern has a fixed length, any word of a different length cannot match. Bucketing words by length before searching removes most of the work when patterns are mostly wildcards.

### Collecting completions, with a limit

The version you would actually ship, because returning everything under `"a"` returns the dictionary.

```

def completions(self, prefix: str, limit: int = 10) → list[str]:
 start = self._walk(prefix)
 if start is None:
 return []
 found: list[str] = []
 stack: list[tuple[TrieNode, str]] = [(start, prefix)]
 while stack and len(found) < limit:
 node, so_far = stack.pop()
 if node.is_end:
 found.append(so_far)
 for character in sorted(node.children, reverse=True):
 stack.append((node.children[character], so_far + character))
 return found[:limit]

```

Explicit stack rather than recursion, so long keys cannot blow the recursion limit, and `reverse=True` on the push so the pops come out alphabetically.



## The complete solution

```
"""Day 121 - trie operations: insert, search, prefix, delete, wildcard."""

from __future__ import annotations

class TrieNode:
 __slots__ = ("children", "is_end")

 def __init__(self) → None:
 self.children: dict[str, TrieNode] = {}
 self.is_end: bool = False

class Trie:
 def __init__(self) → None:
 self.root = TrieNode()
 self.size = 0

 # ----- the shared walk -----

 def _walk(self, prefix: str) → TrieNode | None:
 node = self.root
 for character in prefix:
 child = node.children.get(character)
 if child is None:
 return None
 node = child
 return node

 # ----- insert -----

 def insert(self, word: str) → None:
 """O(L). Inserting an existing word is a no-op."""
 node = self.root
 for character in word:
 node = node.children.setdefault(character, TrieNode())
 if not node.is_end:
 node.is_end = True
 self.size += 1

 # ----- search -----

 def search(self, word: str) → bool:
 """Is this exact word stored? O(L)."""
 node = self._walk(word)
 return node is not None and node.is_end

 def starts_with(self, prefix: str) → bool:
 """Does any word begin with this prefix? O(L). Note: no is_end check."""
 return self._walk(prefix) is not None

 # ----- delete -----

 def delete(self, word: str) → bool:
 """Remove a word and prune dead nodes. Returns False if absent."""
 if not self.search(word):
 return False
 self._delete(self.root, word, 0)
 self.size -= 1
 return True
```

```

def _delete(self, node: TrieNode, word: str, depth: int) → bool:
 """Returns True if this node was removed and the parent may prune too."""
 if depth == len(word):
 node.is_end = False
 return not node.children

 character = word[depth]
 child = node.children[character]
 if self._delete(child, word, depth + 1):
 del node.children[character]
 return not node.children and not node.is_end
 return False

----- wildcard -----

def search_pattern(self, pattern: str) → bool:
 """'.' matches any one character. O(26^wildcards × L) worst case."""
 return self._match(self.root, pattern, 0)

def _match(self, node: TrieNode, pattern: str, index: int) → bool:
 if index == len(pattern):
 return node.is_end
 character = pattern[index]
 if character == ".":
 return any(
 self._match(child, pattern, index + 1)
 for child in node.children.values()
)
 child = node.children.get(character)
 return child is not None and self._match(child, pattern, index + 1)

----- completions -----

def completions(self, prefix: str, limit: int = 10) → list[str]:
 """Up to `limit` words with this prefix, alphabetically. O(L + output)."""
 start = self._walk(prefix)
 if start is None:
 return []
 found: list[str] = []
 stack: list[tuple[TrieNode, str]] = [(start, prefix)]
 while stack and len(found) < limit:
 node, so_far = stack.pop()
 if node.is_end:
 found.append(so_far)
 for character in sorted(node.children, reverse=True):
 stack.append((node.children[character], so_far + character))
 return found[:limit]

----- inspecting -----

def node_count(self) → int:
 total, stack = 0, [self.root]
 while stack:
 node = stack.pop()
 total += 1
 stack.extend(node.children.values())
 return total

def __len__(self) → int:
 return self.size

def __contains__(self, word: str) → bool:

```

```

 return self.search(word)

if __name__ == "__main__":
 trie = Trie()
 for word in ["car", "card", "cat", "do", "dog"]:
 trie.insert(word)
 print(len(trie), trie.node_count()) # 5 9

 # search vs starts_with
 print(trie.search("car"), trie.search("ca")) # True False
 print(trie.starts_with("ca"), trie.starts_with("z")) # True False

 # case 1: absent
 print(trie.delete("cap"), trie.node_count()) # False 9

 # case 2: a prefix of another word – nothing is removed
 print(trie.delete("car"), trie.node_count()) # True 9
 print(trie.search("car"), trie.search("card")) # False True

 # case 3: extends another word – one node goes
 trie.insert("car")
 print(trie.delete("card"), trie.node_count()) # True 8

 # case 4: alone on its branch, then the branch collapses
 print(trie.delete("dog"), trie.node_count()) # True 7
 print(trie.delete("do"), trie.node_count()) # True 5
 print(trie.starts_with("d")) # False

 # wildcards
 for word in ["cat", "cut", "cot", "cast"]:
 trie.insert(word)
 print(trie.search_pattern("c.t")) # True
 print(trie.search_pattern("c..t")) # True (cast)
 print(trie.search_pattern("c.r")) # True (car)
 print(trie.search_pattern("z.t")) # False
 print(trie.search_pattern("ca")) # False (a prefix, not a word)

 # completions
 print(trie.completions("c")) # ['car', 'cast', 'cat', 'cot', 'cut']
 print(trie.completions("c", limit=2)) # ['car', 'cast']

```

Run it and watch `node_count()` in the delete section: 9, 9, 8, 7, 5. Those five numbers are the four cases, and if your implementation produces different ones, the bug is in the pruning rule.

## 6 What it costs

### Time

| OPERATION                           | COST                                    | NOTE                                                |
|-------------------------------------|-----------------------------------------|-----------------------------------------------------|
| <code>insert(word)</code>           | $O(L)$                                  |                                                     |
| <code>search(word)</code>           | $O(L)$                                  |                                                     |
| <code>starts_with(prefix)</code>    | $O(L)$                                  |                                                     |
| <code>delete(word)</code>           | $O(L)$                                  | search once, then unwind — two passes, still $O(L)$ |
| <code>search_pattern</code>         | $O(L)$ best, $O(26^w \times L)$ worst   | $w$ = number of wildcards                           |
| <code>completions(prefix, k)</code> | $O(L + k \times \text{average length})$ | with the limit; unbounded without                   |

**Delete is  $O(L)$ , not  $O(n)$ .** People assume it must be expensive because it changes the structure. It is not: you touch each node on one path at most twice, and the pruning is decided locally at each step.

### The wildcard arithmetic, properly

The worst case is stated as  $26^w$  and almost never happens. Here is why, on a real English trie of 100,000 words:

| pattern                                                                | branches actually explored | why                                              |
|------------------------------------------------------------------------|----------------------------|--------------------------------------------------|
| -----                                                                  | -----                      | -----                                            |
| "cat"                                                                  | 1                          | no forks                                         |
| "c.t"                                                                  | ~8                         | 'c' has ~8 real children                         |
| ".at"                                                                  | ~20                        | 20 letters precede "at"                          |
| "..t"                                                                  | ~350                       | 2 forks, but most pairs do not exist as prefixes |
| "...."                                                                 | ~5,000                     | 4 forks, still pruned hard                       |
| "....." (10 dots)                                                      | most of the trie           | nothing prunes anything                          |
| Theoretical worst for "..t": $26 \times 26 = 676$                      |                            |                                                  |
| Actual: ~350                                                           |                            |                                                  |
| Because "qz", "xj", "vk" and hundreds of others are not real prefixes. |                            |                                                  |

**The trie prunes for you.** That sentence is the right answer to “isn’t the wildcard version exponential?” It is exponential in the number of wildcards, but the base is not 26 — it is the average branching factor of your actual data, which for English is closer to 3 near the bottom.

## Where the wildcards sit

```
"c..." 1 real character first: $\sim 8 \times 8 \times 8 \approx 500$ paths
"...t" 3 wildcards first: $26 \times 26 \times 26 \approx 5,000+$ paths, filtered only at the end

Same number of wildcards. Ten times the work.
A real character early narrows everything after it.
```

If patterns commonly start with wildcards, bucket the words by length first — a five-character pattern can never match a four-character word, and that check is free.

## Memory during delete

Recursive delete uses  $O(L)$  stack. For words that is nothing. For a trie over file paths or DNA sequences it is not:

```
RecursionError: maximum recursion depth exceeded
```

Python's limit is 1000. If keys can be longer than that, use the iterative version.

## The two-pass question

`delete` as written calls `search` first, then recurses. That is two passes over the word. You can do it in one, but the code becomes noticeably harder to follow because the recursion has to report both “was it there?” and “may I be pruned?”.  $O(L)$  either way, and  $L$  is small.

“I would do the two-pass version. It is the same complexity, the search-first check makes the recursion assume the path exists, and it removes an entire class of bug.”

That is the right thing to say. Choosing the clearer of two same-complexity options, out loud, is a signal.

## 7 The traps

**Trap 1: deleting a word that is a prefix of another, and pruning anyway.**

```
>>> trie = Trie()
>>> trie.insert("car"); trie.insert("card")
>>> trie.delete("car")
True
>>> trie.search("card") # if your pruning is wrong:
False
```

You destroyed a word you were not asked to touch. No error, no exception — a silently smaller dictionary. This is the bug the `is_end` half of the rule exists to prevent, and it is the one interviewers probe for.

### Trap 2: deleting a word that is not there.

```
>>> trie = Trie()
>>> trie.insert("car")
>>> trie._delete(trie.root, "cap", 0)
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
KeyError: 'p'
```

The recursion indexes `node.children[character]` assuming the path exists. Guard with `search` first, or check `.get()` at every level.

### Trap 3: checking prunability before removing the child. Order matters:

```
if self._delete(child, word, depth + 1):
 del node.children[character] # first
 return not node.children and not node.is_end # then check
```

Check before the `del` and `node.children` still contains the child you are about to remove, so nothing is ever pruned. The trie stays correct and grows forever — the worst kind of bug, because every test passes.

**Trap 4: forgetting `is_end` in the base case.** If the base case returns `True` unconditionally, deleting `"car"` from `{car, card}` starts pruning from the `r` node upward and takes `"card"` with it. See trap 1.

### Trap 5: the wildcard base case checking the wrong thing.

```
>>> trie.insert("cat")
>>> trie.search_pattern("ca")
True # WRONG - "ca" is not a word
```

The base case must be `return node.is_end`, not `return True`. A pattern matches a *word*, not a prefix.

### Trap 6: the wildcard loop returning too early or too late.

```
if character == ".":
 for child in node.children.values():
 return self._match(child, pattern, index + 1) # WRONG
```

That returns after trying only the first child. It must be `if ...: return True` inside the loop, then `return False` after it — try them all, succeed on any.

**Trap 7: mutating the children dictionary while iterating it.** If a delete runs inside a traversal:

```
RuntimeError: dictionary changed size during iteration
```

Collect what you will remove, then remove it.

**Trap 8: unbounded completions.**

```
>>> len(trie.completions("")) # no limit
100000
```

An empty prefix matches everything. Always take a limit, and stop when it is reached rather than truncating at the end.

**Trap 9: recursion depth on long keys.** See §6. Use the iterative delete when key length is unbounded.

**Trap 10: forgetting that delete does not shrink memory in Python straight away.** Removing nodes drops references; the memory is reclaimed by the garbage collector, not immediately, and the process may not return it to the operating system at all. If you are relying on delete to control memory, measure it.

---

## 8 In the interview

How it gets asked

- “Implement a trie with insert, search and startsWith.” — LeetCode 208. The opener, always.
- “Add delete.” — the real question. Almost never on the LeetCode problem, almost always in an interview.
- “Design a data structure that supports adding words and searching with `.` as a wildcard.” — LeetCode 211, and the standard follow-up.
- “Autocomplete: return the top ten completions for a prefix.”
- “Given a board of letters and a word list, find every word on the board.” — LeetCode 212, where the trie turns an infeasible search into a feasible one.
- “Can a sentence be split into dictionary words?” — word break, with the trie replacing the set lookup.

## The first ninety seconds

For the 208-plus-delete version:

“The three basic operations share one walk, so I will write the walk once. Insert walks and creates missing children, then marks the last node. Search walks and returns `is_end`. `startsWith` walks and returns whether the walk survived — same code, minus the `is_end` check. All three are  $O(L)$  with no dependence on how many words are stored.

Delete is the one worth talking about. The naive version sets `is_end = False` and leaves dead nodes behind, so I also want to prune. The rule is: **a node may be removed only if it has no children and `is_end` is False** — otherwise it is still needed, either because words pass through it or because a word ends there.

The important consequence is that this must be checked bottom-up, since ‘has no children’ is only true after the children have been pruned. So delete is naturally recursive: recurse to the end, unmark, and answer the prune question on the way back.

There are four cases and I would test all four: the word is absent; the word is a prefix of another, where I remove *no* nodes at all; the word extends another, where I remove the tail and stop at the shorter word; and the word is alone, where the whole branch collapses. The second case is the one that breaks naive implementations — deleting `car` must not remove `card`.

Shall I write it?”

That names the rule, the direction, and the four cases before writing a line.

## The follow-ups

“Why must delete be bottom-up?”

“Because the decision depends on what is below. Coming down, I have not looked yet — a node might have a child that is about to be pruned, or one that must stay. Only after the recursive call returns do I know whether the child was removed, and only then can I evaluate ‘do I have children’. That is why the recursive version returns a boolean meaning ‘I was removed, you may consider removing yourself’, and why the iterative version has to record the path on the way down and walk it backwards.”



### “What does delete cost?”

“ $O(L)$ . That surprises people because it changes the structure, but I only ever touch nodes on one path, at most twice each — once going down, once coming back. There is no  $n$  in it. The version I would write does two passes, searching first and then deleting, which is still  $O(L)$  and lets the recursion assume the path exists. One-pass is possible but the recursion then has to report two different things and it gets harder to read for no complexity gain.”

### “Now add a wildcard.”

“That changes the shape from a walk to a search. At a real character I follow one child; at a `.` I have to try every child, so it becomes a depth-first search and the code becomes recursive.

Worst case is  $O(26^w \times L)$  where  $w$  is the number of wildcards. But in practice the trie prunes very hard, because most letter pairs are not real prefixes — on an English dictionary, `..t` explores about 350 branches, not 676. Where the wildcards *sit* matters more than how many there are: a real character early narrows everything after it, so `c...` is roughly ten times cheaper than `...t`.

If patterns were mostly wildcards, I would bucket words by length first, since a five-character pattern can never match a four-character word, and that check is free.”

### “How would you return the top ten completions, not all of them?”

“Returning everything is wrong at the start — an empty prefix returns the whole dictionary. Two options. If ‘top’ means alphabetical, I do a depth-first walk with a limit and stop as soon as I have ten, which is  $O(L + \text{output})$  and never touches the rest of the subtree. If ‘top’ means most popular, the walk is the wrong tool — I would precompute the best ten completions at each node at build time, so a lookup is  $O(L)$  and the answer is already sitting there. That costs memory proportional to nodes times ten, so I would only store it above some depth. For a search box the read path has to be fast and the build is a nightly job, so I would precompute.”

## The model answer

*“Design a data structure that supports `addWord(word)` and `search(word)`, where `search` may contain `.` matching any single character.”*

“This is a trie, and the reason is the `.`. If `search` were exact, a hash set would be better — smaller, simpler,  $O(1)$  average. But a set cannot do wildcards at all: with `c.t` I would have to test all 26 substitutions against the set, which is  $26^w$  lookups with no pruning whatsoever. The trie does the same search but prunes at every step, because it only explores branches that actually exist.

**The structure.** A node with a children map and an `is_end` flag. `addWord` is the plain insert —  $O(L)$ .

**Search.** A depth-first search rather than a walk. At a real character I take that child if it exists and fail if it does not. At a `.` I recurse into every child and succeed if any of them does. The base case, when I have consumed the whole pattern, returns `node.is_end` — not `True`, because the pattern must match a complete word rather than a prefix. That is the bug I would most expect to make.

**Complexity.** `addWord` is  $O(L)$ . Search is  $O(L)$  with no wildcards and  $O(26^w \times L)$  in the worst case. But the practical number is much lower — the base is the real branching factor of the data, around three near the bottom of an English trie, not 26. And the position matters more than the count: a leading `.` forks immediately with nothing to narrow it, so `...t` costs about ten times `c...`.

**What I would add if this were real.** Bucketing by word length, since the pattern length is fixed and that eliminates most candidates for free. And if patterns were dominated by leading wildcards, I would consider indexing the reversed words in a second trie as well, so a pattern ending in real characters could be searched from the other end — twice the memory, but it turns the expensive case into the cheap one.

**The trade-off I would state.** Memory. The trie holds one node per distinct prefix at a couple of hundred bytes each in Python, so for a large word list it is several times the size of a set. I am paying that specifically to get wildcard pruning. If the requirement lost the `.`, I would delete this whole class and use a set.

**What I would ask.** How many words and how long? And do patterns tend to have wildcards at the start or the end? Both answers change what I would optimise, and the second one changes it a lot.”

That answer justifies the structure against the obvious alternative, names the base-case bug before making it, gives honest complexity with the practical correction, proposes two concrete optimisations, and asks the question that would change the design.

## 9 Recall card

**The shared walk** — write it once; three operations sit on top of it.

```
insert: node = node.children.setdefault(c, TrieNode()); ... node.is_end = True
search: node is not None and node.is_end
starts_with: node is not None
```

All three are  $O(L)$ . So is delete. No  $n$  anywhere.

**Delete has two parts:** unmark the word, then prune bottom-up.

**The pruning rule — a node may be removed only if:**

```
len(node.children) == 0 AND node.is_end is False
```

Children  $\Rightarrow$  words pass through it. `is_end`  $\Rightarrow$  a word ends at it. Either one means keep.

**It must be bottom-up**, because “has no children” is only correct after the children are pruned. Hence recursion, returning “I was removed, you may consider removing yourself”.

**The four cases:** | Delete | From | Result | |—|—|—| | absent word | — | do nothing  
| | "car" | {car, card} | clear `is_end`, **remove nothing** | | "card" | {car, card}  
| remove `d`, stop at `r` | | "dog" | {dog} | whole branch collapses |

**Order matters:** `del node.children[c]` **then** check prunability. Reversed, nothing is ever pruned and every test still passes.

**Wildcard search is a DFS, not a walk.** At `.`, try every child; return `True` on the first success. **Base case is** `node.is_end`, **never** `True`.

**Cost:**  $O(26^w \times L)$  worst case, but the trie prunes — the real base is the data’s branching factor ( $\sim 3$  for English). **Position beats count:** `c...`  $\approx 500$  paths, `...t`  $\approx 5,000$ . Bucket by length to help.

**Completions must take a limit.** An empty prefix matches everything. Stop early rather than truncating. For top-*popular* rather than top-*alphabetical*, precompute the best ten at each node at build time.

Use the iterative delete when keys can exceed ~1000 characters — recursion depth.

---

**Next:** [Day 122 — Autocomplete and word dictionaries](#)

**Previous:** [Day 120 — The trie: a tree of characters](#)

## 1 What this is, and why they ask it

Yesterday you saw why two-phase commit is unpopular: it blocks, it holds locks across machines, its availability falls as you add participants, and it couples services together in exactly the way splitting them up was meant to prevent.

So what do people build instead?

A **saga**. The idea is simple enough to state in one sentence:

**Break the distributed transaction into a sequence of local transactions, and for each one, define a compensating action that undoes it.**

No coordinator holding everyone hostage. No locks across services. Each step commits immediately and locally, in its own database, with ordinary ACID guarantees. If a later step fails, you do not roll back — you cannot, the earlier steps are already committed and visible — you run the compensations backwards.

The name comes from a 1987 paper about long-running database transactions, and it has been the default answer for cross-service consistency in microservice architectures for about a decade. Every interviewer who asks “order service and payment service, how do you keep them consistent?” is listening for this word.

But saying “saga” is the easy half. The valuable half is knowing precisely what you gave up, because it is more than people expect:

- **Atomicity is gone.** There are windows where half the work is done and visible.
- **Isolation is completely gone.** Another transaction can read your intermediate state and act on it.
- **Compensation is not rollback.** A refund is not the same as the payment never happening.
- **Some things cannot be compensated at all.** You cannot un-send an email.

By the end you will be able to design a saga, choose between the two ways of running one, handle the failure of a compensation itself, name the isolation problems and their countermeasures, and — the thing that separates a strong answer — say when a saga is the wrong choice.

## 2 The story

Nadia organises weddings. Not the flowers — the logistics. Six hundred guests, three days, four venues.

Every wedding she books involves the same five people: the hall, the caterer, the band, the tent hire and the photographer. All five must be booked for the same weekend, and any one of them can turn out to be unavailable.

For her first two years she tried to do it the sensible-sounding way. She would ring all five, get provisional holds from everyone, and only confirm once all five had said yes. It never worked. The hall would hold a date for forty-eight hours and no longer. The band would not hold at all — first to pay, first served. The caterer wanted a decision the same day. Trying to freeze all five at once meant she usually lost the band while waiting for the tent people to call back.

So she does it differently now, and she explained it to a client last month who was worried about it.

“I book them one at a time, in order, and each booking is real. Not a hold — real, paid, confirmed. Hall first, because it is the hardest to get and everything else depends on the date. Then caterer, then tent, then band, then photographer.

“And before I start, I write down what it costs to *undo* each one. That is the part nobody thinks about. The hall takes twenty percent if I cancel more than a month out. The caterer takes nothing before two weeks. The tent people take nothing. The band keeps the deposit, full stop. The photographer is fine.

“So if I get to the band and the band has gone — I do not panic, and I do not pretend the earlier bookings did not happen, because they did. Money has moved. I go backwards. Cancel the tent, cancel the caterer, cancel the hall. Each one costs what it costs and I know the number before I start.”

The client asked the obvious thing. Why not book the band first, since it is the one that cannot be undone?

Nadia said that was the right instinct and she had learned it the expensive way. “Now I put the ones I cannot undo *last*. If something is going to fail, I want it to fail while everything before it is still cancellable. The moment I pay the band, I am committed, so I do not pay the band until I am certain of everything else.”

The client asked what happens if a cancellation itself fails — if the caterer will not pick up the phone.

“Then I keep ringing,” said Nadia. “Every day. It is on my list and it does not come off my list until it is done. That is the only answer there is. I cannot leave it half-cancelled and forget about it, because in three months somebody will deliver four hundred meals to an empty hall.”

And then she said the thing that the client actually needed to hear.

“There is one more thing and I will not hide it from you. Between me booking the hall and me cancelling it, that hall shows as taken. If another family rings the hall on that Tuesday, they will be told no. They will go somewhere else. And by Thursday it is free again and they never knew. I cannot prevent that. Nobody can. That is simply the cost of booking things one at a time instead of all at once, and anyone who tells you otherwise is selling you something that does not exist.”

### 3 The idea in plain English

Nadia runs sagas, and everything important about the pattern is in what she said.

**The structure.** A saga is a sequence of local transactions  $T_1, T_2, \dots, T_n$ , each in its own service and its own database, each committing immediately. For each  $T_i$  there is a **compensating transaction**  $C_i$  that semantically undoes it.

If everything succeeds, you ran  $T_1 \dots T_n$  and you are done. If  $T_4$  fails, you run  $C_3, C_2, C_1$  — backwards, in reverse order — and end up in a state that is *acceptable*, though not identical to never having started.

**Compensation is not rollback, and this distinction is the heart of the lesson.**

A rollback erases history. The database discards uncommitted changes and nobody ever saw them. A compensation is a **new, forward transaction** that produces an off-setting effect. The original is still in the record.

```
rollback: the payment never happened
compensation: a payment happened, then a refund happened
 - two rows, two ledger entries, a bank fee, and
 possibly a confused customer
```

Nadia’s twenty percent cancellation charge is exactly this. The booking was real. Undoing it costs something and leaves a trace. **Say this in an interview;** it is the sentence that shows you understand sagas rather than having heard of them.

**Order matters, and the rule is counter-intuitive.** Put the steps that are hardest to undo **last**. Nadia's band. If a saga is going to fail, you want it to fail while everything before it is still cheaply reversible. The general rule: **irreversible steps go at the end**, and anything truly irreversible — an email sent, a physical shipment dispatched, money paid to an external party — should be the final step or sit outside the saga entirely.

**Two ways to run one, and interviewers ask you to compare them.**

**Choreography.** No central controller. Each service listens for events and reacts. Order service publishes `OrderCreated` ; payment service hears it, charges, publishes `PaymentTaken` ; inventory hears that, reserves stock, publishes `StockReserved` . Compensations work the same way in reverse — a `PaymentFailed` event triggers whoever needs to undo.

- Simple for three steps. No extra component. Services stay decoupled.
- Becomes unreadable past four or five steps. **The workflow exists nowhere in the code** — you cannot open a file and read what happens. Debugging means correlating logs across services. Cyclic dependencies creep in.

**Orchestration.** One component — the orchestrator — holds the workflow and tells each service what to do, one step at a time, recording progress durably as it goes.

- The whole workflow is in one place, readable and testable. Failure handling is explicit. You can query “where is order 4471?” and get an answer.
- One more service to run, and it must be reliable and durable — if it forgets a half-finished saga, the system is stuck in a partial state forever. It also risks becoming a place where business logic quietly accumulates.

**The practical rule:** choreography up to about three steps, orchestration beyond that. Most real order flows have six or seven steps and end up orchestrated. Say the rule and the reason.

**What you gave up, precisely.**

Atomicity is replaced by *eventual* atomicity: eventually every step either completed or was compensated. In between, the system is visibly half-done.

**Isolation is gone entirely, and this is the underrated problem.** In a real transaction, nobody sees your intermediate state. In a saga, everybody does. Three specific hazards have names:

- **Lost update.** Saga A writes a value; saga B overwrites it; A compensates and clobbers B's write.



- **Dirty read.** Saga B reads data that saga A wrote and will later compensate away. B acted on something that ends up never having been true.
- **Fuzzy read.** Saga A reads the same row twice and gets different answers because B changed it in between.

Nadia's other family ringing the hall is a dirty read with a real-world consequence — they went elsewhere based on a booking that was cancelled two days later. She was right that nobody can prevent it. What you *can* do is limit the damage, and there are four standard countermeasures:

- **Semantic lock.** Mark the record as pending — `status = PENDING_PAYMENT` — so other readers know it is in flight and can decide for themselves whether to wait, refuse or proceed.
- **Commutative updates.** Design operations so order does not matter. `balance = balance - 50` composes safely; `balance = 450` does not. This removes lost updates outright and is the cheapest fix available.
- **Pessimistic view.** Reorder the steps so the risky state is short-lived, or so the reversible-but-visible effect happens as late as possible.
- **Re-read value.** Before writing, re-read and check nothing changed since you read it — optimistic concurrency control, one version column.

**Three rules that make sagas actually work in production**, and they are the ones people forget:

1. **Every step must be idempotent.** Messages get redelivered. A retried “charge the card” must not charge twice. Use a client-supplied key per step and store which keys you have processed.
2. **Compensations must be idempotent too, and must never fail permanently.** They retry forever, with backoff, and end in a dead-letter queue with an alert if they genuinely cannot proceed. A stuck compensation is a data-corruption incident waiting to happen.
3. **Every reservation must expire on its own.** This is the safety net that makes the rest tolerable. If a compensation is lost entirely, a stock reservation that self-expires after fifteen minutes limits the damage to fifteen minutes. Without it, one lost message leaks inventory permanently.

**And the piece that ties it to the database: the outbox pattern.** Every saga step does two things — change local data and publish an event. Those two must be atomic, or you get an order with no event, or an event for an order that was rolled back. You cannot put a database write and a message publish in one transaction. The fix: write the

event into an **outbox table in the same local transaction**, and have a separate process read that table and publish. Two systems become one database, and the problem disappears. It is the single most useful trick in this whole area, and it is a strong thing to mention unprompted.

---

4 The picture

The happy path and the failure path

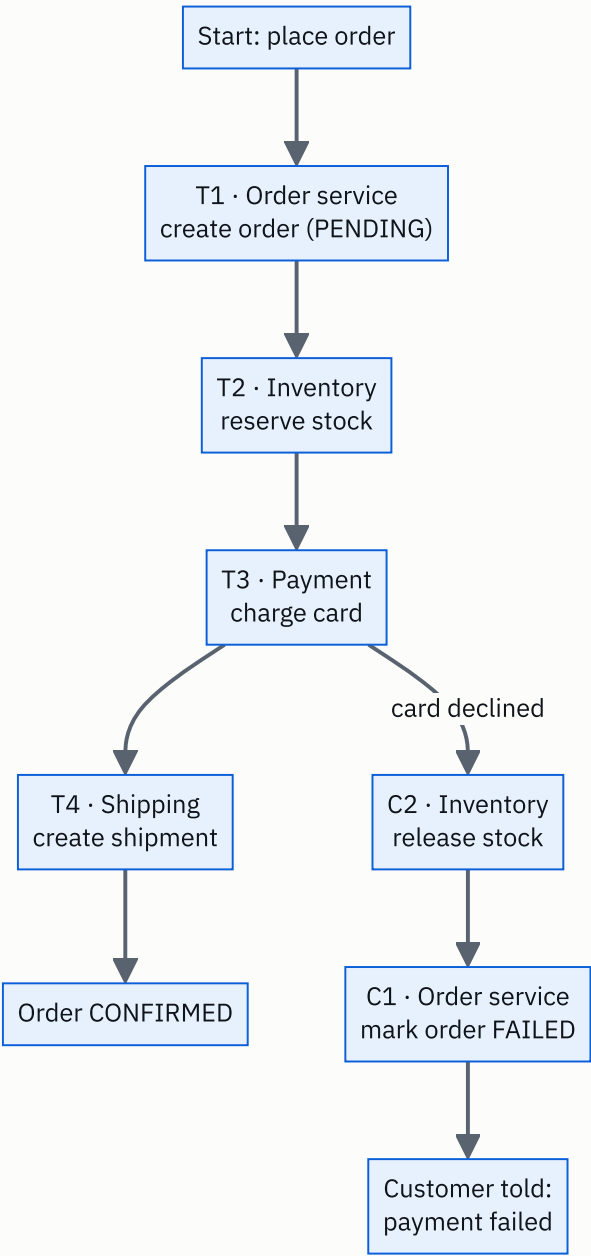


FIGURE 121.2

Notice there is no `C3` on the failure path — the step that failed did not commit, so there is nothing to undo. Compensation starts at the step before the one that failed.

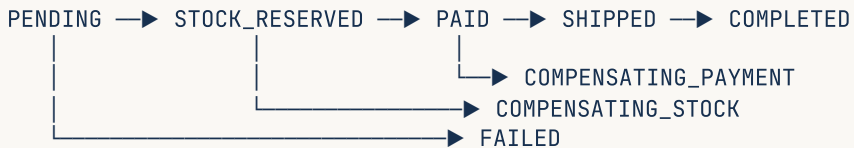
Rollback versus compensation

| ONE DATABASE (2PC or local) | A SAGA                             |
|-----------------------------|------------------------------------|
| -----                       | -----                              |
| T1 begin                    | T1 commits. VISIBLE IMMEDIATELY.   |
| T2                          | T2 commits. VISIBLE.               |
| T3 fails                    | T3 fails.                          |
| ROLLBACK                    | C2 runs – a NEW transaction        |
|                             | C1 runs – a NEW transaction        |
| Nobody ever saw T1 or T2.   | History shows: T1, T2, C2, C1.     |
| History is clean.           | Four ledger entries. A refund fee. |
|                             | Anyone reading in between saw      |
|                             | a state that no longer exists.     |

Choreography versus orchestration

| CHOREOGRAPHY                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | ORCHESTRATION                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Order —OrderCreated—&gt; Payment</p> <p>                                 </p> <p>                                PaymentTaken</p> <p>                                 </p> <p>                                ▼</p> <p>                                Inventory</p> <p>                                 </p> <p>                                StockReserved</p> <p>                                 </p> <p>                                ▼</p> <p>                                Shipping</p> | <div><div>Orchestrator</div><div>↓ ↓ ↓ ↓</div><div>Order Pay Inv Ship</div></div> <p>Workflow lives HERE,<br/>in one readable file.<br/>State is durable.<br/>You can ask: "where is<br/>order 4471?"</p> |
| Workflow lives NOWHERE.<br>To read it, you read four services<br>and correlate their logs.                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                           |
| Fine at 3 steps. Unreadable at 7.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | One more service to run.                                                                                                                                                                                  |

## The state machine an orchestrator actually stores



Every transition is written to durable storage BEFORE the next call is made. If the orchestrator dies, it restarts, reads the state, and continues from exactly there.

The reason: the orchestrator's memory is not a safe place to keep the fact that a payment succeeded.

*Notice that this is a durable state machine, not a function call. That is the whole difference between an orchestrator and a piece of application code that happens to call four services in a row.*

## The outbox pattern

### WRONG

```

BEGIN
 INSERT INTO orders ...
COMMIT
publish("OrderCreated")
```

Crash between the two:  
an order exists, no event.  
The saga never starts.

Or publish first, then crash:  
an event for an order that  
does not exist.

### RIGHT

```

BEGIN
 INSERT INTO orders ...
 INSERT INTO outbox
 (event, payload)
COMMIT
```

A separate poller reads outbox,  
publishes, marks it sent.

Two systems become one database.  
At-least-once delivery – so every  
consumer must be idempotent anyway.

## 5 How it actually works

### A step, and its compensation

```
from dataclasses import dataclass
from typing import Callable

@dataclass(frozen=True)
class Step:
 name: str
 action: Callable[[dict], dict] # do the work, return data for later steps
 compensate: Callable[[dict], None] # undo it – must be idempotent
```

Defining both together is the discipline. If you cannot write the compensation, you have found something that does not belong in a saga, and that is worth discovering at design time rather than in production.

### The orchestrator

```
class SagaOrchestrator:
 def __init__(self, steps: list[Step], store: StateStore) → None:
 self.steps = steps
 self.store = store # durable, not memory

 def run(self, saga_id: str, context: dict) → bool:
 completed: list[int] = []
 for index, step in enumerate(self.steps):
 self.store.record(saga_id, step.name, "STARTED")
 try:
 context = step.action(context)
 self.store.record(saga_id, step.name, "DONE")
 completed.append(index)
 except StepFailed:
 self.store.record(saga_id, step.name, "FAILED")
 self._compensate(saga_id, completed, context)
 return False
 self.store.record(saga_id, "saga", "COMPLETED")
 return True
```

The `store.record` calls are not logging. They are the durable state that lets a restarted orchestrator resume, and they must be written **before** the call they describe, so that a crash mid-call is recoverable.

```

def _compensate(self, saga_id: str, completed: List[int], context: dict) → None:
 for index in reversed(completed):
 step = self.steps[index]
 self._compensate_with_retries(saga_id, step, context)

def _compensate_with_retries(self, saga_id: str, step: Step, context: dict) → None:
 delay = 1.0
 for attempt in range(12):
 # ~1 hour with backoff
 try:
 step.compensate(context)
 self.store.record(saga_id, step.name, "COMPENSATED")
 return
 except Exception:
 sleep(min(delay, 300))
 delay *= 2
 self.store.record(saga_id, step.name, "COMPENSATION_FAILED")
 alert_a_human(saga_id, step.name)
 # this is a real incident

```

**A compensation that fails permanently is not a retry problem, it is an incident.** Alert on it, page someone, and make sure the record survives. Silently swallowing it is how systems end up with money in the wrong place.

#### Idempotency, which every step needs

```

def charge_card(context: dict) → dict:
 key = f"saga-{context['saga_id']}-payment"

 existing = payments.find_by_idempotency_key(key)
 if existing:
 return {"payment_id": existing.id}
 # already done; do not charge again

 charge = gateway.charge(
 amount=context["total"],
 card=context["card_token"],
 idempotency_key=key,
)
 # the gateway checks too
 payments.save(charge, idempotency_key=key)
 return {"payment_id": charge.id}

```

Two layers of protection, and both are wanted. Yours stops the second call reaching the gateway; the gateway's stops a duplicate that slipped past you. Every serious payment API — Stripe among them — accepts an idempotency key for exactly this reason.

#### A compensation, written properly

```

def refund_payment(context: dict) → None:
 payment_id = context.get("payment_id")
 if payment_id is None:
 return
 # never charged; nothing to undo

 if payments.already_refunded(payment_id):
 return
 # idempotent

 gateway.refund(payment_id, idempotency_key=f"refund-{payment_id}")
 payments.mark_refunded(payment_id)

```

Three properties every compensation needs: safe if the original never ran, safe if run twice, and it never raises for a reason a retry cannot fix.

#### The expiry job, which is the real safety net

```
def expire_stale_reservations() → None:
 """Runs every minute. The thing that makes lost compensations survivable."""
 cutoff = now() - timedelta(minutes=15)
 for reservation in inventory.find_reserved_before(cutoff):
 inventory.release(reservation.id)
 log.warning("expired stale reservation %s", reservation.id)
```

If every compensation in the system were lost tomorrow, this job would still return the stock. **Design so that no single lost message causes permanent damage.** That is the sentence to say.

#### The outbox, in SQL

```
BEGIN;
 INSERT INTO orders (id, customer_id, total, status)
 VALUES ('4471', '88', 2499.00, 'PENDING');

 INSERT INTO outbox (id, topic, payload, created_at)
 VALUES (gen_random_uuid(), 'OrderCreated',
 '{"order_id":"4471","total":2499.00}', now());
COMMIT;
```

```
def publish_outbox() → None:
 for event in outbox.fetch_unsent(limit=100):
 broker.publish(event.topic, event.payload) # may duplicate – that is fine
 outbox.mark_sent(event.id)
```

If the process dies between `publish` and `mark_sent`, the event goes out twice. That is at-least-once delivery, and it is why every consumer must be idempotent — which they already are, from the rule above.



## Choreography, for comparison

```
@subscribe("OrderCreated")
def on_order_created(event: dict) → None:
 try:
 inventory.reserve(event["order_id"], event["items"])
 publish("StockReserved", {"order_id": event["order_id"]})
 except OutOfStock:
 publish("StockReservationFailed", {"order_id": event["order_id"]})

@subscribe("StockReservationFailed")
def on_stock_failed(event: dict) → None:
 orders.mark_failed(event["order_id"])
```

Short and clean for two steps. Now imagine seven services and eleven event types, with compensations. The question “what happens when payment fails?” has no single place to look, and that is the honest argument for orchestration.

## 6 The numbers

### Latency

```
Local transaction (all data in one DB): ~5 ms

Saga, 4 steps, orchestrated, synchronous calls:
orchestrator → order service 10 ms (network + local txn)
orchestrator → inventory 15 ms
orchestrator → payment gateway 250 ms ← an EXTERNAL call
orchestrator → shipping 20 ms
state writes, 8 of them 16 ms

total ~310 ms

Same saga, choreographed via a message broker:
4 hops × (publish + poll + process) ~50-200 ms per hop
total ~400 ms - 2 s
```

Two things fall out. **The external payment call dominates everything** — the saga machinery is noise next to it. And choreography is slower because every step goes through a broker; that is the price of decoupling.

Compare with 2PC on the same work: 5-18 ms but with locks held on inventory rows the whole time. **The saga is twenty times slower end to end and holds no cross-service locks at all**, which is why it wins on hot rows.

## Throughput

2PC, hot inventory row, locks held ~10 ms:  
~100 transactions/sec on that row, hard limit

Saga, local transaction only, locks held ~1 ms:  
~1,000 transactions/sec on that row

10× more throughput on exactly the contended rows that matter.

**This is the strongest number in the lesson.** The saga's advantage is not latency — it is worse there. It is contention.

## Availability

2PC: needs ALL participants up.  
4 services at 99.9% →  $0.999^4 = 99.6\%$  (35 hours/year)

Saga: each step retries. A service being down delays the saga;  
it does not fail it.  
Effective completion rate ≈ 99.99%+, with some sagas  
simply taking longer.

The trade: sagas convert an outage into a delay.

## The inconsistency window

The number an interviewer will ask for:

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| happy path:           | order visible as PENDING for ~310 ms                                        |
| payment declined:     | stock reserved but unpaid for ~300 ms,<br>then released                     |
| payment service down: | stock held until it recovers, capped by<br>the 15-minute reservation expiry |
| compensation lost:    | up to 15 minutes (the expiry job)                                           |
| no expiry job:        | forever ← do not build this                                                 |

**The expiry interval is your worst-case inconsistency window.** That is a design parameter you choose, and being able to name it is what makes the answer concrete.

## The cost of compensating

```
1,000,000 orders/month, 2% fail after payment succeeds
= 20,000 refunds/month
```

```
card refund fee (typical) ₹2-20 each
support contacts, ~5% 1,000 tickets/month
customer confusion: a charge and a refund on the statement
```

```
Compare: with 2PC those 20,000 would never have been charged.
```

**Sagas move cost from engineering to operations.** Say this. It is what a senior engineer notices and a junior one does not.

## Sizing the orchestrator

```
1,000 orders/sec × 4 steps × 2 state writes = 8,000 writes/sec
saga state row: ~500 bytes
in-flight sagas at 300 ms each: ~300 rows at any moment — trivial
completed saga history, 90 days: 2.6 bn rows × 500 B ≈ 1.3 TB
→ archive completed sagas after 30 days
```

The orchestrator's *state* is small. Its *history* is not, and history is what you actually need when someone asks why order 4471 is stuck.

## 7 The trade-offs

### The honest summary

A saga trades atomicity and isolation for availability and decoupling. You get a system that keeps working when a service is down, and in exchange you get windows where the data is visibly wrong, and business processes — refunds, apology emails, support tickets — that exist to clean up after it.

## Saga versus 2PC

|                        | 2PC                                            | SAGA                                  |
|------------------------|------------------------------------------------|---------------------------------------|
| Atomicity              | real                                           | eventual                              |
| Isolation              | full                                           | <b>none</b>                           |
| Cross-service locks    | yes, for the whole transaction                 | none                                  |
| Availability           | product of all participants                    | tolerates outages as delays           |
| Latency                | 5-18 ms                                        | 300 ms - 2 s                          |
| Throughput on hot rows | ~100/s                                         | ~1,000/s                              |
| Failure mode           | blocks, holds locks                            | visible inconsistency, then repair    |
| Coupling               | tight — participants expose pre-prepare/commit | loose — plain service calls or events |
| Complexity             | in the protocol                                | <b>in your application code</b>       |

The last row matters. 2PC's complexity lives in the database and is somebody else's problem. A saga's complexity lives in your code — every compensation, every idempotency key, every expiry job is code you write, test and maintain. **A saga is not simpler. It moves the difficulty somewhere you can survive it.**

## Choreography versus orchestration

|                  | CHOREOGRAPHY                   | ORCHESTRATION                           |
|------------------|--------------------------------|-----------------------------------------|
| Workflow visible | nowhere                        | one file                                |
| Extra component  | none                           | the orchestrator                        |
| Coupling         | loosest                        | services couple to the orchestrator     |
| Debugging        | correlate logs across services | query one state table                   |
| Good up to       | ~3 steps                       | any number                              |
| Failure risk     | cyclic event dependencies      | a central place for logic to accumulate |

## When a saga is wrong

- **When compensation is impossible.** You cannot un-send an email, un-ship a parcel, or un-publish a post that has been screenshotted. Restructure so irreversible steps come last, or keep them out.
- **When the inconsistency window is unacceptable.** Some regulated financial operations genuinely cannot show a half-completed state.

- **When the data could just live together.** The best answer to a distributed-transaction question is often that the transaction should not be distributed. Two tables always written together may belong in one database or one shard.
- **When there are only two steps and one is idempotent.** Retry until it succeeds. That is not a saga, it is a retry, and it is much less code.
- **When the team cannot operate it.** A saga needs monitoring for stuck sagas, alerting on failed compensations, a dead-letter queue with a human process behind it, and a way to resume manually. Without those, a saga is worse than a nightly reconciliation job.

The thing everyone underestimates

**Isolation, and it is not close.** Losing atomicity is manageable — eventually everything settles. Losing isolation means *other transactions read your half-finished state and make decisions on it*, and those decisions do not get compensated. Nadia's other family went to a different hall. Nothing in the saga can reach out and un-decide that.

The countermeasures — semantic locks, commutative updates, pessimistic ordering, re-read values — all mitigate. None eliminates. **Be able to say that plainly.** It is the difference between having read about sagas and having run one.

## 8 In the interview

How it gets asked

- “Order service and payment service, separate databases. How do you keep them consistent?”
- “What is a saga? When would you use one instead of a distributed transaction?”
- “Choreography or orchestration — which and why?”
- “What happens if the compensating action itself fails?” — the question that separates people.
- “How do you make sure the database write and the event publish are atomic?” — the outbox.
- “Design a hotel or flight booking system.” — a saga in disguise, every time.

## The first ninety seconds

“Two-phase commit would give real atomicity here and I would not use it, for three reasons: it holds locks on inventory rows across service calls, so throughput on popular items drops by roughly ten times; availability becomes the product of every participant; and it couples the services together in exactly the way splitting them up was meant to prevent.

So: a saga. Break it into local transactions — reserve stock, take payment, create shipment — each committing immediately in its own database. For each one, define a compensating action that undoes it. If payment fails, I do not roll back, because stock is already committed and visible. I run the compensation: release the stock.

The distinction that matters is that **compensation is not rollback**. A rollback erases history. A compensation is a new forward transaction — the charge happened, then a refund happened. Two ledger entries, a bank fee, and a customer who sees both on their statement. That is a real cost and I would name it up front.

Two consequences I would be explicit about. Atomicity becomes eventual — there is a window, a few hundred milliseconds normally, where the order exists and is unpaid. And isolation is gone entirely: another request can read that half-finished state and act on it, and nothing compensates *that*.

With four steps I would orchestrate rather than choreograph, so the workflow lives in one readable place and I can query where any order is. Would you like me to go through the failure handling? That is where the real design is.”

## The follow-ups

“What if the compensation fails?”

“Retry with backoff, forever — a compensation is not allowed to give up, because the alternative is permanently inconsistent data. It must be idempotent, so retrying is always safe. After a bounded number of attempts it goes to a dead-letter queue and pages a human, because at that point it is an incident, not a transient error.

But retrying is not the real answer. The real answer is designing so that a lost compensation is survivable. Every reservation carries an expiry, and a background job reclaims anything older than fifteen minutes. If every compensation in the system vanished tomorrow, the stock still comes back — fifteen minutes late. **That expiry interval is my worst-case inconsistency window, and it is a number I choose deliberately.** Without it, one lost message leaks inventory permanently.”

### “Choreography or orchestration?”

“Choreography for up to about three steps — no extra component, services stay decoupled, and it is easy to follow. Beyond that I orchestrate, and the deciding argument is not performance, it is that with choreography **the workflow does not exist anywhere**. To answer ‘what happens when payment fails’ you read four services and correlate their logs. With an orchestrator it is one file, and I can query one table to find out where order 4471 is stuck. For a seven-step order flow that is worth an extra service.

The cost of an orchestrator is that it becomes a place where business logic accumulates, and that it must itself be durable — it writes its state before every call, so a restart resumes exactly where it stopped. An orchestrator that keeps saga state in memory is not an orchestrator, it is a function call with extra steps.”

### “How do you make the database write and the event publish atomic?”

“You cannot put them in one transaction, so you remove the second system. **The outbox pattern:** write the event into an outbox table in the *same* local transaction as the business data. A separate process polls the outbox and publishes. Now there is one commit, and either both happened or neither did.

The publisher can crash after publishing and before marking the row sent, so events can go out twice. That is at-least-once delivery and it is fine, because every consumer has to be idempotent anyway — a redelivered ‘charge the card’ must not charge twice. I would use a client-supplied idempotency key per step, checked both in my own service and at the payment gateway.”

### “Where does isolation actually bite?”

“Three named hazards. A **dirty read** — someone reads state that gets compensated away, and acts on it. A **lost update** — two sagas write the same row and one compensation clobbers the other’s work. A **fuzzy read** — the same saga reads a row twice and gets different answers.

Four countermeasures, none of which is a cure. Semantic locks: mark the row `PENDING_PAYMENT` so readers know it is in flight. Commutative updates: `balance = balance - 50` composes safely where `balance = 450` does not — that is the cheapest fix and it removes lost updates outright. Pessimistic ordering: put the risky step where its visible window is shortest. And re-read-value with a version column, which is just optimistic concurrency control.

I would be honest that they mitigate rather than solve. If a customer saw an item as out of stock during a 300-millisecond window and went elsewhere, no compensation reaches them.”

### The model answer

*“Design a flight booking system. A booking reserves a seat, charges the card, issues a ticket and emails a confirmation.”*



“Four steps across four systems, and one of them talks to an external airline. Let me set the shape first.

**Why not 2PC.** The airline’s reservation system is not going to join my XA transaction — that settles it immediately. Even if it would, holding a lock on a seat across a 250-millisecond payment call would destroy throughput on a popular flight, which is exactly the flight that matters.

**The saga, and the ordering is the interesting decision.** Reserve seat, charge card, issue ticket, send email. I have ordered it that way deliberately: **the irreversible step goes last.** I can release a seat. I can refund a card, at a cost. I can void a ticket. I cannot un-send an email. So the email is the final step, and if it fails I retry it rather than compensating anything — a failed email is not a reason to cancel a valid booking.

**Orchestrated,** not choreographed. Four steps with an external dependency and real money; I want the workflow in one file and the state in one table, so support can answer ‘where is booking 4471?’ without reading four services’ logs.

**Compensations.** Release the seat. Refund the payment — a new transaction, with a fee, visible on the customer’s statement. Void the ticket. Each idempotent, each retried with backoff, each ending in a dead-letter queue with an alert if it truly cannot proceed.

**The safety net.** Seat reservations expire after fifteen minutes. That single mechanism means a lost compensation costs me fifteen minutes of an unavailable seat rather than a seat that is unsellable forever. It is the most important part of the design and it is three lines of code plus a cron job.

**The outbox** for every event, so the local write and the published event are atomic. At-least-once delivery, so every step carries an idempotency key — the payment one goes to the gateway as well, so a duplicate that gets past me is still caught.

**Numbers.** End to end, dominated by the two external calls: ~250 ms for payment, ~200 ms for the airline, so roughly 500-600 ms per booking. The saga machinery is maybe 30 ms of that. At 100 bookings a second that is 800 state writes a second, trivial. Ninety days of saga history at 500 bytes a row is a few hundred gigabytes, so I would archive after thirty days.

**What I would tell the business, in their words.** If payment fails after the seat is reserved, the seat is unavailable to other customers for a few hundred milliseconds — nobody notices. If the payment service is down, up to fifteen minutes.

And if a card is charged and the airline then rejects the booking, the customer sees a charge and a refund on their statement two days apart, and some of them will ring support. That is a real operational cost — at a million bookings a month with two percent late failures, twenty thousand refunds — and it is the price of not using a distributed transaction. I would rather they know that number in advance than discover it.

**What would change the design.** If seat inventory were mine rather than the airline's, I would look hard at whether the seat and the booking could live in one database, because a local transaction beats every option here. And if the business could not tolerate the charge-then-refund pattern at all, I would reorder to authorise the card first and capture only after the airline confirms — an authorisation hold is reversible in a way a capture is not. That is a better design and it comes from the same rule: put the irreversible thing last.”

That answer rejects 2PC for a concrete reason, orders the steps by reversibility and says so, names the safety net as the most important part, includes the outbox and idempotency, gives real numbers including the operational cost, and ends with a genuinely better variant that falls out of the same principle.

## 9 Recall card

A **saga** = a sequence of local transactions, each with a **compensating transaction** that semantically undoes it.  $T_1 \dots T_n$  forward; on failure at  $T_k$ , run  $C(k-1) \dots C_1$  backwards.

**Compensation is not rollback.** Rollback erases history; compensation is a *new forward transaction*. Payment + refund  $\neq$  no payment — two ledger entries, a fee, a confused customer.

**Order by reversibility: irreversible steps go LAST.** You cannot un-send an email.

**Two styles: choreography** (events, no controller — up to  $\sim 3$  steps; the workflow exists nowhere) and **orchestration** (one component owns the workflow, writes state durably before every call — beyond 3 steps).

**What you gave up:** atomicity becomes eventual; **isolation is gone entirely.**

**Three isolation hazards:** dirty read, lost update, fuzzy read. **Four countermeasures (mitigate, never cure):** semantic lock (`status = PENDING`), commutative updates (`balance - 50`, not `balance = 450`), pessimistic ordering, re-read value.

**Three production rules:** 1. Every step idempotent — idempotency key per step, checked locally *and* at the external service. 2. Compensations idempotent, retried forever with backoff, dead-letter + alert if they truly fail. 3. **Every reservation expires on its own.** The expiry interval is your worst-case inconsistency window.

**The outbox pattern:** write the event to an outbox table in the same local transaction; a poller publishes. Turns two systems into one database. At-least-once  $\Rightarrow$  consumers must be idempotent.

**Numbers:** saga  $\approx$  300 ms - 2 s vs 5-18 ms for 2PC, **but  $\sim 10\times$  the throughput on hot rows** because there are no cross-service locks. 2PC availability is a product ( $4 \times 99.9\% = 99.6\%$ ); a saga turns an outage into a delay. 2% late failures on 1M orders = 20,000 refunds/month.

**Saga vs 2PC in one line:** trades atomicity and isolation for availability and decoupling — and moves the complexity out of the protocol and **into your application code.**

**Wrong choice when:** compensation is impossible, the window is unacceptable, the data could live together, or there are two steps and a retry would do.

---

**Next:** [Day 122 — Idempotency and exactly-once delivery](#)

**Previous:** [Day 120 — Distributed transactions and two-phase commit](#)

**DSA topic:** Insert, search, and prefix search **System design topic:** The saga pattern

### Code these, in this order

One rule for the whole set: **write the four delete cases as tests before you write delete**. Absent word, word that is a prefix of another, word that extends another, word alone on its branch. If your four assertions on `node_count()` are wrong, the implementation will be too.

| # | PROBLEM                                    | SOURCE                | WHAT IT IS REALLY TESTING                                             |
|---|--------------------------------------------|-----------------------|-----------------------------------------------------------------------|
| 1 | Implement Trie (Prefix Tree)               | LeetCode 208 (Medium) | <code>search</code> vs <code>startsWith</code> — one condition apart. |
| 2 | Design Add and Search Words Data Structure | LeetCode 211 (Medium) | Wildcards: a walk becomes a DFS.                                      |
| 3 | Replace Words                              | LeetCode 648 (Medium) | Stopping at the <i>first</i> <code>is_end</code> , not the last.      |
| 4 | Word Search II                             | LeetCode 212 (Hard)   | Pruning a grid search with a trie — the real payoff.                  |
| 5 | Word Break                                 | LeetCode 139 (Medium) | The trie replacing the set lookup, plus memoisation.                  |

**Delete is not on LeetCode. Write it anyway.**

Add `delete` to your problem 1 solution and test all four cases by asserting `node_count()` after each. The five numbers from the lesson are 9, 9, 8, 7, 5. If you get different ones, find out why before moving on — this is the one operation an interviewer will ask for and no online judge will check.

**On problem 2, break the base case deliberately**

Change `return node.is_end` to `return True` and record what `search("ca")` returns when only `"cat"` is stored. Then change the wildcard loop to `return self._match(...)` inside the loop and record which patterns start failing.

#### On problem 4, measure the pruning

Solve it once with a plain word list and once with a trie. Count how many grid cells each version visits. Two numbers. Say why the trie version can abandon a path several characters early.

#### On problem 5, say why a trie helps at all

A hash set already gives `O(1)` lookup here. State what the trie buys you and be honest if the answer is “not much” for the standard version — then say which variant makes it worth it.

---

#### The three-operations drill

1. Write the shared walk from memory.
2. Write insert, search and `starts_with` on top of it.
3. Say the single condition that differs between the last two.
4. Say what makes insert idempotent, and what breaks that.

#### The delete-rule drill

1. State the pruning rule in one line, both conditions.
2. Say what each condition means in terms of who is using the node.
3. Say why it must be evaluated bottom-up.
4. Say what the recursive delete’s return value means, in words.

#### The four-cases drill

1. Name all four cases with an example each.
2. For each, say how many nodes are removed.
3. Say which two are mirror images and which one breaks naive implementations.
4. Give the five `node_count()` values from the lesson’s run.

#### The delete-implementation drill

1. Write the recursive version from memory.
2. Write the iterative version from memory.
3. Point at the line in the iterative version that is the pruning rule.
4. Say why the `del` must come before the prunability check.
5. Say why the two-pass version is the one to write in an interview.

### The wildcard drill

1. Say what changes when a `.` appears — walk to what?
2. Write `_match` from memory.
3. Say what the base case must return, and what happens if it returns `True`.
4. Say what goes wrong if the loop returns instead of continuing.
5. Give the worst-case complexity and then the practical correction.

### The wildcard-cost drill

1. Give the branch counts for `cat`, `c.t`, `.at`, `..t` on an English trie.
2. Say why the practical number is far below `26^w`.
3. Compare `c...` with `...t` and say why they differ by ten times.
4. Give the free optimisation for fixed-length patterns.
5. Say what a reversed second trie would buy, and what it costs.

### The completions drill

1. Say what `completions("")` returns without a limit.
2. Write the bounded version with an explicit stack.
3. Say why the stack pushes in reverse alphabetical order.
4. Say how the design changes if “top” means most popular rather than alphabetical.

### The break-it drill

Trigger each and record the exact output or error:

1. Delete `"car"` from `{car, card}` with the `is_end` check missing from the prune rule.
  2. `_delete` called directly on a word that is not present.
  3. The prunability check placed before the `del`.
  4. `search_pattern("ca")` with a base case of `return True`.
  5. The wildcard loop written with `return` instead of `if ...: return True`.
  6. A recursive delete on a 2,000-character key.
  7. `completions("")` on a 100,000-word trie.
  8. Removing children from a dict while iterating over it.
- 

### The definition drill

1. Define a saga in one sentence.

2. Say what a compensating transaction is.
3. State the difference between compensation and rollback, with the payment example.
4. Say which step you start compensating from when step 4 fails, and why.

#### **The ordering drill**

1. State the ordering rule for saga steps.
2. Say why it is counter-intuitive.
3. Give three examples of steps that cannot be compensated.
4. Say what you do with a step that truly cannot be undone.

#### **The two-styles drill**

1. Describe choreography and orchestration in two sentences each.
2. Give the step-count rule and the reason behind it.
3. State the strongest argument against choreography.
4. State the two costs of an orchestrator.
5. Say what makes an orchestrator different from a function that calls four services.

#### **The isolation drill**

1. Say what a saga gives up beyond atomicity.
2. Name the three hazards and give a scenario for each.
3. Name the four countermeasures and say what each catches.
4. Give the commutative-update example, both the safe and unsafe forms.
5. Say plainly what none of the countermeasures can fix.

#### **The production-rules drill**

1. State the three rules that make sagas work.
2. Say why every step needs an idempotency key, and where it is checked.
3. Say what happens to a compensation that keeps failing.
4. Explain why the expiry job is the most important part of the design.
5. Say what the expiry interval equals, in terms of consistency.

#### **The outbox drill**

1. State the problem the outbox solves.
2. Give both wrong orderings and say what each loses.
3. Write the SQL.

4. Say why events can be published twice and why that is acceptable.
5. Say what property that requirement forces on every consumer.

#### **The numbers drill**

1. Break a four-step saga into its latency components.
2. Say what dominates, and by how much.
3. Compare saga and 2PC on latency and on hot-row throughput.
4. Give the availability figures for both.
5. List the inconsistency windows for five failure scenarios.
6. Compute the monthly refund cost for a million orders at two percent.
7. Size the orchestrator's state and its history.

#### **The trade-offs drill**

1. Fill in the nine-row saga-versus-2PC table.
2. Say where each one's complexity lives, and why that matters.
3. Give five situations where a saga is the wrong choice.
4. Say which loss people underestimate, and why it is worse than the other.
5. Say what a team must have in place before running sagas in production.

#### **The failure drill**

For each, say what happens and what you would build:

1. Payment succeeds, shipping fails, and the refund call times out.
2. A message is redelivered and the card is charged twice.
3. The orchestrator restarts mid-saga.
4. An event is published but the database write was rolled back.
5. A customer sees an item as out of stock during a 300 ms window and leaves.
6. A compensation lands in the dead-letter queue at 3 a.m.
7. Two sagas update the same balance and one compensates.

Two of the seven are not fixed by better retry logic. Name them.

---

#### **Say these out loud**

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.



1. *Implement a trie, then delete a word from it.* The shared walk and the three one-liners, then the pruning rule with both conditions, why it is bottom-up, and all four cases with the prefix case named as the one that breaks naive code.
  2. *Order service and payment service, separate databases. Keep them consistent.* 2PC rejected for three named reasons, the saga described, compensation-is-not-rollback stated plainly, orchestration chosen with the reason, and both losses — eventual atomicity and no isolation — admitted before being asked.
  3. *What if the compensation fails?* Retry forever with backoff, idempotent, dead-letter and alert — then the real answer: design so a lost compensation is survivable, with the expiry job and the window it defines.
- 

### Before you move on

- [ ] I can write the shared walk and all three operations on top of it.
- [ ] I can name the one condition separating `search` from `starts_with`.
- [ ] I can state the pruning rule with both conditions.
- [ ] I can say why delete must be bottom-up.
- [ ] I can name all four delete cases and how many nodes each removes.
- [ ] I know that deleting `car` must not remove `card`.
- [ ] I wrote delete recursively and iteratively.
- [ ] I know the `del` comes before the prunability check.
- [ ] I know delete is  $O(L)$ , not  $O(n)$ .
- [ ] I can write the wildcard match from memory.
- [ ] I know the base case returns `is_end`, never `True`.
- [ ] I can give the worst case and the practical correction for wildcards.
- [ ] I know why wildcard position matters more than wildcard count.
- [ ] My completions take a limit and stop early.
- [ ] I know when to use the iterative delete.
- [ ] I can define a saga and a compensating transaction.
- [ ] I can say why compensation is not rollback, with an example.
- [ ] I know irreversible steps go last, and why.
- [ ] I can choose between choreography and orchestration with a reason.
- [ ] I know an orchestrator must write state durably before every call.
- [ ] I can name the three isolation hazards and the four countermeasures.

- [ ] I can say what none of the countermeasures fix.
- [ ] I know every step and every compensation must be idempotent.
- [ ] I know a failed compensation is an incident, not a retry.
- [ ] I know the expiry job is the safety net and defines the worst-case window.
- [ ] I can describe the outbox pattern and write the SQL.
- [ ] I know sagas win on contention, not latency.
- [ ] I can give the operational cost of compensating at scale.
- [ ] I know five situations where a saga is the wrong choice.
- [ ] I answered all three questions above out loud.

# Autocomplete and word dictionaries

DATA STRUCTURES AND ALGORITHMS

Autocomplete and word dictionaries

SYSTEM DESIGN

Idempotency and exactly-once delivery

## 1 What this is, and why they ask it

Autocomplete is the box that shows you five suggestions after you have typed three letters. Underneath it is the structure you built on [day 120](#) and the walk you wrote on [day 121](#), plus one new step: after you reach the prefix, you have to decide *which* of the words below it to show, and in what order.

That last step is the whole lesson. Reaching the prefix is three lines. Ranking what is underneath it is where the design decisions live, and where the interviewer is actually looking.

They ask this because it is the one trie question that is also a system design question. “Design autocomplete for a search box” is asked at Google, Amazon and Flipkart, in both the coding round and the design round, and the same candidate often gives a good answer in one and a bad answer in the other. It is also LeetCode 642 and LeetCode 1268 almost word for word, so the coding version is a known quantity — you can walk in having already written it.

By the end of this lesson you can walk to a prefix, gather every completion below it, rank them, and then throw that solution away and replace it with the one that stores the answers in advance. You will be able to say what each version costs, in operations and in bytes, and name the exact situation in which the fast one becomes wrong.

## 2 The story

Ramesh has stood behind the counter of the same chemist’s shop for twenty-six years. It is a narrow shop on a busy road, two customers wide, with shelves running from the floor to the ceiling behind him.

The shelves are arranged by the first letters of the name. Everything beginning with A is on the left of the top row. B is beside it. Then C, and so on, down and across, until Z ends at the bottom right, near his knees.

A woman comes in at half past eight with a folded strip of tablets in her hand and starts to say the name. “Amoxi—”

Ramesh has already turned around. He does not wait for the rest of it. Three steps, left hand up, and he is standing in front of the A shelf. Everything he could possibly need is in that one square foot of wood.

There are eleven boxes on that shelf that begin with those five letters. He does not read all eleven. At eye level, in the front row, he keeps the four that people actually ask for — the ones he sells forty times a week. The other seven sit behind them and above them, and he touches those maybe once a month.

So he pulls the front four, holds them up, and says the four names out loud. The woman points at the second one. Eleven seconds, start to finish.

Once a fortnight somebody asks for something that is not in the front row. Then Ramesh has to drag the stool over, climb up, and go through the whole shelf box by box, and that takes two minutes instead of eleven seconds.

His nephew, who covers the shop on Sundays, cannot do this. He knows where the A shelf is. That part is written on the wood. What he does not know is which four of the eleven to hold up first. So he holds up all eleven, and the customer stands there reading them one by one, and the queue behind her grows.

The shelf is the easy half. The front row is the hard half, and it took Ramesh six years to get it right.

---

### 3 The idea in plain English

Take the shop apart, piece by piece.

**The shelves are the trie.** You met a **trie** on [day 120](#): a tree where the path from the root spells a word, one character per edge. Walking “A”, then “m”, then “o”, then “x”, then “i” is exactly Ramesh taking three steps to the A shelf. It costs one move per character, and nothing else.

**Reaching the shelf is `starts_with`.** You wrote that walk yesterday. Follow one child per character of the prefix. If a character is missing, there is no shelf, and the answer is an empty list of suggestions. If you get to the end, the node you are standing on is the root of a **subtree** — the whole set of words that begin with what the user typed. A subtree is a node together with everything hanging below it.

**Gathering the boxes is a traversal.** The words under that node are not stored in a list anywhere. They are spelled out by the paths below it. To read them you walk down every branch and collect the ones marked as whole words. This is the depth-first search you learned on [day 100](#), run on the subtree instead of on the whole tree.

**The front row is ranking.** Ramesh does not hold up the eleven boxes in the order they sit on the shelf. He holds up the four that sell most. To do that in code, every word needs a **weight** — a number saying how often it is chosen. Search suggestions

weight by how many people searched for that term. A code editor weights by how recently you used the symbol.

**Ties need a rule.** Two words with the same weight must still come out in a fixed order, or your function returns different answers on different runs and no test can pin it down. The standard rule, and the one LeetCode 642 uses, is: **higher weight first; when weights are equal, alphabetically smaller first.**

**The nephew is the version without ranking.** He returns everything under the shelf, correct and useless. A suggestion box that shows two thousand suggestions has not answered the question.

**Ramesh's front row is precomputation.** This is the idea worth carrying out of today. He did not decide at half past eight which four boxes to hold up. He decided months ago, and rearranged the shelf then. When the customer speaks, he does no thinking at all — he grabs what is already in front.

In code that means: at every node in the trie, store the top  $k$  words in that node's subtree, worked out in advance. Then answering a query is walk-to-the-node, read the stored list, stop. No traversal, no sorting, nothing proportional to how many words are underneath.

**The cost moves, it does not vanish.** Ramesh pays for the front row every time a new medicine starts selling well and the shelf has to be rearranged. In code you pay on insert: adding one word updates the stored list at every node along its path. That is the trade you are being asked to name — **cheap reads bought with expensive writes, and extra memory at every node.**

**And the stool is the fallback.** Once a fortnight, somebody wants something outside the front row. If your product needs the full ranked set and not just the top five, pre-computation does not help you and you are back to traversing.

#### 4 The picture

Six words, with the number of times each has been searched.

```
cat 15 car 40 card 9 care 21 cart 3 dog 30
```

Here is the trie, with each node labelled by the top two words in its own subtree — Ramesh's front row, written on every shelf rather than only the one he is standing at.

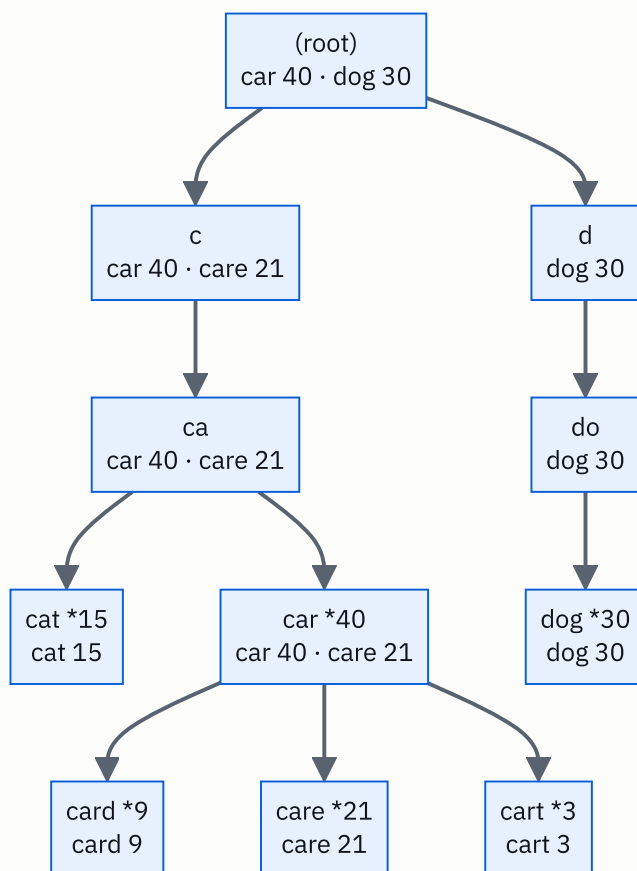


FIGURE 122.1

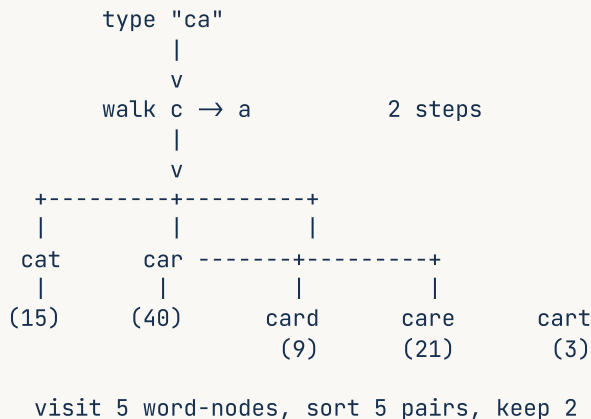
**What to notice.** A star marks a node where a whole word ends. Every node carries the best two words below it, and those lists are *already there* before anybody types anything.

Type `ca` and you land on the `ca` node. Its stored list reads `car 40 · care 21`. You return those two and stop. You never visit `cat`, `card` or `cart`, even though they are all underneath — and that is the point. The unranked version visits all five.

Notice also that `car` is a whole word *and* has three children. Its own front row starts with itself. This is the case people forget: **the prefix can be a word, and it usually deserves the top slot.**

And notice the root. Its list is what an empty search box should suggest — the two most popular things overall. You get that for free.

Now the same trie without precomputation, which is the version you write first:



**What to notice here.** The work is proportional to how much sits under the prefix, not to how much you return. Typing `c` in a hundred-thousand-word dictionary walks one step and then visits six thousand nodes to return five words.

## 5 The code, built step by step

Start with the node. It is yesterday's node with one field added.

```

class Node:
 """One character position in the trie."""

 __slots__ = ("children", "weight", "top")

 def __init__(self) → None:
 self.children: dict[str, Node] = {}
 self.weight: int = 0 # 0 means "no word ends here"
 self.top: list[tuple[int, str]] = [] # the front row, filled in later

```

Two changes from day 121. `is_end` has become `weight`, because a word now needs a popularity number and not just a yes-or-no. Zero doubles as “not a word”, which works as long as a stored word always has a weight of at least one. `top` is empty for now; the second half of this section fills it.

`__slots__` tells Python not to give each object a dictionary of its own attributes. On a trie with hundreds of thousands of nodes it saves real memory, and it is a nice thing to mention out loud.

### Version one: walk, gather, rank

Insert first. It is yesterday's insert with a number on the end.



```
def add(self, word: str, weight: int = 1) → None:
 """Store `word`, adding `weight` to its popularity."""
 node = self.root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.weight += weight
```

`setdefault` returns the existing child if there is one and creates it if there is not, which is the whole of “walk down, building as you go” in one line. Note `+=` rather than `=`: calling `add("car", 1)` forty times should leave `car` with weight 40, not weight 1.

Now the walk to the prefix.

```
def _walk(self, prefix: str) → Node | None:
 """The node at the end of `prefix`, or None if no word starts that way."""
 node = self.root
 for character in prefix:
 if character not in node.children:
 return None
 node = node.children[character]
 return node
```

This is `starts_with` from yesterday, returning the node instead of a boolean. Returning `None` rather than raising keeps the caller simple: no shelf, no suggestions.

Then the gathering. This is the new part.

```
def _collect(self, node: Node, so_far: str, out: list[tuple[int, str]]) → None:
 """Append (weight, word) for every whole word in this subtree."""
 if node.weight:
 out.append((node.weight, so_far))
 for character, child in node.children.items():
 self._collect(child, so_far + character, out)
```

Four lines and both halves of a depth-first search. If the current node ends a word, record it. Then recurse into every child, carrying the string built so far.

`so_far` is the string spelled by the path from the root. You pass it down because the node itself does not know which character reached it — the trie stores characters on edges, not in nodes. Building `so_far + character` copies the string at every step, which is fine for words but not for keys thousands of characters long. The fix, if you ever need it, is a list of characters joined at the end.

Now put the three together.

```
def suggest(self, prefix: str, k: int = 5) → list[str]:
 """The k best completions of `prefix`, most popular first."""
 node = self._walk(prefix)
 if node is None:
 return []
 found: list[tuple[int, str]] = []
 self._collect(node, prefix, found)
 found.sort(key=lambda pair: (-pair[0], pair[1]))
 return [word for _weight, word in found[:k]]
```

The sort key is the tie rule from section 3, written out. `-pair[0]` sorts weights descending, because Python sorts ascending and negating a number reverses it. `pair[1]` then sorts equal weights alphabetically ascending. One tuple, two rules, in priority order.

Passing `prefix` as the starting `so_far` matters. The node you reached does not remember how you got there, so if you pass an empty string you will get back `t`, `rd` and `re` instead of `cat`, `card` and `care`.

Sorting everything to keep five is wasteful. You met the fix on [day 116](#):

```
from heapq import nsmallest

def suggest_with_heap(self, prefix: str, k: int = 5) → list[str]:
 node = self._walk(prefix)
 if node is None:
 return []
 found: list[tuple[int, str]] = []
 self._collect(node, prefix, found)
 best = nsmallest(k, ((-weight, word) for weight, word in found))
 return [word for _negated, word in best]
```

`nsmallest` keeps a heap of size  $k$  and never sorts the rest, turning  $m \log m$  into  $m \log k$ . The negation inside the generator is what makes “smallest” mean “most popular”, and it also keeps the alphabetical tie-break working, because for equal negated weights the smaller string still wins.

That is version one, complete and correct. It is the right answer to LeetCode 1268 and it is what you should write first in an interview.

#### Version two: the front row

Now build Ramesh’s shelf. The plan: every node stores the best  $k$  words in its own subtree, and `add` maintains those lists as it walks down.

```
def add(self, word: str, weight: int = 1) → None:
 node = self.root
 path = [node] # the root counts
 for character in word:
 node = node.children.setdefault(character, Node())
 path.append(node)
 node.weight += weight
 for ancestor in path:
 self._offer(ancestor, node.weight, word)
```

`path` collects every node from the root down to the last character, inclusive. After updating the weight, you offer the word to each of them, because a word is in the subtree of every node along its own path and of no others. That last clause is why this works at all: inserting `card` cannot change the front row of `dog`, so you never have to look at it.

The offer itself:

```
def _offer(self, node: Node, weight: int, word: str) → None:
 """Put (weight, word) into this node's front row, keeping only the best k."""
 node.top = [pair for pair in node.top if pair[1] ≠ word]
 node.top.append((weight, word))
 node.top.sort(key=lambda pair: (-pair[0], pair[1]))
 del node.top[self.k :]
```

Read the first line carefully, because it is the one people leave out. If `car` is already in the list with weight 39 and you are now offering it with weight 40, appending without removing leaves the same word in the list twice, and `suggest("ca")` returns `["car", "car", "care"]`. Remove the old entry, then add the new one.

The list has at most  $k + 1$  entries when you sort it, so the sort is on five or six items. That is not the  $m \log m$  you were trying to avoid — it is a fixed, tiny cost paid per node on the path.

Reading is now trivial.

```
def suggest(self, prefix: str) → list[str]:
 node = self.root
 for character in prefix:
 if character not in node.children:
 return []
 node = node.children[character]
 return [word for _weight, word in node.top]
```

No traversal. No sort. Walk the prefix, read the list, return. This function does the same amount of work for a prefix with six thousand completions as for one with two.

### The complete solution

Both versions, in one file, ready to run.

```

"""Autocomplete over a trie: gather-and-rank, and precomputed top-k."""

from __future__ import annotations

from heapq import nsmallest

class Node:
 """One character position in the trie."""

 __slots__ = ("children", "weight", "top")

 def __init__(self) → None:
 self.children: dict[str, Node] = {}
 self.weight: int = 0
 self.top: list[tuple[int, str]] = []

class Autocomplete:
 """Walk to the prefix, gather everything below it, rank it, keep k.

 Reads are proportional to the size of the subtree. Writes are O(len(word)).
 Use this when the dictionary changes constantly or memory is tight.
 """

 def __init__(self) → None:
 self.root = Node()

 def add(self, word: str, weight: int = 1) → None:
 node = self.root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.weight += weight

 def _walk(self, prefix: str) → Node | None:
 node = self.root
 for character in prefix:
 if character not in node.children:
 return None
 node = node.children[character]
 return node

 def _collect(self, node: Node, so_far: str, out: list[tuple[int, str]]) → None:
 if node.weight:
 out.append((node.weight, so_far))
 for character, child in node.children.items():
 self._collect(child, so_far + character, out)

 def suggest(self, prefix: str, k: int = 5) → list[str]:
 node = self._walk(prefix)
 if node is None:
 return []
 found: list[tuple[int, str]] = []
 self._collect(node, prefix, found)
 # -weight sorts most popular first; word breaks ties alphabetically.
 best = nsmallest(k, ((-weight, word) for weight, word in found))
 return [word for _negated, word in best]

class RankedAutocomplete:
 """Every node stores the best k words beneath it, maintained on insert.

 Reads are O(len(prefix)). Writes are O(len(word) * k log k). Use this when
 reads vastly outnumber writes, which is the normal case for a search box.
 """

 def __init__(self, k: int = 5) → None:

```

```

self.k = k
self.root = Node()

def add(self, word: str, weight: int = 1) → None:
 node = self.root
 path = [node]
 for character in word:
 node = node.children.setdefault(character, Node())
 path.append(node)
 node.weight += weight
 for ancestor in path:
 self._offer(ancestor, node.weight, word)

def _offer(self, node: Node, weight: int, word: str) → None:
 node.top = [pair for pair in node.top if pair[1] ≠ word]
 node.top.append((weight, word))
 node.top.sort(key=lambda pair: (-pair[0], pair[1]))
 del node.top[self.k :]

def suggest(self, prefix: str) → list[str]:
 node = self.root
 for character in prefix:
 if character not in node.children:
 return []
 node = node.children[character]
 return [word for _weight, word in node.top]

if __name__ == "__main__":
 words = [("cat", 15), ("car", 40), ("card", 9), ("care", 21), ("cart", 3), ("dog", 30)]

 slow = Autocomplete()
 fast = RankedAutocomplete(k=2)
 for text, count in words:
 slow.add(text, count)
 fast.add(text, count)

 print(slow.suggest("ca", k=2)) # ['car', 'care']
 print(fast.suggest("ca")) # ['car', 'care']
 print(slow.suggest("car", k=3)) # ['car', 'care', 'card']
 print(fast.suggest("")) # ['car', 'dog']
 print(fast.suggest("xy")) # []

```

Run it and you get:

```

['car', 'care']
['car', 'care']
['car', 'care', 'card']
['car', 'dog']
[]

```

The third line is the one to look at. `car` itself comes back as a suggestion for the prefix `car`, ahead of `care` and `card`, because it is a word and it is the most popular one. If your version drops it, your `_collect` is skipping the node it starts on.

## 6 What it costs

Count version one first. Let  $p$  be the length of the prefix,  $m$  the number of words under it, and  $L$  their average length.

- **The walk:**  $p$  steps. One dictionary lookup each.
- **The traversal:** you visit every node in the subtree. A subtree holding  $m$  words of average length  $L$  has at most  $m \times L$  nodes, and fewer when they share prefixes. Call it  $m \times L$  character steps.
- **Building the strings:** `so_far + character` copies a string of growing length at each step. That adds up to roughly  $m \times L$  character copies as well.
- **The ranking:** sorting is  $m \log m$  comparisons; the heap version is  $m \log k$ .

So `suggest` is  $O(p + m \cdot L + m \log k)$  and the  $m \cdot L$  term dominates in practice.

Put numbers on it. Take an English dictionary of 100,000 words with an average length of 8. Type `c`:

|                       |                                             |           |
|-----------------------|---------------------------------------------|-----------|
| words under "c"       | ~ 6,000                                     |           |
| character steps       | $6,000 \times 8$                            | = 48,000  |
| heap work ( $k = 5$ ) | $6,000 \times \log_2(5) = 6,000 \times 2.3$ | = ~14,000 |
| total                 | ~62,000 operations, for five strings        |           |

Now type `cardio`:

|                      |                 |       |
|----------------------|-----------------|-------|
| words under "cardio" | ~ 40            |       |
| character steps      | $40 \times 8$   | = 320 |
| total                | ~400 operations |       |

**That 150-fold gap is the problem.** The suggestion box is slowest exactly when the user has typed least, which is exactly when they are still typing fast. The first keystroke is the worst one.

Version two:

- **suggest**:  $p$  dictionary lookups, then copy at most  $k$  strings.  $O(p + k)$ . For  $p = 1$  and  $k = 5$  that is about six operations, against sixty-two thousand. Roughly ten thousand times less work.
- **add**: the walk is  $L$  steps. At each of the  $L + 1$  nodes on the path you rebuild a list of at most  $k + 1$  items and sort it:  $k \log k$  plus a scan of  $k$ . So  $O(L \times k \log k)$ . With  $L = 8$  and  $k = 5$  that is about  $8 \times 15 = 120$  operations. Version one's `add` was 8.

**Fifteen times slower on write, ten thousand times faster on read.** For a search box that reads a thousand times per write, that trade is not close.

Space is where it bites. Count nodes first:

```
100,000 words x 8 characters = 800,000 nodes if nothing is shared
shared prefixes in real English ~ 250,000 nodes in practice
```

Version one stores per node: a dictionary of children, an integer, an empty list. Version two adds a list of `k` tuples:

```
250,000 nodes x 5 entries = 1,250,000 tuples
each tuple: 2 references x 8 bytes = 16 bytes, plus ~40 bytes tuple overhead
1,250,000 x 56 = 70 MB
```

The strings themselves are shared, not copied — every entry points at the same string object the dictionary already holds — so you pay for references, not for text. Seventy megabytes to make every keystroke free is a trade most services take without blinking, and it is a number worth saying out loud rather than saying “some extra memory”.

If seventy megabytes is too much, store the front row only on nodes at depth 1, 2 and 3, where the subtrees are huge, and traverse below that, where they are small. Nobody types twelve characters into a search box and waits.

## 7 The traps

The prefix that is also a word

The near-miss:

```
def _collect(self, node, so_far, out):
 for character, child in node.children.items():
 if child.weight:
 out.append((child.weight, so_far + character))
 self._collect(child, so_far + character, out)
```

This looks tidy — check the child before descending into it — and it is wrong. It never examines the node it was called on. Ask for completions of `car` in the sample data and you get `care`, `card` and `cart`, but not `car`, which is the most popular word in the whole trie:

```
>>> slow.suggest("car", k=3)
['care', 'card']
```



Two results where three were asked for, and the missing one is the best one. Check the current node first, then recurse. That is why the working version starts with `if node.weight:`.

### The empty prefix

`suggest("")` walks zero characters, lands on the root, and collects the entire dictionary. On the sample data that is harmless. On a real one:

```
>>> len(dictionary.suggest("", k=100000))
100000
```

A hundred thousand strings built, sorted and returned because a user cleared the box. Decide what an empty prefix means before you ship: either return the global top  $k$ , which version two gives you for free at the root, or return nothing at all. Never let it fall through to the traversal.

### The heap full of nodes

Somebody always tries to push nodes onto the heap to walk the trie by popularity:

```
import heapq
heap = [(-node.weight, node)]
heapq.heappush(heap, (-child.weight, child))
```

The moment two entries have the same weight, Python compares the second element to break the tie:

```
Traceback (most recent call last):
 File "auto.py", line 41, in <module>
 heapq.heappush(heap, (-child.weight, child))
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

Two nodes with weight 0 is not a rare case in a trie; it is almost every pair. Put a tie-breaker between the weight and the object — the string, or a counter — and it goes away. You met the same error on [day 116](#); it will keep finding you.

### The deep word

`_collect` recurses once per character. Feed it a long key, such as a URL or a DNA fragment, and:

```
Traceback (most recent call last):
 File "auto.py", line 33, in _collect
 self._collect(child, so_far + character, out)
 [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

Python's default limit is 1000 frames and you were already several deep before `_collect` started. For ordinary words this never happens. For anything where the key is a path or an address, rewrite `_collect` with an explicit stack — push `(node, so_far)` and pop in a loop.

### The front row that goes stale

This is the real trap of version two, and it is the follow-up an interviewer asks when they want to separate people.

Version two maintains the stored lists correctly when weights only ever go **up**. Now delete a word, or drop a weight:

```
fast.add("car", 40) # every node on c-a-r now lists car first
fast.remove("car") # what should the root's list say now?
```

You cannot fix the ancestors from local information. The root's list said `car 40 · dog 30`. Removing `car` leaves a hole, and the word that should fill it might be anywhere in the trie — you do not know what it is without traversing the subtree you were trying to avoid traversing. The stored list is a cache, and this is cache invalidation.

The three honest answers, in the order you should offer them:

1. **Do not delete.** Set the weight to zero and let the entry age out of the list the next time something more popular is offered. Wrong for a while, then right. Fine for search suggestions, not fine for a blocklist.
2. **Rebuild the affected path.** After a delete, re-traverse the subtree of each node on the removed word's path and recompute its front row. Costs the traversal you were avoiding, but only on writes, and only on the writes that actually remove something.
3. **Rebuild everything, off to one side.** Real suggestion systems rebuild the whole structure every few minutes from a fresh dump and swap it in. Nothing is ever updated in place. This is what production systems do, and saying so is worth more than either of the first two.

Saying “the precomputed lists are a cache, so deletion is cache invalidation, and here is how I would handle it” is the sentence that ends this line of questioning well.

## 8 In the interview

### How it gets asked

- “Design autocomplete for a search box.” — the open version, coding or design round.
- “Given a list of products and a search word, return the top three suggestions after each character typed.” — LeetCode 1268, asked at Amazon almost verbatim.
- “Implement a system that suggests the top three historical queries, ranked by how often they were searched.” — LeetCode 642, with # to end an input.
- “You have a trie. Return the five most popular words under a prefix.” — the follow-up to yesterday’s question, and the one that catches people who only prepared insert and search.
- “The suggestions are slow when the user has typed one letter. Why, and what do you do?”

### The first ninety seconds

“The structure is a trie, and the prefix walk is the easy part — one child per character, so typing `ca` costs two steps regardless of dictionary size. The interesting part is what happens after I land on that node.

The straightforward version traverses the subtree, gathers every word underneath, and takes the top five with a bounded heap. That is correct, and I would write it first. It costs the number of words under the prefix, times their average length. The problem is that the cost is worst when the prefix is shortest — one letter into a hundred-thousand-word dictionary means visiting about six thousand words to return five, and the user is typing fastest at exactly that moment.

So the version I would actually ship stores the top five *at every node*, computed when words are inserted. A query becomes: walk the prefix, read a list of five, return. Constant work after the walk.

I pay for that twice. Inserts get about fifteen times more expensive, because every node on the word’s path updates its list. And memory goes up by about seventy megabytes on a hundred-thousand-word dictionary, which I can compute if you want the arithmetic. For a search box, where reads outnumber writes by a thousand to one, that is an easy trade.

The thing I would flag before you ask is that the stored lists are a cache, so deletion is the hard case. Shall I go into that, or would you rather I write the code first?”

## The follow-ups

**“How would you handle typos? The user types `recieve`.”**

“The trie gives me nothing here, because a typo takes me off the path at character four and I return nothing at all — which is worse than returning something imperfect.

Three options, cheapest first. One: fuzzy walk. Allow up to one edit while walking, so at each step I can also try skipping a character, inserting one, or substituting one. That is a depth-first search over the trie with an edit budget, and it multiplies the work by roughly the alphabet size per allowed edit, so I would cap it at one edit and only for prefixes of four characters or more.

Two: precompute. For the few thousand most common queries, store their common misspellings as extra entries pointing at the correct word. This is what most production systems actually do, because typos follow a very short distribution — a handful of mistakes account for most of them.

Three: fall back to a different structure. Run the corrected-spelling path through something built for approximate matching and keep the trie for exact prefixes. I would not build that in the first pass.

In an interview I would say option two out loud first, because it is the one that admits the distribution is skewed, and then offer option one as the general answer.”

**“The weights change constantly. Every search updates a count. Does your pre-computation survive?”**

“Increases survive fine — that is exactly what `add` does, and it costs  $L \times k \log k$  per update. Eight characters and five slots is about a hundred and twenty operations per search, which at ten thousand searches a second is a million and a bit operations a second on one core. Tight but survivable.

What does not survive is anything that lowers a weight or removes a word, because the stored lists then hold entries that should have been replaced by something I would have to traverse to find.

So I would not update in place at all. I would keep raw counts in a separate store, and rebuild the whole ranked trie from scratch every five minutes in a background process, then swap the new one in atomically. Suggestions that are five minutes stale are completely acceptable — nobody notices — and it removes every invalidation problem in one move. The rebuild for a hundred thousand words is well under a second, so I can afford it far more often than I need it.”

**“A hundred million distinct queries, not a hundred thousand words. Does this still fit in memory?”**

“No, and I would say so rather than pretend. A hundred million queries at an average of twenty characters is two billion nodes at the top end; even with prefix sharing you are in the tens of gigabytes, and the per-node dictionary in Python is most of that.

Three moves. First, prune: strip anything searched fewer than, say, five times in the last month. The distribution is brutally skewed, and that usually removes ninety percent of the distinct queries and one percent of the traffic. Second, compress: use a radix trie, where a chain of single-child nodes collapses into one node holding a whole substring, which cuts the node count several-fold on real text. Third, partition: split by first two characters across machines, so one machine holds everything starting with `ca`, and route the request by prefix — the request already tells you where to send it, so there is no lookup step.

Then the front row is per machine, and the memory is per machine.”

**“What do you return for an empty box?”**

“The root’s stored list, which is the globally most popular queries. In the precomputed version I get that for free — the root is on every word’s path, so its front row is already the global top five. In the traversal version I would special-case it, because otherwise an empty prefix means traversing the entire dictionary on every page load.”

#### The model answer

*“Design autocomplete for a search box with a hundred thousand product names. Show three suggestions per keystroke.”*

“Let me set the shape and then go into each piece.

**The structure is a trie over the product names, with a weight on each word.**

Weight is how many times that product was searched, or clicked; I would use clicks rather than searches if I have them, because a search that led nowhere is not a vote for that product.

**The query path is: walk the prefix, then rank what is below it.** The walk is one step per character and is never the bottleneck. Ranking is.

**I would write the traversal version first, to have something correct.** Gather every completion below the node with a depth-first search, keep the best three with a bounded heap so I am doing  $m \log 3$  and not  $m \log m$ . Tie rule stated up front: higher weight first, alphabetically smaller second, so the output is deterministic and testable.

**Then I would show the number that makes me change it.** One letter into a hundred-thousand-name dictionary is about six thousand names under the prefix, eight characters each — around fifty thousand operations to return three strings. Per keystroke. And a hundred thousand names is a small catalogue.

**So: store the top three at every node, maintained on insert.** Query becomes a walk plus a list read. Insert becomes eight nodes each re-sorting a four-item list, which is about a hundred operations. Memory goes up by roughly  $250,000 \text{ nodes} \times 3 \text{ entries} \times 56 \text{ bytes}$ , call it forty megabytes, and those entries are references to strings the trie already holds, so I am not duplicating text.

**The build is offline.** I do not maintain this structure live. Counts accumulate in whatever store already has them; a background job rebuilds the ranked trie every few minutes and swaps it in behind an atomic reference change. That gives me three things at once: no locking on the read path, no cache invalidation problem on delete, and a trivial rollback if a bad build ships.

**What I would give up.** Suggestions lag reality by one build interval. For a product search box that is invisible. If the requirement were ‘a product listed thirty seconds ago must be suggestable’, I would keep a small second trie holding only the last few minutes of new items, query both, and merge the two three-item lists — which costs six comparisons.

**What I would not do in the first version.** Typo tolerance, personalisation, and per-user history. Each is a real feature and each doubles the design. I would name them as the obvious next steps and ask which one you want me to take further.”

## 9 Recall card

**The shape:** walk the prefix, then rank the subtree. The walk is  $O(p)$  and easy; the ranking is the lesson.

**Two versions:** gather-and-rank costs  $O(p + m \cdot L + m \log k)$  per query and is worst on the first keystroke; precomputed top- $k$  at every node costs  $O(p + k)$  per query and  $O(L \cdot k \log k)$  per insert.

**The tie rule, always stated:** higher weight first, then alphabetically smaller. Without it your output is not deterministic.

**The two bugs:** `_collect` must check the node it starts on, or the prefix itself is never suggested; and the empty prefix must be special-cased, or it traverses the whole dictionary.

**The precomputed list is a cache,** so deletion is cache invalidation. Real systems never update it in place — they rebuild off to one side every few minutes and swap it in.



## 1 What this is, and why they ask it

An operation is **idempotent** when doing it twice has the same effect as doing it once. Setting a light switch to “off” is idempotent. Pressing a doorbell is not. Almost every hard bug in a distributed system comes down to something in the second category being retried.

You met retries in passing on [day 121](#), when the saga had to retry a compensation forever. Today is the missing half: what makes it safe to retry anything at all. And the uncomfortable fact underneath it — that when a request times out, you genuinely do not know whether it happened.

They ask this because it is the question that separates people who have run a payment system from people who have read about one. “The payment request timed out. Is it safe to retry?” has a wrong answer that sounds completely reasonable, and the right answer requires you to say out loud that **exactly-once delivery does not exist**. Interviewers at payment companies, at Amazon, and in any round that touches queues will steer towards this. It is also the single most common production incident in the systems you will build: the same thing happened twice and nobody can explain why.

By the end of this lesson you can design an operation that is safe to retry, size the storage that takes, name what a duplicate costs in rupees, and explain the difference between exactly-once *delivery*, which is impossible, and exactly-once *effect*, which is ordinary engineering.

## 2 The story

The gas cylinder ran out on Tuesday evening, halfway through the cooking, so Farhan rang the distributor first thing on Wednesday morning.

The line was bad. He gave his consumer number, all the way to the last digit, and the man on the other end started to read it back — “Nine four three, one...” — and then the call dropped.

Farhan stood there with the phone still against his ear. He did not know whether the booking had gone through. The man had started reading the number back, which meant he had heard it. But had he finished writing it down? Had he pressed whatever it is he presses?

Ringling again felt risky. Two cylinders would turn up, one of which he would have to pay for and had nowhere to keep. Not ringling felt worse. If nothing had gone through, he would find out at eight in the evening when nothing came, and then he would be cooking on the little induction plate for three days.

He waited twenty minutes and rang again. This time he gave the consumer number first, before anything else, before even saying what he wanted.

“Already booked,” the man said, without much interest. “Ten forty-two this morning. It’s on the van.”

That was the whole answer, and it took four seconds.

The consumer number is not only a way of finding Farhan. It is the thing that made the second call safe. Every booking is filed under it, and the man’s screen will not accept two open bookings against the same number on the same day. So ringling back does not book a second cylinder. It just asks, in a roundabout way, whether the first call worked — and it can be asked as many times as you like.

Mrs D’Souza in the flat below learned the same thing from the other direction. Her own phone had no balance, so she booked from her son’s phone, in his name, because that was quicker than explaining. Then the next morning she was not sure it had worked, so she rang from her own phone, under her own number.

Two cylinders arrived on Thursday. She paid for both.

---

### 3 The idea in plain English

Take the phone call apart.

**The dropped call is a timeout.** Farhan’s phone did not tell him “the booking failed”. It told him nothing. That is exactly what a timeout is: your request went out, and no answer came back within the time you were prepared to wait. It does not mean the work did not happen.

**There are three possibilities and you cannot tell them apart.** The request never arrived. The request arrived, was done, and the reply was lost. Or the request arrived and is *still being worked on right now*. From the caller’s side these look identical. This is the most important sentence in the lesson: **a timeout is not a failure; it is an unknown.**

**Retrying is the only sane response, and retrying is dangerous.** Farhan cannot sit and do nothing. Neither can your service. So it retries — and if the first attempt did succeed, the retry is now a second, unwanted request.

**The consumer number is an idempotency key.** An **idempotency key** is a unique label the caller attaches to a logical operation, and keeps the same across every retry of that operation. Not a new one per attempt — that is the whole point. Farhan's second call carried the same consumer number as his first, so the distributor could recognise it as the same request rather than a new one.

**The distributor's screen is a deduplication store.** Before doing the work, the receiver looks up the key. Seen before? Return the same answer as last time, do nothing new. Not seen? Do the work, record the key with the answer.

**Mrs D'Souza shows the failure mode.** She retried with a *different* key — her son's name the first time, her own the second. To the distributor those were two unrelated requests, and both were honoured correctly. The key must identify the operation, not the attempt and not the caller's mood.

**Now the three delivery guarantees, which are just three positions on this problem.**

- **At-most-once:** send it, never retry. You never duplicate. You sometimes lose the request entirely. Fine for a metric sample, catastrophic for a payment.
- **At-least-once:** retry until acknowledged. You never lose the request. You sometimes duplicate. This is what almost every real system does.
- **Exactly-once:** never lost, never duplicated.

**Exactly-once delivery is not achievable, and you should say so plainly.** The reason is simple enough to say in one breath: the last message in any exchange can always be the one that is lost, and the sender cannot tell a lost reply from a lost request. Adding another acknowledgement just moves the problem to that acknowledgement. No number of round trips fixes it.

**What you can have is exactly-once *effect*.** Deliver at least once, and make the processing idempotent, so that duplicates change nothing. The message may arrive five times; the card is charged once. People call this **effectively-once**, and it is what every system that claims exactly-once is actually doing.

**Some operations are idempotent for free.** “Set the status to `PAID`” can be applied a hundred times with the same outcome. “Add ₹500 to the balance” cannot. When you get to choose, prefer the first shape — it is cheaper than any machinery you could build.

---

**4 The picture**

The three outcomes hidden behind one timeout:

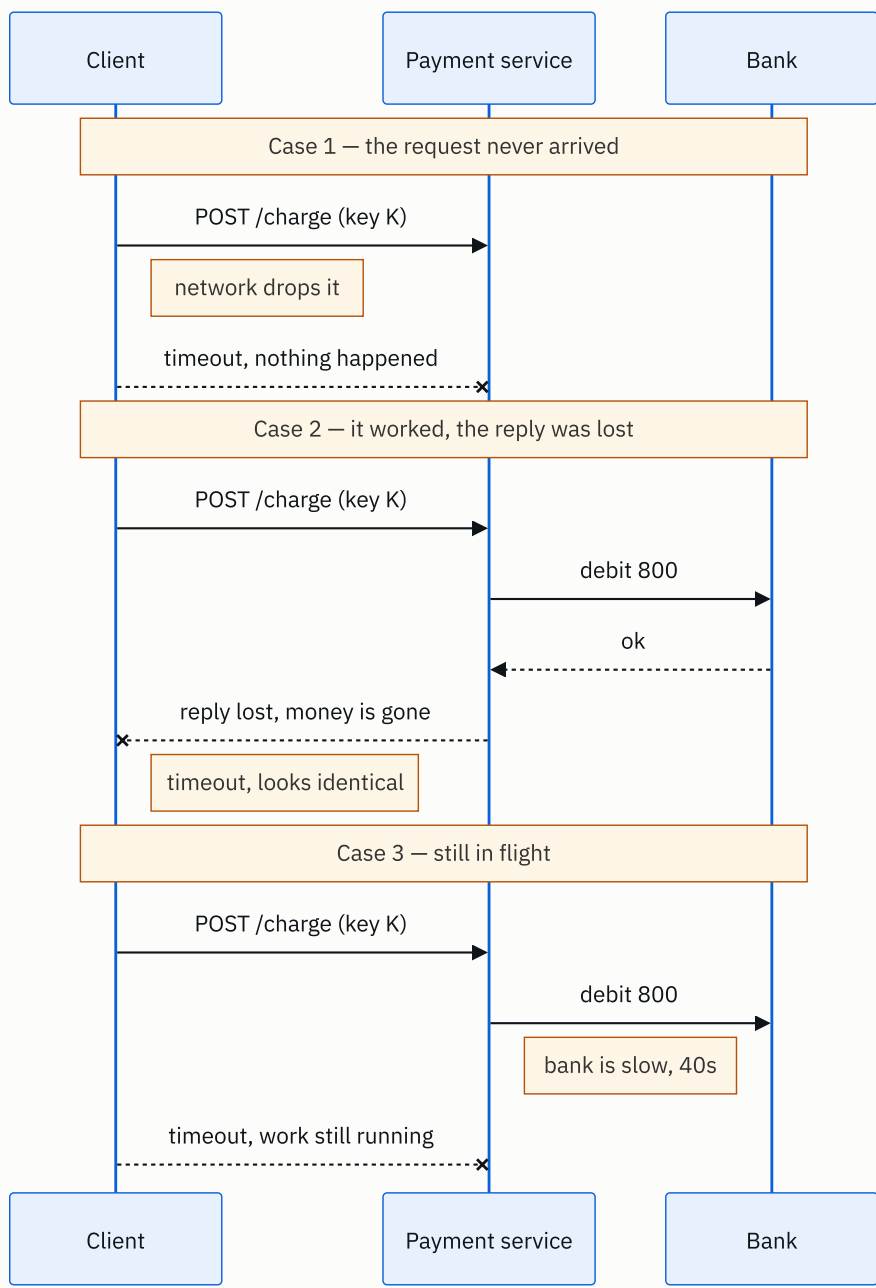


FIGURE 122.2

**What to notice.** The client’s experience is byte-for-byte the same in all three cases. It sees a timeout and nothing else. Any design that says “on timeout, assume it failed” gets case 2 wrong, and case 2 is where the money is.

Now the fix, drawn as the path a retry actually takes:

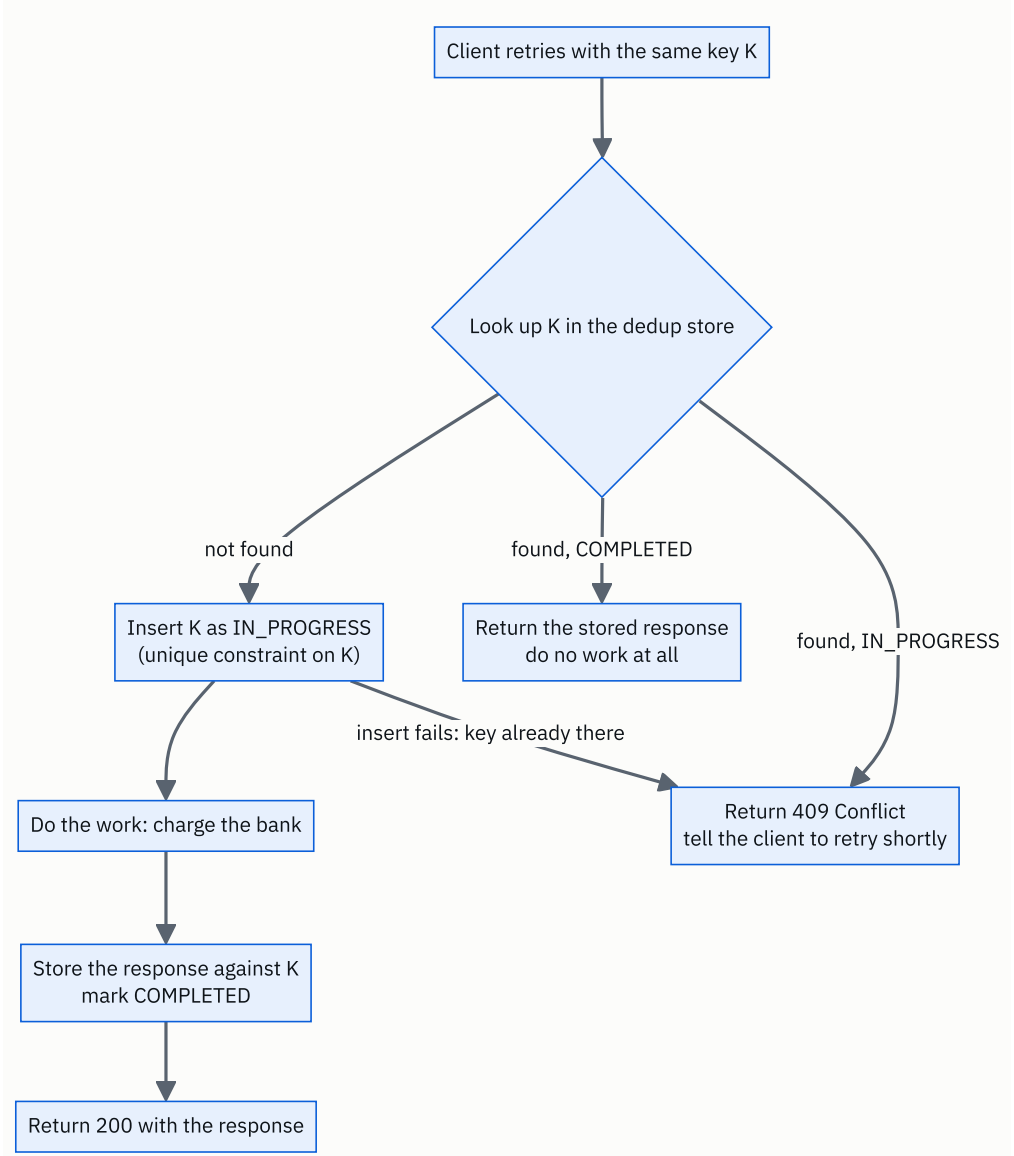


FIGURE 122.3

**What to notice.** There are three branches, not two, and the third is the one people forget. If the first attempt is *still running* when the retry arrives, the key exists but there is no stored response yet. You must not do the work again, and you cannot return the answer, because there is no answer yet. So you refuse and ask the caller to come back. The unique constraint on the key is what makes that safe even when both requests hit different machines at the same millisecond.

## 5 How it actually works

### The key comes from the caller

The receiver cannot generate the key, because two attempts at the same operation would get two different keys and the whole scheme collapses. So the client generates it — a UUID, or a hash of the meaningful fields — once, before the first attempt, and reuses it for every retry of that same operation.

Stripe's API does exactly this, and it is worth naming because interviewers recognise it. You send a header:

```
POST /v1/charges
Idempotency-Key: 3f9a2c10-8e5b-4d1a-9f6c-2b7e5a0d4411
```

Stripe stores the key and the full response for **24 hours**. A retry inside that window gets the original response, including the original charge ID, with no second charge. After 24 hours the key is forgotten, so a retry a week later would charge again — which is why the window has to be longer than any retry policy you allow.

PayPal, Razorpay, Adyen and AWS all have a version of this. AWS calls it a `ClientToken` on several APIs and gives it a similar retention.

### The dedup store is a table with a unique constraint

The whole mechanism, in one table:

```
CREATE TABLE idempotency_keys (
 key TEXT PRIMARY KEY,
 endpoint TEXT NOT NULL,
 request_hash TEXT NOT NULL,
 state TEXT NOT NULL, -- IN_PROGRESS | COMPLETED
 response JSONB,
 created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The order of operations matters, and the order that works is **insert first, then do the work**.

1. `INSERT ... ON CONFLICT DO NOTHING` with state `IN_PROGRESS` . If zero rows were inserted, somebody else owns this key: read the row, and either return its stored response or return `409` .
2. Do the work.
3. `UPDATE` the row with the response and state `COMPLETED` .

Doing the work first and recording the key afterwards is the mistake. Two concurrent retries both look up, both find nothing, and both charge the card. The `PRIMARY KEY` is doing the real work here — it is a lock that costs nothing extra, and it is enforced by the storage engine rather than by your code.

`request_hash` catches a subtler abuse: the same key sent with a *different* body. That means a client bug, and the right response is `422` , not a silent replay of an unrelated charge.

Rows are deleted by a nightly job, or by a TTL if the store is Redis or DynamoDB.

**HTTP already tells you which methods are safe**

This is free marks in an interview.

| METHOD              | IDEMPOTENT? | WHY                                                         |
|---------------------|-------------|-------------------------------------------------------------|
| <code>GET</code>    | Yes         | Reads nothing changes.                                      |
| <code>PUT</code>    | Yes         | “Set the resource to this value” — same value, same result. |
| <code>DELETE</code> | Yes         | Second delete finds nothing to delete; still ends deleted.  |
| <code>POST</code>   | No          | “Create a new one” — twice means two.                       |

So `POST /orders` needs an idempotency key. `PUT /orders/4471` does not, because the caller chose the identifier and repeating the call overwrites with the same content. Letting the client choose the ID and using `PUT` is the cheapest way to make creation idempotent, and it is worth offering as an alternative design.

**Kafka’s version, and what it actually promises**

Kafka is where the phrase “exactly-once” gets thrown around, so know what it means there.

- **The idempotent producer** ( `enable.idempotence=true` ) gives each producer a producer ID and stamps every message with a sequence number per partition. If a producer retries after a lost acknowledgement, the broker sees a sequence number it

has already written and drops it. That removes duplicates caused by producer retries, within one producer session, per partition.

- **Transactions** let a consumer read from one topic, process, and write to another topic *and* commit its own offset in one atomic step. If it crashes halfway, neither the output nor the offset commit is visible.
- Consumers set `isolation.level=read_committed` so they never see the output of an aborted transaction.

What this gives you is exactly-once **within Kafka**. The moment your processing touches anything outside Kafka — charging a card, sending an email, writing to Postgres — the transaction cannot cover it, and you are back to at-least-once plus an idempotency key. Saying that sentence out loud is what shows you understand the boundary rather than the marketing.

#### Making the operation naturally idempotent

Machinery is a last resort. Four ways to avoid needing it:

- **Absolute, not relative.** `SET balance = 4500` is idempotent; `SET balance = balance + 500` is not. Where the domain allows an absolute write, take it.
- **Conditional writes.** DynamoDB's `ConditionExpression: attribute_not_exists(order_id)` makes “create” fail loudly on the second attempt instead of duplicating. Postgres gets the same with a unique index on a business key.
- **State machines.** `UPDATE orders SET state='SHIPPED' WHERE id=? AND state='PAID'` moves the order once and matches zero rows on the retry. The `WHERE` clause is the idempotency.
- **Client-chosen identifiers.** Let the client generate the order ID. Then creation is a `PUT`, and the primary key does your deduplication for you.

#### Where the duplicates actually come from

Not just client retries. In a real system, at least five sources:

1. The client retried on timeout.
2. A load balancer or gateway retried after an upstream timeout.
3. A message queue redelivered because the consumer did not acknowledge in time.
4. The consumer crashed after doing the work and before committing its offset.
5. A human clicked the button twice, or a mobile app resumed and replayed a queued request.



The last one is why the browser’s “resubmit form?” dialogue exists, and it is worth mentioning: **the same defence handles all five**, which is the main argument for putting the key at the edge rather than sprinkling special cases through the code.

## 6 The numbers

Take a payment service handling **ten million payments a day**.

### Baseline load.

|                                 |                                   |
|---------------------------------|-----------------------------------|
| $10,000,000 / 86,400 \text{ s}$ | = 116 payments per second average |
| peak is roughly 5x average      | = 580 per second                  |

**How many requests end in an unknown state.** Assume a conservative 0.1% of calls to the bank time out — network blips, bank slowness, a deploy on either side.

|                           |                                     |
|---------------------------|-------------------------------------|
| $10,000,000 \times 0.001$ | = 10,000 ambiguous requests per day |
|---------------------------|-------------------------------------|

**What they cost if nothing is idempotent.** Suppose the client retries 30% of ambiguous requests, and the original had in fact succeeded 60% of the time — case 2 in the diagram.

|                                       |                                         |
|---------------------------------------|-----------------------------------------|
| $10,000 \times 0.30$ retried          | = 3,000 retries                         |
| $3,000 \times 0.60$ already succeeded | = 1,800 double charges per day          |
| $1,800 \times \text{Rs } 800$ average | = Rs 14,40,000 charged twice, every day |

That is fourteen lakh rupees a day of money you have to find and refund, plus 1,800 support tickets, plus the reputational cost. **This is the number to say in an interview.** It converts “duplicates are bad” into a budget.

### What the fix costs in storage. Per key:

|                         |                         |
|-------------------------|-------------------------|
| key (UUID as text)      | 36 bytes                |
| endpoint + state + hash | ~ 80 bytes              |
| stored response (JSON)  | ~ 500 bytes             |
| row overhead            | ~ 50 bytes              |
|                         | -----                   |
|                         | ~ 670 bytes per request |

|                                       |                       |
|---------------------------------------|-----------------------|
| $10,000,000 \times 670 \text{ bytes}$ | = 6.7 GB per day      |
| 24-hour retention                     | = 6.7 GB steady state |

Under seven gigabytes. That fits in one Redis instance with room to spare, or in one Postgres table with a `created_at` index and a nightly delete. Compare it with fourteen lakh a day and the decision makes itself.

**What the fix costs in latency and load.** One extra round trip to the dedup store on the way in, and one write on the way out:

```
Redis lookup ~ 0.5 ms
Redis write ~ 0.5 ms
added to a payment call ~ 250 ms
 = 0.4% slower
```

```
write load added 580/s x 2 writes = 1,160 extra ops/s at peak
```

Redis does hundreds of thousands of operations a second. This is not a capacity conversation.

**What a retry storm costs, if you get the policy wrong.** Three services deep, each retrying three times:

```
1 user request
x 3 retries at the gateway
x 3 retries at the service
x 3 retries at the payment worker
 = 27 calls to the bank for one payment
```

At 580 requests per second, a bank slowdown turns 580 into 15,660 calls per second aimed at the thing that is already struggling. Idempotency makes those 27 calls *safe*; it does not make them *free*. Bounding them is a separate problem, and it is [day 125](#)'s.

**Kafka's version, sized.** A consumer group processing 50,000 messages a second with a dedup window of five minutes:

```
50,000/s x 300 s = 15,000,000 message IDs in the window
15,000,000 x 40 bytes = 600 MB of dedup state
```

Six hundred megabytes of pure bookkeeping, held in memory, to suppress duplicates for five minutes. That is the real price of “exactly-once”, and it is why the window is always bounded.

## 7 The trade-offs

**You are trading storage and one extra round trip for correctness.** Roughly seven gigabytes and half a millisecond, in the numbers above. There is almost no case where that is the wrong trade for money, orders, or anything a user can see twice. There are cases where it is the wrong trade for high-volume, low-value events — a duplicate page-view metric is not worth 670 bytes of protection.

**The window is finite, and that is a real hole.** Keys expire after 24 hours. A client that retries after 25 hours gets a second charge, correctly according to your design and incorrectly according to the customer. The mitigation is to make the retention longer than the longest retry any client is permitted, and then to enforce that permission. State the window as a number; do not leave it implicit.

**The key is the client's responsibility, and clients get it wrong.** A client that generates a fresh UUID per HTTP attempt has idempotency in name only. A client that reuses one key for two genuinely different payments gets the wrong money moved. Both are common. Defend with the `request_hash` check, and document the rule in one sentence: **one key per logical operation, generated before the first attempt, kept across retries.**

**Concurrent duplicates are harder than sequential ones.** Two retries arriving four milliseconds apart both find no key. Only an atomic insert or a conditional write saves you, and that means the dedup store must be strongly consistent. This rules out an eventually consistent store for the dedup table specifically — [day 115](#)'s eventual consistency is exactly the wrong property here. If the dedup store is Redis, you need one authoritative instance or a proper `SET NX` with a single primary, not a read-replica lookup.

**You have moved the single point of failure.** If the dedup store is down, you must choose: refuse all writes, which is an outage, or process without deduplication, which risks duplicates. Say which one you pick and why. For payments, refuse — a short outage is cheaper than double charges. For a notification service, process — a duplicate push is annoying, a missing one is a broken product.

**Idempotency does not extend past your boundary.** You can make your own writes safe. You cannot un-send the email your retry triggered, or un-fire the webhook. Anything with an external, irreversible effect must sit behind its own key check, and the truly irreversible steps belong last in the sequence — the same ordering rule the saga gave you on [day 121](#).

**When would I not use this?** When the operation is already naturally idempotent. If the endpoint is “set the user’s display name to X”, a retry is harmless and the whole apparatus is dead weight. Reach for the state machine or the conditional write before you reach for a dedup table — those cost nothing and are impossible to get wrong.

---

## 8 In the interview

### How it gets asked

- *“The payment request timed out. Is it safe to retry?”* — the direct version.
- *“How do you guarantee exactly-once processing?”* — a trap, and the answer starts by rejecting the premise.
- *“Your consumer read the message, did the work, and crashed before committing the offset. What now?”*
- *“A user double-clicked the pay button. Design so that only one charge happens.”*
- *“What is the difference between at-least-once and exactly-once?”*
- *“Where does the idempotency key come from, and how long do you keep it?”* — the detail question that finds out whether you have actually built one.

## The first ninety seconds

“The first thing I would say is that the timeout does not tell me the payment failed. It tells me I do not know. There are three possibilities and they look identical from my side: the request never arrived, it arrived and succeeded but the reply was lost, or it is still running at the bank right now. So ‘assume it failed and retry’ is wrong, because in the second case I have just charged the customer twice.

The fix is to make the charge idempotent. The client generates an idempotency key once, before the first attempt — a UUID for that logical payment — and sends it on every retry. My service inserts that key into a dedup table with a unique constraint *before* doing any work. If the insert succeeds, I own the request and I do the work, then store the response against the key. If the insert conflicts, somebody has been here: either I return the stored response, or, if the first attempt is still in flight, I return a 409 and tell the client to come back shortly.

Insert-before-work is the part people get backwards. If you do the work and record the key afterwards, two concurrent retries both see nothing and both charge.

On the phrase exactly-once — I would be direct that exactly-once *delivery* is not achievable, because the last message in any exchange can always be lost and no extra acknowledgement fixes that. What I can build is at-least-once delivery plus idempotent processing, which gives exactly-once *effect*. That is what every system claiming exactly-once is really doing.

Do you want me to size the dedup store, or go into the queue side of it?”

## The follow-ups

“Why can’t you just build exactly-once delivery?”

“Because the sender and the receiver never share knowledge at the same instant. Whatever the last message in the exchange is, it can be lost, and its sender cannot distinguish ‘lost on the way there’ from ‘lost on the way back’. Adding an acknowledgement to the acknowledgement just makes that new message the last one. There is no protocol that terminates.

So I stop trying to fix delivery and fix the effect instead. Deliver at least once — retry until acknowledged, accept duplicates — and make every handler idempotent so duplicates change nothing. The message may arrive five times and the card is debited once. That is achievable, it is what Kafka’s transactional processing is doing behind the phrase, and it is the honest description.”

**“The consumer did the work and crashed before committing its offset. Walk me through it.”**

“The message is redelivered, because from the broker’s point of view it was never acknowledged. That is at-least-once working as designed, and I should not try to prevent it.

What I do instead is make the work idempotent so the redelivery is harmless. Concretely, the message carries a stable ID — the order ID, or a UUID the producer stamped on it. Before processing I do a conditional write: `INSERT ... ON CONFLICT DO NOTHING` on that ID, or in DynamoDB a condition of `attribute_not_exists`. If it conflicts, I have already done this one, so I commit the offset and move on.

The variant worth mentioning is when the work and the offset can live in the same store. If my consumer writes to Postgres, I write the business row and the consumed offset in one transaction. Then the crash either commits both or neither, and the redelivery never even needs the dedup check. That is a genuinely exactly-once pipeline, and it works precisely because there is only one store involved.”

**“Two retries arrive at two different machines at the same millisecond. What happens?”**

“Both look up the key and both find nothing, so a naive check-then-act duplicates. The unique constraint is what saves it: both attempt the insert, exactly one succeeds, and the loser gets a constraint violation rather than a green light. The loser then reads the row — if it is `COMPLETED` it returns the stored response, and if it is still `IN_PROGRESS` it returns a 409 with a `Retry-After` .

That means the dedup store has to be strongly consistent. A read from an eventually consistent replica can return ‘not found’ for a key that was written two milliseconds ago on the primary, and then you have reintroduced the bug you were fixing. So: one authoritative primary for this table, or a conditional write that the storage engine itself serialises.”

**“How long do you keep the keys, and what does that cost?”**

“Twenty-four hours, matching Stripe, and the rule is that the retention must exceed the longest retry window any client is allowed. If a mobile app can queue a request offline for three days and replay it, 24 hours is wrong and I need three days plus a margin.

Cost: about 670 bytes per key including the stored response. At ten million payments a day that is 6.7 GB for a 24-hour window, which is one Redis instance or one modest table. Set against roughly 1,800 double charges a day at an average of ₹800 — fourteen lakh rupees daily — it is not a close decision.

One caveat: I store the *response* as well as the key, not just the key. If I store only the key, the retry knows the work was done but cannot tell the client what the charge ID was, and the client is stuck.”

**“What about the email you already sent?”**

“Idempotency stops at my boundary. I can make my own writes safe; I cannot recall an email or an SMS. Two defences. First, ordering: the irreversible steps go last, so a failure earlier in the sequence never reaches them — the same rule as compensations in a saga. Second, push the key one level further out: the notification service itself takes an idempotency key and refuses to send twice for the same key. That turns the email into an idempotent operation and buys back the safety.

What I would not do is claim the problem away. If the third party has no idempotency support, I say so and put the risk in the design explicitly.”

#### The model answer

*“Design a payment endpoint that a mobile app calls, where the network is unreliable and the user may double-tap the button.”*



“Two different duplicate sources here, and I want to handle both with one mechanism.

**The key is generated on the device, once per payment attempt the user starts.** Not per HTTP request. The app creates a UUID when the payment screen is opened, keeps it in local storage, and sends it on every attempt — the first one, the retry after a timeout, and the one after the app was killed and reopened. A double-tap reuses the same key because it is the same payment; two separate payments of the same amount get two keys because they are two intentions.

**At the service, insert before work.** `INSERT INTO idempotency_keys (key, endpoint, request_hash, state) VALUES (... , 'IN_PROGRESS') ON CONFLICT DO NOTHING`. Zero rows inserted means somebody else owns it. Read the row: `COMPLETED` returns the stored response verbatim, `IN_PROGRESS` returns 409 with a `Retry-After` of two seconds. That branch is what makes concurrent duplicates safe, and it costs one round trip.

**Then the work, then the response is stored against the key** in the same transaction as the business write if they share a store. If they do not, I write the business row and an outbox row in one transaction — the outbox pattern from the saga day — and a publisher moves it on. That keeps ‘the charge happened’ and ‘the world was told’ from diverging.

**The `request_hash` guards against client bugs.** Same key, different amount, means the app is broken. I return 422 rather than silently replaying a charge for a different amount, because the silent version is a support ticket nobody can ever explain.

**Downstream, the bank call carries its own reference.** Most gateways accept a merchant reference and deduplicate on it; I use the same key. If the gateway does not support that, I record the attempt before calling and reconcile against the gateway’s settlement file daily. Reconciliation is not optional here — it is the only thing that catches the cases the key window missed.

**Retention: 24 hours, sized at 6.7 GB for ten million payments a day.** That number is small enough that I would rather over-retain than under-retain, so if the app can replay after being offline for a weekend I would set it to seven days and pay 47 GB for it.

**What I am explicitly not claiming.** This is not exactly-once delivery. The bank may still see two calls; what I guarantee is that the customer is debited once and that a retry is always safe. And if the dedup store is unavailable, I fail the request

rather than process without protection — for payments, a short outage is much cheaper than a double charge, and I would make that policy explicit in the design rather than leave it to whoever writes the error handler.”

## 9 Recall card

**A timeout is not a failure, it is an unknown.** Three indistinguishable cases: never arrived, succeeded but the reply was lost, still running. Assuming failure is how double charges happen.

**Idempotent means doing it twice equals doing it once.** The key is generated by the *client*, once per logical operation, and reused on every retry.

**Insert the key before doing the work**, with a unique constraint, and store the response against it. Check-then-act loses to two concurrent retries; the constraint does not.

**Exactly-once delivery is impossible** — the last message can always be lost. At-least-once delivery plus idempotent processing gives exactly-once *effect*, which is what everyone means.

**The numbers:** ~670 bytes per key, 6.7 GB a day at ten million payments, 24-hour retention, against ~1,800 double charges a day and ₹14 lakh if you skip it.

**DSA topic:** Autocomplete and word dictionaries **System design topic:** Idempotency and exactly-once delivery

Code these, in this order

One rule for the whole set: **write the tie-break rule at the top of the file before you write anything else.** Higher weight first, alphabetically smaller second. Every problem below has a hidden test that depends on it, and every one of them fails silently rather than loudly when you get it wrong.

| # | PROBLEM                           | SOURCE                 | WHAT IT IS REALLY TESTING                                                |
|---|-----------------------------------|------------------------|--------------------------------------------------------------------------|
| 1 | Search Suggestions System         | LeetCode 1268 (Medium) | Walk, gather, rank — the version you write first.                        |
| 2 | Longest Word in Dictionary        | LeetCode 720 (Medium)  | Traversal that stops the moment a node is not a whole word.              |
| 3 | Design Search Autocomplete System | LeetCode 642 (Hard)    | Weights, the <code>#</code> terminator, and state carried between calls. |
| 4 | Top K Frequent Words              | LeetCode 692 (Medium)  | The same tie rule, without a trie, so you can see it alone.              |
| 5 | Replace Words                     | LeetCode 648 (Medium)  | Stopping at the <i>first</i> whole word on the path, not the deepest.    |

On problem 1, solve it twice

Once with a trie, once by sorting the product list and using `bisect` to find the block of names sharing the prefix. Record how many lines each takes. Then say out loud which one you would write in an interview and why — the honest answer is not always the trie.

On problem 3, the `#` is the whole problem

Record what your solution does when `#` arrives and the typed sentence is one that already exists, and when it is brand new. Then break it deliberately: reset the current sentence *before* storing it instead of after, and record which test fails.

### On problem 5, count the visits

Solve it once by checking every dictionary word against every sentence word, and once with a trie that stops at the first root. Count how many character comparisons each version makes on a 1,000-word dictionary. Two numbers, said out loud.

### Then build the precomputed version

Take your problem 1 solution and add a `top` list to every node, maintained on insert. Then:

1. Time 10,000 calls to `suggest("a")` on a 100,000-word dictionary, both versions. Two numbers.
2. Measure the memory of both with `sys.getsizeof` walked over the trie, or `tracemalloc`. Two numbers.
3. Delete the most popular word and print the root's `top` list. Say why it is now wrong.

### The two-versions drill

1. Say what `suggest` costs in version one, with every term named.
2. Say what it costs in version two, and what `add` costs in exchange.
3. Give the crossover: at what read-to-write ratio does version two stop being worth it?
4. Say why the first keystroke is the expensive one.

### The ranking drill

1. State the tie rule in one line.
2. Say what happens to your output if you sort on weight alone.
3. Write the sort key from memory, and say what the minus sign is doing.
4. Say why `nsmallest` with negated weights is correct and `nlargest` without them is not.

### The collect drill

1. Write `_collect` from memory.
2. Say why the check on the current node comes before the loop.
3. Say what `so_far` is and what breaks if you start it as an empty string.
4. Rewrite it with an explicit stack, no recursion.

### The precomputation drill

1. Say which nodes an insert has to update, and why no others.
2. Write `_offer` from memory, including the line that removes the old entry.
3. Say what `suggest("")` returns and why it is free.
4. Say what depth you would stop storing front rows at, and why.

### The staleness drill

1. Say why a deletion cannot be repaired locally.
2. Give the three answers, in the order you would offer them.
3. Say which one real systems use.
4. Say what “swap it in atomically” means in one sentence.

### The break-it drill

Trigger each and record the exact output or error:

1. `_collect` that checks the child instead of the current node, asked for `car`.
  2. `suggest("")` on a 100,000-word dictionary, with no special case.
  3. A heap holding `(weight, node)` tuples where two weights are equal.
  4. `_collect` on a key of 2,000 characters.
  5. `_offer` without the line that removes the existing entry, after `add("car")` twice.
  6. `_collect` called with an empty `so_far` instead of the prefix.
- 

### The definition drill

1. Define idempotent in one sentence, with one example of each kind.
2. Say what a timeout actually tells you, and name the three cases.
3. Say which of the three cases costs money, and why it is invisible to the caller.
4. Define at-most-once, at-least-once and exactly-once in one line each.

### The key drill

1. Say who generates the idempotency key, and when.
2. Say what goes wrong if the receiver generates it.
3. Say what goes wrong if the client generates a fresh one per HTTP attempt.
4. Give the one-sentence rule you would put in the API documentation.

### The ordering drill

1. State the three steps of the insert-before-work sequence.

2. Say what breaks if you do the work first.
3. Say what the unique constraint is doing that your code cannot.
4. Say what you return for `IN_PROGRESS`, and why you cannot return the response.

#### The exactly-once drill

1. Say in two sentences why exactly-once delivery is impossible.
2. Say what exactly-once effect is, and how you build it.
3. Say exactly what Kafka's transactions do guarantee, and where that guarantee stops.
4. Say what makes a read-process-write pipeline into a single store genuinely exactly-once.

#### The natural-idempotency drill

1. Give the absolute and relative versions of a balance update, and say which is safe.
2. Write the state-machine `UPDATE` that is idempotent by construction.
3. Say why `PUT` needs no key and `POST` does.
4. Say what a conditional write buys you over a dedup table.

#### The numbers drill

1. Compute ambiguous requests per day at ten million payments and a 0.1% timeout rate.
2. Compute the daily rupee cost of double charges from those, showing every step.
3. Size the dedup store: bytes per key, per day, at a 24-hour retention.
4. Compute the retry amplification for three tiers retrying three times.
5. Size Kafka's dedup window at 50,000 messages a second over five minutes.

#### The trade-offs drill

1. Say what you give up by adding a dedup store, in latency and in load.
2. Say what happens when the dedup store is down, and which way you fail for payments.
3. Say why the dedup store cannot be eventually consistent.
4. Say what idempotency does *not* protect, with two examples.
5. Name the situation where the whole mechanism is dead weight.

#### The failure drill

For each, say what happens and what you would build:

1. The bank charges the card and the reply is lost.

2. Two retries arrive at two machines four milliseconds apart.
3. A client sends the same key with a different amount.
4. A mobile app replays a queued request 30 hours later.
5. A consumer processes a message and crashes before committing its offset.
6. The dedup store returns “not found” from a stale replica.
7. The retry succeeds, but the first attempt also completes ten seconds later.

Two of the seven are not fixed by an idempotency key at all. Name them.

---

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Design autocomplete for a search box.* Trie plus prefix walk, the traversal version with its cost, the number that makes you change it, the precomputed top-`k` with its memory arithmetic, and the offline rebuild — offered before the interviewer asks about deletion.
  2. *The payment request timed out. Is it safe to retry?* Not as it stands, and say why: three indistinguishable cases, one of which already took the money. Then the idempotency key, insert-before-work with a unique constraint, the three branches including `IN_PROGRESS`, and the retention window as a number.
  3. *How do you guarantee exactly-once processing?* Reject the premise in one sentence, give the reason — the last message can always be lost — then at-least-once plus idempotent handlers, what Kafka actually promises, and the one case that is genuinely exactly-once because only one store is involved.
- 

### Before you move on

- [ ] I can walk to a prefix and gather every completion below it.
- [ ] I can state the tie rule without hesitating.
- [ ] I know `_collect` must check the node it starts on.
- [ ] I know what `so_far` is for and what breaks without it.
- [ ] I can give the cost of the traversal version with every term named.
- [ ] I can say why the first keystroke is the expensive one.
- [ ] I can write `_offer` from memory, including the removal line.

- [ ] I know which nodes an insert must update, and why no others.
- [ ] I can size the memory of the precomputed version out loud.
- [ ] I know the precomputed list is a cache, and deletion is invalidation.
- [ ] I can name the three answers to deletion, and which one production uses.
- [ ] I know what an empty prefix should return.
- [ ] I rewrote `_collect` without recursion.
- [ ] I can define idempotent and give one example of each kind.
- [ ] I can name the three cases a timeout hides.
- [ ] I know the key is the client's, generated once, reused on retries.
- [ ] I can state the insert-before-work order and say what breaks if reversed.
- [ ] I know what the unique constraint is doing for me.
- [ ] I know what to return for a request still in flight.
- [ ] I can say in two sentences why exactly-once delivery is impossible.
- [ ] I can say what Kafka's transactions guarantee, and where that stops.
- [ ] I can name four ways to make an operation naturally idempotent.
- [ ] I can compute the daily cost of double charges from a timeout rate.
- [ ] I can size a dedup store per day at a given volume.
- [ ] I know why the dedup store must be strongly consistent.
- [ ] I know which way I fail when the dedup store is down, and why.
- [ ] I can name two things idempotency does not protect.
- [ ] I answered all three questions above out loud.



# Tries in interviews: word search II

## DATA STRUCTURES AND ALGORITHMS

Tries in interviews: word search II

## SYSTEM DESIGN

Clocks, ordering, and why time is a lie

### 1 What this is, and why they ask it

Word Search II gives you a grid of letters and a list of words, and asks which of those words you can spell by walking from cell to neighbouring cell. It is LeetCode 212, it is marked Hard, and it is the problem the whole trie phase has been building towards.

The reason it is famous is that the obvious solution and the good solution have the same shape. Both do a depth-first search from every cell. The difference is one thing: the good one carries the whole word list *with it* as it walks, so it can stop the moment no word could continue down this path. That single change turns a solution that times out into one that finishes in a fraction of a second.

They ask it because it is the cleanest example in the whole interview canon of **combining two structures to prune a search**. You already know backtracking on a grid from [day 96](#) and you know tries from [day 120](#). Neither alone is interesting. Putting them together is, and the interviewer wants to watch you notice why. Google, Amazon, Microsoft and Uber all ask it, usually as the second question when the first one went well.

By the end of this lesson you can write it from memory, explain in one sentence why searching each word separately is the wrong shape, count the cost of both versions, and name the two optimisations that most candidates miss.

### 2 The story

Balbir has delivered post to the same four buildings for thirty-one years, and Sagar Apartments is the one that taught him everything he knows.

It is seven floors, four flats to a floor, and no lift. Twenty-eight doors, and a staircase that turns twice between every floor.

The way he did it in his first year was one letter at a time. Pick up the top letter, read the flat, climb to that floor, deliver it, come back down, pick up the next. Some mornings he had forty letters for Sagar and he would climb those stairs thirty times. By ten o'clock his knees were finished and he had not started on the other three buildings.

What he does now takes eleven minutes.

He stands at the bottom of the stairs with the whole bundle in his hand and he goes through it once, putting it in order — ground floor first, then first, then second, all the way up. Then he climbs, once, and delivers each floor as he passes it.

But that is only half of what he learned, and the second half is the part he is actually proud of.

At every landing he stops and looks at what is left in his hand. Not at the doors — at the bundle. If there is nothing left for the third floor, he does not walk down the third-floor corridor at all. He does not check the four doors. He does not even look at them. He turns and keeps climbing, and the corridor might as well not exist.

And the bundle keeps getting lighter as he goes, so the higher he climbs, the more corridors he skips. On a Tuesday with only six letters for the building, he walks past five of the seven floors without stepping off the staircase.

The young man who covered for him last summer did it the old way and could not understand why it took him two hours. He was walking every corridor on every floor, knocking to check, then going back to his bundle to see whether anything matched. He had the bundle in one hand the whole time and never once looked at it before walking.

---

### 3 The idea in plain English

Balbir's building is the grid. His bundle is the trie. Take it apart piece by piece.

**The grid is a set of paths.** A **grid** here is a rectangle of letters, and a path is any sequence of neighbouring cells — up, down, left, right, no diagonals — where you never step on the same cell twice within one word. Walking a path spells a string. The question “is **CAT** in the grid” means “is there some path whose letters spell **CAT**”.

**The naive solution searches once per word.** Take the first word. Try starting from all  $M \times N$  cells. From each, walk in all four directions, matching character by character against that word, and back out when it stops matching. Then do the whole thing again for the second word. This is the summer replacement knocking on every door.

It is correct. It is also doing the same walk over and over. If your list has **CAT**, **CAR**, **CARD** and **CARE**, the naive version walks the **C → A** step four separate times.

**The trie carries every word at once.** A **trie** stores all the words in one tree where shared prefixes are shared branches. **CAT**, **CAR**, **CARD** and **CARE** share the path **C → A**. So instead of asking “does this path spell **CAT**”, you ask a much better question: “**is there any word at all that starts with what I have spelled so far?**”

That question is answered by one dictionary lookup. If the current trie node has no child for the letter in the next cell, then no word in the entire list can continue this way, and you stop. This is Balbir looking at the bundle before walking the corridor.

**Walking the grid and walking the trie happen together.** This is the idea to hold on to. At any moment you are standing on a cell *and* on a trie node, and they move in lockstep. Step to a neighbouring cell whose letter is `r` ; move to that node's `r` child. If there is no `r` child, do not take the step at all.

**A word is found when the trie node says so.** Every node that ends a word is marked. When your walk lands on one, you have spelled a complete word and you record it. You keep walking afterwards, because `CAR` being found does not stop `CARD` from being found further along the same path.

**Backtracking is the same as day 94.** Mark the cell as used before you recurse into neighbours, and unmark it after. If you forget the unmark, a cell used by one path is permanently unavailable to every other path, and you silently lose words. This is the bug in this problem.

**The two prunings that matter.** First, once a word is found, remove it from the trie so it cannot be found twice — that also stops you searching for it again on every remaining start cell. Second, when a trie node has no children left and ends no word, it is dead weight, so delete it from its parent. Over a run the trie shrinks, and Balbir's bundle gets lighter as he climbs.

4 The picture

A four-by-four grid and four words.

|        | col 0 | col 1 | col 2 | col 3 |
|--------|-------|-------|-------|-------|
| row 0  | o     | a     | a     | n     |
| row 1  | e     | t     | a     | e     |
| row 2  | i     | h     | k     | r     |
| row 3  | i     | f     | l     | v     |
| words: | oath  | pea   | eat   | rain  |

The trie built from those four words, with the word stored at the ending node:

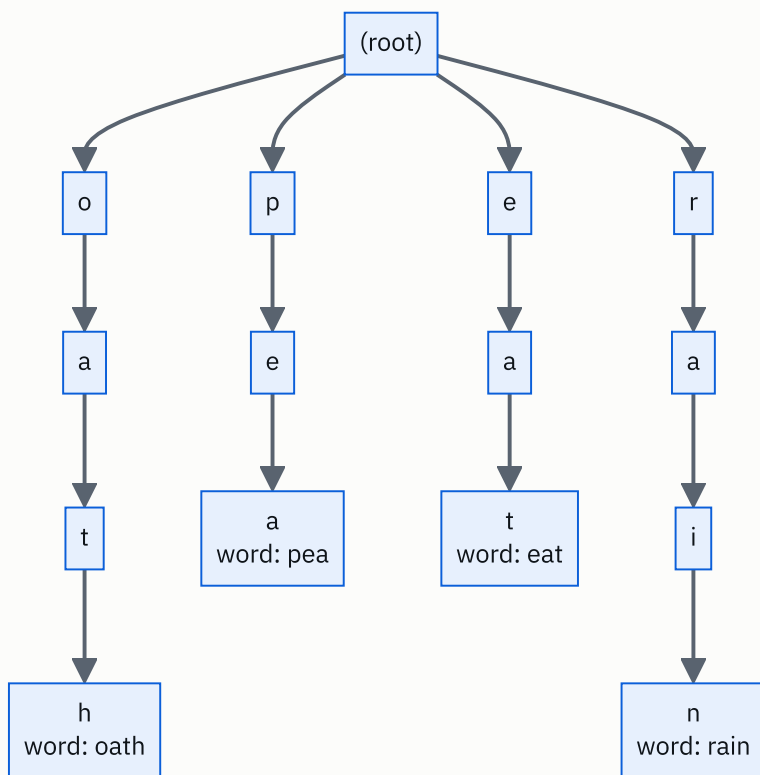


FIGURE 123.1

**What to notice.** The root has four children: `o`, `p`, `e`, `r`. That is the complete list of letters worth starting from. Standing on the `f` at row 3, column 1, you look at the root, find no `f` child, and stop immediately — one dictionary lookup, no walking at all. Twelve of the sixteen cells are eliminated that cheaply.

Now the path that finds `oath`, drawn on the grid:

|       | col 0 | col 1 | col 2 | col 3 |
|-------|-------|-------|-------|-------|
| row 0 | [o] → | [a] → | [a]   | n     |
| row 1 | e     | t     | [t]   | e     |

← wrong: (0,2) is 'a', not 't'

the walk that works:

(0,0) o → (0,1) a → (1,1) t → (2,1) h      spells "oath"  
 root      o              oa              oat → oath \*

**What to notice.** Two positions move together at every step: the cell coordinate and the trie node. When the walk reaches (2,1), the node is oath, which is marked with a word, so oath is recorded. There is no separate “now check if this is a word” pass — the trie already knows.

And the dead end, which is the point of the whole exercise:

```
start at (1,0) 'e' → root has an 'e' child, continue
neighbours of (1,0): (0,0)='o', (2,0)='i', (1,1)='t'
'o' → node 'e' has no 'o' child → stop
'i' → node 'e' has no 'i' child → stop
't' → node 'e' has no 't' child → stop
three lookups, zero recursion
```

**What to notice.** The naive version would have recursed into each of those three neighbours for every one of the four words. The trie version does three dictionary lookups and returns. This is the corridor Balbir never walks down.

## 5 The code, built step by step

Start with the node. One field is different from every trie you have built so far.

```
class Node:
 """A trie node that stores the whole word at its ending position."""
 __slots__ = ("children", "word")

 def __init__(self) → None:
 self.children: dict[str, Node] = {}
 self.word: str | None = None # the full word, not a boolean
```

word instead of is\_end is a small trick with a real payoff. When the search lands on a node that ends a word, you need the word itself to add to the answer. Storing it here means you never have to build the string during the walk — no path + character at every step, no joining a list at the end. It costs one reference per word and saves work on every single step of every walk.

Building the trie is the insert you already know.

```
def build(words: list[str]) → Node:
 root = Node()
 for word in words:
 node = root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.word = word
 return root
```

Nothing new. `setdefault` creates the child if it is missing. The last node gets the whole word written on it.

Now the search from one cell. This is the core, and it is worth reading twice.

```
def explore(row: int, col: int, node: Node) → None:
 character = board[row][col]
 child = node.children.get(character)
 if child is None:
 return # no word continues this way
 if child.word is not None:
 found.append(child.word)
 child.word = None # do not report it twice
```

Three things happen here. The lookup `node.children.get(character)` is the whole optimisation — one dictionary access decides whether this entire branch of the grid is worth exploring. Returning `None` is Balbir turning away from the corridor.

Setting `child.word = None` after recording it is the first pruning. The same word can often be spelled by several different paths, and without this line you would report it once per path. It also means the remaining searches never look for it again.

Then the walk itself.

```
board[row][col] = "#" # mark this cell as in use
for delta_row, delta_col in ((-1, 0), (1, 0), (0, -1), (0, 1)):
 next_row, next_col = row + delta_row, col + delta_col
 if 0 ≤ next_row < rows and 0 ≤ next_col < cols:
 if board[next_row][next_col] ≠ "#":
 explore(next_row, next_col, child)
board[row][col] = character # put it back
```

Overwriting the cell with `#` is how you stop a path from reusing a letter. `#` is never a child of any node, so the check inside the loop is belt and braces — but it makes the intent obvious to a reader, and to an interviewer.

The last line is the one people forget. Restoring the character is what makes this **backtracking** rather than destruction. Without it, the first path through a cell claims it forever, and words that needed it later simply do not appear. There is no error message. The output is just short.

Then the second pruning, which most candidates never mention:

```
if not child.children and child.word is None:
 del node.children[character] # this branch is exhausted
```

Once a node has no children left and no longer ends a word, nothing below it can ever match again. Deleting it from the parent means every future walk fails one step earlier. On a large word list this is the difference between two seconds and two hundred milliseconds, because the trie shrinks as the run proceeds — Balbir’s bundle getting lighter.

Finally, start from everywhere.

```
for row in range(rows):
 for col in range(cols):
 explore(row, col, root)
```

Every cell is a possible first letter, so you try all of them. The root lookup throws out most of them in one step.



## The complete solution

```
"""Word Search II – LeetCode 212. Trie plus backtracking on a grid."""

from __future__ import annotations

class Node:
 """A trie node that stores the whole word at its ending position."""

 __slots__ = ("children", "word")

 def __init__(self) → None:
 self.children: dict[str, Node] = {}
 self.word: str | None = None

class Solution:
 def findWords(self, board: list[list[str]], words: list[str]) → list[str]:
 if not board or not board[0]:
 return []

 root = Node()
 for word in words:
 node = root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.word = word

 rows, cols = len(board), len(board[0])
 found: list[str] = []

 def explore(row: int, col: int, node: Node) → None:
 character = board[row][col]
 child = node.children.get(character)
 if child is None:
 return

 if child.word is not None:
 found.append(child.word)
 child.word = None # each word is reported once

 board[row][col] = "#" # claim the cell for this path
 for delta_row, delta_col in ((-1, 0), (1, 0), (0, -1), (0, 1)):
 next_row, next_col = row + delta_row, col + delta_col
 if 0 ≤ next_row < rows and 0 ≤ next_col < cols:
 if board[next_row][next_col] ≠ "#":
 explore(next_row, next_col, child)
 board[row][col] = character # release it again

 # An exhausted branch can never match: cut it out of the trie.
 if not child.children and child.word is None:
 del node.children[character]

 for row in range(rows):
 for col in range(cols):
 explore(row, col, root)
```

```

 return found

if __name__ == "__main__":
 grid = [
 ["o", "a", "a", "n"],
 ["e", "t", "a", "e"],
 ["i", "h", "k", "r"],
 ["i", "f", "l", "v"],
]
 print(Solution().findWords(grid, ["oath", "pea", "eat", "rain"]))
 print(Solution().findWords(["a"], ["a"]))
 print(Solution().findWords(["a", "b"], ["c", "d"], ["abcb"]))

```

Running it:

```

['oath', 'eat']
['a']
[]

```

The third case is the one to check by hand. `abcb` cannot be spelled, because after `a → b → c` the only `b` is the one you already used, and a cell may not be reused within a word. If your solution returns `['abcb']`, your backtracking is wrong in a way no other test will catch.

## 6 What it costs

Let the grid be  $M \times N$ , the word list have  $W$  words of maximum length  $L$ , and the alphabet have 26 letters.

### The naive version, one search per word.

For each word you start from every cell, and from each cell the walk branches four ways at the first step and three ways afterwards, because you cannot immediately go back where you came from:

|                     |                                                 |
|---------------------|-------------------------------------------------|
| starts              | $M \times N$                                    |
| paths of length $L$ | $4 \times 3^{(L-1)}$                            |
| words               | $W$                                             |
|                     | -----                                           |
| total               | $W \times M \times N \times 4 \times 3^{(L-1)}$ |

Put the LeetCode limits in: a  $12 \times 12$  grid, 3,000 words, maximum length 10.

$$\begin{aligned}
 M \times N &= 144 \\
 4 \times 3^9 &= 4 \times 19,683 = 78,732 \\
 x W = 3,000 &= 144 \times 78,732 \times 3,000 = 34,012,000,000
 \end{aligned}$$

Thirty-four billion steps. That does not finish. This is why the problem is marked Hard — not because the recursion is subtle, but because the obvious version is not a solution.

### The trie version.

The `W` factor disappears entirely, because all the words are searched at once:

|        |                                                                         |
|--------|-------------------------------------------------------------------------|
| starts | $M \times N$                                                            |
| paths  | $4 \times 3^{(L-1)}$                                                    |
| -----  |                                                                         |
| total  | $M \times N \times 4 \times 3^{(L-1)} = 144 \times 78,732 = 11,300,000$ |

Three thousand times less work, and that is before either pruning. `W` **moving out of the multiplication is the whole answer to “why a trie”**. Say it in exactly those words.

That bound is also wildly pessimistic, because the trie cuts a path the moment no word continues. The real number is closer to the number of *prefixes that actually exist*, and on a real word list that is a few hundred thousand steps, not eleven million. Empirically the accepted solution runs in a few hundred milliseconds.

### Building the trie.

$$W \text{ words} \times L \text{ characters} = 3,000 \times 10 = 30,000 \text{ insert steps}$$

Negligible, and paid once.

### Space.

|                 |                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------|
| trie nodes      | $W \times L = 3,000 \times 10 = 30,000$ nodes worst case<br>fewer in practice, since prefixes are shared |
| recursion depth | at most $L = 10$ frames, plus a few                                                                      |
| board mutation  | 0 extra — the “#” is written in place                                                                    |
| output          | at most $W$ strings                                                                                      |

So  $O(W \times L)$  extra space for the trie and  $O(L)$  for the call stack. Writing `#` into the board instead of keeping a separate `visited` grid saves  $M \times N$  booleans and, more importantly, removes an entire parameter from the recursion. Mention that you are mutating the input and restoring it — some interviewers care, and the alternative is one line.

**What the two prunings buy.** Setting `word = None` after a find prevents duplicate reports and stops the search re-finding the same word from a hundred different start cells. Deleting exhausted nodes shrinks the trie as the run proceeds. On the standard test set with 3,000 words, adding the node deletion typically cuts the runtime by half again. Neither is required for correctness of the first, and both are what a strong answer includes.

## 7 The traps

### The cell you never put back

The near-miss:

```
board[row][col] = "#"
for delta_row, delta_col in ((-1, 0), (1, 0), (0, -1), (0, 1)):
 ...
 explore(next_row, next_col, child)
board[row][col] = character ← left out
```

It looks harmless, because within a single path you *do* want the cell to stay claimed. The damage happens after the path returns. On the sample grid:

```
>>> Solution().findWords(grid, ["oath", "eat"])
['oath']
```

`eat` is missing. The search for `oath` walked through `(1,1)`, left a `#` there, and `eat` needed that same cell. No error, no warning, just a shorter list. **This is the bug in this problem**, and the way to catch it is to run a case where two answers share a cell.

### Finding the same word many times

Without `child.word = None`:

```
>>> Solution().findWords([["a","a"],["a","a"]], ["aa"])
['aa', 'aa', 'aa', 'aa', 'aa', 'aa', 'aa', 'aa']
```

Eight paths spell `aa` on a two-by-two grid of `a` s. The judge expects one. Clearing the word after the first find is one line and fixes it exactly, without needing a set — and it doubles as the pruning that stops you searching for a word you have already got.

Using a `set` for `found` instead also produces the right output, and is what most people reach for. It is worse: it hides the fact that you are still doing all that redundant work. Say that out loud if you use it.

### Checking membership per cell

The version that “uses a trie” but has not understood it:

```
for word in words:
 for row in range(rows):
 for col in range(cols):
 if search_grid_for(word, row, col):
 found.append(word)
```

There is a trie in the file somewhere, but the loop over words is still on the outside, so `W` is still in the multiplication. The whole point is that the word list is *inside* the walk, not outside it. On the 3,000-word test case this times out:

```
Time Limit Exceeded
Last executed input:
board = [["a","b","c",...]] (12 x 12)
words = ["aaaaa","aaaab",... 3000 words]
```

### The grid of one letter

A grid entirely of `a` with a word list of long strings of `a` is the classic worst case, and it is on the judge deliberately. Every cell matches, nothing prunes, and the exponent bites:

```
board = 12 x 12 all "a"
words = ["aaaaaaaaaaa"]
```

The node-deletion pruning is what saves this: once the single word is found, its whole branch is deleted, the root has no children left, and every remaining start cell returns after one failed lookup. Without the deletion this case is where solutions time out even *with* a trie. It is worth naming this test in the interview — it shows you thought about the adversary.

### Recursion depth on a big grid

Long words on a large grid:

```
Traceback (most recent call last):
 File "words.py", line 34, in explore
 explore(next_row, next_col, child)
 [Previous line repeated 992 more times]
RecursionError: maximum recursion depth exceeded while calling a Python object
```

On LeetCode's limits this cannot happen — words are at most 10 characters, so the depth is at most 10. It happens the moment someone reuses this code without a word-length limit, because the recursion is bounded by the *trie depth*, and if you accidentally search a trie built from very long strings, you inherit their length. Knowing which bound applies is the answer here, not raising the limit.

### Mutating the caller's board

`board[row][col] = "#"` writes into the list the caller handed you. The code restores it, so the board is correct when the function returns — but it is not correct *during* the call, which matters if anything else is looking, and it means the function cannot take an immutable grid. If asked, say: “I mutate and restore for speed and to avoid a second grid; if you want it pure, I keep a `visited` set of coordinates and pass it down, at the cost of  $M \times N$  extra memory and one more parameter.”

## 8 In the interview

### How it gets asked

- “Given a board of characters and a list of words, find all words that can be formed.” — LeetCode 212, the standard phrasing.
- “You already solved Word Search for one word. Now here are three thousand words.” — the two-part version, and the second part is the whole interview.
- “Find all dictionary words in this letter grid.” — the Boggle phrasing.
- “Your solution times out. What would you change?” — asked after you give the naive answer, which is fine to give first as long as you know it is the naive answer.

## The first ninety seconds

“The single-word version is a straightforward backtracking search: start from every cell, walk to neighbours, match characters, restore the cell on the way out. With three thousand words, running that three thousand times is thirty-four billion steps by my count, and it will not finish.

The observation that fixes it is that those three thousand searches repeat enormous amounts of work. Anything starting `c-a` gets walked once per word beginning with `c-a`. So instead of asking ‘does this path spell CAT’, I want to ask ‘does any word in my list start with what I have spelled so far’ — and a trie answers that in one dictionary lookup.

So: build a trie of all the words up front. Then do exactly one grid search, and carry the current trie node alongside the current cell. They move together — step to a neighbour with letter `r`, move to the node’s `r` child. If there is no `r` child, no word can continue this way and I abandon the branch immediately, without recursing at all.

I store the whole word on the node that ends it rather than a boolean, so when I land on it I can append it without having built the string on the way down.

Two prunings I would put in from the start. After finding a word I clear it from the node, which stops duplicate reports and stops me finding it again from other start cells. And when a node has no children and ends no word, I delete it from its parent, so the trie shrinks as the run goes on. On the all-`a` worst case that second one is the difference between passing and timing out.

Shall I write it?”

## The follow-ups

“What is the actual complexity?”

“ $M \times N \times 4 \times 3^{(L-1)}$  in the worst case, where  $L$  is the length of the longest word. Every cell is a start; from a start you branch four ways, and after that three, because you never immediately step back onto the cell you came from.

The important part is what is *not* in that expression:  $W$ , the number of words. The naive version is that whole thing multiplied by  $W$ . Moving  $W$  out of the multiplication is the entire value of the trie, and on the LeetCode limits it is a factor of three thousand.

I would also say that the bound is very loose. The real cost is bounded by the number of prefixes that exist in the word list, because the trie stops me the instant a prefix does not exist. That is why the accepted solution runs in a couple of hundred milliseconds instead of anything like eleven million steps.

Space is  $O(W \times L)$  for the trie, plus  $O(L)$  for the recursion. I mark cells in the board itself rather than keeping a visited grid, so there is no extra  $M \times N$ .”

“Why store the word on the node instead of `is_end`?”

“Because otherwise I have to build the string as I walk — `path + character` at every step, or a list I join when I find something. Both cost work on every step of every path, and the vast majority of paths never find anything, so it is work spent on nothing.

Storing the word costs one reference on the nodes that end words, which is  $W$  references total, and makes the find a single append. It also makes clearing the word after a find trivial, which is the deduplication.

It is a small thing, but it is the kind of small thing that separates someone who has written this from someone who has read it.”

“How would you handle a grid of a thousand by a thousand?”



“The exponent is in the word length, not the grid size, so a large grid multiplies the number of starts rather than the cost per start. A million cells at, realistically, a few dozen steps each after pruning is tens of millions of operations — big, but linear in the grid.

Two things I would add. First, a cheap frequency filter: count the letters in the grid once, and drop any word that needs more of some letter than the grid contains. On a big grid with a big dictionary that removes a lot of words for free, before any searching. Second, parallelism — the starts are completely independent, so I can partition the cells across workers. The only shared state is the trie, and it is read mostly; if I want the node-deletion pruning I either give each worker its own trie or drop that optimisation, and I would measure before choosing.”

**“The words are a million long and the grid is small. Does the trie still win?”**

“It wins by more, not less. The trie’s cost does not grow with the word count on the search side at all — only the build does, at  $W \times L$ , which is a million times ten, so ten million insert steps and a lot of memory. Meanwhile the naive version would be multiplying its search by a million.

The pressure moves to memory. A million ten-letter words is up to ten million nodes, and a Python dictionary per node is heavy — realistically a gigabyte or more. If that is a problem I would use a fixed 26-slot list per node, or better, a radix trie that collapses single-child chains, which on English cuts the node count several-fold.

I would also point out that with a million words and a  $12 \times 12$  grid, almost every prefix of length two or three exists, so the trie prunes much later than usual. The build cost is real and the pruning is weaker. That is the case where I would put the letter-frequency filter in first.”

**The model answer**

*“Here is a  $12 \times 12$  board and a list of 3,000 words. Return every word that appears.”*

“Let me name the shape before I write anything.

**This is backtracking on a grid, plus a dictionary that tells me when to stop.**

The backtracking half I have done before: from a cell, try four neighbours, mark the cell used, unmark on the way out. The half that makes this problem interesting is the stopping rule.

**The naive version is one grid search per word, and I want to say why it fails rather than just avoid it.** Three thousand words times a hundred and forty-four starting cells times about seventy-eight thousand paths is thirty-four billion steps. It is not slow, it is impossible.

**So I build a trie over all three thousand words first,** costing thirty thousand insert steps, and I do one grid search carrying a trie node beside the cell. If the node has no child for the next letter, I return without recursing. Every path is cut at the first character that no word contains.

**On each node that ends a word I store the word itself,** not a flag, so finding is an append with no string building on the way down.

**Two prunings, both one line.** Clear the word after finding it: that deduplicates and stops the search looking for it again. Delete a node from its parent once it has no children and ends no word: the trie shrinks as the run goes on, and on the pathological all-same-letter grid this is what stops it timing out.

**Cost:**  $M \times N \times 4 \times 3^{(L-1)}$  time, with  $W$  gone from the multiplication, and  $O(W \times L)$  space for the trie. In practice a few hundred milliseconds.

**The bug I am watching for as I write it** is forgetting to restore the cell after the loop. It produces no error — the answer is just missing words that needed a cell an earlier path used. So I will write the restore line at the same time as the mark line, and I will test with a grid where two answers overlap.

**If you want it non-destructive,** I swap the in-place  $\#$  for a set of visited coordinates: one more parameter and  $M \times N$  memory. I would default to the in-place version and mention the trade rather than silently mutate your input.”

---

## 9 Recall card

**The one sentence:** put the word list *inside* the grid walk instead of outside it, so  $W$  leaves the multiplication entirely.

**Walk two things at once** — a cell and a trie node. No child for the next letter means no word continues, so return without recursing.

**Store the word on the node, not a boolean.** Finding becomes an append, with no string built on the way down.

**Two prunings:** clear `node.word` after a find (deduplicates and stops re-finding); delete a childless, wordless node from its parent (the trie shrinks as you go, and this is what survives the all-`a` grid).

**The bug:** forgetting `board[row][col] = character` after the loop. No error — just missing words. Write the restore at the same moment you write the mark.

## 1 What this is, and why they ask it

Every machine has a clock, every machine's clock is wrong, and they are all wrong by different amounts. That is the whole problem. When two machines each write down "this happened at 10:42:31.204", you cannot conclude anything about which one happened first.

This matters because an enormous number of designs quietly assume you can. Last-write-wins conflict resolution assumes it. Expiry checks assume it. "Sort the events by timestamp" assumes it. Every one of those is a bug waiting for a clock to drift.

They ask this because it is the fastest way to find out whether you have thought about distributed systems or only read the vocabulary. "Two events have the same timestamp. Which happened first?" has an answer that most candidates get wrong in a specific, revealing way: they reach for a better clock. The right answer is that you stop asking clocks and start tracking **causality** — what a machine had already seen when it acted. That is the idea Leslie Lamport published in 1978 and it is still the answer.

By the end of this lesson you can say why timestamps do not order events, define happens-before, use Lamport clocks and vector clocks and say which one you need, explain what Google's TrueTime actually buys and what it costs in milliseconds, and name the situations where last-write-wins silently destroys data.

## 2 The story

The printing workshop on the corner opens at seven, and there are two people with keys.

Sunita has had hers for nine years. Deepak got his in March. The arrangement is that whoever arrives first opens the shutter, switches on the big machine so it can warm up, and puts the kettle on.

On Monday they had an argument about it.

Deepak said he had opened up at ten to seven. Sunita said that was not possible, because she had opened up at five to seven and the shutter was already down when she got there, so if he had come at ten to, where was he?

The trouble is the clocks. The big round one on the workshop wall runs fast. Everybody knows it runs fast; nobody knows by how much, and the last time anyone corrected it was before Diwali. Deepak goes by the small display on his scooter, which is slow, and which he reset badly the last time the battery came out. Sunita goes by her phone.

So Deepak's "ten to seven" and Sunita's "five to seven" are not two facts about the same morning. They are two facts about two different instruments, and neither instrument is telling the truth.

They went round it for ten minutes and got nowhere, because there is nothing to get. Two numbers that came from two wrong clocks cannot be compared, however confidently either person says them.

Then Rekha, who does the binding, said the thing that ended it.

"Deepak, was the kettle warm when you got here?"

He thought about it. It was, actually. He remembered noticing, because he did not have to wait for it.

"Then she was here first," Rekha said. "You saw something she had already done."

Nobody had to know what time it was. Nobody had to fix the wall clock. One person had seen the result of the other person's work, and that settled the order completely — it is not possible to see the kettle already warm before somebody has put it on.

Rekha went back to her binding. The wall clock is still fast.

---

### 3 The idea in plain English

Rekha solved in one sentence what two clocks could not.

**Wall-clock time is a measurement, not a fact.** The **wall clock** on a machine — the one that says `2026-09-01 10:42:31` — is a physical device, usually a cheap crystal, that drifts. Left alone it typically gains or loses somewhere between a few seconds and a minute a month. Two machines that were set correctly last week are not set correctly now, and they are wrong in different directions.

**Clock skew is the gap between two clocks at the same instant.** Inside a well-run data centre, machines synchronise against a time source and skew is usually under a few milliseconds. Across the internet, tens to hundreds of milliseconds. It is never zero,

and there is no way to make it zero, because the message that carries the correction takes time to arrive and you do not know exactly how much.

**So a timestamp cannot order two events on two machines.** If machine A stamps `10:42:31.100` and machine B stamps `10:42:31.050`, and the two clocks might be 80 milliseconds apart, then B's event might have happened 30 milliseconds *after* A's. The timestamps say the opposite. This is Deepak's scooter against Sunita's phone.

**Worse, the clock can go backwards.** When the correction arrives and the machine is running fast, the time is pulled back. An event at `10:42:31.200` can be followed by an event at `10:42:31.150` on the same machine. Any code that assumed time only increases is now wrong. This is why every language has a second, separate clock — a **monotonic clock**, which only ever counts forwards and is meaningless as a date, and which is the only correct thing to use for measuring how long something took.

**The kettle is causality.** Rekha did not compare clocks. She found a case where one person *observed the result* of the other's action. That is a fact about the world, not about any instrument, and it cannot be wrong.

**That is the happens-before relation.** Event A **happens-before** event B, written `A → B`, when one of three things is true:

1. A and B are on the same machine and A came first in its own sequence.
2. A is the sending of a message and B is the receiving of that same message.
3. There is a chain: `A → C` and `C → B`.

Nothing in that definition mentions a clock. It is built entirely out of “this machine did this, then that” and “this message went from here to there”.

**When neither happens-before the other, they are concurrent.** Written `A || B`. Concurrent does not mean “at the same time” — it means **no information flowed between them**, so nothing in the system can put them in an order, and you should stop trying. Two people editing different paragraphs of the same document at opposite ends of the world are concurrent, and that is a fact about the system, not about their watches.

**A Lamport clock is a counter that respects causality.** Each machine keeps one number. Increment it before every event. Send it with every message. On receiving a message, set your number to `max(mine, theirs) + 1`. The result has one guarantee, and exactly one: if `A → B`, then A's number is smaller than B's. It gives you a **total order** — sort by the number, break ties by machine name — that never contradicts causality.

**But a Lamport clock cannot tell you the opposite.** A smaller number does *not* mean it happened before. It might just be concurrent. If you need to detect concurrency — “did these two writes conflict, or did one see the other?” — one counter is not enough.

**A vector clock is one counter per machine.** Each machine keeps a list: what it believes every machine’s counter to be. Increment your own entry on every event; send the whole list with every message; on receipt, take the element-wise maximum and then increment your own. Now you can compare two vectors:

- Every entry of  $A$  is  $\leq$  the matching entry of  $B$ , and at least one is strictly smaller  $\rightarrow A \rightarrow B$ .
- Same the other way  $\rightarrow B \rightarrow A$ .
- Neither  $\rightarrow$  **concurrent**, and the system must either merge them or hand them to a human.

That last line is the payoff. **A vector clock is the machinery that lets a system say “these two writes genuinely conflict” instead of silently keeping one.**

#### 4 The picture

Three machines, five events, and the message arrows that create the ordering.

## Syntax error in text mermaid version 10.9.1

FIGURE 123.2

**What to notice.**  $e1 \rightarrow e2$  because a message went from A to B.  $e2 \rightarrow e4$  for the same reason. So  $e1 \rightarrow e4$  by the chain rule. But  $e3$  and  $e1$  have no path between them in either direction: no message went from one to the other before both had happened. They are **concurrent**, and no clock reading can change that. When  $e4$  arrives at C, C is holding two values that nothing in the system can order.

Now the same three machines with the wall clocks written in, to show why the timestamps are useless:

| real time | what each machine's clock says |             |                |           |
|-----------|--------------------------------|-------------|----------------|-----------|
|           | A (+40ms fast)                 | B (on time) | C (-70ms slow) |           |
| 10:00.000 | 10:00.040                      | 10:00.000   | 09:59.930      | ← e1 on A |
| 10:00.020 | 10:00.060                      | 10:00.020   | 09:59.950      | ← e3 on C |
| 10:00.030 | 10:00.070                      | 10:00.030   | 09:59.960      | ← e2 on B |

stamps recorded: e1 = 10:00.040 e3 = 09:59.950 e2 = 10:00.030

sort by timestamp: e3, e2, e1 ← completely wrong

truth: e1, e3, e2

**What to notice.** Sorting by the recorded timestamps puts e3 first and e1 last, which is the exact reverse of what happened. The clocks are only 110 milliseconds apart in total, which is a perfectly ordinary amount of skew across regions, and the events are 30 milliseconds apart, which is a perfectly ordinary gap between related writes. **Skew larger than the gap between events is the whole failure, and it is normal.**

And here is the vector clock doing what timestamps cannot:

|         | A       | B       | C       |                        |
|---------|---------|---------|---------|------------------------|
| start   | [0,0,0] | [0,0,0] | [0,0,0] |                        |
| e1 on A | [1,0,0] |         |         | A writes x=1           |
| e3 on C |         |         | [0,0,1] | C writes x=9           |
| e2 on B |         | [1,1,0] |         | B receives A's message |
| e4 on C |         |         | [1,1,2] | C receives B's message |

compare e3 [0,0,1] with e1 [1,0,0]:

$0 \leq 1$  but  $1 > 0 \rightarrow$  not e3  $\rightarrow$  e1

$1 \leq 0$  is false  $\rightarrow$  not e1  $\rightarrow$  e3

$\Rightarrow$  CONCURRENT. Two real values. The system must decide.

**What to notice.** The vector does not tell you which value to keep. It tells you, correctly and without guessing, that there is a decision to make. Last-write-wins would have thrown one of them away and told nobody.

## 5 How it actually works

**NTP**, and what it can and cannot give you

**NTP** — the Network Time Protocol — is how machines fix their clocks. A machine asks a time source what time it is, measures the round trip, assumes the delay was symmetric, and adjusts. It runs continuously and corrects by small amounts.

What it achieves in practice:



- Within one data centre, against a local stratum-1 source: **0.1 to 5 milliseconds** of skew.
- Over the public internet: **10 to 100 milliseconds**, worse on congested or asymmetric paths.
- A machine that has lost its time source for a day: **seconds**, drifting steadily.

Two properties matter more than the numbers. First, NTP will **step the clock backwards** if it is far enough ahead, so wall-clock time is not monotonic. (`ntpd` slews small corrections gradually and steps large ones; either way you cannot rely on the wall clock only increasing.) Second, NTP gives you no bound you can program against — it does not hand you an error bar, so you never know how wrong you are.

Leap seconds are the famous edge. In 2012 a leap second caused a live-lock in the Linux kernel’s timer code that took down Reddit, Mozilla and several airlines’ booking systems on the same night. Google’s response was **leap smear** — spreading the extra second across 24 hours as a tiny slowdown, so no clock ever repeats or jumps. AWS and Meta do the same now. This is worth knowing because it is a memorable, concrete example of “time is not a reliable input”.

**Monotonic clocks, and the bug everyone writes**

Two clocks exist on every machine and they are for different jobs:

|          | WALL CLOCK                     | MONOTONIC CLOCK                            |
|----------|--------------------------------|--------------------------------------------|
| Python   | <code>time.time()</code>       | <code>time.monotonic()</code>              |
| Means    | Date and time of day           | Seconds since some arbitrary point         |
| Can jump | Yes, forwards and backwards    | No, only increases                         |
| Use for  | Timestamps, expiry dates, logs | Measuring durations, timeouts, rate limits |

The bug: measuring elapsed time with the wall clock. An NTP correction of  $-200$  milliseconds during a measurement yields a negative duration, and code that assumed durations are positive does something strange — a timeout that never fires, a rate limiter that lets everything through, a lock lease that appears to have expired the moment it was taken. Use the monotonic clock for every interval and this class of bug disappears.

**Lamport clocks in practice**

A counter per machine, incremented per event, maxed on receipt. Cheap: eight bytes on the wire, one integer in memory.

Where you actually see it: any system that needs a stable, agreed order but does not need to detect conflicts. Kafka's per-partition offsets are a degenerate case — a single counter defining order within a partition, which is exactly why Kafka can only promise ordering *within* a partition and not across topics. The [replicated log](#) behind Raft is the same idea: the index in the log is the order, and no clock is consulted.

### Vector clocks in practice

One counter per machine, so the metadata grows with the cluster. Amazon's original Dynamo paper used vector clocks and exposed conflicts to the application, which then merged them — the famous shopping-cart example, where merging two carts means taking the union, so a concurrent “add socks” and “add shoes” gives you both. Riak did the same. CouchDB keeps conflicting revisions and makes you pick.

Cassandra explicitly does **not**. Cassandra uses last-write-wins on wall-clock timestamps, and its own documentation warns you about the consequence: two writes within the clock skew window, and one silently disappears. This is a genuine trade — Cassandra chose simplicity and constant-size metadata — but it means **Cassandra can lose writes on a clock problem, by design**. Say that in an interview and you will be taken seriously.

The practical problem with vector clocks is that the vector grows. A cluster of 500 nodes means 500 entries of roughly 16 bytes each, or 8 KB of metadata on every value. The standard fixes are to attach entries only for *client sessions* rather than nodes, and to truncate the oldest entries with a timestamp, accepting a small chance of a false conflict.

### Hybrid logical clocks

An HLC packs a physical time and a logical counter into one value. It behaves like a wall-clock timestamp — you can read it, sort it, and compare it to a real date — while guaranteeing that causally ordered events get increasing values, because the logical part increments when the physical part has not moved enough.

CockroachDB, YugabyteDB and MongoDB (as `clusterTime`) all use HLCs. This is the modern default: you get something close to a real timestamp *and* the causality guarantee, with 8 to 16 bytes of metadata instead of a vector.

### TrueTime, and buying certainty with hardware

Google's Spanner takes the other route. Every data centre has GPS receivers and atomic clocks, and the TrueTime API does not return a time — it returns an **interval**: “now is somewhere between `earliest` and `latest`”. The width of that interval, called epsilon, is typically **1 to 7 milliseconds**.

Spanner then does something blunt and effective. To commit a transaction, it picks a timestamp, and then **waits until the interval has definitely passed** before making the commit visible. That is **commit-wait**, and it costs about  $2 \times \epsilon$ , so roughly **10 milliseconds added to every write transaction**.

What you buy for those 10 milliseconds is enormous: timestamps that can be compared *globally*, so Spanner can offer externally consistent transactions across continents. What you pay is 10 milliseconds and a fleet of atomic clocks. AWS now offers a similar bounded-error clock service, so the technique is no longer unique to Google — but the trade is the same, and it is the cleanest example in the whole field of **spending latency to buy certainty about time**.

## 6 The numbers

**How wrong is a clock?** Start with the raw drift of an ordinary crystal oscillator:

|               |                                                                           |
|---------------|---------------------------------------------------------------------------|
| typical drift | ~ 30 parts per million                                                    |
| per day       | $30 \times 10^{-6} \times 86,400 \text{ s} = 2.6 \text{ seconds per day}$ |
| per month     | ~ 78 seconds                                                              |

So an unsynchronised machine is over a minute out within a month. That is why NTP runs continuously.

### With NTP running:

|                                 |                     |
|---------------------------------|---------------------|
| same data centre, local source  | 0.1 - 5 ms          |
| across regions, public internet | 10 - 100 ms         |
| source unreachable for 24 h     | ~ 2.6 s and growing |

**When does skew actually break ordering?** It breaks whenever skew exceeds the gap between two events you want to order. Take a replicated write:

|                                                                                                 |         |
|-------------------------------------------------------------------------------------------------|---------|
| write on node A, replicate to node B                                                            |         |
| network hop between regions                                                                     | ~ 40 ms |
| clock skew between regions                                                                      | ~ 50 ms |
| 50 ms skew > 40 ms gap → B's timestamp can be EARLIER than A's for an event that happened later |         |

Now put traffic on it. Ten thousand writes a second to one hot key:

```
10,000 writes/s → one write every 0.1 ms
skew → 50 ms
writes inside the skew window: 50 / 0.1 = 500 writes
```

**Five hundred writes are unorderable by timestamp.** Under last-write-wins, any of those 500 could win, and 499 vanish. This is not a rare race; at that rate it is continuous.

**What last-write-wins costs.** Take a system doing 1,000 writes a second where 0.5% of writes touch a key another node wrote within the skew window:

```
1,000/s x 0.005 = 5 conflicting writes per second
x 86,400 = 432,000 conflicts per day
```

With last-write-wins, roughly half of those discard the value that should have survived — **on the order of 200,000 silently lost writes a day.** With vector clocks, all 432,000 are surfaced as conflicts for the application to merge. The second number is much more alarming to look at and much less alarming to live with.

**What metadata costs.**

```
Lamport clock 8 bytes per value
HLC 8 - 16 bytes per value
vector clock, 10 nodes 10 x (8 id + 8 counter) = 160 bytes
vector clock, 500 nodes 500 x 16 = 8,000 bytes
```

On a 200-byte value, a 10-node vector is 80% overhead and a 500-node vector is 4,000% overhead. That single line explains why nobody runs plain vector clocks on a large cluster:

```
1 billion values x 8 KB of vector = 8 TB of pure metadata
1 billion values x 16 B of HLC = 16 GB
```

**What TrueTime costs.**

```
epsilon 1 - 7 ms, call it 5 ms
commit-wait 2 x epsilon = 10 ms per write transaction
```

```
a transaction that would take 5 ms → 15 ms
throughput on one hot row: 1000/5 = 200/s → 1000/15 = 66/s
```

Three times fewer transactions per second on a contended row, in exchange for globally comparable timestamps. For a bank ledger that is a bargain. For a metrics pipeline it is absurd. **Naming which side of that line your system is on is the answer to the trade-off question.**

## 7 The trade-offs

**Last-write-wins is cheap, simple, and loses data.** Constant metadata, no merge logic, no conflicts to show a user. In exchange, any two writes inside the skew window are ordered arbitrarily and one is discarded with no record. Choose it when a lost write is genuinely tolerable — a presence indicator, a “last seen” value, a cached counter that will be recomputed. Do not choose it for anything a user typed. Cassandra’s default is LWW, and that is the single most common way people lose data in Cassandra.

**Vector clocks are correct and do not scale as written.** They detect concurrency exactly, with no false negatives. They cost one entry per participant, which is fine at ten and impossible at five hundred, and they push the merge decision onto the application — which is real work, and sometimes there is no sensible merge. “Union the shopping carts” is easy. “Merge two versions of a user’s address” is not.

**Hybrid logical clocks are the sensible default and they still have a hole.** Readable like a timestamp, causally correct, 16 bytes. What they do not give you is a *bound* on how far ahead of real time you might be — HLC guarantees causality, not accuracy. If your system needs “this certificate expired at 3 p.m. real time”, an HLC does not settle it.

**TrueTime buys certainty and pays in latency and hardware.** Ten milliseconds on every write transaction, plus GPS receivers and atomic clocks in every data centre. Worth it when you need externally consistent global transactions. Not worth it for anything that could be partitioned so that a single machine owns each key — and if you can partition that way, you should, because a single owner gives you order for free.

**Physical timestamps in logs are fine, and you should still keep them.** None of this means removing wall clocks from your log lines. It means not *ordering* by them across machines. Log the wall clock for humans, log a request ID and a sequence number for machines, and put both in every line. Debugging an incident with only causal clocks is miserable; debugging it with only wall clocks is impossible.

**When would I not think about this at all?** When there is a single writer. If one machine owns a key — because it is the leader for that partition, or because the key is sharded to it — then the order is the order that machine applied them, and no clock is involved. **Most systems avoid this entire problem by funnelling writes for a key through one owner, and that is a legitimate answer to give.** The interesting cases are multi-master, offline-capable clients, and cross-region active-active, which is exactly where these problems live.

---

## 8 In the interview

### How it gets asked

- “Two events have the same timestamp. Which happened first?” — the direct version.
- “How do you order events across machines?”
- “Two replicas got different writes to the same key. How do you resolve it?”
- “What is a vector clock and when would you use one over a timestamp?”
- “Your rate limiter lets through twice the limit occasionally. Why?” — usually a wall-clock-vs-monotonic bug.
- “How does Spanner give globally consistent transactions?” — the TrueTime question.

## The first ninety seconds

“The honest answer to ‘which happened first’ is that from the timestamps alone, you cannot tell, and I would not try. Two machines’ clocks differ — a few milliseconds in one data centre, tens to hundreds of milliseconds across regions — and if the skew is larger than the gap between the two events, the timestamps can be in exactly the wrong order. At ten thousand writes a second and fifty milliseconds of skew, five hundred consecutive writes are unorderable by timestamp. So this is not a rare race.

What I use instead is causality. Event A happened-before event B if they are on the same machine in that order, or if A sent a message that B received, or by a chain of those. That definition never mentions a clock — it is built out of what a machine had actually seen when it acted.

The cheap implementation is a Lamport clock: one counter per machine, incremented per event, and on receiving a message you take the max of yours and theirs plus one. That guarantees that if A caused B, A’s number is smaller. It gives me a total order that never contradicts reality.

What it does not give me is the reverse. A smaller number does not prove A came first — the two might be concurrent, meaning no information flowed between them. If I need to *detect* that, which I do for conflict resolution, I need a vector clock: one counter per machine, compared element-wise. If neither vector dominates, the writes genuinely conflict and the system has a decision to make rather than a value to discard.

Do you want me to go into what I would actually deploy, or into how Spanner sidesteps this with hardware?”

## The follow-ups

**“Why not just synchronise the clocks better?”**

“Because ‘better’ has a floor you cannot get under, and more importantly NTP gives you no error bar. It corrects your clock but does not tell you how wrong you still are, so there is no threshold you can program against.

There are two further problems. NTP can step the clock backwards, so wall-clock time is not even monotonic on one machine — an event stamped 31.200 can be followed by one stamped 31.150. And the correction message itself takes an unknown, asymmetric amount of time, which is the irreducible part.

Google did solve it, and the solution shows the price. TrueTime returns an interval rather than a time — ‘now is between these two points’ — with an epsilon of one to seven milliseconds, backed by GPS and atomic clocks in every data centre. Then Spanner waits out that interval before making a commit visible, about ten milliseconds per write transaction. So the answer to ‘synchronise better’ is: yes, with atomic clocks, and it costs you ten milliseconds on every write. That is a real option and I would name it as one, but it is not a free fix.”

**“Two replicas have different values for the same key. Walk me through resolving it.”**



“First I would ask what the metadata is, because that decides everything. Three cases.

If all I have is wall-clock timestamps, it is last-write-wins, and I would say plainly that this can lose data: two writes inside the skew window get ordered arbitrarily and one is discarded with no record. That is Cassandra’s default and it is the most common way people lose writes in Cassandra. Acceptable for a ‘last seen at’ field. Not acceptable for anything a user typed.

If I have vector clocks, I compare element-wise. If one vector dominates the other, that write saw the other and supersedes it — keep it, no conflict. If neither dominates, they are concurrent and both are real, so I keep both as siblings and either merge them semantically or surface them.

Merging semantically is the part worth designing. If the value is a set — a shopping cart, a set of tags — the union is correct and automatic; that is the Dynamo cart example, and the known cost is that a deleted item can come back. If the value is a counter, use a structure that merges by construction. If it is a plain scalar like an address, there is no sensible automatic merge and I would keep both and ask the user, the way CouchDB does.

What I would not do is pretend a policy makes the conflict go away. Picking the larger value or the longer string is still discarding a real write, just with extra steps.”

### **“Our rate limiter occasionally lets through double the limit. Any idea?”**

“Almost certainly the wall clock. If the window is computed with `time.time()`, an NTP correction that steps the clock backwards makes the current window restart, so a fresh allowance is handed out mid-window — and if it steps forward, a window is skipped entirely.

The fix is one line: measure every interval with the monotonic clock, which only increases and is immune to corrections. Wall clocks are for dates you show to humans; monotonic clocks are for durations, timeouts and lease expiry.

This same bug appears in three other places I would check while I am there: lock leases, retry backoff, and anything that computes ‘how long has this been running’. A lock lease measured on a wall clock can appear to expire the instant it is taken, which then hands the lock to a second holder — and that is a correctness bug, not a performance one.”

### “What does a Lamport clock actually guarantee? Be precise.”

“One direction only. If A happens-before B, then  $\text{Lamport}(A) < \text{Lamport}(B)$ . The converse does not hold:  $\text{Lamport}(A) < \text{Lamport}(B)$  tells me nothing, because A and B may be concurrent.

That is enough to build a total order — sort by counter, break ties by machine ID — which is consistent with causality, and that is exactly what you need for something like a replicated state machine, where all you require is that everyone applies operations in the same order and that order never contradicts cause.

It is not enough for conflict detection, which needs the converse. That is the one-sentence reason vector clocks exist.”

### “What would you actually deploy in 2026?”

“Hybrid logical clocks, unless I had a specific reason not to. They give me a value that reads like a real timestamp — sortable, comparable to a date, usable in a log — while guaranteeing that causally ordered events get increasing values, and they cost eight to sixteen bytes rather than a vector that grows with the cluster. CockroachDB, YugabyteDB and MongoDB’s `clusterTime` all use them.

Before that, though, I would try to make the problem not exist: route all writes for a key through a single owner. If one machine applies every write to a key, the order is whatever it applied, and no clock is involved at all. Most well-designed systems dodge this entire topic that way, and I would say so rather than reach for clever clocks first.

Vector clocks I would keep for genuinely multi-master or offline-capable cases — a mobile app that edits while disconnected — where concurrency is normal and merging is a product decision, not a bug.”

### The model answer

*“Design the conflict resolution for a note-taking app that syncs across a phone, a laptop and a web client, and works offline.”*

“Offline editing makes concurrent writes the normal case rather than a rare race, so this is exactly where the clock question earns its keep. Let me be concrete about what I would build.

**First, I reject timestamps outright, and I would say why with a number.** A phone that has been on a plane for four hours has a clock that has drifted seconds, not milliseconds, and it will sync its edits all at once when it reconnects. Ordering those against the laptop’s edits by wall clock is not approximately right, it is arbitrary. Last-write-wins here means a user watches a paragraph they typed on the plane disappear.

**Every device gets a stable ID, and every note carries a version vector** — one counter per device that has ever edited it. On an edit, the device increments its own entry. On sync, the two vectors are compared element-wise: if one dominates, that version is strictly newer and wins with no conflict at all, which is the common case and costs nothing. If neither dominates, the edits are genuinely concurrent.

**Vector size is bounded by devices per note, not by cluster size**, which is the reason this works here and would not work for a 500-node store. A user has three or four devices; that is 64 bytes of metadata on a note. I would prune entries for devices that have not touched the note in ninety days, accepting a small chance of a spurious conflict in exchange for a bounded size.

**For genuine conflicts I merge rather than pick, where the data shape allows it.** Note text is a sequence, so I would use a structure designed to merge — a CRDT for text, which resolves character-level concurrent edits deterministically on every device without a coordinator. Tags are a set, so union, with the known caveat that deletes need tombstones or a removed tag reappears. Anything that genuinely cannot be merged — two different titles — I keep as two versions and show the user, because inventing an answer is worse than asking.

**The server timestamps everything too, and never orders by it.** Wall-clock time goes in the sync log for humans debugging a support ticket. It never decides which edit wins. Those are two different jobs and mixing them is the bug.

**The numbers I would quote.** Version vector:  $4 \text{ devices} \times 16 \text{ bytes} = 64 \text{ bytes}$  on a note averaging 2 KB, so 3% overhead. Conflict rate in practice: with offline editing, a few percent of syncs, versus effectively continuous silent loss under last-

write-wins. And the thing I would flag as the real cost is not storage — it is that merging is now product work. Someone has to decide what merging two versions of a title means, and that decision cannot be made in the storage layer.”

## 9 Recall card

**A timestamp does not order events across machines.** Skew is 0.1–5 ms in one data centre and 10–100 ms across regions; whenever skew exceeds the gap between two events, the timestamps can be exactly backwards.

**Order by causality instead.**  $A \rightarrow B$  if same machine and earlier, or A sent the message B received, or by a chain. No clock appears in that definition. Neither direction holding means **concurrent** — no information flowed, and nothing can order them.

**Lamport clock:** one counter,  $\max(\text{mine}, \text{theirs}) + 1$  on receive. Guarantees  $A \rightarrow B \Rightarrow L(A) < L(B)$  and *only* that. **Vector clock:** one counter per machine, compares element-wise, and is the only one that can *detect* concurrency — at 16 bytes per participant.

**Last-write-wins silently discards data** inside the skew window; that is Cassandra’s default. HLCs (16 bytes, readable, causally correct) are the modern default; TrueTime buys global order for ~10 ms of commit-wait per transaction plus atomic clocks.

**Use the monotonic clock for every duration** — timeouts, leases, rate-limit windows. The wall clock jumps backwards, and that bug shows up as a rate limiter letting through double.

## PRACTICE

## Practice — Tries in interviews: word search II

**DSA topic:** Tries in interviews: word search II **System design topic:** Clocks, ordering, and why time is a lie

Code these, in this order

One rule for the whole set: **write the mark line and the restore line in the same keystroke.** Type `board[r][c] = "#"`, then immediately type `board[r][c] = character` two lines below it, then fill in the loop between them. Every other bug in these problems announces itself. That one does not.

| # | PROBLEM              | SOURCE               | WHAT IT IS REALLY TESTING                                      |
|---|----------------------|----------------------|----------------------------------------------------------------|
| 1 | Word Search          | LeetCode 79 (Medium) | Backtracking on a grid, one word, restore on the way out.      |
| 2 | Word Search II       | LeetCode 212 (Hard)  | Moving the word list <i>inside</i> the walk. The whole lesson. |
| 3 | Word Break II        | LeetCode 140 (Hard)  | A trie plus memoised backtracking, without a grid.             |
| 4 | Concatenated Words   | LeetCode 472 (Hard)  | Searching a trie from inside a trie walk.                      |
| 5 | Stream of Characters | LeetCode 1032 (Hard) | The reversed trie — the trick nobody sees the first time.      |

On problem 2, write the slow version first and time it

Solve it once by calling your problem 1 solution in a loop over the words. Run it on a  $12 \times 12$  grid with 3,000 words and record how long it takes, or record that you gave up waiting. Then write the trie version and record that number. Two numbers, and the ratio between them is the answer to “why a trie”.

On problem 2, remove each pruning and measure

Three runs on the same input:

1. Full solution.
2. Without `child.word = None` after a find.
3. Without the node deletion at the end.

Record the runtime and the output length for each. Version 2 has a wrong output as well as a slow one; say which and why.

### On problem 2, build the adversarial input yourself

A  $12 \times 12$  grid of nothing but `a`, and words `["a"*k for k in range(1, 11)]`. Run all three versions above on it. This is the case the judge uses to fail solutions that have a trie but no node deletion.

### On problem 5, say why the trie is reversed

Write down, in one sentence, what is being stored and why storing the words forwards does not work when characters arrive one at a time and you must answer after each one.

### The two-walks drill

1. Say what two things move together during the search, and what happens when one cannot move.
2. Say what `node.children.get(character)` returning `None` means in words.
3. Say why the word list is inside the walk and not outside it.
4. Give the naive cost and the trie cost, and name the factor that disappears.

### The node drill

1. Say why the node stores the word rather than a boolean.
2. Say what that saves on every step of every path.
3. Say what setting `word = None` after a find does — both effects.
4. Say when the node-deletion pruning fires, and what it buys.

### The backtracking drill

1. Write the mark-loop-restore block from memory.
2. Say what the output looks like when the restore is missing — not what error, what output.
3. Say why `#` is a safe marker and what would not be.
4. Give the non-mutating alternative and its cost.

### The cost drill

1. Derive  $M \times N \times 4 \times 3^{(L-1)}$  out loud, saying where the 4 and the 3 come from.
2. Put the LeetCode limits in and produce the two numbers.
3. Say why the bound is loose, and what the real cost is bounded by.
4. Give the space cost, separating the trie from the call stack.

### The break-it drill

Trigger each and record the exact output or error:

1. The restore line removed, on a grid where two answers share a cell.
  2. `child.word = None` removed, on a  $2 \times 2$  grid of `a` with the word `aa`.
  3. The word loop left on the outside, on the 3,000-word input.
  4. A trie built from 2,000-character strings, searched recursively.
  5. `explore` called without the bounds check.
  6. The node deletion placed *before* the restore line instead of after.
- 

### The clock drill

1. Say how far an unsynchronised clock drifts in a day, and show the arithmetic.
2. Give NTP's realistic skew in one data centre and across regions.
3. Say two things NTP cannot give you, beyond a smaller number.
4. Say what a leap smear is and why Google built one.

### The two-clocks drill

1. Name the two clocks on every machine and what each is for.
2. Say what `time.time()` does that `time.monotonic()` does not.
3. Give three places where using the wrong one is a correctness bug.
4. Describe the rate-limiter symptom and trace it back to the cause.

### The happens-before drill

1. State the three rules of happens-before, without mentioning a clock.
2. Define concurrent, and say what it does *not* mean.
3. Say why “Deepak saw the kettle was warm” settles an order that two clocks could not.
4. Give an example from a real system of each of the three rules.

### The Lamport drill

1. Write the update rule from memory, both on an event and on receipt.
2. State exactly what it guarantees, in one direction.
3. Say what it does not guarantee, and why that matters.
4. Say how you turn it into a total order, and what the tie-break is.

### The vector-clock drill

1. Write the update rule from memory.
2. Give the comparison procedure and its three outcomes.
3. Work through the  $[1, 0, 0]$  versus  $[0, 0, 1]$  example out loud.
4. Say what a vector clock tells you that it does not tell you.
5. Give the metadata size at 10 nodes and at 500 nodes.

### The numbers drill

1. Compute how many writes fall inside a 50 ms skew window at 10,000 writes a second.
2. Compute the daily conflict count at 1,000 writes a second and a 0.5% conflict rate.
3. Compare metadata for Lamport, HLC, and vectors at 10 and 500 nodes, on a 200-byte value.
4. Compute the storage cost of 500-node vectors on a billion values.
5. Compute Spanner's commit-wait cost and its effect on hot-row throughput.

### The trade-offs drill

1. Say what last-write-wins costs and when it is the right choice.
2. Say what vector clocks cost and where they stop scaling.
3. Say what an HLC gives you and the one thing it does not.
4. Say what TrueTime buys and what you pay, in milliseconds and in hardware.
5. Give the design move that makes this entire topic disappear, and when it is available.

### The failure drill

For each, say what happens and what you would build:

1. A phone edits offline for four hours and then syncs.
2. Two nodes write the same key 30 ms apart with 50 ms of skew, under last-write-wins.
3. NTP steps a clock back 200 ms during a duration measurement.
4. A lock lease is measured on the wall clock and the clock jumps forward.
5. A 500-node cluster tries to attach vector clocks to every value.
6. Two users concurrently rename the same document.
7. An event log is sorted by timestamp during an incident review.

Two of the seven are not fixed by any clock, however good. Name them.



---

## Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find all dictionary words in this letter grid.* Backtracking plus a trie, the sentence about **W** leaving the multiplication, the two positions moving together, storing the word on the node, both prunings, the cost, and the restore-the-cell bug named before you write it.
  2. *Two events have the same timestamp. Which happened first?* You cannot tell, and why — skew against event gap, with a number. Then happens-before, then Lamport with its one-directional guarantee, then vector clocks for detecting concurrency.
  3. *Two replicas disagree about a key's value. Resolve it.* The three metadata cases, last-write-wins named as data loss with the Cassandra example, vector-clock comparison with its three outcomes, and merging as a product decision rather than a storage one.
- 

## Before you move on

- ☐ I can say in one sentence why a trie beats searching each word.
- ☐ I know which factor leaves the multiplication, and by how much on the LeetCode limits.
- ☐ I can write the two-positions-moving-together search from memory.
- ☐ I know why the node stores the word and not a boolean.
- ☐ I know both effects of clearing the word after a find.
- ☐ I know when node deletion fires and which test case it saves.
- ☐ I write the restore line at the same time as the mark line.
- ☐ I know the restore bug produces missing words, not an error.
- ☐ I can derive the cost formula and say where the 4 and the 3 come from.
- ☐ I can give the non-mutating alternative and its price.
- ☐ I built the all-**a** adversarial grid and ran it.
- ☐ I can say what NTP achieves and what it cannot give me.
- ☐ I can name the two clocks and what each is for.
- ☐ I can give three bugs caused by using the wall clock for durations.
- ☐ I can state happens-before in three rules with no clock in them.

- [ ] I can define concurrent and say what it does not mean.
- [ ] I can write the Lamport update rule and state its one guarantee.
- [ ] I know why a Lamport clock cannot detect conflicts.
- [ ] I can write the vector clock rule and the three comparison outcomes.
- [ ] I can give vector metadata sizes at 10 and 500 nodes.
- [ ] I know last-write-wins loses data and that it is Cassandra's default.
- [ ] I can say what an HLC gives me and the one thing it does not.
- [ ] I can quote TrueTime's epsilon and the commit-wait cost.
- [ ] I know the single-owner move that makes this problem disappear.
- [ ] I answered all three questions above out loud.

# Tries revision and mock round

## DATA STRUCTURES AND ALGORITHMS

Tries revision and mock round

## SYSTEM DESIGN

Failure detection, heartbeats, and timeouts

### 1 What this is, and why they ask it

Four days ago you had never seen a trie. Today you close the phase, and the way to close it is not to reread [day 120](#) — it is to answer two problems you have not seen, out loud, with a clock running.

This lesson has two halves. The first is the compression: the recognition question, the five templates, the costs, and the five mistakes that account for almost every trie bug. The second is a mock round — two unseen problems, with the script for what to say in the first ninety seconds of each, and what a good answer sounds like when it goes wrong halfway through.

The reason a revision day exists at all is that interviews do not test whether you *know* tries. They test whether you *recognise* one in a problem that never uses the word. Nobody says “use a trie”. They say “given a list of words and a search prefix”, or “the same words keep being checked against a dictionary”, and you have about ninety seconds to notice.

By the end of today you can state the recognition question in one line, write all five templates from memory, give every operation’s cost without thinking, and take two unseen problems from statement to working code while talking.

### 2 The story

Joseph has been fixing wiring in the same three streets for eighteen years, and the thing that makes him fast is that he does almost nothing for the first two minutes.

A woman calls him on a Thursday because the tube light in her kitchen has stopped working. He arrives, and he does not open his bag. He does not touch the switch. He stands in the doorway and asks three questions.

“Did it go off all at once, or was it flickering for a few days first?”

All at once, she says.

“Anything else in the kitchen stop working at the same time? The mixer, the fridge?”

No, just the light.

“And has it happened before in this flat?”

Twice, she says. Both times in the rains.

By now he has stopped listening for anything else. He does not need to check the wiring in the wall, and he does not need to look at the main board, because both of those would have taken the fridge down with it. He does not need to open the switch, because a switch that is going gives you warning — it flickers first, or it gets warm. All at once, and only the one light, and it happens in the rains: that is water getting into the fitting itself and nothing else.

He gets the stool, unscrews the fitting, and there is the green crust on the holder, exactly where he expected it. Eleven minutes from the doorway to done.

His apprentice, who is twenty and quick with his hands, would have been faster to open the bag and slower to finish. He opens things. He checks the switch, then the board, then the wiring, and somewhere in the third hour he gets to the fitting. He is not worse at the work — put the two of them in front of a corroded holder and they will fix it in the same six minutes. What he does not have is the two minutes at the door.

Joseph says the same thing to him about once a month, and the boy has not heard it yet. “Everyone can fix it once you know what it is. Knowing what it is, that is the job.”

---

### 3 The idea in plain English

The two minutes at the door is what this lesson is for.

**The recognition question, in one line:** *am I asked about prefixes, repeatedly, over a fixed set of strings?* If yes, trie. If any of the three parts is missing, probably not.

Take the three parts apart, because each one throws out a different wrong answer.

“**Prefixes.**” Not exact matches — *prefixes*. If the question is “is this exact word in the list”, a **hash set** answers it in one step and a trie is strictly worse. You met sets on [day 62](#); they beat a trie on both time and memory for exact lookup. The trie earns its place the moment the question becomes “does anything start with this”, “what completes this”, or “walk letter by letter and tell me as soon as it becomes impossible”.

“**Repeatedly.**” One prefix query against a list is a loop and a `startswith`. Ten thousand queries against the same list is a trie, because you pay the build once and every query afterwards costs the length of the query and nothing else. If the word list changes on every query, the build cost dominates and the trie loses.

“A fixed set of strings.” The set has to be known and reasonably stable. A trie over a set that changes completely between queries is a structure you are rebuilding rather than using.

**The four shapes it comes in.** Almost every trie interview question is one of these:

1. **Store and look up by prefix.** Insert, search, `starts_with`. LeetCode 208. This is the base, and everything else is built on the same three-line walk.
2. **Match with a hole in it.** A wildcard `.` matches any character, so the walk becomes a search that follows every child. LeetCode 211.
3. **Complete a prefix, ranked.** Walk to the node, then gather and rank what is underneath, or read a precomputed list. LeetCode 1268 and 642, and [day 122](#).
4. **Prune somebody else’s search.** The trie is not the answer; it is the thing that tells another search when to stop. Word Search II, and [day 123](#).

**Shape 4 is the one worth the most.** The other three are “use a trie”. Shape 4 is “use a trie to make a different search cheap”, and that is the shape that turns a Hard problem into a straightforward one. The signal for it is: *a search is being run once per word in a list*. Whenever you see that, ask whether the list can go inside the search instead of outside it.

**What every trie operation shares.** One walk. `node = node.children[character]`, one character at a time. Insert creates missing children on the way down. Search fails when a child is missing. `starts_with` is search without the final `is_end` check. Delete is the walk plus a bottom-up pruning rule. Wildcards are the walk branching. Autocomplete is the walk plus a traversal. **If you can write the walk, you can write all of them**, and that is the thing to have automatic.

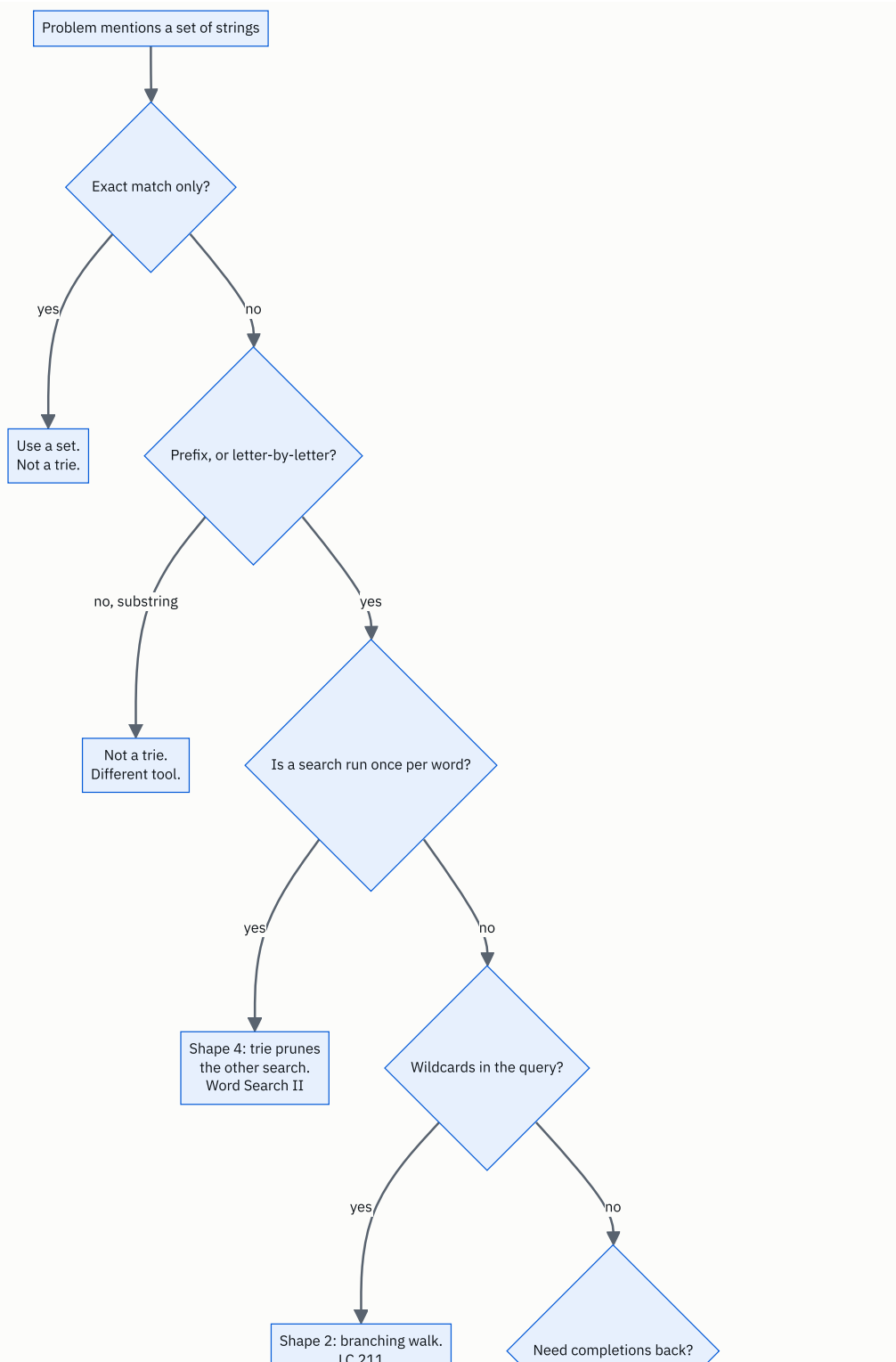
**When the answer is “not a trie”.** Say these out loud too, because knowing the boundary is what makes the recognition trustworthy:

- Exact membership only → set.
- Substring search, not prefix → a suffix structure or a proper string-matching method, not a trie of the words.
- A handful of words → just loop. A trie over five words is complexity for nothing.
- Very long, non-shared keys, like random hashes → a trie degenerates into a linked list per key and costs more memory than the strings did.

---

#### 4 The picture

The recognition path, as a set of questions rather than a diagram of a structure:



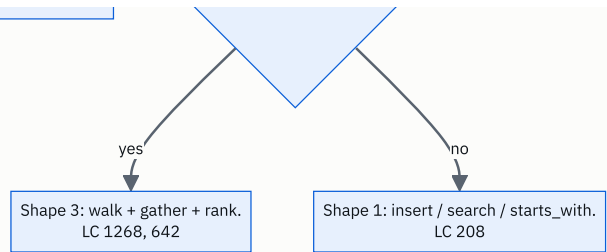


FIGURE 124.1

**What to notice.** The first two questions throw out the cases where a trie is the wrong answer, and they come first on purpose. Reaching for the structure you just learned is the most common way to get a phase's worth of study wrong.

The whole phase, on one picture — the same six words, four ways:

```
words: cat car card care cart dog

SHAPE 1 search("car") walk c-a-r, check is_end → True
 starts_with("ca") walk c-a, ignore is_end → True

SHAPE 2 search("c.t") c → a? o? u? ... → t → True (cat)
 branches at the dot

SHAPE 3 suggest("ca", k=2) walk c-a, then gather below → [car, care]
 or read the stored top-2 list

SHAPE 4 grid + word list walk the GRID, carry the NODE → prune early
 no child for this letter → stop
```

**What to notice.** Every line begins with the same word: *walk*. One motion, four uses. That is what makes the phase memorable — you are not learning four structures, you are learning one walk and three things to do around it.

## 5 The code, built step by step

Five templates. Learn these and you can write any trie problem in a round without designing anything on the spot.

### Template 0 — the node and the walk

Everything else is built on these six lines.



```

class Node:
 __slots__ = ("children", "is_end")

 def __init__(self) → None:
 self.children: dict[str, Node] = {}
 self.is_end: bool = False

```

A dictionary rather than a 26-slot list, because it handles any alphabet, uses memory proportional to the children that actually exist, and is what an interviewer expects. Mention the 26-slot array as the alternative when memory matters and the alphabet is fixed — it is faster and denser but only for lowercase English.

#### Template 1 — insert, search, starts\_with

```

class Trie:
 def __init__(self) → None:
 self.root = Node()

 def insert(self, word: str) → None:
 node = self.root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.is_end = True

 def _walk(self, prefix: str) → Node | None:
 node = self.root
 for character in prefix:
 node = node.children.get(character)
 if node is None:
 return None
 return node

 def search(self, word: str) → bool:
 node = self._walk(word)
 return node is not None and node.is_end

 def starts_with(self, prefix: str) → bool:
 return self._walk(prefix) is not None

```

The three public methods are one line each once `_walk` exists. **The single condition separating `search` from `starts_with` is `and node.is_end`** — say that out loud, because it is the most common one-line question in the whole phase.

## Template 2 — delete, with the pruning rule

```
def delete(self, word: str) → bool:
 """Remove `word`. Returns False if it was not there."""
 path: list[tuple[Node, str]] = []
 node = self.root
 for character in word:
 child = node.children.get(character)
 if child is None:
 return False
 path.append((node, character))
 node = child
 if not node.is_end:
 return False
 node.is_end = False
 for parent, character in reversed(path):
 child = parent.children[character]
 if child.children or child.is_end:
 break
 del parent.children[character]
 return True
```

The pruning rule, in one line: **remove a node only if it has no children and ends no word**. Both conditions, and it must run bottom-up, which is why the loop is `reversed`. The `break` matters — the moment one node is worth keeping, every node above it is too, so stop.

Test it on `{car, card}`: deleting `car` must leave `card` intact and remove nothing at all.

## Template 3 — the wildcard walk

```
def search_pattern(self, pattern: str) → bool:
 def match(node: Node, position: int) → bool:
 if position == len(pattern):
 return node.is_end # NOT True
 character = pattern[position]
 if character == ".":
 return any(match(child, position + 1) for child in node.children.values())
 child = node.children.get(character)
 return child is not None and match(child, position + 1)
 return match(self.root, 0)
```

Two details that are always the bug. The base case returns `node.is_end`, never `True` — otherwise `"ca"` matches when only `"cat"` is stored. And the dot branch must try *every* child and keep going after a failure, which `any` does correctly; a bare `return match(...)` inside a loop gives up after the first child.

#### Template 4 — gather and rank

For this one the node carries `weight: int` instead of `is_end: bool`, exactly as on [day 122](#) — zero means “no word ends here”, and any positive number is both a yes and a popularity.

```
def suggest(self, prefix: str, k: int = 5) → list[str]:
 node = self._walk(prefix)
 if node is None:
 return []
 out: list[tuple[int, str]] = []

 def collect(node: Node, so_far: str) → None:
 if node.weight:
 out.append((node.weight, so_far))
 for character, child in node.children.items():
 collect(child, so_far + character)

 collect(node, prefix) # start string is the PREFIX
 out.sort(key=lambda pair: (-pair[0], pair[1]))
 return [word for _weight, word in out[:k]]
```

Two details, both from [day 122](#). `collect` checks the node it starts on, or the prefix itself is never returned. And `so_far` starts as the prefix, not as an empty string.

## Template 5 — the trie that prunes a grid search

```
def find_words(board: list[list[str]], words: list[str]) → list[str]:
 root = Node()
 for word in words:
 # nodes here store `word`, not is_end
 node = root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.word = word

 rows, cols = len(board), len(board[0])
 found: list[str] = []

 def explore(row: int, col: int, node: Node) → None:
 character = board[row][col]
 child = node.children.get(character)
 if child is None:
 return # nothing continues: stop
 if child.word:
 found.append(child.word)
 child.word = None # report once, stop re-finding
 board[row][col] = "#"
 for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
 r, c = row + dr, col + dc
 if 0 ≤ r < rows and 0 ≤ c < cols and board[r][c] ≠ "#":
 explore(r, c, child)
 board[row][col] = character # RESTORE
 if not child.children and child.word is None:
 del node.children[character] # prune the exhausted branch

 for row in range(rows):
 for col in range(cols):
 explore(row, col, root)
 return found
```

Three lines carry the whole idea: the early `return` when there is no child, the restore after the loop, and the deletion of an exhausted node. If you can type those three from memory you have the shape.

### 6 What it costs

Every operation, with  $L$  the key length,  $n$  the number of keys,  $\Sigma$  the alphabet size, and  $m$  the number of completions under a prefix.



**A trie costs roughly three to four times a set, and buys prefix queries.** If you never ask a prefix question, you paid 27 MB for nothing. That comparison, with those numbers, is a strong thing to have ready.

**When memory is the constraint**, three moves in order of how much they buy:

1. A 26-slot list per node instead of a dict — faster, but wasteful when nodes are sparse; usually a wash on English words and a clear win on dense tries.
2. A **radix trie**, which collapses any chain of single-child nodes into one node holding a whole substring. On English this typically cuts the node count by half or more, because the tails of words are all single-child chains.
3. Do not store the trie at all — go back to a sorted list and **bisect**, which gives prefix ranges in  $O(\log n + m)$  with no structure to maintain. Slower per query, dramatically smaller. Worth naming as the alternative you considered.

## 7 The traps

Five mistakes account for nearly every trie bug. Each is listed with the exact symptom, because in four of the five there is no error message at all.

### 1. `search` that is really `starts_with`

```
def search(self, word: str) → bool:
 return self._walk(word) is not None # missing the is_end check
```

Symptom: `search("ca")` returns `True` when only `"cat"` is stored.

```
>>> t.insert("cat"); t.search("ca")
True
```

Nothing crashes. Every prefix of every stored word is now a member.

### 2. Delete that takes a neighbour with it

Pruning without checking `is_end`:

```
if not child.children:
 del parent.children[character] # forgot: or child.is_end
```

Store `car` and `card`, delete `card`:

```
>>> t.insert("car"); t.insert("card"); t.delete("card"); t.search("car")
False
```

car is gone, because the r node had no children left after the delete and was pruned — even though it ended a word. Both conditions or neither.

### 3. The wildcard base case

```
if position == len(pattern):
 return True # should be node.is_end
```

```
>>> t.insert("cat"); t.search_pattern("ca")
True
```

Same class of bug as number 1, in a different place. Any prefix now matches.

### 4. The cell that is never restored

```
board[row][col] = "#"
for dr, dc in (...):
 explore(...)
missing: board[row][col] = character
```

```
>>> find_words(grid, ["oath", "eat"])
['oath']
```

No error. Just a short answer, and only when two answers share a cell — which most small test cases do not.

### 5. Reaching for a trie when a set would do

The one nobody lists as a bug, and the one that costs marks:

```
trie = Trie()
for word in dictionary:
 trie.insert(word)
return trie.search(target) # exact match, one query
```

Symptom: none. It works. It is 37 MB and forty lines where a set is 10 MB and one line. An interviewer who asks “why a trie here?” and gets no answer has learned something about you.

### And the two real error messages

Both come from the same place — recursion depth — and both are worth recognising instantly:

```
Traceback (most recent call last):
 File "trie.py", line 33, in collect
 collect(child, so_far + character)
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

```
Traceback (most recent call last):
 File "trie.py", line 41, in suggest
 heapq.heappush(heap, (-weight, node))
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

The first is a trie built from long keys — paths, URLs, sequences — being traversed recursively. Rewrite with an explicit stack. The second is a heap with no tie-breaker between the priority and the object. Insert the string, or a counter.

## 8 In the interview

### How it gets asked

Nobody says “trie”. These are the phrasings that mean it:

- “Implement a data structure that supports adding words and searching for them.” — shape 1.
- “...and searching with `.` matching any character.” — shape 2.
- “Given a list of products, return suggestions after each character typed.” — shape 3.
- “Given a board and a list of words, return all words present.” — shape 4.
- “You have a dictionary and a stream of characters. After each one, say whether any word just ended.” — shape 4 with a reversed trie.
- “Replace every word in this sentence with its shortest root from this dictionary.” — shape 1 with an early stop.

The signal that outranks all of them: **a search is being run once per word in a list.** Whenever you hear that, the word list belongs inside the search.

### The mock round

Two problems. Twenty minutes each. Say everything out loud, including the parts where you are stuck.



**Problem 1.** “You are given a dictionary of words and a long string with no spaces. Return true if the string can be split entirely into dictionary words. Then return one such splitting.”

**Minute 0 to 2 — recognise, do not code.**

“Repeated prefix questions against a fixed word list — every time I stand at position `i` in the string I want to know which dictionary words start there. That is a trie.

Before I commit: could a set do it? A set gives me exact membership, so I would have to try every substring `s[i:j]`, which is  $O(n^2)$  substrings each hashed at  $O(\text{length})$ . The trie lets me walk forward from `i` once and find every word starting there in a single pass, so it is  $O(n)$  per start instead of  $O(n^2)$ . That is the reason, and I would say it rather than just asserting the trie.

The second half — ‘can be split’ — is overlapping subproblems, so this is a trie plus memoisation. Let me confirm the constraints: how long is the string, how many words, and can words repeat?”

**Minute 2 to 12 — write it.**

```
def word_break(text: str, words: list[str]) → bool:
 root = Node()
 for word in words:
 node = root
 for character in word:
 node = node.children.setdefault(character, Node())
 node.is_end = True

 from functools import lru_cache

 @lru_cache(maxsize=None)
 def can_split(start: int) → bool:
 if start == len(text):
 return True # consumed everything
 node = root
 for end in range(start, len(text)):
 node = node.children.get(text[end])
 if node is None:
 return False # no word continues: stop early
 if node.is_end and can_split(end + 1):
 return True
 return False

 return can_split(0)
```

Say while typing: “The `return None` inside the loop is the trie earning its place — I stop scanning forward the moment no dictionary word can continue, instead of testing every remaining substring.”

### Minute 12 to 16 — cost, out loud.

“ $n$  start positions, each walking forward at most  $n$  characters, so  $O(n^2)$  character steps, and memoisation means each start is computed once. Building the trie is  $O(\text{total characters in the dictionary})$ . Space is the trie plus  $O(n)$  for the memo and  $O(n)$  for the recursion.

Without the trie it is the same  $O(n^2)$  in the worst case but with a substring construction and a hash at every step, so the constant is much worse, and the early exit does not exist.”

### Minute 16 to 20 — the follow-up they will ask.

“Now return the actual splitting, and then all of them.”

“For one splitting, I return the string instead of a boolean and prepend as I unwind. For *all* splittings, the answer count can be exponential — `aaaa...` with dictionary `["a", "aa"]` gives Fibonacci-many results — so no memoisation on the *results* saves me from that; I would memoise the list of suffix-splittings per start, which shares structure, and I would say up front that the output size is the real bound.”

---

**Problem 2.** “Design a structure that takes characters one at a time. After each character, report whether any word from a fixed dictionary ends at that character.”

### Minute 0 to 3 — the trap, and the recognition.

“The obvious move is a trie of the dictionary, and walking it forward as characters arrive. That fails immediately: the stream is `a, b, c, d` and the word is `cd`. By the time `c` arrives I have already walked off the trie at `b` and there is nothing to walk from.

I could keep a set of *all* the live positions — one walker per possible start — which is correct but grows with the stream.

The move that fixes it is to store the dictionary **reversed**. Then, when a character arrives, I walk *backwards* through the recent characters from the newest, matching against the reversed trie. A word ends here exactly when its reversal starts here going backwards. And I only need to keep as many recent characters as the longest word.

This is LeetCode 1032 and I would say plainly that the reversal is the trick — it is not something you derive under pressure, it is something you recognise.”

### Minute 3 to 12 — write it.

```
from collections import deque

class StreamChecker:
 def __init__(self, words: list[str]) → None:
 self.root = Node()
 self.longest = 0
 for word in words:
 node = self.root
 for character in reversed(word): # REVERSED
 node = node.children.setdefault(character, Node())
 node.is_end = True
 self.longest = max(self.longest, len(word))
 self.recent: deque[str] = deque(maxlen=self.longest)

 def query(self, letter: str) → bool:
 self.recent.appendleft(letter) # newest first
 node = self.root
 for character in self.recent:
 node = node.children.get(character)
 if node is None:
 return False
 if node.is_end:
 return True
 return False
```

Say while typing: “`maxlen` on the deque is doing real work — it bounds memory to the longest word regardless of how long the stream runs, and it means I never have to prune by hand.”

#### Minute 12 to 16 — cost.

“Each query walks at most  $L$  characters where  $L$  is the longest word, so  $O(L)$  per character and completely independent of how many characters have arrived. Space is the trie,  $O(\text{total dictionary characters})$ , plus  $O(L)$  for the buffer.

The early `return False` matters more than it looks: on a real stream most queries die within two or three characters, so the practical cost is far below  $L$ .”

#### Minute 16 to 20 — when it goes wrong.

If you cannot see the reversal, say so and keep working:

“I do not see a way to avoid tracking multiple live positions yet. Let me build the correct version with a set of active nodes first — that is  $O(\text{number of active walkers})$  per character and I can bound it by the longest word — and then see whether I can flip it.”

That is a passing answer. **Being stuck out loud with a working fallback beats silence with a perfect one**, and the interviewer will usually give you the hint once they see you have a correct version.

#### The model answer

“Why did you pick a trie here?” — the question that ends most trie interviews, asked after the code works.

“Three reasons, and I would check them in order.

One, the queries are about prefixes, not exact matches. A set answers exact membership in one step and beats a trie on both time and memory, so if the question were only ‘is this word present’, I would use a set and say so. It is not — I need ‘does anything start with this’ and ‘what continues from here’, and that is the query a set cannot answer without scanning.

Two, the queries are repeated against a stable set of words. I pay  $O(\text{total characters})$  once to build, and every query afterwards costs the length of the query and nothing else — it does not grow with the number of words stored. That is the property I am actually buying.

Three, the shape here is that a search was being run once per word in the list. Putting the word list inside the search removes that factor entirely. On the numbers for this problem that is a factor of three thousand.

The cost is memory: roughly three to four times what a set of the same words would take — about 37 megabytes versus 10 for a hundred thousand English words. If memory were the binding constraint I would use a radix trie, which collapses single-child chains and cuts the node count roughly in half, or fall back to a sorted list with binary search, which gives prefix ranges in  $O(\log n + m)$  with no structure to maintain at all.”

---

## 9 Recall card

**The recognition question:** *prefixes, repeatedly, over a fixed set of strings?* All three parts, or it is not a trie. Exact match only → set. A search run once per word → put the list inside the search.

**One walk, four shapes:** insert/search/ `starts_with`; wildcard branching; gather-and-rank; and prune somebody else’s search. Shape 4 is worth the most and is the hardest to spot.

**Every basic operation is  $O(1)$  and none depends on  $n$ .** That is the property you are buying, and it is what a set cannot give you for prefixes.

**The five bugs, four of them silent:** missing `is_end` in `search`; pruning a node that ends a word; wildcard base case returning `True`; the grid cell never restored; and using a trie where a set would do.

**Memory:** ~150 bytes per node, ~250,000 nodes for 100,000 English words, ~37 MB — about  $3\text{--}4 \times$  a set. Radix trie or sorted-list-plus-`bisect` are the two ways down.

## 1 What this is, and why they ask it

You cannot tell whether a machine has died. You can only tell that it has stopped answering, and those are different facts. A machine that is dead, a machine that is overloaded, and a machine on the far side of a broken network cable all look identical from where you are standing.

Failure detection is the machinery that turns “has not answered for a while” into a decision. Heartbeats, timeouts, health checks and gossip are all versions of the same idea, and every one of them is a bet: you pick a number of seconds, and you accept that sometimes you will declare a healthy machine dead and sometimes you will keep sending work to a dead one.

They ask this because “how do you know a server has died?” is a question with no correct answer, only a trade-off — and watching a candidate discover that live is very informative. It also underpins everything in this phase. [Leader election](#) starts when a failure is detected. A [distributed lock](#) is unsafe precisely because a detected failure might be wrong. Retries, circuit breakers and load-balancer health checks are all downstream of this one number.

By the end of this lesson you can design a failure detector, set the timeout from a real latency distribution rather than a guess, name the cost of getting it wrong in both directions, and explain why the correct answer to “is it dead?” is often “it does not matter, because I fenced it”.

## 2 The story

Mr Pillai has lived on the second floor of the same building for thirty-one years, and he sleeps badly.

Ram Singh walks the lane at night. He starts at eleven, goes up one side and down the other, and every so often he blows his whistle — two short blasts, then a pause, then one long one. You hear him faintly at the top of the lane, louder as he comes past the gate, then faint again as he goes towards the temple. Round and round until five.

Mr Pillai does not really listen for it any more. It is just there, the way the fridge is there. But he notices when it stops.

On Tuesday it stopped.

He was lying awake anyway, and at some point he realised he had not heard the whistle for a while. He tried to remember when the last one had been. Ten minutes? Twenty? He had not been counting, which was the problem — by the time you notice a sound has stopped, you have already lost track of when it stopped.

So he lay there and thought about it. Ram Singh might be asleep on the plastic chair by the gate; he has been known to. He might have gone round the back where the wall blocks the sound, which happens every round and takes about four minutes. He might have walked down to the main road for tea, which takes twelve. He might be lying somewhere.

Four possibilities, and from a second-floor window in the dark there is no way at all to tell them apart. All four sound exactly the same. All four sound like silence.

At twenty-five past two he got up, put on his slippers, and went down.

He could have got up at five past two, and been wrong most nights — Ram Singh goes round the back every round. He could have waited until three, and if something had actually happened, an hour would have been a long time. Twenty-five minutes is not a clever number. It is just the point at which he decided that being wrong and slightly embarrassed was cheaper than being right and too late.

Ram Singh was at the tea stall on the main road. He had been there twenty minutes. He was surprised to see Mr Pillai and slightly annoyed by the implication, and they walked back up the lane together without saying much.

Mr Pillai still goes down about once a month. He has never once found anything wrong. He still goes.

---

### 3 The idea in plain English

Mr Pillai built a failure detector, and every part of one is in that story.

**The whistle is a heartbeat.** A **heartbeat** is a small, regular message that says “I am alive”. It carries almost nothing else. It exists to be missed.

**The heartbeat interval is how often it comes.** Ram Singh whistles roughly every ninety seconds. In a system this is a number you choose — every second, every five seconds — and it is the resolution of your detector. You cannot notice a failure faster than you notice a missing heartbeat.

**The timeout is how many you let go missing before you act.** Mr Pillai waited twenty-five minutes. In a system, the timeout is usually expressed as a multiple of the interval: “three missed heartbeats and you are out”.



**The four possibilities are the whole problem.** Asleep, behind a wall, gone for tea, or lying somewhere. Written properly: the machine has **crashed**; the machine is **slow** because it is overloaded or garbage-collecting; the **network** has dropped the messages; or the machine is fine and the *reply* was lost. From outside, these are indistinguishable. **This is not a limitation of your monitoring. It is a property of the world**, and it has a name — you cannot distinguish a crashed machine from a slow one in an asynchronous network.

**So a failure detector does not report truth. It reports a suspicion.** The vocabulary for how wrong it can be:

- A **false positive** is declaring a healthy machine dead. Mr Pillai walking down and finding Ram Singh at the tea stall. The machine was fine; you removed it from the cluster anyway.
- A **false negative** is failing to notice a machine that really has died. You keep routing work to it, and every request fails.

**You cannot have neither, and shortening the timeout trades one for the other.** A five-minute timeout means Mr Pillai goes down twice a night for nothing. An hour-long timeout means a real problem waits an hour. There is no setting that is right; there is only a setting that matches what each mistake costs you.

**Detection time is the number to state.** If heartbeats come every second and you allow three misses, the worst case is: the machine dies just after sending one, you wait for three intervals plus the time to notice, so detection takes a little over three seconds. **Detection time  $\approx$  interval  $\times$  misses allowed**, and you should be able to say it as a number, not as “quickly”.

**Push and pull are the two shapes.** In a **push** detector, the machine sends heartbeats to a watcher — Ram Singh whistling. In a **pull** detector, the watcher asks the machine “are you alive?” — the watcher calling out. Pull is what a load balancer’s health check does. Push scales better, because the watched machine does the work; pull tells you more, because you can ask a question the machine has to actually do work to answer.

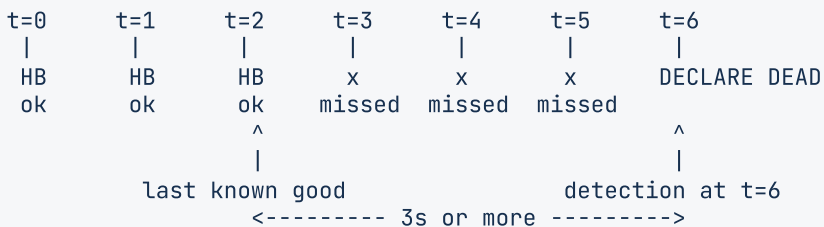
**A better detector reports a degree of suspicion, not a yes or no.** Instead of “dead after three misses”, you can track how long gaps *usually* are and say how surprising the current silence is. If Ram Singh’s whistle normally comes every ninety seconds but sometimes stretches to four minutes when he goes round the back, then four minutes of silence is unremarkable and eleven minutes is not. This is the idea behind the **phi accrual** detector, and its value is that the *application* gets to choose how much suspicion is enough for its own decision.

**And the deepest point: often you should stop trying to know.** Mr Pillai’s real problem is not “is Ram Singh alive” — it is “should I do something”. Systems that go furthest with this stop asking “is it dead?” and instead make it harmless to be wrong: take the work away, revoke its permissions, make its writes get rejected. That is **fen-cing**, and it is why a good answer to “how do you know it died?” often ends with “I do not need to.”

#### 4 The picture

The timeline that shows why the timeout is a bet:

```
heartbeat interval = 1s, timeout = 3 missed
```



CASE A: machine really crashed at t=2.1  
detection took 3.9s. Correct, and 3.9s of failed requests.

CASE B: machine paused for a 2.5s garbage collection at t=2.1,  
came back at t=4.6, heartbeat at t=5 was DROPPED by a  
congested link.  
You declared a perfectly healthy machine dead.  
It does not know. It is still serving traffic.

**What to notice.** Case A and case B are byte-for-byte identical from the watcher’s side. Nothing you can observe distinguishes them, and no shorter timeout helps — a shorter timeout makes case B *more* likely, not less.

Now the two shapes, and the third one that scales:

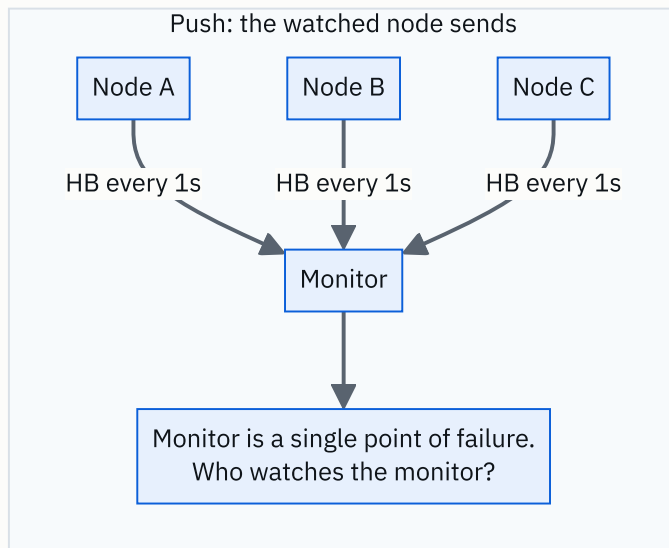


FIGURE 124.2

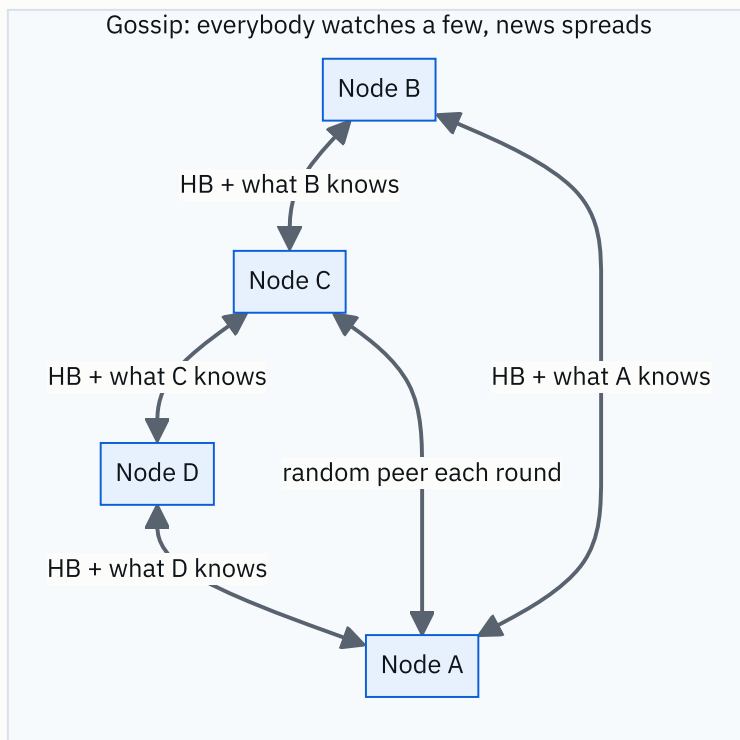


FIGURE 124.3

**What to notice.** In the push diagram every node talks to one monitor, which is simple and has an obvious weakness: the monitor is now the thing whose failure nobody detects. In the gossip diagram no node talks to more than a few peers, each message carries not just “I am alive” but “and here is what I have heard about everyone else”, and news of a failure spreads to the whole cluster in a number of rounds proportional to the *logarithm* of the cluster size. That is why hundred-node and thousand-node clusters gossip.

## 5 How it actually works

Kubernetes probes: the version you will actually configure

Kubernetes runs a pull-based detector against every container, and it distinguishes two questions that people routinely confuse.

```
livenessProbe: # "should I kill and restart this?"
 httpGet: { path: /healthz, port: 8080 }
 initialDelaySeconds: 30
 periodSeconds: 10
 timeoutSeconds: 1
 failureThreshold: 3 # 3 failures → restart

readinessProbe: # "should I send it traffic?"
 httpGet: { path: /ready, port: 8080 }
 periodSeconds: 5
 failureThreshold: 2 # 2 failures → remove from the load balancer
```

**Liveness failing restarts the container. Readiness failing only stops traffic.** Getting these the wrong way round is a classic production incident: a service whose liveness probe checks its database connection will restart every pod in the fleet when the database has a blip, turning a degraded system into a dead one. The rule is that **a liveness probe must only check things the restart can fix**. Dependencies belong in readiness.

Detection time with those settings:  $\text{periodSeconds} \times \text{failureThreshold} = 10 \times 3 = \mathbf{30 \text{ seconds}}$  for liveness,  $5 \times 2 = \mathbf{10 \text{ seconds}}$  for readiness, plus up to one period of luck. Say the number.

### Load balancer health checks

Every load balancer does the same thing with different names. AWS’s ALB defaults are a 30-second interval, a 5-second timeout, 2 consecutive successes to be healthy and 2 failures to be unhealthy — so worst-case detection is about 60 seconds out of the box,

which surprises people. Tightening it to a 5-second interval with a threshold of 2 gives 10-second detection at the cost of six times the health-check traffic.

The subtlety worth naming: a health check that only returns `200 OK` from a static handler tells you the process is running, not that it can do its job. A **deep** health check that touches the database tells you more and creates a new failure mode, because now every instance fails its check at the same instant when the database hiccups. The usual answer is a shallow liveness check and a readiness check that reports degraded dependencies without failing hard.

### Cassandra's phi accrual detector

Cassandra does not use a fixed timeout. It records the intervals between heartbeats from each peer, builds a running distribution, and computes **phi** — a number that grows as the current silence becomes more surprising given that history. Roughly,  $\phi = 1$  means about a 10% chance this is a real failure,  $\phi = 2$  about 1%,  $\phi = 3$  about 0.1%. The threshold is configurable (`phi_convict_threshold`, default 8).

Why this is better than a fixed number: a link that normally delivers in 1 ms and a link that normally delivers in 400 ms get different effective timeouts automatically, and a cluster whose latency degrades gradually does not suddenly start declaring everyone dead. Akka uses the same detector. It is the right answer to “how do you pick the timeout?” when the network is heterogeneous.

### Gossip and SWIM

For large clusters, all-to-all heartbeating does not fit. **SWIM** is the protocol most modern systems use — HashiCorp's Serf and Consul, and the membership layer inside many meshes.

Each round, a node picks one random peer and pings it. If there is no reply, it does not immediately declare failure — it asks `k` other nodes (typically 3) to ping that peer on its behalf. This **indirect probe** is the clever part: it distinguishes “the peer is unreachable *from me*” from “the peer is unreachable from everyone”, which removes a large class of false positives caused by one bad link. Only if all the indirect probes also fail does the node mark the peer suspect, and suspicion spreads by gossip with a timeout before it becomes a confirmed failure.

The cost per node per round is constant — one direct ping plus occasionally `k` indirect ones — regardless of cluster size, which is the entire reason it exists.

### ZooKeeper, Redis Sentinel, Raft

Three more numbers worth having memorised, because they show the range:

- **ZooKeeper sessions.** A client's session has a timeout, negotiated between  $2\times$  and  $20\times$  the server's `tickTime` (default 2 s), so typically 4 to 40 seconds. When the session expires, every ephemeral node the client created is deleted — which is how ZooKeeper-based leader election and locks release automatically.
- **Redis Sentinel.** `down-after-milliseconds`, default 30,000. One sentinel deciding a master is down makes it *subjectively* down; a quorum agreeing makes it *objectively* down, and only then does a failover start. The two-stage rule exists purely to reduce false positives.
- **Raft.** Election timeout of 150–300 ms, randomised per node, with heartbeats every  $\sim 50$  ms. This is two orders of magnitude tighter than the others because Raft nodes are on a fast local network and because an unnecessary election is cheap — it costs one term and a few round trips.

**The range from 150 milliseconds to 40 seconds is not inconsistency. It is each system pricing its own false positive.** An unnecessary Raft election costs milliseconds. An unnecessary Kubernetes restart costs a cold start and dropped connections. An unnecessary Cassandra eviction costs a rebalance of gigabytes.

**And the part that makes detection less important: fencing**

Suppose you get it wrong and declare a live node dead, then hand its work to another node. Now two nodes believe they own the same thing. The answer is not a better detector — it is to make the old one's writes fail.

A **fencing token** is a number that increases every time ownership changes. The new owner gets token 34. The storage layer remembers the highest token it has seen and rejects any write carrying a lower one. The old owner wakes up from its garbage-collection pause holding token 33, writes, and is rejected. It does not need to know it was declared dead.

This is the design that turns an unsolvable problem into a survivable one, and it is the sentence that ends this question well: **“I cannot make detection correct, so I make incorrect detection harmless.”** The blunt hardware version is STONITH — power-cycling the suspected node — which is real, is used in database failovers, and is worth naming as the option when fencing tokens are not available.

## 6 The numbers

**Detection time from the settings.**

|                      |                                              |
|----------------------|----------------------------------------------|
| heartbeat interval   | 1 s                                          |
| failures allowed     | 3                                            |
| worst case detection | 3 x 1 s + up to 1 s of unlucky timing = ~4 s |
| best case detection  | 3 s                                          |

|                              |                 |
|------------------------------|-----------------|
| Kubernetes liveness defaults | 10 s x 3 = 30 s |
| AWS ALB defaults             | 30 s x 2 = 60 s |
| Raft                         | 150-300 ms      |
| Redis Sentinel               | 30 s            |

**What a false negative costs — the time you keep sending work to a dead machine.** Take a service at 10,000 requests per second across 20 machines:

|                           |             |                           |
|---------------------------|-------------|---------------------------|
| per-machine share         | 10,000 / 20 | = 500 requests per second |
| detection time            | 30 s        |                           |
| requests sent to a corpse | 500 x 30    | = 15,000 failed requests  |

Fifteen thousand errors from one machine dying, purely because detection took 30 seconds. Drop detection to 5 seconds:

|         |                         |
|---------|-------------------------|
| 500 x 5 | = 2,500 failed requests |
|---------|-------------------------|

**Six times fewer errors for a six-times-shorter timeout.** This is the argument for tightening.

**What a false positive costs — the other side of the same lever.** Suppose you tighten the timeout to 1 second, and your service has a p99.9 response time of 800 ms with occasional 1.5-second garbage-collection pauses:

|                                |                |                                 |
|--------------------------------|----------------|---------------------------------|
| requests per machine per day   | 500 x 86,400   | = 43,200,000                    |
| health checks per day at 1 s   | 86,400         |                                 |
| fraction exceeding 1 s         | ~ 0.1%         |                                 |
| false failures per machine/day | 86,400 x 0.001 | = 86                            |
| x 20 machines                  |                | = 1,720 false evictions per day |

Seventeen hundred unnecessary evictions a day. If each one triggers a Kubernetes restart with a 20-second cold start:

|              |                                                               |
|--------------|---------------------------------------------------------------|
| 1,720 x 20 s | = 34,400 seconds of lost capacity per day = 9.5 machine-hours |
|--------------|---------------------------------------------------------------|

You have destroyed more capacity than the failures you were detecting. **This is the number that stops people setting aggressive timeouts, and it is the one to bring up when an interviewer suggests one.**

**How to actually pick the timeout.** Not by guessing. From the latency distribution of the check itself:

```
p50 4 ms
p99 60 ms
p999 400 ms
max observed (GC pause) 1,500 ms

timeout = a few times p999, above the known pause
 = 2 s per check
failures allowed = 3
detection = ~6 s, false positive rate < 1 in 10,000 checks
```

**Gossip versus all-to-all.** A thousand-node cluster, heartbeats once a second:

```
all-to-all 1,000 x 999 = 999,000 messages per second
 at 100 bytes = ~100 MB/s of pure heartbeat
```

```
SWIM 1 direct ping + occasionally 3 indirect
 ~ 4 messages per node per round
1,000 x 4 = 4,000 messages per second
at 100 bytes = 400 KB/s
```

**Two hundred and fifty times less traffic**, and it does not get worse as the cluster grows, because the per-node cost is constant. The price is that news spreads in  $O(\log n)$  rounds rather than instantly:

```
1,000 nodes, log2(1000) ~ 10 rounds
at 1 round per second ~ 10 s for the whole cluster to learn
```

So gossip trades about ten seconds of propagation for a hundredfold reduction in traffic. On a thousand nodes that is not a close decision.

**Indirect probing's effect on false positives.** If a single link drops 1% of packets:

```
direct probe alone 1% false suspicion per round
direct + 3 indirect probes 0.01^4 = 0.000001%
```

Assuming the failures are independent — which they are not when the *node* is the problem, and that is exactly the point. Indirect probing removes false positives caused by *one bad link* and leaves genuine node failures detected.



## 7 The trade-offs

**Short timeout versus long timeout is the whole design, and it has no right answer.** Short means fast detection and false positives that cost you capacity and churn. Long means fewer false alarms and a longer window of requests sent into a void. What decides it is which mistake is more expensive *in this system*: for Raft, a false election costs milliseconds, so 150 ms is right; for a Kubernetes pod with a 30-second cold start, a false restart is expensive, so 30 seconds is right.

**Push scales, pull informs.** Push heartbeats put the cost on the watched node and scale to large fleets, but they only prove the heartbeat thread is running — a process whose request handlers are all deadlocked will happily keep sending heartbeats. A pull check that does real work catches that, and costs a request per check per instance. Most systems do both: push for membership, pull for readiness.

**Deep health checks are honest and dangerous.** Checking the database in your health endpoint tells you whether the instance can serve. It also correlates every instance's health with one dependency, so a two-second database blip fails every check simultaneously and empties the entire fleet from the load balancer. Report dependency health, but think very hard before failing on it.

**Centralised monitors are simple and are themselves a single point of failure.** One monitor watching everything is easy to reason about and easy to make the cause of an outage. Gossip removes that but makes the membership view eventually consistent — during those ten seconds, different nodes disagree about who is alive, and any decision made on that view can be made twice.

**Phi accrual is better and harder to explain.** It adapts to real latency and avoids the “one number for a heterogeneous network” problem. It also means your failure behaviour depends on recent history, so it is harder to reason about, harder to test, and produces incidents that are difficult to reproduce. Cassandra and Akka think the trade is worth it; Kubernetes does not.

**And the big one: no detector is correct, so stop relying on it for safety.** Any design where two nodes acting at once is catastrophic must not depend on a detector being right. Add a fencing token, an ownership epoch, or a conditional write on a version. Then a false positive costs you a wasted failover instead of corrupted data. **If your answer to “what if the detector is wrong?” is “it will not be”, you have a correctness bug rather than a tuning problem.**

**When would I not build a failure detector at all?** When something else already has one and is better at it. If the workload runs on Kubernetes, use probes; if state lives in a managed database, use its failover. Writing your own heartbeat protocol for an

application that already sits inside two of them is a common and expensive mistake — you end up with three detectors that disagree, and the disagreement is the outage.

---

## 8 In the interview

### How it gets asked

- *“How do you know a server has died?”* — the direct version.
- *“How long should the timeout be?”* — the version that wants a number and a method, not a number.
- *“Your health check started failing for every instance at once. What happened?”* — almost always a deep check on a shared dependency.
- *“A node was declared dead but was actually just slow, and it came back. What breaks?”* — the fencing question.
- *“How does the cluster agree on who is alive?”* — gossip, quorum, or a coordination service.
- *“Your service restarts every pod when the database is slow. Why?”* — liveness versus readiness.

## The first ninety seconds

“The honest starting point is that I cannot know a machine has died. I can only know it has stopped answering, and four different things produce that: it crashed, it is paused or overloaded, the network dropped my messages, or it answered and the reply was lost. From outside they are indistinguishable, and no amount of better monitoring separates them.

So a failure detector produces a suspicion, and I design it by pricing the two ways it can be wrong. A false positive is declaring a healthy node dead: I lose capacity, I cause a restart or a failover, and if two nodes end up doing the same job, possibly worse. A false negative is not noticing a dead node: every request routed to it fails until I do.

Concretely: heartbeats on a fixed interval, and a threshold of consecutive misses. Detection time is interval times threshold, so one second and three misses gives me roughly four seconds worst case, and I would state it as that number rather than say ‘fast’.

How I choose the interval is the part I would want to be judged on. Not by guessing — from the observed distribution of the check’s own latency. If p999 is 400 milliseconds and I know there are 1.5-second garbage-collection pauses, a 1-second timeout will evict healthy machines all day. I would set the per-check timeout above the known pause, say two seconds, allow three misses, and land on about six seconds of detection with a false positive rate under one in ten thousand.

And the thing I would say before you ask: whatever number I pick, the detector will sometimes be wrong, so I would not build anything whose safety depends on it being right. Do you want me to go into fencing, or into how this scales past a few hundred nodes?”

## The follow-ups

**“A node was declared dead, its work was reassigned, and then it came back. What happens?”**

“Two nodes now believe they own the same work, and that is a correctness problem, not a monitoring one. The old node does not know it was evicted — from its side, it paused for three seconds and woke up.

The fix is fencing rather than better detection. Every ownership handover increments a token. The new owner gets token 34; the storage layer remembers the highest token it has seen and rejects any write carrying a lower one. The old owner writes with token 33 and is refused. It finds out it is no longer the owner from the rejection, which is the only reliable channel.

If the resource cannot check tokens — a legacy device, a third-party endpoint — then the blunt option is STONITH: the new owner power-cycles or network-isolates the old one before taking over. That is genuinely used in database failover, and it is worth naming because it makes the point that when you cannot make the old owner harmless by software, you do it by force.

The sentence I would leave you with is that I cannot make detection correct, so I make incorrect detection harmless.”

#### **“Every pod restarted when the database got slow. Explain.”**

“The liveness probe is checking the database. Liveness answers ‘should this container be killed and restarted’, and a restart cannot fix a slow database — so the check turns a degraded dependency into a fleet-wide outage, and worse, the restarts add a thundering herd of reconnections at exactly the moment the database is struggling.

The rule is that a liveness probe only checks things a restart can fix: is the process wedged, is the event loop blocked, is the internal state corrupt. Anything about a dependency goes in readiness, which removes the instance from the load balancer without killing it, so it recovers on its own when the dependency does.

I would go further and say that even readiness should usually not fail hard on a shared dependency, because if every instance depends on it, failing readiness everywhere empties the load balancer and turns a slow service into a completely unavailable one. Serving degraded is usually better than serving nothing, and that is a product decision I would raise explicitly.”

#### **“How does this work with a thousand nodes?”**

“All-to-all heartbeating does not. A thousand nodes each heartbeating every other node once a second is 999,000 messages a second, about 100 megabytes a second of pure liveness traffic, and it grows with the square of the cluster.

So: gossip, and specifically SWIM, which is what Consul and Serf use. Each node picks one random peer per round and pings it. Constant cost per node regardless of cluster size — about 4,000 messages a second total for the same cluster, four hundred kilobytes.

The part of SWIM worth knowing is the indirect probe. If my ping fails, I do not declare the peer dead — I ask three other nodes to ping it for me. That distinguishes ‘unreachable from me’, which is usually one bad link, from ‘unreachable from everyone’, which is a real failure, and it removes most false positives at the cost of one extra round trip.

What I give up is instant global agreement. News spreads in about  $\log n$  rounds, so ten rounds for a thousand nodes, roughly ten seconds until everyone agrees. During that window different nodes have different views of who is alive, and any decision made on that view can be made twice — which is the fencing problem again.”

**“You said pick the timeout from the distribution. Suppose the distribution changes.”**

“Then a fixed timeout is wrong twice a day, and the answer is an adaptive detector. Cassandra and Akka use phi accrual: instead of a fixed threshold, they keep the recent distribution of inter-heartbeat gaps per peer and compute a suspicion level from how surprising the current silence is given that history. A link that normally takes 1 millisecond and one that normally takes 400 get different effective timeouts, without anyone configuring either.

The other advantage is that it hands the application a *number* rather than a boolean, so a cheap decision can act at low suspicion and an expensive one can wait for high suspicion. That is genuinely useful — you want to stop sending new requests much sooner than you want to trigger a failover.

What I give up is predictability. The behaviour now depends on recent history, so it is harder to test and harder to reproduce in an incident review. Kubernetes deliberately keeps fixed thresholds for that reason, and I would only reach for phi accrual on a heterogeneous network where a single number genuinely does not fit.”

#### The model answer

*“Design failure detection for a fleet of two hundred stateless API servers behind a load balancer, plus a five-node stateful cluster behind them.”*

“Two very different problems, and I would say that first, because the false-positive cost is completely different on the two tiers.

**For the two hundred stateless servers, false positives are cheap.** Removing one healthy instance from a pool of two hundred costs half a percent of capacity for a few seconds. Losing a dead one from the pool slowly costs real errors. So I lean aggressive.

Load-balancer health checks, pull-based, every 2 seconds, 2 consecutive failures to eject, 2 successes to return. That is about 4 to 6 seconds of detection. At 10,000 requests a second across 200 machines, each serves 50 per second, so a dead instance costs me roughly 250 failed requests before ejection — and with retries on the client, most of those are invisible to users.

**The check itself is shallow** — process alive, event loop responsive, no dependency calls — because 200 instances all checking the database every two seconds is 100 extra queries a second for nothing, and because a deep check would eject all 200 at once during a database blip. Dependency health is reported on a separate endpoint that feeds dashboards and alerts, not ejection.

**Liveness is separate and much slower.** 10-second period, 3 failures, so 30 seconds, and it only checks things a restart fixes. I would rather be slow to restart than restart the fleet.

**For the five-node stateful cluster, false positives are expensive** — an unnecessary failover means a leader election, a possible rebalance, and a window where two nodes might both think they are the leader. So: tighter heartbeats but a more careful decision. Heartbeats every 500 milliseconds between peers, and **no single node may declare another dead**. A quorum has to agree, which is exactly the two-stage rule Redis Sentinel uses — subjectively down, then objectively down. That removes the entire class of false positives caused by one bad link.

**And the safety net that makes the numbers less critical.** Every leadership change increments an epoch, and every write to shared storage carries it. A node that was evicted and comes back writes with a stale epoch and gets rejected. That means if my detector is wrong — and over a year it will be — the cost is a wasted failover, not two leaders writing conflicting data.

**What I would measure to know whether the numbers are right:** count of ejections per day per tier, and what fraction of ejected instances came back healthy within a minute. If most of them come back, my timeout is too tight and I am des-

trying capacity chasing phantom failures. That ratio is the feedback loop, and I would put it on a dashboard rather than tune by intuition.”

## 9 Recall card

**You cannot distinguish dead from slow.** Crashed, paused, network-dropped, and reply-lost all look identical. A detector reports suspicion, never truth.

**Two ways to be wrong, and you must price both.** False positive: eject a healthy node, lose capacity, cause churn. False negative: keep sending work to a corpse. Shortening the timeout trades one for the other, and there is no setting that avoids both.

**Detection time = interval × misses allowed.** Say it as a number. Raft 150–300 ms, Kubernetes liveness 30 s, ALB defaults 60 s — the spread is each system pricing its own false positive.

**Pick the timeout from the check’s own latency distribution** — a few times p999, above the known GC pause — not by guessing. Liveness only checks what a restart can fix; dependencies go in readiness.

**Make wrong detection harmless.** Fencing tokens, ownership epochs, quorum before eviction. “I cannot make detection correct, so I make incorrect detection survivable” is the sentence that ends this question.



**DSA topic:** Tries revision and mock round **System design topic:** Failure detection, heartbeats, and timeouts

### Code these, in this order

This is a closing day, so the rule is different: **write every one of these from an empty file, with the lesson closed.** If you have to look something up, finish the problem, then write that template out three times from memory before starting the next one.

| # | PROBLEM                                    | SOURCE                | WHAT IT IS REALLY TESTING                                                                  |
|---|--------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------|
| 1 | Implement Trie (Prefix Tree)               | LeetCode 208 (Medium) | The walk, and the one condition between <code>search</code> and <code>starts_with</code> . |
| 2 | Design Add and Search Words Data Structure | LeetCode 211 (Medium) | The branching walk, and the base case that must not be <code>True</code> .                 |
| 3 | Word Break                                 | LeetCode 139 (Medium) | Trie plus memo — mock problem 1, timed.                                                    |
| 4 | Stream of Characters                       | LeetCode 1032 (Hard)  | The reversed trie — mock problem 2, timed.                                                 |
| 5 | Maximum XOR of Two Numbers in an Array     | LeetCode 421 (Medium) | A trie over <i>bits</i> , not letters. The shape nobody expects.                           |

### Problem 5 is the one to do properly

Nothing in this phase mentioned numbers, and this is a trie problem. Insert each number as its 32 bits, most significant first. Then for each number, walk the trie choosing the *opposite* bit at every level, because the opposite bit is what maximises XOR at that position.

Write down, in one sentence, why greedily taking the opposite bit at the highest position is correct. If you cannot, that is the thing to fix — the structure is easy and the argument is the interview.

### Run the mock round properly

Twenty minutes each for problems 3 and 4, with a timer, standing up, talking the whole time. Record yourself if you can bear to. Then listen back for three things:

1. Did you name the recognition reason before writing any code?

2. Did you say the cost out loud without being asked?
3. When you got stuck, did you keep talking, or go quiet?

The third one is the one that fails interviews.

#### Then the negative drill

Take five problems you have solved with a trie and, for each, write one line saying what you would use instead if the trie were banned, and what it would cost. Two of them have a better non-trie answer than you expect.

---

#### The recognition drill

1. State the recognition question in one line, all three parts.
2. Say what each of the three parts throws out.
3. Name the four shapes and give a problem for each.
4. Say the signal for shape 4 in one sentence.
5. Give four situations where the answer is “not a trie”.

#### The templates drill

Write each from an empty file, timed:

1. Node, insert, `_walk`, `search`, `starts_with`. Target: three minutes.
2. `delete` with the pruning rule. Target: five minutes.
3. `search_pattern` with wildcards. Target: three minutes.
4. `suggest` with gather-and-rank. Target: four minutes.
5. `find_words` on a grid. Target: seven minutes.

#### The costs drill

1. Give the cost of every operation in the table, from memory.
2. Say the sentence about what none of them depend on.
3. Give the wildcard cost and say what matters more than the number of dots.
4. Give the two `suggest` costs and say which is worse on a short prefix.
5. Say what is *not* in Word Search II's time complexity, and why that is the whole point.

#### The memory drill

1. Compute node count for 100,000 English words, worst case and realistic.
2. Compute the trie's memory and the set's memory, and give the ratio.

3. Name the three ways to cut memory, in order of what they buy.
4. Say what a radix trie collapses and roughly how much it saves on English.

#### The five-bugs drill

For each of the five, say the symptom before you say the fix:

1. `search` without `is_end`.
2. Pruning a node that ends a word.
3. Wildcard base case returning `True`.
4. The grid cell never restored.
5. A trie where a set would do.

Four of the five produce no error message. Name which one does.

---

#### The impossibility drill

1. Name the four things that produce “stopped answering”.
2. Say why no better monitoring separates them.
3. Define false positive and false negative in this context.
4. Say which one a shorter timeout increases and which it decreases.

#### The numbers drill

1. Give the detection-time formula and compute it for 1 s and 3 misses.
2. Quote the detection time for Kubernetes liveness defaults and ALB defaults.
3. Compute failed requests from a dead machine at 10,000 QPS over 20 machines, at 30 s and at 5 s.
4. Compute false evictions per day for a 1 s timeout with a 0.1% overrun rate, across 20 machines.
5. Convert those evictions into lost machine-hours at a 20-second cold start.
6. Compare all-to-all and SWIM message rates on 1,000 nodes.
7. Compute gossip propagation time at one round per second on 1,000 nodes.

#### The tuning drill

1. Describe the method for picking a timeout from a latency distribution.
2. Say why p999 alone is not enough, and what else you look at.
3. Say what you would measure to find out whether your timeout is too tight.
4. Give the ratio that tells you that, in one sentence.

### The probes drill

1. State the difference between liveness and readiness in one line each.
2. Say what a liveness probe is allowed to check, and the rule behind it.
3. Explain the fleet-wide restart incident from first principles.
4. Say why even readiness should usually not fail hard on a shared dependency.

### The gossip drill

1. Say why all-to-all heartbeating fails at a thousand nodes, with the arithmetic.
2. Describe one SWIM round.
3. Explain the indirect probe and exactly which false positive it removes.
4. Say what gossip gives up in exchange, and how long that window is.

### The fencing drill

1. Describe the scenario where a node comes back after being declared dead.
2. Explain a fencing token in three sentences.
3. Say where the token is checked, and why it must be there rather than in the caller.
4. Say what STONITH is and when you would reach for it.
5. Give the one-sentence version of why fencing beats a better detector.

### The failure drill

For each, say what happens and what you would build:

1. A 2.5-second garbage-collection pause with a 1-second timeout.
2. A liveness probe that checks the database, during a database blip.
3. One bad network link between two nodes in a ten-node cluster.
4. A single central monitor, and the monitor dies.
5. A node evicted, its work reassigned, and the node returns.
6. A thousand-node cluster using all-to-all heartbeats.
7. A cluster where 90% of ejected instances come back healthy within a minute.

Two of the seven are tuning problems and five are design problems. Sort them.

---

### Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Here is a dictionary and a string with no spaces. Can it be split into words?* The recognition reason first — repeated prefix questions over a fixed list — then why a set is worse, then the trie walk with memoisation, the early exit as the trie earning its keep, and the cost.
  2. *Why did you pick a trie here?* Prefixes not exact matches; repeated queries against a stable set;  $O(L)$  independent of  $n$ . Then the memory cost with the number, and the two alternatives you considered.
  3. *How do you know a server has died?* You do not — four indistinguishable causes. Then suspicion rather than truth, the two errors priced, detection time as a formula and a number, the timeout chosen from a distribution, and fencing as the reason it does not have to be right.
- 

### Before you move on

- ☐ ☐ I can state the recognition question in one line.
- ☐ ☐ I can name the four shapes and give a problem for each.
- ☐ ☐ I can say the signal for shape 4 without hesitating.
- ☐ ☐ I can name four cases where a trie is the wrong answer.
- ☐ ☐ I wrote all five templates from an empty file.
- ☐ ☐ I can give every operation's cost from memory.
- ☐ ☐ I can say what none of the basic operations depend on.
- ☐ ☐ I can do the memory arithmetic for trie versus set.
- ☐ ☐ I know what a radix trie collapses and why it helps.
- ☐ ☐ I can name the five bugs and their symptoms.
- ☐ ☐ I know which four of the five are silent.
- ☐ ☐ I solved the bit-trie problem and can justify the greedy choice.
- ☐ ☐ I ran both mock problems timed and out loud.
- ☐ ☐ I kept talking when I was stuck.
- ☐ ☐ I can name the four causes of “stopped answering”.
- ☐ ☐ I can define both error types and say which way the timeout moves them.
- ☐ ☐ I can give the detection-time formula and three real systems' numbers.
- ☐ ☐ I can compute failed requests from a slow detection.
- ☐ ☐ I can compute lost capacity from an over-tight timeout.
- ☐ ☐ I can describe how to pick a timeout from a distribution.

- [ ] I know the difference between liveness and readiness and the rule behind it.
- [ ] I can explain the fleet-wide restart incident.
- [ ] I can describe a SWIM round and the indirect probe.
- [ ] I can give the message-rate arithmetic for gossip versus all-to-all.
- [ ] I can explain a fencing token and where it is checked.
- [ ] I can say in one sentence why fencing beats a better detector.
- [ ] I answered all three questions above out loud.

# The nine sections, and what each is for

---

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

## 1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

## 2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

## 3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

## 4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

## 5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

## 6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but  $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$ .

### 7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

### 8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

### 9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.



## Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

### Distributed systems core

#### Day 113 · system-design — Why distributed systems are hard

- **Three things stop being true with two machines: an operation has a THIRD outcome (“unknown”), there is NO shared clock, and memory becomes slow, lossy messages** (~1 ns local against ~0.5 ms in a datacentre and ~150 ms across the world — half a million times).
- **The third outcome is the whole subject.** A timeout may mean: never arrived · executed with the reply lost · still running · dead — **indistinguishable from outside**. Therefore **if you cannot tell whether it happened, it must be safe to repeat** — an **idempotency key** on every write.
- **Exactly-once DELIVERY is impossible** (Two Generals: every ack needs an ack).  
\*\*At-least-once delivery
- idempotent processing = exactly-once EFFECT — **which is what Kafka and everyone else actually mean**. At 1M requests/day with 0.1% ambiguous timeouts, that is 1,000 potential double charges a day\*\*.
- **You cannot order events across machines by timestamp** — clock skew (tens of ms) usually exceeds the interval you are ordering, so **last-write-wins silently discards data**. Use **causality**: Lamport, vector clocks, hybrid logical clocks; Spanner buys its way out with **TrueTime** and a deliberate commit-wait.
- **A timeout is a GUESS that a machine is dead** (FLP: crashed and slow are indistinguishable), so fencing, quorums and epochs exist to be **safe when the guess is wrong**. Build in **timeouts · backoff with jitter · idempotency keys · circuit breakers · propagated deadlines · a correlation id**. And **distribution is not a goal** — it is the price of **scale, availability, latency or isolation**, and one machine is faster and more available if you are buying none of them.

## Day 114 · system-design — The CAP theorem, honestly

- **The correct statement: WHEN A PARTITION OCCURS, choose between consistency and availability.** It says nothing about normal operation. “**Pick two of three**” is wrong because **P is not a choice** — cables cut, and a 20-second GC pause is indistinguishable from a partition. **The only genuine CA system is a single node.**
- **The words are unusual.** C = **linearizability** (every read sees the latest write) — not ACID’s C. A = **every non-failing node returns a non-error response** — not **uptime**. A CP system can refuse during partitions and still have **four nines** of ordinary availability. Conflating those is the most common misuse.
- **A system is not CP or AP — OPERATIONS are.** In one shop: browse AP, cart AP (merge), stock check CP, payment CP. **CP: etcd, ZooKeeper, MongoDB, Spanner, HBase.** **AP: Cassandra, DynamoDB, Riak, DNS, CDNs.** And there is a **third option the theorem does not offer: degrade** — serve labelled stale data, accept and confirm later, keep the catalogue and disable checkout.
- **If AP, name the reconciliation mechanism.** Last-write-wins **silently loses data** and with clock skew may keep the **older** write; version vectors detect true concurrency; CRDTs merge by construction; Amazon’s cart **unions** divergent versions on purpose. A 2-hour partition at 1,000 writes/s per side is **7 million writes** to reconcile.
- **PACELC is the more useful framework: if Partitioned, A or C; ELSE, Latency or Consistency.** The else branch applies to **every request** — local replica ~1 ms and stale, quorum ~5–10 ms, cross-region quorum ~150 ms — and **DynamoDB charges 2× for a strongly consistent read**, which is the trade, priced. Quorums need **odd** cluster sizes: 4 nodes tolerate no more failures than 3, and split 2|2 they stop entirely.

## Day 115 · system-design — Consistency models

- **Consistency is a LADDER, not a switch: linearizable → sequential → CAUSAL → session guarantees → eventual.** Going down buys **latency and availability** and costs **anomalies**. The question is never “how consistent can we be” but “**which anomaly can this feature tolerate?**”
- **Linearizable = as if there were ONE copy, in real time.** Needs the leader or a quorum (~5–10 ms in region, ~150 ms across) and **refuses on the minority side of a partition**. Required whenever **two clients can race for the same thing**: seats, stock, locks, balances, leader election.

- **Causal is the underrated middle:** anything that could have influenced anything else is seen in that order — and it stays **AVAILABLE** during a partition, so it is the strongest model that is still AP. It prevents the anomalies users actually report (a reply above its message); staleness is not one of them.
- **The four session guarantees, by the user's own words:** “*I posted it and it's not there*” = **read-your-writes** (route to the leader for ~30 s, ~10–15% of reads); “*it was there, now it's gone*” = **monotonic reads** (pin to one replica); “*my two edits applied backwards*” = **monotonic writes**; “*the reply is above the message*” = **consistent prefix** (a sharding problem). **Cheapest real win in the phase.**
- **Eventual promises ONLY convergence if writes stop** — not your own writes, not monotonic reads, and with **no bound** on “eventually”. Choosing it is not a design until you choose a **conflict resolution** strategy: last-write-wins (**loses data**, and with clock skew keeps the *older* write) · version vectors · **CRDTs** (no conflicts by construction, but cannot express global invariants like “never below zero”) · application merge. **DynamoDB charges 2× for a strongly consistent read**, and **Jepsen** has found violations in most major systems — documentation is not evidence.

#### Day 116 · system-design — Eventual consistency in practice

- **The question is never “is eventual consistency acceptable” — it is “what does a user see in that window, and how bad is it?”** The same two seconds is invisible on a stranger's like count, a reported bug on your own, and a double booking on a seat. **The rule: the DISPLAY may be stale, the DECISION may not.**
- **Four patterns. OPTIMISTIC UI is the highest-value one** — update locally on tap, send in the background, **revert visibly on failure** — because it makes read-your-writes a *client* problem and costs nothing on the server. Never use it where the revert is not honest (payments, bookings). Then: **show the pending state** (the bank's uncleared cheque), **route around it** (leader reads for 30 s, ~10–15% of reads), and **compensate** rather than prevent.
- **Bounded optimism is the underrated move:** act on stale data where the exposure is small and wait where it is large — which is exactly a card terminal's **offline floor limit**. You do not have to choose between fully trusting stale data and fully refusing.
- **“Eventually” is bimodal and has three mechanisms:** read repair (milliseconds, but only for keys that get READ) · hinted handoff (seconds after a node returns; hints expire) · anti-entropy (minutes to hours, comparing **MERKLE TREES** to find differing ranges in  $O(\log n)$  instead of  $O(n)$ ). The data that stays wrong longest is the data nobody reads.

- **Never eventual: two users can get the same thing, or the mistake cannot be corrected without harm** — seats, stock at purchase, unique constraints, permissions, balances at transfer. **Compensate when the expected correction is cheaper than the coordination** (airlines:  $\sim 40\times$  cheaper). And a **counter** is not trivial: read-modify-write is a **lost update** — use an atomic increment, a sharded counter, or **per-replica counters summed on read**, which is a CRDT and needs no coordination at all.

## Day 117 · system-design — Quorums: why R plus W must exceed N

- **$R + W > N$ , and the proof is PIGEONHOLE:** the write set and the read set together exceed the number of replicas, so they **must share at least one** — and that replica has the newest value.  $N=5, W=3, R=3 \rightarrow 6 > 5 \checkmark$ .  $W=2, R=2 \rightarrow 4 \leq 5 \times$ , and you can write  $\{A,B\}$  then read  $\{D,E\}$  and miss it entirely.
- **The overlap is useless without VERSIONS** — every record carries one and the reader takes the highest. A timestamp gives last-write-wins (**silently loses a concurrent write**, and with clock skew keeps the *older*); a **vector clock** distinguishes concurrent from ordered and surfaces the conflict.
- **Never  $W = N$ .** At 99.9% per replica,  $N=3, W=3$  gives  **$\sim 26$  hours/year** of blocked writes — *worse than one machine* — against  **$\sim 1.6$  minutes** for  $W=2$ . And a **quorum's latency is its SLOWEST member**: with responses of 2/3/5/40/200 ms,  $W=3$  costs 5 ms and  $W=5$  costs 200. **Tail-latency insulation is the underrated half of why quorums exist.**
- **A QUORUM IS NOT LINEARIZABILITY.** It guarantees a value at least as new as the last *completed* write, not a global order. Three holes: a **partial write** from a dead client is neither committed nor rolled back; **concurrent writes** have no ordering; and **reads can go backwards** after partial read repair. For a total order you need **consensus** (Paxos/Raft) — Cassandra's lightweight transactions,  $\sim 4\times$  the cost.
- **A SLOPPY quorum accepts  $W$  acks from ANY nodes with hints for the absent owners** — the write survives a partition and the inequality's guarantee is **suspended** until hints replay. Dynamo-lineage systems default to this. Healing is **read repair** (hot keys, milliseconds) · **hinted handoff** (a returning node) · **anti-entropy with Merkle trees** (cold keys). Across regions use **LOCAL\_QUORUM**:  $\sim 5$  ms instead of  $\sim 150\text{--}300$  ms, with the guarantee holding only within the region. **Even  $N$  buys nothing.**

## Day 118 · system-design — Leader election

- **A leader exists to make coordination cheap: one writer means ordering is free and conflicts are impossible.** That is why replication, sharding, locks and schedulers are built on one. The everyday case is a **nightly job on 50 machines running 50 times**.
- **The election: heartbeats stop → a RANDOMISED timeout (150–300 ms) fires → a candidate increments the TERM, votes for itself, and needs a MAJORITY.** One vote per term, so two candidates cannot both win — at most one leader per term. **Randomisation prevents split votes** and makes elections finish in one round.
- **The election is the EASY half. The hard half: a paused leader is indistinguishable from a crashed one** — a GC pause longer than the timeout is the most common real cause of split-brain — **and it wakes up confident**.
- **FENCING is the answer, and the STORAGE must enforce it, not the leader:** every write carries a monotonically increasing term/epoch, and anything older is rejected. Plus a **quorum requirement** so a partitioned leader stops on its own (hence **odd cluster sizes**), and **STONITH** where available. **A lock without a fencing token is not a lock.**
- **Do not implement Raft — take a lease from etcd or ZooKeeper** (ephemeral sequential nodes, each watching only its predecessor to avoid a herd) and **use their revision/zxid as the token**. The cost is a critical-path dependency — Kubernetes stopping when etcd loses quorum is correct, not a bug. And the leader is a **write bottleneck**, answered by **one leader per shard; client redirection, not the election**, usually dominates failover time.

## Day 119 · system-design — Consensus, and Raft in plain English

**Consensus:** a group agrees on a value and stays agreed, despite crashes, delays and lost messages. Agreement, validity, termination.

**The real problem is a sequence, not a value.** Hence the **replicated log**: same start state + same commands + same order = same end state. That is **state machine replication**, and it is the whole idea.

**Raft has three parts:** 1. **Election** — one leader per term, majority vote, one vote per term. 2. **Replication** — leader appends, pushes out; a majority storing it makes it **committed**. 3. **Safety** — restrictions so a new leader cannot lose committed data.

**The two safety rules:** - **Election restriction** — vote only for a candidate whose log is at least as up to date as yours (compare by term first, then index). “Whose book is longest.” - **Current-term rule** — a leader only advances the commit point on an entry from its *own* term.

**Log matching:** same index + same term  $\Rightarrow$  identical logs up to there. One check verifies all history.

**Commit = a majority has it stored.** Permanent, because any future majority overlaps this one.

**A leader seeing a higher term steps down immediately.** No argument.

**Numbers:** one write = 4-12 ms in one region, **dominated by the disk sync, not the network**;  $\sim$ 10-40k writes/sec for etcd; heartbeat 50 ms; election timeout 150-300 ms randomised; algorithmic failover  $\sim$ 200-300 ms but **observed failover 1-10 s because clients must rediscover the leader**; cross-region latency = round trip to your second-nearest peer; etcd holds  $\sim$ 2 GB, config not data.

**Cluster sizes are odd.** 3 tolerates 1, 5 tolerates 2. Beyond 5, latency grows for little gain.

**Raft is CP.** The minority side refuses writes rather than diverging.

**It does not give you:** exactly-once execution (add client IDs), free fast reads (leases or a round trip), protection from bad commands, tolerance of malicious machines, or scale (shard into many groups).

**Quorum vs consensus:** no order and possible conflicts, versus total order and none. Choose by what a wrong answer costs.

**The best answer to “would you implement Raft?” is no.** Run etcd, or use a database that has it inside.

---

**Next:** [Day 120 — Distributed transactions and two-phase commit](#)

**Previous:** [Day 118 — Leader election](#)

## **Day 120 · system-design — Distributed transactions and two-phase commit**

**The problem:** each machine guarantees all-or-nothing locally; nobody guarantees it across machines.

**2PC has one coordinator and several participants.**

**Phase 1 — prepare:** “can you commit, for certain?” Participant does the work, writes it durably, **takes locks**, votes. **A yes is an irrevocable promise** and must survive a crash — fsync before voting.

**Phase 2 — commit/abort:** all yes → coordinator writes `COMMIT` to its log (**this write is the decision point**) → tells everyone. Any no or any silence → abort.

**Phase 1: silence** = NO (safe, nothing committed). **Phase 2: retry forever** (the decision exists).

**The flaw: 2PC blocks.** Coordinator dies after the votes, before the decision → participants cannot commit (maybe a no), cannot abort (maybe a commit), cannot decide alone (they promised). **Locks held until a human resolves it.** No protocol avoids this under partitions.

**The asymmetry:** participant in doubt must **ask**; coordinator in doubt **decides ABORT**.

**Numbers:** ~5-18 ms vs 1-5 ms locally, **four fsyncs**; locks held ~10× longer → ~10× less throughput on hot rows; in-doubt window minutes to hours. Cross-region ≈ 450 ms, set by the **slowest** participant.

**Availability is a product:** 3 participants at 99.9% → 99.7%. **Consensus improves with more machines; 2PC gets worse.** Not the same problem: consensus = majority on a value; 2PC = unanimity on “did it happen”.

**The modern fix:** replicate the coordinator’s log with Raft. In-doubt window becomes one election. Spanner, CockroachDB, TiDB.

**3PC:** shrinks the window, fails under partitions, nobody uses it. Know it, do not propose it.

**XA** is the standard; `PREPARE TRANSACTION` in Postgres, where `max_prepared_transactions` defaults to 0.

**2PC gives atomicity, not isolation** across participants — a global consistent read needs more.

**Avoid it for:** services you do not own, wide areas, long transactions, many participants, microservices. **The real objection is coupling**, not speed.

**First question to ask: why is the data distributed at all?** Co-locating beats optimising.

---

**Next:** [Day 121 — The saga pattern](#)

**Previous:** [Day 119 — Consensus, and Raft in plain English](#)

## Day 121 · system-design — The saga pattern

A **saga** = a sequence of local transactions, each with a **compensating transaction** that semantically undoes it.  $T_1 \dots T_n$  forward; on failure at  $T_k$ , run  $C(k-1) \dots C_1$  backwards.

**Compensation is not rollback.** Rollback erases history; compensation is a *new forward transaction*. Payment + refund  $\neq$  no payment — two ledger entries, a fee, a confused customer.

**Order by reversibility: irreversible steps go LAST.** You cannot un-send an email.

**Two styles: choreography** (events, no controller — up to  $\sim 3$  steps; the workflow exists nowhere) and **orchestration** (one component owns the workflow, writes state durably before every call — beyond 3 steps).

**What you gave up:** atomicity becomes eventual; **isolation is gone entirely.**

**Three isolation hazards:** dirty read, lost update, fuzzy read. **Four countermeasures (mitigate, never cure):** semantic lock ( `status = PENDING` ), commutative updates ( `balance - 50` , not `balance = 450` ), pessimistic ordering, re-read value.

**Three production rules:** 1. Every step idempotent — idempotency key per step, checked locally *and* at the external service. 2. Compensations idempotent, retried forever with backoff, dead-letter + alert if they truly fail. 3. **Every reservation expires on its own.** The expiry interval is your worst-case inconsistency window.

**The outbox pattern:** write the event to an outbox table in the same local transaction; a poller publishes. Turns two systems into one database. At-least-once  $\Rightarrow$  consumers must be idempotent.

**Numbers:** saga  $\approx 300$  ms - 2 s vs 5-18 ms for 2PC, **but  $\sim 10 \times$  the throughput on hot rows** because there are no cross-service locks. 2PC availability is a product ( $4 \times 99.9\% = 99.6\%$ ); a saga turns an outage into a delay. 2% late failures on 1M orders = 20,000 refunds/month.

**Saga vs 2PC in one line:** trades atomicity and isolation for availability and decoupling — and moves the complexity out of the protocol and **into your application code.**

**Wrong choice when:** compensation is impossible, the window is unacceptable, the data could live together, or there are two steps and a retry would do.

---

**Next:** [Day 122 — Idempotency and exactly-once delivery](#)



**Previous:** [Day 120 — Distributed transactions and two-phase commit](#)

## Day 122 · system-design — Idempotency and exactly-once delivery

**A timeout is not a failure, it is an unknown.** Three indistinguishable cases: never arrived, succeeded but the reply was lost, still running. Assuming failure is how double charges happen.

**Idempotent means doing it twice equals doing it once.** The key is generated by the *client*, once per logical operation, and reused on every retry.

**Insert the key before doing the work**, with a unique constraint, and store the response against it. Check-then-act loses to two concurrent retries; the constraint does not.

**Exactly-once delivery is impossible** — the last message can always be lost. At-least-once delivery plus idempotent processing gives exactly-once *effect*, which is what everyone means.

**The numbers:** ~670 bytes per key, 6.7 GB a day at ten million payments, 24-hour retention, against ~1,800 double charges a day and ₹14 lakh if you skip it.

## Day 123 · system-design — Clocks, ordering, and why time is a lie

**A timestamp does not order events across machines.** Skew is 0.1–5 ms in one data centre and 10–100 ms across regions; whenever skew exceeds the gap between two events, the timestamps can be exactly backwards.

**Order by causality instead.**  $A \rightarrow B$  if same machine and earlier, or A sent the message B received, or by a chain. No clock appears in that definition. Neither direction holding means **concurrent** — no information flowed, and nothing can order them.

**Lamport clock:** one counter,  $\max(\text{mine}, \text{theirs}) + 1$  on receive. Guarantees  $A \rightarrow B \Rightarrow L(A) < L(B)$  and *only* that. **Vector clock:** one counter per machine, compares element-wise, and is the only one that can *detect* concurrency — at 16 bytes per participant.

**Last-write-wins silently discards data** inside the skew window; that is Cassandra's default. HLCs (16 bytes, readable, causally correct) are the modern default; TrueTime buys global order for ~10 ms of commit-wait per transaction plus atomic clocks.

**Use the monotonic clock for every duration** — timeouts, leases, rate-limit windows. The wall clock jumps backwards, and that bug shows up as a rate limiter letting through double.

## Day 124 · system-design — Failure detection, heartbeats, and timeouts

**You cannot distinguish dead from slow.** Crashed, paused, network-dropped, and reply-lost all look identical. A detector reports suspicion, never truth.

**Two ways to be wrong, and you must price both.** False positive: eject a healthy node, lose capacity, cause churn. False negative: keep sending work to a corpse. Shortening the timeout trades one for the other, and there is no setting that avoids both.

**Detection time = interval × misses allowed.** Say it as a number. Raft 150–300 ms, Kubernetes liveness 30 s, ALB defaults 60 s — the spread is each system pricing its own false positive.

**Pick the timeout from the check's own latency distribution** — a few times p999, above the known GC pause — not by guessing. Liveness only checks what a restart can fix; dependencies go in readiness.

**Make wrong detection harmless.** Fencing tokens, ownership epochs, quorum before eviction. “I cannot make detection correct, so I make incorrect detection survivable” is the sentence that ends this question.

## Heaps and priority queues

### Day 113 · dsa — The heap: a tree stored in an array

- **TWO rules, and the second is the one people forget.** **VALUE:** every parent  $\leq$  both children — much weaker than a BST, because siblings are **UNORDERED**. **SHAPE:** a **COMPLETE** tree, every level full except the last, filling left to right. The shape rule is why the structure exists.
- **Completeness means no gaps, so the tree lives in a plain ARRAY:**  $\text{left} = 2i + 1$ ,  $\text{right} = 2i + 2$ ,  $\text{parent} = (i - 1) // 2$  (0-based; the 1-based forms are  $2i$ ,  $2i+1$ ,  $i // 2$  — **never mix them**). No pointers, no node objects, and the height is  $\log n$  **guaranteed**, not expected.
- **The promise is narrow:**  $O(1)$  **peek**,  $O(\log n)$  **push and pop**,  $O(n)$  **build** — and  $O(n)$  **for everything else**. Finding an arbitrary value is a **linear scan**, because no pruning is possible when siblings are unordered. **A heap knows its minimum and nothing else**, and its array is **not sorted**.
- **Memory is the underrated win:** 1,000,000 integers  $\approx$  8 MB as a heap against  $\sim$ 120 MB as a **pointer tree**. For “insert a million, take the smallest thousand”, a heap is  $\sim$ 1 million operations against 20 million for a sort.

- Use it whenever the question is “what is next?” — schedulers, Dijkstra, top-k, merging k sorted lists, running median, heapsort. **Do not use it to search.** If you need find-and-update too, that is an **indexed heap** (heap plus a value→index map). And Python’s `heapq` is min-only, operates on a plain list, and breaks silently if you mutate that list directly.

#### Day 114 · dsa — Push, pop, and heapify

- Both operations move ONE value along ONE root-to-leaf path —  $O(\log n)$ . Sift **up** compares with the single parent (one comparison per level); sift **down** compares with both children and must swap with the **smaller** — swapping with the larger leaves the property broken, silently.
- Insert APPENDS then sifts up; extract moves the LAST element to the root then sifts down. Both are forced by completeness: the end is the only position you can add to or remove from without leaving a hole the array cannot represent.
- Heapify is  $O(n)$ , not  $O(n \log n)$ . Start at the last parent  $(n//2 - 1)$  and go backwards (a node’s subtrees must already be heaps). Half the nodes are leaves and do zero work; a quarter sink  $\leq 1$ ; the far travellers are the rare ones at the top.  

$$\sum (n/2^{(d+1)})(h-d) = n \cdot \sum k/2^k = 2n.$$
- The asymmetry is the clincher: top-down with sift-UP (n pushes) really is  $O(n \log n)$ , because then the many bottom nodes can travel the full height. Measured at  $n = 1,000,000$ : ~1M swaps against ~4M. If you have all the items, `heapify` — never  $n$  pushes.
- Heapsort is  $O(n \log n)$  worst case with  $O(1)$  extra space — the only common sort with both — and still 2–3× slower than quicksort because sift-down strides  $2i+1$ ,  $4i+3$ ,  $8i+7$  and destroys cache locality. Changing a priority is  $O(\log n)$  only once you know the index (decrease → sift up, increase → sift down); finding it is  $O(n)$ , hence indexed heaps or lazy deletion.

#### Day 115 · dsa — Python’s heapq, and the min-heap-only problem

- `heapq` is MIN-ONLY. Two fixes: NEGATE (numbers only — `push(-v)`, `pop` and negate back) or INVERT `__lt__` (any type, and the only way to express compound orderings). `heapq` uses only `<`, so `__lt__` is the only method a class needs. The private `_heapify_max` family is incomplete — name it and reject it.
- Always push (priority, counter, payload). Tuples compare left to right, so equal priorities make Python compare the payload → `TypeError: '<' not supported between instances of 'dict' and 'dict'` — and only on a collision,

so it passes tests and fails in production. The tie-breaker needs **both**: always comparable (an int — a *name* fails) and never equal (`itertools.count()` — a *timestamp* fails). Bonus: FIFO within a priority.

- **A heap cannot find or update — so changing a priority is LAZY DELETION:** push the new version, leave the old, skip it when it surfaces via a live sequence number. Costs  $O(\text{pushes})$  entries instead of  $O(\text{live})$  — and it is exactly what a Python Dijkstra does.
- **There is no heap object — the list IS the heap.** `append`, `h[i] = x`, or sorting it yourself breaks the invariant **with no exception**; so does mutating an item already inside; so does `float('nan')`, which compares `False` both ways. Assert `all(h[(i-1)//2] ≤ h[i] ...)` in tests.
- **heapify is  $O(n)$ ;  $n$  pushes is  $O(n \log n)$  — measured  $\sim 4\text{--}5\times$  slower for the same result.** Use `heapreplace` (one sift-down) rather than pop-then-push in a fixed-size top-k loop. And know where heaps stop winning:  $k = 1 \rightarrow \text{max}()$ ;  $k \leq n/10 \rightarrow \text{nlargest} (O(n \log k))$ ;  $k$  near  $n \rightarrow \text{sorted}()$ . `nlargest(..., key=)` avoids tuples and tie-breakers entirely, and `heapq.merge` is **lazy**.

## Day 116 · dsa — Top K problems

- **Three approaches, and the answer is which one and why:** sort ( $O(n \log n)$ , ordered, simplest) · size-k heap ( $O(n \log k)$ ,  $O(k)$  space, the **ONLY streaming option**) · quickselect ( $O(n)$  average,  $O(1)$  space, but it mutates the input). Ask: how big is  $k$  relative to  $n$  · in memory or a stream · do you need them in order?
- **For the  $k$  LARGEST, keep a MIN-heap of size  $k$  —** the heap holds candidates, and the only question you ask is “*better than the worst I am keeping?*”, so you need the **minimum** at the root. Mirror rule:  $k$  smallest  $\rightarrow$  a max-heap. Use `heapreplace`, not pop-then-push.
- **The `v > heap[0]` test is where the speed is.** On random data only  $\sim k \cdot \ln(n/k)$  items ever enter the heap —  $\sim 100$  insertions out of 1,000,000 at  $k = 10$  — so the practical cost is far below the bound. Worst case is **ascending input**, where every item is inserted.
- **Quickselect is  $O(n)$  because it recurses into ONE side:**  $n + n/2 + n/4 + \dots = 2n$ . It is  $O(n^2)$  when pivots are extreme, and **sorted input with a fixed pivot does exactly that — so randomise the pivot, always.** Name **median-of-medians** (guaranteed  $O(n)$ , bad constants) and **introselect** (what real libraries do).

- Crossover is around  $k \approx n/10$ , decided by constants.  $k = 1 \rightarrow \max()$ . Both the heap and quickselect return results UNORDERED — add  $O(k \log k)$  if order matters. For **top-k frequent**, the heap is over the **distinct** items, and **bucket sort by count** is  $O(n)$  because the key is bounded — that is the “can you do better?” answer.

#### Day 117 · dsa — Merging K sorted lists

- The invariant IS the proof: each list is sorted, so nothing behind a list’s front is smaller than that front — therefore the overall smallest is one of the  $k$  fronts, and there is nowhere else it can be. Keep exactly those  $k$  candidates in a heap.
- The heap holds  $k$ , never  $n$  — every pop is followed by at most one push from the same list, so the size is invariant. That gives  $O(n \log k)$  time (7M comparisons against 20M for a sort at  $n = 1M$ ,  $k = 100$ ) and — the part that matters more —  $O(k)$  space, which is what lets it work on data larger than memory.
- The entry is `(value, list_index, position)`. The index says which list to advance, and because it is always **distinct** it doubles as the **tie-breaker** — equal values across lists are common, and without it you get `TypeError` on unorderable payloads.
- There is a second  $O(n \log k)$  with no heap: a pairwise TOURNAMENT —  $\log k$  rounds,  $O(n)$  each. Better cache locality, trivially parallel,  $O(n)$  space ( $O(1)$  for linked lists, which is why the classic problem uses them), but it cannot stream. The trap is SEQUENTIAL pairwise merging, which re-walks the accumulator and is  $O(n \cdot k)$ .
- This is the merge phase of an EXTERNAL SORT: sort runs that fit in memory, then k-way merge holding one block per run. It is why **merge is the disk world’s sorting algorithm and quicksort is memory’s**. Edge cases that bite: **skip empty lists** when building the heap (`IndexError` otherwise), bound-check before pushing, and expect the heap to **shrink** as lists are exhausted.

#### Day 118 · dsa — Two heaps: the running median

- Keep the numbers already SPLIT so the median sits at the boundary. `low` is a **max-heap** of the smaller half (its **largest** on top), `high` is a **min-heap** of the larger half (its **smallest** on top) — those two are the middle values.  $O(\log n)$  add,  $O(1)$  median.

- **Two invariants, and everything follows: ORDER** (  $\max(\text{low}) \leq \min(\text{high})$  ) and **BALANCE** (sizes differ by  $\leq 1$ ). Assert both in tests — every bug here breaks one of them **silently**.
- **PUSH, SHIFT, REBALANCE, as one unit:** push into `low` unconditionally · move `low`'s top into `high` (this is what guarantees the ordering) · move one back if `high` is bigger. **No branches, no case analysis.** The branching version is also correct, faster on random data, and has an empty-heap trap on the first insert.
- **Python has no max-heap, so `low` stores NEGATED values — two negations in and out, one when moving across.** And the median function must agree with the rebalance rule about which heap holds the extra element, or you are right on even counts and wrong on odd. Use `/`, not `//`.
- **Forgetting the rebalance is the classic bug:** an increasing sequence drifts everything into one heap and the median reads plausibly and wrongly. Variants: **sliding window** = the same + **lazy deletion**, where you must track **logical** sizes separately from `len(heap)`; **any percentile** = change the balance ratio; **bounded values** → a **count array**, `0(1)`; **at scale** → **t-digest** / **HdrHistogram**, which is what your metrics system really does.

## Day 119 · dsa — Heaps revision and mock round

**The recognition question:** do I repeatedly need the smallest or largest, from a collection that keeps changing? Yes → heap. Fixed collection → sort. Whole order → sort. Once → one pass.

**The three phrasings:** a superlative asked repeatedly; a schedule; a boundary.

**The five shapes:**

| # | SHAPE          | HEAP HOLDS               | COST                         | EXAMPLE                  |
|---|----------------|--------------------------|------------------------------|--------------------------|
| 1 | Priority queue | everything               | $O(n \log n)$                | scheduler, Dijkstra      |
| 2 | Fixed-size k   | exactly k, opposite kind | $O(n \log k)$ , $O(k)$ space | top k frequent           |
| 3 | Two heaps      | the two halves           | $O(\log n)$ in, $O(1)$ read  | running median           |
| 4 | K-way merge    | the k fronts             | $O(n \log k)$                | merge k lists            |
| 5 | Lazy deletion  | stale entries too        | $O(\log n)$ amortised        | Dijkstra, sliding median |

**The costs:** push  $O(\log n)$ , pop  $O(\log n)$ , peek  $O(1)$ , heapify  $O(n)$ , find anything  $O(n)$ .

**Heapify is  $O(n)$**  because half the nodes are leaves that move zero steps:  $n/2 \times 0 + n/4 \times 1 + n/8 \times 2 + \dots \rightarrow n$ .

**The k trap:**  $k$  largest  $\rightarrow$  min-heap. The top is the weakest survivor, the one you evict.

**The `heapq` six:** min-heap only (negate twice); `heapify` returns `None`; `heap[0]` peeks; tuples compare left to right so add a tie-breaker; `len(heap)` lies when deleting lazily; no arbitrary delete.

**The array is not sorted.** Only `heap[0]` is guaranteed. Second-smallest is `min(heap[1], heap[2])`.

**Lead with space, not time.**  $O(k)$  versus  $O(n)$  is the strong argument, and it is the one that makes streams possible at all.

**Not a heap when:** the data is fixed, you need the whole order,  $k \approx n$ , or you must update items by value.

---

**Next:** [Day 120 — The trie: a tree of characters](#)

**Previous:** [Day 118 — Two heaps: the running median](#)

## Scaling fundamentals

### Day 098 · system-design — Vertical versus horizontal scaling

- **Vertical = a bigger machine. Horizontal = more machines.** Ask first: *is one request slow, or are there too many?* **Horizontal only answers the second — throughput scales horizontally, latency does not.**
- **Default to vertical, and name the stopping condition.** It is one command and a restart, changes no code, and adds no failure modes. Stop when **(a)** the price curve bends — flat cost per vCPU to about **32–64 vCPU**, rising sharply above — **(b)** availability is required, at any size, or **(c)** the working set exceeds the biggest machine (**~4 TB RAM**) or one disk's write rate (**~50,000 IOPS**).
- **The real prize of horizontal is availability, not throughput.** One machine at 99.9% is **8.8 hours down a year**; two where either can serve is **~32 seconds** — a  $1000\times$  change no bigger box can buy. But only if they fail **independently**: same rack, same zone, same bad deploy breaks the arithmetic.
- **Horizontal demands statelessness, a load balancer and health checks.** For a database it is two separate steps: **replication** (scales reads, costs lag) then **sharding** (scales writes, costs cross-shard joins and transactions **for ever**). **Sharding a database you did not need to shard is one of the most expensive mistakes in the subject.**

- **The answer is per tier, not per system.** App tier horizontal and auto-scaled; cache horizontal, sharded by key; **primary database vertical, as large as it needs to be**; read replicas a small fan-out; media in object storage. Going from 1 to 50 app servers breaks three things in order: **connections** (need a pooler), **in-process state** (sessions, counters, cron), and **observability**.

#### Day 099 · system-design — Load balancers

- **Two jobs, and the second matters more: spread load, and hide failure.** A balancer turns  $n$  unreliable servers into one apparently reliable service — but only if it knows which are healthy, so **health checks are the real mechanism**, not the algorithm.
- **Least connections by default, not round robin.** Round robin is fair in request *count* and blind to *cost*, so **a server that is slow but alive keeps getting one request in every  $n$ , for ever**. Least connections self-corrects. **Power of two choices** (pick two at random, take the lighter) gets nearly the same quality with no herd effect. **Consistent hashing** when backends hold per-client state.
- **Detection time** = `interval * unhealthy_threshold + timeout` —  $5\text{ s} \times 2 + 2 = 12\text{ seconds}$ , which at 1,000 req/s to that backend is **~12,000 failed requests**. Thresholds exist to stop **flapping**. Halving the interval halves the damage and doubles the false removals.
- **The health endpoint must check only what THIS instance needs.** A check that touches the shared database fails **every** server at once during one hiccup, and the balancer removes the whole pool — degraded becomes down, caused by the safety mechanism. Deep checks go on a separate endpoint for monitoring.
- **Layer 7 for HTTP** — path routing, TLS termination, and **retries** (safe for `GET`, not for `POST`); **layer 4** for raw speed and millions of connections. **Never run one balancer** — two zones behind DNS or anycast. **Connection draining** is how you deploy without dropping requests. **Sticky sessions are a compromise**, not a design: uneven load, sessions lost on failure, and no clean draining.

#### Day 100 · system-design — Stateless services and why they scale

- **Stateless means any instance can serve any request, and killing one loses nothing that mattered.** It does **not** mean there is no state — the state moved out of the server into a shared store. The test: *if this machine lost its memory now, would the answer be wrong or just slow?* Slow is fine.



- **Four things break without it, and the list is the answer:** you cannot **add** a machine (new users logged out), cannot **remove** one (their sessions gone), cannot **deploy** without disruption, and a **crash becomes user-visible** to exactly  $1/n$  of users.
- **Session in Redis by default** — one lookup of  $\sim 0.5$  ms against a 50–200 ms request, **under 1%**, and 1M sessions  $\times$  1 KB = **1 GB**. A **signed token** removes the store but **cannot be revoked**, cannot be updated server-side, and costs  $\sim 800$  bytes on **every** request ( $\sim 69$  GB/day at 1,000 req/s). Usual answer: short-lived token plus a refresh token that is checked.
- **The state people forget: rate-limit counters** (10 servers each allowing 100 = an effective limit of **1,000**, silently — a correctness bug), **scheduled jobs** (one run per machine), local **file uploads**, and **per-process id counters** (two servers issue the same order number). An in-process cache is allowed if a miss is only slow.
- **Sticky sessions are not a scaling technique — they are a symptom of state in the wrong place.** They cost a  $2-3 \times$  **spread** between busiest and quietest server (so 30–50% wasted capacity), user-visible crashes, no draining, and broken auto-scaling. And the honest cost of going stateless: **you swap  $n$  partial failures for one shared dependency**, so the store must be replicated with failover. **WebSocket connections are pinned — keep the connection on the machine and the meaning in the store.**

#### Day 101 · system-design — Caching: the single biggest win

- **A cache works because of two facts: the layers are orders of magnitude apart** (memory  $\sim 100$  ns, Redis  $\sim 0.5$  ms, DB  $\sim 20$  ms, cross-world  $\sim 150$  ms) **and access is extremely uneven** — the top 10% of items serve  $\sim 90\%$  of requests. **If access were uniform, caching would barely work** — say that assumption aloud.
- **Four layers, not one: browser · CDN · application (in-process or Redis) · database buffer pool.** Work outside in; for media-heavy products the **CDN is the biggest win** and the one candidates skip. The buffer pool is **free**.
- **Do the load arithmetic, not just latency: 6,000 reads/s at a 90% hit rate leaves 600/s on the database —  $10 \times$ , the difference between sharding and not.** 95% halves it again, because **at high hit rates the misses are the entire cost**. Sizing: cache  $\sim 10\%$  of the data for  $\sim 90\%$  hits (20 GB  $\rightarrow$  2 GB).
- **Use cache-aside: on a miss, query and populate. Then a cache failure makes the system slow, not broken** — read-through makes it broken. Write path: **write-around by default**, write-through when reads follow writes, **write-back only for data you can afford to lose**.

- The TTL is a dial between staleness and load, and nothing else. And the stampede is the failure that turns a cache into an outage — a popular key expires, 1,000 in-flight requests issue 1,000 identical queries in 50 ms. Fix with a **per-key lock** and **jittered TTLs**. Finally: **a cache miss must make the answer slow, never wrong** — cache the *display*, never the *decision*.

## Day 102 · system-design — Cache invalidation and eviction policies

- **Invalidation is correctness (the entry is wrong); eviction is capacity (the entry is right but does not fit)**. Different mechanisms, different failures. Default: **write-around** — write the DB, then **DELETE** the key — plus a **TTL on every key** as the bound on every invalidation bug you have not found.
- **Delete, do not update, on a write, and write the database FIRST, then invalidate** — deleting first leaves a window as long as the DB write in which a reader caches the old value with a fresh TTL.
- **The race to be able to draw**: reader misses → reads 60 from the DB → writer sets 70 and deletes → reader sets 60. **Cache 60, DB 70, and nothing will correct it until the TTL**. On a key read  $100\times/s$ , a write has  $\sim 20\%$  chance of hitting the window. Fixes in order: **TTL · delete-not-set · delayed double delete · per-key lock or leases**.
- **Write-through** when a read always follows the write (a user's own profile). **Write-around** by default. **Write-back only for data you can lose** — a counter watched 10,000 times a minute becomes **one** DB write instead of 10,000.
- **LRU by default, and `allkeys-lru` in Redis — the default is `noeviction`, which returns errors when full**. Redis's LRU is **approximate** (sampled). Its weakness is **scans**: a nightly export evicts the whole working set, hit rate  $95\% \rightarrow 5\%$ , DB load  $\sim 10\times$ , recovery 30–90 minutes — fixed by **admission control**, not a bigger cache. And for **one write invalidating many keys**: publish an **event**, or **version a namespace** so one `INCR` invalidates thousands.

## Day 103 · system-design — Content delivery networks

- **A CDN exists because distance is physics, not engineering**. India ↔ Virginia is  **$\sim 200$  ms round trip** and no code change removes it. Edges put a copy  **$\sim 8$  ms** away. The **TLS handshake** (2–3 round trips) terminates at the edge too — often a **bigger saving** than the content itself.
- **Quote the byte split**:  **$\sim 95\%$  of a page's bytes are shared assets** (images, JS, CSS, video), so a CDN removes almost all origin bandwidth **before Redis is consulted** — 200 GB/hour → 10 GB/hour at 100k page loads. And CDN bandwidth is **cheaper**

than cloud egress, so it usually saves money too.

- **Origin pull by default, plus an ORIGIN SHIELD** — without it, 300 edges each cold-miss and you get a  $300 \times$  burst at the origin per new object. Push distribution for large predictable objects (game patches, film releases; Netflix Open Connect ships boxes into ISPs).
- **What it cannot do: personalised content and writes.** A shared cache has nothing to share if the response differs per user; a `POST` always travels the full distance. **Split the page** — static shell at the edge, personal data via an uncached API call. **Caching a personalised response or a `Set-Cookie` is a data-breach bug, not a performance bug.**
- **Do not purge — rename.** Content hashes ( `app.a3f9c2.js` ) make a new build a new key: no invalidation, `max-age` of a year, and **old and new coexist safely** mid-session. And when the hit rate is bad, **check the cache key first:** `utm_*` and `fbclid` create a new object per share link (99% → 25%), and `Vary: User-Agent` effectively disables caching.

#### Day 104 · system-design — Database replication

- **Leader-follower: exactly ONE machine accepts writes and ships an ORDERED log of changes.** Order is not negotiable. **Row-based** replication by default — **statement-based breaks on anything non-deterministic** ( `NOW()` , `RAND()` , auto-increment races) and diverges silently. PostgreSQL ships the WAL.
- **Semi-synchronous is the real answer: wait for ONE follower, let the rest lag.** Asynchronous loses the lag on a crash (~200 writes at 1,000/s and 200 ms lag; 5,000 during a bulk job). Fully synchronous loses nothing but makes **every follower a single point of failure for writes**. Cross-region synchronous is ~200 ms per write — never.
- **Failover is five steps: detect · elect · FENCE · promote · redirect.** Quote **RPO** (data lost = the lag) and **RTO** (10–90 s automated, minutes to hours manual). **Client redirection through DNS makes  $RTO \geq \text{TTL}$**  — use a proxy or virtual IP.
- **Split-brain is the worst failure and it cannot be auto-repaired:** an old leader that was only *unreachable* keeps accepting writes, and two leaders assign the same ids to different data. Defend with **fencing**, a **quorum** (leader must see a majority — hence odd cluster sizes), and **epoch numbers** so followers reject a stale term.
- **Replication is NOT a backup** — a `DELETE` with no `WHERE` reaches every follower in milliseconds. Machine failure → replication; human error → **point-in-time backups**, plus a **deliberately delayed follower** as a cheap partial defence. And

replication does **nothing for write throughput** — that is sharding. Adding a follower to a 2 TB database takes **~5–6 hours**, so it is not a lever you pull during an incident.

#### Day 105 · system-design — Read replicas and replication lag

- A read replica is a follower you also query. Reads scale near-linearly; writes do not scale at all — every replica applies every write, so at 4,000 w/s on a 5,000 w/s machine each replica has only 20% left for reads, and the eleventh replica adds nothing. Past that, the answer is **sharding**.
- The failure users report is a read-your-own-writes violation, and nothing is broken — the write went to the leader, the read went to a replica that had not applied it. The UI reloads immediately after a write, so the read lands **inside** the lag window by design.
- Four fixes: read from the leader for ~30 s after a write (default; costs ~10–15% of reads back on the leader) · object-based routing (only what the user just touched; a few percent) · a consistency token — the write returns a log position, the read waits for a replica that has reached it (DynamoDB, Aurora) · sleeping, which is not a fix.
- Monotonic reads is the uglier sibling: two reads on replicas with different lag make a comment appear and then vanish — it looks like data loss. Fix by pinning a user to one replica (hash their id); the cost is uneven load and one backwards jump if that replica dies.
- Measure lag with a heartbeat row, not just `Seconds_Behind_Master`, and evict a replica from the read pool automatically past a threshold — a replica minutes behind is worse than one fewer replica. And triage reads like a cache: the display may be stale, the decision may not — balances, stock at checkout and permission checks always come from the leader.

#### Day 106 · system-design — Sharding, part one: choosing the key

- Replication copies all the data everywhere and scales READS; sharding splits the data and scales WRITES and STORAGE. They are orthogonal — real systems shard into N groups and replicate within each.
- The question that chooses the key: what is the dominant query, and what does it filter by? Then four requirements: in the dominant query · high cardinality · evenly distributed · never changes. That is usually `user_id`, `conversation_id`, or — cleanest of all — `tenant_id`.

- **Hash by default; range only when range queries dominate AND the key is not sequential.** Range on anything sequential gives an **append hotspot**: at 10,000 writes/s over 4 shards, **one machine takes all 10,000 and three sit idle** — quadruple the cost, no capacity. The fix is a **composite key: partition by something that spreads, order by time within it.**
- **What breaks, permanently: no cross-shard joins · no cross-shard transactions · no unique constraint except on the key.** Plus **auto-increment ids collide** — use **Snowflake ids** (timestamp + machine + counter in 64 bits). **Co-locate related tables on the same key**, and replicate small reference tables to every shard instead of splitting them.
- **A query without the shard key is a scatter-gather, and it costs twice: latency is the SLOWEST shard** (16 shards with a 10 ms p99 → ~15% chance one is slow, so your p50 becomes their p99) **and availability is the product** ( $16 \times 99.9\% = 98.4\%$ , i.e. 140 hours a year). **Do not shard until a single machine is out of write capacity (~5,000 writes/s) or disk** — the cost is not the migration, it is that every future feature lives inside today's key.

#### Day 107 · system-design — Sharding, part two: rebalancing and hot spots

- **Never run plain `hash % N`: going from 8 machines to 9 moves ~89% of the data.** Fix with an **indirection** — a large fixed number of **logical shards** (1,024, a power of two) hashed once and never changed, plus a small **map** from logical shard to machine. Then adding a machine moves  $1/(N+1)$  of the data and the routing rule is untouched. **The logical shard count is a permanent ceiling on machine count** — Redis Cluster's is 16,384.
- **Resharding live is four phases and the order is the safety: DUAL WRITE → BACKFILL → VERIFY → CUT OVER gradually.** Everything is reversible until the last two. **Rate-limit the backfill** (it competes with live traffic) and **do not shorten verification** — that is where every failed migration went wrong. A single logical shard moves with a **millisecond** write pause.
- **A hot spot has FOUR causes and you must diagnose before prescribing: uneven map (free) · one hot key (hours) · skewed key distribution (a quarter) · sequential key (a quarter).** The metric that decides it is **top keys within the shard**, not just per-shard traffic.
- **A hot shard caps the whole cluster:** 60% on one of four shards means the ceiling is that machine's capacity ÷ 0.6 — about a third of the hardware you are paying for. **Caching one hot key** at a 95% hit rate can take a shard from 6,000 to 2,675

req/s in ten minutes. For a **write-heavy** hot key, **salt** it ( `key#0..key#9` ) and pay a 10-way fan-out on reads.

- **The unbounded key has no after-the-fact fix** — a shard key value is atomic, so every row with it is on one machine. **Design a composite key** ( `(tenant_id, month)` , `(conversation_id, message_id/10000)` ) so no single value can grow without limit. And version the shard map, with the old shard **forwarding** during a grace period, or a stale client writes to the wrong machine mid-move.

## Day 108 · system-design — Consistent hashing

- **The problem:** `hash % N` puts the node count **INSIDE** the calculation, so  $8 \rightarrow 9$  nodes relocates **~89%** of keys — for a cache, the hit rate collapses and the origin takes **9,000 req/s** instead of 500. **The fix:** hash keys **AND** nodes into one space, join it into a circle, and a key belongs to the first node **CLOCKWISE**. The node count never appears.
- **Adding a node moves**  $1/(N+1)$  of the keys — **~11%** at  $8 \rightarrow 9$  — **and they all come from ONE neighbour**. That is the theoretical minimum. Removing a node sends its arc to its successor and disturbs nothing else.
- **Virtual nodes (100–256 per machine, Cassandra defaults to 256) exist for TWO reasons, and the second is the one people miss.** Distribution: one point per node lands randomly, so one node can own **63% of the ring**; 200 points brings the spread to  $\sim 1.2\times$  the mean. **Failure spreading:** with one point, a dead machine's entire load lands on one successor and **doubles** it  $\rightarrow$  cascade; with 200, it spreads across the cluster and each survivor takes **~1% more**.
- **Implementation:** sorted positions + binary search,  $O(\log(N \times V)) \approx 15$  comparisons; the ring is under a megabyte. Two real bugs: the **wrap-around** (a key past the last node belongs to the first) and a **non-deterministic hash** — Python's `hash()` is randomised per process, so use MurmurHash or xxHash. Replication falls out for free: the next **R** distinct nodes clockwise.
- **It balances KEYS, not TRAFFIC** — **it does not solve hot keys**, and it gives up **placement control** (hence directory sharding for big tenants and jurisdictions). **Rendezvous hashing is simpler and more even at  $O(N)$  per lookup** — **genuinely better under ~50 nodes**; jump hash is memory-free but only allows removing the last node; Maglev gives  $O(1)$ .

## Day 109 · system-design — Back-of-the-envelope estimation

- **Six steps, in the same order, every time:** users → actions/day → QPS ( $\div 100,000$ ,  $\times 3$  for peak) → read:write split → storage ( $\times$  bytes  $\times 365 \times 3$ ) → bandwidth. Then the **seventh and most important:** say what each number *means* for the design. A number with no consequence attached was not worth computing.
- **Round to one significant figure and SAY that you are** — “86,400 rounded to 100,000, so I am  $\sim 15\%$  low, which changes nothing”. Round **up** for capacity, **down** for savings. The target is the right **order of magnitude**:  $\pm 20\%$  changes nothing,  $\times 10$  changes the design.
- **Sanity-check against something known, every time.** Twitter  $\sim 6,000$  tweets/s; Instagram  $\sim 100\text{M}$  photos/day; WhatsApp  $\sim 100\text{B}$  messages/day; YouTube  $\sim 500$  hours uploaded/minute; Stack Overflow ran on  $\sim$  a dozen servers. The errors that actually happen are **factors**: registered-as-daily-active ( $10\times$ ), per-day-as-per-second ( $86,400\times$ ), forgetting peak ( $3\times$ ), replication ( $3\times$ ), fan-out or transcoding ( $2\text{--}100\times$ ).
- **Reason about actions per user; never guess a number.** “Three or four sessions, five server requests each, images go to the CDN — so twenty.” **The derivation is the answer.** And when you do not know, **ask**; if refused, **assume and label it**.
- **Memorise the consequence table.** Peak QPS:  $< 100 \rightarrow$  one server  $\cdot 1,000 \rightarrow$  a few + one DB  $\cdot 10,000 \rightarrow$  LB + cache + replicas  $\cdot 100,000 \rightarrow$  sharding. Storage: 1 TB  $\rightarrow$  one DB  $\cdot 100$  TB  $\rightarrow$  sharded  $\cdot 1$  PB  $\rightarrow$  object storage. Writes:  $> 10,000/\text{s} \rightarrow$  sharded, no other option. And watch the three invisible constraints: **fan-out**, **connections** (not requests), and **storage accumulating** while QPS does not.

## Day 110 · system-design — Capacity planning: QPS, storage, bandwidth

- **Peak QPS  $\div$  per-server QPS is NOT the answer** — it is the answer to “what is theoretically possible”. Three multipliers follow:  $\div$  **utilisation (0.65)**  $\cdot$  + **redundancy (N + 1)**  $\cdot$   $\times$  **growth**. 6 machines becomes 14.
- **The one idea that matters: queueing delay grows as**  $u / (1 - u)$ . Flat to  $\sim 60\%$ , then vertical:  $80\% = 4\times$  the wait,  $90\% = 9\times$ ,  $99\% = 99\times$ . So **size for 60–70% for user-facing work, 80–90% for batch behind a queue** — that difference is the biggest cost lever available.
- **Verify the failure case before answering.** 14 machines, one dies  $\rightarrow$  13 at 46%; a whole AZ dies  $\rightarrow$  9 at 67%. And **redundancy is proportionally cheaper in a big fleet**: losing 1 of 30 costs the survivors 3.4%, losing 1 of 3 costs them 50%.



- **Autoscaling does not remove headroom.** Metric window + decision + boot + warm-up + health checks = **3–4 minutes**, and a spike arrives in seconds — so **the existing fleet takes the whole spike alone**. Autoscaling saves money in the **trough**; a **queue** is the only thing that absorbs a spike instantly.
- **Three separate plans, and the binding one is rarely the obvious one.** Compute scales with traffic; **storage scales with time and is a ratchet** (retention is the plan — and fully loaded it is  $\sim 5\times$  the raw data after replication, +**30% indexes** and  $\div$  **0.8 free space**); **bandwidth binds first for media** — 6,000 QPS  $\times$  5 KB is 30 MB/s, but  $\times$  2 MB is **12 GB/s**, twelve machines' worth of link. And the per-server figure is an assumption: **load-test to the knee where p99 degrades**, and take the QPS just below it.

#### Day 111 · system-design — Single points of failure

- **An SPOF is any component whose failure stops the system.** Find them two ways: **point at every box and ask what still works**, and **trace one critical path backwards until you leave the system** — the second pass finds what the diagram does not show.
- **The ones that cause real outages are undrawn: DNS · config and secrets · certificates · the deploy pipeline · monitoring · a shared library · one person.** Config being down does not break running instances — it breaks every **restart, deploy and autoscale**, so you cannot recover during the incident.
- **Series multiply availabilities, parallel multiply failure rates.** Six components at 99.9% give  $\sim$ **99.4%** — **two days a year**, so every dependency on the critical path costs availability and **making one optional removes it from the product entirely**. Two parallel copies at 99.9% give **99.9999%** — **nine hours becomes 32 seconds**.
- **That parallel figure assumes INDEPENDENCE.** Same rack, same zone, same deploy, same config push, same certificate — and it collapses: **10% correlation turns 32 seconds into  $\sim$ 53 minutes**. A **bad deploy or config push is the correlated failure redundancy cannot help with**, which makes canaries and fast roll-back availability work.
- **Redundancy  $\neq$  failover  $\neq$  degradation.** Two of something with no tested switching is **a second thing that might also be broken** — and DNS TTL is often the largest term in failover time. **Degradation is usually the best value**. And a **slow dependency is more dangerous than a dead one**: use timeouts, **circuit breakers** and **bulkheads**, then **cells** and **shuffle sharding** to shrink blast radius from 100% of users to a fraction of a percent.



## Day 112 · system-design — Scaling revision and interview questions

- **Diagnose before prescribing, in four questions: one slow request or too many · which resource is saturated · reads or writes · steady or a spike.** *“Throughput scales horizontally; latency does not”* disposes of the most common misdiagnosis, and a **lock** is contention that none of the moves fix.
- **The ladder, cheapest and most reversible first: measure → fix the obvious → scale up → CACHE → CDN → stateless + LB → read replicas → queue → SHARD → multi-region.** Rungs 3 and 4 hold the leverage: a **90% hit rate is 10×** and a **CDN removes ~95% of the bytes**, both in an afternoon.
- **“I’d shard it” as a first answer is the failure the question exists to catch.** Sharding is the only thing that scales **writes**, so shard only above **~5,000 writes/s** or when the working set exceeds the biggest machine — and its cost is **not the migration but every feature afterwards**.
- **Name the price with every move:** cache → staleness and stampedes · CDN → slow purges, nothing personalised · stateless → a hot shared store · replicas → **lag and read-your-own-writes** · queue → at-least-once, caller blind · shard → **no joins, transactions or unique constraints, permanently**.
- **The numbers: 1 billion/day  $\approx$  10,000 QPS · app server  $\sim$ 1,000 QPS · DB  $\sim$ 10,000 reads and  $\sim$ 5,000 writes · Redis  $\sim$ 100,000 ops/s · peak  $\times 3$  · size for 65% utilisation because delay is  $u/(1-u)$  · every replica applies every write.** And check the direction you moved availability: **components in series multiply**, so six at 99.9% is  $\sim$ 99.4% — make the cache **optional**, and prefer **degradation** over redundancy.

## Trees and binary search trees

### Day 098 · dsa — What a tree is, and the vocabulary you need

- **A tree: every node has exactly one parent except the root, and no cycles — so there is exactly one path from the root to any node.** That is why tree walks need **no visited set** (a graph does), and why every tree function is recursive: **each child is the root of a subtree that looks like the original problem.** A tree with  $n$  nodes has exactly  $n - 1$  edges.
- **Depth belongs to a node and is measured DOWN from the root (root = 0). Height belongs to a node and is measured UP from its deepest leaf (leaf = 0). The height of the *tree* is the height of the root.** “The depth of the tree” is not a thing.

- **Height is computed UP from the children — a node can work it out alone. Depth must be passed DOWN as an argument.** That asymmetry decides how nearly every tree problem is written.
- **Say the convention before the base case: edges ( `None` → -1, single node → 0) or nodes ( `None` → 0, single node → 1).** LeetCode 104 uses nodes. The code differs by one character; mixing them makes only *some* branches off by one.
- **Every full walk is  $O(n)$  time and  $O(\text{height})$  space** — ~20 frames on a balanced million-node tree, **1,000,000 on a degenerate one** ( `RecursionError` at ~1,000). **Never assume balance**, and never confuse a **binary tree** ( $\leq 2$  children) with a **binary search tree** (ordered values): finding a value in a plain binary tree is  $O(n)$ , not  $O(\log n)$ . DFS costs  $O(\text{height})$ ; **BFS costs  $O(\text{width})$** , and the widest level of a perfect tree is  $n/2$ .

#### Day 099 · dsa — Binary trees in code

- **Four lines: `val`, `left`, `right`.** There is no `Tree` class — a tree is a reference to its root, and `None` is both “empty tree” and “no child here”, which is why one base case handles both. Use `val`, not `value`, and add a one-line `__repr__` or every debugging print is a memory address.
- **[3, 9, 20, null, null, 15, 7] is level order with a `null` for every missing child, and the nulls are load-bearing** — drop them and 15 and 7 move under 9, silently. Trailing nulls are trimmed. It is **not** the  $2^{i+1} / 2^{i+2}$  array layout: that needs  $2^{(h+1)} - 1$  slots, so a **degenerate tree of 40 nodes would need  $\sim 10^{12}$  entries**.
- **Write `from_list` and `to_list` once and reuse them for every tree problem**, plus the **nine-line sideways printer** (recurse right, print, recurse left) — the best thirty seconds available in a tree round. Make test trees **lopsided, with a one-child node and a repeated value**; a symmetric three-node tree passes almost any wrong solution.
- **There is no parent pointer.** So you cannot go up, cannot delete a node given only that node, and a node cannot know its own depth. **Carry the parent down as an argument** rather than adding a field that every mutation must maintain.
- **`TreeNode(1) == TreeNode(1)` is `False`** — Python compares identity, so structural equality is its own recursion — and `backup = root` **copies the reference, not the tree**. `find` in a plain binary tree is  $O(n)$ , not  $O(\log n)$ . A million nodes is **~120 MB** in Python (~56 with `__slots__`), against **~8 MB** for the same as a complete-tree array.

## Day 100 · dsa — Depth-first traversal: preorder, inorder, postorder

- All three traversals are ONE function with the visit line in a different place — before the children (**preorder**: node, left, right), between them (**inorder**: left, node, right), after them (**postorder**: left, right, node). Do not memorise three functions; move one line.
- Choose by reason, not by name. Parent needs to act first → **preorder** (copying, serialising, carrying a depth/path/range *down*). Answer computed from the children → **postorder** (height, size, sums, diameter, balance, and **deletion** — freeing a parent first is a use-after-free). **BST and you want sorted order** → **inorder**; on a non-BST, inorder means nothing.
- Iterative: **preorder pops and visits, pushing RIGHT then LEFT** (a stack reverses). **Inorder is a shape** — go left pushing everything, pop when you cannot, then go right — and the condition must be `while stack or node` or the loop never starts and you get `[]`. **Postorder = node-right-left preorder, reversed**; the honest version needs a `last_visited` pointer.
- `O(n)` time for all of them; `O(height)` space — ~20 frames on a balanced million-node tree, **1,000,000 on a degenerate one** (`RecursionError` past ~1,000). **Depth-first is `O(height)`, breadth-first is `O(width)`** — on a perfect million-node tree that is 20 against 500,000, in opposite directions depending on shape.
- **Morris inorder is `O(1)` space** by temporarily threading each node's rightmost left-descendant back to it — still `O(n)` time, and the catch is that it **mutates the tree while running**. And **one traversal never determines the tree**: you need two, one of which must be **inorder**, or a traversal that records the `None` s.

## Day 101 · dsa — Breadth-first traversal: level order

- **Breadth-first is iterative preorder with a QUEUE instead of a STACK**. That one swap is the whole difference: last-in-first-out goes deep, first-in-first-out goes wide. Use `collections.deque` — `list.pop(0)` is `O(n)` and silently makes the traversal `O(n2)` (~5 billion moves at  $n = 100,000$ ).
- **The one line that matters**: `level_size = len(queue)`, captured **BEFORE draining**. The invariant: *at the top of the outer loop the queue holds exactly one complete level*, so its length is that level's size and everything appended inside belongs to the next. Without it, the first “level” is the entire tree.
- `O(n)` time — same as DFS. Space is `O(width)`, not `O(n)` — ~500,000 nodes for a perfect million-node tree against ~20 DFS frames, and 1 node for a chain where DFS raises `RecursionError`. Opposite directions; neither wins in general.

- **Use BFS for: levels are the answer** (level order, right side view, averages, zigzag, next-right pointers), **the shallowest anything**, and **shortest paths in an unweighted graph**. **Minimum depth** is where it is genuinely faster — 3 nodes against 10,002. **Use DFS for anything computed from the children** (height, sums, diameter, balance): BFS has nowhere to combine them.
- **Check children for `None` before appending** (a `None` in the queue corrupts the level sizes too), guard the empty tree before the loop, and **zigzag by reversing the LIST, not the traversal**. And the recursive `min_depth` trap: `1 + min(left, right)` is wrong — a node with one child is not a leaf, and it returns 1 instead of 2.

## Day 102 · dsa — Height, depth, and diameter

- **Diameter = the longest path between any two nodes, and it does NOT have to pass through the root** — that is the most common wrong answer and it passes on symmetric tests. Count **edges** by default (LeetCode 543): a single node is 0, and `nodes = edges + 1`.
- **The key observation: every path has exactly one highest node**, so evaluating every node as the top of a path considers every path exactly once. At a node the bent path is `left height + right height + 2` (edge convention, empty = -1).
- **The naive version calls `height` at every node** → `O(n log n)` **balanced**, `O(n2)` **on a chain** — 50 million operations at  $n = 10,000$  against 10,000. **The same skewed input is also the stack-overflow case.**
- **The trick, and it is the whole phase: RETURN one quantity, RECORD another.** Return the **height** (what the parent needs); record the **bent path** in an outer variable (the parent cannot use it — a path bending here has already spent both children). Returning the bent path instead gives an answer that grows with depth. Python needs `nonlocal`, or you get `UnboundLocalError` or a silent 0.
- **The same shape solves five problems: diameter, balanced in one pass** (return a `-2` sentinel — `-1` is a real height and would silently pass), maximum path sum, longest univalue path, largest BST subtree. `O(n)` time, `O(height)` space — and returning the *path* rather than the length costs `O(n)` space instead. Bounds: `height ≤ diameter ≤ 2 * height`, both tight.

## Day 103 · dsa — Same tree, symmetric tree, and subtree

- **All three problems are one recursion over TWO trees, and the base cases are the whole difficulty. THREE of them: both `None` → True · exactly one `None` → False · neither → compare values then both child pairs.** Omitting the middle

line gives `AttributeError: 'NoneType' object has no attribute 'val'`, not a wrong answer. Write both base lines as a unit first.

- **Symmetry is `is_same` with the arguments CROSSED** — left against right, right against left. It is **not** `is_same(root.left, root.right)`: identical halves are a *translation*, symmetry is a *reflection*, and the wrong version returns `True` on `[1, 2, 2, null, 3, null, 3]`.
- **“Subtree” means a node AND EVERYTHING below it** — nothing extra may hang off the bottom, so a `1` with a left child is not a subtree of a `1` with both children. And its base cases are the **opposite** way round: **empty small** → **True**, **empty big** → **False**.
- **Same tree is `O(min(n, m))`** because `and` short-circuits. **Subtree is `O(n * m)`** — bad in the worst case (a chain of identical values), fine in practice because most attempts fail on the first value.
- **The `O(n + m)` alternative: serialise both trees and search for one string inside the other** — and it needs **BOTH** fixes, each silent if missed. Null markers, or a left-only and a right-only child give the same string; a **delimiter before every value**, or `"2"` matches inside `"12"`. Say `O(n + m)` with KMP, and note the cost is `O(n + m)` extra memory.

#### Day 104 · dsa — Path problems, and the return-value trick

- **Pin down “path” first: root-to-leaf** (state goes **DOWN** as an argument, **pre-order**, **no undo**) or **any-to-any** (arms come **UP** as return values, **postorder**, the **record/return split**). They are different problems, and root-to-leaf has **no floor** because you cannot decline the only branch.
- **Every path has exactly one highest node**, so evaluate every node as the turning point. Then keep two quantities apart: **RECORD** `left + right + val` (the bend — the answer candidate) and **RETURN** `val + max(left, right)` (one arm — all the parent can extend). Merging them lets a parent build on a path that has already turned around: too large, no error.
- **Floor each arm:** `max(0, arm)` — you are never obliged to take a branch, so a negative arm contributes nothing. Without it, `[5, -100, 3]` gives **−92 instead of 8**. This is Kadane’s rule on two branches.
- **But initialise `best` to `-inf`, not 0** — the floor applies to the **arms**, not the **answer**. A path must contain a node, so `[-3]` answers **−3**, and starting at 0 answers 0, which is not a path.

- A leaf is a node with BOTH children `None` — `node is None` makes a one-sided node look like a leaf. And **path sum III is not this shape**: it is **prefix sums with a hash map** on the current root-to-node path (`seen[0] = 1`, `count += seen[running - target]`), with `seen[running] -= 1` on the way out — which keeps it  $O(n)$  time,  $O(\text{height})$  space, and without it you silently count paths that bend.

## Day 105 · dsa — Lowest common ancestor

- LCA = the deepest node with both targets beneath it, and a node counts as its own descendant — so if one is an ancestor of the other, that one is the answer. Ask two questions first: **plain tree or BST?** and **are both nodes guaranteed present?**
- General tree, six lines: base case `node is None or node is p or node is q` → **return it; recurse BOTH sides; if left and right: return node** (they split here); else `return left or right`. The return value carries **three meanings**: nothing found · exactly one target found · the answer itself. Use `is`, not `=` — values may repeat.
- The early return on `node is p` is correct **ONLY** because both nodes are guaranteed present. Without that guarantee it silently returns a non-ancestor: count what you found and require 2, tallying *after* recursing, not before an early return.
- In a BST there is no search: walk down while both targets are on the same side, and the first split — or a node that IS a target — is the answer.  $O(\text{height})$  time,  $O(1)$  space, no recursion — about 20 comparisons against 1,000,000 node visits on a balanced million-node tree. Use strict `<` and `>`, or a node that is a target gets walked past.
- With parent pointers it becomes linked-list intersection — the two-pointer swap (walk up from `p`, then continue from `q`, and vice versa) meets at the LCA in  $O(1)$  space. And for many queries, switch from walking to preprocessing: **binary lifting**,  $O(n \log n)$  build and  $O(\log n)$  per query —  $10^{10}$  operations becomes a few million.

## Day 106 · dsa — Binary search trees: the ordering property

- The invariant, with the word that matters: at every node, ALL values in the LEFT SUBTREE are smaller and ALL values in the RIGHT SUBTREE are larger. Not the immediate children — that answer permits a globally broken tree in which `search` returns `None` for a value that is present, with no error.

- **Equivalent formulations:** every node lies inside a **RANGE** inherited from **ALL** its ancestors (left narrows the upper bound, right narrows the lower), and an **INORDER** traversal gives **sorted order**. Both are the same rule and both give correct validations.
- **One comparison discards a whole subtree**  $\rightarrow O(\text{height})$ . Say  $O(\text{height})$ , not  $O(\log n)$  — that needs **balance**, which nothing guarantees. **Sorted insertion builds a chain** (height  $n-1$ , every operation  $O(n)$ ), and sorted input is the *common* case. A **randomly** built BST is fine: expected height  $\sim 2 \log n$ . **The shape depends on insertion order, not on the set of values** — the same 7 values give height 6 or height 2.
- **Use a hash map unless you need order.** A hash map wins every single-key operation ( $O(1)$ ) and cannot do **ranges, min/max, successor/ceiling, sorted iteration or k-th smallest** at all. Those are what a BST is for. Against a sorted array: same  $O(\log n)$  search, but insertion is  $O(\log n)$  **versus up to a million element moves**.
- **State a duplicates policy:** disallow, always-left, always-right, or — best — **a count on each node**. Equal values on both sides means a search cannot know which way to go and finds some occurrences and not others. And **build a balanced BST from sorted data by taking the middle as the root** —  $O(n)$ , height exactly  $\lceil \log_2 n \rceil$ .

#### Day 107 · dsa — BST insert, search, and delete

- **The idiom is the technique:** `node.left = delete(node.left, key)`. Every recursive call returns the new subtree root and the caller **reattaches** it — that is how you restructure without parent pointers, and it is why the top level must be `root = delete(root, key)` or **deleting the root silently does nothing**. Forgetting the assignment leaves the tree unchanged, with no error.
- **Three cases, two lines:** `if node.left is None: return node.right` covers both “no children” and “only right”; then `if node.right is None: return node.left`.
- **Two children: there are EXACTLY TWO legal replacements** — the **inorder neighbours**. The value must be greater than *all* of the left subtree and less than *all* of the right, and only the largest-on-the-left and smallest-on-the-right qualify. Copy the **successor's value** into the node, then **delete the successor from the right subtree**.
- **That inner delete goes at most one level deeper**, because a minimum **has no left child** — so it is always case 1 or case 2. Say this unprompted. And **the predecessor is equally valid**; always using one side makes the tree lean, so real implementa-

tions alternate.

- **All three are**  $O(\text{height})$  —  $O(\log n)$  balanced,  $O(n)$  on a chain, and nothing here balances anything. Against a sorted array:  $O(\log n)$  **versus up to a million element moves** for one deletion. In production, prefer **lazy deletion** — mark a tombstone, rebuild when tombstones outnumber live nodes — which needs no pointer surgery and is safe when others hold node references.

## Day 108 · dsa — Validating a binary search tree

- **The classic wrong answer checks each node against its immediate CHILDREN.** Have the killer input ready and produce it unprompted: `[10, 5, 15, null, null, 6, 20]` — every parent-child pair passes, and 6 sits in the right subtree of 10. Worse than a wrong answer: `search(6)` **cannot find a value that is in the tree.**
- **Method one — carry the RANGE down.** Every node lives in an open interval inherited from **all** ancestors: going left tightens `high` to the parent's value, going right tightens `low`. **This IS the invariant restated**, so writing it explains the definition.
- **Use `None` (or `-inf`) for unbounded, NEVER `INT_MIN` / `INT_MAX`** — node values may equal them, so a single node `-231` gets rejected. And use `≤` / `≥`, not strict comparisons, or `[2, 2, 2]` passes. State the duplicates convention; it lives entirely in one operator.
- **Method two — inorder must STRICTLY increase, keeping only the previous value** ( $O(\text{height})$  space, not  $O(n)$  for a list). It needs no knowledge of the rules — anything wrong shows up as the sequence going backwards — and it is the basis for **LeetCode 99**: every backwards step is a violation, there are **one or two**, and you take the **first node of the first** and the **second node of the second**.
- **Both are  $O(n)$  time and  $O(\text{height})$  space; the local check is not slower, it is WRONG.** Offer the **iterative** inorder version when the tree may be a chain — which is not exotic, since a valid BST built from sorted input is a chain. And **largest BST subtree** is the day-102 trick again: return `(is_bst, size, min, max)` upward for  $O(n)$  instead of  $O(n^2)$ .

## Day 109 · dsa — Balanced trees, and why balance matters

- **The worst case is a CHAIN, and the input is SORTED DATA — which is the normal case**, since data arrives ordered by id or timestamp far more often than randomly. At  $n = 1,000,000$  that is **1,000,000 comparisons instead of ~20**, plus `RecursionError` past ~1,000. **A randomly built BST is fine** (expected height  $\sim 2 \log n$ ): the enemy is order, not adversaries.



- **Balanced means the subtree heights differ by  $\leq 1$  at EVERY node, not just the root** — and check it in **one pass with a sentinel**, because calling `height` at every node is  $O(n^2)$  on a chain.
- **A rotation rewires three pointers to move a child up and its parent down, and it is legal because IT DOES NOT CHANGE THE INORDER SEQUENCE.**  $O(1)$ , touches nothing outside the subtree, and the caller must **reattach** the returned root. Four cases: **LL and RR need one rotation**; **LR and RL are zig-zags and need two**, because one rotation on a zig-zag gives the mirror-image zig-zag.
- **AVL: height  $\leq 1.44 \log n$ , faster reads, more rotations on writes. Red-black: height  $\leq 2 \log n$ ,  $\leq 2$  rotations per insert** — which is why `std::map`, Java's `TreeMap` and the Linux scheduler use it. **Do not hand-write either.** Write a **treap** (BST by key, heap by a *random* priority,  $\sim 25$  lines,  $O(\log n)$  expected, immune to insertion order) or a **skip list** (what Redis uses for sorted sets).
- **Databases do not use binary trees — they use B-trees**, because on disk the cost is the number of node visits (**10 ms a read against 100 ns in memory**). A node is sized to one page and holds hundreds of keys: **1,000,000 keys is height 20 in a binary tree and height 3 in a B-tree**; a billion rows is four levels.

## Day 110 · dsa — Building a tree from its traversals

- **One traversal NEVER determines a tree** — a left-only and a right-only child both give preorder `[1, 2]`. The two lists do different jobs: **preorder identifies the ROOT (first element)**; **inorder locates the BOUNDARY between the subtrees**.
- **The non-obvious step: preorder finishes the ENTIRE left subtree before starting the right**, so the size learned from inorder cuts the preorder list at the same place. One number splits both lists.
- **Two separate  $O(n^2)$  s, and both must be fixed: `inorder.index()` (a scan per node)  $\rightarrow$  a value-to-index map built once; and SLICING (a new list per node)  $\rightarrow$  pass INDEX RANGES.** Together:  $O(n)$  time,  $O(n)$  space. Requires **unique values** — say so.
- **With a cursor into preorder, the LEFT recursion must run first** — it consumes exactly the left subtree's elements. Swapping the lines builds a valid tree from the wrong values, silently. For **postorder + inorder**, mirror it: root is the **last** element, walk **backwards**, and build the **RIGHT** subtree first.
- **Preorder + postorder is AMBIGUOUS** — both say where the root is, neither says where the boundary is — **except for a full binary tree** (every node has 0 or 2 children). And **a BST needs only ONE traversal**, because inorder is implied by the

ordering: walk the preorder with a permitted **range**,  $O(n)$  and no map. Verify any answer by **round-tripping** both traversals.

#### Day 111 · dsa — Serialising and deserialising a tree

- It is a **DESIGN** question with four decisions: **traversal order** · **null marker** · **delimiter** · **how you consume the string**. Say all four as choices, with reasons — that is the performance.
- A **single traversal does not determine a tree**; a **single traversal PLUS explicit null markers does**. The marker says *which* child is missing, which is exactly what distinguishes `1→left 2` from `1→right 2`.  $n$  nodes have  $n+1$  empty slots, so the output is  $2n + 1$  tokens.
- Choose **preorder** because the reader consumes tokens in the order the writer wrote them — one forward walk, no index arithmetic, no queue. Level order is equally valid and is the platform's display format; it is shorter for wide shallow trees and longer for deep narrow ones.
- The **null marker must not be parseable as a value** (`#`, never `-1`) and the **delimiter must not appear in a value**. Without a delimiter, `12` reads back as `1`, `2` and `-1` as `-`, `1` — silent corruption that passes every single-digit test.
- Two silent  $O(n^2)$  traps: `string +=` instead of `join` (~35 GB of copying at 100k tokens) and `list.pop(0)` instead of an iterator (~5 billion element moves). Both directions are  $O(n)$ . And a **BST needs NO markers** —  $n$  tokens instead of  $2n+1$ , **roughly half** — rebuilt by walking the values with a permitted **range**. The theoretical floor is  $2n$  bits for the shape, since there are  $\sim 4^n$  binary trees.

#### Day 112 · dsa — Trees revision and mock round

- One question picks the shape: which way does the information travel? **Up, combined** → simple postorder (the five-line skeleton). **Up, but you need TWO things** → the **return-value trick** (diameter, max path sum, balanced, largest BST subtree). **Down from an ancestor** → carry it as an argument (paths, depths, BST ranges — and no undo needed). **Across** → BFS with `len(queue)` captured first. **Between two trees** → three base cases. **A BST** → walk down, do not search.
- The **universal skeleton**: handle `None`, recurse both children, combine. The base case is the **identity** of the combining operation — `0` for sums, `-1` for max-of-heights, `True` for “all”.

- **The five bugs, and only the first one raises.** Missing `None` case · **local check where the property is global** (BST and balance) · returning the answer instead of what the parent needs · **recomputing what you already have** (  $O(n^2)$  in balance, diameter and reconstruction) · **reference instead of copy.**
- **Time is the whole tree; space is the deepest path — but DFS is  $O(\text{height})$  and BFS is  $O(\text{width})$ , in opposite directions.** Perfect million-node tree: 20 frames against 500,000 queued. Chain: 1 queued against `RecursionError`.  **$O(\text{height})$  is  $O(\log n)$  only if balanced, and sorted input builds a chain.**
- **Say the convention before the base case** (edges or nodes; empty tree `-1` or `0`), **state both complexities as you write, and run the five-bug checklist before declaring done** — four of the five are silent.

## Tries

### Day 120 · dsa — The trie: a tree of characters

**A trie is a tree of characters.** The **path spells the key**. No node stores a word.

**A node holds exactly two things:** `children: dict[str, TrieNode]` and `is_end: bool`.

```
class TrieNode:
 def __init__(self) → None:
 self.children: dict[str, TrieNode] = {}
 self.is_end: bool = False
```

**Insert:** walk, creating missing children, then set `is_end = True` on the last node.

**Search vs starts\_with — the whole lesson in two lines:**

```
search: node is not None and node.is_end
starts_with: node is not None
```

**Existence and word-ness are different facts.** With `"card"` stored, the node at `"car"` exists and is not a word. The only guarantee: **every leaf is a word.**

**Costs:** insert, search, prefix — all  $O(L)$ . **No `n` anywhere.** Collecting words costs  $O(L + \text{characters returned})$ .

**Three things a hash set cannot do:** prefix queries, sorted output for free, guaranteed worst case.

**The price is memory.** One node per distinct prefix,  $\sim 150\text{--}250$  bytes each in Python. English words:  $\sim 5\times$  a hash set. Random identifiers:  $50\text{--}80\times$  worse and slower. **Only use a trie when the keys share prefixes and you need prefix queries.**

**The real competitor is a sorted array + binary search** —  $O(L \log n + k)$ , far less memory. The trie wins on frequent updates and on wildcard or pruned searches.

**Shrink it with:** a radix tree (collapse single-child chains,  $\sim 60\text{--}80\%$  nodes), a DAWG (also merge suffixes, static only), or a ternary search tree.

**Store `words_below` on every node** to answer “how many start with this?” in  $O(L)$ .

**A trie does prefixes, not substrings.** “Contains” needs a suffix structure.

**Next:** [Day 121 — Insert, search, and prefix search](#)

**Previous:** [Day 119 — Heaps revision and mock round](#)

## Day 121 · dsa — Insert, search, and prefix search

**The shared walk** — write it once; three operations sit on top of it.

```
insert: node = node.children.setdefault(c, TrieNode()); ... node.is_end = True
search: node is not None and node.is_end
starts_with: node is not None
```

**All three are  $O(L)$ . So is delete.** No `n` anywhere.

**Delete has two parts:** unmark the word, then prune bottom-up.

**The pruning rule — a node may be removed only if:**

```
len(node.children) == 0 AND node.is_end is False
```

Children  $\Rightarrow$  words pass through it. `is_end`  $\Rightarrow$  a word ends at it. Either one means keep.

**It must be bottom-up**, because “has no children” is only correct after the children are pruned. Hence recursion, returning “I was removed, you may consider removing yourself”.

**The four cases:** | Delete | From | Result | `—|—|—|` | absent word | `—` | do nothing  
| | `"car"` | `{car, card}` | clear `is_end`, **remove nothing** | | `"card"` | `{car, card}`  
| remove `d`, stop at `r` | | `"dog"` | `{dog}` | whole branch collapses |

**Order matters:** `del node.children[c]` then check prunability. Reversed, nothing is ever pruned and every test still passes.

**Wildcard search is a DFS, not a walk.** At `.`, try every child; return `True` on the first success. **Base case is** `node.is_end`, never `True`.

**Cost:**  $O(26^W \times L)$  worst case, but the trie prunes — the real base is the data's branching factor ( $\sim 3$  for English). **Position beats count:** `c...`  $\approx 500$  paths, `...t`  $\approx 5,000$ . Bucket by length to help.

**Completions must take a limit.** An empty prefix matches everything. Stop early rather than truncating. For top-*popular* rather than top-*alphabetical*, precompute the best ten at each node at build time.

**Use the iterative delete when keys can exceed  $\sim 1000$  characters** — recursion depth.

---

**Next:** [Day 122 — Autocomplete and word dictionaries](#)

**Previous:** [Day 120 — The trie: a tree of characters](#)

## Day 122 · dsa — Autocomplete and word dictionaries

**The shape:** walk the prefix, then rank the subtree. The walk is  $O(p)$  and easy; the ranking is the lesson.

**Two versions:** gather-and-rank costs  $O(p + m \cdot L + m \log k)$  per query and is worst on the first keystroke; precomputed top- $k$  at every node costs  $O(p + k)$  per query and  $O(L \cdot k \log k)$  per insert.

**The tie rule, always stated:** higher weight first, then alphabetically smaller. Without it your output is not deterministic.

**The two bugs:** `_collect` must check the node it starts on, or the prefix itself is never suggested; and the empty prefix must be special-cased, or it traverses the whole dictionary.

**The precomputed list is a cache,** so deletion is cache invalidation. Real systems never update it in place — they rebuild off to one side every few minutes and swap it in.

## Day 123 · dsa — Tries in interviews: word search II

**The one sentence:** put the word list *inside* the grid walk instead of outside it, so `W` leaves the multiplication entirely.

**Walk two things at once** — a cell and a trie node. No child for the next letter means no word continues, so return without recursing.

**Store the word on the node, not a boolean.** Finding becomes an append, with no string built on the way down.

**Two prunings:** clear `node.word` after a find (deduplicates and stops re-finding); delete a childless, wordless node from its parent (the trie shrinks as you go, and this is what survives the all-`a` grid).

**The bug:** forgetting `board[row][col] = character` after the loop. No error — just missing words. Write the restore at the same moment you write the mark.

## Day 124 · dsa — Tries revision and mock round

**The recognition question:** *prefixes, repeatedly, over a fixed set of strings?* All three parts, or it is not a trie. Exact match only → set. A search run once per word → put the list inside the search.

**One walk, four shapes:** insert/search/ `starts_with`; wildcard branching; gather-and-rank; and prune somebody else's search. Shape 4 is worth the most and is the hardest to spot.

**Every basic operation is  $O(L)$  and none depends on `n`.** That is the property you are buying, and it is what a set cannot give you for prefixes.

**The five bugs, four of them silent:** missing `is_end` in `search`; pruning a node that ends a word; wildcard base case returning `True`; the grid cell never restored; and using a trie where a set would do.

**Memory:** ~150 bytes per node, ~250,000 nodes for 100,000 English words, ~37 MB — about  $3-4 \times$  a set. Radix trie or sorted-list-plus-`bisect` are the two ways down.

# The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

| DAY | DATA STRUCTURES AND ALGORITHMS                      | SYSTEM DESIGN                                 |
|-----|-----------------------------------------------------|-----------------------------------------------|
| 098 | What a tree is, and the vocabulary you need         | Vertical versus horizontal scaling            |
| 099 | Binary trees in code                                | Load balancers                                |
| 100 | Depth-first traversal: preorder, inorder, postorder | Stateless services and why they scale         |
| 101 | Breadth-first traversal: level order                | Caching: the single biggest win               |
| 102 | Height, depth, and diameter                         | Cache invalidation and eviction policies      |
| 103 | Same tree, symmetric tree, and subtree              | Content delivery networks                     |
| 104 | Path problems, and the return-value trick           | Database replication                          |
| 105 | Lowest common ancestor                              | Read replicas and replication lag             |
| 106 | Binary search trees: the ordering property          | Sharding, part one: choosing the key          |
| 107 | BST insert, search, and delete                      | Sharding, part two: rebalancing and hot spots |
| 108 | Validating a binary search tree                     | Consistent hashing                            |
| 109 | Balanced trees, and why balance matters             | Back-of-the-envelope estimation               |
| 110 | Building a tree from its traversals                 | Capacity planning: QPS, storage, bandwidth    |
| 111 | Serialising and deserialising a tree                | Single points of failure                      |
| 112 | Trees revision and mock round                       | Scaling revision and interview questions      |
| 113 | The heap: a tree stored in an array                 | Why distributed systems are hard              |
| 114 | Push, pop, and heapify                              | The CAP theorem, honestly                     |
| 115 | Python's heapq, and the min-heap-only problem       | Consistency models                            |
| 116 | Top K problems                                      | Eventual consistency in practice              |
| 117 | Merging K sorted lists                              | Quorums: why R plus W must exceed N           |
| 118 | Two heaps: the running median                       | Leader election                               |
| 119 | Heaps revision and mock round                       | Consensus, and Raft in plain English          |
| 120 | The trie: a tree of characters                      | Distributed transactions and two-phase commit |
| 121 | Insert, search, and prefix search                   | The saga pattern                              |
| 122 | Autocomplete and word dictionaries                  | Idempotency and exactly-once delivery         |
| 123 | Tries in interviews: word search II                 | Clocks, ordering, and why time is a lie       |



| DAY | DATA STRUCTURES AND ALGORITHMS | SYSTEM DESIGN                               |
|-----|--------------------------------|---------------------------------------------|
| 124 | Tries revision and mock round  | Failure detection, heartbeats, and timeouts |
|     |                                |                                             |

# Glossary

---

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

## 4

**4 kb** [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

## A

**a replicated log** [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

**abstract class** [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

**acid** [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [\[day 033\]](#) ([../day-033-window-with-a-map/README.md](#)).

**adaptee** [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

**adapter** [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

**add a tie-breaker** [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

**algorithm** [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

**always comparable** [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

**amortised** [Day 005](#) — The word for that is amortised: `list.append` is  $O(1)$  amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

**anaemic model** [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

**and a direction** [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

**anti-diagonals** [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of  $r + c$ .

**anti-sniping** [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

**approximate** [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

**array** [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [day 009](../day-009-what-an-array-is/README.md).

**artefact** [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

**assignment rule** [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

**at-least-once** [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

**atomic** [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

**atomicity** [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

**availability** [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *\*every\** non-failing node answers.

**aws ecs** [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

## B

**backend** [Day 002](#) — The backend is the code that never leaves the kitchen.

**bearer token** [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

**best case** [Day 002](#) — The best case is 1, the worst case is n.

**binary tree** [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

**blast radius** [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

**block** [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

**blocked** [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

**both** [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

**boundary** [Day 118](#) — The unifying idea worth stating: *\*two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.\**

**bounded context** [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

**bounded optimism** [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

**breadth-first** [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [tomorrow](../day-101-bfs-level-order/README.md).

**break point** [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

**broken links** [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

**bucket by time** [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel\_id, month).

**bucket counts** [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's histogram\_quantile does, and it is why counters and histograms are the metric types that aggregate and...

**bulkhead** [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

**burst size** [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

## C

**cache** [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

**cache it** [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

**cache stampede** [Day 090](#) — And the first cold morning is a cache stampede.

**candidates** [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

**capacity** [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

**cdn** [Day 070](#) — CDN — a geographically distributed caching proxy.

**certificate** [Day 006](#) — When your browser connects to google.com, the machine sends back a certificate — a small document containing the name it claims (google.com), a public key, an expiry date, and a signature from a Certificate Authority.

**chaining** [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

**channels** [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

**checks permission** [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

**chunks** [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

**circular buffer** [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

**class** [Day 043](#) — A class is the description; an object is one thing made from that description.

**client** [Day 068](#) — The client — the wall socket, which will only accept one shape.

**collapse key** [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

**collapsing writes** [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

**collection** [Day 016](#) — A collection is the set of things: /bags.

**column** [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

**compact** [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

**comparator** [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

**comparison** [Day 052](#) — A comparison is one question: \*is this one bigger than that one?\* A swap is exchanging two values so each sits where the other was.

**complete** [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

**component** [Day 069](#) — The tumblers are the component — the real object that does the actual job.

**composite** [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

**composite key** [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

**composition** [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

**composition root** [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually main() or a startup module.

**connection** [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

**connection-oriented** [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

**consequence** [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

**consistency** [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that \*every\* non-failing node answers.

**constraints** [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

**constructor** [Day 044](#) — `_init_` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

**container** [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

**container image** [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

**containerd** [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

**context** [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

**coordinator** [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

**correlation id** [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

**cost** [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

**count map** [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

**counter** [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

**cri-o** [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

**critical path** [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

**cycle** [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

## D

**dangerous** [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

**dashed** [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

**dau** [Day 097](#) — DAU — daily active users.

**deep-copied** [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

**deleted** [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

**delta of delta** [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

**dependency** [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

**depth** [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

**depth-first** [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

**deque** [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [\[day 074\]\(../day-074-deques-and-window-max/README.md\)](#) — holding indices, arranged so that their values are decreasing from front to back.

**diminishing returns** [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

**direct addressing** [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

**domain name** [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

**draw two diagrams** [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

**duck typing** [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

**dummy node** [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

## E

**earlier** [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

**edge cases** [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

**edge compute** [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

**effect** [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

**elastic scaling** [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

**election timeout** [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

**empty** [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

**etl** [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

**event object** [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

**eventual consistency** [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

**exactly** [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

**exactly correct** [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

**exactly one** [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

**exclusive lock** [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

**expiry** [Day 090](#) — The dates are expiry — a completely separate mechanism.

## F

**facts** [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

**false negative** [Day 124](#) — A false negative is failing to notice a machine that really has died.

**false positive** [Day 124](#) — A false positive is declaring a healthy machine dead.

**fanout** [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

**fee** [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.\*

**fifo** [Day 090](#) — FIFO — evict the oldest inserted.

**file** [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

**filled diamond** [Day 050](#) — A filled diamond means \*composition\* — the parts die with the whole, so Order ♦ — OrderLine because a line item has no meaning without its order.

**finite state machine** [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

**float** [Day 081](#) — Devil's second tin is a float: coins reserved so tomorrow can start.

**four-direction move** [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

**fragile** [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

**frontend** [Day 002](#) — The frontend is the code that crosses the counter.

## G

**galloping mode** [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

**gap** [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

**generator expression** [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

**greedy simulation** [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

**guest operating system** [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

## H

**hash map** [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

**hash ring** [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

**hateoas** [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.



**header interface** [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

**heap** [Day 117](#) — The six letters on the table are the heap: one candidate per list.

**heapsort** [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A\* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

**heartbeat** [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

**hidden spof** [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

**hiding mechanism** [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

**highest node** [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

**historical snapshot** [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment\_count | COUNT(\*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer\_name | joining to customers | a historical snapshot: the name at the time of the order | | order\_items.unit\_price | joining to products | the price \*then\*, which must...

**hollow triangle** [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

**hook** [Day 075](#) — award is a hook: the base class implements it as \*doing nothing\*, so a subclass may override it and most do not.

**horizontal scaling** [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

**hypervisor** [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

## I

**image** [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

**implicit graph** [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [[day 125](#)](../day-125-what-a-graph-is/README.md).

**in parallel** [Day 111](#) — Components in parallel — any one will do — multiply their \*failure\* probabilities, which is a completely different shape.

**in series** [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

**in-degree** [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

**index** [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

**indexed heap** [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

**inheritance** [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

**inorder** [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

**input** [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

**integrity** [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

**intent plus multiplicity** [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

**interface** [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

**invariant** [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

**is possible** [Day 046](#) —  $> *$ Find the smallest  $X$  such that some task is possible. $*$   $> > *$ Find the largest  $X$  such that some condition still holds. $*$

**item** [Day 016](#) — An item is one of them: `/bags/213`.

**iterator** [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

## J

**json** [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [[day 007](#)](../day-007-space-complexity/README.md); it looks like `{"id": 42, "name": "Zoya"}`.

**jump** [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

**jwt** [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from *\*somebody else\**, so a user can log in with Google without your service ever...

## K

**k closest points** [Day 116](#) — k closest points — the same shape with a computed key.

**k most frequent** [Day 116](#) — k most frequent — count first, then top-k on the counts.

**k-th largest specifically** [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

**key** [Day 023](#) — The key is as much of the design as the algorithm.

**key property** [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

## L

**lag** [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

**largest** [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position  $k - 1$ ; - the 1st largest is at position  $n - 1$ , so the k-th largest is at position  $n - k$ .

**latency** [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

**lazy** [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

**lazy deletion** [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

**lazy loading** [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

**lazy refill** [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

**leader** [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

**leaf** [Day 098](#) — He is a leaf — an item with no children.

**lease** [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

**ledger entries** [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

**left** [Day 040](#) — Remove the strip to the left: `prefix[r2 + 1][c1]`.

**lfu** [Day 090](#) — LFU — evict the least frequently used.

**linear scan** [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

**linearizability** [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

**list** [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

**list of lists** [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

**load balancer** [Day 098](#) — Sending students to the right counter is a load balancer.

**load factor** [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

**load shedding** [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

**log-structured merge tree** [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

**logical cohesion** [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

**logical time** [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by \*causality\* rather than by wall time.

**loop** [Day 001](#) — A loop is a statement that says, "do the following again and again".

**loop body** [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

**low cohesion** [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

**lru** [Day 090](#) — LRU — evict the least recently used.

## M

**manifest** [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS `.m3u8` or DASH `.mpd` — listing every rendition and every segment.

**many times** [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

**many-to-many** [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: `posts.author_id` | | many-to-many | a post has many tags, a tag has many posts | a third table: `post_tags(post_id, tag_id)` | | one-to-one | a user has one profile | either side; usually the optional one |

**matrix** [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

**mau** [Day 097](#) — MAU — monthly active users.

**max-heap** [Day 116](#) — The mirror rule: to find the  $k$  smallest, keep a max-heap of size  $k$  — in Python, a min-heap of negated values.

**method resolution order** [Day 052](#) — The Python answer. The method resolution order — `D._mro_` — is computed by a rule called C3 linearisation, and `super()` follows that order rather than jumping straight to a parent.

**milliseconds** [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

**min-heap** [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

**minimum spanning forest** [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than  $V - 1$  edges taken means the graph was disconnected.

**mixins** [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

**monotone** [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

**monotonic deque** [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is  $O(n)$  overall.

## N

**name mangling** [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

**namespace** [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

**negative** [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

**negative infinity** [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

**negatives** [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

**nested loop** [Day 001](#) — A nested loop is a loop inside another loop.

**network partition** [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

**never equal** [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

**never forgets** [Day 090](#) — Needs a third structure and a min\_frequency counter to stay  $O(1)$ , and it never forgets: an entry that was hot last month holds a huge count and will not leave.

**never persisted** [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

**no children** [Day 121](#) — It has no children — nothing continues past it.

**node** [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

**not a map** [Day 067](#) — | not a map — heap or sorted array |

## O

**o(1)** [Day 079](#) — tail — a reference to the last node, which makes append  $O(1)$  instead of  $O(n)$ .

**object** [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

**observers** [Day 072](#) — The objects on the list are the observers.

**occupied** [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

**offset** [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

**on first touch** [Day 041](#) — Each `post.author` is a \*different table\*, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

**one-shot** [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

**opens** [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

**orchestrator** [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

**origin** [Day 101](#) — The storeroom is the origin — the source of truth.

**origin pull** [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

**orphan rows** [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

**orphans** [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

**out-degree** [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

**overloading** [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

**overriding** [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

## P

**page** [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

**parallel edges** [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely *\*is\** a cycle, and skipping "the parent vertex" wrongly ignores it.

**parameters** [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

**parent** [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

**partial failure** [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, *"I do not know"*, and almost every difficult problem in the rest of this phase is a consequence of it.

**participants** [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

**partition key** [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows *\*sorted inside\** that bucket.

**partitions** [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

**path** [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

**port** [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

**prefix sums** [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

**preorder** [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

**priority queue** [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

**procedure** [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

**process** [Day 008](#) — A process is a running program with its own private memory.

**program** [Day 001](#) — A program is a sequence of statements.

**protecting state** [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

## Q

**qps** [Day 097](#) — QPS stands for *\*queries per second\** — the number of requests arriving at your system each second.

**queue** [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

**quorum** [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

## R

**race condition** [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

**ram** [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

**rank** [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

**rebalancing** [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

**rebuilding the tree** [Day 109](#) — Repacking the whole cart is rebuilding the tree —  $O(n)$ .

**recursion** [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

**references** [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

**refill rate** [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

**register** [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

**relative epsilon** [Day 048](#) — The fix is a relative epsilon — stop when the gap is small \*compared to the value\*:

**removal** [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

**replica** [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

**replica divergence** [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

**request-aware routing** [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

**resolver** [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

**resources** [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

**returned** [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."\*

**ring buffer** [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position  $7 + 1$  is position 0.

**role interface** [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

**root** [Day 093](#) — The root is the loft itself, written /.

**rotation** [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

**routine** [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

**row** [Day 026](#) — A row is one of those things.

**run** [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

## S

**same remainder** [Day 038](#) — A stretch is divisible by  $k$  when the prefixes at its two ends leave the same remainder — so carry running  $\% k$  and count matching remainders.

**schema** [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

**scoping** [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are *\*not\** building.

**script** [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

**seam** [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

**search space** [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

**selectivity** [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

**sentinel** [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

**separator** [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `",".join` puts a comma and a space between each pair.

**sequence diagram** [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

**serialisable** [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

**service credits** [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

**session** [Day 020](#) — A session is a record the server keeps about a logged-in user.

**session guarantees** [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

**set** [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

**shallow-copied** [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *\*reference\** to the same arrangement, and changing it from either end changed it for both.

**shape** [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.



**shard** [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

**shared lock** [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

**shared store** [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

**singleton** [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

**singly linked list** [Day 076](#) — A singly linked list — where each node knows only the *\*next\** one — cannot do that.

**slice** [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

**smallest** [Day 046](#) — `>` *\*Find the smallest X such that some task is possible.\** `>` *\*Find the largest X such that some condition still holds.\**

**snapshot** [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

**snipe** [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

**sorted region** [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *\*which end\** the region grows from and *\*how\** the next value gets there.

**sorting everything** [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

**splits** [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

**spof** [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

**squares** [Day 003](#) — | Example | `|---|---|---|` |  $O(1)$  | constant | does not change | reading items[5] | |  $O(\log n)$  | logarithmic | goes up by one step | halving until you reach 1 | |  $O(n)$  | linear | doubles | one loop over the list | |  $O(n \log n)$  | linearithmic | slightly more than doubles | sorting | |  $O(n^2)$  | quadratic | goes up four times | every...

**stable** [Day 051](#) — Python's sort is Timsort:  $O(n \log n)$  worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to  $O(n)$ .

**stack** [Day 075](#) — The fridge is a stack: last in, first out.

**stack frame** [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

**stack overflow** [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

**state** [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

**statements** [Day 001](#) — A program is a sequence of statements.

**status** [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

**still one machine** [Day 112](#) — | Move | Price | `|---|---|` | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

**stonith** [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

**strategy** [Day 071](#) — Strategy turns an algorithm into a value.

**strictly shorter** [Day 072](#) — `left[i]` — the index of the nearest bar to the left that is strictly shorter.

**structural** [Day 058](#) — In Python, `typing.Protocol` is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([\[day 052\]\(../day-052-quadratic-sorts/README.md\)](#)).

**subarray** [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

**subject** [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

**subscription list** [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

**substring** [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

**subtree** [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

**surge multiplier especially** [Day 088](#) — The surge multiplier especially is frozen.

**swap** [Day 052](#) — A comparison is one question: `*is this one bigger than that one?*` A swap is exchanging two values so each sits where the other was.

**system call** [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

## T

**table** [Day 026](#) — A table holds one kind of thing.

**tail call** [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

**tail-recursive** [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

**tcp** [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

**template** [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

**test double** [Day 053](#) — A test double is a stand-in you pass in during a test.

**the return address** [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

**the running median** [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and `A\*` — the nearest unvisited node - top-k problems — [\[day 116\]\(../day-116-top-k/README.md\)](#) - merging k sorted lists — [\[day 117\]\(../day-117-merge-k-sorted/README.md\)](#) - the running median — [\[day 118\]\(../day-118-two-heaps/README.md\)](#), with two heaps -...

**third table** [Day 026](#) — | Shape | Example | Where the key goes | `|-|-|-|-|` | one-to-many | a user has many posts | on the many side: `posts.author_id` | | many-to-many | a post has many tags, a tag has many posts | a third table: `post_tags(post_id, tag_id)` | | one-to-one | a user has one profile | either side; usually the optional one |

**thread** [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

**three distinct orderings** [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

**three-way partition** [Day 054](#) — The fix is a three-way partition: split into \*less than\*, \*equal to\*, and \*greater than\*, and recurse only into the outer two.

**tight coupling** [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

**time-to-live** [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

**tiny constraint** [Day 097](#) — A tiny constraint:  $n \leq 8$  means factorial,  $n \leq 20$  means  $2^n$ , target  $\leq 500$  with small candidates means combination sum.

**toast** [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

**token** [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

**tombstone** [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

**top-k problems** [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and  $A^*$  — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

**total order** [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

**transitions itself** [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

**transitive dependency** [Day 029](#) — This is a transitive dependency:  $id \rightarrow department\_id \rightarrow department\_name$ .

**tree** [Day 040](#) — A domain of independent records — a tree — tolerates anything.

**ttl** [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

**tuple** [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

**two** [Day 087](#) — In fib, each call makes two — so the work explodes.

**two implementations** [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

## U

**udp** [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

**under-fetching** [Day 021](#) — And from [[day 015](#)](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the  $N+1$  problem when it happens in a list.

**union filesystem** [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

**upper bound** [Day 044](#) — The upper bound is the first index where  $\text{nums}[i] > 4$ , which is index 4 — one past the run.

**uri** [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

**url** [Day 001](#) — A URL is an address for something on the internet.

## V

**value object** [Day 051](#) — A value object is defined entirely by its values.

**version** [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

**vertex** [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

**vertical scaling** [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

**victim** [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

**virtual address space** [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

**virtual machine** [Day 013](#) — A virtual machine is an emulated computer.

**volatile** [Day 009](#) — It is also volatile: switch the power off and it is gone.

## W

**weight** [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

**which class** [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

**which is hours** [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

**within** [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

**worst case** [Day 002](#) — The best case is 1, the worst case is  $n$ .

**write skew** [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

**write-around with invalidation** [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

**write-through** [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

## Y

**yes** [Day 093](#) — | Problem | Recurse on | Duplicates in input | Extra rule |  $|\dots|\dots|\dots|$  | Combinations (LC 77) |  $i + 1$  | no | stop at size  $k$  | | Combination Sum (LC 39) |  $i$  — reuse allowed | no | stop when remaining  $= 0$  | | Combination Sum II (LC 40) |  $i + 1$  — each used once | yes | sort, and skip when  $i > \text{start}$  and  $c[i] == c[i-1]$  | |...

# References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

## FOR THE WHOLE BOOK

**Python Software Foundation.** *The Python Language Reference and Standard Library*. <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

**Python Wiki.** *TimeComplexity - cost of the built-in container operations*. <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

**David Galles, University of San Francisco.** *Data Structure Visualizations*. <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

**Steven Halim and others.** *VisuAlgo - visualising data structures and algorithms*. <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

**MIT OpenCourseWare.** *6.006 Introduction to Algorithms*. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

**LeetCode.** *Problem set*. <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

## DATA STRUCTURES AND ALGORITHMS, BY PART

### PART XII · TREES AND BINARY SEARCH TREES (DAYS 98–112)

**Wikipedia.** *Binary search tree*. [https://en.wikipedia.org/wiki/Binary\\_search\\_tree](https://en.wikipedia.org/wiki/Binary_search_tree) [CC BY-SA]

The invariant, and what goes wrong when the tree degenerates.

**Wikipedia.** *Tree traversal*. [https://en.wikipedia.org/wiki/Tree\\_traversal](https://en.wikipedia.org/wiki/Tree_traversal) [CC BY-SA]

All four orders, iteratively and recursively.

**Wikipedia.** *Self-balancing binary search tree*. [https://en.wikipedia.org/wiki/Self-balancing\\_binary\\_search\\_tree](https://en.wikipedia.org/wiki/Self-balancing_binary_search_tree) [CC BY-SA]

AVL and red-black, for the follow-up about guaranteeing  $O(\log n)$ .

### PART XIII · HEAPS AND PRIORITY QUEUES (DAYS 113–119)

**Python Software Foundation.** *heapq - heap queue algorithm*. <https://docs.python.org/3/library/heapq.html> [PSF Licence]

The module docstring explains the array layout used in Part XIII.

**Wikipedia.** *Binary heap*. [https://en.wikipedia.org/wiki/Binary\\_heap](https://en.wikipedia.org/wiki/Binary_heap) [CC BY-SA]  
sift-up, sift-down, and the linear-time build.

## PART XIV · TRIES (DAYS 120–124)

**Wikipedia.** *Trie*. <https://en.wikipedia.org/wiki/Trie> [CC BY-SA]  
Includes compressed tries, the usual follow-up on memory.

**Wikipedia.** *Aho-Corasick algorithm*. [https://en.wikipedia.org/wiki/Aho%E2%80%93Corasick\\_algorithm](https://en.wikipedia.org/wiki/Aho%E2%80%93Corasick_algorithm) [CC BY-SA]  
Multi-pattern search built on a trie. A strong answer to 'can you do better'.

## SYSTEM DESIGN, BY PHASE

### SCALING FUNDAMENTALS (DAYS 97–112)

**Google.** *Site Reliability Engineering - the SRE Book*. <https://sre.google/sre-book/table-of-contents/> [Free to read online]  
Chapters on load balancing, overload and cascading failure.

**NGINX.** *Load balancing documentation*. <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/> [Free to read]  
Round robin, least connections and hashing, as actually configured.

**Redis.** *Redis documentation*. <https://redis.io/docs/latest/> [Free to read]  
Eviction policies and expiry, which is most of the caching chapter.

**Wikipedia.** *Cache replacement policies*. [https://en.wikipedia.org/wiki/Cache\\_replacement\\_policies](https://en.wikipedia.org/wiki/Cache_replacement_policies) [CC BY-SA]  
LRU, LFU and the others, compared.

**Colin Scott.** *Latency numbers every programmer should know*. [https://colin-scott.github.io/personal\\_website/research/interactive\\_latency.html](https://colin-scott.github.io/personal_website/research/interactive_latency.html) [Free to use]  
An interactive version of Jeff Dean's table. Learn the orders of magnitude.

### DISTRIBUTED SYSTEMS CORE (DAYS 113–128)

**Diego Ongaro and John Ousterhout.** *In Search of an Understandable Consensus Algorithm (Raft)*. <https://raft.github.io/raft.pdf> [Free to read]  
The clearest consensus paper written. Read it once end to end.

**Giuseppe DeCandia and others, Amazon.** *Dynamo: Amazon's Highly Available Key-value Store*. <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf> [Free to read]  
Consistent hashing, vector clocks and quorums, in one paper.

**Eric Brewer.** *CAP Twelve Years Later: How the Rules Have Changed*. <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/> [Free to read]  
Brewer correcting the popular misreading of his own theorem.

**MIT PDOS.** *6.824 Distributed Systems*. <https://pdos.csail.mit.edu/6.824/> [Free to read]  
Lecture notes and the paper list. The best free syllabus on the subject.

**Kyle Kingsbury.** *Jepsen analyses*. <https://jepsen.io/analyses> [Free to read]  
What real databases actually do under partition, as opposed to what they claim.

# Colophon: how this book was made

---

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

## TYPE, TOOLS AND LIBRARIES

---

**SIL International, after Matthew Carter.** *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

**Bold Monday for IBM.** *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

**JetBrains.** *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

**Google.** *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

**Knut Sveidqvist and contributors.** *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

**Georg Brandl and contributors.** *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

**Waylan Limberg and contributors.** *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

**Martin Thoma and contributors.** *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

**The Chromium Authors.** *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

## HOW A PAGE WAS MADE

---

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

## Problem index

---

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.



| DAY | PROBLEM                                  | SOURCE                | WHAT IT IS REALLY TESTING                                                     |
|-----|------------------------------------------|-----------------------|-------------------------------------------------------------------------------|
| 098 | Maximum Depth of Binary Tree             | LeetCode 104 (Easy)   | The five-line shape, and which convention LeetCode uses.                      |
| 098 | Count Complete Tree Nodes                | LeetCode 222 (Easy)   | The same shape with + instead of max — then the follow-up about doing better. |
| 098 | Minimum Depth of Binary Tree             | LeetCode 111 (Easy)   | The trap: min is wrong when one child is missing.                             |
| 098 | Binary Tree Level Order Traversal        | LeetCode 102 (Medium) | Capturing the queue length before the loop, which is the level boundary.      |
| 099 | Invert Binary Tree                       | LeetCode 226 (Easy)   | Mutating a tree in place, and whether you can see the result.                 |
| 099 | Merge Two Binary Trees                   | LeetCode 617 (Easy)   | Building new nodes, and the four None cases.                                  |
| 099 | Search in a Binary Search Tree           | LeetCode 700 (Easy)   | Why this is O(height) and plain-tree search is O(n).                          |
| 099 | Construct String from Binary Tree        | LeetCode 606 (Medium) | A different serialisation format, and why the brackets are load-bearing.      |
| 100 | Binary Tree Preorder Traversal           | LeetCode 144 (Easy)   | The iterative version, and pushing right before left.                         |
| 100 | Binary Tree Inorder Traversal            | LeetCode 94 (Easy)    | The go-left loop shape, and while stack or node.                              |
| 100 | Binary Tree Postorder Traversal          | LeetCode 145 (Easy)   | The reversal trick, then the honest version.                                  |
| 100 | Kth Smallest Element in a BST            | LeetCode 230 (Medium) | Why inorder is the answer, and why iterative beats recursive here.            |
| 101 | Binary Tree Level Order Traversal        | LeetCode 102 (Medium) | Capturing len(queue) before draining.                                         |
| 101 | Binary Tree Right Side View              | LeetCode 199 (Medium) | The same loop; the last node of each level.                                   |
| 101 | Minimum Depth of Binary Tree             | LeetCode 111 (Easy)   | Where BFS is genuinely faster, and the min trap.                              |
| 101 | Binary Tree Zigzag Level Order Traversal | LeetCode 103 (Medium) | Reversing the list, not the traversal.                                        |
| 102 | Maximum Depth of Binary Tree             | LeetCode 104 (Easy)   | The base case, and which convention you are using.                            |
| 102 | Balanced Binary Tree                     | LeetCode 110 (Easy)   | Spotting the repeated height call, and the sentinel.                          |
|     |                                          |                       |                                                                               |

| DAY | PROBLEM                        | SOURCE                 | WHAT IT IS REALLY TESTING                                  |
|-----|--------------------------------|------------------------|------------------------------------------------------------|
| 102 | Diameter of Binary Tree        | LeetCode 543 (Easy)    | Return one thing, record another.                          |
| 102 | Longest Univalue Path          | LeetCode 687 (Medium)  | The same shape with a condition on each arm.               |
| 103 | Same Tree                      | LeetCode 100 (Easy)    | The three base cases, and the and short-circuit.           |
| 103 | Symmetric Tree                 | LeetCode 101 (Easy)    | Crossed arguments — mirror, not identical.                 |
| 103 | Subtree of Another Tree        | LeetCode 572 (Easy)    | Different base cases, and what "subtree" strictly means.   |
| 103 | Invert Binary Tree             | LeetCode 226 (Easy)    | Producing a mirror rather than testing for one.            |
| 104 | Path Sum                       | LeetCode 112 (Easy)    | The leaf test, and state carried down as an argument.      |
| 104 | Binary Tree Paths              | LeetCode 257 (Easy)    | Collecting paths, and why passing a string needs no undo.  |
| 104 | Path Sum III                   | LeetCode 437 (Medium)  | Prefix sums on a tree, and the un-choose on the map.       |
| 104 | Binary Tree Maximum Path Sum   | LeetCode 124 (Hard)    | Record versus return, the floor, and the -inf start.       |
| 105 | LCA of a Binary Search Tree    | LeetCode 235 (Medium)  | Noticing that the ordering removes the search entirely.    |
| 105 | LCA of a Binary Tree           | LeetCode 236 (Medium)  | The overloaded return value, and recursing both sides.     |
| 105 | LCA of a Binary Tree II        | LeetCode 1644 (Medium) | What breaks when the nodes may be absent.                  |
| 105 | LCA of Deepest Leaves          | LeetCode 1123 (Medium) | The same postorder shape returning a pair.                 |
| 106 | Search in a Binary Search Tree | LeetCode 700 (Easy)    | One comparison discarding a subtree; write it iteratively. |
| 106 | Convert Sorted Array to BST    | LeetCode 108 (Easy)    | Why taking the middle prevents the degenerate case.        |
| 106 | Range Sum of BST               | LeetCode 938 (Easy)    | Skipping whole subtrees using the ordering.                |
| 106 | Kth Smallest Element in a BST  | LeetCode 230 (Medium)  | Inorder as sorted order, and stopping early.               |
|     |                                |                        |                                                            |

| DAY | PROBLEM                                           | SOURCE                 | WHAT IT IS REALLY TESTING                                            |
|-----|---------------------------------------------------|------------------------|----------------------------------------------------------------------|
| 107 | Insert into a Binary Search Tree                  | LeetCode 701 (Medium)  | The reattachment idiom, and that a new value is always a leaf.       |
| 107 | Delete Node in a BST                              | LeetCode 450 (Medium)  | The three cases, and deriving the successor rather than reciting it. |
| 107 | Inorder Successor in BST                          | LeetCode 285 (Medium)  | The two cases, and finding the last left turn on the way down.       |
| 107 | Trim a Binary Search Tree                         | LeetCode 669 (Medium)  | The same reattachment idiom, used for something else.                |
| 108 | Validate Binary Search Tree                       | LeetCode 98 (Medium)   | The range method, and the counter-example to the local check.        |
| 108 | Minimum Absolute Difference in BST                | LeetCode 530 (Easy)    | Inorder plus a previous value — the same one-variable idea.          |
| 108 | Recover Binary Search Tree                        | LeetCode 99 (Medium)   | One or two violations, and which nodes to take from each.            |
| 108 | Largest BST Subtree                               | LeetCode 333 (Medium)  | The postorder return-value trick returning four things.              |
| 109 | Balanced Binary Tree                              | LeetCode 110 (Easy)    | Balance at <i>*every*</i> node, in one pass with a sentinel.         |
| 109 | Convert Sorted Array to BST                       | LeetCode 108 (Easy)    | The $O(n)$ rebuild, and why the middle is the root.                  |
| 109 | Balance a Binary Search Tree                      | LeetCode 1382 (Medium) | Flatten to sorted order, then rebuild — the whole cart repacked.     |
| 109 | Insert into a Binary Search Tree                  | LeetCode 701 (Medium)  | Then extend it into an AVL insert with the four cases.               |
| 110 | Construct Binary Tree from Preorder and Inorder   | LeetCode 105 (Medium)  | The split, the position map, and the cursor ordering.                |
| 110 | Construct Binary Tree from Inorder and Postorder  | LeetCode 106 (Medium)  | The mirror: last element, backwards cursor, right subtree first.     |
| 110 | Construct BST from Preorder                       | LeetCode 1008 (Medium) | Why one traversal suffices, and the range walk that avoids sorting.  |
| 110 | Construct Binary Tree from Preorder and Postorder | LeetCode 889 (Medium)  | Why this is ambiguous in general, and what makes it work here.       |
| 111 | Serialize and Deserialize Binary Tree             | LeetCode 297 (Hard)    | Null markers, the delimiter, and consuming with an iterator.         |
| 111 | Serialize and Deserialize BST                     | LeetCode 449 (Medium)  | Why a BST needs no markers, and the range walk.                      |
|     |                                                   |                        |                                                                      |

| DAY | PROBLEM                            | SOURCE                 | WHAT IT IS REALLY TESTING                                         |
|-----|------------------------------------|------------------------|-------------------------------------------------------------------|
| 111 | Find Duplicate Subtrees            | LeetCode 652 (Medium)  | Serialisation as a *key* — where the delimiter bug bites hardest. |
| 111 | Construct String from Binary Tree  | LeetCode 606 (Medium)  | A different format, and why its brackets are load-bearing.        |
| 112 | Count Good Nodes in Binary Tree    | LeetCode 1448 (Medium) | Recognising "carry state down" from an unfamiliar phrasing.       |
| 112 | Maximum Level Sum of a Binary Tree | LeetCode 1161 (Medium) | Recognising a level problem, and the boundary capture.            |
| 112 | Sum Root to Leaf Numbers           | LeetCode 129 (Medium)  | State down again, and the leaf test.                              |
| 112 | Binary Tree Maximum Path Sum       | LeetCode 124 (Hard)    | The return-value trick, cold, with the floor and the -inf.        |
| 113 | Kth Largest Element in a Stream    | LeetCode 703 (Easy)    | The O(1) peek, and why a heap of size k is the right shape.       |
| 113 | Last Stone Weight                  | LeetCode 1046 (Easy)   | Repeated extract-max, and Python's min-only gotcha.               |
| 113 | Kth Largest Element in an Array    | LeetCode 215 (Medium)  | Heap against sort against quickselect, argued with numbers.       |
| 113 | Top K Frequent Elements            | LeetCode 347 (Medium)  | A heap over tuples, and where the tie-breaker matters.            |
| 114 | Sort an Array                      | LeetCode 912 (Medium)  | Heapsort in place, and why it loses to quicksort anyway.          |
| 114 | Kth Largest Element in an Array    | LeetCode 215 (Medium)  | heapify + k pops against a full sort, argued with numbers.        |
| 114 | Find K Pairs with Smallest Sums    | LeetCode 373 (Medium)  | heapreplace on a fixed-size heap, rather than push-then-pop.      |
| 115 | Kth Largest Element in an Array    | LeetCode 215 (Medium)  | The max-heap workaround, and heap-versus-sort with numbers.       |
| 115 | Top K Frequent Elements            | LeetCode 347 (Medium)  | Tuples, the tie-breaker, and nlargest with a key.                 |
| 115 | Task Scheduler                     | LeetCode 621 (Medium)  | A max-heap of counts, and re-pushing after a cooldown.            |
| 115 | Network Delay Time                 | LeetCode 743 (Medium)  | Dijkstra with lazy deletion, written the Python way.              |
| 116 | Kth Largest Element in a Stream    | LeetCode 703 (Easy)    | The one case where the heap is forced, and why.                   |

| DAY | PROBLEM                                       | SOURCE                | WHAT IT IS REALLY TESTING                                             |
|-----|-----------------------------------------------|-----------------------|-----------------------------------------------------------------------|
| 116 | Top K Frequent Elements                       | LeetCode 347 (Medium) | Counting first, and the $O(n)$ bucket answer.                         |
| 116 | K Closest Points to Origin                    | LeetCode 973 (Medium) | A computed key, and not taking the square root.                       |
| 116 | Kth Largest Element in an Array               | LeetCode 215 (Medium) | Quickselect with a random pivot, written iteratively.                 |
| 117 | Merge Two Sorted Lists                        | LeetCode 21 (Easy)    | The primitive the tournament is built from.                           |
| 117 | Merge k Sorted Lists                          | LeetCode 23 (Hard)    | The heap of k fronts, and both $O(n \log k)$ solutions.               |
| 117 | Kth Smallest Element in a Sorted Matrix       | LeetCode 378 (Medium) | A k-way merge that stops after k pops.                                |
| 117 | Smallest Range Covering Elements from K Lists | LeetCode 632 (Hard)   | The same heap plus one extra variable.                                |
| 118 | Find Median from Data Stream                  | LeetCode 295 (Hard)   | The two invariants, the negation, and push-shift-rebalance.           |
| 118 | IPO                                           | LeetCode 502 (Hard)   | Two heaps facing each other with a different question.                |
| 118 | Sliding Window Median                         | LeetCode 480 (Hard)   | Lazy deletion, and logical sizes versus <code>len(heap)</code> .      |
| 118 | Meeting Rooms II                              | LeetCode 253 (Medium) | The simpler cousin: one heap of end times against a stream of starts. |
| 119 | Kth Largest Element in an Array               | LeetCode 215 (Medium) | Shape 2, and whether you reach for quickselect.                       |
| 119 | Task Scheduler                                | LeetCode 621 (Medium) | Shape 1 plus a cooldown queue. Today's mock 1.                        |
| 119 | Find K Pairs with Smallest Sums               | LeetCode 373 (Medium) | Shape 4 in disguise. Today's mock 2.                                  |
| 119 | Network Delay Time                            | LeetCode 743 (Medium) | Shape 5 — Dijkstra with lazy deletion.                                |
| 119 | Reorganize String                             | LeetCode 767 (Medium) | Shape 1, and knowing when it is impossible.                           |
| 120 | Implement Trie (Prefix Tree)                  | LeetCode 208 (Medium) | The node, insert, and search vs <code>startsWith</code> .             |
| 120 | Longest Common Prefix                         | LeetCode 14 (Easy)    | Whether you know a trie is *overkill* here.                           |
|     |                                               |                       |                                                                       |

| DAY | PROBLEM                                    | SOURCE                 | WHAT IT IS REALLY TESTING                                           |
|-----|--------------------------------------------|------------------------|---------------------------------------------------------------------|
| 120 | Longest Word in Dictionary                 | LeetCode 720 (Medium)  | Every prefix must also be a word — is_end used as a real condition. |
| 120 | Replace Words                              | LeetCode 648 (Medium)  | Stopping the walk at the first is_end.                              |
| 120 | Map Sum Pairs                              | LeetCode 677 (Medium)  | The words_below augmentation, with sums instead of counts.          |
| 121 | Implement Trie (Prefix Tree)               | LeetCode 208 (Medium)  | search vs startsWith — one condition apart.                         |
| 121 | Design Add and Search Words Data Structure | LeetCode 211 (Medium)  | Wildcards: a walk becomes a DFS.                                    |
| 121 | Replace Words                              | LeetCode 648 (Medium)  | Stopping at the *first* is_end, not the last.                       |
| 121 | Word Search II                             | LeetCode 212 (Hard)    | Pruning a grid search with a trie — the real payoff.                |
| 121 | Word Break                                 | LeetCode 139 (Medium)  | The trie replacing the set lookup, plus memoisation.                |
| 122 | Search Suggestions System                  | LeetCode 1268 (Medium) | Walk, gather, rank — the version you write first.                   |
| 122 | Longest Word in Dictionary                 | LeetCode 720 (Medium)  | Traversal that stops the moment a node is not a whole word.         |
| 122 | Design Search Autocomplete System          | LeetCode 642 (Hard)    | Weights, the # terminator, and state carried between calls.         |
| 122 | Top K Frequent Words                       | LeetCode 692 (Medium)  | The same tie rule, without a trie, so you can see it alone.         |
| 122 | Replace Words                              | LeetCode 648 (Medium)  | Stopping at the *first* whole word on the path, not the deepest.    |
| 123 | Word Search                                | LeetCode 79 (Medium)   | Backtracking on a grid, one word, restore on the way out.           |
| 123 | Word Search II                             | LeetCode 212 (Hard)    | Moving the word list *inside* the walk. The whole lesson.           |
| 123 | Word Break II                              | LeetCode 140 (Hard)    | A trie plus memoised backtracking, without a grid.                  |
| 123 | Concatenated Words                         | LeetCode 472 (Hard)    | Searching a trie from inside a trie walk.                           |
| 123 | Stream of Characters                       | LeetCode 1032 (Hard)   | The reversed trie — the trick nobody sees the first time.           |
|     |                                            |                        |                                                                     |

| DAY | PROBLEM                                    | SOURCE                | WHAT IT IS REALLY TESTING                                       |
|-----|--------------------------------------------|-----------------------|-----------------------------------------------------------------|
| 124 | Implement Trie (Prefix Tree)               | LeetCode 208 (Medium) | The walk, and the one condition between search and starts_with. |
| 124 | Design Add and Search Words Data Structure | LeetCode 211 (Medium) | The branching walk, and the base case that must not be True.    |
| 124 | Word Break                                 | LeetCode 139 (Medium) | Trie plus memo — mock problem 1, timed.                         |
| 124 | Stream of Characters                       | LeetCode 1032 (Hard)  | The reversed trie — mock problem 2, timed.                      |
| 124 | Maximum XOR of Two Numbers in an Array     | LeetCode 421 (Medium) | A trie over *bits*, not letters. The shape nobody expects.      |

## H

# Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

## #

4 kb, 514, 1294, 1295, 1306

## A

a replicated log, 1294

abstract class, 1294

acid, 759, 768, 769, 772, 773, 815, 1051, 1056, 1063, ...

adaptee, 1294

adapter, 1294

add a tie-breaker, 988, 1267, 1294

algorithm, 17, 26, 45, 46, 67, 70, 73, 74, 79, ...

always comparable, **787**, 790, 1294, 1305

amortised, 972, 1294

anaemic model, 1294

and a direction, 1294

anti-diagonals, 1294

anti-sniping, 1294

approximate, **344**, 935, 1294

array, 7, 19, 41, 44, 49, 59, 61, 63, 65, ...

artefact, 1295, 1297

assignment rule, 1295

at-least-once, 664, 673, 675, 679, 714, 716, 718, 720, 722, ...

atomic, 433, 445, 717, 718, 723, 725, 852, 855, 858, ...

atomicity, 1294, 1295

availability, 28, 31, 42, 76, 619, 622, 623, 664, **733**, ...

aws ecs, 1295, 1305

## B

backend, 68, 69, 70, 71, 73, 74, 75, 76, 77, ...

bearer token, 1295

best case, 328, 713, 725, 1050, 1235, 1295, 1312

binary tree, 1, 6, 11, 15, 17, 45, 46, 55, 56, ...

blast radius, 393, **614**, 617, 621, 623, 627, 630, 1276, 1295

block, 150, 297, 303, 304, 305, 307, 341, 453, 454, ...

blocked, 898, 908, 911, 912, 1040, 1041, 1240, 1256, 1295

both, 4, 5, 6, 7, 16, 17, 18, 19, 28, ...

boundary, 12, 121, 135, 138, 174, 230, 548, 553, **925**,

...

bounded context, 1295

bounded optimism, **851**, 1295

breadth-first, 1, 15, 19, **95**, 105, 109, 112, 131, 135, ...

break point, 1295

broken links, 1296, 1305

bucket by time, 1296

bucket counts, 985, 1296

bulkhead, **617**, 714, 1296

burst size, 1296, 1307

## C

cache, 28, 33, 34, 36, 38, 42, 70, 71, 72, ...

cache it, 160, 163, 201, 216, 261, **433**, 435, 436, 440,

...

cache stampede, 1296

candidates, 525, 673, 948, 949, 1294, 1296, 1311, 1312

capacity, 2, 28, 32, 76, 122, 343, 346, 347, 391, ...

cdn, 160, 162, 165, 166, 169, 170, 172, 173, 206, ...

certificate, 615, 616, 620, 1296

chaining, 1296

channels, 1296

checks permission, 1296

chunks, 255, 885, 1296

circular buffer, 1296, 1307

class, 5, 6, 9, 11, 23, 45, 46, 47, 48, ...

client, 72, 73, 74, 76, 249, 296, 297, 298, 299, ...

collapse key, 1297

collapsing writes, 1297

collection, 11, 55, 99, 140, 142, 144, 150, 156, 179, ...

column, 390, 524, 532, 671, 1010, 1093, 1108, 1169, 1297, ...

compact, 596, 601, 633, 1297, 1298

comparator, 1297

comparison, 15, 57, 73, 191, 238, 322, 326, 329, 367,

...

complete, 6, 8, 11, 13, 55, 57, 59, 99, **740**, ...

component, 28, 591, 619, 620, 622, 672, 1297, 1301, 1304, ...

composite, 389, 1297



composite key, 1297  
composition, 1297, 1300  
composition root, 1297  
connection, 73, 74, 75, 76, 77, 389, 438, 528, 663, ...  
connection-oriented, 1297  
consequence, 132, 211, **528**, 531, 542, 695, 715, 852, 1297, ...  
consistency, 343, 344, 671, 673, 710, **761**, 763, 765, 766, ...  
constraints, 389, 671, 852, 1045, 1297, 1299  
constructor, 1294, 1297  
container, 366, 1295, 1297, 1298, 1313  
container image, 1295, 1297  
containerd, 1298  
context, 121, 1098, 1099, 1295, 1298  
coordinator, 851, 861, 899, 902, 909, **1041**, 1042, 1043, 1044, ...  
correlation id, 1298  
cost, 3, 14, 19, 21, 22, 23, 24, 25, 26, ...  
count map, 1298  
counter, 24, 67, 68, 69, 116, 118, 122, 123, 125, ...  
cri-o, 1298  
critical path, 615, 622, 625, 626, 627, 629, 630, 676, 679, ...  
cycle, 3, 18, 19, 22, 43, 247, 997, 1056, 1240, ...

## D

dangerous, 299, 1298  
dashed, 1298  
dau, 407, 408, 421, 524, 525, 527, 528, 530, 541, ...  
deep-copied, 1298  
deleted, 204, 217, 410, 411, 417, 418, 423, 424, 425, ...  
delta of delta, 1298  
dependency, 120, 123, 617, 619, 719, 951, 1298, 1305, 1311  
depth, 1, 2, 3, 5, 7, 8, 9, 10, 11, ...  
depth-first, 1, 15, 19, **90**, 91, 95, 105, 112, 129, ...  
deque, 11, 12, 53, 55, 56, 57, 99, 103, 138, ...  
diminishing returns, 1299  
direct addressing, 1299  
domain name, 74, 1299  
draw two diagrams, 1299  
duck typing, 1299  
dummy node, 1299

## E

earlier, 275, 282, 289, 311, 448, 454, 493, 568, **943**, ...  
edge cases, 138, 453, 457, 465, 558, 604, 838, 886, 892, ...  
edge compute, 1299

effect, 31, 73, 77, 83, 88, 122, 125, 128, **714**, ...  
elastic scaling, 1299  
election timeout, **947**, 952, 953, 1001, 1299  
empty, 9, 11, 16, 18, 21, 39, 46, 47, 55, ...  
etl, 31, 34, 69, 107, 113, 123, 155, 258, 261, ...  
event object, 1299  
eventual consistency, **780**, 814, 826, 847, 860, 1255, 1292, 1299  
exactly, 3, 4, 6, 7, 9, 11, 12, 15, 16, ...  
exactly correct, 1299  
exactly one, 3, 4, 6, 7, 11, 12, 18, 19, 22, ...  
exclusive lock, 1299  
expiry, 120, 121, 163, 615, 671, 946, 947, 957, 959, ...

## F

facts, 6, 40, 158, 173, 176, 179, 185, 594, 689, ...  
false negative, 1300  
false positive, 717, 1236, 1300  
fanout, 1300  
fee, 3, 9, 31, 68, 94, 161, 166, 175, 177, ...  
fifo, 205, 785, 788, 791, 793, 797, 799, 801, 802, ...  
file, 28, 42, 77, 84, 88, 116, 119, 125, 126, ...  
filled diamond, 1300  
finite state machine, 1300  
float, 145, 275, 279, 281, 286, 456, 463, 464, 467, ...  
four-direction move, 1300  
fragile, 1300  
frontend, 1300

## G

galloping mode, 1300  
gap, 46, 47, 49, 53, 55, 56, 59, 60, 257, ...  
generator expression, 1300  
greedy simulation, 1300  
guest operating system, 1300, 1301

## H

hash map, 275, 282, 289, 292, 365, 368, 369, 370, 374, ...  
hash ring, 476, 1300  
hateos, 1300  
header interface, 1301  
heap, 3, 7, 15, 19, 25, 26, 29, 35, **872**, ...  
heapsort, 693, 700, 701, 707, 709, 738, 740, 743, 744, ...  
heartbeat, 299, 302, 717, 948, 951, 952, 996, 1001, 1207, ...  
hidden spof, 1301  
hiding mechanism, 1301  
highest node, 183, 185, 189, 196, 198, 199, 200, 220, 274, ...

historical snapshot, 1301  
hollow triangle, 1301  
hook, 1151, 1301  
horizontal scaling, 2, 23, 25, 28, 30, 1267, 1292, 1301  
hypervisor, 1300, 1301

## I

image, 69, 170, 213, 228, 246, 248, 250, 253, 254, ...  
implicit graph, 1301  
in parallel, 392, 619, 672, 997, 1301  
in series, 615, 619, 626, 629, 672, 673, 676, 679, 680, ...  
in-degree, 1301, 1305  
index, 26, 28, 37, 49, 51, 59, 61, 63, 65, ...  
indexed heap, 706, 709, 739, 745, 751, 753, 756, 776, 785, ...  
inheritance, 1301  
inorder, 1, 90, 91, 92, 93, 94, 96, 98, 545, ...  
input, 40, 193, 197, 199, 200, 220, 238, 243, 265, ...  
integrity, 1302  
intent plus multiplicity, 1302  
interface, 1298, 1300, 1301, 1302, 1307  
invariant, 141, 144, 175, 360, 363, 367, 370, 375, 401, ...  
is possible, 1302, 1309  
item, 3, 4, 5, 7, 15, 122, 159, 164, 166, ...  
iterator, 63, 93, 111, 594, 595, 596, 599, 600, 605, ...

## J

json, 524, 575, 577, 581, 583, 1146, 1149, 1302  
jump, 68, 340, 350, 353, 479, 484, 488, 490, 494, ...  
jwt, 117, 120, 121, 124, 1302

## K

k closest points, 827, 831, 866, 982, 1302, 1321  
k most frequent, 977, 1302  
k-th largest specifically, 1302  
key, 30, 33, 36, 37, 38, 50, 114, 119, 120, ...  
key property, 1302

## L

lag, 29, 30, 35, 37, 38, 65, 101, 105, 657, ...  
largest, 5, 21, 23, 25, 27, 29, 37, 181, 182, ...  
latency, 26, 35, 37, 38, 42, 44, 70, 75, 77, ...  
lazy, 163, 411, 417, 420, 421, 423, 425, 427, 873, ...  
lazy deletion, 411, 417, 420, 421, 745, 785, 788, 791, 792, ...  
lazy loading, 1303  
lazy refill, 1303  
leader, 116, 119, 124, 125, 127, 270, 293, 294, 337, ...  
leaf, 2, 4, 5, 6, 7, 8, 9, 11, 12, ...

lease, 165, 210, 215, 217, 218, 249, 263, 297, 946, ...  
ledger entries, 1091, 1096, 1106, 1110, 1260, 1303  
left, 6, 7, 8, 9, 10, 11, 12, 13, 16, ...  
lfu, 205, 206, 210, 213, 215, 1303, 1314  
linear scan, 549, 557, 883, 888, 1303  
linearizability, 899, 912, 1256, 1303  
list, 3, 6, 7, 8, 12, 13, 14, 41, 46, ...  
list of lists, 1303  
load balancer, 27, 28, 45, 67, 72, 118, 162, 478, 532, ...  
load factor, 1303  
load shedding, 1303  
log-structured merge tree, 1303  
logical cohesion, 1303  
logical time, 712, 715, 1303  
loop, 3, 4, 6, 12, 17, 19, 39, 40, 91, ...  
loop body, 106, 1024, 1303  
low cohesion, 1303  
lru, 158, 161, 205, 206, 207, 210, 213, 214, 215, ...

## M

manifest, 255, 1303  
many times, 110, 227, 236, 535, 971, 1139, 1142, 1176, 1304, ...  
many-to-many, 1304, 1310  
matrix, 885, 886, 972, 1303, 1304, 1321  
mau, 1304  
max-heap, 509, 517, 691, 694, 701, 704, 729, 738, 829, ...  
method resolution order, 1304  
milliseconds, 140, 149, 159, 436, 813, 851, 852, 854, 857, ...  
min-heap, 687, 689, 690, 691, 694, 698, 700, 708, 920, ...  
minimum spanning forest, 1304  
mixin, 195, 1304  
monotone, 1304  
monotonic deque, 1304

## N

name mangling, 1304  
namespace, 161, 203, 208, 213, 216, 218, 223, 1270, 1304  
negative, 274, 279, 281, 283, 285, 598, 604, 607, 649, ...  
negative infinity, 279, 1304  
negatives, 274, 598, 649, 790, 1299, 1304  
nested loop, 1304  
network partition, 946, 1305  
never equal, 782, 785, 787, 790, 799, 801, 802, 1264, 1294, ...

never forgets, 1305  
never persisted, 1305  
no children, 11, 16, 412, 414, **1071**, 1303, 1305  
node, 2, 5, 6, 7, 8, 9, 10, 11, 12, ...  
not a map, 1305

## O

o(1), 103, 104, 105, 108, 109, 112, 140, 147, 149, ...  
object, 6, 9, 16, 33, 36, 38, 46, 49, 50, ...  
observers, 1305  
occupied, 1298, 1299, 1305  
offset, 1305  
on first touch, 1305  
one-shot, 1305  
opens, 74, 204, 214, 217, 526, 689, 1019, 1072, 1184, ...  
orchestrator, 1096, 1097, 1098, 1101, 1103, 1295, 1305  
origin, 159, 162, 165, 206, 248, 249, 252, 253, 254, ...  
origin pull, 1305  
orphan rows, 1305  
orphans, 1296, 1305  
out-degree, 1301, 1305  
overloading, 321, 325, 1305  
overriding, 1306

## P

page, 23, 24, 60, 161, 165, 166, 169, 172, 177, ...  
parallel edges, 1306  
parameters, 1306  
parent, 2, 11, 17, 49, 57, 58, 61, 65, 74, ...  
partial failure, **713**, 1306  
participants, **1046**, 1047, 1048, 1049, 1102, 1298, 1306, 1308  
partition key, 1296, 1306  
partitions, 759, 763, 767, 830, 1306  
path, 1, 3, 4, 5, 6, 11, 13, 18, 19, ...  
port, 11, 29, 35, 47, 55, 68, 69, 71, 79, ...  
prefix sums, 282, 284, 1304, 1306, 1318  
preorder, 1, 90, 91, 92, 93, 96, 97, 98, **545**, ...  
priority queue, 687, 790, 971, 972, 973, 977, 1262, 1306  
procedure, 574, 615, 1306  
process, 116, 123, 135, 159, 162, 165, 167, 320, 329, ...  
program, 1297, 1306, 1307, 1309, 1314  
protecting state, 1306

## Q

qps, 32, 38, 76, 88, 120, 121, 165, 166, 167, ...  
queue, 3, 12, 14, 15, 19, 23, 39, 40, 53, ...

quorum, 297, 298, 301, 302, 306, 307, 308, 710, **944**, ...

## R

race condition, 1307  
ram, 14, 15, 17, 18, 19, 21, 22, 25, 29, ...  
rank, 79, 88, 1000, 1049, 1120, 1121, 1122, 1123, 1124, ...  
rebalancing, **406**, 429, 436, 661, 671, 934, 936, 1273, 1292, ...  
rebuilding the tree, **1307**  
recursion, 12, 14, 15, 17, 19, 21, 22, 40, 57, ...  
references, 58, 149, 375, 411, 417, 620, 1132, 1296, 1305, ...  
refill rate, 1296, 1307  
register, 158, 337, 524, 525, 527, 528, 531, 533, 534, ...  
relative epsilon, 1307  
removal, 49, 71, 73, 78, 82, 83, 332, 408, **924**, ...  
replica, 29, 33, 166, 270, 293, 295, 297, 299, 301, ...  
replica divergence, **1307**  
request-aware routing, **1307**  
resolver, 1307  
resources, 567, 571, 574, 588, 589, 590, 720, 1005, 1307, ...  
returned, 184, 186, 197, 219, 220, 272, 279, 284, 289, ...  
ring buffer, 1296, 1307  
role interface, 1307  
root, 2, 4, 5, 6, 7, 8, 9, 10, 11, ...  
rotation, **501**, 502, 504, 505, 506, 507, 509, 510, 511, ...  
routine, **521**, 522, 523, 526, 527, 533, 541, 543, 544, ...  
row, 6, 7, 25, 33, 36, 46, 47, 51, 54, ...  
run, 23, 26, 31, 32, 33, 35, 36, 39, 41, ...

## S

same remainder, 1308  
schema, 301, 304, 312, 394, 1307, 1308  
scoping, 1308  
script, 77, 170, 246, 248, 260, 620, 627, 787, 790, ...  
seam, 1308  
search space, 1308  
selectivity, 1308  
sentinel, 190, 195, 220, 492, 507, 650, 856, 1233, 1235, ...  
separator, 1308  
sequence diagram, 1308, 1315  
serialisable, 1000, 1308  
service credits, 1308

session, 74, 90, 115, 116, 117, 118, 119, 121, 122, ...  
session guarantees, 805, 806, 814, 1308  
set, 6, 18, 19, 22, 29, 30, 32, 35, 36, ...  
shallow-copied, 1308  
shape, 3, 6, 7, 8, 9, 10, 11, 21, 24, ...  
shard, 23, 26, 29, 30, 32, 33, 35, 36, 360, ...  
shared lock, 1039, 1309  
shared store, 30, 75, 81, 113, 114, 115, 117, 118, 120, ...  
singleton, 119, 1309  
singly linked list, 1309  
slice, 30, 432, 434, 437, 439, 443, 474, 481, 486, ...  
smallest, 102, 104, 299, 370, 371, 373, 374, 409, 410, ...  
snapshot, 300, 999, 1301, 1309  
snipe, 1309  
sorted region, 741, 1309  
sorting everything, 1309  
splits, 394, 397, 398, 437, 482, 547, 552, 562, 563, ...  
spof, 613, 614, 618, 625, 629, 630, 633, 636, 1276, ...  
squares, 1309  
stable, 120, 205, 213, 395, 749, 877, 892, 1139, 1154, ...  
stack, 3, 14, 15, 18, 19, 21, 30, 41, 44, ...  
stack frame, 155, 329, 368, 369, 465, 1309  
stack overflow, 525, 531, 1309  
state, 18, 23, 25, 26, 28, 30, 31, 32, 33, ...  
statements, 1306, 1309, 1316  
status, 433, 441, 444, 1093, 1100, 1110, 1143, 1260, 1309  
still one machine, 25, 37, 304, 308, 395, 664, 1302, 1309  
stonith, 297, 947, 949, 956, 960, 1234, 1240, 1248, 1257, ...  
strategy, 208, 222, 383, 510, 513, 518, 765, 768, 807, ...  
strictly shorter, 1310  
structural, 57, 225, 617, 1306, 1310  
subarray, 282, 1310  
subject, 70, 1310, 1314  
subscription list, 1310  
substring, 230, 235, 238, 1211, 1310  
subtree, 1, 2, 14, 52, 54, 56, 94, 98, 1125, ...  
surge multiplier especially, 1310  
swap, 15, 105, 124, 128, 136, 144, 152, 155, 156, ...  
system call, 1310

## T

table, 24, 42, 89, 116, 120, 122, 157, 160, 193, ...  
tail call, 133, 1310  
tail-recursive, 1310

tcp, 77, 254, 1297, 1310, 1311  
template, 95, 105, 1212, 1213, 1214, 1215, 1216, 1246, 1301, ...  
test double, 1310  
the return address, 1310  
the running median, 687, 919, 920, 929, 961, 988, 1265, 1267, 1292, ...  
third table, 543, 1304, 1310  
thread, 103, 112, 300, 340, 467, 617, 621, 627, 716, ...  
three distinct orderings, 1311  
three-way partition, 1311  
tight coupling, 1311  
time-to-live, 1311  
tiny constraint, 1311  
toast, 1311  
token, 116, 117, 118, 119, 120, 121, 122, 124, 125, ...  
tombstone, 411, 417, 418, 420, 421, 1311  
top-k problems, 1301, 1310, 1311  
total order, 376, 691, 695, 705, 904, 906, 912, 916, 1186, ...  
transitions itself, 1311  
transitive dependency, 1311  
tree, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...  
ttl, 40, 69, 80, 119, 120, 121, 123, 160, 161, ...  
tuple, 57, 101, 105, 185, 190, 191, 192, 195, 197, ...  
two, 3, 4, 5, 6, 8, 9, 13, 16, 17, ...  
two implementations, 559, 1311

## U

udp, 1310, 1311  
under-fetching, 1311  
union filesystem, 1311  
upper bound, 198, 363, 366, 378, 380, 381, 455, 456, 459, ...  
uri, 141, 144, 150, 211, 258, 301, 339, 344, 346, ...  
url, 210, 248, 249, 250, 251, 254, 255, 257, 260, ...

## V

value object, 1312  
version, 98, 101, 105, 129, 183, 188, 189, 193, 883, ...  
vertex, 1298, 1305, 1306, 1312  
vertical scaling, 25, 29, 391, 672, 1312  
victim, 1312  
virtual address space, 1312  
virtual machine, 1295, 1305, 1312  
volatile, 206, 1312

## W

weight, 70, 86, 139, 153, 156, 175, 478, 479, 1121, ...  
which class, 1312  
which is hours, 1312

within, 33, 47, 50, 122, 125, 139, 146, 160, 203, ...

worst case, 14, 18, 63, 71, 73, 76, 79, 80, 193, ...

write skew, 1312

write-around with invalidation, 1312

write-through, 163, 164, 173, 180, 202, 204, 207, 212, 214, ...



yes, 28, 35, 46, 49, 60, 67, 76, 82, 109, ...

# Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.